

PEARSON

MySQL, Fifth Edition

MySQL 技术内幕

(第5版)

[美] Paul DuBois 著
张雷平 何莉莉 陶虹 译



人民邮电出版社



人民邮电出版社
POST & TELECOM PRESS

PEARSON

MySQL® Fifth Edition

MySQL 技术内幕

(第5版)

【美】Paul DuBois 著
张雪平 何莉莉 陶虹 译



中国水利水电出版社



人民邮电出版社
PEOPLE'S POSTAL AND TELEGRAPH PRESS



版权信息

书名：MySQL技术内幕(第5版)

作者：[美] 保罗·迪布瓦(Paul DuBois)

排版：咪奥

出版社：人民邮电出版社

出版时间：2015-07-01

ISBN：9787115388445

— · 版权所有 侵权必究 · —

前言

关系数据库管理系统（Relational Database Management System，RDBMS）是一种极为重要的工具，其应用非常广泛，从商业、科研和教育领域里的应用，到基于因特网（Internet）的内容发布，无处不在。尽管良好的数据库系统对于信息资源的管理和访问来说至关重要，但是许多组织机构还是觉得自己财力不够，难以负担。在过去，数据库系统给人的印象一直都是价格昂贵：不管是软件本身还是后续的技术支持，服务商总是漫天要价。此外，为了获得令人满意的性能表现，数据库引擎对计算机硬件通常都有很高的要求，从而导致整个成本变得更高。

随着时间的推移，计算机在硬件和软件两个方面都有了长足发展。小型桌面系统和服务器的价格都变得更加低廉，但功能却变得更加强大。而且，还出现了一种致力于为其编写高性能操作系统的蓬勃运动。这些操作系统都可以通过因特网或价格低廉的CD免费获得。其中，包括多种BSD Unix派生系统和众多的Linux发行版本。

免费操作系统的发行总是伴随着免费开源（Open Source）工具的发展，这些工具包括GNU C语言编译器gcc、因特网上使用最广的Web服务器Apache，以及像Perl、PHP、Python和Ruby这样被广泛认可的通用型脚本语言。与此形成鲜明对比的则是那些专属方案，它们会将你封锁在服务商所提供的高价产品里，他们甚至还不提供源代码。

现在，获得数据库软件已不再是什么难事，而且很多开源数据库系统还是免费的。其中，最为重要的一个就是MySQL，这是一套源自Scandinavia的SQL“客户端/服务器”关系数据库管理系统。MySQL包含的组件有：SQL服务器、用于访问服务器的客户端程序、管理工具，以及可以让用户自行编写程序的编程接口。

MySQL起源于Michael Widenius（外号Monty）在1979年为瑞典公司TcX所创建的数据库工具UNIREG。1994年，TcX开始着手研究具有SQL接口的RDBMS，用于开发Web应用程序。他们发现所有测试过的商业服务器对于他们的庞大数据表都表现不佳，而且免费的mSQL还缺少多项他们所必需的功能。于是，Monty准备开发出一套新的服务器。

1995年，Detron HB公司的David Axmark开始积极说服TcX公司将MySQL发布到因特网上。David还编写了文档，并使用GNU的配置工具autotools来构建MySQL。适用于Linux和Solaris系统的MySQL 3.11.1的二进制发行版于1996年公诸于世。为提供MySQL的发行，以及商业化的服务，专门成立了MySQL AB公司。2008年，Sun Microsystems公司收购了MySQL AB公司，2010年，Oracle公司又收购了Sun Microsystems公司。如今，MySQL既有二进制版本，又有源代码版本，可以在更多的平台上工作。

早期，MySQL广受欢迎的原因在于它快速、简单。但也有人提出批评：说它缺少特性，如事务处理和外键支持。随着MySQL的不断发展，它不仅增加了这些特性，还增加了许多其他特性，如复制、子查询、存储例程、触发器和视图。

这些功能将MySQL带入了企业应用程序的领域。最终，那些曾经只将“大块头”数据库系统用于其应用程序的人们，现在也开始对在普通硬件一直到企业服务器上都可以运行的MySQL另眼相看了。MySQL的性能可以傲视其他任何竞争对手，而且它可以处理具有数亿行数据的大型数据库。在商业领域，MySQL继续让更多的公司了解到它能够满足他们的数据库需求，同时只需要花费他们为商业许可和支持所支付的费用中的一小部分。

MySQL展现在我们面前的是一片宽广的前景：免费的操作系统，运行在强大但价格低廉的硬件上；同时，在比以往任何时候都要广泛的系统上，将大量的处理能力交给企业和个人。计算经济壁垒的降低让更多（比过去任何时候都要多）的组织机构拥有了高性能RDBMS的强大能力，而且花费极少。这点也同样适用于个人。例如，我在我的苹果笔记本电脑上（运行的操作系统是Mac OS X）使用了MySQL、Perl、PHP和Apache，这使得我能随处工作。全部成本仅仅只有笔记本电脑这一项而已。

选用MySQL的原因

可供选用的免费或低成本数据库管理系统有好几种，如MySQL、PostgreSQL或SQLite。当将MySQL与其他数据库系统进行比较时，需要考虑的

重点是什么呢？性能、功能（如对SQL的兼容性或扩展性）、技术支持、许可情况和价格，所有这些都是要考虑的因素。就此而言，MySQL具有许多让人垂涎的特性。

运行速度。MySQL的运行速度非常快，而且还会变得更快，更多详情请参考<http://www.mysql.com/why-mysql/benchmarks>。最近就有许多改进，尤其是在InnoDB（它现在是默认的存储引擎了）和查询优化器方面。

易用性。MySQL是一个相对简单的、高性能数据库系统，与其他大型数据库系统相比，它的设置和管理都要简单很多。

查询语言支持。MySQL支持的结构化查询语言（Structured Query Language, SQL），是所有现代数据库系统都选用的标准语言。

功能性。MySQL服务器支持多线程，因此允许多个客户端同时与它连接。每个客户端可以同时使用多个数据库。与MySQL进行交互的接口很多，通过它们可以输入查询命令，并查看查询结果。主要的接口类型包括命令行客户端、Web浏览器和GUI客户端等。此外，它还提供了多种语言编程接口，其中包括C、Perl、Java、PHP、Python和Ruby。也可以通过支持ODBC和.NET（由微软公司开发的协议）的应用程序来访问MySQL。也就是说，你可以选择使用预先打包好的客户端软件，也可以编写自己的应用程序。

连接性与安全性。MySQL完全支持网络化，用户可以从因特网上的任何地点来访问数据库，因此你可以与任何地方的任何人共享数据。由于MySQL拥有访问控制功能，因此一方完全无法看到另一方的数据。为了提供更多的安全防护，MySQL还支持使用安全套接层（Secure Sockets Layer, SSL）协议加密的连接。

可移植性。MySQL除了可以运行在Windows系统上以外，还可以运行在各种版本的Unix和Linux系统上。可以运行MySQL硬件包括各种类似路由器和个人电脑那样的小型设备，也包括那些拥有多个CPU和大容量内存的高端服务器。

可用性和成本。MySQL是一个开源项目，有多种许可条款可供选择。首先，可以选择GNU通用公共许可（General Public License, GPL）。这意味着MySQL对于大部分内部使用都是免费的。其次，对于那些愿意接受或者需要正规服务的组织，以及那些不想受GPL许可约束的机构，可以选择商业许可。

公开发行和源代码。获取MySQL的方法很简单，只需要使用Web浏览器即可。如果对某些工作原理有不太明白的地方，或者对某个算法感到好奇，或者想要进行安全审查，你可以获取它的源代码，并对它进行检查。如果你认为自己发现了某个bug，那么请立即报告它，因为开发人员会很想要知道具体情况是怎样的。

MySQL的技术支持做得怎么样？这是个很不错的問題！没有技术支持的数据库系统毫无用处。事实上，本书也算是一种支持形式，希望它可以满足大家的需要。（本书已出版到第5版，这正好表明它实现了该目标。）此外还有很多其他开放资源可供使用，你会发现MySQL拥有很好的技术支持。

MySQL的发行版本里有MySQL参考手册，可以在线获得。该参考手册在MySQL用户社区一直受到很好的评价。这点很重要，因为对于一个好产品，如果没有人能弄清如何使用它，那么它的价值会大打折扣。

Oracle公司提供技术支持合同和教育资源，如培训课程。

MySQL的邮件列表和论坛都是无价的支持资源，任何人都可以访问。这些资源聚积了很多参与者的努力，其中不乏一些MySQL的开发人员。

在MySQL社区，无论是开发人员还是非开发人员，对各种提问的回复都很积极。邮件列表上的问题，通常在几分钟之内即可得到答复。当有bug报告时，开发人员通常都能迅速修复它们，并定期发布新的发行版本。

如果你正在挑选数据库产品，那么MySQL就是一个理想的候选评估对象。你可以无风险地免费试用它。安装和设置时间都要少于许多其他数据库系统。如果遇到什么困难，你可以使用邮件列表寻求帮助。

或许，你当前正在运行另一套数据库系统，但感觉受到严重束缚：当前系统的性能让人揪心；它是专属产品，你不想被困在这点上；你想在当前系统不支持的硬件上运行；提供给你的软件是二进制只读格式的，但你想要获得源代码；或者只是嫌它太贵！所有这些都是你选择MySQL的理由。你可以先通过本书来了解MySQL的功能，然后与MySQL销售人员进行联系，在邮件列表里提问咨询，最后你会找到自己要的答案。

本书的主要内容

本书能让你学会：如何有效地使用MySQL提高工作效率、如何把信息录入到数据库，以及如何通过构建查询语句来向数据“提问”，并获得你想要的答案。

你不需要成为一名专业的SQL程序员。本书会向你展示它是如何工作的。不过，与了解SQL语法相比，明白如何正确使用数据库系统则更有意义。因此，本书着重强调的是MySQL的特有功能，并展示了如何使用它们。你还能看到MySQL是如何与其他工具集成的。本书展示了如何自己编写程序来访问MySQL数据库，以及如何将Perl、PHP与MySQL一起使用，根据数据库查询的结果生成动态网页。

对于负责MySQL安装的人员，可以通过本书来了解具体的职责，以及如何实施。本书还会介绍如何创建用户账号、备份数据库、设置复制，以及如何保证网站的安全。

各章路线图

本书由4个部分组成。第一部分，集中介绍与数据库使用相关的基本概念。第二部分，重点关注自己如何动手编写使用MySQL的程序。第三部分，主要面向那些肩负数据库管理职责的读者。第四部分，提供的是一些参考附录。

第一部分：MySQL基本用法

第1章讨论MySQL的用途，提供了一个介绍交互式客户端程序MySQL的教程，讲解了SQL的基础知识，并演示了MySQL的基本功能。

现在，每一种主流RDBMS都能理解SQL，但是每个数据库引擎所实现的都是稍有不同的SQL方言。第2章讨论SQL，并特别强调了MySQL与众不同的那些特性。

第3章讨论MySQL提供的用于储存信息的数据类型、各种类型的属性和局限性、何时和如何使用它们、表达式计算，以及类型转换。

第4章讨论如何编写和使用存储在服务器端的SQL对象。其中包括视图（虚表）和存储程序（存储函数、存储过程、触发器和事件）。

第5章讨论如何让查询运行得更快。

第二部分：使用MySQL编程接口

第6章讨论MySQL提供的部分应用程序编程接口（Application Programming Interface，API），综合比较了本书详细介绍的那些API。

第7章讨论如何利用MySQL的C语言客户端开发库所提供的API来编写C语言程序。

第8章讨论如何利用DBI模块编写Perl脚本，其中包括独立的命令行脚本和适用于网站编程的脚本。

第9章讨论如何利用PHP脚本语言和PHP数据对象（PHP Data Object，PDO）数据库访问扩展，编写可以访问MySQL数据库的动态Web页面。

第三部分：MySQL管理

第10章概述数据库管理员的职责，以及成功运行一个数据库站点所应了解的事项。

第11章深入研究数据目录的组织和内容，数据目录是MySQL用来存储各种数据库、日志和状态文件的地方。

第12章讨论如何在操作系统启动和关闭时正确启动和停止MySQL服务器。同时，还将讨论配置存储引擎、优化数据库服务器、日志维护，以及运行多个服务器实例等内容。

第13章讨论为让MySQL安装变得更加安全，免受入侵攻击（包括来自数据库服务器主机上的其他用户和通过网络连接进入的客户端）所需要了解的事情。其中，还将讨论如何设置MySQL用户账户，解释用于控制客户端访问MySQL服务器的权限表的结构，以及描述如何设置服务器才能支持基于SSL的安全连接。

第14章讨论如何通过预防性维护减少发生灾难的可能性；如何备份数据库；如何在采取预防性措施之后，灾难仍然发生时恢复系统；以及如何设置复制服务器。

第四部分：附录

附录A介绍在哪里可以获得本书所提到的主要工具和数据库示例文件。

附录B介绍MySQL数据类型的特点。

附录C介绍编写SQL语句里表达式所用到的运算符和函数。

附录D介绍MySQL服务器所维护的各种变量，以及如何在SQL语句里使用自己的变量。

附录E介绍MySQL所支持的所有SQL语句。

附录F介绍MySQL发行版中提供的各种程序。

附录G介绍MySQL的C语言客户端开发库所包含的数据类型和函数。

附录H介绍Perl语言的DBI模块所提供的各种方法和属性。

附录I介绍在PHP语言里为支持MySQL通过PDO扩展而提供的各种方法。

说明

附录G、H、I需要上网获取。可以访问
www.informit.com/title/9780672329388，注册后可获取，也可以访问
www.kitebird.com/mysql-book来获取[1]。

附录G介绍MySQL C客户端库所提供的数据类型和函数。

附录H介绍Perl DBI模块提供的方法和属性。

附录I介绍PDO扩展为在PHP中支持MySQL而提供的方法。

阅读本书的方法

不管你是从本书的什么地方开始阅读，都最好能练习一下阅读过程中遇到的示例。这意味着你要先做好两件事情。

如果在你的系统上还未安装MySQL，那么请把它安装好，或者请其他人帮你装上它。

获取用于设置本书所使用到的sampdb示例数据库所需要的文件。

附录A给出了获得所有这些必要组件的办法和步骤。

如果你对MySQL或SQL还不太熟悉，那么请从第1章开始。它提供了一个入门教程，让你先了解MySQL与SQL的基本概念，从而加快本书其余章节的阅读速度。接着，再进入到第2章、第3章和第4章，了解如何描述和操作你自己的数据，这样你就可以在自己的应用程序里使用MySQL的功能了。

即使你对SQL已有所了解，但是仍然可以从第2章和第3章开始阅读。SQL的实现各不相同，而你要首先弄清：是什么让MySQL的实现与其他你所熟悉的SQL实现相比显得很独特。如果你对MySQL已经非常熟悉，但是还想要进一步改善其运行性能，那么可以把本书当作参考书，根据实际需要查询相关的章节。书后的附录可以提供很多信息。

如果你想要自己编写能访问MySQL数据库的程序，那么请从第6章开始，阅读与API有关的章节。如果想要为你的数据库开发一个基于Web的前端，以便更好地访问它们，或者与之相反，想要为你的Web站点提供一个数据库后台，以便能让站点增添动态内容，那么请阅读第8章和第9章。

如果你的职责是管理MySQL的安装，那么请从第10章开始阅读。

如果你正在对MySQL进行评估，以便得出“与你当前的RDBMS相比，它表现如何”的结论，那么本书中有几个章节可供参考。如果你想要了解MySQL与你习惯的SQL版本相比有何差异，那么请阅读本书第一部分里与SQL语法和数据类型有关的章节。如果你需要开发自己的应用程序，那么请阅读第二部分中与编程有关的章节。如果你想要评估MySQL安装所需要的管理支持级别，那么请阅读第三部分中与管理有关的章节。如果你目前还没使用数据库，但是正在将MySQL与其他数据库系统一起进行对比分析，以便从中做出选择，那么这些信息也能派上用场。

本书涉及的软件版本

本书第1版讲的是MySQL 3.22，并开始涉及MySQL 3.23。第2版则把讲解范围扩大到了MySQL 4.0，以及MySQL 4.1的第一个发行版本。第3版讲的是MySQL 4.1，以及MySQL 5.0的几个初始发行版本。第4版讲的是MySQL 5.0，以及MySQL 5.1的几个初始发行版本。

对于第5版，最低版本要求是MySQL 5.5。也就是说，本书讲解的内容是MySQL 5.5，以及MySQL 5.6的早期发行版本。如果你的版本比5.5还旧，那么这本书的大部分内容仍然适用，但针对旧版本的差异一般没有明确指出来。

MySQL 5.5系列已经达到了普遍适用（General Availability, GA）阶段，也就是说，它完全适用于生产环境。与5.5之前的发行版本相比，新版本已经发生了很多变化，因此请大家尽可能选用最新的版本（在我编写本书时，最新版本为5.5.30）。当前，MySQL 5.6系列还处在开发当中（还不适合用于生产用途），但很快就会达到GA阶段，而且很可能在你阅读本书时就已经达到了。

与旧版本有关的更多信息，请访问MySQL网站<http://dev.mysql.com/doc>，在那里可以访问到所有版本的参考手册。

每次用新的材料对本书的各个版本进行更新时，想要控制书的篇幅一直是件极具挑战性的事情。出于篇幅的考虑，我删除了一些在之前的版本里已有的信息。相比之前的版本，MySQL最大的变化在于：InnoDB现在成为了默认的存储引擎（不再是MyISAM了）。因此，本书为了迎合重点强调InnoDB的需要，减少了与MyISAM相关的内容。此外，对于不太重要的存储引擎（如FEDERATED和BLACKHOLE），则一笔带过。本书还删除了与libmysqld（嵌入式服务器）、mysqlhotcopy、myisampack、空间数据类型和函数有关的信息，并且使用通用性的说明代替了详尽的安装材料。更多相关信息请阅读MySQL参考手册。

对于书中未曾涉及的以下几项内容，大家需要留意一下。

MySQL连接器。用于为Java、ODBC和.NET程序提供客户端访问。

NDB存储引擎和MySQL集群。它们提供基于内存的存储、高可用性和冗余。更多详细信息请参考MySQL参考手册。

图形用户界面（Graphical User Interface, GUI）工具MySQL Workbench。它可以让你在窗口环境里使用MySQL。

MySQL企业版。这是MySQL的商业版本，其中包含像MySQL企业版监控器（MySQL Enterprise Monitor）和MySQL企业版备份（MySQL Enterprise Backup）这样的功能。MySQL企业版监控器能提供服务器监控和诊断功能，MySQL企业版备份能实现热备份。

如果想要下载这些产品或者查看它们的文档，可以访问<http://www.mysql.com/products/>或<http://dev.mysql.com/doc/>。

对于本书中涉及的其他主要软件包，只要是最近的版本对于示例展示来说就应该足够了。下面表格中列出了在编写本书时各软件包的最新版本。

软件包	版 本
Perl DBI模块	1.623
Perl DBD::mysql模块	4.020
PHP	5.4.10
Apache	2.4.3
CGI.pm	3.63

本书所提到的所有软件都可以在网上下载到。附录A提供了将MySQL、Perl DBI、PHP、PDO、Apache和CGI.pm部署至系统里的相关帮助。该附录还包含了“如何获得全书所用示例数据库sampdb”的说明，以及在本书编程章节所开发的各个程序。

如果你使用的是Windows，那么本书会假设你的系统为Windows 2000或更高的版本。本书涉及的某些功能，如命名管道和Windows服务，在早期的版本里并不支持。

排版约定

本书使用了下列印刷约定。

主机名、文件名、目录名、命令、选项和网站皆用Courier字体表示。

在命令示例中需要读者输入的内容，则用加粗的Courier字体表示。

在命令中需要读者替换为自己所选值的那部分，则用Courier斜体字表示。

对于交互示例，则假设是在终端窗口或控制台窗口里输入各种命令。为反映出具体的环境，命令示例里的提示符则用来表示运行你的命令所需要的程序环境。例如，在mysql客户端程序里执行的SQL语句，其前面会显示mysql>提示符。对于通过命令解释器执行的命令，其提示符通常为%。一般情况下，此提示符表示的是：这些命令可以同时使用在Unix系统和Windows系统下使用，只

是你真正看到的提示符样子，取决于具体的命令解释器。（在Unix系统里，命令解释器指的是你登录所用的那个shell程序；在Windows系统里，则指的是cmd.exe。）另外还有两个比较特殊的命令行提示符，即#和C:\>，前者表示该命令需要在Unix系统上通过su或sudo命令以root用户身份来执行，后者表示该命令专门用于Windows系统。

下面的示例展示了一条应从命令解释器里输入的命令。%是提示符（不需要你输入）。为执行该命令，你需要输入如下所示的那些粗体字符，并将那个斜体字替换成你自己的用户名：

```
% mysql --user=user-name sampdb
```

在SQL语句里，SQL的关键字和函数名要用大写字母来表示。而数据库名、表名和列名则一般用小写字母来表示。

在语法描述里，方括号（[]）用于表示可选内容。备选列表则以竖线（|）进行分隔。包括在方括号内的列表是可选项，可以从中选择一项。包括在花括号{}内的列表是必选项，必须从中选择一项。

其他资源

如果你在本书中未能找到某个问题的答案，该怎么办呢？下面的表格列出了一些文档资源，其中包括相关软件及其站点，希望对你有所帮助。

软件包	官方网站
MySQL	http://dev.mysql.com/doc
Perl DBI	http://dbi.perl.org
PHP	http://www.php.net
Apache	http://httpd.apache.org
CGI.pm	http://search.cpan.org/dist/CGI.pm

这些站点提供的信息包括参考手册、常见问题（Frequently Asked-Question, FAQ）和邮件列表。

参考手册。包含在MySQL里的主要文档就是参考手册。它提供有多种格式，其中包括在线版本和可下载版本。

手册页。对于DBI模块及其MySQL专用的驱动程序DBD::mysql的文档，可以在命令行输入perldoc命令来查阅。请尝试输入perldoc DBI和perldoc DBD::mysql命令。DBI文档提供的是一些基本概念，而MySQL驱动程序的文档讨论的是与MySQL相关的各种功能。

常见问题。具体包括DBI、PHP和Apache的常见问题列表。

邮件列表。这里主要提供了几个与本书所涉及软件有关的邮件列表。订阅与你所用工具相关的邮件列表，是个不错的选择。使用这些邮件列表的归档列表，也是个很好的做法。如果你对某个工具不太了解，那么总会有很多的问题要问，而这些问题通常都早已被很多人问过（和回答过）无数次，因此当你能在归档列表中快速搜索到答案时，就没有必要再提出相同的问题。

不同邮件列表的订阅说明也各有不同。下面的表格列出了可以帮你找到必要信息的网址。

软件包	邮件列表说明
MySQL	http://lists.mysql.com
Perl DBI	http://dbi.perl.org/support
PHP	http://www.php.net/mailling-lists.php
Apache	http://httpd.apache.org/lists.html

辅助网站。除了官方网站之外，这里所涉及的某些工具还另外有许多辅助站点。这些站点可以提供更多信息，如样例源代码或专题文章。在你访问的官方网站上，到“链接”（Links）区域找找，即可发现很多相关站点。

关于作者

Paul DuBois既是作家和数据库管理员，又是开源社区和MySQL社区的领导者。他为MySQL在线文档作出过很多贡献，其著作包括New Riders出版的《MySQL and Perl for the Web》，以及O'Reilly出版的《MySQL Cookbook》、《Using csh and tcsh》和《Software Portability with imake》。目前，他是Oracle公司MySQL文档团队的技术作家。

致谢

首先要感谢的是技术审稿人Stephen Frein，他为本书的改进提供了许多很好的看法和建议。另外，如果没有之前的几个版本作为基础，也就不会有这一版本的诞生，因此我还要继续感谢那些在之前版本里帮助过我的技术审稿人和耐心回答我问题的人。

Pearson公司参与本版出版工作的员工有：策划编辑Mark Taber、项目编辑Tonya Simpson、文字编辑Sarah Kearns、排版员Kim Scott、校对员Jess DeGabriele、索引员Heather McNeill，以及封面设计者Chuti Prasertsith。在此，也要对他们表示感谢。

最后要感谢我的妻子Karen。感谢她在本版编写过程中给予的最大支持和鼓励。

读者服务

访问Pearson的网站，并在www.informit.com/register注册本书，可获得更多信息，其中包括本书提供的各种更新、下载资源或勘误。

[1].相应的中文译稿可在人民邮电出版社网站本书主页上免费注册下载。

——编者注

第1章

MySQL入门

本章主要介绍MySQL关系数据库管理系统（Relational Database Management System, RDBMS）和MySQL所使用的结构化查询语言（Structured Query Language, SQL）。其中，列出了应该掌握的基本术语和概念，描述了示例所要用到的样本数据库sampdb，并且提供了一个用于展示如何使用MySQL创建数据库并与之进行交互操作的教程。

如果你对数据库系统还不甚了解，或者还不是很肯定自己是否需要学习它，甚至还不肯定是否需要使用它，那么请从本章开始。如果你对MySQL和SQL还一无所知，并且需要一个入门指南，那么也请从本章开始。对MySQL或其他数据库系统已有一定经验的读者，则可以略读一下本章的内容。不过，为了你能对全书所用那个数据库sampdb的目的和内容更熟悉，希望可以阅读一下1.2节。

1.1 MySQL的用途

本节描述的是MySQL数据库系统的用途，具体描述了“MySQL可以干什么”，以及“它如何才能给你提供帮助”。如果你已经明白数据库的用途（也许你正在思考某个问题，到这里来只是想要找出“如何使用MySQL来解决它”的答案），那么可以直接翻阅到1.2节。

数据库系统在本质上是一种高效的管理大量列表信息的办法。这些信息的来源可能多种多样。它可能是研究数据、业务记录、客户需求、体育统计、销售报告、个人信息、人事档案、bug报告或者学生成绩。发挥数据库系统强大作用的时机在于：想要组织和管理的信息非常庞大或复杂，以至于所有记录采用手工处理会变得异常繁重。对于每天处理上百万条事务的大公司来说，数据库是必不可少的。不过，即使只有一个人的小公司，也可能会维

护很多的信息，甚至多到需要用一个数据库来管理它。假设有下面这些情况。

你在牙科诊所工作，在那里需要管理好患者的跟踪记录：何人何时到访、做了些什么、下次预约信息、保险信息等。

你收集了多年的研究数据，为了发表而需要对它们进行分析。你要从大量原始数据里提炼出摘要性的信息，并取出选中的观察子集进行详细的统计分析。

你是一名教师，需要跟踪学生的成绩和考勤。每次考试结束，你都需要记录每一个学生的分数。虽然将成绩记录到成绩册上很简单，但之后的成绩分析却很繁琐。你很想避免为了确定分数曲线，而对每次考试的分数进行排序；也真的很不愿意在期末时为了确定最终成绩，而把每个学生的成绩都加起来。统计每个学生的考勤也很无趣。

你在某个组织机构（可能是一个专业团体、一个俱乐部、一个交响乐团或者一个健身俱乐部）担任秘书一职，具体负责维护机构成员名录的工作。你每年都要为所有成员生成一份打印名录，名录是用文字处理软件管理的，每当成员资料有变化时你都得编辑更新。你非常厌倦这种维护名录的方式，因为它限制了你的发挥，主要表现在：很难对名录条目按不同方式进行排序；无法轻松地选到每一条目的指定部分，如列出人名和电话号码；更不能轻松地找出一组的成员，如需要尽快更新成员资格的成员。如果有办法，就可以省却每月通过翻阅这些条目来找出那些需要更新成员资格的成员的工作。你听说过“无纸化办公”，知道它是电子化记录发展的结果，但你还未见过它所带来的任何好处。虽然成员资格记录是电子化的，但具有讽刺意味的是，它们记录的形式除了能将名录打印成纸质的以外，很难用作他途！

上面这些场景所涉及的信息量有大有小。但它们都有一个共同的特点，即这些工作都可以通过手工来完成，但使用数据库系统来管理会更高效。

在使用像 MySQL 这样的数据库系统时，你希望从中获得哪些具体好处呢？这取决于你的特殊需要和需求，并且，如同上面示例所示，具体的好处都各不相同。不过，在一般情况下，适合于用数据库管理系统来处理任务的人群是那些不使用数据库管理系统就要使用文件柜的人。事实上，数据库系统就像一个内置了复杂文件系统的巨大文件柜。与手工管理记录的方式相比，以电子化方式来管理记录存在有许多优势。下面来看看前面描述过的那个牙科诊所的场景。在用于管理患者记录的文档系统能力方面，MySQL可以为你带来下面这些帮助。

(1) 缩短记录归档时间。你不用在文件柜里挨个拉抽屉找存放新记录的地方。你只需将它提交给MySQL，MySQL会为你找到正确的存放这条记录地

方。

(2) 缩短记录检索时间。当进行记录查找时，不必为了想要找到的记录而亲自动手去逐个搜索。为了给那些最近一段时间没来做检查的患者发个提醒信息，你可以让 MySQL 来帮你找出这些记录。当然，这与让你告诉另一个人“请帮忙确认一下最近6个月哪些患者没来参加检查”的情况有所不同。事实上，你“念出的”是一段奇怪的“咒语”：

```
SELECT last_name, first_name, last_visit FROM patient
```

```
WHERE last_visit < DATE_SUB(CURDATE(), INTERVAL 6 MONTH);
```

如果你以前从没见过类似的内容，那么它初看起来可能会相当吓人。不过，它的效果却是相当吸引人：你不用再花费一小时来翻看你的记录，只需一两秒即可得到想要的结果。不管怎样，无需多久你就会习惯这种奇形怪状的表达。等阅读完本章，你便会明白它真正的含义。

(3) 灵活的检索排序。不用严格按照记录存储的顺序（如按患者的名字排序）来检索它们。MySQL可以按任何你想要的顺序来提取记录，如按名字、保险公司名字、上次就诊时间等。

(4) 灵活的输出格式。在找到感兴趣的记录之后，你无需手动复制这些信息，MySQL会为你生成一个列表。有时，你可能只是想打印这些信息；有时，你可能会想在另一个程序里使用它们。例如，在生成了最近逾期未能复诊的患者名单后，你便可以把这些资料输送到某个文字处理软件，让它打印出你想要发送给那些患者的通知单。或许，你只是对如同“选中记录统计”这样的汇总信息比较感兴趣。你不用亲自来统计记录，MySQL会替你生成汇总信息。

(5) 多用户同时访问记录。对于纸质记录，如果有两个人同时想要查看某个记录，那么其中一个人就必须等待另一个人将这个记录归还之后才能查看。而 MySQL 则可以允许这两个人同时访问这个记录。

(6) 记录的远程访问与电子传输。想用纸质记录，就得亲自跑到存放它们的地方，或者让人将它们复印之后再发送给你。电子记录则为远程访问记录或者电子传输记录提供了可能。如果你的牙科集团设有许多分支机构，那么这些机构里的人员便可以在当地访问到你的资料。你完全不用再通过快递来传送这些副本。如果有人需要记录，但却又没有与你一样的数据库软件，那么你可以选择那些所需的记录，将其中的内容通过网络发送给他。

如果你曾经用过数据库管理系统，那么肯定对刚才描述的种种好处深有体会，而且可能也在思索如何才能超越常规“代替文件柜”应用程序的限

制。有许多组织机构将数据库与网站结合在一起使用，这就是一种很好的方式。假设贵公司有一个商品库存数据库，每当顾客打电话过来询问仓库里是否有货、其价格是多少时，服务台的员工便会用到它。这是一种比较传统的数据库使用方式。不过，如果贵公司搭建一个供顾客访问的网站，那么就可以提供一项新的服务：创建一个搜索页面，让顾客可以查看条目，从而确定价格、可用性以及商品的库存情况。如果支持在线订购，那么顾客甚至不用离开家门便能购买到你的产品。这让顾客获得了他们想要的信息，而数据库却是根据提问自动搜索商品库存信息来提供这些内容的。顾客立即获得了所关心的信息，不用一边听着恼人的录音，一边傻等着；也不用受限于服务台的上下班时间。并且，每当有一位顾客使用贵公司网站，就意味着会少一个电话，而这是需要由一个在服务台拿工资的人来处理的事情。如此看来，该网站或许可以为自己买单。

不过，你还可以更进一步地发挥数据库的作用。基于Web的库存搜索请求，不仅可以把信息提供给顾客，也能提供给贵公司。这些查询可以让你了解到顾客正在查找哪些商品，而查询结果则会让你知道是否能够满足他们的需求。在某种程度上，如果你没有顾客想要的东西，那么你可能会错失这笔生意。因此，记录下库存搜索信息非常有意义，通过它可以了解到：顾客正在找寻什么商品，而你是否还有存货。接着，就可以根据这些信息来调整库存，并向顾客提供更好的服务。

说了半天，那么 MySQL 是如何工作的呢？找到答案的最好方式就是自己动手体验一下。为此，我们需要有一个可以操作的数据库。

1.2 示例数据库

本节将描述本书所使用的示例数据库。在你学习使用MySQL的过程中，可以用这个数据库提供的示例进行尝试，该数据库是针对之前描述过的两种情况设计的。

“机构秘书”场景。该机构有一些特点：其成员对美国历史很感兴趣（因没有一个更好的名字，所以姑且称其为“美史联盟”）。所有成员都要定期缴纳一定的费用以维持其成员资格。缴纳的费用会用于一些正常开支，如出版通讯——《Chronicles of U. S. Past 》（美国编年史）。该联盟运营着一个小型网站，但还未被充分开发出来，而你很想要扭转这一局面。

“成绩考评”场景。你是一名教师，要负责考评期间的各种考试与测验，记录分数和打分。之后，你要确定出最终成绩，并把它们随同考勤情况一同上交到学校办公室。

下面，我们进一步分析一下这两个场景的需求。

你必须决定数据库里的哪些内容是你所想要的——即那些你要实现的目标。

你必须决定需要将哪些内容放入数据库——即你想要跟踪的数据是什么。

在考虑“将什么放入数据库”之前考虑“要从数据库里获得什么样的内容”，这似乎有些本末倒置。毕竟，大家都是这么认为的：要先输入数据，然后才能检索它。不过，你使用数据库的方式取决于你的目标是什么。而且，与想要放入数据库的内容相比，想要从数据库检索的内容则与那些目标的关系更加密切。只有在计划今后使用这些信息之后，你才会想要花费时间和精力把它们放入数据库。

1.2.1 美史联盟项目

该项目的场景是：你是联盟秘书，眼下正使用某个文字处理文档维护那份成员资格名单。对于像“生成打印名录”这样的事情，处理起来当然没什么问题。但是，当你想要获得其他更多信息时，就会受到限制。你还有几个目标要实现。

你想要以不同格式输出名录，同时按照不同的应用程序来定制信息。有一个目标是每年生成打印好的名录——这是联盟一直以来的一个需求，要继续实施下去。你可能还会想到名录信息的其他用途，例如向出席联盟年会的人员提供一份打印好的最新成员名单。这两个应用程序涉及的信息是不同的。打印名录程序需要用到每个成员条目的所有内容。而年会程序则只需要提取出成员的姓名即可（使用文字处理软件无法轻松完成这项工作）。

你想要在名录里搜索出满足不同条件成员。例如，你要知道近期有哪些成员需要更新成员资格。你还需要另一个搜索相关的应用程序，用来维护每个成员各自的关键字列表。这些关键字描述了所有成员都特别感兴趣的一些美国历史时期，如南北战争（Civil War）、经济大萧条（Depression）、民权法案（civil right）、托马斯·杰斐逊（Thomas Jefferson）总统的生平事迹。有时，有些成员会要求你为他们提供一份与其志趣相投的其他成员名单，而你也很愿意满足这些需求。

你想要将名录发布到联盟网站上去。这样做能让你和所有成员都受益。如果你能通过某些自动化的过程将名录转换为Web页面，那么名录的在线版本将总能保持最新，而这是纸质形式无法做到的。如果在线名录能支持搜索功能，那么成员们便能轻松地自行查找信息。例如，如果某成员想要知道其他还有谁对“南北战争”感兴趣，那么他便可以自行查找，完全用不着等你帮他搜索，而你也不用抽时间去处理这件事情。

数据库不是世界上最令人兴奋的工具，因此我不会鼓吹像“使用数据库能激发人的创造性思维”这样不切实际的话。不过，如果你不再把信息看成是一种累赘（如同你在使用文字处理文档时所想的那样），而是把它想成是可相对轻松处理的事情（如同你希望使用MySQL来处理一样），那么你自然会释放出自己的潜能，找到更多新的使用这些信息的方法。

如果数据库中的信息能够以在线名录的形式放到网站上去，那么你也可以让这些信息以其他方式流转。例如，让成员能够在线修改他们自己的资料，并更新到数据库。这样，你就不用自己负责所有的编辑工作，而且这也能让名录里的信息更加准确。

如果你把电子邮件地址也存储到数据库里，那么你就可以利用它们来给成员发送电子邮件，提醒他们及时更新自己的资料。邮件内容可以显示出成员的当前资料，请成员们进行检查，并提示他们如何使用网站提供的功能完成必要的修改。

数据库还可以在很多方面拓展联盟网站的用途，而并不仅限于成员资格列表。联盟通讯《Chronicles of U. S. Past》有一个儿童专栏，其中每一期都会包含一个历史知识测验。

最近几期的重点是美国总统的传记。联盟网站上也可以设置一个儿童专区，把那些测验题目放在上面。或许这个专区还可以弄成互动的，比如将做过的测验信息放入数据库，让Web服务器在数据库里查询问题的答案，然后呈现给访客。

好了！此时此刻，你所想到的数据库应用数量可能让你有些忘乎所以了。在稍息片刻之后，重新回到现实，你开始问一些比较实际的问题，例如：

是不是想太多了？实现起来工作量会很大吧？

当然，所有事情只想不做都会变得很容易，而这些想法对于具体实现根本不重要。不过，在读完本书之后，你就能实现我们刚才罗列出来的所有需求。但是请记住，没有必要一下子完成所有需求。要将整个工作分解成若干需求，一次只解决一个需求。

MySQL能实现所有这些目标吗？

不，它不能，至少单靠它是不行。例如，MySQL没有内置Web程序开发工具。不过，你可以将它与其他工具相结合，借助它们来补充和扩展其功能。

我们将使用脚本语言Perl和它的数据库接口（Database Interface, DBI）模块来编写访问MySQL数据库的脚本程序。Perl语言有着强大的文本处理能力，它能以极其灵活的方式对数据库的查询结果进行处理，并产生各式各样的输出。例如，我们可以用Perl语言生成一份富文本格式（Rich Text Format, RTF）格式的成员名录，该格式可供各种文字处理软件读取；另外，还可以生成适用于Web浏览器的HTML格式的名录。

我们还要用到另一种脚本语言 PHP。PHP语言特别适用于编写 Web应用程序，并且它也易与数据库进行交互。有了它，你就能够通过Web页面来启动MySQL查询，然后生成新的包含数据库查询结果的页面。有很多支持 PHP语言的Web服务器（其中包括世界上最流行的Web服务器Apache），因此像“呈现一个搜索表单，并显示搜索结果”这样的事情，对它来说就是“小菜一碟”。

MySQL 与这些工具可以很好地集成在一起，因此你可以自由地选择组合方式，以便达到你心中设定的目标。别太相信那些一体化的套装组件，它们通常都大肆鼓吹自己具有“集成”功能，但是它们只有彼此配合才能更好地发挥作用。

最后，还有一个很重要的问题：总计要花费多少钱？毕竟联盟的预算是有限的。

答案可能令人难以置信，但实际上它可能不会有任何成本。如果你对数据库系统有所了解，那么你应该知道它们通常都很昂贵。相比之下，MySQL通常是可免费使用的。即便是在需要有技术支持和维护承诺保证的企业环境里，将 MySQL作为数据库系统，其成本也是很低的。（想了解更多详情，请访问www.mysql.com。）我们将用到的其他工具（其中包括Perl、DBI、PHP和Apache）都是免费的。因此，综合考虑一下，你完全能够以相当低的成本组建一个有用的系统。

用于开发数据库的操作系统则由你来选择。几乎所有我们将讨论的软件都能在UNIX（它指代了BSD UNIX、Linux、Mac OS X等）和Windows上运行。极少数的例外情况一般是UNIX或Windows特有的shell脚本或批处理脚本。

1.2.2 成绩考评项目

现在一起来看另一个要使用示例数据库的情况。该项目的场景是：你是一名负责成绩考评的教师。你想将成绩处理工作，从使用成绩册的手工操作方式转换成使用MySQL的电子表示方式。在这种情况下，从数据库获取信息的方式隐含在你目前使用成绩册的方式中。

对于每一次测验或考试，你都要记录分数。如果是考试，你还需要对分数进行排序，以便查看它们并确定每一个字母成绩（包括A、B、C、D和F）所代表的界线。

在期末，你需要把计算出每一个学生的总分数，对这些总分数进行排序，并以此为基础确定出成绩。总分数可能需要加权计算，因为你可能需要让考试比测验的权重更大。

在期末，你还要向学校办公室提供学生的考勤情况。

最终目标是要避免手动排序、避免手动汇总分数和考勤情况。换句话说，你希望 MySQL 可以对分数排序，并在期末完成所有与每个学生的总分和缺席次数相关的计算。为实现这些目标，你需要班里学生的名单、每次考试和测验的分数，以及全部学生的缺勤日期。

1.2.3 如何运用示例数据库

如果你对这里的“美史联盟”和“成绩考评”两个项目都不感兴趣，那么你可能会想“还有什么场景一定适合你呢”。答案就是“无穷无尽”。其实，这两个项目可以说明你用MySQL和与之相关的工具可以做什么事情。稍微想象一下，你便能看到示例数据库查询是如何应用于你所要解决的那些特定问题的。假设你正好就在我前面提到的那个牙科诊所里工作。虽然在这本书里不会看到很多与牙科学有关的查询，但是会看到这里所发现的很多查询都可以应用到患者记录维护、办公记载等那些工作中。例如，“确定联盟里哪些成员需要更新他们的成员资格”与“确定最近哪些患者未来复诊”这两个任务便非常相似。它们都是基于日期的查询，因此只要你学会了“成员资格更新”的查询，那么便可以将该技巧用于编写“久未复诊患者”的查询，从而带来更多的收获。

1.3 基本数据库术语

你可能已经注意到了，尽管这是一本关于数据库的书籍，但到目前为止，你还未遇到过多少晦涩难懂的专业技术术语。事实上，尽管我们对如何使用示例数据库有过大致描述，但关于数据库到底是什么样子，我却只字未提。可是，既然我们要设计数据库，并实现它，那么我们就不能再避而不谈有关的术语了。这正是本节的主要内容。本节所描述的术语都是本书要用到的，因此希望大家能够熟悉它们。庆幸的是，关系数据库的许多概念都很简单。人们之所以喜欢关系数据库，很大程度上便是因为其基本概念都简明易懂。

1.3.1 结构术语

在数据库领域，MySQL被划分为关系数据库管理系统（RDBMS）。我们下面来拆解一下。

数据库（DataBase，即RDBMS里的DB）是一个用来存储信息的仓库，它的结构简单、规则。

数据库里的数据集都被组织成表（table）。

每个表由多个行（row）和列（column）组成。

表中的每一行称为一条记录（record）。

记录可以包含多项信息；表里的每一列对应于其中的一项。

管理系统（Management System，即RDBMS里的MS）是一个软件，我们可以通过它来插入（insert）、检索（retrieve）、修改（modify）或删除（delete）记录。

关系（Relational，即RDBMS里的R）一词表示这是一种特殊的DBMS，其长处在于通过查找两个表里的共同元素，将分别存放于两个表里的信息联系（即匹配）起来。

RDBMS的强大之处在于：它能方便地将这些表里的数据提取出来，并把相关表里的信息结合起来生成答案，回答那些只靠单个表无法回答的问题。

（事实上，“关系”的正式定义与我在本书中用它的方式有所不同。为此，我先向那些纯粹主义者道歉。不过，我的定义更有助于表达出RDBMS的用途。）

关系数据库是如何把数据组织到表里的呢？又是如何把不同表的信息关联在一起的呢？下面来看一个例子。假设你在经营一个网站，它有一项横幅广告服务。你与多家想要刊登广告的公司签订了合同，满足它们想要在人们访问你网站页面时显示其广告的需要。每当有访客点击其中的某个页面时，你就提供一个嵌有广告的页面发送给访客浏览器，这样你就能从刊登这条广告的公司那里获得一点费用。这就是所谓的广告“点击”。为了表示这些信息，你需要用到3个表（见图1-1）。第1个company表由这样几列构成：公司名称、编号、地址和电话号码。第2个ad表的构成列有：广告编号、“拥有”该广告的那家公司的编号，以及每点击一次的收费数目。第3个hit表需要记录广告点击量和广告点击日期。

company表

company_name	company_num	address	phone
Big deal, Ltd.	13	14 Grand Blvd.	875-2934
Pickles, Inc.	14	59 Cucumber Dr.	884-2472
Real Roofing Co.	17	928 Shingles Rd.	882-4173
GigaFred & Son	23	2572 Family Ave.	847-4738

ad 表

company_num	ad_num	hit_fee
14	48	0.01
23	49	0.02
17	52	0.01
13	55	0.03
23	62	0.02
23	63	0.01
23	64	0.02
13	77	0.03
23	99	0.03
14	101	0.01
13	102	0.01
17	119	0.02

hit 表

ad_num	date
49	July 13
55	July 13
48	July 14
63	July 14
101	July 14
62	July 14
119	July 14
102	July 14
52	July 14
48	July 14
64	July 14
119	July 14
48	July 14
101	July 14
63	July 15
49	July 15
77	July 15
99	July 15

图1-1 横幅广告的表

有些问题只用一个表便能回答。例如，想要知道与你签订了合同的公司有多少家，那么你只需统计一下company表共有多少行即可。同样地，想要了解在某段指定的时间里有多少点击量，则只需检查表hit即可。其他有些问题则可能比较复杂，需要查询多个表才能得到答案。例如，想要知道在7月14日那天Pickles公司（Pickles, Inc.）的每一条广告分别被点击了多少次，那么就需要像下面那样使用所有的3个表。

（1）在company表里查找公司名称（Pickles, Inc.），从而查出该公司的编号（14）。

（2）在ad表里，利用这个公司编号找出与之匹配的行，从而可以确定出所有相关广告的编号。最后找到两条广告，即编号48和101。

（3）对于在ad表里匹配到的每一行，使用该行里的广告编号在hit表里找出日期介于给定日期范围内的所有匹配行；然后，再统计这些匹配行的数量。最后查询出的结果是：48号广告有3个匹配；101号广告有2个匹配。

这些听起来好像很复杂！但这正是关系数据库系统最擅长做的事情。这种复杂性只是表面现象，因为刚才描述的每一步骤都是一个简单的匹配操作：将一个表的行值与另一个表的行值，通过匹配关联起来。这种简单的操作可以有多种变化，用于回答各式各样的问题，如各家公司分别投放了多少个不同的广告？哪家公司的广告最受欢迎？每个广告带来的收益是多少？在当前结算期内，各家公司应该支付你多少广告费？

有了之前讲述的这些关系数据库理论，你就能读懂本书后续的内容了；我们不必了解“第三范式”（Third Normal Form）、“实体联系图”

（Entity-Relationship Diagram）等这些枯燥乏味的内容。（如果了解它们，请阅读C. J. Date或E. F. Codd的著作。）

1.3.2 查询语言术语

为了与MySQL交互，需要使用一种名为SQL的语言。所有主流数据库系统都支持SQL，但各个服务商的实现都各不相同。SQL支持许多不同的语句，可以让你用非常有趣和实用的方式与数据库进行交互。

与其他计算机语言一样，初次接触SQL的人往往会觉得它很奇怪。例如，在创建表时，你必须告诉MySQL该表的结构是什么样子的。很多人都会想到把表与图表或图片联系起来。但是MySQL不是这样子的，因此在创建表时，你必须告知MySQL一些类似下面那样的内容：

```
CREATE TABLE company
```

```
(  
  
company_name VARCHAR(30),  
  
company_num INT,  
  
address VARCHAR(30),  
  
phone VARCHAR(12)  
);
```

如果对SQL还不太熟悉，那么可能会对这样的语句心生畏惧。不过请放心，你不用成为程序员就能学会如何熟练地使用SQL。随着对SQL的深入了解，你对CREATE TABLE的看法也会悄然地发生变化——它不再是一组怪诞的表达，而是一种有助于信息描述的强大工具。

1.3.3 MySQL的体系结构术语

当使用MySQL的时候，你实际上至少会用到两个程序，因为MySQL采用的是“客户端/服务器”体系结构。有一个程序是MySQL服务器，即mysqld。它运行在存放数据库的那台机器上，主要负责监听网络上的客户端请求，并根据这些请求去访问数据库内容，然后向客户端提供它们所查询的信息。另一个程序是MySQL客户端，它主要负责连接到数据库服务器，并发起查询，以便将自己想要的信息告知服务器。

MySQL的大部分发行版本都包括数据库服务器程序和客户端程序。（在Linux下使用RPM包时，会有一个单独的服务器RPM包和客户端RPM包，因此应该安装两种包。）请根据自己的实际情况使用合适的客户端程序。最常用的客户端程序是mysql，它是一个交互式客户端程序，你可以用它来发起查询并查看结果。另外还有两个主要用于管理的客户端程序，即mysqldump和mysqladmin。前者主要用来把表的内容导出到文件里；后者主要用来检查数据库服务器的工作状态，以及执行一些数据库管理相关的任务，如通知服务器关闭。MySQL发行版本里还包括一些其他的客户端程序。此外，MySQL还提供了一个客户端开发库，如果它自带的标准客户端程序无法满足你的应用需求，你可以利用这个库来编写自己的程序。这个开发库可以被C语言程序直接使用。如果你不习惯C语言，那么也可以选择适用于其他语言（如Perl、PHP、Python、Java和Ruby）的编程接口。

本书讨论的客户端程序都是基于命令行的。你也可以试试MySQL工作台（MySQL Workbench），这是一个基于图形用户界面（Graphical User

Interface, GUI) 的工具, 提供了即点即击 (point-and-click) 功能。关于此工具的更多信息, 请访问<http://www.mysql.com/products/tools/>。

MySQL的“客户端/服务器”体系结构有以下几个好处。

服务器强制执行并发控制, 可以防止两个用户同时修改同一条记录。所有客户端请求都要经过服务器, 因此服务器会负责安排处理它们的先后顺序。即使出现多个客户端同时访问同一个表的情况, 它们也不用先找到对方进行协商。它们只需把自己的请求发往服务器, 然后由服务器来决定执行这些请求的顺序。

不是只有在数据库所在的那台机器上才能登录。MySQL工作在网络环境里, 因此你可以在任意地方运行MySQL客户端程序, 它都能够通过网络连接到服务器。距离不是问题! 你可以在世界的任何地方访问服务器。例如, 你的服务器位于澳大利亚, 那么就算你带着笔记本电脑旅行到了冰岛, 你也仍然可以访问自己的数据库。这是否意味着任何人都能够通过互联网看到你的数据呢? 答案是“不能”。MySQL有一套灵活的安全机制, 你可以设定只有得到授权的人才能访问。此外, 你还可以进一步限制这些人的操作。例如, 财务部的Sally应该有查看和更新(修改)记录的权限; 而服务台的Phil却只应该有查看记录的权限。总之, 你可以把这种访问权限控制细化到每一个人。如果你只想运行一个自属的系统, 那么你完全可以把访问权限设置为只允许服务器上的客户端程序进行连接。

MySQL与mysql之间的差异

为避免混淆, 我们使用MySQL指代完整的MySQL RDBMS, 而mysql指代特定客户端程序的名字。虽然它们的发音相同, 但我们通过字母大小写和字体差异对它们进行了区分。

说到发音, MySQL被读作“my-ess-queue-ell”。具体细节可以在MySQL参考手册(MySQL Reference Manual)里查阅到。此外, SQL的读法有“sequel”和“ess-queue-ell”两种, 具体是哪种由读它的人决定。本书假定SQL的发音为“ess-queue-ell”。

1.4 MySQL教程

现在你已具备了所有的预备知识。下面来实际操作一下MySQL。

本节提供了一个带有多个示例的教程, 其目的在于帮助你熟悉MySQL。主要内容包括创建一个示例数据库和多个表, 通过对这些表里的信息执行插

入、检索、删除和修改操作，练习与示例数据库的交互。在此过程中，你将学会以下内容。

了解MySQL所能理解的SQL基础知识。（MySQL与其他RDBMS所使用的SQL有所不同，因此你最好也能快速浏览一下本节的内容，从而确认一下MySQL的SQL实现与你熟悉的版本是否存在差异。）

了解MySQL自带的标准客户端程序是如何与MySQL服务器进行通信的。前一节讲过，MySQL采用的是“客户端/服务器”体系结构。其中，服务器运行在数据库所在的机器上；而客户端则是通过网络连接到服务器。本教程主要依赖于客户端程序mysql，它首先读取你输入的SQL查询语句，把它们发送到服务器执行，然后把执行结果显示在你面前。客户端mysql可以在MySQL所支持的所有平台上运行，并且提供了与服务器进行交互的最直接的方式。不过根据需要，有些示例会使用mysqlimport或者mysqlshow来代替。

本书将示例数据库命名为sampdb。如果在你的系统上已经有人占用了这个名字，或者你的MySQL管理员为你分配了另一个名字，那么你需要换一个名字。不管是何种情况，你都需要把本书示例中的sampdb替换为你实际使用的数据库名称。

即使你的系统里有多个用户，且他们都各自拥有自己的示例数据库，本节所有示例里的表名也都可以直接使用。在MySQL里，只要每个人都用自己的数据库，那么大家完全可以使用相同的表名，不会有任何问题。MySQL将表限制在各自的数据库里，防止了相互干扰。

1.4.1 获取示例数据库发行包

本教程在好几个地方都要用到“示例数据库发行包”（也称作“sampdb数据库的发行版”）。该发行包里包含了许多用来安装示例数据库的查询语句和数据。发行包的获取办法和安装步骤可以在附录A里查到。在发行包解压之后，它会自动创建一个名为sampdb的子目录，其中包含了你所需要的许多文件。顺便提个建议：每次你操作数据库里的示例时，最好都能切换到一个新的目录。

如果想要在任何目录里都可以方便地运行MySQL程序，那么你最好能把位于MySQL目录下包含着这些程序的bin目录，添加到命令解释器的搜索路径里。具体操作方法是，参考本书A.3.3节，将该目录的路径名称添加到环境变量PATH里。

1.4.2 基本配置要求

为练习本教程里的各个示例，除获得示例数据库发行包外，还必须满足以下几项基本要求。

必须安装MySQL软件。

要有一个能够连接数据库服务器的MySQL账号。

要有一个用来操作的数据库。

所需的软件包括MySQL客户端和MySQL服务器。客户端程序必须安装在你自己要用的机器里。服务器可以选择安装在你的机器上，这点不强求。只要你有权限连接它，那么服务器可以位于任何地方。如果你的因特网服务提供商（Internet Service Provider, ISP）提供了MySQL服务，那么也可以申请使用它。想要自己获得和安装MySQL，请参考附录A。

除了MySQL软件，还必须要有一个MySQL账户，这样才能连接到服务器，并创建示例数据库及其表。（如果你已有MySQL账户，则可以直接用它。不过，建议你另外建立一个专用于本书学习的账户。）

此时此刻，我们遇到了一个“是先有鸡，还是先有蛋”的难题：为了建立一个用于连接服务器的MySQL账户，你必须要先连接到该服务器。通常情况下，你需要在运行MySQL服务器的主机上，以root用户身份登录，然后执行CREATE USER语句和GRANT语句来创建新的MySQL账户，并为其分配数据库权限。如果你是在自己的机器上安装了MySQL服务器，并且正在运行，那么你可以像下面那样用root身份连接服务器，然后创建一个新的示例数据库管理员账户，其用户名为sampadm，密码为secret。你也可以把其中的名字和密码更改成自己想用的，但在本书后面用到它们的地方也要做相应的修改。

```
% mysql -p -u root
```

```
Enter password: *****
```

```
mysql> CREATE USER 'sampadm'@'localhost' IDENTIFIED BY 'secret';
```

```
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> GRANT ALL ON sampdb.* TO 'sampadm'@'localhost';
```

```
Query OK, 0 rows affected (0.01 sec)
```

命令mysql有一个选项-p，它能让mysql提示输入root用户的MySQL密码。如上例所示，输入的密码会被显示为一串星号，即*****。这里假设你已经为MySQL的root用户设置了密码。如果你还未设置，则请在Enter Password: 提示出现后直接按回车（Enter）键。不过，如果root用户无密码，那么这是个很大的安全漏洞，你应该尽快给它设置一个。想要了解更多

关于CREATE USER语句、GRANT语句、设置MySQL用户账户以及修改密码的信息，请参考第13章。

在创建sampadm账户之后，请输入quit，并按回车键，然后退出mysql程序。

刚才展示那些语句适合于在运行服务器的那台机器上连接MySQL。它们让你可以通过用户名sampadm和密码secret来连接服务器，并让你拥有完全访问sampdb数据库的权限。不过，GRANT语句并不会创建数据库（你可以在创建数据库之前为它分配权限）。我们稍后会讨论与数据库创建相关的操作。

如果你打算通过网络从另一台主机连接到MySQL服务器，那么需要把示例中的localhost更改为你正使用的那台主机的名字。例如，你想要从主机boa.example.com连接到MySQL服务器，那么相应的语句则应类似这样：

```
mysql> CREATE USER 'sampadm'@'boa.example.com' IDENTIFIED BY 'secret';
```

```
mysql> GRANT ALL ON sampdb.* TO 'sampadm'@'boa.example.com';
```

如果你对服务器没有控制权限，无法创建用户，那么请求MySQL 管理员，让他为你建立一个账户。然后，将本书各示例里的sampadm、secret、sampdb 分别替换为管理员分配给你的用户名、密码和数据库名。

1.4.3 建立和断开MySQL服务器连接

为连接到 MySQL服务器，请在命令提示符（即 Unix系统的shell提示符，或者 Windows下的控制台窗口提示符）里调用mysql程序。命令如下：

```
% mysql options
```

本书使用%来表示命令提示符。它是 Unix 系统的其中一个标准提示符；另一个是\$。在Windows系统里，你所看到的提示符有点像C:\>。当输入这些示例里所示的命令时，请不要输入提示符本身。

mysql命令行里的options部分可以为空。但下面这种命令形式更为常见：

```
% mysql -h host_name -p -u user_name
```

在执行mysql时，不用提供所有的选项，但通常至少要指定用户名和密码。下面是全部选项的具体含义。

`-h host_name`（另一种形式是：`--host=host_name`）

指定运行MySQL服务器的那台主机。如果它与你运行mysql程序的那台机器相同，就可以省略此选项。

`-u user_name`（另一种形式是：`--user=user_name`）

指定MySQL用户名。如果你使用的是Unix系统，并且你的MySQL用户名与登录名完全一样，就可以省略此选项。mysql将自动将你的登录名当作MySQL用户名。

在Windows系统上，默认用户名为ODBC，它有可能无法使用。你可以在命令行通过-u选项来指定，也可以通过设置环境变量USER 来添加一个默认用户名。例如，你可以用下面的set命令来指定一个用户名sampadm：

```
C:\> set USER=sampadm
```

如果你通过控制面板（Control Panel）里的系统（System）项目设置环境变量USER，那么该设置对每一个控制台窗口都会起作用，你就不必在命令提示符里执行这条命令了。

`-p`（另一种形式是：`--password`）

此选项会让mysql显示Enter password: 提示符，并要求你输入MySQL密码。例如：

```
% mysql -h host_name -p -u user_name
```

```
Enter password:
```

当你看到Enter password: 提示符时，请输入你的密码。（输入的密码不会显示到屏幕上，以免被人偷看到。）请注意：MySQL密码并不一定与登录Unix或Windows系统的密码相同。

如果你省略选项-p，那么mysql将认为你不需要密码，因此不会提示你输入它。

在命令行上直接提供密码的另一种方式是输入-pyour_pass 选项（另一种形式是：`--password=your_pass`，其中的your_pass即为你的密码）。不过，出于安全的考虑，最好别这样做。因为在输入时，你身边的其他人能看到屏幕上的密码。另外，在Unix系统里，其他用户可能也能够使用系统工具查看到命令行。

如果你确实想要在命令行直接输入密码，那么请注意：在-p选项和后面的密码值之间没有空格。-p 选项的这种行经常会造成混乱，因为它与选项-h和-u的习惯要求有所不同：它们都是与跟在后面的单词相关联的，不管其间是否有空格。

假设MySQL的用户名和密码分别是sampadm和secret。如果运行MySQL服务器的那台主机与你运行 mysql 程序的主机相同，那么你可以省略选项-h，此时mysql 命令会像下面那样连接服务器：

```
% mysql -p -u sampadm
```

```
Enter password: *****
```

在输入完这条命令之后，mysql会显示Enter password:，提示你输入密码，然后你便可以输入它（输入的secret会在屏幕上显示为6个星号*****）。

如果一切顺利，mysql将会显示出一条欢迎消息和一个mysql>提示符，此时表明它在等你发起查询命令。完整的启动过程近似如下：

```
% mysql -p -u sampadm
```

```
Enter password: *****
```

```
Welcome to the MySQL monitor. Commands end with ; or \g.
```

```
Your MySQL connection id is 13762
```

```
Server version: 5.5.30-log
```

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

```
mysql>
```

如果想要连接运行于另一台机器上的MySQL服务器，那么必须使用-h选项来指定其主机名。假设该主机名为cobra.example.com，那么相应的命令如下所示：

```
% mysql -h cobra.example.com -p -u sampadm
```

为简洁起见，在后面大部分显示mysql命令行的示例里，选项-h、-u和-p都会被省略，并且假定你会提供所有必要的选项。在运行其他MySQL程序（如

mysqlshow) 时, 你也会用到这些选项。

在连接到MySQL服务器之后, 你可以随时输入quit命令来终止会话:

```
mysql> quit
```

Bye

也可以输入exit或\q来退出。在Unix系统里, 可以按组合键Ctrl+D来退出。

在刚开始学习MySQL时, 很多人都可能会对它的安全系统感到烦恼, 因为它很难操作。(必须要有创建和访问数据库的权限; 还有, 不管何时连接服务器, 都必须指定正确的用户名和密码。) 不过, 在练习完本书提供的示例数据库, 并开始输入和使用自己的数据记录之后, 你的看法便会迅速发生转变。到那时, 你就会欣赏MySQL的这种方式, 因为它可以防止他人窥视(或者更为糟糕的情况, 破坏) 你的信息。

如何设置工作环境, 才能不用在每次运行mysql时都需要在命令行指定连接参数? 关于这个问题请参考 1.5 节。简化服务器连接过程的最常见办法是, 将连接参数存放到一个选项文件里。如果想现在就了解如何建立这样的文件, 那么请直接跳转到1.5节。

1.4.4 执行SQL语句

在连接上服务器以后, 你便可以发起SQL语句, 让服务器执行它。本节将介绍一些与mysql进行交互的一般原则。

想要在mysql里输入语句, 只需要直接输入即可。在语句的结尾, 请输入一个分号 (;), 并按下回车键。该分号会告知 mysql——语句到此结束。在输入完一条语句之后, mysql 会将它发送到服务器执行。服务器接着处理它, 并把结果发送回mysql, 然后mysql会显示结果。

下面这个示例展示了一条用于查询当前日期和时间的简单语句:

```
mysql> SELECT NOW();
```

```

-----
| NOW()
|
-----
| 2013-01-08 17:42:33
|
-----

```

1 row in set (0.00 sec)

除使用分号外，还有另一种终止语句的方法，即使用\g（意思是go）：

mysql> SELECT NOW()\g

```

-----
| NOW()
|
-----
| 2013-01-08 17:42:40
|
-----

```

1 row in set (0.00 sec)

也可以使用\G。它会以“垂直”方式显示结果，每行显示一个值：

mysql> SELECT NOW(), USER(), VERSION()\G

***** 1.row *****

NOW() : 2013-01-08 17:54:24

USER() : sampadm@localhost

VERSION(): 5.5.30-log

1 row in set (0.00 sec)

如果语句的输出行比较短，那么\G就没什么用处；但如果输出行很长，而且会在屏幕上回绕显示，那么使用\G便能让输出内容更易于阅读。

如上所示，mysql 会显示语句结果和一行统计信息，其中包括该结果所包含的行数，以及语句执行所花费的时间。在后面的示例里，一般情况下将不再显示统计信息行。

因为mysql会等待语句结束符，所以你不用在单独一行里输入一条语句的全部内容。你可根据需要，将它分隔成多行进行输入：

```
mysql> SELECT NOW(),
```

```
-> USER(),
```

```
-> VERSION()
```

```
-> ;
```

NOW()	USER()	VERSION()
2013-01-08 17:54:56	sampadm@localhost	5.5.30-log

请注意，在输入该语句的第一行时，提示符从mysql>变为了->。这是在提醒你：mysql认为你还要继续输入语句内容。这是个很重要的反馈。如果你忘了在语句末尾加上分号，那么这个变化的提示符会提醒你注意：mysql 仍在耐心等待你继续输入语句内容。否则，你会很不耐烦地等在一边，心里疑惑：为何MySQL执行你的语句要花这么长的时间。命令mysql还有另外几种提示符，附录F对它们进行了详细介绍。

如果你已经输入了一条多行语句，但突然决定不执行它了，那么可以输入\c来清除（即取消）它：

```
mysql> SELECT NOW(),
```

```
-> VERSION(),
```

```
-> \c
```

```
mysql>
```

请注意，提示符是如何变回mysql>的。这种变化表明mysql已准备接收一条新的语句。

与将一条语句分成多行输入相反的操作是，在单独一行里输入多条语句，两条语句中间用终止符隔开：

```
mysql> SELECT NOW();SELECT USER();SELECT VERSION();
```

```
-----  
NOW()  
-----  
2013-01-08 17:55:20  
-----  
-----  
USER()  
-----  
sampadm@localhost  
-----  
-----  
VERSION()  
-----  
5.5.30-log  
-----
```

大多数情况下，输入语句使用大写、小写或者大小写混用都可以。例如，下面几条语句检索到的是相同的信息（虽然显示结果里列标题的大小写有所不同）：

```
SELECT USER();
```

```
select user();
```

```
SeLeCt UsEr();
```

本书的所有示例都将用大写字母来表示SQL关键字和函数名，用小写字母来表示数据库、表和列的名字。

当在语句里调用函数时，请不要在函数名和后面的括号之间加入空格。如果有空格，则会导致语法错误。

将多条语句存储在一个文件里可以创建一个SQL脚本，然后让mysql从该文件（而不从键盘）读取语句。请使用shell的输入重定向功能来实现这种操作。例如，假设语句都存储在一个名为myfile.sql的文件里，那么我们可以使用下面这样的命令来执行其中的语句（请记住指定所有必需的连接参数选项）：

```
% mysql < myscript.sql
```

你可以为该文件随便取一个名。我给它加上了一个“.sql”后缀，以表明里面存放的是SQL语句。

像这种调用mysql来执行文件里的语句的做法，会在1.4.7节再次用到，到时，我们会用这种办法来往sampdb数据库里输入数据。与逐条手工输入相比，让mysql从某个文件里读取多条INSERT语句来加载表要方便很多。

本教程的其余部分展示了很多 SQL 语句，你可以自行练习。在这些语句前面都有提示符mysql>作为标志，并且这些示例都提供了语句输出结果。如果你输入与示例显示一样的语句，那得到的输出结果也应该是相同的。前面未加提示符的语句主要是用来做说明用的，你不需要执行它们。当然，如果愿意，执行一下也无妨。提醒一下，请记住在每条语句的末尾加上一个分号作为结束符。

1.4.5 创建数据库

数据库的使用涉及以下几个步骤。

- (1) 创建一个数据库。
- (2) 在该数据库里创建多个表。
- (3) 对表里的数据执行插入、检索、修改或删除操作。

想要创建新的数据库，请先使用mysql连接到服务器。然后执行CREATE DATABASE语句，并指定新的数据库名字：

```
mysql> CREATE DATABASE sampdb;
```

在创建可进入或对其内容进行操作的表之前，必须先创建sampdb数据库。

你可能会希望在创建某个数据库的同时，让它成为默认（或当前）数据库。但这是行不通的。看看下面这条检查默认数据库的语句，你就会明白这一点：

```
mysql> SELECT DATABASE();
```

```
-----  
| DATABASE ()  
-----  
| NULL  
-----
```

NULL表示“未选择到数据库”。如果想要把sampdb设置为默认选择数据库，那么还需要执行一条USE语句：

```
mysql> USE sampdb;
```

```
mysql> SELECT DATABASE();
```

```
-----  
| DATABASE ()  
-----  
| sampdb  
-----
```

另一种选择默认数据库的办法是，调用mysql的时候在命令行上给该数据库取一个名字：

```
% mysql sampdb
```

事实上，这就是在选择数据库时用得最多的办法。如果还需要使用连接参数，那么可以在命令行指定它们。例如，下面的命令可以让用户sampadm连接到本地主机（如果未指定主机名字，则默认是它）上的sampdb数据库：

```
% mysql -p -u sampadm sampdb
```

如果需要连接到运行于远程主机上的MySQL服务器，那么需要在命令行指定该主机：


```
% mysql -h cobra.example.com -p -u sampadm sampdb
```

如果无特别说明，后面所有的示例都将假定：当调用mysql时，命令行里指定的默认数据库就是sampdb。如果在调用mysql时忘记在命令行指定数据库，那么请在mysql>提示符处输入一条USE sampdb语句。

1.4.6 创建表

本节将创建示例数据库 sampdb 所需要的那些表。首先，创建“美史联盟”场景所需要的表，然后，再为“成绩考评”项目创建所需的表。有些数据库书讲到在此处便会开始讨论“数据库的分析与设计”、“实体联系图”、“规范化过程”（Normalization Procedure）等内容。有很多书专门讲解这些内容，所以本书在这里只想说明我们的数据库应该是个什么样子，具体来讲涉及这样几个方面：它应该包含哪些表、每个表都应包含什么内容，以及在决定如何表示这些数据时需要考虑哪些问题。

这里所选择的表示方式并不是绝对的。在其他场合，你可以选用不同的方式来表示相似的数据。到底选择哪一种，需要由应用程序和数据的具体用途来决定。

1.4.6.1 美史联盟表

美史联盟的表相当简单，包括以下两个表。

president表。其中包含美国历任总统的描述性记录。我们需要用它来实现联盟网站上的在线小测验（对联盟通讯儿童专栏里出现的小测验进行交互式模拟）。

member表。用于保存联盟每位成员的最新个人资料。我们可以用它来创建成员名录的印刷版本和在线版本，用它来向到期成员自动发送提醒通知，还可以用它做很多其他事情。

1. president表

president表包含一些与美国历任总统生平相关的基本信息。

姓名。在表里，表示姓名的方式有好几种，如使用单列包含整个名字，或用不同的列分别表示姓（last name）和名（first name）。使用单列来表示当然更简单一些，但这种做法不够灵活，存在一些限制。

如果优先输入名，就无法按姓排序。

如果优先输入姓，就无法按名在前、姓在后的顺序显示它们。

难以查找姓名。例如，想要查找某人的姓，那么必须先使用一种模式，然后去查找与之匹配的姓名。与直接查找姓的做法相比，这种做法的效率很低，速度很慢。

为避免出现这些限制，`president`表将使用两列分别表示姓和名。

那个“名”列还要存放中间名或其缩写。这种做法不会影响到以后的姓名排序操作，因为我们不太可能对中间名进行排序（也不太可能对名进行排序）。另外，姓名也可以正常显示，因为无论是以“Bush, George W.”或“George W. Bush”中的哪一种格式来输出，姓名里的中间名总是排在名字的后面。

还有一点需要注意。如果一位总统（如Jimmy Carter）的姓名后面带有一个“Jr.”，那么这个“Jr.”应该放在哪里呢？根据英文姓名输出格式习惯，这位总统的名字可以显示为“James E. Carter, Jr.”，或者“Carter, James E., Jr.”。这里的“Jr.”与名和姓没有联系，因此我们还需要另外创建一个列，用它来保存姓名后缀。这种情况表明：当你在选择数据的表示形式时，即使只是一个单值，也可能会引发某些问题。它同时还表明：在把数据值放入数据库之前，最好尽可能多地对数据值进行深入的了解。如果对数据值的真正含义了解不够，那么在表启用之后，可能还要被迫更改它的结构。虽然这种事不能算作是一场灾难，但总的说来还是尽量避免为妙。

出生地（城市和州）。与姓名的情况类似，它既可以用一列来表示，也可以用多列表示。采用单列来表示的做法显得更简单些；但与姓名的情况一样，使用多列能实现某些更复杂的操作。例如，若把州名与市名分开表示，那么像“找出出生在某个州的总统共有多少位”这种类似的操作便能轻易地实现。我们将使用两个单独的列来分别存放州名与市名。

出生日期和逝世日期。这里唯一需要特殊处理的事情是：因为有些总统依然健在，所以我们不能要求必须填上逝世日期。特殊值NULL的意思即表示“无值”，因此我们可以在逝世日期列里用它来表示该位总统“依然健在”。

2. member 表

从每条记录都保存着单独某个人的个人资料这一角度来看，存放“美史联盟”成员列表的member表与刚才介绍的president表很相似。只是member表的每一行还包含了以下这些列。

姓名。我们将沿用与president表相同的3列表示法：姓、名和姓名后缀。

ID编号。这是一个唯一值，为每个首次加入的成员分配一个。联盟此前从未对成员编过号，但现在需要所有的记录都更系统化，因此这里需要这个值。（希望你能不断发现MySQL的好处，并找到更多将编号应用于联盟记录的方式。当想要将member表里的行，与你所创建的成员相关的其他表建立关联时，使用编号则会比使用姓名更容易实现。）

有效期。所有成员必须定期更新其成员资格，以避免过期。对于某些应用程序，可能还需要把最近一次资格更新后的起始日期存储起来，但“美史联盟”不需要这样做。成员资格的有效期是一个变值（通常有1年、2年、3年或者5年之分），而最近一次的资格更新日期也并不能说明该成员下一次的资格更新日期一定是在什么时候。因此，我们需要把成员资格的截止日期存储起来。此外，联盟还提供了终身成员资格。虽然我们可以用一个很遥远的日期来表示这种情况，但使用NULL会更合适，因为“无值”在逻辑上正好对应于“永不失效”。

电子邮件地址。公开电子邮件地址可以使兴趣相投的成员交流起来更方便。作为联盟的秘书，在有了这些地址之后，就可以用电子邮件来向成员发送成员资格更新通知，而不用邮寄信件了。与到邮局寄信相比，这种做法既方便又省钱。你还可以利用电子邮件把每位成员的个人最新资料发送给他们，让他们在必要时更新信息。

通信地址。当与那些没有电子邮件（或者长期没有回复你邮件）的成员进行联系时，你会需要这条信息。我们将使用多个列来分别存储街道地址、城市名、州名和邮政编码。

我们假设所有联盟成员都居住在美国。当然，对于那些成员遍布于世界各地的组织机构来说，这个假设可能显得有些简单。如果想要处理涉及多个国家的地址，那么你将遭遇到一些棘手的问题，必须要解决好不同国家所使用的地址格式不同的问题。例如，邮政编码就不是一项国际标准；另外，有很多国家都只有“省”的概念，而无“州”的说法。

电话号码。与地址列相似，主要用于联系成员。

特殊兴趣关键字。联盟的每位成员对美国历史肯定都很感兴趣，但他们的兴趣却可能集中于某些特定的历史时期。此列便是用于记录这些兴趣。每位成员都可以利用这些信息来寻找与自己兴趣相投的其他成员。（严格来讲，建立一个独立的表可能会更好些，表中的每一行都由一个关键字和相关成员的ID组成。这点有些复杂，我们暂不作处理。）

3. 创建美史联盟表

现在，我们准备创建美史联盟表。为此，我们需要使用CREATE TABLE语句，该语句具有以下格式：

```
CREATE TABLE tbl_name (column_specs);
```

其中，tbl_name指的是你要给表指定的名字；column_specs指的是该表的各列。该语句还会包括各种索引的定义，如果有的话。索引能够加快查找速度，关于这一点请参考第5章的介绍。

下面是针对president表的CREATE TABLE语句：

```
CREATE TABLE president
(
    last_name    VARCHAR(15) NOT NULL,
    first_name   VARCHAR(15) NOT NULL,
    suffix       VARCHAR(5) NULL,
    city         VARCHAR(20) NOT NULL,
    state        VARCHAR(2) NOT NULL,
    birth        DATE NOT NULL,
    death        DATE NULL
);
```

执行此语句的方法有两种：可以自己手动输入；也可以使用sampdb 发行包中create_president.sql文件里包含的预先编写好的语句。

如果想要自己输入这条语句，那么请先调用mysql，同时将sampdb设置为默认数据库：

```
% mysql sampdb
```

接着，输入上面的CREATE TABLE 语句。请记得在语句末尾输入分号，以便将这条语句的结束位置告知 mysql 程序。没有缩进格式也没有关系，你不用像上面那样换行。你完全可以在同一行输入整条语句。

如果想用预先编写好的描述来创建 `president` 表，那么可以使用 `sampdb` 发行包里的 `create_president.sql` 文件。当你解压发行包时，解压程序会自动创建一个 `sampdb` 目录，并将那个文件放在这个目录里。先进入该目录，然后执行下面这条命令：

```
% mysql sampdb < create_president.sql
```

无论你采用何种方式调用 `mysql`，都请记得在命令行的命令名后面指定你可能需要的连接参数（其中包括主机名、用户名和密码）。

现在让我们来仔细看一下 `CREATE TABLE` 语句。该语句会为每一列指定列名、数据类型（列要存放的值的类型），以及可能有的某些列属性。

`president` 表用到了两种数据类型：`VARCHAR` 和 `DATE`。`VARCHAR(n)` 表示的是：该列可以存放长度可变的字符型值，且最大长度为 `n` 个字符。也就是说，它们包含了长度不定的字符串，但其长度存在上限。那个 `n` 的值即表明了你所期望的数据长度。例如，将 `state` 定义为 `VARCHAR(2)` 类型，即表示我们需要以两个字母的缩写形式输入州名。而其他值为字符串类型列，其数据长度必须要再长一点才能容纳下更长的值。

我们用到的另一个数据类型是 `DATE`。很显然，此类型表示该列用于保存日期值。不过，日期的表示格式可能会让你感到意外。`MySQL` 要求将日期表示为 `'CCYY-MM-DD'` 格式，其中的 `CC`、`YY`、`MM`、`DD` 分别代表了世纪、世界的年份、月和日。这也是 `SQL` 标准规定的日期表示格式（也叫做 `ISO 8601` 格式）。例如，想要在 `MySQL` 里指定“2013年7月18日”这样一个日期，则需要使用 `'2013-07-18'`，而不能用 `'07-18-2013'` 或 `'18-07-2013'`。

在 `president` 表里，我们用到的列属性只有 `NULL`（可以没有值）和 `NOT NULL`（必须要有值）。大部分数据列的属性都为 `NOT NULL`，因为我们要求它们必须要有值。可以有 `NULL` 属性的两列是：`suffix`（大部分总统的姓名都无后缀）和 `death`（如果总统还健在，则无逝世日期）。

对于 `member` 表，其 `CREATE TABLE` 语句如下：

```
CREATE TABLE member
```

```
(
```

```
member_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
```

```
PRIMARY KEY (member_id),
```

```
last_name  VARCHAR(20) NOT NULL,  
  
first_name VARCHAR(20) NOT NULL,  
  
suffix     VARCHAR(5) NULL,  
  
expiration DATE NULL,  
  
email      VARCHAR(100) NULL,  
  
street     VARCHAR(50) NULL,  
  
city       VARCHAR(50) NULL,  
  
state      VARCHAR(2) NULL,  
  
zip        VARCHAR(10) NULL,  
  
phone      VARCHAR(20) NULL,  
  
interests  VARCHAR(255) NULL  
  
);
```

与之前一样，你可以手动将这些语句输入mysql中，也可以利用预先编写好的文件。发行包sampdb中的creat_member.sql文件包含了member表的CREATE TABLE语句。要使用它，需要执行以下命令：

```
% mysql sampdb < create_member.sql
```

就数据类型而言，member表里的大部分列（有两个例外）的类型都很普通，都是可变长度的字符串类型。列member_id和expiration是例外，它们分别保存的是序号和日期。

使用成员资格编号列member_id的主要考虑是：为了避免成员之间产生混淆，其中每一个值都应该是唯一的。此时，AUTO_INCREMENT列正好可以派上用场，因为当我们添加新成员时，MySQL便能为我们自动生成一个唯一编号。虽然 member_id 列只是包含数字，但其定义却包含有好几个部分。

INT。它表示该列用于存放整数（无小数部分的数值）。

UNSIGNED。它表示该值不能为负数。

NOT NULL。它表示该列必须要填值，这可以防止创建的成员没有ID号。

AUTO_INCREMENT。它是MySQL的一个特殊属性，表示该列存放的是序号。

AUTO_INCREMENT的工作原理为：当往member表里添加新记录时，如果没有为member_id列提供值，那么MySQL将自动生成下一个编号，并将它赋给该列。如果你显式地将NULL赋给该列，结果也是一样的。AUTO_INCREMENT的这种特性使得为每一位成员分配一个唯一的ID变得很简单，因为MySQL会替我们生成这些值。

PRIMARY KEY子句表示为member_id列创建索引，这样能加快查找速度。同时，它还设置了约束：要求该列里的所有值都必须唯一。后面这个属性正好符合对成员ID值的要求，因为它可以防止我们两次误用相同的ID。此外，MySQL要求AUTO_INCREMENT列必须有某种索引，如果没有索引，那么表的定义就是不合法的。（任何一个PRIMARY KEY 列也必须是 NOT NULL的，因此就算我们在member_id定义里省略了NOT NULL，MySQL也会自动加上。）

如果你还不太明白AUTO_INCREMENT和PRIMARY KEY是怎么回事，那么可以把它们想象成一种能生成索引ID号的神奇魔法。这些值到底是什么并不重要，只要这些ID号对每位成员来说都是唯一的就可以了。（关于AUTO_INCREMENT列的更多信息请参考第3章。）

expiration列的数据类型是DATE。它允许值为NULL，且默认也为NULL，即表示可以不输入日期。正如前面所提到的，我们将使用这样一个约定：当expiration值为NULL时，表明该成员拥有终身成员资格。

到目前为止，你已经让 MySQL 创建了两个表，现在让我们来确认一下结果是否正确。在mysql里，调用下面这条命令可以查看president表的结构：

```
mysql> DESCRIBE president;
```

Field	Type	Null	Key	Default	Extra
last_name	varchar(15)	NO		NULL	
first_name	varchar(15)	NO		NULL	
suffix	varchar(5)	YES		NULL	
city	varchar(20)	NO		NULL	
state	varchar(2)	NO		NULL	
birth	date	NO		NULL	
death	date	YES		NULL	

调用DESCRIBE member语句，则可以看到有关member表的类似信息。（如果你想知道为什么未定义为允许NULL的Default列会显示NULL，那么我在这里告诉你，那是因为NULL也可用于表明该列没有显式的DEFAULT子句。）

如果你不记得一个表中的列名、想知道其数据类型，或者想了解其宽度是多少，那么可以使用DESCRIBE。你还可以利用它来查看 MySQL 在表行里存储各列的先后顺序。当你执行INSERT或LOAD DATA语句时，这个顺序很重要，因为这些语句要求各列的值是以它们的默认列顺序列出的。

你也可以通过其他方式来获得 DESCRIBE 生成的信息。既可以是简写的DESC，也可以是EXPLAIN语句或SHOW语句。下面这些语句具有相同的作用：

```
DESCRIBE president;
```

```
DESC president;
```

```
EXPLAIN president;
```

```
SHOW COLUMNS FROM president;
```

```
SHOW FIELDS FROM president;
```

这些语句还允许对输出列加以限制。例如，可以在SHOW语句的末尾加上一个LIKE子句，这样便只能看到与给定模式相匹配的那些列的信息：

```
mysql> SHOW COLUMNS FROM president LIKE '%name';
```


Field	Type	Null	Key	Default	Extra
last_name	varchar(15)	NO			
first_name	varchar(15)	NO			

使用DESCRIBE president '%name'也可以得到同样的结果。这里用到的百分号(%)是一个特殊的通配符，我们将在后面的1.4.9.7节中介绍它。

SHOW FULL COLUMNS与SHOW COLUMNS很像，不同之处在于它会显示附加的列信息。你可以现在试一下，看看结果。

SHOW语句还有其他几种形式，可用于从MySQL获得不同类型的信息。SHOW TABLES语句会列出默认数据库里的表。到目前为止，我们已在数据库 sampdb里创建了两个表，因此执行该语句将得到如下输出：

```
mysql> SHOW TABLES;
```

```

Tables_in_sampdb
+-----+
member
president

```

SHOW DATABASES语句会列出当前连接到的服务器上的所有数据库：

```
mysql> SHOW DATABASES;
```

```
Database
-----
information_schema
mysql
sampdb
test
```

这个列表可能会因服务器不同而有所差异，但是应该能看到 `information_schema` 和 `sampdb`。其中，`information_schema` 是一个事先存在的特殊数据库；而 `sampdb` 则是刚才创建的。另外还有一个数据库 `test`，它是 MySQL 在安装过程中创建的。如果你的访问权限足够高，你可能还会看到名为 `mysql` 的数据库，它是一个权限分配表。

客户端程序 `mysqlshow` 提供了一个命令行接口，通过它所获得的信息和使用 `SHOW` 语句所显示的一样。请记住：当你运行 `mysqlshow` 时，还需要提供正确的命令行选项，其中包括用户名、密码和主机名。这些选项与你在运行 `mysql` 时所用的一样。

当不带参数时，`mysqlshow` 会显示出一个数据库列表：

```
% mysqlshow
```

```
Databases
-----
information_schema
mysql
sampdb
test
```

在带上数据库名之后，`mysqlshow` 会列出给定数据库里的所有表：

```
% mysqlshow sampdb
```

```
Database: sampdb
```

```
Tables
-----
member
president
```

如果同时带上数据库名和表名，那么mysqlshow会显示出该表里各列的信息，这时等同于SHOW FULL COLUMNS语句。

1.4.6.2 成绩考评项目表

要想确定成绩考评项目需要用到哪些表，得先弄清在用纸质成绩册来记录考生成绩时可能会怎么做。图1-2展示了成绩册中一页的内容。该页的主体部分是用于记录分数的表格。该页里面还包含了其他一些让分数有意义的信息。学生的姓名和ID号列在表格的左侧。（为了简洁，这里只列出了4位学生。）考试或测验的举行日期则列在表格的顶部。从该表格可以看出：在9月的3、6、16、23日有测验；在9月9日和10月1日有考试。

students		scores						
ID	name	Q	Q	T	Q	Q	T	
		9/3	9/6	9/9	9/16	9/23	10/1	...
1	Billy	14	10	73	14	15	67	...
2	Missy	17	10	68	17	14	73	...
3	Johnny	15	10	78	12	17	82	...
4	Jenny	14	13	85	13	19	79	...
...	...	...	...	...	...	...	...	...

图1-2 成绩册示例

为使用数据库持续跟踪这些信息，我们需要一个score（分数）表。那么，在这个表里应该包含哪些行呢？此问题不难回答。在每一行里，需要列出学生姓名、考试或测验日期，以及学生的考试分数。图1-3展示了成绩册里

的部分分数在表里的表现形式。（其中，日期是按MySQL的方式来表现的，即'CCYY-MM-DD'格式。）

score 表

name	date	score
Billy	2012-09-23	15
Missy	2012-09-23	14
Johnny	2012-09-23	17
Jenny	2012-09-23	19
Billy	2012-10-01	67
Missy	2012-10-01	73
Johnny	2012-10-01	82
Jenny	2012-10-01	79

图1-3 最初的score表

可惜的是，采用这种方式得到的表遗漏了一些信息。例如，请仔细看看图1-3 中的各行，它还未清楚地表明该成绩是考试分数还是测验分数。一般来说，在评定的期末总成绩时，考试分数与测验分数的权重是有明显区别的，因此有必要将考分类别标识出来。当然，我们也可以根据某给定日期的分数范围（在数值上，测验分数通常要比考试分数低很多）来推测其类型，但这种不采用具体数据明确标识的做法会带来问题。

可以在每行记录里将各类考分区别开来，具体方法就是：给score表增加一列，用T或Q分别表示考试（test）或测验（quiz），如图1-4所示。这种做法的优点是考分类别能直接体现在数据上；缺点是显得有些冗余。看看那些日期相同的行就能发现，考分类别栏里的值都是相同的。所有9月23日的考分都属于Q类，所有10月1日的考分都属于T类。没人会喜欢这个样子。如果按这种办法来记录学生们的考试分数，那么我们不仅要反复输入相同的日期，而且还要多次输入一个相同的考分类别。天啊！谁会愿意输入这么多的冗余信息呢？

score 表

name	date	score	category
Billy	2012-09-23	15	Q
Missy	2012-09-23	14	Q
Johnny	2012-09-23	17	Q
Jenny	2012-09-23	19	Q
Billy	2012-10-01	67	T
Missy	2012-10-01	73	T
Johnny	2012-10-01	82	T
Jenny	2012-10-01	79	T

图1-4 修改后包含分数类型的score表

我们再来试试另一种表示方式。这次不把考分类别放到score表里，而是将它与考试日期对应起来。我们可以把考试日期列出来，然后用它来跟踪在各个日期发生过什么样的“考试事件”（包括测验和考试）。这样，我们就可以根据 score 表里的日期在grade_event 表里查出当天的考试事件类型，从而得知某个分数是来自测验，还是来自考试。将score表里的日期与grade_event表进行匹配，便能获得考试的类别。图1-5列出了这个表的结构，并展示了它与score表之间是如何与9月23日那天关联起来的。通过将该行与grade_event表里的相应行相匹配，我们便能看出该分数是一次测验成绩。

score 表			grade_event 表	
name	date	score	date	category
Billy	2012-09-23	15	2012-09-03	Q
Missy	2012-09-23	14	2012-09-06	Q
Johnny	2012-09-23	17	2012-09-09	T
Jenny	2012-09-23	19	2012-09-16	Q
Billy	2012-10-01	67	2012-09-23	Q
Missy	2012-10-01	73	2012-10-01	T
Johnny	2012-10-01	82		
Jenny	2012-10-01	79		

图1-5 score表与grade_event表，通过日期链接

与通过推测来判断考分类别的做法相比，这种方法更好一些。因为现在能够从记录在数据库里的数据直接得出考试分数的类型。与将考分类别直接

记录在score表里的做法相比，这种方法也更易让人接受。现在，我们只需记录一次考分类别，再也不用为每个考分都记录一次了。

不过，我们现在需要把多个表的信息进行组合。如果你是我，那么在第一次听说这种事的时候，或许会想：“嘿，这个主意真不错。但是，有这么多的表，想查什么东西会不会太费事？这会不会把事情搞得更复杂了？”

在某种程度上，这种担心是有道理的，它需要做更多的工作。记录两个表肯定会比记录一个表复杂。但是，请再仔细看看当初的成绩册（如图1-2所示），你不是已经在记录两组信息了吗？请看下面两个事实。

你需要把考试成绩记录在表格的每一个格里，这些格都要按学生姓名和考试日期进行排列（姓名由上往下排列，考试日期则由左往右排列）。这正是我刚才讲过的两组信息当中的一组，而它正好对应于score表里的内容。

你要如何才能知道各日期所对应的考试类型呢？你或许会这样做：在日期上方写一个T或Q。于是，你又在表格的顶部把考试日期与考试类型关联起来了。这正是我刚才讲过的两组信息当中的第二组，而它也正好对应于grade_event表里的内容。

换句话说，尽管你可能还未意识到这一点，但你在成绩册里所做的事，与我把信息放到两个表里的做法并无多大差异。唯一的区别在于，纸质成绩册里的两类信息没有明确地分离开来。

成绩册里的页的概念体现了我们对信息的思维方式，同时这也表明：把信息妥善地放到数据库里并不是一件简单的事情。在日常生活中，人们习惯于把不同信息综合起来，然后把它们当作一个整体来考虑。但数据库无法那样工作，这也正是它们看起来不太自然的原因。我们习惯把信息统一起来，这也使得我们有时很难清楚地分辨出自己所拥有的信息是有多种类型，还是只有一种。正因如此，“以数据库系统的方式进行思考”，考虑如何表现数据才具有挑战性。

图1-5里的grade_event表还隐含了这样一个要求：所有的日期必须唯一，因为每个日期要用于链接score和grade_event两个表里的各个行。换句话说，不能在同一天进行两场测验，或者一次测验加一次考试。如果这样做的话，那么对于同一个日期，在score表里将会出现两组记录，而在grade_event表里也会有两条记录，而且你也无法说清如何将score中的这些行与grade_event中的那两行进行匹配。

假如你每天最多只进行一场考试，那么这个问题就绝不会出现。但是，可否假设这一情况永远不会发生呢？似乎可行。毕竟，心地善良的你应该不会对学生过于苛刻，要对他们每天进行两场考试。不过，我还是会经常听到

有人声称：对于他们的数据，“这种情况永远不会发生”。然而，事实证明，这种情况偶尔也会出现；而这时为了弥补这一奇怪情况所引发的各种问题，你便不得不重新设计相关表。

最好能防患于未然，事先想好如何处理这些问题。因此，我们现在假设：你有时也会需要在同一天记录两组分数。这一问题该如何解决呢？事实证明，这个问题并不是那么难以解决。只需对有关数据的布局结构作一点小小的改动，就能处理在同一天存在多个事件的情况了，如下所示。

(1) 在grade_event表里增加一列，用它来为该表里的每一行分配一个唯一的编号。这样，每次事件就都拥有自己的ID编号了，我们可以将这一列命名为event_id。这种做法看起来有点奇怪，不过在图1-2的成绩册里其实已经隐式地用到了这个属性：这里的事件ID与成绩册分数表格里的列序号是相当的。虽说这个列序号未被显式地写出来，并标注为“event ID”，但它实际就是列序号。

(2) 当把各个分数放到score表中的时候，记录事件ID，不记录日期。

完成上述改动之后，我们会得到图1-6所示的结果。现在要用event_id来链接score表和grade_event表，不再使用date了。使用grade_event表不仅能查出每个考分的类别，还能查出它具体发生的日期。还有，在grade_event表里，具有唯一性的不再是日期，而是事件ID。这意味着，在同一天可以进行多次考试和测验（在听到这个消息之后，你的学生肯定会欣喜若狂），而且你也能把它们直接记录下来。

score 表

name	event_id	score
Billy	5	15
Missy	5	14
Johnny	5	17
Jenny	5	19
Billy	6	67
Missy	6	73
Johnny	6	82
Jenny	6	79

grade_event 表

event_id	date	category
1	2012-09-03	Q
2	2012-09-06	Q
3	2012-09-09	T
4	2012-09-16	Q
5	2012-09-23	Q
6	2012-10-01	T

图1-6 score表与grade_event 表，通过事件ID链接

必须承认，图1-6里的表结构不如前面的那几个看起来顺眼。score 表变得更加抽象，因为它包含的列越来越让人看不懂。请看图1-4里的score表，

里面既有考试日期又有考分类别，让人一眼即能看明白。但在图1-6所示的score表里，那两列都不见了，我们看到的是一个高度抽象化的信息表示形式。谁愿意看一个只包含“事件ID”的score表呢？它毫无意义。

此时此刻，我们到了一个十字路口。大家之前还对电子化的成绩考评系统充满希望，觉得很快就能从繁琐的评分工作中解脱出来。可是，在了解到“在数据库里分数信息实际上是如何表示的”之后，你却因这些信息表示起来相当抽象和分散而开始踌躇不前。

这自然会引出一个问题：“也许MySQL不适合我。不用数据库会不会好些？”想必大家已猜到了，我对此持反对意见。原因很简单，你看本书的厚度就知道了。但是，当你在考虑如何开始某项工作之前，多考虑几种情况，以及想一下“如果使用像MySQL这样的数据库系统，或者使用其他像电子表格程序那样的工具，事情是否会变得更好”，这些都是不错的做法。对比情况如下。

成绩册由行和列构成，电子表格也是如此。这使得它们在概念和外观上都很相似。

电子表格程序能够执行计算，因此可以使用计算字段来统计每个学生的分数。将测验分数和考试分数按不同权重来统计可能会有点棘手，但相信你能办到。

另一方面，如果你想只对一部分数据进行操作（如只统计测验分数，或者只统计考试分数），进行对比分析（如男生与女生的对比），或者想以灵活方式显示统计信息，那么情况就有所不同了。这些工作不是电子表格所擅长的，而关系数据库系统却能轻易地完成。

另外一个需要考虑的地方是：在关系数据库里表示具有抽象和分散特性的数据，也不是什么大问题。在数据库建立之初，仔细考虑好信息在数据库里的表示方式是很有必要的，这样你才能按照最符合你目标的方式来安排你的数据。不过，确定好如何表示信息之后，你便需要依赖数据库引擎来收集数据，并把它以一种对你来讲很有意义的方式呈现出来。这样，你所看到的就不会是一组分散的数据块了。

例如，当从score表检索分数时，你想要看考试日期，而不想看到事件ID。这点很容易做到。数据库将在grade_event表里根据事件ID查出考试日期，并显示在你面前。你可能还想要知道考试分数是属于测验的，还是属于考试的。这点也很容易做到。数据库可以采用同样的方式（根据事件ID）查出考分类别。别忘了，像MySQL这类的关系数据库系统最擅长的就是：将一个事物与另一样事物进行关联，从而在多个信息源里把你最想知道的信息提取

出来。在成绩考评这个示例里，MySQL会负责考虑通过事件ID将信息汇集到一起，你不必去关心其中的细节。

现在，为了提前让大家了解到如何让 MySQL 实现这种事物之间的关联，假设你想要查看2012年9月23日的考试分数。下面这个查询可以将指定日期的考试分数查出来：

```
SELECT score.name, grade_event.date, score.score, grade_event.category  
  
FROM score INNER JOIN grade_event  
  
ON score.event_id = grade_event.event_id  
  
WHERE grade_event.date = '2012-09-23';
```

相当可怕，哈？这个查询通过将score表中的各行和event表中的各行进行连接（关联），检索得出学生姓名、考试日期、考试分数和考分类别等信息。结果如下所示：

name	date	score	category
Billy	2012-09-23	15	Q
Missy	2012-09-23	14	Q
Johnny	2012-09-23	17	Q
Jenny	2012-09-23	19	Q

是不是觉得上面这个表格有点面熟？你应该很熟悉，它与图1-4 里的表格布局是一样的。你不必知道事件ID就能获得结果。你只要指定你感兴趣的那个日期，然后MySQL便会根据这个日期将考试分数找出来。因此，当从数据库里以某种对我们很意义的形式将信息提取出来的时候，如果你还在担心那种抽象和分散是否会让我们迷失，那么到这个时候你应该可以看到根本不会出现这种情况。

当然，在仔细查看该查询之后，你或许又会产生一些新的疑问。换句话说，这个查询看起来又长又复杂。只是要查出某一天的考试分数，就要写得这么复杂？没错，它是有点复杂。不过，在每次你想要调用某个查询时，也

有很多方法可以避免输入占用很多行的SQL语句。一般情况下，在执行完某个类似这样的查询之后，你可以将它保存起来；然后，在必要时你便可以轻易地重复使用它。关于这一做法的更多内容请参考1.5节。

为了让大家对查询过程先有所了解，我提前展示了这个示例。事实上，与我们真正用来检索考试分数的查询相比，它算是简单的。因为我们还需要对表的结构再做一次较大的改动。首先，我们将score表里的学生姓名替换成具有唯一性的学生ID。（也就是说，我们将使用成绩册里“ID”栏的值，而不使用“名称”栏里的值。）我们还要另外新建一个名为student的表，其中包含学生姓名（name）和学号（student_id）两列（见图1-7）。

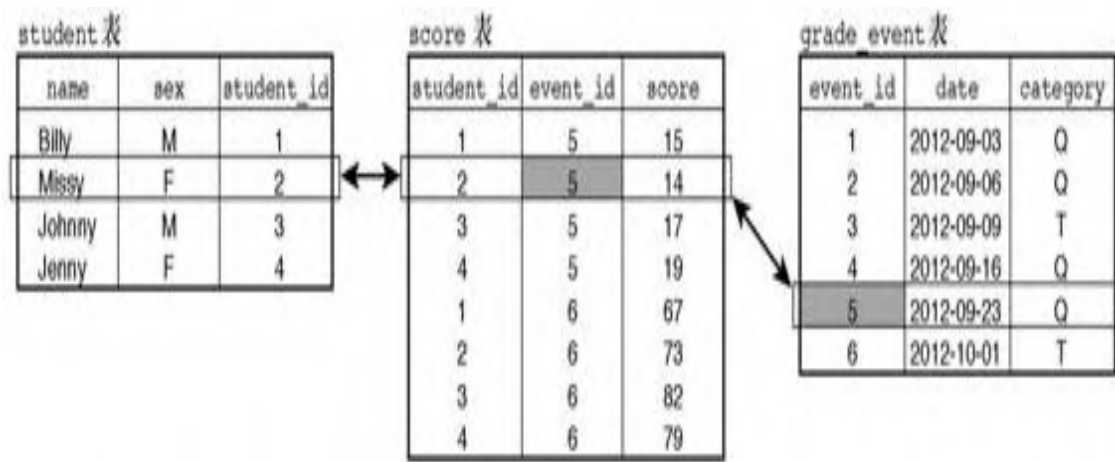


图1-7 score表、student表和grade_event表，通过学生ID和事件ID链接

为什么要做这样的改动呢？只为解决可能会出现的两名学生同名的情况。使用唯一的学生ID编号有助于把他们的分数区别开来。这与我们不用日期而是使用唯一的事件ID来区分同一天进行考试和测试的分数是一样的道理。在对表的结构做了上述改动之后，根据指定日期查询考试分数的语句又复杂了一点：

```
SELECT student.name, grade_event.date, score.score, grade_event.category
FROM grade_event INNER JOIN score INNER JOIN student
ON grade_event.event_id = score.event_id
AND score.student_id = student.student_id
WHERE grade_event.date = '2012-09-23';
```

如果你现在为看不懂这个查询命令而担心，那么请放松。大部分人都看不懂。在本教程的后半部分，我们还会遇到这个查询命令，不过前后的差异在于后面的那个版本会让你眼前一亮。真的，不开玩笑。

从图1-7里可以看到，student 表里增加了一些在成绩册里没有的内容：它多了一个用于记录性别的列。可以利用这个列来统计班级里男女生的人数；也可以用它来做一些复杂的事情，如比较男女生的成绩。

至此，我们几乎完成了成绩考评项目所有表的创建。最后还需再增加一个表，用来记录考勤情况。这个表的内容相对比较简单，只包含一个学生ID和一个日期（见图1-8）。这个表的每一行都代表了一位在指定日期有缺席的学生。到期末的时候，我们将利用MySQL的统计功能来对表里的数据进行汇总，从而统计出每位学生的缺勤天数。

absence 表

student_id	date
2	2012-09-02
4	2012-09-15
2	2012-09-20

图1-8 absence表

1. student表

现在，我们知道了那些成绩考评表的样子，下面来创建它们。用于创建student表的CREATE TABLE语句如下：

```
CREATE TABLE student  
  
(  
  
    name          VARCHAR(20) NOT NULL,  
  
    sex           ENUM('F','M') NOT NULL,  
  
    student_id INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  
    PRIMARY KEY (student_id)  
  
) ENGINE = InnoDB;
```

请注意观察，我在CREATE TABLE 语句里加入了一些新的内容（在末尾加上了ENGINE 子句）。稍后我会解释它的用途。

你可以在mysql客户端程序里输入上面的CREATE TABLE语句，也可以在命令行里执行下列命令：

```
% mysql sampdb < create_student.sql
```

这条 CREATE TABLE 语句将创建一个名为 student 的表，其中包含三列：name、sex 和 student_id。

name 是一个长度可变的字符串列，它最多可以存储 20 个字符。这里的名字表示比“美史联盟”表里的简单，它只使用了一个单列，并没有将名和姓单独分开。这样做的原因在于：我事先知道成绩考评查询示例不需要用到多个列来表示名字。（没错，这是骗人的。我承认！实际上你可能需要使用多个列。）

sex 用于表明某位学生是男生还是女生。这是一个 ENUM（枚举）列，其取值只能是在该列的规范里列出的那些值当中的一个：'F' 代表女生，'M' 代表男生。当你想把某列的可取值限定在某个有限集合内时，ENUM 会非常管用。当然，我们也可以把该列定义为 CHAR(1)，但 ENUM 可以让被允许的列值更加明确。如果你忘了它都有哪些可取值，就可以调用 DESCRIBE 命令来查看。MySQL 会列出 ENUM 列的合法枚举值：

```
mysql> DESCRIBE student 'sex';
```

Field	Type	Null	Key	Default	Extra
sex	enum('F','M')	NO			

ENUM 列的值不一定非得是单个字符。例如，还可以把 sex 列定义为：ENUM ('female', 'male')。

student_id 是一个整型列，用于保存唯一的学生 ID 编号。通常情况下，ID 编号来源于某个权威机构，如学校办公室。但本书出于示例目的，将自己编造它们。我们将使用一个 AUTO_INCREMENT 列，其定义与前面创建 member 表时所用的 member_id 列相类似。

如果的确需要从学校办公室获得学生 ID 编号，而不能自动生成它们，那么在定义 student_id 列时请不要为它加上 AUTO_INCREMENT 属

性。不过，需要保留PRIMARY KEY子句，其目的在于要避免ID值出现重复或为NULL。

现在，CREATE TABLE语句末尾的ENGINE子句有什么作用呢？如果存在这个子句，那么它将在创建新表时，为MySQL应该使用的存储引擎指定名字。“存储引擎”就是一种用来管理某种表的处理器。MySQL有好几种存储引擎，都各有特色。其中两个最常用的引擎是 InnoDB（MySQL 5.5版本的默认引擎）和MyISAM（MySQL 5.5版本之前的默认引擎）。

关于这两个引擎之间的不同之处请参考2.6.1节。现在，只需说明成绩考评项目表的定义显式指定了InnoDB引擎就行了，因为我们需要InnoDB引擎所提供的称为“引用完整性（referential integrity）”的功能。该功能是通过使用外键（foreign key）来实现的。也就是说，我们可以使用MySQL将某些约束施加到两个表之间的相互关系上，这种做法对成绩考评项目的各个表来讲是很有必要的。

考试成绩与考试事件和学生都有关联：只有当与考试成绩相关联的学生ID和考试事件ID分别在student表和grade_event表里存在时，才允许考试成绩进入score表。

类似地，考勤记录与学生有关联：只有与考勤相关联的学生ID在student表里存在时，才允许考勤情况进入absence表。

为了实施这些约束，我们需要建立一些外键关系。这里的“外”表示的意思是“在另一个表里”，而“外键”指的是必须与另一个表里的某个键值相匹配的键值。随着后面更多成绩考评项目表的创建，这些概念将会变得越来越清晰。

在早些时候，我们在创建美史联盟的表（president和member）时，并没有使用ENGINE子句，因此服务器会使用默认存储引擎来创建它们。如前所述，默认存储引擎是 InnoDB（除非服务器被重新配置过）。student表的定义显式地包括了ENGINE = InnoDB，防止了服务器为其配置与此不同的默认值。

2. grade_event表

grade_event表的定义如下所示：

```

CREATE TABLE grade_event

(

date          DATE NOT NULL,

category ENUM('T','Q') NOT NULL,

event_id INT UNSIGNED NOT NULL AUTO_INCREMENT,

PRIMARY KEY (event_id)

) ENGINE = InnoDB;

```

要创建grade_event表，可以在mysql客户端程序里输入上述CREATE TABLE语句，也可以在命令行上执行下面这条命令：

```
% mysql sampdb < create_grade_event.sql
```

date列用于存放标准的MySQL的DATE（日期类型）值，格式为'CCYY-MM-DD'（年在前）。

category表示的是分数类别。与student表里的sex列一样，category也是一个枚举列。其允许的取值是'T'和'Q'，分别代表测试（test）和测验（quiz）。

event_id是一个AUTO_INCREMENT列，同时也被定义为了PRIMARY KEY。它与student表里的student_id列类似。利用AUTO_INCREMENT属性，我们能方便地生成唯一事件ID的值。与student表里的student_id列类似，其特定的值并不重要，重要的是它们必须唯一。

因为这些列必须都要有值，所以它们全部被定义成了NOT NULL。

3. score表

score表的创建语句如下所示：

```
CREATE TABLE score
```

```
(
    student_id INT UNSIGNED NOT NULL,

    event_id    INT UNSIGNED NOT NULL,

    score       INT NOT NULL,

    PRIMARY KEY (event_id, student_id),

    INDEX (student_id),

    FOREIGN KEY (event_id) REFERENCES grade_event (event_id),

    FOREIGN KEY (student_id) REFERENCES student (student_id)

) ENGINE = InnoDB;
```

这个表的定义又包含了新内容：FOREIGN KEY结构。我们稍后会讲到它。

要创建score表，可以在mysql客户端程序里输入上面的语句，也可以在命令行里执行下面的命令：

```
% mysql sampdb < create_score.sql
```

score是一个INT列，用于保存整型分数值。如果想要保存像58.5那样带有小数部分的分数，那么最好使用能表示它们的数据类型，如DECIMAL。

student_id列和event_id列都是整型列，它们分别表示每一个考试分数所对应的学生和考试事件。通过它们与student表和grade_event表里的相应ID值链接起来，我们就能够查出学生姓名和考试日期。关于student_id列和event_id列有两个要点需要注意一下。

我们已将这两列的组合设置成了一个PRIMARY KEY。这样可以确保我们不会重复记录某位学生在某次考试或测验的分数。请注意，只有event_id和student_id的组合才具有唯一性。在score表里，这两个ID

值自身都不具备唯一性。对于每一个event_id值（每位学生有一个）都会有多个分数行与之对应；对于每一个student_id值（每次考试或测验有一个）也会有多行记录相对应。

每一个ID列都需要用FOREIGN KEY子句来定义约束条件。此子句的REFERENCES部分表明这个ID列是与哪个表的哪一列相对应。event_id列的约束条件为：该列里的每一个值都必须与grade_event表里的某个event_id值相匹配。类似地，score表里的每一个student_id值都必须与student表里的某个student_id值相匹配。

那个PRIMARY KEY定义可以确保我们不会创建重复的分数行。而FOREIGN KEY定义可以确保在我们的记录行不会有虚假的ID值，即要求它们必须存在于grade_event表和student表里。

为什么student_id列会有一个索引呢？这是因为，对于FOREIGN KEY定义里的任何列，都应该有一个关于它们的索引，或者它们应该是某个多列索引里被首先列出的列，这样能加快查找速度。对于event_id列的FOREIGN KEY，该列被优先列在PRIMARY KEY里。对于student_id列的FOREIGN KEY，则无法使用PRIMARY KEY，因为student_id列未被首先列出来。因此，我们需要在student_id列上单独创建一个索引。

如有必要，InnoDB存储引擎会自动为出现在外键定义里的那些列创建一个索引，但它使用的索引定义不一定是你所期望的（更多信息请参考 2.13 节）。显式地定义这个索引可以避免这一问题。

4. absence表

absence表用于记录学生的考勤情况，其创建语句如下所示：

```
CREATE TABLE absence
```

```
(
```

```
student_id INT UNSIGNED NOT NULL,
```

```
date          DATE NOT NULL,
```

```
PRIMARY KEY (student_id, date),
```

```
FOREIGN KEY (student_id) REFERENCES student (student_id)
```

```
) ENGINE = InnoDB;
```

要创建absence表，可以在mysql客户端程序里输入上述语句，也可以在命令行上执行以下命令：

```
% mysql sampdb < create_absence.sql
```

将student_id列和date列都定义为NOT NULL，这样可以防止有缺失值。为了避免出现重复行，我们将这两列的组合定义为一个主键（primary key）。在同一天统计两次学生缺勤情况肯定是不公平的，对吧？

absence表也包含有一个外键关系，其目的在于确保每一个student_id值都与student表里的一个student_id值相匹配。

在成绩考评项目的各个表里设置外键关系，可以在让这些约束条件在数据录入阶段发挥作用，如我们只想插入那些包含合法考试事件ID值和学生ID值的记录行。不过，外键关系还有另外一种效果。它们会形成某些依赖关系，让你按照一定的顺序来创建和删除表，如下所示。

score表依赖于grade_event表和student表，因此在创建score表之前必须先创建其依赖的表。类似地，adsence表依赖于student表，因此在创建adsence表之前，student表必须已存在。

在删除表时，必须把上面的顺序颠倒过来。如果不先删除score表，就无法删除grade_event表；如果不先删除score表和absence表，也无法删除student表。

1.4.7 添加新行

至此，我们的数据库和表就都创建好了。接下来，我们需要往表里添加一些行。不过，在往表里放入某些内容之后，得能知道如何检查表里的内容，因此，尽管有关检索操作的详细介绍要在1.4.9节才会

讲到，此时也至少应该知道下面这条语句是用来查看tbl_name表里的全部内容的：

```
SELECT * FROM tbl_name;
```

例如：

```
mysql> SELECT * FROM student;
```

```
Empty set (0.00 sec)
```

现在，mysql报告说该表为空，但在练习完本节的几个示例之后，你会看到不一样的结果。

往数据库里添加数据的办法有好几种。可以用INSERT 语句将行手工插到表中，也可以利用文件把行添加到表里。该文件的内容既可以是一系列事先编写好的能直接提供给 mysql的INSERT语句，也可以是通过LOAD DATA语句或mysqlimport客户端程序来加载的原始数据值。

本节将演示各种把记录插到表中的方法。大家应该多练习这些方法，熟悉和掌握它们的工作原理以及用法。在练习完这些方法之后，再转到1.4.8节，运行那里的命令。那些命令可以用来删除这些表，然后再重建它们，并将本书提供的数据加载到这些表里。这样，你的数据库所包含的内容就会与我在后面示例中用到的数据保持一致，而你在练习本书其他示例时也会看到相同的结果。如果你已知道如何插入行，那么可以直接跳过本节。

1.4.7.1 利用INSERT添加行

我们先使用INSERT语句来添加行，这是一条SQL语句，你可用它来指定要插入数据行的那个表，以及要插入的数据行和该行的各个列值。INSERT语句有多种格式。

（1）一次性指定全部列值。语法如下：

```
INSERT INTO tbl_name VALUES(value1, value2, ...);
```

例如：

```
mysql> INSERT INTO student VALUES('Kyle', 'M', NULL);
```

```
mysql> INSERT INTO grade_event VALUES('2012-09-03', 'Q',  
NULL);
```

在使用此语法时，VALUES 列表必须包含表中每一列的值，并且值的顺序要与各列在表里的存储顺序保持一致。（通常情况下，该顺序就是各列在该表的CREATE TABLE语句里指定的顺序。）如果你不太确定列的顺序，那么可以利用DESCRIBE tbl_name语句来查明。

在MySQL 里，你可以使用单引号或双引号将字符串和日期值括起来，不过使用单引号会更标准些。NULL 值对应于 student 表和 grade_event 表里的AUTO_INCREMENT 列。在AUTO_INCREMENT列里插入一个“缺失值”（missing value），可以让MySQL为该列自动生成下一个序号。

MySQL还支持使用一条INSERT语句，同时指定多个值列表的方式，将多个行插入一个表里：

```
INSERT INTO tbl_name VALUES(...),(...),...;
```

例如：

```
mysql> INSERT INTO student VALUES('Avery','F',NULL),  
('Nathan','M',NULL);
```

与使用多条 INSERT 语句的方式相比，这种方式不仅能让你少打字，还能提高服务器的执行效率。请注意，将每行各列的值括起来的那对括号不可少。下列语句是非法的，因为它括号内包含的列值的个数不正确。

```
mysql> INSERT INTO student  
VALUES('Avery','F',NULL,'Nathan','M',NULL);
```

ERROR 1136 (21S01): Column count doesn't match value count at row 1

(2) 命名赋值列，并列出它们的值。当你创建的行只有少数几列需要初始化时，这种方式特别有用。语法如下：

```
INSERT INTO tbl_name (col_name1,col_name2,...)
VALUES(value1,value2,...);
```

例如：

```
mysql> INSERT INTO member (last_name,first_name)
VALUES('Stein','Waldo');
```

这种形式的INSERT语句也可以一次插入多个值列表：

```
mysql> INSERT INTO student (name,sex) VALUES('Abby','F'),
('Joseph','M');
```

对于没在INSERT语句中指定的列，将被赋予默认值。例如，上面两条语句都未给member_id和event_id赋值，因此MySQL会将默认值NULL赋给它们。又因为member_id和event_id都是AUTO_INCREMENT列，因此最后的结果是这两列都会被分别赋予各自的下一个序号，这与显式地将NULL赋给它们是一样的。

(3) 使用一系列的“列/值”形式进行赋值。此语法使用SET子句实现，其中包含多个col_name= value的赋值形式，没有使用VALUES()列表的形式。

```
INSERT INTO tbl_name SET col_name1=value1, col_name2=value2,
...;
```

例如：

```
mysql> INSERT INTO member SET
last_name='Stein',first_name='Waldo';
```

对于没在SET子句里指定的列，将被赋予默认值。这种形式的INSERT语句无法用于一次插入多个行的情形。

既然已对 INSERT 语句的工作原理有所了解，那么现在使用它来检查一下，我们所建立的外键关系是否真的能够防止将不规范的行录入score表和absence表里。试着在grade_event表和student表里找几个不存在的ID值，然后插入几条分别包含这些值的行：

```
mysql> INSERT INTO score (event_id,student_id,score)
VALUES(9999,9999,0);
```

```
ERROR 1452 (23000): Cannot add or update a child row: a foreign key
constraint fails (`sampdb`.`score`, CONSTRAINT `score_ibfk_1`
FOREIGN
```

```
KEY (`event_id`) REFERENCES `grade_event` (`event_id`))
```

```
mysql> INSERT INTO absence SET student_id=9999, date='2012-09-
16';
```

```
ERROR 1452 (23000): Cannot add or update a child row: a foreign key
constraint fails (`sampdb`.`absence`, CONSTRAINT `absence_ibfk_1`
FOREIGN KEY (`student_id`) REFERENCES `student`
(`student_id`))
```

有错误消息出现，即表明这些约束发挥了作用。

1. 4. 7. 2 利用文件添加新行

另一种把行载入表中的方法是，直接从文件里读取它们。该文件可以包含 INSERT 语句或原始数据。例如，在sampdb发行版里就有一个名为insert_president.sql的文件，它包含一系列用于将新行添加到president表里的INSERT语句。如果你是在与该文件相同的目录里，那么你可以像下面那样直接执行这些语句：

```
% mysql Sampdb < insert_president.sql
```

如果你已经运行了mysql，那么可以用一条source命令来读取这个文件：

```
mysql> source insert_president.sql;
```

如果文件里存储的行不是INSERT语句而是原始数据，那么可以利用LOAD DATA语句或客户端程序mysqlimport来加载它们。

LOAD DATA语句是一个从文件里读取数据的批量加载程序。它需要在mysql里运行：

```
mysql> LOAD DATA LOCAL INFILE 'member.txt' INTO TABLE  
member;
```

假设 member.txt 文件位于客户端主机的当前目录里，那么上面这条语句会读取该文件，并将其内容发送至服务器，加载到member表里。member.txt文件可在sampdb发行版里找到。

默认情况下，LOAD DATA语句会假设各列的值是以制表符分隔的，各行末尾都是换行符。同时，假设这些值的顺序都与表里存储的列的顺序相同（文件里的\N值表示的是NULL）。你也可以用它来读取其他格式的文件，或者指定不同的列顺序。有关LOAD DATA的更多细节请参考附录E。

LOAD DATA语句里的关键字LOCAL会引发客户端程序（在本示例里，指的是mysql）读取数据文件，并把文件内容发送到服务器进行加载。如果省略了LOCAL，那么数据文件必须存在于服务器主机上，并且你需要拥有FILE服务器访问权限（大部分MySQL用户都没有这样的权限）。另外，你还要指定完整的文件路径，以便服务器能找到它。

如果在LOAD DATA语句里使用LOCAL时遇到以下错误信息，那么很可能是因为在默认情况下LOCAL功能被禁用了：

```
ERROR 1148 (42000): The used command is not allowed with this  
MySQL version
```

可以在mysql之后加上--local-infile选项再试一次。例如：

```
% mysql --local-infile sampdb
```

```
mysql> LOAD DATA LOCAL INFILE 'member.txt' INTO TABLE  
member;
```

如果这招也不管用，那么说明服务器在启动时需要带上`--local-infile`选项。

另一种加载数据文件的方法是在命令提示符里使用客户端程序`mysqlimport`。它会为你生成一条`LOAD DATA`语句：

```
% mysqlimport --local sampdb member.txt
```

与程序`mysql`的用法一样，请根据需要在命令行里指定连接参数，并把它们放置在那个数据库名字的前面，紧挨着它。

对于上面这条命令，`mysqlimport`程序将生成一条能将`member.txt`文件里的内容加载到`member`表里的`LOAD DATA`语句。这是因为`mysqlimport`程序是根据数据文件的名称来确定表名的，同时它会把文件名中第一个句号（.）之前的所有内容都当作表名。例如，`mysqlimport`会把名为`member.txt`和`president.txt`的文件分别加载到`member`表和`president`表里。这意味着，你应该仔细挑选数据文件名，否则，`mysqlimport`将无法使用正确的表名。如果想要加载文件`member1.txt`和`member2.txt`，那么`mysqlimport`会认为是要把这两个文件分别加载到名为`member1`和`member2`的表里去。如果你真的想将这两个文件加载到`member`表里，那么可以分别将它们命名为`member.1.txt`和`member.2.txt`，或者`member.txt1`和`member.txt2`。

1.4.8 重置sampdb数据库

在练习完成上面介绍的这几种添加行的方法之后，为了顺利进行后面的学习，你应该重新建立和加载`sampdb`数据库里的各个表，把整个数据库恢复为原样。请在包含`sampdb`发布版文件的目录下，运用`mysql`程序来执行下面这些语句：

```
% mysql sampdb
```

```
mysql> source create_member.sql;
```



```
mysql> source create_president.sql;

mysql> source insert_member.sql;

mysql> source insert_president.sql;

mysql> DROP TABLE IF EXISTS absence, score, grade_event,
student;

mysql> source create_student.sql;

mysql> source create_grade_event.sql;

mysql> source create_score.sql;

mysql> source create_absence.sql;

mysql> source insert_student.sql;

mysql> source insert_grade_event.sql;

mysql> source insert_score.sql;

mysql> source insert_absence.sql;
```

如果不想单独输入这么多条语句，那么在Unix系统上，可以执行下面这条命令：

```
% sh init_all_tables.sh sampdb
```

而在Windows系统上，可以执行下面这条命令：

```
C:\> init_all_tables.bat sampdb
```

无论使用哪条命令，如果需要在命令行里指定连接参数，那么请把它们放到命令名的后面，且紧挨着它。

1.4.9 检索信息

现在，我们的表都建好了，并且加载了数据。下面一起来看看如何使用这些数据。使用SELECT语句可以检索和显示表里的信息。你可以根据自己的需要以常规或特定的方式来检索信息。例如，可以把表里的所有内容都显示出来：

```
SELECT * FROM president;
```

也可以只显示很少的数据，如一行中的一列：

```
SELECT birth FROM president WHERE last_name = 'Eisenhower';
```

SELECT语句拥有几个子句，你可以根据需要组合它们，用于检索你感兴趣的信息。这些子句可以很简单，也可以很复杂，因此语句SELECT 也会随之变得简单或者复杂。不过，本书中绝对没有整页长的、需要花费一个钟头才能搞明白的查询语句。当我遇到长长的（arm-length）查询语句时，通常会跳过它们，我想你也会这样做。

SELECT语句的简化语法如下：

```
SELECT what to retrieve
```

```
FROM table or tables
```

```
WHERE conditions that data must satisfy;
```

在写SELECT语句时，需要先指定检索的内容，然后再加上一些可选的子句。上面显示的两个子句（FROM和WHERE）是最为常见的，尽管还可以指定其他子句，如GROUP BY、ORDER BY和LIMIT。请记住，SQL语言对语句格式并没有严格的要求，因此写你自己的SELECT语句时，不必严格像本书示例那样换行排列。

FROM子句通常是不可少的，但当你不需要给出表名时，可以省略它。例如，下面这条查询语句只显示一些表达式的值。这些值的计算并未涉及任何表，因此这里不需要FROM子句：

```
mysql> SELECT 2+2, 'Hello, world', VERSION();
```

2+2	Hello, world	VERSION()
4	Hello, world	5.5.30-log

当的确需要使用FROM 子句来指定要从哪个表检索数据时，还需要指明要查看哪些列。SELECT 语句最常见的一种形式是使用一个星号（*）作为列说明符，代表“所有列”。下面这条查询语句将显示 student表里的所有列：

```
mysql> SELECT * FROM student;
```

name	sex	student_id
Megan	F	1
Joseph	M	2
Kyle	M	3
Katie	F	4

...

这些列将按它们在表里的存储顺序依次显示出来。这个顺序与你用DESCRIBE student语句看到的列顺序是一致的。（示例末尾处的省略号“...”表示该查询返回的行实际上有很多。）

也可以把自己想要查看的那些列的名字列出来。例如，只想查看学生姓名，则可以这样做：

```
mysql> SELECT name FROM student;
```

```

      name
-----
      Megan
      Joseph
      Kyle
      Katie

...

```

如果要列出多个列名，那么需要使用逗号把它们分隔开。下面这条语句等价于 `SELECT *FROM student` 语句，但它把各列的名字明确地列了出来：

```
mysql> SELECT name, sex, student_id FROM student;
```

name	sex	student_id
Megan	F	1
Joseph	M	2
Kyle	M	3
Katie	F	4

...

你可以按任意顺序列出各个列名：

```
SELECT name, student_id FROM student;
```

```
SELECT student_id, name FROM student;
```

只要你愿意，甚至还可以重复列出某一列的名字，只是这样做通常没什么意义。

另外，还可以从一个以上的表里选取列，即多表“连接”（join）。更多关于连接的信息请参考1.4.9.10节。

一方面，在MySQL里，列名不区分大小写，因此下面这些检索语句都是等价的：

```
SELECT name, student_id FROM student;
```

```
SELECT NAME, STUDENT_ID FROM student;
```

```
SELECT nAmE, sTuDeNt_Id FROM student;
```

另一方面，数据库名和表名可能是区分大小写的。具体情况取决于服务器主机所使用的文件系统，以及MySQL的配置。Windows系统的文件名不区分大小写，所以运行在它上面的服务器也不区分数据库名和表名的大小写。在Unix系统上，文件名通常都区分大小写，因此运行在它上面的服务器会区分数据库名和表名的大小写。Mac OS X系统的扩展文件系统比较特殊，它不区分大小写。

如果想让MySQL服务器不区分数据库名和表名的大小写，那么可以对它进行配置。更多详细信息请参考11.2.6节。

1.4.9.1 指定检索条件

要想限制SELECT语句检索出来的行数，可以使用WHERE子句，指定列值所必须满足的检索条件。例如，可以搜索某个范围内的数值：

```
mysql> SELECT * FROM score WHERE score > 95;
```

student_id	event_id	score
5	3	97
18	3	96
1	6	100
5	6	97
11	6	98
16	6	98

可以查找包含字符数据的字符串值。对于默认的字符集和排序方式，字符串的比较操作通常不区分大小写：

```
mysql> SELECT last_name, first_name FROM president
-> WHERE last_name='ROOSEVELT';
```

last_name	first_name
Roosevelt	Theodore
Roosevelt	Franklin D.

```
mysql> SELECT last_name, first_name FROM president
-> WHERE last_name='roosevelt';
```

last_name	first_name
Roosevelt	Theodore
Roosevelt	Franklin D.

也可以查找日期:

```
mysql> SELECT last_name, first_name, birth FROM president
```

```
-> WHERE birth < '1750-1-1';
```

last_name	first_name	birth
Washington	George	1732-02-22
Adams	John	1735-10-30
Jefferson	Thomas	1743-04-13

甚至还可以查找组合值:

```
mysql> SELECT last_name, first_name, birth, state FROM president
```

```
-> WHERE birth < '1750-1-1' AND (state='VA' OR state='MA');
```

last_name	first_name	birth	state
Washington	George	1732-02-22	VA
Adams	John	1735-10-30	MA
Jefferson	Thomas	1743-04-13	VA

WHERE子句里的表达式允许使用算术运算符（见表1-1）、比较运算符（见表1-2）和逻辑运算符（见表1-3）。在表达式里还可以使用括号。在运算时，可以使用常量、表列和函数调用。本教程的语句里会用到一些 MySQL的函数，但由于函数比较多，这里无法一一列出。有关这些函数的详细信息请参考附录C。

表1-1 算术运算符

运算符	含 义
+	加法
-	减法
*	乘法
/	除法
DIV	整除
%	模运算（除法余数）

表1-2 比较运算符

运算符	含 义
<	小于
<=	小于等于（不大于）
=	等于
<=>	等于（可用于NULL值）
<> 或 !=	不等于
>=	大于等于（不小于）
>	大于

表1-3 逻辑运算符

运算符	含 义
AND	逻辑与
OR	逻辑或
XOR	逻辑异或
NOT	逻辑非

当需要在查询语句里使用逻辑运算符时，千万要注意：逻辑运算符AND与人们日常生活中所说的“和”在含义上是不一样的。假设你要找出“出生于弗吉尼亚州和马萨诸塞州的总统”。这里用到了“和”字，它似乎是在暗示你该编写如下查询语句：

```
mysql> SELECT last_name, first_name, state FROM president
```

```
-> WHERE state='VA' AND state='MA';
```

```
Empty set (0.01 sec)
```

空的结果清楚表明，这条语句没起作用。为什么没起作用呢？因为这条查询语句的真正含义是“把同时出生于弗吉尼亚州和马萨诸塞州的总统”找出来，而这是不可能的。在日常生活里，你可以用“和”来表达你的查询条件；但在SQL里，必须使用OR来连接这两个条件：

```
mysql> SELECT last_name, first_name, state FROM president
```

```
-> WHERE state='VA' OR state='MA';
```

last_name	first_name	state
Washington	George	VA
Adams	John	MA
Jefferson	Thomas	VA
Madison	James	VA
Monroe	James	VA
Adams	John Quincy	MA
Harrison	William H.	VA
Tyler	John	VA
Taylor	Zachary	VA
Wilson	Woodrow	VA
Kennedy	John F.	MA
Bush	George H. W.	MA

请大家务必注意日常语言与SQL语言之间的差异，不只是在自己编写查询语句时要引起注意，在为其他人编写查询时也要注意。一定要仔细倾听别人对查询内容的描述，不能将他们的描述照搬成SQL的逻辑运算符。以刚描述的那个查询为例，与查询语句相当的自然语言表述应该是：“把出生于弗吉尼亚州或马萨诸塞州的总统找出来”。

当在组织可以查找到多个独立值的查询语句时，你可能会发现，使用IN()运算符更简洁。在使用IN()之后，前面的那个查询可以改写成下面这个样子：

```
SELECT last_name, first_name, state FROM president  
  
WHERE state IN('VA','MA');
```

当把一个列与大量值进行比较时，使用IN()会特别方便。

1.4.9.2 NULL值

NULL 值很特殊。其含义是“无值”或“未知值”，所以不能采用两个“已知值”的比较方式，将它与“已知值”进行比较。如果试图将 NULL 与常规的算术比较运算符一起使用，那么其结果将是未定义的（undefined）：

```
mysql> SELECT NULL < 0, NULL = 0, NULL <> 0, NULL > 0;
```

NULL < 0	NULL = 0	NULL <> 0	NULL > 0
NULL	NULL	NULL	NULL

事实上，你也不能让 NULL 与其自身进行比较，因为两个“未知值”的比较结果是无法确定的：

```
mysql> SELECT NULL = NULL, NULL <> NULL;
```

NULL = NULL	NULL <> NULL
NULL	NULL

如果需要测试多个NULL值是否相等，那么必须使用IS NULL或IS NOT NULL，而不能使用=、<>或者!=。例如，对于目前仍然健在的美国总统，其逝世日期在president表里表示为NULL。如果想要找到他们，可以使用下面这条查询语句：

```
mysql> SELECT last_name, first_name FROM president WHERE
death IS NULL;
```

last_name	first_name
Carter	James E.
Bush	George H. W.
Clinton	William J.
Bush	George W.
Obama	Barack H.

IS NOT NULL可以用来查找非NULL值。下面这条查询语句可以找到那些具有后缀名的名字：

```
mysql> SELECT last_name, first_name, suffix
-> FROM president WHERE suffix IS NOT NULL;
```

last_name	first_name	suffix
Carter	James E.	Jr.

MySQL特有的<=>比较运算符可用于NULL与NULL的比较。将前面两个查询语句改写成使用这个运算符的查询语句：

```
SELECT last_name, first_name FROM president WHERE death <=>
NULL;
```

```
SELECT last_name, first_name, suffix
```

```
FROM president WHERE NOT (suffix <=> NULL);
```

1. 4. 9. 3 对查询结果排序

每位MySQL用户最终都会注意到这样一种情况：在你创建一个表，并存入一些行之后，使用“SELECT * FROM tbl_name”语句查询出的行，其顺序通常与插入它们时的顺序一致。这很符合人们的思维习惯，人们自然会认定查询出的行的顺序与插入它们时的顺序是相同的。但实际情况并非这样。因为在表数据初始加载完之后，删除和插入行都会改变服务器返回行的顺序。

关于行检索顺序，请记住这样一条原则：服务器不会保证返回行的先后顺序，除非你自己指定顺序。要想结果有序，需要在查询语句后面增加一条 ORDER BY 子句。下面这条查询语句会按姓的字母顺序返回美国总统的姓名：

```
mysql> SELECT last_name, first_name FROM president
-> ORDER BY last_name;
```

```

      last_name | first_name
-----+-----
Adams          | John Quincy
Adams          | John
Arthur         | Chester A.
Buchanan       | James

```

...

ORDER BY子句的默认排序方式是升序排列。在其中的列名后面加上关键字ASC或DESC，可以指定是按照升序排列还是按照降序排列。例如，想让美国总统的姓名按姓的逆序（降序）排列显示，那么就要使用DESC关键字：

```
mysql> SELECT last_name, first_name FROM president
-> ORDER BY last_name DESC;
```

```

      last_name | first_name
-----+-----
Wilson         | Woodrow
Washington     | George
Van Buren      | Martin
Tyler          | John

```

...

你可以对多列进行排序，而且每一列单独地按升序或降序排列。下面的查询语句用于检索president表里的行，先按出生地所在州进行逆序排列，然后在每一个相同的州里再按姓升序排列：

```
mysql> SELECT last_name, first_name, state FROM president  
-> ORDER BY state DESC, last_name ASC;
```

last_name	first_name	state
Arthur	Chester A.	VT
Coolidge	Calvin	VT
Harrison	William H.	VA
Jefferson	Thomas	VA
Madison	James	VA
Monroe	James	VA
Taylor	Zachary	VA
Tyler	John	VA
Washington	George	VA
Wilson	Woodrow	VA
Eisenhower	Dwight D.	TX
Johnson	Lyndon B.	TX

...

在一个列里，对于升序排列，NULL值总是出现在开头；而对于降序排列，它总是出现在末尾。为确保NULL值出现在指定排列顺序的末

尾，需要额外增加一个可以区分NULL值和非NULL值的排序列。例如，想按逝世日期降序排列所有总统，那么当前健在的（逝世日期为NULL的）那些总统就应该出现在结果顺序的末尾。而如果想让他们出现在开头，就要使用下面这条查询语句：

```
mysql> SELECT last_name, first_name, death FROM president  
-> ORDER BY IF(death IS NULL,0,1), death DESC, last_name;
```

```
-----+-----+-----  
last_name | first_name | death  
-----+-----+-----  
  
Bush      | George W.  | NULL  
Bush      | George H.W. | NULL  
Carter    | James E.   | NULL  
Clinton   | William J. | NULL  
Obama     | Barack H.  | NULL  
Ford      | Gerald R.  | 2006-12-26  
Reagan    | Ronald W.  | 2004-06-05  
Nixon     | Richard M. | 1994-04-22  
  
...  
  
Adams     | John       | 1826-07-04  
Jefferson | Thomas     | 1826-07-04  
Washington | George    | 1799-12-14  
-----+-----+-----
```


其中， IF () 函数的作用是计算第一个参数给出的那个表达式的值，然后根据计算结果的真假来决定是返回第二个参数（为真），还是返回第三个参数（为假）。对于这条查询语句，当遇到NULL值时，IF () 函数的计算结果为0；当遇到非NULL值时，它计算结果为1。最终结果会把所有的NULL值放到非NULL值的前面。同时，对于death值相同的行，将last_name作为辅助列按姓继续进行排序。

1.4.9.4 限制查询结果

查询结果往往有很多行，如果只想看到其中的一小部分，那么可以在查询命令里增加一条LIMIT子句。如果将它与ORDER BY子句联合使用，效果会特别好。MySQL允许限制查询输出的行数，只输出结果中前面的n行。下面的查询语句将把按出生日期排在前5位的总统列出来：

```
mysql> SELECT last_name, first_name, birth FROM president  
-> ORDER BY birth LIMIT 5;
```

last_name	first_name	birth
Washington	George	1732-02-22
Adams	John	1735-10-30
Jefferson	Thomas	1743-04-13
Madison	James	1751-03-16
Monroe	James	1758-04-28

如果用DESC来逆序排列查询结果，那么可以得到最晚出生的那5位总统：

```
mysql> SELECT last_name, first_name, birth FROM president
```

-> ORDER BY birth DESC LIMIT 5;

last_name	first_name	birth
Obama	Barack H.	1961-08-04
Clinton	William J.	1946-08-19
Bush	George W.	1946-07-06
Carter	James E.	1924-10-01
Bush	George H. W.	1924-06-12

LIMIT 子句还允许从查询结果的中间抽出部分行。此时需要指定两个值：第一个，给出从查询结果的开头部分跳过的行数；第二个，需要返回的行数。下面这条查询语句与前面那条很相似，但它返回的是跳过前面10行之后的5行：

```
mysql> SELECT last_name, first_name, birth FROM president
```

-> ORDER BY birth DESC LIMIT 10, 5;

last_name	first_name	birth
Eisenhower	Dwight D.	1890-10-14
Truman	Harry S	1884-05-08
Roosevelt	Franklin D.	1882-01-30
Hoover	Herbert C.	1874-08-10
Coolidge	Calvin	1872-07-04

如果想从某个表里随机抽取出一行或几行，那么可以联合使用
LIMIT 子句和ORDER BY RAND() 子句：

```
mysql> SELECT last_name, first_name FROM president
-> ORDER BY RAND() LIMIT 1;
```

last_name	first_name
Johnson	Lyndon B.

```
mysql> SELECT last_name, first_name FROM president
-> ORDER BY RAND() LIMIT 3;
```

```

-----+-----
last_name | first_name
-----+-----
Harding   | Warren G.
Bush      | George H. W.
Jefferson | Thomas

```

1.4.9.5 对输出列进行计算和命名

到目前为止，大部分查询语句都是直接通过检索表中的值来获得输出结果。MySQL也支持根据表达式的结果计算输出值，引不引用表都可以。下面的这条查询语句计算了一个简单表达式（一个常量）和一个复杂表达式（它使用了几种算术运算和两个函数调用，这两个函数一个用于计算表达式的平方根，另外一个用于将结果格式化保留3位小数）：

```
mysql> SELECT 17, FORMAT(SQRT(25+13),3);
```

```

-----+-----
17 | FORMAT(SQRT(25+13),3)
-----+-----
17 | 6.164
-----+-----

```

在表达式里也可以使用表列，如下所示：

```
mysql> SELECT CONCAT(first_name,' ',last_name), CONCAT(city,' ',state)
```

```
-> FROM president;
```

```

CONCAT(first_name,' ',last_name) | CONCAT(city,', ',state)
-----+-----
George Washington                | Wakefield, VA
John Adams                       | Braintree, MA
Thomas Jefferson                 | Albemarle County, VA
James Madison                   | Port Conway, VA
...

```

这个查询对总统的名字进行了格式化，将名和姓用空格连接成了一个字符串。对他们的出生地也进行了格式，将所在城市和州用逗号和空格连接成了一个字符串。

计算某列的值的表达式会成为该列的名字，并被用作输出结果的标题。如果表达式很长（如前面那个查询示例所示），那么它会使输出列的宽度变得很大。为使输出更具意义和可读性，你可以利用AS name结构为该列分配另一个名字（也称“别名”）：

```

mysql> SELECT CONCAT(first_name,' ',last_name) AS Name,
-> CONCAT(city,', ',state) AS Birthplace
-> FROM president;

```

Name	Birthplace
George Washington	Wakefield, VA
John Adams	Braintree, MA
Thomas Jefferson	Albemarle County, VA
James Madison	Port Conway, VA

...

如果输出列的别名里包含空格，那么必须给它加上引号：

```
mysql> SELECT CONCAT(first_name,' ',last_name) AS 'President
Name',
```

```
-> CONCAT(city,' ',state) AS 'Place of Birth'
```

```
-> FROM president;
```

President Name	Place of Birth
George Washington	Wakefield, VA
John Adams	Braintree, MA
Thomas Jefferson	Albemarle County, VA
James Madison	Port Conway, VA

...

在为列提供别名时，可以省略关键字AS：

```
mysql> SELECT 1 one, 2 two, 3 three;
```

```
-----+-----+-----
one | two | three
-----+-----+-----
 1  |  2  |    3
-----+-----+-----
```

如果一个查询结果的列名有误，或者有列丢失，那么请检查在某两个列之间是否忘了加上逗号。如果真是这种情况，那么第二列会被当成第一列的别名。例如，你原打算编写一个查询语句，选取下列总统的姓名，可惜不小心漏掉了first_name和last_name两列之间的逗号。结果，first_name列被误命名为last_name，而列last_name列却不见了：

```
mysql> SELECT first_name last_name FROM president;
```

```
-----
last_name
-----
George
John
Thomas
James
```

...

1. 4. 9. 6 处理日期

在MySQL里使用日期时，千万要记住的是年份总是在最前面。在写2012年7月27日这个日期时，请将其写成'2012-07-27'。不要像日常生活中那样，将它写成'07-27-2012'或'27-07-2012'。对于其他格式的输入值，或许能使用STR_TO_DATE()函数来进行转换。具体示例请参考3.2.6节。

MySQL支持多种类型的日期运算。

按日期排序（我们已经操作过多次）。

搜索特定日期或日期范围。

从日期值里提取各组成部分，如年、月或日。

计算两个日期之间的时间差。

通过将一个日期加上或减去一个时间间隔，计算出另一个日期。

下面是一些与日期运算有关的查询示例。

为了通过确切值或通过另一日期的相对值，查询出特定的日期来，可以将某个DATE 列与你感兴趣的那个日期值进行比较：

```
mysql> SELECT * FROM grade_event WHERE date = '2012-10-01';
```

date	category	event_id
2012-10-01	T	6

```
mysql> SELECT last_name, first_name, death
```

```
-> FROM president
```

```
-> WHERE death >= '1970-01-01' AND death < '1980-01-01';
```


last_name	first_name	death
Truman	Harry S	1972-12-26
Johnson	Lyndon B.	1973-01-22

为测试或检索日期的各个部分，可以使用像 YEAR()、MONTH()、DAYOFMONTH() 这样的函数。例如，下面这个查询可以把生于3月的美国总统查找出来：

```
mysql> SELECT last_name, first_name, birth
-> FROM president WHERE MONTH(birth) = 3;
```

last_name	first_name	birth
Madison	James	1751-03-16
Jackson	Andrew	1767-03-15
Tyler	John	1790-03-29
Cleveland	Grover	1837-03-18

也可以用月份名称来改写这个查询：

```
mysql> SELECT last_name, first_name, birth
-> FROM president WHERE MONTHNAME(birth) = 'March';
```

last_name	first_name	birth
Madison	James	1751-03-16
Jackson	Andrew	1767-03-15
Tyler	John	1790-03-29
Cleveland	Grover	1837-03-18

再进一步，把函数MONTH()和DAYOFMONTH()结合起来使用，找出生于3月某一天的总统：

```
mysql> SELECT last_name, first_name, birth
```

```
-> FROM president WHERE MONTH(birth) = 3 AND
DAYOFMONTH(birth) = 29;
```

last_name	first_name	birth
Tyler	John	1790-03-29

你偶尔会看到类似“今日名人”之类的信息，用上面的查询语句就能生成一份这样的名单。不过，如果你的查询与“当前日期”有关，那么大可不必像前面的例子那样插入一个具体的日期值。不管今日是一年里的哪一天，只要将各位总统的生日与CURDATE()函数（该函数总是返回当前日期）里的月和日进行比较，便可查出“今日出生的”总统，如下所示：

```
SELECT last_name, first_name, birth
```

```
FROM president WHERE MONTH(birth) = MONTH(CURDATE())
```

```
AND DAYOFMONTH(birth) = DAYOFMONTH(CURDATE());
```

如果想知道两个日期值之间的时间间隔，可以将它们相减。例如，想要知道哪位总统活得最久，那么可以用他们的逝世日期减去出生日期。此时，TIMESTAMPDIFF() 函数就派上用场了，因为它有一个参数，可以指定计算结果的单位（在本例里为年，即YEAR）：

```
mysql> SELECT last_name, first_name, birth, death,  
-> TIMESTAMPDIFF(YEAR, birth, death) AS age  
-> FROM president WHERE death IS NOT NULL  
-> ORDER BY age DESC LIMIT 5;
```

last_name	first_name	birth	death	age
Reagan	Ronald W.	1911-02-06	2004-06-05	93
Ford	Gerald R.	1913-07-14	2006-12-26	93
Adams	John	1735-10-30	1826-07-04	90
Hoover	Herbert C.	1874-08-10	1964-10-20	90
Truman	Harry S	1884-05-08	1972-12-26	88

如果想要计算相差的天数，则需要使用另一种计算两个日期之间时间间隔的方法：使用TO_DAYS() 函数，将日期转换为天数。检测出与某个参考日期相差有多少天是该函数的一种主要应用。例如，为找出近期需要更新成员资格的“美史联盟”成员，可以将成员的有效日期与当前日期相减。如果其结果小于某个阈值，即表明该成员的资格快要到期了。下面这条查询语句可以将成员资格已过期的以及在60天内将到期的成员查找出来：

```
SELECT last_name, first_name, expiration FROM member  
WHERE (TO_DAYS(expiration) - TO_DAYS(CURDATE())) < 60;
```

使用TIMESTAMPDIFF() 函数的等效语句，如下所示：

```
SELECT last_name, first_name, expiration FROM member
```

```
WHERE TIMESTAMPDIFF(DAY, CURDATE(), expiration) < 60;
```

要根据某个日期计算出另一个日期，可以使用函数DATE_ADD()或DATE_SUB()。这两个函数的参数都是一个日期值和一个时间间隔，然后返回一个新的日期值。例如：

```
mysql> SELECT DATE_ADD('1970-1-1', INTERVAL 10 YEAR);
```

```
-----  
DATE_ADD('1970-1-1', INTERVAL 10 YEAR)  
-----
```

```
1980-01-01  
-----
```

```
mysql> SELECT DATE_SUB('1970-1-1', INTERVAL 10 YEAR);
```

```
-----  
DATE_SUB('1970-1-1', INTERVAL 10 YEAR)  
-----
```

```
1960-01-01  
-----
```

在本节里的前面有一个查询，它选取那些逝世于20世纪70年代美国总统，使用文字量型的日期值作为选取范围的结束点。可以重写该查询语句，使用一个文字量型的起点日期，然后在起点日期上加上一个时间间隔计算出终点日期：

```
mysql> SELECT last_name, first_name, death
```

```
-> FROM president
```

```
-> WHERE death >= '1970-1-1'
```

```
-> AND death < DATE_ADD('1970-1-1', INTERVAL 10 YEAR);
```

last_name	first_name	death
Truman	Harry S	1972-12-26
Johnson	Lyndon B.	1973-01-22

另外还有一种编写“成员资格更新”查询的方法，即使用DATE_ADD()函数：

```
SELECT last_name, first_name, expiration FROM member
WHERE expiration < DATE_ADD(CURDATE(), INTERVAL 60 DAY);
```

如果 expiration 列经过了索引，那么这个查询将会比前一个更有效率。具体原因请参考第5章。

大概在本章开头，有下面这样一个查询，用于找出“在牙医诊所的患者中，有哪些患者在到期后还未来复诊”：

```
SELECT last_name, first_name, last_visit FROM patient
WHERE last_visit < DATE_SUB(CURDATE(), INTERVAL 6 MONTH);
```

或许当时你还不太能看懂这个查询。那么现在是否已完全明白了呢？

1.4.9.7 模式匹配

MySQL支持模式匹配操作，这使我们能够在未给出精确比较值的情况下把行查出来。模式匹配需要使用像LIKE和NOT LIKE那样的运算符，并且需要指定一个包含通配字符的字符串。下划线“_”可以匹配任何的单个字符，百分号“%”则能匹配任何字符序列（其中包括空序列）。

下面这个模式能匹配到所有以字母W或w开头的姓：

```
mysql> SELECT last_name, first_name FROM president
-> WHERE last_name LIKE 'W%';
```

last_name	first_name
Washington	George
Wilson	Woodrow

下面这个查询展示了一种常见错误。这个“模式匹配”不会返回任何内容，因为它没有使用带LIKE的模式，而是使用了带算术比较运算符的模式。

```
mysql> SELECT last_name, first_name FROM president
```

```
-> WHERE last_name = 'W%';
```

Empty set (0.00 sec)

上面这个比较表达式唯一成功的地方是该列正好包含了字符串'W%'或'w%'。

下面这个模式将与那些包含有'W%'或'w%'（并不仅限于第一个字母）的姓相匹配：

```
mysql> SELECT last_name, first_name FROM president
```

```
-> WHERE last_name LIKE '%W%';
```

last_name	first_name
Washington	George
Wilson	Woodrow
Eisenhower	Dwight D.

下面这个模式将与恰好包含4个字母的姓相匹配，如下所示：

```
mysql> SELECT last_name, first_name FROM president
```

```
-> WHERE last_name LIKE '____';
```

```
-----+-----  
last_name | first_name  
-----+-----  
Polk      | James K.  
Taft      | William H.  
Ford      | Gerald R.  
Bush      | George H. W.  
Bush      | George W.  
-----+-----
```

MySQL还提供另一种基于正则表达式 (regular expression) 和REGEXP运算符的模式匹配。关于LIKE和REGEXP的更多信息请参考3.5.1.1节和附录C。

1.4.9.8 设置和使用自定义变量

MySQL支持你定义自己的变量。这些变量可以被设置为查询结果，这使我们可以方便地把一些值存储起来以供今后的查询使用。假设想知道在Andrew Jackson总统之前出生的总统有哪些。可以先将他的出生日期检索出来，并存储到一个变量里，然后再将出生日期早于该变量值的其他总统查找出来：

```
mysql> SELECT @Jackson_birth := birth FROM president
```

```
-> WHERE last_name = 'Jackson' AND first_name = 'Andrew';
```

```
-----  
@jackson_birth := birth  
-----  
1767-03-15  
-----
```

```
mysql> SELECT last_name, first_name, birth FROM president
```

```
-> WHERE birth < @jackson_birth ORDER BY birth;
```

last_name	first_name	birth
Washington	George	1732-02-22
Adams	John	1735-10-30
Jefferson	Thomas	1743-04-13
Madison	James	1751-03-16
Monroe	James	1758-04-28

自定义变量的语法为“@变量名”。在SELECT语句里，赋值语法是形如“@变量名:=值”的表达式。因此，上面的第一个查询主要负责把Andrew Jackson总统的出生日期查出来，并把它赋给一个名为@jackson_birth的变量。这条SELECT语句的查询结果仍会被显示出来；而将查询结果赋给变量的过程并不会阻碍该查询的输出显示。第二个查询会引用该变量，并在president表里找出birth值小于该变量值的那些行。

实际上，前面的那个问题可以通过一条使用连接或子查询的查询语句来解决，但这里不想对此有过多的讨论。有时，使用一个变量可能会更容易让人理解。更多关于子查询的信息请参考1.4.9.10节。

也可以使用SET语句来对变量进行赋值。此时，“=”和“:="都可以用作赋值运算符：

```
mysql> SET @today = CURDATE();
```

```
mysql> SET @one_week_ago := DATE_SUB(@today, INTERVAL 7 DAY);
```

```
mysql> SELECT @today, @one_week_ago;
```


@today		@one_week_ago
2012-04-21		2012-04-14

1.4.9.9 生成统计信息

MySQL最有用的一项功能就是，能够对大量原始数据进行归纳和统计。大家都明白，单纯依靠人工手段来生成统计信息是一项既枯燥耗时，又易出错的工作。如果大家能学会使用MySQL来生成各种统计信息，那么它将会成为你的得力助手。

在一组值里把各个唯一值找出来，这是一项典型的统计工作。可以使用DISTINCT 关键字清除查询结果里重复出现的行。例如，下面的查询可以将美国历任总统出生地所在的州不加重复地列举出来：

```
mysql> SELECT DISTINCT state FROM president ORDER BY state;
```

```

-----
state
-----

AR
CA
CT
GA
HI
IA
IL
KY
MA
MO

...

```

另一种形式的统计是计数，需要使用COUNT () 函数。如果使用COUNT (*)，那么计算出来的结果将是查询所选取到的行数。如果查询语句没有带WHERE子句，那么它会查询所有行，因此，COUNT (*) 计算出来的结果就是表的行数。下面这个查询可以显示出“美史联盟”的member表里包含多少行：

```
mysql> SELECT COUNT(*) FROM member;
```

```

-----
COUNT (*)
-----

102
-----

```

如果查询语句带有 WHERE 子句，那么 COUNT (*) 计算出来的结果就是该子句匹配到了多少行。下面这个查询可以显示出“到目前为止，你的班级已

进行了多少次测验”：

```
mysql> SELECT COUNT(*) FROM grade_event WHERE category = 'Q';
```

```
-----  
COUNT(*)  
-----  
4  
-----
```

COUNT(*) 函数会统计所有被查询到的行数。与之相对的是，COUNT(列名) 只会统计所有非NULL值的数目。下面这个查询展示了两个函数的区别：

```
mysql> SELECT COUNT(*), COUNT(email), COUNT(expiration) FROM  
member;
```

```
-----+-----+-----  
COUNT(*) | COUNT(email) | COUNT(expiration)  
-----+-----+-----  
102 | 80 | 96  
-----+-----+-----
```

从上面的查询结果可以看出，member表目前共有102行，其中只有80行在email列里有值。它同时还显示出有6名成员具有终身成员资格。

(expiration 列里的NULL 值表示具有终身成员资格，并且由于在102条记录里有96条不为NULL，因此剩下的6条必然属于终身成员。)

组合使用COUNT()与DISTINCT，可以统计出在查询结果里有多少个不同的非NULL值。例如，想要知道美国共有多少个不同的州曾经诞生过总统，那么可以使用下面这条查询语句：

```
mysql> SELECT COUNT(DISTINCT state) FROM president;
```

```

-----
COUNT(DISTINCT state)
-----
21
-----

```

你可以对某个数据列进行全面统计，也可以对该列进行分类统计。例如，使用下面这个查询，可以确定出班级里总共有多少名学生：

```
mysql> SELECT COUNT(*) FROM student;
```

```

-----
COUNT(*)
-----
31
-----

```

不过，班级里的男生和女生分别是多少呢？有一种办法可以找到答案，即按性别分别进行统计：

```
mysql> SELECT COUNT(*) FROM student WHERE sex='f';
```

```

-----
COUNT(*)
-----
15
-----

```

```
mysql> SELECT COUNT(*) FROM student WHERE sex='m';
```

```

-----
COUNT(*)
-----
16
-----

```

这个办法可行，但比较麻烦，而且很不适合于有多种不同列值的情形。假设如何采用这种方式来确定出生自美国各个州的总统人数。首先，你必须一个不少地把涉及的州全部找出来（使用查询语句 `SELECT DISTINCT state FROM president`），然后，再针对各州执行查询语句 `SELECT COUNT(*)` 来统计出最终结果。显然这是你想避免的事情。

幸运的是，只用一个查询便可以统计出某一系列里的不同值分别出现过多少次。对于那个学生列表，可以使用 `GROUP BY` 子句来分别统计男、女学生的人数，如下所示：

```
mysql> SELECT sex, COUNT(*) FROM student GROUP BY sex;
```

```

-----+-----
sex | COUNT(*)
-----+-----
F   |      15
M   |      16
-----+-----

```

使用同样的查询形式，可以分别统计出生自各州的总统人数，如下所示：

```
mysql> SELECT state, COUNT(*) FROM president GROUP BY state;
```

-----+-----	
state	COUNT(*)
-----+-----	
AR	1
CA	1
CT	1
GA	1
HI	1
IA	1
IL	1
KY	1
MA	4
MO	1

...

在采用这种方式进行分组统计时，GROUP BY子句会告知MySQL在统计之前应该如何对值进行分组。

与分别统计某列的不同值所出现次数的做法相比，将COUNT(*)函数与GROUP BY子句结合在一起用于进行分组统计的做法有很多优点。

不用事先知道被统计列里有什么值。

只需一个查询语句。

因为只用一个查询便能获得所有的结果，所以可以对输出进行排序。

前两项优点的重要性体现在：它们有助于简化查询语句的表达。第3项优点的重要性则体现在：它能以更灵活的方式显示查询结果。默认情况下，MySQL会根据GROUP BY子句里的列名来对查询结果进行排序，但你也可以用ORDER BY子句指定一个特定的排序顺序。例如，你想获得按出生地所在州分组后的总统人数，并按人数从多到少的顺序排列出来，于是可以多加一个ORDER BY子句，如下所示：

```
mysql> SELECT state, COUNT(*) AS count FROM president
```

```
-> GROUP BY state ORDER BY count DESC;
```

-----+-----	
state	count
-----+-----	
VA	8
OH	7
MA	4
NY	4
NC	2
VT	2
TX	2
GA	1
IL	1
SC	1

...

如果用于排序的列是由某个汇总函数产生的，那么不能直接在ORDER BY子句里引用该函数。而是应该先为该列取一个别名，然后再在ORDER BY子句里引用这个别名。上面那个查询就是这样做的，其中的COUNT(*)列的别名为count。

要使用GROUP BY子句来对某个计算列的结果进行分组，需要使用别名或列位置来引用它，具体实现方法与ORDER BY相类似。下面这个查询可以确定出在一年的每个月分别有多少位总统出生：

```
mysql> SELECT MONTH(birth) AS Month, MONTHNAME(birth) AS Name,
```

```
-> COUNT(*) AS count
```

```
-> FROM president GROUP BY Name ORDER BY Month;
```

Month	Name	count
1	January	4
2	February	4
3	March	4
4	April	4
5	May	2
6	June	1
7	July	4
8	August	5
9	September	1
10	October	6
11	November	5
12	December	3

`COUNT()` 函数可以与`ORDER BY`和`LIMIT`组合在一起使用。例如，想要在 `president` 表里找出哪4个州出生的总统最多，那么可以使用下面这条查询语句：

```
mysql> SELECT state, COUNT(*) AS count FROM president
```

```
-> GROUP BY state ORDER BY count DESC LIMIT 4;
```


state	count
VA	8
OH	7
MA	4
NY	4

如果不是想用LIMIT子句来限制查询结果中的记录条数，而是想把COUNT()的某些特定值找出来，那么需要用到HAVING子句。该子句与WHERE相类似，它们都可用来设定输出行所必须满足的查询条件。与WHERE子句的不同之处在于，它可以引用像COUNT()那样的汇总函数输出的结果。下面这个查询会告诉你“哪些州曾经出现过两位及以上总统”：

```
mysql> SELECT state, COUNT(*) AS count FROM president
```

```
-> GROUP BY state HAVING count > 1 ORDER BY count DESC;
```

state	count
VA	8
OH	7
MA	4
NY	4
NC	2
VT	2
TX	2

一般情况下，带有 HAVING 子句的查询语句，特别适合于查找在某个数据列里重复出现的值。也可用于查找不重复出现的值，此时使用HAVING count = 1即可。

除COUNT()以外，还有其他几个汇总函数。函数MIN()、MAX()、SUM()和AVG()可分别用于确定某个数据列的最小值、最大值、总计和平均值。你甚至可以同时在一个查询语句里使用它们。下面这个查询可以显示出已进行过的每次考试或测验的各种数值特征。它也会显示出有多少分数参与了各个值的计算。（可能有的学生缺勤或未被统计。）

```
mysql> SELECT
-> event_id,
-> MIN(score) AS minimum,
-> MAX(score) AS maximum,
-> MAX(score)-MIN(score)+1 AS span,
-> SUM(score) AS total,
-> AVG(score) AS average,
-> COUNT(score) AS count
-> FROM score
-> GROUP BY event_id;
```

event_id	minimum	maximum	span	total	average	count
1	9	20	12	439	15.1379	29
2	8	19	12	425	14.1667	30
3	60	97	38	2425	78.2258	31
4	7	20	14	379	14.0370	27
5	8	20	13	383	14.1852	27
6	62	100	39	2325	80.1724	29

很明显，如果从中还能明确地知道 event_id 列的值是表示考试还是表示测验，那么这些信息会更具有意义。想要获得该信息，还需要查询 grade_event表。关于此操作的更多细节请参考1.4.9.10节。

如果想要生成额外的输出行，显示出“统计结果的统计”，那么还需要增加一条WITH ROLLUP子句。它会让 MySQL 计算各分组行的“超集”（super-aggregate）值。这里有个简单的示例，它是基于先前那个按性别统计学生人数的示例改进的。WITH ROLLUP子句将生成另外一行，对两类性别的人数进行汇总：

```
mysql> SELECT sex, COUNT(*) FROM student GROUP BY sex WITH ROLLUP;
```

sex	COUNT(*)
F	15
M	16
NULL	31

分组列里的NULL表明，相应的计数结果就是其前面那些分组统计的汇总值。

WITH ROLLUP子句还可以与其他聚集函数搭配使用。下面这条语句，除了像前面几个段落那样可以对考试成绩进行了汇总以外，还可以产生一个额外的超集行：

```
mysql> SELECT
-> event_id,
-> MIN(score) AS minimum,
-> MAX(score) AS maximum,
-> MAX(score)-MIN(score)+1 AS span,
-> SUM(score) AS total,
-> AVG(score) AS average,
-> COUNT(score) AS count
-> FROM score
-> GROUP BY event_id WITH ROLLUP;
```

event_id	minimum	maximum	span	total	average	count
1	9	20	12	439	15.1379	29
2	8	19	12	425	14.1667	30
3	60	97	38	2425	78.2258	31
4	7	20	14	379	14.0370	27
5	8	20	13	383	14.1852	27
6	62	100	39	2325	80.1724	29
NULL	7	100	94	6376	36.8555	173

在上面这个输出里，最后一行显示出了一些聚集值，它们都是根据其前面的全部分组统计值计算出来的。

WITH ROLLUP子句很有用，因为它可以让你不必为了获得一些额外的信息，而执行另外一条查询语句。只用一条查询语句就能达到目的，当然效率会更高，因为服务器无需对数据进行两次检查。如果GROUP BY子句指定了多列，那么WITH ROLLUP还会再生成其他的超集行，其中会包含更高层的汇总值。

1.4.9.10 从多个表里检索信息

到目前为止，我们查询出来的信息都是来自一个表。不过，MySQL的能耐远不止于此。前面说过，DBMS的威力在于它们可以把源自多个表的信息结合起来，从而解答那些只靠单个表而无法解答的问题。本节将介绍如何编写涉及多个表的查询语句。

在从多个表中查询信息时，有一种类型的操作叫连接（join）。之所以叫这个名字，是因为必须把一个表与另一个表中的信息连接起来才能得到结果。此操作是通过匹配多个表里的公共值实现的。另一种类型的多表操作是将一条SELECT语句嵌套在另一条SELECT语句里使用。这种嵌套的SELECT语句叫子查询（subquery）。本节将对这两种类型的操作进行介绍。

先一起来看一个关于连接的例子。1.4.6.2 节给出了一个用来检索给定日期考试或测验分数的查询命令，但在那里并未对它进行解释。现在可以对

它进行解释了。那条查询语句实际上涉及了一个三方的连接操作，因此我们将它分成两步来实现。第一步，构造一条可以查出给定日期的分数的查询语句：

```
mysql> SELECT student_id, date, score, category
-> FROM grade_event INNER JOIN score
-> ON grade_event.event_id = score.event_id
-> WHERE date = '2012-09-23';
```

student_id	date	score	category
1	2012-09-23	15	Q
2	2012-09-23	12	Q
3	2012-09-23	11	Q
5	2012-09-23	13	Q
6	2012-09-23	18	Q

...

这个查询先查出给定日期（'2012-09-23'）的grade_event行，再利用此行里的事件ID把score表里拥有同一事件ID的分数都查询出来。对于grade_event和score两个表里相匹配的每一个行组合，把其中的学生ID、分数、日期和事件类别都显示出来。

这个查询与之前介绍的查询语句在以下两个方面有着显著的区别。

FROM子句指定了多个表名，因为需要从多个表里检索信息：

```
FROM grade_event INNER JOIN score
```

ON子句指定了表grade_event和score的连接条件，即这两个表的event_id值必须相互匹配：

```
ON grade_event.event_id = score.event_id
```

请注意，我们是如何通过`grade_event.event_id`和`score.event_id`来引用`event_id`列的，其语法形式为：`tbl_name.col_name`。这样，MySQL 便能知道我们是在引用哪个表。因为这两个表都有 `event_id` 列，所以在不限定表名时，会产生二义性。这条查询语句里的其他列（`date`、`score`和`category`）可以直接使用，不用限定表名，因为它们只存在于其中的一个表里，不会产生二义性。

在连接语句里，我个人比较喜欢在每个列的前面都加上表名，从而可以让每一列属于哪个表变得更加清楚。在后面的连接语句里，我将一直沿用这个习惯。在为每列加上完整的表名之后，这个查询现在变成了下面这个样子：

```
SELECT score.student_id, grade_event.date, score.score, grade_event.category
FROM grade_event INNER JOIN score
ON grade_event.event_id = score.event_id
WHERE grade_event.date = '2012-09-23';
```

在第一阶段的查询里，我们利用`grade_event` 表将日期映射到了事件ID，并使用这个事件ID在`score`表里找到了与之匹配的分值。这个查询的输出只包含了`student_id`值，但是如果能把学生的姓名直接显示出来则会更具意义。第二阶段，我们将利用`student`表，把学生ID映射成他们的姓名。`score`表和`student`表都有`student_id`列，通过它可以两个表的各个行链接起来，显示出学生的名字。最终的查询语句如下所示：

```
mysql> SELECT
-> student.name, grade_event.date, score.score,
grade_event.category
-> FROM grade_event INNER JOIN score INNER JOIN student
-> ON grade_event.event_id = score.event_id
-> AND score.student_id = student.student_id
-> WHERE grade_event.date = '2012-09-23';
```

name	date	score	category
Megan	2012-09-23	15	Q
Joseph	2012-09-23	12	Q
Kyle	2012-09-23	11	Q
Abby	2012-09-23	13	Q
Nathan	2012-09-23	18	Q

...

这个查询与以前介绍的查询命令在以下几方面有区别。

FROM子句现在包含了student表，因为这条查询语句除了要用到grade_event表和score表以外，还需要用到它。

在前一个查询里，student_id列不会产生二义性，因此在引用它时，既可以不限定表名(student_id)，也可以限定表名(score.student_id)。但在这个查询里，因为score表和student表都有student_id列，所以肯定会出现二义性。于是，为了避免产生二义性，必须将它们分别限定为score.student_id和student.student_id。

ON子句里多了一个查询条件，用于指定score表里的行与student表里的行必须基于学生ID匹配在一起：

ON ...score.student_id = student.student_id

这个查询会显示出学生的姓名，而不显示学生的ID。（如果想要两者都显示，只需要在输出列的列表里加上student.student_id即可。）

对于这个查询，只要插入任何日期，即可获得那天对应的分数、参加考试的学生姓名，以及考试的类别。你根本不用了解学生ID和事件ID，因为MySQL会自动查出有关的ID值并利用它们把你想要的信息找出来。

在前面的1.4.9.9节，我们运行了一个查询，它可以将score表里各类数据的数值特征统计出来。在那个查询的输出结果里，只列出了事件ID，而没有列出事件日期或类别，因为我们当时还不知道如何将score表连接到

grade_event表上，将事件ID映射到考试日期和类别上去。现在，我们来实现这一效果。下面这个查询与前面那个相差无几，但它显示出来的是考试日期和类别，而不只是一些数字形式的事件ID：

```
mysql> SELECT  
  
-> grade_event.date, grade_event.category,  
  
-> MIN(score.score) AS minimum,  
  
-> MAX(score.score) AS maximum,  
  
-> MAX(score.score)-MIN(score.score)+1 AS span,  
  
-> SUM(score.score) AS total,  
  
-> AVG(score.score) AS average,  
  
-> COUNT(score.score) AS count  
  
-> FROM score INNER JOIN grade_event  
  
-> ON score.event_id = grade_event.event_id  
  
-> GROUP BY grade_event.date;
```

date	category	minimum	maximum	span	total	average	count
2012-09-03	Q	9	20	12	439	15.1379	29
2012-09-06	Q	8	19	12	425	14.1667	30
2012-09-09	T	60	97	38	2425	78.2258	31
2012-09-16	Q	7	20	14	379	14.0370	27
2012-09-23	Q	8	20	13	383	14.1852	27
2012-10-01	T	62	100	39	2325	80.1724	29

虽然GROUP BY列带有限定符，但对于这条查询语句来说并不是必须的。GROUP BY引用了多个输出列，不过名叫date的列只有一个，因此MySQL可以清楚地知道你所指的是哪一个。

即使有源自多个表的多个列，你也可以使用像 COUNT() 和AVG() 这样的函数，为它们生成汇总信息。下面这个查询可以为事件日期与学生性别的每种组合，确定出各分数的数目，以及平均分数：

```
mysql> SELECT grade_event.date, student.sex,  
-> COUNT(score.score) AS count, AVG(score.score) AS average  
-> FROM grade_event INNER JOIN score INNER JOIN student  
-> ON grade_event.event_id = score.event_id  
-> AND score.student_id = student.student_id  
-> GROUP BY grade_event.date, student.sex;
```

date	sex	count	average
2012-09-03	F	14	14.6429
2012-09-03	M	15	15.6000
2012-09-06	F	14	14.7143
2012-09-06	M	16	13.6875
2012-09-09	F	15	77.4000
2012-09-09	M	16	79.0000
2012-09-16	F	13	15.3077
2012-09-16	M	14	12.8571
2012-09-23	F	12	14.0833
2012-09-23	M	15	14.2667
2012-10-01	F	14	77.7857
2012-10-01	M	15	82.4000

我们可以用一条类似的查询语句来完成成绩考评项目的其中一项任务，即在期末的时候计算每个学生的总成绩：

```
SELECT student.student_id, student.name,
SUM(score.score) AS total, COUNT(score.score) AS n
FROM grade_event INNER JOIN score INNER JOIN student
ON grade_event.event_id = score.event_id
AND score.student_id = student.student_id
GROUP BY score.student_id
```

```
ORDER BY total;
```

成绩考评项目的另一项任务是汇总所有学生的缺勤情况。所有缺勤情况都记录在absence表里，其中包括学生ID和日期。为获得学生的名字（不只是ID），我们必须基于student_id的值，将absence表连接到student表上。下面这个查询可以列出学生的ID号和姓名，以及缺勤情况：

```
mysql> SELECT student.student_id, student.name,
```

```
-> COUNT(absence.date) AS absences
```

```
-> FROM student INNER JOIN absence
```

```
-> ON student.student_id = absence.student_id
```

```
-> GROUP BY student.student_id;
```

student_id	name	absences
3	Kyle	1
5	Abby	1
10	Peter	2
17	Will	1
20	Avery	1

如果你只想知道有哪些学生缺勤，那么这个查询输出已能满足需要。如果要把这个列表交到学校办公室，那么他们可能会问：“其他学生的情况怎么样呢？我们希望看到每位学生的情况。”这就是个有所不同的问题了。它既要求统计缺勤学生的数量，也要求统计无缺勤情况学生的数量。既然这个问题有所不同，那么回答这个问题的查询也就会有所不同。

使用内连接（inner join）来回答此问题并不合适，我们需要使用LEFT JOIN子句。该子句会告知MySQL，对于连接里的第一个表（即LEFT JOIN关键字左边的那个表），为从其里面查询出的每一行产生一个输出行。通过将student表指定为第一个表，我们将能获得每位学生的输出，其中甚至包括那

些absence表里没有的学生。在编写此查询语句时，可以在FROM子句里的两个表之间使用LEFT JOIN（而不是用逗号把这两个表分开），然后使用ON子句说明如何匹配这两个表中的行。此查询如下所示：

```
mysql> SELECT student.student_id, student.name,  
-> COUNT(absence.date) AS absences  
-> FROM student LEFT JOIN absence  
-> ON student.student_id = absence.student_id  
-> GROUP BY student.student_id;
```

student_id	name	absences
1	Megan	0
2	Joseph	0
3	Kyle	1
4	Katie	0
5	Abby	1
6	Nathan	0
7	Liesl	0

...

连接操作并非只能用于两个不同的表。这乍听起来有点儿奇怪，但你完全可以把某个表与其自身连接起来。例如，想确定是否有某位总统与另一位总统出生在同一个城市，这时便需要检查每位总统的出生地与其他总统的出生地是否一致：

```
mysql> SELECT p1.last_name, p1.first_name, p1.city, p1.state  
-> FROM president AS p1 INNER JOIN president AS p2
```

-> ON p1.city = p2.city AND p1.state = p2.state

-> WHERE (p1.last_name <> p2.last_name OR p1.first_name <> p2.first_name)

-> ORDER BY state, city, last_name;

last_name	first_name	city	state
Adams	John Quincy	Braintree	MA
Adams	John	Braintree	MA

这条查询命令有以下两个地方需要特别注意。

它需要引用同一个表中的两个实例，因此我们必须为它创建两个别名（p1和p2），并用它们来将表中的同名列区别开来。由于列已有别名，所以在为表指定别名时， AS关键字就是可选项了。

每位总统的记录都与其本身相匹配，但这并不是我们想要的输出结果。在确保参与比较的总统名字都不相同的情况下， WHERE子句便能防止出现“行与其本身相匹配”的情况。

用一个类似的查询可以查出在同月同日出生的总统。不过，如果直接比较某两位总统的出生日期，那么查询结果里就会缺少那些生于同月同日但不同年的总统。因此，我们必须用函数MONTH() 和DAYOFMONTH() 来比较出生日期里的月和日：

```
mysql> SELECT p1.last_name, p1.first_name, p1.birth
```

-> FROM president AS p1 INNER JOIN president AS p2

-> WHERE MONTH(p1.birth) = MONTH(p2.birth)

-> AND DAYOFMONTH(p1.birth) = DAYOFMONTH(p2.birth)

-> AND (p1.last_name <> p2.last_name OR p1.first_name <> p2.first_name)

-> ORDER BY p1.last_name;

last_name	first_name	birth
Harding	Warren G.	1865-11-02
Polk	James K.	1795-11-02

使用DAYOFYEAR()来代替MONTH()与DAYOFMONTH()的组合，可以得到一个稍微简单一点儿的查询语句。但其查询结果却可能不正确，因为它没有考虑到出现闰年的情况。

另一种类型的多表检索操作是使用“子查询”，即把一条SELECT语句嵌套在另一条SELECT语句里。子查询有几种类型，详细内容将在2.9节进行讨论。我们现在只看两个示例。假设需要把全勤的学生都找出来。此要求等价于把没在absence表里出现过的学生都找出来，因此我们可以这样做：

```
mysql> SELECT * FROM student
```

-> WHERE student_id NOT IN (SELECT student_id FROM absence);

name	sex	student_id
Megan	F	1
Joseph	M	2
Katie	F	4
Nathan	M	6
Liesl	F	7

...

嵌套于内层的SELECT语句会确定出absence表里的student_id值集合，而外层的那个SELECT语句则会检索出student表里与该集合中的ID值都不匹配的那些行。

子查询还能解决1.4.9.8节提出的那个问题（即有哪些总统出生在Andrew Jackson总统之前）提供一种单语句解决方案。当时的解决方案是使用两条语句和一个用户变量，而现在可以用一条子查询语句来解决，如下所示：

```
mysql> SELECT last_name, first_name, birth FROM president
```

```
-> WHERE birth < (SELECT birth FROM president
```

```
-> WHERE last_name = 'Jackson' AND first_name = 'Andrew');
```

last_name	first_name	birth
Washington	George	1732-02-22
Adams	John	1735-10-30
Jefferson	Thomas	1743-04-13
Madison	James	1751-03-16
Monroe	James	1758-04-28

内层的SELECT语句用来确定Andrew Jackson的出生日期，外层SELECT语句负责检索出生日期早于该日期的总统。

1.4.10 删除或更新已有行

有时候，你会需要删除某些行，或者修改其内容。这时便需要用到DELETE语句和UPDATE语句。

DELETE语句的基本格式如下所示：

```
DELETE FROM tbl_name
```

```
WHERE which rows to delete;
```


其中，WHERE子句是可选的，它用于指定需要删除掉哪些行。如果没有WHERE子句，那么将删除表里的所有行。这意味着，越简单的DELETE语句越危险，例如下面这条语句：

```
DELETE FROM tbl_name;
```

它会彻底删除表里的所有内容，因此请务必小心使用！如果只想删除特定的行，那么必须先使用WHERE子句把它们筛选出来。这种做法与在SELECT语句里使用WHERE子句来避免查询出整个表的做法相类似。例如，在president表里，只想将出生于Ohio的美国总统删除掉，那么可以使用下面这条查询语句：

```
mysql> DELETE FROM president WHERE state='OH';
```

```
Query OK, 7 rows affected (0.01 sec)
```

如果你不太清楚某条 DELETE 语句到底会删除哪些行，那么最好先把这条语句的WHERE子句放到一条SELECT语句里，看看这条SELECT语句能查出哪些行。这有助于你确认：它们的确是你想要删除的那些行，而且只有这些，不能多也不能少。假设想要删除Teddy Roosevelt总统对应的那一行。那么下面这条语句是否能完成此项工作呢？

```
DELETE FROM president WHERE last_name='Roosevelt';
```

这条语句确实能删除掉 Teddy Roosevelt 总统对应的那一行，但它同时也会删除掉 Franklin Roosevelt总统对应的那一行。因此，出于安全的考虑，最好先把这个WHERE子句放到SELECT语句里检查一下，就像这样：

```
mysql> SELECT last_name, first_name FROM president
```

```
-> WHERE last_name='Roosevelt';
```

```
-----+-----  
last_name | first_name  
-----+-----  
Roosevelt | Theodore  
Roosevelt | Franklin D.  
-----+-----
```

从上面的查询结果可以看出，还需要在条件里加上总统的名，从而让它更具体一点：

```
mysql> SELECT last_name, first_name FROM president
```

```
-> WHERE last_name='Roosevelt' AND first_name='Theodore';
```

```
-----+-----  
last_name | first_name  
-----+-----  
Roosevelt | Theodore  
-----+-----
```

现在，你应该知道如何使用WHERE子句正确地标识那些想要删除的行。下面是更正后的DELETE语句：

```
mysql> DELETE FROM president
```

```
-> WHERE last_name='Roosevelt' AND first_name='Theodore';
```

表面看起来似乎是为了删除一行而输入了很多的内容，但事实上稳妥一点总是胜过事后来后悔。如果真的遇到这种情况，则可以通过复制粘贴或者输入行编辑（line-editing）技术，尽可能减少输入操作。更多信息请参考1.5节。

如果想修改已有记录行，则需要使用UPDATE语句，其基本格式如下：

```
UPDATE tbl_name
```

```
SET which columns to change
```

```
WHERE which rows to update;
```

这里WHERE子句的情况与DELETE语句里的相似。它是可选的，因此在没有指定它时，会更新表里的每一行。例如，下面这个查询会把每一位学生的名字都更改为George：

```
mysql> UPDATE student SET name='George';
```

很显然，必须谨慎对待这类查询，所以通常情况下都需要加上一个WHERE子句，用它来更具体地指出需要更新哪些行。假设你最近往“美史联盟”里新增了一名成员，但只填写了很少的几栏信息：

```
mysql> INSERT INTO member (last_name,first_name)
-> VALUES('York','Jerome');
```

接着，你发现自己忘了为他设置成员资格有效日期。此时，可以用一条UPDATE语句来进行修复，其中包含一条WHERE子句，用以标识需要更新的那行：

```
mysql> UPDATE member
-> SET expiration='2013-7-20'
-> WHERE last_name='York' AND first_name='Jerome';
```

你可以在一条语句里同时更新多个列。下面这条UPDATE语句将更新Jerome的电子邮件地址和邮箱地址：

```
mysql> UPDATE member
-> SET email='jeromey@aol.com', street='123 Elm St',
-> city='Anytown', state='NY', zip='01003'
-> WHERE last_name='York' AND first_name='Jerome';
```

你也可以将某列的值设置为NULL（如果该列允许使用NULL值的话），从而使它处于“未设置”状态。例如，Jerome在后来的某个时候支付了一大笔成员资格更新费用，足以让他成为终身成员。那么，这时你可以通过将其有效日期设置为NULL，把他的那一行标记为“永不过期”：

```
mysql> UPDATE member
-> SET expiration=NULL
-> WHERE last_name='York' AND first_name='Jerome';
```

对于UPDATE语句，与DELETE语句一样，为了确保能准确地把你想要更新的行全都筛选出来，最好先把它的WHERE子句放到SELECT语句里去确认一下。

否则，如果检索条件过于严格或过于宽松，便会出现少更新行或多更新行的情况。

如果你练习过本节这些语句，那么你肯定已在sampdb数据库的许多表里，删除或者更新了某些行。在进入下一节之前，你应该恢复这些修改。请参考1.4.8节中的说明，使用之前提供的那些命令重新加载所有表。

1.5 mysql交互技巧

本节将介绍一些与客户端程序mysql进行交互的技巧，这些技巧能帮助我们更有效率地使用它，减少文字输入。此外，还会描述“怎样更加方便地连接到服务器”，以及“如何在不经手工输入的情况下输入语句”。

1.5.1 简化连接过程

在启动mysql程序时，通常都需要设定某些连接参数，如主机名、用户名或密码。如果在每次启动mysql程序时都输入这么多的内容，那么你很快就会感到厌烦。其实，在连接MySQL服务器时，有好几种办法可以减少打字输入的需要。

把连接参数存储在一个选项文件里。

利用shell的历史命令功能输入重复命令。

利用shell别名或脚本定义一个mysql命令行快捷方式。

1.5.1.1 使用选项文件

MySQL允许把连接参数存储到一个选项文件里。这样，你就不用在每次启动mysql时都输入这些参数了。它们的使用效果与在命令行上输入它们是一样的。这样做的好处在于，其他MySQL客户端程序（如mysqlimport或mysqlshow）也能使用这些参数。换句话说，选项文件不仅能简化mysql的启动过程，也可以让许多其他程序受益。本节对“如何设置客户端程序所用的选项文件”进行了简要介绍。有关更多详细信息请参考F.2.2节。

在Unix系统上，可以创建一个名为~/.my.cnf的文件（即在你的主目录下创建一个名为.my.cnf的文件）来作为选项文件。在Windows系统上，可以在MySQL安装目录下或者在C盘根目录下，创建一个名为my.ini的选项文件（如果在C盘根目录下，就是C:\my.ini）。选项文件是一个纯文本文件，因此可以使用任何一种文本编辑器来创建。选项文件的内容近似如下：

```
[client]
```

```
host=server_host
```

```
user=your_name
```

```
password=your_pass
```

其中，`[client]`行是`client`选项组的开始标记。MySQL客户端程序会由该行开始读入各行，一直到文件末尾或遇到一个不一样的选项组起始标记为止，从而获得各个选项值。请把其中的`server_host`、`your_name`和`your_pass`分别替换为你在连接MySQL服务器时所用的主机名、用户名和密码。例如，服务器运行在`cobra.example.com`主机上，MySQL的用户名和密码分别是`sampadm`和`secret`，那么请在`.my.cnf`文件里放入下面几行内容：

```
[client]
```

```
host=cobra.example.com
```

```
user=sampadm
```

```
password=secret
```

那个`[client]`行是必需的，用于定义选项组的起始位置。不过，那些用来定义参数值的文本行都是可选的，你可以只列出所需要的那些参数值。例如，你使用的是Unix系统，并且MySQL用户名就是Unix的登录名，那么可以不用包含`user`那一行。默认的主机是`localhost`，因此如果你要连接的服务器运行在本地主机上，那么`host`那一行也可以不要。

如果是在Unix系统上，那么在创建选项文件之后还需要为它设置访问权限，从而保证其他人不能读取和修改它。下面两条命令都可以实现让选项文件仅限你本人访问：

```
% chmod 600 .my.cnf
```

```
% chmod u=rw,go-rwx .my.cnf
```

1.5.1.2 利用shell的历史命令功能

有些shell程序，如`tcsh`或`bash`，能把你在命令行上输入过的命令记录在一个历史清单里，从而可以让你重复其中的命令。如果你使用的是这类shell程序，那么这个历史清单可以帮你避免输入整条的命令。例如，在你刚使用过`mysql`之后，你可以像下面那样来执行它：

```
% !my
```

感叹号字符(!)的作用是告知 shell 程序，让它在命令历史清单里，把你最近执行过的以my 开头的命令找出来，再执行一次，就好像你在命令行上再次自己输入了它一样。有些 shell 程序还支持使用键盘上的上下箭头键（或组合键Ctrl+P和Ctrl+N）在命令历史清单里前后移动。这时，在当中选择一条命令之后，按Enter键即可执行它。tcsh和bash具有这样的功能，其他 shell 可能也有。如果你想了解更多如何使用历史命令列表的信息，可以查阅你所用的shell程序的帮助文档。

1.5.1.3 利用shell别名和脚本

如果shell程序支持别名机制，那么你可以将一个较短的命令名映射到一条较长的命令上。例如，在csh或tcsh里，你就可以像下面这样用alias命令来创建一个名为sampdb的别名：

```
alias sampdb 'mysql -h cobra.example.com -p -u sampadm sampdb'
```

bash的语法稍有不同：

```
alias sampdb='mysql -h cobra.example.com -p -u sampadm sampdb'
```

利用别名定义之后，下面两条命令完全等价：

```
% sampdb
```

```
% mysql -h cobra.example.com -p -u sampadm sampdb
```

很明显，第一条命令比第二条简短很多。如果想让这个别名在你每次登录系统时都能生效，那么还需要把alias命令放到shell程序的启动文件里（例如，tcsh的启动文件是.tcshrc，而bash下的启动文件是.bashrc或.bash_profile）。

在Windows系统上，也有类似的技巧，即可以先创建一个快捷方式指向mysql程序，然后再编辑该快捷方式的属性，让它包含相关的连接参数。

另一种能让你在调用命令时少打字的方法是，创建一个脚本，让它带着恰当的选项执行mysql。在Unix系统上，一个shell脚本与刚才定义的命令别名sampdb是等价的：

```
#!/bin/sh
```

```
exec mysql -h cobra.example.com -p -u sampadm sampdb
```

如果把这个脚本命名为sampdb，并将其设置为可执行的（使用chmod +x sampdb命令），那么在命令提示符里输入sampdb，即可启动mysql，并连接到sampdb数据库。

在Windows系统上，可以使用批处理文件来完成同样的事情。先创建一个名为sampdb.bat的批处理文件，然后再把下面的内容放入其中：

```
mysql -h cobra.example.com -p -u sampadm sampdb
```

执行这个批处理文件的办法有两种：一种是在控制台窗口的提示符处输入sampdb；另一种是双击这个批处理文件的Windows图标。

如果需要访问多个数据库或连接多个主机，那么你可以多定义几个别名、快捷方式或者脚本，每一个都以不同的选项参数来调用mysql程序。

1.5.2 在调用命令时减少打字输入

从对数据库进行交互式查询的角度讲，mysql 是一个极其有用的程序，但其操作界面却主要适合于输入短小的单行查询命令。虽然mysql自身并不关心输入的查询命令是否会延续多少行，但输入一条长长的查询命令却不是件让人高兴的事。如果因为语法错误而不得不重新输入一遍，那么肯定会让人更郁闷不已。有几种技术可以帮助我们减少不必要的录入工作。

利用mysql的输入行编辑功能。

利用复制和粘贴。

利用批处理运行mysql程序。

1.5.2.1 利用mysql的输入行编辑器

mysql 程序支持输入行编辑。你可以编辑当前正在输入的那一行，也可以把以前的输入行调出来，直接再次输入它们，或者在进一步修改之后输入它们。当在输入的命令行中有打字错误的时候，使用这个功能会很方便：在按 Enter 键之前，可以把光标移到出错位置，对问题进行修正。如果你输入了一个有错误的查询命令，那么可以把它重新调出来，编辑修正问题，然后再提交。（如果你是在一行里输入了整条查询命令，那么修改起来是最容易的。）

表1-4列出了一些适用于Unix系统的按键组合。

表1-4 mysql程序的输入编辑命令

按键组合	含 义
向上键或Ctrl+P	调出前一个输入行
向下键或Ctrl+N	调出后一个输入行
向左键或Ctrl+B	向左移动光标
向右键或Ctrl+F	向右移动光标
Esc b	把光标向后移动一个单词
Esc f	把光标向前移动一个单词
Ctrl+A	把光标移到输入行的开头
Ctrl+E	把光标移到输入行的末尾
Ctrl+D	删除光标位置上的那个字符
Delete	删除光标前面（左侧）的那个字符
Esc D	删除单词
Esc Backspace	删除光标前面（左侧）的那个单词
Ctrl+K	删除从光标位置到输入行末尾的所有内容

在Windows平台上，mysql程序并没有提供行编辑功能。不过，Windows自身支持许多编辑命令，如表1-5所示，因此mysql程序可以直接使用它们。

表1-5 Windows的输入编辑命令

按键组合	含 义
向上键	调出前一行
向下键	调出后一行
向左键	光标左移一个字符（后退）
向右键	光标右移一个字符（前进）
Ctrl + 向左键	光标左移一个单词
Ctrl + 向右键	光标右移一个单词
Home	光标移动到行首

续表

按键组合	含 义
End	光标移动到行尾
Delete	删除光标处的字符
Backspace	删除光标左边的字符
Esc	删除整行
Page Up	调出最早输入的命令
Page Down	调出最后输入的命令
F3	调出最后输入的命令
F7	弹出命令菜单，用向上键/向下键选择
F9	弹出命令菜单，用命令编号选择
F8, F5	循环显示命令列表

下面的示例演示了一个输入行编辑功能的简单用法。假设你在mysql程序里输入了如下所示的查询命令：

```
mysql> SHOW COLUMNS FROM persident;
```

在按Enter键之前，你突然发现自己把president错拼成了persident，那么可以像下面这样修改这条查询命令。

- (1) 先按几次向左键，把光标左移到字符s的位置上。
- (2) 按两次Delete键或Backspace键（这两个键都可以删除光标左侧的字符），删除er。
- (3) 再重新输入re，修正错误。
- (4) 然后按下Enter键来执行该查询命令。

如果在按Enter键之后才发现打错了字，那么也不要紧。等到mysql显示出错信息之后，按向上键调出刚才输入的那条查询命令，然后像刚才描述的那样将其修改好即可。

1.5.2.2 利用复制和粘贴来执行语句

如果你使用的是窗口化操作环境，那么可以把你认为有用的那些语句保存到某个文件里，其后通过复制和粘贴操作再次调用它们。

- (1) 在某个终端或控制台窗口里启动mysql程序。
- (2) 在文档窗口里打开那个用来存放语句的文件。（例如，我在Unix系统上用vi，在Windows系统上用gvim。）
- (3) 在这个文件里找到你想要执行的语句，选取并复制它。然后切换到终端窗口，把刚才复制的那条语句粘贴到mysql程序里。

这一过程看起来比较繁琐，但在掌握之后却相当快捷。它提供了一种无需打字，即可快速输入命令的方式。稍加练习，即可熟练掌握。

你也可以将复制和粘贴的顺序反过来使用（把终端窗口里的语句复制粘贴到你的文件里）。在Unix系统上，当在mysql里输入语句时，这些语句会被保存到位于主目录下的一个名为.mysql_history的文件里。如果你手工输入了一条语句之后，想把它保存起来以供今后使用，那么可以这样做：先退出mysql，再用一个编辑器打开.mysql_history，然后将.mysql_history里的语句复制粘贴到你的那个语句存档文件里。

1.5.2.3 利用mysql执行脚本文件

mysql程序并非只能在交互模式下运行，它也可以在非交互模式（即批处理）下从某个文件里读取输入。如果你有一些需要定期运行的语句，那么

此方法会特别有用，因为你不用每次都来重新输入它们。只要在一个文件中保存一次这些语句，就可以根据需要反复地让mysql执行它们。

假设你有一条查询命令，它可以通过member表里的interests列，将那些对某个特定时期的美国历史感兴趣的“美史联盟”成员找出来。例如，想要将那些对Great Depression（美国20世纪 30年代的那个萧条时期）感兴趣的成员找出来，那么可以编写出类似下面这样的查询命令：

```
SELECT last_name, first_name, email, interests FROM member  
  
WHERE interests LIKE '%depression%'  
  
ORDER BY last_name, first_name;
```

把这条查询命令保存在interests.sql 文件里，然后像下面这样将它提供给 mysql 程序执行：

```
% mysql sampdb < interests.sql
```

默认情况下，当以批处理模式运行时，mysql 程序产生的输出内容是以制表符分隔的。如果想得到交互方式运行mysql时输出的格式效果，可以加上一个-t选项：

```
% mysql -t sampdb < interests.sql
```

如果想把输出结果保存起来，那么可以将它重定向至一个文件：

```
% mysql -t sampdb < interests.sql > interests.out
```

如果已经运行了mysql，那么可以通过source命令来执行指定文件里的内容：

```
mysql> source interests.sql
```

如果需要将那些对Thomas Jefferson总统的生平感兴趣的成员查找出来，那么可以先编辑这个文件，把其中的depression改为Jefferson，然后再次运行mysql即可。不过，这个方法只有在你不会频繁地使用这条查询命令时才会显现出优势来。如果要频繁地运行某条查询命令，那么需要另谋出路。在Unix系统上，有一种让查询命令变得更加灵活的方法：先把它保存为一个可以接受脚本命令行参数的shell脚本；然后，用这些命令行参数来更改查询命令的文本内容。这种参数化查询命令的方式，让你可以在运行脚本时指定

interests 值。通过下面这个shell脚本interests.sh, 可以看出其中的工作原理:

```
#!/bin/sh

# interests.sh - find USHL members with particular interests

if [ $# -ne 1 ]; then echo 'Please specify one keyword'; exit; fi

mysql -t sampdb <<QUERY_INPUT

SELECT last_name, first_name, email, interests FROM member

WHERE interests LIKE '%$1%'

ORDER BY last_name, first_name;

QUERY_INPUT
```

其中的第 3行用于确保命令行参数只有一个; 否则, 它就会输出一条简短的出错信息, 然后退出执行。在<<QUERY_INPUT和最后那个QUERY_INPUT 之间的所有内容, 都会成为 mysql的输入。shell 程序会把这段查询命令文本里的脚本变量\$1 替换为源自命令行的那个参数(在shell程序里, 变量\$1、\$2、... 依次对应于命令行里的各个参数)。这样, 在运行这个脚本时, 命令行的参数将成为这条查询命令中的检索关键字。

在运行该脚本之前, 还必须把它设置为可执行的:

```
% chmod +x interests.sh
```

现在, 你不用在每次运行这个脚本时都去编辑它了。只需通过命令行参数, 告诉它你想查找什么东西即可:

```
% ./interests.sh depression
```

```
% ./interests.sh Jefferson
```

你可以在sampdb发行版的misc目录里找到这个interests.sh脚本。另外, 在那里还提供了与之等效的Windows批处理文件interests.bat。

说明

强烈建议大家不要把这类脚本安装在公共区域里，因为它们在执行时不会对参数进行安全检查，因而很容易遭遇SQL注入攻击。假设有人调用类似下面的脚本发起攻击：

```
% ./interests.sh "Jefferson';DROP DATABASE sampdb;"
```

此脚本的后果是会有一条DROP DATABASE语句被注入脚本语句中，成为mysql的输入内容。如此一来，这条命令会真的被执行，其后果可想而知。

1.6 何去何从

通过本章的学习，相信你对 MySQL 的使用已有所了解。你可以创建数据库和表，向表中插入行，用各种方式检索这些行，修改它们，或者删除它们。但本章的这个教程只能算是“蜻蜓点水”，还有很多内容并未讲到。请仔细想想，sampdb数据库当前到什么状态了呢？我们把它和它的表创建好了，并且还使用了一些初始数据来填充其中的各个表。在这个过程中，我们还编写了一些查询命令，用于回答那些与数据库信息有关的问题。不过，还有很多事情在等着我们去完成。例如，到目前为止，我们还没有一种简便的交互方式，用于为成绩考评项目输入新的分数行，或者为“美史联盟”添加新的成员。我们还没有可以方便地修改已有行的方式。而且，我们还不能以打印版和在线版两种形式生成“美史联盟”的成员名录。在后面的章节（尤其是第8章和第9章）中，我们将继续完成这些任务。

接下来你想要跳转到本书的哪一章节呢？这要取决于你的兴趣。如果你只想知道如何完成“美史联盟”和“成绩考评项目”里的各项任务，那么可以直接跳转到本书的第二部分，那里会讲到如何编写MySQL应用程序。如果你打算成为网站的MySQL数据库管理员，那么可以直接跳转到本书的第三部分，那里会对数据库的管理工作进行探讨。不过，建议大家先按部就班地读完第一部分，多积累一些 MySQL 在使用方面的背景知识。这些内容将帮助大家进一步了解SQL语句的语法和用法，弄清 MySQL是如何处理数据的，怎样才能让查询执行得更快。无论你是想要运行 mysql，想要自己编写这样的程序，还是想要成为一名数据库管理员，牢牢掌握这些内容总会有收获。

第2章

使用SQL管理数据

MySQL服务器支持SQL。因此，我们可以通过SQL来告诉MySQL服务器如何完成各种数据管理操作。要想与服务器进行有效的交流，则必须熟练掌握它。当使用某个程序（如mysql客户端）时，它本质上就是一种把SQL语句发送到服务器执行的工具。如果使用某种带有MySQL接口（如Perl语言的DBI模块，或者PHP语言的PDO扩展）的编程语言来编写程序，那么这些接口能让你通过调用SQL语句来与服务器进行交流。

第1章介绍了MySQL的许多功能，以及SQL的某些基本用法。现在，我们将在那个基础上深入介绍更多与SQL语言有关的内容，主要涉及以下几个方面。

- 调整SQL模式以影响服务器的行为。

- 引用数据库的各种元素。

- 使用多种字符集。

- 创建和销毁数据库、表和索引。

- 获得关于数据库及其内容的信息。

- 使用连接、子查询和联合来检索数据。

- 多表删除和更新。

- 利用事务一次性执行或撤销多条语句。

- 建立外键关系。

使用FULLTEXT搜索引擎。

上面列出的这些条目覆盖了 SQL 语言的众多应用领域。其他章节也提供了一些附加的与SQL相关的信息。

第4章将讨论如何创建和使用视图（这是种虚表，它提供了另一种查看数据的方式），以及如何创建和使用存储程序（包括函数、过程、触发器和事件）。

第12章将会讲述如何使用各种管理语句（如GRANT和REVOKE）去管理用户账户。还会讨论到权限控制系统，此系统控制着各个账户都被允许执行什么样的操作。

附录E列出了通过MySQL实现的各条SQL语句的语法，以及使用它们所要求的权限。它还介绍了在SQL语句里使用注释的语法。

如果想要了解更多相关信息，尤其想要了解 MySQL的最新版本所做的那些更改，则可以参考MySQL参考手册（MySQL Reference Manual）。

2.1 服务器的SQL模式

MySQL服务器的SQL模式对SQL语句的执行会产生几个方面的影响，在服务器上有一个名为 `sql_mode` 的系统变量，你可以用它来配置这个模式。可以全局性地设置这个变量，让它对所有客户端产生影响；也可以让每个客户端自己更改这个模式，从而只对其自己与（它所连接的那个）服务器的会话产生影响。这意味着，任何客户端都可以在不影响其他客户端的前提下，更改服务器对待它自己的行为。

SQL模式会影响一些行为，比如，如何处理数据录入过程中的无效值，以及标识符的引号方式。下面列出了几种可能的SQL模式设置值。

`STRICT_ALL_TABLES`和`STRICT_TRANS_TABLES`将启用“严格”模式。在严格模式下，MySQL服务器将会更加严格地对待接受到的坏（bad）数据。（具体来讲，它不会把这些坏数据值转换为最为接近的有效值，而是会拒绝接受它们。）

`TRADITIONAL`是一个组合模式。它类似于严格模式，但它还会启用几种其他模式，这些模式会引入更多的附加限制条件，从而实现更加严格的数据检查。`TRADITIONAL` 模式可以让MySQL服务器在处理坏数据时，其行为更接近于那些传统的SQL服务器。

ANSI_QUOTES会告诉MySQL服务器，把双引号识别为一个标识符引用字符。

PIPES_AS_CONCAT会让服务器把“||”当成一个标准的SQL字符串连接运算符，而不会把它当成是OR运算符的同义词。

ANSI也是一个组合模式。它会同时启用ANSI_QUOTES、PIPES_AS_CONCAT和其他的几个模式值，其结果是让MySQL服务器的行为比它的默认运行状态更接近于标准SQL。

在设置SQL模式时，需要指定一个由单个模式值或多个模式值（用逗号分隔）构成的值，或者指定一个空串，用以清除该值。模式值不区分大小写。

如果想在启动服务器时设置SQL模式，那么可以在mysqld命令行，或者在某个选项文件里设置系统变量sql_mode。在这个命令行里，可以使用类似下面这样的设置：

```
--sql-mode="TRADITIONAL"
```

```
--sql-mode="ANSI_QUOTES,PIPES_AS_CONCAT"
```

如果想在运行时更改SQL模式，那么可以使用一条SET语句来设置sql_mode系统变量。任何客户端都可以为其自己设置一个本次会话特定的SQL模式：

```
SET sql_mode = 'TRADITIONAL';
```

如果想设置全局性的SQL模式，则需要加上GLOBAL关键字：

```
SET GLOBAL sql_mode = 'TRADITIONAL';
```

设置全局变量需要具备SUPER管理权限。新设置的全局变量值将成为此后连入客户端的默认SQL模式。

如果想知道当前会话或全局的SQL模式值，则可以使用如下语句：

```
SELECT @@SESSION.sql_mode;
```

```
SELECT @@GLOBAL.sql_mode;
```

其返回值由当前设置为启用的所有模式构成，在两个模式之间以逗号隔开。如果当前没有启用任何模式，则返回一个空值。

在3.3节将讨论一些SQL模式值，它们会影响到对“数据录入期间出现的值错误或缺失等问题”的处理。附录D对sql_mode变量所支持的所有模式值进行了描述。关于使用系统变量的更多信息请参考12.3.1节。

2.2 MySQL的标识符语法和命名规则

几乎所有的SQL语句都会以某种方式使用到标识符，用以引用某个数据库或其构成元素，如表、视图、列、索引、存储例程、触发器或事件。在引用数据库的各个元素时，相应的标识符必须遵从以下几条规则。

(1) 标识符里的字符要合法。不加引号的标识符可以由大小写形式的字母a-z、数字0-9、美元符号、下划线，以及范围在U+0080到U+FFFF之间的Unicode扩展字符构成。标识符的第一个字符可以是标识符所允许的任何一种字符，其中包括数字。不过，不加引号的标识符不允许完全由数字字符构成，因为那样会使它难以与数字区分开来。MySQL支持标识符以数字字符开头的做法，在各类数据库系统中并不常见。如果想要使用这样的标识符，那么必须特别留意它是否还包含了一个“E”或“e”字符，因为这种组合很容易导致表达式出现歧义。例如，表达式23e + 14（在“+”号的两边都有空格）表示的是列23e加上数字14，那么23e+14又该如何解释呢？它表示的是同一件事，还是一个用科学计数法表示的数字呢？

标识符可以用反引号“`”引起来，它支持使用除NUL字节和Unicode补充字符（范围在U+10000及以上）以外的任何字符：

```
CREATE TABLE `my table` (`my-int-column` INT);
```

如果标识符是SQL保留字，或者包含有空格或其他特殊字符，那么引号就会很有用。为标识符加上引号，可以让它完全由数字字符构成；这一点对不加引号的标识符来说是不允许的。如果想在加上引号的标识符里使用标识符的引号字符，那么只需把重复一下即可。

对于数据库名和表名这样的标识符，操作系统可能会有其他的一些限制。更多相关信息请参考11.2.6节。

列名和表名的别名没什么限制。如果要使用的别名是SQL保留字，或者完全由数字构成，或者包含空格或其他特殊字符，那么应该用标识符引号把它引起来。列的别名还可以使用单引号或双引号。

(2) 服务器的SQL模式。如果启用了SQL的ANSI_QUOTES模式，那么可以使用双引号将标识符引起来（也可以使用反引号）。

```
CREATE TABLE "my table" ("my-int-column" INT);
```

启用ANSI_QUOTES会有副作用，即字符串文字量必须使用单引号引起来。如果使用了双引号，那么服务器会将这个值解释为标识符，而不会把它当作一个字符串。

内建函数的名字通常都不是保留字，因此可以不加引号直接将其用作标识符。不过，如果启用了SQL的IGNORE_SPACE模式，那么函数名就会变成保留字。这时，如果想要把它们当作标识符，则必须使用引号将它们引起来。

关于SQL模式设置的指导请参考2.1节。

(3) 标识符的长度。大部分标识符的最大长度是64个字符。别名的最大长度是256个字符。

(4) 标识符限定符。根据具体的上下文，标识符可能需要加以限定，以明确它所引用的是什么内容。如果想要引用数据库，那么只需直接指定其名字即可：

```
USE db_name;
```

```
SHOW TABLES FROM db_name;
```

如果想要引用表，则有两种选择。

使用一个完全限定的表名，即用数据库标识符和表标识符共同指定：

```
SHOW COLUMNS FROM db_name.tbl_name;
```

```
SELECT * FROM db_name.tbl_name;
```

表标识符自己可以引用默认（当前）数据库里的表。如果sampdb是默认数据库，那么下面两条语句是等效的：

```
SELECT * FROM member;
```

```
SELECT * FROM sampdb.member;
```

如果没有选择数据库，那么在没有给出数据库限定符的情况下引用某个表，就会出现错误；因为还不知道这个表到底属于哪个数据库。

为表名加上限定符的原因同样适用于视图（它们是“虚拟”表）和存储程序的名字。

对于表列的引用，有如下3种选择。

使用完全限定的列名：`db_name.tbl_name.col_name`。

对于默认数据库中命名表里的某一个列，可以使用部分限定名：
`tbl_name.col_name`。

只简单地写一个无限定名来引用具体环境上下文所确定的那个表：
`col_name`。下面两条查询语句都使用了相同的列名，但每条语句的FROM子句所提供的上下文会指出，实际应该选择哪个表里的列：

```
SELECT last_name, first_name FROM president;
```

```
SELECT last_name, first_name FROM member;
```

通常情况下，尽管提供完全限定名总是合法的，但没有必要那样做。如果用USE语句选择一个数据库，那么该数据库就会成为默认数据库，并隐含在你此后所引用的每一个不完整的表名里。如果在某条 SELECT 语句里只引用了一个表，那么该表则会隐含在该条语句所包含的每一个列名里。只有在无法根据上下文确定出是哪一个表或哪一个数据库时，才必须使用完整的标识符。例如，有一条语句涉及多个数据库里的表，那么必须以 `db_name.tbl_name` 语法形式来引用那些不属于默认数据库的表。这样才能让MySQL知道是哪个数据库包含了该表。类似地，如果某条查询语句使用了多个表，并引用了某个在多个表里存在的列名，那么必须用一个表标识符来限定这个列标识符，以明确到底要使用哪一个列。

如果在引用一个完全限定名时使用了引号，那么需要给该名字里的每一个标识符分别加上引号。例如：

```
SELECT * FROM `sampdb`.`member` WHERE  
`sampdb`.`member`.`member_id` > 100;
```

请不要把这种名字当作一个整体用引号引起来。下面这条语句是不正确的：

```
SELECT * FROM `sampdb.member` WHERE `sampdb.member.member_id` >  
100;
```

如果需要把某个保留字用作标识符，那么必须给它加上引号。不过当这个保留字紧跟在一个句号限定符后面时，则不能这样做，因为这时的上下文已说明这个保留字其实就是一个标识符。

2.3 SQL语句的大小写规则

SQL语句中的大小写规则因语句各个组成元素的不同而有所差异，同时还取决于你正引用的内容和MySQL服务器主机所运行的操作系统。

(1) SQL关键字和函数名。SQL的那些关键字和函数名不区分大小写。它们可以为任意的大小写组合。下面这些语句都是等价的：

```
SELECT NOW();
```

```
select now();
```

```
sElEcT nOw();
```

(2) 数据库名、表名和视图名。MySQL使用了服务器主机上的底层文件系统所包含的目录和文件来表示数据库和表。因此，库名和表名的默认大小写取决于服务器主机上的操作系统在文件名方面的规定。Windows系统的文件名不区分大小写，所以运行在它上面的MySQL服务器也不区分库名和表名的大小写。而运行于 Unix 主机上的MySQL 服务器，通常需要区分库名和表名的大小写，因为Unix文件系统区分大小写。对于Mac OS X平台，其扩展文件系统中的名字是个例外，它们不区分大小写。

MySQL会使用一个文件来表示一个视图，所以刚才与表有关的讨论也同样适用于视图。

(3) 存储程序的名字。存储函数、存储过程和事件的名字都不区分大小写。触发器的名字要区分大小写，这一点与标准SQL的行为有所不同。

(4) 列名和索引名。在MySQL里，列名和索引名都不区分大小写。下面这些语句都是等价的：

```
SELECT name FROM student;
```

```
SELECT NAME FROM student;
```

```
SELECT nAmE FROM student;
```

(5) 别名的名字。默认情况下，表的别名要区分大小写。可以使用任意的大小写组合（大写、小写或大小写混用）来指定一个别名，但如果需要在同一条语句里多次用到同一个别名，则必须让它们保持同样的大小写组合。如果lower_case_table_names系统变量为非零值，那么表的别名将不区分大小写。

(6) 字符串值。字符串值是否区分大小写，具体取决于它是二进制还是非二进制串，并且对于非二进制串，还取决于字符集的排序规则

(collation)。对于文字串和字符串类型列的内容，这一点也同样成立。更多相关信息请参考3.1.2节。

当你在一台文件名区分大小写的机器上创建数据库和表时，应该要认真考虑一下大小写的问题，如“它们以后是否有可能迁移到一台不区分文件名大小写的机器上”。假设你在一台Unix服务器上创建了两个名字分别为abc和ABC的表，此时，这两个名字会被认为是不同的。当你想把这两个表迁移到一台Windows机器上时，就会出现问题。因为那台新的机器并不区分大小写，所以abc和ABC无差别。当你把表从一台Unix主服务器复制到一台Windows从服务器时，也同样会遇到麻烦。

想要避免大小写问题演变成一个棘手难题，可行的办法是：先选定一种大小写方案，然后一直按照该方案去创建数据库和表。这样一来，等你日后想把某个数据库迁移到不同的服务器上时，名字的大小写问题就不存在了。个人建议统一使用小写字母，尤其是，如果你要使用InnoDB表的话，因为InnoDB引擎在其内部都是以小写字母方式来存储数据库名和表名的。

如果想统一使用小写字母来创建数据库名和表名，即使在CREATE 语句里没有特意指定，也想要这样的效果，那么可以通过设置lower_case_table_name 系统变量来配置服务器。更多信息请参考11.2.6节。

不管你的系统是否会区分数据库名和表名的大小写，在给定的查询语句里都必须使用与之相同的大小写来引用它们。对于SQL关键字、函数名、列名和索引名，则不必如此，因为MySQL允许在查询命令里使用不同的大小写风格来引用它们。

2.4 字符集支持

MySQL支持多种字符集(character set)，而且允许在服务器、数据库、表、列和字符串常量等不同层次单独指定字符集。例如，你想让某个表的列默认使用latin1 字符集，但同时还想包含一个支持希伯来语(Hebrew)的列和一个支持希腊语(Greek)的列，这都是允许的。此外，你还可以显式地指定排序规则。可以找出 MySQL 具体都支持哪些字符集和排序规则，也可以将数据从一种字符集转换为另一种。

本节提供了与 MySQL的字符集支持有关的基本背景知识。第3章将会具体讨论字符集、排序规则、二进制串和非二进制串，以及“如何定义和使用基于字符的表列”等内容。

MySQL提供了以下这些字符集特性。

MySQL服务器允许同时使用多种字符集。

一个给定的字符集可以有一种或多种排序规则。你可以为自己的应用挑选一种最适用的排序规则。

支持 Unicode的字符集有：utf8和ucs2 字符集，它们包括基本多文种平面（Basic Multilingual Plane, BMP[1]）字符；以及 utf16、utf32和 utf8mb4 字符集，它们包括BMP字符和补充字符。MySQL 5.6.1增加了 utf16le。该字符集与utf16很像，主要差异在于其使用的编码是低字节优先（little-endian），而非高字节优先（big-endian）。

你可以分别在服务器、数据库、表、列和字符串常量等这些层次上指定字符集。

MySQL服务器有一个默认字符集。

可以使用CREATE DATABASE语句来设置数据库的字符集，使用ALTER DATABASE语句来更改它。

CREATE TABLE和ALTER TABLE有专门的子句，用于设定表和列的字符集。

字符串常量的字符集既可以通过上下文来确定，也可以显式指定。

还有几个函数和运算符可用来将一些单独的值从一种字符集转换成另一种。CHARSET()函数可返回某个给定值的字符集。类似地，COLLATE 运算符可更改某个字符串的排序规则，而COLLATE()函数能返回某个给定字符串的排序规则。

SHOW 语句和INFORMATION_SCHEMA 库里的数据表能提供与可用字符集和可用排序规则相关的信息。

当更改某个索引过的字符列时，MySQL服务器会自动对索引进行重新排序。

在一个字符串内部不能混用不同的字符集，也不能为某个给定列的不同行使用不同的字符集。不过，可以使用Unicode字符集（可以用一种编码表示多种语言的字符）来实现多语言支持。

2.4.1 指定字符集

字符集和排序规则可以在多个层次（从 MySQL 服务器所使用的默认字符集，一直到单个字符串所使用的字符集）进行设定。

服务器的默认字符集和排序规则是在编译时构建好的。但你可以在服务器启动或运行时，通过设置系统变量`character-set-server`和`collation-server`来改写它们（相关信息请参考12.6.2节）。如果只是指定字符集，那么其默认排序规则将会成为服务器的默认排序规则。如果指定了排序规则，那么它必须与字符集兼容。如果某种排序规则的名字是以某个字符集的名字作为开头，那么它们就是兼容的。例如，排序规则`utf8_danish_ci`与字符集`utf8`相兼容，但与字符集`latin1`不兼容。

在用于创建数据库和表的SQL语句里，有两个子句可用于指定数据库、表和列的字符集和排序规则，即：

```
CHARACTER SET charset
```

```
COLLATE collation
```

其中，`CHARACTER SET`可用`CHARSET`来代替。`charset`是服务器支持的某个字符集的名字，而`collation`是该字符集的某种排序规则的名字。这两个子句可同时使用，也可分开使用。在同时使用它们两时，必须保证排序规则的名字与字符集相兼容。如果只给出了`CHARACTER SET`子句，则表示使用默认排序规则。如果只给出了`COLLATE`子句，则使用由给定排序规则的名字的开头部分所确定的那个字符集。这些规则适用于几个层次。

如果想在创建数据库时为它指定默认的字符集和排序规则，则可以使用下面这条语句：

```
CREATE DATABASE db_name CHARACTER SET charset COLLATE  
collation;
```

如果没有指定字符集和排序规则，那么这个数据库会使用服务器的默认设置。

如果想为某个表指定默认的字符集和排序规则，则可以在创建该表时使用表选项`CHARACTER SET`和`COLLATION`：

```
CREATE TABLE tbl_name (...) CHARACTER SET charset COLLATE  
collation;
```

如果没有指定字符集和排序规则，那么这个表会使用数据库的默认设置。

对于表里的列，可以使用属性CHARACTER SET和COLLATION显式地为它指定字符集和排序规则。例如：

```
c CHAR(10) CHARACTER SET charset COLLATE collation
```

如果没有给定字符集和排序规则，那么列会使用表的默认设置。这些属性适用于CHAR、VARCHAR、TEXT、ENUM和SET等数据类型。

也可以通过使用COLLATE运算符，按照特定的排序规则对字符串值进行排序。例如，c是一个列，它的字符集为 latin1，排序规则为 latin1_swedish_ci。但是，你想按照西班牙语（Spanish）的排序规则对它排序，这时可以像下面这样做：

```
SELECT c FROM t ORDER BY c COLLATE latin1_spanish_ci;
```

2.4.2 确定可用字符集和当前设置

如果想知道有哪些字符集和排序规则可供选用，可使用下面这些语句：

```
SHOW CHARACTER SET;
```

```
SHOW COLLATION;
```

这两条语句都支持 LIKE 子句，它可以把查询结果的范围缩小到那些字符集或排序规则的名字可以与某个给定的模式相匹配。例如，下面这面这些语句可以分别列出基于拉丁语的字符集以及utf8字符集所提供的那些排序规则：

```
mysql> SHOW CHARACTER SET LIKE 'latin%';
```

Charset	Description	Default collation	Maxlen
latin1	cp1252 West European	latin1_swedish_ci	1
latin2	ISO 8859-2 Central European	latin2_general_ci	1
latin5	ISO 8859-9 Turkish	latin5_turkish_ci	1
latin7	ISO 8859-13 Baltic	latin7_general_ci	1


```
mysql> SHOW COLLATION LIKE 'utf8%';
```

Collation	Charset	Id	Default	Compiled	Sortlen
utf8_general_ci	utf8	33	Yes	Yes	1
utf8_bin	utf8	83		Yes	1
utf8_unicode_ci	utf8	192		Yes	8
utf8_icelandic_ci	utf8	193		Yes	8
utf8_latvian_ci	utf8	194		Yes	8
utf8_romanian_ci	utf8	195		Yes	8
utf8_slovenian_ci	utf8	196		Yes	8

...

排序规则的名字总是以字符集的名字作为开头。每种字符集都最少拥有一种排序规则，并且其中有一种即为它的默认排序规则。

与可用字符集和排序规则有关的信息，还可以从 INFORMATION_SCHEMA 库的 CHARACTER_SETS 表和 COLLATIONS 表里获得（参考 2.7 节）。

如果想显示服务器当前的字符集和排序规则设置，则可以使用 SHOW VARIABLES：

```
mysql> SHOW VARIABLES LIKE 'character\_set\__%';
```

Variable_name	Value
character_set_client	utf8
character_set_connection	utf8
character_set_database	latin1
character_set_filesystem	binary
character_set_results	utf8
character_set_server	latin1
character_set_system	utf8

```
mysql> SHOW VARIABLES LIKE 'collation\_%';
```

Variable_name	Value
collation_connection	utf8_general_ci
collation_database	latin1_swedish_ci
collation_server	latin1_swedish_ci

其中有几个系统变量，会在客户端与服务器的连接建立好之后，对它们之间的通信产生影响。更多信息请参考3.1.2.2节。

2.4.3 Unicode支持

之所以会有这么多字符集，原因之一就是人们为不同的语言制定了不同的字符编码方案。从而会导致好些问题。例如，如果某个给定字符存在于好几种语言里，它在不同的编码方案里就有可能被用不同的数字表示。还有，不同的语言往往要求使用数目不同的字节去表示一个字符。latin1字符集很小，每个字符只使用一个字节来表示即可。但对于某些语言，如日语和汉语，由于包含了非常多的字符，它们的每个字符都需要使用多个字节来表示。

Unicode的目标是提供一种统一的字符编码系统，让所有语言的字符集都能以一种统一的方式进行表示。

utf8与ucs2这两种类型的Unicode字符集，都只包括了BMP里定义的字符，即最多只有65 536个字符。它们都不支持BMP之外的那些补充字符。

ucs2字符集与Unicode的UCS-2编码方案相对应。它使用2个字节来表示1个字符，并且最高有效字节（most significant byte）优先。UCS是通用字符集（Universal Character Set）的缩写。

utf8 字符集采用了一种长度可变的格式，使用1 到 3个字节来表示一个字符。它与UTF-8编码方案相对应。UTF是统一编码转换格式（Unicode Transformation Format）的缩写。

从MySQL 5.5.3版本开始，其他的Unicode字符集都包括了BMP之外的那些补充字符。

字符集 utf16和utf32 类似于 ucs2，只是它们增加了对补充字符的支持。对于utf16，那些BMP字符仍然占2个字节（与usc2一样），补充字符占4个字节。对于utf32，所有字符都占4个字节。

utf8mb4字符集包含了所有的utf8字符（其中，每个字符占1到3个字节），另外也包含了补充字符，其中，每个字条占4个字节。

MySQL 5.6.1增加了对utf16le的支持。该字符集与utf16很像，主要差异在于它使用的是低字节优先，而非高字节优先。

2.5 数据库的选择、创建、删除和更改

MySQL 提供了几条数据库级的语句：USE 用于选定一个默认数据库；CREATE DATABASE用于创建数据库；DROP DATABASE用于删除数据库；ALTER DATABASE用于更改数据库的全局属性。

在涉及DATABASE的所有语句里， SCHEMA关键字与DATABASE同义。

2.5.1 选择数据库

使用USE语句可以选择数据库，并把它指定为MySQL服务器连接的默认（当前）数据库：

```
USE db_name;
```

要想选择数据库，你必须具备相应的访问权限；否则，会出现错误提示。

不是一定要显式地选择数据库。在你没有选择数据库的情况下，只要用数据库名来限定表名，就可以引用该库里的表。例如，你事先没有把 sampdb 选为默认数据库，但是想检索其中 president 表里的内容，那么可以使用如下所示的查询语句：

```
SELECT * FROM sampdb.president;
```

选择了一个数据库，并不意味着它就是此次会话期间的默认数据库。你可以根据需要调用 USE 语句，从而实现在两个数据库之间的任意切换。同时，选择了一个数据库也并不意味着只能使用该库里的表。由于已有了一个默认数据库，因此如果想引用其他库里的表，则需要使用与这些表相对应的那个库标识符来限定它们的名字。

当断开与服务器的连接时，针对该会话的那个服务器默认数据库概念也将不复存在。也就是说，当你再次连接上该服务器时，它并不会记得你上一次所选择的那个默认数据库。

2.5.2 创建数据库

创建数据库的方法是，使用 CREATE DATABASE 语句：

```
CREATE DATABASE db_name;
```

先决条件：这个数据库不能是已存在的，并且必须要拥有 CREATE 权限。

CREATE DATABASE 语句有几个可选子句。完整语法如下：

```
CREATE DATABASE [IF NOT EXISTS] db_name  
[CHARACTER SET charset] [COLLATE collation];
```

在默认情况下，当你试图创建一个已存在的数据库时，系统会报错。如果想要避免这类错误，并且只想在给定数据库尚不存在时才创建它，那么可以加上 IF NOT EXISTS 子句：

```
CREATE DATABASE IF NOT EXISTS db_name;
```

在默认情况下，服务器的字符集和排序规则将成为新建数据库的默认字符集和排序规则。显式设置数据库属性的方法是，使用 CHARACTER SET 子句和

COLLATE子句。例如：

```
CREATE DATABASE mydb CHARACTER SET utf8 COLLATE
utf8_icelandic_ci;
```

如果只给出了CHARACTER SET，而没有提供COLLATE，则意味着使用这个给定字符集的默认排序规则。如果只给出了COLLATE，而没有提供CHARACTER SET，则意味着使用排序规则的名字的开头部分来确定的字符集。

这个字符集必须是服务器支持的若干字符集中的一个，如latin1或sjis。排序规则应该是那个给定字符集的排序规则。关于字符集和排序规则的进一步讨论请参考2.4节。

在创建表时，MySQL 服务器会在其数据目录下创建一个目录，其名与数据库名相同。这个新目录被称为数据库目录。服务器还会在这个目录下创建一个名为db.opt的文件，用于存储一些属性，如数据库的字符集和排序规则。当以后在这个数据库里创建表时，如果表的定义里没有指定其自己的默认字符集和排序规则，那么数据库的这些默认设置都会成为该表的默认设置。

查看现有数据库定义的方法是，使用SHOW CREATE DATABASE语句：

```
mysql> SHOW CREATE DATABASE mydb\G

***** 1.row *****

Database: mydb

Create Database: CREATE DATABASE `mydb`

/*!40100 DEFAULT CHARACTER SET utf8

COLLATE utf8_icelandic_ci */
```

2.5.3 删除数据库

如果你有DROP权限，那么删除数据库会跟创建数据库一样简单：

```
DROP DATABASE db_name;
```

请注意，千万不要随意使用DROP DATABASE语句。因为它会删掉数据库和其中的所有内容（表、存储例程等）。此时，这个数据库也会永远消失，除

非你定期对它做过备份，才有可能把它恢复过来。

一个数据库就是 MySQL 数据目录里的一个目录，这个目录主要用于存放对象，如表、视图和触发器。如果DROP DATABASE语句执行失败，那么通常是因为那个数据库的目录里还包含有一些与数据库对象无关的文件。DROP DATABASE语句不会删除这类文件，因而也就不会删除那个目录。这意味着，那个数据库目录依然存在，并且如果调用SHOW DATABASES语句，它仍然会显示出来。想要真正删除那个数据库，就要手动删除该数据库子目录里残留文件和目录，然后再调用DROP DATABASE语句。

2.5.4 更改数据库

如果拥有ALTER权限，则可以使用ALTER DATABASE语句来更改数据库的全局属性。目前，数据库的全局属性只包含默认字符集和排序规则：

```
ALTER DATABASE [db_name] [CHARACTER SET charset] [COLLATE
collation];
```

在前面对CREATE DATABASE的讨论里，讲述了CHARACTER SET子句和COLLATE子句的效果，并且要求至少给定它们其中的一个。

如果省略了数据库名，那么ALTER DATABASE会应用到默认数据库。

2.6 表的创建、删除、索引和更改

MySQL允许使用CREATE TABLE语句和DROP TABLE语句来创建、删除表，使用ALTER TABLE语句更改其结构。CREATE INDEX语句和DROP INDEX语句则可以用来为已有表增加或删除索引。下面几节的内容将详细解释这些语句，但首先需要讨论一下 MySQL 支持的用于管理不同类型表的那些存储引擎。

2.6.1 存储引擎的特性

MySQL支持多种存储引擎（以前被称为“表处理器”）。每一种存储引擎所实现的表都具有一组特定的属性或特性。表2-1简要描述几种存储引擎，稍后会对其中的几种（主要是InnoDB和MyISAM）进行详细讨论。其他未介绍到的引擎或者一般很少用到，或者像 NDB这样的，需要更多扩展性的讨论，这超出了本书介绍的范畴。因此，在本书的后面部分对它们并不会谈及太多。

表2-1 MySQL的存储引擎

存储引擎	描 述
ARCHIVE	用于数据存档的引擎（行被插入后就不能再修改了）
BLACKHOLE	这种存储引擎会丢弃写操作，读操作会返回空内容
CSV	这种存储引擎在存储数据时，会以逗号作为数据项之间的分隔符
FEDERATED	用来访问远程表的存储引擎
InnoDB	具备外键支持功能的事务处理引擎
MEMORY	置于内存的表
MERGE	用来管理由多个 MyISAM 表构成的表集合
MyISAM	主要的非事务处理存储引擎
NDB	MySQL 集群专用存储引擎

有几种存储引擎的名字还有同义词。例如，MRG_MyISAM和NDBCLUSTER分别是MERGE和NDB的同义词。存储引擎MEMORY和InnoDB在早期分别称为HEAP和Innobase。虽然后面两个名字仍能被识别，但已废弃了。

在一开始，MySQL服务器都是把所有这些存储引擎编译在内部，以供使用的。现在，它采用了一种“可插入”的架构，可以实现选择性的插件加载，并且许多存储引擎都被构建为了插件。这样，DBA可以把这些引擎当成是可选的，然后再根据需要进行加载。这个插件接口还允许将第三方开发人员的存储引擎集成到服务器里。关于该接口的更多信息请参考12.4节。

2.6.1.1 查看有哪些存储引擎可用

给定一个服务器，它实际包含哪些可用的存储引擎，具体取决于：所用的MySQL 版本，在编译该服务器时所用的具体配置，以及启动该服务器时所使用的选项。关于存储引擎选择的更多信息请参考12.5节。

查出服务器都知道哪些存储引擎的方法是，使用SHOW ENGINES语句：

```
mysql> SHOW ENGINES\G
```

```
***** 1.row *****
```

```

Engine: InnoDB

Support: DEFAULT

Comment: Supports transactions, row-level locking, and foreign keys

Transactions: YES

XA: YES

Savepoints: YES

...

***** 8.row *****

Engine: MyISAM

Support: YES

Comment: MyISAM storage engine

Transactions: NO

XA: NO

Savepoints: NO

...

```

其中，Support栏的值为YES或NO，分别代表了该存储引擎是否可用；DISABLED的意思是该存储引擎存在，但它已被关闭；DEFAULT 表示它是服务器默认使用的存储引擎。标记为DEFAULT的存储引擎应该是肯定可用的。另外，Transactions栏表示的是存储引擎是否支持事务处理。XA和Savepoints栏表示的是存储引擎是否支持分布式事务（本书并未涉及）和部分事务回滚。

在INFORMATION_SCHEMA数据库的ENGINES表里，提供了与SHOW ENGINES语句所示内容完全一样的信息。不过由于需要使用SELECT 来访问它，因此你可以加上查询条件，以便只选择那些你感兴趣的信息。例如，下面这条查询语句会使用ENGINES表来检查有哪些可用引擎支持事务处理：


```
mysql> SELECT ENGINE FROM INFORMATION_SCHEMA.ENGINES
-> WHERE TRANSACTIONS = 'YES';
```

ENGINE
InnoDB

2. 6. 1. 2 表的磁盘存储方式

每次在创建表时，MySQL都会创建一个磁盘文件，用于保存该表的格式（即它的定义）。这个格式文件的基本名与表名相同，扩展名为.frm。例如，有一个名为t的表，其格式文件的名称将是t.frm。服务器会在包含这个表的那个数据库所对应的数据库目录里，创建这个文件。.frm文件不会变化，因为无论管理这个表的存储引擎是哪一种，每个表都只会有一个与之对应的.frm文件。如果表名包含的字符会引起文件名问题，那么 SQL语句里使用的表名可能与相应的.frm文件的基本名并不一致。更多关于SQL名字与文件名之间的映射规则请参考11. 2. 6节。

各个存储引擎还可能会为表再创建几个特定的文件，用以存储表的内容。对于某个给定的表，与之相关的所有文件都集中存放在包含这个表的那个数据库所对应的目录里。表 2-2列出了几种由某个存储引擎创建的与表相关的文件扩展名。

表2-2 存储引擎创建的表文件

存储引擎	磁盘文件
InnoDB	.ibd (数据和索引)
MyISAM	.MYD (数据)、.MYI (索引)
CSV	.CSV (数据)、.CSM (元数据)

对某些存储引擎而言，格式文件是与某特定表相关联的唯一文件。其他存储引擎可能会把表的内容保存到磁盘上的其他地方，或者可能使用一个或多个表空间（由多个表共享的存储区域）。

MEMORY存储引擎会把表存放在内存里，而非磁盘上。

默认情况下，InnoDB 引擎会把表的数据和索引存储在它的系统表空间里。也就是说，所有InnoDB表的内容都集中存储在一个共享存储区域里，而不是与某个特定表所特有的文件里。另外，如果你对InnoDB引擎进行了配置，让它为每个表分别创建一个表空间，那么这时它会创建出 .ibd 文件。

在接下来的几节里，我们将有选择地介绍几种 MySQL 存储引擎的功能和行为。更多关于这些引擎如何在物理上表示表的信息请参考11.2.3节。

2.6.1.3 InnoDB存储引擎

InnoDB存储引擎是MySQL的默认引擎，除非你给服务器配置了其他引擎。它具有以下几项功能。

其表在执行提交和回滚操作时是事务安全的。可以通过创建保存点（savepoint）来实现部分回滚。

在系统崩溃后可以自动恢复。

外键和引用完整性支持，包括级联删除和更新。

基于行级别的锁定和多版本化，使得在执行同时包含有检索和更新操作的组合条件查询时，可以表现出很好的并发性能。

从MySQL 5.6开始，InnoDB支持全文搜索和FULLTEXT索引。

默认情况下，InnoDB存储引擎会把表集中存储在一个系统表空间里，而不是像大多数其他存储引擎那样为不同的表创建不同的文件。InnoDB的表空间由多个文件构成，并且可以包含多个原始分区。实际上，InnoDB存储引擎会把表空间当作是一个虚拟的文件系统，它会在其中存储和管理所有 InnoDB表的内容。这样一来，表的长度就可以超出文件系统对各个文件的最大长度限制。你也可以配置InnoDB，让它为每个表分别创建一个表空间。这时，在包含它的那个数据库目录里，每个表都会有一个 .ibd 文件。

配置各个表空间的方法是，在服务器启动或者运行时，启用 innodb_file_per_table 系统变量。在启用此变量的同时，也会启用InnoDB的其他功能，如快速的表截断和行存储格式化（它们为某些类型数据的处理提供了更好的性能）。更多相关信息请参考12.5.3.1节的第4小节。

2.6.1.4 MyISAM存储引擎

MyISAM存储引擎提供了如下这些功能。

当保存连续相似的字符串索引值时，它会对键进行压缩。此外，MyISAM还可以压缩相似的数字索引值，因为这些数值都是按高字节优先的方式来保存的。（以低位字节优先表示索引值时，检索速度更快；因此以高字节优先表示时，更易于压缩。）如果想启用数字压缩功能，那么请在创建MyISAM表时使用PACK_KEYS=1选项。

与其他存储引擎相比，它为 AUTO_INCREMENT 列提供了更多的功能。更多相关信息请参考3.4节。

每个MyISAM表都有一个标志，它会在执行表检查操作时被设置。MyISAM表还有一个标志，用于表明该表在上次使用后是否被正常关闭。如果服务器意外宕机或机器崩溃，那么可用这个标志来判断表是否需要检查和修复。如果想进行自动检查，则需要在启动服务器时，将myisam_recover_options系统变量设置为一个包含有FORCE选项的值。这样，在每次打开 MyISAM 表时，服务器便会自动检查该表的标志，并根据需要进行表修复。更多相关信息请参考14.3.1节。

支持全文检索和FULLTEXT索引。

支持空间数据类型和SPATIAL索引。

2.6.1.5 MEMORY存储引擎

MEMORY存储引擎会把表储存在内存里，并且这些表的行长度固定不变。这两个特性使得它的相关操作非常快。

在某种意义上，MEMORY表是临时性的，当服务器掉电时，其内容也会消失。也就是说，MEMORY表在服务器重启之后依然存在，只是它们的内容为空。不过，与用CREATE TEMPORARY TABLE语句创建出来的临时表相比，MEMORY表的另一个特点是其内容对其他客户端来说是可见的。

MEMORY表的以下几个特点使它们比其他类型的表更易处理，因此速度也更快。

默认情况下，MEMORY 表使用的是散列索引——这种索引对于“相等比较”非常快，但对于“范围比较”非常慢。因此，散列索引只适合用于相等运算符“=”和“<=>”的比较操作，不太适合像“<”或“>”这样的比较运算符。同样地，散列索引也不适合用在ORDER BY子句里。

为更便于处理，存储在MEMORY表里的行使用长度固定不变的格式。这意味着，你不能使用长度可变的数据类型BLOB和TEXT。VARCHAR是一种长度可变

的类型，但由于在MySQL内部，它被当作是一种长度固定不变的CHAR类型，所以可以在MEMORY表里使用。

当使用像“<”、“>”或 BETWEEN 这样的运算符来查找某个范围内的值时，如果是使用MEMORY表，那么可以用BTREE 索引来代替散列索引，以加快比较速度。更多相关信息请参考2.6.4.2节和5.1.3节。

2.6.1.6 NDB存储引擎

NDB是MySQL的集群（cluster）存储引擎。对于这个存储引擎，MySQL服务器实际上变成了一个其他进程（它们提供了对NDB表的访问）的集群客户端。集群节点会处理彼此间的通信，从而在内存中实现对表的管理。为了实现冗余，这些表会在集群进程之间被复制。内存存储提供了高性能，而集群机制则提供了高可用性，因为即使某个节点发生了故障，整个系统也不会崩溃。

NDB的配置和使用已超出了本书的讨论范围，因此在这里就不再多讲。更多相关信息请参考MySQL参考手册。

2.6.1.7 其他存储引擎

MySQL还提供了其他几种存储引擎，归纳如下。

ARCHIVE 引擎提供了数据归档存储功能。它主要适合于大批量存储那些“写了就不会再更改”的行。因此，它所支持的SQL语句很有限：可以使用INSERT和SELECT；但REPLACE与INSERT总是很像；不能使用DELETE或UPDATE。为了节省空间，在存储时会进行压缩；而在检索时，再对它们进行解压缩。ARCHIVE 表可以包含一个带索引的AUTO_INCREMENT列，但其他列不能被索引。

BLACKHOLE引擎所创建的表，其写操作会被忽略，读操作是返回空内容。这个数据库等同于Unix系统上的/dev/null设备。

CSV 引擎在存储数据时，会用逗号分隔值。对于每个表，它会在数据库目录里创建一个.CSV 文件。这是一种普通文本文件，其中每个表行占用一个文本行。CSV存储引擎不支持索引。

FEDERATED引擎提供了访问由其他 MySQL服务器进行管理的表的能力。也就是说， FEDERATED表的内容实际上位于远程。当创建FEDERATED表时，需要指定一台运行着其他服务器的主机，并提供该服务器上的某个账户的用户名和密码。当要访问FEDERATED表时，本地服务器将使用这个账户连接那台远程服务器。

MERGE 引擎提供了一种把多个MyISAM 表合并为一个逻辑单元的手段。查询一个MERGE表时，相当于查询其所有的成员表。这种做法的好处之一是，可以突破文件系统对单个MyISAM表的尺寸所设定的限制。分区表可以替换MERGE表，并且不会受限于MyISAM表。更多相关信息请参考2.6.2.5节。

2.6.2 创建表

创建表的方法是，使用CREATE TABLE 语句。你必须要拥有表的CREATE 权限。这条语句的完整语法相当复杂，因为它的可选子句实在是太多。不过，在实际使用时，通常都比较简单。例如，在第1章所用到的大部分CREATE TABLE语句，都算不上复杂。只要从比较基本的语法形式开始循序渐进，就不会遇到太大的麻烦。

在最简单的情况下， CREATE TABLE语句只须指定表名和其中的各个列即可。例如：

```
CREATE TABLE mytbl  
  
(  
  
    name      CHAR(20),  
  
    birth     DATE NOT NULL,  
  
    weight    INT,  
  
    sex       ENUM('F','M')  
  
);
```

在创建表时，除了各列的定义，还可以指定它要如何进行索引。另一种做法是，先创建一个不带任何索引的表，然后再为它加上索引。对于MyISAM表来说，如果你计划在开始对它查询之前先填入大量的数据，那么第二种做法通常会更好些。与“先把数据加载到一个不带任何索引的MyISAM表里，然后再去创建它的索引”这种做法相比，“在每次插入一个行时都进行一次索引”的做法则要慢得多。

在第1章，我们已经对CREATE TABLE语句的基本语法进行了介绍。关于列定义的细节请参考第3章。本节将重点介绍CREATE TABLE语句的几种重要扩展，它们可以帮助你灵活地构造表。

修改存储特性的表选项。

只在表不存在时才创建它。

临时表——在客户端会话结束时服务器会自动删除它们。

根据另一个表或是根据某次SELECT查询的结果来创建表。

使用分区表。

2.6.2.1 表选项

如果要想更改某个表的存储特性，可以在CREATE TABLE语句中的右括号之后加上一个或多个表选项。有关选项的完整列表请参考附录E。

一个表选项是 ENGINE = engine_name，它可以为表指定要用的存储引擎。例如，要创建MEMORY表或MyISAM表，可以编写下面这样的语句：

```
CREATE TABLE mytbl (...) ENGINE = MEMORY;
```

```
CREATE TABLE mytbl (...) ENGINE = MyISAM;
```

存储引擎的名字不区分大小写。如果没有给出 ENGINE 选项，那么服务器将使用默认的存储引擎来创建表。内建的默认存储引擎是InnoDB，但可以根据12.5.2节的指导来告诉服务器使用不同的默认引擎。

如果所给名字指定的那个存储引擎未启用，则会看到两条警告信息：

```
mysql> CREATE TABLE t (i INT) ENGINE = ARCHIVE;
```

```
Query OK, 0 rows affected, 2 warnings (0.01 sec)
```

```
mysql> SHOW WARNINGS;
```

Level	Code	Message
Warning	1286	Unknown storage engine 'ARCHIVE'
Warning	1266	Using storage engine InnoDB for table 't'

如果能让某个表使用某个特定的存储引擎，那么一定要包含 `ENGINE` 表选项。由于默认的存储引擎可以更改，因此如果省略 `ENGINE` 选项，则可能无法获得你所期望的默认存储引擎。此外，一定要保证 `CREATE TABLE` 语句没有产生任何警告消息——它们经常是提示所指定的存储引擎不可用，并且提示使用了默认的存储引擎来代替。

如果能让 MySQL 在指定的存储引擎不可用时产生一个错误（而不是使用默认的存储引擎来代替），则需要启用 SQL 的 `NO_ENGINE_SUBSTITUTION` 模式。

如果想要确定某个表使用的是哪一种存储引擎，那么可以执行 `SHOW CREATE TABLE` 语句，并查看其输出内容里的 `ENGINE` 选项：

```
mysql> SHOW CREATE TABLE t\G

***** 1.row *****

Table: t

Create Table: CREATE TABLE `t` (

  `i` int(11) DEFAULT NULL

) ENGINE=MyISAM DEFAULT CHARSET=latin1
```

存储引擎还可以在 `SHOW TABLE STATUS` 语句或 `INFORMATION_SCHEMA.TABLES` 表的输出内容里看到。

选项 `MAX_ROWS` 和 `AVG_ROW_LENGTH` 可用于调整 MyISAM 表的大小。默认情况下，MyISAM 表具有的大小与内部行指针长度一致，即允许表文件增长到 256 TB。如果指定了选项 `MAX_ROWS` 和 `AVG_ROW_LENGTH`，那么这就是在告知 MyISAM：对于一个至少能够容纳 `MAX_ROWS` 个行的表，应该使用与一个指针长度一样的大小。

如果想要修改某个已有表的存储特性，可以在 `ALTER TABLE` 语句里使用表选项。例如，想把 `mytbl` 表的当前存储引擎更改为 InnoDB，那么可以执行下面这条语句：

```
ALTER TABLE mytbl ENGINE = InnoDB;
```

更多关于如何改变存储引擎的信息请参考 2.6.5 节。

2.6.2.2 暂时性表的创建

如果只想创建原本不存在的表，那么可以使用CREATE TABLE IF NOT EXISTS语句。这条语句可以让你的应用程序无需假设它将使用到的表是否已事先建立好。应用程序会正常执行，并且会根据情况决定是否要创建这个表。在通过mysql把脚本当作批处理作业运行时，IF NOT EXISTS修饰符会非常有用。对于这种情况，不适合用常规的CREATE TABLE语句。原因是：在作业第一次运行时，它能正常创建表；但在第二次运行时，则会出错，因为那个表已经存在了。如果使用IF NOT EXISTS，就不会出现这样的问题。在作业第一次运行时，它会像之前那样创建表；在第二次和以后运行时，试图创建表的操作会被毫无声息地忽略掉，而且也不会产生任何错误提示。这使得作业可以继续执行，仿佛创建操作已成功执行一样。

在使用IF NOT EXISTS时，需要注意的是：MySQL不会把CREATE TABLE语句里的表结构与已有表的结构进行比较。即使已有表的名字相同但结构不同，这条语句也不会失败。如果不想冒这个风险，那么最好在CREATE TABLE语句之前放置DROP TABLE IF EXISTS语句。

2.6.2.3 TEMPORARY表

如果在表创建语句里加上TEMPORARY关键字，那么服务器将创建一个临时表，它会在你与服务器的会话终止时自动消失：

```
CREATE TEMPORARY TABLE tbl_name ...;
```

这么做的好处在于你不必惦记着调用DROP TABLE 语句来删除那个表，并且即使你与服务器的会话意外终止，那个表也不会遗留在那里。例如，你在一个批处理文件里存储了一个复杂的查询命令，你通过mysql来运行它，并且决定不想等它执行完毕，这时你可以毫无顾虑地取消那个脚本，服务器会删除由该脚本创建的所有TEMPORARY表。

如果想要创建使用某种存储引擎的临时表，则可以为CREATE TEMPORARY TABLE语句加上ENGINE表选项。

尽管服务器会在客户端会话结束时自动删除TEMPORARY表，但最好是能在用完它之后立即显式地删除它，这可以让服务器尽快释放与之有关的资源。如果与服务器之间的会话还需要持续一段时间，那么将不再需要的资源（尤其是那些临时性的MEMORY表）及时释放掉，是一个很好的习惯。

TEMPORARY表只对创建该表的客户端可见。因为每个客户端都只能看到它自己创建的表，所以不同的客户端可以创建具有相同名字的TEMPORARY表，同时不会发生任何冲突。

TEMPORARY表的名字可以与某个已有的永久表的相同。这不会有任何问题，那个已有的永久表也不会因此遭到损坏。事实上，对于创建TEMPORARY表的那个客户端来说，当有TEMPORARY表存在时，那个永久表会隐藏起来（即无法访问）。假设，在sampdb数据库中创建了一个名为member的TEMPORARY表。这时，原有的那个member表将被隐藏起来，而对member表的引用也会转向这个新建的TEMPORARY表。如果调用DROP TABLE member语句，那么被删除掉的将会是那个TEMPORARY表，而原有的那个member表此时会“再次显现”。如果在没有删除TEMPORARY表的情况下便断开了与服务器的连接，那么服务器将会自动为你删除它。当你下次再连接时，原有的那个member表又会是可见的。（如果重新命名了TEMPORARY表，那么原有的那个被隐藏的表也会变得可见。）

这种隐藏名字的机制只对一层操作有效。也就是说，你无法创建两个同名的TEMPORARY表。

请在考虑是否需要使用TEMPORARY表时，注意以下几个因素。

如果客户端程序在与服务器的连接意外断开时自动重建连接，那么在你重新连接之后，上次创建的那个TEMPORARY表将不复存在。如果你之前在用TEMPORARY表来“隐藏”某个与之同名的永久表，那么这个永久表现在会变成你所使用的那个表。例如，在重新连接后，在未做检测的情况下调用了DROP TABLE语句，这时将会导致那个永久表被删除。为避免这类问题，则需要使用DROP TEMPORARY TABLE语句来代替。

因为TEMPORARY表只在创建它们的会话里是可见的，所以它们与连接池机制（它不保证为你调用的每一条语句都使用相同的连接）一起使用时没什么用处。

如果使用连接池或永久性连接，那么当你的应用程序终止时，你与MySQL服务器之间的连接不一定会被关闭。那些机制可能会保持连接处于打开状态，以供其他客户端使用，这表示，当应用程序终止时，你不能假定那个TEMPORARY表会自动消失。

2.6.2.4 根据其他表或查询结果来创建表

有时候，为某个表创建一份副本会很有用。例如，你有一个数据文件，并且想用LOAD DATA语句把它添加到某个表里，但你对用来指定数据格式的选项没什么把握。万一这些选项设置不正确，那么你就会把一些乱七八糟的行插到那个原有的表里。如果有一份那个原有表的空副本（empty copy），那么你就可以放心地指定不同的列和行分隔符，尝试LOAD DATA的各种选项，最终达到正确分隔输入行的目的。然后，再次运行LOAD DATA语句，将这个数据文件加载到原有的那个表里。

有时也会想把查询结果存储到某个表里，而不是把它们显示到显示器上。在存储了查询结果之后，你可以不用再运行原来那条查询语句，也能引用它们，甚至还可以对它们做进一步的分析。

MySQL提供了两条语句，可用于根据其他的表或根据查询结果创建新表。这两条语句各有各的优缺点。

`CREATE TABLE...LIKE` 会根据原有表创建一个新表，该表是原有表的一个空副本。它会把原有表的结构丝毫不差地复制过来，而且会保留各列的所有属性。索引结构也照样会被复制。不过，由于这个新表是空的，因此如果想把数据填充到里面，还需要另外一条语句（如 `INSERT INTO...SELECT`）。请注意，`CREATE TABLE...LIKE` 不能根据原有表的列子集创建出新表，它也不能使用除原有表以外的任何其他表里的列。

`CREATE TABLE...SELECT` 可以根据任意一条 `SELECT` 语句的查询结果创建一个新表。默认情况下，这条语句不会复制所有的列属性，如 `AUTO_INCREMENT` 列就不会被复制。根据查询数据来自动创建表，并把数据填入其中，这种做法也不会把原有表里任何索引复制过去，因为结果集不会自己索引。另一方面，`CREATE TABLE...SELECT` 只用一条语句即可完成创建新表和填充新表的操作。它也可以用原有表的某个子集来创建新表，并且可以包括来自其他表的列，或者以表达式结果形式创建出来的列。

如果要使用`CREATE TABLE...LIKE`来创建某个已有表的空副本，可以编写如下语句：

```
CREATE TABLE new_tbl_name LIKE tbl_name;
```

如果要创建某个表的空副本，并根据原有表的数据来填充它，那么可以先调用`CREATE TABLE...LIKE`，再调用`INSERT INTO...SELECT`：

```
CREATE TABLE new_tbl_name LIKE tbl_name;
```

```
INSERT INTO new_tbl_name SELECT * FROM tbl_name;
```

如果要把某个表创建为其自身的一个临时副本，则需要加上关键字 **TEMPORARY**：

```
CREATE TEMPORARY TABLE new_tbl_name LIKE tbl_name;
```

```
INSERT INTO new_tbl_name SELECT * FROM tbl_name;
```

当想要尝试某些会修改表内容的语句，而又不想改变原有表的内容时，便可以利用与原有表同名的TEMPORARY表。如果要使用那些事先编写好的使用了原有表名的脚本，那么你不必为了使用另外的表而去编辑它们。只要在脚本的开头加上 CREATE TEMPORARY TABLE 语句和INSERT语句即可。该脚本将创建一个临时副本，并在该副本上执行各种操作，服务器会在脚本结束时自动删除这个临时副本。（不过，请一定要对2.6.2.3节提到的自动重连问题多加注意。）如果只想把原有表里的部分行插到新表中，则可以增加WHERE子句来标识要选取的那些行。下面的语句会创建一个名为student_f的新表，并且它只会包含student表里的女生行：

```
CREATE TABLE student_f LIKE student;
```

```
INSERT INTO student_f SELECT * FROM student WHERE sex = 'f';
```

如果不太关心新表是否完全保留了原有表里的列定义，那么 CREATE TABLE...SELECT 有时要比CREATE TABLE...LIKE更易使用，因为它可以在一条语句里创建并填充新表：

```
CREATE TABLE student_f SELECT * FROM student WHERE sex = 'f';
```

CREATE TABLE...SELECT也可以创建这样的新表，它并不完全包含已有表的所有列。你可以随时根据需要用它来快速创建一个新表，用于保存任意SELECT 查询的结果。这让我们可以非常简便快速地创建一个新表，并把我们的感兴趣的数据填充到它里面，以便后面的语句使用。不过，如果不够小心的话，这个新表可能会包含一些奇怪的列名。当你通过把选择数据放入其中的方式来创建表时，新表的列会根据你所选择的那些列来命名。如果某个列是以表达式的计算结果生成的，那么该列的名字将是这个表达式的文本表示，从而会使创建出的表包含一个不同寻常的列名：

```
mysql> CREATE TABLE mytbl SELECT PI() * 2;
```

```
mysql> SELECT * FROM mytbl;
```

```
-----  
PI() * 2  
-----  
6.283185  
-----
```

这显然不太妙，因为要想直接引用这样的列名，必须得把它当成用引号引起来的标识符才行：

```
mysql> SELECT `PI() * 2` FROM mytbl;
```

```
-----  
PI() * 2  
-----  
6.283185  
-----
```

为避免这个问题，就需要使用别名来提供一个便于使用的列名：

```
mysql> DROP TABLE mytbl;
```

```
mysql> CREATE TABLE mytbl SELECT PI() * 2 AS mycol;
```

```
mysql> SELECT mycol FROM mytbl;
```

```
-----  
mycol  
-----  
6.283185  
-----
```

如果从具有相同列名的不同表里进行查询，则会遇到一个难题。假设表 t1 和 t2 都有一个列为 c，并且你想要根据这两个表里的所有行的组合创建表。由于这时是在试图创建一个具有两个列名都为 c 的表，所以下面的语句将会以失败告终：

```
mysql> CREATE TABLE t3 SELECT * FROM t1 INNER JOIN t2;
```

```
ERROR 1060 (42S21): Duplicate column name 'c'
```

解决这个问题的办法是，提供必要的别名，让每个列在新表里都各有一个唯一的名字：

```
mysql> CREATE TABLE t3 SELECT t1.c, t2.c AS c2
```

```
-> FROM t1 INNER JOIN t2;
```

如前面所提到的，`CREATE TABLE...SELECT`的一点不足在于：它不会把原有数据的所有特征全部复制到新表的结构里去。例如，通过把查询数据放入其中的方式来创建表，就不会把原有表里的索引复制过去，而且还可能会丢失列属性。可以保留的属性包括有：该列是 `NULL` 的还是 `NOT NULL` 的、字符集和排序规则、默认值，以及列注释。

在某些场合，你可以采用在语句的 `SELECT` 部分使用 `CAST()` 函数的方式，在新表里强制使用某些特定的属性。下面的 `CREATE TABLE...SELECT` 语句，将强制把 `SELECT` 子句生成的各个列分别视为 `INT UNSIGNED`、`TIME` 和 `DECIMAL(10,5)`。结果与使用 `DESCRIBE` 语句来验证时看到的内容一样：

```
mysql> CREATE TABLE mytbl SELECT
```

```
-> CAST(1 AS UNSIGNED) AS i,
```

```
-> CAST(CURTIME() AS TIME) AS t,
```

```
-> CAST(PI() AS DECIMAL(10,5)) AS d;
```

```
mysql> DESCRIBE mytbl;
```

Field	Type	Null	Key	Default	Extra
i	int(1) unsigned	NO		0	
t	time	YES		NULL	
d	decimal(10,5)	NO		0.00000	

允许强制转换的类型包括：`BINARY`（二进制串）、`CHAR`、`DATE`、`DATETIME`、`TIME`、`SIGNED`、`SIGNED INTEGER`、`UNSIGNED`、`UNSIGNED INTEGER` 和 `DECIMAL`。

也可以在 `CREATE TABLE` 部分提供显式的列定义，以便将它们用在 `SELECT` 部分检索的列里。这两个部分里的列名是相匹配的（与位置无关），因此必须在 `SELECT` 部分提供别名，才能让它们正确地匹配：

```
mysql> CREATE TABLE mytbl (i INT UNSIGNED, t TIME, d
DECIMAL(10,5))
```

```
-> SELECT
```

```
-> 1 AS i,
```

```
-> CAST(CURTIME() AS TIME) AS t,
```

```
-> CAST(PI() AS DECIMAL(10,5)) AS d;
```

```
mysql> DESCRIBE mytbl;
```

Field	Type	Null	Key	Default	Extra
i	int(10) unsigned	YES		NULL	
t	time	YES		NULL	
d	decimal(10,5)	YES		NULL	

这种提供显式列定义的技术，可以让你实现许多操作，其中包括创建具有特定精度和小数位数的数字列、创建不同于结果集中最长值的宽度的字符列等。此外，请注意在这个例子里有几个列的Null和Default 属性，与前面几个例子里的情况有所不同。如有必要，你可以在CREATE TABLE部分为这些属性提供显式的定义。

2.6.2.5 使用分区表

MySQL支持表分区，从而可以让表的内容分散存储在不同的物理存储位置。在对表存储进行分区后，得到分区表，使用分区表有很多好处，其中两个好处如下。

表存储可以分布在多个设备上，这样可以通过I/O并行机制来缩短访问时间。

优化器可以把本地化搜索限定在某个特定的分区，或者并行搜索多个分区。

如果要创建分区表，可以像往常一样，在CREATE TABLE语句里提供一系列的列和索引。此外，还要指定PARTITION BY子句（它会定义一个可以把行分配到各个分区的分区函数），以及其他一些与分区有关的选项。分区函数可以根据范围、值列表或散列值来分配各行。

根据范围来分区。适用这种方式的情形是，行包含的是一些可划分为多个离散区间的值域，如日期、收入水平或重量。

根据列表来分区。适用这种方式的情形是，每个分区都分别对应于某些明确的列表值，如邮政编码表、电话号码区号、按地理区域划分出的各实体的编号。

根据散列值来分区。适用这种方式的情形是，根据从行键计算出的散列值，把行分布到各个分区。你可以自行提供一个散列函数；也可以告知MySQL要使用到的有哪些列，然后它会基于那些列使用内建的散列函数计算出散列值。

分区函数必须具有确定性，这样，在分配行时，才会把相同的输入值分配到同一个分区。有些函数不适合这个规则，如RAND()和NOW()。

假设你想要创建一个表，用于存储一些简单的由日期和描述性字符串构成的条目，并且假设已有了累计多年的条目需要加载到这个表里。对于每一个包含有日期的数据条目，使用范围分区方式是最适合的。为实现按年将各行分配到某个给定的分区，可以使用相应日期值里的“年”：

```
CREATE TABLE log_partition  
  
(  
  
  dt      DATETIME NOT NULL,  
  
  info    VARCHAR(100) NOT NULL,  
  
  INDEX (dt)  
  
)  
  
PARTITION BY RANGE(YEAR(dt))  
  
(  
  
  PARTITION p0 VALUES LESS THAN (2010),
```

```
PARTITION p1 VALUES LESS THAN (2011),  
  
PARTITION p2 VALUES LESS THAN (2012),  
  
PARTITION p3 VALUES LESS THAN (2013),  
  
PARTITION pmax VALUES LESS THAN MAXVALUE  
  
);
```

2014年及以后的行都将被分配到MAXVALUE区。当到了2014年时，可以再把这个分区进行划分，从而让所有2014年的行都有了它们自己的分区；而对于2015年及以后的行，则会继续分到MAXVALUE区：

```
ALTER TABLE log_partition REORGANIZE PARTITION pmax  
  
INTO (  
  
PARTITION p4 VALUES LESS THAN (2014),  
  
PARTITION pmax VALUES LESS THAN MAXVALUE  
  
);
```

默认情况下，MySQL会将分区存储在专属于分区表的数据库目录里。若想将存储分散到其他地方（如放置到不同的物理设备上），则需要用到分区选项 `DATA_DIRECTORY`和`INDEX_DIRECTORY`。关于这两个以及其他分区选项的语法，请参考附录E中对`CREATE TABLE`语句的描述。

2.6.3 删除表

删除表要比创建它容易很多，因为你不需要给出任何有关其内容格式的信息。如果你拥有这个表的`DROP`权限，那么只要给出其名字即可：

```
DROP TABLE tbl_name;
```

在MySQL里，`DROP TABLE`语句提供了几种有用的扩展。如果想要删除多个表，则可以把它们依次列在同一条语句里：

```
DROP TABLE tbl_name1, tbl_name2, ...;
```


在默认情况下，当你试图删除一个并不存在的表时，系统将会报错。为了不让这个错误出现，并在表不存在时只是生成一条警告信息，可以在语句里加上IF EXISTS子句：

```
DROP TABLE IF EXISTS tbl_name;
```

如果语句产生了警告信息，那么可以使用SHOW WARNINGS来查看它们。

IF EXISTS子句非常适合于用在与mysql客户端一起使用的脚本里。默认情况下，当有错误发生时，如果这个错误是在试图删除某个并不存在的表时发生的，那么mysql会立即退出。例如，你有一个安装脚本，它会创建许多表，以便其他脚本可以做进一步的处理。在这种情形下，你想要确保这个安装脚本在开始运行时有一个干净的环境。如果你在这个脚本的开头使用的是常规的DROP TABLE语句，那么在第一次运行时它就会失败，因为它想要删除的那个表还没有创建出来。如果使用了IF EXISTS子句，则不会出问题。如果表已经存在，那么它们会被删除。如果不存在，那么也不会有错误出现，并且脚本会继续执行。

如果只想删除临时表，则可以加上TEMPORARY关键字：

```
DROP TEMPORARY TABLE tbl_name;
```

2.6.4 索引表

索引是加快访问表内容的基本手段，尤其是在涉及多个表的关联查询里。这是一个非常重要的话题，我们在后面会用一整章的篇幅来讨论“为什么要使用索引”、“它们是如何工作的”，以及“怎样才能最大限度地利用索引去优化查询”（参考第5章）。本节的主要内容是介绍各种适用于不同表类型的索引特性，以及创建和删除索引的语法。

2.6.4.1 存储引擎的索引特性

MySQL提供了多种灵活的索引创建办法。

可以对单个列或多个列建立索引。多列索引也被称作复合索引。

索引可以只包含唯一值，也可以包含重复值。

可以为同一个表创建多个索引，帮助优化对表的不同类型的查询。

对于除ENUM和SET以外的字符串类型，可以利用列的前缀创建索引。也就是说，只对列名的最左边n个字符（对于非二进制串类型）或n个字节（对于二进制串类型）创建索引。（对于BLOB和TEXT列，只有在指定了前缀长度的

情况下才能创建索引。) 如果在前缀长度范围内, 大部分的列都具有唯一性, 那么查询性能通常不会受到影响, 并且还可能会得到改善, 具体表现在: 为列的前缀而不是整个列进行索引, 可以让索引变得更小, 访问速度更快。

并非所有存储引擎都提供了所有的索引功能。表2-3对MySQL的几种存储引擎所具有的索引特性进行了汇总。这个表没有包括MERGE存储引擎, 因为MERGE表是根据MyISAM表创建出来的, 它们有相似的索引特性。这里也没有包括 ARCHIVE、BLACKHOLE、CSV 或EXAMPLE引擎, 它们要么根本不支持索引, 要么支持得很有限。

表2-3 存储引擎的索引特性

索引特性	InnoDB	MyISAM	MEMORY
是否允许使用NULL值	是	是	是
每个索引的列数	16	16	16
每个表的索引数	64	64	64
索引行的最大长度 (字节)	3072	1000	3072
是否支持列前缀索引	是	是	是
列前缀的最大长度 (字节)	767	1 000	3 072
是否支持BLOB/TEXT索引	是	是	否
是否支持FULLTEXT索引	5.6.4版本以上	是	否
是否支持SPATIAL索引	否	是	否
是否支持HASH索引	否	否	是

不同的存储引擎具有不同的索引特性, 其隐含意思是: 如果要求某个索引必须具备某种特性, 那么你可能无法使用特定类型的表。例如, 想要使用HASH索引, 则必须使用MEMORY表; 如果想对某个TEXT列建立索引, 则必须使用MyISAM或InnoDB表。

如果想将某个已有表转换成使用另一种存储引擎的, 这种存储引擎具有更为合适的索引特性, 那么可以使用ALTER TABLE语句来实现。假设你有一个MySQL 5.5版本的InnoDB表, 但是需要使用FULLTEXT 索引来完成搜索, 应该怎么做呢? 在MySQL 5.5里, 只有MyISAM存储引擎才支持这种功能。可以使用下面的语句来转换这个表:

```
ALTER TABLE tbl_name ENGINE = MyISAM;
```

2.6.4.2 创建索引

MySQL可以创建多种类型的索引。

唯一索引。对于单列索引，不允许有重复值出现；对于多列（复合）索引，不允许出现重复的组合值。

常规（非唯一性）索引。它可以让你获得索引的好处，但会出现重复值的情况。

FULLTEXT 索引。它可用于完成全文检索。这种类型的索引只适用于MyISAM 表（在MySQL 5.6.4以上的版本，可用于InnoDB表）。更多相关信息请参考2.14节。

SPATIAL 索引。这种索引只适用于包含空间值的MyISAM表。更多关于空间值的信息请参考3.1.4节。

HASH索引。这是MEMORY表的默认索引类型，不过可以通过创建BTREE索引来改写它。

你可以在使用CREATE TABLE语句创建新表时，包含索引定义。相关示例请参考1.4.6节。如果想为已有表添加索引，则可以使用语句ALTER TABLE或CREATE INDEX。（MySQL会在其内部把CREATE INDEX语句映射为ALTER TABLE操作。）

ALTER TABLE语句比CREATE INDEX语句更灵活多变，因为它可用来创建MySQL所支持的任何一种索引。例如：

```
ALTER TABLE tbl_name ADD INDEX index_name (index_columns);
```

```
ALTER TABLE tbl_name ADD UNIQUE index_name  
(index_columns);
```

```
ALTER TABLE tbl_name ADD PRIMARY KEY (index_columns);
```

```
ALTER TABLE tbl_name ADD FULLTEXT index_name  
(index_columns);
```

```
ALTER TABLE tbl_name ADD SPATIAL index_name  
(index_columns);
```

其中，tbl_name是要添加索引的那个表的名字，index_columns是要进行索引的列（如果要索引多个列，则需要使用逗号隔开）。索引名index_name是可选的。如果没指定它，那么MySQL会根据第一个索引列的名字选取一个名字。

如果某个索引列在索引时使用了PRIMARY KEY或SPATIAL，则它必须为NOT NULL的。其他索引列允许包含NULL值。

如果使用逗号把它们隔开，则可以在同一条ALTER TABLE语句里包括多个对表的改动。因此，你可以同时创建多个索引，这会比使用多条ALTER TABLE语句逐个添加索引快很多。

如果想要限制某个索引，让它只包含唯一值，那么可以把这个索引创建为PRIMARY KEY或UNIQUE索引。这两种索引很相似，主要区别有以下两点。

每个表只能包含一个PRIMARY KEY。因为PRIMARY KEY的名字总是为PRIMARY，而同一个表不允许有两个同名的索引。可以在一个表里放置多个UNIQUE索引。

PRIMARY KEY不可以包含NULL值，而UNIQUE索引可以。如果某个UNIQUE索引包含了NULL值，那么它就可以包含多个NULL值。这是因为，NULL值不会与任何值相等，甚至与另一个NULL值也一样不相等。

除PRIMARY KEY以外，大部分索引类型都可以用CREATE INDEX语句来添加：

```
CREATE INDEX index_name ON tbl_name (index_columns);
```

```
CREATE UNIQUE INDEX index_name ON tbl_name  
(index_columns);
```

```
CREATE FULLTEXT INDEX index_name ON tbl_name  
(index_columns);
```

```
CREATE SPATIAL INDEX index_name ON tbl_name  
(index_columns);
```

其中，tbl_name、index_name和index_columns的含义与ALTER TABLE语句里的一样。与ALTER TABLE语句不同的是，CREATE INDEX语句里的索引名不是可选的，并且不能在一条语句里创建多个索引。

如果想在使用CREATE TABLE语句创建新表时创建索引，那么相应的语法与ALTER TABLE语句所用语法很相似，只不过需要在定义各列的基础上再增加一些索引创建子句：

```
CREATE TABLE tbl_name
```

```
(
```

```
...列定义 ...
```

```
INDEX index_name (index_columns),
```

```
UNIQUE index_name (index_columns),
```

```
PRIMARY KEY (index_columns),
```

```
FULLTEXT index_name (index_columns),  
  
SPATIAL index_name (index_columns),  
  
...  
  
);
```

与ALTER TABLE语句相类似，其中的index_name也是可选的。如果省略它，那么MySQL会从索引名里挑选一个。

作为一个特殊情况，通过在某列的定义末尾加上PRIMARY KEY或UNIQUE子句，可以创建一个单列的PRIMARY KEY或UNIQUE索引。例如，下面两条CREATE TABLE语句是等价的：

```
CREATE TABLE mytbl  
  
(  
  
i INT NOT NULL PRIMARY KEY,  
  
j CHAR(10) NOT NULL UNIQUE  
  
);
```

```
CREATE TABLE mytbl  
  
(  
  
i INT NOT NULL,  
  
j CHAR(10) NOT NULL,  
  
PRIMARY KEY (i),  
  
UNIQUE (j)
```

);

MEMORY 表的默认索引类型是 HASH。散列索引对于精确值的查询速度非常快，这也是MEMORY表使用的典型方式。不过，如果想要把MEMORY表用于范围比较（如id < 100），那么散列索引的效果就没那么理想了。此时，最好是创建一个BTREE索引来代替它。实现方式是在索引定义里增加USING BTREE子句：

```
CREATE TABLE namelist  
  
(  
  
id INT NOT NULL,  
  
name CHAR(100),  
  
INDEX (id) USING BTREE  
  
) ENGINE = MEMORY;
```

如果只想对字符串列的前缀建立索引，那么在索引定义里为该列命名的语法是col_name(n)，而非简单的col_name。前缀值n 表示的是，索引应该包括列值的前n个字节（二进制串类型）或前n个字符（非二进制串类型）。例如，下面这条语句创建出来的表有一个CHAR列和一个BINARY列。它对CHAR列的前10个字符和BINARY列的前15个字节建立了索引：

```
CREATE TABLE addresslist  
  
(  
  
name CHAR(30) NOT NULL,  
  
address BINARY(60) NOT NULL,  
  
INDEX (name(10)),
```

```
INDEX (address(15))
```

```
);
```

在对某个字符串列的前缀进行索引时，与列的长度一样，前缀长度的计量单位必须与该列的数据类型相同。也就是说，二进制串是以字节为单位，非二进制串是以字符为单位。不过，索引项自身的最大长度单位是字节。这两种长度计量办法对单字节字符集来说没有什么区别；但对于多字节字符集则影响较大。对于拥有多字节字符集的非二进制串，MySQL会在最大允许字节长度内尽可能多地把所有字符都存储进索引值。

在某些场合，你可能会发现对列前缀（而不是对整个列）进行索引，不仅仅是想不想那样做的问题，而是必须那样做。

对于BLOB或TEXT列，只能创建前缀型索引。

索引行的长度等于构成该索引的各列的索引部分的长度总和。如果总长度超过了索引行所能容纳的最大字节数，那么可以通过索引列的前缀来“缩短”这个索引。假设有一个MyISAM表，它使用的是latin1 单字节字符集，并且包含了 4个CHAR(255)列（其列名分别为c1到c4）。这时，每个满列值的索引值都会占用255字节，因此所有4个列的索引就会占用1 020字节。可是，因为一个MyISAM索引的最大长度是1 000字节，所以你无法创建能把4个列的完整内容都包括在内的复合索引。不过，你可以采用对其中的几个或全部列的较短部分进行索引的方式来创建索引。例如，可以只为每列的前250个字符进行索引。

FULLTEXT索引里的列是以满列值方式进行索引的，不能进行前缀索引。即使为FULLTEXT索引里的某个列指定了前缀长度，MySQL也会忽略它。

2.6.4.3 删除索引

删除索引的方法是，使用DROP INDEX或ALTER TABLE语句。如果要使用DROP INDEX语句，则必须给出那个要被删除的索引的名字：

```
DROP INDEX index_name ON tbl_name;
```


如果要用DROP INDEX语句来删除PRIMARY KEY，则必须以带引号标识符的形式给指定名字PRIMARY：

```
DROP INDEX `PRIMARY` ON tbl_name;
```

这条语句没有任何歧义，因为一个表只有一个PRIMARY KEY，其名字也始终为PRIMARY。

与CREATE INDEX语句相类似，DROP INDEX语句会被内部处理成ALTER TABLE语句。前面的那条DROP INDEX语句等价于下面的ALTER TABLE语句：

```
ALTER TABLE tbl_name DROP INDEX index_name;
```

```
ALTER TABLE tbl_name DROP PRIMARY KEY;
```

如果不知道表的索引名字，则可以使用SHOW CREATE TABLE或SHOW INDEX来查询它们。

当从表里删除列时，索引也会隐式地受到影响。如果删除的列是某个索引的组成部分，那么也会从该索引里删除这个列。如果你删除了某个索引的所有列，那么MySQL会删除整个索引。

2.6.5 更改表结构

ALTER TABLE是一条万能型的语句，拥有许多用途。在本章前半部分，我们已见识过它的一些本领（如更改存储引擎、创建和删除索引等）。也可用ALTER TABLE语句来完成重命名表、添加或删除列、更改列的数据类型等操作。本节只介绍它的部分功能。有关其语法的更多信息请参考附录E。

当你发现某个表的结构不再满足需要时，ALTER TABLE语句就会很有用。有时你会想要在表里记录一些额外的信息，或者发现表里包含的信息都已过时。有时已有的列可能太小，或者最终证明当初定义的列宽过大，完全超出实际的需要，你想要把它们缩短一些，以便节省空间和改善查询性能。下面列出的是一些能体现ALTER TABLE语句价值的情况。

在某个研究项目中，你使用了一个AUTO_INCREMENT列来记录研究案例编号。你并没有想过会有太多经费，使生成的研究记录超过 50 000 条，因此你将这列的数据类型定为SMALLINT UNSIGNED（它最多只能容纳 65 535条唯一值）。可是，项目的研究经费有了追加，现在足以让你再生成 50 000 条记录。这时，你就需要一个更大的类型来容纳更多的案例编号。

大小调整也可能会有其他情况。有时，创建了一个CHAR(255)列，但后来却发现表里的那些值都没有超过 100个字节那么长。这时，你可以缩短该列，或者将它转换为VARCHAR(255)类型，以节省空间。

为了获得另一种存储引擎所提供的功能，你想将一个表转换成使用那种引擎的。例如，MyISAM 表不具备事务安全性，但你有一个应用程序需要事务处理功能。这时，你可以让受到影响的表改用InnoDB，它支持事务处理。或者，你在MySQL 5.5 里使用了MyISAM，因为它提供支持FULLTEXT功能，但是，现在你已升级到MySQL 5.6，它已将FULLTEXT功能支持扩展到了InnoDB。

下面是ALTER TABLE语句的语法：

```
ALTER TABLE tbl_name action [, action] ...;
```

其中的每个动作（action）指的是对表所做的修改。有些数据库系统只允许一条 ALTER TABLE语句带一个动作，但是MySQL支持带上多个动作，以逗号隔开。

提示

在使用ALTER TABLE之前，如果需要查看表的当前定义，可以执行SHOW CREATE TABLE语句。在执行完 ALTER TABLE 之后，也可以用这条语句来验证你所做的改动是否如预期的那样已对表定义的产生了影响。

下面几个示例演示了ALTER TABLE语句的某些功能。

（1）更改列的数据类型。如果要更改某列的数据类型，可以使用CHANGE子句或MODIFY子句。假设在mytbl表里，列i的数据类型为

SMALLINT UNSIGNED。如果要把它更改为MEDIUMINT UNSIGNED，可以使用下面两条语句中的任意一条：

```
ALTER TABLE mytbl MODIFY i MEDIUMINT UNSIGNED;
```

```
ALTER TABLE mytbl CHANGE i i MEDIUMINT UNSIGNED;
```

在使用CHANGE 子句时，为何需要把列名写两遍呢？这是因为CHANGE 子句能够做到一件MODIFY子句无法做到的事情，即在更改数据类型的同时将该列重新命名。如果想在改变其数据类型的同时，把i重新命名为k，可以这样做：

```
ALTER TABLE mytbl CHANGE i k MEDIUMINT UNSIGNED;
```

请记住，在使用CHANGE 子句时，是先给出想要更改的那个列的名字，然后再指定其新的名字和定义。即使不想重新命名那个列，也需要把它的名字写两遍。

如果想要在不更改其数据类型的情况下更改列名，则可以使用CHANGE old_name new_name，后面紧跟列的当前定义。

如果要更改列的字符集，则可以在列的定义里使用CHARACTER SET 属性：

```
ALTER TABLE t MODIFY c CHAR(20) CHARACTER SET ucs2;
```

更改数据类型的一个重要原因，是要提高多表连接（它会对两个表的列进行比较）的查询效率。索引经常可用于两个相似列类型之间的连接比较，但是如果这两个列的类型完全相同的话，比较速度会更快。假设要运行如下查询语句：

```
SELECT ...FROM t1 INNER JOIN t2 WHERE t1.name = t2.name;
```

如果t1.name是CHAR(10)，而t2.name是CHAR(15)，那么查询速度将不如它们都是CHAR(15)时那么快。使用下面两条语句中的任何一条来更改t1.name，可以让它们变成一样的：

```
ALTER TABLE t1 MODIFY name CHAR(15);
```

```
ALTER TABLE t1 CHANGE name name CHAR(15);
```

(2) 让表改用另一种存储引擎。如果想让表改用另一种存储引擎，则可以使用ENGINE子句来指定一个新的存储引擎名字：

```
ALTER TABLE tbl_name ENGINE = engine_name;
```

其中，engine_name是一个诸如InnoDB、MyISAM或MEMORY之类的名字。不区分大小写。

让表改用另一种存储引擎的一个原因，是让它具有事务安全的特性。假设你有一个MyISAM表，并且发现有一个用到这个表的应用程序需要进行事务操作，包括以防失败发生的事务回滚机制。MyISAM表不支持事务处理，但你可以让它改用InnoDB，从而让它是事务安全的：

```
ALTER TABLE tbl_name ENGINE = InnoDB;
```

当你打算让表改用另一种存储引擎时，能否达到最终目的，还要取决于新旧两种存储引擎的功能是否兼容。例如，你有一个包含有一个BLOB列的表，你将无法把它转换成使用MEMORY引擎的，因为MEMORY表不支持BLOB列。

还有一些不应使用ALTER TABLE语句来更改表使用另一种存储引擎的情况。例如以下几种情况。

可以让 InnoDB表改用另一种存储引擎。不过，如果这个表定义了外键约束条件，那么这些约束条件在转换后将不复存在，因为只有InnoDB才支持外键。

MEMORY 表存储在内存之中，在服务器退出时会消失。因此，如果你希望某个表的内容在服务器重新启动后仍然存在，那么就不要再让它改用MEMORY引擎。

(3) 重新命名表。用RENAME子句可以为表指定一个新的名字：

```
ALTER TABLE tbl_name RENAME TO new_tbl_name;
```

另一种重新命名表的办法是使用RENAME TABLE语句。语法如下：

```
RENAME TABLE tbl_name TO new_tbl_name;
```

有一件事情是RENAME TABLE语句可以做、但ALTER TABLE语句无法做到的，即它每次可以对多个表进行重新命名。例如，你可以像下面这样交换两个表的名字：

```
RENAME TABLE t1 TO tmp, t2 TO t1, tmp TO t2;
```

如果在表名前面使用了数据库名进行限定，就可以通过对它重新命名达到把它从一个数据库移动到另一个数据库的目的。下面两条语句都可以把表t从sampdb数据库移到test数据库：

```
ALTER TABLE sampdb.t RENAME TO test.t;
```

```
RENAME TABLE sampdb.t TO test.t;
```

重新命名表时不能使用已有的名字。

2.7 获取数据库元数据

MySQL提供了多种获取数据库元数据（即有关数据库的信息与它里面的各种对象）的方式。

各种SHOW语句，如SHOW DATABASES或SHOW TABLES。

INFORMATION_SCHEMA数据库里的表。

命令行程序，如mysqlshow或mysqldump。

接下来的几节将描述使用这些信息源来访问元数据的一些具体做法。

2.7.1 使用SHOW语句获取元数据

MySQL提供的SHOW语句，可以显示许多类型的元数据。SHOW语句有助于我们及时了解数据库内容的变化情况，以及查看有关的表结构。接下来的几个示例演示了 SHOW 语句的一些常见用法。

列出可以访问的数据库：

```
SHOW DATABASES;
```

显示数据库的CREATE DATABASE语句：

```
SHOW CREATE DATABASE db_name;
```

列出默认数据库或给定数据库里的表：

```
SHOW TABLES;
```

```
SHOW TABLES FROM db_name;
```

SHOW TABLES语句无法显示出TEMPORARY表。

显示表的CREATE TABLE语句：

```
SHOW CREATE TABLE tbl_name;
```

显示表里的列或索引信息：

```
SHOW COLUMNS FROM tbl_name;
```

```
SHOW INDEX FROM tbl_name;
```

语句DESCRIBE tbl_name和EXPLAIN tbl_name与语句SHOW COLUMNS FROM tbl_name是同一个意思。

显示默认数据库或某给定数据库里的表描述信息：

```
SHOW TABLE STATUS;
```

```
SHOW TABLE STATUS FROM db_name;
```

有几种SHOW语句形式还可以带上一条LIKE 'pattern'子句——它可以通过一个模式来限制SHOW语句的输出范围。MySQL会将'pattern'解释成一个允许包含通配符“%”和“_”的SQL模式。例

如，下面这条语句可以把student表里名字以“s”打头的所有列显示出来：

```
mysql> SHOW COLUMNS FROM student LIKE 's%';
```

Field	Type	Null	Key	Default	Extra
sex	enum('F','M')	NO		NULL	
student_id	int(10) unsigned	NO	PRI	NULL	auto_increment

如果需要在LIKE模式里使用通配符自身，则可以在它的前面加上一个反斜线进行转义。常见的情形是匹配字符“_”，因为这个字符经常出现在数据库、表和列的名字里。

所有支持LIKE子句的SHOW语句，都可以改写为使用WHERE子句。它们显示出的列都一样，只是WHERE子句在指定要返回哪些行方面更为灵活一点。WHERE子句应该引用SHOW语句的列名。如果列名是一个保留字，如KEY，那么它必须以标识符引起来的方式进行指定。下面这条语句可以确定哪一列是student表里的主键：

```
mysql> SHOW COLUMNS FROM student WHERE `Key` = `PRI`;
```

Field	Type	Null	Key	Default	Extra
student_id	int(10) unsigned	NO	PRI	NULL	auto_increment

有时候它会非常有用，能在应用程序里识别出某个给定的表是否存在。使用SHOW TABLES语句可以找出结果（不包括TEMPORARY表）：

```
SHOW TABLES LIKE 'tbl_name';
```

```
SHOW TABLES FROM db_name LIKE 'tbl_name';
```

如果SHOW TABLES语句列出了表的信息，则说明该表存在。也可以使用下面两条语句中的一条来确定表是否存在（甚至可用于TEMPORARY表）：

```
SELECT COUNT(*) FROM tbl_name;
```

```
SELECT * FROM tbl_name WHERE FALSE;
```

如果表存在，这两条语句就成功；如果不存在，就失败。第一条语句最适合于MyISAM表，因为不带WHERE子句的COUNT(*)函数已被高度优化过。它不太适合于InnoDB表，因为在行统计时会要求扫描全表。第二条语句则更为通用，它对于任何存储引擎都执行得很快。这些语句最适合于在像Perl或PHP这样的语言里使用，因为你可以测试查询执行是否成功，然后执行相应的动作。它们不太适合用在通过mysql来运行的批处理脚本里，因为当有错误发生时你无法做任何事情，只有强行终止脚本的运行（或者忽略那个错误，但这时继续运行查询显然已无任何意义）。另一种策略是查询 INFORMATION_SCHEMA 数据库，这种做法可用于任何上下文，同时不会出现失败。更多相关信息请参考2.7.2节。

如果想要确定各个表所使用的存储引擎，则可以使用SHOW TABLE STATUS或SHOW CREATE TABLE。这两条语句的输出里都会包含有关于存储引擎的指示信息。

2.7.2 借助INFORMATION_SCHEMA获取元数据

获取数据库信息的另一个办法是访问 INFORMATION_SCHEMA 库。这个数据库是以 SQL标准为基础构建的。也就是说，虽然有些内容是MySQL特有的，但对它的访问机制是标准化的。这使得 INFORMATION_SCHEMA有着优于各种SHOW语句（它们都是MySQL特有的）的可移植性。

通过SELECT语句可以访问INFORMATION_SCHEMA库，并且这种方式非常灵活。在SHOW语句的输出里，列的个数是固定的，而且无法把输出内容捕获到表里去。利用INFORMATION_SCHEMA数据库，SELECT语句可以选取特定的输出列，而且WHERE子句可以让你通过各种表达式挑选

你真正需要的信息。不仅如此，你还可以使用连接或子查询，也可以使用CREATE TABLE...SELECT或INSERT INTO...SELECT把检索结果存储到另一个表，以便进一步处理。

可以把 INFORMATION_SCHEMA 库想象成一个虚拟的库，其中的表是各种不同数据库元数据的视图。如果想要知道 INFORMATION_SCHEMA 库包含了哪些表，可以使用SHOW TABLES语句：

```
mysql> SHOW TABLES IN INFORMATION_SCHEMA;
```

```
Tables_in_information_schema
-----
CHARACTER_SETS
COLLATIONS
COLLATION_CHARACTER_SET_APPLICABILITY
COLUMNS
COLUMN_PRIVILEGES
ENGINES
EVENTS
FILES
GLOBAL_STATUS
GLOBAL_VARIABLES
KEY_COLUMN_USAGE
PARAMETERS
PARTITIONS
PLUGINS
PROCESSLIST
```

PROFILING
REFERENTIAL_CONSTRAINTS
ROUTINES
SCHEMATA
SCHEMA_PRIVILEGES
SESSION_STATUS
SESSION_VARIABLES
STATISTICS
TABLES
TABLESPACES
TABLE_CONSTRAINTS
TABLE_PRIVILEGES
TRIGGERS
USER_PRIVILEGES
VIEWS

下面简要说明一下 INFORMATION_SCHEMA 库里的各个表。

SCHEMATA、TABLES、VIEWS、ROUTINES、TRIGGERS、EVENTS、PARAMETERS、PARTITIONS、COLUMNS 与数据库、表、视图、存储例程、触发器、事件、表分区和列有关的信息。

FILES

与用于存储表空间数据的那些文件有关的信息。

TABLE_CONSTRAINTS、KEY_COLUMN_USAGE

与具有约束条件（如唯一值索引或外键）的表和列有关的信息。

STATISTICS

与表索引特性有关的信息。

REFERENTIAL_CONSTRAINTS

与外键有关的信息。

CHARACTER_SETS、COLLATIONS、 COLLATION_CHARACTER_SET_APPLICABILITY

与所支持的字符集、每种字符集的排序规则，以及每种排序规则与其字符集之间的映射关系有关的信息。

ENGINES、PLUGINS

与存储引擎和服务器插件有关的信息。

USER_PRIVILEGES、SCHEMA_PRIVILEGES、TABLE_PRIVILEGES、COLUMN_PRIVILEGES与全局、数据库、表和列的权限分配有关的信息，分别来自于 mysql 数据库里的user、db、tables_priv和column_priv表。

GLOBAL_VARIABLES、SESSION_VARIABLES、GLOBAL_STATUS、 SESSION_STATUS

全局和会话的系统变量值和状态变量值。

PROCESSLIST

与服务器内的执行线程有关的信息。

个别存储引擎还会在INFORMATION_SCHEMA 里增加它们自己的表。例如，InnoDB就会这样做。

如果想要确定某个给定的INFORMATION_SCHEMA 里表都包含有哪些列，则可以使用语句SHOW COLUMNS或DESCRIBE：

```
mysql> DESCRIBE
INFORMATION_SCHEMA.CHARACTER_SETS;
```

Field	Type	Null	Key	Default	Extra
CHARACTER_SET_NAME	varchar(32)	NO			
DEFAULT_COLLATE_NAME	varchar(32)	NO			
DESCRIPTION	varchar(60)	NO			
MAXLEN	bigint(3)	NO		0	

如果想要显示某个表的信息，可以使用SELECT语句。
(INFORMATION_SCHEMA数据库及其里面的所有表和列的名字都不区分大小写。) 用于查看 INFORMATION_SCHEMA 里的某个给定表的所有列的通用查询语句，如下所示：

```
SELECT * FROM INFORMATION_SCHEMA.tbl_name;
```

如果想要有选择地查看，则可以加上一条WHERE子句。

前面介绍了如何利用SHOW语句去检查某个表是否存在，以及它所用的存储引擎是哪一种。INFORMATION_SCHEMA 里的表可以提供同样的信息。下面的查询语句使用了INFORMATION_SCHEMA 里的表来测试某个特定的表是否存在。如果该表存在，则返回 1；如果它不存在，则返回0。

```
mysql> SELECT COUNT(*) FROM
INFORMATION_SCHEMA.TABLES
```

```
-> WHERE TABLE_SCHEMA='sampdb' AND TABLE_NAME='member' ;
```

```

-----
COUNT(*)
-----
1
-----

```

下面这条查询语句可用于检查某个表所使用的存储引擎是哪一种：

```
mysql> SELECT ENGINE FROM
INFORMATION_SCHEMA.TABLES
```

```
-> WHERE TABLE_SCHEMA='sampdb' AND TABLE_NAME='student';
```

```

-----
ENGINE
-----
InnoDB
-----

```

2.7.3 从命令行获取元数据

mysqlshow 命令提供的信息与某些 SHOW 语句很相似，因此你可以从命令行提示符获取数据库和表的信息。

列出服务器所管理的数据库：

```
% mysqlshow
```

列出数据库里的表：

```
% mysqlshow db_name
```

显示表里的列信息：

```
% mysqlshow db_name tbl_name
```

显示表里的索引信息：

```
% mysqlshow --keys db_name tbl_name
```

显示数据库里所有表的描述性信息：

```
% mysqlshow --status db_name
```

客户端程序mysqldump能够让你看到CREATE TABLE语句（与SHOW CREATE TABLE语句很像）所定义的表结构。如果使用mysqldump来查看表结构，请切记加上--no-data选项，否则看到的内容将是表里的数据！

```
% mysqldump --no-data db_name [tbl_name] ...
```

如果你只指定了数据库的名字，而没有给出任何表名，那么mysqldump将把该数据库里所有表的结构显示出来。否则，它将只会显示那些通过名字指定的那些表的信息。

在使用mysqlshow和mysqldump时，请记得要指定必要的连接参数选项，如--host、--user或--password。

2.8 使用连接实现多表检索

如果只把记录存入数据库，而不对它们进行检索或是好好利用，那就没意义了。那正是SELECT语句的目的所在：帮你找到所需要的数据。与SQL语言中的其他语句相比，SELECT语句应该是最常用的，同时也应该是最不容易掌握的。它用来筛选行的条件有时会非常复杂，并且需要对多个表里的列进行比较。

SELECT语句的基本语法，如下所示：

```
SELECT select_list           # 所选择的列
```

```
FROM table_list             # 要查询的那些表
```

WHERE row_constraint # 行必须满足的条件

GROUP BY grouping_columns # 结果如何分组

ORDER BY sorting_columns # 结果如何排序

HAVING group_constraint # 分组必须满足的条件

LIMIT count; # 限制结果里的行数

除了单词SELECT和用于指定想要输出哪些内容的select_list部分以外，这个语法中的所有内容都是可选的。有些数据库系统要求FROM子句也必不可少。但MySQL没有那样做，因为这样可以实现在没有引用任何表的情况下对表达式进行求值：

```
SELECT SQRT(POW(3,2)+POW(4,2));
```

在第 1 章，我们重点讨论了只涉及一个表的SELECT 语句，并把注意力主要集中在了输出列的列表，以及WHERE、GROUP BY、ORDER BY、HAVING和LIMIT等子句上。本节将讨论SELECT语句中最不易掌握的地方：编写连接查询（即编写可以从多个表检索行的SELECT语句）。我们将讨论 MySQL 支持的连接操作的类型、它们的含义，以及如何使用它们。这些内容有助于让你更有效率地使用MySQL，因为在许多情况里，真正的问题是在于要如何编写出查询命令，将多个表正确地连接在一起。

使用SELECT语句的一个问题是，当你第一次遇到某个新问题时，想要弄清如何编写SELECT查询来解决它，并不总是件容易的事情。不过，在你弄明白之后，将来再遇到类似的问题时，就可以使用这个经验。对于 SELECT 语句，过往的经验很可能要扮演最重要的角色，只有这样才能有效地利用它。原因很简单，用它来解决的问题实在是千变万化。随着经验的积累，你会发现有很多新问题都可以通过连接操作轻松解决，并且你也会发现自己在以这样的方式进行思考，如“噢，这是一个LEFT JOIN问题”或者“啊哈，这就是一个通过公共的键列配对进行限制的三方连接”。（希望你在听到“经验可以帮上大忙”时可以受到一些鼓舞。当然，在想到要那样做时，你也可能会感到惊讶。）

有许多演示如何使用MySQL所支持的连接操作的示例，都用到了下列两个表t1和t2：

Table t1:

Table t2:

i1	c1	i2	c2
1	a	2	c
2	b	3	b
3	c	4	a

由于这两个表都很小，因此我们可以很清楚地看到每种类型的连接操作所执行的效果。

其他类型的多表SELECT语句是子查询（将SELECT语句彼此嵌套一起使用）和UNION语句。这些内容将会在2.9节和2.10节涉及。

MySQL所支持的与多表操作有关的功能是，基于另外一个表的内容来删除或更新某个表的行。例如，你可能需要从某个表里把它在另一个表里没有任何匹配的记录全部删掉，或者需要把某个表里的列值复制到另一个表的列里。更多相关细节请参考2.11节。

2.8.1 内连接

如果在SELECT语句的FROM子句里列出了多个表的名称，并且这些名称是由INNER JOIN隔开的，那么MySQL就会执行一个内连接（inner join）操作——它会把一个表里的行与另一个表里的行进行匹配，然后产生结果。例如，如果像下面这样把t1和t2连接起来，那么t1里的每一行将与t2里的每一行进行组合：

```
mysql> SELECT * FROM t1 INNER JOIN t2;
```


i1	c1	i2	c2
1	a	2	c
2	b	2	c
3	c	2	c
1	a	3	b
2	b	3	b
3	c	3	b
1	a	4	a
2	b	4	a
3	c	4	a

在这条语句里，SELECT *的含义是“从FROM子句列出的每个表里选取每个列”。也可以把它写成SELECT t1.*, t2.*:

```
SELECT t1.*, t2.* FROM t1 INNER JOIN t2;
```

如果不想选取所有的列，或者想按不同的从左至右的顺序来显示它们，那么可以给出每一个想选的列的名字，并用逗号隔开。

根据某个表里的每一行与另一个表里的每一行得到全部可能组合的连接操作，叫做生成笛卡儿积（cartesian product）。按这种方式来连接表，很可能会产生数量巨大的行，因为最终的行总数是所有表的行数的乘积。假设，有3个表分别包含有100行、200行和300行，那么它们的连接将返回6百万（100 × 200 × 300）行。这可是个相当庞大的数字，尽管那3个表本身很小。在这种情况下，通常需要增加WHERE子句，从而将结果集减少到一个更适合管理的大小。

如果增加WHERE子句，实现表之间基于某些列值的匹配，那么整个连接操作将只会选取那些列值彼此相等的行：

```
mysql> SELECT t1.*, t2.* FROM t1 INNER JOIN t2 WHERE t1.i1 = t2.i2;
```

i1	c1	i2	c2
2	b	2	c
3	c	3	b

连接类型CROSS JOIN和JOIN都等同于INNER JOIN，因此下面的语句是等价的：

```
SELECT t1.*, t2.* FROM t1 INNER JOIN t2 WHERE t1.i1 = t2.i2;
```

```
SELECT t1.*, t2.* FROM t1 CROSS JOIN t2 WHERE t1.i1 = t2.i2;
```

```
SELECT t1.*, t2.* FROM t1 JOIN t2 WHERE t1.i1 = t2.i2;
```

连接运算符“,”（逗号）也与之类似：

```
SELECT t1.*, t2.* FROM t1, t2 WHERE t1.i1 = t2.i2;
```

不过，逗号运算符的优先级和其他连接类型的有所不同。有时，在其他连接类型的都正常的情况下，它会导致语法错误。个人建议尽量避免使用逗号运算符。

在指定如何对表里的列进行匹配方面，INNER JOIN、CROSS JOIN和JOIN（未包括逗号运算符）还支持另外几种语法格式。

有一种语法是用ON子句代替WHERE子句。下面示例使用INNER JOIN展示了这一点：

```
SELECT t1.*, t2.* FROM t1 INNER JOIN t2 ON t1.i1 = t2.i2;
```

不管被连接的列是否同名，都可以使用ON。

另一种语法是使用USING()子句。它在概念上类似于ON子句，但要求被连接的列必须同名。例如，下面这条查询语句可以将mytbl1.b连接到mytbl2.b：

```
SELECT mytbl1.*, mytbl2.* FROM mytbl1 INNER JOIN mytbl2
USING (b);
```

2.8.2 对被连接表里的列引用进行限定

SELECT语句里的每一个表列的引用，都必须毫无歧义地只对应FROM子句里的一个表。如果FROM子句中只提到一个表，那么肯定不会有歧义，所有列肯定都来自这个表。如果提到多个表，那么只在一个表里出现的列名才不会产生歧义。不过，如果某个列名出现在多个表里，则在引用这个列时必须使用tbl_name.col_name语法，指定你所想用的那个表。假设，表mytbl1里包含列a和b，表mytbl2里包含列b和c。此时，引用a或c不会产生歧义，但在引用b时则必须将它限定为mytbl1.b或mytbl2.b：

```
SELECT a, mytbl1.b, mytbl2.b, c FROM mytbl1 INNER JOIN mytbl2
...;
```

有时，用表的名字进行限定仍不足以解决列的歧义问题。例如，你正在进行一个自连接操作（即把一个表与它自身连接起来），那么这会在同一条查询命令里多次用到同一个表，此时使用表名来限定列名已无济于事。在这种情况下，可以使用表的别名来解决问题。只需为该表的某个实例取一个别名，便可以通过alias_name.col_name语法来引用该实例里的列。下面这条查询语句将把某个表与其自身连接起来，不过它为该表的一个实例分配了一个别名，从而消除了在列引用时可能产生的歧义问题：

```
SELECT mytbl.col1, m.col2 FROM mytbl INNER JOIN mytbl AS m
WHERE mytbl.col1 > m.col1;
```

2.8.3 左（外）连接和右（外）连接

内连接只会显示在两个表里都匹配上的行。外连接除了显示同样的匹配结果，还可以把其中一个表在另一个表里没有匹配的行也显示

出来。外连接分为左连接和右连接两种。本节里的绝大多数示例使用的是LEFT JOIN，即会把左表里的在右表里未匹配上的行也显示出来。RIGHT JOIN在表的顺序上刚好与之相反。

LEFT JOIN的工作方式是：先指定用于对两个表里的行进行匹配的列；接着，当左表的某行与右表的某行相匹配时，两行的内容会被选取为一个输出行；当左表的某行在右表里无匹配时，它仍然会被选取为一个输出行，只是与它连接的是右表的一个“假”行，其中的每一列都包含NULL。

换句话说，对于左表查询出的每一行，不管它在右表里是否有匹配，LEFT JOIN都会强制结果集包含一行记录。在结果集里，对于在右表里没有匹配的结果行，它们会有这样一个特征，即来自右表的所有列都为NULL。这一特征可以让你了解到右表里缺少了哪些行。这是一个既有意思又很重要的特征，因为这种问题会在许多不同的环境里遇到。例如，还没有为哪些顾客指派服务代表？哪些库存商品一件也没卖出？或者，回到我们的sampdb 数据库：哪些学生没有参加过某次特定的考试？哪些学生在absence表里没有任何行（即哪些学生属于全勤）？

仍以刚才的那两个表t1和t2为例：

Table t1:

Table t2:

t1		t2	
i1	c1	i2	c2
1	a	2	c
2	b	3	b
3	c	4	a

如果我们使用内连接，并根据t1.i1和t2.i2来匹配这两个表，那么只能得到值为2和3的输出，因为只有它们才是这两个表里都拥有的值：

```
mysql> SELECT t1.*, t2.* FROM t1 INNER JOIN t2 ON t1.i1 = t2.i2;
```

i1	c1	i2	c2
2	b	2	c
3	c	3	b

左连接会为t1表里每一行生成一个输出，不管它在t2里是否有匹配。为了编写左连接，需要列出这两个表，并在它们之间放置INNER JOIN来代替LEFT JOIN：

```
mysql> SELECT t1.*, t2.* FROM t1 LEFT JOIN t2 ON t1.i1 = t2.i2;
```

i1	c1	i2	c2
1	a	NULL	NULL
2	b	2	c
3	c	3	b

现在，结果集里对于t1. i1取值为1的情况也有了一个输出行，而它在t2里并无任何匹配。这一行里与t2的列相对应的所有列，其值都为NULL。

在使用LEFT JOIN 时需要注意这样一个问题，即只有将右表里的列全部定义成 NOT NULL的，结果集里的行才会没什么问题。例如，右表里包含值为 NULL的列，那么你就无法将这些NULL值与标识未匹配行的那些NULL值区别开来。

之前提到过，RIGHT JOIN与LEFT JOIN很相似，只是表的角色需要对换一下。下面两条语句是等价的：

```
SELECT t1.*, t2.* FROM t1 LEFT JOIN t2 ON t1.i1 = t2.i2;
```

```
SELECT t1.*, t2.* FROM t2 RIGHT JOIN t1 ON t1.i1 = t2.i2;
```

下面的讨论将围绕LEFT JOIN展开。如果要调整成使用RIGHT JOIN的，则只须将表的角色调换一下即可。

当只想将左表里的那些在右表里无匹配的行找出来时，LEFT JOIN非常有用。要实现这个功能，只需增加一条WHERE子句，让它把右表列为NULL的（也就是说，这些行只在一个表里存在，在另一个表里无匹配）行筛选出来：

```
mysql> SELECT t1.*, t2.* FROM t1 LEFT JOIN t2 ON t1.i1 = t2.i2
```

```
-> WHERE t2.i2 IS NULL;
```

i1	c1	i2	c2
1	a	NULL	NULL

通常情况下，在编写这样的查询命令时，真正让人感兴趣的内容是左表里未被匹配到的值。把来自右表值为NULL的列显示出来毫无意义，因此可以把它们从输出列的列表里省略掉：

```
mysql> SELECT t1.* FROM t1 LEFT JOIN t2 ON t1.i1 = t2.i2
```

```
-> WHERE t2.i2 IS NULL;
```

i1	c1
1	a

LEFT JOIN与INNER JOIN相类似，也可以通过ON子句或USING()子句来指定匹配条件。与 INNER JOIN的情况一样，不管被连接的列是否拥有相同的名字，都可以使用ON 子句；但USING()子句要求被连接的列必须有相同的名字。

NATURAL LEFT JOIN类似于LEFT JOIN。它会按照LEFT JOIN规则对左右两个表里所有同名的列进行匹配。（因此，它不需要指定任何ON或USING子句。）

之前提到过，LEFT JOIN非常适合用于解决像“缺失了哪些值”这样的问题。接下来，我们将把这个原则应用到sampdb数据库上，并一起来看一个比早先使用t1和t2的那些示例更为复杂的示例。

对于第1章中提到过的那个成绩考评项目，我们有一个记录学生的student表，有一个记录已发生考试或测验事件的grade_event表，还有一个记录每位学生的每次考试或测验成绩的score表。不过，如果某个学生在某次考试或测验的当天生病了，那么score表里就不会有该学生该次事件的成绩。因故误考的学生需要参加补考，可是要怎样才能把这些缺失的行找出来呢？要解决的问题是：确定哪些学生在给定考试事件里没有成绩。为此，需要针对每一个考试事件执行一遍。也就是说，我们需要找出哪些学生和事件的组合没在score表里出现过。这种“哪些值未出现过”的说法即是需要执行 LEFT JOIN 操作的一种提示。这种连接操作可不像前面那几个示例那么简单。我们不只是要查找那些在某个列里不存在的值，而且要查找不存在于两个组合列的那些值。我们想要的组合是所有的“学生+事件”。可以通过连接student表和grade_event表，来产生这些组合：

```
FROM student INNER JOIN grade_event
```

接下来，我们需要利用这个连接结果，与score表一起执行一个LEFT JOIN操作，以便找到“学生ID/考试ID”对的所有匹配：

```
FROM student INNER JOIN grade_event  
  
LEFT JOIN score ON student.student_id = score.student.id  
  
AND grade_event.event_id = score.event_id
```

请注意，ON子句会导致score表里的行按照该连接操作前面列出的不同表里的匹配情况进行连接。这是解决此问题的关键。这个LEFT JOIN操作会针对student表和grade_event表通过连接而得到的每一个行，强制生成一个行，即使在score表里没有对应的行也没关系。缺少分数行的结果行可以根据这样一个事实标识出来，即来自score表的列将全部为NULL。我们可以通过在WHERE子句里增加一个条件，把这些行找出来。来自score表的所有列都可以这样做，但因为我们正在查找缺失的考试分数行，所以对score列进行测试是顺理成章的事：

```
WHERE score.score IS NULL
```

我们还可以用一个ORDER BY子句对结果进行排序。最符合逻辑的两种顺序是：按先学生后考试的顺序和按先考试后学生的顺序。这里选择的是第一种：

```
ORDER BY student.student_id, grade_event.event_id
```

现在，所有需要做的事情就是列出想要在输出里看到的那些列的名字。下面是最终的语句：

```
SELECT  
  
student.name, student.student_id,  
  
grade_event.date, grade_event.event_id, grade_event.category  
  
FROM  
  
student INNER JOIN grade_event
```



```
LEFT JOIN score ON student.student_id = score.student_id  
AND grade_event.event_id = score.event_id  
  
WHERE  
  
score.score IS NULL  
  
ORDER BY  
  
student.student_id, grade_event.event_id;
```

下面是运行这条查询语句所得到的结果：

name	student_id	date	event_id	category
Megan	1	2012-09-16	4	Q
Joseph	2	2012-09-03	1	Q
Katie	4	2012-09-23	5	Q
Devri	13	2012-09-03	1	Q
Devri	13	2012-10-01	6	T
Will	17	2012-09-16	4	Q
Avery	20	2012-09-06	2	Q
Gregory	23	2012-10-01	6	T
Sarah	24	2012-09-23	5	Q
Carter	27	2012-09-16	4	Q
Carter	27	2012-09-23	5	Q
Gabrielle	29	2012-09-16	4	Q
Grace	30	2012-09-23	5	Q

这里有个小细节需要解释一下。输出结果里显示了学生ID和事件ID。由于student_id列在student 表和score 表里都有，因此你或许会认为在输出结果里也应该列出名字student.student_id或score.student_id。但事实并非如此，之所以能找到我们感兴趣的行，那是因为对于 score 表里的全部列，LEFT JOIN 返回的都是NULL 值。如果选择score.student_id，那么在输出结果里只会产生一个全部值都为NULL的列。在决定要显示哪个表的event_id列时，也需要考虑相同的原则。它在grade_event 表和score表里都有出现，但因为score.event_id值总是为NULL，所以这条查询命令选择了grade_event.event_id列。

2.9 使用子查询实现多表检索

子查询指的是用括号括起来，并嵌入另一条语句里的那条SELECT语句。下面有一个示例，它实现的是找出与考试类别（'T'）相对应的所有考试事件行的ID，然后利用它们来查找那些考试的成绩：

```
SELECT * FROM score  
  
WHERE event_id IN (SELECT event_id FROM grade_event WHERE  
category = 'T');
```

子查询可以返回各种不同类型的信息。

标量子查询返回一个值。

列子查询返回一个由一个值或多个值构成的列。

行子查询返回一个由一个值或多个值构成的行。

表子查询返回一个由一个行或多个行构成的表，而行则由一个或多个列构成。子查询结果的测试方法各不相同。

可以用诸如“=”或“<”之类的关系比较运算符来评估标量子查询的结果。

可以用运算符IN和NOT IN来测试某给定值是否包含在子查询的结果集里。

可以用运算符ALL、ANY和SOME把某给定值与子查询的结果集进行比较。

可以用运算符EXISTS和NOT EXISTS来测试子查询的结果是否为空。

标量子查询是最严格的，因为它只会产生一个值。正因如此，标量子查询的适用范围也最大。从理论上讲，标量子查询可以出现在任何允许使用标量操作数的地方，如出现在某个表达式里、作为函数参数或者放置在输出列的列表里。列、行和表这3类子查询会返回更多的信息，因此不能用在要求必须使用单个值的环境里。

子查询要不要相关都可以。也就是说，子查询既可以引用和依赖外层查询里的值，也可以不这样做。

除了 SELECT 语句，子查询还可以用在其他语句里。不过，如果把子查询用在一条会改变表内容的语句里（如 DELETE、INSERT、REPLACE、UPDATE、LOAD DATA），那么MySQL会强行限制这个子查询，不允许它查询正被修改的那个表。

有些子查询可以改写为连接操作。你会发现掌握子查询的改写技术很有用，通过这门技术可以看出MySQL优化器在使用连接的情况下，是否比使用等价的子查询表现得更好。

接下来的几节讨论的内容是：多种可用来测试子查询结果的操作、如何编写相关子查询，以及如何把子查询改写为连接。

2.9.1 带关系比较运算符的子查询

运算符=、<>、>、>=、<和<=可用来对值之间的关系进行比较。当与标量子查询配合使用时，它们会将外层查询里所有与子查询的返回值有着特定关系的行找出来。例如，想要查看'2012-09-23'那天的测验成绩，则可以先用一个标量子查询来确定该次测验的事件 ID，然后再在外层的SELECT里，针对那个ID与score表的所有行进行匹配：

```
SELECT * FROM score

WHERE event_id =

(SELECT event_id FROM grade_event

WHERE date = '2012-09-23' AND category = 'Q');
```

在这种形式的语句里，子查询的前面有一个值和一个关系比较运算符，因此这个子查询一定是只产生一个值。也就是说，它一定是个标量子查询；如果它返回了多个值，那么整条语句将以失败告终。有时，为了满足只返回一个值的要求，也可以使用LIMIT 1来限制子查询的结果。

如果遇到的问题可以通过在WHERE子句里使用某个聚合函数来解决，那么可以考虑用带关系比较运算符的标量子查询。例如，要知道在president表里的哪位总统出生得最早，则可能会试着编写出下面这条语句：

```
SELECT * FROM president WHERE birth = MIN(birth);
```

可这个办法是行不通的，因为你不能在WHERE子句里使用聚合函数。（WHERE子句的用途是确定应该选取哪些行，但MIN()的值只有在选取行之后才能确定下来。）不过，你可以像下面这样用一个子查询来产生一个最小的出生日期：

```
SELECT * FROM president
```

```
WHERE birth = (SELECT MIN(birth) FROM president);
```

也可以用其他的聚合函数来解决类似的问题。下面这条语句使用了一个子查询来选取某次考试中高于平均分数分数：

```
SELECT * FROM score WHERE event_id = 5
```

```
AND score > (SELECT AVG(score) FROM score WHERE event_id = 5);
```

如果子查询返回的是一个行，那么可以用一个行构造器来实现一组值（即元组）与子查询结果的比较。下面这条语句会返回多行与John Adams总统出生于同一个州和城市的那些总统的行：

```
mysql> SELECT last_name, first_name, city, state FROM president
```

```
-> WHERE (city, state) =
```

```
-> (SELECT city, state FROM president
```

```
-> WHERE last_name = 'Adams' AND first_name = 'John');
```

last_name	first_name	city	state
Adams	John	Braintree	MA
Adams	John Quincy	Braintree	MA

也可以使用ROW(city, state) 表示法，它等价于(city, state)。两者都可用作行构造器。

2.9.2 IN和NOT IN子查询

当子查询要返回多个行来与外层查询进行比较运算时，可以使用运算符IN和NOT IN。它们会测试某个给定的比较值是否存在于某一组值里。如果外层查询里的行与子查询返回的某一个行相匹配，那么IN的结果即为真。如果外层查询里的行与子查询返回的所有行都不匹配，那么NOT IN的结果即为真。下面两条语句分别使用IN和NOT IN来查找那些在absence表里有缺勤记录的学生和没有缺勤记录（也就是全勤）的学生：

```
mysql> SELECT * FROM student
```

```
-> WHERE student_id IN (SELECT student_id FROM absence);
```

name	sex	student_id
Kyle	M	3
Abby	F	5
Peter	M	10
Will	M	17
Avery	F	20

```
mysql> SELECT * FROM student
```

```
-> WHERE student_id NOT IN (SELECT student_id FROM
absence);
```

name	sex	student_id
Megan	F	1
Joseph	M	2
Katie	F	4
Nathan	M	6
Liesl	F	7

...

IN和NOT IN还可以用于会返回多个列的子查询。也就是说，可以把它们与表的子查询一起使用。此时，需要使用一个行构造器来指定与各列进行测试的比较值。

```
mysql> SELECT last_name, first_name, city, state FROM president
-> WHERE (city, state) IN
-> (SELECT city, state FROM president
-> WHERE last_name = 'Roosevelt');
```

last_name	first_name	city	state
Roosevelt	Theodore	New York	NY
Roosevelt	Franklin D.	Hyde Park	NY

IN和NOT IN实际上就是= ANY和<> ALL的同义词，相关内容会在下一节进行讨论。

2.9.3 ALL、ANY和SOME子查询

运算符ALL和ANY常与某个关系比较运算符结合在一起使用，以便测试列子查询的结果。它们会测试比较值与子查询返回的全部或部分值是否匹配。例如，当比较值小于或等于子查询返回的每个值时，<= ALL的结果为真；当比较值小于或等于子查询返回的任意值时，<= ANY的结果为真。SOME是ANY的同义词。

下面这条语句可以用来检索最早出生的总统，具体做法是选取出生日期小于或等于president表里所有出生日期（只有最早的那个出生日期才会满足这一条件）的那个行：

```
mysql> SELECT last_name, first_name, birth FROM president
-> WHERE birth <= ALL (SELECT birth FROM president);
```


last_name	first_name	birth
Washington	George	1732-02-22

下面这条语句的用处就不大了，它会返回所有的行，因为对于每个日期，都至少会有另一个日期（它自身）小于或等于它：

```
mysql> SELECT last_name, first_name, birth FROM president
```

```
-> WHERE birth <= ANY (SELECT birth FROM president);
```

last_name	first_name	birth
Washington	George	1732-02-22
Adams	John	1735-10-30
Jefferson	Thomas	1743-04-13
Madison	James	1751-03-16
Monroe	James	1758-04-28

...

当ALL、ANY或SOME与比较运算符“=”配合使用时，子查询可以是表子查询。此时，需要使用一个行构造器来提供比较值。

```
mysql> SELECT last_name, first_name, city, state FROM president
```

```
-> WHERE (city, state) = ANY
```

```
-> (SELECT city, state FROM president
```

```
-> WHERE last_name = 'Roosevelt');
```

last_name	first_name	city	state
Roosevelt	Theodore	New York	NY
Roosevelt	Franklin D.	Hyde Park	NY

前一节提到过，IN和NOT IN是= ANY和<> ALL的简写。也就是说，IN的含义是“等于子查询所返回的某个行”，而NOT IN的含义是“不等于子查询所返回的任何行”。

2.9.4 EXISTS和NOT EXISTS子查询

运算符EXISTS和NOT EXISTS只会测试某个子查询是否返回了行。如果有返回，则EXISTS的结果为真，而NOT EXISTS的结果为假。下面两条语句展示了这两个运算符的具体用法。如果absence表为空，那么第一条语句将返回0，第二条语句将返回1：

```
SELECT EXISTS (SELECT * FROM absence);
```

```
SELECT NOT EXISTS (SELECT * FROM absence);
```

EXISTS和NOT EXISTS实际上在相关子查询里比较常见。相关示例请参考2.9.5节。

在使用EXISTS和NOT EXISTS时，子查询通常将“*”用作输出列的列表。因为这两个运算符是根据子查询是否返回了行来判断真假的，并不关心行所包含的具体内容，所以没必要显式地列出列名。事实上，可以在子查询的列选取列表里编写任何东西，但如果想要确保在子查询成功时返回一个真值，则可以把它写成SELECT 1，而不要写成SELECT *。

2.9.5 相关子查询

子查询要不要相关都可以。

不相关的子查询不会引用外层查询里的值，因此它自己可以作为一条的单独查询命令去执行。例如，下面这条语句里的子查询就是不相关的，它只引用了t1，而没有引用t2：

```
SELECT j FROM t2 WHERE j IN (SELECT i FROM t1);
```

相关子查询则引用了外层查询里的值，所以它也就依赖于外层查询。因为有了这种联系，所以相关子查询不能脱离外层查询作为一条独立的查询语句去执行。例如，对于下面这条语句里的子查询，其作用是把t2中j列的每一个值与t1中i列的值进行匹配：

```
SELECT j FROM t2 WHERE (SELECT i FROM t1 WHERE i = j);
```

相关子查询通常用在EXISTS和NOT EXISTS 子查询里，这类子查询主要用于在某个表里查找在另一个表里有匹配行或没有匹配行的行。相关子查询的工作原理是：把值从外层查询传递到子查询，并检查它们是否满足子查询里指定的那些条件。因此，如果列名会引起歧义（在多个表里有同名列），那么必须使用表名来限定这些列名。

下面的EXISTS 子查询可以标识出两个表之间的匹配情况——即在两个表里都存在的那些值。整条语句的作用是将absence表里至少有一条缺勤记录的学生查询出来：

```
SELECT student_id, name FROM student WHERE EXISTS  
  
(SELECT * FROM absence WHERE absence.student_id =  
student.student_id);
```

NOT EXISTS 则会标识出那些无匹配的情况——即那些在一个表里有，但在另一个表里不存在的值。下面这条语句将把没有缺勤记录的学生查询出来：

```
SELECT student_id, name FROM student WHERE NOT EXISTS
```

```
(SELECT * FROM absence WHERE absence.student_id =  
student.student_id);
```

2.9.6 FROM子句里的子查询

子查询可以用在FROM子句里，以生成某些值。此时，子查询的结果就像一个表。FROM子句里的子查询可以参与关联操作，其值可以在WHERE子句里进行测试，等等。在使用这种类型的子查询时，必须提供一个表别名，用作子查询结果的名字：

```
mysql> SELECT * FROM (SELECT 1, 2) AS t1 INNER JOIN (SELECT 3, 4)
AS t2;
```

```
-----+-----+
1 | 2 | 3 | 4
-----+-----+
1 | 2 | 3 | 4
-----+-----+

```

2.9.7 将子查询改写为连接

有相当一部分使用了子查询的查询命令，可以改写为连接；并且，在检查那些倾向于编写成子查询的那些查询时，不妨考虑这样做。连接的效率有时会比子查询更好些，因此如果某条使用子查询的SELECT 语句需要花费很长时间才能执行完毕，那么可以尝试把它改写为一个连接，看看执行效果是否有所改善。本节将讨论具体该如何做。

2.9.7.1 改写用来查询匹配值的子查询

下面这条示例语句包含有一个子查询，它只会把score表里的考试（不含测验）成绩查询出来：

```
SELECT * FROM score

WHERE event_id IN (SELECT event_id FROM grade_event WHERE category
= 'T');
```

在编写这条语句时，可以不使用子查询，而是把它转换为一个简单的连接：

```
SELECT score.* FROM score INNER JOIN grade_event

ON score.event_id = grade_event.event_id WHERE grade_event.category = 'T';
```

再来看另一个示例。下面这条查询语句可以把女生们的考试成绩查询出来：

```
SELECT * from score
```

```
WHERE student_id IN (SELECT student_id FROM student WHERE sex = 'F');
```

这条语句可以像下面这样转换成连接：

```
SELECT score.* FROM score INNER JOIN student
```

```
ON score.student_id = student.student_id WHERE student.sex = 'F';
```

这里有一个模式。这些子查询语句都遵从这样一种形式：

```
SELECT * FROM table1
```

```
WHERE column1 IN (SELECT column2a FROM table2 WHERE column2b =  
value);
```

这类查询都可以被转换为下面这种形式的连接查询：

```
SELECT table1.* FROM table1 INNER JOIN table2
```

```
ON table1.column1 = table2.column2a WHERE table2.column2b = value;
```

在某些场合，子查询和关联查询可能会返回不同的结果。当table2包含column2a的多个实例时，会发生这种情况。那种形式的子查询只会为每个column2a 值生成一个实例；而连接操作会为所有值生成实例，并且其输出会包含重复行。如果想要防止这种重复记录出现，就要在编写连接查询语句时使用SELECT DISTINCT，而不能使用SELECT。

2.9.7.2 改写用来查询非匹配（缺失）值的子查询

另一种常见的子查询语句类型是：把存在于某个表里，但在另一个表里并不存在的那些值查找出来。我们前面见过，与“哪些值不存在”有关的问题通常都可以用LEFT JOIN来解决。下面是一个我们曾经见过的子查询，它用来测试哪些学生没有出现在absence表里（它可以把那全勤学生查找出来）：

```
SELECT * FROM student
```

```
WHERE student_id NOT IN (SELECT student_id FROM absence);
```

这条查询语句可以像下面那样使用LEFT JOIN来改写：

```
SELECT student.*
```

```
FROM student LEFT JOIN absence ON student.student_id =  
absence.student_id
```

```
WHERE absence.student_id IS NULL;
```

通常情况下，如果子查询语句符合如下所示的形式：

```
SELECT * FROM table1
```

```
WHERE column1 NOT IN (SELECT column2 FROM table2);
```

那么可以把它改写为下面这样的连接查询：

```
SELECT table1.*
```

```
FROM table1 LEFT JOIN table2 ON table1.column1 = table2.column2
```

```
WHERE table2.column2 IS NULL;
```

这里需要假设table2.column2被定义成了NOT NULL的。

与LEFT JOIN相比，这个子查询的好处更直观。大部分人都可以毫无困难地理解“没被包含在……里面”的含义，因为它不是数据库编程技术带来的新概念。概念“左连接”有所不同，难以用自然语言直观地描述出它的含义。

2.10 使用UNION实现多表检索

如果想把多个查询的结果合并成一个结果集，那么需要使用UNION语句。在本节的各个示例里，假设你有3个表，分别为t1、t2和t3：

```
mysql> SELECT * FROM t1;
```

i	c
1	red
2	blue
3	green

```
mysql> SELECT * FROM t2;
```

j	c
-1	tan
1	red

```
mysql> SELECT * FROM t3;
```

d	k
1904-01-01	100
2004-01-01	200
2004-01-01	200

表t1和t2拥有整数列和字符列，表t3拥有日期列和整数列。如想要编写一条UNION语句把多个检索合并在一起，首先需要写出多条SELECT语句，然后把关键字UNION放置到它们的中间。每条 SELECT 语句都必须检索出相同数量的列。例如，下面这条语句将选取各个表的整数列，并把它们合并在一起：

```
mysql> SELECT i FROM t1 UNION SELECT i FROM t2 UNION SELECT i
FROM t3;
```



```

-----
i
-----
1
2
3
-1
100
200

```

UNION有以下几种特性。

(1) 列名和数据类型。UNION结果集里的列名来自第一个SELECT里的列名。UNION中的第二个及后面的SELECT必须选取相同个数的列，但各相应列不必具有相同的名字和数据类型。（通常情况下，同一条UNION语句里的各相应列都具有相同的数据类型。如果它们不一样，那么MySQL会进行必要的类型转换。）列是根据位置，而不是根据名字来匹配。这也正是下面两条语句会返回不同结果的原因，尽管它们从两个表里查询到的是相同的值：

```
mysql> SELECT i, c FROM t1 UNION SELECT i, d FROM t3;
```

```

-----+-----
i      | c
-----+-----
1      | red
2      | blue
3      | green
100    | 1904-01-01
200    | 2004-01-01
-----+-----

```

```
mysql> SELECT i, c FROM t1 UNION SELECT d, i FROM t3;
```

i	c
1	red
2	blue
3	green
1904-01-01	100
2004-01-01	200

在各条语句的结果里，每一列的数据类型都是根据所选值的类型来确定的。在第一条语句里，我们为第二个列选取的类型是字符串和日期。而结果是一个字符串列。在第二条语句里，第一个列选取的类型是整数和日期，而第二个列选取的类型是字符串和整数。对于这两种情况，最后的结果都是一个字符串列。

（2）重复行处理。在默认情况下，UNION会将结果集里的重复行剔除掉：

```
mysql> SELECT * FROM t1 UNION SELECT * FROM t2 UNION SELECT
* FROM t3;
```

i	c
1	red
2	blue
3	green
-1	tan
1904-01-01	100
2004-01-01	200

表t1和t2都有一个行包含的值是1和'red'，但在输出结果里只存在一行这样的记录。此外，表t3有两个行包含'2004-01-01'和200，而其中的一个也被剔除了。

UNION DISTINCT是UNION的同义词，它们都只会保留不同的行。

如果你想保留重复的行，则需要把所有的UNION改为UNION ALL：

```
mysql> SELECT * FROM t1 UNION ALL SELECT * FROM t2 UNION ALL  
SELECT * FROM t3;
```

i	c
1	red
2	blue
3	green
-1	tan
1	red
1904-01-01	100
2004-01-01	200
2004-01-01	200

如果把UNION或UNION DISTINCT与UNION ALL混合使用，那么所有的“不同联合”操作都会比其左边的所有UNION ALL操作获得更高的优先级。

(3) ORDER BY和LIMIT处理。如果想将UNION结果作为一个整体进行排序，那么需要用括号把每一个SELECT语句括起来，并在最后那个SELECT语句的后面再加上一个ORDER BY子句。因为UNION会使用第一个SELECT语句里的列名，所以ORDER BY子句必须引用那些名字，而不能引用最后那个SELECT语句里的列名：

```
mysql> (SELECT i, c FROM t1) UNION (SELECT i, d FROM t3)
```

```
-> ORDER BY c;
```

i	c
100	1904-01-01
200	2004-01-01
2	blue
3	green
1	red

如果某个排序列是别名，那么在UNION末尾的ORDER BY子句必须引用这个别名。此外，ORDER BY不能引用表名。如果需要按第一个SELECT语句里某个以table_name.col_name的形式指定的列来进行排序，那么必须为此列取一个别名，并在ORDER BY子句里引用此别名。

类似地，如果想要限制UNION所返回行的数量，那么可以在语句末尾加上LIMIT子句：

```
mysql> (SELECT * FROM t1) UNION (SELECT * FROM t2) UNION
(SELECT * FROM t3)
```

```
-> LIMIT 2;
```

i	c
1	red
2	blue

子句 ORDER BY和LIMIT 还可以用在被括号括起来的单个SELECT 里，从而只会对这条SELECT语句起作用：

```
mysql> (SELECT * FROM t1 ORDER BY i LIMIT 2)
```

```
-> UNION (SELECT * FROM t2 ORDER BY j LIMIT 1)
```

-> UNION (SELECT * FROM t3 ORDER BY d LIMIT 2);

-----+-----	
i	c
-----+-----	
1	red
2	blue
-1	tan
1904-01-01	100
2004-01-01	200
.	

在用括号括起来的单个SELECT语句里，ORDER BY只能与LIMIT一起使用，用以确定LIMIT将作用于哪些行。此时，它不会影响UNION的最终结果行的先后顺序。

2.11 多表删除和更新

有时，需要根据某些行是否与另一个表里的行相匹配来删除它们。类似地，也可能需要使用一个表里的内容去更新另一个表。本节将讨论如何完成涉及多个表的DELETE操作和UPDATE操作。这些类型的语句都会大量用到那些用于连接操作的概念，因此你一定要熟悉前面 2.8 节里讨论过的那些内容。

要完成单个表的DELETE和UPDATE操作，只需要引用一个表的列即可，并不需要使用表名来限定列名。例如，下面这条语句会将表t里的id值大于100的所有行删除掉：

```
DELETE FROM t WHERE id > 100;
```

如果需要根据一个表里的行与另一个表里的行之间的关系，而不是根据它们自身的特性，来删除它们，那该怎么办呢？假设要从表t里删除这样一些行，其id值可以在另一个表t2里找到（或者不在这个表里），该怎么做呢？

在编写涉及多表的DELETE语句时，需要把所有涉及的表全部列在FROM子句里，并把用来匹配表中各行的检索条件写在WHERE子句里。下面这条语句，将从表t1里把其id值可以在另一个表t2里找到的所有行全部删掉：

```
DELETE t1 FROM t1 INNER JOIN t2 ON t1.id = t2.id;
```

请注意，如果某个列名出现在多个表里，则可能导致歧义问题，这时必须用表的名字对它加以限定。

DELETE语句还支持一种语法，可以一次性删除多个表里的行。如果要从两个表里把id值相匹配的行都删掉，那么必须在DELETE关键字的后面写出这两个表的名字：

```
DELETE t1, t2 FROM t1 INNER JOIN t2 ON t1.id = t2.id;
```

如果想要删除不匹配的行，又该怎么办呢？在涉及多个表的DELETE 语句里，可以使用任意一种 SELECT 语句支持的连接操作，因此，按照相同的策略，同样可以使用在编写查找多个表里不匹配行的SELECT语句时所采用的方法。也就是说，使用LEFT JOIN或者RIGHT JOIN。例如，想要从表 t1 里把在表 t2 中没有匹配的行找出来，那么可以编写下面这样的SELECT语句：

```
SELECT t1.* FROM t1 LEFT JOIN t2 ON t1.id = t2.id WHERE t2.id IS NULL;
```

同样地，从表t1找出并删除那些行的DELETE语句，也要用到LEFT JOIN操作：

```
DELETE t1 FROM t1 LEFT JOIN t2 ON t1.id = t2.id WHERE t2.id IS NULL;
```

MySQL还支持另一种多表DELETE语法。这种语法会使用一个FROM子句，列出那些需要删除行的表；并使用一个USING子句来连接这些表，从而确定出哪些行是需要删除的。采用这种语法，可以将前面那几条多表DELETE语句改写成下面这个样子：

```
DELETE FROM t1 USING t1 INNER JOIN t2 ON t1.id = t2.id;
```

```
DELETE FROM t1, t2 USING t1 INNER JOIN t2 ON t1.id = t2.id;
```

```
DELETE FROM t1 USING t1 LEFT JOIN t2 ON t1.id = t2.id WHERE t2.id IS NULL;
```

那些编写多个表的UPDATE语句所涉及的原则，与编写多个表的DELETE语句所涉及的原则很相似。即同样需要列出所有涉及的表，同样需要用表名来限定列名。假设你在2012 年 9 月23日对学生进行的一次测验中，有一道题，所有学生都没有答对，后来你却发现是因为你把标准答案弄错了，所以才会出现那样的情况。于是，你决定为每位学生的考试成绩都加上 1分。使用多表UPDATE语句，可以做到这一点，如下所示：

```
UPDATE score, grade_event SET score.score = score.score + 1  
  
WHERE score.event_id = grade_event.event_id  
  
AND grade_event.date = '2012-09-23' AND grade_event.category = 'Q';
```

对于这个问题，使用单表更新和子查询同样可以达到目的：

```
UPDATE score SET score = score + 1  
  
WHERE event_id = (SELECT event_id FROM grade_event  
  
WHERE date = '2012-09-23' AND category = 'Q');
```

但其他情况的更新操作则不能使用子查询来改写。例如，你不仅想要根据另外一个表的内容来标识需要更新的那些行，而且还想要把另一个表的列值复制到这个表里来。下面这条语句可以把id列值相匹配的各行从t1.a列复制到t2.a列：

```
UPDATE t1, t2 SET t2.a = t1.a WHERE t2.id = t1.id;
```

如果要对InnoDB类型的表进行多表删除和更新操作，那么可以不使用刚才介绍的语法。更好的办法是，在这些表之间建立一个外键关系，并让它包含约束条件 ON DELETE CASCADE 或ON UPDATE CASCADE。更多详细信息请参考2.13节。

2.12 事务处理

事务（transaction）指的是一组SQL语句，它们是一个执行单位，且在必要时还可以取消。并非所有的语句每次都能执行成功，也可能它们都没什么影响。事务处理是通过使用提交（commit）和回滚（rollback）功能来实现的。如果某个事务里的所有语句都成功执行，那么你可以把它提交到数据库永久性地记录下来。如果在事务执行过程中发生了错误，则可以通过回滚操作取消该事务。在事务里，所有在出错之前执行的语句都将被还原；而数据库也会恢复到事务开始执行之前的那个状态。

提交和回滚所提供的功能，能够确保尚未全部完成的操作不会影响到数据库，而且不会让数据库处于部分更新的（不一致）状态。财务转账，即把钱从一个账户转移到另一个账户，便是一个涉及事务处理的典型例子。假设Bill向Bob开了一个100美元的支票，然后Bob去兑现这个支票。这时，Bill的账户里就应该减少100美元，而Bob的账户则应该增加相同数量的存款：

```
UPDATE account SET balance = balance - 100 WHERE name = 'Bill';
```

```
UPDATE account SET balance = balance + 100 WHERE name = 'Bob';
```

如果在执行这两条语句时正好发生了故障，那么这个操作就不能算完成。根据这些语句在执行时的具体顺序不同，可能出现这样一些情况：Bill的账户可能少了100美元，而 Bob的账户金额没增加；或者Bob的账户多了100美元，而Bill的账户金额没减少。这两种情况都不正确。如果没有使用事务机制，那么你将不得不以手动方式分析你的日志，以便查明在出故障时都有哪些操作正在进行，从而确定出要如何还原或继续完成这些操作。事务的回滚功能可以让你正确地处理好这样的状况，只需把发生错误之前已经执行完的语句所产生的效果还原即可。（你可能还需要确定出哪些事务不能再输入，以及哪些事务需要再次执行。但不管怎样，你至少不用担心那些未能完全成功执行的事务会损害到数据库的完整性。）

事务的另一个用途是，确保某个操作所涉及的行不会在你正在使用它们时被其他客户端修改。MySQL在执行每一条 SQL语句时，会自动对该语句所涉及的资源进行锁定，以避免各个客户端之间相互干扰。但这仍不足以保证每一个数据库操作都总是能够得到其预想的结果，因为有些数据库操作需要多条语句才能完成。在这种情况下，不同的客户端之间就有可能出现相互干扰。事务把多条语句定义为一个执行单位，便可以防止在多客户端环境里可能会发生的并发问题。

事务系统通常具有“ACID”的四种特性，即Atomic（原子性）、Consistent（一致性）、Isolated（独立性）和Durable（持久性）。对各个特性的具体描述如下。

原子性。构成事务的所有语句应该是一个独立的逻辑单元。你不能只执行它们当中的一部分。

一致性。数据库在事务的执行前后都必须是一致的。例如，对于在某个表里的各行，其ID必须能在另外一个表里找到；否则，一个事务在试图插入带无效ID的行时会出现失败，然后会执行回滚操作。

独立性。事务之间不应该相互影响，这样事务在并发执行时才会得到与挨个依次执行一样的效果。

持久性。当事务执行成功完成时，其影响将被永久性地记录到数据库里。

事务处理为数据库操作的结果提供了强有力的保证，但这也对CPU周期、内存和磁盘空间等方面的开销提出了更多的需求。MySQL 提供了几种存储引

擎，有的是事务安全的（如InnoDB），有的则不是（如MyISAM和MEMORY）。事务的这四个特性对于有些应用来说是必不可少的，但对于其他应用来说则没那么重要。你可以根据自己的具体情况来进行选择。金融业务通常需要事务，并且对保证数据完整性的需求远大于对额外开销所占成本的顾虑。另一方面，对于需要将网页的访问情况记录到数据库表的应用，在服务器主机崩溃时会损失少量的几行记录，这是可以接受的。在这种情况下，使用非事务存储引擎则可以避免因事务处理所需而产生的额外开销。

2.12.1 利用事务保证语句安全执行

要想使用事务，就必须选用一种支持事务处理的存储引擎，如 InnoDB。像 MyISAM和MEMORY这样的存储引擎就不行。如果不太清楚自己的MySQL服务器是否支持事务存储引擎，请参考2.6.1.1节。

默认情况下，MySQL的运行模式是自动提交，即每条语句所做的更改会立刻提交到数据库，并永久保存下来。事实上，这相当于每条语句都被隐式地当作了一个事务来执行。如果想要显式地执行事务，那么需要禁用自动提交模式，并主动告知MySQL何时提交更改或何时回滚更改。

一种常用的执行事务的办法是：先调用START TRANSACTION（或BEGIN）语句，挂起自动提交模式；接着，再执行构成本次事务的各条语句；最后，用COMMIT 语句结束事务，从而让所有修改持久化。如果在事务处理过程中发生错误，那么可以调用ROLLBACK 语句来撤销所有的更改。START TRANSACTION语句会挂起当前的自动提交模式，因此在事务被提交或回滚之后，该模式又会恢复到执行 START TRANSACTION 之前的那个状态。如果自动提交模式原来是启用的，那么事务结束时还会回到自动提交模式。如果它原来是禁用的，那么在结束当前事务时会开始下一个事务。

下面的示例对这一方法进行了说明。首先，需要创建一个供使用的表：

```
mysql> CREATE TABLE t (name CHAR(20), UNIQUE (name)) ENGINE = InnoDB;
```

接下来，用START TRANSACTION语句初始化事务，然后往表里添加一些行，再提交事务，最后来看看表变成了什么样子：

```
mysql> START TRANSACTION;
```

```
mysql> INSERT INTO t SET name = 'William';
```

```
mysql> INSERT INTO t SET name = 'Wallace';
```

```
mysql> COMMIT;
```

```
mysql> SELECT * FROM t;
```

```
+-----+
```

```
| name      |
```

```
+-----+
```

```
| Wallace   |
```

```
| William   |
```

```
+-----+
```

你可以看到表里已记录到了几行内容。如果你启动第二个mysql程序实例，并且在插入之后、提交之前查询了表 t的内容，那么你将看不到这些行。那些行在第一个mysql 进程发出COMMIT语句之前，对第二个mysql进程来说是不可见的。

如果事务执行过程中发生了错误，那么你可以用ROLLBACK语句来取消它。仍以表t为例，在调用下面这些语句时你就会看到这种情况：

```
mysql> START TRANSACTION;
```

```
mysql> INSERT INTO t SET name = 'Gromit';
```

```
mysql> INSERT INTO t SET name = 'Wallace';
```

```
ERROR 1062 (23000): Duplicate entry 'Wallace' for key 'name'
```

```
mysql> ROLLBACK;
```

```
mysql> SELECT * FROM t;
```

```

-----
name
-----
Wallace
William
-----

```

第二条 INSERT 语句会试图往表里插入一行，而它与这个表里已有的某个行存在相同的name值。由于name列有一个UNIQUE索引，因此这条语句在执行时会失败。在调用ROLLBACK语句之后，这个表只包含在这个事务失败之前被插入的那两个行。准确地说，在发生错误之前已经成功执行的那条INSERT语句被还原了，它的影响并没有被记录到表里。

如果在事务处理过程中调用了START TRANSACTION语句，那么它将隐式地提交当前事务，然后开始一个新事务。

另一个执行事务的办法是，利用SET语句直接操作自动提交模式的状态：

```
SET autocommit = 0;
```

```
SET autocommit = 1;
```

将 autocommit 变量设置为 0，可以禁用自动提交模式。最后的效果是，后面的所有语句都将成为当前事务的一部分，语句范围一直延伸到你调用COMMIT 或 ROLLBACK 语句来提交或还原它为止。如果使用这个办法，那么自动提交模式的状态将一直保持下去，直到你把它设置回原来的状态。因此，在结束一个事务的同时，又将开始下一个事务。你也可以通过重新激活自动提交模式的办法来提交一个事务。

我们来看看这个办法是如何实施的。首先，创建一个与前面那个示例一样的表：

```
mysql> DROP TABLE t;
```

```
mysql> CREATE TABLE t (name CHAR(20), UNIQUE (name)) ENGINE =
InnoDB;
```

接下来，禁用自动提交模式，并插入一些行，然后提交此事务：

```
mysql> SET autocommit = 0;
```

```
mysql> INSERT INTO t SET name = 'William';
```

```
mysql> INSERT INTO t SET name = 'Wallace';
```

```
mysql> COMMIT;
```

```
mysql> SELECT * FROM t;
```

```
+-----+
```

```
| name      |
```

```
+-----+
```

```
| Wallace |
```

```
| William |
```

```
+-----+
```

此时，有两个行被插入了表，但自动提交模式仍处于禁用状态。如果你继续调用更多的语句，那么它们将成为另一个新事务的组成部分，它们的提交或还原与第一个事务不会有任何关系。下面这些语句可以说明自动提交模式仍处于被禁用状态，并且 `ROLLBACK` 将取消那些尚未被提交的语句：

```
mysql> INSERT INTO t SET name = 'Gromit';
```

```
mysql> INSERT INTO t SET name = 'Wallace';
```

```
ERROR 1062 (23000): Duplicate entry 'Wallace' for key 'name'
```

```
mysql> ROLLBACK;
```

```
mysql> SELECT * FROM t;
```

```
+-----+
```

```
| name      |
```

```
+-----+
```

| Wallace |

| William |

+-----+

要想重新启用自动提交模式，可以使用下面这条语句：

```
mysql> SET autocommit = 1;
```

正如刚才所述，无论是调用COMMIT或ROLLBACK语句，还是在自动提交模式处于禁用模式时再次启用它，当前事务都会结束。在其他情形里，事务也会结束。除了 SET autocommit、START TRANSACTION、BEGIN、COMMIT和 ROLLBACK等语句会显式地对事务产生影响外，还有另外一些语句，因为不能成为事务的一部分，而只会对事务产生隐式的影响。通常情况下，那些用来创建、更改或删除数据库或其中的对象的数据定义语言（Data Definition Language, DDL）语句，以及那些与锁定有关的语句都不能成为事务的一部分。例如，你在事务处理过程中调用了下列语句当中的某一条，那么服务器在执行这条语句之前将先提交当前事务：

ALTER TABLE

CREATE INDEX

DROP DATABASE

DROP INDEX

DROP TABLE

LOCK TABLES

RENAME TABLE

SET autocommit = 1 (if not already set to 1)

TRUNCATE TABLE

UNLOCK TABLES (if tables currently are locked)

针对你正在使用的那个MySQL 版本，具体有哪些语句会隐式地提交当前事务请参考MySQL参考手册。

如果在事务被提交之前，客户端结束了会话，或者因意外而中断，都会导致事务结束。此时，服务器会自动回滚该客户端正提交的所有事务。

如果客户端程序在与服务器的连接断开后，又再次自动重新连接上，那么这个新建的连接将会把默认状态重置为启用自动提交模式。

在各种场合里，事务都能派上用场。假设你正在处理成绩考评项目里那个score表，然后你突然发现有两名学生的分数搞混了，需要对换一下。那些输入有误的成绩如下所示：

```
mysql> SELECT * FROM score WHERE event_id = 5 AND student_id IN (8,9);
```

student_id	event_id	score
8	5	18
9	5	13

想要修复这个错误，应该把8号学生的成绩改成13，把9号学生的成绩改为18。使用下面两条语句即可轻松地完成：

```
UPDATE score SET score = 13 WHERE event_id = 5 AND student_id = 8;
```

```
UPDATE score SET score = 18 WHERE event_id = 5 AND student_id = 9;
```

不过，你必须保证这两条语句是作为一个单元成功执行的。这是一个需要使用事务来解决的问题。先用START TRANSACTION语句来实现：

```
mysql> START TRANSACTION;
```

```
mysql> UPDATE score SET score = 13 WHERE event_id = 5 AND student_id = 8;
```

```
mysql> UPDATE score SET score = 18 WHERE event_id = 5 AND student_id = 9;
```

```
mysql> COMMIT;
```

下面再通过直接显式地设置自动提交模式来完成同样的事情：

```
mysql> SET autocommit = 0;
```

```
mysql> UPDATE score SET score = 13 WHERE event_id = 5 AND student_id = 8;
```

```
mysql> UPDATE score SET score = 18 WHERE event_id = 5 AND student_id = 9;
```

```
mysql> COMMIT;
```

```
mysql> SET autocommit = 1;
```

这两种办法都可以保证两名学生的分数被正确地交换过来：

```
mysql> SELECT * FROM score WHERE event_id = 5 AND student_id IN (8,9);
```

student_id	event_id	score
8	5	13
9	5	18

2.12.2 使用事务保存点

MySQL可以让你对事务进行部分回滚。具体做法是，在事务里调用 `SAVEPOINT` 语句来设置一些命名标记。在事务的后续部分，如果想要回滚到那个标记点位置，那么需要使用 `ROLLBACK` 语句来指定那个保存点。下列语句说明了整个过程是如何工作的：

```
mysql> CREATE TABLE t (i INT) ENGINE = InnoDB;
```

```

mysql> START TRANSACTION;

mysql> INSERT INTO t VALUES(1);

mysql> SAVEPOINT my_savepoint;

mysql> INSERT INTO t VALUES(2);

mysql> ROLLBACK TO SAVEPOINT my_savepoint;

mysql> INSERT INTO t VALUES(3);

mysql> COMMIT;

mysql> SELECT * FROM t;

+-----+
| i      |
+-----+
|      1 |
|      3 |
+-----+

```

在执行完这些语句之后，表里只有第一条和第三条INSERT语句插入的那些行；而第二条INSERT语句所插入的行被取消了，因为后面执行了那个部分回滚到my_savepoint保存点的操作。

2.12.3 事务隔离

因为 MySQL 是一个多用户数据库系统，所以不同的客户端可能会在同一时间试图访问同一个表。诸如MyISAM之类的存储引擎使用了表级的锁定机制，以此保证不同客户端不能同时修改同一个表。但这种做法在大量更新时操作时，难以提供很好的并发性能。InnoDB存储引擎采用的是另外一方法。它使用了比较底层的锁定方式，为客户端提供了更细致的表访问控制。在一个客户端修改某个行的同时，另一个客户端可以读取和修改同一个表里的另一个行。如果有两个客户端想要同时修改某个行，那么先锁定该行的那个客户端则可以先修改它。这比表级的锁定方式提供了更好的并发性能。不过，

这里还有一个问题，即一个客户端的事务是否应该看到另一个客户端的事务所做的修改。

InnoDB存储引擎实现的事务隔离级别功能，能够让客户端对它们想要看到的由其他事务所做的修改类型进行控制。它提供了多种不同的隔离级别，可以允许或预防在多个事务同时运行时可能出现的各类问题。

脏读（dirty read）。它指的是，在某个事务所做的修改尚未提交时，其他事务就能看到这些修改。其他事务因此认为这些行已被修改；即使那个对行进行修改的事务后来被回滚，从而导致这些行并未真的被修改，其他事务也会那样认为。

不可重复读（nonrepeatable read）。它指的是，同一个事务使用同一条SELECT语句在每次读取时得到的结果都不一样。如果有一个事务两次执行了同一条SELECT语句，但另一个事务在这条语句的两次执行期间修改了某些行，那么就会发生这种情况。

幻影行（phantom row）。它指的是，一个事务突然看到一个以前没有见过的行。假设某个事务在刚执行完一条SELECT语句之后，接着就有另一个事务插入了一个新的行。如果第一个事务再执行同样的一条SELECT语句，则可能会看到这个新增行，而它其实是一个幻影。

为了解决这些问题，InnoDB存储引擎提供了4种事务隔离级别。这些隔离级别确定了一个事务所做的哪些修改可以被其他与之同时执行的事务所看到。

READ UNCOMMITTED。它允许某个事务看到其他事务尚未提交的行修改。

READ COMMITTED。它只允许某个事务看到其他事务已经提交的行修改。

REPEATABLE READ。如果某个事务两次执行同一条 SELECT 语句，其结果是可重复的。也就是说，即使有其他事务在同时插入或修改行，这个事务所看到的结果也是一样的。

SERIALIZABLE。它与 REPEATABLE READ 很相似，但对事务的隔离更彻底，主要表现在：对于某个事务正在查看的行，只有等到该事务完成才能被其他事务所修改。也就是说，如果某个事务正在读取某些行，那么在它完成之前，其他事务都将无法对这些行修改。

表 2-4列出了这 4种隔离级别与脏读、不可重复读或幻影行这三个问题之间的认可关系。这个表格只适用于InnoDB存储引擎，其中REPEATABLE READ

级别不允许出现幻影行。而有些数据库系统的REPEATABLE READ级别却允许出现幻影行。

表2-4 隔离级别所认可的问题

隔离级别	脏读	不可重复读	幻影行
READ UNCOMMITTED	是	是	是
READ COMMITTED	否	是	是
REPEATABLE READ	否	否	否
SERIALIZABLE	否	否	否

InnoDB 存储引擎默认的隔离级别是 REPEATABLE READ。更改它的方式有两种：在服务器启动时使用--transaction-isolation选项，或者在服务器运行时使用SET TRANSACTION语句。此语句有以下3种形式：

```
SET GLOBAL TRANSACTION ISOLATION LEVEL level;
```

```
SET SESSION TRANSACTION ISOLATION LEVEL level;
```

```
SET TRANSACTION ISOLATION LEVEL level;
```

拥有SUPER权限的客户端，可以使用SET TRANSACTION语句来改变全局隔离级别，之后它会作用于后续的所有客户端连接。此外，所有客户端都可以修改其自己的事务隔离级别。用SET SESSION TRANSACTION语句做出的修改，将会对服务器当前会话里的所有后续事务起作用；而用SET TRANSACTION 语句做出的修改，则只会对下一个事务起作用。客户端在修改其自己的隔离级别时，不需要任何特殊的权限。

事务表和非事务表可以混用吗？

在某次事务中可以将事务表和非事务表混合使用，但最终的结果不一定是你所期望的。对非事务表进行操作的语句总是会立刻生效，即便是自动提交模式处于禁用状态也是如此。事实上，非事务表永远待在自动提交模式下，每条语句都会在执行完毕后立刻提交。因此，如果你在某个事务里修改了一个非事务表，那么这个修改将无法被还原。

2.13 外键和引用完整性

利用外键关系，你可以在一个表里声明与另一个表里的某个索引相关联的索引。你还可以把自己想要施加在表上的约束条件放到外键关系里。数据库会根据这个关系里的规则来维护数据的引用完整性。例如，`sampdb`数据库里的`score`表包含一个`student_id`列，我们要用它来把`score`表里的考试成绩与`student`表里的学生关联在一起。我们在第1章创建这些表时，在它们之间建立了一些明确的关系。例如，把 `score.student_id` 定义为了 `student.student_id`列的一个外键。这样可以确保：只把那些其`student_id`值在`student`表里存在的行，插到`score`表里去。也就是说，这个外键可以防止为不存在的学生输入成绩这一情况的出现。

外键不仅在行的插入操作中很有用，在删除和更新操作中也有用处。例如，我们可以建立这样一个约束条件，即从`student`表里删除某个学生时，也要同时自动删除`score`表里与这个学生有关的所有行。这就是所谓的级联删除（`cascaded delete`），删除操作的效果就像瀑布那样从一个表“流淌”到另外一个表。另外还可以进行级联更新。例如，有了级联更新，那么当在`student`表里更改某个学生的`student_id`时，`score`表里与这个学生相对应的所有行的这个值也会发生相应的变化。

外键可以帮我们维护数据的一致性，并且它们用起来也很方便。如果不使用外键，那么你必须自己负责保证表之间的依赖关系和维护它们的一致性，而这意味着你的应用程序必须增加一些必要的代码。在某些情况下，这样做也不需要花费太多的工夫，你只需额外调用几条`DELETE`语句，确保在你删除某个表里的行时，其他表里与之相对应的行也会随之一起被删除。但毕竟这是多出来的工作，而且既然数据库引擎能够替你完成数据的一致性检查，那么为何不使用它呢？如果你的表关系非常复杂，那么自动检查功将会特别有用。你完全不用在自己的应用程序里去负责实现这些依赖关系。

在MySQL里，`InnoDB`存储引擎提供了对外键的支持。本节将讨论如何设置`InnoDB`表来实现外键的定义，并且将讨论外键是如何影响表的使用方式的。首先，需要定义几条术语。

父表。它指的是包含原始键值的表。

子表。它指的是引用了父表中键值的相关表。

父表中的键值可以用来关联两个表。具体来讲，子表里的索引会引用父表里的索引。子表的索引值必须与父表中的索引值相匹配；或者被设置为`NULL`，以表明在父表里不存在与之对应的行。子表里的这个索引就是所谓的“外键”。也就是说，这个键处在父表的外部，但它又包含了指向父表的值。一个外键关系可以被设置为不接受 `NULL` 值，此时所有的外键值都必须与父表里的某个值相匹配。

InnoDB存储引擎会通过这些规则来保证：在外键关系里不会出现不匹配的情况。这就是所谓的参照完整性（referential integrity）。

下面所示内容为在子表里定义外键的语法：

```
[CONSTRAINT constraint_name]
```

```
FOREIGN KEY [fk_name] (index_columns)
```

```
REFERENCES tbl_name (index_columns)
```

```
[ON DELETE action]
```

```
[ON UPDATE action]
```

```
[MATCH FULL | MATCH PARTIAL | MATCH SIMPLE]
```

这个语法自身很完备，但InnoDB存储引擎目前还没有实现所有的子句语义：它目前还不支持MATCH子句，即使指定了MATCH子句，它也会被忽略掉。另外，还有几种action值可以被识别出来，但不会有任何效果。（对于除InnoDB 以外的其他存储引擎，它们可以解析整个FOREIGN KEY定义，但会把它完全忽略掉。）

InnoDB存储引擎着重关注的外键定义成分包含下面这样几个。

CONSTRAINT 子句。如果提供它，那么它会为外键约束提供一个名字。如果省略它，那么InnoDB存储引擎会创建一个名字。

FOREIGN KEY子句。它会列出子表里的索引列，这些列必须与父表里的索引值相匹配。

fk_name 是外键ID。即使提供了这种ID，它也会被忽略掉。只有InnoDB为这个外键自动创建了索引时，它才会起作用；在这种情况下，该索引名即为fk_name。

REFERENCES 子句。它会列出父表及其索引列的名字，让子表里的外键可以引用它们。

REFERENCES子句的index_columns部分必须与FOREIGN KEY关键字的index_columns部分拥有相同的列数。

ON DELETE子句。可以用它来指定在删除父表里的行时，子表应该做些什么。如果没有指定 ON DELETE 子句，那么其默认行为是“拒绝从父表里删除仍被子表的行所引用的那些行”。如果想显式地指定一种action值，那么可以使用下列子句的其中一条。

ON DELETE NO ACTION 子句和ON DELETE RESTRICT 子句。它们的含义与省略 ON DELETE子句一样。有些数据库系统提供了延迟检查功能，而NO ACTION就是一种延迟检查。对于InnoDB，由于外键约束条件会被立即检查，所以NO ACTION和RESTRICT的含义完全一样。

ON DELETE CASCADE 子句。它表示的是这样一种行为，即在删除父表里的行时，子表里与之相关联的行也会被删除掉。在本质上，删除产生的效果将从父表蔓延到子表。

这样一来，你只需删除父表里的行，然后让 InnoDB存储引擎负责从子表里删除相关的行，即可实现一个涉及多表的删除操作。

ON DELETE SET NULL子句。它表示的是这样一种行为，即在删除父表里的行时，子表里与之相关联的索引列会被设置为 NULL。如果想使用这个选项，那么必须把外键定义里指定的所有子表索引列，定义为允许有NULL值。（使用这种行为，即暗示着你不能把外键定义为PRIMARY KEY，因为主键不允许有NULL值。）

ON DELETE SET DEFAULT 子句。它可以被 MySQL正确识别，但尚未实现；InnoDB会将它报告为一个错误。

ON UPDATE子句。它用来指定：当更新父表里的行时，子表应该做些什么。如果没有指定这条子句，那么其默认行为是：在子表里，拒绝执行那些会导致其外键值在父表索引里无任何匹配的插入和更新操作；并且会阻止更新那些仍然被子表所引用的父表索引值。可供选用的action值与ON DELETE子句的相同，并且具有相同的效果。

如果想要建立外键关系，那么请遵从下面这些指导。

子表必须建立索引，且外键列需要放在首位。父表也必须建立索引，且REFERENCES子句里的列需要放在首位。（也就是说，外键关系所涉及的两个表里的外键列必须被索引。）在定义外键关系之前，必须显式地创建父表的索引。如果在创建子表的CREATE TABLE语句里没有包括这样的索引，那么InnoDB存储引擎将自动在子表里为外键（即引用列）创建索引。有时，不包括索引，可以让CREATE TABLE语句写起来更容易。不过，由InnoDB自动创建的索引是一个非唯一的索引，并且只包含外键列。如果想让它成为一个

PRIMARY KEY或UNIQUE索引，或者想让它能够包括在外键列之外的其他列，那么你就应该在子表里显式地定义它。

父表和子表索引里的对应列必须类型兼容。例如，不能用一个INT 列去匹配一个CHAR列。对应的字符列必须拥有相同的长度。对应的整型列必须拥有相同的大小，并且要么都是有符号数，要么都是UNSIGNED的。

不能对外键关系里的字符串列的前缀进行索引。（也就是说，对于字符串列，必须对整个列进行索引，而不能只对其前面的几个字符进行索引。）

在第 1章，我们为成绩考评项目创建了几个带有简单外键关系的表。现在来看一个更复杂的例子。首先，创建两个名字分别为parent和child的表。其中，child表包含一个外键，它会引用parent表里的par_id列：

```
CREATE TABLE parent
(
    par_id INT NOT NULL,
    PRIMARY KEY (par_id)
) ENGINE = INNODB;

CREATE TABLE child
(
    par_id INT NOT NULL,
    child_id INT NOT NULL,
    PRIMARY KEY (par_id, child_id),
    FOREIGN KEY (par_id) REFERENCES parent (par_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE
) ENGINE = INNODB;
```

这里定义的外键使用ON DELETE CASCADE子句指定了：当从parent表里删除某个行时，MySQL也应该自动从child表里删除与par_id值相匹配的行。ON UPDATE CASCADE子句表示的是，如果更改了parent表里某个行的par_id值，那么MySQL将自动把child表里所有匹配到的par_id值也更改为这个新值。

现在，在parent表里插入一些行，然后在child表里插入一些与之拥有相关键值的行：

```
mysql> INSERT INTO parent (par_id) VALUES(1),(2),(3);
```

```
mysql> INSERT INTO child (par_id,child_id) VALUES(1,1),(1,2);
```

```
mysql> INSERT INTO child (par_id,child_id) VALUES(2,1),(2,2),(2,3);
```

```
mysql> INSERT INTO child (par_id,child_id) VALUES(3,1);
```

这些语句将产生下面所示的表内容，其中，child表里的每个par_id值都分别与parent表里的某个par_id值相匹配：

```
mysql> SELECT * FROM parent;
```

par_id
1
2
3

```
mysql> SELECT * FROM child;
```

par_id	child_id
1	1
1	2
2	1
2	2
2	3
3	1

为证明InnoDB在插入新记录时会遵从外键关系的约束，我们现在来往child表插入一个行，而其par_id值在parent表里并无匹配：

```
mysql> INSERT INTO child (par_id,child_id) VALUES(4,1);
```

ERROR 1452 (23000): Cannot add or update a child row: a foreign key

constraint fails (`sampdb`.`child`, CONSTRAINT `child\_ibfk\_1` FOREIGN
KEY (`par\_id`) REFERENCES `parent` (`par\_id`) ON DELETE CASCADE
ON UPDATE CASCADE)

如果想测试级联删除的效果，可以看看在删除parent表里的一个行时会发生什么情况：

```
mysql> DELETE FROM parent WHERE par_id = 1;
```

MySQL会将从表parent里删除那一行：

```
mysql> SELECT * FROM parent;
```


par_id
2
3

此外，MySQL还会把这条DELETE语句的效果级联到child表：

```
mysql> SELECT * FROM child;
```

par_id	child_id
2	1
2	2
2	3
3	1

如果想测试级联更新的效果，可以看看在更新parent表里的一个行时会发生什么情况：

```
mysql> UPDATE parent SET par_id = 100 WHERE par_id =2;
```

```
mysql> SELECT * FROM parent;
```

par_id
3
100

```
mysql> SELECT * FROM child;
```

par_id	child_id
3	1
100	1
100	2
100	3

上面的两个示例演示了在删除和更新 parent 表里的行时，需要如何安排那些操作，才能让它们级联删除或更新child表里的相关行。ON DELETE子句和ON UPDATE子句还支持其他动作。例如，你可以让 child 表里的行都保持原样，而只把它们的外键列设置为 NULL。为实现这一点，需要对child表的定义做一些修改。

使用ON DELETE SET NULL 来代替 ON DELETE CASCADE。这将使 InnoDB 把外键列（par_id）设置为NULL，而不会删除这些行。

使用ON UPDATE SET NULL来代替ON UPDATE CASCADE。这将使InnoDB在更新parent表里的行时，把child表里与之匹配的行的外键列（par_id）设置为NULL。

child表里的par_id列最初被定义成NOT NULL。这无法与ON DELETE SET NULL和ON UPDATE SET NULL配合使用，因此必须把该列的定义更改为允许NULL值。

在child表的最初定义中，将par_id列定义成PRIMARY KEY的一部分。不过PRIMARY KEY 不允许包含 NULL 值。因此，在把 par_id 改成允许 NULL 值的同时，还需要把PRIMARY KEY改成UNIQUE索引。UNIQUE索引要求索引值必须是唯一的，但NULL值除外，它可以在索引里多次出现。

想要查看这些修改的效果，则需要使用最初的定义重新创建 parent 表，并在表中加载相同的初始行。然后，用如下所示的新定义创建一个新的child表：

```
CREATE TABLE child
```

```
(
```

```
par_id INT NULL,  
  
child_id INT NOT NULL,  
  
UNIQUE (par_id, child_id),  
  
FOREIGN KEY (par_id) REFERENCES parent (par_id)  
  
ON DELETE SET NULL  
  
ON UPDATE SET NULL  
  
) ENGINE = INNODB;
```

现在，当往 child 表里插入新行时，其行为和最初的定义基本一样。也就是说，只有其par_id值在parent表里能找到的行，才允许被插到child表里；否则，将拒绝插入：

```
mysql> INSERT INTO child (par_id,child_id) VALUES(1,1),(1,2);  
  
mysql> INSERT INTO child (par_id,child_id) VALUES(2,1),(2,2),(2,3);  
  
mysql> INSERT INTO child (par_id,child_id) VALUES(3,1);  
  
mysql> INSERT INTO child (par_id,child_id) VALUES(4,1);  
  
ERROR 1452 (23000): Cannot add or update a child row: a foreign key  
constraint fails ('sampdb'. 'child', CONSTRAINT 'child_ibfk_1' FOREIGN  
KEY ('par_id') REFERENCES 'parent' ('par_id') ON DELETE SET NULL  
ON UPDATE SET NULL)
```

请注意，往child表里插入新的行时，与原来相比还是有一点区别。因为par_id列现在被定义为允许NULL值，所以现在可以把包含NULL值的新行插到child表里，而不会出现错误。此外，当从parent表里删除行时，具体的表现行为也存在一点区别。请试着从parent表里删除一行记录，然后检查一下child表的内容，看看会发生什么事情：

```
mysql> DELETE FROM parent WHERE par_id = 1;
```

```
mysql> SELECT * FROM child;
```

par_id	child_id
NULL	1
NULL	2
2	1
2	2
2	3
3	1

此时，在child表里，所有par_id列的值为1的行都未被删除。而par_id列的值都被设置为了NULL，这正是ON DELETE SET NULL约束所指定的效果。

更新parent表里的行也会有类似的效果：

```
mysql> UPDATE parent SET par_id = 100 WHERE par_id = 2;
```

```
mysql> SELECT * FROM child;
```

par_id	child_id
NULL	1
NULL	1
NULL	2
NULL	2
NULL	3
3	1

如果想要查看某个InnoDB表都有哪些外键关系，可以使用SHOW CREATE TABLE语句。

如果在创建一个带有外键关系的表时遇到了问题，那么可以使用SHOW ENGINE INNODB STATUS语句来查看完整的出错消息。

2.14 使用FULLTEXT搜索

MySQL具备全文搜索的能力，它可以让你在不使用模板匹配操作的情况下进行单词或短语的查找。全文搜索有3种类型。

自然语言搜索（默认类型）。MySQL 会把搜索字符串解析成一系列的单词，然后搜索出包含这些单词的那些行。

布尔模式搜索。在搜索字符串里那些单词可以包含修饰字符，用以表明某些特定的要求，如某个给定的单词必须出现（或不出现）在匹配行里，或者某些行必须恰好包含某个短语。

查询扩展搜索。这种搜索分两阶段进行。第一阶段是自然语言搜索。第二阶段则先把原来的搜索字符串，与在第一阶段的搜索里高度匹配的那些行，连接在一起，然后再进行一次搜索。这种做法扩大了搜索范围，可以把那些与原有搜索字符串相关、但用这些字符串却匹配不到的那些行也找出来。

要想对某个给定表进行全文搜索，则必须事先为它创建一个特殊类型的索引，这种索引具有以下几个方面特点。

全文搜索基于FULLTEXT索引。在MySQL 5.5里，这些索引只能针对MyISAM类型的表创建。MySQL 5.6 引入了对 InnoDB的全文搜索支持，但在这里我们仍然是针对MyISAM 类型的表，因为你可能还没有 5.6 版本。在FULLTEXT 索引里，只能包含CHAR、VARCHAR和TEXT这几种类型的列。

全文搜索将会忽略掉那些常见词。这里的“常见”指的是“至少在一半的行里都出现过”。牢记这一点很重要，尤其是当你在建立测试表，以体验FULLTEXT 功能的时候。在测试表里，你至少需要插入3个行。如果那个表只有一两个行，那么它里面的每个单词将至少有50%的出现几率，所以对它进行全文搜索将不会得到任何结果！

有些内建的常用单词，如“the”、“after”和“other”等。它们都被称为“停用词”（stopword），在进行全文搜索时总是会被忽略掉。

太短的单词也会被忽略。默认情况下，“太短”指少于4个字符。不过，你可以重新配置服务器，把这个最小长度设置为其他值。更多信息请参考2.14.4节。

全文搜索对“单词”的定义是，它们是由字母、数字、撇号和下划线构成的字符序列。这意味着会把像“full-blooded”这样的字符串当成两个单词——“full”和“blooded”。一般情况下，全文搜索会匹配整个单词，而不会匹配部分单词。通常情况下，只要在某个行里找到了搜索字符串里的某个单词，那么FULLTEXT引擎便会认为这个行与搜索字符串相匹配。如果你使用的是布尔式全文搜索，那么你可以加上一些额外的约束条件，如要求所有的单词都必须存在（或者以任何顺序，或者严格按照搜索字符串里的顺序进行短语搜索）。还可以用布尔搜索来匹配那些不包含特定单词的行，或者通过添加一个通配符来匹配所有以一个给定前缀开头的单词。

可以为单个列或多个列创建FULLTEXT索引。如果它涉及多个列，基于该索引的搜索将在所有列上同时进行。反过来也就是说，在进行全文搜索时，你给出的列列表必须和某个FULLTEXT索引所匹配的那些列精确匹配。例如，有时想搜索 col1，有时想搜索col2，而有时想同时搜索col1和col2；那么，这时你需要创建3个索引，即两个列各占一个，而两个列的组合需要占一个。

接下来的几个示例将会演示如何使用全文搜索，这些示例首先创建了几个FULLTEXT索引，然后使用MATCH运算符来查询它们。在sampdb发行版的full-text目录里，提供了一个用来创建表的脚本，以及一些需要加载到其中的样本数据。

FULLTEXT索引与其他索引的创建方法大同小异。你可以在开始创建表的时候，在CREATE TABLE语句里定义它；也可以先创建表，然后再用ALTER TABLE或CREATE INDEX来添加它。因为FULLTEXT索引要求必须使用MyISAM类型的表，所以如果你正在创建一个新的需要使用全文搜索的表，那么可以顺便利用一下MyISAM存储引擎的其中一个特性：如果采用先填充表，然后再添加索引的方式，而不是采用将数据加载到已经索引过的表中的方式，那么表的加载过程会变得更快速一些。假设有一个名为apothegm.txt的数据文件，其中内容是一些名人及他们的名言信息：

Aeschylus	Time as he grows old teaches many lessons
-----------	---

Alexander Graham Bell	Mr.Watson, come here.I want you!
-----------------------	----------------------------------

Benjamin Franklin	It is hard for an empty bag to stand upright
-------------------	--

Benjamin Franklin	Little strokes fell great oaks
-------------------	--------------------------------

Benjamin Franklin	Remember that time is money
Miguel de Cervantes	Bell, book, and candle
Proverbs 15:1	A soft answer turneth away wrath
Theodore Roosevelt	Speak softly and carry a big stick
William Shakespeare	But, soft! what light through yonder window breaks?
Robert Burton	I light my candle from their torches.

如果按“名人”、“名言”和“名人加名言”来搜索，那么你需要创建3个FULLTEXT索引：两个列各算一个，这两列加起来算一个。根据下面这些语句，可以创建一个名为 apothegm的表，并对它进行填充和索引：

```
CREATE TABLE apothegm (attribution VARCHAR(40), phrase TEXT)
ENGINE = MyISAM;

LOAD DATA LOCAL INFILE 'apothegm.txt' INTO TABLE apothegm;

ALTER TABLE apothegm

ADD FULLTEXT (phrase),

ADD FULLTEXT (attribution),

ADD FULLTEXT (phrase, attribution);
```

2.14.1 自然语言FULLTEXT搜索

在创建好表之后，便可以用运算符MATCH列出一个或多个用于搜索的列，并用AGAINST()指定搜索字符串，对表进行自然语言类型的全文搜索。例如：

```
mysql> SELECT * FROM apothegm WHERE MATCH(attribution)
AGAINST('roosevelt');
```

attribution	phrase
Theodore Roosevelt	Speak softly and carry a big stick

```
mysql> SELECT * FROM apothegm WHERE MATCH(phrase)
AGAINST('time');
```

attribution	phrase
Benjamin Franklin	Remember that time is money
Aeschylus	Time as he grows old teaches many lessons

```
mysql> SELECT * FROM apothegm WHERE MATCH(attribution, phrase)
-> AGAINST('bell');
```

attribution	phrase
Alexander Graham Bell	Mr. Watson, come here. I want you!
Miguel de Cervantes	Bell, book, and candle

在最后这个示例里，要注意这条查询语句是如何在不同的列里将包含搜索单词的各个行查找出来的，其中也演示了利用FULLTEXT 索引同时搜索多个列的功能。另外还要注意，在这条查询语句里以名字方式列出来的各个列的顺序是：attribution, phrase。这与在创建索引时列出来的顺序（即 phrase, attribution）并不相同。由此说明这里的顺序并不重要。重要的是，这里必须要有一个FULLTEXT索引，而它正好是由查询命令里列出的那些列组成的。

如果只想看看某个搜索可以匹配到多少个行，那么可以使用COUNT(*)：


```
mysql> SELECT COUNT(*) FROM apothegm WHERE MATCH(phrase)
AGAINST('time');
```

```
-----
COUNT(*)
-----
2
-----
```

当你在WHERE子句里使用MATCH表达式时，对于自然语言类型的全文搜索，其输出行是按照相关程度递减顺序排列的。相关度是一个非负浮点数，其中零代表“毫不相关”。要想查看这些值，可以在输出列的列表里加上一个MATCH表达式：

```
mysql> SELECT phrase, MATCH(phrase) AGAINST('time') AS relevance
-> FROM apothegm;
```

phrase	relevance
Time as he grows old teaches many lessons	1.3253291845321655
Mr.Watson, come here.I want you!	0
It is hard for an empty bag to stand upright	0
Little strokes fell great oaks	0
Remember that time is money	1.340062141418457
Bell, book, and candle	0
A soft answer turneth away wrath	0
Speak softly and carry a big stick	0
But, soft! what light through yonder window breaks?	0
I light my candle from their torches.	0

自然语言搜索能够将包含所有搜索单词的那些行找出来，因此下面这条查询语句将把所有包含单词“hard”或“soft”的行找出来：

```
mysql> SELECT * FROM apothegm WHERE MATCH(phrase)
```

```
-> AGAINST('hard soft');
```

attribution	phrase
Benjamin Franklin	It is hard for an empty bag to stand upright
Proverbs 15:1	A soft answer turneth away wrath
William Shakespeare	But, soft! what light through yonder window breaks?

自然语言模式是默认的全文搜索模式。如果想要显式地指定这个模式，那么可以在搜索字符串的后面加上IN NATURAL LANGUAGE MODE。下面这条语句可以完成与前面那个示例完全一样的事情：

```
SELECT * FROM apothegm WHERE MATCH(phrase)
```

```
AGAINST('hard soft' IN NATURAL LANGUAGE MODE);
```

2.14.2 布尔模式的全文搜索

通过使用布尔模式的全文搜索，可以获得对多单词（multiple-word）搜索的更多控制。要完成这种类型的搜索，需要在AGAINST()函数里搜索字符串的后面添加IN BOOLEAN MODE。布尔模式搜索具有以下几个特点。

“50%规则”不再起作用。即使找到的单词会出现在一半以上的行里，仍然会把它们搜索出来。

查询结果不再按相关程度排序。

搜索可以要求短语里的所有单词都必须是按某种特定的顺序出现。如果想要匹配一个短语，那么需要把它用双引号引起来。只有在行包含的那些单词及其顺序与短语里列出的内容一致时，才会被认为是匹配上了：

```
mysql> SELECT * FROM apothegm
```

```
-> WHERE MATCH(attribution, phrase)
```

```
-> AGAINST('"bell book and candle"' IN BOOLEAN MODE);
```

-----+-----	
attribution	phrase </p>
-----+-----	
Miguel de Cervantes	Bell, book, and candle
-----+-----	

也可以对未被包括在FULLTEXT索引里的那些列，进行布尔模式的全文搜索，只是这样做会比对索引过的列进行搜索要慢一些。

对于布尔模式搜索，还可以为搜索字符串里的单词加上一些修饰符。在单词的前面加上一个加号，则表示该单词必须出现在匹配行里；而加上一个减号，则表示该单词不能出现在匹配行里。例如，搜索字符串'bell'会匹配到所有包含“bell”的行；但在布尔模式下，搜索字符串'+bell -candle'则只会与那些包含了“bell”但不包含“candle”的行相匹配。

```
mysql> SELECT * FROM apothegm
```

```
-> WHERE MATCH(attribution, phrase)
```

```
-> AGAINST('bell');
```

-----+-----	
attribution	phrase
-----+-----	
Alexander Graham Bell	Mr.Watson, come here.I want you!
Miguel de Cervantes	Bell, book, and candle
-----+-----	

```
mysql> SELECT * FROM apothegm
```

```
-> WHERE MATCH(attribution, phrase)
```

```
-> AGAINST('+bell -candle' IN BOOLEAN MODE);
```

attribution	phrase
Alexander Graham Bell	Mr.Watson, come here.I want you!

单词末尾的星号“*”是一个通配符，只要行包含的单词是以搜索单词开头的，那么这些行就是匹配的。例如，'soft*'与“soft”、“softly”、“softness”等都匹配：

```
mysql> SELECT * FROM apothegm WHERE MATCH(phrase)
```

```
-> AGAINST('soft*' IN BOOLEAN MODE);
```

attribution	phrase
Proverbs 15:1	A soft answer turneth away wrath
William Shakespeare	But, soft! what light through yonder window breaks?
Theodore Roosevelt	Speak softly and carry a big stick

不过，不能用这个通配符功能来匹配那些比最小索引单词长度还要短的单词。

在附录C的Match条目里，可以查到所有的布尔模式修饰符。

与自然语言全文搜索相似，布尔模式的全文搜索也会把所有停用词忽略掉，即使它们被标记为“必需的”也是一样。例如，搜索'+Alexander +the +great'将把那些包含“Alexander”和“great”的行找出来，而“the”则会因其是一个停用词而被忽略。

2.14.3 查询扩展全文搜索

支持查询扩展的全文搜索会分两个阶段完成搜索。第一阶段搜索与自然语言搜索一样。在这次搜索里，那些相关程度最高的行会被用于第二阶段的搜索。这些行里的单词会与原来那些搜索单词一起使用，完成第二阶

段的搜索。因为整个搜索单词的集合变得更大了，所以在最终的结果里会包含一些在第一阶段未被找到，但又与它们有着密切联系的行。

要想完成这种搜索，需要在搜索字符串的后面加上 WITH QUERY EXPANSION。下面这个示例可用于说明整个过程。第一条查询命令展示的是一次自然语言搜索。第二条查询命令展示的是一次查询扩展搜索。这次查询多找到了一行记录，而它并没有包含原始搜索字符串里的那些单词。之所以能找到该行，是因为它包含单词“candle”，而这个单词存在于那个自然语言搜索所找到的行里。

```
mysql> SELECT * FROM apothegm
```

```
-> WHERE MATCH(attribution, phrase)
```

```
-> AGAINST('bell book');
```

-----+-----	
attribution	phrase
-----+-----	
Miguel de Cervantes	Bell, book, and candle
Alexander Graham Bell	Mr.Watson, come here.I want you!
-----+-----	

```
mysql> SELECT * FROM apothegm
```

```
-> WHERE MATCH(attribution, phrase)
```

```
-> AGAINST('bell book' WITH QUERY EXPANSION);
```

-----+-----	
attribution	phrase
-----+-----	
Miguel de Cervantes	Bell, book, and candle
Alexander Graham Bell	Mr.Watson, come here.I want you!
Robert Burton	I light my candle from their torches.
-----+-----	

2.14.4 配置全文搜索引擎

全文搜索有几个参数是可配置的，并且可以通过设置系统变量的办法进行修改。变量`ft_min_word_len`和`ft_max_word_len`用于确定FULLTEXT索引里的那些单词所具有的最小长度和最大长度。在创建FULLTEXT索引时，会忽略掉其长度超出这两个参数所确定的范围的那些单词。默认的最小值和最大值分别是4和84。

假设想要把最小单词长度从4改成3，那么可按以下步骤进行操作。

(1) 把`ft_min_word_len`变量设置为3，重启服务器。如果想让这个设置在每次服务器重启时都能生效，那么最好的办法是把这个设置放到某个选项文件里，如`/etc/my.cnf`文件：

```
[mysqld]
```

```
ft_min_word_len=3
```

(2) 对于那些已经建立了 FULLTEXT 索引的表，必须重建这些索引。你可以先删除这些索引，然后再重建它们，但更为简易有效的办法是执行一次快速修复操作：

```
REPAIR TABLE tbl_name QUICK;
```

(3) 所有在更改参数后新创建的FULLTEXT索引，将自动使用这个新值。

关于设置系统变量的更多信息请参考附录D。关于使用选项文件的更多信息请参考附录F。

说明

如果想使用`myisamchk`程序为某个包含有FULLTEXT索引的表重建索引，那么可以参考附录F，其中在对`myisamchk`进行描述时包含了一些与FULLTEXT索引有关的注意事项。

[1].也常被称为零号平面（Plane 0），它是Unicode中的一个编码区段，编码范围包括U+0000到U+FFFF。——译者注

第3章 数据类型

事实上，在MySQL里做的所有事情或多或少都会涉及数据，因为从定义即可看出，数据库管理系统的目的就是管理数据。即使像SELECT 1这样的简单语句，也涉及了表达式的计算，它会产生一个整数值。

MySQL里的每一个数值都拥有一个类型。例如，37.4是一个数字，而'abc'是一个字符串。有些时候，数据的类型比较直观。例如，当你用CREATE TABLE语句来创建表时，就必须为这个表里的每一个列定义一种类型：

```
CREATE TABLE mytbl

(
    int_col    INT,           # 整数值列
    str_col    CHAR(20),      # 字符串值列
    date_col   DATE           # 日期值列
);
```

而其他时候，数据的类型则比较隐蔽。例如，有时你会想在表达式里引用文本值，将值传递给某个函数或者使用某个函数的返回值。下面的INSERT语句能够完成所有这些事情：

```
INSERT INTO mytbl (int_col, str_col, date_col)
```

```
VALUES(14, CONCAT('a','b'), 20130815);
```

这条语句完成了以下几个操作，每一个操作都会涉及数据类型。

它会把整数14赋给整型列int_col。

它会把字符串'a'和'b'传递给字符串拼接函数CONCAT()；然后，将其返回的字符串值'ab'赋给字符串列str_col。

它会把整数值 20130815 赋值给日期列 date_col。该赋值操作会出现类型不匹配的问题，但由于整数值可以合理地解释为日期值，因此MySQL会执行自动类型转换，将整数20130815转换为日期'2013-08-15'。

要想高效地使用MySQL，你必须要透彻理解“MySQL是如何处理数据的”。本章将描述你会用到的数据值类型。

MySQL能表示的一般类别的数据值，包括NULL值。

MySQL 专门为表的列提供的数据类型，以及描述每种数据类型的属性。有些 MySQL数据类型很常见，如BLOB字符串类型。但你需要对某些类型的特殊行为有所了解，否则可能遭遇意外。具有特殊行为的数据类型有：TIMESTAMP和拥有AUTO_INCREMENT属性的整数类型。

本章还将讨论使用这些类型时可能会遇到的一些问题。

服务器的SQL模式对坏数据值的处理方式有何影响，它又是如何使用“严格”模式拒绝坏数据值的。

如何生成和处理序列。

MySQL的表达式计算规则是什么。在表达式里，你可以广泛地使用各种运算符和函数来检索、显示和操作数据。表达式计算包含了很多约束类型转换的规则。当某种类型值用于与其类型不相同的环境时，这些规则就会发挥作用。理解类型转换什么时候会发生，以及它是如何工作的，这是件很重要的事情，因为有些类型转换毫无意义，而且会导致不合理的结果值。例如，当把字符串'13'赋给一个整型列时，MySQL 会自动将其转换为整数值 13。但是，当你把字符串'abc'赋给一个整型列时，MySQL的转换结果是 0 值（在严格模式下会产生一个错误），因为'abc'根本就不是一个数字。更糟的是，如果对MySQL的类型转换规则不熟悉，则可能会造成非常严重的破坏，例如，当你只是想对特定的几行进行调整时，有可能修改或删除了表里的所有行。

如何为表中的列选择适合的数据类型。在创建表时，以及当有好多个类型都满足你所要存储的那些值的类型时，该如何选择适合的类型，这是很值

得了解的事情。

作为对本章关于MySQL的数据类型、运算符和函数等内容的补充，附录B和附录C提供了更多的参考信息。

本章的示例大量使用了CREATE TABLE语句和ALTER TABLE语句来创建和修改表。这些语句在第1章和第2章都讨论过。如有必要，也可以参考附录E。

在某些情况下，数据处理取决于两个方面：默认值是如何定义的；当前的SQL模式。关于SQL模式设置的详细信息请参考2.1节。在本章中，3.2.3节讲到了默认值的处理，3.3节则讲到了严格模式，以及坏数据的处理规则。

3.1 数据值类别

MySQL支持多种常规类别的数据值。其中包括数值、字符串值、日期/时间这样的时态值、空间值，以及NULL值。

3.1.1 数值

数值是指诸如48、193.62或-2.378E12之类的值。MySQL所能识别的数字包括整数（无小数位）、定点数或浮点数（可以有小数部分），以及位域值（bit-field value）。

当你指定某个数字时，请不要将逗号作为分隔符包括在内。例如，12345678.90是合法的，但12,345,678.90是非法的。

3.1.1.1 精确值数和近似值数

MySQL支持对精确值数的精确运算，以及对近似值数的近似运算。

精确值数在使用时与指定值是完全一致的。精确值包括整数（如 0、14、-382）和带小数点的数（如0.0、38.5、-18.247）。

整数可以表示成十进制或十六进制两种格式。在十进制格式下，每个整数由一个不包含小数点的数字序列构成。十六进制值会被默认为字符串，但在进行数值运算的时候，十六进制常量会被视为64位的整数。例如，0x10即为十进制数16。关于十六进制值的语法描述请参考3.1.2节。

带小数部分的精确值由 3个部分组成：一个数字序列、一个小数点和另一个数字序列。小数点前后的数字序列可以有一个为空，但不能同时为空。

近似值是采用科学计数法表示的浮点数，它们带有一个底数和一个指数。具体表示方法为：在整数或浮点数的后面紧跟着字母e或E；接着是一个正号（+）或负号（-）；然后是一个表示指数的整数。底数和指数的正负号可以任意组合，如1.58E5、-1.58E5、1.58E-5、-1.58E-5。

十六进制数不能用科学计数法来表示，这是因为指数部分的开始字母“e”，也是一个合法的十六进制数字，这会产生二义性。你可以在任何一数值的前面加上一个正号（+）或负号（-），以此表明它是一个正值或负值。

用精确值计算得到的结果也是精确的，只要没有超出那些数值的精度范围，就不会丢失精度。例如，你不能原封不动地把1.23456插到只允许两位小数的列里。近似值的计算则是近似的，而且会有舍入误差。

在计算表达式时，MySQL会根据以下规则来决定是使用精确计算还是使用近似计算。

只要表达式里有近似值，那么它便会被当作浮点（近似）表达式来进行计算。

如果表达式只包含整数精确值，那么它会以BIGINT（64位）精度来进行计算。

如果表达式只包含精确值，但其中有的值带有小数部分，那么它会以具有 65 位精度的DECIMAL算法来进行计算。

在表达式里，如果存在字符串必须转换成一个数才能进行计算的情况，那么该字符串会被转换成一个双精度浮点值。因此，该表达式会按照前面的规则进行近似计算。

3.1.1.2 位域值

位域值可以写成 b'val' 或 0bval，其中，val 由一个或多个二进制数字（0或1）构成。例如，b'1001' 和 0b1001 即为十进制数9。

在结果集里，BIT 值会被显示为一个二进制串，不过其输出格式并不太好。让它加上零或者使用CAST() 函数，可以将其转换为一个整数：

```
mysql> SELECT b'1001' + 0, CAST(b'1001' AS UNSIGNED);
```

-----+-----	
b'1001' + 0 CAST(b'1001' AS UNSIGNED)	
-----+-----	
9	9
-----+-----	

3.1.2 字符串值

字符串指的是这样一些值，如'Madison, Wisconsin'、'patient shows improvement'或'12345'（虽然它看起来像一个数值，但实际上不是）。字符串两端的引号既可以是单引号，也可以是双引号。请尽量使用单引号，具体原因有两个。

SQL 语言标准规定使用单引号，因此使用单引号字符串的语句，能够更好地移植到其他数据库引擎。

如果启用了SQL模式ANSI_QUOTES，那么MySQL会将双引号处理成将标识符引起来的符号，而不会把它只当成将字符串引起来的符号。也就是说，双引号里的值必须是数据库名或表名。请看下面这条语句：

```
SELECT "last_name" from president;
```

若启用了ANSI_QUOTES模式，那么此语句将从president表里选取last_name列的值。

若未启用ANSI_QUOTES模式，那么此语句将为该表的每行选取一次字符串"last_name"。在后面的示例里，会假设未启用ANSI_QUOTES模式，即双引号是一个字符串引号符。

MySQL能够识别出字符串里用来代表特殊字符的转义序列，如表3-1所示。转义序列是以一个反斜线字符（\）开始，表示将要临时脱离常规的字符解释规则。请大家注意，NUL字节与SQL语言中的NULL值是不同的：NUL代表的是零值字节，而NULL值代表的是“没有值”。

表3-1 字符串转义序列

转义序列	含 义
\0	NUL (零值字节)
\'	单引号
\"	双引号
\b	退格符
\n	换行符
\r	回车符
\t	制表符
\\	反斜线
\Z	Ctrl+Z (Windows系统中的EOF字符)

此表里的转义序列是区分大小写的。所有未列在此表里的字符，即使其前面加上一个反斜线字符，也仍然会被解释为该字符本身。例如，\t是一个制表符，而\T是一个普通的'T'字符。

从表 3-1 可以看出，你可以用反斜线序列来将单引号或双引号转义，但实际上还有几种方法可以在字符串内嵌入引号。

如果嵌入的引号与字符串的引号相同，则重复两次即可：

'I can"t'

"He said, ""I told you so.""

如果嵌入的引号与字符串的引号不相同，那么不用重复即可嵌入：

"I can't"

'He said, "I told you so."'

对嵌入的引号使用反斜线进行转义；此办法不受字符串引号的限制：

'I can\'t'

"I can\'t"

```
"He said, \"I told you so.\""
```

```
'He said, \"I told you so.\"'
```

如果需要取消反斜线字符的特殊含义，并把它当作一个普通字符，那么请启用SQL的NO_BACKSLASH_ESCAPES模式。

在使用引号书写字符串值时，还有两种形式的十六进制记法可供选择。第一种是使用标准的SQL 记法X'val'，其中val由多对十六进制数字（包括“0”～“9”和“a”～“f”）组成。例如，X'0a'即为十进制的10，而X'ffff'即为十进制的65535。前导字符“X”和字符串里的十六进制数字字符（“a”～“f”）都不区分大小写：

```
mysql> SELECT X'4A', x'4a';
```

-----+-----	
X' 4A'	x' 4a'
-----+-----	
J	J
-----+-----	

在字符串上下文中，每两个十六进制数字会被解释为一个8 位数字字节值，其取值范围是0～255；而其结果会被当作一个字符串。在数字上下文中，十六进制常量会被当成一个数。下列语句展示了十六进制常量在两种上下文中的解释情况：

```
mysql> SELECT X'61626364', X'61626364'+0;
```

-----+-----	
X' 61626364'	X' 61626364'+0
-----+-----	
abcd	1633837924
-----+-----	

第二种记法是以“0x”开头，后面跟着一个或多个十六进制数字。前缀“0x”需要区分大小写。“0x0a”和“0x0A”是合法的十六进制数值；而“0X0a”和“0X0A”则是非法的。

与X'val'记法的情况类似，0x值会被默认解释为字符串，但在数值上下文中，则可能被用作数字：

```
mysql> SELECT 0x61626364, 0x61626364+0;
```

-----+-----	
0x61626364	0x61626364+0
-----+-----	
abcd	1633837924
-----+-----	

X'val'记法要求构成val的数字个数为偶数。像X'a'这样的值是非法的。如果用0x记法写的十六进制值只有奇数个十六进制数字，那么 MySQL 将会在其前面增加一个字符“0”。例如，0xa会被视为0x0a。

3.1.2.1 字符串类型与字符集支持

字符串值一般可以分为两类，即二进制串和非二进制串。

二进制串是一组字节序列。对这些字节的解释不牵涉任何字符集概念。二进制串没有特殊的比较或排序属性。比较操作是基于各字节的数值逐个字节实现的。所有字节都有意义，甚至包括结尾的空格。

非二进制串是一个字符序列。每一个非二进制串都与字符集相关，字符集决定了：哪些字符可以用；MySQL 会如何解释字符串的内容。每个字符集都有一种或多种排序规则。字符串所使用的排序规则决定了字符在字符集里的先后顺序，而这会对比较操作产生影响。默认的字符集和排序规则分别为 latin1 和 latin1_swedish_ci。

非二进制串里的尾部空格不会参与比较，但 TEXT 类型除外。由于基于索引的比较会在尾部补足一些空格，因此当你试图往一个具有唯一性的 TEXT 索引里插入某个值时，如果它与另一个已有值只是在尾部空格的个数上不同，那么会导致一个“关键字重复”错误。

字符单位在占用存储空间方面存在差异。对于像 latin1 这样的单字节字符集，每个字符只占用一个字节空间；而对于那些多字节字符集，有的是部分，有的甚至是全部字符都需要占用一个字节以上的空间。例如，MySQL 里的 Unicode 字符集是多字节的。ucs2 是双字节字符集，其每个字符需要两个字节来存储。utf8 是变长多字节字符集，其每个字符需要占用 1~3 个字节空间。有的 Unicode 字符集还会占到每字符 4 个字节的空间。更多详细信息请参考 2.4.3 节。

下面两条语句可用来查看服务器上提供有哪些字符集，以及它们的排序规则是什么：

```
mysql> SHOW CHARACTER SET;
```

Charset	Description	Default collation	Maxlen
big5	Big5 Traditional Chinese	big5_chinese_ci	2
dec8	DEC West European	dec8_swedish_ci	1
cp850	DOS West European	cp850_general_ci	1
hp8	HP West European	hp8_english_ci	1
koi8r	KOI8-R Relcom Russian	koi8r_general_ci	1
latin1	cp1252 West European	latin1_swedish_ci	1
...			
utf8	UTF-8 Unicode	utf8_general_ci	3
ucs2	UCS-2 Unicode	ucs2_general_ci	2
...			

```
mysql> SHOW COLLATION;
```

Collation	Charset	Id	Default	Compiled	Sortlen
big5_chinese_ci	big5	1	Yes	Yes	1
big5_bin	big5	84		Yes	1
..</p>					
latin1_german1_ci	latin1	5		Yes	1
latin1_swedish_ci	latin1	8	Yes	Yes	1
latin1_danish_ci	latin1	15		Yes	1
latin1_german2_ci	latin1	31		Yes	2
latin1_bin	latin1	47		Yes	1
latin1_general_ci	latin1	48		Yes	1
latin1_general_cs	latin1	49		Yes	1
latin1_spanish_ci	latin1	94		Yes	1

...

从SHOW COLLATION语句的输出内容里可以看到，每种排序规则都捆绑在某个特定的字符集上，而每个给定的字符集可以有多种排序规则。排序规则名由字符集名、语言名和一个附加的后缀构成。例如，utf8_icelandic_ci是Unicode字符集utf8的一种排序规则，其比较操作遵从的是冰岛

（icelandic）排序规则，并且被比较字符都要区分大小写。排序规则名里的后缀具有下列含义。

_ci表明排序规则不区分大小写。

_cs表明排序规则要区分大小写。

_bin 表明这是一种二进制排序规则。也就是说，比较操作是基于数字字符编码值进行的，与语言无关。因此，_bin的排序规则名里不包括语言名，如 latin1_bin和utf8_bin。

二进制串和非二进制串有着不同的排序特性。

二进制串是逐字节进行比较的，其结果只取决于每个字节的数值大小。因此，二进制串看起来像是区分大小写的（如'abc' <> 'ABC'），但这只是因为同一个字符的大写和小写形式有着不同的字节数值而已。二进制串其实没有大小写的概念。区分大小写是排序规则的一项功能，只适用于字符（非二进制）字符串。

非二进制串是按字符进行比较的，每一个字符的相对值取决于当前所用字符集的排序规则。大多数排序规则都把同一个字符的大小写形式设定为相同的排序值，所以非二进制串的比较一般情况下不区分大小写。当然，这一定论不适用于那些区分大小写的排序规则或者是二进制排序规则。

因为排序规则应用于比较和排序，所以它们会对以下操作造成影响。

比较运算符：<、<=、=、< >、>=、>和LIKE。

排序：ORDER BY、MIN()和MAX()。

分组：GROUP BY和DISTINCT。

你可以使用函数CHARSET()或COLLATION()来确定某个字符串的字符集和排序规则。

引号里的字符串会根据服务器的当前设置来解释。默认的字符集和排序规则分别为 latin1和latin1_swedish-ci。不过，当你将环境变量LANG或LC_ALL设置为某个指定语言区域时，mysql和其他的客户程序可以检测到它们，并进行相应的调整。更多详细信息请参考3.1.2.2节。

默认情况下，MySQL把十六进制常量当作二进制串对待：

```
mysql> SELECT CHARSET(X'0123'), COLLATION(X'0123');
```

```
-----+-----  
CHARSET(X' 0123' ) | COLLATION(X' 0123' )  
-----+-----  
binary             | binary
```

有两种记法约定可用于将某个字符串常量强制解释为某种指定的字符集。第一种记法如下所示：

```
_charset str
```

其中，charset 是某种被支持的字符集名。_charset 记法称为字符集引导符（character set introducer）。紧跟引导符后面的字符串，既可以用引号形式，也可以是一个十六进制值。下面示例演示的是如何将字符串解释为 latin2或utf8字符集：

```
_latin2 'abc'
```

```
_latin2 X'616263'
```

```
_latin2 0x616263
```

```
_utf8 'def'
```

```
_utf8 X'646566'
```

```
_utf8 0x646566
```

对于引号里的字符串，字符集引导符和字符串之间的空白是可选的。对于以十六进制值形式给出的字符串，这个空白是必需的。

第二种记法是N'str'，它等价于_utf8'str'。请注意，在N（不区分大小写）的后面必须紧跟一个引号形式的字符串，其间不能有任何空白。

引导符记法可用于引号形式的字符串或十六进制常量，但不能用于字符串表达式或列值。不过，可以使用CONVERT()函数将任意字符串转换为另一种指字符集形式的字符串：

```
CONVERT(str USING charset);
```

引导符和CONVERT()函数是不一样的。引导符只会改变对字符串的解释，并不会改变字符串的值（不过，对于多字节字符集，当该字符串包含的字节数不足时，它可能会在其尾部增加一些空格）。而 CONVERT()函数则是以某个字符串作为输入参数，并根据给定的字符集生成一个新的字符串。通过下面的示例可以看出字符集引导符和CONVERT()函数之间的区别，它们都使用了ucs2双字节字符集：

```
mysql> SET @s1 = _ucs2 'ABCD';
```

```
mysql> SET @s2 = CONVERT('ABCD' USING ucs2);
```

假设当前字符集为latin1（一种单字节字符集）。第一条语句将把字符串'ABCD'中的每一对字符解释为一个双字节ucs2字符，结果即为包含两个字

符的ucs2字符串。第二条语句将把字符串'ABCD'中的每个字符转换为相应的ucs2字符，结果为包含4个字符的ucs2字符串。

字符串的“长度”如何定义？这要视具体情况而定。如果用CHAR_LENGTH()函数来测量，那么长度的单位是字符。如果用LENGTH()函数来测量，那么长度的单位是字节。对包含有多字节字符的字符串来说，这两个值是不一样的：

```
mysql> SELECT CHAR_LENGTH(@s1), LENGTH(@s1),  
CHAR_LENGTH(@s2), LENGTH(@s2);
```

-----+-----+-----+-----			
CHAR_LENGTH(@s1)	LENGTH(@s1)	CHAR_LENGTH(@s2)	LENGTH(@s2)
-----+-----+-----+-----			
2	4	4	8
-----+-----+-----+-----			

这里还有一个更容易让人迷糊的问题，即二进制串与使用某种二进制排序规则的非二进制串是不一样的。

二进制串没有字符集的概念。它会被解释成字节，并且比较的是单字节的数字代码。

使用了二进制排序规则的非二进制串，会被解释成字符，并且比较的是它们的数字字符值，这种值通常是基于每个字符多个字节算出的。

通过下面这个示例，我们可以看到二进制串和非二进制串在大小写方面的区别。它会先创建一个二进制串和一个使用了某种二进制排序规则的非二进制串，然后把它们传递给UPPER()函数：

```
mysql> SET @s1 = BINARY 'abcd';
```

```
mysql> SET @s2 = _latin1 'abcd' COLLATE latin1_bin;
```

```
mysql> SELECT UPPER(@s1), UPPER(@s2);
```

UPPER(@s1)	UPPER(@s2)
abcd	ABCD

UPPER() 函数为何没有把二进制串转换为大写呢？这是因为二进制串没有任何字符集的概念，因而无从得知哪些字节值对应着大写或小写字符。如果需要把某个二进制串传递给诸如UPPER() 或LOWER() 之类的函数，那么必须先把它转换为非二进制串：

```
mysql> SELECT @s1, UPPER(CONVERT(@s1 USING latin1));
```

@s1	UPPER(CONVERT(@s1 USING latin1))
abcd	ABCD

3.1.2.2 字符集相关的系统变量

MySQL服务器有几个系统变量，它们涉及字符集支持的各个方面。其中大部分变量与字符集有关，其余的与排序规则有关。每个排序规则变量都有一个相应的字符集变量与之相连。

有些字符集变量表示的是服务器或当前数据库的属性。

character_set_system表示的是用于存储标识符的字符集。这始终为utf8。

character_set_server和collation_server分别表示服务器的默认字符集和排序规则。

character_set_database和collation_database 分别表示默认数据库的字符集和排序规则。这两个变量都是只读的，每当你选择一个默认数据库时，服务器都会自动将它们设置好。如果没有默认数据库，那么它们会被设置为服务器的默认字符集和排序规则。当你创建某个表时，如果没有明确地为其指定字符集和排序规则，那么这些变量便会发挥作用。这时，表的默认设置将会沿用数据库的默认设置。

其他的字符集变量将影响客户和服务器之间通信。

`character_set_client`表示的是客户端向服务器发送SQL语句时使用的字符集。

`character_set_results` 表示的是服务器向客户端返回结果时使用的字符集。这里所说的“结果”包括数据值和诸如列名之类的元数据。

`character_set_connection` 是服务器使用的变量。当服务器接收到来自客户端的语句字符串时，会将该字符串从 `character_set_client` 转换为 `character_set_connection`，并使用后者的字符集来处理该语句。（这里有个例外，即对于语句里的所有文字串，如果它由字符集引导符引导，那么它会使用该字符集引导符所表示的字符集来进行解释。）

`collation_connection`适用于语句字符串里的两个文字串值之间的比较。

`character_set_filesystem`表示的是文件系统字符集。MySQL会用它来解释在SQL语句（如 `LOAD DATA` 语句）里代表文件名的文字串。在该文件被打开之前，其文件名字符串会从 `character_set_client` 转换为 `character_set_filesystem`。此变量的默认值为 `binary`（即不进行转换）。

很可能你会发现，绝大多数字符集和排序规则变量都被默认设置成了相同的值。例如，下面的输出即表明：客户端和服务器之间使用了 `latin1` 字符集来进行通信。

```
mysql> SHOW VARIABLES LIKE 'character\_set\__%';
```

Variable_name	Value
character_set_client	latin1
character_set_connection	latin1
character_set_database	latin1
character_set_filesystem	binary
character_set_results	latin1
character_set_server	latin1
character_set_system	utf8

```
mysql> SHOW VARIABLES LIKE 'collation\_%';
```

Variable_name	Value
collation_connection	latin1_swedish_ci
collation_database	latin1_swedish_ci
collation_server	latin1_swedish_ci

如果某个客户端想使用另一种字符集来与服务器进行通信，那么它需要修改与通信有关的变量。例如，想要使用utf8字符集，则需要更改3个变量：

```
mysql> SET character_set_client = utf8;
```

```
mysql> SET character_set_results = utf8;
```

```
mysql> SET character_set_connection = utf8;
```

不过，使用SET NAMES语句可以更方便地达到同样的效果。下面这条语句与上面的3条SET语句是等效的：

```
mysql> SET NAMES 'utf8';
```

在设置与通信有关的字符集时会有一个限制，即不能使用ucs2、utf16、utf16le或utf32。

有许多客户端程序都支持--default-character-set选项，此选项可以获得与SET NAMES语句相同的效果，达到将你所用通信字符集通知服务器的目的。

对于像mysql和mysqladmin这种标准的MySQL客户端程序，甚至还有一种更简单的方法。将环境变量LANG或LC_ALL设置成指定某一个语言区域，然后这些客户端会检测这些变量，并进行相应的调整。例如，如果将LC_ALL设置为en_US.UTF-8，那么文字量的默认字符集和排序规则会受到类似下面那样的影响：

```
mysql> SELECT CHARSET('abcd'), COLLATION('abcd');
```

```
-----+-----  
CHARSET('abcd') | COLLATION('abcd')  
-----+-----  
utf8           | utf8_general_ci |<
```

对于那些成对出现的变量（一个字符集变量和一个排序规则变量），它们之间的影响关系如下。

在设置字符集变量的同时，也会将相关的排序规则变量设置为该新字符集默认的排序规则。

在设置排序规则变量的同时，也会将相关的字符集变量设置为该新排序规则名的第一部分所暗示的那个字符集。

例如，当把character_set_connection变量设置为utf8时，会将collation_connection设置为utf8_general_ci。将collation_connection变量设置为latin1_spanish_ci时，会将character_set_connection设置为latin1。

3.1.3 时态（日期/时间）值

时态值包括日期值或时间值，如'2012-06-17'或'12:30:43'。MySQL也支持将日期和时间合并在一起的值，如'2012-06-17 12:30:43'。需要特别注意

的是，MySQL 是按“年-月-日”的顺序来表示日期的，并且输入值也必须是这样的顺序。虽然这是标准的SQL格式（也叫“ISO 8601”格式），但还是会有很多MySQL初学者对此感到不习惯。可以利用DATE_FORMAT()函数，按任意方式来显示日期值，但默认显示格式是先显示年。对于其他格式的输入值，需要使用STR_TO_DATE()函数来进行转换。具体示例请参考3.2.6节。

对于组合后的日期时间值，允许在日期和时间之间加一个字符“T”，但不能用空格，如'2008-12-31T12:00:00'。

时间值或日期时间组合值的语法还支持在时间后面紧跟一个小数形式的秒，其中包含有一个小数点和多达6位数字（微秒）的精度，如'12:30:15.000045'或'2008-06-15 10:30:12.5'。

3.1.4 空间值

MySQL支持空间值，不过仅限于InnoDB、MyISAM、NDB和ARCHIVE这几种引擎。这一功能让我们可以直接表示像点、线和多边形这样的值。这些类型都是每一个OpenGIS规范的实现。关于这些规范的更多信息，可以在“Open Geospatial Consortium”的官网（<http://www.opengeospatial.org>）上获得。例如，有一个X和Y坐标的值为（10，20）的点，下面语句利用其文本表示形式，创建了一个POINT类型，并把结果赋值给了一个自定义变量：

```
SET @pt = POINTFROMTEXT('POINT(10 20)');
```

本书只涉及了很少的空间值应用。关于这些类型的更多信息请参考MySQL参考手册。

3.1.5 布尔值

在表达式里，零会被当成假，任何非零、非NULL的值会被当成真。

布尔常量TRUE和FALSE将分别被当作1和0。它们不区分大小写。

3.1.6 NULL值

NULL是一种“没有类型的”值。它通常用来表示“无值”、“未知值”、“缺失值”、“超界”、“不适用”和“不在其中”等。可以将NULL值插到表中，从表里检索它们，以及测试某个值是否为NULL。但不能对NULL进行算术运算。如果用它进行算术运算，那么结果将是NULL。此外，有许多函数，在你使用NULL或非法参数调用它们时，也会返回NULL。

在书写关键字NULL时，不需要加引号，也不用区分大小写。MySQL还会把单独的\N（区分大小写）当作NULL：


```
mysql> SELECT \N, ISNULL(\N);
```

-----+-----	
NULL	ISNULL(\N)
-----+-----	
NULL	1
-----+-----	

3.2 MySQL数据类型

每一个数据库里的表都由一个或者多个列构成。在用CREATE TABLE语句来创建表时，必须为它的每一个列定义一种类型。列的数据类型要比一般的分类（如“数”或“字符串”）更具体。它精确刻画了列的类型，如SMALLINT 或VARCHAR(32)。同时，它也决定了MySQL将如何对待那些值。例如，你既可以把数值存储在一个数字列里，也可以把它们存储在一个字符串列里，不过MySQL会根据这些值的实际存储方式来区别对待。每种数据类型都有以下几个特点。

它可以表示何种类型的值。

这种类型的值要占用多少存储空间。

值的长度是固定的（即该类型的每个值都将占用同样数量的存储空间），还是可变的（不同的值会占用不同数量的存储空间）。

MySQL会如何比较这种类型的值，如何对它进行排序。

是否可以对这种类型进行索引，如何索引。

接下来的讨论将概述 MySQL的各种数据类型，然后详细描述它们的定义语法和每种类型的特点，如取值范围和存储要求。我们将以各种数据类型在CREATE TABLE语句里的用法为例，展示其规范。其中，方括号([])表示可选信息。例如，语法MEDIUMINT [(M)]表示的是该类型的最大显示宽度(M)是可选的。反之，对于 VARCHAR (M)，这种没有方括号的，表示(M)必不可少。

3.2.1 数据类型概述

MySQL的数字类型包括整数、定点数、浮点数和位值，如表3-2所示。除BIT之外，其他的数类型可以带有正负号，也可以不带正负号。有一种特殊的

属性，它可以让列自动生成一组有序的整数或浮点值。在你需要一组唯一标识编号的时候，可以使用它。

表3-2 数字数据类型

类型名称	含 义
TINYINT	非常小的整数
SMALLINT	小整数
MEDIUMINT	中等大小的整数
INT	标准的整数
BIGINT	大整数
DECIMAL	定点数
FLOAT	单精度浮点数
DOUBLE	双精度浮点数
BIT	位域

表3-3显示的是MySQL的字符串数据类型。字符串可以容纳任何内容，甚至能容纳那些用于表示图像和声音的二进制数据。字符串可以按照大小写进行比较。此外，还可以对字符串进行模式匹配。（事实上，数字类型也支持模式匹配，只不过字符串类型的模式匹配更为常见一点。）

表3-3 字符串数据类型

类型名称	含 义
CHAR	固定长度的非二进制（字符）字符串
VARCHAR	可变长度的非二进制串
BINARY	固定长度的二进制串

续表

类型名称	含 义
VARBINARY	可变长度的二进制串
TINYBLOB	非常小型的BLOB（二进制大对象）
BLOB	小型BLOB
MEDIUMBLOB	中型BLOB
LONGBLOB	大型BLOB
TINYTEXT	非常小型的非二进制串
TEXT	小型非二进制串
MEDIUMTEXT	中等大小的非二进制串
LONGTEXT	大型非二进制串
ENUM	枚举集合。每个列的取值将是这个枚举集合中的某一个元素
SET	集合。每个列的取值可以为零个或者多个集合元素

表3-4显示的是MySQL的时态类型，其中的CC、YY、MM、DD、hh、mm和ss分别代表世纪、年、月、日、时、分和秒。对于时态值，MySQL提供的类型有：日期与时间（合并或分开表示）、时间戳（一种专门用来记载某个行最近一次修改时间的类型）。此外，在不需要完整日期时，还可以使用专用于表示年的类型。

表3-4 时态数据类型

类型名称	含 义
DATE	日期值，格式为'CCYY-MM-DD'
TIME	时间值，格式为'hh:mm:ss'
DATETIME	日期加时间值，格式为'CCYY-MM-DD hh:mm:ss'
TIMESTAMP	时间戳值，格式为'CCYY-MM-DD hh:mm:ss'
YEAR	年份值，格式为CCYY或YY

3.2.2 表定义里的特殊列类型

想要创建表，可以调用CREATE TABLE语句，并列出表中所有的列。下面这个示例会创建一个名为mytbl的表，该表由3个列构成，分别为f、c和i：

```
CREATE TABLE mytbl  
  
(  
  
f FLOAT(10,4),  
  
c CHAR(15) NOT NULL DEFAULT 'none',  
  
i TINYINT UNSIGNED NULL  
  
);
```

每个列都有一个名字、一个类型，以及一些可能有的可选属性。列定义的语法如下：

```
col_name col_type [type_attrs] [general_attrs]
```

列名，col_name，总是放在列定义的最前面，并且必须是合法的标识符。关于标识符的命名规则，可参考2.2节。简单来讲，列标识符的长度可以多达64个字符，并且可以包含的字符包括英文字母、数字、下划线（_）、美元符号（\$），以及Unicode扩展字符。关键字（如SELECT、DELETE和CREATE）通常都是保留字，不得用作列名。在标识符内，允许包含其他字符，或者把保留字用作标识符。但是每次在引用它的时候，都会稍微麻烦一点，即使用引号括起来。使用反引号（`）可以将标识符引起来。如果启用了SQL的ANSI_QUOTES模式，则可以使用双引号（"）来代替。

col_type表示列类型，即表明这一列可以用来容纳何种类型的值。有些类型说明符会表明存储在列里的值所允许的最大长度。而对于其他的类型说明符，所允许的长度则隐含在它们的名称里。例如，CHAR(10)显式地指定了一个具有 10个字符的长度；而 TINYTEXT 则隐含表示列的最大长度为 255个字符。有些类型说明符允许你指定一个最大显示宽度（用于显示值的字符有多少个）。对于定点类型和浮点类型，还可以指定有效位数和小数位。

在列的数据类型后面，除了指定多个通用属性外，还可以指定一些类型特有的可选属性。这些属性的作用是对该类型做进一步的修饰和限定，它们会对 MySQL 处理这个列的方式产生影响。

所允许的类型特有属性取决于具体的数据类型。例如，只有数字类型才有UNSIGNED和ZEROFILL属性；只有非二进制串类型才有CHARACTER SET和

COLLATE属性。

通用属性可用于任何一种数据类型，除少数例外。例如，可以指定 `NULL` 或 `NOT NULL` 来表明某列是否允许 `NULL` 值。对于大部分的数据类型，都可以通过 `DEFAULT` 子句来定义列的默认值。有关默认值的处理方式请参考3.2.3节。

如果存在多个列属性，那么它们的先后顺序必须遵从一定的规则。一般情况下，比较安全的做法是，将数据类型特有的属性（如 `UNSIGNED` 或 `ZEROFILL`）放在通用属性（如 `NULL` 和 `NOT NULL`）的前面。

3.2.3 指定列的默认值

除了 `BLOB` 与 `TEXT` 类型、空间类型，或者具有 `AUTO_INCREMENT` 属性的列以外，你还可以指定 `DEFAULT def_value` 子句，以此来表明：当在创建新的行时，如果没有显式地指定某个值，那么该列将被赋值为默认值 `def_value`。除了 `TIMESTAMP` 列和 `DATETIME` 列有限制以外，这里的默认值 `def_value` 必须是一个常量。它不能为表达式，也不能引用其他列。

如果某个列定义没有显式包含 `DEFAULT` 子句，并且该列允许为 `NULL` 值，那么其默认值为 `NULL`。否则，如果该列不允许为 `NULL` 值，并且在定义里没有显示包含 `DEFAULT` 子句，那么在创建它时也不会带有 `DEFAULT` 子句。也就是说，它没有默认值。在往表里插入新的行时，如果没有为列指定具体值，这会影响MySQL对列的处理。

如果没有启用SQL的严格模式，那么该列将被设置成其数据类型的隐含默认值。（各种隐含默认值将在稍后讨论。）

在启用SQL的严格模式之后，如果表是事务性的，就会出现错误。这条语句会中止执行，然后回滚。对于非事务性的表，如果这行是该语句插入的第一个行，那么会出现一个错误，并且该语句会中止执行。如果插入的不是第一行，那么你可以选择让语句中止执行，或者选择把这列设置为它的隐含默认值，同时发一条警告消息。具体的选择需要参考哪一种严格模式在起作用。更多相关细节请参考3.3节。更多与事务有关的信息请参考2.12节。列的隐含默认值取决于它的数据类型。

对于数字列（不包括那些具有 `AUTO_INCREMENT` 属性的列），其默认值为 0。对于 `AUTO_INCREMENT` 列，其默认值是下一个列序号。

对于大多数时态类型列，其默认值为该类型的“零”值（如 `DATE` 类型的“零”值是 `'0000-00-00'`）。对于 `TIMESTAMP` 列（以及MySQL 5.6里的 `DATETIME` 列），其自动初始化规则比较特殊。更多相关信息请参考3.2.6.6节。

对于字符串类型（不包括 ENUM 类型）列，其默认值为空串。对于 ENUM 列，其默认值为枚举集里的第一个元素。对于 SET 列，如果不允许包含 NULL 值，那么默认值将是一个空集，不过它等价于空串。

使用 SHOW CREATE TABLE tbl_name 语句，可以查看到表里的哪些列带有 DEFAULT 子句，以及它们的默认值是什么。

3.2.4 数字数据类型

MySQL 的数字数据类型分为以下 3 大类。

精确值类型，它包括整数类型和 DECIMAL。整数类型用来存放没有小数部分的数字，如 43、-3、0 或 -798432。只要是能用整数来表示的数据（如精确到磅的重量值、家庭人口数或者库存量），都可以用一个整型列来保存。DECIMAL 类型可以保存带有小数部分的精确值，如 3.14159、-.00273 或 -4.78。这种类型非常适合用来保存像币值之类的数据。整数和 DECIMAL 类型的值都会被精确地保存起来，不会进行四舍五入；并且关于它们的计算也是精确的。

浮点类型，它细分为单精度（FLOAT）和双精度（DOUBLE）。这些类型和 DECIMAL 一样，也可以用来存放带有小数部分的数字，但它们存储的是可能发生四舍五入的近似值，如 3.9E+4 或 -0.1E-100。如果对数值精确度的要求不太严格，或者因为数值很大，以至于 DECIMAL 类型无法表示，那么浮点类型则是个不错的选择。诸如粮食平均亩产、星球距离等类型的数据都很适合使用浮点值来表示。

BIT 类型，用于存储位域值。

带小数部分的值可以赋值给一个整型列，但会使用“四舍五入”规则近似成整数：如果小数部分大于等于 0.5，则舍弃小数部分，在整数部分上加 1（对于负数，则是减 1）。类似地，整数值也可以赋给允许有小数部分的类型列，只不过它们的小数部分等于零。

表 3-5 列出了每种数字类型的名称和取值范围。表 3-6 列出了各种数字类型所要求的存储空间大小。M 表示整型的最大显示宽度、浮点类型和 DECIMAL 类型的精度（有效位数），以及 BIT 类型的位数。对于那些带有小数部分的数据类型，D 代表的是数学精确度（小数点后面的数字个数），也称为“小数精度”（scale）。

表 3-5 数字数据类型的取值范围

类型名称	取值范围
TINYINT[(M)]	有符号值: -128~127 ($-2^7 \sim 2^7 - 1$) 无符号值: 0~255 ($0 \sim 2^8 - 1$)
SMALLINT[(M)]	有符号值: -32768~32767 ($-2^{15} \sim 2^{15} - 1$) 无符号值: 0~65535 ($0 \sim 2^{16} - 1$)
MEDIUMINT[(M)]	有符号值: -8388608~8388607 ($-2^{23} \sim 2^{23} - 1$) 无符号值: 0~16777215 ($0 \sim 2^{24} - 1$)
INT[(M)]	有符号值: -2147683648~2147683647 ($-2^{31} \sim 2^{31} - 1$) 无符号值: 0~4294967295 ($0 \sim 2^{32} - 1$)
BIGINT[(M)]	有符号值: -9223372036854775808~9223372036854775807 ($-2^{63} \sim 2^{63} - 1$) 无符号值: 0~18446744073709551615 ($0 \sim 2^{64} - 1$)
DECIMAL[(M[, D])]	由M和D的值决定
FLOAT[(M, D)]	最小非零值: $\pm 1.175494351\text{E}-38$ 最大非零值: $\pm 3.402823466\text{E}+38$
DOUBLE[(M, D)]	最小非零值: $\pm 2.2250738585072014\text{E}-308$ 最大非零值: $\pm 1.7976931348623157\text{E}+308$
BIT[(M)]	$0 \sim 2^M - 1$, $1 \leq M \leq 64$

表3-6 数字数据类型的存储空间要求

类型名称	存储空间占用量
TINYINT[(M)]	1个字节
SMALLINT[(M)]	2个字节
MEDIUMINT[(M)]	3个字节
INT[(M)]	4个字节
BIGINT[(M)]	8个字节
DECIMAL[(M[D])]	取决于M和D
FLOAT[(M,D)]	4个字节
DOUBLE[(M,D)]	8个字节
BIT[(M)]	取决于M

类型DECIMAL的存储空间要求取决于小数点左右两侧的数字个数。对于每一侧，每9位数字需要4个字节，最后剩下的数字需要1~4个字节。每一个值的存储空间都等于小数点左右两边字节存储占用总和，是小数点左右两侧数值所需存储空间的总和。

一个BIT(M)值大约需要 $(M+7)/8$ 个字节的存储空间。

3. 2. 4. 1 精确值数字数据类型

精确值数据类型包括整数类型和定点DECIMAL类型。

MySQL中的整数类型包括TINYINT、SMALLINT、MEDIUMINT、INT和BIGINT。其中，INTEGER是INT的同义词。这几种数据类型之间的主要区别在于，它们可以表示的那些值的取值范围和它们所需要的存储空间都各不相同。（取值范围越大，所需的存储空间也会越多。）可以将整型列定义为UNSIGNED，以禁止出现负数，这也将其取值范围上移到了从0开始的区间。

在定义整型列时，你可以为它指定一个可选的显示宽度 M，该值必须是1~255的整数。它决定着MySQL将用多少个字符来显示该列里的值。例如，MEDIUMINT(4)会指定一个显示宽度为4个字符的MEDIUMINT列。如果没有显式地为整型列声明宽度，那么MySQL将自行确定一个默认的宽度。这个默认宽度通常是该列里“最长”值的长度。

整型列的显示宽度 M 只与MySQL用于显示列值时所使用到的字符数相关，而与存储该值所需要占用的字节存储空间无关。例如，不管其显示宽度是多少个字符，BIGINT值都要占用8个字节的存储空间。即使把它定义为BIGINT(4)，但BIGINT列所需要占用的存储空间也不会神奇地缩小一半。这个 M 与列的取值范围没有任何关系。例如，把某列定义为INT(3)，这并不会把这列里的最大值限定为999。此外，显示值也不会被强行截断为 M 个字符。如果某个特定值的正常显示要求超过了 M 个字符，那么MySQL会把该值完整显示出来。

DECIMAL是一种定点类型，即所有值都有一个固定的小数位数。这一点的重要性在于DECIMAL值不会像浮点数那样存在四舍五入的问题——此特性使得DECIMAL非常适合于保存货币值。

NUMERIC和FIXED都是DECIMAL的同义词。

DECIMAL列也可以被定义成UNSIGNED。与整数类型不同的是，把一个DECIMAL类型定义为UNSIGNED不会扩大该类型的取值范围，而只会“砍掉”整个负数部分。

在DECIMAL列的定义里，可以包含一个最大有效位数 M 和一个小数位数 D （即精度和小数位数）。其中， M 的取值范围是1~65； D 的取值范围是0~30，且不超过 M 。

这里的 M 和 D 都是可选的。如果省略了 D ，那么其默认值为0。如果省略了 M ，那么其默认值为10。换句话说，下列等式是成立的：

$$\text{DECIMAL} = \text{DECIMAL}(10) = \text{DECIMAL}(10,0)$$

$$\text{DECIMAL}(n) = \text{DECIMAL}(n,0)$$

DECIMAL类型的最大取值范围取决于 M 和 D 的值。如果在 D 保持不变的情况下调整 M ，则取值范围将随着 M 的变大而增加（见表3-7）。如果在 M 保持不变的情况下调整 D ，则取值范围将随着 D 的变大而减小（见表3-8）。

表3-7 M 对DECIMAL(M , D)取值范围的影响

类型定义	取值范围
DECIMAL(4,1)	-999.9~999.9
DECIMAL(5,1)	-9999.9~9999.9
DECIMAL(6,1)	-99999.9~99999.9

表3-8 D对DECIMAL(M, D)取值范围的影响

类型定义	取值范围
DECIMAL(4,0)	-9999~9999
DECIMAL(4,1)	-999.9~999.9
DECIMAL(4,2)	-99.99~99.99

3. 2. 4. 2 近似值数字数据类型

MySQL提供了两种浮点类型——FLOAT和DOUBLE，用于存储近似值数。DOUBLE PRECISION是DOUBLE的同义词。在默认情况下，REAL类型是DOUBLE的同义词；如果启用了SQL的REAL_AS_DEFAULT模式，则它会成为FLOAT的同义词。

浮点类型也可以定义为UNSIGNED。这样，该类型的负数部分将被“砍掉”。

与DECIMAL类型相似，在浮点类型列的定义里，可以包含一个最大有效位数M和一个小数位数D（即精度和小数位数）。其中，M的取值范围是1~255；D的取值范围是0~30，且不超过M。

这里的M和D都是可选的。如果在定义列时省略了它们，那么这些值会按你的硬件所支持的最大精度来存储。

使用FLOAT(p)语法也是可以的。不过，这里的p在标准SQL里表示的是所要求的精度位数，这点与MySQL的处理方式有所差异。p的取值范围是0~53，并且只能用于确定此列存储的是单精度值，还是双精度值。如果p值落在0~24的区间内，则此列会被视为单精度列；如果落在25~53的区间内，则此列将被视为双精度列。也就是说，此列会根据是否带有用M和D值，来确定是FLOAT类型，还是DOUBLE类型。

3.2.4.3 BIT数据类型

BIT数据类型可用于存放位域值。BIT列的定义里可以包含一个可选的最大宽度M，它表示的是按二进制位计算列的“宽度”。M的取值必须是一个1~64的整数。如果省略，M的默认值将为1。

在默认情况下，从BIT列检索出来的值不能打印显示。要显示一个位域值的可打印形式，需要加上零或是使用CAST()函数：

```
mysql> CREATE TABLE t (b BIT(3)); # 列宽为3位，取值范围为0~7
```

```
mysql> INSERT INTO t (b) VALUES(0),(b'11'),(b'101'),(b'111');
```

```
mysql> SELECT b+0, CAST(b AS UNSIGNED) FROM t;
```

b+0 CAST(b AS UNSIGNED)	
0	0
3	3
5	5
7	7

如果需要以二进制表示法来显示某些位域值或是它们以二进制形式计算后的结果，那么可以使用BIN()函数：

```
mysql> SELECT BIN(b), BIN(b & b'101'), BIN(b | b'101') FROM t;
```

-----+-----+-----		
BIN(b)	BIN(b & b'101')	BIN(b b'101')
-----+-----+-----		
0	0	101
11	1	111
101	101	101
111	101	111
-----+-----+-----		

对于八进制和十六进制的显示，可以分别使用函数OCT()和HEX()。

3.2.4.4 数字数据类型的属性

UNSIGNED 属性可以防止出现负值。它适用于除 BIT 以外的所有数字类型，但大部分情况是与各种整数类型一起使用。让一个整型列具有 UNSIGNED 属性，并不会改变底层数据类型的取值范围的宽度，而只是把这个范围向右平移。请看下面这个表声明：

```
CREATE TABLE mytbl
(
  itiny TINYINT,
  itiny_u TINYINT UNSIGNED
);
```

itiny和itiny_u都是范围跨度为256的TINYINT列，只是允许的值集不一样。itiny的取值范围是-128~127，而itiny_u的取值范围则左移到了0~255。

如果数据不可能出现负数值（如人口统计数字或观众人数），则应考虑给相应的整型列加上UNSIGNED属性。如果用一个带正负号的列类型来保存这类数据，则只能使用到该数据类型的一半取值范围。在该列加上UNSIGNED属性之后，可用范围则立刻会增加一倍。例如，想用某列来保存序列编号，那么在加上UNSIGNED属性之后，可用编号的长度会翻一倍。

你也可以为DECIMAL或浮点列加上UNSIGNED属性，但最终效果与整型列的情况稍有不同。具体表现在：浮点值的取值范围不会朝正数方向平移；相反，原取值范围的上端将保持不变，而下端会变成0（在效果上，即是将整个取值范围“砍掉”了一半）。

所有支持UNSIGNED属性的数字类型，也都可以用SIGNED属性。因为这些类型在默认情况下都是带符号的，所以即使为允许使用负数的列加上SIGNED属性，也没有什么效果。

ZEROFILL属性适用于除BIT以外的所有数字类型。它会在列里的显示值前面填充若干0，使其宽度最终达到显示宽度。当你想要让列值按指定的显示宽度进行显示时，就可以使用ZEROFILL。确切来讲，显示宽度其实是“一个给定的最小数字个数”，因为那些长度大于显示宽度的数值，会全部显示出来，而不会被截断。可以调用下面的语句来查看效果。请注意，最后那个值是完整显示的，因为它比列的显示宽度宽。

```
mysql> DROP TABLE IF EXISTS mytbl;

mysql> CREATE TABLE mytbl (my_zerofill INT(5) ZEROFILL);

mysql> INSERT INTO mytbl VALUES(1),(100),(10000),(1000000);

mysql> SELECT my_zerofill FROM mytbl;
```

```
-----
my_zerofill
-----

    00001
    00100
    10000
1000000
```

如果给某个列指定了ZEROFILL属性，那么它将自动转换成一个UNSIGNED列。

另外一个属性是 AUTO_INCREMENT，它可用于整数或浮点数类型（不过，用于浮点数类型的情况比较少见）。当想要生成一组唯一标识符或序列值时，可以指定AUTO_INCREMENT属性。当把NULL插到AUTO_INCREMENT列时，

MySQL会自动生成下一个序列值，并把它存储到该列中。通常情况下，在没有做出另外设定的时候，AUTO_INCREMENT 列的值将从 1 开始，每新增一行，序列值也加1。如果在表里删除了某几行，那么这个序列可能也会受到影响。也就是说，序列值可能会被再次使用，但是这种情况是否会发生，还要看具体的存储引擎。

每个表最多只能有一个AUTO_INCREMENT列。该类型的列应该具备NOT NULL约束，并且必须被索引。一般情况下，最好能把AUTO_INCREMENT列声明为一个PRIMARY KEY或UNIQUE索引。此外，因为序列值不可能是负数，所以通常还应该为该列加上 UNSIGNED 属性。例如，可以用下列任意一条语句来声明一个AUTO_INCREMENT列：

```
CREATE TABLE ai (i INT UNSIGNED NOT NULL AUTO_INCREMENT  
PRIMARY KEY);
```

```
CREATE TABLE ai (i INT UNSIGNED NOT NULL AUTO_INCREMENT  
UNIQUE);
```

```
CREATE TABLE ai (i INT UNSIGNED NOT NULL AUTO_INCREMENT,  
PRIMARY KEY (i));
```

```
CREATE TABLE ai (i INT UNSIGNED NOT NULL AUTO_INCREMENT,  
UNIQUE (i));
```

其中，前面两种形式将索引信息指定为列定义的一部分。紧接着的两种形式将索引指定为CREATE TABLE语句的一个独立子句。如果这个索引只包含AUTO_INCREMENT列，那么使用独立子句的方式则是可选的。如果想要创建一个涉及多个列且包含AUTO_INCREMENT列的索引，那么必须使用独立子句。更多相关示例请参考3.4.2.1节。

AUTO_INCREMENT列可以显式地声明为NOT NULL。即使你省略了NOT NULL，MySQL也会自动把它加上。

3.4节将进一步讨论AUTO_INCREMENT列的行为特点。

以上介绍的各种属性都是数字类型列所特有的。在这些属性后面，你可以继续指定 NULL或NOT NULL属性。如果没有为数字类型列指定NULL或NOT NULL，那么它将默认允许使用NULL值（通常情况是这样。不过，如上所述，AUTO_INCREMENT列默认使用的是NOT NULL）。

你还可以利用DEFAULT属性来指定默认值。下面这个表包含了3个INT列，它们的默认值分别为-1、1和NULL：

```
CREATE TABLE t
(
    i1 INT DEFAULT -1,
    i2 INT DEFAULT 1,
    i3 INT DEFAULT NULL
);
```

如果列定义没有包含DEFAULT子句，那么MySQL将根据3.2.3节描述的规则，为它选择一个默认值。

3.2.4.5 选择数字数据类型

在为数字列选择数据类型时，需要考虑数据的取值范围，并选择一个能够覆盖该范围的最小类型。选择较大的类型，会浪费存储空间；会导致表毫无必要地变得很大；也会导致在处理这样的表时，无法像处理选择了更小类型的表那样有效率。如果数据的取值范围很小（如人的年龄或兄弟姐妹人数），则选择TINYINT类型是最佳的。MEDIUMINT类型能够表示上百万的值，它可用于很多类型的信息，只是在存储空间方面会有些额外的开销。BIGINT类型的取值范围最大，但它占用的存储空间是最小整数类型（INT）的两倍，所以应该只在真的需要时才使用它。对于浮点值，DOUBLE 类型所占用的存储空间是 FLOAT 类型的两倍。因此，只有在需要非常高的精确度或者数据的取值范围非常大的情况下，才需要使用DOUBLE 来代替 FLOAT；否则，会浪费掉一半的存储开销。

每个数字列的取值范围都由其类型决定。如果想要把某个超出了列的取值范围的值插入这个列里，那么最终的结果将取决于是否已启用严格的SQL模式。如果已启用，则取值超出范围时将导致一个错误。如果未启用，则会进行截断：MySQL会先把这个值替换为该列的取值范围的相应端点值，然后将处理后的结果值插入列中，同时生成一条警告消息。

值的截断处理依据的是数据类型的取值范围，而不是它的显示宽度。例如，SMALLINT(3)列的显示宽度只有3个字符，但它的取值范围却为-32768～32767。因为数值12345比列的显示宽度更宽，但仍在该列的取值范围内，所以不需要对它进行截断即可插入它，而且在检索时得到的结果也是 12345。

对于数值 99999，由于它超出了该列的取值范围，所以它在插入时会被截断为32767。以后的检索结果也会是32767。

对于定点或浮点列，如果存储值的小数位数多于该列定义所规定的值，那么会出现取舍情况。例如，把1.23456插到一个FLOAT(8, 1)列里，其结果将是1.2。如果把这个值插到一个FLOAT(8, 4)列里，其结果将是1.2346。也就是说，必须根据自己对精确度的要求来选定一个小数点后面的位数。如果需要精确到千分之一，则不能只把小数点后面的位数声明为2位。

3.2.5 字符串数据类型

MySQL提供了多种用于存储字符串值的数据类型。字符串通常用于表示下面这样的文本值：

```
'N.Bertram, et al.'

'Pencils (no.2 lead)'

'123 Elm St.'

'Monograph Series IX'
```

不过，字符串可以用于表示任何值；从这个意义上讲，它实际上是一种“范型”类型。例如，可以用二进制串类型来存储二进制数据，如图像、声音、压缩后的gzip输出。

表3-9列出了MySQL提供的所有用于定义字符串值列的类型，以及这些类型的最大大小和存储空间要求。其中，M 表示列值的最大长度（二进制串以字节为单位，非二进制串以字符为单位）；L表示某个给定值的字节长度；w表示列字符集里最“宽”字符所占用的字节数。类型BLOB和TEXT分别有多种变体，其主要区别在于它们所能容纳的字符串的最大长度不相同。

表3-9 字符串类型

类型名称	最大长度	存储空间要求
BINARY[(M)]	M个字节	M个字节
VARBINARY(M)	M个字节	L+1或L+2个字节
CHAR[(M)]	M个字符	M×w个字节
VARCHAR(M)	M个字符	L+1或L+2个字节
TINYBLOB, TINYTEXT	2 ⁸ -1个字节	L+1个字节
BLOB, TEXT	2 ¹⁶ -1个字节	L+2个字节
MEDIUMBLOB, MEDIUMTEXT	2 ²⁴ -1个字节	L+3个字节
LONGBLOB, LONGTEXT	2 ³² -1个字节	L+4个字节
ENUM('value1', 'value2', ...)	65 535个成员	1或2个字节
SET('value1', 'value2', ...)	64个成员	1、2、3、4或8个字节

有些类型是用于保存二进制串（字节串），而其他类型则用于保存非二进制串。例如，BINARY(20)可以容纳20个字节，而CHAR(20)也可以容纳20个字符（对于多字节字符，其存储空间要求肯定会超过 20个字节）。关于二进制串和非二进制串在字节和字符语义方面的区别请参考3.1.2节。每种二进制串类型都有一种与之对应的非二进制串类型，如表3-10所示。

表3-10 二进制串类型和非二进制串类型之间的对应关系

二进制串类型	非二进制串类型
BINARY	CHAR
VARBINARY	VARCHAR
BLOB	TEXT

每一种非二进制串类型，以及ENUM和SET类型，都可以指定具体的字符集和排序规则。对不同的列，可以指定不同的字符集。关于如何指定字符集的更多信息请参考3.2.5.5节。

BINARY和CHAR 都是固定长度的字符串类型。对于这两种类型的列，MySQL将为每个值分配同样数量的存储空间，并且会对那些比列的长度更短的值进行补齐。对于 BINARY 类型，是使用0x00进行补齐；对于CHAR类型，则

是使用空格。因为CHAR(M)列必须能够表示由列的字符集所确定的最大可能字符串，所以这种类型列需要占用 $M \times w$ 个字节，其中，w是字符集里最宽字符所占用的字节数。例如，1个ujis字符会占用1~3个字节，因此CHAR(20)必须分配60个字节的空間，以应对20个字符全部占用3个字节的情况。

其他字符串类型的长度都是可变的。对于每个值所占用的存储空间大小，各个行也各不相同；具体是多少，则取决于存储在列里的那些值的最大允许长度。对于可变长度的类型，这个长度在表3-9里被表示为L。除去L以外的额外要求字节空间，与用来存储值的长度所需的字节数相等。在处理长度可变的数据时，MySQL会把数据内容和数据长度都存储起来。这些额外的“长度字节”前缀，会被当作无符号整数来对待。在可变类型的最大长度（即该类型所要求的长度字节数）与占用相同字节数的无符号整数类型的取值范围之间，存在着一定的对应关系。例如，MEDIUMBLOB类型值的最大长度是 $2^{24}-1$ 个字节，并且需要3个字节来记录这个长度。而对于占用3个字节的整数类型MEDIUMINT，其最大无符号值也为 $2^{24}-1$ 。这可不是什么巧合！

对于VARBINARY和VARCHAR类型，如果列值的最大字节长度小于256，那么其长度前缀将占用1个字节；否则，将占用2个字节。

除了ENUM和SET这两种类型以外，MySQL会将其他所有字符串类型的值存储为一串字节；并且它还会根据所存储字符串的具体类型（二进制或非二进制），把这些字节解释为字节串或字符串。如果这些值的长度太长，导致无法存储，那么会截断它们。（在严格模式下，只有截断后的字符为空格的情况下，才会有错误提示。）不过，因为各种字符串类型的大小各异（最大的类型能够容纳将近4 GB的数据），所以在理论上完全可以找到一种很长的字符串类型，用于存储各种长数据，从而避免信息被截断。（实际上，字符串类型列的最大有效长度，取决于MySQL的“客户端/服务器”通信协议所支持的最大数据包长度，其默认值为1 MB。）

对于ENUM和SET类型，其列定义里都有一个合法字符串值的列表，但ENUM和SET的值在内部都被存储为数字，更多相关细节请参考3.2.5.4节。在未启用严格模式的情况下，如果把某个未出现在列表里的值存储到ENUM或SET类型列里，则会导致该值被转换为一个空串（''）。在严格模式下，这会导致出现错误。

3.2.5.1 CHAR和VARCHAR数据类型

字符串类型CHAR和VARCHAR主要用于保存非二进制串，因此它们有字符集和排序规则。

CHAR和VARCHAR之间的主要区别在于：它们的长度是固定的，还是不变的；如何对待尾部空格。

CHAR是一种长度固定的类型，而VARCHAR是一种长度可变的类型。

从CHAR列检索出来的值，其尾部空格会被移除。对于CHAR(M)列，如果其值的长度小于M个字符，那么在存入时会用空格将长度补齐到M。但是，当在检索这些值时，其尾部的空格会被移除。如果启用SQL的PAD_CHAR_TO_FULL_LENGTH模式，那么在检索CHAR列值时就可以保留尾部空格。

对于VARCHAR列，其尾部空格在存储和检索时都会被保留。

在定义CHAR列时，其最大长度M的取值范围是0~255。对于CHAR类型，M是可选的；如果省略，则其默认值为1。请注意，CHAR(0)也是合法的。如果你允许CHAR(0)列为NULL，那么它可以用它来表示“开/关”值。这种列只有两种取值：NULL或空串。在表里，CHAR(0)列只占用非常少的存储空间——仅一个二进制位。

对于VARCHAR(M)，其中的M在语法上的取值范围是1~65 535，但它实际能够容纳的最大字符个数肯定小于65 535。这是因为在MySQL里，行的最大长度为65 535个字节。下面是一些潜在的影响点。

一个长VARCHAR列需要2个字节来存放字符串的长度，最终长度不能超过行的总长。

使用多字节字符可以减少字符个数，从而使字符总长度不会超过最大行长度。

表里的其他列也会减少行里VARCHAR列的可用空间量。

当在CHAR和VARCHAR这两种数据类型之间犹豫不决时，请记住下面两条通用原则。

如果所有值的长度都是M个字符，那么VARCHAR(M)列会比CHAR(M)列多占用存储空间，因为前一种类型还需要用额外的字节来记录值的长度。反之，如果数据长短不一，那么选用VARCHAR类型可以节省空格占用的存储空间。CHAR(M)列始终会占用M个字符的空间，即使其值为空或NULL也一样。

如果使用的是MyISAM表，并且各个值的长度差别不大，那么选用CHAR往往会比选用VARCHAR更好些，因为MyISAM存储引擎对固定长度行的处理效率，要比对长度可变行的高。更多相关信息请参考5.4节。

3.2.5.2 BINARY和VARBINARY数据类型

BINARY和VARBINARY类型与CHAR和VARCHAR相似，不同之处在于以下几点。

CHAR和VARCHAR都是用于存储字符的非二进制类型，并且都有字符集和排序规则。比较操作依据的是排序序列。

BINARY和VARBINARY都是用于存储字节的二进制类型，它们没有字符集和排序规则。比较操作依据的是字节值的大小。

对于BINARY (M) 列，当存储那些长度短于M个字节的值时，它们会用0x00字节进行补齐，以保证长度为 M。在检索时，不会去除任何内容。对于VARBINARY，在存储值时，不会进行补齐；在检索时，也不会去除任何内容。

3. 2. 5. 3 BLOB和TEXT数据类型

“blob”指的是二进制大对象 (binary large object)，它实际上是一个能够存放任何内容的容器，并且可以大到超出你的想象，多达4 GB。在MySQL里，BLOB类型实际上是一个类型家族，它包括TINYBLOB、BLOB、MEDIUMBLOB和LONGBLOB。除各自所能存放的最大信息量不同以外，这些类型在其他方面完全一致（见表3-9）。BLOB列存储的是二进制串。如果要保存的信息有可能会急剧膨胀，或者各行的长短差异很大，那么 BLOB 类型会非常适合。像压缩数据、加密数据、图像和声音，都适合使用BLOB。

MySQL还有一个TEXT类型家族，它包括TINYTEXT、TEXT、MEDIUMTEXT和LONGTEXT。它们与相应的BLOB类型有很多相似之处，但TEXT类型存储的是非二进制串，而非二进制串。也就是说，它们存储的是字符，而不是字节，它们与字符集和排序规则相关联。这也是二进制串和非二进制串的不同之处（参考3.1.2节）。例如，BLOB值的比较是以字节为单位的；而TEXT值的比较则是以字符为单位，并且依据的是列的排序规则。

不过，TEXT 类型的最大长度与 BLOB 类型的是一样的。也就是说，它们都是以字节为单位来衡量最大长度的，而不是以字符为单位（对于 BLOB 类型）和以字符为单位（对于 TEXT类型）进行衡量（见表3-9）。

BLOB和TEXT列能否被索引，具体取决于所使用的存储引擎。

存储引擎InnoDB和MyISAM都支持对BLOB和TEXT列进行索引。但必须指定一个前缀长度，以方便索引使用。这样可以避免创建出可能会过于庞大的索引，如果出现这样的情况，则会完全抵消掉索引所带来的好处。不过凡事有例外，TEXT 列的FULLTEXT索引并不会使用那个前缀长度，因为FULLTEXT索引是以索引列的完整内容为基础的，指定的前缀长度会被忽略。

MEMORY表不支持 BLOB和TEXT 索引，因为 MEMORY引擎根本不支持 BLOB和TEXT列。

对于BLOB或TEXT列，需要特别注意以下几点。

由于BLOB和TEXT 列里的值在长度方面的差异通常很大，因此在多次执行删除和修改操作之后，表里容易产生大量碎片。如果是使用MyISAM表来存储BLOB或TEXT值，那么定期运行OPTIMIZE TABLE 命令可以减少碎片和改善系统性能。更多相关信息请参考第5章。

max_sort_length系统变量会对BLOB和TEXT类型值的比较和排序操作产生影响。对于每个值，只有其前面的max_sort_length个字节才会被使用到。

（这意味着，对于使用了多字节字符集的TEXT列，参与比较的字符数会少于max_sort_length。）如果max_sort_length的默认值（为1024）会造成问题，则可以在执行比较之前把它调大。

对于数据量非常大的值，可能需要配置MySQL服务器，增大max_allowed_packet参数的值。更多相关信息请参考12.7.1节。对于所有想要使用大数据量值的客户端程序，则需要在客户端增大数据包的大小。客户端程序mysql和mysqldump都支持直接使用启动选项来设置这个值。

3.2.5.4 ENUM和SET数据类型

ENUM和SET是比较特殊的字符串数据类型，它们只能从一个固定的（预先定义好的）字符串列表里取值。这两种类型的主要区别是：ENUM列值必须包含且只能包含一个值列表成员，而SET 列值则允许包含任意多个值列表成员（可以为空，也可以是全体成员）。换句话说，ENUM类型的值不允许同时出现，而SET类型的值允许同时出现。

ENUM类型定义的是枚举集合，它最多允许有65 535个成员。赋给ENUM列的值只能是你创建表时指定的值列表中的一个成员。枚举类型通常用于表示类别值。例如，对于某个定义为ENUM('N', 'Y')的列，其值可以是'N'或'Y'。另外，你也可以将ENUM 类型用于表示某种产品的尺寸或颜色，或者用于表示某次调查问卷中的多重选择题的答案（仅限单选）：

```
employees ENUM('less than 100','100-500','501-1500','more than 1500')
```

```
color ENUM('red','green','blue','black')
```

```
size ENUM('S','M','L','XL','XXL')
```

```
vote ENUM('Yes','No','Undecided')
```

如果你正在处理来自Web页面的选择项，而该页面包含了很多互斥的单选按钮，那么你可以使用ENUM 来表示这些让网站的访问者可以从中进行选择的

选项。例如，你正在运营一个在线比萨饼订购服务，这时就可以用一个ENUM列来表示顾客所订比萨饼的口味和大小：

```
crust ENUM('thin','regular','pan style','deep dish')
```

```
size ENUM('small','medium','large')
```

SET类型与ENUM类型有相似之处，如在创建SET列时，同样需要为它指定一个所允许的集合成员列表。SET类型最多允许有64个成员。它与ENUM类型有所不同，其每一个值列都可以由集合的任何成员构成。如果有一组固定值，但与在ENUM 列里的情况不同，它们不是互斥的，这时就可以使用SET。例如，那个在线比萨饼服务可能会有一组多选框，用于表示顾客想要的用作馅料的配料，而且可以有多种选择。这时就可以使用SET来表示这些配料：

```
SET('pepperoni','sausage','mushrooms','onions','ripe olives')
```

于是，特定的SET值就代表了顾客实际所选择的那些配料：

```
'pepperoni,mushrooms'
```

```
'sausage,onions'
```

```
'sausage,mushrooms,ripe olives'
```

```
'onions'
```

```
"
```

上面显示的最后那个值（空串）表示的是，顾客在订购时没有选择任何配料。SET 列允许有这样的值。

SET列的定义，可以写成以逗号分隔的单个字符串列表，表示所有的集合成员。另一方面，SET 列的值，必须是一个字符串。如果这个值由集合里的多个成员构成，则必须用逗号把各个成员分隔开。这意味着，你不应该把一个包含逗号的字符串用作SET的成员。

在为ENUM或SET列定义合法取值列表时，必须考虑到以下几个因素。

这个列表确定了列的所有允许值，之前已经讨论过这一点。

如果ENUM或SET列带有一个不区分大小写的排序规则，那么在插入合法值时也不用区分大小写，它们都能够被识别出来。但是，当在检索ENUM或SET列

里的数据时，它们将按照列定义的合法取值列表里的字母大小写形式来显示。例如，定义了一个ENUM('Y', 'N')列，那么完全可以插入'y'和'n'；但在检索时，它们会显示为'Y'和'N'。如果这个列带有区分大小写的排序规则或二进制排序规则，那么在插入数据时，必须严格按照列定义里的大小写字母来书写；否则，它们会被服务器当作是非法的。

另一方面，的确可以根据成员大小写的不同来区分它们，但是当你使用不区分大小写的排序规则时，这种做法就不可行了。

ENUM 定义里的值的顺序就是排序所用的顺序。SET 定义里的值的排序也确定了排序顺序，只是关系更为复杂些，因为其列值可能包含多个集合成员。

当 MySQL 显示某个由多个集合成员构成的SET 值时，这些成员的列出顺序由它们在SET列定义里的顺序确定。

在创建ENUM和SET类型列时，需要以字符串的形式列出枚举和集合成员，因此它们都被归类为字符串类型。但是，ENUM和SET类型实际上有点“人格分裂”：它们的成员在内部的存储形式是数字，并且也可以把它们当作数字来对待。这意味着，ENUM和SET类型要比其他字符串类型有着更好的处理性能，因为它们可以使用数字操作来代替字符串操作进行处理。同时也意味着，ENUM和SET值可以用在字符串环境里，也可以用在数字环境里。最后还有一点，如果在字符串环境里使用ENUM和SET列，但是希望它们表现得像数字（或者是情况刚好相反），那么它们可能会引起混乱。

MySQL将从1开始依次对ENUM列定义里的成员进行顺序编号。（编号0被保留为出错代码，其字符串形式即为一个空串。）ENUM 列占用的存储空间大小由枚举值的个数确定。1个字节可以表示256个值，2个字节能表示65 536个值。（请试着将它们分别与占用1个字节的整数类型TINYINT UNSIGNED的取值范围和占用2个字节的整数类型SMALLINT UNSIGNED的取值范围作个对比。）因此，在加上那个出错代码之后，枚举成员的最大数目是 65 536个；而对于它所占用的存储空间是1个字节还是2个字节，则取决于成员数是否超过256个。因为MySQL需要为出错代码预留 1个位置，并将它作为每一个枚举的隐含成员，所以在ENUM 定义里最多能够指定的是65 535（而非65 536）个成员。当把某个非法值放入ENUM列时，MySQL会赋值为那个出错成员。（在严格模式下，会产生一个错误。）

下面这个示例演示了以字符串方式和数字方式检索 ENUM 值的情况（它会显示枚举成员的序号；NULL值没有序号）：

```
mysql> CREATE TABLE e_table (e ENUM('jane','fred','will','marcia'));
```

```
mysql> INSERT INTO e_table
```

```
-> VALUES('jane'),('fred'),('will'),('marcia'),(NULL);
```

```
mysql> SELECT e, e+0, e+1, e*3 FROM e_table;
```

e	e+0	e+1	e*3
jane	1	2	3
fred	2	3	6
will	3	4	9
marcia	4	5	12
NULL	NULL	NULL	NULL

可以按名字或编号来对ENUM成员进行比较：

```
mysql> SELECT e FROM e_table WHERE e='will';
```

```
+-----+
```

```
| e      |
```

```
+-----+
```

```
| will |
```

```
+-----+
```

```
mysql> SELECT e FROM e_table WHERE e=3;
```

```
+-----+
```

```
| e      |
```

```
+-----+
```

```
| will |
```

```
+-----+
```

也可以把空串定义为有效的枚举成员，不过这个时候需要多加注意。这个字符串被赋值为一个非零值，与该定义里列出的其他成员没什么两样。但是，空串也被用于那个数值为0的错误成员，因此它会对应到两个内部数字元素值。在下面的示例里，当把非法枚举值'x'插到ENUM列时，它会被赋值为出错成员。只有以数字形式检索，才能把列定义里的空串成员区别出来：

```
mysql> CREATE TABLE t (e ENUM('a','b'));
```

```
mysql> INSERT INTO t VALUES('a'),(''),('b'),('x');
```

```
mysql> SELECT e, e+0 FROM t;
```

e		e+0
a		1
		2
b		3
		0

在严格模式下，赋值成非法值'x'会导致错误，并且不会存储任何值。

SET列的数字表示与ENUM列稍有不同。SET成员并没有按顺序编号。每个SET成员对应着SET值里的一个二进制位。第一个成员对应于第0位，第二个成员对应于第1位，依此类推。换句话说，SET成员的数字值都是2的幂。空串对应的SET数字值为0。

SET的值都被存储为二进制位值。每个字节对应8个SET成员，所以一个SET列占用的存储空间取决于它的成员个数，最多只能有64个成员。如果SET的成员数量范围分别为1~8个、9~16个、17~24个、25~32个、33~64个，那么与之对应的占用字节空间分别为1、2、3、4或8个字节。

既然一个SET可以由一组二进制位表示，那么一个SET值就可以由多个集合成员组成。由于这样的值可以展开为某种二进制位的组合，因此它也可以由SET定义里与这些二进制位相对应的那些字符串组合而成。

下面这个示例演示了SET列的字符串形式与数字形式之间的关系。它会以十进制和二进制形式来显示具体的数值：

```
mysql> CREATE TABLE s_table (s SET('table','lamp','chair','stool'));

mysql> INSERT INTO s_table
-> VALUES('table'),('lamp'),('chair'),('stool'),(''),(NULL);

mysql> SELECT s, s+0, BIN(s+0) FROM s_table;
```

s	s+0	BIN(s+0)
table	1	1
lamp	2	10
chair	4	100
stool	8	1000
	0	0
NULL	NULL	NULL

当把值 'lamp, stool' 赋给 s 列时，MySQL 会在其内部把它存储为 10（二进制数 1010），因为 'lamp' 的数值是 2（第 1 位），'stool' 的数值是 8（第 3 位）。

在对 SET 列进行赋值时，其中各个子字符串的顺序不用与定义这个列时所用的顺序完全一样。不过，在以后的检索操作中，这个成员是按它们在声明里的先后顺序显示的。此外，在给 SET 列赋值时，如果这个值包含的某些子字符串不属于集合成员，那么这些子字符串会被剔除掉，剩余的子字符串会被赋给这个列。以后检索这个值时，也看不到那些非法子字符串。

如果把 'chair, couch, table' 赋值给 s_table 表里的 s 列，那么会发生两件事。

'couch' 会被剔除掉，因为它不是集合成员。之所以如此，是因为在赋值过程中，MySQL 会为每一个值的子字符串确定出与之对应的二进制位，并把它们组合成存储值。既然不存在与 'couch' 相对应的二进制位，那么它便会被忽略掉了。

将来检索这个值时，它将显示为 'table, chair'。在检索时，MySQL 会按顺序扫描各个位，并根据数字值构造出对应的字符串值；同时，还会按照列定义里的顺序对子字符串自动重排。这种重排行为还意味着：即使你把某个值多次赋给一个 SET 列，它在检索结果里也只会出现一次。也就是说，如果把 'lamp, lamp, lamp' 赋值给 SET 列，那么在检索时也只会看到一个 'lamp'。

在严格模式下，使用非法 SET 成员将导致错误，并且这个值也不会被存储。在前面这个示例里，当赋予某个包含 'couch' 的值时，会引发一个错误，并且赋值操作也会失败。

MySQL会重排SET列值里的成员，这一事实说明：如果要搜索那些使用字符串的值，那么必须按正确的顺序列出各个成员。例如，先插入'chair, table'，然后搜索'chair, table'，那么最终是无法找到这行的；必须使用'table, chair'来查找才行。

ENUM和SET列的排序和索引操作都是按列值的内部值（即数字值）执行的。下面这个示例表面看起来不正确，因为这些值没有按字母顺序显示，但它实际毫无问题：

```
mysql> SELECT e FROM e_table ORDER BY e;
```

```
-----  
e  
-----  
  
NULL  
  
jane  
  
fred  
  
will  
  
marcia  
-----
```

为了更好看清是怎么回事，下面同时检索ENUM值的字符串形式和数字形式：

```
mysql> SELECT e, e+0 FROM e_table ORDER BY e;
```

-----+-----	
e	e+0
-----+-----	
NULL	NULL
jane	1
fred	2
will	3
marcia	4
-----+-----	

如果有一组固定值，而你想按某种顺序对它们进行排序，那么可以利用 ENUM 类型的排序特性。即先将这些值表示为某个表里的一个ENUM 列，并在该列的定义里按照你想要的顺序依次列出各个值。假设你用一个表来表示某个运动组织（如橄榄球队）的个人信息，而且你想要根据个人的位置来对输出结果进行排序，以便按特定的顺序（如教练、助理教练、四分卫、跑锋、接球手、边线防守队员等）进行输出。那么，你可以把这个列定义为 ENUM 类型的，并按你想要的顺序列出各个枚举成员。然后，在各种排序操作中，这些列值便会自动按事先设定的顺序输出。

如果想让某个ENUM列按常规的字母表顺序排序，那么可以先用CAST() 函数把这个列的值转换为一个非ENUM的字符串，然后再对结果进行排序：

```
mysql> SELECT CAST(e AS CHAR) AS e_str FROM e_table ORDER BY
e_str;
```

e_str

NULL
fred
jane
marcia
will

CAST() 函数并没有更改这些显示值，它在这条语句里的作用只是完成了一个从 ENUM 到字符串的转换操作——它会改变这些值的排序特性，从而可以按字符串方式进行排序。

3.2.5.5 字符串数据类型属性

字符串数据类型特有的属性有 CHARACTER SET（或 CHARSET）和 COLLATE，它们分别用于指定字符集和排序规则。你可以把它们指定为表自身的默认选项；也可以指定为各个列的单独选项，从而改写表的默认选项。（事实上，与服务器一样，每个数据库都有默认的字符集和排序规则。这些默认选项在创建表时能发挥一定作用，稍后即将看到。）

CHARACTER SET 和 COLLATION 属性适用的数据类型有 CHAR、VARCHAR、TEXT、ENUM 和 SET。它们并不适用于二进制串数据类型（BINARY、VARBINARY 和 BLOB），因为这些类型包含的是字节串，而非字符串。

在指定 CHARACTER SET 和 COLLATION 属性时，不管是在列、表和数据库这三级别中的哪一级，都要遵从以下规则。

使用的字符集必须是服务器支持的。使用 SHOW CHARACTER SET 语句可以查出当前可用的字符集。

如果定义里同时使用了 CHARACTER SET 和 COLLATION 属性，那么它们所代表的字符集和排序规则必须要相兼容。例如，对于字符集 latin2，排序规则可以使用 latin2_croatian_ci，但不能使用 latin1_bin。使用 SHOW COLLATION 语句可以把每一种字符集所支持的排序规则显示出来。

如果定义里只有 CHARACTER SET，而没有 COLLATION，则使用默认排序规则。

如果定义里只有 COLLATION，而没有 CHARACTER SET，那么具体的字符集就要由排序规则名的第一部分来确定。

通过下面这条语句，可以看到上述规则的应用情况。该语句会创建一个使用多种字符集的表。

```
CREATE TABLE mytbl
```

```
(
```

```
  c1 CHAR(10),
```

```

c2 CHAR(40) CHARACTER SET latin2,

c3 CHAR(10) COLLATE latin1_german1_ci,

c4 BINARY(40)

) CHARACTER SET utf8;

```

最终的表将使用utf8 作为其默认字符集。因为没有给出表选项 COLLATION，所以默认的表排序规则为utf8字符集的默认排序规则（即 utf8_general_ci）。因为c1列的定义里没有包含CHARACTER SET属性和 COLLATION属性，所以它将使用表的默认设置。c2、c3和c4列没有使用表级的字符集和排序规则：因为c2和c3列有它们自己的字符集信息，而c4列有二进制串类型，因此那些字符集属性对它们不适用。c2 列的排序规则是 latin2_general_ci，这是字符集 latin2的默认排序规则。c3 列的字符集是 latin1，它是根据该列的排序规则名latin1_german_ci确定的。

如果想要查看某个已有表的字符集信息，那么可以使用SHOW CREATE TABLE语句：

```

mysql> SHOW CREATE TABLE mytbl\G

***** 1.row *****

Table: mytbl

Create Table: CREATE TABLE `mytbl` (

  `c1` char(10) DEFAULT NULL,

  `c2` char(40) CHARACTER SET latin2 DEFAULT NULL,

  `c3` char(10) CHARACTER SET latin1 COLLATE latin1_german1_ci
DEFAULT NULL,

  `c4` binary(40) DEFAULT NULL

) ENGINE=InnoDB DEFAULT CHARSET=utf8

```

如果 SHOW CREATE TABLE 没有显示列的字符集，则表明它与表的默认字符集相同。如果它没有显示列的排序规则，则表明该列的排序规则与该列的

字符集默认排序规则相同。

也可以给SHOW COLUMNS语句加上关键字FULL，用于显示排序规则信息（可以根据它推断出具体的字符集）：

```
mysql> SHOW FULL COLUMNS FROM mytbl;
```

Field	Type	Collation	Null	Key	Default
c1	char(10)	utf8_general_ci	YES		NULL
c2	char(40)	latin2_general_ci	YES		NULL
c3	char(10)	latin1_german1_ci	YES		NULL
c4	binary(40)	NULL	YES		NULL

前面的讨论只提到了列和表的字符集设置情况，其实字符集还可以在列、表、数据库和服务等多个级别进行指定。在MySQL 处理字符列的定义时，会依次根据下述原则为它确定使用何种字符集。

（1）如果列的定义里指定了字符集，那么使用这个字符集。这里包括只给出 COLLATON 属性的情况，因为MySQL可以根据排序规则名确定使用哪一种字符集。

（2）否则，如果表的定义里有字符集选项，那么使用这个字符集。

（3）否则，把数据库的字符集作为表的默认字符集，该字符集还将成为列的默认字符集。如果以前没有为数据库显式指定字符集，那么数据库的字符集将沿用服务器的。

也就是说，MySQL在确定列的字符集时，会试着向上搜索可能指定字符集的各个层级，直到它找到某个定义的字符集为止，然后将它用于该列。数据库始终有一个默认字符集，因此即使在任何低层级没有显示指定字符集，这一搜索过程也肯定会在到达数据库级别之后终止。

字符集名binary比较特殊。当将某个非二进制串列指定为binary字符集时，相当于把该列定义为相应的二进制串类型。在下面几对列定义里，每一对里的那两条语句都是等价的：

c1 CHAR(10) CHARACTER SET binary

c1 BINARY(10)

c2 VARCHAR(10) CHARACTER SET binary

c2 VARBINARY(10)

c3 TEXT CHARACTER SET binary

c3 BLOB

如果为某个二进制串列指定了 CHARACTER SET binary，那么它将被忽略，因为该类型已经是二进制类型了。如果为ENUM或SET列指定了CHARACTER SET binary，那么它将像前面讨论的那样发挥作用。

如果把 binary 字符集赋值为表选项，那么它将应用到所有在其定义里没有指定任何字符集信息的字符串列。

对于字符串列定义的属性，MySQL还提供了多种简写形式。

ASCII属性是CHARACTER SET latin1的简写。

UNICODE属性是CHARACTER SET ucs2的简写。

对于非二进制串列、ENUM列或SET列，如果指定BINARY属性，则等同于在指定该列字符集的二进制排序规则。例如，某个表的默认字符集为 latin1，则以下这些定义是等效的：

c1 CHAR(10) BINARY

c2 CHAR(10) CHARACTER SET latin1 BINARY

c3 CHAR(10) CHARACTER SET latin1 COLLATE latin1_bin

如果为某个二进制串列指定了 BINARY，那么它将被忽略，因为该类型已经是二进制类型了。

任何字符串类型都能使用通用属性NULL或NOT NULL。如果没有指定，则默认为NULL。不过，把一个字符串列声明为 NOT NULL，并不意味着它不能存储空字符串（即''）。在MySQL里，空值与没有值是有区别的。因此，千万不

要因为某个字符串列声明成NOT NULL，就错误地认为它包含的是非空值。如果想让所有字符串值全部为非空，则必须在应用程序里实施这样的限制。

也可以使用DEFAULT子句，为除BLOB和TEXT类型外的其他字符串数据类型指定默认值。在3.2.3节描述了在某个列定义没有包含DEFAULT子句时，MySQL所使用的默认值赋值规则。

3.2.5.6 选择字符串数据类型

在为字符串列选择数据类型时，请认真考虑以下几个问题。

(1) 这些值是要表示为字符数据，还是二进制数据？如果答案是字符数据，则选择非二进制串类型最为合适。如果答案是二进制数据，则选用二进制串类型。

(2) 比较操作需要区分大小写吗？如果是，则应该选用一种非二进制串类型，因为这些存储字符，与字符集和排序规则是相关联的。

非二进制串值在比较和排序时是否区分大小写，受制于为它们指定的排序规则。如果不想区分大小写，则应该选用一种不区分大小写的排序规则。否则，应该选用一种二进制排序规则或区分大小写的排序规则。二进制排序规则在比较两个字符时，使用的是它们的数值编码。区分大小写的排序规则在比较两个字符时，使用的是该种排序规则特有的字符顺序——它不一定与字符编码的大小顺序相同。对于这两种情况，给定字符的小写和大写版本在比较操作中会被认为是不同的。假设，字符串'mysql'、'MySQL'和'MYSQL'都是latin1字符集。如果使用某种不区分大小写的排序规则，如latin1_swedish_ci，那么它们将被认为是相同的。如果使用二进制排序规则latin1_bin或某种区分大小写的排序规则latin1_general_cs，则它们是三种不同的字符串。

如果需要使用到的列里既有区分大小写的比较，又有不区分大小写的比较，则使用的排序规则要与你最常执行的比较类型相对应。对于其他类型的比较，则可以利用COLLATE运算符来更改排序规则。例如，如果mycol是一个使用latin1字符集的CHAR列，则可以选择latin1_swedish_ci作为不区分大小写比较的默认排序规则。下面的比较操作不区分大小写：

```
mycol = 'ABC'
```

当需要区分大小写的比较时，可以使用排序规则latin1_general_cs或latin1_bin。下面的比较操作会区分大小写（COLLATE运算符可以放在左边的字符串上，也可以放在右边的字符串上，效果是一样的）：

```
mycol COLLATE latin1_general_cs = 'ABC'
```

```
mycol COLLATE latin1_bin = 'ABC'
```

```
mycol = 'ABC' COLLATE latin1_general_cs
```

```
mycol = 'ABC' COLLATE latin1_bin
```

（3）想要少占用存储空间吗？如果是，则可以选用一种可变长度的类型，而不要选用固定长度的类型。

（4）列的取值总是从固定的某些值里选取吗？如果是，则ENUM或SET可能是最好的选择。

如果字符串值是一个有限集合，并且想按照某种非字母表顺序对它们进行排序，那么ENUM类型会非常有用。对 ENUM 值的排序依据的是列定义里那些枚举值的排列顺序，因此可以按你所期望的任何顺序让这些值进行排序。

（5）尾部的填充值很重要吗？如果要求数据必须原样存入和取出，不能增加和移除尾部的空格（对二进制数据类型，具体值为字节0x00），那么应该选用TEXT 或VARCHAR 列来存放非二进制串，而选用BLOB 或VARBINARY 列来存放二进制串。当想要存储压缩数据、散列数据或加密数据时，这一点非常关键，因为它们的编码方法可能会导致尾部空格。表 3-11 展示了MySQL如何处理各种字符串数据类型在存储和检索数据时的尾部填充问题。

表3-11 处理字符串数据类型的填充值

数据类型	存储	检索	结果
CHAR	填充空格	去掉	检索值无尾部填充
BINARY	填充0x00	不处理	检索值会保留尾部填充
VARCHAR, VARBINARY	不处理	不处理	尾部填充无变化
TEXT, BLOB	不处理	不处理	尾部填充无变化

启用SQL模式PAD_CHAR_TO_FULL_LENGTH，可以让检索出来的CHAR列值保留尾部空格。

3.2.6 时态（日期/时间）数据类型

MySQL提供了多种存储时态（与时间相关的）值的类型：DATE、TIME、DATETIME、TIMESTAMP和YEAR。在MySQL 5.6版本里，对这些类型进行了多项重要的改进。

对于数据类型TIME、DATETIME和TIMESTAMP，MySQL 5.6.4增加了对小数秒的支持。这些类型现在允许的可选小数部分多达6位（微秒）精度。

MySQL 5.6.5引入了扩展支持：自动把当前时间戳作为初始值并进行更新。在以前的版本里，这些属性只能用于表里的大部分单个TIMESTAMP 列。现在，它们可以用于任何TIMESTAMP列，并且也可用于DATETIME列。

MySQL 5.6.6丢弃了对YEAR(2)的支持，取而代之的是允许创建像YEAR(4)那样的列。

表3-12列出了这些时态数据类型，以及每种类型的合法取值范围。这些范围同时也反映出了在MySQL 5.6.4里对小数秒的额外支持。（对于5.6.4之前的版本，可以将小数秒部分忽略掉。）

表3-12 时态数据类型的取值范围

类型名称	取值范围
DATE	'1000-01-01' ~ '9999-12-31'
TIME	'-838:59:59[.000000]' ~ '838:59:59[.000000]'
DATETIME	'1000-01-01 00:00:00[.000000]' ~ '9999-12-31 23:59:59[.999999]'
TIMESTAMP	'1970-01-01 00:00:01[.000000]' ~ '2038-01-19 03:14:07[.999999]'
YEAR	1901~2155

如果要声明包含小数秒部分的时态类型列，则需要把定义写成type_name(fsp)，其中，type_name为TIME、DATETIME或TIMESTAMP，fsp为小数秒精度。例如，下面的TIME列允许的小数位数分别为3位和6位：

t1 TIME(3)

t2 TIME(6)

fsp值的取值范围必须为0~6。如果未给定，则默认为0。更多相关信息请参考3.2.6.5节。

表3-13展示的是每一种时态数据类型的存储空间要求。表3-14展示的是那些声明中带有小数秒部分的类型所具有的额外存储空间要求。

表3-13 时态数据类型的存储空间要求

类型名称	存储要求 (5.6.4以前的版本)	存储要求 (5.6.4及以后版本)
DATE	3字节	3字节
TIME	3字节	3字节+fsp的存储要求
DATETIME	8字节	5字节+fsp的存储要求
TIMESTAMP	4字节	4字节+fsp的存储要求
YEAR	1字节	1字节

表3-14 小数秒部分的存储空间要求

小数秒精度	存储要求
0	0字节
1, 2	1字节
3, 4	2字节
5, 6	3字节

当为某种日期/时间类型插入非法值时，该类型会把它存储为一个“零”值。表3-15列出了各种日期/时间类型的零值情况。如果想要把非法值处理为错误，并拒绝接受，则需要设置相应的SQL模式，更多信息请参考3.3节。“零”值也是那些声明时带有NOT NULL属性的日期/时间类型列的默认值。

表3-15 时态类型的“零”值

类型定义	零 值
DATE	'0000-00-00'
TIME	'00:00:00[.000000]'
DATETIME	'0000-00-00 00:00:00[.000000]'
TIMESTAMP	'0000-00-00 00:00:00[.000000]'
YEAR	0000

遵照标准SQL和ISO 8601规范的要求，MySQL的日期表示顺序为“年-月-日”。例如，2015年12月3日将被表示为'2015-12-03'。为满足检索显示要求，可以使用DATE_FORMAT()函数和TIME_FORMAT()函数来显示各种格式的日期和时间。

在输入日期方面，MySQL的处理方式则比较灵活。例如，它可以把两位数的年份转换为四位数的年份，并且对于小于10的月和日，不用提供前导数字0。不过，这些值的顺序必须为“年-月-日”。对于平时已习惯性的一些格式，如'12-3-99'或'3-12-99'，解释情况有可能与你想象的样子有所不同。此时，可以借助于STR_TO_DATE()函数，把非ISO格式的字符串转换为ISO格式的日期值。例如，mytbl表拥有一个日期列date_col，你可以像下面那样插入各个值：

```
mysql> INSERT INTO mytbl (date_col)
-> VALUES (STR_TO_DATE('12-3-99', '%m-%d-%Y'));

mysql> SELECT * FROM mytbl;
```

```
-----
date_col
-----
1999-12-03
-----
```

在3.2.6.7节，会进一步讨论MySQL所使用的日期解释规则。

3.2.6.1 DATE、TIME和DATETIME数据类型

DATE和TIME类型分别用于保存日期值和时间值，而DATETIME类型则用于保存日期和时间的组合值。这 3 种类型值的格式分别是'CCYY-MM-DD'、'hh:mm:ss[.uuuuuu]'和'CCYY-MM-DD hh:mm:ss[.uuuuuu]'，其中的CC、YY、MM、DD、hh、mm、ss和uuuuuu分别代表世纪、年、月、日、时、分、秒和微秒。对于MySQL 5.6.4之前的版本，TIME和DATETIME值也有小数秒部分，只是在存储时它们会被丢弃掉。

自MySQL 5.6.5起，DATETIME列会自动把当前时间戳作为初始值，并进行更新。更多信息请参考3.2.6.6节。

如果把DATE值赋值给DATETIME列，那么MySQL会自动把时间部分补足为'00:00:00'。反方向的转换也同样有效。如果把DATETIME值赋值给DATE或TIME列，那么MySQL会把不相干的部分去掉：

```
mysql> CREATE TABLE t (dt DATETIME, d DATE, t TIME);

mysql> INSERT INTO t (dt,d,t) VALUES(NOW(), NOW(), NOW());

mysql> SELECT * FROM t;
```

dt	d	t
2013-01-09 15:33:24	2013-01-09	15:33:24

从TIME到DATETIME的转换，依赖于具体的MySQL版本：自MySQL 5.6.4起，当前日期会添加上时间。对于以前的版本，这个转换并不一定会产生有意义的结果。

在MySQL里，DATETIME类型里的时间值与TIME值略有不同。对于DATETIME类型，时间部分表示的是一天里的时间，且必须是在'00:00:00'～'23:59:59'的范围内。但是，TIME值表示的则是一段逝去的时间。这也正是表3-13里所示的TIME列的取值范围为何可以包含负值，以及为何可以大于'23:59:59'的具体原因。

当往表里插入“短”（不完整）的TIME值时，一定要小心。它们可能不会被解释成你所期望的样子。例如，把'30'和'12:30'插到某个TIME 列，那么最终的存储值为'00:00:30'和'12:30:00'。如果想把'12:30'当作“12分30秒”，那么需要以完整的形式（如'00:12:30'）来提供它。

3.2.6.2 TIMESTAMP数据类型

TIMESTAMP是一种时态数据类型，用于储存日期和时间的组合值。（单词timestamp的中文意思是“时间戳”，它的字面意思很容易让人误以为该类型只与时间有关，其实不然。）TIMESTAMP数据类型有一些特殊的属性，我们接下来就会对它们进行讨论。

TIMESTAMP 类型列的取值范围是'1970-01-01 00:00:00[.000000]'\~'2038-01-19 03:14:07[.999999]'\。与DATETIME一样，在MySQL 5.6.4之前，TIMESTAMP值也允许有小数秒部分，但在存储时会被丢弃。它的取值范围与Unix时间密切相关，其中规定1970年的第一天为“零日”，也称作“纪元”。对于每一个TIMESTAMP值，MySQL会用4个字节来把它存储为自纪元以来总共逝去的秒数。1970年的起始确定了TIMESTAMP类型的取值范围下限值。（不过请注意，TIMESTAMP的取值范围并非起始于'1970-01-01 00:00:00'。你可能会想当然地认为它就是纪元以来的那个0秒，但0表示的时间戳是'0000-00-00 00:00:00'。）取值范围上限值则与4个字节所能表示的最大Unix时间相对应。

MySQL会按世界标准时间（Universal Coordinated Time, UTC）来存储TIMESTAMP值。当保存这样的值时，服务器会把它从会话时区转换为UTC。当以后检索该值时，服务器又会把它从UTC转换回会话时区，从而让你看到与你存储结果一样的时间值。不过，如果另一个客户端使用了另一个时区去连接服务器，并检索该值，那么它所看到的值则是调整为其所设置时区的那个值。事实上，只要更改一下会话时区设置，就可以在一个会话里看到这样的效果：

```
mysql> CREATE TABLE t (ts TIMESTAMP);
```

```
mysql> SET time_zone = '+00:00';    # 将时区设置为UTC
```

```
mysql> INSERT INTO t VALUES('2000-01-01 00:00:00');
```

```
mysql> SELECT ts FROM t;
```

```
-----  
ts  
-----  
  
2000-01-01 00:00:00  
-----
```



```
mysql> SET time_zone = '+03:00';    # 将时区前调3个小时
```

```
mysql> SELECT ts FROM t;
```

```
-----  
ts  
-----  
2000-01-01 03:00:00  
-----
```

在上面示例里，指定时区所采用的方式为：相对于UTC的小时和分钟的有符号偏移量。如果服务器的时区表已按照12.6.1节所描述那样进行了设置，那么还可以使用'Europe/Zurich'这样的时区名来更改时区设置。

TIMESTAMP列会自动把当前时间戳作为初始值，并进行更新。此外，如果在定义TIMESTAMP列时为了允许存放NULL值而带有NULL属性，那么当把NULL存储到该列时，该列值会被设置为当前时间戳。更多相关信息请参考3.2.6.6节。

3.2.6.3 YEAR数据类型

YEAR是单字节数据类型，其用意在于提高年值的表示效率。在声明YEAR列时，可以指定一个显示宽度M，M值只能为4或2。如果在YEAR列的定义里忽略掉M，那么其默认值将为4。YEAR类型的取值范围是1901~2155年。如果只会用到日期里的年份，如出生年份、政府选举年份等，那么使用YEAR就足够了。如果不需要完整的日期值，那么用YEAR会比用其他日期类型更省存储空间。

YEAR(2)只显示最后两位数，并且这种类型实际只能存储从1970年到2069年之间的值。如果使用YEAR(2)来存储该范围之外的值，那么最终的显示值将不确定。例如，YEAR(2)列里存储的1970和2070都会显示为70。避免这类问题的最简单方法是避免使用YEAR(2)，用YEAR(4)来代替。由于这样的存储问题，自MySQL 5.6.6起，就废弃了YEAR(2)：在已有表里，YEAR(2)列会继续保持不变；但对于新表，这样的列会被创建为YEAR(4)。

MySQL会使用其年值判断规则，将输入的2位YEAR值转换成4位值（参考3.2.6.8节）。例如，97和14会变成1997和2014。不过请注意，当把数值00插到4位的YEAR列时，最终存储的值将会是0000，而不是2000。如果想要把值00转换为2000，那么需要以形如'0'或'00'的字符串来指定它。

TINYINT 类型的存储空间占用量与 YEAR 类型（只有1个字节）的一样，但取值范围不一样。如果想要用一个整数类型来覆盖YEAR类型所能表示的年份范围，那么需要使用SMALLINT（它将占用两倍的空间）。如果需要表示的年份包含在YEAR 类型所能表示的年份范围内，那么与使用SMALLINT相比，使用YEAR更节省空间。

3.2.6.4 时态数据类型的属性

时态列的定义可以包含通用属性 NULL 或 NOT NULL。如果都不指定，则默认为 NULL，TIMESTAMP类型除外，其默认值为NOT NULL。

也可以用DEFAULT子句来设定默认值。在3.2.3描述了在列定义没有包含DEFAULT子句时，MySQL所使用的设置默认值的规则。

大部分情况下，这些默认值都必须为常量。除了TIMESTAMP（自MySQL 5.6.5起）和DATETIME以外，你都不能使用像CURRENT_TIMESTAMP这样的函数来将DATETIME列的默认值设置为“当前日期和时间”。TIMESTAMP和DATETIME列之所以比较特殊，是因为它们的默认值可以为当前日期和时间。（更多与掌控这些类型默认值的规则相关的信息请参考3.2.6.6节。）如果其他类型想要得到这种结果，那么可以在每次创建新行时，显式地将该列的值设置为CURRENT_TIMESTAMP。另外，也可以使用TIMESTAMP 列或DATETIME 列来代替；或者设置一个触发器，让它将该列初始化为适当的值。更多相关信息请参考4.2.3节。

3.2.6.5 时态类型的小数秒功能

本节重点描述MySQL如何处理时间值里的小数秒。它主要适用于MySQL 5.6.4及以上的版本。

在TIME、DATETIME和TIMESTAMP 类型的声明语法中，允许设置一个可选的小数秒精度（fsp），精度值最高可达6位数字。fsp值必须是0~6，0表示没有小数部分，而6则表示精度为微秒。如果没有指定fsp，则默认值为0。例如，TIME和TIME(0)是等价的，都没有小数部分。DATETIME(1)允许日期和时间值精确到十分之一秒。TIMESTAMP(6)允许时间戳的精确值达到微秒级。

对于带时态参数的函数，其接受或返回的时态值中都带有小数秒部分。在某些情况下，MySQL 5.6.4之前的版本有些不带参数的函数，现在也开始接受fsp参数，以方便对可能有的返回值进行小数秒位数的控制。例如，CURTIME()返回的是不带小数秒部分的当前时间，而CURTIME(3)返回的时间则包括了一个精度高达千分之一秒的小数秒部分：

```
mysql> SELECT CURTIME(), CURTIME(3);
```

-----+-----
CURTIME() CURTIME(3)
-----+-----
12:22:31 12:22:31.475
-----+-----

在C. 2. 5节里，对各个函数的描述表明了什么时候允许使用fsp参数。

MySQL 5. 6. 4之前的版本，只提供了对微秒值的有限支持。有一些时间函数，如DATE_ADD()，使用了这些值，但是如果你想要在某个时间列里存储包含微秒部分的值，那么 MySQL 会将那个微秒部分丢弃。

3. 2. 6. 6 时态类型的自动特性

TIMESTAMP和DATETIME列可以有自动初始化属性和自动更新特性。

“自动初始化”即意味着，对于新行，如果在INSERT语句里省略了这两种类型的列，那么列会被设置为当前时间戳。

“自动更新”即意味着，对于已有的行，当把任何其他列更改为不同值时，这两种类型的列都会被更新为当前时间戳。（将列设置成它的当前值不算自动更新；这种做法实际上是在防止自动更新。）

对于MySQL 5. 6. 5之前的版本，可以为表里任何单个的TIMESTAMP列指定两种特性中的一种或全部。不能出现一个TIMESTAMP列拥有自动初始化特性，而另一个列拥有自动更新特性的情况。也不能让多个列拥有自动初始化特性，或者让多个列拥有自动更新特性。

MySQL 5. 6. 5扩展并推广了对自动特性的支持：任何TIMESTAMP列都可以有这两种特性中的一种或全部，和DATETIME列一样。

还有一个只适用于 TIMESTAMP 列的特殊特性：如果将列设置为 NULL，那么它的值会被设置为当前时间戳。要想允许TIMESTAMP列存储NULL值，在定义它时需要带有NULL属性。

请使用下列语法来指定TIMESTAMP列：

```
col_name TIMESTAMP [DEFAULT default_value] [ON UPDATE
CURRENT_TIMESTAMP]
```

如果同时指定 `DEFAULT`和`ON UPDATE` 属性，它们的顺序无关紧要。默认值可以是`CURRENT_TIMESTAMP`、像0那样的常量值，或者是格式为'`CCYY-MM-DD hh:mm:ss`'的值。也可以使用`CURRENT_TIMESTAMP`的同义词，如`NOW()`。

自MySQL 5.6.5起，`DATETIME`列便开始允许这些与`DEFAULT`和`ON UPDATE`一样的属性。在5.6.5版本之前，对于`DEFAULT`属性，`DATETIME`只允许常量，并且不支持`ON UPDATE`。

如果想让表里的第一个`TIMESTAMP` 列具有一个或两个自动特性，那么可以组合使用`DEFAULT`属性和`ON UPDATE`属性来定义它。

如果使用`DEFAULT CURRENT_TIMESTAMP`，那么列将拥有自动初始化特性。如果指定`ON UPDATE CURRENT_TIMESTAMP`，那么它还会拥有自动更新特性。

如果两种属性都没有指定，那么MySQL会将列定义为具有`DEFAULT CURRENT_TIMESTAMP`和`ON UPDATE CURRENT_TIMESTAMP`两种属性。

如果使用指定了常量值的`DEFAULT constant_value`属性，那么列将不具有自动初始化特性。如果指定`ON UPDATE CURRENT_TIMESTAMP`，那么它会有自动更新特性。

如果没有`DEFAULT`，但有`ON UPDATE CURRENT_TIMESTAMP`，那么具体的默认值为0，并且列有自动更新特性。

对于MySQL 5.6.5之前的版本，如果要想让某个`TIMESTAMP`列（而非第一列）使用自动初始化或自动更新特性，那么必须使用`DEFAULT constant_value`属性显式地定义第一列，同时不带`CURRENT_TIMESTAMP`属性。然后，对于任何其他的单个`TIMESTAMP`列，可以使用`DEFAULT CURRENT_TIMESTAMP`或`ON UPDATE CURRENT_TIMESTAMP`（或者同时使用这两者）。

自MySQL 5.6.5起，对于任何`TIMESTAMP`列，可以自由地使用这两个属性当中的一个，也可以两个都用。此外，`DATETIME`列也可以有这些属性。

如果不想让拥有自动初始化特性或自动更新特性的`TIMESTAMP`或`DATETIME`列，受到这些特性的影响，那么在执行插入或更新操作时，可以显式地把列设置成所期望的值。例如，为防止在更改列时自动更新，可以将列设置成它的当前值。

`TIMESTAMP`和`DATETIME`列的定义也可以包含`NULL`或`NOT NULL`属性。`TIMESTAMP`的默认属性是 `NOT NULL`。这会产生一种特殊效果，即当你把列显式地设置成 `NULL` 时，MySQL会将它设置成当前时间戳。（对于插入和更新操

作，都是如此。）如果在列的定义里指定NULL，那么将该列设置为NULL时，存储的将是NULL，而不是当前时间戳。DATETIME的默认属性是NULL，并且在把DATETIME列设置为NULL时，没有产生特殊效果。

下面来看一个表，它含有一列。在新增行时，列会被设置成当前时间戳，并且之后不会被自动更新：

```
CREATE TABLE t1

(

    ts_created TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    ...other columns ...

);
```

如果要新建一个行，则需要通过以下方式将列初始化为当前时间戳：将列设置为NULL，或者在INSERT语句里省略它。对于后续的更新操作，列会保持它的价值，一直到显式更改它为止。

再来看一个包含两个列的表，这两个列分别用于存储创建时间和最后修改时间，它们都是TIMESTAMP列：

```
CREATE TABLE t2

(

    ts_created TIMESTAMP DEFAULT 0,

    ts_modified TIMESTAMP DEFAULT CURRENT_TIMESTAMP

ON UPDATE CURRENT_TIMESTAMP,

    ...other columns ...

);
```

如果要插入一个新行，则需要把这两个TIMESTAMP 列都设置为 NULL，从而将它们设置成当时的插入时间戳。如果要更新已有的行，那么可以不用管这两列。如果任何其他列的值发生变化，那么ts_modified会被自动更新为当时的修改时间戳。

3. 2. 6. 7 处理时态值

MySQL可以正确解释各种格式的日期和时间列的输入值，其中包括字符串形式和数值形式。此外，自MySQL 5.6.4起，TIME、DATETIME和TIMESTAMP数据类型，还可以带有一个可选的精度高达6位数字（微秒）的小数秒部分。例如，DATETIME和TIMESTAMP值支持的格式有：

`'CCYY-MM-DD hh:mm:ss[.uuuuuu]'`

`'YY-MM-DD hh:mm:ss[.uuuuuu]'`

`'CCYYMMDDhhmmss[.uuuuuu]'`

`'YYMMDDhhmmss[.uuuuuu]'`

`CCYYMMDDhhmmss[.uuuuuu]`

`YYMMDDhhmmss[.uuuuuu]`

对于DATE、TIME和YEAR类型值，也有类似的表示。

MySQL也可以解释那些没有世纪部分（即CC）的MySQL也可以解释那些没有世纪部分（即CC）的日期和时间格式，所用规则请参考3.2.6.6节。对于带分隔符的字符串格式，日期里可以不用“-”，时间里可以不用“:”。任何一种标点符号都可以用作分隔符。对这些值的解释，依据的是其上下文，而非分隔符。例如，虽然人们习惯于把“:”当作时间值的分隔符，但是在期望是日期值的场合，MySQL也不会把一个包含“:”的值解释为具体的时间。此外，对于包含有分隔符的字符串格式，当月、日、时、分或秒的值小于10时，可以不用指定2位数字。例如，下面这些值都是等价的。

`'2012-02-03 05:04:09'`

`'2012-02-03 05:04:9'`

`'2012-02-03 05:4:9'`

`'2012-02-03 5:4:9'`

`'2012-02-3 5:4:9'`

`'2012-2-3 5:4:9'`

对于带有前导零的日期和时间值，MySQL有多种不同的解释方式，具体取决于这些值是以字符串形式指定的，还是以数字形式指定的。例如，字符串'001231'会被看作是一个有6位数字的值，并且如果将其当作 DATE 类型，那么它会被解释成'2000-12-31'；而如果将其当作DATETIME 类型，那么它会被解释成'2000-12-31 00:00:00'。另外，在解析器将其处理成数字之后，数字001231会被看作1231，于是，对它的解释就会变得不确定。此时，最好是提供一个字符串值'001231'。如果的确需要使用数字，则需要使用完整形式（对于 DATE 类型，即为20001231；而对于DATETIME类型，即为200012310000）。

一般情况下，可以在DATE、DATETIME和TIMESTAMP类型之间随意地相互赋值，但也请记住，存在以下这样几条限制。

如果把DATETIME或TIMESTAMP值赋给DATE列，则其中的时间部分会被丢失。

如果把DATE值赋值给DATETIME或TIMESTAMP列，则结果值的时间部分将会被设置为零（即'00:00:00'）。

各个类型的取值范围有所不同。尤其是 TIMESTAMP 类型，其取值范围仅限于1970～2038。因此，不能把一个早于1970年的DATETIME值赋给TIMESTAMP列。如果强行那样做，最终的结果也不会是你所期望的那样。同样，也不能把遥远未来的值赋给TIMESTAMP列。

MySQL提供了很多用于处理日期和时间值的函数。更多相关信息请参考附录C。

3.2.6.8 解释模糊年份值

对于所有那些带有年份值的日期和时间类型（包括DATE、DATETIME、TIMESTAMP和YEAR），MySQL在处理那些包含2位数字的年份值时，会把它们转换成4位数字。

年份在00～69之间的值，会被转换为2000～2069。

年份在70～99之间的值，会被转换为1970～1999。

把各种不同的2位数字值存储到YEAR(4)列，然后再把它们检索出来，便可以看到这些转换规则的实际效果：

```
mysql> CREATE TABLE y_table (y YEAR(4));
```

```
mysql> INSERT INTO y_table VALUES(68),(69),(99),(00),('00');
```

```
mysql> SELECT * FROM y_table;
```

```
-----  
y  
-----  
2068  
2069  
1999  
0000  
2000  
-----
```

前面这个示例也展示了某些应该引起注意的地方：00会被转换为0000，而不是2000。如果把数字 0 插到 YEAR(4) 列，得到的也将是 0000。如果要利用某个没有包含世纪部分的值得到2000年的表示结果，那么必须使用字符串 '0' 或 '00'。如果想确保MySQL看到的是字符串，而非数字，则可以使用 `CAST(value AS CHAR)` 函数来插入YEAR值，不管value是字符串，还是数字，其结果都一律是字符串。

需要注意的是，把2位数字年份值转换为4位数字年份值的转换规则，只是提供了一种比较合理的猜测。当未指定世纪部分时，MySQL并不知道你的2位数字年份具体指的是哪一年。MySQL的年份转换规则适用于很多情形，但如果它们产生的结果与你预期的不一致，那就必须得提供一个无歧义的4位数字年份值。例如，想把自18世纪以来的美国总统的出生日期和去世日期都录到 `president` 表里，那么必须使用4位数字的年份值。这两个列的值跨越了好几个世纪，因此让MySQL根据2位数的年份值去猜测它们属于哪个世纪，绝非明智之举。

3.3 MySQL如何处理无效数据值

在过去，MySQL处理数据的基本原则是“垃圾进来、垃圾出去”。换句话说，你给MySQL什么样的数据，它就会存储什么样的数据。但是，如果在存储数据时没有对它们进行验证，那么在把它们检索出来时得到的就不一定是你所期望的内容。还好，有几种SQL模式可以让你拒绝“坏”值，并且会在遇到“坏”值时抛出一个错误，这种行为与其他数据库系统的很像，如果你对其

他数据库系统比较熟悉的话。接下来将首先讨论 MySQL 默认是如何处理非正常数据的，然后会讨论当启用各种SQL模式时会对数据处理产生哪些影响。

默认情况下，MySQL会按照以下规则处理越界（即超出取值范围的）值和其他非正常值。

对于数值列或TIME列，超出合法取值范围的那些值将被截断到取值范围最近的那个端点，并把结果值存储起来。

对于除 TIME 以外的其他时态类型列，非法值会被转换成与该类型相一致的“零”值（见表3.15）。

对于字符串列（不包括ENUM或SET），过长的字符串将被截断到该列的最大长度。

给ENUM或SET类型列进行赋值时，需要根据列定义里给出的合法取值列表进行。如果把不是枚举成员的值赋给ENUM列，那么列的值会变成“出错”成员（即与零值成员相对应的空字符串）。如果把包含非集合成员的子字符串的值赋给SET列，那么这些字符串会被清理，剩余的成员才会被赋值给列。

如果在执行INSERT、REPLACE、UPDATE、LOAD DATA和ALTER TABLE等语句时发生了上述转换，那么 MySQL 会给出警告消息。在执行完其中的某一条语句之后，便可以使用SHOW WARNINGS语句来查看这种警告消息的内容。

如果需要在插入或更新数据时执行更严格的检查，那么可以启用以下两种SQL模式中的一种：

```
mysql> SET sql_mode = 'STRICT_ALL_TABLES';
```

```
mysql> SET sql_mode = 'STRICT_TRANS_TABLES';
```

对于支持事务的表，这两种模式都是一样：如果发现某个值无效或缺失，那么结果会产生一个错误，语句会中止执行，并进行回滚，就像什么事都未发生过一样。对于不支持事务的表，这两种模式有以下效果。

对于这两种模式，如果在插入或修改第一个行时，发现某个值无效或缺失，那么结果会产生一个错误，语句会中止执行，就像什么事都未发生过一样。这跟事务表的行为很相似。

在用于插入或修改多个行的语句里，如果在第一行之后的某个行出现了错误，那么会出现某些行被修改的情况。这两种严格模式决定着，这条语句此时此刻是要停止执行，还是要继续执行。

在STRICT_ALL_TABLES 模式下，会抛出一个错误，并且语句会停止执行。因为受该语句影响的许多行都已被修改，所以这将会导致“部分更新”问题。

在STRICT_TRANS_TABLES 模式下，对于非事务表，MySQL会中止语句的执行，只有这样做，才能达到事务表那样的效果。只有当第一个行发生错误时，才能达到这样的效果；如果错误在后面的某个行上，那么就会出现某些行被修改的情况。由于对于非事务表，那些修改是无法撤销的，因此MySQL会继续执行该语句，以避免出现“部分更新”的问题。它会把所有的无效值转换为与其最接近的合法值，如本节之前所定义的那样。对于缺失的值，MySQL会把该列设置成其数据类型的隐式默认值，如3.2.3节所述。

严格模式实际上并不是 MySQL 能够执行的最严格检查。通过以下模式当中的任何一个或全部，都可以对输入数据进行更加严格的检查。

ERROR_FOR_DIVISION_BY_ZERO：在严格模式下，如果遇到以零为除数的情况，它会阻止数值进入数据库。（如果不在严格模式下，那么会产生一条警告消息，并插入NULL。）

NO_ZERO_DATE：在严格模式下，它会阻止“零”日期值进入数据库。

NO_ZERO_IN_DATE：在严格模式下，它会阻止月或日部分为零的不完整日期值进入数据库。

例如，想让所有的存储引擎启用严格模式，并对“被零除”错误进行检查，那么可以像下面这样设置SQL模式：

```
mysql> SET sql_mode =  
'STRICT_ALL_TABLES,ERROR_FOR_DIVISION_BY_ZERO';
```

如果想启用严格模式，以及所有的附加限制，那么最为简单的办法是启用TRADITIONAL模式：

```
mysql> SET sql_mode = 'TRADITIONAL';
```

TRADITIONAL模式的含义是“启用两种严格模式，外加一大堆的其他限制”。这与其他“传统的”SQL DBMS在数据检查方面的行为比较接近。

也可以选择性地在某些方面弱化严格模式。如果启用了SQL的ALLOW_INVALID_DATES模式，那么MySQL将不会对日期部分做全面检查。相反，它只会要求月份值在1~12，而天数处于1~31（即允许像'2000-02-30'或'2000-06-31'这样的无效值）。另一个制止错误的办法是，在INSERT或

UPDATE语句里使用IGNORE关键字。有了IGNORE之后，那些会因无效值而导致错误的语句，将只会导致警告的出现。

这些选项能让你灵活地为你的应用选择正确的有效性检查级别。

3.4 处理序列

为达到标识的目的，许多应用都需要生成唯一编号，如成员编号、样品编号或批号、顾客ID、bug报告或故障报告表标签等。

MySQL提供唯一编号的机制是使用MySQL提供唯一编号的机制是使用AUTO_INCREMENT列属性——它会自动生成序列编号。不过，MySQL所支持的那些存储引擎，处理AUTO_INCREMENT列的方式有所不同。下面的讨论将描述：在一般情况下，AUTO_INCREMENT列是如何工作的；对于特定的存储引擎，它又是如何工作的。有了这些知识，你就可以高效地使用它们，而不会遭遇那些让人感到意外的陷阱。下面还会对“不使用AUTO_INCREMENT列的情况下，如何生成序列”进行讨论。与MySQL里可用存储引擎有关的更多信息请参考2.6.1节。

3.4.1 通用的AUTO_INCREMENT属性

AUTO_INCREMENT列必须按照以下条件进行定义。

每个表只能有一个列具有AUTO_INCREMENT属性，并且它应该为整数数据类型。（AUTO_INCREMENT也支持浮点类型，但很少那样使用。）

列必须建立索引。最常见的情况是使用PRIMARY KEY或UNIQUE索引，但是也允许使用不唯一的索引。

列必须拥有NOT NULL约束条件。即使没有显式地这样声明，MySQL也会自动把列设置为NOT NULL。

在创建之后，AUTO_INCREMENT列将具有以下行为。

把NULL值插入AUTO_INCREMENT列将引发MySQL自动生成下一个序列编号，并把它插入列。AUTO_INCREMENT序列通常是从1开始，并依次单步递增，因此连续插入表的各行的序号值将为1、2、3，如此等等。在某些场合，根据所用存储引擎的不同，可以显式地设置或重置下一个序号，或者重复使用已从序列顶端删除的那些序号值。

要获得最近生成的序号值，可以调用LAST_INSERT_ID()函数。这样，即使在你根本不知道AUTO_INCREMENT值到底是多少的时候，也可以在后续的语言

句里引用它。如果在当前会话里，还没有生成过AUTO_INCREMENT值，那么LAST_INSERT_ID()将返回0。

LAST_INSERT_ID()只会依赖于与服务器的当前会话连接所生成的AUTO_INCREMENT值。你可以生成一个序号，接着在同一个会话连接里调用LAST_INSERT_ID()来检索它。即使其他客户在此期间生成了它们自己的序号值，也不会出现任何问题。

一次插入多个行的INSERT语句，将生成多个AUTO_INCREMENT值，LAST_INSERT_ID()只会返回其中的第一个。

如果使用INSERT DELAYED，那么要直到实际插入行时，才会生成AUTO_INCREMENT值。这时，就不能依靠LAST_INSERT_ID()来返回序号值了。

在插入一行时，如果不为AUTO_INCREMENT列指定值，则等同于向该列插入一个NULL值。如果ai_col是一个AUTO_INCREMENT列，那么下面这两条语句就是等效的：

```
INSERT INTO t (ai_col,name) VALUES(NULL,'abc');
```

```
INSERT INTO t (name) VALUES('abc');
```

默认情况下，把0插入AUTO_INCREMENT列，等效于插入NULL值。如果启用了SQL的NO_AUTO_VALUE_ON_ZERO模式，那么插入0则会导致存储值为0，而非下一个序号值。

如果要插入一行，并为某个拥有唯一索引的AUTO_INCREMENT列指定一个既不为NULL，也不为0的值，那么将发生这样两种情况中的某一种。如果已存在一行使用该值的记录，那么将会发生键重复错误。否则，这行会被正常插入，而那个AUTO_INCREMENT列会被设置为那个给定值。如果该值大于当前的下一个序号值，那么这个序列将被重置，对于随后的行，所使用的编号会继续由该值开始生成。换句话说，即可以插入一行，让其序号值大于当前计数器值，以此达到增大计数器的目的。

对于某些存储引擎，从序列顶端删除的值可以被重用。如果把包含最大AUTO_INCREMENT列值的那行删除，那么在下次生成新值时可以重用这个最大值。如果把表里的所有记录都删除，那么所有值都可以重用，并且这个序列会重新从1开始。

如果使用UPDATE命令把AUTO_INCREMENT列的值设置成某个正被其他行使用的值，并且这个列拥有唯一索引，那么会出现一个键重复错误。如果把这个列设置成某个大于所有已有列值的编号值，那么对于随后的行，这个序列

从那个值开始继续生成下一个编号。如果把该列更新为 0，那么它会被设置为 0（不管是否启用了 NO_AUTO_VALUE_ON_ZERO都一样）。

如果根据 AUTO_INCREMENT 列的值，使用REPLACE 来更新行，那么这个行的AUTO_INCREMENT值将保持不变。如果根据其他具有PRIMARY KEY或UNIQUE索引的列的值，使用REPLACE命令来更新行，那么当你把AUTO_INCREMENT列设置为NULL，或者把它设置为0，而且没有启用NO_AUTO_VALUE_ON_ZERO 时，该列的值将被更新为一个新的序号值。

3.4.2 存储引擎特有的AUTO_INCREMENT属性

刚才介绍的通用AUTO_INCREMENT特征，是理解各种存储引擎特有序列行为的基础。绝大多数引擎实现的行为与上面的描述大体是一致的，因此大家在继续往前阅读时，请一定要好好记住刚才的讨论。MyISAM为序列处理提供了最大的灵活性，因此这里的讨论便由该引擎开始。

3.4.2.1 MyISAM表的AUTO_INCREMENT列

MyISAM存储引擎拥有以下AUTO_INCREMENT特征。

MyISAM 表里的序列一般是单调的。在一个自动生成的序列里，这些值都是严格递增的；并且在行被删除之后，不会被重用。如果最大值是 143，而你删除了包含这个值的行，那么MySQL生成的下一个值仍然是144。不过，这种行为存在以下两种例外情况。

如果使用TRUNCATE TABLE清空了表，那么计数器将被重置为从1开始。

如果在表里使用了复合索引来生成多个序列，那么从序列顶端删除的那些值将被重用。（此项技术随后将被讨论到。）

MyISAM 序列将默认从 1 开始，不过你可以在CREATE TABLE 语句里，通过AUTO_INCREMENT = n选项显式地指定初始值。下面这个示例会创建一个MyISAM表，其中带有一个名为seq的AUTO_INCREMENT列，其起始值为1 000 000：

```
CREATE TABLE mytbl  
(  
    seq INT UNSIGNED NOT NULL AUTO_INCREMENT,  
    PRIMARY KEY (seq)
```

```
) ENGINE = MYISAM AUTO_INCREMENT = 1000000;
```

一个表只会有一个AUTO_INCREMENT列，因此末尾的那个AUTO_INCREMENT = n选项会应用到哪一列，绝不会引起歧义。

可以使用ALTER TABLE来更改某个已有MyISAM表的当前序列计数器。例如，序列的当前编号为1000，那么下面这条语句将使下一个生成的编号值变为2000：

```
ALTER TABLE mytbl AUTO_INCREMENT = 2000;
```

如果想要重用那些从序列顶端删除了的值，你也可以用这个语句来设置。把编号计数器设置到最低，使得下一个编号只比当前最大序号值多1：

```
ALTER TABLE mytbl AUTO_INCREMENT = 1;
```

不能使用AUTO_INCREMENT选项来把当前计数值设置得，比表里当前的最大计数值还小。

如果某个AUTO_INCREMENT列包含的值为1~10，那么使用AUTO_INCREMENT = 5来设置这个计数器，得到的下一个自动值仍然为11，而非5。

MyISAM存储引擎支持在同一个表里使用复合（多列）索引，以创建多个相互独立的序列。为利用这个特性，为表创建一个由多列组成的PRIMARY KEY或UNIQUE索引，并把包含AUTO_INCREMENT的那个列作为其中的最后一个。对于该索引最左边的列构成的每一个相异键，AUTO_INCREMENT列将生成一组彼此互不干扰的序列值。例如，有一个名为bugs的表，你需要用它来同时记录多个软件项目的bug报告，该表的定义如下：

```
CREATE TABLE bugs
(
    proj_name VARCHAR(20) NOT NULL,
    bug_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
    description VARCHAR(100),
    PRIMARY KEY (proj_name, bug_id)
) ENGINE = MYISAM;
```

其中，proj_name列用于标识项目名，description列用于存放bug描述。bug_id列是一个AUTO_INCREMENT列，通过创建一个与proj_name列相关联的索引，可以为各个项目分别生成一个互不干扰的序列编号。假设，需要把下面几行输到表里，记录SuperBrowser项目的3个bug，以及SpamSquisher项目的2个bug：

```
mysql> INSERT INTO bugs (proj_name,description)
```

```
-> VALUES('SuperBrowser','crashes when displaying complex  
tables');
```

```
mysql> INSERT INTO bugs (proj_name,description)
```

```
-> VALUES('SuperBrowser','image scaling does not work');
```

```
mysql> INSERT INTO bugs (proj_name,description)
```

```
-> VALUES('SpamSquisher','fails to block known blacklisted  
domains');
```

```
mysql> INSERT INTO bugs (proj_name,description)
```

```
-> VALUES('SpamSquisher','fails to respect whitelist  
addresses');
```

```
mysql> INSERT INTO bugs (proj_name,description)
```

```
-> VALUES('SuperBrowser','background patterns not displayed');
```

表的最终内容如下所示：

```
mysql> SELECT * FROM bugs ORDER BY proj_name, bug_id;
```

proj_name	bug_id	description
SpamSquisher	1	fails to block known blacklisted domains
SpamSquisher	2	fails to respect whitelist addresses
SuperBrowser	1	crashes when displaying complex tables
SuperBrowser	2	image scaling does not work
SuperBrowser	3	background patterns not displayed

这个表为每一个项目的bug_id 值进行了单独的编号，整个过程与这些项目的行输入顺序毫无关系。在输入另一个项目的行之前，你不用先全部输入某个项目的所有行。

如果使用复合索引来创建多个序列，那么从各个序列顶端删除的值都可以被重用。这一点与MyISAM表的不重用序号值的行为有所不同。

3.4.2.2 InnoDB表的AUTO_INCREMENT列

InnoDB存储引擎拥有以下AUTO_INCREMENT特征。

在CREATE TABLE语句里，可以使用AUTO_INCREMENT = n表选项来设置初始序列值，并且在表创建之后，还可以使用ALTER TABLE来进行更改。

从序列顶端删除的值通常不能再重用。不过，如果使用TRUNCATE TABLE清空表，那么序列将被重置，并重新从 1 开始编号。此外，在满足后面几个条件时也可以重用。首先，在首次为AUTO_INCREMENT列生成序号值时，InnoDB会把列的当前最大值加上1，得到的结果作为新生成的序号值（如果表此前为空，那么新的序号值为1）。其次，为满足生成后续序号值的需要，InnoDB 是在内存里维护这个计数器——它并未存储在表内部。这意味着，如果从这个序列的顶端删除了某些值，然后又重启了服务器，那么这些删除的值将会被重用。重启服务器还将取消在CREATE TABLE或ALTER TABLE语句里使用AUTO_INCREMENT表选项所带来的效果。

如果生成AUTO_INCREMENT值的事务被回滚，那么在序列里可能会出现断裂。

在表里，不能使用复合索引生成多个独立的序列。

3.4.2.3 MEMORY表的AUTO_INCREMENT列

MEMORY存储引擎拥有以下AUTO_INCREMENT特征。

在CREATE TABLE语句里，可以使用AUTO_INCREMENT = n表选项来设置初始序列值，并且在表创建之后，还可以使用ALTER TABLE来进行更改。

从序列顶端删除的值通常不能再重用。不过，如果使用TRUNCATE TABLE清空表，那么序列将被重置，并重新从1开始编号。

在表里，不能使用复合索引生成多个独立的序列。

3.4.3 使用AUTO_INCREMENT列需要考虑的问题

在使用AUTO_INCREMENT列时，为避免陷入不必要的麻烦，请牢记以下几点。

AUTO_INCREMENT机制的主要用途是生成一个正整数序列。AUTO_INCREMENT列不支持使用非正数。因此，我们还可以把AUTO_INCREMENT列定义为UNSIGNED类型。对于整型列，使用UNSIGNED还有一个好处：在达到该数据类型的范围上限前，可以获得两倍的序列编号。

把 AUTO_INCREMENT 添加到列的定义里，并不能得到无穷尽的序列编号。AUTO_INCREMENT序列总是会受到底层数据类型的取值范围的约束。例如，如果你使用的是一个TINYINT列，那么序列编号的最大值即为127。一旦达到这个上限，应用程序便会因“键重复”错误而失败。如果使用的是TINYINT UNSIGNED，那么序列编号的上限会扩充到255，不过它还是很有限。

使用TRUNCATE TABLE语句来清除某个表的内容，可以把该表的计数序列重置为重新从1开始。即使对于那些通常不会重用AUTO_INCREMENT值的存储引擎，也是如此。之所以会重置序列，是因为MySQL需要对整表的删除操作进行优化。只要有可能，它会快速丢弃全部的行和索引，并且从头开始重建该表，而不是一次删除一行。这样做将导致序列编号信息出现丢失。如果想在删除所有行的同时把序列信息保留下来，那么可以使用带有一个永远为真的WHERE子句的DELETE语句来阻止这个优化操作的执行。该语句会迫使MySQL针对每行计算一次条件，然后单独地删除每行：

```
DELETE FROM tbl_name WHERE TRUE;
```

3.4.4 AUTO_INCREMENT列的使用提示

本节将讨论使用AUTO_INCREMENT列时可以用到的一些实用技术。

3.4.4.1 为表增加一个序列编号列

假设，你创建并填充了一个表：

```
mysql> CREATE TABLE t (c CHAR(10));
```

```
mysql> INSERT INTO t VALUES('a'),('b'),('c');
```

```
mysql> SELECT * FROM t;
```

```
+-----+
```

```
| c      |
```

```
+-----+
```

```
| a      |
```

```
| b      |
```

```
| c      |
```

```
+-----+
```

接着，你想要为这个表增加一个序列编号列。为做到这一点，可以执行 ALTER TABLE 语句来增加一个 AUTO_INCREMENT 列，所定义的类型与 CREATE TABLE 语句里使用的类型一样。MySQL 会自动把序列值赋给 AUTO_INCREMENT 列：

```
mysql> ALTER TABLE t ADD i INT UNSIGNED NOT NULL  
AUTO_INCREMENT PRIMARY KEY;
```

```
mysql> SELECT * FROM t;
```

c	i
a	1
b	2
c	3

3.4.4.2 重置已有列的序列编号

如果表已经有了一个AUTO_INCREMENT列，但是你想要对其进行重新编号，以便消除因删除行而在序列里产生的断裂。实现这一目标的最简单办法是：先删除该列，然后再重新添加它。当MySQL添加列时，会自动分配新的序列号。

假设，有这样一个表t，其中的i为一个AUTO_INCREMENT列：

```
mysql> CREATE TABLE t (c CHAR(10), i INT UNSIGNED
AUTO_INCREMENT
```

```
-> NOT NULL PRIMARY KEY);
```

```
mysql> INSERT INTO t (c)
```

```
-> VALUES ('a'), ('b'), ('c'), ('d'), ('e'), ('f'), ('g'), ('h'), ('i'),
('j'), ('k');
```

```
mysql> DELETE FROM t WHERE c IN('a','d','f','g','j');
```

```
mysql> SELECT * FROM t;
```

-----+-----		
c		i
-----+-----		
b		2
c		3
e		5
h		8
i		9
k		11
-----+-----		

下面的ALTER TABLE语句将依次删除列，再重加它，重回的过程中会对列重新编号：

```
mysql> ALTER TABLE t
```

```
-> DROP PRIMARY KEY,
```

```
-> DROP i,
```

```
-> ADD i INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
```

```
-> AUTO_INCREMENT = 1;
```

```
mysql> SELECT * FROM t;
```

-----+-----	
c	i
-----+-----	
b	1
c	2
e	3
h	4
i	5
k	6
-----+-----	

AUTO_INCREMENT = 1子句将把序列的起始编号重置为1。对于MyISAM、MEMORY或InnoDB表，可以把序列的起始编号设置为不等于1的其他数。对于其他的存储引擎，则可以忽略AUTO_INCREMENT子句，因为它们不允许使用这样的方式设定序列的初始值。这个序列将从1开始编号。

虽然可以轻易地重新设置列的编号，但是“要怎么才能做到呢？”这个问题仍然很常见，而且通常没有必要这么做。MySQL并不会在意序列里是否存在有断裂，并且在重置序列编号后也不会获得任何性能上的提高。此外，如果在另一个表里有行引用了AUTO_INCREMENT列里的值，那么调整该列的编号将破坏这两个表之间的对应关系。

3.4.5 在无AUTO_INCREMENT的情况下生成序列

MySQL支持一种生成序列编号的办法，它根本不需要使用AUTO_INCREMENT列。事实上，它使用的是 LAST_INSERT_ID() 函数的另一种带参数的形式。如果在插入或修改一个列时使用了LAST_INSERT_ID(expr) 函数，那么下次不带参数调用LAST_INSERT_ID() 时，它会返回表达式expr的值。换句话说，MySQL会把表达式expr当作是一个新生成的AUTO_INCREMENT值。因此，你可以先创建一个序列编号，然后在随后的会话中检索它，同时不用担心这个编号值会受到其他客户端程序活动的影响。

这种策略的一种用途是，创建一个只有一个行的表，其中包含一个在每次需要该序列里下一个值时都会进行更新的值。例如，可以像下面这样创建和初始化这个表：

```
CREATE TABLE seq_table (seq INT UNSIGNED NOT NULL);
```

```
INSERT INTO seq_table VALUES(0);
```

上面这些语句会创建一个只有一个行的seq_table表，其中seq的值为0。为生成下一个序列编号，并对它进行检索，可以这样做：

```
UPDATE seq_table SET seq = LAST_INSERT_ID(seq+1);
```

```
SELECT LAST_INSERT_ID();
```

上面的UPDATE语句将检索seq列的当前值，并把它加上1，从而产生该序列的下一个编号。利用LAST_INSERT_ID(seq+1)生成的新编号值，与AUTO_INCREMENT 值很相像，因此可以通过不带参数调用LAST_INSERT_ID() 的方式来检索它。LAST_INSERT_ID() 函数是客户端专用的，因此即使在UPDATE和SELECT两个操作之间的时间间隔里，其他客户端程序又生成多个序列编号，你也会检索到正确值。

利用这种方法，可以生成步长不为 1（甚至可以为负值）的序列编号。例如，反复执行下面这条语句，将生成一个步长为100的序列：

```
UPDATE seq_table SET seq = LAST_INSERT_ID(seq+100);
```

反复执行下面这条语句将生成一个递减序列：

```
UPDATE seq_table SET seq = LAST_INSERT_ID(seq-1);
```

你也可以生成一个从任意值开始编号的序列，只要给seq列设置一个合适的初始值就行。

前面的讨论描述了如何利用一个只包含一行的表来建立一个计数器。如果需要多个计数器，则可以这样做：首先给这个表增加一列，用作计数器的标识符；然后在这个表里为每个计数器单独增加一行。假设，你有一个网站，想要实现“本网页已被访问过 n 次”的页计数器。为此，创建一个具有两个列的表：一列用于保存计数器的唯一标识；而另一列则用于保存计数器的当前值。你仍可以使用LAST_INSERTED_ID()函数，但需要根据计数器的名字来确定它应该应用于哪一行。例如，可以用下面的语句来创建这样一个表：

```
CREATE TABLE counter

(

    name VARCHAR(255) CHARACTER SET latin1 COLLATE latin1_general_cs
    NOT NULL,

    value INT UNSIGNED,

    PRIMARY KEY (name)

);
```

其中，name列是一个字符串，因此你可以把计数器命名为你想要的任何名字。同时，为了避免计数器的名字出现重复，它还被定义成了PRIMARY KEY。这里假定使用这个表的应用程序，对它们所使用的名字有所了解。就Web计数器而言，可以使用每个页面在文档树里的路径名作为计数器名，以此保证计数器名的唯一性。name 列的排序规则是区分大小写的，所以路径名的值也是区分大小写的。如果你的系统不区分路径名的大小写，则需要使用不区分大小写的排序规则。

在使用counter表时，INSERT ... ON DUPLICATE KEY UPDATE语句会非常有用，因为它既可以为一个未曾统计过的页面插入一个新行，也可以更新已有页面的计数值。此外，通过使用LAST_INSERT_ID(expr)函数来生成计数值，你还可以在更新当前计数器后，轻易地将其值检索出来。例如，为了初始化或递增网站主页的计数器，然后检索计数器显示结果，可以像下面这样做：

```
INSERT INTO counter (name, value)
```

```
VALUES('index.html', LAST_INSERT_ID(1))
```

```
ON DUPLICATE KEY UPDATE value = LAST_INSERT_ID(value+1);
```

```
SELECT LAST_INSERT_ID();
```

在不使用LAST_INSERT_ID()函数的情况下，另一种递增已有页面计数器的方法是：

```
UPDATE counter SET value = value+1 WHERE name = 'index.html';
```

```
SELECT value FROM counter WHERE name = 'index.html';
```

不过，在执行UPDATE语句之后，且在执行SELECT语句之前，如果有另一个客户端递增了这个计数器，那么这种方法就无法正确地工作。可以通过LOCK TABLE和UNLOCK TABLE把这两条语句围起来，解决此问题。或者，选用支持事务处理的存储引擎来创建这个表，并且在一个事务里更新这个表。这两种办法都可以在你使用这个计数器时，阻断其他客户端，但是使用LAST_INSERT_ID()函数可以更容易地完成这件事情。因为计数器的值是客户端专用的，所以你总是能获得你刚插入的那个值，而不会获得来自其他客户端插入的值；并且，你不用为了达到排斥其他客户端的目的，而使用表锁定或事务来把代码弄得很复杂。

3.5 表达式计算和类型转换

表达式包含子项和运算符，并且经过计算可以产生具体值。子项可以包含值，如常数、函数调用、表列引用，以及标量子查询。这些值可以用不同种类的运算符（如算术运算符或比较运算符）组合在一起，表达式的子项可以用括号进行分组。绝大部分情况下，表达式都是出现在输出列的列表里，以及SELECT语句的WHERE子句里。例如，下面有一个查询，它与1.4.9.6节里用来计算年龄的那个查询非常相似：

```
SELECT
```

```
  CONCAT(last_name, ', ', first_name),
```

```
  TIMESTAMPDIFF(YEAR, birth, death)
```

```
FROM president
```

```
WHERE
```



```
birth > '1900-1-1' AND DEATH IS NOT NULL;
```

每一个选择值都代表着一个表达式，与 WHERE 子句的内容一样。表达式也可以出现在DELETE和UPDATE语句的WHERE子句里、INSERT语句的VALUE () 子句里等。

当MySQL遇到表达式时，它会计算这个表达式，并产生一个结果。例如，表达式 (4 * 3) DIV (4-2) 的计算结果为6。表达式的计算过程可能涉及类型转换。例如，当数字用在需要DATE值的上下文中时，MySQL会把960821这样的数字转换成日期值 '1996-08-21'。

本节将讨论如何在MySQL 里编写表达式，还将讨论在表达式计算过程中掌控各种类型转换的规则。这里列出了MySQL的所有运算符，但因为MySQL的函数实在是太多，所以这里只涉及其中的小部分。更多相关信息请参考附录C。

3.5.1 编写表达式

表达式可以很简单，甚至允许只为一个常量，如数字0和字符串 'abc'。

表达式可以使用函数调用。有些函数带有实参（即括号里的值），有些则没有。函数的多个实参之间应该用逗号隔开。在调用内建函数时，它的参数之间允许出现空格。但在函数名和紧随其后的左括号之间，不允许出现空格，因为 MySQL的解析器可能会错误地解释函数名。常见的后果即是报语句出错。不过，通过启用SQL的IGNORE_SPACE模式，可以让MySQL允许内建函数名的后面出现空格，但这又会造成把函数名视为保留字的问题。

表达式里可以包含对表列的引用。对于最简单的情形，当可以根据上下文清楚地知道列所属的那个表时，只需给出该列的名字即可。在下面的两条SELECT 语句里，由于它们都只用到了一个表，所以尽管这两条语句里的列名完全一样，但这些列引用不会产生歧义：

```
SELECT last_name, first_name FROM president;
```

```
SELECT last_name, first_name FROM member;
```

如果还不清楚应该使用哪个表，则需要在列名的前面加上正确的表名进行限定。如果无法知道应该使用哪一个数据库，则需要在表名前面加上数据库名进行限定。即使在不会产生歧义的上下文里，也可以使用这种更为具体的限定形式，以便看起来更加明确：

```
SELECT
```

```
president.last_name, president.first_name,  
  
member.last_name, member.first_name  
  
FROM president INNER JOIN member  
  
WHERE president.last_name = member.last_name;  
  
SELECT sampdb.student.name FROM sampdb.student;
```

关于限定符的更多信息请参考2.2节。

在表达式里，标量子查询可以用于提供单个值。这种子查询必须用括号括起来：

```
SELECT * FROM president WHERE birth = (SELECT MAX(birth) FROM  
president);
```

最后，可以根据所有这些类型的值（常量、函数调用、列引用和子查询）组合成各种更加复杂的表达式。

3.5.1.1 运算符类型

可以使用各种运算符把表达式的各个子项组合在一起。本节将描述这些运算符的功能，下一节将讨论它们的计算优先级。

如表3-16所示，算术运算符包括常见的加法、减法、乘法、除法和取模。当两个操作数都是整数时，对于运算符+、-和*，整个运算过程所使用的都是BIGINT（64位）整数值。如果两个操作数都是整数，且其中有一个是无符号数，那么结果也将是无符号数。对于除DIV以外的所有运算符，如果某个操作数是近似值，那么整个计算过程会遵循双精度浮点运算规则。对于字符串转换为数字的情形也是如此，因为字符串总是会被转换为双精度数。需要特别注意的是，如果某个整数运算涉及了很大的值（如超出了 64位所能表示的范围），那么得到的结果是不可预测的。（事实上，应尽量避免使用超过 63位的整数，因为正负号也需要占一位。）

表3-16 算术运算符

运算符	语法	含义
+	$a + b$	加法，即两个操作数之和
-	$a - b$	减法，即两个操作数之差

续表

运算符	语法	含义
-	$-a$	取反，即操作数的负值
*	$a * b$	乘法，即两个操作数之积
/	a / b	除法，即两个操作数之商
DIV	$a \text{ DIV } b$	整除，即两个操作数之商的整数部分
%	$a \% b$	取模，即相除之后的余数

如表3-17所示，逻辑运算符用于确定逻辑表达式的结果为真（非0值），还是为假（0值）。如果逻辑表达式的值无法确定，则其计算结果为NULL。例如，表达式`1 AND NULL`即为一个不确定的值。

表3-17 逻辑运算符

运算符	语法	含义
AND, &&	$a \text{ AND } b, a \&\& b$	逻辑与。若两个操作数同时为真，则结果为真
OR,	$a \text{ OR } b, a b$	逻辑或。只要有一个操作数为真，则结果为真
XOR	$a \text{ XOR } b$	逻辑异或。若有且仅有一个操作数为真，则结果为真
NOT, !	$\text{NOT } a, !a$	逻辑非。若操作数为假，则结果为真

MySQL还支持运算符AND、OR和NOT的替代形式，即`&&`、`||`和`!`，其用法与C语言里的一样。需要特别注意运算符`||`。标准SQL把`||`规定为字符串的连接运算符，但在MySQL里它表示的却是逻辑 OR 运算。如果你在使用下面的表达式时，期望它会执行字符串的连接，那么你可以会意外地发现：其返回结果是0。

'abc' || 'def' → 0

之所以会发生这种情况，是因为'abc'和'def'在进行运算时都被转换成了整数，即都会转换为0。在MySQL里，必须使用CONCAT('abc','def')或其他办法来连接字符串：

CONCAT('abc','def') → 'abcdef'

'abc' 'def' → 'abcdef'

如果想让运算符||具有标准SQL的行为，则必须启用SQL的PIPES_AS_CONCAT模式。

如表3-18所示，按位运算符可以完成二进制位的与、或和异或运算。其中，最终结果的每一位，都是按两个操作数相对应的二进制位，执行逻辑与、逻辑或、逻辑异或运算而得到的。你还可以执行位的左移或右移操作。按位运算所用的值都是BIGINT（64位）整数值。

表3-18 按位运算符

运算符	语法	含义
&	a & b	按位与。若两个操作数的相应位皆为1，则结果中的相应位为1
	a b	按位或。若两个操作数的相应位有一个为1，则结果中的相应位为1
^	a ^ b	按位异或。若两个操作数的相应位分别为1和0，则结果中的相应位为1
<<	a << b	把a按位左移b位
>>	a >> b	把a按位右移b位

如表3-19所示，比较运算符包括用于测试数字和字符串的相对大小或词法顺序的运算符、用于模式匹配的运算符，以及用于测试NULL值的运算符。其中，<=>运算符是MySQL特有的。更多关于字符串比较属性的信息请参考3.1.2节。

表3-19 比较运算符

运算符	语法	含义
=	a = b	若两个操作数相等，则结果为真
<=>	a <=> b	若两个操作数相等（即使为NULL），则结果为真
<>, !=	a <> b, a != b	若两个操作数不等，则结果为真
<	a < b	若a小于b，则结果为真
<=	a <= b	若a小于或等于b，则结果为真
>=	a >= b	若a大于或等于b，则结果为真
>	a > b	若a大于b，则结果为真
IN	a IN (b1, b2, ...)	若a等于b1、b2、...中的某一个，则结果为真
BETWEEN	a BETWEEN b AND c	如果满足b <= a <= c，则结果为真
NOT BETWEEN	a NOT BETWEEN b AND c	如果不满足b <= a <= c，则结果为真
LIKE	a LIKE b	SQL模式匹配。若a匹配b，则结果为真
NOT LIKE	a NOT LIKE b	SQL模式匹配。若a不匹配b，则结果为真
REGEXP	a REGEXP b	正则表达式匹配。若a匹配b，则结果为真
NOT REGEXP	a NOT REGEXP b	正则表达式匹配。若a不匹配b，则结果为真
IS NULL	a IS NULL	若操作数为NULL，则结果为真
IS NOT NULL	a IS NOT NULL	如果操作数不为NULL值，则结果为真

模式匹配允许我们在不指定精确字面值的情况下查找值。MySQL提供了两种类型的模式匹配。一种是利用LIKE 运算符以及通配符“%”（能匹配任意字符序列）和“_”（只能匹配单个字符）实现的SQL模式匹配。另一种是利用REGEXP运算符和正则表达式（它与grep、sed和vi等UNIX程序所使用的正则表达式很相似）实现的模式匹配。必须使用其中的一种运算符来完成模式匹配，并且不能使用=运算符。要颠倒模式匹配的含义，可以使用运算符NOT LIKE或NOT REGEXP。

这两种类型的模式匹配，除了使用的运算符和模式字符不同之外，还有以下两个重要的差异。

SQL的LIKE模式只用于匹配整个字符串。正则表达式REGEXP可以匹配字符串的任何部分。

LIKE运算符是多字节安全的。REGEXP运算符只能正确地处理单字节字符集，并且不会考虑排序规则。

与 LIKE 运算符配合使用的模式，可以包括通配符 “%” 和 “_” 。例如，模式'Frank%'将匹配所有以'Frank'开头的字符串：

'Franklin' LIKE 'Frank%' → 1

'Frankfurter' LIKE 'Frank%' → 1

通配符 “%” 能匹配任意长度的字符序列，其中包括空序列，因此模式'Frank%'能够匹配'Frank'：

'Frank' LIKE 'Frank%' → 1

这意味着模式'%'能够匹配任何字符串，包括空字符串。但是，'%'不能匹配NULL。事实上，其中一个操作数为NULL的所有模式匹配都会失败：

'Frank' LIKE NULL → NULL

NULL LIKE '%' →
NULL

如果其中有一个操作数为二进制串，那么MySQL的LIKE运算符可以把它两边的操作数当作二进制串来进行比较。如果这两个操作数都不是二进制串，那么 LIKE 将根据它们的排序规则来进行比较：

'Frankly' LIKE 'Frank%' → 1

'frankly' LIKE 'Frank%' → 1

BINARY 'Frankly' LIKE 'Frank%' → 1

BINARY 'frankly' LIKE 'Frank%' → 0

'Frankly' COLLATE latin1_general_cs LIKE 'Frank%' → 1

'frankly' COLLATE latin1_general_cs LIKE 'Frank%' → 0

这种行为与标准SQL的LIKE运算符有所差异，即在标准SQL里是区分大小写的。

LIKE运算符的另一个通配符是下划线“_”，它可以匹配任何单个字符。因此，模式'___'将与任何恰好包含有3个字符的字符串相匹配。例如，'c_t'将与'cat'、'cot'、'cut'，甚至'c_t'（因为“_”可以自我匹配）相匹配。

通配符可以指定在模式里的任何位置。例如，模式'%bert'可以与'Englebert'、'Bert'和'Albert'相匹配。模式'%bert%'除了可以与刚才这些字符串相匹配之外，还能匹配像'Berthold'、'Bertram'和'Alberta'这样的字符串。模式'b%t'可以与'Bert'、'bent'和'burnt'相匹配。

如果想匹配字符“%”或“_”本身，则需要在其前面加上反斜线字符（即“\%”或“_”），以取消其特殊含义：

'abc' LIKE 'a%c' → 1

'abc' LIKE 'a\%c' → 0

'a%c' LIKE 'a\%c' → 1

'abc' LIKE 'a_c' → 1

'abc' LIKE 'a_c' → 0

'a_c' LIKE 'a_c' → 1

MySQL的另一种模式匹配形式是使用正则表达式。运算符为REGEXP，而非LIKE。下面的示例演示了几种比较常用的正则表达式模式字符。

点号字符“.”是一个可以匹配任何单个字符的通配符：

'abc' REGEXP 'a.c' → 1

方括号“[...]”，可以匹配列在方括号内部的任何字符：

'e' REGEXP '[aeiou]' → 1

'f' REGEXP '[aeiou]' → 0

连接号“-”可用于指定字符范围，其两端分别为范围的起始和结束字符。如果要把这个字符范围的含义变反（即要匹配未列在其中的字符），那么可以在其前面加上一个“^”：

'abc' REGEXP '[a-z]' → 1

'abc' REGEXP '[^a-z]' → 0

星号 “*” 表示 “匹配任意个其前面的字符”。例如，模式 'x*' 将匹配任意个数的 “x” 字符：

'abcdef' REGEXP 'a.*f' → 1

'abc' REGEXP '[0-9]*abc' → 1

'abc' REGEXP '[0-9][0-9]*' → 0

这里的 “任意个” 包括 0 个，而这也正是上面第二个表达式会匹配成功的原因。如果要匹配 1 个或多个其前面的字符，则需要使用 “+” 来代替 “*”：

'abc' REGEXP 'cd*' → 1

'abc' REGEXP 'cd+' → 0

'abcd' REGEXP 'cd+' → 1

形如 '^pattern' 和 'pattern\$' 的模式表示的是：当 pattern 出现在字符串的开头或结尾时，模式匹配成功。而对于形如 '^pattern\$' 的模式，只有在 pattern 匹配整个字符串时，才能匹配成功：

'abc' REGEXP 'b' → 1

'abc' REGEXP '^b' → 0

'abc' REGEXP 'b\$' → 0

'abc' REGEXP '^abc\$' → 1

'abcd' REGEXP '^abc\$' → 0

MySQL 的正则表达式匹配还有其他一些特殊模式字符。更多相关信息请参考附录 C。

列值可以用在 LIKE 或 REGEXP 模式里，不过如果列包含了几种不同的值，那么整个匹配过程会比常量模式更慢。每当列值发生变化时，模式都必须重

重新检查一次，并且需要转换为内部形式。

3.5.1.2 运算符优先级

当MySQL计算表达式时，其中的运算符决定了表达式中子项的分组顺序。有些运算符拥有较高的优先级，也就是说，它们会先于其他运算符进行计算。例如，乘法和除法高于加法和减法的优先级。下面两个表达式是等价的，因为“*”和“DIV”都会在“+”和“-”之前进行计算：

$3 + 4 * 2 - 10 \text{ DIV } 2 \rightarrow 6$

$3 + 8 - 5 \rightarrow 6$

下面按优先级从高到低的顺序列出了各个运算符。其中，处于同一行的运算符拥有相同的优先级。优先级高的运算符会在优先级低的运算符之前进行计算。优先级相同的运算符则按从左至右的顺序依次进行计算。

INTERVAL

BINARY COLLATE

!

-（负号） ~（按位求反）

^

* / DIV % MOD

+ -

<< >>

&

|

< <= = <=> <> != >= > IN IS LIKE REGEXP
RLIKE

BETWEEN CASE WHEN THEN ELSE

NOT

AND &&

XOR

OR ||

:=

根据SQL模式或MySQL的版本差异，有些运算符具有不同的优先级顺序。更多相关信息请参考附录C。

要改写运算符的优先级，更改表达式子项的计算顺序，可以使用括号：

$1 + 2 * 3 - 4 / 5 \rightarrow 6.2000$

$(1 + 2) * (3 - 4) / 5 \rightarrow -0.6000$

3.5.1.3 表达式里的Null值

在表达式里使用NULL 值时，要小心，因为其结果往往会出乎你的意料。下面这些指南能帮助你避免意外情况的发生。

当把NULL用作算术运算符或位运算符的操作数时，其计算结果皆为NULL：

$1 + \text{NULL} \rightarrow \text{NULL}$

$1 | \text{NULL} \rightarrow \text{NULL}$

当把NULL与逻辑运算符一起使用时，除非其计算结果可以确定，否则其结果皆为NULL：

$1 \text{ AND } \text{NULL} \rightarrow \text{NULL}$

$1 \text{ OR } \text{NULL} \rightarrow 1$

$0 \text{ AND } \text{NULL} \rightarrow 0$

0 OR NULL →
NULL

当把NULL用作比较运算符或模式匹配运算符的操作数时，其计算结果皆为NULL，不过，运算符<=>、IS NULL和IS NOT NULL例外，因为它们都是专门用于处理NULL值的：

1 = NULL →
NULL

NULL = NULL →
NULL

1 <=> NULL → 0

NULL LIKE '%' →
NULL

NULL REGEXP '*' →
NULL

NULL <=> NULL → 1

1 IS NULL → 0

NULL IS NULL → 1

当把NULL用作函数参数时，函数通常会返回NULL。不过，那些专门用于处理NULL参数的函数例外。例如，IFNULL()便可以处理NULL参数，并且会根据具体情况返回真值或假值。另外，STRCMP()期望的是非 NULL 参数；如果把 NULL 作为参数传递给它，那么它返回的结果将为NULL，而不是真值或假值。

在排序运算中，NULL值会一起排序。当按升序排序时，它们会出现在最前面；当以降序排序时，会出现在最后面。

3.5.2 类型转换

只要某个值的类型与上下文所要求的类型不相符，MySQL就会根据操作的类型自动进行类型转换。发生类型转换的场合有以下几种。

把操作数转换成适合运算符计算的相应类型。

把函数参数转换成函数所期望的类型。

把某个值转换成适合于存储在列里的另一种类型。

另外，还可以利用类型转换运算符或函数来完成显式类型转换。

可以根据系统变量`character_set_connection`和`collation_connection`所指定的字符集和排序规则，将数字转换成字符串或时态值。只有`character_set_connection`为`binary`时，转换结果才会是一个二进制串；否则，结果为非二进制串。（在MySQL 5.5.3之前，数字转换为字符串，总是会得到一个二进制串。）

下面这个表达式涉及一个隐式类型转换。它由一个加法运算符（+）和两个操作数（1和'2'）构成：

1 + '2' → 3

由于这两个操作数的类型不一致（一个是数字，另一个是字符串），因此MySQL会对它们当中的某一个进行类型转换，以使它们的类型一致。但是，应该转换哪一个呢？在本例里，+是一个数字运算符，所以MySQL会要求两个操作数都是数字类型，因此会把字符串'2'转换为数字2。然后表达式的计算结果为3。

下面来看另一个示例。`CONCAT()`函数能够把多个字符串连接成一个更长的字符串。不管参数是什么类型，此函数都会把它们当作字符串来处理。当把一组数字传递给`CONCAT()`时，它会先把它们转换成字符串，然后再返回连接后的结果：

`CONCAT(1,23,456)` →
'123456'

如果对`CONCAT()`函数的调用又属于某个更大的表达式，那么可能会发生进一步的类型转换。请看下面这个表达式及其结果：

`REPEAT('X',CONCAT(1,2,3)/10)` →
'XXXXXXXXXXXXX'

首先，`CONCAT(1, 2, 3)`会返回字符串'123'。接着，'123'/10会转换成123/10，因为相除是一个算术运算符。此表达式的结果为12.3。由于

REPEAT() 函数需要的重复次数是一个整数，所以这个结果会四舍五入为整数 12。最后，REPEAT('X', 12) 会把字符 'X' 重复 12 次。

如果 CONCAT() 的所有参数都是非二进制串，那么其结果也将是一个非二进制串。如果有参数为二进制串，那么其结果将是一个二进制串。其中，数字参数会像前面所讨论的那样转换为字符串，然后应用前面的这些规则。

请记住，默认情况下，MySQL 会尽量把值转换为表达式所需要的类型，而不是产生错误。根据具体上下文的不同，MySQL 会让值在三种通用类别（即数字、字符串、日期/时间）之间进行转换。不过，值并不是总能从一种类型转换成另外一种类型。如果某个值在类型转换后，看起来并不像目标类型的合法值，那么这个转换就是失败的。例如，把看起来并不像数字的某个值（如字符串 'abc'）转换为数字类型，那么最终得到的结果将为 0。如果某个看起来并不像日期或时间的值转换为日期或时间类型，那么最终得到的结果将为该类型的“零”值。例如，把字符串 'abc' 转换成日期类型，则得到的结果为“零”日期：'0000-00-00'。另外，由于任何值都可以处理成字符串，因此把一个值转换成字符串，在通常情况下都不会有问题。

在数据输入操作期间，为防止把非法值转换成最相近的合法值，可以启用严格模式，以便把此类问题当作错误报告出来。更多相关细节请参考 3.3 节。

MySQL 还会执行某些微小的类型转换。如果在整数上下文中使用浮点数值，那么浮点数值会被转换（通过四舍五入）。反之亦可，整数可以毫无问题地当作浮点数来使用。

除非上下文明确地表明需要一个数字，否则十六进制常量会被当作二进制串来对待。在字符串上下文里，每两个十六进制数字会被转换成一个字符，最终结果将为一个字符串，如下列示例所示：

0x61	→ 'a'
0x61 + 0	→ 97
X'61'	→ 'a'
X'61' + 0	→ 97
CONCAT(0x61)	→ 'a'
CONCAT(0x61 + 0)	→ '97'

CONCAT(X'61') → 'a'

CONCAT(X'61' + 0) → '97'

对于比较操作中的十六进制常量，上下文会根据具体情况来确定是把它当成二进制串，还是把它当成数字。

下面的表达式会把操作数当作二进制串，并按字节进行比较。

0x0d0a = '\r\n' → 1

下面的表达式比较的是一个十六进制常量和一个数字，因此为了进行比较，该常量会先转换成数字。

0x0a = 10 → 1

下面的表达式执行的是二进制串的比较。左操作数的第一个字节值比右操作数的第一个字节值小，因此结果为假。

0xee00 > 0xff → 0

在下面的表达式里，右操作数中的十六进制常量会被转换成一个数字，因为那是个算术运算。接着，为完成比较，左操作数也会转换成一个数字。最后，由于 0xee00（60928）在数字上大于0xff（255），所以最终的结果为真。

0xee00 > 0xff+0 → 1

使用字符集引导符或CONVERT()，可以把十六进制常量强制转换成一个非二进制串：

0x61 → 'a'

0x61 = 'A' → 0

_latin1 0x61 = 'A' → 1

CONVERT(0x61 USING latin1) = 'A' → 1

有些运算符会把操作数强制转换成它所期望的类型，而不管该操作数原来是什么类型。以算术运算符为例。它们期望的是数字，因此其操作数都会进行相应的转换：

$3 + 4 \rightarrow 7$

$'3' + 4 \rightarrow 7$

$'3' + '4' \rightarrow 7$

对于“字符串到数字”的转换，只是字符串里的某个地方包含一个数字还远远不够。MySQL并不会检查整个字符串，以期望找到某个数字；它只会检查字符串的开头。如果字符串的开头部分不是数字，那么转换的结果即为0。

$'1973-2-4' + 0 \rightarrow 1973$

$'12:14:01' + 0 \rightarrow 12$

$'23-skidoo' + 0 \rightarrow 23$

$'-23-skidoo' + 0 \rightarrow -23$

$'carbon-14' + 0 \rightarrow 0$

MySQL的“字符串到数字”的转换规则，适用于把看起来像数字的字符串转换为数字值：

$'-428.9' + 0 \rightarrow -428.9$

$'3E-4' + 0 \rightarrow 0.0003$

不过，这种转换对于看起来像十六进制常量的字符串不起作用。只有使用前导“0”的才可转换：

$'0xff' + 0 \rightarrow 0$

位运算符甚至比算术运算符还严格。它们不仅要求操作数得是数字，而且还要求它们是整数（根据情况进行相应的类型转换）。这意味着，对于像0.3这样的小数，即使它为非零值，也不会被当成逻辑真值。因为当它转换成整数时，所得结果为0。在下面几个表达式里，只有那些值大于1的操作数才会被认为是逻辑真值：

$0.3 \mid .04 \rightarrow 0$

$1.3 \mid .04 \rightarrow 1$

0.3 & .04 → 0

1.3 & .04 → 0

1.3 & 1.04 → 1

模式匹配运算符希望操作的是字符串。这意味着，可以把 MySQL 的模式匹配运算符用于数字，因为在查找匹配的过程中，它会把这些数字转换成字符串！

12345 LIKE '1%' → 1

12345 REGEXP '1.*5' → 1

大小比较运算符（<、<=、=等）跟上下文密切相关，也就是说，它们会根据操作数的类型来进行计算。下列表达式会比较两个操作数的数字大小，因为它们都是数字：

2 < 11 → 1

下列表达式涉及的操作数都是字符串，因此它们会按词法进行比较：

'2' < '11' → 0

在下列表达式里，两个操作数的类型各不相同，因此 MySQL 会按数字方式对它们进行比较。于是，这两个表达式的结果都为真：

'2' < 11 → 1

2 < '11' → 1

在进行比较运算时，MySQL 会根据需要按以下规则对操作数进行类型转换。

除<=>运算符以外，所有涉及NULL值的比较结果都为NULL。（另外，<=>与=很像，只是NULL <=> NULL的结果为真，而NULL = NULL的结果为NULL。）

如果两个操作数都是字符串，那么它们将按字符串进行词法比较。对于二进制串，会按字节逐个比较各个对应字节的数值。对于非二进制串，会使用表示字符串的字符集的排序序列，逐个字符地进行比较。如果字符串使用了不同的字符集，那么这个比较可能导致错误或者产生毫无意义的结果。对于非二进制串和二进制串之间的比较，按二进制串方式进行。

如果两个操作数都是整数，那么它们将按整数方式进行数字比较。

与数字进行比较的十六进制常量，会按二进制串进行比较。

除IN()以外，如果有一个操作数为TIMESTAMP或DATETIME，而另一个操作数为常量，那么两个操作数会按TIMESTAMP值进行比较。这种做法很适合于各种ODBC应用程序的比较操作。

如果有一个操作数为小数，而另一个操作数为小数或整数，那么两个操作数会按小数进行比较；否则，按双精度浮点值进行比较。

除上述情况以外，比较操作中的操作数都将按双精度浮点值进行数字比较。请注意，字符串与数字之间的比较也属于这种情况。字符串将被转换成一个双精度数，如果字符串看起来不像数字，那么转换结果将为0。例如，'14.3'将被转换成14.3，但'L4.3'会被转换成0。

3.5.2.1 时态值的解释规则

根据表达式中上下文的需要，MySQL 会把字符串和数字自动转换成日期/时间值，或者进行相反转换。在数字上下文中，日期/时间值将自动转换成数字；在日期或时间上下文中，数字将自动转换成日期或时间。当把某个值赋给日期或时间类型列，或者当某个函数需要日期或时间值时，会发生到日期或时间类型的转换。在比较操作中，一般的规则是把日期/时间值当作字符串来比较。

如果 mytbl 表里有一个名为date_col的DATE列，那么下列语句都是等价的：

```
INSERT INTO mytbl SET date_col = '2025-04-13';
```

```
INSERT INTO mytbl SET date_col = '20250413';
```

```
INSERT INTO mytbl SET date_col = 20250413;
```

在下列示例里，TO_DAYS()函数的3种表达式参数都将解释成相同的值：

```
TO_DAYS('2025-04-13') → 739719
```

```
TO_DAYS('20250413') → 739719
```

```
TO_DAYS(20250413) →  
739719
```

3.5.2.2 测试或强制类型转换

如果想看看某个表达式会发生什么样的类型转换，那么可以在SELECT 语句里计算这个表达式，并检查其结果：

```
mysql> SELECT X'41', X'41' + 0;
```

```
-----+-----  
X' 41' | X' 41' + 0  
-----+-----  
A      |          65  
-----+-----
```

如果无法直观地看出表达式的类型，那么可以把查询结果放到一个新表里，然后查看这个表的定义：

```
mysql> CREATE TABLE t SELECT X'41' AS col1, X'41' + 0 AS col2;
```

```
mysql> DESCRIBE t;
```

```
-----+-----+-----+-----+-----+-----  
Field | Type          | Null | Key | Default | Extra  
-----+-----+-----+-----+-----+-----  
col1  | varbinary(1)  | NO   |     |         |  
col2  | double(17,0)  | NO   |     | 0       |  
-----+-----+-----+-----+-----+-----
```

对于那些用来删改行的DELETE或UPDATE语句，提前对表达式计算进行测试特别有用，因为你会想要确保这些语句将只会对你想要进行删改的行产生影响。有一种检查表达式的方式是，在执行DELETE或UPDATE语句之前，先运行一条带有相同WHERE子句的SELECT语句，以确认该子句选中的行正是你想要的。假设，mytbl表有一个名为char_col的CHAR列，其中包含以下几个值：

'abc'

'00'

'def'

'00'

'ghi'

给定这些值之后，下面这条语句会产生什么样的效果呢？

```
DELETE FROM mytbl WHERE char_col = 00;
```

其本意可能只是想删除包含'00'值的那两行。而实际的执行效果却是把所有的行都删除了——非常意外吧！这完全是MySQL的类型转换规则惹的祸。char_col是一个字符串列，但上面这条语句里的00并没有用引号引起来，因此会被当作一个数字。根据MySQL的比较规则，涉及字符串和数字的比较会被当成两个数字的比较。在MySQL执行 DELETE 语句时，它会把char_col 列的每个值进行转换，然后再与数字 0 进行比较。很不幸，虽然'00'会转换成0，但是所有看起来不像数字的字符串也会转换成0。于是，该WHERE子句对于每一行都为真，从而导致DELETE语句会清空整个表。此时，最好能在执行DELETE语句之前，使用SELECT语句来测试一下它的WHERE子句，因为这样便可以让你看到这个表达式选择了过多的行：

```
mysql> SELECT char_col FROM mytbl WHERE char_col = 00;
```

```
-----  
char_col  
-----  
  
abc  
00  
def  
00  
ghi  
-----
```

在对某个值的使用方式不太确定时，可以利用MySQL的类型转换机制，把表达式强制转换为某个特定类型的值；或者调用函数执行你需要的转换。下面列出了多项有用的类型转换技术。

往表达式里添加子项+0或者+0.0，将其强制转换成数值：

0x65

→ 'e'

0x65 + 0 → 101

0x65 + 0.0 → 101.0

如果要去掉某个数字的小数部分，那么可以使用函数FLOOR()或CAST()。如果想为某个整数增加小数部分，那么可以让它加上一个带有所需小数位数的精确零值：

FLOOR(13.3) → 13

CAST(13.3 AS SIGNED) → 13

13 + 0.0 → 13.0

13 + 0.0000 →
13.0000

如果要带有四舍五入，则需要使用ROUND()来代替CAST()。

使用CAST()或CONCAT()，可以把一个值转换成字符串：

14 → 14

CAST(14 AS CHAR) → '14'

CONCAT(14) →
'14'

使用HEX()，可以把数字转换成十六进制字符串：

HEX(255) → 'FF'

HEX(65535) →
'FFFF'

也可以把HEX()用于字符串，把字符串转换成一个由十六进制数字构成的字符串，其中每两个数字对应于原字符串里的一个字节：

HEX('abcd'); →
'61626364'

使用ASCII(), 可以把一个单字节字符转换成它的ASCII编码值:

'A' → 'A'

ASCII('A') → 65

如果想根据ASCII码反向转换成字符, 则需要使用CHAR() 函数:

CHAR(65) → 'A'

使用DATE_ADD() 或INTERVAL运算时, 会把字符串或者数字强制转换成日期:

DATE_ADD(20130101, INTERVAL 0 DAY) → '2013-01-01'

20130101 + INTERVAL 0 DAY → '2013-01-01'

DATE_ADD('20130101', INTERVAL 0 DAY) → '2013-01-01'

'20130101' + INTERVAL 0 DAY → '2013-01-01'

一般情况下, 可以采用让日期值加上一个0的方式, 把它转换成数字形式:

CURDATE() → '2013-01-09'

CURDATE()+0 → 20130109

带有时间部分的时态值可以转换成带有微秒部分的值:

NOW() → '2012-06-25 09:35:17'

NOW()+0 → 20120625093517.000000

CURTIME() →
'09:35:17'

CURTIME()+0 →
93517.000000

如果想去掉小数秒部分，可以把值转换成整数：

CAST(NOW() AS UNSIGNED) →
20130109153603

CAST(CURTIME() AS UNSIGNED) →
153603

如果想把字符串从一个字符集转换到另一个，可以使用CONVERT()。如果想检查结果是否有所要的字符集，可以使用CHARSET()函数：

'abcd' → 'abcd'

CONVERT('abcd' USING ucs2) → 'abcd'

CHARSET('abcd') →
'latin1'

CHARSET(CONVERT('abcd' USING ucs2)) → 'ucs2'

在字符串的前面带上字符集引导符，并不会导致该字符串发生转换，但MySQL 会把它解释成那个引导符所指示的字符集：

CHARSET(_ucs2 'abcd') → 'ucs2'

如果想确定某个给定十六进制UCS-2字符所对应的UTF-8字符的十六进制值，可以组合使用函数CONVERT()和HEX()。下列表达式可以确定注册商标符号的UTF-8值：

HEX(CONVERT(_ucs2 0x2122 USING utf8)) → 'E284A2'

如果想更改字符串的排序规则，可以使用COLLATE运算符。如果想检查结果是否有所要的排序规则，可以使用COLLATION()函数：

`COLLATION('abcd')` →
`'latin1_swedish_ci'`

`COLLATION('abcd' COLLATE latin1_bin)` → `'latin1_bin'`

字符集和排序规则必须是兼容的。如果它们不兼容，则可以先用 `CONVERT()` 来转换字符集，然后再用 `COLLATE` 来更改排序规则：

`CONVERT('abcd' USING latin2) COLLATE latin2_bin`

如果想把二进制串转换成具有给定字符集的非二进制串，则可以使用 `CONVERT()`：

`0x61626364` →
`'abcd'`

`0x61626364 = 'ABCD'` → 0

`CONVERT(0x61626364 USING latin1) = 'ABCD'` → 1

另外，对于用引号引起来的二进制串或十六进制串，可以用字符集引导符来更改 MySQL 对这种二进制串的解释：

`_latin1 0x61626364 = 'ABCD'` → 1

如果想把非二进制串强制转换成二进制串，则可以使用 `BINARY` 关键字：

`'abcd' = 'ABCD'` → 1

`BINARY 'abcd' = 'ABCD'` → 0

`'abcd' = BINARY 'ABCD'` → 0

3.6 选择数据类型

在3.2节，我们对各种可用的数据类型及其基本特性进行了描述，其中包括它们能够存放的值的类型，它们占用多少存储空间，等等。但在创建表时使用何种类型又该如何抉择呢？本节将对那些有助于大家做出选择的因素进行讨论。

最为“通用”的数据类型是字符串类型。任何内容都可以保存在字符串中，因为数字和日期都可以表示成字符串形式。既然如此，难道就应该把所有的列定义为字符串吗？非也。我们来看一个简单示例。假设有一些看起来像是数字的数据。你可以把它们表示成字符串，但应该那么做吗？如果这么做了，会发生什么事情呢？

首先，你可能会使用更多的空间，因为使用数字类型列来存储数字，比使用字符串类型更有效率。另外需要注意的是，由于对数字和字符串的处理方式不同，查询结果也会存在差异。例如，对数字的排序与对字符串的排序就不一样。数字 2 小于数字 11，但字符串 '2' 在词法上却比字符串 '11' 更大。此问题可以通过把列放到数字上下文中来解决：

```
SELECT col_name + 0 as num ...ORDER BY num;
```

让列加上 0，可以强制列按数字方式来排序，但这么做合理吗？在某些场合，这个技术很有用，但没必要每次需要按数字方式排序时都用这一招。让 MySQL 把一个字符串列当作一个数字列来对待，会引发两个很严重的问题。一个问题是，这将迫使 MySQL 对列里的每一个值都执行从字符串到数字的转换，而此操作的效率很低。另一个问题是，在计算过程中使用这样的列，会导致 MySQL 不会使用这些列上的任何索引，从而进一步降低查询的速度。如果从一开始就把值存储成数字，那么这两个性能降低问题都将不复存在。

刚才的示例揭示了几个在选择数据类型时需要考虑的问题。只是简单地选用某种表示，而弃用另一种表示，这种做法往往预示着在存储要求、查询处理和整体性能方面存在问题。下面列出了几个在为列挑选类型时需要思考的问题。

列要存放什么类型的值？是数字？字符串？还是日期？你完全可以把任何类型的值都表示成字符串，只是，正如我们刚才看到的那样，如果使用另一种更适合的类型，则有可能获得更好的性能。不过，对要处理的值的类型进行评估，尤其是要评估那些源自其他人的数据的值，并不是件轻而易举的事情。如果你正在为别人设计表，那么把“列将存放什么样的数据”这样一个问题询问清楚将非常重要。一定要多问，并获得必要的信息，以便做出最佳的选择。

值是否都在某个特定区间内？如果它们都是整数，那么是否总为非负值？如果是，则可以考虑选用 UNSIGNED。如果它们是字符串，那么是否总是来自某个固定且有限的值集合？如果是，则使用 ENUM 或 SET 更为合适。

数据类型的取值范围和空间占用量是相互影响的。实际需要使用多“大”的类型才算合适呢？对于数字，可以选择一种取值范围有限的小类

型，也可以选择一种取值范围很大的大类型。对于字符串，应该根据它们的长短来选择。如果要存储的值所包含的字符不超过10个，那么不要选择CHAR(255)。

在性能和效率方面要考虑哪些问题？有些类型处理起来的效率比其他类型要高。数字操作的执行速度通常都比字符串操作要快。短字符串的比较速度，比长字符串要快，而且磁盘开销也更小。对于MyISAM表，长度固定的行的性能，比长度可变的行更好。

下面几节将对这些问题进行更加详细的讨论，其中性能问题将放到5.3节讨论。

尽管你很想创建表时就做出最好的数据类型选择，但实际选择的类型可能并不是最优的，即便如此，也不用发愁。你可以利用ALTER TABLE语句，把它更改成一种更好的类型。当你发现某个数据集包含的值超出当初的设想时，你要做的更改，可能就像把SMALLINT更改为MEDIUMINT那样简单。有时会稍微复杂点，如把CHAR更改为具有特定允许值的ENUM。

3.6.1 列要存放什么类型的值

在选择数据类型时，首先要考虑的事情是列要存放什么类型的值，因为这对于选择的类型有着最直接的影响。一般来说，应该做简明的决定，即用数字类型列存储数字、用字符类型列存储字符串、用时态类型列存储日期和时间。如果数字带有小数，就选用DECIMAL或浮点类型，不用整数类型。但凡事也有例外。这里的原则是，必须了解数据的本质，才能更好地选出适合的类型。如果存储的是你自己的数据，那么你可能对使用什么样的类型心中有数。如果你是在为别人设计表，那么情况就不一样了。想了解你要处理的内容，恐怕就没那么容易了。一定要多问，以便弄清这个表到底需要包含什么样的值。

假设，有人让你帮忙设计一个表，其中需要用一个列来记录“降雨量”。它是数字吗？或者它“几乎”（即通常会编码为数字，但并非总是如此）是数字的吗？例如，在看电视新闻时，天气预报一般都会报降雨量。有时，它是一个数字（如“降雨量为0.25英寸”）。但有时，它是微量降雨，也就是说“几乎没有”降雨。这种说法对于天气预报来说很正常，但在数据库里要如何存储它呢？一种办法是，把“微量”量化成一个数字，这样可以用数字类型来记录降雨量。另一种办法是，选用字符串类型，这样可以把单词“微量”记录下来。当然，还可以想出一些更为复杂的办法，例如：同时使用数字列和字符串列，在填充其中一列时，让另一列为NULL。只要有可能，请尽量不要选择最后一种做法。因为它会让表变得难以理解，并且大大增加编写查询代码的难度。

如果让我来设计这个表，我可能会尽量把所有的行都存储成数字形式，然后根据需要对它们进行转换，以便于显示。例如，如果把所有小于0.01英寸的非零降雨量都认为是微量，那么可以用下面这样的代码来显示这个列的值：

```
SELECT IF(precip>0 AND precip<.01,'trace',precip) FROM ...;
```

有些值一眼就能看出是数字，但你必须确定是选用整数类型还是非整数类型。你必须弄清数据的计量单位和精度。精确到整数能满足需求吗？还需要精确到小数吗？这类问题有助于让你把整数和定点数或浮点数类型区分开来。例如，如果想要记录的“重量”是精确到磅，那么可以选用整数类型列。如果想要记录精确到小数，则应该选用一个定点数或浮点数类型列。在某些场合，你甚至可以使用多个列（如以磅和盎司来记录重量）。

高度是一种数字类型的信息，它有以下几种可能的表示形式。

用'6-2'这样的字符串来表示像“6 英尺，2 英寸”这样的值。这种表示形式的优点是易于阅读和理解（肯定比“74英寸”这种形式好），但这种值在像求和或求平均值之类的算术运算中很难使用。

用一个数字列来表示英尺，用另一个列来表示英寸。这种做法让算术运算变得稍微容易一点，但使用两个列比使用一个列更复杂。

用一个数字列来表示英寸。这种做法对数据库来说最容易处理，对人来说却意义不大。但不要忘了，值的展示格式并非一定要与用于处理它们的格式保持一致。你可以用许多MySQL里的函数来重新格式化这些值，让显示变得更有意义。这意味着，这可能是表示“高度”的最佳办法。

另一种数字类型的信息是货币，如美元。在计算时，使用到的值常带有美元和美分两个部分。它们看起来像是浮点值，但FLOAT和DOUBLE类型存在四舍五入的误差问题，因此不太适合，除非你只需要近似精度的行。因为人们对自己的金钱都很敏感，所以你可能更需要可以提供完美精度的数据类型。你有下面两种选择。

把货币表示成DECIMAL(M, 2)类型，其中M 为所需取值范围的最大宽度。这种类型的数值可以精确到小数点后2位。DECIMAL的优点在于：值不存在舍入误差，计算是精确的。

内部用整数类型来表示所有货币换算成“分”后的值。这种表示方法的优点是，计算过程在内部都是使用整数完成的，速度很快。不足之处在于：在输入过程中原有值需要乘以100，而在输出过程中原有值需要除以100。

某些“数字”其实不是真正的数字。电话号码、信用卡号和社会保险号都会使用非数字字符，如空格和短划线，因此不能直接存储到数字类型列里，除非去掉其中的非数字字符。但即使去掉了其中的非数字字符，恐怕还是会想要把它们存储成字符串，而不是数字，以避免丢失开头的“零”。

如果需要保存日期信息，那么这个值包括时间吗？也就是说，它们还需要包含时间值吗？MySQL没有提供时间部分为可选的日期类型：DATE没有时间部分，DATETIME一定有时间部分。如果时间部分确实是可选的，那么可以使用DATE列来记录日期，再用一个单独的TIME列来记录时间。然后，让那个TIME列可以为NULL，并把它解释为“无时间”：

```
CREATE TABLE mytbl  
(  
    date DATE NOT NULL,      # 日期是必需的  
    time TIME NULL           # 时间可选（可能为NULL）  
);
```

在某些场合，确定是否需要时间值尤其重要，如需要基于日期信息，把两个具有“主从”关系的表“链接”起来。假设，你正在进行某项涉及多项试验课题的研究。在完成一组标准初始试验之后，你可能还需要立即开展另外一些试验。根据初始试验结果的不同，对附加试验的选择也会不同。可以用一个“主从”关系来表示这种信息：其中的试验课题标识信息和标准初始试验都存储在主表的行里，而所有的附加试验都存储在从表的行里。然后，通过具体试验所给定的课题ID和试验日期把这两个表链接在一起。

在这个场景里，必须回答的问题是：是否可以只用日期，或者是否同时需要使用日期和时间。这取决于你是否会在同一天多次进行同一项试验。如果是这样，那么需要记录一个时间值（如试验的开始时间）。所用方法既可以是只用一个DATETIME列，也可以是使用分开的DATE列和TIME列（两列都必须填写）。在没有记录时间值时，如果有课题一天之内进行了两次试验，那么你将无法把该课题具体的行与相应的主表行关联起来。

我曾听到有人声称：“我不需要记录时间值，我在同一天绝不会做两次相同的试验。”在有些情况下，他们是正确的。但是，我也见过很多与之相同的人，他们后来在一天之内做了多次同样的试验，并在输入这些课题的数据之后，开始变得焦头烂额，想方设法要防止从表的行与主表的行错误地交织在一起。很遗憾，为时已晚！

有时候，可以通过往表里增加一个TIME 列来解决此问题。不幸的是，如果你没有某种独立的数据源，如原始书面记录，那么将很难修复已有的行。另外，你根本无法理清从表行与主表行之间的关联关系。即使你保留了一份独立的信息记录，情况也是乱如麻，并且很可能导致你已编写好的使用到这些表的应用程序出问题。最好的解决办法是，把这类问题向表的拥有者解释清楚，并确信在为特定数据创建表之前，已对它们的各项特征了然于心。

有时，你所掌握的数据并不完整，而这也影响到你对数据类型的选择。假设，为了家谱研究，你正在收集某些人的出生和死亡日期。有时，你所能收集到的资料只包括某人的出生或死亡年份（或年和月），而非精确的日期。如果选用DATE列来记录这些信息，那么你能只能输入具有完整日期格式的日期值。如果需要把你拥有的信息（哪怕它不完整）都记录下来，那么必须将其分成年、月和日三个列来使用。然后，对于你所拥有的日期，输入其存在部分，其余部分则输入NULL。另一种可能是使用DATE值，并将其中的年、月或日这几个部分设置为0。这种“模糊”日期可以用来表示不完整的日期值。

3.6.2 所有值是否都在某个特定的区间内

如果你已确定了一个大致范畴，计划从中选择列的数据类型，那么多考虑一下所要表示的值的范围，会有助于把选择缩小到该范畴里的某个特定类型上。假设，需要存储某些整数值。值的范围则决定了可选用的类型。如果取值范围是0~1000，那么可以选择SMALLINT~BIGINT之间的任何一种类型。如果取值范围超过了200万，则不能使用SMALLINT，可以选择的类型变为从MEDIUMINT到BIGINT之间的某一种。

当然，你完全可以为你要存储的那类值选择一种最“大”的类型（如在上一段的示例里选择 BIGINT）。但是，如果选用足以满足使用目的的最小类型，那么将可以最小化表所使用的存储空间。此外，你还将获得更好的性能，因为与较长的列相比，较短的列的处理速度更快。当读取较短的值时，所需的磁盘读写操作会更少，并且可以把更多的键值放入内存索引缓冲区里，进而加快索引搜索的速度。

如果无法获知各种可能值的范围，则只能靠猜测，或者使用BIGINT 以满足最坏情况的需要。如果猜测的类型偏小，那么也不是就无药可救。将来，还可以使用ALTER TABLE让该列变得更大些。

有时，你也会发现可以把某个列调整得小一点。在1.4.6.2节的3小节里，我们为成绩考评项目创建了一个score表，它有一个用于记录学生考试和测验成绩的score列。当时，为了便于讨论，我们在创建该列时选用的类型是INT。但是，你现在可以看到，如果成绩的范围都是在0~100，那么选择TINYINT UNSIGNED即可满足需要，因为这样占用的存储空间会更少。

数据的取值范围，也会对与数据类型一起使用的属性产生影响。如果所有值都不可能为负数，那么可以使用UNSIGNED属性；否则，不可使用。

字符串类型没有像数字类型列那样的“取值范围”，但它们都有长度的概念，并且实际需要的最大长度也会对能用的列类型产生影响。如果需要存储的字符串短于 256个字符，那么可以使用CHAR、VARCHAR或TINYTEXT。如果需要存储更长一点的字符串，则可以选择用VARCHAR或某种更长的TEXT类型。

如果某个字符串列用于表示某种固定集合的值，那么可以考虑使用数据类型ENUM或SET。它们之所以是很好的选择，那是因为它们在内部都被表示为数字。对它们的操作都是按数字方式进行，这使它们的处理效率比其他字符串类型的更高。它们也比其他字符串类型更紧凑，从而可以节省更多空间。此外，如果启用了SQL的严格模式，那么还可以防止那些并未列在允许值列表里的字符串进入数据库。更多相关细节请参考3.3节。

在确定取值范围时，最常用的两个词“总是”和“绝不”（如“总是小于1 000”或“绝不为负”），因为它们可以进一步约束你对数据类型的选择。但千万注意不要把这两个词用在并不恰当的地方。当你在了解他人数据的情况时，如果对方说出了这两个词，则需要引起特别的注意。当人们在说“总是”或“绝不”时，一定要确认他们真的那个意思。有时，当人们在说自己的数据“总是”具备某种特征时，其实是想说“几乎总是”。

假设，你正在为一组调查员设计表，他们告诉你：“我们的考试成绩总是在0 ~100。”因为有了这句话，所以你决定选用TINYINT类型；又因为这些值总是为非负数，所以你把它设置为了UNSIGNED。后来你才发现，这些人在用程序把些数据录入数据库时，有时也会用“-1”，用于表示“学生因病缺席”。糟糕！他们当初可没提及这一点！使用NULL来表示这种值，可能会被对方接受。但是，如果对方不接受，那么你将不得不记录“-1”，即不能使用UNSIGNED列。（这时，可以使用ALTER TABLE来补救。）

有时，顺便多问一个问题（如“会有例外吗？”），这种情况便可能会变得易于处理。只要存在某种例外情况，哪怕只有一次，你也必须考虑到它。你可能会发现，那些与你讨论数据库设计的人们总是会有这样的认识误区：只要例外情况不是经常发生，那么它们就无足轻重。当你在创建表时，可千万不要有这样的想法。你需要提出的问题不是“例外会经常发生吗？”，而是“例外会发生吗？”只要有例外发生，你就必须考虑它们。

第4章

视图和存储程序

MySQL支持多种类型的服务器端对象。这些是你创建的对象，服务器会把它们存储起来，以后执行。

视图（view）是一种类型的存储对象。视图是一个虚拟表。也就是说，它像一个表，但实际上又不包含数据。其实，它是根据表或其他视图来进行定义的，并且提供了另一种查看表数据的方式。视图提供了一种简单的运行复杂查询的方式，通过这种方式可以简化应用程序的开发。

存储程序（stored program）是另一种类型的存储对象。它们有多种形式，有些可以按需调用，而其他的则可以在修改表或达到预定时间时自动执行。

存储函数（stored function）。它可以用在表达式里返回某个计算的结果。

存储过程（stored procedure）。它不会直接返回结果，但可以用来完成一般的运算，或者生成可以传递回到客户端的结果集。

触发器（trigger）。它与表关联在一起，当这个表在使用INSERT、DELETE 或 UPDATE语句进行修改时，它会自动执行。

事件（event）。它会根据计划在预定时刻自动执行。

存储程序有以下几个优点和功能。

存储对象的可执行部分可以使用复合语句——它对 SQL 语法进行了扩展，可以包括代码块、循环和条件语句。

所有用于定义存储程序的代码，都会在这个程序创建时通过网络一次性发送出去，而不是每一次需要执行它时才发送。这样可以减少执行时间上的开销。

存储程序可以把复杂的计算封装成一个程序单元，从而可以轻易地通过名字来调用它。

存储程序提供了一种标准化计算操作的方式，因为所有使用它的应用程序都会以相同的方式来执行这些操作。

它们提供了一种错误处理机制。

它们可以提高数据库的安全性，因为它们可以限制具有哪些权限的存储程序才能执行，从而启用对敏感数据的访问控制。

本章会用到以下术语。

存储程序。它是存储函数、存储过程、触发器和事件的总称。

存储例程 (stored routine)。它主要特指存储函数和存储过程。这两种对象的定义语法很相似，因此会把它们放在一起讨论。事实上，也有许多数据库文献使用“存储过程”这一术语来同时指代存储函数和存储过程。但个人认为那样很容易引起混淆，所以本书并没有按照那种方式来使用这个术语。

本章展示了如何创建和使用视图与存储程序。还介绍了DEFINER子句——这是视图和存储程序的共同之处，并且常用于控制对数据的访问。

这里所讨论的是与 CREATE 语句有关的语法描述，其中只展示了最为重要的那几个子句。更多语法有关的信息请参考附录E。

4.1 使用视图

视图是一个虚表，它是在表或其他视图的基础上，使用SELECT语句来定义的。查询视图就等效于查询定义它的那条语句，只是视图将细节隐藏了起来。视图定义里可以包含表达式计算和连接等操作，因此查询视图更易于编写出简单的用于获得所要信息的查询。

一个基本的视图可以只选择表的部分列。假设你经常想要从 `president` 表里只选取 `last_name`、`first_name`、`city` 和 `state` 这4列，但是并不想使用下面这种方式将这几个列都写出来：

```
SELECT last_name, first_name, city, state FROM president;
```

同时，你也不想使用 `SELECT *`。虽然这个写起来更简单，但是用*会把那些你并不想要的列也检索出来。具体的解决办法是定义一个视图，让它只检索想要的那些列：

```
CREATE VIEW vpres AS
```

```
SELECT last_name, first_name, city, state FROM president;
```

现在，这个视图就像一个“窗口”，透过它只能看到你想的那些列。这意味着，在视图里也可以使用 `SELECT *`，并且只会获得在视图定义里通过名字指定的那些列：

```
mysql> SELECT * FROM vpres;
```

last_name	first_name	city	state
Washington	George	Wakefield	VA
Adams	John	Braintree	MA
Jefferson	Thomas	Albemarle County	VA
Madison	James	Port Conway	VA
Monroe	James	Westmoreland County	VA

...

在创建视图时，必须拥有对视图的CREATE VIEW权限，拥有对SELECT语句所选列的操作权限，以及拥有对该语句里其他地方所引用的所有列的SELECT权限。

如果在查询视图时包含了WHERE子句，那么MySQL在执行该查询时，会把这个子句添加到视图的定义里，以便进一步限制检索结果：

```
mysql> SELECT * FROM vpres WHERE last_name = 'Adams';
```

last_name	first_name	city	state
Adams	John	Braintree	MA
Adams	John Quincy	Braintree	MA

如果添加ORDER BY、LIMIT等子句，也会有同样的效果。

在使用视图时，只能引用在该视图定义里列出的那些列。也就是说，即使某个列属于基表（base table），但它未列在视图定义里，你也不能引用它：

```
mysql> SELECT * FROM vpres WHERE suffix <> '';
```

```
ERROR 1054 (42S22): Unknown column 'suffix' in 'where clause'
```

在默认情况下，视图里的列名与其 SELECT 语句里列出的输出列名相同。如果想要显式地改用其他的列名，那么需要在视图定义里，紧跟视图名字的后面用括号列出那些新名字：

```
mysql> CREATE VIEW vpres2 (ln, fn) AS
```

```
-> SELECT last_name, first_name FROM president;
```

当引用此视图时，必须使用括号里给出的列名，而不能使用视图定义的SELECT 部分里列出的那些名字：

```
mysql> SELECT last_name, first_name FROM vpres2;
```

```
ERROR 1054 (42S22): Unknown column 'last_name' in 'field list'
```

```
mysql> SELECT ln, fn FROM vpres2;
```

ln	fn
Washington	George
Adams	John
Jefferson	Thomas
Madison	James
Monroe	James

...

除视图的SELECT 语句较为复杂以外，前面的视图定义还是相对比较简单。这样可以隐藏复杂性，并轻易地查询到重要的输出信息。在1.4.9.10 节，我们为成绩考评项目开发了一个查询，用于检索考试和测验的统计信息。我们可以把这个查询定义为一个视图，从而能够更轻易地检索到相同的信息：

```
CREATE VIEW grade_stats AS

SELECT

grade_event.date, grade_event.category,

MIN(score.score) AS minimum,
```

```

MAX(score.score) AS maximum,

MAX(score.score)-MIN(score.score)+1 AS span,

SUM(score.score) AS total,

AVG(score.score) AS average,

COUNT(score.score) AS count

FROM score INNER JOIN grade_event

ON score.event_id = grade_event.event_id

GROUP BY grade_event.date;

```

通过这个视图进行查询，会执行连接操作，并检索计算的结果：

```
mysql> SELECT * FROM grade_stats;
```

date	category	minimum	maximum	span	total	average	count
2012-09-03	Q	9	20	12	439	15.1379	29
2012-09-06	Q	8	19	12	425	14.1667	30
2012-09-09	T	60	97	38	2425	78.2258	31
2012-09-16	Q	7	20	14	379	14.0370	27
2012-09-23	Q	8	20	13	383	14.1852	27
2012-10-01	T	62	100	39	2325	80.1724	29

如果只想看到某些列，那么可以列出它们的名字。如果只想看到某个特定事件的信息，那么只需要指定日期即可，当在执行它时，

MySQL会将WHERE 子句添加到视图的定义里。视图让查询真正需要的信息变得很简单：

```
mysql> SELECT date, category, count, average  
-> FROM grade_stats WHERE date = '2012-09-16';
```

date	category	count	average
2012-09-16	Q	27	14.0370

有些视图是可更新的，也就是说，你可以通过操作视图，对底层表（underlying table）里的那些行实现插入、更新或删除操作：

```
mysql> CREATE TABLE t (i INT);  
mysql> INSERT INTO t (i) VALUES(1),(2),(3);  
mysql> CREATE VIEW v AS SELECT i FROM t;  
mysql> SELECT i FROM v;
```

i
1
2
3

```
mysql> INSERT INTO v (i) VALUES(4);
```

```
mysql> DELETE FROM v WHERE i < 3;
```

```
mysql> SELECT i FROM v;
```

```
-----  
i  
-----  
3  
4  
-----
```

```
mysql> UPDATE v SET i = i + 1;
```

```
mysql> SELECT i FROM v;
```

```
-----  
i  
-----  
4  
5  
-----
```

如果想要让一个视图可更新，那么必须让它直接映射到一个表上，它所选择的那些列必须是对表中各列的简单引用，并且对视图某行的任何操作必须与底层表中某一行的操作相对应。例如，如果某个视图里有一个由聚合函数计算得到的“汇总”列，那么此视图的每一行都将涉及底层表的多个行。在这种情况下，这个视图是不可更新的，因为无法辨别出应该更新底层表的哪一行。

4.2 使用存储程序

本节主要描述如何编写和使用各种类型的存储程序：存储函数、存储过程、触发器和事件。但是，在深入讲解与各个特定类型的存储

程序有关的细节之前，我们首先要讨论一个它们共有的话题：如何写复合语句。

4.2.1 复合语句和语句分隔符

对于由单个SQL语句构成程序体的存储程序，在编写时没有特别要求。下面这个过程使用了一条SELECT语句，用于显示sampdb数据库里的各个表的名字：

```
CREATE PROCEDURE sampdb_tables ()

SELECT TABLE_NAME FROM
INFORMATION_SCHEMA.TABLES

WHERE TABLE_SCHEMA = 'sampdb' ORDER BY TABLE_NAME;
```

不过，存储程序并不局限于是一条简单的声明。里面的代码可以包含多条SQL语句，并且可以使用许多类型的构建形式，如局部变量、条件语句、循环和嵌套块。更多相关信息请参考E.2节。

如果想要让编写出的存储程序使用这些特性，那么可以使用一个复合语句，这种语句由一个BEGIN和END块构成，其间可以包含任意数量的语句。下面这个存储过程会显示出一条消息，并用你的名字（对于匿名用户，则以“earthling”代替）向你问好：

```
CREATE PROCEDURE greetings ()

BEGIN

# 77 = 16 for username + 60 for hostname + 1 for '@'

DECLARE user CHAR(77) CHARACTER SET utf8;

SET user = (SELECT CURRENT_USER());

IF INSTR(user, '@') > 0 THEN

SET user = SUBSTRING_INDEX(user, '@', 1);
```

```

END IF;

IF user = " THEN # 匿名用户

SET user = 'earthling';

END IF;

SELECT CONCAT('Greetings, ',user, '!')AS greeting;

END;

```

对于复合语句，在块内的语句之间必须使用分号“;”进行分隔。由于它同时又是客户端程序mysql的默认语句分隔符，因此在尝试使用mysql定义存储程序时，会引起冲突。为解决这一问题，可以使用 `delimiter` 命令重新定义 mysql的默认语句分隔符，以此保证它不会出现在这个例程的定义里。这样便可以让mysql不用将分号解释为结束符，而是将整个对象定义作为单独的一个语句传递到服务器。在定义完存储程序之后，还可以再次将结束符定义成分号。下面这个示例展示的是：在定义存储过程时，如何将mysql分隔符临时更改为美元符号“\$”；接着，在恢复成默认分隔符之后，开始执行该过程。

```

mysql> delimiter $

mysql> CREATE PROCEDURE show_times()

-> BEGIN

->     SELECT CURRENT_TIMESTAMP AS 'Local Time';

->     SELECT UTC_TIMESTAMP AS 'UTC Time';

-> END$

mysql> delimiter ;

mysql> CALL show_times();

```

```
-----  
Local Time  
-----  
2012-05-03 18:18:19  
-----  
-----  
UTC Time  
-----  
2012-05-03 23:18:19  
-----
```

```
mysql> delimiter EOF
```

```
mysql> CREATE PROCEDURE show_times()
```

```
-> BEGIN
```

```
->     SELECT CURRENT_TIMESTAMP AS 'Local Time';
```

```
->     SELECT UTC_TIMESTAMP AS 'UTC Time';
```

```
-> END EOF
```

```
mysql> delimiter ;
```

需要遵循的原则是，如果存储程序的程序体包含了某些内部分号，那么在定义这个程序时，应该重新定义分隔符。

复合语句不一定非要用在复杂的存储程序里。不管程序体有没有语句，都可以使用复合语句：

```
CREATE PROCEDURE do_little ()
```

```
BEGIN
```



```
DO SLEEP(1);

END;

CREATE PROCEDURE do_nothing ()

BEGIN

END;
```

为保证风格的一致性，即使在没有严格要求的情况下，你可能也会使用BEGIN和END关键字来封闭每一个存储程序定义的程序体。

4.2.2 存储函数和存储过程

存储函数常用在表达式里，它会执行计算并返回一个值，有点像COS()或HEX()这样的内建函数。存储过程是使用CALL语句来调用的独立操作，不能用在表达式里。适合于使用存储过程的情况有两种：只需通过运算来实现某种效果或动作，不需要返回值；运算结果需要返回多个结果集（函数无法做到这一点）。

存储函数可以用CREATE FUNCTION语句来创建；存储过程可以用CREATE PROCEDURE语句来创建。它们的基本语法如下所示：

```
CREATE FUNCTION func_name ([param_list])

RETURNS type

routine_stmt

CREATE PROCEDURE proc_name ([param_list])

routine_stmt
```

下面这个示例会创建一个函数，它带有一个表示年份的整数值参数。（为与表或列的名字有所区别，在给参数命名时将使用了前缀p_。）该函数使用了一个子查询，用于确定有多少位总统出生于给定年份，并会返回统计值：

```
mysql> delimiter $
```

```
mysql> CREATE FUNCTION count_born_in_year(p_year INT)
```

```
-> RETURNS INT
```

```
-> READS SQL DATA
```

```
-> BEGIN
```

```
-> RETURN (SELECT COUNT(*) FROM president WHERE  
YEAR(birth) = p_year);
```

```
-> END$
```

```
mysql> delimiter ;
```

这个函数有一条用来表明其返回值数据类型的RETURNS子句，以及一个用来计算那个值的函数体。函数体至少要包含一条RETURN语句，用来向调用者返回一个值。将一个计算定义为函数，可以方便地执行它，而无需每次都写出所有的逻辑，并且可以像使用内建函数那样来调用它：

```
mysql> SELECT count_born_in_year(1908);
```

```
-----  
count_born_in_year(1908)  
-----  
1  
-----
```

```
mysql> SELECT count_born_in_year(1913);
```

```
-----  
count_born_in_year(1913)  
-----
```

```
2  
-----
```

这些语句都自己调用了那个函数，不过存储函数也可以用在任何复杂的表达式里。

函数无法返回多个值。要让函数返回多个值，一种办法是编写多个函数，然后在同一条语句里调用所有这些函数；另一种办法是使用一个存储过程，通过它的OUT参数“返回”这些值。存储过程应该负责计算这些期望值，并把它们赋值给各个参数，在过程返回后，调用者即可访问这些参数。更多相关信息请参考4.2.2.2节。

如果定义的存储函数与某个内建的函数同名，那么在调用它时必须用数据库名对函数名进行限定，以避免出现歧义。例如，要在 `sampdb` 数据库里定义了一个名为 `PI()` 的存储函数，则必须使用 `sampdb.PI()` 来调用它，以此表明调用的不是同名的内建函数。当然，避免这种歧义的最好办法是，不要使用内建函数的名字来命名存储函数。

存储过程与存储函数很相似，不同之处在于它没有返回值。因此，它没有 `RETURNS` 子句或任何 `RETURN` 语句。下面这个简单的存储过程与 `count_born_in_year()` 函数很相似，只是它会显示一个结果集，而不会把计算结果作为其返回值。在结果集里，包含的是与每一位在给定年份出生的总统有关的信息行：

```
mysql> delimiter $
```

```
mysql> CREATE PROCEDURE show_born_in_year(p_year INT)
```

```
-> BEGIN
```

```
->     SELECT first_name, last_name, birth, death
```

```
->     FROM president
```

```
-> WHERE YEAR(birth) = p_year;
```

```
-> END$
```

```
mysql> delimiter ;
```

与存储函数有所不同，存储过程不能用在表达式里，只能通过CALL语句来调用它：

```
mysql> CALL show_born_in_year(1908);
```

first_name	last_name	birth	death
Lyndon B.	Johnson	1908-08-27	1973-01-22

```
mysql> CALL show_born_in_year(1913);
```

first_name	last_name	birth	death
Richard M.	Nixon	1913-01-09	1994-04-22
Gerald R.	Ford	1913-07-14	2006-12-26

在本例里，过程体执行了一条 SELECT 语句。如本例所示，语句的结果集并不是以过程值方式返回的，而是被发送给到了客户端。一个过程可以生成多个结果集，其中的每一个结果会被依次发送到客户端。

到目前为止，所有的示例都只是在查询信息，不过，存储例程也可用来修改表。下面这个例程 `update_expiration()`，会根据美史联盟成员的ID号，用给定的到期日期去更新相应的成员行：

```

CREATE PROCEDURE update_expiration (p_id INT UNSIGNED,
p_date DATE)

BEGIN

UPDATE member SET expiration = p_date WHERE member_id =
p_id;

END;

```

下面的update_expiration() 调用，会把成员的到期日期设置成明年的今天或者“终身成员”（NULL表示“永不过期”）：

```

mysql> CALL update_expiration(61, CURDATE() + INTERVAL 1
YEAR);

mysql> CALL update_expiration(87, NULL);

```

存储函数需要服从这样一条限制，即它们不能对调用该函数的那条语句正读取或正写入的那个表进行修改。存储过程通常没有这个限制，但如果它们是从存储函数里面调用的，则需要服从这条限制。例如，有一条查询 member 表的语句调用了某个存储函数，那么不能在这个存储函数里调用update_expiration()。

4. 2. 2. 1 存储函数和存储过程的权限

存储函数和存储过程隶属于某个数据库。如果要想创建存储函数或存储过程，则必须拥有该数据库的CREATE ROUTINE 权限。默认的情况下，在创建存储例程时，如果你还没有获得EXECUTE和ALTER ROUTINE权限，那么服务器会自动为你授予这些权限，以便你可以执行或删除该例程。在删除例程时，服务器将自动撤销那些权限。如果不想使用这种自动化的权限授予/撤销机制，那么可以把系统变量automatic_sp_privileges设置成0。

如果服务器启用了二进制日志功能，那么存储函数还需要服从另外一些限制条件，这些条件主要用于保证二进制日志在执行备份和复制操作时的安全性，具体做法是，不允许创建不确定或者会修改数据的存储函数。（如果某个函数针对给定的输入值会生成不同的结果，

那么通过再次执行二进制日志来恢复数据，将不能保证把数据恢复到原来的样子；而且该函数还可能导致在主、从服务器上的复制结果不一致。）这些限制条件也适用于触发器的创建过程。具体如下所示。

如果未启用 `log_bin_trust_function_creators` 系统变量，你就必须要具备 `SUPER` 权限才能创建存储函数。另外，你创建的所有函数都必须是确定的，并且不得修改数据。为了表明这一点，需要使用 `DETERMINISTIC`、`NO SQL` 或 `READS SQL DATA` 这三者之一来定义存储函数。例如：

```
CREATE FUNCTION half (p_value DOUBLE)

RETURNS DOUBLE

DETERMINISTIC

BEGIN

RETURN p_value / 2;

END;
```

如果启用了 `log_bin_trust_function_creators` 系统变量，就没有任何限制了。只有在你相信MySQL服务器上的所有用户都不会去定义不安全的存储函数时，才可以使用这种设置。

4.2.2.2 存储过程的参数类型

存储过程的参数分为3种类型。对于 `IN` 参数，调用者会把一个值传递到过程里。这个过程可以对这个值进行修改，但在过程返回之后任何修改对调用者来说都是不可见的。`OUT` 参数则正好相反。过程把某个值赋给这个参数，但在过程返回后该值可以被调用者访问。`INOUT` 参数则允许调用者向过程传递一个值，然后再取回一个值。

显式指定参数类型的方法是，在参数列表里的参数名前面使用 `IN`、`OUT` 或 `INOUT`。如果没有为参数指定类型，则其默认类型为 `IN`。

使用OUT或INOUT参数的方法是，在调用过程时指定一个变量名。过程可以设置参数值，相应的变量将在过程返回时获得那个值。如果想让某个存储过程返回多个结果值，那么参数类型OUT和INOUT将非常有用。（由于存储函数只能返回一个值，因此它并不适用于这种情形。）下面这个过程演示了OUT参数的用法。它可以分别统计出student表里的男生和女生人数，并通过参数返回这两个统计值，让调用者可以访问它们：

```
CREATE PROCEDURE count_students_by_sex (OUT p_male INT,  
OUT p_female INT)  
  
BEGIN  
  
SET p_male = (SELECT COUNT(*) FROM student WHERE sex =  
'M');  
  
SET p_female = (SELECT COUNT(*) FROM student WHERE sex =  
'F');  
  
END;
```

在调用此过程时，请把各个参数替换成相应的用户定义变量。此过程将把统计值放到这些参数里。在它返回之后，这些变量会包含那些统计值：

```
mysql> CALL count_students_by_sex(@male_count, @female_count);
```

```
mysql> SELECT @male_count, @female_count;
```

@male_count	@female_count
16	15

难道只有用户定义变量才能作为参数进行传递吗？非也。如果在另一个存储程序里调用`count_students_by_sex()`，那么在这个程序中定义的局部变量或参数可以作为参数传递给`count_students_by_sex()`。

需要额外参数的例子有很多。例如，你可以编写一个存储过程，让它具有一个IN参数，用于指明某次测试或测验在score表里的ID。

这个存储过程可以根据相关的分数计算描述性统计（如平均值、标准偏差、极差等），然后通过OUT参数把所有这些值传递给调用者。

关键字IN、OUT和INOUT都不能用于存储函数、触发器和事件。对于存储函数，所有参数都像IN参数。触发器和事件则根本就没有参数。

4.2.3 触发器

触发器是与特定表相关联的存储过程，其定义会在执行表的INSERT、DELETE 或 UPDATE语句时，被自动激活。触发器可被设置成语句处理各行之前或之后激活。触发器的定义包含有一条会在触发器被激活时执行的语句。

触发器具有以下几个好处。

触发器可以检查或修改将被插入或用来更新行的那些新的数据值。这意味着我们可以利用触发器来实现数据完整性约束，如检查某个百分比数值是不是落在了0~100这样一个区间内。触发器还可以用来对输入数据进行过滤。

触发器可以基于某个表达式来为列提供默认值，甚至可以为那些只能使用常量默认值进行定义的列类型提供值。

触发器可以在行删除或更新之前先检查行的当前内容。这种能力能完成许多任务，如记录已有行的更改情况。

要创建触发器，可以使用CREATE TRIGGER语句。在触发器的定义里，需要指明：触发它的语句类型（INSERT、UPDATE 或 DELETE）；以及是在行被修改之前触发，还是在之后触发。触发器创建语句的基本语法如下所示：


```

CREATE TRIGGER trigger_name          # 触发器的名字

{BEFORE | AFTER}                     # 触发器激活的时机

{INSERT | UPDATE | DELETE}          # 激活触发器的语句

ON tbl_name                           # 关联表

FOR EACH ROW trigger_stmt;          # 触发器内容

```

其中，`trigger_name`是触发器的名字；`tbl_name`是与触发器相关的那个表的名字。在给触发器命名时，我习惯于在名字中体现触发器的用途和关联的表，如`bi_tbl_name`表示的是一个与 `tbl_name` 表有关的BEFORE INSERT 触发器；而 `ai_tbl_name` 表示的则是一个与 `tbl_name`表有关的AFTER INSERT触发器。

`trigger_stmt` 是触发器的语句体，即在触发器被激活时需要执行的语句。在触发器的语句体里，可以使用`NEW.col_name`语法来引用在INSERT或UPDATE触发器里将被插入或修改的那个新行里的列。类似地，`OLD.col_name`语法可以用来引用在DELETE或UPDATE触发器里将被删除或修改的原行里的列。如果想要用BEFORE 触发器改变列值，而且想在值存储到表中之前改变它，那么可以使用`SET NEW.col_name = value`。

在下面这个示例里，表`t`的INSERT语句有一个名为`bi_t`的触发器。其中，表`t`有一个整型列`percent`，用于存储百分比值（范围为0～100）；另外还有一个DATETIME列。该触发器使用的是BEFORE，因此它可以在数据值被插入进表之前对它们进行检查。

```

mysql> CREATE TABLE t (percent INT, dt DATETIME);

mysql> delimiter $

mysql> CREATE TRIGGER bi_t BEFORE INSERT ON t

->     FOR EACH ROW BEGIN

->         IF NEW.percent < 0 THEN

```

```
->          SET NEW.percent = 0;

->      ELSE IF NEW.percent > 100 THEN

->          SET NEW.percent = 100;

->      END IF;

->  END$  NEW.dt = CURRENT_TIMESTAMP;

->  END$
```

```
mysql> delimiter ;
```

这个触发器将完成以下两个动作。

如果要插入的百分比值超出了 0~100的范围，那么这个触发器将把该值转换成最靠近端点的那个值。

这个触发器将自动为那个DATETIME列提供一个CURRENT_TIMESTAMP值。实际上，这绕开了“列的默认值必须是一个常数”的限制，让DATETIME列具备了类似于TIMESTAMP列的自动初始化能力。（自 MySQL 5.6.5开始，DATETIME 列可以被初始化为当前的时间戳，因此这里的触发器已没什么用处。不过，它在MySQL 5.6 以及更早一点的版本里一样可以如设计的那样工作，因此你可以在其他依赖于具体版本的环境里达到版本无关的效果。）

我们来看看这个触发器是如何工作的，先往表里插入几行，然后检索表的内容：

```
mysql> INSERT INTO t (percent) VALUES(-2); DO SLEEP(2);
```

```
mysql> INSERT INTO t (percent) VALUES(30); DO SLEEP(2);
```

```
mysql> INSERT INTO t (percent) VALUES(120);
```

```
mysql> SELECT * FROM t;
```

percent	dt
0	2012-05-19 11:14:54
30	2012-05-19 11:14:56
100	2012-05-19 11:14:58

触发器属于表，因此你必须要拥有表的TRIGGER 权限，才能为表创建或删除触发器。（如果你删除某个表，那么MySQL会删除所有与之关联的触发器。）

触发器创建操作具有与存储函数创建操作一样的约束。更多信息请参考4.2.2.1节。

4.2.4 事件

MySQL有一个事件调度器，它可以定时激活多个数据库操作。事件就是一个与计划相关联的存储程序。计划会定义事件执行的时间或次数，并且还可以定义事件何时强行退出。事件非常适合于执行那些无人值守的系统管理任务，如汇总报告定期更新、旧数据过期清理或者日志表轮换等。本节将演示如何完成行过期处理操作。与如何完成基于事件日志表轮换操作有关的示例请参考12.8.7.4节。

默认情况下，事件调度器并不会运行，因此必须先启用它才能使用事件。请把下面两行内容放到某个在服务器启动时会被读取的选项文件里：

```
[mysqld]
event_scheduler=ON
```

如果想在运行时查看事件调度器的状态，那么可以使用下面这条语句：

```
SHOW VARIABLES LIKE 'event_scheduler';
```

如果想在运行时停止或启动事件调度器，那么可以更改event_scheduler系统变量（它是一个GLOBAL变量，因此必须要拥有SUPER权限才能修改它）的值：

```
SET GLOBAL event_scheduler = OFF;      # 或者0
```

```
SET GLOBAL event_scheduler = ON;       # 或者1
```

如果你停止了调度器，则所有事件都不会运行。也可以让事件调度器保持运行，但禁用各个事件。关于这一点，稍后再做讨论。

说明

如果在服务器启动时把变量event_scheduler设置成DISABLED，那么在运行时只能查看其状态，而不能更改它。你可以创建事件，但它们不会执行。

事件调度器会把执行情况写到服务器的错误日志里，通过它，你可以查看到事件调度器正在做什么。在事件执行期间，只要有错误发生，事件调度器都会把它们记录下来。如果事件调度器没有如预期的那样运行，那么查看错误日志里的消息，从而找出具体的原因。

创建事件的方法是，使用CREATE EVENT语句，其基本语法如下：

```
CREATE EVENT event_name

ON SCHEDULE

    {AT datetime | EVERY expr interval [STARTS datetime]
    [ENDS datetime]}

DO event_stmt
```

事件属于数据库，因此你必须要拥有数据库的EVENT权限才能创建或删除其触发器。

下面这个示例演示的是：如何创建一个简单的事件，并用它来删除表里的旧行。假设有一个名为web_session的表，其存储的内容是与

网站访问用户相关的会话状态信息，并且该表有一个名为last_visit的DATETIME列，用于记录每位用户的最近一次访问时间。为防止表里的行越积越多，可以创建一个事件，用于定期清理它们。为实现事件每 4小时执行一次，同时将超过一天的行清除掉，可以编写如下事件定义：

```
CREATE EVENT expire_web_session  
  
ON SCHEDULE EVERY 4 HOUR  
  
DO  
  
DELETE FROM web_session  
  
WHERE last_visit < CURRENT_TIMESTAMP - INTERVAL 1 DAY;
```

其中， EVERY n interval子句用于指定事件定期执行的时间间隔。interval值与DATE_ADD()函数里的参数值（HOUR、DAY或MONTH）相似。在EVERY子句的后面，还可以包含可选子句STARTS datetime和ENDS datetime，用于指定事件的第一次和最后一次执行时间。默认情况下， EVERY事件在被创建后会立刻开始它的第一次执行，并且会定时持续执行下去，永不停止。

DO子句负责定义事件的语句体部分，它是一条在事件被触发时执行的SQL语句。与其他类型的存储程序一样，它可以是一条简单的语句，也可以是一条使用BEGIN和END编写出的复合语句。

如果想创建一个只执行一次的事件，则应该使用AT调度类型，而不要使用EVERY。像下面的定义便会创建一个在一个小时后只执行一次的事件：

```
CREATE EVENT one_shot  
  
ON SCHEDULE AT CURRENT_TIMESTAMP + INTERVAL 1  
HOUR  
  
DO ...;
```

如果要禁用某个事件，让它不再执行，或者要重新激活某个被禁用的事件，可以使用ALTER EVENT：

```
ALTER EVENT event_name DISABLE;
```

```
ALTER EVENT event_name ENABLE;
```

4.3 视图和存储程序的安全性

在定义视图时，需要设置一条用于以后调用的SELECT 语句。在定义存储程序时，也有相同的操作，即需要定义一个以后执行的对象。这里的“以后执行”预示着实际执行这些对象的用户并不一定就是当初创建它们的那个用户，而这会引出一个很重要的问题：服务器在执行时应该使用什么样的安全上下文来检查访问权限呢？也就是说，应该应用哪个账户的权限呢？

默认情况下，服务器会使用定义该对象的那个用户的账户。假设我定义了一个存储过程p()，用于访问我的表。如果我把p()的EXECUTE 权限授予你，那么你可以使用CALL p()来调用该过程，进而访问到我的表，因为它是以我的权限来运行的。这种类型的安全上下文有利有弊：

有利的一面是：精心编写的存储程序可以把表开放给那些无法直接访问它们的用户，并可对访问进行控制。

有弊的一面是：如果某个用户创建了某个可以访问到敏感数据的存储程序，但却忘记了其他可以调用该对象的人也会访问到那些敏感数据，则会造成安全漏洞。

在定义存储程序或视图时，显式指定定义者的方法是，在该对象的CREATE 语句里包含一个DEFINER = account 子句。这样，这里列出的账户就能跟定义者完全一样，从而达到在执行时检查访问权限的目的。例如：

```
CREATE DEFINER = 'sampadm'@'localhost' PROCEDURE  
count_students()
```

```
SELECT COUNT(*) FROM student;
```

在DEFINER子句里，定义者的具体值可以是格式为'user_name'@'host_name'的账户名，该格式与账户管理语句（如CREATE USER）里所用的格式一样。更多信息请参考 13.2.1.1 节。对于这种格式，其中的user_name和host_name必不可少。此外，那个具体值还可以是CURRENT_USER或CURRENT_USER()。它表明定义者就是执行这条CREATE语句的那个用户的账户。如果没有给出DEFINER子句，则默认使用同一个账户。

如果你拥有SUPER权限，则可以使用任何一种语法正确的账户名来作为DEFINER值。当账户不存在时，会出现一条警告消息。如果你没有SUPER权限，则只能把定义者设置为你自己的账户。既可以使用完整的账户名，也可以使用CURRENT_USER。

对于视图和存储例程（包括存储函数和存储过程），还可以使用SQL SECURITY特性，用于实现对执行时访问检查的附加控制。SQL SECURITY 特性的允许值为 DEFINER（以定义者的权限执行）或 INVOKER（以对象调用者的权限执行）。

适合使用SQL SECURITY INVOKER的场合是：只想以调用者所拥有的权限来执行视图或存储程序。例如，下面这个视图将访问mysql 数据库里的某个表，但是是以调用者的权限来运行。这样一来，如果调用者本人无权访问mysql.user表，那么即使他访问这个视图，也无法突破权限限制。

```
CREATE SQL SECURITY INVOKER VIEW v
AS SELECT CONCAT(User,'@',Host) AS Account, Password FROM
mysql.user;
```

服务器会自动调用触发器和事件，所以这里的“调用者”概念对它们并不适用。因此，它们没有SQL SECURITY特性，总是以定义者的权限来执行。

如果某个视图或存储程序以定义者的权限来运行，但那个定义者账户并不存在，那么会出现一个错误。

在视图或存储程序里，CURRENT_USER()函数会默认返回与该对象的DEFINER属性相对应的账户。对于视图和定义时带有SQL SECURITY

INVOKER 特性的存储例程，CURRENT_USER() 会返回调用用户的账户。

第5章 查询优化

关系数据库理论涉及的主要内容有表、集合，以及针对表和集合的操作。数据库是表的集合，表是行和列的集合。当调用SELECT语句检索表中的行时，得到的结果则是行和列的另一个集合，即另一个表。这些都是很抽象的概念，并不涉及数据库系统用来操作表中数据的底层表示。集合理论里的另一个抽象概念是，对表的操作都是立刻发生的；查询在概念上就是集合操作，其中没有时间的概念。

不过，真实世界完全是另一回事。数据库管理系统的确能实现这些抽象概念，但依赖于实际硬件的具体实现会受到各种真实物理条件的约束。因此，查询需要占用一定的时间——有时会长到让人厌烦。急躁的我们并不喜欢等待，于是会把对集合的瞬时数学操作抽象搁置一边，去寻找能够加快查询的方法。所幸的是，还真有一些技术可以实现这个目标。

为表创建索引，能让数据库服务器查找行的速度更快。

认真考虑如何编写能够最大程度利用这些索引的查询，并且使用EXPLAIN 语句来检查MySQL服务器是否真的那样做了。

选择能够让服务器进行高效处理的数据类型和表存储格式。

本章将着重讨论上述几个方面的内容，以帮助你优化数据库系统的性能，使它尽可能快速地处理查询。MySQL数据库已经很快了，但是如果你能“施以援手”，那么即使最快的数据库也可以更加快速地运行查询。

5.1 使用索引

用来加速查询的技术有很多，其中最为重要的就是索引。也就是说，在通常情况下，造成查询速度差异的最大因素就是索引是否使用得当。多数情况下，当查询速度很慢时，加上索引便能立马解决问题。但也并非总是如此，因为优化通常都不是件简单的事情。然而，如果你不使用索引，那么在许多情况下，通过其他途径来提高性能的各种尝试都纯粹是在浪费时间。应该首先使用索引来最大程度地改进性能，然后再看看是否还有其他有用的技术。

本节将描述索引是什么，以及索引是怎样改进查询性能的。另外，还会讨论到索引可能降低性能的情况，并对如何为表合理地选择索引提供指导。在下一节，我们将讨论 MySQL 的查询优化程序，该程序会尝试找出最为有效的查询执行方式。除了解如何创建索引之外，若能对优化程序也有所理解，那就更好了。因为这样一来，你就能更加充分地利用创建的索引了。有些查询编写方法实际上会阻碍你对索引的有效利用，你肯定想避免这种情况。

5.1.1 索引的优点

让我们先从一个没有索引的表开始，一起来看看索引是如何工作的。无索引的表就只是一组无序的行。图5-1展示了ad表，我们在第1章中讨论过它。因为这个表没有索引，所以，如果我们想要找到某个公司的行，那么需要检查表的每一行，看看它是否与那个期望值相匹配。这是一个全表扫描操作，其效率很低，如果表很大，而且仅有少数几个行与搜索条件相匹配，那么整个扫描过程的效率将会超级低。

ad 表

company_num	ad_num	hit_fee
14	48	0.01
23	49	0.02
17	52	0.01
13	55	0.03
23	62	0.02
23	63	0.01
23	64	0.02
13	77	0.03
23	99	0.03
14	101	0.01
13	102	0.01
17	119	0.02

图5-1 无索引的ad表

图5-2显示的是同一个表，不同之处在于ad表的company_num列增加了一个索引。该索引包含ad表里每个行的一个项，而且这些索引项按company_num值排了序。现在，不用为了查找匹配项，一行一行地搜索整个表了，我们可以使用这个索引。假设，我们要找出公司编号为13的所有行。我们开始扫描索引，便会找到3个属于该公司的值。然后，我们会到达公司编号为14的索引值，该值比我们正查找的值要大一点。由于索引值已是有序的，因此当我们读到那条包含 14的索引行时，我们便知道再也无法找到更多与 13匹配的内容了，于是可以退出查找过程。由此可见，一种使用索引提高效率的做法是，我们可以得知匹配行在什么位置结束，从而跳过其余部分。另一种使用索引提高效率的做法是，利用定位算法，不用从索引开始位置进行线性扫描，即可直接找到第一个匹配项（例如，二分搜索比扫描要快很多）。这样，我们便可以快速地定位到第一个匹配值，从而节省大量的搜索时间。许多数据库都使用了各种各样的技术来快速定位索引值，但这些技术具体是什么对本书的读者来讲并不重要。重要的是，我们要知道这些技术很有效，并且还要知道使用索引的确是个好方法。

ad 表

index	company_num	ad_num	hit_fee
13	14	48	0.01
13	23	49	0.02
13	17	52	0.01
14	13	55	0.03
14	23	62	0.02
17	23	63	0.01
17	23	64	0.02
23	13	77	0.03
23	23	99	0.03
23	14	101	0.01
23	13	102	0.01
23	17	119	0.02

图5-2 索引后的ad表

为何不直接对数据行排序，省掉索引操作呢？这样做难道不是同样能够加快搜索速度吗？如果表只有一个索引，那么回答是肯定的。但是你可能还想添加第二个索引，但又不能同时按两种方式对数据行进行排序。例如，可能需要一个有关顾客姓名的索引，以及一个有关顾客ID号或电话号码的索引。将索引从数据行中整体分离出来，便可以解决这个问题，并且还可以创建出多个索引。此外，索引里的行通常都要比表里的数据行更短。当插入或

删除新值时，为保持排序顺序，来回移动较短的索引值，要比来回移动较长的数据行更加容易。

对于不同的MySQL存储引擎，索引的具体实现细节会有所不同。例如，对于MyISAM表，其数据行保留在数据文件里，而索引值则保留在索引文件里。一个表可以有多个索引，但它们都保存在同一个索引文件里。索引文件里的每一个索引都由一组有序的关键字行构成，这个组关键字行主要用于快速访问数据文件。

与之相比，InnoDB存储引擎没有按照上面的方法将行和索引值分开放置，尽管它也是把索引值当作是一组有序值。默认情况下，InnoDB存储引擎只使用一个表空间，在这个表空间的内部，管理着所有InnoDB表的数据存储和索引存储。可以配置InnoDB，让它创建的每个表都有自己的表空间，但是即便如此，给定表的数据和索引也同样保存在同一个表空间文件里。

上述讨论描述的是，在查询单个表时，索引所具有的优势，即使用索引可以不再需要执行全表扫描，从而可以极大地提高搜索速度。在执行连接多个表的查询时，索引发挥的作用更大。在执行单表全扫描查询时，你必须不断地检查行数，以确保它就是表的行数。在多表查询里，这个数可能是一个天文数字，因为它是所有这些表中所有行的乘积。

假设，你有3个无索引表t1、t2和t3，其中每个表都包含有一个列，它们分别为i1、i2和i3，并且每个列都有1 000个行，这些行中包含的是1~1000的数字。如果要在这些表的各个行的所有组合中，将其值彼此相等的行找出来，那么可以用如下语句进行查询：

```
SELECT t1.i1, t2.i2, t3.i3  
  
FROM t1 INNER JOIN t2 INNER JOIN t3  
  
WHERE t1.i1 = t2.i2 AND t2.i2 = t3.i3;
```

此查询的结果应该有1 000个行，其中每一个行都包含有3个相等的值。如果我们不使用索引来进行查询，那么根本无法知道哪些行包含了哪些值。于是，我们必须尝试所有组合，以找到那些与WHERE子句相匹配的行。可能的组合总共有 $1000 \times 1000 \times 1000$ （共10亿！）种，这比匹配数目多了100万倍。这可是一种巨大的浪费。随着表的生长，情况会变得更糟。如果不使用索引，那么处理表之间的连接所花费的时间将会大大增加。为表建立索引，可以大幅提高查询的速度，因为有了索引，查询就可以像下面这样进行处理。

(1) 从表t1中选择第一个行，看该行包含的是何值。

(2) 利用表t2的索引，直接转到与表t1的值相匹配的那个行。类似地，利用表t3的索引，直接转到与表t1的值相匹配的那个行。

(3) 继续选择表t1的下一个行，然后重复前面的过程。一直持续到表t1的所有行都被检查完为止。

此时，我们对表t1执行的仍然是全扫描操作，但是为了将那些行直接挑选出来，对表t2和t3执行的是索引查找。这种查询方式的运行速度比不使用索引的方式要快100万倍——毫不夸张。虽然这个例子是用来说明索引用途的，但是它所揭示的问题却是真实存在的，而且为表加上索引通常能够使性能得到显著提升。

MySQL使用索引的方式有以下几种。

如上所述，索引可以用于加快对 WHERE 子句匹配的行进行搜索的速度，或者用于加快对，与另一个连接表里的行匹配的行，进行搜索的速度。

对于使用MIN() 或 MAX() 函数的查询，MySQL可以在不用逐行检查的情况下，快速找到索引列里的最小值或最大值。

对于ORDER BY和GROUP BY 子句，MySQL经常使用索引来高效地完成分类和分组操作。

有时，MySQL 也会通过索引来读取查询所请求的所有信息。假设有一个InnoDB 或MyISAM 表，你要对这个表中的某个索引数字类型列进行查询，而且你并不打算查询该表的其他列。在这种情况下，MySQL读取索引值获得的值，跟通过读取相应的数据行得到的值一样。没理由读取两次那些值，因此根本就不用访问这个数据行。

5.1.2 索引的代价

一般来讲，如果 MySQL 知道如何通过索引来更快速地处理查询，那么它都会那样做。这意味着，在大部分情况下，如果不为表创建索引，那么你就是自残。对于索引的优点，我已描绘了一幅玫瑰般漂亮的画面。它没有缺点吗？有，它们在时间和空间上都有成本。在实际当中，这些缺点常被其优点所掩盖，但是你应该对此有所了解。

首先，索引可以加快检索速度，但是它同时也降低了索引列插入、删除和更新值的速度。也就是说，索引降低了大部分与写入相关的操作的速度。之所以会出现这样的情况，是因为写入一个行不仅是写入一个数据行，还要

更改索引。表的索引越多，需要做出的更改就越多，平均性能下降得也就越多。大部分的表都是读操作多写操作少，但对于那些写操作较多的表来说，在索引更新方面的开销会非常大。5.5节将讨论如何减少这方面的开销。

其次，索引也会占用磁盘空间，多个索引会占据更大的空间。与没有索引相比，使用索引会让你很快便达到表的大小极限。

对于 MyISAM 表，大量地对它进行索引，有可能导致索引文件比数据文件更快地到达其最大大小。

存储在 InnoDB 系统表空间里的所有 InnoDB 表，都共享同一个存储空间池，添加索引会使表空间里用于存储的空间减少得更快。不过，与 MyISAM 表所用的文件不同，InnoDB 系统表空间不会受到操作系统文件大小的限制，因为可以配置它使用多个文件。如果有其他的磁盘空间，那么你就可以采用将其添加为新部件的方式来扩展表空间。

使用独立表空间的 InnoDB 表，会把数据和索引集中存储在同一个文件里，因此增加索引将导致表的大小更加快速地到达文件的最大大小。

上述两个因素的现实指导意义是什么呢？如果不需要某个特定的索引来加快查询速度，那么就不要再创建它。

5.1.3 挑选索引

与索引创建有关的语法，已在 2.6.4.2 节里讲过了。这里假定你已阅读了该节的内容。但是只了解语法，并不能帮助你确定要创建哪些索引。该操作需要你认真考虑各个表的具体用途。本节将在如何确定出索引候选列方面给出一些指导。

(1) 为用于搜索、排序或分组的列创建索引，而对于用作输出显示的列则不用创建索引。也就是说，最佳索引候选列是那些出现在 WHERE 子句中的列、连接子句中的列，或者出现在 ORDER BY 或 GROUP BY 子句中的列。而那些只是出现在 SELECT 关键字后面的输出列表里的列，则不是很好的索引候选列：

```
SELECT
```

```
col_a
```

← 不是候选列

```
FROM
```



```
tbl1 LEFT JOIN tbl2
```

```
ON tbl1.col_b = tbl2.col_c ← 候选列
```

```
WHERE
```

```
col_d = expr; ← 候选列
```

当然，输出显示的列与WHERE子句里使用的列可能是相同的。重点在于，那个输出列的列表里出现的列自己并不足以说明它应该被索引。

出现在连接子句中的列，或者出现在WHERE子句中形如 `col1 = col2` 的表达式里的列，都是特别好的索引候选列。在上面的查询里，`col_b`和`col_c`都是这种例子。如果MySQL能使用多个连接列来优化查询，那么它不会使用全表扫描，从而大大减少潜在的“表-行”组合。

(2) 认真考虑数据列基数。列的基数 (cardinality) 是指它所容纳的所有非重复值的个数。例如，某个列包含值1、3、7、4、7、3，那么它的基数为4。相对于表里行的总数来说，列的基数越高（也就是说，它包含的唯一值多，重复值少），索引的使用效果越好。对于包含许多不同年龄值的列，索引可以很容易地将各个行区分开来。但是对于记录性别的列，其中只会包含两个值——'M'和'F'，索引操作毫无用处。如果这两个值出现的频率大致一样，那么不管搜索哪个值，你得到的都是近乎一半的行。在这种情况下，索引可能根本无法使用。因为当查询优化程序确定出某个值在表的行里出现频率很大时，它会跳过索引，转去执行全表扫描操作。

```
mysql> SELECT COUNT(*), COUNT(DISTINCT state) FROM member;
```

COUNT(*)	COUNT(DISTINCT state)
102	46

(3) 索引短小值。应尽量选用较小的数据类型。例如，当使用MEDIUMINT列便足以容纳你需要存储的数据时，就不要选用BIGINT；如果值的长度都不会超过25个字符，那么就不要用CHAR(100)。较短小的值可以在以下几个方面提高索引的处理性能。

短小值可以让比较操作更快，从而加快索引查找速度。

短小值可以让索引短小，从而减少对磁盘I/O请求。

对于更短小的键值，键缓存里的索引块可以容纳更多的键值。如果MySQL能在内存里同时容纳更多的键，那么就可以在不从磁盘读取更多索引块的前提下，提高找到键值的几率。

对InnoDB存储引擎，因为它使用的是聚簇索引（clustered index），所以让主键尽量短小会带来更多好处。聚簇索引会把数据行和主键值存储在一起（即聚集在一起）。其他的索引都是二级索引，即它们把主键值和二级索引值存储在一起。在二级索引里进行查找，会先得到主键值，然后再通过它定位到相应的行。这意味着，主键值在每一个二级索引里都会重复出现。因此，如果主键值较长，则会导致每一个二级索引需要占用更多的存储空间。

（4）索引字符串值的前缀。想要对字符串列进行索引，应当尽可能指定前缀长度。例如，你有一个CHAR(200)列，如果大多数值的前10个或20个字符都是唯一的，那么就可以不用为整个列进行索引。而只为前面的10或20个字符进行索引，这样可以节省大量的索引空间，而且还能使查询变得更快。（对较短小的值进行索引，可以获得上一条指导中描述过的好处，即较少磁盘I/O操作，加快索引速度。）当然，你可能会想继续沿用某些惯用做法。不过，只索引数据列的第一个字符恐怕不行，因为这样做会导致索引无法获得大量的唯一值。

可以对CHAR、VARCHAR、BINARY、VARBINARY、TEXT和BLOB类型列的前缀进行索引，具体语法请参考2.6.4.2节。

（5）利用最左前缀。当创建包含n个列的复合索引时，实际上会创建n个专供MySQL使用的索引。复合索引相当于多个索引，因为索引中最左边的任意数据列集合都可用于匹配各个行。这样的集合即为“最左前缀”。（这与对列的前缀进行索引有所不同，它会使用列值的前 n个字符或字节来创建索引。）

假设有一个表，它拥有一个多个列的复合索引，这些列的名字分别为country、state和city。在索引里，行的排列顺序为country/state/city，因此，行首先会自动按country/state顺序排序，然后再按 country 进行排序。这意味着，即使在查询里只指定了 country 值，或者只指定了country值和state值，MySQL也可以充分利用索引。因此，索引可用于搜索下面几列组合：

Country, State, city

Country, State

Country

对于没有包含最左边前缀的那些搜索，如按照state或city来搜索，MySQL无法使用索引。如果要搜索某个给定的州和某个特定的市（该索引的第1个列和第3个列），那么，尽管使用索引找到那些与州匹配的行可以缩小MySQL 搜索的范围，但是该索引还是不能用于这些值的组合。

（6）不要建立过多的索引。不要以为索引“越多越好”，然后就为你所能看到的所有数据列都建立索引。前面提到过，每增加一个索引都需要占据额外的磁盘空间，而且都会影响写入操作的性能。在对表做了修改之后，索引就会更新，并且可能还会重组，索引越多，整个过程所占用的时间就越长。很少使用或从不使用索引，会大大降低表的修改速度。此外，在为检索生成执行计划时，MySQL会对索引进行仔细推敲。创建多余的索引，会为查询优化程序增加更多的工作。当表有太多的索引时，MySQL还有可能（只是存在可能）无法使用最好的索引。为避免造成这样的错误，请维护好那些真正需要的索引。

如果你正在考虑要为某个有索引的表添加索引，那么请考虑这个需要添加的索引是否是已有索引的最左前缀。如果是这样，那么就不用再费心思去添加它，因为，事实上，你已有了这个索引。例如，已有一个对country、state和city的索引，那么再添加一个对country的索引，就没有意义了。但FULLTEXT 索引是一个例外，对于想要搜索的每一个不同的列集合，它们必须拥有单独的索引。

（7）让参与比较的索引类型保持匹配。在创建索引时，大部分存储引擎都会选择它们要使用的索引实现。例如，InnoDB总会使用B树索引。MyISAM也会使用B树索引，但对于空间类型则会改用R树索引。MEMORY存储引擎默认会使用散列索引，但它也支持B树索引，并允许你在这两者之间进行选择。在选择索引类型时，请考虑计划在索引列上执行的是什么类型的比较操作。

对于散列索引，会有一个散列函数应用于每一个列值。最终的散列值都会被存入索引，用于执行查找。（散列函数实现的算法，会尽量为不同的输入值生成不同的散列值。使用散列值的好处是它们之间的比较比其原始值更有效率。）在使用运算符=或<=>完成精确匹配的比较操作里，散列索引的速度非常快。但在那些用于查找一个范围内的值的比较操作里，它们却表现欠佳，如下列表达式：

id < 30

weight BETWEEN 100 AND 150

在使用<、<=、=、>=、>、<>、!=和BETWEEN运算符，进行精确比较或范围比较时，使用B 树索引会带来高效。如果匹配模式是以一个纯字符串，而不是一个通配符作为开头的，那么B树索引还可在使用运算符LIKE进行模式匹配的操作里。

如果只是为了精确查找才使用MEMORY表，那么使用散列索引则会是一个不错的选择。这也是 MEMORY 表的默认索引类型，因此不需要做什么特别的事情。如果通常是使用MEMORY表来进行范围比较，那么请改用B树索引。如果想要指定这种类型的索引，可以在索引定义里加上USING BTREE。例如：

```
CREATE TABLE lookup
(
    id          INT NOT NULL,
    name        CHAR(20),
    PRIMARY KEY (id) USING BTREE
) ENGINE = MEMORY;
```

如果所用的搜索类型允许，那么可以让一个MEMORY表同时拥有一个散列索引和一个B树索引，甚至可以对同一列进行索引。

有些类型的比较操作不能使用索引。如果只是通过把列值传递给STRCMP()那样的函数来进行比较，那么对列进行索引就毫无价值。服务器必须为每一个行计算一次函数值，这样就排除了在该列使用索引的可能性。

(8) 利用慢查询日志找出那些性能低劣的查询。这个日志可以帮助我们找出从索引当中获益的查询。（更多与MySQL日志有关的讨论请参考12.8节。）慢查询日志是一个文本文件，它可以用任何文件显示程序打开查看，也可以用mysqldumpslow工具来汇总其内容。如果某个给定的查询在这个日志里频繁出现，那么这个查询可能就不是最优的，需要改写，以让它运行得更快。在查看慢查询日志时，请记住“慢”是实时测量出来的，因此，与处于低负载时相比，服务器处于高负载时，会往慢查询日志里写入更多的查询。

5.2 MySQL查询优化程序

当你调用语句时，MySQL会先对它进行分析，判断出可以使用哪些优化操作来加快处理速度。本节将讨论查询优化程序是如何工作的。更多与MySQL所采用的优化度量方法有关的资料请参考MySQL参考手册中有关优化的章节。

当然，MySQL的查询优化程序会充分利用索引，但它同时也会利用其他的信息。例如，如果调用下面这条查询语句，那么不管表有多大，MySQL都能执行得很快：

```
SELECT * FROM tbl_name WHERE FALSE;
```

在这个示例里，MySQL会检查 WHERE 子句，并发现没有符合查询条件的行，因此根本就不会去搜索表。通过执行EXPLAIN语句，可以看到这一点。该语句会告知MySQL，把SELECT如何执行的（并非实际执行）相关信息显示出来。（自 MySQL 5.6.3开始，也可以将 EXPLAIN与DELETE、INSERT、REPLACE和UPDATE语句一起使用。）

要使用EXPLAIN，只需将单词EXPLAIN放在SELECT语句的前面即可。

```
mysql> EXPLAIN SELECT * FROM tbl_name WHERE FALSE\G
```

```
***** 1.row *****
```

```
id: 1
```

```
select_type: SIMPLE
```

```
table: NULL
```

```
type: NULL
```

```
possible_keys: NULL
```

```
key: NULL
```

```
key_len: NULL
```

```
ref: NULL
```

```
rows: NULL
```

Extra: Impossible WHERE

通常情况下，EXPLAIN 语句返回的信息比上面的多，包含信息更为丰富的值，而非只是一些 NULL 值。具体来讲，这些信息会包括将用于扫描各个表的索引、将要用到的连接类型，以及每个表都必须检查的行数的粗略估计。更多相关示例请参考5.2.2节。

在某些情形下，如果某个查询的FROM子句里包含了子查询，那么EXPLAIN会先执行这个查询的一部分：在对主SELECT语句进行分析之前，EXPLAIN必须先执行子查询，才能知道它们返回的具体内容。（自MySQL 5.6.3开始，不会再出现这种行为。）

5.2.1 查询优化程序的工作原理

查询优化程序有几个目标，但是其主要目标是：尽可能使用索引，并且使用最严格的索引来消除对行数量随时可能快速增加的顾虑。其中最后那个观点，听起来有悖常理。毕竟，调用SELECT语句的目标是要找到行，而不是要排除它们。相比之下，查询优化程序之所以要尽力排除行，是因为从搜索范围中排除行的速度越快，那么找到那些与搜索条件匹配的行的速度也会越快。如果首先执行最严格的条件测试，那么查询处理的速度也会更快一些。假设，有一个查询需要对两个列进行测试，那么它们当中的每一个都会有一个索引：

```
SELECT col3 FROM mytable
```

```
WHERE col1 = 'some value' AND col2 = 'some other value';
```

再假设，对col1列的测试会匹配到900个行，对col2列的测试会匹配到300个行；而同时测试这两个列，会匹配到30个行。如果首先测试col1列，得到900个行，那么为找到与col2列相匹配的那30个行，也必须对这900个行进行检查。也就是说，会有870个行测试失败。如果首先测试col2列，得到300个行，那么为找到与col1列相匹配的那30个行，只需对这300个行进行检查。也就是说，只有 270个行会测试失败，从而减少了对计算和磁盘 I/O操作的需求。最终，查询优化程序会首先检验col2，因为这样做的工作量更少。

下面描述的这些指导意见，有助于优化程序对索引的充分利用。

(1) 分析表。这将生成关于键值分布情况的统计数据，它们可以帮助优化程序更准确地评估索引效率。默认情况下，当把索引列里的值与某个常量进行比较时，优化程序会假设相关索引里的键值都是均匀分布的。当在判

断索引是否应该与常量进行比较时，优化程序还会对索引进行一次快速检查，以估算出会涉及多少索引项。对于InnoDB和MyISAM 表，可以使用ANALYSE TABLE语句告知服务器执行键值分析。

如果表在填充好数据之后一直未被改动，那么只需在加载后对它做一次分析即可。如果表经常会被更新，则应该间歇性地（与数据的更新频率保持一致）对它进行分析。

（2）使用EXPLAIN验证优化程序的操作。使用EXPLAIN语句可以了解到各个索引是否会被用到。当你尝试使用各种不同的办法来编写某一条语句时，或者很想知道增加索引对于查询命令的执行效率是否会有所改善时，这类信息可以派上用场。

（3）在必要时给予优化程序提示或改写它。在连接操作的表列表里，可以在表名的后面加上FORCE INDEX、USE INDEX 或 IGNORE INDEX，以告知服务器期望使用哪些索引。更多相关信息请参考附录E。

也可以通过 STRAIGHT_JOIN 强制要求优化程序按特定的顺序使用各个表。一般情况下，MySQL优化程序会自行确定各个表的扫描顺序，以求用最快的速度把各个行检索出来。有时，优化程序做出的选择不是最优的。如果遇到此种情况，可以用STRAIGHT_JOIN关键字来改写优化程序的选择。带有STRAIGHT_JOIN的连接操作，会强制各个表按照它们在FROM子句里的排列顺序进行连接。

如果真想这么做，那么可以尝试把各个表排列好，从而达到优先执行最严格查询的目的。也就是说，对表进行排序，让那个能使查询出的行数最小的表，排在最前面。查询性能越好，你就能越早地缩小可能的候选行范围。

在SELECT语句里，有两个地方可以指定STRAIGHT_JOIN。第一个地方在SELECT关键字与查询列的列表之间，对语句里的所有连接都起作用，具有全局效果。第二个地方在FROM 子句里。下面两条语句完全等效：

```
SELECT STRAIGHT_JOIN ...FROM t1 INNER JOIN t2 INNER JOIN t3
...;
```

```
SELECT ...FROM t1 STRAIGHT_JOIN t2 STRAIGHT_JOIN t3 ...;
```

一定要对比试试有和没有STRAIGHT_JOIN的情况。MySQL可能有更好的理由，不按你认为是最好的顺序使用索引，并且可能STRAIGHT_JOIN实际也起不到任何作用。这时，可以使用EXPLAIN语句检查一下具体的执行计划，看看MySQL是如何处理每一条语句的。

(4) 比较拥有相同数据类型的列。在对带有索引的列进行比较时，如果其数据类型相同，那么与类型不等的情况相比，查询性能会提高很多。例如，INT 与 BIGINT 不同，因此 INT/INT比较或BIGINT/BIGINT比较的速度，会比INT/BIGINT比较的更快。可以认为CHAR(10)，与CHAR(10)或VARCHAR(10)是同类型的，但是CHAR(10)与CHAR(12)或VARCHAR(12)是不同类型的。如果频繁进行比较的列拥有不同的类型，那么可以使用ALTER TABLE语句来修改其中的一个，从而使它们的类型相匹配。

(5) 让索引列在比较表达式中单独出现。如果你在函数调用中使用了列，或者将列当作是复杂算术表达式中的一部分，那么MySQL将不会使用该索引，因为它必须计算每一个行的表达式值。虽然有时这是无法避免的，但是在许多情况下，你都可以改写查询，让这个索引列单独出现。

下面两条WHERE子句说明了整个工作过程。它们在算术上是等效的，但其优化目的却大不一样：

```
WHERE mycol * 2 < 4
```

```
WHERE mycol < 4 / 2
```

对于第一行，MySQL必须检索每一个行的mycol值，然后乘以2，再将结果与4进行比较。在此情况下，不会使用索引。因为必须检索列中的每一个值，才能计算出比较操作中左侧表达式的值。对于第二行，优化程序将表达式4/2简化为值2，然后使用mycol的索引，快速地找到了那些小于2的值。因此，这里的第二行要好于第一行。

让我们来看另外一个示例。假设，有一个DATE类型索引列date_col。如果执行如下所示的查询，那么索引将不会被用到：

```
SELECT * FROM mytbl WHERE YEAR(date_col) < 1990;
```

这个表达式不会把1990与索引列进行比较；它会把1990与根据该列计算出来的那个值进行比较，而且每一个行都要计算那个值。因此，date_col的索引不会被用到，并且这个查询的执行过程也会要求进行全表扫描。要如何解决这个问题呢？可以使用文字型日期来解决这个问题。然后，优化程序就会使用date_col的索引来查找相匹配的列值：

```
WHERE date_col < '1990-01-01'
```

但是，如果没有准确的日期，又该怎么办呢？或许你只是想要把那些从今天算起一定天数以内的行找出来。有多种方式可以表达这种类型的比较操

作，但效果不尽相同。这里列出了其中的3种：

```
WHERE TO_DAYS(date_col) - TO_DAYS(CURDATE()) < cutoff
```

```
WHERE TO_DAYS(date_col) < cutoff + TO_DAYS(CURDATE())
```

```
WHERE date_col < DATE_ADD(CURDATE(), INTERVAL cutoff DAY)
```

其中，第一行不会使用索引，因为必须逐行检索这个列，才能算出 `TO_DAYS(date_col)` 的值。第二行则要好一些。其中的 `cutoff` 和 `TO_DAYS(CURDATE())` 都是常量，因此比较表达式右边的值可以在处理查询之前由优化程序一次性计算出来，而不用逐行计算。但是 `date_col` 列仍然会出现在函数调用里，阻止索引的使用。第三行是最好的。同样地，表达式右边的值可以在执行查询之前，被一次性地计算成一个常量，但是该值现在是一个日期。它可以直接与 `date_col` 列的值进行比较，而无需再转换成具体的天数。在此情况下，索引即可派上用场。

（6）不要在LIKE模式的开始位置使用通配符。有些字符串搜索会使用下列形式的WHERE子句：

```
WHERE col_name LIKE '%string%'
```

如果想要把无论是否出现在该列里的所有字符串都找出来，那么用这个子句是正确的。但不要习惯性地“%”放在字符串的两侧。如果只想把出现在该列开始位置的字符串找出来，则需要删除第一个“%”。假设，需要在包含姓的列里，把以'Mac'开头的那些姓（如 MacGreGor和MacDougall）都查找出来。此时，即可以使用类似下面这样的WHERE子句：

```
WHERE last_name LIKE 'Mac%'
```

优化程序会查看这个模式的文字初始部分，并使用索引来找到匹配的行，跟下面编写的表达式一样，这种形式的表达式允许使用 `last_name` 的索引：

```
WHERE last_name >= 'Mac' AND last_name < 'Mad'
```

这种优化方式不能用于使用REGEXP运算符的模式匹配。REGEXP表达式决不会被优化。

（7）利用优化程序的长处。MySQL 支持连接和子查询，但子查询支持是最近才开始有的功能。因此，优化程序在许多场合对连接的优化效果，要

比子查询的优化效果更好一些。在某个子查询运行得很慢时，这一点很有现实指导意义。正如2.9.7节讨论过的，有些子查询可以改写为逻辑上等价的连接操作。如果你的慢速子查询属于其中的一种情况，那么应该尝试把它改写成连接操作，看是否能运行得更快。（对于 MySQL 5.6之前的版本，这种策略的效果可能比较明显，因为在MySQL 5.6版本里，为提高子查询性能做了大量的改进工作。）

（8）测试查询的各种替代形式，并多次运行它们。当测试查询的各种替代形式（如子查询和等效的连接操作）时，对每种形式都要反复多次运行。如果让这两种查询都只运行一次，那么你会发现第二种查询方式通常会快一些，因为第一次查询的信息仍然保留在磁盘的高速缓存内，不需要从磁盘里读取。还应该在系统负载相对稳定的时候运行查询，这样可以避免系统上的其他活动对测试结果产生影响。

（9）避免过多使用自动类型转换。MySQL可以实现类型的自动转换，但是如果避免这类转换，则可能获得更好的性能。例如，num_col是一个整型列，下面两个查询都可以返回相同的结果：

```
SELECT * FROM mytbl WHERE num_col = 4;
```

```
SELECT * FROM mytbl WHERE num_col = '4';
```

但是第二个查询涉及类型转换。这个类型转换自身又涉及一个很小的性能损失问题，因为它需要将整数和字符串转换成双精度数，以完成比较操作。更为严重的问题是，如果num_col列带有索引，那么涉及类型转换的比较可能会阻止使用索引。

与之相对的比较操作（把一个字符串类型列与一个数值相比较）也会阻止使用索引。假设编写了下面这样一条查询命令：

```
SELECT * FROM mytbl WHERE str_col = 4;
```

此时，str_col列上的索引将无法使用，因为在str_col列里可能会有许多不同的字符串值在转换成具体数字后等于4（如'4'、'4.0'和'4th'）。区别它们的唯一办法是：依次逐个读取，转换它，再进行比较。如果想要在寻找某个特定的字符串值（如'4'）时避免上述问题，那么可以像下面这样在查询命令里指定它：

```
SELECT * FROM mytbl WHERE str_col = '4';
```


5.2.2 使用EXPLAIN检查优化程序的操作

EXPLAIN 语句提供的信息，有助于我们了解优化程序为处理各种语句而生成的执行计划。本节将展示EXPLAIN语句的两种用途。

确定以不同方式编写出来的查询命令，是否会影响索引的使用。

研究给表增加索引，对优化程序生成高效执行计划的能力会产生什么影响。

本节只描述与具体示例有关的那些EXPLAIN输出字段。对EXPLAIN输出内容的详细讨论请参考附录 E。书中给出的输出内容是在我的系统上得到的。根据服务器版本和配置的不同，你所看到的结果也会略有不同。

在之前的5.2.1节讨论过，编写表达式的方式可以决定优化程序能否使用已有的索引。具体来讲，那个讨论所用示例里涉及了 3个逻辑上等价的 WHERE 子句，而只有第三个才允许使用索引：

```
WHERE TO_DAYS(date_col) - TO_DAYS(CURDATE()) < cutoff
```

```
WHERE TO_DAYS(date_col) < cutoff + TO_DAYS(CURDATE())
```

```
WHERE date_col < DATE_ADD(CURDATE(), INTERVAL cutoff DAY)
```

EXPLAIN 语句可以让我们了解到：以何种方式编写表达式会更优。为了让大家有个直观印象，我们一起来尝试使用这三条WHERE子句，搜索member表里expiration列的值。其中，cutoff的值设置为30天。不过，在刚开始创建member表时，其expiration列并没有创建索引。为能够看到索引的使用情况与表达式编写方式之间的联系，需要先对expiration列进行索引：

```
mysql> ALTER TABLE member ADD INDEX (expiration);
```

接着，用EXPLAIN语句依次检查各种形式的表达式，看看优化程序为其生成了什么样的执行计划。

```
mysql> EXPLAIN SELECT * FROM member
-> WHERE TO_DAYS(expiration) - TO_DAYS(CURDATE()) < 30\G
***** 1.row *****
      id: 1
  select_type: SIMPLE
        table: MEMBER
         type: ALL
```

```

possible_keys: NULL
      key: NULL
    key_len: NULL
      ref: NULL
     rows: 102
    Extra: Using where
mysql> EXPLAIN SELECT * FROM member
      -> WHERE TO_DAYS(expiration) < 30 + TO_DAYS(CURDATE())\G
***** 1.row *****
      id: 1
    select_type: SIMPLE
      table: MEMBER
        type: ALL
possible_keys: NULL
      key: NULL
    key_len: NULL
      ref: NULL
     rows: 102
    Extra: Using where
mysql> EXPLAIN SELECT * FROM member
      -> WHERE expiration < DATE_ADD(CURDATE(), INTERVAL 30 DAY)\G
***** 1.row *****
      id: 1
    select_type: SIMPLE
      table: member
        type: range
possible_keys: expiration
      key: expiration
    key_len: 4
      ref: NULL

rows: 6

```

Extra: Using where

从前两条语句的输出结果可以看出，它们都没有使用索引。其中，type 的值表明了会如何从表读取各个值。ALL表示“将检查所有行”。也就是说，将执行全表扫描，用不上索引。对于名字里带有key字样的各个字段，其值为NULL，同样也表明不会使用任何索引。

相比之下，第三条语句的输出结果表明：像那样编写的WHERE 子句，会让优化程序使用expiration列上的索引。

type的值表明，优化程序可以使用索引来搜索特定范围内的值（即那些小于表达式的右半部分所给出的日期的值）。

possible_keys和key的值表明，优化程序会把expiration 列上的索引视为候选索引，而实际上也会使用这个索引。

row的值表明，优化程序估计完成此次查询需要检查6个行。这比前两个执行计划里要检查102个行好很多。

EXPLAIN的第二个用途是，验证增加索引能否有助于优化程序更有效地执行某条语句。例如，我起初只使用了两个无索引的表。这足以满足对简单连接操作创建索引的效果进行演示的需要。同样的原则也适用于涉及许多个表的更复杂的连接操作。

这个示例的输出源自 MyISAM 表，当涉及全表扫描时，用它们能更清晰地估算出要检查1 000个行，从而更易于把事情说清楚。对于InnoDB表，结果大致相同，只是全扫描的估算不是1 000个行，因为InnoDB使用了不同的估算机制。

假设，有两个表t1和t2，它们各有1 000个行，其中行包含的值为1～1000。下面是用于检验那些行的查询命令，它会把两个表里对应值相等的那些行找出来：

```
mysql> SELECT t1.i1, t2.i2 FROM t1 INNER JOIN t2
```

```
-> WHERE t1.i1 = t2.i2;
```

i1	i2
1	1
2	2
3	3
4	4
5	5

...

在两个表都没有索引的情况下，EXPLAIN的输出结果如下。

```
mysql> EXPLAIN SELECT t1.i1, t2.i2 FROM t1 INNER JOIN t2
```

```
-> WHERE t1.i1 = t2.i2\G
```

```
***** 1.row *****
```

```
id: 1
```

```
select_type: SIMPLE
```

```
table: t1
```

```
type: ALL
```

```
possible_keys: NULL
```

```
key: NULL
```

```
key_len: NULL
```

```
ref: NULL
```

```
rows: 1000
```

```
Extra:
```

```
***** 2.row *****
```

```
id: 1
```

```
select_type: SIMPLE
```

```
table: t2
```

```
type: ALL
```

```
possible_keys: NULL
```

```
key: NULL
```

```
key_len: NULL
```

ref: NULL

rows: 1000

Extra: Using where; Using join buffer

其中，type栏里的ALL值表明，将执行一次全表扫描，检查所有行。possible_keys栏的NULL 值表明，没有找到可以加快查询速度的候选索引。由于没有合适的索引，因此 key、key_len和ref栏的值全部为NULL。而Using where表明， WHERE子句里的信息将被用于标识出符合条件的行。

以上几段信息告诉我们，优化程序并没有发现任何有助于提高这条查询命令执行效率的信息，将进一步采取以下措施。

对t1表执行一次全表扫描。

针对t1表里的每个行，对t2表执行一次全表扫描；同时，使用WHERE子句里的信息来寻找符合条件的行。

rows栏里的值表明，优化程序对于该查询的每一个阶段所需要检查的行数的估算值。因为需要执行一次全表扫描，所以t1表的估算值是1 000。类似地，t2表的估算值也是1 000，但是t1表里的每一个行都会有这样一个值。换句话说，按照优化程序的估算，在处理这个查询命令时需要检查处理的行组合数为1 000×1 000，即100万。这是个非常浪费资源的操作，因为实际上只有1 000种组合满足WHERE子句里的那个条件。

为了让查询更有效率，可以为相关联的其中一个列增加一个索引，然后再次执行 EXPLAIN语句。

```
mysql> ALTER TABLE t2 ADD INDEX (i2);
```

```
mysql> EXPLAIN SELECT t1.i1, t2.i2 FROM t1 INNER JOIN t2
```

```
-> WHERE t1.i1 = t2.i2\G
```

```
***** 1.row *****
```

id: 1

select_type: SIMPLE

```

        table: t1

type: ALL

        possible_keys: NULL

key: NULL

        key_len: NULL

ref: NULL

rows: 1000

Extra:

***** 2.row *****

id: 1

        select_type: SIMPLE

        table: t2

type: ref

        possible_keys: i2

key: i2

        key_len: 5

ref: sampdb.t1.i1

rows: 10

Extra: Using where; Using index

```

情况有了明显改善。t1 表的EXPLAIN 结果表明，对它执行仍是全表扫描，但优化程序对t2表的处理有了差异。

type的值从ALL变成了ref，意思是说可以通过引用值（来自t1表的值）来完成索引查找，以在t2表里定位到符合条件的行。

该引用值被指定在ref字段里，即sampdb.t1.i1。

row的值从1000下降到了10，这表明优化程序相信：对于t1表里的每个行，在t2表里只需要检查 10个行即可。（这是个保守估计。事实上，对于t1 表里的每个行，在t2 表里只会有 1个行与之匹配。稍后会看到，我们将如何帮助优化程序改进这个估计值。）现在，行组合的总估算值为1000×10 = 10 000。这比之前没有任何索引时的100万估算值减少许多。

有必要为t1表也加上索引吗？要知道，对于这个特定的连接操作，无论如何也需要对其中一个表执行全表扫描，而全表扫描是不需要索引的。为看看它到底有无影响，可先给t1.i1列加上索引，然后再次运行EXPLAIN。

```
mysql> ALTER TABLE t1 ADD INDEX (i1);
```

```
mysql> EXPLAIN SELECT t1.i1, t2.i2 FROM t1 INNER JOIN t2
```

```
-> WHERE t1.i1 = t2.i2\G
```

```
***** 1.row *****
```

```
id: 1
```

```
select_type: SIMPLE
```

```
table: t1
```

```
type: index
```

```
possible_keys: i1
```

```
key: i1
```

```
key_len: 5
```

```
ref: NULL
```

```
rows: 1000
```

Extra: Using index

***** 2.row *****

id: 1

select_type: SIMPLE

table: t2

type: ref

possible_keys: i2

key: i2

key_len: 5

ref: sampdb.t1.i1

rows: 10

Extra: Using where; Using index

这里的输出结果与前一条EXPLAIN的结果很相似。不过，在新增索引之后，t1表的输出内容有了一些变化。其中，type的值从NULL变成了index，而Extra的值从空白变成了Using index。这些变化表明，虽然仍需要对已索引过的那些值进行完整的扫描，但是优化程序现在可以直接从这个索引里读取值，而不用再去读取数据文件了。对于MyISAM表，当优化程序知道可以从索引文件里获得所需要的全部信息时，你便会看到这样的结果。对于InnoDB表，当优化程序只使用来自索引的信息即可完成查询任务，而无需进行查找便能获得行时，你也会看到这样的结果。

接下来，为了帮助优化程序获得更有价值的估算值，将进一步运行ANALYSE TABLE命令。这将使服务器生成关于键值分布情况的统计数据。分析这些表，并再次运行EXPLAIN，便可得到更准确的rows估算值。

```
mysql> ANALYZE TABLE t1, t2;
```

```
mysql> EXPLAIN SELECT t1.i1, t2.i2 FROM t1 INNER JOIN t2
```


-> WHERE t1.i1 = t2.i2\G

***** 1.row *****

id: 1

select_type: SIMPLE

table: t1

type: index

possible_keys: i1

key: i1

key_len: 5

ref: NULL

rows: 1000

Extra: Using index

***** 2.row *****

id: 1

select_type: SIMPLE

table: t2

type: ref

possible_keys: i2

key: i2

key_len: 5

ref: sampdb.t1.i1

rows: 1

Extra: Using where; Using index

此时，优化程序估计：对于t1表里的每一个值，在t2表里只会有一行与之匹配。

5.3 选择利于高效查询的数据类型

对数据类型的选择，会在许多方面影响查询性能。本节提供了一些与数据类型选择有关的建议，它们有助于提高查询命令的运行速度。

多用数字运算，少用字符串运算。数字运算通常比字符串运算更快。以比较运算为例。数字之间的比较，可以在一个运算里完成。而字符串之间的比较，则需要多次进行字节与字节，或字符与字符的比较才能完成，而且字符串越长，比较的次数越多。

如果字符串类型列的取值个数有限，那么可以选用ENUM或SET类型，这样可以获得数字运算所带来的好处。这些类型在内部是以数字形式表示的，处理效率更高。

一起来看看字符串的其他表示方法。有时候，把字符串值表示成数字可以显著改善性能。例如，以点记号表示的IP地址，如 192.168.0.4，通常就是一个字符串。但这种 IP地址很容易转换成整数形式，即可以把构成IP地址的4组数字依次存入INT UNSIGNED 类型的4个字节里。整数不仅节约存储空间，而且还可以加快检索速度。但从另一方面来看，把IP地址表示成INT值又会给模式匹配运算带来困难，例如，你想要查找给定子网里的IP地址，就要用到模式匹配。或许，你还可以用位掩码运算来做这样的事情。这类问题说明：不能只能考虑空间问题，还必须根据数据的具体用途和用法，为它们选择一种最合适的表示形式。（对于 IP地址这个示例，不管你做何选择，INET_ATON()和INET_NTOA()函数都可以帮你完成两种表示形式之间的转换。）

当较小类型够用时，就不用较大类型。MySQL 处理较小类型的速度比较大类型快。尤其是对于字符串，其处理时间与长度有直接关系。选用较小类型的另一个好处是，可以让整个表变得更小，从而减少在磁盘读写方面的开销。对于那些带索引的列，较短小的值还能进一步提升性能。除了索引能加快查询速度以外，短键值的处理速度也较长键值的快。

对于使用固定长度数据类型的列，应该根据其取值范围选用最小的类型。如果MEDIUMINT够用，就不选BIGINT。如果FLOAT足以满足精度要求，就

不选DOUBLE。如果选用固定长度的CHAR 列，则不要把它们设置得太长。如果存储在列里的最长值有40个字符，那么就把它定义成CHAR(40)，而不要定义为CHAR(255)。

对于可变长度的类型，选用较小的类型仍可以节约空间。BLOB类型会使用2个字节来记录值的长度，而LONGBLOB类型会使用4个字节。如果要存储的值的长度没有超过64 KB，那么选用BLOB可以让每个值节省2个字节。（对于TEXT类型，也有类似的考虑。）

把数据列声明成NOT NULL。如果数据列为NOT NULL，那么MySQL对它的处理速度也会更快。这是因为在查询处理期间，不再需要检查该列的值是否为NULL。这样可以让你编写出更为简单的查询，因为不用再把NULL当作一种特例去检查，并且更简单的查询处理起来通常会更快。

考虑使用ENUM 列。如果字符串列的基数低（即差异值的个数很少），则可以考虑把它转换成ENUM列。ENUM值的处理速度很快，因为它们的内部表示形式皆为数字。

使用PROCEDURE ANALYSE()。运行 PROCEDURE ANALYSE()，可以看到许多关于表里各列的信息：

```
SELECT * FROM tbl_name PROCEDURE ANALYSE();
```

```
SELECT * FROM tbl_name PROCEDURE ANALYSE(16,256);
```

输出结果里的一列，对适用于表里各个列的优化数据类型给出了建议。第二个示例会告知PROCEDURE ANALYSE() 不要建议这样的ENUM 类型：它包含的值超过 16个，或者总长度超过256个字节（可以根据实际情况更改这些值）。如果不加这些限制，那么输出结果可能会很长；对于ENUM类型的定义，其可读性很差。

基于PROCEDURE ANALYSE() 的输出，你可能会发现，可以修改一下自己的表，利用效率更高的类型。想要更改列的类型，可以使用ALTER TABLE。

整理表碎片。对于那些频繁修改的表，尤其是那些包含有可变长度数据列的表，往往会产生大量碎片。碎片是有害的，因为它会造成存储表的磁盘块空间的浪费。随着时间的推移，你必须读取更多的磁盘块，才能获得有效的行，而这无疑会降低性能。包含可变长度行的表都会产生碎片，但是BLOB或TEXT列受到的影响特别显著，因为它们的大小变化非常大。

定期使用OPTIMIZE TABLE，可以消除或减少碎片化的MyISAM或InnoDB表里的空间浪费，并有助于防止性能降低。适用于各种存储引擎的碎片整理方法是：先用mysqldump转储表，然后再利用这个转储文件重建它：

```
% mysqldump db_name tbl_name > dump.sql
```

```
% mysql db_name < dump.sql
```

把数据压缩到BLOB或TEXT列。使用BLOB或TEXT列来存储那些可在应用程序里对其进行压缩和解压缩的数据，能够达到使用单个检索操作（而非多个操作）找出所有内容的目的。此办法特别适用于存储那些难以用标准表结构表示的数据，或者那些会随时间变化的数据。在第2章对ALTER TABLE语句的讨论里，其中有一个示例处理了一个表，这个表中存储了Web问卷里各个题目的答案。该示例讨论了，当需要往那个问卷里增加新题目时，如何用ALTER TABLE来往表里增加列。

这类问题的另一种解决方案是：让负责处理Web表单的应用程序把答案数据压缩成某种结构的数据，然后再把它插到单个的BLOB或TEXT列里。例如，可以使用XML来表示各个问卷的答案，然后把这个XML字符串存储到一个TEXT列里。这种做法会增加客户端应用的负担，需要客户端对数据进行编码（当从表中检索行时，还需要对数据进行解码），但是它大大简化了表的结构，并且在修改问卷时不需要更改表的结构。

另一方面，BLOB和TEXT 值也会带来一些麻烦，尤其是在执行大量的DELETE 或 UPDATE操作时。删除这些值会在表里留下大量的空白，而这些空白在将来会被一个行或者长短不同的多个行所填补。可以使用前面讨论过的碎片处理技术来解决这个问题。

使用合成索引。合成索引列有时很有用。一种做法是，先根据表里的其他列计算出一个散列值，把它存储到一个单独的列里。然后通过搜索散列值来检索行。不过，这个技术只适用于精确匹配型查询。（散列值对于使用像“<”或“>=”这样的运算符实现的范围搜索毫无用处。）散列值可以通过MD5()函数来生成。也可以选择使用函数SHA1()或CRC32()。当然，还可以在应用程序里用你自己的算法来计算散列值。请记住，数字型散列值的存储效率非常高。如果选用的散列算法有可能生成带有尾部空格的字符串，那么请不要选用那些会自动去除尾部空格的数据类型来存储它们。

合成散列索引对BLOB和TEXT列非常有用。与直接搜索BLOB或TEXT列自身相比，把散列值当作标识符值进行查找，可以让查找速度更快。

避免检索很大的BLOB或TEXT值，除非迫不得已。例如，只有在确定WHERE子句所限制的结果就是你想要找的那些行的时候，才可以使用检索所有行的SELECT *查询。否则，你可能会毫无目的地通过网络拉取一些很大的值。此时，把BLOB或TEXT值的散列标识符信息存储在合成索引列中，会很有帮助。可以先搜索列，以确定想要的行；然后再从这些行里把那些BLOB或TEXT值检索出来。

把BLOB或TEXT列剥离出来形成一个单独的表。在某些场合，把表里的BLOB或TEXT列剥离出来，存入一个附表，可能会更有意义，前提是，这样做可以让你把这个表的其他列转换成行固定长度的格式。这样能够减少主表里的碎片，并且可以让你享受到行固定长度所带来的好处。此外，它还能让你在主表上运行SELECT *查询，不用通过网络拉取大的BLOB或TEXT值。

5.4 选择利于高效查询的表存储格式

有些存储引擎实现了多种存储格式，其中每一种都有其自己的性能特性。下列建议为选择合适的格式提供了一些一般性的指导，但对于某个具体的应用，最好的做法是执行测试，以进一步验证某种格式是否真的比另一个更优秀。

如果所有列的长度固定，那么MyISAM存储引擎默认会使用固定长度的行，但如果任何一列的长度都是可变的，那么它会使用可变长度的行。对于字符串列，可以通过是选用CHAR 还是 VARCHAR（或者是选用BINARY，还是选用VARBINARY）来改变这一情况。在空间和时间之间要多做综合考虑。对于最大长度为n的列，CHAR在每一行都要占用n个字符空间。而VARCHAR平均占用的空间只有它的一半，因此如果空间很紧张，则需要使用VARCHAR列。如果速度是主要关心的内容，并且可以负担空间代价，那么可以使用（固定长度的）CHAR列，因为MyISAM处理固定长度的行的速度，比变长的行快。对于那些会频繁修改，并因此而产生大量碎片的表，更是如此。

对于变长行，由于各行的大小不一，因此当执行过多的删除或更新操作时，会产生更多的碎片。为维护好性能，你需要定期运行OPTIMIZE TABLE。对于固定长度的行，这一点不是问题。

当表崩溃时，具有固定长度行的表更易于重建。对于固定长度的行，每一行的开始位置都是确定的，因为它们都是行大小的倍数，而对于变长行，情况则有所不同。这个问题与查询处理的性能无关，但是它可以加速表的修复过程。

由于MEMORY表使用的是固定长度的行，而且其中的CHAR和VARCHAR列都被隐式地当作CHAR来对待，因此不管你选择哪一个都没关系。

InnoDB并不会区别对待固定长度列和可变长度列（所有行都使用一个指向列值的头指针），所以使用CHAR列在本质上不会比使用VARCHAR列更简单。因此，主要的性能因素在于行占用的存储量。对于最大长度为n的列，VARCHAR平均占用的空间比CHAR更少，并且减少了存储量以及为处理行而发生的磁盘I/O数量。即使对于NULL值，CHAR也会占用n个字符空间，因此，对于有许多NULL值的列，使用VARCHAR的优势会更加明显。

当创建InnoDB表时，请选择其特性与存储在表中的数据完美匹配的行存储格式。

默认情况下，InnoDB会使用COMPACT行格式。这种选择适合于大部分的情形。

对于包含重复数据的表，使用COMPRESSED 行格式，能带来更多的好处。这种表占用的空间更少（从而减少了读取它的那些I/O操作的数量），而节省下来的读取数据的时间，远远超过解压它所需的CPU时间。压缩格式对于存储随机值或已压缩过的值的表没有用。

对于带有长BLOB或TEXT值的表，DYNAMIC行格式最有效。

想要为新的InnoDB表指定行格式，可以使用ROW_FORMAT表选项。例如：

```
CREATE TABLE t1 (...) ENGINE=InnoDB
ROW_FORMAT=COMPRESSED;
```

想要检查已有表的行存储格式，可以使用SHOW TABLE STATUS。想要更改表的格式，可以使用ALTER TABLE：

```
ALTER TABLE t1 ROW_FORMAT=DYNAMIC;
```

行格式COMPRESSED和DYNAMIC需要Barracuda文件格式，而它又要求正确设置系统变量innodb_file_per_table和innodb_file_format。更多相关信息请参考 12.5.3.1 节的第 4小节。

5.5 高效加载数据

在大部分时间里，你最关心的问题很可能是对 SELECT 语句的优化，因为它们是最常用的查询语句，并且想要弄明白如何优化它们并不总是件简单

的事情。相比之下，将数据加载到数据库则较为简单。尽管如此，还是有许多策略可用于提高数据加载的效率。这些基本原则如下所述。

把数据从缓存刷新到磁盘的次数越少，那么数据加载的速度就越快。因此，批量加载的效率比单行加载的效率更高，因为插入的行可以先缓存，然后在加载操作结束时再刷新到磁盘。与逐行刷新到磁盘相比，在加载结束时一次性刷新，能够显著地减少磁盘I/O操作。

表的索引越少，加载速度越快。如果有多个索引，那么不仅要把行的内容添加到表里，而且要修改每个索引，这样才能反映出有新的行加入。因此，请不要创建毫无必要的索引，或者，如果这些索引已存在，请删除它们。

与长 SQL 语句相比，短语句的数据加载速度更快。这是因为它们在服务器上的解析操作会更少，并且它们可以更加快速地通过网络从客户端发送到服务器。

在上面这几个因素中，有的看上去微不足道（尤其是最后一个）。但是如果加载的数据量很大，那么即使很小的效率影响因素也会造成很大的影响。依据这里讨论的一般性原则，我们可以得出几个有关如何快速加载数据的实用结论。

LOAD DATA（包括各种形式）的效率比INSERT的高，因为它是批量加载行。服务器一次只能解析和解释一条语句，而不会同时处理几条语句。另外，只有在所有行都处理完之后，索引才会需要刷新，而不是在每个行处理完之后，便立即去刷新。

LOAD DATA在不带LOCAL的情况下效率会更高。如果不带LOCAL，那么数据文件必须加载到服务器上，并且必须要有FILE权限，这时，服务器可以直接从磁盘读取这个文件。对于LOAD DATA LOCAL，客户端程序会先读取文件，然后通过网络将它发送到服务器。因此，这种方式会慢一些。

如果必须使用INSERT，那么请尝试使用那种允许在单条语句里插入多个行的格式：

```
INSERT INTO tbl_name VALUES(...),(...),...;
```

在这种语句里，指定的行越多，效果越好。这样可以减少总的所需要语句数量，并大大减少索引刷新的次数。与前面讨论过的那条原则（即短语句处理起来比长语句更快）相比，这条原则看上去与之存在矛盾。但实际上并不矛盾。这里强调的是：与使用一组等效的插入单行的INSERT语句相比，使

用单条可插入多个行的INSERT语句，总体上花费的时间会更短，而且服务器在处理多行插入语句时，需要的索引刷新次数会更少。

如果使用mysqldump程序来生成数据库备份文件，那么它会默认生成可插入多行的INSERT语句：启用--opt（优化）选项，此选项会打开--extended-insert选项，产生可插入多行的INSERT语句；此外，还会启用一些其他选项，使得在重新加载转储文件时，能更高效地处理转储文件。

要避免在mysqldump 程序里使用--complete-insert 选项。否则，最终生成的INSERT语句将是针对单行的，与可插入多行的语句相比，它们更长，且需要执行更多的解析处理。

如果必须使用多条 INSERT 语句，则请尽可能把它们分组，以减少刷新索引的次数。对于支持事务的存储引擎，则可以在单个事务里执行这些INSERT 语句，而不要以自动提交的方式来执行：

```
START TRANSACTION;

INSERT INTO tbl_name ...;

INSERT INTO tbl_name ...;

INSERT INTO tbl_name ...;

COMMIT;
```

对于不支持事务的存储引擎，可以先占用对表的写入锁，然后在表被锁定时，执行 INSERT语句：

```
LOCK TABLES tbl_name WRITE;

INSERT INTO tbl_name ...;

INSERT INTO tbl_name ...;

INSERT INTO tbl_name ...;

UNLOCK TABLES;
```

无论哪种情况，你都能获得同样的好处：只有在所有语句都执行完以后，MySQL才会刷新一次索引；而不是在每个INSERT 语句执行完之后都刷新

一次。当在自动提交模式下，或者表未被锁定时，才会出现第二种情况。

对于MyISAM表，减少索引刷新次数的另一种策略是，使用DELAY_KEY_WRITE表选项。使用这个选项，行便可以像往常一样被立刻写入数据文件，但键缓存只有在必要时才会刷新一次，而不是每次插入之后都会立刻刷新。为了在服务器范围的基础上使用延迟索引刷新，可以先将delay_key_write系统变量设置成ALL，然后再启动mysqld。此时，服务器会把表的索引块写入操作，延迟到为了给其他索引值腾出空间而必须把存储块刷新到磁盘为止，或者延迟到执行FLUSH TABLES语句为止，或者延迟到表被关闭为止。

如果对MyISAM表使用“键写入延迟”功能，那么当服务器意外关机时，可能会导致索引值的丢失。这并不是一个致命问题，因为MyISAM表的索引可以根据其数据行进行修复。为了确保真的进行修复，在启动服务器时，可以把myisam_recover_options系统变量设置成某个包含FORCE选项的值。这个选项会强制服务器在打开MyISAM表时对它们进行检查，并根据需要自动修复它们。

对于复制机制中的从服务器，你可能会把delay_key_write设置成ALL，以延迟所有MyISAM表的索引刷新，不管它们在主服务器上是如何创建的。

使用压缩的客户端/服务器协议，可以减少通过网络传送的数据量。对于大部分的MySQL客户端程序，都可以使用--compress命令行选项来指定。通常情况下，这种做法只适合用在传输速率较低的网络上，因为压缩过程会占用一部分处理器的时间。

让MySQL插入默认值。也就是说，不在INSERT语句里指定各个列，让其随意分配为默认值。平均起来看，你的语句会更短一些，从而可以减少通过网络发往服务器的字符数。此外，因为这些语句包含的值较少，所以服务器执行解析和值转换的操作也会较少。

对于MyISAM表，如果你需要将大量的数据加载到某个新表里，那么可以先创建不带索引的表，然后再创建索引。一次性创建全部的索引要比逐行修改它们更快一些。对于已带有索引的表，如果事先删除索引或关闭它，事后再重建索引或重新激活它，那么数据的加载速度可能会更快。

如果要删除和重建索引，可以使用DROP INDEX和CREATE INDEX语句，或者使用ALTER TABLE语句与索引有关的形式。如果要关闭索引或重新激活它，那么可以使用ALTER TABLE的DISABLE KEYS和ENABLE KEYS形式，它们可以分别打开和关闭表的非唯一性索引：

```
ALTER TABLE tbl_name DISABLE KEYS;
```

...多条用于加载表内容的语句 ...

```
ALTER TABLE tbl_name ENABLE KEYS;
```

如果使用LOAD DATA 语句来把数据加载到一个空白的MyISAM表里，那么服务器会自动执行索引关闭和激活。另外，mysqldump默认也会添加ALTER TABLE语句。

对于InnoDB表，删除和添加附表的速度很快，因此在加载大量数据之前可以考虑这样做。这种做法并不适用于主（集群）索引，因此请不要删除和添加那个索引。

如果你正在考虑使用这种删除或关闭索引的策略来把数据加载到表，那么请对总体情况进行仔细评估，看看是否能从中获得某些好处。如果只是加载少量的数据到某个大表里，那么与不做任何特殊处理只是加载数据相比，重建索引可能会花费更长的时间。

前面所讨论的那些数据加载原则，也适用于混合查询环境：其中涉及需要执行多种不同类型操作的多个客户端程序。例如，你通常会希望尽量避免对那些经常会变化（即写入）的表，长时间运行 SELECT 查询。因为那样做会造成多个写入者之间的竞争，会降低性能。如果大部分的写入动作都是 INSERT 操作，那么这个问题也许能这样解决：先把各个新行添加到一个辅助表里，然后再将这些行定期地添加到主表。如果你需要能够立即访问新行，那么就不能使用这种方法。但是，如果你能保证在较短时间内不去访问它们，那么使用辅助表会有两个好处。首先，可以减少发生在主表上的多条 SELECT 查询语句之间的竞争，让它们执行得更快。其次，从辅助表里把行批量加载到主表，总体来讲，会比单独加载各行更省时，因为大批量加载操作的速度更快。

此策略的一种应用场景是：你想要把Web页面的访问记录从Web服务器写入MySQL数据库。此时，确定哪些条目需要立即进入主表，可能就不是一件需要优先考虑的事情。

5.6 调度、锁定和并发

前面几节的重点是如何加快单个查询的速度。本节将深入了解 MySQL 的调度策略，以及存储引擎锁定层在多个客户端之间的并发性方面所具有的常规影响。当需要针对具体应用选择存储引擎时，可以参考一下本节的信息。

对于给定应用里处于主导地位的查询类型，可能会有一种存储引擎比其他引擎更适合用于处理它。为了方便讨论，我们把负责检索（操作语句为SELECT）的客户端定义为读取者，把修改表（操作语句为DELETE、INSERT、REPLACE或UPDATE）的客户端定义为写入者。

MySQL的调度策略总结如下。

写入的优先级比读取的高。

表的写入操作一次只能进行一个，多个写入请求按其到达的先后顺序依次处理。

可以同时处理多个对同一个表的读取操作。

InnoDB存储引擎利用行级的锁定操作，实现这种调度策略，但InnoDB只有在必要时才会锁定行。在许多情况下，如只读的操作完成之时，InnoDB根本不使用锁定操作。

存储引擎MyISAM、MERGE和MEMORY，实现锁定操作的层级有所不同，使用了表锁定。因此，就竞争管理而言，它们具有完全不同的性能特征。只要客户端访问表，就必须要先锁定它。当这个客户端完成了对表的操作时，才会解除锁定。执行LOCK TABLES和UNLOCK TABLE语句可以显式地获得和解除锁定。但在通常情况下，服务器的锁定管理器都可以在需要时自动获得锁定，而在不需要时，解除它们。所需的锁定类型取决于客户端执行的是写入操作，还是读取操作。

为把数据写入表，客户端必须具有对表进行互斥访问的锁定。在写入操作处理的过程中，表处于一种不一致的状态，因为其中的数据行可能正在被删除、添加或更改，并且，为了保持匹配，表上的所有索引可能也需要随之更新。当表在不断变化时，允许其他客户端访问它，则会引发许多问题。允许两个客户端同时对一个表进行写入，显然是件很糟的事情，因为该表很快就会变得混乱不堪，最终导致无法使用。另外，允许客户端从一个不断变化的表中读取数据，也不是件好事，因为表有可能就在正被读取的那个位置进行更改，而此时的读取结果也会不准确。

为从表里读取数据，客户端必须锁定它，以防止其他客户端把数据写入表，或者在读取数据期间修改它。但是，这个锁定操作不需要互斥访问。因为读取数据并不会更改表，所以一个读取者没有理由禁止其他读取者访问这个表。因此，读取锁定可以让其他客户端同时读取表的数据。

存储引擎使用的锁定级别，对客户端之间的并发性有显著的影响。假设有两个客户端，它们各自都想要在给定的表里更新某行。为完成这个更新，每个客户端都会需要一个写入锁定。此时，InnoDB表的并发性明显要比MyISAM表的并发性好很多。对于InnoDB表，只要两个客户端不会同时更新同一行，那么两个更新操作就可以同时处理。对于MyISAM表，其存储引擎会为第一个客户端获得表锁定，从而导致第二个客户端阻塞，一直到第一个客户端操作完成为止。

一般情况下，更精细的锁定会有更好的并发性，因为如果各个客户端使用的是表的不同部分，那么可以让更多的客户端同时使用这个表。实际的影响是：不同的存储引擎适合于不同的查询语句混合情况。

当有许多更新操作时，InnoDB表可以提供更好的性能。因为完成锁定操作的级别是行级，而非表级，所以表被锁定的范围相对较小。这种做法可以减少锁定竞争，从而增强并发性。

MyISAM表的检索速度极快。但是，在有多个检索和更改操作混杂在一起的环境里，特别是当检索需要运行较长时间时，使用表级锁定可能会引发问题。在这些情况下，更新操作可能需要等待很长时间才能得到处理。

就死锁防止而言，表锁定能比更精细的锁定带来更多的好处。使用表锁定，不会出现死锁问题。服务器可以通过查看语句来确定需要哪些表，并提前把它们全部都锁定。对于InnoDB表，可能会出现死锁问题，因为在事务开始的时候，该存储引擎还没有获得所有必要的锁定。事实上，在事务处理过程中，只有在必要时才需要获得锁定。因此，有可能出现这种情况，即两个查询都获得了锁定，然后它们会试图进一步获得这样的锁定：它们每一个都在期望那些已占用的锁定被解除。结果，每一个客户端都在它能继续下一步操作之前，占据着其他客户端所需要的锁定。这就会导致死锁，并且只有服务器中止其中一个事务才能解决此问题。

第6章

MySQL程序设计入门

本章讲述为何要自己编写基于 MySQL 的程序，而不使用MySQL发行版里包含的那个标准化客户端程序的原因。另外，本章还对随后几章将会涉及的3种程序设计语言（C、Perl和PHP）的编程接口做了概念性的介绍，并对选择程序设计语言时需要考虑的因素进行了讨论。

6.1 为何要自己编写MySQL程序

MySQL发行版中有一组与服务器进行通信的客户端程序。例如，`mysqldump`可用于导出表的定义和内容；`mysqldamin`可用于管理操作；`mysql`可用于执行任意的SQL语句。

这些标准客户端程序足以处理 MySQL 用户日常遇到的绝大多数问题，但有些应用的需求则远远超出这些程序的能力范围。为解决此类问题，MySQL提供了一个客户端应用程序编程接口（Application Programming Interface, API），用于满足应用程序可能会有的任何特殊要求。客户端API提供了对MySQL服务器进行访问的能力，其开放程度完全超出你的想象。

本章将讨论，如何编写基于 MySQL 的可对数据库进行访问的程序。为让大家明白这样做的好处，下面把你自己编写程序所能完成的任务，与`mysql`客户端程序的能力和其访问MySQL服务器的便捷接口进行对比。

定制输入处理。使用`mysql`，只能输入原始的SQL语句。使用自己的程序，可以为用户提供更直观、更容易使用的输入方法。使用编写程序的用户，不需要掌握 SQL 语言，甚至不用关心完成任务所需的数据库角色。输入形式可以丰富多样，从简单的命令行接口（提示用户、读取值），到更复杂

的屏幕录入表单，都能实现。例如，大部分人发现：通过填写表单的方式来指定搜索参数，比通过执行SELECT语句来指定，容易得多。

用户输入验证。可以要求某些字段必须以某种方式进行填充。这样可以增强应用程序的安全性。

定制输出。mysql的输出基本上是无格式的，只能选择制表符分隔或表格样式。如果想获得更美观的输出，则只能自己动手格式化。有简单一点的，比如可以把NULL输出为“缺失”。也有复杂一点的，比如票据输出，每个票据的表头都需要与客户的信息以及每个定单项的信息关联起来。这类报告很容易超出mysql的格式化能力。

可以克服SQL语言的先天不足。多数情况下，SQL脚本都是由一组语句构成，它们从头到尾一次性执行完成，并具有少量的出错检查功能。如果使用mysql以批处理模式执行一个SQL查询文件，那么在遇到第一个错误时，mysql要么选择退出，要么不管出现了多少错误，不间断地执行完所有的查询（如果指定了--force选项）。如果自己编写程序，那么可以在语句执行操作的前后进行流程控制，有选择地对成功或失败情况做相应的处理。你可以根据某个查询的成败转去执行另一个查询，也可以根据上一个查询的结果，决定下一步该怎么做。

MySQL支持存储程序——它所提供的流程控制和出错处理机制，在SQL这一层提供了更多的灵活性。但是，这些机制所提供的灵活性，远远无法与各种通用程序设计语言所提供的灵活性相提并论。

总之，需要一个不同于mysql的工具，用于完成涉及“主从”关系的任务，以及有着复杂输出格式要求的任务。程序像“粘合剂”一样，把多个查询连接在一起，让你能够把一个查询的输出用作另一个查询的输入。

把MySQL集成到任何应用程序里。许多程序都享受到了利用数据库来提供信息的好处。

客户端编程接口为你提供了具体的实现方式。对于必须核对顾客编号，或者检查库存情况的应用程序，可以通过执行一条简单的查询来实现。对于能让客户通过特定作者来查询所有书籍的Web应用程序，可以在数据库里把书籍查找出来，然后把结果发送回该客户的浏览器。

在第1章，我们列举了几个与样本数据库 sampdb 有关的目标，其中即包含了编写程序与MySQL服务器进行交互的要求。下面列出的是其中几个目标。

格式化输出美史联盟的成员名录。

提供在线查看和搜索成员名录的功能。

发送电子邮件提醒成员更新成员资格。

使用Web浏览器简便地把学生成绩输入成绩册。

还有一个需要我们深入思考的问题，即如何把 MySQL的各项功能集成到 Web 环境中。MySQL不直接支持Web应用程序，但是，将MySQL与某些合适的工具结合在一起，你便可以代表客户端用户从你的Web服务器上执行查询，并把结果发送给用户浏览器。这样一来，便可以很容易地通过Web访问数据库。

下面从彼此互补的观点来看待MySQL和Web的结合。

利用Web服务器提供对 MySQL的增强访问。从此角度来看，你的主要兴趣是数据库，并且想要把Web当作一种工具，以便更容易地访问数据库里的数据。这应该是MySQL系统管理员常采用的视角。在此场景里，数据库的地位是显而易见的，因为它是你关注的重点。例如，你可以编写几个Web 页面，让你能够看到数据库里的所有表，并且能够检查每个表的结构或内容。

利用MySQL增强Web服务器的功能。从这个角度来看，你的主要兴趣是Web 站点，并且想要把MySQL用作一种工具，以便让站点向访问者提供更有价值的内容。这应该是Web 开发人员常采用的视角。例如，如果网站上有一个留言板或讨论列表，那么你可以用一个数据库来跟踪这些消息。此时，MySQL 便是一个“配角”，网站用户甚至根本不会从网站提供的服务里察觉到数据库的存在。

这两个观点并不矛盾。例如，在“美史联盟”示例里，我们把Web当作是这样一种方式：把所有成员名录都发布到网上，让成员能够方便地访问到成员名录的内容。这就是利用Web来提供对数据库的访问功能。与此同时，把名录内容发布到联盟的网站上，也会提升网站对成员的价值。而这就是使用数据库来增强网站的服务水平。

无论如何看待MySQL与Web的结合，具体的实现都是相似的。你要以Web服务器为桥梁，把前端的Web站点与后端的MySQL连接在一起。Web服务器负责从客户端用户那里收集信息，并把它以查询形式发送到 MySQL 服务器，然后再把检索出来的结果返回给客户端的浏览器，以供用户查看。

当然，你完全可以不把数据发布到网上。但是，与通过标准 MySQL 客户端程序来访问数据的方式相比，把数据发布到网上通常会有以下几点好处。

那些通过 Web 访问数据的人们，可以在他们喜欢的任何一种平台上，使用他们喜欢的任何一种浏览器来访问数据。他们不必局限于运行标准MySQL客

户端程序的系统。无论MySQL客户端的传播范围有多大，始终无法与Web浏览器的传播范围相提并论。

Web应用程序的操作界面，可以比单独的命令行MySQL客户端程序的更简单。

Web 应用程序的操作界面，可以根据具体的应用要求进行定制。MySQL客户端程序都是一些通用的工具，操作界面比较固定。

动态Web页面扩展了MySQL的能力，让我们可以完成许多只使用标准MySQL客户端程序难以完成或不可能完成的任务。例如，想要这些客户端程序集成为一个具备购物车功能的应用程序——这一点真的无法做到。

任何一种编程语言都可用于编写基于Web的应用程序，但是相比之下，有些语言则更为适合。在6.3节会讨论到这个问题。

6.2 MySQL提供的API

为支持应用程序的开发，MySQL提供了一个用C语言编写的客户端开发库。通过它，我们可以在任何的C语言程序里访问MySQL数据库。这个客户端开发库实现了一个API，其中包含一组数据结构和函数。

其他编程语言的MySQL接口，可以把这个C语言客户端开发库链接到语言处理器里。因此，这个客户端开发库提供了一种MySQL绑定方式，让其他编程语言可以构建在这个C语言API之上。Perl、PHP、Python、Ruby、C++、Tcl以及其他语言，都有这种类型的接口。

每一种编程语言绑定都定义了其自己的接口，用于指定访问MySQL的规则。MySQL提供了多种API。我们这里主要介绍3种最为流行的API。

C语言客户端开发库API。这是最基础的MySQL编程接口。在MySQL发行版里，许多标准客户端程序都是用它来实现的，如mysql、mysqladmin和mysqldump。

Perl语言DBI API。数据库接口（Database Interface，DBI）是一个Perl模块，用于在数据库驱动程序（Database Driver，DBD）层与其他模块（其中的每一个都能提供对特定数据服务器的访问）进行交互。这里所用的DBD模块支持MySQL。我们将使用MySQL和DBI，创建多个可以从命令行直接调用的独立脚本，以及多个可以从Web服务器（它提供了通过Web访问MySQL的功能）进行调用的脚本。

PHP语言API。PHP是一种脚本语言，它提供了一种简便的把编程结构嵌入Web页面的方法。在把这种页面发送给客户端之前，PHP 会在服务器主机上对它进行处理。如此一来，便可以让脚本生成动态的内容，如把MySQL的查询结果包含在页面里。与DBI相类似，PHP还支持访问除MySQL以外的其他几种数据库服务器。它既有服务器特定的接口，也有许多独立于服务器的接口。本书将使用后者中的一种，即众所周知的PHP数据对象（PHP Data Object，PDO）。

本章后面对这3种API做了概要性的对比：首先，描述了它们的一般特点；然后，想让你对“在某些特定的应用程序里，为何要选这个，而不选另一个”这一做法的原因有所了解。接下来的三章会详细讨论它们。

当然，把自己锁定在某个API里是毫无理由的。对多个API有所了解，并用这些知识把你自己武装起来，可以让你在它们之间做出明智的选择。对于包含多个组成部分的大项目，可以选用多种API，并根据工作的具体要求选择最合适的语言，编写各个部分。

如果你还缺少使用某个API所必需的软件，则可以参考附录A。

6.2.1 C语言API

C语言API主要用在编译过的C语言程序的上下文里。它是一个客户端开发库，提供了与MySQL服务器进行对话的接口，让你能够与服务器建立连接，并进行通信。

在MySQL发行版里提供了多个C语言客户端程序，它们都是基于这个API开发的。C语言客户端开发库也是其他编程语言实现大部分 MySQL绑定的基础。例如，Perl的DBI模块里的那个MySQL驱动程序，便是通过链接适用于MySQL的C语言客户端开发库的代码，才实现了对MySQL的访问。

6.2.2 Perl语言DBI API

在Perl脚本语言里，DBI API用于编写数据库应用程序。这套API一边要尽可能多地处理数据库服务器，一边又要把服务器有关的细节隐藏起来，不让脚本的编写者看到。为实现这一目标，DBI使用了多个Perl模块。这些模块分属在两层架构里，协同工作（见图6-1）。

数据库接口（DBI）层。它负责为客户端脚本提供通用接口。这是一个抽象层，与具体的数据库服务器无关。

数据库驱动程序（DBD）层。它以驱动程序（特定于服务器）的形式为各种数据库服务器提供支持。对MySQL来说，DBD层的模块名字为DBD:mysql。

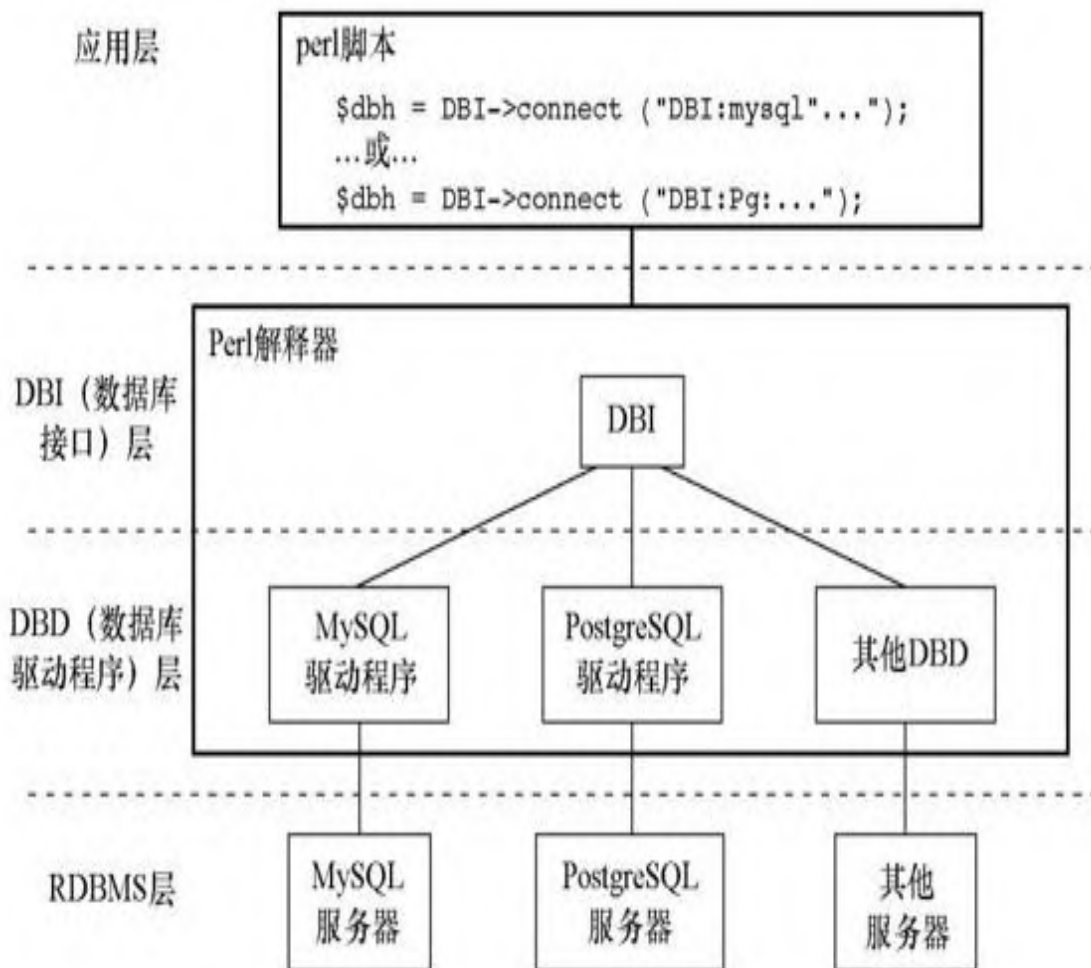


图6-1 DBI架构

DBI架构可以让你以相对通用的方式编写应用程序。在编写DBI脚本时，可以使用一套标准的数据库访问调用。DBI层会在DBD层调用正确的驱动程序来处理你的请求，并且驱动程序会处理，与你所用的特定数据库服务器进行通信时所产生的问题。DBD层会把服务器返回的数据传递回DBI层，而DBI层会把数据呈现给应用程序。于是，不管数据来自哪一个数据库系统，数据的形式都是统一的。

从应用程序开发人员的角度来看，DBI 接口不仅掩盖了数据库服务器之间的差异，而且还能处理很多种不同的服务器（与用于它们的驱动程序的种类相当）。DBI提供的统一客户端接口改善了程序的可移植性，因为我们可以用一种统一的方式去访问各个数据库服务器。

在脚本编写方面，只有一种情况与特定服务器密切相关，即当你连接某个数据库服务器的时候，因为，这时你必须指明选用哪种驱动程序来建立连接。例如，想要使用MySQL数据库，可以这样连接：

```
$dbh = DBI->connect ("DBI:mysql:...");
```

如果要使用PostgreSQL或Oracle，则可以这样连接：

```
$dbh = DBI->connect ("DBI:Pg:...");
```

```
$dbh = DBI->connect ("DBI:Oracle:...");
```

在连接建立之后，就不用再引用任何特定的驱动程序了。DBI和驱动程序自己会把与特定数据库有关的全部细节处理好。不管怎样，理论上应该如此。不过，在DBI脚本的可移植性方面，有两个因素需要特别注意。

如果两个数据库系统具有不同的SQL 实现，那么完全有可能出现这样的情况：为一个数据库服务器编写的SQL语句，另一个服务器理解不了。如果所用的SQL具有较强的通用性，那么用它们编写的脚本在数据库服务器之间也会有更好的可移植性。但是，如果所用的SQL 依赖于服务器，那么用它编写的脚本也会依赖于这种服务器。例如，如果某个脚本使用了MySQL特有的SHOW VARIABLES语句，那么此脚本在其他数据库服务器上便无法正常工作。

DBD 模块经常会提供一些与数据库服务器特定相关的信息，以便让脚本开发人员使用给定数据库系统特有的功能。例如，针对MySQL的DBD模块，便提供了这样的支持：允许在查询结果里访问数据列的属性（如所有值的最大长度、是否为数字类型列等）。其他数据库服务器未必会提供这种类型的信息。这种 DBD特有的功能会对可移植性产生影响。如果使用了这种功能，那么当想要把为MySQL编写的脚本，移植到其他的数据库系统上使用时，整个工作会变得更加困难。

尽管上述两个因素会潜在影响到DBI脚本的可移植性，但是以抽象方式提供数据库访问的DBI 机制，仍然是提高可移植性的有效途径。现在轮到你自己来决定要使用多少个不可移植的功能。在第 8章你会发现，我基本没怎么避免使用这些 MySQL DBD提供的、专属于 MySQL的功能。这是因为你应该知道这些功能是什么，只有这样，你才能决定是否要使用它们。更多相关信息请参考附录H。

6.2.3 PHP语言API

与Perl语言一样，PHP也是一种脚本语言。与Perl语言不一样的是，PHP的设计目标不是要成为一种通用语言，而是要成为一种专门编写Web应用程序的语言。PHP的API主要用于把可执行脚本嵌入Web页面里。这样，Web开发人员便可以很容易地编写出带有动态生成内容的页面。当某个客户浏览器向Web服务器发送一个PHP页面请求时，PHP将执行在该页面里找到的所有脚本，并把它们替换为该脚本的输出内容。最终的结果被发送到浏览器。如此一来，

实际上显示在浏览器里的页面，会根据页面请求所发生的具体环境产生变化。例如，当把下面这段简短的PHP脚本嵌入某个Web页面时，它便会显示请求该页面的那台客户端主机的IP地址：

```
<?php echo $_SERVER["REMOTE_ADDR"]; ?>
```

举一个不太重要但比较让人感兴趣的例子，你可以用一个脚本，把数据库内容的最新信息提供给访问者。下面这个示例显示了一个简单的脚本，可以用在“美史联盟”的Web站点上。这个脚本调用一个查询，用以确定当前的联盟成员人数，并且会把它报告给网站的访问者。

```
<html>

<head>

<title>U.S.Historical League</title>

</head>

<body bgcolor="white">

<h2>U.S.Historical League</h2>

<p>Welcome to the U.S.Historical League Web Site.</p>

<?php

# USHL 主页

try

{

    $dbh = new PDO("mysql:host=localhost;dbname=sampdb", "sampadm",
"secret");

    $dbh->setAttribute(PDO::ATTR_ERRMODE,
PDO::ERRMODE_EXCEPTION);

    $sth = $dbh->query ("SELECT COUNT(*) FROM member");
```

```

$count = $sth->fetchColumn (0);

print ("<p>The League currently has $count members.</p>");

$dbh = NULL; # 关闭连接

}

catch (PDOException $e) { } # 清空处理器（捕捉并忽略错误）

?>

</body>

</html>

```

PHP脚本一般看上去很像HTML页面，只是在标签<?php和?>之间嵌入了一些可执行代码。一个页面可以包含任意多个代码段。这提供了一种极其灵活的脚本开发方法。例如，可以像编写普通HTML页面那样，先编写PHP脚本，搭建起页面的基本框架，然后再添加代码，生成页面的动态部分。

PHP实际上有多种类型的数据库接口。这些接口中包含的是一组底层的函数库，每一组函数库都对应一种数据库服务器——这与DBI接口的做法有所不同，它并没以费尽心思去统一针对不同服务器的接口。实际上，各个服务器的接口，看起来更像是为该服务器实现了底层 API 的那个C语言库所对应的接口。例如，在PHP脚本里，用来访问MySQL数据库的那个底层函数的名字，与MySQL中C语言客户端开发库里实现该功能的那个函数的名字非常相似。

在PHP里，比较像DBI的方法是由PHP数据对象（PHP Data Object, PDO）扩展模块提供的。这个扩展模块利用与DBI相类似的两层架构，提供了更加抽象的数据库系统接口。第9章的PHP脚本将使用PDO扩展模块来访问数据库。

6.3 选择API

本节提供的通用指南，可以帮助你为各种类型的应用选择合适的API。其中，将对C、DBI和PHP中的API功能进行对比，以便让你对它们的优点和缺点都有所了解，并且还会指明何时选择何种更合适。

虽然我有自己偏好的语言，但我不会偏重于这三种语言中的任何一种语言。你可能会像本书的技术评审那样，也拥有自己的偏好。其中一位评审觉得，我应该着重强调使用C语言编写MySQL程序的重要性，而另一位却认为，

我应该不鼓励使用这种语言。请自己权衡本节所讨论的各种因素，得出自己的结论。

在评估哪种API最适合某项特定任务时，值得考虑的因素有很多。

期望的执行环境。你想让应用程序用在什么样的上下文里。

性能。当用某种API语言编写应用程序时，其执行效率如何。

开发的难易程度。如果使用某种API及其相关的语言，编写应用程序，其方便性如何。

可移植性。这个应用程序会不会用于除MySQL以外的其他数据库系统。

请注意，这其中的一些因素是相互影响的。例如，你很想让编写的应用程序具有很高的性能，但是，选用一种能让你快速开发应用程序的编程语言，可能也同样重要，哪怕应用程序的性能没那么高。

6.3.1 执行环境

在编写应用程序时，都要考虑其使用环境。例如，它可能是一个可以从shell启动的报表生成程序，也可能是一个在每个月底以 cron 作业方式运行的应付账款汇总程序。不管是从shell启动的命令，还是从 cron 作业运行的命令，它们通常都比较独立，对执行环境并无过多的要求。另一种情况是，你可能正在编写一个供Web服务器调用的应用程序。这类程序往往需要从其执行环境提取很多非常特定的信息，如客户端使用的是什么浏览器？输入到邮件列表订阅申请表里的参数是什么？客户端是否提供了访问个人信息所需要的正确密码？

每种API语言所适合编写的应用程序，随环境的不同而不同。

C 语言是一种通用语言，因此在理论上，可以用它来编写任何应用程序。但实际上，C语言更适合于编写独立程序，而不太适合于 Web编程。其中一个原因是，在C语言里进行文本处理或内存管理，并不像在Perl语言或 PHP语言里那样容易，而这些功能恰恰是Web应用程序频繁使用到的。

与C语言类似，Perl语言也适合于编写独立程序。不过碰巧的是，Perl语言在Web站点开发方面也非常有用，例如，借助 CGI.pm 模块。这让 Perl 语言很适合于编写需要把MySQL和Web结合在一起的应用程序。这种程序可以通过CGI.pm模块与Web进行交互，通过DBI与MySQL进行交互。

PHP语言是专门为编写Web应用程序而设计的，因此Web环境显然是最适合它的。不仅如此，数据库访问也是PHP的强项之一，因此，对于要完成MySQL

相关任务的Web应用程序，PHP 语言是个不错的选择。你也可以用PHP 语言来编写命令行脚本，但这种用法比较少见。

综上所述，C语言和Perl语言都是编写独立应用程序的最佳候选语言。Perl语言或PHP语言则最适合于编写Web应用程序。如果你需要同时编写这两种类型的应用程序，而你对这些语言又都不太熟悉，想稍加学习就能编程，那么Perl语言将是最佳选择。

6.3.2 性能

在同等条件下，我们当然希望应用程序能够以最快的速度运行。但是，性能的实际重要性往往与程序的使用频率有关。对于每月只会在午夜运行一次的cron 作业，其性能问题可能无关紧要。但是，如果某个程序在一个访问量极大的Web站点上，每秒运行多次，那么其效率的微小变化即可造成巨大的差异。在后一种情况里，性能对提高网站的可用性和吸引力有着显著影响。不管网站提供的内容是什么，一个反应迟钝的网站肯定会让访问者感到厌烦。如果你的收入来源依赖这个网站，那么性能的低劣将直接导致利润的减少。如果你的网站不能同时支持许多连接，那么厌烦等待的访问者便会放弃访问，再去寻找其他网站。

性能评估是一个复杂的问题。如果想要知道“某个具体API编写出来的应用程序的性能如何”，那么最好是用这个API实际编写一个程序，然后测试一下。最好的对比测试是，用不同的API实现出多个不同的版本，再对它们一一比较。当然，实际开发通常不会这么做。很多时候，都是先写出程序。在它工作之后，再去考虑优化它，看看它是否还可以运行得更快、占用的内存更少，或者是否还有其他可以改进的地方。不过，在以相对一致的方式对性能产生影响方面，至少有两个基本因素值得考虑。

编译型程序比解释型脚本的执行速度更快。在实现同样功能时，编译型程序比使用脚本语言编写的程序更有效率，所用的内存更少，执行得更快。这是因为它们是直接执行的，没有用于执行脚本的中间语言解释器所导致的开销。C 语言是编译型语言，Perl语言和PHP语言都是解释型语言，因此C语言程序的运行速度一般比Perl或PHP脚本更快。由此，对于使用频率特别高的程序，C语言应该是最佳选择。不过，仍然有必要好好考虑一下程序具体所做的事情。如果脚本型应用程序在大部分时间里，都是在执行那些经过编译的MySQL客户端开发库里的例程（它们被链接在解释器引擎里），那么编译型程序和解释型程序之间的差距会大大缩小。

对于在Web上下文里使用的解释型语言，如果解释器是作为 Web服务器的模块来运行的，而不是以单独的进程来运行的，那么性能会好一点。例如，如果使用的是Apache，那么可以安排 Apache以单独进程方式调用脚本解释器。在这种操作模式下，当Apache需要运行Perl或PHP脚本时，它将启动相应

的解释器，并让它去执行那个脚本。此时，Apache是把解释器当作CGI程序来使用的，也就是说，Apache与这些程序之间使用了通用网关接口（Common Gateway Interface, CGI）协议来进行通信。与此相反的是，解释器也可以当作模块来使用，这个模块会被直接链接到 Apache的二进制执行文件里，并作为Apache进程的一部分来运行。按照Apache规定，Perl和PHP解释器采用的模块形式分别是mod_perl和mod_php。

6.3.3 开发时间

刚才讨论的因素会对应用程序的性能产生影响，但是单纯地追求执行效率不应该是唯一的目标。你的时间以及编程工作的难易程度都很重要，因此在为MySQL应用程序选择API时，另一个需要考虑的因素是，需要多长时间能完成开发工作。如果为完成相同的功能，编写Perl或PHP脚本所花费的时间只有开发C语言程序所用时间的一半，那么即使最终的程序并没有运行得那么快，你仍有可能不会选用C语言API。少关注应用程序的执行时间，多关注编写它所花费的时间，这通常是合情合理的。尤其是对于那些执行频率并不高的应用程序，更是如此。你的一个小时比计算机的一个小时更加宝贵！

一般来讲，脚本语言可以让你更快地完成程序的开发，尤其是在进行原型设计的时候。至少有两个因素可用于说明其中的原因。首先，与编译型语言相比，脚本语言提供的语法结构通常更加高级。这使你能够在更高的抽象层上思考问题，从而可以把注意力集中在方向性的“应该做什么”上面，而不是去关注细节性的“如何去做”上面。例如，在处理涉及“键/值”关系（如“学生ID/学生姓名”）的数据时，PHP中的关联数组和Perl中的散列都可以帮助程序员节约很多时间。C语言并没有提供任何类似的功能。如果使用C语言来实现，那么你必须编写代码，用于处理诸如内存管理之类的底层细节，并且还需要对它们进行调试。这些事情都需要花费时间。Perl和PHP的另一个优势是，它们可进行模式匹配和文本处理。在这些方面，C语言的处理都很初级。

其次，与使用编译型语言相比，使用脚本语言的程序开发周期更少。如果使用C语言，那么程序开发周期将是“编辑-编译-测试”。程序每修改一次，都要重新编译一次，然后才能测试。如果使用Perl或PHP，那么程序开发周期将简化成“编辑-测试”，脚本修改后，无需编译即可测试。另一方面，C语言编译器会对编写的程序实施更多的约束，即执行更严格的类型检查。编译器对代码的严格检查，有助于避免那些在“宽松”语言（如Perl和PHP）里难以捕捉到的错误。在C语言里，如果某个变量名拼错了，那么编译器会向你发警告。PHP和Perl则不会这样做，除非你要求它们这么做。当应用程序变得越来越大，并且越来越难以维护时，这种严格的约束将会变得更为重要。

一般情况下，在编译型语言 and 解释型语言之间，需要权衡的是开发时间和性能。你是想使用编译型语言开发程序，获得更快的执行速度，花费更少

的编程时间呢？还是想把程序编写成一个脚本，在最短时间内让它运行起来，提高执行速度呢？

6.3.4 可移植性

可移植性指的是，把某个为了使用MySQL 而编写的程序，修改成可以使用另外一种数据库系统。你可能从未关心过这个问题。不过，除非你能预知未来，否则说出像“我绝不会把这个程序用在除MySQL以外的任何数据库上”这样的话，就有些冒险。假设你刚换了个新工作，并打算继续使用原有的程序，但是你的新老板使用的是另一种数据库系统，这时你要怎么办？或者，还能怎么做呢？如果需要优先考虑可移植性，那么必须弄清这些不同API之间的差异。

C语言API根本不可移植。从本质上讲，它就是专门为MySQL设计的。

Perl语言DBI API具有很好的可移植性，因为与数据库无关便是DBI标明的设计目标。

如果使用前面提到的PDO数据库访问扩展模块，那么PHP脚本的可移植性与DBI很相近。

当你在同一个应用程序里访问多种类型的数据库系统时，以数据库无关的形式实现可移植性尤为重要。例如，需要把数据从一个数据库系统导出，并导入另外一个系统里；或者需要把源自多个数据库系统的信息整合在一起生成一份报告。

第7章

用C语言编写MySQL程序

MySQL提供了一个用C语言写成的客户端开发库，你可以用它来编写访问MySQL数据库的客户端程序。这个库定义了一套应用程序编程接口，其中包括以下功能。

用于打开和关闭服务器会话的连接管理例程。

用于构造SQL语句，并把这些语句发送到服务器，然后还要对结果进行处理的例程。

在API调用失败时，用于确定出错原因的状态检查函数和出错报告函数。

用于处理在选项文件里或在命令行上给出的各种选项的例程。

本章主要展示，如何使用C语言客户端开发库来编写自己的程序，使用的约定与MySQL发行版里客户端程序所使用的基本一致。

本章的第1部分将开发一系列的短程序。这个系列最终会演变成一个简单的程序，它只包含与服务器建立和断开连接的功能，可用作客户端程序的框架。这么做的原因在于，虽然各个MySQL客户端程序的用途各异，但有一点是共通的，即都必须连接到服务器。

虽然最终的骨架程序是通用的，但是它也包含了用于处理选项和错误的代码。因此，它可用作许多其他客户端程序的基础。在把它开发完成之后，我们将暂停片刻，想想如何执行各种类型的SQL语句。首先，我们将探讨如何处理特定的硬编码语句，然后再动手开发可用于处理任意语句的代码。之后，往这个骨架程序里增加一些语句处理代码，进而开发出一个与mysql客户端程序非常相似的程序，可用于交互执行语句。

接着，本章将演示另外几种客户端开发库所支持的功能。

使用安全套接字层（Secure Socket Layer，SSL）协议，编写出可以通过安全连接与服务器进行通信的客户端程序。

一次性向服务器发送多条语句，并对返回的结果集进行处理。

使用服务器端预处理语句。

对于客户端开发库的各个函数和数据类型，本章只会讨论那些示例程序中会用到的。想要了解完整的函数与类型列表，请参考附录 G。该附录可用作参考手册，在需要用到客户端开发库里的某个部分时，可从中了解更多信息。

这里的示例程序都有现成的代码，不用你自己录入即可直接使用。更多关于如何获得包含这些程序的sampdb发行版的说明请参考附录A。所有程序都放置在该发行版的capi目录下。

示例程序的存放位置

在MySQL邮件列表上，经常会看到这样的问题：“在什么地方可以找到用C语言编写的客户端示例程序？”当然，我的答案是：“就在本书里！”此外，在MySQL源代码发行版里包含的几个客户端程序，碰巧也是用C语言编写的（如mysql、mysqladmin和mysqldump）。因为该发行版是现成的，所以它也提供了相当多的客户端示例代码。如果你还没用过，那么什么时候下载一个源代码发行版，看看client目录和tests目录里的那些程序。

7.1 编译和链接客户端程序

本节将对“如何编译和链接使用MySQL 客户端开发库的程序”进行介绍。对于不同的系统，用于构建客户端程序的命令也往往存在差异，你甚至可能还需要对本节所展示的命令进行适当的调整。不过，讲述的内容都是通用的，你完全可以将其应用到所编写的大部分客户端程序里。

当使用C语言编写MySQL客户端程序时，显然需要一个C语言编译器。本节示例使用的是gcc，它是Unix系统上最常见的编译器。此外，你还需要MySQL头文件和客户端开发库。

头文件和客户端开发库是开发MySQL客户端程序的基础。如果你系统上的MySQL是通过源代码或二进制发行版安装的，那么支持客户端程序开发的所有内容，应该已作为该过程的一部分安装完成。如果安装使用的是RPM包，那么

只有在安装了开发者RPM之后，才会安装所有支持内容。想要获得MySQL的头文件和开发库请参考附录A。

想要编译和链接客户端程序，就必须指定 MySQL 头文件和客户端开发库的存放位置，因为它们并没有安装在编译器和链接器的默认搜索路径上。在下面的示例里，假设头文件和客户端开发库的存储位置分别为`/usr/local/include/mysql`和`/usr/local/lib/mysql`。请将这两个路径名修改成你自己系统使用的路径名。

当把源文件编译成目标文件时，如果想要告知编译器如何找到 MySQL 头文件，那么可以向它传递一个可以指定正确目录的`-I`选项。例如，要将`myclient.c`编译成`myclient.o`，则可以使用如下命令：

```
% gcc -c -I/usr/local/include/mysql myclient.c
```

在把目标文件链接成可执行的二进制文件时，如果想要把开发库的存放位置和名称告知链接器，那么可以向它传递参数`-L/usr/local/lib/mysql`和`-lmysqlclient`，如下所示：

```
% gcc -o myclient myclient.o -L/usr/local/lib/mysql -lmysqlclient
```

如果你的客户端程序由多个文件构成，则需要在链接命令行列出所有的目标文件。

链接步骤可能会因“函数未找到”之类的错误而失败。此时，需要提供附加选项`-l`，用于指明包含这些缺失函数的库。如果出错消息与`compress()`或`uncompress()`有关，则可以尝试添加`-lz`或`-lgz`来告知链接器去搜索`zlib`压缩库：

```
% gcc -o myclient myclient.o -L/usr/local/lib/mysql -lmysqlclient -lz
```

如果出错消息涉及`floor()`函数，则需要添加`-lm`，以便链接数学函数库。你可能还需要添加其他的函数库。

另一种自己确定编译和链接 MySQL 程序所需正确标志的方法是，使用`mysql_config` 工具。例如，该工具可能会表明需要如下选项：

```
% mysql_config --include  
  
-I'/usr/local/mysql/include'  
  
% mysql_config --libs
```

```
-L'/usr/local/mysql/lib' -lmysqlclient -lpthread -lz -lm -lrt -ldl
```

如果想在编译或链接命令里直接使用`mysql_config`，则需要把它放到一对反引号里：

```
% gcc -c `mysql_config --include` myclient.c
```

```
% gcc -o myclient myclient.o `mysql_config --libs`
```

系统的shell程序会执行`mysql_config`，并用其输出替换反引号里的命令。这样，便能自动为gcc提供恰当的编译和链接标记。

如果你还未用make 来建立程序，那么建议你先学习一下如何使用它，这样你就不用手动输入一大堆的程序建立命令。假设，你要开发一个名为myclient的客户端程序，它由2个源文件（main.c和lib.c）和1个头文件（myclient.h）构成。下面这个简单的Makefile文件可用于建立这个程序。请注意，该文件里的行缩进使用的是制表符，而没有使用空格。

```
CC = gcc
```

```
INCLUDES = -I/usr/local/include/mysql
```

```
LIBS = -L/usr/local/lib/mysql -lmysqlclient
```

```
all: myclient
```

```
main.o: main.c myclient.h
```

```
$(CC) -c $(INCLUDES) main.c
```

```
lib.o: lib.c myclient.h
```

```
$(CC) -c $(INCLUDES) lib.c
```

```
myclient: main.o lib.o
```

```
$(CC) -o myclient main.o lib.o $(LIBS)
```

```
clean:
```

```
rm -f myclient main.o lib.o
```

在使用Makefile文件之后，每次修改源文件，都可以通过运行make简单地重建这个程序。make将显示并执行必要的命令：

```
% make
```

```
gcc -c -I/usr/local/mysql/include/mysql myclient.c
```

```
gcc -o myclient myclient.o -L/usr/local/mysql/lib/mysql -lmysqlclient
```

与需要键入长长的gcc 命令相比，这种办法既简单又不易出错。Makefile 文件还可以让建立过程变得更易于修改。例如，你的系统还需要链接其他库，如数学库和压缩库，那么可以在Makefile文件的LIBS行添加-lm和-lz：

```
LIBS = -L/usr/local/lib/mysql -lmysqlclient -lm -lz
```

如果还需要其他库，也可以把它们添加到LIBS行。此后，当运行make时，它将自动使用更新后的LIBS值。

除直接编辑Makefile文件外，另一种更改make变量的办法是，在命令行上指定它们。例如，C语言编译器的名字现在不是gcc，而是cc，那么可以这样做：

```
% make CC=cc
```

如果mysql_config可用，那么可以用它来避免在Makefile文件里明确地写出包含文件和库的路径名称。可以像下面这样使用INCLUDES和LIBS行：

```
INCLUDES = ${shell mysql_config --include}
```

```
LIBS = ${shell mysql_config --libs}
```

当 make 运行时，它会执行每一个mysql_config 命令，并用其输出来设置相应的变量值。其中的\${shell}结构是受GNU make支持的，如果你的make版本不是基于GNU make构建的，那么可能需要使用一种稍有差异的语法。

如果你使用的是集成开发环境（Integrated Development Environment，IDE），那么你可能根本不会用到Makefile文件。具体情况则取决于你所用的IDE版本。

7.2 连接服务器

我们的第一个MySQL客户端程序非常简单：它可以连接MySQL服务器、断开连接和退出。这些操作本身很普通，但是你必须要知道如何做，因为在操作 MySQL 数据库之前，你必须先连接服务器。连接服务器是一个很常见的操作，你所开发的用于建立连接的代码也可用于你所编写的所有客户端程序。此外，这项任务也可以把出发点降低。这段代码在后面会被不断改进，以便完成一些更有用的事情。

我们的第一个客户端程序connect1，只包含一个源文件connect1.c。

```
/*

* connect1.c - 建立和断开与MySQL 服务器的连接

*/

#include <my_global.h>

#include <my_sys.h>

#include <mysql.h>

static char *opt_host_name = NULL;      /* 服务器主机（默认为localhost）
*/

static char *opt_user_name = NULL;      /* 用户名（默认为login name） */

static char *opt_password = NULL;      /* 密码（默认为空） */

static unsigned int opt_port_num = 0; /* 端口号（使用内建值） */

static char *opt_socket_name = NULL; /* 套接字名（使用内建值） */

static char *opt_db_name = NULL;      /* 数据库名（默认为空） */

static unsigned int opt_flags = 0;     /* 连接标志（为空） */

static MYSQL *conn;                   /* 连接处理器指针 */

int

main (int argc, char *argv[])
```

```

{
MY_INIT (argv[0]);

/* 初始化客户端开发库 */

if (mysql_library_init (0, NULL, NULL))

{
fprintf (stderr, "mysql_library_init() failed\n");
exit (1);
}

/* 初始化连接处理器 */

conn = mysql_init (NULL);

if (conn == NULL)

{
fprintf (stderr, "mysql_init() failed (probably out of
memory)\n");
exit (1);
}

/* 连接服务器 */

if (mysql_real_connect (conn, opt_host_name, opt_user_name, opt_password,
opt_db_name, opt_port_num, opt_socket_name, opt_flags) == NULL)

{
fprintf (stderr, "mysql_real_connect() failed\n");
mysql_close (conn);
}

```



```

    exit (1);

}

/* 断开服务器连接，终止客户端开发库 */

mysql_close (conn);

mysql_library_end ();

exit (0);

}

```

这个源文件的开头包含了头文件`my_global.h`、`my_sys.h`和`mysql.h`。根据MySQL客户端程序的具体用途，它可能还需要包含其他头文件，但是这3个是最基本的。

`my_global.h` 文件中包含其他几个常用的头文件，如 `stdio.h`。如果你是在Windows系统上编译这个程序，那么它还会包含与Windows相兼容的信息。（你本人可能不想使用Windows，但是如果你打算对外发布代码，那么使用`my_global.h` 文件能够为其他想要在Windows下编译代码的人们带来帮助。）

`my_sys.h`中包含可移植性的宏和定义，主要用于客户端开发库所使用的结构和函数。

`mysql.h`定义了许多基本的、MySQL相关的常量和数据结构。

这些文件的包含顺序非常重要：应该把 `my_global.h` 放在所有其他与MySQL有关的头文件之前。

接着，这个程序声明了一组变量，依次与用于连接 MySQL 服务器的各种参数相对应。在这个客户端程序里，这些参数都硬编码在代码里，且带有默认值。随后，我们将开发一种更灵活的方法，通过在选项文件或命令行里指定值，来改写这些参数的默认值。（这也正是为何变量名全都以 `opt_` 作为开头的原因，其含义是，那些变量最终可以通过命令选项来设定）。此程序还声明了一个指向MySQL结构的指针，用作连接处理器。

此程序的`main()` 函数负责建立和断开与服务器的连接。建立连接需要执行以下两个步骤。

(1) 调用mysql_init()函数，获得一个连接处理器。

(2) 调用mysql_real_connect()函数，建立与服务器的连接。

当把NULL传递给mysql_init()时，该函数将自动分配一个MYSQL结构，并初始化它，然后返回一个指向它的指针。数据类型MYSQL是一个包含连接信息的结构。此类型的变量被称为“连接处理器”。

另一种方法是传递一个指向某个已有 MYSQL 结构的指针。此时，mysql_init()自己不分配这种结构，它会初始化该结构，并返回一个指向它的指针。

函数mysql_real_connect()可以接受很多形式的参数。

指向连接处理器的指针。它应该是mysql_init()的返回值。

服务器主机。对这个值的解释与具体的操作平台有关。在Unix 系统里，如果你指定的字符串里包含主机名或者IP地址，那么客户端程序将使用TCP/IP连接协议来连接该主机。如果你指定的是NULL或主机"localhost"，那么客户端程序将通过Unix套接字文件来连接运行于本地主机的服务器。

在Windows系统里，操作也相似，只是对 "localhost"的处理不太一样，它使用的连接方式是共享内存或TCP/IP连接，而非Unix套接字文件连接。在Windows系统里，如果指定的主机名为"."或NULL，并且服务器支持命名管道连接，那么客户端程序将会尝试使用命名管道来连接本地服务器。

使用MySQL账户的用户名和密码。如果用户名为NULL，那么客户端开发库会将你的登录名发送给服务器（对于Windows系统，是发送给ODBC）。如果密码为NULL，则不发送密码。

数据库名称指定的数据库将是连接建立之后的默认数据库。如果这个值为 NULL，则不选择数据库。

端口号。它用于TCP/IP连接。如果值为0，则表示让客户端开发库使用默认端口号。

套接字文件名。在Unix系统里，它被用于Unix套接字文件连接。在Windows系统里，它会被解释为管道连接所用的名字。如果其值为 NULL，则客户端开发库将使用默认套接字（或管道）名。

标志值。程序connect1传递的值为0，因为它没有用到任何特殊的连接选项。

更多关于mysql_real_connect()的信息请参考附录G。其中，对“主机名参数对端口号和套接字文件名参数的影响”进行了详细的描述，并且列出了标志参数里指定的各种选项。该附录还对mysql_options()函数进行了介绍。在调用mysql_real_connect()之前，可以用它来指定其他与连接相关的选项。

如果要终止连接，可以调用mysql_close()，并把指向连接处理器的指针传递给它。如果连接处理器是采用向mysql_init()传递NULL的方式自动分配的，那么当连接终止时，mysql_close()会自动把这个处理器释放掉。在调用mysql_close()之后，该处理器不能再用于后续的服务器通信。

除包含用于建立连接的代码以外，connect1.c还用到了其他3个调用。

MY_INIT()是一个初始化宏。它会把一个全局变量设置成程序名（以参数形式传递的），以方便MySQL库在出错消息里使用。它还将调用my_init()来完成某些设置操作。

mysql_library_init()会初始化客户端开发库。请在调用其他mysql_xxx()函数之前，调用它。

mysql_library_end()会终止使用客户端开发库，并执行一些必要的清理工作。当使用完客户端开发库之后，需要调用它。

如果想要试试connect1，可以按照本章前面介绍过的用于建立客户端程序的步骤，先编译和链接它，然后运行它。在Unix系统里，可以使用下面的命令来运行程序：

```
% ./connect1
```

在Unix系统里，如果你的shell程序不会搜索当前目录（“.”），那么就必须在命令的开头使用“./”。如果当前目录包含在搜索路径里，或者使用的是Windows系统，那么可以省略命令名里的“./”：

```
% connect1
```

如果 connect1 没有产生任何输出，则表明连接成功。如果有信息输出，那么你看到的可能是这样的：

```
% ./connect1
```

```
mysql_real_connect() failed
```

这种输出表明连接没有建立上，但并没有告诉你具体原因。导致连接失败的最常见原因是，默认的连接参数（主机名、用户名等）不正确。如果真是如此，则一种解决办法是：在编辑这些参数变量的初始值，并把它们更改为可以访问服务器的值之后，重编译这个程序。这种做法的好处是，至少你可以建立连接。但是，这个程序仍然含有硬编码值，如果有其他人要使用你的程序，会很不方便。这种做法也不够安全，因为它会暴露你的密码。你可能会认为，在程序编译成二进制可执行文件之后，这些密码就会隐藏起来，但实际上根本就不是这么回事，只要有人运行strings工具来查看这个二进制文件，密码就会显露出来。此外，所有对此文件拥有读取权限的人，都可以轻而易举获得这个密码。

上面这段揭示了connect1程序的两个明显不足之处。

出错信息没多大用，不能让人了解导致问题的具体原因。

对于用户来讲，没有通过指定连接参数来灵活运行程序的方式。这些参数都硬编码在源代码里。如果能让用户通过命令行或者选项文件来改写这些参数，会更好。

下一节就来解决这些问题。

7.3 出错处理和命令选项处理

我们的下一个客户端程序为connect2，它在表面上与connect1很相似：它也可以连接服务器、断开连接和退出。但是，connect2有两项重要的改进。

它可以提供更好的出错报告。它使用MySQL客户端开发库函数，返回了更为具体的关于出错原因的信息。

它提供了默认连接参数，但可以让用户通过命令行或选项文件来改写它们。

7.3.1 出错检查

我们先来看看与出错处理有关的论题。首先要强调一点：只要你调用的MySQL函数会失败，对它进行错误检查就是重要的。许多编程书中常这样讲：“出错检查留给读者作为练习。”个人认为他们之所以这么做，是因为出错检查——真的——很乏味。不管怎样，让MySQL客户端程序测试出错条件，并做出适当的处理很有必要。那些返回各种状态值的客户端开发库函数，之所以那么做，是有道理的，忽略它们会很危险。例如，有一个函数在发生错误时，会返回一个指向某个数据结构或者NULL的指针，那么你最好能

检查一下这个返回值。当程序后面的代码，需要一个指向某种有效数据结构的指针时，如果实际使用的是NULL，则会导致奇怪的结果，甚至导致整个程序崩溃。

不对返回值进行检查，往往会增加编程难度。MySQL邮件列表上有很多求助帖子都与此有关。典型的问题是，“为什么我的程序会在执行这条语句时崩溃？”或者“为什么我的查询命令没返回任何内容？”在许多时候，程序之所以会出现这样的问题，可能有两种原因：一是，在执行语句前，忘了检查连接是否已成功建立；二是，在检索查询结果前，忘记了检查服务器是否已成功执行语句。

千万不要想当然地认为，客户端开发库里的函数调用每次都会成功。如果不检查返回值，那么结果可能是：你最终还是要设法找出程序里出现的问题；或者程序的用户会对程序的异常表现产生疑惑；或者两种情况兼有。

MySQL客户端开发库里那些带有返回值的例程，通常会根据返回值是指针还是整数，来表明执行是否成功。

对于返回指针值的函数，若返回非NULL指针，则表示成功；若返回NULL，则表示失败。（这里的NULL指的是“C语言里的NULL指针”，而非“MySQL的NULL列值”。）

以我们使用过的客户端开发库例程 `mysql_init()` 和 `mysql_real_connect()` 为例，若执行成功，则都会返回一个指向连接处理器的指针；若执行失败，则都会返回NULL。

对于返回整数值函数，通常是用返回 0 来表示成功，用返回非零值来表示失败。对于某些特定的非零值，如-1，不进行测试也会造成严重影响。客户端开发库里的函数在调用失败时是否会返回某个特定值，是没有保证的。有时，你可能会看到下面这样的代码，它根本不能对C语言的API函数 `mysql_XXX()` 所返回的值进行正确的测试：

```
if (mysql_XXX () == -1)           /* 此测试不正确 */  
  
    fprintf (stderr, "something bad happened\n");
```

这个测试不一定有效。MySQL的API并没有说返回的非零错误值一定是某个特定值，它只是说那个返回值（明显）不为零。因此，正确的测试代码可以这样写：

```
if (mysql_XXX () != 0)           /* 此测试正确 */
```

```
fprintf(stderr, "something bad happened\n");
```

另外，也可以像下面这样写，效果也是一样的，而且写起来更简单些：

```
if (mysql_XXX ())                /*此测试正确 */
```

```
fprintf(stderr, "something bad happened\n");
```

如果你浏览过MySQL本身的源代码，你会发现其中普遍使用的是第二种测试形式。

并非所有API调用都有返回值。前面使用过的客户端开发库例程 `mysql_close()` 就没有返回值。（如果它失败该怎么办呢？如果它真失败了，那又会怎样？你已不再需要这个连接，就别管它了。）

如果客户端开发库的函数调用真的失败了，那么可以用API里的3个函数来查明原因。

`mysql_error()` 会返回一个包含出错消息的字符串。

`mysql_errno()` 会返回一个MySQL特有的数字出错代码。

`mysql_sqlstate()` 会返回一个SQLSTATE代码。SQLSTATE值与数据库软件的提供商无关，因为它依据的是ANSI SQL和ODBC标准。

这几个函数的参数都是指向连接处理器的指针。在错误发生后，应立即调用它们。如果你执行了另一个会返回状态信息的API调用，那么从 `mysql_error()`、`mysql_errno()` 或 `mysql_sqlstate()` 获取的出错信息将对应于后面那个API调用。

一般情况下，程序的用户都认为，出错消息比出错代码更易于理解，因此，如果你只返回某个值来报告错误，那么个人建议你换成返回出错消息。为完整起见，本章的示例将报告所有的三个值。不过，在每个可能发生错误的地方都写出三个函数调用会很繁琐。实际上，我们可以编写一个名为 `print_error()` 的辅助函数，用于输出：我们所提供的出错消息，以及MySQL客户端开发库的例程所提供的出错值。换句话说，在每次测试错误时，不用再像下面这样调用 `mysql_errno()`、`mysql_error()` 和 `mysql_sqlstate()` 了：

```
if (...某个MySQL 函数调用失败...)
```

```
{
```

```
fprintf(stderr, "...some error message...:\nError %u (%s): %s\n",  
mysql_errno (conn), mysql_sqlstate (conn), mysql_error (conn));  
  
}
```

通过辅助函数可以更容易地报告错误，调用如下：

```
if (...某个MySQL函数调用失败...)  
{  
  
    print_error (conn, "...some error message...");  
  
}
```

`print_error()` 函数会输出出错消息，并调用MySQL的那几个出错报告函数。`print_error()` 调用比 `fprintf()` 调用简单，编写也更容易，并且还可以让程序代码更易于阅读。此外，如果能把 `print_error()` 编写得更细致一点，即使当 `conn` 为 `NULL` 时也能做些有实际意义的事情，那么我们便可以把它用在像 `mysql_init()` 调用失败这样的场合里。然后，我们就不会再像以前那样，把出错报告调用混杂在一起：有的使用 `fprintf()`，有的使用 `print_error()`。

有人可能会对此提出质疑：“其实，并不是每次报告时都必须调用所有的出错报告函数。你故意夸大了出错报告操作的繁琐，其目的只不过是让你的辅助函数显得更有价值。再说了，你也用不着每次都去键入出错信息输出语句，你可以只编写一次，然后在需要它时使用复制粘贴即可。”这种说法的确有一定的道理，但我对此的答复如下。

即使使用复制粘贴，也只是对较短的代码比较方便而已。

如果报告错误很容易，那么在可能的情况下，你更应该坚持检查它们。

在每次报告错误时，不管你是否喜欢调用所有的出错报告函数，把所有的出错报告代码都写出来的方式，迟早会让你禁不住想要采取某些偷懒的做法，进而导致你在报告错误时出现前后不一致的情况。把出错报告代码“打包”放在一个易于调用的辅助函数里，能防微杜渐，提高代码的一致性。

如果你想要修改出错消息的格式，那么只修改一个地方，肯定比修改整个程序更省事。或者，你决定把出错消息写到某个日志文件里，不想再将它们写到 `stderr`（或者想要同时写到这两个地方），这时如果能只需修改

`print_error()` 函数即可达到目的, 那么事情便会容易很多。这种方法能减少犯错, 让你的出错报告工作不会半途而废, 且保持前后一致。

如果在测试程序时使用了调试器, 那么你可以很方便地在出错报告函数里设置一个断点, 让程序在每次检测到有错误发生时都能及时地切换到调试器。

基于这些原因, 本章后面的程序示例都将使用 `print_error()` 来报告与 MySQL 有关的错误。

下面是 `print_error()` 的定义, 它具备了刚才所讨论的那几个优点:

```
static void  
  
print_error (MYSQL *conn, char *message)  
{  
  
    fprintf (stderr, "%s\n", message);  
  
    if (conn != NULL)  
  
    {  
  
        fprintf (stderr, "Error %u (%s): %s\n",  
mysql_errno (conn), mysql_sqlstate (conn), mysql_error (conn));  
  
    }  
  
}
```

在 `connect2.c` 程序里, 用于出错检查的部分与 `connect1.c` 程序里相对应的代码很相似, 并且我们可以像下面这样使用 `print_error()`:

```
/* 初始化连接处理器 */  
  
conn = mysql_init (NULL);  
  
if (conn == NULL)  
  
{
```



```

print_error (NULL, "mysql_init() failed (probably out of memory)");

exit (1);

}

/* 连接服务器 */

if (mysql_real_connect (conn, opt_host_name, opt_user_name, opt_password,
opt_db_name, opt_port_num, opt_socket_name, opt_flags) == NULL)
{
print_error (conn, "mysql_real_connect() failed");

mysql_close (conn);

exit (1);

}

```

出错检查逻辑的前提是，`mysql_init()` 和 `mysql_real_connect()` 在失败时都会返回 `NULL`。请注意，如果 `mysql_init()` 失败，那么我们将把 `NULL` 传递给 `print_error()` 的第一个参数。这将导致 `print_error()` 不会再去调用 MySQL 的出错报告函数，因为无法确保传递给那些函数的连接处理器还包含着有意义的信息。相比之下，如果 `mysql_real_connect()` 失败，那么我们一定会把连接处理器传递给 `print_error()` 函数。虽然这个处理器并没有包含对应于有效连接的信息，但是它将包含一些可以被递给出错报告类函数的诊断信息。这个处理器还可以传递给 `mysql_close()` 函数，以释放 `mysql_init()` 已自动为它分配的各种内存。（请切记，千万不要把这个处理器传递给任何其他客户端例程！因为它们中的大部分都只有在连接有效的前提下才能正常工作，否则你的程序可能会崩溃！）

本章的其他示例程序都会进行出错检查，而你的程序也应如此。从长远来看，这是一种省事的做法，因为你不必花费大量的时间去跟踪那些细小的问题。

7.3.2 实时获取连接参数

现在我们来完成这样一个任务：让用户在运行时可以指定连接参数，而不是使用硬编码的默认参数。程序 `connect1` 有一个显著的缺陷，即连接参

数被直接写在源代码里。不管改变它们当中的哪一个，你都必须编辑源文件，并重新编译它。这很不方便，如果你的程序还有其他使用者，那么事情将变得更麻烦。实时设定连接参数的一个常用方法是使用命令行选项。例如，很多MySQL发行版里的程序都能接受两种形式的参数，如下表所示。

参数	长选项形式	短选项形式
主机名	--host=host_name	-h host_name
用户名	--user=user_name	-u user_name
密码	--password 或 --password=your_pass	-p 或 -pyour_pass
端口号	--port=port_num	-P port_num
套接字名	--socket=socket_name	-S socket_name

为了与标准的MySQL客户端程序保持一致，客户端程序connect2将采用与之相同的格式。这事做起来并不难，因为客户端开发库已支持选项处理。此外，connect2还将具备从选项文件里提取信息的能力。这让我们可以把连接参数放到~/.my.cnf文件（即主目录里的.my.cnf文件）中，或者放到某个全局选项文件里。然后，你便不用在每次启动程序时在命令行上设定那些选项。客户端开发库已让“对 MySQL 选项文件进行检查和从中提取相关值的操作”变得很容易。只需在程序里增加几行代码，便可以让它支持选项文件，无须自己动手编写代码，做无谓的重复劳动。（更多与选项文件语法有关的信息请参考F. 2. 2节。）

在展示 connect2 里的选项处理是如何工作的之前，我们会先开发两个程序，用于演示所涉及的一般性原理。从这些程序可以看到，选项处理工作其实相当简单，并且不会增加MySQL服务器连接和语句处理的复杂性。

注意

MySQL 另外还提供了两个与服务器连接建立有关的选项：用于指定连接协议（如TCP/IP、Unix套接字文件等）的选项--protocol，以及在Windows系统上，用于指定共享内存连接建立所需要的共享内存名字的选项--shared-memory-base-name。本章没有对这两个选项进行介绍，但是sampdb发行版里有protocol程序的源代码，该程序展示了如何使用这些选项，有兴趣的读者请自行参阅。

7. 3. 2. 1 访问选项文件的内容

如果要从选项文件读取连接参数值，可以调用load_defaults()。此函数会查找选项文件，解析其内容，从而找出你所感兴趣的选项组，并改写程序的参数向量（即 argv[] 数组）。它会把选项组里的信息以命令行选项的形式放在argv[]的开头。如此一来，这些选项看起来就像是在命令行指定的一样。当你解析这些命令行选项时，你便会在普通的选项处理代码里看到那些连接参数。这些选项会被添加到argv[]里，并且正好处于命令名的后面，其他参数的前面（不在末尾）。这样，命令行指定的任何连接参数都将出现后面部分，从而可以改写通过load_defaults()添加的任何选项。

下面是一个小程序show_argv，它演示了如何使用load_defaults()，并且举例说明了如何修改参数向量。

```
/*  
  
 * show_argv.c - 演示load_defaults()对参数向量的影响  
  
 */  
  
#include <my_global.h>  
  
#include <my_sys.h>  
  
#include <mysql.h>  
  
static const char *client_groups[] = { "client", NULL };  
  
int  
  
main (int argc, char *argv[])  
  
{
```

```

int i;

printf ("Original argument vector:\n");

for (i = 0; i < argc; i++)

printf ("arg %d: %s\n", i, argv[i]);

MY_INIT (argv[0]);

load_defaults ("my", client_groups, &argc, &argv);

printf ("Modified argument vector:\n");

for (i = 0; i < argc; i++)

printf ("arg %d: %s\n", i, argv[i]);

exit (0);

}

```

这段选项文件处理代码包含了以下几个部分。

`client_groups[]`。这是一个字符串数组，用于存放可以获得选项的那些选项文件组的名字。一般情况下，客户端程序至少应该把"client"包含在这个列表里（它代表的是选项组[client]），不过你可以根据需要列出多个选项组。此数组的最后一个元素必须是NULL，以此表明列表到此为止。

`MY_INIT()`。它是一个初始化宏，我们之前用过它。这里要强调的重点是，`MY_INIT()`会调用`my_init()`来完成`load_default()`要求的某些设置工作。

`load_defaults()`。它用于读取选项文件，有4个参数：选项文件名的前缀（它应该一直是"my"）、列出你所感兴趣的那些选项组名的数组，以及程序的参数计数器和向量的地址。请注意，不要直接传递计数器和向量的值。要传递它们的地址，因为`load_defaults()`需要更改它们的值。特别是对于`argv`，虽然它已经是一个指针，但你仍然要传递`&argv`，即该指针的地址。

为展示 `load_defaults()`对参数数组的影响，`show_argv` 程序分两次对其参数进行了输出。首先，当这些参数在命令行进行指定时，程序输出了它

们。接着，在调用`load_defaults()`之后，它再一次对参数数组进行了输出。

如果想看`load_defaults()`的工作情况，则必须确保你的主目录里有一个名为`.my.cnf`的文件，该文件的`[client]`选项组里必须有某些选项。（在Windows系统里，可以使用`C:\my.cnf`文件。）假设此文件有如下所示的内容：

```
[client]

user=sampadm

password=secret

host=some_host
```

然后，执行`show_argv`应该会得到如下所示的输出结果：

```
% ./show_argv a b

Original argument vector:

arg 0: ./show_argv

arg 1: a

arg 2: b Modified argument vector:

arg 0: ./show_argv

arg 1: --user=sampadm

arg 2: --password=secret

arg 3: --host=some_host

arg 4: a

arg 5: b
```

在`show_argv`第二次输出参数向量时，从上面的显示效果来看，选项文件里的那些值已成为参数列表的一部分。你还可能会看到一些未在命令行和`~/.my.cnf`文件里指定的选项。如果出现这种情况，你应该能够在某个系统

级选项文件的[client]选项组里找到它们。之所以会这样，是因为load_defaults()实际上会在多个位置查找选项文件。（更多与这些位置有关的信息请参考F.2.2节。）

使用load_defaults()的客户端程序，通常都会在选项组列表里包含"client"（以便获得各选项文件里通用的客户端设置）。但是，你也可以建立自己的选项文件处理代码，以便获得其他选项组里的选项。假设，想让show_argv 读取选项组[client]和[show_argv]里的各个选项。此时，需要先在show_argv.c里找到下列代码：

```
const char *client_groups[] = { "client", NULL };
```

接着，将其更改为下面这个样子：

```
const char *client_groups[] = { "show_argv", "client", NULL };
```

然后，重新编译show_argv。这个修改后的程序将会从两个选项组里读取选项。为验证这一点，可以在你的~/.my.cnf文件里增加一个[show_argv]选项组：

```
[client]
```

```
user=sampadm
```

```
password=secret
```

```
host=some_host
```

```
[show_argv]
```

```
host=other_host
```

有了这些修改之后，再次调用show_argv，便会产生与之前不一样的结果：

```
% ./show_argv a b
```

```
Original argument vector:
```

```
arg 0: ./show_argv
```

```
arg 1: a
```

arg 2: b

Modified argument vector:

arg 0: ./show_argv

arg 1: --user=sampadm

arg 2: --password=secret

arg 3: --host=some_host

arg 4: --host=other_host

arg 5: a

arg 6: b

各个选项值在参数数组里的先后顺序，取决于它们在选项文件里的顺序，而非选项组名在`client_groups[]`数组里的顺序。这意味着，你可能会想要在选项文件里的`[client]`组的后面指定程序特有的选项组。采用这种方式之后，如果你在两个选项组里都指定了某个选项，那么程序的特有值会获得比`[client]`组里的通用值更高的优先级。从刚才的示例里可以看到这一点：选项组`[client]`和`[show_argv]`都指定了`host`选项，但因为`[show_argv]`选项组位于选项文件的最后，所以它的`host`设置会最后出现在参数向量里，并因此获得优先。

`load_defaults()` 函数不会通过环境设置获得值。如果需要用到环境变量（如`MYSQL_TCP_PORT` 或 `MYSQL_UNIX_PORT`）的值，则必须使用`getenv()` 函数自己动手实现。这里并不打算把此项功能添加到我们的客户端里，不过下面这一小段代码演示了“如何对两个MySQL相关的环境变量的值进行检查”：

```
extern char *getenv();

char *p;

int port_num = 0;

char *socket_name = NULL;

if ((p = getenv ("MYSQL_TCP_PORT")) != NULL)
```

```
port_num = atoi (p);

if ((p = getenv ("MYSQL_UNIX_PORT")) != NULL)

socket_name = p;
```

在标准 MySQL 客户端程序里，环境变量值的优先级要低于通过选项文件或命令行指定的值。如果你想在自己的程序里检查环境变量，并且想与上述惯例保持一致，那么请在调用 `load_defaults()` 或者处理命令行选项之前（而不是之后），对环境变量进行检查。

`load_defaults()` 与安全性

在多用户系统上，像 `ps` 程序这样的实用工具，能够把任何进程（包括其他用户所运行的）的参数列表显示出来。正因如此，你可能会担心，`load_defaults()` 的进程嗅探功能是否会提取它在选项文件里找到的密码，并把它们放在参数列表里。这一点其实并不是个问题，因为 `ps` 程序只显示 `argv[]` 的最初内容。由 `load_defaults()` 创建的密码参数，将指向它自己分配的内存区域。这个区域并不是原来那个向量的一部分，因此 `ps` 程序绝对看不到它。

另一方面，在命令行上给出的密码确实会被 `ps` 程序显示出来，所以在命令行指定密码不是个好主意。为降低风险，程序可以采取的预防措施是，它一开始执行，就从参数列表里删除密码。具体做法请参考 7.3.2.2 节。

7.3.2.2 处理命令行参数

利用 `load_defaults()`，我们便可以把所有的连接参数都放到参数向量里，但是我们还需要一种处理向量的方法。`handle_options()` 函数就是专门为此设计的。它是 MySQL 客户端开发库的一部分，因此只要你链接了这个函数库，便可以访问它。

客户端开发库的选项处理例程有以下几个特点。

精确规范了合法值的选项类型和取值范围。例如，你不仅可以指明某个选项必须为一个整数值，而且还可以指明它必须是一个正整数，且是 1024 的倍数。

集成了帮助文本。如此一来，输出帮助消息的操作变得更加简便，只需调用标准库函数即可。你不用再自己编写代码来产生帮助消息。

提供了对标准选项`--no-defaults`、`--print-defaults`、`--defaults-file`和`--defaults-extra-file`的内建支持。更多相关信息请参考F. 2. 2节。

支持标准的选项前缀，如`--disable-`、`--enable-`和`--loose-`，从而使布尔（开/关）型选项和可忽略型选项更易实现。（本章没有用到这个功能，更多相关信息请参考F. 2节。）

为向大家演示如何使用MySQL的选项处理机制，本节将会对`show_opt`程序进行讲解。这个程序会调用`load_defaults()`，以便读取选项文件和设置参数向量，然后会使用`handle_options()`来处理结果。

`show_opt`程序可以让你体验到各种不同的指定连接参数的方法（不管是通过选项文件，还是通过命令行）；也可以把那些用于连接MySQL服务器的参数值都显示出来，让你看到最终结果。`show_opt`程序能帮助大家初步了解下一个客户端程序`connect2`的处理过程，它会把这里的选项处理代码与实际连接服务器的代码挂接起来。

为说明在参数处理的各个阶段都发生了哪些事情，`show_opt`程序将执行下面几个动作。

（1）把主机名、用户名、密码和其他连接参数初始化为默认值。

（2）输出原始的连接参数值和参数向量值。

（3）调用`load_defaults()`来改写参数向量，以便反映出选项文件的内容，然后输出修改后的最终向量。

（4）调用选项处理例程 `handle_options()`来处理参数向量，然后输出最终的连接参数值和参数向量里最后剩余的内容。

在对`show_opt`的工作流程做出解释说明之前，先来看一下它的源文件——`show_opt.c`。

```
/*  
  
* show_opt.c - 使用load_defaults()和  
  
* handle_options()来演示选项的处理过程  
  
*/  
  
#include <my_global.h>
```

```

#include <my_sys.h>

#include <mysql.h>

#include <my_getopt.h>

static char *opt_host_name = NULL;    /* 服务器主机（默认为localhost）
*/

static char *opt_user_name = NULL;    /* 用户名（默认为login name） */

static char *opt_password = NULL;    /* 密码（默认为空） */

static unsigned int opt_port_num = 0; /* 端口号（使用内建值） */

static char *opt_socket_name = NULL; /* 套接字名（使用内建值） */

static const char *client_groups[] = { "client", NULL };

static struct my_option my_opts[] =    /* 选项信息结构 */

{

{"help", '?', "Display this help and exit",

NULL, NULL, NULL,

GET_NO_ARG, NO_ARG, 0, 0, 0, 0, 0, 0},

{"host", 'h', "Host to connect to",

(uchar **) &opt_host_name, NULL, NULL,

GET_STR, REQUIRED_ARG, 0, 0, 0, 0, 0, 0},

{"password", 'p', "Password",

(uchar **) &opt_password, NULL, NULL,

GET_STR, OPT_ARG, 0, 0, 0, 0, 0, 0},

{"port", 'P', "Port number",

```

```

    (uchar **) &opt_port_num, NULL, NULL,
    GET_UINT, REQUIRED_ARG, 0, 0, 0, 0, 0, 0},
    {"socket", 'S', "Socket path",
    (uchar **) &opt_socket_name, NULL, NULL,
    GET_STR, REQUIRED_ARG, 0, 0, 0, 0, 0, 0},
    {"user", 'u', "User name",
    (uchar **) &opt_user_name, NULL, NULL,
    GET_STR, REQUIRED_ARG, 0, 0, 0, 0, 0, 0},
    { NULL, 0, NULL, NULL, NULL, NULL, GET_NO_ARG, NO_ARG, 0, 0, 0,
0, 0, 0 }

};

static my_bool

get_one_option (int optid, const struct my_option *opt, char *argument)

{

switch (optid)

{

case '?':

my_print_help (my_opts);    /* 输出帮助消息 */

exit (0);

}

return (0);

}

```

```

int

main (int argc, char *argv[])

{

int i;

int opt_err;

printf ("Original connection parameters:\n");

printf ("hostname: %s\n", opt_host_name ? opt_host_name : "(null)");

printf ("username: %s\n", opt_user_name ? opt_user_name : "(null)");

printf ("password: %s\n", opt_password ? opt_password : "(null)");

printf ("port number: %u\n", opt_port_num);

printf ("socket filename: %s\n",

opt_socket_name ? opt_socket_name : "(null)");

printf ("Original argument vector:\n");

for (i = 0; i < argc; i++)

printf ("arg %d: %s\n", i, argv[i]);

MY_INIT (argv[0]);

load_defaults ("my", client_groups, &argc, &argv);

printf ("Argument vector after calling load_defaults():\n");

for (i = 0; i < argc; i++)

printf ("arg %d: %s\n", i, argv[i]);

if ((opt_err = handle_options (&argc, &argv, my_opts, get_one_option)))

```

```

exit (opt_err);

printf ("Connection parameters after calling handle_options():\n");

printf ("hostname: %s\n", opt_host_name ? opt_host_name : "(null)");

printf ("username: %s\n", opt_user_name ? opt_user_name : "(null)");

printf ("password: %s\n", opt_password ? opt_password : "(null)");

printf ("port number: %u\n", opt_port_num);

printf ("socket filename: %s\n",

opt_socket_name ? opt_socket_name : "(null)");

printf ("Argument vector after calling handle_options():\n");

for (i = 0; i < argc; i++)

printf ("arg %d: %s\n", i, argv[i]);

exit (0);

}

```

show_opt.c所展示的选项处理方式涉及的几个方面，对于任何使用MySQL客户端开发库来处理命令行选项的程序来讲，都是相同的。在你自己的程序里，也应做同样的事情。

(1) 除了我们前面包含的那几个文件外，还要包含 my_getopt.h 文件，该文件定义了MySQL选项处理例程的接口。

(2) 定义my_option结构的数组。在show_opt.c里，此数组的名字为my_opts。对于程序能够识别的每一个选项，这个数组都应该拥有一个结构。每个结构都要提供一些信息，如选项的短名字和长名字、其默认值，以及该数值是数字还是字符串，等等。

(3) 在调用load_defaults()读取选项文件并设置参数向量之后，便可以调用handle_options()来处理这些选项。handle_options()的前两个参数分别是程序的参数计数和向量（与load_options()的情形一样，你必须传递这些变量的地址，而不能传递它们的值）。第三个参数指向my_option结构的

数组。第四个参数是一个指向辅助函数的指针。设计handle_options()例程和my_option结构，使绝大多数选项处理操作都可以由客户端开发库来替你自动完成。不过，如果想要执行某些客户端开发库处理不了的特殊动作，那么你的程序应该定义一个辅助函数，供handle_options()调用。在show_opt.c里，这个辅助函数的名字为get_one_option()。

my_option结构定义了一些必需的信息类型，这些类型是为程序所能解读的每一个选项指定的：

```
struct my_option

{

    const char *name;                /* 选项的长名 */

    int          id;                  /* 选项的短名或代码 */

    const char *comment;             /* 提供帮助消息的选项描述 */

    uchar        *value;              /* 用于存储值的那个变量的指
针 */

    uchar        *u_max_value;        /* 用户定义的最大变量值 */

    struct st_typelib *typelib;       /* 指向各种可能值（未使用）的指针 */

    ulong        var_type;            /* 选项值的类型 */

    enum get_opt_arg_type arg_type; /* 选项值是否必需 */

    longlong     def_value;           /* 选项的默认值 */

    longlong     min_value;           /* 选项的最小允许值 */

    longlong     max_value;           /* 选项的最大允许值 */

    longlong     sub_size;            /* 值的移动数量 */

    long          block_size;         /* 选项值的倍数 */

    void         *app_type;           /* 专为应用程序保留的 */
```

```
};
```

关于my_option结构的各个成员的具体用法，如下所示。

name是选项的长名。它是--name形式的选项，不包括开头的短划线。例如，长选项为--user，则在my_option结构里将被列为"user"。

id是选项的短名（单个字母）；如果选项没有单字母名字，那么id就是与选项相关联的代码值。例如，短选项为-u，那么在my_option结构里将被列为'u'。如果选项只有长名，而没有对应的单字符名字，那么你应该指定一组选项代码值，以便内部用于短名。代码值必须是唯一的，并且与所有单字符名字不同。（为了满足后面那条要求，可以让代码都大于255，这是单字符值的最大可能取值。具体示例请参考7.6节。）

comment是一个描述该选项用途的解释性字符串。这里的文字可以显示在帮助消息里。

value 是一个泛型指针的地址，其声明为一个uchar**值。如果这个选项带有参数，那么 value 会指向这个你想要存储的参数的变量。在各个选项都处理完成之后，你可以检查这个变量，看看此选项到底被设置了什么内容。被指向的那个变量的数据类型必须与 var_type 成员的值保持一致。如果此选项不带任何参数，那么 value 为NULL。

u_max_value 也是一个泛型指针地址，但它只被服务器使用。对于客户端程序，请把u_max_value设置为NULL。

typelib 目前尚未使用。在将来的MySQL 发行版本里，有可能被用于指定一组合法值，从而要求所有给定的选项值都必须与这些合法值中的某一个相匹配。

var_type 表明命令行里紧跟在选项名后面的那个值的类型。下表列出了这些类型，以及它们的含义和相应的C语言类型。

var_type值	含义	C语言类型
GET_NO_ARG	没有值	
GET_BOOL	布尔值	my_bool
GET_INT	整数值	int
GET_UINT	无符号整数值	unsigned int
GET_LONG	长整数值	long
GET_ULONG	无符号长整数值	unsigned long
GET_LL	长长整数值	long long
GET_ULL	无符号长长整数值	unsigned long long
GET_STR	字符串值	char*
GET_STR_ALLOC	字符串值	char*
GET_DISABLED	选项被禁用	
GET_ENUM	枚举值（当前未使用）	
GET_SET	集合值（当前未使用）	
GET_DOUBLE	双精度（浮点）值	double

GET_STR和GET_STR_ALLOC的区别是：对于GET_STR，客户端开发库会设置选项变量直接指向参数向量里的值；对于GET_STR_ALLOC，它会把这个参数复制一份，并设置选项变量指向这个副本。

GET_DISABLED 类型可以用来表明某个选项已不再可用，或是只在以某种特定方式建立程序的时候（如启用了调试支持功能）才可用。更多关于示例的信息请参考MySQL源代码发行版里的mysql.cc文件。

arg_type表明选项名后面是否跟着选项值，其可能的取值如下表所示。如果arg_type的取值为NO_ARG，则var_type应该为GET_NO_ARG。

arg_type值	含义
NO_ARG	选项后面不带参数
OPT_ARG	选项后面可能带有参数
REQUIRED_ARG	选项要求后面带有参数

def_value 适用于数字型选项。如果参数向量里没有显式指定值，则此选项会被设置成默认值。

min_value 适用于数字型选项。它是最小许可值。比它小的值将自动增大为这个最小值。如果为0时，则表示“没有最小值”。

max_value 适用于数字型选项。它是最大许可值。比它大的值将自动减小为这个最大值。如果为0时，则表示“没有最大值”。

sub_size 适用于数字型选项。它是一个偏移值，用于把参数向量里给定范围内的值，转换成内部所用范围内的值。例如，如果命令行给定的值的范围是1~256，但是程序内部实际使用的范围却是0到255，那么sub_size会被设置为1。

block_size 适用于数字型选项。如果它不等于零，则它所表示的是单个块的大小。如有必要，用户给出的选项值将会向下近似为最接近其倍数的那个值。例如，如果选项值必须为偶数，那么请把单个块的大小设置为 2。handle_options() 函数将把奇数值向下近似为最接近的那个偶数。

app_type为专为应用程序保留的。

对于每一个有效选项，my_opts数组都应该有一个与之对应的my_option结构，后面还紧跟一个用于表明数组结束的终止结构：

```
{ NULL, 0, NULL, NULL, NULL, NULL, GET_NO_ARG, NO_ARG, 0, 0, 0,
0, 0, 0 }
```

当调用handle_options()处理参数向量时，它将跳过向量中的第一个参数（即程序名），然后再开始处理选项参数——即以短划线开头的参数。整个处理过程会一直持续到这个向量的末尾，或者遇到专门的用双短划线（即“--”）表示的“选项结束”参数为止。当handle_options()遍历参数向量时，一旦遇到某个启用了辅助函数的选项，它便会调用这个函数来完成特殊的处理。handle_options()将向这个辅助函数传递 3个参数：短选项值、一

个指向该选项的`my_option`结构的指针，以及一个指向在参数向量里紧跟该选项后面的那个参数的指针（如果这个选项后面不带选项值，那么这个指针将为`NULL`）。

当 `handle_options()` 返回时，参数计数器和向量会被适当地重置，用以表示一个其中只包含非选项参数的参数列表。

下面是调用`show_opt`之后的输出结果（假设，`~/.my.cnf`的内容仍然与7.3.2.1节里的最后那个`show_argv`示例所用的文件内容一样）：

```
% ./show_opt -h yet_another_host --user=bill x
```

Original connection parameters:

hostname: (null)

username: (null)

password: (null)

port number: 0

socket filename: (null)

Original argument vector:

arg 0: ./show_opt

arg 1: -h

arg 3: yet_another_host

arg 3: --user=bill

arg 4: x

Argument vector after calling `load_defaults()`:

arg 0: ./show_opt

arg 1: --user=sampadm

arg 2: --password=secret

arg 3: --host=some_host

arg 4: -h

arg 5: yet_another_host

arg 6: --user=bill

arg 7: x

Connection parameters after calling handle_options():

hostname: yet_another_host

username: bill

password: secret

port number: 0

socket filename: (null)

Argument vector after calling handle_options():

arg 0: x

从上面的输出结果里可以看到：主机名取自命令行（它会改写选项文件里那个值），而用户名和密码则来自选项文件。无论是以短选项形式进行指定（如-h yet_another_host），还是以长选项形式进行指定（如--user=bill），handle_options()函数都可以正确地解析出这些选项。

辅助函数get_one_option()主要配合handle_options()一起使用。对show_opt程序来讲，它现在还相当小，并且只能处理选项--help或-?（为此，handle_options()会传递一个值为'?'的optid参数）：

```
static my_bool
```

```
get_one_option (int optid, const struct my_option *opt, char *argument)
```

```

{

switch (optid)

{

case '?':

my_print_help (my_opts); /* 输出帮助消息 */

exit (0);

}

return (0);

}

```

其中，`my_print_help()` 函数是一个客户端开发库例程，它会根据 `my_opts` 数组里的选项名和注释串自动生成一条帮助消息。如果想看看它是如何工作的，那么可以试试下面这条命令：

```
% ./show_opt --help
```

你还可以根据需要，把其他情况添加到 `get_one_option()` 里的 `switch()` 语句中（稍后我们将在 `connect2` 里这样做）。例如，`get_one_option()` 便可用于处理密码选项。当指定此选项时，可以选择给不给出密码值，如同选项信息结构里的 `OPT_ARG` 所表明的那样。也就是说，如果使用了长选项形式，那么可以把这个选项指定为 `--password` 或 `--password=your_pass`；如果使用了短选项形式，那么可以写成 `-p` 或 `-pyour_pass`。MySQL 客户端程序通常允许你在命令行省略密码值，然后提示你输入它。这样可以让你避免在命令行给出密码，从而防止他人看到你的密码。在后面的程序里，我们将用 `get_one_option()` 来检查是否给出了密码值。如果给出了密码值，那么我们会把它保存起来；否则，会设置一个标志，表明这个程序在开始连接服务器之前，应该提示用户输入密码。

修改一下 `show_opt.c` 中的选项结构，看看你的更改会对这个程序的行为产生什么样的影响。通过这种方式，你将学到更多。例如，把 `--port` 选项的最小值、最大值和块大小分别设置成 100、1000 和 25，然后重新编译这个程序，你便会发现，无法把端口号设置成 100 到 1000 这个范围以外的值，并且还会发现，这些值都自动向下近似为与之最接近的那个 25 的倍数值。

这些选项处理例程还能自动处理--no-defaults、--print-defaults、--defaults-file和--defaults-extra-file选项。请大家带上所有这些选项调用show_opt，看看会发生什么事情。

7.3.3 把选项处理整合进客户端程序

现在，我们来编写connect2.c。它具有以下特点。

它可以连接MySQL服务器、断开连接和退出。这与connect1.c很像，但是它使用了之前开发的print_error()函数来报告错误。

它可以处理命令行或选项文件里的各个选项。为实现这个功能，使用的代码与show_opt.c的相似，但略有修改，以便在必要时提示用户输入密码。

最终的源文件connect2.c，如下所示：

```
/*  
  
* connect2.c - 使用选项文件或命令行指定的连接参数，连接MySQL 服务器  
  
*/  
  
#include <my_global.h>  
  
#include <my_sys.h>  
  
#include <m_string.h>    /* 用于strdup() */  
  
#include <mysql.h>  
  
#include <my_getopt.h>  
  
static char *opt_host_name = NULL;    /* 服务器主机（默认为localhost） */  
  
static char *opt_user_name = NULL;    /* 用户名（默认为login name） */
```

```

static char *opt_password = NULL;      /* 密码（默认为空） */

static unsigned int opt_port_num = 0; /* 端口号（使用内建值） */

static char *opt_socket_name = NULL; /* 套接字名（使用内建
值） */

static char *opt_db_name = NULL;      /* 数据库名（默认为
空） */

static unsigned int opt_flags = 0;     /* 连接标志（为空） */

static int ask_password = 0;          /* 是否提示输入密码 */

static MYSQL *conn;                   /* 指向连接处理
器的指针 */

static const char *client_groups[] = { "client", NULL };

static struct my_option my_opts[] =    /* 选项信息结构 */

{

{"help", '?', "Display this help and exit",

NULL, NULL, NULL,

GET_NO_ARG, NO_ARG, 0, 0, 0, 0, 0},

{"host", 'h', "Host to connect to",

(uchar **) &opt_host_name, NULL, NULL,

GET_STR, REQUIRED_ARG, 0, 0, 0, 0, 0},

{"password", 'p', "Password",

```

```

(uchar **) &opt_password, NULL, NULL,

GET_STR, OPT_ARG, 0, 0, 0, 0, 0, 0},

{"port", 'P', "Port number",

(uchar **) &opt_port_num, NULL, NULL,

GET_UINT, REQUIRED_ARG, 0, 0, 0, 0, 0, 0},

{"socket", 'S', "Socket path",

(uchar **) &opt_socket_name, NULL, NULL,

GET_STR, REQUIRED_ARG, 0, 0, 0, 0, 0, 0},

{"user", 'u', "User name",

(uchar **) &opt_user_name, NULL, NULL,

GET_STR, REQUIRED_ARG, 0, 0, 0, 0, 0, 0},

{ NULL, 0, NULL, NULL, NULL, NULL, GET_NO_ARG,
NO_ARG, 0, 0, 0, 0, 0, 0 }

};

static void

print_error (MYSQL *conn, char *message)

{

fprintf (stderr, "%s\n", message);

if (conn != NULL)

```

```

{

    fprintf (stderr, "Error %u (%s): %s\n",

mysql_errno (conn), mysql_sqlstate (conn), mysql_error
(conn));

}

}

static my_bool

get_one_option (int optid, const struct my_option *opt, char *argument)

{

switch (optid)

{

case '?':

my_print_help (my_opts);    /* 输出帮助消息 */

exit (0);

case 'p':                    /* 密码 */

if (!argument)              /* 如果未指定值，则在随后提
示输入它 */

ask_password = 1;

else                          /* 复制密码，改写原有密码
*/

{

```



```

opt_password = strdup (argument);

if (opt_password == NULL)

{

print_error (NULL, "could not allocate password buffer");

exit (1);

}

while (*argument)

*argument++ = 'x';

ask_password = 0;

}

break;

}

return (0);

}

int

main (int argc, char *argv[])

{

int opt_err;

MY_INIT (argv[0]);

```

```
load_defaults ("my", client_groups, &argc, &argv);

if ((opt_err = handle_options (&argc, &argv, my_opts, get_one_option)))

exit (opt_err);

/* 如有必要则提示输入密码 */

if (ask_password)

opt_password = get_tty_password (NULL);

/* 如果命令行指定了数据库名，则获取它 */

if (argc > 0)

{

opt_db_name = argv[0];

--argc; ++argv;

}

/* 初始化客户端开发库 */

if (mysql_library_init (0, NULL, NULL))

{

print_error (NULL, "mysql_library_init() failed");

exit (1);

}

/* 初始化连接处理器 */
```

```
conn = mysql_init (NULL);

if (conn == NULL)

{

    print_error (NULL, "mysql_init() failed (probably out of memory)");

    exit (1);

}

/* 连接服务器 */

if (mysql_real_connect (conn, opt_host_name, opt_user_name,
opt_password,

    opt_db_name, opt_port_num, opt_socket_name, opt_flags) ==
NULL)

{

    print_error (conn, "mysql_real_connect() failed");

    mysql_close (conn);

    exit (1);

}

/* ...在此处执行语句和处理结果 ...*/

/* 断开与服务器的连接，终止客户端开发库 */

mysql_close (conn);

mysql_library_end ();
```

```
exit (0);
```

```
}
```

与此前开发的connect1和show_opt程序相比，connect2新做了如下几件事情。

允许以命令行参数的方式指定默认数据库。这与 MySQL 发行版里的标准客户端是一致的。

如果密码值存在于输入参数向量里，那么get_one_option()将复制它，并且会改写原来那个。这样可以大大缩小时间窗口——在这期间，在命令行指定的密码会被 ps 或其他系统状态程序看到。（时间窗口只能压缩到最小，但不能彻底消除。在命令行上指定密码的做法仍然存在安全风险。）

如果在命令行只给出了密码选项，而没有给出密码值，那么get_one_option()将设置一个标志，用以表明程序应该记得提示用户输入密码。这项工作是在main()里完成的，并且是在所有选项都被处理完之后调用get_tty_password()来完成的。这是一个客户端开发库里的辅助例程，它会提示你输入密码，同时不会把密码回显到屏幕上。你可能会问“为什么不调用getpass()呢？”答案就是，并非所有系统都有这个函数（例如，Windows系统就没有）。get_tty_password()可跨系统移植，因为它可通过配置来适应特定的系统。

在编译并链接connect2之后，便可以试着运行它：

```
% ./connect2
```

如果connect2程序没有产生任何输出（如上所示），则表明连接成功。如果有输出，你可能会看到下面这样的消息：

```
% ./connect2
```

```
mysql_real_connect() failed:
```

```
Error 1045 (28000): Access denied for user 'sampadm'@'localhost'
```

(using password: NO)

这种输出消息表明没有建立连接，并且它还说明了具体原因。此时，Access denied意味着你还需要提供一些必要的连接参数。对于connect1 程序，你将别无选择，只有通过编辑源文件并重新编译它来解决问题。但connect2程序是根据你指定的选项来连接MySQL服务器的。把问题简化一下，假设没有使用选项文件。如果不带任何参数运行 connect2，那么它会连接localhost主机，并且会把你的Unix登录名和空密码传递到服务器。但如果使用如下所示的命令来运行connect2程序，那么它将提示你输入密码（因为-p选项的后面没有紧跟密码值），接着会连接some_host主机，并把用户名some_user以及你在提示时输入的密码传递给服务器：

```
% ./connect2 -h some_host -p -u some_user some_db
```

connect2还会把数据库名some_db传递给mysql_real_connect()，以便让它成为默认数据库。如果带有选项文件，那么 connect2 还会处理它的内容，并且对连接参数进行相应的修改。

让我们稍停片刻，回想一下到目前为止，我们都取得了哪些成果。主要的成果体现在，让connect2具备了每一个MySQL客户端程序都应该具备的功能，即使用适当的参数去连接服务器。它还做了一件很漂亮的事情，即在连接失败时报告出错。我们现在有了一个框架，可用作各种不同客户端程序的基础。如果要编写新的客户端，可以这样做。

(1) 将connect2.c复制一份。

(2) 如果新程序还需要接受，除 connect2.c 已实现的那些标准选项以外的其他选项，那么请把这些新增选项添加到my_opts数组里，并对那个选项处理循环进行修改。

(3) 把你自己的应用程序所特有的代码添加到连接调用和断开连接调用之间。

至此，大功告成！

应用程序的所有实际动作都将发生在`mysql_real_connect()`调用和`mysql_close()`调用之间。不过，在有了可重用的编程骨架之后，你便可以吧注意力集中在你真正感兴趣的内容上——访问数据库的内容。

7.4 处理SQL语句

连接MySQL服务器的目的在于：在连接打开的时候，通过执行语句与其进行“对话”。本节的主要内容是展示具体该如何做。每条语句的执行都会涉及以下步骤。

（1）构造语句。实现的方式取决于语句本身的内容——具体地说，即它是否包含二进制数据。

（2）把语句发送给服务器执行。服务器将执行这条语句，并生成结果。

（3）对结果进行处理。这一步取决于你要执行的语句类型。例如，`SELECT`语句通常都会返回多行数据供你处理。`INSERT`语句则不返回数据。

MySQL客户端开发库包含两组用于语句执行的例程。第一组例程，会把每条语句当作一个字符串发送给服务器，并且会按字符串格式返回所有数据列的结果。第二组例程，使用的是二进制协议，能以原始格式发送和返回非字符串数据，不需要来回转换字符串格式。

本节将对第一种处理SQL语句的办法进行讨论。到7.8节才会讲到二进制协议。

在构造语句时，需要考虑的一个因素是，使用哪个函数来把它们发往服务器。最常用的语句调用例程是 `mysql_real_query()`。借助这个例程，可以把语句当作一个计数字符串（字符串加上长度值）。你必须计算出语句字符串的长度，并把这个长度值随同字符串一起传递给 `mysql_real_query()`。因为这种语句将被视为计数字符串，而不是以 `null` 结尾的字符串，所以它可以包含任何内容，包括二进制数据或 `null` 字节。

另一个语句调用函数是 `mysql_query()`，它对语句字符串里所允许的内容有更多的限制，但更易于使用。传递给 `mysql_query()` 的语句，必须是一个以 `null` 结尾的字符串。这意味着，语句文本不得包含 `null` 字节，因为 `null` 字节会导致语句被错误地解释成比实际长度更短。一般来讲，如果语句可以包含任意二进制数据，那么它就有可能包含 `null` 字节，因此这时不应使用 `mysql_query()`。另一方面，如果你正在处理以 `null` 结尾的字符串，那么你可以利用C语言标准函数库中的字符串函数来构造语句，其中有些函数你可能已经熟悉了，像 `strcpy()` 和 `sprintf()`。

在构造语句时，需要考虑的另一个因素是，是否需要执行某些字符转义操作。如果想构造的语句使用的值中包含二进制数据，或者其他具有特殊含义的字符，如引号或反斜线，那么必须对它们进行转义处理。7.4.7.1节将会对此问题进行讨论。

下面是一个简单的语句处理框架：

```
if (mysql_query (conn, stmt_str) != 0)

{

    /* 若失败，则报告错误 */

}

else
```

```
{  
  
/* 若成功，则弄清语句所带来的影响 */  
  
}
```

如果语句执行成功，`mysql_query()` 和 `mysql_real_query()` 都将返回零值；否则，它们都将返回一个非零值。所谓的语句“成功”指的是，MySQL服务器认为它没有语法错误，并且能够执行；与语句的执行效果没有任何关系。例如，“语句成功”并不表示，SELECT语句查询了多少个行；也不表示，DELETE语句删除了多少个行。对语句的实际执行效果进行检查，还会涉及其他的处理操作。

语句失败的原因有很多。下面列出一些较为常见的失败原因。

语句本身有语法错误。

语句在语义上有错误——例如，语句用了某个其实并不存在的表。

要访问语句所引用的那个表，但权限不够。

所有语句可以粗略地划分为两大类：一类用于修改行；另一类则会返回一个结果集（一组行）。像INSERT、DELETE和UPDATE这样的语句，可以修改行，并返回一个计数值，用以表明有多少个行受到了影响。

像SELECT和SHOW这样的语句，会返回一个结果集。在MySQL的C语言API里，这类语句所返回的结果集，将会通过MYSQL_RES数据类型来表示。这种数据类型其实是一个结构，它里面包含了各个行的数据值，以及关于这些值的元数据（如列名和数据值长度）。结果集也允许为空，即包含0个行。

7.4.1 处理那些修改行的语句

如果想要处理修改行的语句，那么可以使用`mysql_query()` 或 `mysql_real_query()` 来调用它。如果语句成功，那么调用 `mysql_affected_rows()` 可以查明实际插入、删除或修改了多少个行。

下面这个示例演示了如何处理修改行的语句：

```
if (mysql_query (conn, "INSERT INTO my_tbl SET name = 'My
Name'") != 0)

{

    print_error (conn, "INSERT statement failed");

}

else

{

    printf ("INSERT statement succeeded; number of rows affected:
%lu\n",

(unsigned long) mysql_affected_rows (conn));

}
```

请注意，为了便于输出显示，这段代码是如何把 `mysql_affected_rows()` 的返回值强制转换成 `unsigned long` 值的。此函数返回的是一个 `my_ulonglong` 类型的值，但是这种类型的值在某些数据库系统上无法直接输出。因此，需要把这个返回值强制转换成 `unsigned long` 类型的，并使用 `%lu` 输出格式来解决此问题。此原则也适用于其他会返回 `my_ulonglong` 类型值的函数，如 `mysql_num_rows()` 和 `mysql_insert_id()`。如果想让编写的客户端程序具备跨数据库系统的可移植性，那么请牢记这一点。

`mysql_affected_rows()` 函数会返回受语句影响的行数，不过“受影响的行”的具体含义取决于语句的类型。对于语句 `INSERT`、`REPLACE` 和 `DELETE`，它指的是插入、替换和删除的行数。对于 `UPDATE` 语句，它指的是实际修改的行数。如果行原有的内容与你要更新的内容相同，那么MySQL并不会更新这个行。这意味着，即使更新操作可能会（通过

UPDATE语句的WHERE子句) 查询到某个行, 但这个行也有可能实际并未改变。

对于UPDATE语句, “受影响的行” 的含义有时也存在争议, 因为有人会希望它的含义是 “匹配到行” ——即更新时查询到的所有行数, 即使这个更新操作没有实际更改它们的值, 也将其计算在内。如果某个应用程序确实需要用到这种含义, 那么只要在连接服务器时, 在标志参数里把CLIENT_FOUND_ROWS值传递给mysql_real_connect(), 服务器便可支持这种行为。

7.4.2 处理那些返回结果集的语句

有些语句会返回一个结果集。在调用这类语句时, 如果使用了mysql_query()或mysql_real_query(), 那么检索出来的数据将以结果集的形式返回。值得注意的是, 在MySQL里能返回行的语句并非只有SELECT。像SHOW、DESCRIBE、EXPLAIN和CHECK TABLE这样的语句, 也都会返回结果集。对于这些语句, 在调用语句之后, 必须执行附加的行处理操作。

(1) 调用mysql_store_result()或mysql_use_result()可以生成结果集。这两个函数在调用成功时, 会返回一个MYSQL_RES 指针; 失败时, 则会返回 NULL。我们稍后会对这两个函数之间的区别和它们各自的适用场合进行介绍。当前, 我们的示例使用的是mysql_store_result(), 这个函数会立刻从服务器检索出行, 并把它们缓存在客户端的内存里。

(2) 调用mysql_fetch_row()函数可以依次取回结果集里的每个行。这个函数在调用成功时, 会返回一个MYSQL_ROW值; 如果已没有行, 则会返回NULL。返回的MYSQL_ROW值其实是一个字符串数组指针, 其中的字符串分别代表着这个行里各个列的值。要如何处理这些列值, 则由应用程序来决定。例如, 你可以输出列值, 也可以对它们进行统计分析。

(3) 当处理完结果集之后, 请调用mysql_free_result()释放它所使用的内存资源。如果忘记做这件事, 那么应用程序会存在内存泄漏的问题。对于需要长时间运行的应用程序, 及时把结果集占用的内存资源释放掉是很重要的一步操作。否则, 系统会随着资源消耗量的增长, 变得越来越慢。

下面是一个对返回结果集的语句进行处理的示例：

```
MYSQL_RES *res_set;

if (mysql_query (conn, "SHOW TABLES FROM sampdb") != 0)

    print_error (conn, "mysql_query() failed");

else

{

    res_set = mysql_store_result (conn);    /* 生成结果集 */

    if (res_set == NULL)

        print_error (conn, "mysql_store_result() failed");

    else

    {

        /* 处理结果集，然后释放它 */

        process_result_set (conn, res_set);

        mysql_free_result (res_set);

    }

}
```

这段代码把对结果集的处理细节，都隐藏在另一个名为 `process_result_set()` 的函数里，只是我们还未定义它。一般来讲，那些处理结果集的操作都是基于一个如下的循环结构展开的：

```
MYSQL_ROW row;
```

```

while ((row = mysql_fetch_row (res_set)) != NULL)

{

/* 处理行的内容 */

}

```

其中，`mysql_fetch_row()` 将返回一个 `MYSQL_ROW` 值——它是一个数组指针。如果把这个返回值赋给一个名为 `row` 的变量，那么可以使用 `row[i]` 语法来访问行中的各个值，这里 `i` 的范围是从 0 到行里的列数少 1。关于数据类型 `MYSQL_ROW` 有以下几个值得注意的地方。

`MYSQL_ROW` 是指针类型，因此应该把这种类型的变量定义为 `MYSQL_ROW row`，而不能定义为 `MYSQL_ROW *row`。

在 `MYSQL_ROW` 数组里，对于所有类型的值，甚至包含数字类型的，都是以字符串方式返回的。如果想把某个值当作数字，则必须自己对字符串进行转换。

`MYSQL_ROW` 数组里的字符串都是以 `null` 结尾的。不过，如果某列可以包含二进制数据，那么它可能包含 `null` 字节，因此不能把这个值当作是以 `null` 结尾的字符串来对待。请通过数据列的长度来弄清这个数据列的值有多长。（更多关于如何确定数据列长度的信息请参考 7.4.6 节。）

在 `MYSQL_ROW` 数组里，SQL 的 `NULL` 值将被表示成 C 语言的 `NULL` 指针。如果数据列未被定义成 `NOT NULL`，那么你应该在程序里检查该数据列里的值是否为 `NULL`；否则，你的程序可能会因试图解引用 `NULL` 指针而崩溃。

要如何处理各个行，则取决于应用程序的具体目标。作为演示，我们只会把各个行都输出来，其中的各个列值之间用制表符隔开。如果要想做到这一点，则必须知道行由多少个列构成。这种信息可以使用另一个客户端开发库函数 `mysql_num_fields()` 来查询。

下面是 `process_result_set()` 的代码：

```

void

process_result_set (MYSQL *conn, MYSQL_RES *res_set)

{

MYSQL_ROW      row;

unsigned int i;

while ((row = mysql_fetch_row (res_set)) != NULL)

{

for (i = 0; i < mysql_num_fields (res_set); i++)

{

if (i > 0)

fputc ('\t', stdout);

printf ("%s", row[i] != NULL ? row[i] : "NULL");

}

fputc ('\n', stdout);

}

if (mysql_errno (conn) != 0)

print_error (conn, "mysql_fetch_row() failed");

else

printf ("Number of rows returned: %lu\n",

```

```
(unsigned long) mysql_num_rows (res_set));  
  
}
```

`process_result_set()` 函数将以制表符分隔（NULL值会被显示成单词“NULL”）的形式依次输出各个行的内容，以及检索到的行数。这个行数可以通过调用`mysql_num_rows()`获得。与`mysql_affected_rows()`相似的是，`mysql_num_rows()`返回的也是`m_ulonglong`值，因此需要先把它强制转换成`unsigned long`类型，再用格式符`%lu`来输出它。与`mysql_affected_rows()`不同的是，`mysql_num_rows()`接受的参数是一个结果集指针，而不是一个连接处理器。

循环结束后的那段代码包含一个用作预防措施的错误检测。如果使用`mysql_store_result()`来创建结果集，那么`mysql_fetch_row()`的返回值NULL将始终表示“已无更多行”。不过，如果使用`mysql_use_result()`来创建结果集，那么`mysql_fetch_row()`的返回值NULL则表示“已无更多行”或者有错误发生。因为`process_result_set()`并不知道其调用者在生成结果集时，使用的是`mysql_store_result()`，还是`mysql_use_result()`，因此，这个测试可以让它在两种情况下都能正确地检测到出错情况。

这里展示的`process_result_set()`版本只显示了最简单的输出各个列值的方法——当然肯定有缺点。请执行下面这个查询，看看会得到什么结果：

```
SELECT last_name, first_name, city, state FROM president  
  
ORDER BY last_name, first_name
```

你将得到下面这样的输出——它可能不太易读：

Adams	John	Braintree	MA
Adams	John Quincy	Braintree	MA
Arthur	Chester A.	Fairfield	VT

Buchanan	James	Mercersburg PA
Bush	George H.W.Milton	MA
Bush	George W.	New Haven CT
Carter	James E.	Plains GA
...		

我们可以让这个输出变得更美观一点，具体的做法是：提供一些信息，如列标签；让这些值都垂直对齐。为此，我们需要标签，并且必须知道每一个列的最大宽度。这些信息都可以获得，但它们并不包含在列的数据值里——而是在结果集的元数据（即关于数据的数据）里。在对语句处理器略作概括之后，我们将在7.4.6节编写一个显示格式美化程序。

输出二进制数据

包含二进制数据的列会包含null字节。对于这些数据列，使用printf()的格式说明符%s无法正确地进行输出。printf()期望的是以null结尾的字符串，并且只会输出列值里第一个null字节之前的内容。对于二进制数据，最好是使用可以接受列长度参数的函数，这样才能输出完整的值。例如，可以使用fwrite()。

7.4.3 通用的语句处理器

前面两节里的语句处理示例，利用了“语句是否应该返回某些数据”这样一个知识点。这种做法之所以能够奏效，原因在于这些语句都被硬编码在代码里：我们使用了一条不会返回结果集的INSERT语句，以及一条会返回结果集的SHOW TABLES语句。

不过，你不可能一直知道给定的语句所表示的是何种语句。例如，你执行的语句是从键盘或者某个文件读入的，那么这条语句可能是任何内容。你无法事先知道它是否期望返回行，甚至连它是否是合法的，也无法知道。那么，还能怎么做呢？你当然不想通过解析语句来确定它到底是何种类型。不管怎样，那是服务器应该干的事。

幸运的是，你不必事先知道语句的类型，便能对它作出正确的处理。利用MySQL的C语言API可以编写出一个通用的语句处理器，它可以正确地处理各种SQL语句。不管它是否会返回结果集，也不管它是否会执行成功，都能正确处理。在编写这个处理程序之前，我们先列出它实现的过程。

(1) 执行语句。如果执行失败，则结束。

(2) 如果语句执行成功，则调用`mysql_store_result()`从服务器检索行，并创建一个结果集。

(3) 如果 `mysql_store_result()` 函数调用成功，则语句返回一个结果集。调用`mysql_fetch_row()`函数来处理各个行，一直到它返回NULL为止，然后释放这个结果集。

(4) 如果 `mysql_store_result()` 调用失败，则具体原因可能是，语句根本不会返回结果集，也可能是它需要返回结果集，但在试图创建结果集时发生了错误。这两种情况都可以利用`mysql_field_count()`来加以区别，即把连接处理器传递给`mysql_field_count()`，然后检查它的返回值。

如果`mysql_field_count()`返回0，则表明这条语句没有返回任何列，因此也就没有结果集。（这也表明，此语句为INSERT、DELETE或UPDATE这样的语句。）

如果`mysql_field_count()`返回的是非零值，则表明有错误发生，因为该语句应该返回一个结果集，但却没有返回。发生这类错误的原因很多。例如，结果集大小过大，导致内存分配失败，或者客户端和服务端之间的网络连接在获取行时意外中断。

下面这个函数能够处理任何语句，前提是给定一个连接处理器和一个以 `null` 结尾的语句字符串：

```
void  
  
process_statement (MYSQL *conn, char *stmt_str)  
  
{
```



```

MYSQL_RES *res_set;

if (mysql_query (conn, stmt_str) != 0)    /* 语句执行失败 */
{
    print_error (conn, "Could not execute statement");
    return;
}

/* 语句执行成功；检查它是否返回了数据 */

res_set = mysql_store_result (conn);

if (res_set)        /* 返回了一个结果集 */
{
    /* 处理各个行，然后释放结果集 */
    process_result_set (conn, res_set);
    mysql_free_result (res_set);
}

else                /* 无结果集返回 */
{
    /*
    * 当没有结果集时，是因为不返回，
    * 还是因为应该返回，但出现了错误呢？
    */
}

```

```

*/

if (mysql_field_count (conn) == 0)

{

/*

* 语句不生成结果集（它不是SELECT、SHOW、DESCRIBE
等）；

* 只是报告受影响的行数。

*/

printf ("Number of rows affected: %lu\n",
(unsigned long) mysql_affected_rows (conn));

}

else /* 有错误发生 */

{

print_error (conn, "Could not retrieve result set");

}

}

}

```

7.4.4 另一种语句处理方法

上面展示的这个`process_statement()`版本有以下3个特点。

它使用了`mysql_query()`来执行语句。

它使用了mysql_store_result()来检索结果集。

在没有获得结果集时，它利用mysql_field_count()来判断：是出现了错误，还是原本就不会有结果集返回。

如果对语句处理的这3个方面加以调整，就能得到另一种处理方案。

在执行语句时，用一个计数字符串和mysql_real_query()来代替那个以null结尾的字符串和mysql_query()。

通过调用mysql_use_result()，而非调用mysql_store_result()来创建结果集。

调用mysql_error()或mysql_errno()，而非调用mysql_field_count()来确定是结果集检索失败，还是根本就没有供检索的结果集。

这几种方法可用来代替 process_query()里使用的对应方法。下面的process_real_query()函数与process_query()相似，但是使用了上述3个替换方法：

```
void
process_real_statement (MYSQL *conn, char *stmt_str, unsigned int len)
{
    MYSQL_RES *res_set;

    if (mysql_real_query (conn, stmt_str, len) != 0) /* 语句执行失败 */
    {
        print_error (conn, "Could not execute statement");
        return;
    }
}
```

```

}

/* 语句执行成功；检查它是否返回了数据 */

res_set = mysql_use_result (conn);

if (res_set)          /* 返回了一个结果集 */

{

/* 处理各个行，然后释放结果集 */

process_result_set (conn, res_set);

mysql_free_result (res_set);

}

else                  /* 无结果集返回 */

{

/*

* 当没有结果集时，是因为不返回，

* 还是因为应该返回，但出现了错误呢？

*/

if (mysql_errno (conn) == 0)

{

/*

* 语句不生成结果集（它不是SELECT、SHOW、DESCRIBE

等）；

```

```

    * 只是报告受影响的行数。

    */

    printf ("Number of rows affected: %lu\n",
(unsigned long) mysql_affected_rows (conn));

}

else    /* 有错误发生 */

{

    print_error (conn, "Could not retrieve result set");

}

}

}

```

7.4.5 mysql_store_result() 与 mysql_use_result() 的对比

函数mysql_store_result() 与 mysql_use_result() 有两个相似点，即都会接受一个连接处理器参数，并返回一个结果集。不过，它们之间的差异实际相当大。最本质的差异是，它们从服务器检索结果集的行的方式不同。当你调用mysql_store_result()时，它会立刻检索出所有的行；而 mysql_use_result() 则只会对检索进行初始化，并不会实际取回任何行。正是这一差异，导致了这两个函数在其他方面的种种差异。本节将对这两个函数进行比较，以便你能知道，如何为一个给定的应用程序选择一个最适当的函数。

当从服务器检索结果集时，mysql_store_result() 会取回行，并为它们分配内存，然后把它们在客户端缓存起来。此后的mysql_fetch_row() 调用决不会返回出错消息，因为它们只是从一个已

经包含结果集的数据结构里提取出行。因此，当`mysql_fetch_row()`返回NULL值时，则肯定是表示“已到达结果集的末尾”。

相比之下，`mysql_use_result()`本身并不会检索任何行。它只是逐行完成对检索的初始化工作，而你必须自己针对每一个行调用`mysql_fetch_row()`来完成检索操作。因此，虽然当`mysql_fetch_row()`返回NULL值时，通常是表示“已到达结果集的末尾”，但也有可能是表示“与服务器的通信出现了错误”。通过调用`mysql_error()`或`mysql_errno()`可以区分出这两种情况。

因为`mysql_store_result()`需要把完整的结果集保存在客户端，所以它的内存和处理需求都比`mysql_use_result()`大。内存分配和数据结构创建所需的开销都会更大，并且如果客户端检索的是大型结果集，那么会有内存耗尽的风险。如果你要检索某个包含大量行的结果集，那么应该使用`mysql_use_result()`。

因为`mysql_use_result()`每次只会取回一个行进行处理，所以它对内存的要求很低。同时，因为不必为创建结果集而建立各种复杂的数据结构，所以其内存分配速度也更快。另一方面，`mysql_use_result()`却加重了服务器的负担，因为服务器必须保存结果集里的行，一直到客户端程序认为适合检索所有这些行那一刻为止。因此，`mysql_use_result()`不适合用在以下两类客户端程序里。

根据用户请求，逐个遍历各有关行的交互式客户端程序。（你肯定不想让服务器因为用户工间休息，而一直等着发送下一个行吧！）

在两次行检索操作之间，需要执行大量处理操作的客户端程序。

在上述两种情况里，客户端程序无法迅速检索完结果集里的所有行。这对服务器和其他客户端程序都有很大的负面影响。特别是，如果你使用的存储引擎像MyISAM一样，使用了表锁定功能，那么这种影响会表现得更突出。你从中检索数据的那些表，在查询期间，将一直处于读操作锁定状态。其他试图修改这些表的客户端程序都将因此而被阻塞。

虽然`mysql_store_result()`会消耗较多的内存，但也有好处——可以立刻访问整个结果集。因为结果集的所有行都可用，所以可以随机访问它们：函数`mysql_data_seek()`、`mysql_row_seek()`和

`mysql_row_tell()`可以让你按任意顺序访问各个行。如果使用`mysql_use_result()`，则只能按`mysql_fetch_row()`取回行时的顺序访问各个行。如果想按任意顺序，而不是按它们从服务器取回的顺序去处理各个行，则必须使用`mysql_store_result()`。例如，想让应用程序允许用户跳跃式地前后浏览用某个查询选取出来的各个行，则应该首选`mysql_store_result()`。

使用`mysql_store_result()`，可以访问某些使用`mysql_use_result()`时访问不到的列信息。调用`mysql_num_rows()`可以获得结果集里行的数量。而每一个列的值的最大宽度都存储在`MYSQL_FIELD`列信息结构的`max_width`成员里。如果使用`mysql_use_result()`，那么`mysql_num_rows()`会等你取回所有行之后，才返回正确的值。类似地，因为`max_width`的值也只有在行全部取回之后，才能正确地计算出来，所以在此之前，它是不可用的。

因为`mysql_use_result()`做的事情比`mysql_store_result()`少，所以它会提出一条`mysql_store_result()`不会提出要求：客户端程序必须通过`mysql_fetch_row()`调用取回结果集里的每个行。如果你在执行另一条语句之前，忘了这么做，那么当前结果集里尚未来得及取回的行将与下一条语句的结果集混杂在一起，此时会出现“out of sync”（数据不同步）的出错信息。（如果要避免出现这一问题，则需要执行第二条语句之前，调用`mysql_free_result()`函数来取回和丢弃尚未被取回的行。）这种处理模型的一种“潜台词”是，如果使用`mysql_use_result()`，那么每次只能处理一个结果集。

`mysql_store_result()`函数并不会导致数据不同步的问题，因为当它返回时，服务器上不会再有尚未被取回的行。事实上，如果使用的是`mysql_store_result()`，那么根本用不着显式调用`mysql_fetch_row()`。不过，如果你感兴趣的是结果集是否为空，而不是结果集里有什么样的数据，那么这个函数还是很有用的。例如，为了弄清`mytbl`表是否存在，可以执行下面这条语句：

```
SHOW TABLES LIKE 'mytbl'
```

如果在调用`mysql_store_result()`之后，`mysql_num_rows()`返回的是一个非零值，那么表明这个表存在。并不需要调用`mysql_fetch_row()`。

虽然应该在某个时候调用`mysql_free_result()`函数，释放在`mysql_store_result()`生成的结果集，但绝对不是在执行下一条语句之前。这意味着你可以同时生成并使用多个结果集，这与使用`mysql_use_result()`时所提出的“每次只能使用一个结果集”的约束要求是相反的。

为了提供最大限度的灵活性，可以让用户选择结果集的处理方案。`mysql`和`mysqldump`便是两个照这种方法做的程序。默认情况下，它们会使用`mysql_store_result()`，不过，当在命令行上给出`--quick`选项时，它们则会使用`mysql_use_result()`。

7.4.6 使用结果集元数据

结果集不仅包含从数据行里检索出来的列值，还包含与这些数据有关的信息。这种信息即为所谓的结果集“元数据”（metadata），其中包括以下内容。

结果集里的行数和列数。调用`mysql_num_rows()`和`mysql_num_fields()`函数即可获得。

当前行里各列值的长度。调用`mysql_fetch_lengths()`即可获得。

关于各列的信息，如列的名字和类型、每个列的值最大宽度，以及包含这个列的表。

这些信息都保存在`MYSQL_FIELD`结构里，调用`mysql_fetch_field()`即可获得。与所有用于访问这些列信息相关的函数信息请参考附录G。

元数据是否可用，取决于所用的结果集处理方法。7.4.5节曾介绍过，如果想使用行计数或者各个列的最大长度值，那么在创建结果集时，必须使用`mysql_store_result()`，而不能使用`mysql_use_result()`。

结果集元数据能帮助我们决定如何处理结果集数据。

列的名字及宽度有助于生成漂亮格式的输出——垂直排列，并带有列标题。

列计数则表明，那个依次处理各个行中各个列的循环，要迭代多少次才能结束。

行和列的计数，有助于为那些依赖于结果集维度的数据结构分配内存。

列的数据类型，可以让你清楚地辨别出很多情况，如列表示的是不是数字，它是否包含二进制数据，等等。

在之前的7.4.2节，我们编写了`process_result_set()`函数的一个版本，它可以把结果集的各行各列以制表符分隔的格式进行输出。这个程序很有用（例如，可以用它来把数据导入电子表格），但它的显示格式却不太美观，不便于直观查看或打印输出。再来看看之前那个`process_result_set()`版本产生的输出结果：

```
Adams      John      Braintree  MA
Adams      John Quincy Braintree  MA
Arthur     Chester A.  Fairfield  VT
Buchanan   James      Mercersburg PA
Bush       George H.W.Milton  MA
Bush       George W.    New Haven  CT
Carter     James E.     Plains     GA
...
```

下面，我们来编写 `process_result_set()` 函数的另一个版本，它会产生一个带有表头的表格形式的输出，并将每个列“框起来”。这个版本将以易于解释的格式显示相同的结果：

last_name	first_name	city	state
Adams	John	Braintree	MA
Adams	John Quincy	Braintree	MA
Arthur	Chester A.	Fairfield	VT
Buchanan	James	Mercersburg	PA
Bush	George H.W.	Milton	MA
Bush	George W.	New Haven	CT
Carter	James E.	Plains	GA

...

具体的显示算法执行了以下几个步骤。

- (1) 确定各列的显示宽度。
- (2) 输出一行框起来的列标签（用垂直线分隔，前后再加上短横线）。
- (3) 输出结果集每一个行里的各个值，同时把每个列都框起来（即以垂直线进行分隔），垂直对齐。对于数字，按右对齐方式输出。对于NULL值，输出单词“NULL”。
- (4) 最后，输出检索到的行总数。

这个练习很好地演示了如何使用结果集元数据，因为它要求了解相当多的结果集信息，而非只是了解它的行所包含的数据值。

你也许会在想：“嗯，这好像与mysql程序生成输出报告的方式差不多嘛。”是的，的确如此。希望你能把mysql程序的源代码与process_result_set()函数的最终代码做一个对比。其实它们并不相同，但这种对比肯定会给你带来更多启发。

首先，有必要确定各个列的显示宽度。下面列出了具体的做法。
请注意：其中的各种计算全部是依据结果集元数据完成的，并未引用任何的行值。

```
MYSQL_FIELD      *field;

unsigned long col_len;

unsigned int      i;

/* 确定列的显示宽度；要求结果集是由 */

/* mysql_store_result() 生成的，而非使用mysql_use_result()来生成 */

mysql_field_seek (res_set, 0);

for (i = 0; i < mysql_num_fields (res_set); i++)

{

    field = mysql_fetch_field (res_set);

    col_len = strlen (field->name);

    if (col_len < field->max_length)

        col_len = field->max_length;

    if (col_len < 4 && !IS_NOT_NULL (field->flags))

        col_len = 4;    /* 单词“NULL”的长度为4 */

    field->max_length = col_len;    /* 重置列信息 */

}
```

为计算各列的显示宽度，这段代码将遍历与结果集里的各个列相对应的MYSQL_FIELD结构。在进入循环之前，先通过调用mysql_field_seek()定位到第一个结构处。紧接着，调用mysql_fetch_field()返回指向后续列结构的指针。列的显示宽度是下面所示3个值中的最大值，其中的每一个值都取决于这个列结构信息里的元数据。

列标题field->name的长度。

field->max_length，是列中最长数据值的长度。

字符串“NULL”的长度。如果 field->flags 表明这个列允许包含 NULL 值，那么这个长度值有效。

请注意，在把列的显示宽度确定下来之后，我们把它赋给了max_length，它是我们从客户端开发库获得的MYSQL_FIELD 结构的一个成员。这个结构的内容允许更改，还是应该为只读呢？通常情况下，我会说“只读”。但是，MySQL发行版的某些客户端程序，会以类似的方式更改max_length的值，所以我认为这种做法也是可以的。（如果你不想修改max_length，那么可以分配一个unsigned long数组，并把计算出来的显示宽度存储在这个数组里。）

在计算显示宽度时，有一个细节很关键，即当使用mysql_use_result()创建结果集时，max_length还没有任何意义。因为我们需要使用max_length来确定列值的显示宽度，所以正确的算法操作需要这个结果集使用mysql_store_result()来创建。在使用mysql_use_result()，而非mysql_store_result()的程序里，可采用一种变通的方法来实现，即使用MYSQL_FIELD结构的length成员，通过它，可以获得列值的最大长度。

在知道列的宽度后，便为输出结果做好了所有准备。标题很容易处理。对于某个给定的列，利用field变量所指向的那个列信息结构，输出name成员即可实现。再带上刚才计算出的那个宽度值，就能实现输出了：

```
printf (" %-*s |", (int) field->max_length, field->name);
```

对于数据，循环遍历结果集里的各个行，在每次迭代时输出当前行的列值。在输出行的各个列值时，也需要一点技巧，因为这个值可能为 NULL，也可能代表某个数字（它必须按右对齐方式输出）。各个列值的输出结果如下，其中，row[i]用于存放数据值，而 field 则指向列信息：

```
if (row[i] == NULL)                /* 输出单词"NULL" */

printf (" %-*s |", (int) field->max_length, "NULL");

else if (IS_NUM (field->type))    /* 以右对齐方式输出值 */

printf (" %-*s |", (int) field->max_length, row[i]);

else                             /* 以左对齐方式输出值 */

printf (" %-*s |", (int) field->max_length, row[i]);
```

如果field->type表明的列数据类型为某种数字类型，如INT、FLOAT或DECIMAL，那么IS_NUM()宏的值为真。

用于显示结果集的最终代码如下。因为需要多次输出短划线，所以更为简便的方法是编写一个print_dashes()函数，让它实现这一功能，这样就不用多个地方重复这种短划线输出代码了。

```
void

print_dashes (MYSQL_RES *res_set)

{

    MYSQL_FIELD      *field;

    unsigned int      i, j;

    mysql_field_seek (res_set, 0);
```

```

fputc ('+', stdout);

for (i = 0; i < mysql_num_fields (res_set); i++)

{

field = mysql_fetch_field (res_set);

for (j = 0; j < field->max_length + 2; j++)

fputc ('-', stdout);

fputc ('+', stdout);

}

fputc ('\n', stdout);

}

void

process_result_set (MYSQL *conn, MYSQL_RES *res_set)

{

MYSQL_ROW      row;

MYSQL_FIELD    *field;

unsigned long col_len;

unsigned int    i;

/* 确定列的显示宽度；要求结果集是由 */

/* mysql_store_result()生成的，而非使用mysql_use_result()来生成 */

```

```

mysql_field_seek (res_set, 0);

for (i = 0; i < mysql_num_fields (res_set); i++)

{

field = mysql_fetch_field (res_set);

col_len = strlen (field->name);

if (col_len < field->max_length)

col_len = field->max_length;

if (col_len < 4 && !IS_NOT_NULL (field->flags))

col_len = 4;  /* 单词"NULL"的长度为4 */

field->max_length = col_len;  /* 重置列信息 */

}

print_dashes (res_set);

fputc ('|', stdout);

mysql_field_seek (res_set, 0);

for (i = 0; i < mysql_num_fields (res_set); i++)

{

field = mysql_fetch_field (res_set);

printf (" %-*s |", (int) field->max_length, field->name);

}

```

```

fputc ('\n', stdout);

print_dashes (res_set);

while ((row = mysql_fetch_row (res_set)) != NULL)
{
    mysql_field_seek (res_set, 0);

    fputc ('|', stdout);

    for (i = 0; i < mysql_num_fields (res_set); i++)
    {
        field = mysql_fetch_field (res_set);

        if (row[i] == NULL)          /* 输出单词"NULL" */
            printf (" %-*s |", (int) field->max_length, "NULL");

        else if (IS_NUM (field->type)) /* 以右对齐方式输出值 */
            printf (" %-*s |", (int) field->max_length, row[i]);

        else                        /* 以左对齐方式输出值 */
            printf (" %-*s |", (int) field->max_length, row[i]);
    }

    fputc ('\n', stdout);
}

print_dashes (res_set);

```



```

printf ("Number of rows returned: %lu\n",
(unsigned long) mysql_num_rows (res_set));

}

```

MySQL客户端开发库还提供了其他几种访问列信息结构的方法。例如，上面示例里的代码便使用下面这样的循环语句，多次访问了这些结构：

```

mysql_field_seek (res_set, 0);

for (i = 0; i < mysql_num_fields (res_set); i++)

{

field = mysql_fetch_field (res_set);

...

}

```

不过，`mysql_field_seek()`加`mysql_fetch_field()`的组合，并非是获得MySQL_FIELD结构的唯一方式。关于其他方式的更多信息请参考附录 G 中函数 `mysql_fetch_fields()`和`mysql_fetch_field_direct()`的描述。

使用metadata程序显示结果集元数据

sampdb发行版包含一个metadata程序的源代码，你可以编译运行它，查看各种语句产生的元数据。它会提示用户输入并执行SQL语句，但显示的是结果集的元数据，而非结果集的内容。如果要进行对比，则可以在调用mysql时，带上`--column-type-info`选项，然后执行相同的语句。

7.4.7 对特殊字符和二进制数据进行编码

执行语句的程序必须要小心处理某些特殊字符。例如，如果在某个用引号引起来的字符串里，包含一个引号字符，那么可以采用双写这个引号，或者在其前面加上一个反斜线的方式来解决：

```
'O"Malley'
```

```
'O\'Malley'
```

本节将对“如何处理字符串里的引号问题”和“如何处理二进制数据”进行讨论。

7.4.7.1 处理包含特殊字符的字符串

如果在语句里单纯以字面方式插入包含有引号、null字节或反斜线的数据值，那么在试图执行这条语句时便会引起各种问题。下面将对这一问题的本质和解决办法进行讨论。

假设，你想基于以null结尾的字符串的内容（指向它的是一个名为name_val的变量），构造SELECT语句：

```
char stmt_buf[1024];

sprintf(stmt_buf, "SELECT * FROM mytbl WHERE name='%s'",
name_val);
```

如果 name_val 变量的值为“O'Malley, Brian”之类的内容，那么最终构造出的语句将是非法的，因为在用引号引起来的字符串里出现了一个引号：

```
SELECT * FROM mytbl WHERE name='O'Malley, Brian'
```

对字符串内部的那个引号必须特殊对待，以便服务器不会把它解释成名字的结尾。标准SQL语言对此的习惯做法是，在这个字符串里双写引号。MySQL既支持这种惯例，也允许使用反斜线来对引号字符进行转义，因此你可以用下面两种格式中的任何一种来写出这条语句：

```
SELECT * FROM mytbl WHERE name='O"Malley, Brian'
```

```
SELECT * FROM mytbl WHERE name='O\'Malley, Brian'
```

这类问题可以用`mysql_real_escape_string()`来解决，此函数会对特殊字符进行编码，让它们在用引号引起来的字符串里变得可用。`mysql_real_escape_string()`会把空字节、单引号（'）、双引号（"）、反斜线（\）、换行符、回车符和Ctrl+Z当作特殊字符。（最后那个Ctrl+Z是Windows里的特殊字符，它有时候表示文件结尾。）

应该在什么时候使用`mysql_real_escape_string()`呢？最安全的答案是“一直都用”。不过，如果对数据的格式和内容很有把握，如事前执行了某些验证检查，那么可以不必对它们进行编码。例如，如果你正处理的字符串所代表的是一些合法的电话号码，即仅由数字和短划线组成，那么你当然没必要调用`mysql_real_escape_string()`。至于其他情况，则可能需要调用它。

`mysql_real_escape_string()`函数会对有问题的字符进行编码，它会把它们转换成两个以反斜线开头的字符序列。例如，null字节会被编码成“\0”，其中的“0”是可输出的ASCII字符0，而不再是null字节。反斜线、单引号和双引号将分别被编码成“\\”、“\'”和“\"”。

`mysql_real_escape_string()`函数的调用语法如下所示：

```
to_len = mysql_real_escape_string(conn, to_str, from_str, from_len);
```

`mysql_real_escape_string()`会对`from_str`进行编码，把结果写入`to_str`。它还会在结尾添加null，这将为把最终的字符串与像`strcpy()`、`strlen()`或`printf()`这样的函数一起使用带来方便。

`from_str`指向的是一个char缓冲区，其中包含要被编码的字符串。这个字符串可能包含任何内容，其中包括二进制数据。`to_str`指向的是一个已经存在的char缓冲区，其中写入的是编码后的字符串。注意，千万不要传递未经初始化的指针或者 NULL 指针，因为`mysql_real_escape_string()`并不会替你分配内存！`to_str`指向的那个缓冲区的长度，必须至少为 $(\text{form_len} * 2) + 1$ 个字节。（因为`from_str`里的每个字符，可能都需要被编码成一个由两个字符构成的序列；那个额外的字节是为结尾的null准备的。）

from_len和to_len的类型都是unsigned long。其中，from_len表示缓冲区from_str里的数据长度。它是提供长度值所必需的，因为from_str允许包含null字节，并且不能把它当作是以null结尾的字符串。to_len是mysql_real_escape_string()的返回值，是编码结果字符串的实际长度，它并未把结尾的null计算在内。

当mysql_real_escape_string()返回时，因为from_str里的null字节都会被编码成可打印的“\0”序列，所以to_str里的编码结果可以被当作是以null字节结尾的字符串。

为改写那段 SELECT 构造的代码，让它在遇到包含引号的名字值时也能正确地工作，我们可以像下面这样做：

```
char stmt_buf[1024], *p;

p = strcpy (stmt_buf, "SELECT * FROM mytbl WHERE name=");

p += strlen (p);

p += mysql_real_escape_string (conn, p, name_val, strlen (name_val));

*p++ = '\\';

*p = '\\0';
```

没错，这个样子的确很难看。为把这段代码变得简洁一点，也可以像下面这样做，代价就是需要使用第二个缓冲区：

```
char stmt_buf[1024], buf[1024];

(void) mysql_real_escape_string (conn, buf, name_val, strlen (name_val));

sprintf (stmt_buf, "SELECT * FROM mytbl WHERE name='%s'", buf);
```

有一点需要特别注意：传递给mysql_real_escape_string()的缓冲区，必须是真实存在的。请看下面这个示例，它便违反了这一原则：

```
char *from_str = "some string";

char *to_str;

unsigned long len;

len = mysql_real_escape_string (conn, to_str, from_str, strlen
(from_str));
```

问题出在哪里？to_str本该指向一个已经存在的缓冲区，但是它没有这样做——它未被初始化，因而可能指向任何一个位置。如果不想把内存搞得一团糟，请千万不要把一个未经初始化的指针传递给mysql_real_escape_string()的to_str参数。

7.4.7.2 处理二进制数据

另一个存在问题的情形是，在语句里使用了任意的二进制数据。例如，在把图像存入数据库的应用里，便会出现这种情况。因为二进制值可以包含任何字符（其中包括null字节、引号或反斜线），所以把它们直接嵌到语句里并不安全。

对二进制数据的处理离不开mysql_real_escape_string()。本节将利用从文件中读取的图像数据来演示具体的做法。这里的讨论也同样适用于其他任何形式的二进制数据。

假设，需要从文件里读出图像，并把它们存入一个名为picture的表。当然，同时还会存入一个唯一标识符。对长度小于16 MB的二进制值来说，MEDIUMBLOB类型是个很好的选择，因此你可以定义一个像下面这个样子的表：

```
CREATE TABLE picture

(

    pict_id      INT NOT NULL PRIMARY KEY,

    pict_data    MEDIUMBLOB
```

```
);
```

为把文件里的图像加载到picture表，可以使用下面这个load_image()函数来实现。同时，还需要给定一个标识号，以及一个指针，指向一个打开的包含有图像数据的文件：

```
int

load_image (MYSQL *conn, int id, FILE *f)

{

char          stmt_buf[1024*1024], buf[1024*10], *p;

unsigned long from_len;

int           status;

/* 开始创建INSERT 语句，同时添加id 值 */

sprintf (stmt_buf,

"INSERT INTO picture (pict_id,pict_data) VALUES (%d,' ",

id);

p = stmt_buf + strlen (stmt_buf);

/* 分块从文件读取数据，对每一块进行编码， */

/* 并添加到语句的末尾 */

while ((from_len = fread (buf, 1, sizeof (buf), f)) > 0)

{

/* 千万不要超出语句缓冲区的末尾！ */
```

```

    if (p + (2*from_len) + 3 > stmt_buf + sizeof (stmt_buf))
    {
        print_error (NULL, "image is too big");

        return (1);
    }

    p += mysql_real_escape_string (conn, p, buf, from_len);
}

*p++ = '\\';

*p++ = ')';

status = mysql_real_query (conn, stmt_buf, (unsigned long) (p -
stmt_buf));

return (status);
}

```

这个load_image()不会分配太大的语句缓冲区(1 MB)，因此它只适用于相对较小的图像。在实际的应用里，你可以根据图像文件的大小动态地分配这个缓冲区。

从数据库取回图像值(或任意二进制值)时，没有当初把它存入数据库时遇到的那么多问题。这个数据值是以原始格式存放在MYSQL_ROW 变量里的，其长度可以通过调用mysql_fetch_lengths()函数来获得。请记住把这个值当作一个计数字符串来处理，而不要把它当作一个以null结尾的字符串。

7.5 交互式语句执行程序

本节将把此前开发的许多代码结合起来，编写一个简单的交互式语句执行客户端程序——`exec_stmt`。这个程序可以让你输入语句，并使用通用语句处理器`process_statement()`来执行它们，然后使用前面开发的显示格式处理器`process_result_set()`来显示结果。

`exec_stmt`程序与客户端程序`mysql`很相似，当然在功能上没那么丰富。`exec_stmt`允许的输入内容有以下几项限制。

每一个输入行都必须包含一条完整的语句。

语句不应该以分号或`\g`作为结尾。

能识别出的唯一非SQL命令是`quit`和`\q`，这条命令用于终止程序。也可以使用`Ctrl+D`组合键来退出。

事实证明，编写 `exec_stmt` 完全不费力气（大概只有十几行的新代码）。我们的客户端程序骨架（即 `connect2.c`）和已完成的其他几个函数，几乎提供了我们所需要的所有内容。唯一需要我们添加的内容是一个用于接收输入并执行它们的循环。

在构建`exec_stmt`时，第一步是把客户端程序骨架`connect2.c`复制成`exec_stmt.c`。然后再为函数`process_statement()`、`process_result_set()`和`print_dashes()`添加代码。最后，在`exec_stmt.c`的`main()`函数里找到下面这行：

```
/* ...issue statements and process results here (在此处执行语句和处理结果) ...*/
```

把该行替换成如下所示的`while`循环：

```
while (1)

{

    char    buf[10000];

    fprintf(stderr, "query> ");                                /* 显示提示符
*/
```



```

if (fgets (buf, sizeof (buf), stdin) == NULL) /* 读取语句 */

break;

if (strcmp (buf, "quit\n") == 0 || strcmp (buf, "\\q\n") == 0)

break;

process_statement (conn, buf);          /* 执行语句 */

}

```

把exec_stmt.c编译成exec_stmt.o，再把exec_stmt.o与客户端开发库链接起来生成exec_stmt，就完成任务了。现在，你便有了一个能够执行任何语句并显示其结果的交互式MySQL客户端程序。下面这个示例演示了这个程序的工作情况，其中涉及的语句有SELECT语句和非SELECT语句，以及几条错误的语句：

```
% ./exec_stmt
```

```
query> USE sampdb
```

```
Number of rows affected: 0
```

```
query> SELECT DATABASE(), USER()
```

```

-----+-----
DATABASE() | USER()
-----+-----
sampdb     | sampadm@localhost
-----+-----

```

```
Number of rows returned: 1
```

```
query> SELECT COUNT(*) FROM president
```

```

-----
COUNT(*)
-----
43
-----

```

Number of rows returned: 1

```

query> SELECT last_name, first_name FROM president ORDER BY
last_name LIMIT 3

```

```

-----+-----
last_name | first_name
-----+-----
Adams     | John
Adams     | John Quincy
Arthur    | Chester A.
-----+-----

```

Number of rows returned: 3

```

query> CREATE TABLE t (i INT)

```

Number of rows affected: 0

```

query> SELECT j FROM t

```

Could not execute statement

Error 1054 (42S22): Unknown column 'j' in 'field list'

```

query> USE mysql

```

Could not execute statement

Error 1044 (42000): Access denied for user 'sampadm'@'localhost' to
database 'mysql'

7.6 编写支持SSL的客户端程序

MySQL包含对SSL的支持，并且你可以用它来编写自己的程序，让它通过安全连接来访问服务器。为展示如何实现这一功能，本节将描述如何修改 `exec_stmt`，并产生一个与之类似的客户端程序 `exec_stmt_ssl`。这两个程序看上去很相似，但后面那个使用了加密方式来建立连接。为了让 `exec_stmt_ssl` 正常工作，构建MySQL时也必须带有SSL支持功能，并且服务器在启动时，必须通过适当的选项设定好自己的证书和密钥文件。客户端也需要证书和密钥文件。更多相关信息请参考13.5节。

这个程序的源文件 `exec_stmt_ssl.c` 包含在 `sampdb` 发行版里，你可以直接用它来建立客户端程序 `exec_stmt_ssl`。下面的这个过程描述的是，如何从 `exec_stmt.c` 文件开始，一步步创建 `exec_stmt_ssl.c` 文件的步骤。

(1) 将 `exec_stmt.c` 复制成 `exec_stmt_ssl.c`。以下步骤都是针对 `exec_stmt_ssl.c` 执行的。(2) 为让编译器检测到SSL支持是否可用，MySQL在头文件 `my_config.h` 里定义了一个名为 `HAVE_OPENSSL` 的符号。这意味着，在编写与SSL有关的代码时，可以使用下面的结构，这样，在SSL不可用的时候，可以忽略这段代码。

```
#ifdef HAVE_OPENSSL

...此处是与SSL 相关的代码...

#endif
```

因为 `my_global.h` 包含 `my_config.h`，而 `exec_stmt_ssl.c` 文件又包含 `my_global.h`，所以你不必再显式地包含 `my_config.h` 文件。

(3) 修改那个包含选项信息结构的my_opts数组，让它包含一些标准的与SSL相关的选项（如--ssl-ca、--ssl-key 等）。最简便的办法是，用#include 指令把 sslopt-longopts.h的内容包括到my_opt数组里。修改后的my_opts如下所示：

```
static struct my_option my_opts[] =    /* 选项信息结构 */

{

{"help", '?', "Display this help and exit",

NULL, NULL, NULL,

GET_NO_ARG, NO_ARG, 0, 0, 0, 0, 0, 0},

{"host", 'h', "Host to connect to",

(uchar **) &opt_host_name, NULL, NULL,

GET_STR, REQUIRED_ARG, 0, 0, 0, 0, 0, 0},

{"password", 'p', "Password",

(uchar **) &opt_password, NULL, NULL,

GET_STR, OPT_ARG, 0, 0, 0, 0, 0, 0},

{"port", 'P', "Port number",

(uchar **) &opt_port_num, NULL, NULL,

GET_UINT, REQUIRED_ARG, 0, 0, 0, 0, 0, 0},

{"socket", 'S', "Socket path",

(uchar **) &opt_socket_name, NULL, NULL,
```

```

GET_STR, REQUIRED_ARG, 0, 0, 0, 0, 0, 0},

{"user", 'u', "User name",

(uchar **) &opt_user_name, NULL, NULL,

GET_STR, REQUIRED_ARG, 0, 0, 0, 0, 0, 0},

#include <sslopt-longopts.h>

{ NULL, 0, NULL, NULL, NULL, NULL, GET_NO_ARG,
NO_ARG, 0, 0, 0, 0, 0, 0 }

};

```

sslopt-longopts.h是一个公共的MySQL头文件。其内容如下所示（格式上稍有调整）：

```

#ifdef HAVE_OPENSSL

{"ssl", OPT_SSL_SSL,

"Enable SSL for connection (automatically enabled with other flags).

Disable with --skip-ssl.",

(uchar **) &opt_use_ssl, (uchar **) &opt_use_ssl, 0,

GET_BOOL, NO_ARG, 0, 0, 0, 0, 0, 0},

{"ssl-ca", OPT_SSL_CA,

"CA file in PEM format (check OpenSSL docs, implies --ssl).",

(uchar **) &opt_ssl_ca, (uchar **) &opt_ssl_ca, 0,

GET_STR, REQUIRED_ARG, 0, 0, 0, 0, 0, 0},

```

```

{"ssl-capath", OPT_SSL_CAPATH,
"CA directory (check OpenSSL docs, implies --ssl).",
(uchar **) &opt_ssl_capath, (uchar **) &opt_ssl_capath, 0,
GET_STR, REQUIRED_ARG, 0, 0, 0, 0, 0, 0},

{"ssl-cert", OPT_SSL_CERT, "X509 cert in PEM format (implies --
ssl).",
(uchar **) &opt_ssl_cert, (uchar **) &opt_ssl_cert, 0,
GET_STR, REQUIRED_ARG, 0, 0, 0, 0, 0, 0},

{"ssl-cipher", OPT_SSL_CIPHER, "SSL cipher to use (implies --ssl).",
(uchar **) &opt_ssl_cipher, (uchar **) &opt_ssl_cipher, 0,
GET_STR, REQUIRED_ARG, 0, 0, 0, 0, 0, 0},

{"ssl-key", OPT_SSL_KEY, "X509 key in PEM format (implies --ssl).",
(uchar **) &opt_ssl_key, (uchar **) &opt_ssl_key, 0,
GET_STR, REQUIRED_ARG, 0, 0, 0, 0, 0, 0},

#ifdef MYSQL_CLIENT

{"ssl-verify-server-cert", OPT_SSL_VERIFY_SERVER_CERT,

"Verify server's \"Common Name\" in its cert against
hostname used

when connecting. This option is disabled by default.",
(uchar **) &opt_ssl_verify_server_cert,

```

```

    (uchar **) &opt_ssl_verify_server_cert, 0,

    GET_BOOL, NO_ARG, 0, 0, 0, 0, 0, 0},

#endif

#endif /* HAVE_OPENSSL */

```

(4) `sslopt-longopts.h` 所定义的选项结构引用了 `OPT_SSL_SSL`、`OPT_SSL_KEY` 等多个值。这些值的用途是充当短选项代码，并且必须由你的程序来定义。具体来讲，只要把下面这些代码添加到 `my_opts` 数组定义的前面即可：

```

#ifdef HAVE_OPENSSL

enum options_client

{

    OPT_SSL_SSL=256,

    OPT_SSL_KEY,

    OPT_SSL_CERT,

    OPT_SSL_CA,

    OPT_SSL_CAPATH,

    OPT_SSL_CIPHER,

    OPT_SSL_VERIFY_SERVER_CERT

};

#endif

```

在编写你自己的程序时，如果这些程序也为其他选项定义了代码，那么请确保这些代码与OPT_SSL_XXX符号所对应的值不相同。

(5) 在sslopt-longopts.h 里与SSL有关的选项结构，会引用一组用于保存这些选项值的变量。为了声明这些变量，需要用一条#include指令把sslopt-vars.h的内容包括在程序里，并且是在my_opts数组的定义前面。sslopt-vars.h的内容如下所示：

```
#ifdef HAVE_OPENSSL

static my_bool opt_use_ssl    = 0;

static char *opt_ssl_ca      = 0;

static char *opt_ssl_capath = 0;

static char *opt_ssl_cert    = 0;

static char *opt_ssl_cipher = 0;

static char *opt_ssl_key     = 0;

#ifdef MYSQL_CLIENT

static my_bool opt_ssl_verify_server_cert= 0;

#endif

#endif
```

(6) 在get_one_option() 例程的末尾增加一行，用于包含sslopt-case.h文件：

```
static my_bool

get_one_option (int optid, const struct my_option *opt, char *argument)
```



```
{
```

```
switch (optid)
```

```
{
```

```
case '?':
```

```

my_print_help (my_opts);    /* 输出帮助消息 */

exit (0);

case 'p':                    /* 密码 */

    if (!argument)          /* 如果未给定值，则在随后提示输入它*/

        ask_password = 1;

    else                    /* 复制密码，改写原有密码 */

        {

            opt_password = strdup (argument);

            if (opt_password == NULL)

                {

                    print_error (NULL, "could not allocate password buffer");

                    exit (1);

                }

            while (*argument)

                *argument++ = 'x';

            ask_password = 0;

        }

    break;

```

```

#include <sslopt-case.h>

}

return (0);

}

```

sslopt-case.h包含了switch()语句的另外几种情况，用于检测是否给定了SSL选项。

如果有SSL选项，则设置opt_use_ssl变量。此文件的内容如下所示：

```

#ifdef HAVE_OPENSSL

case OPT_SSL_KEY:

case OPT_SSL_CERT:

case OPT_SSL_CA:

case OPT_SSL_CAPATH:

case OPT_SSL_CIPHER:

/*

如果使用了ssl选项，则启用SSL功能

以后可以使用--skip-ssl或--ssl=0来禁用SSL功能

*/

opt_use_ssl= 1;

break;

```

```
#endif
```

这种做法所造成的影响是：在选项处理工作结束后，程序可以通过检测`opt_use_ssl`值来判断用户是否想要使用安全连接。

如果使用以上步骤，那么经常用到的两个例程`load_defaults()`和`handle_options()`会负责解析 SSL 相关的选项，并自动设置它们的值。你唯一需要做的事情是，如果这些 SSL选项表明用户想要建立SSL连接，那么需要在连接到服务器之前，把这些信息传递到客户端开发库。这个工作可以在调用`mysql_init()`之后、调用`mysql_real_connect()`之前，通过调用`mysql_ssl_set()`来完成。整个操作序列如下所示：

```
/* 初始化连接处理器 */

conn = mysql_init (NULL);

if (conn == NULL)

{

    print_error (NULL, "mysql_init() failed (probably out of memory)");

    exit (1);

}

#ifdef HAVE_OPENSSL

/* 把SSL 信息传递到客户端开发库 */

if (opt_use_ssl)

    mysql_ssl_set (conn, opt_ssl_key, opt_ssl_cert, opt_ssl_ca,

opt_ssl_capath, opt_ssl_cipher);
```

```

mysql_options (conn,MYSQL_OPT_SSL_VERIFY_SERVER_CERT,
(char*)&opt_ssl_verify_server_cert);

#endif

/* 连接服务器 */

if (mysql_real_connect (conn, opt_host_name, opt_user_name,
opt_password,

opt_db_name, opt_port_num, opt_socket_name, opt_flags) ==
NULL)

{

print_error (conn, "mysql_real_connect() failed");

mysql_close (conn);

exit (1);

}

```

这段代码并没有通过测试 `mysql_ssl_set()` 来判断它是否返回了错误。当你调用`mysql_real_connect()`时，如果传递给此函数的信息有任何问题，就会导致出错。

把`exec_stmt_ssl.c`编译成`exec_stmt_ssl`程序，然后运行它。如果`mysql_real_connect()`调用成功，那么继续执行各条语句。如果在调用`exec_stmt_ssl`时，适当使用了SSL选项，那么与服务器的通信便是通过加密连接进行的。如果想确定一下是否果真如此，则可以执行下面这条语句：

```
SHOW STATUS LIKE 'Ssl_cipher'
```

如果Ssl_cipher的值非空，则说明正在使用某种加密算法。（要想更简单点儿，可以使用包含在sampdb发行版里的exec_stmt_ssl，替你执行这条语句，并报告结果。）

7.7 一次执行多条语句

MySQL客户端开发库支持一次执行多条语句的功能。这样，你便可以把一个由分号隔开的多条语句构成的字符串，发送给服务器，然后依次检索各个结果集。

一次执行多条语句的功能，默认是禁用的，因此当你想要使用它时，必须明确地告诉服务器。有两种告知办法。第一种是，在连接服务器时，把CLIENT_MULTI_STATEMENTS选项添加到mysql_real_connect()的标志参数里：

```
opt_flags |= CLIENT_MULTI_STATEMENTS;

if (mysql_real_connect (conn, opt_host_name, opt_user_name,
opt_password,

opt_db_name, opt_port_num, opt_socket_name, opt_flags) ==
NULL)

{

print_error (conn, "mysql_real_connect() failed");

mysql_close (conn);

exit (1);

}
```

第二种是，用mysql_set_server_option()来为现有连接启用这项功能。例如：

```
if (mysql_set_server_option (conn,  
MYSQL_OPTION_MULTI_STATEMENTS_ON) != 0)
```

```
print_error (conn, "Could not enable multiple-statement execution");
```

哪一种办法更好呢？如果程序没有使用存储过程，那么这两种办法都适用。如果程序使用了存储过程，并且调用了 CALL 语句来返回结果集，那么用第一种办法更好些。这是因为 CLIENT_MULTI_STATEMENTS 选项会顺便打开 CLIENT_MULTI_RESULTS 选项，后面这个选项必须启用，否则，当存储过程试图返回结果集时，会出现错误。（把 CLIENT_MULTI_STATEMENTS 选项添加到 mysql_real_connect() 的标志参数里，这种做法更可取，因为它可以明白无误地表明你启用了此选项。）

在处理多个结果集时，可以使用下面这两个函数，对结果检索的当前状态进行基本的检查。

mysql_more_results()。如果还有更多的结果，则返回非零值；否则，返回0。

mysql_next_result()。如果还有更多的结果，则返回一个状态值，同时初始化下一个结果集的检索工作。如果还有更多的结果，则状态值为 0；如果没有，则返回-1；如果发生了错误，则返回一个大于零的值。

在使用这些函数时，可以把结果检索代码放在某个循环里。在使用常规代码检索出一个结果之后，请记得检查一下是否还有结果需要检索。如果有，则再循环一次。如果没有，则退出循环。根据循环语句具体结构的不同，你可能根本不需要调用 mysql_more_results()，因为你可以根据 mysql_next_result() 的返回值来判断是否还有更多的结果。

在7.4.3节，我们编写了一个 process_statement() 函数，它可以执行一条语句，并检索其结果，或者显示受其影响的行数。通过把结果检索代码放入循环，并使用 mysql_next_result()，我们可以编写一个与之类似的 process_multi_statement() 函数，用于检索多个结果集：

```

void

process_multi_statement (MYSQL *conn, char *stmt_str)

{

MYSQL_RES *res_set;

int          status;

int          keep_going = 1;

if (mysql_query (conn, stmt_str) != 0)   /* 有语句执行失败 */
{

print_error (conn, "Could not execute statement(s)");

return;

}

/* 语句执行成功；进入结果检索循环 */

do {

/* 确定当前语句是否有数据返回 */

res_set = mysql_store_result (conn);

if (res_set)          /* 返回了一个结果集 */

{

/* 处理各个行，然后释放结果集 */

process_result_set (conn, res_set);

```



```

mysql_free_result (res_set);

}

else                                /* 无结果集返回 */

{

/*

* 当没有结果集时，是因为不返回，

* 还是因为出现了错误呢？

*/

if (mysql_field_count (conn) == 0)

{

/*

* 语句不生成结果集（它不是SELECT、SHOW、DESCRIBE 等）；

* 只是报告受影响的行数。

*/

printf ("Number of rows affected: %lu\n",

(unsigned long) mysql_affected_rows (conn));

}

else /* 有错误发生 */

{

```

```

print_error (conn, "Could not retrieve result set");

keep_going = 0;

}

}

/* 确定是否还有更多的结果存在*/

/* 0 表示有； -1 表示无； >0 表示出错 */

status = mysql_next_result (conn);

if (status != 0)      /*无结果集，或者出错 */
{

keep_going = 0;

if (status > 0)      /* 出错 */

print_error (conn, "Could not execute statement");

}

} while (keep_going);

}

```

如果愿意，你也可以只测试`mysql_next_result()`的返回值是否为0。如果不为0，则退出循环。这个更为简单的策略有一个不足之处：当没有检索到更多结果时，你无法知道是正常到达了末尾，还是发生了错误。也就是说，你无法判断是否应该显示一条出错消息。

7.8 使用服务器端预处理语句

在本章的前面几节中，用于处理SQL语句的代码都是基于MySQL客户端开发库所提供的那些函数编写的，它们都是以字符串的形式来发送和检索所有信息。本节将讨论如何使用二进制的客户端/服务器协议。二进制协议支持服务器端预处理语句，并且可以按本地格式来传输数据值。

并非所有语句都能被预处理。预处理语句 API 适用于这样几种语句：CREATE TABLE、DELETE、DO、INSERT、REPLACE、SELECT、SET、UPDATE和绝大多数的SHOW变体。预处理机制所支持的语句种类还在不断增加。更多关于最新语句列表的信息请参考MySQL参考手册。

为使用二进制协议，必须创建一个语句处理器。有了这个处理器，便可以把语句发送到服务器进行预处理。服务器会分析语句、记住它，并把有关它的信息（客户端开发库存储在语句处理器里的）发送回去。对这条语句的进一步处理会用到这个处理器。

用于预处理的语句可以被参数化，即包含问号（?），以此来表示，当你将来执行该语句时所提供的数据值会出现在该处。例如，可以像下面这样预处理语句：

```
INSERT INTO score (event_id,student_id,score) VALUES(?,?,?)
```

这条语句包含有3个“?”字符，它们的作用是充当参数标记或占位符。将来，你可以提供数据值绑定到这些占位符上。而当执行它时，这些数据便会让语句变得完整。预处理语句的这种“参数传递”机制使它们可以重复使用：只要把各有关“参数”替换成某些不同的值，便可以反复多次地执行同一条语句。这意味着，语句文本只需发送一次，并且每次执行该语句时，也只需发送数据值即可。对于重复执行的语句，这种做法可以大大改善性能。

服务器只需对语句分析一次，而不用每次执行时都分析一次。

能降低网络开销，因为每次执行时只需发送数据值，而不是整条语句。

数据值在发送时无需转换成字符串形式，这样可以降低执行开销。例如，上面那条INSERT 语句里的3个列全部都是 INT 列。每当使用mysql_query() 或 mysql_real_query() 执行类似的INSERT 语句时，

只需把这几个数据值转换成字符串，并嵌入语句文本里即可。在有了预处理语句接口之后，便可以用二进制格式来发送这些数据值。

不必对检索结果进行转换。在预处理语句的结果集里，非字符串值都是以二进制格式返回的，无需转换成字符串形式。

与最初的非二进制协议相比，二进制协议有以下几个缺点。

难以使用，因为在传输和接收数据值时需要更多的准备工作。

二进制协议并不支持所有的语句。例如，USE语句便不能被预处理。

对于交互式程序，最好还是使用原来的非二进制协议。在交互式程序里，从用户那里接收到的语句都只会执行一次。因此，使用预处理语句，并没有什么优势，它只能为那些需要反复执行的语句，带来最大限度的性能提升。

使用预处理语句的基本流程，包括以下几个步骤。

(1) 调用`mysql_stmt_init()`，分配一个语句处理器。这个函数会返回一个指向处理器的指针，后续步骤会用到这个处理器。

(2) 调用`mysql_stmt_prepare()`，把语句发送到服务器，让其接受预处理，并与语句处理器相关联。服务器会分析该语句的特征，如语句类型，它包含多少个参数标记，在执行时它是否会生成一个结果集，等等。

(3) 如果该语句包含占位符，那么在执行它之前，必须为每个占位符提供相应的数据。具体做法是，为每个参数创建一个`MYSQL_BIND`结构。每个结构表明一个参数的数据类型、它的值，以及它是否为`NULL`。然后，调用`mysql_stmt_bind_param()`函数，把这些结构与该语句绑定在一起。

(4) 调用`mysql_stmt_execute()`函数，执行该语句。

(5) 如果该语句只会修改数据，而不会生成结果集（如`INSERT`或`UPDATE`语句），那么调用`mysql_stmt_affected_rows()`函数来确定该语句影响的行数。

(6) 如果该语句会生成结果集，那么可以调用 `mysql_stmt_result_metadata()` 来获得关于该结果集的元数据。为获得这些行，需要再次使用 `MYSQL_BIND` 结构，但是，它们这次是用于接收从服务器返回的数据，而不是向服务器发送数据。你必须为结果集里的每一个列创建一个 `MYSQL_BIND` 结构。它们包含与每一个行的数据值有关的信息，这些值都是你期望从服务器接收到的。先调用 `mysql_stmt_bind_result()`，把这些结构绑定到语句处理器上，然后，反复调用 `mysql_stmt_fetch()`，依次取回每一个行。在取回每一个行之后，你便可以访问当前行的各个列值。

在调用 `mysql_stmt_fetch()` 之前，可以选择调用 `mysql_stmt_store_result()`。如果调用了，则可以从服务器里一次性取回全部结果集的行，并缓存在客户端的内存里。另外，也可以通过调用 `mysql_stmt_num_rows()`，确定结果集里的行数。如果结果集为空，则该函数返回零。

在取回结果集之后，请记得调用 `mysql_stmt_free_result()`，释放与之相关的内存。

(7) 如果想再次执行刚才那条语句，可以返回到第3步，同时指定新的参数值。

(8) 如果想使用这个处理器来预处理另一条语句，则可以返回到第2步。

(9) 在使用完语句处理器之后，请别忘了调用 `mysql_stmt_close()` 来释放它。如果客户连接关闭时，服务器仍然把预处理语句与该连接关联在一起，那么服务器会自动释放它们。

在同一个客户端应用程序里，可以先准备多条语句，然后按照应用程序要求的顺序逐个执行它们。

接下来，我们将编写一个简单的程序，把多个行插入表中，然后再检索它们。在这个程序里，处理 `INSERT` 语句的那个部分演示的是：如何在语句里使用占位符，以及如何在执行预处理语句时，把数据值传递给与它绑定在一起的那个服务器。处理 `SELECT` 语句的那个部分演示的是：如何检索预处理语句所产生的结果集。在 `sampdb` 发行版的 `capi` 目录下，可以找到两个文件：`prepared.c` 和

process_prepared_statement.c，它们是这个程序的源文件。这里没有展示用于建立连接的代码，因为它与前几个示例程序里的代码相似。

建立此程序的目的是使用预处理语句，其主要部分如下所示：

```
void
process_prepared_statements (MYSQL *conn)
{
    MYSQL_STMT *stmt;

    char          *use_stmt = "USE sampdb";

    char          *drop_stmt = "DROP TABLE IF EXISTS t";

    char          *create_stmt =

"CREATE TABLE t (i INT, f FLOAT, c CHAR(24), dt DATETIME)";

    /* 打开数据库，并创建测试表 */

    if (mysql_query (conn, use_stmt) != 0
        || mysql_query (conn, drop_stmt) != 0
        || mysql_query (conn, create_stmt) != 0)
    {
        print_error (conn, "Could not set up test table");
        return;
    }
}
```

```

stmt = mysql_stmt_init (conn);  /* 分配语句处理器 */

if (stmt == NULL)

{

    print_error (conn, "Could not initialize statement
handler");

    return;

}

/* 插入并检索记录 */

insert_rows (stmt);

select_rows (stmt);

mysql_stmt_close (stmt);      /* 释放语句处理器 */

}

```

首先，我们打开了一个数据库，并创建了一个测试表。这个表包含4个不同类型的列：INT、FLOAT、CHAR和DATETIME。随后你将看到，这几种不同的数据类型会被区别对待。

在创建测试表之后，先调用了mysql_stmt_init()，用于分配一个预处理语句处理器；接着，插入和检索了多个行；最后释放了处理器。所有的实际工作都发生在insert_rows()和select_rows()函数里，很快便会讲到它们。为进行出错处理，程序还使用了一个名为print_stmt_error()的函数，它与前几个示例程序所用到的print_error()函数相似；不同之处在于，它会调用特定于预处理语句的出错报告函数：

```

static void

```

```

print_stmt_error (MYSQL_STMT *stmt, char *message)

{

fprintf (stderr, "%s\n", message);

if (stmt != NULL)

{

    fprintf (stderr, "Error %u (%s): %s\n",
mysql_stmt_errno (stmt),
mysql_stmt_sqlstate (stmt),
mysql_stmt_error (stmt));

}

}

```

insert_rows() 函数会负责把新行添加到测试表里：

```

static void

insert_rows (MYSQL_STMT *stmt)

{

    char                *stmt_str = "INSERT INTO t (i,f,c,dt)
VALUES(?,?,?,?)";

    MYSQL_BIND          param[4];

    int                 my_int;

```



```

float          my_float;

char          my_str[26]; /* ctime()会返回长度为26 字符串 */

MYSQL_TIME    my_datetime;

unsigned long my_str_length;

time_t        clock;

struct tm     *cur_time;

int           i;

printf ("Inserting records...\n");

if (mysql_stmt_prepare (stmt, stmt_str, strlen (stmt_str)) != 0)

{

    print_stmt_error (stmt, "Could not prepare INSERT
statement");

    return;

}

/*

* 将参数结构置0, 然后初始化所有常量以及在

*每一行都不会发生变化的参数

*/

memset ((void *) param, 0, sizeof (param));

/* 设置INT 参数 */

```

```
param[0].buffer_type = MYSQL_TYPE_LONG;

param[0].buffer = (void *) &my_int;

param[0].is_unsigned = 0;

param[0].is_null = 0;

/* buffer_length和length 不需要设置 */

/* 设置FLOAT 参数 */

param[1].buffer_type = MYSQL_TYPE_FLOAT;

param[1].buffer = (void *) &my_float;

param[1].is_null = 0;

/* is_unsigned、buffer_length和length 都不需要设置 */

/* 设置CHAR 参数 */

param[2].buffer_type = MYSQL_TYPE_STRING;

param[2].buffer = (void *) my_str;

param[2].buffer_length = sizeof (my_str);

param[2].is_null = 0;

/* is_unsigned 不需要设置，而length 则会在随后设置 */

/* 设置DATETIME 参数 */

param[3].buffer_type = MYSQL_TYPE_DATETIME;

param[3].buffer = (void *) &my_datetime;

param[3].is_null = 0;
```

```
/* is_unsigned、buffer_length和length 都不需要设置 */  
if (mysql_stmt_bind_param (stmt, param) != 0)  
{  
    print_stmt_error (stmt, "Could not bind parameters for INSERT");  
    return;  
}  
for (i = 1; i <= 5; i++)  
{  
    printf ("Inserting record %d...\n", i);  
    (void) time (&clock); /* 获得当前时间 */  
    /* 设置那些与每个参数关联在一起的变量 */  
    /* param[0]: 设置my_int的值 */  
    my_int = i;  
    /* param[1]: 设置my_float的值 */  
    my_float = (float) i;  
    /* param[2]: 把my_str 设置成ctime()的当前字符串值 */  
    /* 并且把length 指向表示my_str的长度的那个变量 */  
    (void) strcpy (my_str, ctime (&clock));  
    my_str[24] = '\0'; /* 把尾部的换行符去除掉 */
```

```
my_str_length = strlen (my_str);

param[2].length = &my_str_length;

/* param[3]: 把my_datetime 设置成当前的日期和时间 */

cur_time = localtime (&clock);

my_datetime.year = cur_time->tm_year + 1900;

my_datetime.month = cur_time->tm_mon + 1;

my_datetime.day = cur_time->tm_mday;

my_datetime.hour = cur_time->tm_hour;

my_datetime.minute = cur_time->tm_min;

my_datetime.second = cur_time->tm_sec;

my_datetime.second_part = 0;

my_datetime.neg = 0;

if (mysql_stmt_execute (stmt) != 0)

{

print_stmt_error (stmt, "Could not execute statement");

return;

}

sleep (1); /* 暂停一下（以便更改生效） */

}
```

```
}
```

`insert_rows()` 函数的目的是，往测试表里插入5个包含下列值的行。

一个1~5之间的INT值。

一个1.0~5.0之间的FLOAT值。

一个CHAR值。为生成这些值，需要调用系统函数`ctime()`来获得“当前时间”的字符串形式。`ctime()`返回值的格式如下：

```
Sun Sep 19 16:47:23 CDT 2004
```

一个DATETIME值。这也是“当前时间”的值，但存储在MYSQL_TIME结构里。二进制协议使用MYSQL_TIME结构来传输DATETIME、TIMESTAMP、DATE和TIME值。

`insert_rows()` 函数里做的第一件事是准备一条 INSERT 语句，采用的方式是将其传递给`mysql_stmt_prepare()` 函数。这条语句如下所示：

```
INSERT INTO t (i,f,c,dt) VALUES(?,?,?,?)
```

这条语句包含4个占位符，因此每次执行这条语句时，需要提供4个数据值。占位符通常代表 `VALUES()` 里列出的数据值，或者 `WHERE` 子句里的数据值。但有些地方不能使用占位符，具体如下所示。

作为标识符，如表名或列名。下面这条语句便是非法的：

```
SELECT * FROM ?
```

可以把占位符用在操作符的任何一侧，但不允许同时用在两侧。下面这条语句是合法的：

```
SELECT * FROM student WHERE student_id = ?
```

不过，下面这条语句是非法的：

```
SELECT * FROM student WHERE ? = ?
```

这条限制很有必要，这样才能让服务器确定参数的数据类型。

下一步，设置MYSQL_BIND结构的数组，其中每个元素都对应一个占位符。如insert_rows()里所演示的，设置操作涉及以下两个阶段。

(1) 初始化这些结构里对于所有新插入的行都相同的部分。

(2) 执行行插入循环——它会逐行初始化这些结构里对于各行都不相同的部分。

实际上，也可以在循环体内执行所有的初始化工作，但这种做法比较低效。

在初始化的第一个阶段里，对包含MYSQL_BIND结构的param数组进行清零。此程序使用的是memset()，如果你的系统没有memset()，也可以用bzero()。下面两条语句是等效的：

```
memset ((void *) param, 0, sizeof (param));
```

```
bzero ((void *) param, sizeof (param));
```

清空param数组，会把所有结构成员隐式地设置成零。接下来的代码会把某些成员设置成零，以显式地表明正在发生的事情，但这不是必须的。实际上，在清空结构之后，就必要再把该结构的任何成员赋为零了。

下一步是把正确的信息赋给 MYSQL_BIND 数组里的每一个参数。对于每个参数，需要设置的结构成员则取决于你要传输的值的类型。

buffer_type 成员必须设置；它用于表明参数值的数据类型。附录 G中有一张表，其中完整地列出了每一个允许使用的类型代码，并且还显示了与每个代码相对应的SQL类型和C语言类型。

buffer 成员应该设置成那个保存有数据值的变量的地址。insert_rows()声明了 4个变量来保存行值：my_int、my_float、my_str和my_datetime。每一个param[i].buffer 值会被设置成指向

相应的变量。在插入新行时，我们会把这4个变量设置成表的列值，并用它们来创建新行。

`is_unsigned`成员只适用于整数数据类型。它应该被设置成真（非零）或假（零），以表明参数是否为 `UNSIGNED` 整数类型。我们的表包含一个有符号 `INT` 列，因此`is_unsigned` 会被设置为 0。如果这个列为 `INT UNSIGNED` 类型，那么除了要把`is_unsigned`设置为1以外，还需要把`my_int`变量声明成`unsigned int`类型，而不能是`int`类型。

`is_null` 成员表明是否正在传输一个`NULL` 值。一般情况下，应该把这个成员设置成`my_bool` 变量的地址。然后，在插入任何给定行之前，把这个变量设置成真或假，以表明将要插入的值是否为 `NULL`。如果不会发送 `NULL` 值（如本例一样），那么可以把`is_null`设置成0，并且不需要`my_bool`变量。

对于字符串值或二进制数据（`BLOB`值），还需要设置两个 `MYSQL_BIND`成员。它们分别表明，用于存储字符串的那个缓冲区的长度，以及要被传输的那个值的实际长度。

在许多情况下，它们都是相同的。但是，如果使用了固定长度的缓冲区，并且发送值会因行不一样而有所不同，那么它们的值也会存在差异。`buffer_length` 成员用来给出缓冲区的长度。`length`是一个指针，它应该被设置成某个`unsigned long`变量的地址，该变量包含了传输值的实际长度。

对于数字类型和时态类型，不用设置`buffer_length`和`length`。这些类型的长度都是固定不变的，因而可以根据`buffer_type`的值来确定。例如，`MYSQL_TYPE_LONG`和`MYSQL_TYPE_FLOAT`分别表示值的长度为4个字节和8个字节。

在完成对 `MYSQL_BIND` 数组的初始化之后，便可以把该数组传递到 `mysql_stmt_bind_param()`，将它与预处理语句绑定起来。接着，把各个值赋给`MYSQL_BIND`结构所指向的那些变量，并执行该语句。这个过程会循环执行 5次。循环的每次迭代都将把相应的值赋给语句的各个参数。

对于整数和浮点参数，只需要把各个值赋给关联的`int`和`float`变量即可。

对于字符串参数，需要把字符串格式的当前时间赋给关联的char变量。要获得这个值，可以调用ctime()，并去掉换行字符。

日期时间参数也会被赋值成当前时间，但采用的方式是，把这个时间的各个部分，分别赋给与MYSQL_TIME结构有关的各个成员。

在把各个参数值设置好之后，调用mysql_stmt_execute()便可以执行这条语句。这个函数会把当前值传递给服务器，而服务器将把它们整合进预处理语句，然后执行该语句。

当insert_rows()返回时，测试表已被填充，可以调用select_rows()来检索它们了：

```
static void  
  
select_rows (MYSQL_STMT *stmt)  
  
{  
  
    char                *stmt_str = "SELECT i, f, c, dt FROM t";  
  
    MYSQL_BIND          param[4];  
  
    int                  my_int;  
  
    float                my_float;  
  
    char                 my_str[24];  
  
    unsigned long my_str_length;  
  
    MYSQL_TIME           my_datetime;  
  
    my_bool              is_null[4];  
  
    printf ("Retrieving records...\n");
```



```

if (mysql_stmt_prepare (stmt, stmt_str, strlen (stmt_str)) != 0)

{

    print_stmt_error (stmt, "Could not prepare SELECT
statement");

    return;

}

if (mysql_stmt_field_count (stmt) != 4)

{

    print_stmt_error (stmt, "Unexpected column count from
SELECT");

    return;

}

/*

* 初始化结果列的结构

*/

memset ((void *) param, 0, sizeof (param)); /* 结构清零 */

/* 设置INT 参数 */

param[0].buffer_type = MYSQL_TYPE_LONG;

param[0].buffer = (void *) &my_int;

param[0].is_unsigned = 0;

```

```
param[0].is_null = &is_null[0];

/* buffer_length和length 不需要设置 */

/* 设置FLOAT 参数 */

param[1].buffer_type = MYSQL_TYPE_FLOAT;

param[1].buffer = (void *) &my_float;

param[1].is_null = &is_null[1];

/* is_unsigned、 buffer_length和length 都不需要设置 */

/* 设置CHAR 参数 */

param[2].buffer_type = MYSQL_TYPE_STRING;

param[2].buffer = (void *) my_str;

param[2].buffer_length = sizeof (my_str);

param[2].length = &my_str_length;

param[2].is_null = &is_null[2];

/* is_unsigned 不需要设置 */

/* 设置DATETIME 参数 */

param[3].buffer_type = MYSQL_TYPE_DATETIME;

param[3].buffer = (void *) &my_datetime;

param[3].is_null = &is_null[3];

/* is_unsigned、 buffer_length和length 都不需要设置 */
```

```

if (mysql_stmt_bind_result (stmt, param) != 0)

{

    print_stmt_error (stmt, "Could not bind parameters for
SELECT");

    return;

}

if (mysql_stmt_execute (stmt) != 0)

{

    print_stmt_error (stmt, "Could not execute SELECT");

    return;

}

/*

* 获取结果集，放置到客户端内存里；这一步是可选的，

* 不过允许调用mysql_stmt_num_rows()来确定

* 结果集里的行数

*/

if (mysql_stmt_store_result (stmt) != 0)

{

    print_stmt_error (stmt, "Could not buffer result set");

    return;

```

```

    }

    else

    {

        /* mysql_stmt_store_result() 让行计数变得可用*/
        printf ("Number of rows retrieved: %lu\n",
(unsigned long) mysql_stmt_num_rows (stmt));

    }

    while (mysql_stmt_fetch (stmt) == 0)    /* 获取每一个行*/

    {

        /* 显示各个行的值 */

        printf ("%d    ", my_int);

        printf ("%0.2f    ", my_float);

        printf ("%*.s    ", (int) my_str_length, (int)
my_str_length, my_str);

        printf ("%04d-%02d-%02d %02d:%02d:%02d\n",
my_datetime.year,
my_datetime.month,
my_datetime.day,
my_datetime.hour,
my_datetime.minute,

```

```
my_datetime.second);
```

```
}
```

```
mysql_stmt_free_result(stmt);          /* 释放结果集 */
```

```
}
```

`select_rows()` 函数由 3 个部分组成：准备一条 `SELECT` 语句、执行它，然后检索结果。此时，这条语句并没有包含任何占位符：

```
SELECT i, f, c, dt FROM t
```

这意味着，在执行这条语句之前，无需设置任何 `MYSQL_BIND` 结构。但是我们并未摆脱困境。与 `insert_rows()` 一样，`select_rows()` 的大量工作也是设置一个 `MYSQL_BIND` 结构的数组。不同之处在于，这次是在执行此语句之后，接收来自服务器的各个数据值，而不是在执行语句之前，把要被发送到服务器的数据值设置好。

尽管如此，设置 `MYSQL_BIND` 数组的过程还是与 `insert_rows()` 里的对应代码很相似，具体过程如下所示。

(1) 对数组清零。

(2) 把每个参数的 `buffer_type` 成员设置成相应的类型代码。

(3) 把每个参数的 `buffer` 成员指向这样一个变量：当取回行时，相应的列值会存入其中。

(4) 把整数参数的 `is_unsigned` 成员设置成零。

(5) 对于字符串参数，把 `buffer_length` 设置成应该取回的最大字节数，并且把 `length` 设置成某个 `unsigned long` 变量的地址。在取回行时，这个变量将被设置成实际取回的字节数。

(6) 对于每一个参数，把 `is_null` 成员设置成某个 `my_bool` 变量的地址。在取回行时，这些变量将被设置成具体值，用于表明取回的值是否为 `NULL`。（对于本例，在取回行之后，程序将忽略这些变

量，因为我们已知道测试表不包含任何NULL值。但一般情况下，你应该检查它们。)

在设置好参数之后，需要调用`mysql_stmt_bind_result()`把这个数组与语句绑定起来，然后执行该语句。

至此，我们可以调用`mysql_stmt_fetch()`来取回各个行了。这个程序演示了一个可以先执行的步骤，即调用`mysql_stmt_store_result()`取回整个结果集，并把它缓存在客户端的内存里。这么做的好处是，可以调用`mysql_stmt_num_rows()`查出结果集里有多少个行。而坏处就是，需要占用更多的客户端内存。

用于取回行的那个循环会反复调用`mysql_stmt_fetch()`，一直到它返回一个非零值为止。在每次取回之后，与参数结构相关联的变量将包含当前行的各个列值。

在把所有行全部取回之后，便可以调用`mysql_stmt_free_result()`来释放与结果集相关联的所有内存了。

此时，`select_rows()`会返回到调用函数，从而调用`mysql_stmt_close()`函数来释放预处理语句处理器。

前面对预处理语句接口和某些关键函数进行了概要介绍。客户端开发库还包含其他几个相关函数，更多相关信息请参考附录G。

7.9 使用预处理CALL支持

预处理语句支持在MySQL 5.5里得到了加强和改进，具体表现在：可以使用预处理 CALL语句来调用存储过程；可以访问OUT和INOUT类型过程参数的返回值。此前，预处理CALL语句不能产生多个结果集，并且调用者不能访问返回的参数值。

出于演示的目的，假设有下面这样一个存储过程：

```
CREATE PROCEDURE grade_event_stats  
  
(IN p_event_id INT, OUT p_min INT, OUT p_max INT)
```

```

BEGIN

-- display scores for event

SELECT student_id, score

FROM score

WHERE event_id = p_event_id

ORDER BY student_id;

-- store min/max event scores in OUT parameters

SELECT MIN(score), MAX(score)

FROM score

WHERE event_id = p_event_id

INTO p_min, p_max;

END;

```

此存储过程会接收一个IN参数和两个OUT参数。如果把某个考试事件ID作为其参数，那么这个存储过程会显示该次考试的分数，并在那两个OUT参数里返回考试的最低分数和最高分数。如果我们从 `mysql` 客户端调用 `grade_event_stats()`，那么这条语句看起来会像下面这个样子：

```

mysql> SET @p_min = NULL, @p_max = NULL;

mysql> CALL grade_event_stats(4, @p_min, @p_max);

```

student_id	score
2	7
3	17
4	16
5	20

...

```
mysql> SELECT @p_min, @p_max;
```

@p_min	@p_max
7	20

下面的讨论将展示如何使用C 语言 API 来做同样的事情。此程序的源文件是prepared_call.c，在sampdb发行版的capi目录里。为创建此存储程序，可以使用相同目录下的prepared_call_setup.sql文件：

```
% mysql sampdb < prepared_call_setup.sql
```

预处理CALL支持需要MySQL 5.5.3或更高的版本。在连接服务器之后，为验证它具有此项功能，可以使用prepared_call来检查服务器的版本：

```
if (mysql_get_server_version (conn) < 50503)
{
    print_error (NULL, "Prepared CALL requires MySQL 5.5.3 or higher");
}
```



```

mysql_close (conn);

exit (1);

}

```

此程序接下来会初始化一个预处理语句处理器，并使用它来执行预处理 CALL 语句。如果操作成功，它还会处理存储过程的结果：

```

stmt = mysql_stmt_init (conn);

if (!stmt)

    print_error (NULL, "Could not initialize statement handler");

else

{

    if (exec_prepared_call (stmt) == 0)

        process_call_result (conn, stmt);

    mysql_stmt_close (stmt);

}

```

预处理CALL语句与其他预处理语句很像。请把语句串与“?”占位符代表的数值一起发送给 `mysql_stmt_prepare()`。对于 CALL 而言，这些占位符便代表了那些要传递给存储过程的参数值。参数设置肯定与存储过程所接收的参数个数和类型特定相关。为了简单一点，假设 `grade_event_stats()` 函数的所有参数都是整数。

```

static int exec_prepared_call (MYSQL_STMT *stmt)

{

    MYSQL_BIND params[3];    /* 参数缓冲区 */

```

```

int          int_data[3]; /* 参数值 */

int          i;

/* 准备CALL 语句 */

if (mysql_stmt_prepare (stmt, "CALL grade_event_stats(?, ?, ?)", 31))
{
print_stmt_error (stmt, "Cannot prepare statement");
return (1);
}

/* 初始化参数结构，并绑定到语句 */

memset (params, 0, sizeof (params));

for (i = 0; i < 3; ++i)
{

params[i].buffer_type = MYSQL_TYPE_LONG;

params[i].buffer = (char *) &int_data[i];

params[i].length = 0;

params[i].is_null = 0;

}

if (mysql_stmt_bind_param (stmt, params))
{

```

```

print_stmt_error (stmt, "Cannot bind parameters");

return (1);

}

/* 给参数值赋值，并执行语句 */

int_data[0]= 4; /* p_event_id */

int_data[1]= 0; /* p_min (OUT 参数；存储过程会忽略初始值) */

int_data[2]= 0; /* p_max (OUT 参数；存储过程会忽略初始值) */

if (mysql_stmt_execute (stmt))

{

print_stmt_error (stmt, "Cannot execute statement");

return (1);

}

return (0);

}

```

在执行CALL语句之后，接下来处理其结果，处理过程可分为以下3个部分。

(1) 在此存储过程中执行的语句，每一个都会产生一个结果集。这类语句包括 SELECT、SHOW等。

(2) 如果这个存储过程有OUT或INOUT参数，那么还会有一个附加的单行结果集——其中包含最终的参数值，并且排列顺序与它们出现在过程定义中的顺序一致。

(3) 最终状态包。其中并没有任何结果集与它关联在一起，因此你可以把它与语句或参数结果集区别开来，因为其最终列的个数为0。

必须对所有存在的CALL 结果进行处理。语句和参数结果集是可选的，具体取决于存储过程的编写方法。无论前面是否有结果集，存储过程总是会返回那个最终状态包。

`process_call_result()` 函数里有一个通用的循环结构，用于检索存储过程的结果。这个存储过程是否会产生某些声明或参数结果集，这段代码对此并未做任何假设。那个检索循环的逻辑如下所示。

(1) 通过获得下一个结果的列数，可以确定它是否包含有结果集，或者是否为最终状态包。如果列数为0，则意味着它是状态包，不需要进一步进行处理。

(2) 如果列数大于 0，则表示有一个拥有很多个列的结果集。这个结果集可以由某个语句产生，也可能包含存储过程返回的那些参数值。这个连接处理器有一个包含标志的状态成员，可用于区分这两种情况。（`process_call_result()` 只是把这个标志当作信息来用，通过它来宣称每个结果集是如何产生的。）在这两种情况下，都需要获取结果集。

(3) 通过调用`mysql_stmt_next_result()`，可以检查是否还有更多的结果。如果还有更多的结果，则此函数会返回 0。如果没有，则返回-1。如果有错误发生，则返回一个大于 0的值。因此，如果结果为0，则返回第1步，继续获取下一个结果。

```
static void process_call_result (MYSQL *conn, MYSQL_STMT *stmt)
```

```
{
```

```
int status;
```

```
int num_cols;
```

```
/*
```

```
* 检查每一个结果的列数。如果没有，
```

* 则结果为最终状态包，不需要做任何事情。

* 否则，获取结果集。

*/

do {

if ((num_cols = mysql_stmt_field_count (stmt)) > 0)

{

/* 声明结果集是包含参数还是数据集 */

if (conn->server_status & SERVER_PS_OUT_PARAMS)

printf ("OUT/INOUT parameter values:\n");

else

printf ("Statement result set values:\n");

if (process_result_set (stmt, num_cols))

break; /* 有错误发生 */

}

/* 如果状态为-1，则表示完成；若为0，则表示有还更多结果；
若>0，则表示出错 */

status = mysql_stmt_next_result (stmt);

if (status > 0)

print_stmt_error (stmt, "Error checking for next result");

} while (status == 0);

```
}
```

为获得语句和参数结果集，这个循环还需要调用 `process_result_set()`。不过，这里并没把它显示出来。请检查 `prepared_call.c` 的源代码，看看它是如何做的。为简单一点，它假定所有检索回的值都是整数。你可以把它修改成一个更通用的程序，让它可以处理其他数据类型。

下面是运行 `prepared_call` 程序之后得到的输出结果：

```
% ./prepared_call sampdb
```

```
Statement result set values:
```

```
val[1] = 2; val[2] = 7;
```

```
val[1] = 3; val[2] = 17;
```

```
val[1] = 4; val[2] = 16;
```

```
val[1] = 5; val[2] = 20;
```

```
...
```

```
OUT/INOUT parameter values:
```

```
val[1] = 7; val[2] = 20;
```

各参数在存储过程定义里的位置与在参数结果集里的位置存在差异，请务必小心对待。观察一下前面的输出，可以看到：那些返回参数值的位置分别为 1 和 2，尽管它们实际对应的是存储过程 `grade_event_stats()` 里的参数 2 和 3。为什么会这样呢？因为那个返回参数值的最终结果集只包含了 OUT 和 INOUT 参数，而没有包含 IN 参数。在存储过程里对 IN 参数的任何更改，对于调用者来说都是不可见的。因此，它们在调用前后都拥有同样的值，并不需要在存储过程的结果里进行返回。

第8章

用Perl DBI编写MySQL程序

本章将对如何使用MySQL的Perl DBI接口进行介绍。这里并不会讨论DBI的工作原理或体系结构。关于DBI的这几方面内容（尤其是DBI与C语言API和PHP语言API的对比），请参考第6章。

本章讲到的各个示例将会用到示例数据库sampdb，并且会用到那些在第1章为成绩考评项目和美史联盟创建的表。

尽管大部分材料都可应用于更早一点的版本，但这里还是要假定使用的最低 DBI 版本为1.50。DBI 1.50对Perl的版本的最低要求为5.6.0（首选5.6.1）。自DBI 1.611起，要求的最低Perl版本为5.8.1。除了安装MySQL的C语言客户端开发库和头文件以外，还必须安装Perl语言的DBD:mysql模块。如果想利用本章介绍的内容编写基于Web的DBI脚本，则还需要安装CGI.pm模块。在本章中，CGI.pm主要是与Web服务器Apache配合一起使用。如果需要这些软件包，请参考附录A，其中，还有对如何获得sampdb发行版（它包含本章开发的各个示例脚本）的指导说明。在该发行版的perlapi目录下能找到这些脚本。

基本上，本章只会对那些需要讨论的Perl DBI方法和变量进行介绍。想了解更多综合性的方法和变量列表，请参考附录H。可把该附录当作参考手册，以便在需要用到DBI的某个部分时，对其进行查询。通过运行下面几条命令，或者访问 <http://dbi.perl.org> 即可获得帮助文档：

```
% perldoc DBI
```

```
% perldoc DBI::FAQ
```

```
% perldoc DBD::mysql
```

在数据库驱动程序（DataBaseDriver，简称为 DBD）层，针对 MySQL 的驱动程序都是在 MySQL 的 C 语言客户端开发库的基础上构建的，因此这种驱动程序也拥有这个开发库的一些特点。关于该开发库的更多信息请参考第 7 章。

8.1 Perl 脚本的特点

Perl 脚本是文本文件，因此你可以用任何文本编辑器来创建它们。本章的所有 Perl 脚本都遵循了 Unix 的脚本约定：在第一行用 “#!” 指定用于执行脚本的那个程序的路径名。本书使用了这样一行：

```
#!/usr/bin/perl
```

在 Unix 系统里，如果 Perl 的路径名在你的系统上有所不同，如 /opt/bin/perl，那么需要修改这个 #! 行。否则，Perl 脚本在你的系统上将无法正确执行。

在任何系统上，都可以像下面这样调用 Perl 脚本 myscript.pl：

```
% perl myscript.pl
```

在执行脚本时，也可以不用 perl 字样来显式地说明执行的是 perl 程序。为了不用显示说明，在 Unix 里，可以用 chmod 命令来修改这个文件的模式，使脚本变得可执行：

```
% chmod +x myscript.pl
```

然后，在运行此脚本时，只需输入其名字即可：

```
% ./myscript.pl
```

本章的示例都将采用这种脚本调用风格。如果脚本文件位于当前目录（即 “.”），并且 shell 程序不搜索当前目录，那么请在脚本名的前面加上 “./”。否则，可以省略命令名里的 “./”：

```
% myscript.pl
```

在 Windows 里，可以在程序 Perl 和以 “.pl” 结尾的文件名之间建立起文件名关联。例如，你安装的是 ActiveState Perl，那么其安装程序会允许你建立一个文件名关联，这样以 “.pl” 结尾的文件就可以按 Perl 程序来运行。此时，只需在命令行上给出 Perl 脚本的名字，便能顺利执行它们：

C:\> myscript.pl

8.2 Perl DBI概述

本节将介绍DBI的背景知识，无论是编写自己的脚本，还是理解别人所写的脚本，都要用到它DBI。如果你已对DBI有所了解，则可以直接跳到8.3节。

8.2.1 DBI数据类型

Perl DBI的API与第7章介绍的C客户端开发库有很多相似的地方。在使用C客户端开发库时，需要调用各种函数，并通过指向结构或数组的指针来访问与MySQL有关的数据。在使用DBI API时，也需要调用函数，并使用指向各种结构的指针。不同之处在于：函数被称作“方法”（method），指针被称作“引用”（reference），指针变量被称作“句柄”（handle），而句柄所指向的结构被称作“对象”（object）。

DBI使用了多种句柄。它们在DBI文档里大都有约定俗成的称呼，如表8-1所示。随着后面的深入介绍，这些句柄的用法会变得越加清晰。此外，有些常用的非句柄变量也有约定俗成的称呼（见表8-2）。本章实际上不会全部用到它们，但是当你阅读别人写的DBI脚本时，可能会用到。

表8-1 Perl DBI句柄变量的惯用名

名字	含义
<code>\$dbh</code>	指向数据库对象的句柄
<code>\$sth</code>	指向语句（查询）对象的句柄
<code>\$fh</code>	指向打开文件的句柄
<code>\$h</code>	“泛型”句柄；具体含义视上下文而定

表8-2 Perl DBI非句柄变量的惯用名

名字	含义
\$rc	会返回true或false值的操作所返回的代码
\$rv	会返回整数的操作所返回的值
\$rows	会返回行计数的操作所返回的值
\$ary	一个数组（列表），代表着查询命令所返回的一个行的值

8.2.2 一个简单的DBI脚本

我们先从一个简单的脚本 `dump_members.pl` 开始。这个脚本演示了DBI程序设计中的几个标准概念，如与MySQL服务器的连接和断开、发出SQL语句、检索数据等。它将以制表符分隔格式输出一份美史联盟的成员资格名单。输出格式本身没什么意思。眼下最重要的是，弄清如何使用DBI脚本，而不是要产生漂亮的输出。`dump_members.pl`的内容看起来像下面这样。

```
#!/usr/bin/perl

# dump_members.pl - 导出美史联盟的成员资格名单

use strict;

use warnings;

use DBI;

# 数据源名字、用户名、密码和连接属性

my $dsn = "DBI:mysql:sampdb:localhost";

my $user_name = "sampadm";

my $password = "secret";

my %conn_attrs = (RaiseError => 1, PrintError => 0, AutoCommit => 1);

# 连接数据库

my $dbh = DBI->connect ($dsn, $user_name, $password, \%conn_attrs);
```

```

# 执行查询

my $sth = $dbh->prepare ("SELECT last_name, first_name, suffix, email,"
. " street, city, state, zip, phone FROM member ORDER BY last_name");

$sth->execute ();

# 读取并显示查询结果

while (my @ary = $sth->fetchrow_array ())

{

print join ("\t", @ary), "\n";

}

$sth->finish ();

$dbh->disconnect ();

```

如果你想自己试试这个脚本，那么可以使用sampdb 发行版里包含的副本，也可以用文本编辑器来创建它。如果使用的是字处理器程序，那么一定要记得把脚本保存为普通文本格式。请不要把它保存为文字处理器的默认格式。另外，你可能至少还需要修改某些连接参数，如主机名、数据库名、用户名或密码。（对于本章其他用到连接参数的脚本，也需要做适当的修改。）在8.2.9节，我们将会了解到如何从选项文件里获得这些参数，而不是把它们直接放置在脚本里。

下面来逐行分析这个脚本。第一行是一个标准的Perl路径指示符：

```
#!/usr/bin/perl
```

这一行是本章所讨论的每个脚本的一部分。因此，在后面将不再赘述。

在脚本里至少包含一个简单的用途描述，是一种不错的做法。因此，接下来的一行是一个注释，它能帮助那些查看此脚本的人了解其用途：

```
# dump_members.pl - 导出美史联盟的成员资格名单
```

“#”字符后面的那行文本会被认为是注释。在脚本里使用注释来说明脚本是如何工作的，这是一种很好的编程习惯。

接下来是几条use语句：

```
use strict;
```

```
use warnings;
```

```
use DBI;
```

`use strict` 用于告诉 Perl 来要求你，在使用变量之前必须先声明。在编写脚本时，可以不写这条语句，但它有助于捕获错误，所以建议你总是包含它。例如，如果你声明了变量 `$my_var`，但在后面的代码里错误地把它写成了 `$mv_var`，那么在严格模式下运行此脚本，就会显示下面这样的信息：

```
Global symbol "$mv_var" requires explicit package name at line n
```

当你看到这条信息时，你便会想：“怎么回事？我从未用过名为 `$mv_var` 的变量呀！”然后，你会检查脚本程序的第 `n` 行，发现自己把 `$my_var` 错误地写成了 `$mv_var`，于是把它改正过来。如果不使用严格模式，那么 Perl 不会把 `$mv_var` 当作是个错误，它会创建一个名为 `$mv_var` 的新变量，并把它赋为 `undef`（即“未定义”），然后正常使用它，不报错。而你只能在那里疑惑：为什么脚本不工作呢？

`use warnings` 用于告诉 Perl，在发现你使用了有问题的语言结构，或者执行了有问题的操作（如输出未初始化的变量）时，发出警告。这会很有用，因为它会警告你在编写代码时应该更认真。

`use DBI` 用于告诉 Perl 解释器，记得加载 DBI 模块。如果缺少这一行，那么只要一执行脚本中与 DBI 相关的部分，就会出现错误。你不用指明要使用哪一个 DBD 层的驱动程序模块，因为在你连接数据库时，DBI 会帮你激活那个正确的驱动程序模块。

因为我们是在严格模式下执行操作的，所以必须使用 `my` 关键字声明此脚本要用到的变量。这个脚本仿佛在说：“我在明确声明，这些就是我的变量。”在脚本接下来的部分，会设置那些用于指定连接参数的变量，然后用它们来连接数据库：

```
# 数据源名称、用户名、密码和连接属性
```

```
my $dsn = "DBI:mysql:sampdb:localhost";
```

```
my $user_name = "sampadm";

my $password = "secret";

my %conn_attrs = (RaiseError => 1, PrintError => 0, AutoCommit => 1);

# 连接数据库

my $dbh = DBI->connect ($dsn, $user_name, $password, \%conn_attrs);
```

因为connect()调用是 DBI类的一个方法，所以需要使用DBI->connect()的形式来进行调用。（你可以不用关心它到底是什么意思。其实，它只是一种会让你感到头痛不已的面向对象术语。如果你真的想知道，那么它表示connect()是一个“属于”DBI的函数。）connect()可以接受多个参数。

数据源，也叫数据源名称（data source name），简称 DSN。DSN 指明了要使用哪一个DBD模块，以及可能的其他参数。

MySQL账户的用户名和密码。

一个可选参数，用于指明一些其他的连接属性。如果有这个参数，则此参数应该是一个散列引用，其中指定了连接属性的名字和值。

可以根据你使用的DBI模型的具体需求来确定数据源格式。对于MySQL驱动程序，允许的DNS格式有下列两种：

```
DBI:mysql:db_name
```

```
DBI:mysql:db_name:host_name
```

DBI的大小写无关紧要，但mysql必须是小写。db_name代表要使用的数据库名，而host_name表示的是运行MySQL服务器的主机的位置。如果省略主机名，那么默认为localhost。（更多与数据源格式有关的信息请参考8.2.9节。）

我们指定连接属性散列作为%conn_attrs的值，会启用RaiseError 属性，而禁用PrintError 属性。这些设置会使DBI检查与数据库有关的错误。如果有检测到错误，则退出执行，并给出一条出错信息。（这也正是在dump_members.pl脚本里看不到任何出错检查代码的原因，检查全由DBI代劳了。）8.2.3节还会介绍另外的出错响应方法。

这个属性散列还会启用AutoCommit属性。目前，根本不必启用它（它默认就是启用的），但是这个设置可以明确地表明，此脚本在事务处理时将启用自动提交模式。这个脚本并没有包含任何显式的事务操作，但是将来DBI有可能会要求脚本显式地指定AutoCommit属性。现在在脚本里这样做，可以做到未雨绸缪。

如果要设定连接属性，那么也可以在调用connect()时直接提供散列引用：

```
my $dbh = DBI->connect ($dsn, $user_name, $password,  
{ RaiseError => 1, PrintError => 0, AutoCommit => 1 });
```

上面两种方法在实际操作上没有任何差别，不同的人可以有不同的选择，适合自己就好。

如果connect()调用成功，它会返回一个数据库句柄，我们再把这个句柄赋给变量\$dbh。在默认情况下，如果调用失败，connect()会返回undef。但是，因为这个脚本启用了RaiseError属性，所以如果在connect()的调用过程中真的出现了错误，那么DBI会显示出错信息，然后退出。（其他DBI方法也是如此。我将描述它们在执行出错时会返回什么内容，但是如果启用了RaiseError属性，那么它们根本就不会返回。）

在连接到数据库之后，dump_members.pl将执行一条SELECT语句，检索成员资格名单，然后执行一个循环来处理返回的每一个行。这些行构成了结果集。想执行SELECT，需要先预处理语句，然后再执行它：

```
# 执行查询
```

```
my $sth = $dbh->prepare ("SELECT last_name, first_name, suffix, email,"  
."  
." street, city, state, zip, phone FROM member ORDER BY last_name");  
  
$sth->execute ();
```

在调用prepare()时，使用到了数据库句柄。它会把SQL语句传递给驱动程序，以便在执行之前进行预处理。在这个时候，有些驱动程序实际上会处理这条语句。而另外一些则会记住这条语句，一直到调用execute()方法来执行它为止。prepare()的返回值是一个语句句柄，在这里它被赋给了\$sth。所有与这条语句有关的进一步处理工作，都将通过这个语句句柄来完成。

请注意，这个语句字符串的结尾并没有带分号。在长时间用mysql程序进行交互开发之后，你可能已习惯于在SQL语句的末尾加上一个分号“;”来表示语句结束。不过，在使用DBI时，最好能改掉这个习惯，因为分号经常会导致语句因语法错误而失败。把\g或\G结束符添加到语句串里，也会有同样的问题。这些语句结束符只适合于mysql程序，当在DBI脚本中执行语句时，请不要使用它们。语句串的结尾会隐式地终止语句，并不需要显式地加上结束符。

如果在调用方法时，不向它传递任何参数，那么可以把它后面的括号省略掉。下面这两个调用是等价的：

```
$sth->execute ();
```

```
$sth->execute;
```

我喜欢留着这些括号，因为它们可以帮我区分脚本中的方法调用和变量引用。你也可以选择不同的方式。

在调用execute()之后，便可以开始处理成员资格名单的各个行了。在dump_members.pl脚本里，用于获取行的循环语句，只是简单地输出了每一行的内容，同时用制表符把各个值分隔开来：

```
# 读取并显示查询结果

while (my @ary = $sth->fetchrow_array ())

{

    print join ("\t", @ary), "\n";

}

$sth->finish ();
```

当有更多行时，fetchrow_array()函数会返回值一个包含当前行的各个列值的数组，否则，会返回一个空数组。因此，这个循环语句将依次取回SELECT 语句所返回的每一个行，并在输出每一个行时，在各列值之间加上制表符。

在Perl脚本里，数据库里的NULL值会被返回为undef值，但是它们作为空字符串输出的，而不是输出为单词“NULL”。在你运行脚本时，undef 值

还会产生另外一个效果，即它们会导致Perl解释器给出如下所示的警告信息：

```
Use of uninitialized value in join at dump_members.pl line n.
```

这些警告信息是因包含了`use warnings`语句而产生的。如果去掉这条语句，再运行脚本，那么这些警告信息便会消失。不过，警告模式有助于发现问题（如输出了一个未初始化的变量！）。比较好的消除警告的办法是，检测并处理`undef`值。8.2.5节将会介绍一些具体的处理技术。

在`print`语句里，制表符和换行符（分别表示为`\t`和`\n`序列）都加上了双引号。在Perl语言里，转义序列只有出现在双引号里时，才能被解释；在单引号里，无法解释。如果已使用了单引号，那么字符`\t`和`\n`都将会原样输出。

在获取行的循环结束之后，调用`finish()`是想表明：不再需要这个语句句柄，可以释放所有分配给它的临时资源。在此脚本里，调用`finish()`只是为了演示而已。其实这里用不着调用它，因为当“获取行”的调用遇到结果集末尾时，它会自动执行这些操作。如果你只是取回部分的结果集，并不会到达其末尾（如只想获取第一个行），那么`finish()`便能发挥作用。在以后的示例里，如果没必要，将不再使用`finish()`。

在输出成员资格名单之后，我们的事就做完了，可以断开与服务器的连接，并退出了：

```
$dbh->disconnect ();
```

`dump_members.pl`脚本演示了几个大部分DBI程序都会涉及的概念。从现在开始，你不必再了解更多的内容，便可以开始编写自己的DBI脚本。例如，要输出其他表的内容，那么只需要修改那个传递给`prepare()`方法的`SELECT`语句即可。事实上，如果你急于想看采用了此项技术的应用程序，那么可以直接跳到8.3节，该节将讨论，如何为美史联盟年会议程和打印名录生成成员名单。不过，DBI还提供了许多其他有用的功能。下一节将对其中的部分内容进行更详细的讲解，以便你了解如何在Perl脚本里完成比运行`SELECT`语句更复杂的操作。

8.2.3 出错处理

在调用`connect()`方法时，`dump_members.pl`启用了`RaiseError`出错处理属性。如此一来，这些出错，便会自动终止脚本，同时显示出错信息；而不是仅仅返回出错代码。也有可能还有其他的出错处理办法。例如，你完全可以不用DBI代劳，自己来处理这些错误。

为了更好地控制DBI的出错处理行为，现在来仔细研究一下传递给connect()的最后那个参数，它是一个连接属性散列：

```
# 数据源名字、用户名、密码和连接属性

my $dsn = "DBI:mysql:sampdb:localhost";

my $user_name = "sampadm";

my $password = "secret";

my %conn_attrs = (RaiseError => 1, PrintError => 0, AutoCommit => 1);

# 连接数据库

my $dbh = DBI->connect ($dsn, $user_name, $password, \%conn_attrs);
```

与出错处理相关的两个属性是：RaiseError和PrintError。

如果启用了RaiseError属性（即将其设置为非零值），那么在某个DBI方法出错时，DBI会发出异常消息。默认情况下，它会调用die()来显示出错信息，然后退出脚本。

如果启用了PrintError属性，那么在某个DBI方法出错时，DBI会调用warn()方法来显示出错信息，但脚本会继续往下执行。

在默认情况下，RaiseError 是禁用的，而 PrintError 是启用的。在这种情况下，如果connect()调用失败，那么DBI会显示出错信息，但会继续往下执行。因此，即使省略了connect()的第4个参数，你也可以使用默认的出错处理行为：

```
my $dbh = DBI->connect ($dsn, $user_name, $password)

or exit (1);
```

在这种情况下，如果出现了错误，那么connect()将返回undef，以表明执行失败，而这会触发对exit()的调用。你不用输出出错信息，因为DBI已经输出了。

如果你想显式地把出错处理属性指定为默认值，那么可以把传递给connect()的设置写成下面这个样子：

```
my %conn_attrs = (RaiseError => 0, PrintError => 1, AutoCommit => 1);

my $dbh = DBI->connect ($dsn, $user_name, $password, \%conn_attrs)

or exit (1);
```

虽说这样会让你多写很多内容，但是它能让别人更清楚地了解具体的出错处理行为。

如果你想检查错误，并输出自己的信息，那么需要同时禁用RaiseError和PrintError属性：

```
my %conn_attrs = (RaiseError => 0, PrintError => 0, AutoCommit => 1);

my $dbh = DBI->connect ($dsn, $user_name, $password, \%conn_attrs)

or die "Could not connect to server: $DBI::err ($DBI::errstr)\n";
```

上面这段代码里所使用的变量\$DBI::err和\$DBI::errstr，对于构造出错信息非常有用。它们包含 MySQL 出错代码和出错字符串，这一点与 C 语言的API 函数 mysql_errno()和mysql_error()很像。如果没有发生错误，那么\$DBI::err将为0或undef，而\$DBI::errstr将是空字符串或undef。（换句话说，这两个变量都会为假。）

如果想让DBI替你处理错误，则需要启用RaiseError，并禁用PrintError：

```
my %conn_attrs = (RaiseError => 1, PrintError => 0, AutoCommit => 1);

my $dbh = DBI->connect ($dsn, $user_name, $password, \%conn_attrs);
```

这是一个最省事的做法，本章的大部分脚本都是这样写的。之所以在启用RaiseError的同时禁用PrintError，是为了避免两次输出相同的出错信息。（在某些特定的情形下，如果同时启用这两个属性，那么这个DBI处理器可能会被调用两次。）

如果想让脚本在退出时做些自己定义的清理工作，那么启用RaiseError的做法则不可取，尽管在这种情况下，可能可以通过重新定义\$SIG{__DIE__}信号处理器来达到目的。另一个避免启用RaiseError的理由是，DBI输出的信息过于专业：

```
disconnect(DBI::db=HASH(0x197aae4)) invalidates 1 active statement.Either
```

destroy statement handles or call finish on them before disconnecting.

这条信息对程序员非常有用，但你可能想避免把它展示给普通用户。在这种情况下，最好是自己检查错误，并为目标用户显示一些更具意义的信息。在当前的这个示例里，你可能还会考虑重新定义\$SIG{__DIE__}处理器。这样做的好处是，你既可以启用RaiseError来简化出错处理工作，也可以把DBI提供的默认出错信息修改成你自己的内容。如果要重新定义__DIE__处理器，则需要在执行任何DBI调用之前，做好类似下面这样的工作：

```
$SIG{__DIE__} = sub { die "Sorry, an error occurred\n"; };
```

你也可以像平常那样先定义一个子例程，并把信号处理器设置成对这个子例程的引用：

```
sub die_handler  
{  
    die "Sorry, an error occurred\n";  
}  
  
$SIG{__DIE__} = \&die_handler;
```

下面这个dump_members2.pl脚本，演示了如何编写一个用于检查错误并输出其信息的脚本。dump_members2.pl处理的语句与dump_members.pl相同，只是显式地禁用了RaiseError和PrintError属性，并且会测试每个DBI调用的结果。一旦出现错误，脚本就会调用bail_out()子例程来输出信息以及\$DBI::err和\$DBI::errstr的内容，然后退出：

```
#!/usr/bin/perl  
  
# dump_members2.pl - 导出美史联盟的成员资格名单  
  
use strict;  
  
use warnings;  
  
use DBI;  
  
# 数据源名字、用户名、密码和连接属性
```

```

my $dsn = "DBI:mysql:sampdb:localhost";

my $user_name = "sampadm";

my $password = "secret";

my %conn_attrs = (RaiseError => 0, PrintError => 0, AutoCommit => 1);

# 连接数据库

my $dbh = DBI->connect ($dsn, $user_name, $password, \%conn_attrs)
or bail_out ("Cannot connect to database");

# 执行查询

my $sth = $dbh->prepare ("SELECT last_name, first_name, suffix, email,
    ." street, city, state, zip, phone FROM member ORDER BY last_name")
or bail_out ("Cannot prepare query");

$sth->execute ()

or bail_out ("Cannot execute query");

# 读取并显示查询结果

while (my @ary = $sth->fetchrow_array ())

{

    print join ("\t", @ary), "\n";

}

!$DBI::err

or bail_out ("Error during retrieval");

$dbh->disconnect ()

```

```

or bail_out ("Cannot disconnect from database");

# bail_out()子例程 - 输出错误代码和字符串，其后退出

sub bail_out

{

my $message = shift;

die "$message\nError $DBI::err ($DBI::errstr)\n";

}

```

`bail_out()` 与我们在第7章编写C语言程序时使用的那个`print_error()`函数很像，只是 `bail_out()` 执行的操作是退出，而非返回到调用者。`bail_out()` 给我们带来了两个好处：首先，不必在每次想要输出错误信息时都显示`$DBI::err`和`$DBI::errstr`的值；其次，若把出错信息的输出功能封装在子例程里，那么只需修改这个子例程，便能统一地改变整个脚本的出错信息输出格式。

`dump_members2.pl`脚本在行获取循环语句的后面有一个测试。因为这个脚本在`fetchrow_array()`出错时不会自动退出，所以比较谨慎的做法是确定这个循环终止的原因是什么：到底是因为到达了结果集的末尾（正常结束），还是因为在执行过程中发生了错误（非正常结束）。当然，这两种情况都会导致循环结束，但是如果有错误发生，那么脚本的输出结果会被截断。如果不进行出错检查，那么运行这个脚本的人将根本无法知道是否已出现了问题！如果你打算自行编写代码来检查错误，那么千万要记得测试你的那个数据取回循环的结果。

8.2.4 处理那些修改行的语句

与返回行的语句（如`SELECT`、`DESCRIBE`和`SHOW`）相比，修改行的语句（如`DELETE`、`INSERT`和`UPDATE`）更易于处理。为处理非`SELECT`语句，需要使用数据库句柄把它传递给`do()`方法。`do()`方法会在一个步骤里预处理并执行这条语句。例如，要为Marcia Brown创建一个新的成员条目，其成员资格的到期日期是2012年6月3日，可以使用下面的脚本：

```

$rows = $dbh->do ("INSERT INTO member (last_name,first_name,expiration)"
. " VALUES (' Brown', ' Marcia', ' 2012-06-03' )");

```

`do()` 方法返回的是受影响的行数。若执行出错，则返回`undef`；若因某种原因而无法确定受影响的行数，则返回`-1`。出错的原因各不相同。（如语句本身有错误或者你没有访问表的权限。）对于非 `undef` 返回值，应该特别注意无受影响行的情况。当出现这种情况时，`do()` 并不会返回数字`0`，而是会返回字符串`"0E0"`（即Perl语言对`0`的科学计数法表示形式）。在数字上下文中，`"0E0"`会被计算为`0`；但在条件表达式里，它会被计算为真值。这样一来，我们便可以轻易地把它与 `undef` 区分开来。如果 `do()` 返回 `0`，则很难区分“出错”（`undef`）和“无受影响行”这两种情况。也可以使用下面两个测试当中的任何一种来检查是否出现了错误：

```
if (!defined ($rows))

{

    print "An error occurred\n";

}

if (!$rows)

{

    print "An error occurred\n";

}
```

在数字上下文中，`"0E0"`会被计算为`0`，因此，下面这段代码能够根据 `$rows` 的任何非`undef`值，正确地输出行数：

```
if (!$rows)

{

    print "An error occurred\n";

}

else

{


```

```

$rows += 0; # force conversion to number if value is "0E0"

print "Number of rows affected: $rows\n";

}

```

在输出\$rows时，也可以使用printf()函数的%d格式说明符，强制把它隐式转换为一个数字：

```

if (!$rows)

{

    print "An error occurred\n";

}

else

{

    printf "Number of rows affected: %d\n", $rows;

}

```

do()方法等价于依次使用prepare()和execute()。也就是说，对于前面那条INSERT语句，除了通过调用do()方法来执行它以外，还可以使用下面的方式来执行：

```

$sth = $dbh->prepare ("INSERT INTO member
(last_name,first_name,expiration)"

. " VALUES (' Brown', ' Marcia', ' 2012-06-03' )");

$rows = $sth->execute ();

```

8.2.5 处理那些返回结果集的语句

针对SELECT语句（或者像SELECT那样能返回行的其他语句，如DESCRIBE、EXPLAIN和SHOW）执行行获取循环，你可以有许多选择。本节提供了许多与之相关的信息。此外，本节还会将讨论：如何获得结果集里的行

数；在不需要使用循环时，如何处理结果集；以及如何一次性检索整个结果集。

8.2.5.1 编写行获取循环

dump_members.pl脚本使用了一系列DBI方法来检索数据：prepare()让驱动程序预处理语句，execute()方法开始执行语句，fetchrow_array()用于获取结果集里的每一个行。

在对有结果集的任何语句进行处理时，prepare()和execute()都是相当标准的组成部分。但是，对于获取行来说，fetchrow_array()实际只是以下几种可用方法（见表8-3）当中的一种。

表8-3 DBI的行获取方法

方法名	返回值
fetchrow_array()	行值数组
fetchrow_arrayref()	对行值数组的引用
fetch()	等同于fetchrow_arrayref()
fetchrow_hashref()	对行值散列的引用，列名为键

下面几个示例演示了如何使用各个行获取方法。这些示例将循环遍历结果集里的每一个行，并用逗号分隔方式输出各个列的值。有可能的情况下，还可以把这些代码编写得更有效率一点，但因为这里的主要目的是演示（用于展示访问各个列值的语法），所以并未考虑效率的问题。

fetchrow_array() 的用法如下所示：

```
while (my @ary = $sth->fetchrow_array ())
{
    my $delim = "";
    for (my $i = 0; $i < @ary; $i++)
    {
```



```

    $ary[$i] = "" if !defined ($ary[$i]); # NULL value?

    print $delim, $ary[$i];

    $delim = ", ";

}

print "\n";

}

```

每次调用`fetchrow_array()`时，如果还有更多行，会返回一个行值数组；否则，返回一个空数组。内循环将依次测试各个列值是否已被定义，如果没有定义，则把它设置为空字符串。这样将把NULL值（它们在DBI脚本里被表示为`undef`）转换为空字符串。这项工作看起来好像有些多余，毕竟，对于`undef`和空字符串，Perl都不会输出任何内容。不过，如果在运行这个脚本时启用了警告模式，那么当你试图输出 `undef` 值时，Perl会产生警告信息——“Use of uninitialized value”，即使用了未经初始化的值。将`undef`转换为空字符串，可以消除警告。在本章的许多示例里，你都能看到类似的代码结构。

如果想把`undef`输出为另外的值，如字符串“NULL”，那么稍微修改那个`if`测试即可：

```

while (my @ary = $sth->fetchrow_array ())
{

    my $delim = "";

    for (my $i = 0; $i < @ary; $i++)
    {

        $ary[$i] = "NULL" if !defined ($ary[$i]); # 为NULL 值?

        print $delim, $ary[$i];

        $delim = ", ";

    }
}

```

```
print "\n";

}
```

当处理值的数组时，可以利用map一次性转换数组中的所有undef元素，从而让代码稍微简洁一点：

```
while (my @ary = $sth->fetchrow_array ())

{

    @ary = map { defined ($_) ? $_ : "NULL" } @ary;

    print join (",", @ary), "\n";

}
```

map 会使用花括号里的表达式来处理数组里的每一个元素，并且会返回一个含有转换结果值的数组。

除把fetchrow_array()的返回值赋给数组变量以外，还可以把列值放入一组标量变量中。这样，我们便能够用更有意义的变量名来代替\$ary[0]、\$ary[1]等。假设，想要把成员的姓名和电子邮件地址检索出来，放到相应的变量里。像下面这样，使用fetchrow_array()便可以选择和获取各个行：

```
my $sth = $dbh->prepare ("SELECT last_name, first_name, suffix, email"
. " FROM member ORDER BY last_name");

$sth->execute ();

while (my ($last_name, $first_name, $suffix, $email)

= $sth->fetchrow_array ())

{

    # 处理各个变量

}
```

如果想照上面这种方式来使用一组变量，那么一定要保证语句中给出的列名称的顺序，与你用来保存数据的那组变量的排列顺序是相符的。DBI并不清楚你的SELECT 语句应该以什么样的顺序列出各个列名，因此给变量正确赋值的责任就落在了你的身上。我们还可以利用一种称作“参数绑定”（参考8.2.7节）的技术，让列的取值自动赋给各个变量。

如果要把获取到的一个值放入一个变量，那么在编写这种赋值表达式时就要多加小心了。如果像下面这样编写循环的开头部分，那么它可以正确工作：

```
while (my ($val) = $sth->fetchrow->array ()) ...
```

由于值是采用列表方式获取的，因此只有在没有更多的行时，这个测试才会失败。但是，如果把这个测试写成下面这样，那么它便会莫名其妙地失败：

```
while (my $val = $sth->fetchrow->array ()) ...
```

此处值是采用标量方式获取的，因此如果\$val的值恰好为0、undef或空字符串，那么即使没有到达结果集的最后一行，这个测试的结果也会为假，从而终止循环。

第二个行获取方法fetchrow_arrayref()，与fetchrow_array()很相似，但它返回的不是一个由当前行的各个列值所构成的数组，而是对该数组的引用，或者 undef（在没有更多行的情况下）。其使用方法如下所示：

```
while (my $ary_ref = $sth->fetchrow_arrayref ())
{
    my $delim = "";

    for (my $i = 0; $i < @{$ary_ref}; $i++)
    {
        $ary_ref->[$i] = "" if !defined ($ary_ref->[$i]); # 为NULL 值?
        print $delim, $ary_ref->[$i];

        $delim = ", ";
    }
}
```

```

}

print "\n";

}

```

可以通过数组引用`$ary_ref`，访问数组元素。这有点像解引用（dereferencing）指针，因此需要使用`$ary_ref->[$i]`形式，而不能用`$ary[$i]`。如果想把这种引用转换为数组，那么可以使用`@{$ary_ref}`结构。

`fetchrow_arrayref()` 不适合用列表方式来存放获取变量。例如，下面这个循环便无法工作：

```

while (my ($var1, $var2, $var3, $var4) = @{$sth->fetchrow_arrayref ()})
{
    # 处理各个变量
}

```

只要`fetchrow_arrayref()`能实际取回一个行，这个循环就能正确执行。可是，如果没有更多的行，那么`fetchrow_arrayref()`便会返回`undef`，而`@{undef}`是非法的（它与C语言程序里的解引用NULL指针的情况相似）。

第三个行获取方法`fetchrow_hashref()`的使用方式，如下所示：

```

while (my $hash_ref = $sth->fetchrow_hashref ())
{
    my $delim = "";

    foreach my $key (keys (%{$hash_ref}))
    {
        $hash_ref->{$key} = "" if !defined ($hash_ref->{$key}); # 为
        NULL 值?

        print $delim, $hash_ref->{$key};
    }
}

```

```

$delim = ",";

}

print "\n";

}

```

每次调用`fetchrow_hashref()`，它都会返回一个散列引用（该散列由行的值构成，其键为各个列名），或者`undef`（若没有更多的行）。在此情况下，列值并无特定顺序，因为Perl散列的元素是无序的。不过，因为DBI是把各个列名作为散列元素的键，所以我们完全能够通过`$hash_ref`变量访问列值。这意味着，你完全可以按任意顺序提取各个值（或者它们的任何子集），可以不用理会SELECT语句在检索这些列时的顺序。例如，想要访问成员的姓名和电子邮件，那么可以采用下面的做法：

```

while (my $hash_ref = $sth->fetchrow_hashref ())
{
    my $delim = "";

    foreach my $key ("last_name", "first_name", "suffix", "email")
    {
        $hash_ref->{$key} = "" if !defined ($hash_ref->{$key}); # 为NULL 值?

        print $delim, $hash_ref->{$key};

        $delim = ",";
    }

    print "\n";
}

```

`fetchrow_hashref()` 特别适用于这样的场合：需要把行值传递给某个函数，同时又不要求它了解 SELECT 语句里给出的列的顺序。在这种情况下，你可以调用`fetchrow_hashref()`来检索各个行，并编写一个函数，让它通过列名来访问行值。

在使用`fetchrow_hashref()`时，请注意以下几个问题。

如果想获得最佳性能，那么使用`fetchrow_hashref()`并不是最佳选择。它的执行效率比`fetchrow_array()`或`fetchrow_arrayref()`低。

在默认情况下，充当键值的那些列名将沿用它们在SELECT语句里的大小写。在MySQL里，列名不区分大小写，所以不管使用什么样的大小写组合来写列名，语句都能正确地执行。但是，Perl语言的散列键名是区分大小写的，这可能会导致你出现某些问题。为了避免出现潜在的大小写不匹配的问题，你可以向`fetchrow_hashref()`传递一个`NAME_lc`或`NAME_uc`属性，强制要求它统一列名的大小写：

```
$hash_ref = $sth->fetchrow_hashref("NAME_lc"); # 使用小写字母
```

```
$hash_ref = $sth->fetchrow_hashref("NAME_uc"); # 使用大写字母
```

散列中的元素必须对应着一个唯一的列名。如果同时对多个表进行关联查找，并且这些表的列名有重复，那么你将无法访问所有这些重复的列值。例如，当执行下面语句时，`fetchrow_hashref()`将返回一个只包含一个元素（即name）的散列：

```
SELECT a.name, b.name FROM a INNER JOIN b WHERE a.name = b.name
```

为避免出现这种问题，应该使用别名来保证每个列名都唯一。如果把刚才那个语句改写为下面这个样子，那么`fetchrow_hashref()`会返回一个散列引用，而散列里将包含`name`和`name2`两个元素：

```
SELECT a.name, b.name AS name2 FROM a INNER JOIN b WHERE a.name = b.name
```

8.2.5.2 确定语句返回的行数

怎样才能知道SELECT语句或者类似于SELECT的语句，到底返回了多少个行呢？一种办法是，在获取行时对它们进行统计。事实上，这是DBI里唯一一种可移植的、可以了解到SELECT语句返回了多少个行的手段。MySQL驱动程序提供了一个`row()`方法，在调用完`execute()`之后，可以调用它来获得行数。但这个办法不能移植到其他数据库系统，并且DBI的文档里也明确表示不提倡把`row()`用于SELECT语句。即使在MySQL里，如果设置了`mysql_use_result`属性，那么也要等到把所有行都取回之后，`row()`才能返回正确的结果。因此，最保险的做法还是在取回行时对它们进行计数（。更多有关`mysql_use_result`属性的详细信息请参考附录H。）

8.2.5.3 获取单行结果

如果结果集里只有一个行，那么就没必要使用循环来获得结果了。假设，想要编写一个脚本 `count_members.pl`，用它来统计美史联盟中当前的成员人数。执行这条语句的参考代码如下所示：

```
# 执行查询

my $sth = $dbh->prepare ("SELECT COUNT(*) FROM member");

$sth->execute ();

# 读取并显示查询结果

my $count = $sth->fetchrow_array ();

$sth->finish ();

$count = "can't tell" if !defined ($count);

print "$count\n";
```

在上面这段代码里，`SELECT` 语句只会返回一个行，所以没必要使用循环，只需调用一次 `fetchrow_array()` 即可。此外，因为我们只选取了一个列，所以甚至不必把返回值赋给数组。当在标量环境里调用 `fetchrow_array()` 方法时（即期望的是一个值，而非一个列表），它会在有行时，返回这个行的一个列；没有行时，返回 `undef`。DBI 并没有规定 `fetchrow_array()` 在标量环境里会返回行的哪一列，但是这对上面这条语句来说丝毫没有影响。这段代码只会检索一个值，因此返回值不可能产生二义性。

虽然结果集里只有一个行，但这段代码还是会调用 `finish()` 方法来释放结果集。（当 `fetchrow_array()` 注意到已经到达最后一行时，它会隐式地释放结果集。但是，在这里，只有当你第二次调用它时，才会有这样的效果。）

另外还有一种最多只会返回一个行的查询，即它会包含 `LIMIT 1`，用以限制返回的行数。这种查询经常被用来查找某个列里的最大值或者最小值。例如，下面这个查询可以把最近出生的那位总统的姓名和出生日期显示出来：

```
my $stmt = "SELECT last_name, first_name, birth FROM president"
```

```

." WHERE birth = (SELECT MAX(birth) FROM president)";

my $sth = $dbh->prepare ($stmt);

$sth->execute ();

my ($last_name, $first_name, $birth) = $sth->fetchrow_array ();

$sth->finish ();

if (!defined ($last_name))

{

print "Query returned no result\n";

}

else

{

print "Most recently born president: $first_name $last_name ($birth)\n";

}

```

使用MAX()或MIN()函数的语句只会返回一个行，因此它是另一种不必使用循环的语句。对于所有这些情况，有一种更加简便的获得单行结果的方法，即通过数据库句柄来调用selectrow_array()方法，它把prepare()、execute()和行获取这三项操作合并到了一起。如果执行成功，它将返回一个数组（而不是引用）；如果没有返回任何行，或者在执行过程中出现了错误，那么它将返回一个空数组。前一个示例可以用selectrow_array()来重写成下面这样：

```

my $stmt = "SELECT last_name, first_name, birth FROM president"

." WHERE birth = (SELECT MAX(birth) FROM president)";

my ($last_name, $first_name, $birth) = $dbh->selectrow_array ($stmt);

if (!defined ($last_name))

```



```

{

print "Query returned no result\n";

}

else

{

print "Most recently born president: $first_name $last_name ($birth)\n";

}

```

8.2.5.4 处理整个结果集

当使用循环来获取行时，只能按循环返回的顺序依次处理各个行，因为 DBI 没有提供任何其他顺序。此外，如果没有采取适当的内存维护措施，那么在你获取了一个行之后，上一个行便会丢失。这些行为并不总是我们所希望的。例如，当需要对行进行多次遍历以完成某项统计计算时，它们便不适合。（或许你想先遍历一次结果集，评估数据的一般数字特征；然后再遍历一次各个行，进行精确分析。）

可以用两种不同的方式来把结果集当作整体来访问。第一种，你可以使用常规的获取循环，每获取一个行，便立刻把它保存起来。第二种，可以调用某个方法一次性返回整个结果集。不管采用哪种方式，最终都会得到一个这样的矩阵：它的每一个行分别对应着结果集里的一个行，它包含的列也与你选择的那些列相同。然后，你便可以按想要的顺序、想要的次数来处理矩阵的元素。接下来分别对这两种方式进行讨论。

第一种方式是用获取循环来捕获结果集，即调用 `fetchrow_array()` 方法，保存一个行引用数组。下面这段代码与 `dump_members.pl` 脚本里的那个“获取加输出”循环所做的事情基本一致。不同之处在于，它会把所有行保存起来，然后再输出这个矩阵。下面这段代码主要演示了：如何确定矩阵的行数和列数，以及如何访问矩阵里的各个元素。

```

my @matrix = (); # 数组引用的数组

while (my @ary = $sth->fetchrow_array ()) # 获取每一个行
{

```

```

push (@matrix, [ @ary ]); # 保存对刚获取到的那个行的引用
}

# 确定矩阵的维度

my $rows = scalar (@matrix);

my $cols = ($rows == 0 ? 0 : scalar (@{$matrix[0]}));

for (my $i = 0; $i < $rows; $i++)      # 输出每一个行
{
    my $delim = "";

    for (my $j = 0; $j < $cols; $j++)
    {
        $matrix[$i][$j] = "" if !defined ($matrix[$i][$j]); # 为NULL
值?

        print $delim, $matrix[$i][$j];

        $delim = ", ";
    }

    print "\n";
}

```

在检查矩阵的维度时，必须先确定矩阵的行数，因为在计算矩阵的列数时，需要知道矩阵是否为空。如果\$row等于0，则矩阵为空，\$col也将变为0。因为矩阵的列数等于它的第一个行里的元素个数，所以利用语法@{\$matrix[0]}以整体方式访问这一个行，便能把列数确定下来。

上面的示例在获取每一个行时，都会把它当作一个数组，并且会保存对它的引用。你或许会认为像下面这样调用fetchrow_arrayref()会比直接检索行引用更有效率：

```

my @matrix = (); # 数组引用的数组

while (my $ary_ref = $sth->fetchrow_arrayref ())
{

    push (@matrix, $ary_ref); # 保存对刚获取到的那个行的引用

}

```

很可惜，这种做法行不通，因为`fetchrow_arrayref()`会反复使用引用所指向的那个数组。结果矩阵是一个引用数组，其中的每一个引用都指向同一个行——即检索回的最后一个行。因此，如果想用每次获取一个行的办法来构造矩阵，那么一定要使用`fetchrow_array()`，而不要使用`fetchrow_arrayref()`。

第二种使用获取循环的方式是，调用某个可以返回整个结果集的DBI 方法。例如，`fetchall_arrayref()`方法会返回一个对引用数组的引用，而数组引用中的每一个引用都会指向结果集里某一个行的内容。（听起来很复杂，简单来讲，其含义即为：它的返回值是一个矩阵引用。）如果要使用`fetchall_arrayref()`，则需要依次调用`prepare()`和`execute()`，然后再像下面这样检索结果：

```

# 获取所有行，并保存为一个对引用数组的引用

my $matrix_ref = $sth->fetchall_arrayref ();

确定结果集矩阵维度并访问其中元素的代码，如下所示：

# 确定矩阵的维度

my $rows = (!defined ($matrix_ref) ? 0 : scalar (@{$matrix_ref}));

my $cols = ($rows == 0 ? 0 : scalar (@{$matrix_ref->[0]}));

for (my $i = 0; $i < $rows; $i++)          # 输出每一个行
{

    my $delim = "";

```

```

    for (my $j = 0; $j < $cols; $j++)

    {

        $matrix_ref->[$i][$j] = "" if !defined ($matrix_ref->[$i][$j]);
# 为NULL 值?

        print $delim, $matrix_ref->[$i][$j];

        $delim = ", ";

    }

    print "\n";

}

```

如果结果集为空，那么`fetchall_arrayref()`会返回一个对空数组的引用。如果执行出错，则返回`undef`。因此，如果没有启用`RaiseError`属性，那么在使用返回值之前，必须先对其进行检查。

可以通过检查结果矩阵是否为空来确定行数和列数。如果想把结果矩阵的某一个行`$i`当作一个数组来访问，则需要使用`@{$matrix_ref->[$i]}`语法形式。

很显然，用`fetchall_arrayref()`来检索结果集，比使用行获取循环的做法更简单，只是访问矩阵元素的语法略微麻烦一点。`selectall_arrayref()`方法与`fetchall_arrayref()`相似，但它替你做了更多的事。它可以为你完成`prepare()`、`execute()`和行获取循环等一整套的流程。如果打算使用`selectall_arrayref()`，则需要使用数据库句柄把你的语句直接传递给它：

```

# 获取所有行，并保存为一个对引用数组的引用

my $matrix_ref = $dbh->selectall_arrayref ($stmt);

# 确定矩阵的维度

my $rows = (!defined ($matrix_ref) ? 0 : scalar (@{$matrix_ref}));

my $cols = ($rows == 0 ? 0 : scalar (@{$matrix_ref->[0]}));

for (my $i = 0; $i < $rows; $i++)          # 输出每一个行

```

```

{
my $delim = "";

for (my $j = 0; $j < $cols; $j++)

{
    $matrix_ref->[$i][$j] = "" if !defined ($matrix_ref->[$i][$j]);
# 为NULL 值?

    print $delim, $matrix_ref->[$i][$j];

    $delim = ", ";

}

print "\n";

}

```

8.2.5.5 检测NULL值

当从数据库检索信息时，经常需要把列里的NULL 值与数字 0或空字符串区分开来。因为DBI会把NULL值返回为undef，所以这件事情比较好办。不过，你必须保证使用了正确的测试。如果执行下面的代码段，则会输出3个“false!”：

```

$col_val = undef; if (!$col_val) { print "false!\n"; }

$col_val = 0;      if (!$col_val) { print "false!\n"; }

$col_val = "";     if (!$col_val) { print "false!\n"; }

```

以上测试表明，测试形式无法把 undef、数字 0和空字符串区分开来。下面这段代码的两个测试都输出了“false!”，表明这个测试无法区分undef和空字符串：

```

$col_val = undef; if ($col_val eq "") { print "false!\n"; }

$col_val = "";    if ($col_val eq "") { print "false!\n"; }

```

下面这段代码会输出同样的结果，从而说明第二个测试不能把数字0和空字符串区分开来：

```
$col_val = "";

if ($col_val eq "") { print "false!\n"; }

if ($col_val == 0) { print "false!\n"; }
```

如果想要区分列中的undef (NULL) 值和非undef 值，则必须使用 `defined()`。只有在知道了值不为NULL之后，才能通过适当的测试来区分各个值之间的类型。例如：

```
if (!defined ($col_val)) { print "NULL\n"; }

elsif ($col_val eq "") { print "empty string\n"; }

elsif ($col_val == 0) { print "zero\n"; }

else { print "other\n"; }
```

这些测试的顺序非常重要，如果 `$col_val` 是一个空字符串，那么第二个比较操作和第三个比较操作都将为真。如果颠倒这这些比较的顺序，那么你会错误地把空字符串解释成数字0。

8.2.6 在语句串里引用特殊字符

到目前为止，我们构造出来的语句都是一些用引号引起来的简单字符串。当在带引号的字符串里包含带引号的值时，会在Perl词法层面引起混乱。如果试图插入或者选取某些包含引号、反斜线或二进制数据的值，那么又会在SQL 层面遇到麻烦。当把某条语句构造成一个在Perl语言里带引号的字符串时，必须对语句字符串里每一个引号字符进行转义：

```
$stmt = 'INSERT INTO absence VALUES(14,\''2012-09-16\')';

$stmt = "INSERT INTO absence VALUES(14,\"2012-09-16\")";
```

Perl和MySQL都支持使用单引号或双引号来把字符串引起来，因此在有些时候，你可以通过混用单、双引号来避免转义：

```
$stmt = 'INSERT INTO absence VALUES(14,"2012-09-16")';
```

```
$stmt = "INSERT INTO absence VALUES(14,'2012-09-16');"
```

请注意，在使用引号时，必须保证字符串可以按照你所期望的那样进行解释。下面几个因素需要着重考虑一下。

这两种引号在Perl语言里不等效。只有双引号里的变量引用才能被解释。因此，如果想采用在语句字符串里嵌入变量引用的方式来构造语句，那么不能用单引号。例如，如果变量\$var的值为14，那么下面两个字符串并不等价：

```
"SELECT * FROM member WHERE member_id = $var"
```

```
'SELECT * FROM member WHERE member_id = $var'
```

Perl会把它们解释成下面这个样子：

```
"SELECT * FROM member WHERE member_id = 14"
```

```
'SELECT * FROM member WHERE member_id = $var'
```

很显然，第一个字符串更像是你想传递到 MySQL服务器的。至于第二个字符串，服务器将会把其中的\$var解释成member表里的某个列名。

在MySQL里，单引号和双引号的含义并不总是相同。如果服务器运行在ANSI_QUOTES被禁用的SQL模式下，那么使用这两种引号当中的任何一种都可以。但是，如果启用了SQL的ANSI_QUOTES 模式，那么对于字符串必须使用单引号；而对于标识符（如数据库名或表名），则只能使用双引号。因此，使用单引号来引用字符串是最安全的，因为无论是否设置了ANSI_QUOTES，它都可以使用。

在Perl层面，除把字符串放在双引号里外，还可以使用qq{}结构，它会告诉Perl把qq{和}之间的内容当作用双引号引起来的字符串（两个q的意思即为“双引号”）。例如，下面两行代码是等价的：

```
$date = "2012-09-16";
```

```
$date = qq{2012-09-16};
```

在构造语句时，只要使用qq{}，便可以不用考虑引号的问题，因为你可以任意地在语句里使用引号（单引号或双引号），同时不必对它们进行转义。此外，变量引用也可以被正确解释。qq{}的这两个特性在下面的INSERT语句里得到了充分的体现：

```
$id = 14;
```

```
$date = "2012-09-16";
```

```
$stmt = qq{INSERT INTO absence VALUES($id,$date)};
```

qq 结构的分隔符并非只能使用 “{” 和 “}” 。你完全可以写成 qq() 或 qq//, 只需这种成对的分隔符不会出现在字符串里即可。我个人比较喜欢使用 qq {}, 因为在语句字符串里, 字符 “}” 出现的几率比 “)” 或 “\” 都小, 并且不容易被误认为是语句的结束字符。例如, 在上面这条 INSERT 语句里, 便出现了 “)” , 因此用 “qq()” 来引用这个语句串就会出现这个问题。

这种 qq {} 结构允许代码跨行。这样一来, 我们便可以把语句串与外围的 Perl 代码区别开来:

```
$id = 14;
```

```
$date = "2012-09-16";
```

```
$stmt = qq{
```

```
INSERT INTO absence VALUES($id,$date)
```

```
};
```

可以利用这一特性, 把语句格式化多行, 以增加代码的可读性。例如, dump_members.pl 脚本里有一条 SELECT 语句, 如下所示:

```
$sth = $dbh->prepare ("SELECT last_name, first_name, suffix, email,"
```

```
." street, city, state, zip, phone FROM member ORDER BY last_name");
```

利用 qq {}, 可以把它改写为下面这个样子:

```
$sth = $dbh->prepare (qq{
```

```
SELECT
```

```
last_name, first_name, suffix, email,
```

```
street, city, state, zip, phone
```



```
FROM member

ORDER BY last_name

});
```

虽然放在双引号里的字符串也允许跨行，但是个人认为 qq{和} 比两个孤零零的" 字符更醒目，能让语句更容易阅读。这两种格式在本书都有使用，看看你更喜欢哪一个。

这种qq{}结构解决了Perl词法层面上的引号使用问题，它让你可以在字符串里随意使用引号，而不会引起Perl的抱怨。但是，你也必须认真考虑SQL层面的语法问题。看看下面这段代码，它想要把一个新行插到member表里：

```
$last = "O'Malley";

$first = "Brian";

$expiration = "2013-09-01";

$rows = $dbh->do (qq{

INSERT INTO member (last_name,first_name,expiration)

VALUES('$last','$first','$expiration')

});
```

do()方法发送给MySQL的结果字符串将会像下面这样：

```
INSERT INTO member (last_name,first_name,expiration)

VALUES('O'Malley','Brian','2013-09-01')
```

这条SQL语句是非法的，因为在'O'Malley'这个单引号字符串里又出现了单引号字符。我们在第7章里曾遇到过这种引号问题，当时使用了mysql_real_escape_string()函数来解决问题。DBI也提供了类似的机制：当需要在语句里使用带引号的值时，可以调用quote()方法，然后使用这个方法的返回值。上面那个示例可以正确地写成下面这样：

```
$last = $dbh->quote ("O'Malley");
```

```

$first = $dbh->quote ("Brian");

$expiration = $dbh->quote ("2013-09-01");

$rows = $dbh->do (qq{

INSERT INTO member (last_name,first_name,expiration)

VALUES($last,$first,$expiration)

});

```

现在， `do()` 方法发送给MySQL的字符串将变成下面这个样子，其中，带引号的字符串里的引号字符都得到了正确的转义处理：

```

INSERT INTO member (last_name,first_name,expiration)

VALUES(O\'Malley','Brian','2013-09-01')

```

请注意，语句串里的 `$last` 和 `$first` 都不应该再用引号引起来，因为 `quote()` 方法会替你把它们处理好。如果你自己添加了引号，那么这条语句里的引号就会过多，如下例所示：

```

$value = "paul";

$quoted_value = $dbh->quote ($value);

print "The quoted value is: $quoted_value\n";

print "The quoted value is: '$quoted_value'\n";

```

这些语句将产生下列输出结果：

```

The quoted value is: 'paul'

The quoted value is: ''paul''

```

很明显，在第二种情况里，字符串包含了多余的引号。

8.2.7 占位符与预处理语句

在前面几节，我们在构造语句时，都是把将被插入或者将被用在选择条件里的数据值，直接放置到语句字符串里。这种做法不是必需的。DBI 允许

在语句串里，使用被称作“占位符”的特殊标记符，然后在执行这个语句时，会使用具体的值来替换这些占位符。这就是所谓的“将值绑定到语句”。采用这种做法，你可以获得字符引用的好处，即不需要显式地调用`quote()`便能获得同样的字符引用效果。此外，如果需要在循环里反复地执行某条语句，那么你可以先预处理它，然后再反复执行它。这样可以避免“在每次执行之前都需要预处理语句”所带来的开销。

为了说明占位符是如何工作的，这里做一个假设：现在正值新学期的开始，你需要把成绩册里的`student`表清理干净，并使用保存在某个文件里的新的学生名单来初始化这个表。如果不使用占位符，那么你可能会像下面这样删除已有表里的内容，并加载新的名字：

```
$dbh->do (qq{ DELETE FROM student } );    # 删除已有行

while (<>)                                  # 读取每一个输入
行，

    {                                       # 把它添加到一个
个新行

    chomp;

    $_ = $dbh->quote ($_);

    $dbh->do (qq{ INSERT INTO student SET name = $_ });

    }
```

这种方法需要你自己调用`quote()`方法来处理各个数据值里的特殊字符。这种做法的执行效率很低，因为 `INSERT` 语句的基本格式每次都一样，但在循环的过程中，`do()` 每次都会调用`prepare()`和`execute()`。如果在进入循环之前，只调用一次`prepare()`来建立`INSERT`语句，并且在循环里，只调用`execute()`，那么执行效率会提高很多。这样，只需调用一次`prepare()`，从而省去所有其他调用。DBI可以像下面这样实现这一点：

```
$dbh->do (qq{ DELETE FROM student } );    # 删除已有行

my $sth = $dbh->prepare (qq{ INSERT INTO student SET name = ? });

while (<>)                                  # 读取每一个输入
行，
```

```

    {
# 把它添加到一个新行

    chomp;

    $sth->execute ($_);

}

```

一般情况下，如果你发现需要在一个循环里多次调用`do()`，那么最好把`prepare()`调用安排在循环的前面，并在循环里调用`execute()`。请注意INSERT语句里的那个问号“?”。它就是一个占位符。当调用`execute()`，把语句发送给服务器时，需要把这个占位符替换成具体的值。DBI会自动为这个值里的特殊字符加上引号，因此无需调用`quote()`。

在使用占位符时，有以下几点需要引起注意。

不要为语句字符串里的占位符加上引号。否则，DBI不会把它识别为占位符。

不要使用`quote()`方法来指定占位符的值。否则，要插入的那个值里会出现多余的引号。

在同一个语句串里允许出现多个占位符，只是对于传递给`execute()`函数的那些值，必须保证其数量与占位符标记的个数是相同的。

每一个占位符指定的都只能是一个值，而不能是值列表。例如，想要指定多个数据值，那就不能像下面这样预处理和执行语句：

```

my $sth = $dbh->prepare (qq{

INSERT INTO member last_name, first_name VALUES (?)

});

$sth->execute ("Adams,Bill,2014-07-19");

```

必须单独指定各个值，并且为每个值提供一个占位符：

```

my $sth = $dbh->prepare (qq{

INSERT INTO member last_name, first_name VALUES (?, ?, ?)

```

```
});
```

```
$sth->execute ("Adams","Bill","2014-07-19");
```

如果需要把某个占位符的值替换为NULL值，那么要用undef。

占位符和quote()方法都只适用于数据值。请不要把占位符用于像SELECT之类的关键字，或者像数据库名、表名、列名之类的标识符。这种做法之所以行不通，是因为这种关键字或标识符在插到语句后，都会被加上引号，从而导致这条语句因语法错误而执行失败。

对于某些数据库服务器，使用占位符除了提高循环语句的效率以外，还有另外一个与执行效率有关的好处。某些数据库服务器会缓存预处理语句，可能还会缓存语句执行计划。这样，当服务器再次接收到同样的语句时，便可以更加快速地重用这条语句，而不必再进行初始预处理。把查询缓存起来的做法对复杂的SELECT语句特别有用，因为预处理语句和生成最佳执行计划，需要花费很多时间。在让语句变得更适合缓存方面，占位符提供了较好的机会。因为与采用在语句字符串里直接嵌入特定数据值来构造语句的做法相比，利用占位符构造出的语句更具有通用性。

MySQL并不会缓存执行计划。MySQL有一个查询缓存，但它只是用于缓存查询字符串的结果集，而不会缓存执行计划。更多相关信息请参考12.7.3节。

默认情况下，MySQL也不会缓存预处理语句。把参数绑定到占位符，这个动作发生在客户端的DBD::mysql模块里。不过，在MySQL的C语言客户端开发库里所实现的二进制协议，既支持在服务器端预处理语句，也支持由服务器来处理参数绑定的事宜。

DBD::mysql模块可以利用这种功能。为了打开服务器端的预处理语句和参数绑定功能，所需做的就是启用mysql_server_prepare选项。例如，给定一个数据库句柄\$dbh，那么可以像下面这样做：

```
$dbh->{mysql_server_prepare} = 1;
```

如果要禁用服务器端的预处理语句功能，则只需把这个选项设置为0。

即使不想使用MySQL的服务器端预处理语句功能，利用占位符来编写语句也是有好处的：当把脚本移植到某个支持执行计划缓存的数据库服务器时，有占位符的语句的执行效率会比没有占位字符的语句的高。

神秘的undef

某些执行语句字符串的DBI方法，如do()和selectrow_array()，支持为语句里的任意多个“?”字符提供占位符值。例如，可以像下面这样更新行：

```
my $rows = $dbh->do (
    "UPDATE member SET expiration = ? WHERE member id = ?",
    undef, "2007-01-01", 14);
```

或者像下面这样获取行：

```
my $ref = $dbh->selectrow_arrayref (
    "SELECT * FROM member WHERE member id = ?",
    undef, 14);
```

请注意观察，这两段代码都使用一个看起来毫无意义的神秘参数undef作为占位符的值。具体原因在于：对于那些支持使用占位符作为参数的语句执行方法，在某些参数的前面可能是另一个可用来设定语句处理属性的参数。虽然这些属性很少会被使用到（必要时会用到），但与之对应的那个参数却必须存在，因此可以把它指定为undef。

8.2.8 把查询结果绑定到脚本变量

在语句字符串里有了占位符，你便可以在执行语句时把它们替换为具体值。换句话说，你可以参数化这条语句的“输入”。DBI也提供了一种相应的名为“参数绑定”的输出操作。在获取行时，它可以让你通过检索各个列值的方式，把“输出”自动参数化为变量，而你完全不用自己对这些变量进行赋值。

假设你有一个查询，用于检索member表里的姓名。你可以让DBI把所选列的值自动赋给Perl变量。当获取行时，这些变量将自动更新为相应的列值，从而让检索操作变得非常有效率。下面这个示例演示了如何把列绑定到某个变量，以及如何在行获取循环里访问这些变量：

```
my ($last_name, $first_name, $suffix);

my $sth = $dbh->prepare (qq{
    SELECT last_name, first_name, suffix
```

```
FROM member ORDER BY last_name, first_name
});
```

```
$sth->execute ();
```

```
$sth->bind_col (1, \ $last_name);
```

```
$sth->bind_col (2, \ $first_name);
```

```
$sth->bind_col (3, \ $suffix);
```

```
print "$last_name, $first_name, $suffix\n" while $sth->fetch ();
```

请在`execute()`之后和获取行之前调用`bind_col()`。每次调用都应该指定一个列编号和一个对绑定到该列的那个变量的引用。列编号从1开始。

有一种方法可以取代这种单个调用`bind_col()`的做法，即把所有变量引用一次性全部传递给`bind_columns()`：

```
my ($last_name, $first_name, $suffix);
```

```
my $sth = $dbh->prepare (qq{
```

```
SELECT last_name, first_name, suffix
```

```
FROM member ORDER BY last_name, first_name
```

```
});
```

```
$sth->execute ();
```

```
$sth->bind_columns (\ $last_name, \ $first_name, \ $suffix);
```

```
print "$last_name, $first_name, $suffix\n" while $sth->fetch ();
```

请在`execute()`之后和获取行之前调用`bind_columns()`。

8.2.9 指定连接参数

与服务器建立连接的最直接办法是，把所有的连接参数设定为`connect()`方法的参数：

```
my $dsn = "DBI:mysql:db_name:host_name";
```

```
my $dbh = DBI->connect ($dsn, user_name, password);
```

如果省略了连接参数，那么DBI将会根据以下规则进行确定使用什么值。

如果定义了环境变量DBI_DSN，并且数据源名称（DSN）未定义或者为一个空串，则会使用环境变量 DBI_DSN。如果定义了环境变量 DBI_USER和DBI_PASS，并且没有定义用户名和密码（不包括它们为空串的情况），则会使用环境变量DBI_USER和DBI_PASS。

在Windows系统上，如果没有定义用户名，则使用环境变量USER。

如果省略了主机名，则DBI会尝试连接本地主机。

如果用户名设定为undef或一个空串，则在Unix上，默认为登录名；而在Windows系统上，默认为ODBC。

如果密码设定为undef或一个空串，则不发送密码。

可以在DSN里设定某些选项，方法就是把它们追加到整个字符串的最开头部分，并且各自用分号隔开。例如，像下面这样用mysql_read_default_file选项来指定选项文件的路径名：

```
my $dsn = "DBI:mysql:sampdb;mysql_read_default_file=/home/paul/.my.cnf";
```

当这个脚本在执行时，会到指定的文件读取连接参数。假设，/home/paul/.my.cnf文件的内容如下：

```
[client]
```

```
host=localhost
```

```
user=sampadm
```

```
password=secret
```

在这种情况下，connect()调用会尝试使用用户名 sampdb和密码 secret来连接位于主机localhost上的MySQL服务器。在Unix系统上，还可以让脚本使用这样的选项文件：它属于任何运行该脚本的用户，而其文件名则像下面这样被参数化：


```
my $dsn =  
"DBI:mysql:sampdb;mysql_read_default_file=$ENV{HOME}/.my.cnf";
```

`$ENV{HOME}` 包含运行此脚本的那个用户的主目录路径名，因此它所使用的连接参数都将来自于当前用户自己的选项文件。如果采用这种方式来编写脚本，则可以不用在脚本里写出各个连接参数。

使用 `mysql_read_default_file` 选项会导致让脚本只读取指定的选项文件。如果还想让它读取全局性选项文件（如Unix系统上的 `/etc/my.cnf`，或者Windows系统上的 `C:\my.ini`），则难以做到。如果想让脚本从所有标准选项文件里读取连接参数，则应该使用 `mysql_read_default_group` 选项。这个选项使得在这个选项文件里指定的那个组以及 `[client]` 组里的参数会被用上。例如，你有某些选项，专门用于与 `sampdb` 数据库有关的脚本，则可以把它们列在一个 `[sampdb]` 组里，然后像下面这样使用数据源值：

```
my $dsn = "DBI:mysql:sampdb;mysql_read_default_group=sampdb";
```

如果只想读取标准选项文件里的 `[client]` 组，则可以像下面这样指定选项：

```
my $dsn = "DBI:mysql:sampdb;mysql_read_default_group=client";
```

与指定数据源字符串的选项相关的更多信息请参考附录 H。与 MySQL 选项文件格式相关的更多信息请参考附录 F。

在Windows系统上使用 `mysql_read_default_file` 选项会遇到这样一个难题，即文件路径名通常以一个驱动器盘符和一个冒号开始。之所以会成为一个问题，那是因为DBI会把这个冒号解释为分隔DSN字符串的各个部分的字符。有可能可以绕过这个问题，虽然解决办法比较笨拙。

(1) 利用 `chdir()` 把位置切换到选项文件所在的驱动器的根目录，以便那些不带驱动器盘符的路径名可以被解释为该驱动器的相对路径。

(2) 把DSN里 `mysql_read_default_file` 选项的值设置为这个文件名，但不带驱动器盘符或冒号。

(3) 如果不想让连接操作影响到当前目录，则需要在调用 `connect()` 之前，把当前目录的路径保存起来，接着，在连接完成之后，通过 `chdir()` 重新返回该目录。

下面这段代码演示了怎样使用选项文件 `C:\my.ini`。（请注意，Windows路径名里的反斜线“`\`”，必须改写为Perl语言字符串里的斜线

“/” 。)

保存当前目录路径名

```
use Cwd;
```

```
my $orig_dir = cwd ();
```

切换到文件所在的那个驱动器的根目录

```
chdir ("C:/") or die "Cannot chdir: $!\n";
```

使用C:\my.ini里的参数进行连接

```
my $dsn = "DBI:mysql:sampdb:localhost:mysql_read_default_file=/my.ini";
```

```
my %conn_attrs = (RaiseError => 1, PrintError => 0, AutoCommit => 1);
```

```
my $dbh = DBI->connect ($dsn, undef, undef, \%conn_attrs);
```

切换回原来的那个目录

```
chdir ($orig_dir) or die "Cannot chdir: $!\n";
```

使用选项文件，并不会妨碍你在connect()调用里指定连接参数（例如，想让脚本以某个特定的用户进行连接）。在调用connect()时，显式指定的连接参数，如主机名、用户名和密码值，将改写那些在选项文件找到的连接参数。例如，你可能会想让脚本解析来自命令行的选项，如--host和--user，并且使用那些值。如果命令行里给出了这些选项，那么它们将优先于在选项文件里找到的任何相同选项。这么做是有道理的，因为这正是标准的MySQL客户端程序所具有的行为。你的DBI脚本也应该与这种行为保持一致。

对于本章开发的其他命令行脚本，我将使用标准的连接设置，从而省掉有关的代码。我只在这里展示一次连接代码，以便我们在编写脚本时，把精力集中在脚本的主体上面：

```
#!/usr/bin/perl
```

```
use strict;
```

```
use warnings;
```

```

use DBI;

# 如果在命令行给定了连接参数，则解析它们

use Getopt::Long;

$Getopt::Long::ignorecase = 0; # 所有选项都区分大小写

$Getopt::Long::bundling = 1;    # -uname 表示的是 -u name，而非 -u -n -a -m
-c

# 默认参数- 开始时全部未定义

my ($host_name, $password, $port_num, $socket_name, $user_name);

GetOptions (

# =i 表示在选项后面需要提供一个整数值

# =s 表示在选项后面需要提供一个字符串值

"host|h=s"      => \$host_name,

"password|p=s"  => \$password,

"port|P=i"      => \$port_num,

"socket|S=s"    => \$socket_name,

"user|u=s"      => \$user_name

) or exit (1);

# 构造数据源

my $dsn = "DBI:mysql:sampdb";

$dsn .= ";host=$host_name" if $host_name;

$dsn .= ";port=$port_num" if $port_num;

```

```

$dsn .= ";mysql_socket=$socket_name" if $socket_name;

$dsn .= ";mysql_read_default_group=client";

# 连接服务器

my %conn_attrs = (RaiseError => 1, PrintError => 0, AutoCommit => 1);

my $dbh = DBI->connect ($dsn, $user_name, $password, \%conn_attrs);

```

这段代码首先会初始化 DBI，接着会查找命令行里的连接参数，然后使用来自命令行的连接参数或者在标准选项文件的[client]组里找到的连接参数，建立与MySQL服务器的连接。如果你的连接参数放置在选项文件里，则在运行使用了这段代码的脚本时，不必输入这些参数。

每个脚本的结尾部分都相似，只需要在退出之前断开连接即可：

```

$dbh->disconnect ();

```

当进入8.4节进行Web编程时，我们将对这段连接代码稍做修改，但其基本思想是相同的。MySQL的标准客户端程序与Getopt模块在处理命令行选项方面有所不同。MySQL的标准客户端程序的选项处理代码有点特别：在指定密码选项（--password 或-p）时，其后面可以带上密码值，也可以不带；并且如果没有给出密码值，则会提示用户输入。

对于Getopt模块，如果想让提供给--password和-p的密码值变成可选的，那么在不提供密码值时，可能会导致歧义，除非它是命令行的最后一个参数，或者在该选项的后面紧跟着另一个选项，才不会出现该问题。假设有一个脚本，它期望在这些选项后面提供的参数是一个表名。如果像下面这样执行此脚本，那么Getopt模块将错误地把mytbl解释为密码值，而不会提示你输入密码：

```

% ./myscript.pl -u paul -p mytbl

```

对于像前面所示的那段代码，在Perl语言框架里，为避免出现此类问题，要求在给出密码选项时必须带上一个值。

8.2.10 调试

如果要调试不能正常工作的DBI脚本，则通常会用到两种技术，它们既可以单独使用，也可以配合使用。第一种技术是，在脚本里随处放置一些输出语句。这样可以让你随意输出调试信息，只是你必须手工添加这些语句。第

二种技术是，使用DBI内建的跟踪功能。这种方式更通用、更系统化，并且可以在启用之后自动完成。DBI的跟踪功能，还能让你看到驱动程序的操作信息，这是其他调试手段无法做到的。

8.2.10.1 使用输出语句进行调试

这里有一个常见的问题：“我的语句在mysql客户端程序里执行得挺好的，可是在DBI脚本里就不行了。为什么会这样？”最为常见的情况是：DBI脚本执行的语句并非是你所想的那条。如果在执行这条语句之前先输出它，你便会惊奇地看到实际发送到服务器的内容是什么。假设在客户端程序mysql里输入了下面这样一条语句：

```
mysql> INSERT INTO member (last_name,first_name,expiration)
-> VALUES('Brown','Marcia','2012-06-03');
```

接着，在DBI脚本里试图去做同样的事情（当然，要去掉语句末尾的分号）：

```
$last = "Brown";

$first = "Marcia";

$expiration = "2012-06-03";

$stmt = qq{

INSERT INTO member (last_name,first_name,expiration)

VALUES($last,$first,$expiration)

};

$rows = $dbh->do ($stmt);
```

尽管语句都相同，但这一次却不工作了。或者两条语句是否真的相同呢？试着用下面这条语句输出来看看是什么情况：

```
print "$stmt\n";
```

结果如下所示：

```
INSERT INTO member (last_name,first_name,expiration)
```

```
VALUES(Brown,Marcia,2012-06-03)
```

从这个输出结果可以清楚地看到，前后两条语句根本不一样。VALUES()列表里的那些值都没有引号。一种正确指定这类语句的方式是像下面这样使用quote()：

```
$last = $dbh->quote ("Brown");
```

```
$first = $dbh->quote ("Marcia");
```

```
$expiration = $dbh->quote ("2012-06-03");
```

```
$stmt = qq{
```

```
INSERT INTO member (last_name,first_name,expiration)
```

```
VALUES($last,$first,$expiration)
```

```
};
```

```
$rows = $dbh->do ($stmt);
```

另一种方式是，使用占位符来指定语句，并把将要插到语句中的那些值，以参数形式传递给do()方法：

```
$last = "Brown";
```

```
$first = "Marcia";
```

```
$expiration = "2012-06-03";
```

```
$stmt = qq{
```

```
INSERT INTO member (last_name,first_name,expiration)
```

```
VALUES(?,?,?)
```

```
};
```

```
$rows = $dbh->do ($stmt, undef, $last, $first, $expiration);
```

不幸的是，当使用第二种方式时，无法利用输出语句查看整条语句的内容，因为占位符对应的值只有在调用`do()`时才会被计算出来。当使用占位符时，跟踪调试功能往往是更有效的调试方法。

8.2.10.2 利用跟踪机制进行调试

DBI 提供了一个能够生成调试信息的跟踪机制，能找出脚本行为发生异常的原因。该机制有0（关闭）到15（信息量最大）共16个级别。一般情况下，1~4级最有用。例如，2级能让你看到你正在执行的语句的文本（包括已完成占位符替换后的结果）、`quote()`方法的调用结果等这样一些内容。这些信息对于跟踪问题有着巨大的帮助。

你可以在各个脚本里使用`trace()`方法对跟踪机制进行控制，也可以通过设置环境变量`DBI_TRACE`来对你所运行的所有DBI脚本的跟踪机制施加影响。

在调用`trace()`方法时，需要提供一个跟踪级别参数和一个可选的文件名。如果没有指定文件名，那么调试信息将被发送到`STDERR`；如果指定了文件名，则调试信息会被发送到这个文件。下面这个调用会建立一个1级跟踪，而调试信息会被送往`STDERR`：

```
DBI->trace (1);
```

下面这个调用会建立一个2级跟踪，并且调试信息会被送往`trace.out`文件：

```
DBI->trace (2, "trace.out");
```

如果要禁用跟踪，可以把跟踪级别设定为0：

```
DBI->trace (0);
```

利用`DBI->trace()`，可以跟踪所有的DBI操作。对于某个拥有更多细致操作的办法，可以启用单独的句柄级的跟踪功能。当你对脚本存在的问题比较清楚，不想浪费时间去查看大段的跟踪报告时，这种做法非常有用。例如，发现某个特定的`SELECT` 查询有问题，那么可以只跟踪与该查询相关联的语句句柄：

```
$sth = $dbh->prepare (qq{ SELECT ...}); # 创建语句句柄
```

```
$sth->trace (1); # 启用对语句的跟踪
```

```
$sth->execute ();
```

如果在调用`trace()`时，指定了一个文件名参数，那么不管跟踪输出是来自DBI全局，还是来自某个特定句柄，所有的内容都将写到这个文件。

除使用`trace()`外，还可以使用`TraceLevel`属性。此属性可以用于设置或读取给定句柄的跟踪级别：

```
$dbh->{TraceLevel} = 3;           # 设置数据库句柄跟踪级别  
  
my $cur_level = $sth->{TraceLevel}; # 获得语句句柄跟踪级别
```

如果想启用全局跟踪，使它能够对你所运行的所有脚本产生影响，那么可以在shell程序里设置环境变量`DBI_TRACE`。设置语法与所使用的shell程序有关。

对于csh或tcsh：

```
% setenv DBI_TRACE value
```

对于sh、bash或ksh：

```
$ export DBI_TRACE=value
```

对于Windows系统：

```
C:\> set DBI_TRACE=value
```

其中，`value`的格式对于所有shell程序都一样：如果它为一个数字`n`，则启用级别为`n`的跟踪，调试信息会输出到`STDERR`；如果它为一个文件名，则启用级别为2的跟踪，调试信息会输出到指定文件；如果它为`n=file_name`，则启用级别为`n`的跟踪，调试信息会输出到指定文件。下面是一些示例，用的是tcsh语法。

跟踪级别为1，输出到`STDERR`：

```
% setenv DBI_TRACE 1
```

跟踪级别为1，输出到`trace.out`文件：

```
% setenv DBI_TRACE 1=trace.out
```

跟踪级别为2，输出到`trace.out`文件：


```
% setenv DBI_TRACE trace.out
```

使用DBI_TRACE的好处在于，不用修改脚本便能启用跟踪调试机制。但是，如果在shell程序里启用了文件跟踪功能，那么千万要记得在解决问题之后禁用它。由于调试输出会一直追加到那个跟踪文件末尾（而不是覆盖它），因此如果不加注意，那么这个文件可能会变得非常大。因此，在shell启动文件（如.cshrc、.tcshrc、.login或.profile）里定义DBI_TRACE，是极不可取的！

使用下面这些命令，可以关闭用于各种命令解释器的DBI_TRACE。

对于csh或tcsh：

```
% setenv DBI_TRACE 0
```

```
% unsetenv DBI_TRACE
```

对于sh、bash或ksh：

```
$ unset DBI_TRACE
```

```
$ export DBI_TRACE=0
```

对于Windows系统：

```
C:\> unset DBI_TRACE
```

```
C:\> set DBI_TRACE=0
```

8.2.11 使用结果集元数据

DBI 提供了对结果集元数据的访问，这里的元数据是有关查询命令所选取的行的描述性信息。为获得这些信息，需要访问与生成结果集的那个查询相关联的语句句柄的属性。其中，有些属性是所有数据库驱动程序都有的标准DBI属性（如NUM_OF_FIELDS，即结果集里的列数）；而有些则是DBD::mysql（DBI提供的MySQL驱动程序）专为MySQL提供的属性。这类专用属性，如mysql_max_length，即各列值的最大宽度，不适用于其他数据库系统。如果在脚本里使用了MySQL特有的属性，则你的脚本可能存在无法移植到其他数据库的风险。但从另一方面来讲，它们又能让你更加容易地获得想要的信息。

你必须在正确的时间参考访问元数据。一般情况下，只有在调用完 `prepare()` 和 `execute()` 之后，才可以使用 `SELECT` 语句的结果集属性。此外，在使用某个行获取函数到达结果集的末尾之后，或者在调用 `finish()` 之后，这些属性可能不再有效。

下面的示例演示的是，如何把MySQL特有的元数据属性 `mysql_max_length`，与更为通用的属性 `NUM_OF_FIELDS`（即结果集里的列数）和 `NAME`（用于保存列名）一起使用。我们可以把这几个属性提供的信息综合起来编写一个脚本 `box_out.pl`，让它根据 `SELECT` 查询生成输出报告，输出报告的表格样式，与以交互式模式运行的客户端程序 `mysql` 的一样。下面是 `box_out.pl` 的主体部分。你可以把其中的 `SELECT` 语句替换为其他语句。这个输出例程与具体的语句无关。

```
my $sth = $dbh->prepare (qq{

SELECT last_name, first_name, suffix, city, state

FROM president ORDER BY last_name, first_name

});

$sth->execute (); # 在此调用之后，各个属性便应该可用了

# 结果集里各个列值的实际最大宽度

my @wid = @{$sth->{mysql_max_length}};

# 结果集里的列数

my $ncols = $sth->{NUM_OF_FIELDS};

# 如果数据值比列标题窄，或者比单词"NULL"窄（对于NULL列），

# 则调整列宽度

for (my $i = 0; $i < $ncols; $i++)

{

my $name_wid = length ($sth->{NAME}->[$i]);

$wid[$i] = $name_wid if $wid[$i] < $name_wid;
```

```

$wid[$i] = 4 if $sth->{NULLABLE}->[$i]δδ$wid[$i] < 4;

}

# 以表格形式输出结果

print_dashes (\@wid, $ncols);          # 短划线行

print_row ($sth->{NAME}, \@wid, $ncols); # 列标题

print_dashes (\@wid, $ncols);          # 短划线行

while (my $ary_ref = $sth->fetchrow_arrayref ())

{

print_row ($ary_ref, \@wid, $ncols);    # 行数据

}

print_dashes (\@wid, $ncols);          # 短划线行

```

在使用`execute()`初始化语句之后，我们便可以抓取所需要的元数据。`$sth->{NUM_OF_FIELDS}`是一个标量，它表示结果集里有多少个列。`$sth->{NAME}`和`$sth->{mysql_max_length}`则分别表示列名和每一个列值的最大宽度。这两个属性的值都是数组引用，其中包含的每一个元素都对应着结果集的各个列，而它们的排列顺序与这些列在语句里的顺序相同。

剩下的计算，与第7章开发的那个`exec_stmt` 程序所使用的计算大同小异。例如，为避免出现输出不整齐的问题，需要在列名比列中的数据值更宽时，增加列宽值。

输出函数`print_dashes()`和`print_row()`，也与`exec_stmt`里的相应代码相似：

```
sub print_dashes

{

my $wid_ary_ref = shift;  # 对列宽数组的引用

my $cols = shift;        # 列数

for (my $i = 0; $i < $cols; $i++)

{

print "+", "-" x ($wid_ary_ref->[$i]+2);

}

print "+\n";

}

# print row of data.  (doesn't right-align numeric columns)

sub print_row
```

```

{
my $val_ary_ref = shift;  # 对列值数组的引用
my $wid_ary_ref = shift;  # 对列宽数组的引用
my $cols = shift;        # 列数
for (my $i = 0; $i < $cols; $i++)
{
printf "| %-*s ", $wid_ary_ref->[$i],
defined ($val_ary_ref->[$i]) ? $val_ary_ref->[$i] : "NULL";

}

print "|\\n";

}

```

tabular.pl 脚本的输出如下所示：

last_name	first_name	suffix	city	state
Adams	John	NULL	Braintree	MA
Adams	John Quincy	NULL	Braintree	MA
Arthur	Chester A.	NULL	Fairfield	VT
Buchanan	James	NULL	Mercersburg	PA
Bush	George H. W.	NULL	Milton	MA
Bush	George W.	NULL	New Haven	CT
Carter	James E.	Jr.	Plains	GA

...

我们的下一个脚本将利用列的元数据来产生另一种格式的输出。这个脚本为 `show_member.pl`，它可以让你在不输入任何查询语句的情况下，快速查看各个美史联盟成员条目。如果给定了某位成员的姓，那么这个脚本将会像下面这样显示查询到的条目：

```
% ./show_member.pl artel

member_id: 63

last_name: Artel

first_name: Mike

suffix:

expiration: 2016-04-16

email:      mike_artel@venus.org

street:     4264 Lovering Rd.
```

city: Miami

state: FL

zip: 12777

phone: 075-961-0712

interests: Civil Rights,Education,Revolutionary War

在执行show_member.pl脚本时，也可以使用成员编号，或者使用匹配多个姓的SQL模式。在下面两条命令里，第一条会显示23号成员的条目，第二条会显示出那些姓以字母“C”开头的条目：

```
% ./show_member.pl 23
```

```
% ./show_member.pl C%
```

下面是脚本 show_member.pl的主体部分。它会使用NAME 属性来确定各输出行所用的标签，并使用NUM_OF_FIELDS属性来弄清结果集包含多少个列：

```
my $count = 0; # 当前总计输出的条目数

my @label = (); # 列标签数组

my $label_wid = 0;

while (@ARGV) # 根据命令行的每一个参数运行一次查询
{
    my $arg = shift (@ARGV);

    # 默认是按姓进行模式搜索...

    my $clause = "last_name LIKE " . $dbh->quote ($arg);
```

```

# ... 如果参数为数字，则搜索ID

$clause = "member_id = " . $dbh->quote ($arg) if $arg =~
/^\d+$/;

# 执行查询

my $sth = $dbh->prepare (qq{

SELECT * FROM member

WHERE $clause

ORDER BY last_name, first_name

});

$sth->execute ();

# 把获取到的列名用作标签，

# 并且为格式化确定列名的最大宽度

# （但是在循环里只执行一次即可）

if ($label_wid == 0)

{

    @label = @{$sth->{NAME}};

    foreach my $label (@label)

    {

        $label_wid = length ($label) if $label_wid < length ($label);

    }

}

```



```

}

# 读取并显式查询结果

my $matches = 0;

while (my @ary = $sth->fetchrow_array ())
{
    # 在第2个及后续条目之前输出换行
    print "\n" if ++$count > 1;

    foreach (my $i = 0; $i < $sth->{NUM_OF_FIELDS}; $i++)
    {
        # 输出标签
        printf "%-*s", $label_wid+1, $label[$i] .":";

        # 如果有一个行，则输出其中的值
        print " ", $ary[$i] if defined ($ary[$i]);

        print "\n";
    }

    ++$matches;
}

print "\nNo match was found for \"$arg\"\n" if $matches
== 0;

}

```

`show_member.pl`的目的是显示一个条目的所有内容，不管它有多少个字段。先使用`SELECT*`检索出所有的列，再利用`NAME`属性来找出具体的列；采用这种方式之后，即使`member`表在今后增加或者删除了某些列，这个脚本也无需进行修改便能工作。

如果你只想知道表里到底有哪些列，而不打算检索其中的行，那么可以使用下面这条语句：

```
SELECT * FROM tbl_name WHERE FALSE
```

其中，`WHERE FALSE`子句对所有行来说结果都为`FALSE`，因此这条语句的执行效果是，在不返回任何行的情况下生成列的元数据。当像往常那样调用`prepare()`和`execute()`方法之后，便可以通过`@{$sth->{NAME}}`获得各列的名字。请注意，虽然这种利用“空”查询来获得列名的小技巧在MySQL里可以工作，但是在其他数据库系统里却可能无法工作。

更多关于DBI和`DBD::mysql`所提供的属性的信息请参考附录H。是要避免使用MySQL特有的属性来保证可移植性，还是要以可移植性为代价来使用这些属性，现在由你来确定！

8.2.12 执行事务

用DBI脚本来执行事务的一种办法是，显式地执行`SET autocommit`、`START TRANSACTION`、`COMMIT`和`ROLLBACK`语句。（与这几条语句有关的信息请参考2.12节。）不过，DBI模块提供了一个事务处理抽象（用DBI方法和属性来表达的），它会自动为你调用正确的与事务有关的SQL语句。它也可以移植到其他支持事务的数据库系统，然而换作SQL语句却可能无法做到。

要想使用DBI的事务机制，你的应用程序必须使用事务安全类型的表。如果它们不是事务安全的表，则必须使用`ALTER TABLE`来更改它们的类型。例如，把给定表`tbl_name`更改为InnoDB表，则可以使用下面这条语句：

```
ALTER TABLE tbl_name ENGINE = InnoDB;
```

在DBI脚本里，事务处理遵循以下几个步骤。

(1) 禁用（或临时挂起）自动提交模式，以便由你自己来控制何时提交SQL语句。

(2) 在一个eval 块内执行那些构成某个事务的语句，同时启用RaiseError 属性，禁用PrintError 属性，以便任何错误都可以在不输出出错信息的情况下，终止块语句的执行。如果块语句执行成功，则它内部的那个操作应该是，调用commit()来提交本次事务。

(3) 当eval块执行完成时，立刻检查其终止状态。如果出现错误，则调用rollback()来取消事务（并在需要时报告相应的错误）。

(4) 根据需要恢复自动提交模式和出错处理属性。

下面的这个示例实现了此方法。它基于第2章的一个场景，即如何从mysql客户端程序以手动方式执行与事务相关的语句。此处的场景是，你发现自己在score表里把两名学生的分数弄混了，需要把它们交换过来。其中，8号学生的分数变成了18，9号学生的分数变成了13，而实际情况是这两个分数应该对换一下。为修正这个问题，需要用到的两个UPDATE 语句如下所示：

```
UPDATE score SET score = 13 WHERE event_id = 5 AND
student_id = 8;
```

```
UPDATE score SET score = 18 WHERE event_id = 5 AND
student_id = 9;
```

你很想用正确的考试分数来更新这两个行，但是必须保证它们作为一个整体同时被更新成功。在前一章里，那个示例的更新操作周围充斥着多条SQL语句，用于完成设置自动提交模式、提交和回滚等操作。在使用DBI事务机制的Perl脚本里，可以像下面这样来完成更新操作：

```
my $orig_re = $dbh->{RaiseError}; # 存储出错处理属性
```

```
my $orig_pe = $dbh->{PrintError};
```

```
my $orig_ac = $dbh->{AutoCommit}; # 存储自动提交模式
```

```

$dbh->{RaiseError} = 1;           # 出错时抛出异常

$dbh->{PrintError} = 0;           # 但是不允许出现错误信
息

$dbh->{AutoCommit} = 0;           # 直到许可时才提交

eval

{

# 执行属于事务部分的语句

my $sth = $dbh->prepare (qq{

UPDATE score SET score = ?

WHERE event_id = ? AND student_id = ?

});

$sth->execute (13, 5, 8);

$sth->execute (18, 5, 9);

$dbh->commit();                   # 提交事务

};

if ($?)                           # 事务失败了吗?

{

print "A transaction error occurred: $@\n";

# 执行回滚，但是使用eval 来防止回滚失败

eval { $dbh->rollback (); }

```

```
}
```

```
$dbh->{AutoCommit} = $orig_ac;      # 恢复自动提交模式
```

```
$dbh->{RaiseError} = $orig_re;      # 恢复出错处理属性
```

```
$dbh->{PrintError} = $orig_pe;
```

其中，`eval`语句块负责执行事务，它的终止状态被保存在`$@`变量里。如果这些`UPDATE`语句和`commit()`方法的执行都没有出错，则`$@`为空。否则，`eval`语句块失败，`$@`将包含相应的出错信息。此时，紧跟在`eval`之后的那段代码会输出这条出错信息，并且会调用`rollback()`方法来取消事务。（为防止因回滚操作失败而导致脚本终止，我们把回滚操作单独放在了 `eval` 块里。）

在本章中，DBI 脚本所使用的出错处理模式基本上都启用了 `RaiseError`，但禁用了 `PrintError`。这意味着，它们已经有了执行事务所要求的那些值。因此，在上面示例里没有必要保存、设置和恢复那两个属性。不过，这是一种很有效的通用做法，即使在你不太确定出错处理属性的设置情况时，也可以这样做。

8.3 DBI脚本实践

此刻，你已经了解了DBI程序设计的许多概念，下面来实际处理一下我们的示例数据库。在第1章列出的众多目标当中，有几项是要求通过编写DBI脚本来实现的。

对于成绩考评项目，我们希望能够检索到任意给定考试或测验的分数。

对于美史联盟项目，我们想要实现以下几项任务。

按不同格式生成成员名录。我们想要的格式有两种。一种是只包含成员姓名的列表，用在联盟年会分发的议程里。另一种是可用于打印的成员名录。

把那些快要到期的成员查找出来，向他们发送电子邮件，通知他们更新成员资格。

编辑成员条目。（当他们更新成员资格时，我们还需要更新他们的到期日期。）

查找兴趣相投的成员。

把成员名录放到网上去。

在这些任务当中，有些可以通过编写在命令行运行的脚本来解决。而另外一些，则需要创建脚本，配合Web服务器一起来解决（参考8.4节）。其中的部分目标在本章的最后部分即可实现。所有的目标要等到第9章才能全部完成。

8.3.1 生成美史联盟成员名录

我们的目标之一是，根据美史联盟名录按不同的格式生成各种信息。最简单的格式是生成一份成员名单——它可用在打印的议程上，而这些议程会分发给那些出席联盟年会的人员。这种格式可以是一个简单的普通文本清单，作为用于创建会议议程的更大文档的一部分。我们只需要某些能直接粘贴进该文档的内容。

对于打印的成员名录，我们想要格式能更漂亮点，因此需要一种比普通文本格式更好的表示方法。比较合理的选择是RTF（Rich Text Format），这是一种由微软公司开发的格式，许多字处理软件都支持它。Word当然是其中一个，但是还有许多其他的软件，如OpenOffice，也支持这种格式。不同的字处理软件对RTF的支持程度会有所不同，但我们只会使用全套RTF技术规范的一个子集，任何支持RTF的程序都应该能正确读取它。例如，在Mac OS X系统上，TextEdit编辑器和Safari浏览器都可以正确读取我们生成的RTF输出。

生成年会人员名单（普通文本）和RTF格式名录的过程在本质上是一样的：先用一个查询命令检索各成员条目，然后再用一个循环获取和格式化每一个条目。既然它们基本相似，那么就没必要为每一种格式编写不同的脚本。最后，我们只编写一个脚本 `gen_dir.pl`，让它可以生成不同类型的输出。此脚本的构建步骤如下所示。

(1) 在输出成员条目之前，先初始化输出格式要求。对于普通文本格式的名单，不需要进行初始化。但是对于RTF版本，则必须写入某些初始控制语言。

(2) 获取各个条目，根据输出格式要求输出它。

(3) 在处理完各条目之后，执行某些必要的清理工作，然后终止。同样地，普通文本格式不需要特殊处理；但对于RTF版本，则需要写入某些结尾控制语言。

预计将来还可能需要利用这个脚本来生成其他格式的输出，因此我们需要增强它的扩展性，方法是建立一个“开关盒”。这种“开关盒”其实是一个散列，其中的每一个元素对应着一种输出格式。对于某种给定的格式，其中的每个元素规定了，要调用哪些函数去完成各个输出的生成阶段：一个初始化函数、一个条目输出函数和一个清理函数。

```
# 包含每一种输出格式的格式化函数的开关盒

my %switchbox =

(

"text" =>                                # 普通文本格式的处理函数

{

"init"      => undef,          # 不需要初始化

"entry"     => \&text_format_entry,

"cleanup"   => undef          # 不需要进行清理

},

"rtf" =>                                # RTF 格式的处理函数

{
```

```

"init"      => \&rtf_init,
"entry"     => \&rtf_format_entry,
"cleanup"   => \&rtf_cleanup

    }

);

```

这个开关盒的每一个元素都是以格式名称（"text"或"rtf"）作为键。我们编写的脚本要能够处理用于表明所要格式的命令行参数：

```

% ./gen_dir.pl text

% ./gen_dir.pl rtf

```

照此方式建立开关盒之后，可以很容易地根据需要添加对新格式的支持。

(1) 针对输出生成的不同阶段编写3个格式化函数。

(2) 往开关盒里增加一个新元素，定义一个格式名称，并指向输出函数。

(3) 当需要按新格式生成输出时，可以执行gen_dir.pl，同时在命令行上指定那个新格式名称。

下面这段代码将根据命令行上的第一个参数，选择正确的开关盒条目。如果在命令行没有给出格式名称，或者给出了无效的格式名称，那么此脚本将生成一条出错信息，并会显示出所有可用的名称。如果给出格式名称有效，那么\$func_hashref将被设置为相应的开关盒条目：

```

my $formats = join (" ", sort (keys (%switchbox)));

# 确保命令行指定了一个参数

```



```
@ARGV == 1
```

```
or die "Usage: gen_dir.pl format_type\nAllowable formats:\n$formats\n";
```

```
# 根据命令行参数确定正确的开关盒条目;
```

```
# 如果未找到条目, 则指定的格式类型无效
```

```
my $func_hashref = $switchbox{$ARGV[0]};
```

```
defined ($func_hashref)
```

```
or die "Unknown format: $ARGV[0]\nAllowable formats:\n$formats\n";
```

这段格式选择代码基于这样一个事实, 即输出格式名都是%switchbox 散列里的键。对于每一个有效的格式名称, 对应的开关盒条目都会指向输出函数。对于无效的格式名称, 则不会有任何条目存在。这样便可以不必把格式名称硬编码在格式选择代码里, 从而在往这个开关盒里添加新条目时, 代码无需更改, 便能自动检测它。

当在命令行上指定的格式名称有效时, 上面这段代码会设置\$func_hashref。这个变量的值是一个散列引用, 它指向的是针对所选格式的输出函数。下面这段代码将使用它来调用初始化函数, 然后获取并输出条目, 最后调用清除函数:

```
# 如果有初始化函数, 则调用它
```

```
&{$func_hashref->{init}} if defined ($func_hashref->{init});
```

```
# 如果有条目格式化函数, 则获取并输出条目
```

```
if (defined ($func_hashref->{entry}))
```

```
{
```

```

my $sth = $dbh->prepare (qq{
SELECT * FROM member ORDER BY last_name, first_name
});

$sth->execute ();

while (my $entry_ref = $sth->fetchrow_hashref ("NAME_lc"))
{
# 通过对格式化函数的引用传递条目
&{$func_hashref->{entry}} ($entry_ref);
}
}

# 如果有清理函数，则调用它

&{$func_hashref->{cleanup}} if defined ($func_hashref->{cleanup});

```

这个条目获取循环使用`fetchrow_hashref()`是有原因的。如果这个循环获取了一个数组，那么每一个格式函数就必须要知道各个列的顺序。不难想象，通过访问`$sth->{NAME}`属性（它包含列名的顺序与它们返回这些列的顺序一样）也可以达到目的，可为何要多费周折呢？因为使用散列引用之后，每个格式函数都可以利用`$entry_ref->{col_name}`语法，通过指定列名来获得它想要的列值。

现在剩下的工作就是，为每一种输出格式编写函数，并用开关盒的条目来命名函数。

8.3.1.1 生成普通文本格式的成员名单

文本输出格式不需要进行初始化或清理调用。我们只需要一个条目格式化函数`text_format_entry()`，它会接受一个对成员条目的引

用，并输出成员姓名。输出成员的棘手部分是要处理成员姓名的后缀部分。像“Jr.”或“Sr.”这样的姓名后缀，其前面要有一个逗号和一个空格；而对于像“II”或“III”这样的姓名后缀，其前面却只有一个空格：

Michael Alvis IV

Clarence Elgar, Jr.

Bill Matthews, Sr.

Mark York II

字母“I”、“V”、“X”是仅有的表示辈分的罗马数字，它们可表示第1代到第39代。我们不可能使用超出这个范围的任何数字，因此我们可以根据下面这个模式，通过对姓名后缀的检查来决定是否需要加上逗号：

```
/^[IVX]+$/
```

`text_format_entry()` 函数会负责把人名的各个部分按适当顺序组织在一起，函数中的代码在后面生成RTF格式的成员名录时还会用到。因此，为了避免重复编写`rtf_format_entry()`里的代码，我们可以把它单独写成一个辅助函数：

```
sub format_name
{
    my $entry_ref = shift;

    my $name = $entry_ref->{first_name} ." " . $entry_ref->{last_name};

    if (defined ($entry_ref->{suffix}))      # 有一个姓名后缀
    {
```

```
# 像I、II、III 等这样的姓名后缀前面没有逗号

$name .= ", " unless $entry_ref->{suffix} =~ /^[IVX]+$;/

$name .= " " . $entry_ref->{suffix}

}

return ($name);

}
```

有了 `format_name()` 函数之后，实现用于输出条目的 `text_format_entry()` 函数，就变成一件很容易做到的事情了：

```
sub text_format_entry

{

    printf "%s\n", format_name ($_[0]);

}
```

8.3.1.2 生成RTF格式的成员名录

与生成用于年会议程的成员列表相比，生成RTF版本的成员名录则要复杂一些。原因之一是，每个条目都需要输出更多的信息。原因之二是，为实现格式效果，需要在每个条目上都加一些RTF控制语言，并且在文档的开头和结尾都要加上某些控制语言。最小的RTF文档框架如下所示：

```
{\rtf0

{\fonttbl {\f0 Times;}}

\plain \f0 \fs24
```

...文档内容从此处开始...

```
}
```

文档以花括号 “{” 和 “}” 作为开始和结束。RTF的关键字都以 “\” 作为开头，并且文档的第一个关键字必须是\rtfn，其中，n 是该文档所使用的RTF规范的版本号。版本0已足以满足我们的需要。

在这个文档内，我们还需要指定一个字体表，用以表明各个条目所使用的字体。字体表信息列在一组花括号里，其中包含打头的关键字\fonttbl和字体信息。上面显示的这段代码框架，把0号字体定义成Times字体。（我们只需要一种字体，但是如果你想把文档弄得更花哨一点，可以增加更多字体信息。）

接下来的几条指令用于设置默认的格式风格：\plain表示选择普通格式；\f0表示选择0号字体（在字体表里它被定义为Times字体）；\fs24表示把字体大小设置为12点（紧跟在\fs后面的数字表示大小单位为半个点）。没必要设置页边距，因为大多数的字处理软件都能提供一组合理的默认值。

初始化函数和清理函数负责生成文档框架，如下所示（请注意，为了在输出结果里获得反斜线 “\”，需要在代码里双写反斜线，即使使用 “\\” ）：

```
sub rtf_init

{

print "{\\rtf0\n";

print "{\\fonttbl {\\f0 Times;}}\n";

print "\\plain \\f0 \\fs24\n";

}

sub rtf_cleanup

{
```

```
print "}\n";  
  
}
```

那个条目格式化函数负责生成文档的内容。我们采取了一种非常简单的方法，即把每一个条目输出成一系列的行，并给每一行加上一个标签。如果对应于某个输出行的信息缺失，则忽略这一行。例如，某位成员没有电子邮件地址，则不需要输出“Email:”行。由于有些输出行，如“Address:”行，是由多个列（street、city、state、zip）里的信息构成，因此这个脚本还必须处理各种数据值缺失的情况。下面是输出格式的一个样例：

Name: Mike Artel

Address: 4264 Lovering Rd., Miami, FL 12777

Telephone: 075-961-0712

Email: mike_artel@venus.org

Interests: Civil Rights,Education,Revolutionary War

对于这个条目，其RTF表示形式如下所示：

\b Name: Mike Artel\b0\par

Address: 4264 Lovering Rd., Miami, FL 12777\par

Telephone: 075-961-0712\par

Email: mike_artel@venus.org\par

Interests: Civil Rights,Education,Revolutionary War\par

如果想让“Name:”行以粗体显示，那么需要让它以\b加空格作为开头，表示粗体显示开始；并以\b0 作为结尾，表示粗体显示结束。这里的成员姓名是用8.3.1.1节所展示的那个format_name()函数来格式化的。每一行的末尾都有一个段落标记（\par），用于告诉字处理

软件移到下一行——没有太复杂的内容。主要的难点在于，格式化地址串和确定哪些行需要输出：

```
sub rtf_format_entry

{

my $entry_ref = shift;

printf "\\b Name: %s\\b0\\par\\n", format_name ($entry_ref);

my $address = "";

$address .= $entry_ref->{street}

if defined ($entry_ref->{street});

$address .= ", " . $entry_ref->{city}

if defined ($entry_ref->{city});

$address .= ", " . $entry_ref->{state}

if defined ($entry_ref->{state});

$address .= " " . $entry_ref->{zip}

if defined ($entry_ref->{zip});

print "Address: $address\\par\\n"

if $address ne "";

print "Telephone: $entry_ref->{phone}\\par\\n"

if defined ($entry_ref->{phone});

print "Email: $entry_ref->{email}\\par\\n"
```

```

if defined ($entry_ref->{email});

    print "Interests: $entry_ref->{interests}\\\par\n"

if defined ($entry_ref->{interests});

    print "\\par\n";

}

```

当然，你不用拘泥于这种特殊的格式化样式。如果要更改这个输出名录的样式，只需简单地更改 `rtf_format_entry()` 即可。如果这个名录是以原始形式（即字处理文档）提供的，事情就没那么容易处理了。

到此，`gen_dir.pl` 脚本就算是大功告成了，你通过运行下面这些命令，生成普通文本格式或RTF格式的成员名录：

```

% ./gen_dir.pl text > names.txt

% ./gen_dir.pl rtf > directory.rtf

```

此时，无论是把那个普通文本姓名文件粘贴到年会议程文档里，还是把那个RTF文件与某个支持RTF的程序一起使用，都只有一步之遥。

DBI使得从 MySQL提取我们想要的信息变得简单易行，并且 Perl 语言的文本处理能力也大大简化了我们格式化这些信息的工作。尽管 MySQL 没有提供花哨的格式化输出功能，但是没有关系，因为你可以轻易地把 MySQL的数据库处理能力，集成到像 Perl那样的语言里，充分利用这类语言的出色文本处理能力。

8.3.2 发送成员资格更新通知

在使用原始形式（即字处理文档）维护美史联盟成员名录时，想要查出需要通知哪些成员延长自己的成员资格，是项既费时、又易出错的工作。既然已把这些信息放到了数据库里，那么便有可能实现自动发送成员资格更新通知。具体可以这样做，先标识需要更新的成

员，然后再通过电子邮件给他们发送通知，以便我们不用通过电话或者信件去联系他们。

我们需要做的是，确定哪些成员资格已过期，以及哪些成员资格需要在指定的日期内更新。这种查询涉及一个相对简单的日期计算：

```
SELECT ...FROM member

WHERE expiration < DATE_ADD(CURDATE(), INTERVAL cutoff
DAY)
```

其中，cutoff表示宽限天数。这个查询将把那些应该在这个宽限期内续期（或者已过期）的成员记录查询出来。如果想要知道哪些成员的资格已过期，只需把宽限期设置为0即可。

在标识那些需要发送通知的行之后，下一步该怎么办呢？一种方案是，直接用同一个脚本发送电子邮件，但是如果能在发送通知之前，先检查一下列表，则可能会更实用。为此，我们分两步来完成。

(1) 运行need_renewal.pl脚本，生成一个需要更新的成员列表。你可以通过检查这个列表来完成对它的核对或编辑工作，然后把它用作下一步（即发送更新通知）的输入内容。

(2) 运行 renewal_notify.pl 脚本，让它通过电子邮件向成员发送一个“请更新”的通知。如果某些成员没有电子邮件地址，那么脚本应该给出警告，以便让你可以采用其他方式联系这些成员。

任务第一步，need_renewal.pl脚本必须标识出哪些成员需要更新。此脚本的主要代码如下所示：

```
# 默认30天的宽限时间...
```

```
my $cutoff = 30;
```

```
# ...不过，如果在命令行给出了某个数字参数，则可以重置它
```

```
$cutoff = shift (@ARGV) if @ARGV && $ARGV[0] =~ /^\\d+$/;
```

```

# 提醒用户，本脚本所设置的宽限期限是多少

warn "Using cutoff of $cutoff days\n";

my $sth = $dbh->prepare (qq{

SELECT

member_id, email, last_name, first_name, expiration,

TO_DAYS(expiration) - TO_DAYS(CURDATE()) AS days

FROM member

WHERE expiration < DATE_ADD(CURDATE(), INTERVAL ? DAY)

ORDER BY expiration, last_name, first_name

});

$sth->execute ($cutoff);    # 传递宽限期限

while (my $entry_ref = $sth->fetchrow_hashref ())

{

    # 将undef 值转换为空串，以便于输出

    foreach my $key (keys (%{$entry_ref}))

    {

        $entry_ref->{$key} = "" if !defined ($entry_ref->{$key});

    }

    print join ("\t",

$entry_ref->{member_id},

```

```

$entry_ref->{email},
$entry_ref->{last_name},
$entry_ref->{first_name},
$entry_ref->{expiration},
$entry_ref->{days} ." days"),
"\n";
}

```

need_renewal.pl 脚本的输出如下所示（你看到的输出也许会有所不同，因为结果都是依据当前日期来确定的，你阅读本章时的日期与我撰写本章时的日期肯定不一样）：

```

89   g.steve@pluto.com           Garner   Steve    2012-08-03
-38 days

18   york_mark@earth.com        York     Mark     2012-08-24
-17 days

82   john_edwards@venus.org     Edwards John    2012-09-12   2
days

```

可以看到，有些成员资格的有效天数为负数。这意味着，他们的成员资格已过期！（当你采用手工方式维护成员记录时，常会出现这种情况；人们会钻漏洞。既然数据库里有这些信息，我们就能找出之前漏掉的少许人。）

成员更新通知任务的第二步是，调用renewal_notify.pl 脚本，以电子邮件的方式发送这些通知。为了让renewal_notify.pl脚本的使用能更容易一点，我们可以让它接受三种命令行参数：成员ID编号、电子邮件地址和文件名。数字参数用于表示成员ID值，而包含“@”字符的参数用于表示电子邮件地址。其他情况将被当作文件名，通过这个文件可以读取到ID编号或电子邮件地址。这种方法让你可以

通过ID编号或电子邮件地址来指定成员，并且可以在命令行上直接指定，也可以把它们列在文件里。（特别地，你可以把need_renewal.pl的输出保存在一个文件里，然后用这个文件当作renewal_notify.pl的输入。）

对于每一位需要收到通知的成员，renewal_notify.pl脚本将会在member表里查找出相应的条目，提取邮件地址，并向该地址发送邮件。如果这个条目没有包含邮件地址，那么renewal_notify.pl将会生成一条警告信息，提醒你需要通过其他手段与这位成员进行联系。

下面是参数处理主循环。如果在命令行没有指定任何参数，那么此脚本将从标准输入设备上读取输入。如果指定了参数，那么这些参数将被逐个传递给interpret_argument()，从而确定出它是ID编号、电子邮件地址，还是文件名：

```
if (@ARGV == 0)    # 如果没指定参数，则从STDIN 读取各个值

{

    read_file (\*STDIN);

}

else

{

    while (my $arg = shift (@ARGV))

    {

        # 对参数进行解释，同时递归处理文件名

        interpret_argument ($arg, 1);

    }

}
```

`read_file()` 函数负责读取文件的内容（假设该文件已打开），并查看每一行的第一个字段。（如果我们把 `need_renewal.pl` 的输出提供给 `renewal_notify.pl` 作为输入，那么每一行便会有多个字段。但是我们只需要第一个字段，因为它就是成员ID编号。）

```
sub read_file

{

my $fh = shift;    # 已打开文件的句柄

my $arg;

while (defined ($arg = <$fh>))

{

# 去除第1个列以外的所有内容，包括换行符

$arg =~ s/\s.*//s;

# 对参数进行解释，同时不再递归处理文件名

interpret_argument ($arg, 0);

}

}
```

`interpret_argument()` 函数会对每一个参数加以区分，确定它是ID编号、电子邮件地址，还是文件名。对于ID编号或电子邮件地址，它会查出相应的成员条目，并将其传递给 `notify_member()`。对于那些以电子邮件地址指定的成员，我们需要加以注意。有可能出现两位成员使用同一个电子邮件地址的情况（如他们是一对夫妻），我们不想把更新通知发给无关的人。为避免出现这种情况，我们需要根据电子邮件地址查找与之对应的ID编号，并确保恰好只有一个编号存在。如果某个地址匹配到了多个ID编号，那么这会产生歧义，此时，我们会输出一条警告信息，然后忽略它。

如果某个参数既不像ID编号，也不像电子邮件地址，那么我们将把它视为文件名，并继续从中读取输入数据。此处也必须多加小心，为避免陷入无限循环，如果我们已在读取某个文件，则不应该再去读取另一个文件。

```
sub interpret_argument

{

my ($arg, $recurse) = @_;

if ($arg =~ /^\\d+$/)      # 数字型的成员ID

{

    notify_member ($arg);

}

elsif ($arg =~ /@/)      # 电子邮件地址

{

    # 获得与地址相关联的member_id

    # （理论上应该只有一个）

    my $stmt = qq{ SELECT member_id FROM member WHERE email =
? };

    my $ary_ref = $dbh->selectcol_arrayref ($stmt, undef,
$arg);

    if (scalar (@{$ary_ref}) == 0)

    {

        warn "Email address $arg matches no entry: ignored\\n";

    }

}
```

```
}

elsif (scalar (@{$ary_ref}) > 1)

{

warn "Email address $arg matches multiple entries: ignored\n";

}

else

{

notify_member ($ary_ref->[0]);

}

}

else                                     # 文件名

{

if (!$recurse)

{

warn "filename $arg inside file: ignored\n";

}

else

{

open (IN, $arg) or die "Cannot open $arg: $!\n";
```

```

read_file (\*IN);

close (IN);

}

}

}

```

`notify_member()` 函数负责实际发送更新通知。如果某位成员没有电子邮件地址，那么`notify_member()` 函数不会发送任何消息，而只会输出一条警告信息，提醒你需要采用其他方式去联系这位成员。利用这种信息中显示的成员ID编号执行`show_member.pl`脚本，便可以查看到成员的完整资料——例如，可以从中找到成员的电话号码和住址。（关于`show_member.pl`的描述请参考8.2.11节。）

下面是`notify_member()` 的全部代码：

```

sub notify_member

{

my $member_id = shift;

warn "Notifying $member_id...\n";

my $stmt = qq{ SELECT * FROM member WHERE member_id = ? };

my $sth = $dbh->prepare ($stmt);

$sth->execute ($member_id);

my @col_name = @{$sth->{NAME}};

my $entry_ref = $sth->fetchrow_hashref ();

```



```

$sth->finish ();

if (!$entry_ref)                                     # 未找到成员!

{

warn "NO ENTRY found for member $member_id!\n";

return;

}

if (!defined ($entry_ref->{email}))                  # 成员条目里没有电子邮件地
址

{

warn "Member $member_id has no email address; no message
was sent\n";

return;

}

open (OUT, "| $sendmail") or die "Cannot open mailer\n";

print OUT <<EOF;

To: $entry_ref->{email}

Subject: Your USHL membership is in need of renewal

Greetings.    Your membership in the U.S.Historical League is

due to expire soon.    We hope that you'll take a few minutes to

contact the League office to renew your membership.    The

```

contents of your member entry are shown below. Please note

particularly the expiration date.

Thank you.

EOF

```
foreach my $col_name (@col_name)

{

    printf OUT "$col_name:";

    printf OUT " $entry_ref->{$col_name} "

if defined ($entry_ref->{$col_name});

    printf OUT "\n";

}

close (OUT);

}
```

notify_member() 函数发送电子邮件的办法是：打开一个连通 sendmail 程序的管道，然后把邮件信息塞入这个管道。在 renewal_notify.pl 脚本的开头部分，有一个参数设置了 sendmail 程序的路径名。因为 sendmail 程序的位置会因系统的不同而有所差异，所以你可能需要修改这个路径：

根据你的系统修改下面的路径

```
my $sendmail = "/usr/sbin/sendmail -t -oi";
```

如果你的系统上没有 sendmail 程序，那么这个脚本将无法正确工作。（例如，Windows 系统通常不会安装 sendmail 程序。）为应对此

种情况，sampdb 发行版包含了一个renewal_notify.pl脚本的修改版——renewal_notify2.pl，它使用了Mail::sendmail模块，可以保证在没有sendmail程序的情况下也可以发送邮件。如果你安装了Mail::sendmail模块，就可以使用renewal_notify2.pl来代替。

你还可以进一步优化这个脚本。例如，可以在member 表里增加一列，用于记录最近一次发送更新通知的具体日期，然后修改renewal_notify.pl脚本，让它在发送邮件时更新该列。这么做，有助于保证你不会过于频繁地发送通知。其实，我们这里会假设你运行这个脚本的频率不会超过每月一次。

现在，这两个脚本都已创建完成。你可以按以下步骤来使用它们。

(1) 运行need_renewal.pl脚本，生成一个成员列表，其中每个成员的资格都已过期或将要过期：

```
% ./need_renewal.pl > tmp
```

(2) 检查tmp，看是否存在不合理的地方。

(3) 如果无误，则把它输入renewal_notify.pl脚本，发送更新通知：

```
% ./renewal_notify.pl tmp
```

如果想单独通知个别成员，可以利用他们的ID编号或电子邮件地址来指定它们。例如，下面的命令将通知ID编号为18的成员，以及电子邮件地址为g.steve@pluto.com的成员。

```
% ./renewal_notify.pl 18 g.steve@pluto.com
```

8.3.3 编辑美史联盟成员条目

在开始发送更新通知之后，肯定会有一些收到通知的成员来更新他们的成员资格。如果有人更新，我们就需要用新的到期日期来更新他们的条目。在下一章里，我们将设法利用Web来编辑成员信息。但是本节将展示如何编写命令行脚本 edit_member.pl，让你能用一种简单

的方式，即按提示输入成员条目的各个新值，来更新成员条目。其工作原理如下所示。

如果在调用`edit_member.pl`时未在命令行上指定任何参数，那么它会假设你想要输入一个新成员。于是，它会提示你输入必要的信息，并创建一个新条目。

如果在调用`edit_member.pl`时在命令行指定了成员ID编号，那么它会查找该成员条目的现有内容，然后提示你对每一列进行更新。如果你输入了列值，那么它会把该列的当前值替换掉。如果直接按下Enter键，那么该列的值不会发生变化。如果输入了单词“none”，则会清除该列的当前值。如果输入了单词“Exit”，则会直接退出，不创建此条目。（如果你不知道成员的ID编号，那么可以执行`show_member.pl last_name`命令，查看那些与给定姓相匹配的条目，并以此确定正确的ID编号值。）

如果只是想更新成员的资格到期日期，那么像这样编辑整个条目，可能就有些小题大做了。但从另一个角度来讲，像这样的脚本也提供了一种简单而通用的解决方案，让不懂SQL的人也能修改条目的各项内容。（有一个例外，即`edit_member.pl`不会允许你修改`member_id`字段，因为这是脚本创建条目时自动生成的，之后不应该被更改。）

`edit_member.pl`脚本最先要做的事情就是，了解`member`表里的列名，以及它们是否可以赋值为NULL。在清除列值时，会用到后面那个特性（如果列允许使用NULL值，则把它赋值为NULL；否则，赋值为一个空串）。从 `INFORMATION_SCHEMA` 数据库的 `COLUMNS` 表里，可以查到所需要的这些信息：

```
my @col_name = ();          # 列名数组

my %nullable = ();          # 列是否允许为(), 以列名为键

# 获得member表的所有列名

my $sth = $dbh->prepare (qq{

SELECT COLUMN_NAME, UPPER(IS_NULLABLE)
```

```
FROM INFORMATION_SCHEMA.COLUMNS  
WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ?  
));
```

```
$sth->execute ("sampdb", "member");  
  
while (my ($col_name, $is_nullable) = $sth->fetchrow_array ())  
{  
  
    push (@col_name, $col_name);  
  
    $nullable{$col_name} = ($is_nullable eq "YES");  
  
}
```

在得到列信息之后，这个脚本会生成一个数组，依次包含各个列的名字；并且，还会生成一个以列名为键的散列，用以表明各个列是否允许赋值为NULL。然后，`edit_member.pl`进入主循环：

```
if (@ARGV == 0) # 如果未指定参数，则创建新条目  
{  
  
    # 传递对列名数组的引用  
  
    new_member (\@col_name);  
  
}  
  
else # 否则，把参数当作成员ID，编辑各个条目  
{
```

把@ARGV 保存起来，然后清空它，以便脚本在从STDIN 读取输入时，

不会把@ARGV的内容解释为文件名

```
my @id = @ARGV;
```

```
@ARGV = ();
```

对于每一个ID 值，查找相应的条目，然后编辑它

```
while (my $id = shift (@id))
```

```
{
```

```
    $sth = $dbh->prepare (qq{
```

```
SELECT * FROM member WHERE member_id = ?
```

```
});
```

```
    $sth->execute ($id);
```

```
    my $entry_ref = $sth->fetchrow_hashref ();
```

```
    $sth->finish ();
```

```
    if (!$entry_ref)
```

```
    {
```

```
        warn "No member exists with member ID = $id\n";
```

```
    next;
```

```
    }
```

传递列名数组引用和条目引用

```

edit_member (\@col_name, $entry_ref);

}

}

```

下面是用来创建新成员条目的代码，它会提示你输入 member 表中各列的值，然后执行INSERT语句来插入新行：

```

sub new_member

{

my $col_name_ref = shift; # 对列名数组的引用

my $entry_ref = { };      # 把新条目创建为一个散列

return unless prompt ("Create new entry (y/n)? ") =~ /^y/i;

```

提示输入新值。用户可以选择：输入新值；或者按Enter键以保留原值；

```

# 输入"NONE"来清除原值；或者输入"EXIT"来退出编辑，

```

```

# 同时不创建新记录。

```

```

foreach my $col_name (@{$col_name_ref})

```

```

{

```

```

    next if $col_name eq "member_id";      # 跳过关键字字段

```

```

    my $col_val = col_prompt ($col_name, undef);

```

```

    next if $col_val eq "";                  # 用户按下了Enter

```

键

```

return if uc ($col_val) eq "EXIT";    # 提前退出

if (uc ($col_val) eq "NONE")

{

# 如果列可以为空，则输入NULL；否则，输入空串

$col_val = ($nullable{$col_name} ? undef : "");

}

$entry_ref->{$col_name} = $col_val;

}

# 显示各个值，并在正式插入条目之前要求确认

show_member ($col_name_ref, $entry_ref);

return unless prompt ("\nInsert this entry (y/n)? ") =~ /^y/i;

# 构造一条INSERT 查询语句，然后执行它。

my $stmt = "INSERT INTO member";

my $delim = " SET "; # 在第1 列之前放置"SET"，在其他列之前放置", "

foreach my $col_name (@{$col_name_ref})

{

# 仅为事先给出的列指定具体值

next if !defined ($entry_ref->{$col_name});

# quote() 会把undef 转换为单词NULL（注意，没有引号），

```


这正是我们想要的。

那些为NOT NULL的值会被赋成它们的默认值。

```
$stmt .= sprintf ("%s %s=%s", $delim, $col_name,  
$dbh->quote ($entry_ref->{$col_name}));  
  
$delim = ", ";  
  
}  
  
$dbh->do ($stmt) or warn "Warning: new entry not created!\n"  
  
}
```

edit_member.pl脚本使用了两个例程来显示用户提示信息。
prompt()函数会先提问，然后再返回答案：

```
sub prompt  
  
{  
  
my $str = shift;  
  
print STDERR $str;  
  
chomp ($str = <STDIN>);  
  
return ($str);  
  
}
```

col_prompt()函数可以接受一个列名作为其参数。它会显示这个
列名，提示输入新值，并会返回用户输入的那个值：

```
sub col_prompt
```

```

{

my ($col_name, $entry_ref) = @_;

my $prompt = $col_name;

if (defined ($entry_ref))

{

my $cur_val = $entry_ref->{$col_name};

$cur_val = "NULL" if !defined ($cur_val);

$prompt .= " [$cur_val]";

}

$prompt .= ": ";

print STDERR $prompt;

my $str = <STDIN>;

chomp ($str);

return ($str);

}

```

`col_prompt()` 函数的第二个参数是一个散列引用，表示的是成员条目。在创建新条目时，这个值为 `undef`，但在编辑已有行时，它会指向当前条目的内容。对于编辑已有行的情况，`col_prompt()` 函数会在提示字符串里包含当前列值，以便用户可以看到它。用户直接按下 Enter 键，即可接受这个值。

用于编辑已有成员信息的代码，与用于创建新成员的代码相似。不过，由于我们有要处理的条目，因此提示例程会显示条目的各个当

前值，`edit_member()` 函数会调用UPDATE 语句，而不是INSERT：

```
sub edit_member

{

# 列名数组引用和条目散列引用

my ($col_name_ref, $entry_ref) = @_;

# 显示初始值，并询问是继续还是编辑

show_member ($col_name_ref, $entry_ref);

return unless prompt ("\nEdit this entry (y/n)? ") =~ /^y/i;

# 提示输入新值。用户可以选择：输入新值；或者按Enter 键以保留原值；

# 输入"NONE"来清除原值；或者输入"EXIT"来退出编辑，

# 同时不创建新记录。
```

```

foreach my $col_name (@{$col_name_ref})
{
    next if $col_name eq "member_id";    # 跳过关键字字段
    my $col_val = col_prompt ($col_name, $entry_ref);
    next if $col_val eq "";              # 用户按下了Enter
键
    return if uc ($col_val) eq "EXIT";   # 提前退出
    if (uc ($col_val) eq "NONE")
    {
        # 如果列可以为空，则输入NULL；否则，输入空串
        $col_val = ($nullable{$col_name} ? undef : "");
    }
    $entry_ref->{$col_name} = $col_val;
}

# 显示新值，并在更新条目之前要求确认
show_member ($col_name_ref, $entry_ref);

return unless prompt ("\nUpdate this entry (y/n)? ") =~ /^y/i;

# 构造一条UPDATE 查询语句，然后执行它。
my $stmt = "UPDATE member";

```

```
my $delim = " SET "; # 在第1 列之前放置"SET", 在其他列之前放置","
```

```
foreach my $col_name (@{$col_name_ref})  
{  
    next if $col_name eq "member_id"; # 跳过关键字字段  
  
    # quote()会把undef 转换为单词NULL (注意, 没有引号) ,  
    # 这正是我们想要的。  
  
    $stmt .= sprintf ("%s %s=%s", $delim, $col_name,  
$dbh->quote ($entry_ref->{$col_name}));  
  
    $delim = ",";  
  
}  
  
$stmt .= " WHERE member_id = " . $dbh->quote ($entry_ref->  
{member_id});  
  
$dbh->do ($stmt) or warn "Warning: entry not undated!\n"  
  
}
```

edit_member.pl 脚本的一个不足之处在于, 它没有验证输入值。member 表里的大部分字段, 都没有多少需要验证的, 因为它们都只是一些字符串字段。但是成员资格到期日期列的输入值, 真的应该验证, 要保证它们看起来像日期。在具有通用性的数据录入程序里, 你可能会想要获得与表有关的信息, 并以此来确定它的各个列的类型。然后, 你便可以根据这些类型来实现输入验证功能。这些内容已超出此处所要讨论的范围, 不过还是可以往 col_prompt() 函数里添加一个简单的日期检查功能, 以检查 expiration 列的输入值格式:

```

sub col_prompt

{

my ($col_name, $entry_ref) = @_;

loop:

my $prompt = $col_name;

if (defined ($entry_ref))

{

my $cur_val = $entry_ref->{$col_name};

$cur_val = "NULL" if !defined ($cur_val);

$prompt .= " [$cur_val]";

}

$prompt .= ": ";

print STDERR $prompt;

my $str = <STDIN>;

chomp ($str);

# 对到期日期进行基本检查

if ($str && $col_name eq "expiration")    # 检查到期日期的格式

{

if ($str !~ /\d+\D\d+\D\d+$/ )

```

```

{

warn "$str is not a legal date, try again\n";

goto loop;

}

}

return ($str);

}

```

上面代码里的模式，会测试 3组由非数字字符分隔的数字序列。这只是个部分检查，因为它会把"1999-14-92"之类的值判定为合法值。如果想要改进这个脚本，那么可以为它提供更严格的日期检查，或者其他检查，如要求名和姓字段的值都不能为空。

对此脚本的其他可能改进如下所示。

如果用户未对某个已有条目做任何修改，则跳过此次更新操作。为实现这一点，需要把各成员条目列的原始值保存起来，然后编写 UPDATE语句，让它只更新那些被用户更改过的列。如果没有做任何更改，则完全无需执行该语句。

当用户正在编辑某行时，如果另外有人更改了它，则需要及时通知用户。为实现这一点，可以在WHERE 子句里为各列的原始值加上 AND col_name = col_value。这样，一旦有人更改了正在编辑的行，那么UPDATE语句便会执行失败，由此表明有两个人正在试图更改这个条目。

启用SQL的严格模式和其他输入限制，这样可以让MySQL本身来拒绝接受不符合要求的数据值，并且在输入数据无法使用时返回错误消息：

```
$dbh->do ("SET sql_mode = 'TRADITIONAL'");
```

`edit_member.pl`脚本还有另一个不足之处，你可能考虑到了如何应付，这个不足就是：如上面代码所示，它会在进入提示循环之前打开数据库连接，然后直到循环输出该行之后才关闭。如果用户在输入或更新行时花费了很长时间，或者碰巧有其他事情需要处理，那么这个连接将在很长一段时间内一直处于打开状态。你要如何修改 `edit_member.pl` 脚本，让它只在需要时才保持与数据库的连接打开呢？

8.3.4 寻找志趣相投的联盟成员

美史联盟秘书的一大职责是，根据成员请求，列出对美国历史事件（如大萧条时期或亚伯拉罕·林肯总统的生平）有着共同兴趣的其他成员名单。当成员名录是以字处理文档格式进行维护时，可以利用字处理软件提供的“查找”功能，因此找出这些成员的工作还不算太难。不过，如果想要生成一份仅由符合条件的那些成员条目所构成的名单，就没那么容易了，因为它会涉及大量的复制和粘贴操作。在有了 MySQL 之后，这项工作就变得简单多了，我们只需执行类似下面这样的查询即可：

```
SELECT * FROM member WHERE interests LIKE '%lincoln%'

ORDER BY last_name, first_name
```

不幸的是，如果在mysql客户端程序里执行这个查询，那么结果并不怎么好看。下面，我们组织一个小的DBI 脚本 `interests.pl`，用于完成搜索和美化输出报告的工作。首先，`interests.pl` 会检查确认我们在命令行至少给出了一个参数，因为没有参数的话，就没什么可查的了。然后，这个脚本会针对每个参数去搜索member表的interests列：

```
@ARGV or die "Usage: interests.pl keyword\n";

search_members (shift (@ARGV)) while @ARGV;
```

在搜索关键字字符串时，首先在它的两端分别加上一个通配符“%”，完成一个模式匹配，以便可以在interests列里的任何位置找到这个字符串。然后，输出匹配到的条目：


```

sub search_members

{

my $interest = shift;

print "Search results for keyword: $interest\n\n";

my $sth = $dbh->prepare (qq{

SELECT * FROM member WHERE interests LIKE ?

ORDER BY last_name, first_name

});

# 在interest 字段的任何位置查找字符串

$sth->execute ("%". $interest ."%");

my $count = 0;

while (my $hash_ref = $sth->fetchrow_hashref ())

{

format_entry ($hash_ref);

++$count;

}

print "Number of matching entries: $count\n\n";

}

```

`format_entry()` 函数负责把条目转换为其可打印的表示形式。这里没有显示这个函数，因为它与`gen_dir.pl`脚本里的

rtf_format_entry() 函数基本一致，只要去掉其中的RTF控制字即可。查看sampdb发行版里的interests.pl脚本，即可看到它的具体实现。

8.3.5 把美史联盟成员名录放到网上

在8.4节里，我们将开始编写这样的脚本：连接MySQL服务器来提取信息，并且把这些信息以网页的形式显示在客户端的Web浏览器里。那些脚本将根据客户请求动态生成HTML文档。在那之前，我们先来编写一个DBI脚本，用它来生成一个可被加载到Web服务器的文档树里的静态HTML文档，从而加深对HTML的了解。一个很好的候选任务就是，生成一份HTML格式的美史联盟成员名录（别忘了，把成员名录放到网上是我们的目标之一）。

一个简单的HTML文档结构如下所示：

<html>	← 文档起始标记
<head>	← 文档头起始标记
<title>My Page Title</title>	← 文档标题
</head>	← 文档头结束标记
<body bgcolor="white">	← 文档主体起始标记（白色背景）
My Level 1 Heading	← 一级标题
...文档主体内容 ...	
</body>	← 文档主体结束标记
</html>	← 文档结束标记

按HTML格式生成成员名录并不需要从头编写脚本。别忘了，我们此前编写过gen_dir.pl脚本。当时，我们使用了一个可扩展的框架，因此完全可以往它里面插入按其他格式生成成员名录的代码。现在，我们一起来利用这种可扩展性所带来的便利，添加一段可以输出HTML格式的代码。为此，我们需要对gen_dir.pl脚本做以下修改。

编写新文档初始化函数和清理函数。

编写一个用于格式化各成员行的函数。

增加一个新的开关盒元素，用于标识此格式的名称，并且把它与那个用于输出此格式的函数关联在一起。

上面给出的这个HTML文档框架，无疑可以划分为开头、中间和结尾。其中，开头部分和结尾部分，分别由初始化函数和清理函数来处理，而中间部分则由条目格式化函数来生成。HTML 初始化函数将负责生成文档头标记之前的所有内容，而清理函数则负责生成结尾的</body>和</html>标记：

```
sub html_init

{

    print "<html>\n";

    print "<head>\n";

    print "<title>U.S.Historical League Member Directory</title>\n";

    print "</head>\n";

    print "<body bgcolor=\"white\">\n";

    print "<h1>U.S.Historical League Member Directory</h1>\n";

}

sub html_cleanup
```

```

{

print "</body>\n";

print "</html>\n";

}

```

与之前一样，主要工作是格式化各个条目。但是，这部分代码并不难写。我们可以把`rtf_format_entry()`函数复制为`html_format_entry()`函数，然后对它进行修改，以保证条目里的所有特殊字符都会被编码，并且还要把RTF控制字替换为HTML标记：

```

sub html_format_entry

{

my $entry_ref = shift;

# 把&、"、>和<转换成相应的HTML 实体

# (&amp;、&quot;、&gt;、&lt;)

foreach my $key (keys (%{$entry_ref}))

{

next unless defined ($entry_ref->{$key});

$entry_ref->{$key} =~ s/&/&amp;/g;

$entry_ref->{$key} =~ s/"/&quot;/g;

$entry_ref->{$key} =~ s/>/&gt;/g;

$entry_ref->{$key} =~ s/</&lt;/g;

}

}

```

```
    printf "<strong>Name: %s</strong><br />\n", format_name
($entry_ref);
```

```
    my $address = "";
```

```
    $address .= $entry_ref->{street}
```

```
if defined ($entry_ref->{street});
```

```
    $address .= ", " . $entry_ref->{city}
```

```
if defined ($entry_ref->{city});
```

```
    $address .= ", " . $entry_ref->{state}
```

```
if defined ($entry_ref->{state});
```

```
    $address .= " " . $entry_ref->{zip}
```

```
if defined ($entry_ref->{zip});
```

```
    print "Address: $address<br />\n"
```

```
if $address ne "";
```

```
    print "Telephone: $entry_ref->{phone}<br />\n"
```

```
if defined ($entry_ref->{phone});
```

```
    print "Email: $entry_ref->{email}<br />\n"
```

```
if defined ($entry_ref->{email});
```

```
    print "Interests: $entry_ref->{interests}<br />\n"
```

```
if defined ($entry_ref->{interests});
```

```
    print "<br />\n";
```

```
}
```

这个函数的输出结果如下所示：

```
<strong>Name: Mike Artel</strong><br />
```

```
Address: 4264 Lovering Rd., Miami, FL 12777<br />
```

```
Telephone: 075-961-0712<br />
```

```
Email: mike_artel@venus.org<br />
```

```
Interests: Civil Rights,Education,Revolutionary War<br />
```

```
<br />
```

此脚本是遵照XHTML标准（它比HTML更严格）生成的文档，因此换行标记用的是
，而不是
。8. 4. 2. 2节将简要讨论HTML与XHTML之间的差异。

对gen_dir.pl脚本的最后一项修改是，在开关盒里增加一个指向各个HTML输出函数的新元素。下面是修改之后的开关盒代码，其中，最后那个元素定义了一个名为 html的格式，指向用来生成HTML文档的各部分的函数：

```
# 包含各种输出格式的格式化函数的开关盒
```

```
my %switchbox =
```

```
(
```

```
"text" =>                                # 普通文本格式的处理函数
```

```
{
```

```
"init"      => undef,                    # 不需要初始化
```

```

"entry"    => \&text_format_entry,

"cleanup" => undef          # 不需要清理

},

"rtf" =>                                # RTF 格式的处理函数

{

"init"     => \&rtf_init,

"entry"    => \&rtf_format_entry,

"cleanup" => \&rtf_cleanup

},

"html" =>                                # HTML 格式的处理函数

{

"init"     => \&html_init,

"entry"    => \&html_format_entry,

"cleanup" => \&html_cleanup

}

);

```

为把成员名录输出为HTML格式，可以执行下面这条命令，并把结果输出文件directory.html安装到Web服务器的文档树里：

```
% ./gen_dir.pl html > directory.html
```

无论何时更新数据库里的member 表，你都可以再次运行这条命令，并使用这个输出结果来更新在线成员名录。如果不想以手动方式执行这条命令，那么可以设置一个任务，让它定时自动更新这个在线成员名录。在Unix上，可以使用cron来完成此操作。假设gen_dir.pl脚本安装在/usr/local/bin 里，美史联盟在Web 服务器文档树的/usr/local/apache/htdocs/ushl里。于是，下面这条crontab条目将会在每天凌晨4点更新成员名录（请在一行里输入整条命令）：

```
0 4 * * * /usr/local/bin/gen_dir.pl  
  
> /usr/local/apache/htdocs/ushl/directory.html
```

运用这个cron作业的用户必须拥有在文档树目录里写入文件的权限。

8.4 使用DBI 来开发Web应用程序

我们此前开发的DBI脚本都是用于命令行环境的。其实，DBI也可以用在其他环境里，如用于开发基于Web的应用程序。当你编写的DBI脚本，能够被Web服务器调用来响应Web浏览器发出的请求时，你便可以向用户展示新鲜有趣的数据库交互操作了。例如，你编写了一个用表格形式显示数据的脚本，它能很容易地把每个列标题转换为一个链接，通过链接，可以对相应的列数据重新排序。这样一来，用户无需输入任何查询，只要轻轻一点鼠标，便可以用不同的方式查看数据。你还可以提供一个表单，让用户在其中输入数据库搜索条件，然后显示一个包含该搜索结果的网页。虽然这种功能很简单，但是它却大大提升了你能提供的访问数据库内容的交互级别。此外，Web浏览器的显示能力一般也优于终端窗口，因此你还可以创建出更加美观的输出效果。

在本节中，我们将创建下面几个基于Web的脚本。

一个通用性的表浏览器，适用于sampdb数据库。这个脚本与我们计划中的数据库任务无任何关系。它只是用于演示几个Web程序设计中常用的概念，并且会提供一种便捷的查看表内容的方式。

一个分数浏览器，用于查看任意一次考试或测验的分数。这不仅为成绩考评项目的考试分数核对工作提供了方便，而且也能为我们绘制考试分数曲线提供帮助，让我们可以更加准确地在考卷上标记出字母分数。

一个用于把拥有共同爱好的联盟成员找出来的脚本。具体实现方法为：先让用户输入一个搜索短语，然后在member表的interests列中搜索那个短语。其实，我们在前面8.3.4节里编写那个的名为interests.pl的命令行脚本，即可完成此任务。但是，这个命令行版本只能被那些在安装着该脚本的机器上拥有登录账户的人执行。如果提供一个基于Web的版本，那么这个成员名录便可以开放给每一位拥有Web浏览器的人。创建另外一个版本，还可以提供一种有益的参考，让我们可以对完成同一任务的不同方法进行对比。（事实上，我们开发了两个基于Web的实现：第一个基于模式匹配功能，有点儿像interests.pl脚本，另一个执行FULLTEXT搜索。）

编写这些脚本需要用到Perl语言的CGI.pm模块，它提供了一种简单的把DBI连接到Web的方式。（更多与CGI.pm模块的获得和安装有关的信息请参考附录A。）之所以称为CGI.pm，是因为用它编写出来的DBI脚本，能够使用公共网关接口（Common Gateway Interface protocol, CGI）协议，该协议定义了Web服务器如何与其他程序进行通信。CGI.pm模块负责处理一些常见内务任务所涉及的细节，如收集被Web服务器以输入形式传递给脚本的各个参数值。CGI.pm还提供了多种用于生成HTML输出的便捷方法，与手工编写原始的HTML标记相比，这些方法有助于降低HTML文档格式错误的概率。

本章介绍的CGI.pm模块的知识，已经足以让你编写出自己的Web应用程序，不过这里介绍的CGI.pm模块的功能并不全。如果想进一步了解此模块，请参考Lincoln Stein编写的《Official Guide to Programming with CGI.pm》（John Wiley, 1998），或者在<http://cpan.perl.org>网站上搜索“CGI”，阅读在线文档。

我的另一本专门讨论MySQL与DBI的书——《MySQL and DBI for the Web》（New Rider, 2000），也介绍了CGI.pm。

本章的后续部分所介绍的各个基于Web的脚本，都可以在sampdb发行版中的/perlap/web目录里找到。

8.4.1 配置Apache服务器来处理CGI脚本

在编写基于Web的脚本时，除使用DBI和CGI.pm模块以外，还需要一个组件：Web服务器。这里的指导内容是，将脚本与Apache服务器配合一起使用。不过，只要把这些指导内容稍微调整一下，或许就能使用另一种Web服务器。

假设你已把Apache服务器安装在/usr/local/apache（对于Unix系统）或者C:\Apache（对于Windows系统）下面。对于我们的目标而言，在Apache的顶层目录下面，最为重要的子目录是htdocs（用于存放HTML文档树）、cgi-bin（用于存放将由Web服务器调用的可执行脚本和程序）和conf（用于存放配置文件）。在你的系统上，这些子目录可能会稍有不同。如果真是这样，请根据下面的提示做相应的调整。

应该确保 cgi-bin 目录不在Apache的文档树里。这是一种出于安全考虑的做法，它可以防止客户查看脚本的源代码。你肯定不想让恶意客户获得脚本的文本内容并对它们进行分析，从而发现脚本可能存在的漏洞。

如果想要安装一个与Apache服务器一起使用的CGI脚本，则需要把它复制到cgi-bin目录下。在Unix系统上，脚本的第一行必须以#!开头，并且需要将其文件模式设置为可执行，如同命令行脚本一样。此外，最好能够把脚本的所有者设置为运行Apache的用户，并且只允许该用户访问。例如，如果运行Apache的用户的名字为www，那么下面两条命令将把脚本myscript.pl的所有者设置为www，并只允许该用户执行和读取：

```
# chown www myscript.pl
```

```
# chmod u=rx,go-rwx myscript.pl
```

你可能需要以root身份来运行这两条命令。如果你没有把脚本安装到cgi-bin目录里的权限，那么可以要求系统管理员来帮你完成。

在Windows系统里，不需要chown和chmod命令，但是脚本的第一行仍然应访以#!作为开头。这一行要列出你的Perl程序的完整路径名。

例如，Perl的安装路径为C:\Perl\bin\perl.exe，则#!行需要写成下面这个样子：

```
#!C:/Perl/bin/perl
```

另外，在Windows系统里，如果Perl的安装路径包含在环境变量PATH里，那么这一行还可以简单地写成下面这个样子：

```
#!perl
```

sampdb 发行版里的所有 Perl 语言脚本，都把#!行里的Perl 路径名指定成了/usr/bin/perl。如果需要一个对应于你的系统的路径，则需要修改所有的脚本。

在把脚本安装到cgi-bin目录之后，要请求脚本，可以通过Web浏览器向Web服务器发送相应的URL。例如，Web 服务器运行于本地主机，则可以使用下面这个URL 来请求myscript.pl脚本：

```
http://localhost/cgi-bin/myscript.pl
```

请记得更改本章所有示例里的URL地址，以便指向你自己的Web服务器主机，而非指向localhost。

用Web浏览器来请求脚本，会引发Web服务器去执行它。脚本的输出结果会发回给你，并显示为浏览器的一个页面。

当在命令行运行DBI脚本时，警告和出错信息都将被送往你的终端。在Web环境里，并没有终端，因此这些信息将被送往Apache的出错日志。这些信息有助于你对脚本进行调试，你一定要把这个日志的存放地点找出来。在我的系统里，它是 Apache 根目录/usr/local/apache下的logs目录里的error_log文件。你的系统情况可能与我的有所不同。日志文件的存放位置可以通过配置文件httpd.conf（位于Apache的conf目录）里的Errorlog指令来设定。

8.4.2 CGI.pm模块简介

如果在编写Perl脚本时要用到CGI.pm模块，则必须在脚本的开头放上一条use CGI语句，以导入模块里的函数名。可以像下面这样导入

那些最常用的函数：

```
use CGI qw(:standard);
```

然后，你便可以调用CGI.pm模块中的函数，生成各种HTML结构。一般情况下，那些函数的名字与相应的HTML元素是一致的。例如，想生成一个一级标题和一个段落，可以调用h1()和p()函数：

```
print h1 ("This is a header");
```

```
print p ("This is a paragraph");
```

CGI.pm还支持面向对象的编程风格，这使得你可以在不导入函数名的情况下，调用它们。为实现这一点，需要包含一条use语句，并创建一个CGI对象：

```
use CGI;
```

```
my $cgi = new CGI;
```

通过这个对象可以访问CGI.pm里的函数，就像调用对象的方法一样：

```
print $cgi->h1 ("This is a header");
```

```
print $cgi->p ("This is a paragraph");
```

这种面向对象接口要求你每次都要写出\$cgi->前缀。本书使用了更为简单的函数调用接口。不过，这种函数调用接口有一个弊端，即一旦 CGI.pm里的函数与 Perl语言的内置函数发生同名现象，那么你必须用一种不会产生冲突的办法来调用它。例如，CGI.pm模块里有一个名为tr()的函数，它会生成<tr>和</tr>标记，用来围住HTML表格的行里面的单元格。此函数与Perl语言中用于翻译的内置函数tr出现了名字冲突。为解决此问题，在使用CGI.pm模块的函数调用接口时，必须使用Tr()或TR()来调用tr()。当使用面向对象接口时，不会出现这种问题。因为调用tr()时，是把它当作\$cgi 对象的某个方法（即调用方式为\$cgi->tr()），这能够清楚地表明此时并没有引用Perl语言的内置函数。

8.4.2.1 检查Web输入参数

CGI.pm模块能为你做的一件事情是，处理所有在收集由 Web服务器提供给脚本的输入信息时涉及的各种琐事。为获得该信息，你所需要做的是调用param()函数。如果想获得所有可用参数的名字数组，可以像下面这样做：

```
my @param = param ();
```

如果想检索某个参数的值，则需要把它的名字传递给 param()。如果该参数设置有值，则param()将返回它的值；否则，返回undef：

```
my $my_param = param ("my_param");
```

```
print "my_param value: ", (defined ($my_param) ? $my_param : "not  
set"), "\n";
```

8.4.2.2 生成Web输出

CGI.pm 模块里的很多函数都可以生成发送到客户端浏览器的输出。请看下面这段 HTML文档：

```
<html>

<head>

<title>My Simple Page</title>

</head>

<body bgcolor="white">

<h1>Page Heading</h1>

<p>Paragraph 1.</p>

<p>Paragraph 2.</p>
```

```
</body>
```

```
</html>
```

下面这个脚本将使用CGI.pm输出函数来生成一份等效的文档：

```
#!/usr/bin/perl
```

```
# simple_doc.pl - 生成简单的HTML页面
```

```
use strict;
```

```
use warnings;
```

```
use CGI qw(:standard);
```

```
print header ();
```

```
print start_html (-title => "My Simple Page", -bgcolor => "white");
```

```
print h1 ("Page Heading");
```

```
print p ("Paragraph 1.");
```

```
print p ("Paragraph 2.");
```

```
print end_html ();
```

header () 函数负责在页面内容的前面，生成一个Content-Type: 头标记。如果网页是由脚本生成的，那么这个头标记必不可少，其作用是让浏览器知道，紧随其后的文档属于何种类型。（这与我们编写静态HTML页面的做法稍有不同。静态HTML页面不需要有头标记，因为Web服务器会自动给浏览器发一个。）默认情况下，header () 函数输出的头标记如下所示：

```
Content-Type: text/html
```

紧随 `header()` 调用之后的是另外一些函数调用，用于生成页面内容。`start_html()` 负责生成`<html>`起始标记和`<body>`起始标记之间的各种标记，`h1()`和`p()`负责写入各种标题和段落元素，而`end_html()`负责加入各个文档结束标记。

如 `start_html()` 调用所展示的那样，很多 `CGI.pm` 函数都允许你指定参数，指定格式为`-name => value`。这种做法对那些可以接受许多可选参数的函数特别有用，因为你不仅可以只列出你需要的参数，还可以按任意顺序来列出它们。

使用`CGI.pm`模块的输出生成函数，并不会妨碍你在需要时编写自己的原始 HTML。你可以混用这两种方法，把对 `CGI.pm` 函数的调用与生成文本标记的输出语句结合在一起。不过，使用`CGI.pm` 函数来生成输出的一个优势在于，你可以按逻辑单元来进行思考，而不用过多关注那些单个的标记，并且，最后生成的HTML也不太容易包含错误。

（我说的是“不太容易”，因为`CGI.pm`模块本身并不能阻止你做出奇怪的事情，如在标题里放一个列表）。

`CGI.pm`模块还能带来更好的可移植性——自己编写HTML难以做到。例如，自2.69版开始，`CGI.pm`可以自动生成XHTML格式的输出。如果你使用的早期的`CGI.pm`模块版本，输出的是普通的HTML，那么只需升级`CGI.pm`，你的脚本便能输出XMTML格式的文档了。

XHTML与HTML很相似，但其格式定义更加完善。HTML易于学习和使用，但有一个问题，即不同的浏览器对HTML的解释往往也不同。例如，它们对不规范的HTML文档便有着不同的处理方式。也就是说，对于不太规范的页面，它们在有的浏览器里能正确显示，而在另一个浏览器里却无法正确显示。XHTML的要求更为严格，有助于保证文档的规范性。下面是HTML和XHTML之间的几点区别。

与HTML有所不同，XHTML要求文档中的每一个起始标记，都必须有一个与之配对的结束标记。例如，段落本应该被放在`<p>`和`</p>`标记之间，但是在HTML文档里，`</p>`标记经常会被省略。在XHTML 里，`</p>`标记必不可少。对于没有任何内容的HTML标记，如`<br>`和`<hr>`，XHTML要求它们必须是闭合的，即应该把它们写成像`<br></br>`和`<tr></tr>`这个样子。为简化表示，XHTML也允许使用单个标记的简洁形式（如`<br/>`和`<hr/>`），同时充当起始标记和结束标记。不过，某些早期的浏览器在看到这类标记时，有时会将其误认为是`br/`和`hr/`。因此，在

斜线前面加一个空格，把它们写成`<br />`和`<hr />`，有助于减少这类错误的发生。

在HTML里，标记和属性的名字不区分大小写。例如，`<BODY BGCOLOR= "white">`和`<body bgcolor="white">`是一样的。在XHTML里，标记和属性的名字都要求使用小写，因此只有`<body bgcolor="white">`是被允许的。

在HTML里，属性值可以不用引号引起来，甚至可以不提供。例如，下面这个表数据单元的构造在HTML里是合法的：

```
<td width=40 nowrap>Some text</td>
```

在XHTML里，属性必须要有值，而且必须用引号引起来。对于那些在HTML里通常不带值的属性，一种常见的约定是，将其名字用作它们的值。在XHTML文档里，上面那个`<tr>`元素的等效表示如下所示：

```
<td width="40" nowrap="nowrap">Some text</td>
```

本书里所有Web脚本所生成的输出都符合XHTML规范。在本章中，我们将根据CGI.pm模块生成格式正确的XHTML标记。第9章里所讨论的那些脚本也会生成XHTML标记，但它们是靠自己生成的，因为PHP没有提供像CGI.pm模块那样的标记生成函数。

8.4.2.3 对HTML和URL文本进行转义

如果写到Web页面里的文本包含特殊字符，那么应该使用`escapeHTML()`函数来处理该文本，以确保这些特殊字符能被正确转义。如果构造出来的URL可能包含特殊字符，那么也需要对它进行转义处理，只是此时应该使用`escape()`函数。因为这两个函数所能识别的特殊字符集不同，并且对它们的编码方式也不一样，所以必须根据具体情况选用正确的编码函数。`escapeHTML()`的用途是，对HTML特殊字符进行转义。例如，“`<`”会被转义成“`<`”实体。`escape()`的用途是，把每一个特殊字符转义成以“`%`”引导的两位十六进制数字（代表了字符的数字编码），因此“`<`”将被转义成“`%3C`”。下面来看一个简短的Perl脚本`escape_demo.pl`，它演示了上述两种转义操作。

```
#!/usr/bin/perl
```



```
# escape_demo.pl - 演示CGI.pm的“输出编码”函数

use strict;

use warnings;

use CGI qw(escapeHTML escape); # 导入escapeHTML()和escape()

# 分配默认字符串值；如果存在命令行参数，则使用该参数

my $s = "1<=2, right?";

$s = shift (@ARGV) if @ARGV;

print "Unencoded string:           ", $s, "\n";

print "Encoded for use as HTML text: ", escapeHTML ($s), "\n";

print "Encoded for use in a URL:      ", escape ($s), "\n";
```

这个脚本分别使用了刚才介绍的那两个函数，对字符串\$s进行编码，并输出结果。当运行这个脚本时，它将产生下面的输出结果，从中可以清楚地看到，对HTML文本的编码约定与对URL的编码约定并不相同。

```
unencoded string:           1<=2, right?

encoded for use as HTML text: 1&lt;=2, right?

encoded for use in a URL:      1%3C%3D2%2C%20right%3F
```

如果向escape_demo.pl脚本提供一个命令行参数，那么此脚本将对那个参数（而不是默认字符串）进行编码。这样，你能看到对自己选择的字符串进行编码的情况。

escape_demo.pl脚本通过use CGI语句导入编码函数的名称。由于CGI.pm模块版本的不同，这两个编码函数有可能并没有包括在标准函

数集里，因此即使你导入了标准函数集，也需要像下面这样另外导入这两个函数：

```
use CGI qw (:standard escapeHTML escape);
```

8.4.2.4 生成多用途的页面

为什么要用基于Web的脚本来生成HTML页面，而不是编写静态的HTML文档呢？其中一个重要的原因在于，脚本能够根据不同的调用方式生成不同的页面。后面要编写的所有 CGI脚本都具有这一特性。它们的操作流程如下所示。

当你通过浏览器第一次请求脚本时，它会生成一个初始页面，让你选择想要的信息类型。

在你作出选择后，浏览器将会向Web服务器发出一个请求，再次调用刚才的脚本。接着，该脚本将从数据库里检索出你请求的具体信息，并把它们显示在第二个页面里。

这里有一个必须强调的问题：你想根据第一个页面里作出的选择来确定第二个页面里的内容，但是各个Web页面之间通常是彼此无关的，除非特意安排。解决办法就是，让脚本在生成页面时把某个参数设定为一个特定值，用以把你再次调用它的意图告知它。当你第一次调用该脚本时，这个参数还没有值，于是该脚本将生成一个初始页面。当你指明了要看的信息之后，该脚本将被再次调用，只是这次这个参数被设定成了一个用以指明该脚本应该做什么的值。

让Web页面向脚本传递参数值的办法有很多种。一种是，在页面里提供一个表单，让用户填写。当用户提交此表单时，它的内容会被传递到Web服务器。Web服务器再把信息传递给脚本，脚本通过调用 `param()` 函数可以获得提交的内容。我们实现的对美史联盟名录的关键字搜索，便是采用的这种办法：那个搜索页面里包含一个表单，用户可以在其中填写他们要搜索的关键字。

另一种向脚本传递参数值的办法是，把参数值追加到 URL 尾部。当向 Web 服务器请求脚本时，那些参数值将被传递到Web服务器。在实现 `sampdb` 表浏览器和成绩浏览器的脚本时，我们将使用这种办法。这种办法可行的原因在于，脚本会生成一个包含超链接的页面。当用户

选中某个链接时，该脚本会被再次调用，不过这次该链接会包含一个用以指明脚本应该做些什么的参数。实际上，这等效于让脚本以不同方式来调用自己，根据用户选择的链接，提供各种不同的结果。

采用向浏览器发送一个包含自引用超链接（即链接到其自己的URL地址）页面的方式，脚本可以实现对自己的调用。例如，如果在Web服务器的cgi-bin 目录里安装有一个脚本myscript.pl，那么它可以生成一个包含如下链接的页面：

```
<a href="/cgi-bin/myscript.pl">Click Me!</a>
```

当用户点击该页面里的“Click Me !”文字时，浏览器会向 Web 服务器发送一个对myscript.pl 脚本的请求。当然，就此链接自身而言，因为其 URL地址没有提供任何其他信息，所以它能做就是再次发出相同的页面。可是，如果为它加上一个参数，那么用户点击链接时，该参数也会被送回Web服务器。接着，服务器调用脚本，脚本通过param() 函数检测到该参数已被设置，并根据其值采用相应的动作。

如果想把参数添加到URL的末尾，则需要添加一个问号“?” 和一个name=value形式的名值对，后者用以指明参数的名字和值。例如，想要添加一个值为large的size参数，则可以把URL写成下面这样：

```
/cgi-bin/myscript.pl?size=large
```

如果想添加多个参数，则需要用字符“;” 或 “&” 来分隔：

```
/cgi-bin/myscript.pl?size=large;color=blue
```

CGI.pm模块能把“;” 或 “&” 识别为参数分隔符。不同程序设计语言的Web编程API有着不同的约定，因此在构造URL之前，一定要搞清楚它们期望的分隔符是“;” 还是“&”。我们此处使用的是“;”。

如果想构造一个带有参数的自引用URL，那么脚本应该先调用CGI.pm模块的url() 函数，获得其自己的URL，然后再像下面这样向URL里添加参数：

```
$url = url ();          # 获得脚本的URL
```

```
$url .= "?size=large"; # 添加第一个参数
```

```
$url .= ";color=blue"; # 添加第二个参数
```

使用url()来获得脚本路径，可以避免把路径硬编码在代码里。

如果想生成一个超链接，可以把这个URL传递给CGI.pm模块中的a()函数：

```
print a ({-href => $url}, "Click Me!");
```

这条输出语句会生成一个下面这样的超链接：

```
<a href="/cgi-bin/url.pl?size=large;color=blue">Click Me!</a>
```

上面这个例子在构造\$url值时比较随意，因为它并没有考虑到数值或链接标签可能包含特殊字符的情况。除非你百分之百地确定不需要对这些值和标签进行编码，否则最好能用CGI.pm模块的编码函数来处理它们。escape()函数可用于对需要添加到URL末尾的值进行编码，而escapeHTML()函数则用于对普通的HTML文本进行编码。例如，如果这个超链接标签的值存储在\$label变量里，而size和color参数的值存储在\$size和\$color变量里，那么正确的编码操作如下所示：

```
$url = sprintf ("%s?size=%s;color=%s",
```

```
url (), escape ($size), escape ($color));
```

```
print a ({-href => $url}, escapeHTML ($label));
```

请看下面这个简短的CGI脚本flip_flop.pl，从中可以看到，在具体的应用程序环境里，自引用的URL构造是如何工作的。当第一次调用该脚本时，它会生成的一个包含一个超链接的页面，我们称之为“页面A”。当选择该链接时，会再次调用这个脚本，只是链接中还包含一个pageb参数，用以告知flip_flop.pl显示页面B。（在这种情况下，我们并不关心这个参数的具体值，而只在意它是否有被设置）。页面B里也包含了一个对该脚本的链接，但是没有pageb参数。这意味着，如果选择页面B里的链接，则会重新显示那个原始页面。换句话说，连续调用此脚本，将会使显示结果在页面A和页面B之间来回交替：

```
#!/usr/bin/perl

# flip_flop.pl - 简单地支持多个输出页面的CGI.pm 脚本

use strict;

use warnings;

use CGI qw(:standard);

my $url;

my $this_page;

my $next_page;

# 根据参数pageb的存在与否来确定要显示哪一个页面

if (!defined (param ("pageb"))) # 显示页面A，并可链接到页面B

{

    $this_page = "A";

    $next_page = "B";

    $url = url () . "?pageb=1";

}

else # 显示页面B，并可链
接到页面A

{

    $this_page = "B";
```

```
$next_page = "A";

$url = url ();

}

print header ();

print start_html (-title => "Flip-Flop: Page $this_page",
-bgcolor => "white");

print p ("This is Page $this_page.To select Page $next_page, "
.a ({-href => $url}, "click here"));

print end_html ();
```

把这个脚本安装到 `cgi-bin` 目录，然后在浏览器里，使用下面这样的URL来请求它，记得把`localhost`替换为你自己的Web服务器的名字：

```
http://localhost/cgi-bin/flip_flop.pl
```

在页面里多次选择那个链接，便可以看到这个脚本会如何交替显示它所生成的那两个页面。

现在，假设有另一个客户端也开始请求`flip_flop.pl`脚本。结果会怎么样呢？彼此会相互干扰吗？答案是不会，因为各自的首次请求都没有包含`pageb`参数，而这个脚本只会用其初始页面进行响应。因此，每个客户端各自发出的请求会根据当前看到的页面来决定是否需要包含这个参数。`flip_flop.pl`脚本也会正确地为每一个客户端生成交替页面，而与另一个客户端所采取的动作无关。

8.4.3 从Web脚本连接MySQL服务器

在前面的8.3节里，开发了几个命令行脚本，它们在与MySQL服务器建立连接时都使用了相同的代码。大部分的CGI脚本也可以共享这种

代码，只是稍有不同：

```
#!/usr/bin/perl

use strict;

use warnings;

use DBI;

use CGI qw(:standard);

use Cwd;

# 对于UNIX 系统，选项文件应该包含连接参数

my $option_file = "/usr/local/apache/conf/sampdb.cnf";

my $option_drive_root;

# 对于Windows 系统，改写文件的位置

if ($^O =~ /^MSWin/i || $^O =~ /^dos/)

{

    $option_drive_root = "C:/";

    $option_file = "/Apache/conf/sampdb.cnf";

}

# 构造数据源，并连接到服务器（在Windows 里，

# 先存储当前工作路径，接着把位置改成选项文件

# 所在的那个驱动器，开始连接数据库，其后还原到当前路径）
```

```

my $orig_dir;

if (defined ($option_drive_root))

{

$orig_dir = cwd ();

chdir ($option_drive_root)

or die "Cannot chdir to $option_drive_root: $!\n";

}

my $dsn = "DBI:mysql:sampdb:mysql_read_default_file=$option_file";

my %conn_attrs = (RaiseError => 1, PrintError => 0, AutoCommit =>
1);

my $dbh = DBI->connect ($dsn, undef, undef, \%conn_attrs);

if (defined ($option_drive_root))

{

chdir ($orig_dir)

or die "Cannot chdir to $orig_dir: $!\n";

}

```

上面这段代码，与我们在前面那个命令行脚本里所使用的代码，存在以下几点差异。

上面代码的开头部分包含use CGI语句和use Cwd语句。第一条语句用于导入CGI.pm模块。第二条用于导入可以返回当前工作目录名字

的模块（如同稍后所述，使用它的原因在于预防脚本在Windows里运行）。

上面代码不会解析来自命令行的连接参数。实际上，它会假设这些参数都列在选项文件里。

上面代码并没有使用`mysql_read_default_group` 来读取标准选项文件，而是使用`mysql_read_default_file` 读取了一个选项文件——它专供访问 `sampdb` 数据库的Web 脚本使用。如上所示，这段代码会在Unix 下的`/usr/local/apache/conf/sampdb.cnf` 文件或Windows下的`C:\Apache\conf\sampdb.cnf` 文件里，查找各个选项。请注意，在Windows下，这段代码会在连接之前，把当前路径切换到选项文件所在驱动器的根目录，待连接成功之后，再切换回原来的那个目录。关于这样做的具体原因请参考8.2.9节。

在`sampdb` 发行版里有一个现成的`sampdb.cnf` 文件，你可以把它直接安装到系统里，供基于DBI的Web脚本使用。其内容如下所示：

```
[client]

host=localhost

user=sampadm

password=secret
```

如果你想在自己的系统上使用本章开发的这些基于Web的脚本，请把各段代码里选项文件的位置更改为你所使用的位置。你还得把选项文件`sampdb.cnf`安装到恰当的位置，并在其中把有关参数修改为你要使用的MySQL服务器主机名、MySQL账户名和密码。

在Unix下，还应该把这个选项文件的所有者设置为用于运行Apache服务器的那个用户，并把这个文件的访问模式设置为 400或600，以阻止其他用户读取它。这种做法可以防止出现一种安全漏洞，因为它可以防止其他登录到Web服务器主机的用户直接读取这个选项文件。

令人遗憾的是，那些能在Web服务器上安装并执行脚本的用户，仍然能够读取这个选项文件。Web服务器所调用的脚本在执行时，拥有用于运行Web服务器的登录账户的权限。这意味着，另一个能安装Web脚本的用户，能编写这个脚本，让它打开那个选项文件，并把里面的内容显示到某个Web页面上。因为那个脚本拥有Web服务器用户的所有权限，所以它完全拥有读取选项文件的权限，而你用来连接MySQL服务器和访问sampdb数据库的连接参数就此暴露无遗。如果只有你能访问Web服务器主机，那么这个问题无关紧要。但是，如果有其他你信不过的用户可以登录访问你的主机，那么应该考虑创建一个对sampdb数据库拥有只读（即SELECT）权限的MySQL账户。然后，把这个账户的用户名和密码（并非你自己的MySQL用户名和密码）列在sampdb.cnf文件里。这样便不会存在这样的风险，即允许脚本通过有权修改其表的MySQL账户连接到你的数据库。第13章将讨论如何创建一个权限受限的MySQL用户账户。这一策略的缺陷在于：在使用一个具有只读权限的MySQL账户之后，你编写的脚本只能进行数据检索，而无法进行数据录入。

另一种方法是，设法在Apache的suEXEC机制下执行脚本。如此一来，你便能以指定信任的用户身份来运行脚本，从而可以编写脚本，从那个只有该用户能读取的选项文件里读取连接参数。

还有一种编写脚本的方法，即让脚本要求客户端提供MySQL账户的用户名和密码，并用这些值来连接MySQL服务器。这种方法比较适用于那些使用频率不高的系统管理类脚本，不太适用于日常使用的脚本。在任何情况下，你都应该知道，这些要求使用者提供用户名和密码的方法，容易受到攻击，攻击者会在Web服务器与你的浏览器之间的网络上放置嗅探器，因此你很可能还需要建立一个安全连接。这些内容已超出本书的讨论范围。

从上面几段的内容可以看出，Web脚本安全是一件非常棘手的事情。安全问题是涉及范围很广的话题，在这里我很难给出更好的建议，因此你需要自行阅读更多的资料。在之前提及的《MySQL and DBI for the Web》一书中，有一章对网络安全进行了专门的讲解，其中包括如何使用SSL来建立安全连接。其他可供参考的信息资源有，Apache使用手册里的安全材料，以及<http://www.w3.org/Security/Faq>上的WWW安全常见问题。

8.4.4 基于Web的数据库浏览器

我们第一个基于Web的MySQL应用程序是一个简单的脚本，`db_browse.pl`，它能让你看到`sampdb`库里现在有哪些表，并能让你从Web浏览器上交互地查看其中任何一个表的内容。此脚本的工作原理如下。

当你从浏览器第一次请求`db_browse.pl`脚本时，它将连接到MySQL服务器，检索出`sampdb`库里现有的表，并向浏览器发送一个页面，其中把每一个表都显示成一个超链接。当选中这个页面里某个表名所对应的链接时，你的浏览器将向Web服务器发送一个请求，要求`db_browse.pl`显示该表的内容。

如果在`db_browse.pl`被调用时，它发现你已选择了某个表名，那么它会把这个表的内容检索出来，并展现在你的Web浏览器里。其中，每列数据的头部即为表里那个列的名字。所有表头都被显示为超链接。如果你选择了某个表头，那么浏览器会向Web服务器发送请求，要求重新显示该表的内容，同时将按你所选择的那个列进行排序。

警告

在继续往前讲解之前，想先提醒大家一下：虽然`db_browse.pl`脚本很好地说明了Web程序设计中的很多重要概念，但是它也展示了一个安全漏洞。这个脚本可以显示出`sampdb`数据库里的任何表，这会有麻烦的。例如，在第9章，我们将编写一个能够让美史联盟成员通过Web编辑其成员信息的脚本。此脚本对成员条目的访问是由存储在`member_pass`表里的密码进行控制的。如果到时还用`db_browse.pl`脚本，那么任何人都能查看这个密码表里的内容，进而可以访问到那些编辑`member`表的任何条目所必需的信息！因此，在你试用完此脚本，并理解了它是如何工作的之后，请立刻将其从`cgi-bin`目录里删除。

（或者，把这个脚本安装到一个非信任用户不可访问的私有Web服务器上。）

假设你还没被上面这段警告吓倒，接下来一起看看`db_browse.pl`脚本是如何工作的。这个脚本的主体部分会首先生成这个Web页面的开头部分，然后检查`tbl_name`参数，看看是否需要显示某个特定的表：

```
#!/usr/bin/perl
```

```
# db_browse.pl - 允许通过Web 浏览sampdb 数据库

use strict;

use warnings;

use DBI;

use CGI qw (:standard escapeHTML escape);

# ...建立数据库连接（未显示） ...

my $db_name = "sampdb";

# 生成页面的初始部分

my $title = "$db_name Database Browser";

print header ();

print start_html (-title => $title, -bgcolor => "white");

print h1 ($title);

# 在URL 里查找参数

my $tbl_name = param ("tbl_name");

my $sort_col = param ("sort_col");

# 如果$tbl_name 没有值，则显示一串可点击的表名。

# 否则，显示给定表的内容。如果设置了$sort_col,

# 则表明要按该列进行排序。

if (!defined ($tbl_name))
```

```

{

display_table_names ($dbh, $db_name)

}

else

{

    display_table_contents ($dbh, $db_name, $tbl_name,
$sort_col);

}

print end_html ();

```

要获得参数的值很容易，因为CGI.pm模块就能完成所有的工作——弄清Web服务器传递给脚本的信息内容。我们只需用自己感兴趣的参数的名字来调用param()函数即可。在db_browse.pl脚本的主体部分，该参数被命名为tbl_name。如果它没有值，则说明这是脚本的初始调用，脚本会显示一个表的清单。否则，它会显示参数 tbl_name 所指定的那个表的内容，同时按sort_col参数指定的那个列进行排序。

display_table_names()函数负责生成初始页面。此函数会检索表列表，并输出成一个项目列表，其中每一项即为sampdb库里的表名：

```

sub display_table_names

{

my ($dbh, $db_name) = @_;

print p ("Select a table by clicking on its name:");

# 检索由表名构成的单列数组的引用

```

```

my $sth = $dbh->prepare (qq{
SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA = ? ORDER BY TABLE_NAME
});

$sth->execute ($db_name);

# 使用函数ul()（无序列表）和li()（列表项）来构建一个项目列表，

# 每一项都是一个超链接，

# 可以通过它来再次调用脚本，显式特定的表。

my @item;

while (my ($tbl_name) = $sth->fetchrow_array ())
{

    my $url = sprintf ("%s?tbl_name=%s", url (), escape
($tbl_name));

    my $link = a ({-href => $url}, escapeHTML ($tbl_name));

    push (@item, li ($link));

}

print ul (@item);

}

```

其中，li() 函数负责为每个列表项添加和标记，而ul() 函数负责为整组列表添加和标记。此列表里的每一个表

名都是一个超链接，点击它可以再次调用此脚本，并显示出指定表的内容。`display_table_names()` 函数所生成的结果列表如下所示：

```
<ul>

  <li><a href="/cgi-bin/db_browse.pl?
tbl_name=absence">absence</a></li>

  <li><a href="/cgi-bin/db_browse.pl?
tbl_name=grade_event">grade_event</a></li>

  <li><a href="/cgi-bin/db_browse.pl?
tbl_name=member">member</a></li>

  ...

</ul>
```

当调用`db_browse.pl` 时，如果 `tbl_name` 参数有值，那么此脚本会把这个值传递给`display_table_contents()` 函数，如果还给定了列的名字，那么它会随同一起传递，用以对查询结果进行排序：

```
sub display_table_contents

{

my ($dbh, $db_name, $tbl_name, $sort_col) = @_;

my $sort_clause = "";

my @rows;

my @cells;

# 指定了排序列，则使用它来对结果排序

if (defined ($sort_col))
```

```

    {

        $sort_clause = " ORDER BY " . $dbh->quote_identifier
($sort_col);

    }

    # 呈现一个链接，将用户转向表列表页面

    print p (a ({-href => url ()}, "Show Table List"));

    print p (strong ("Contents of $tbl_name table:"));

    my $sth = $dbh->prepare (

"SELECT * FROM "

.$dbh->quote_identifier ($db_name, $tbl_name)

."$sort_clause LIMIT 200"

);

    $sth->execute ();

    # 把数据库表里各个列的名字当作

    # HTML 表格的表头。让每一个名字都成为一个超链接，

    # 可以让脚本再次被调用，重新显示这个表，

    # 并按指定的列进行排序。

    foreach my $col_name (@{$sth->{NAME}})

    {

        my $url = sprintf ("%s?tbl_name=%s;sort_col=%s",

```



```
url (),
escape ($tbl_name),
escape ($col_name));

    my $link = a ({-href => $url}, escapeHTML ($col_name));
    push (@cells, th ($link));
}

push (@rows, Tr (@cells));

# 显示表的各行

while (my @ary = $sth->fetchrow_array ())
{
    @cells = ();

    foreach my $val (@ary)
    {
        # 如果为非空，则显示值；否则，显示不间断空格

        if (defined ($val) && $val ne "")
        {
            $val = escapeHTML ($val);
        }

        else
```

```

{

$val = "&nbsp;";

}

push (@cells, td ($val));

}

push (@rows, Tr (@cells));

}

# 显示带边框的表格

print table ({-border => "1"}, @rows);

}

```

这个查询还包含了一个LIMIT 200子句，用以防止脚本向浏览器发送大量的数据。（sapmdb库里的表不太可能发生这种情况；但是如果用这个脚本来显示其他库里的表的内容，那么就有可能出现这种情况。）display_table_contents()函数将把表里的各行显示为一个HTML表格，即用th()和td()函数来生成HTML表格的表头和数据单元，然后用Tr()函数把单元格组成行，最后用table()函数生成<table>标记，把所有行包在其中。

HTML表格的各个列标题，展示成了超链接，单击它们，可以重新显示这个表格。这些链接都包含一个显式指定排序列的sort_col参数。例如，有一个显示grate_event表的内容的页面，那么其列标题链接将会是下面这个样子的：

```

<a href="/cgi-bin/db_browse.pl?
tbl_name=grade_event&sort_col=date">

date</a>

```

```
<a href="/cgi-bin/db_browser.pl?tbl_name=grade_event&sort_col=category">
```

```
category</a>
```

```
<a href="/cgi-bin/db_browser.pl?tbl_name=grade_event&sort_col=event_id">
```

```
event_id</a>
```

`display_table_contents()` 函数还使用一个小技巧：把空值转换为一个不间断空格（即` `）。对于带边框的HTML表格，有些浏览器无法正确地显示空单元格的边框。因此，要在这种单元格里设置一个不间断空格，以解决此问题。

如果想编写一个更为通用的脚本，可以修改`db_browser.pl`脚本，让它可以浏览多个数据库。例如，可以让这个脚本一开始便显示一个服务器里的数据库列表，而不是显示特定库里的表列表。然后，你可以从中选择一个数据库，获取它的表列表，并从此继续与前面类似的操作。

8.4.5 成绩考评项目：分数浏览器

我们要设计的下一个Web脚本是`score_browser.pl`，用于显示成绩考评项目所记录的考试成绩。严格来讲，我们应该先想办法录入考分，然后再想方法来检索它们。不过，录入考分的脚本要等到下一章才会进行介绍。在这个时候，我们在数据库里提前准备了几组前面学分的分数。虽然我们还没有一个方便的分数录入方法，但是我们可以编写脚本来显示这些分数。此脚本可以用有序列表来显示测验或考试分数，这种做法可以为评分曲线的确定和字母等级的分配提供方便。

`score_browser.pl`与 `db_browser.pl` 有很多相似之处（即同为信息浏览器），但是其用途更为具体，即要检查某次考试或测验分数。初始页面展现的是一个可以选择的考试事件列表，用户可以选择任意一个考试事件，查看该事件的分数。一个给定事件的分数，会按从高到低的顺序进行排列，以便你可以利用这个结果来确定评分曲线。

score_browse.pl脚本只需检查event_id这一个参数，看是否指定了考试事件。如果未指定，score_browse.pl会把grade_event表里的各个行显示出来供你选择；如果有指定，它会显示出与所选考试事件相关联的分数：

```
# ...建立数据库连接（未显示） ...

# 生成页面的初始部分

my $title = "Grade-Keeping Project -- Score Browser";

print header ();

print start_html (-title => $title, -bgcolor => "white");

print h1 ($title);

# 此参数用于指明要显示哪次考试事件的分数

my $event_id = param ("event_id");

# 如果$event_id无值，则显示事件列表。

# 否则，显示给定考试的分数。

if (!defined ($event_id))

{

    display_events ($dbh)

}

else

{
```

```
display_scores ($dbh, $event_id);

}

print end_html ();
```

其中，`display_events()` 函数负责从 `grade_event` 表里提取信息，将其显示为一个表格，用查询里的列名作为该表格的列标题。在每一行里，`event_id` 值都会显示为一个超链接，在点击它之后可以触发一个查询，检索出该次考试分数。与各次考试事件相对应的URL仅仅是一个指向 `score_browse.pl` 的路径，外加一个用于指定考试事件编号的参数：

```
/cgi-bin/score_browse.pl?event_id=n
```

下面是 `display_events()` 的全部代码：

```
sub display_events
{
    my $dbh = shift;

    my @rows;

    my @cells;

    print p ("Select an event by clicking on its number:");

    # 获得考试事件列表

    my $sth = $dbh->prepare (qq{

SELECT event_id, date, category

FROM grade_event

ORDER BY event_id
```

```

});

$sth->execute ();

# 把列名用作表格的列标题

for (my $i = 0; $i < $sth->{NUM_OF_FIELDS}; $i++)

{

push (@cells, th (escapeHTML ($sth->{NAME}->[$i])));

}

push (@rows, Tr (@cells));

# 把每一个考试事件的信息显示为表格的一行

while (my ($event_id, $date, $category) = $sth->fetchrow_array ())

{

@cells = ();

# 把考试事件ID 显示为一个超链接，用它来重新调用脚本，

# 显示该次考试的分数

my $url = sprintf ("%s?event_id=%d", url (), event_id);

my $link = a ({-href => $url}, escapeHTML ($event_id));

push (@cells, td ($link));

# 显示考试事件的日期和类别

push (@cells, td (escapeHTML ($date)));

```

```

push (@cells, td (escapeHTML ($category)));

push (@rows, Tr (@cells));

}

# 显示带边框的表格

print table ({-border => "1"}, @rows);

}

```

当用户选择一个考试事件时，浏览器会发送一个调用 `score_browse.pl` 脚本的请求，其末尾会带有一个事件 ID。`score_browse.pl` 脚本会发现设置了 `event_id` 参数，并调用 `display_scores()` 函数把指定考试的所有分数列出来。这个函数还会把文本“Show Events List”（显示考试事件列表）显示为一个可以返回初始页面的超链接，以使用户能迅速返回考试事件列表页面，选择另一次考试。

```

sub display_scores

{

my ($dbh, $event_id) = @_;

my @rows;

my @cells;

# 生成一个指向该脚本的链接，不包含

# 任何event_id 参数。如果用户点击了这个链接，

# 则此脚本会显示考试事件列表

print p (a ({-href => url ()}, "Show Event List"));

```

查询给定考试事件的分数

```
my $sth = $dbh->prepare (qq{
```

```
SELECT
```

```
student.name,
```

```
grade_event.date,
```

```
score.score,
```

```
grade_event.category
```

```
FROM
```

```
student INNER JOIN score INNER JOIN grade_event
```

```
ON
```

```
student.student_id = score.student_id
```

```
AND score.event_id = grade_event.event_id
```

```
WHERE
```

```
grade_event.event_id = ?
```

```
ORDER BY
```

```
grade_event.date ASC,
```

```
grade_event.category ASC,
```

```
score.score DESC
```

```
});
```


`$sth->execute ($event_id);` # 把考试事件编号与查询里的点位符进行绑定

```
print p (strong ("Scores for grade event $event_id"));
```

```
# 把列名用作表格的列标题
```

```
for (my $i = 0; $i < $sth->{NUM_OF_FIELDS}; $i++)
```

```
{
```

```
push (@cells, th (escapeHTML ($sth->{NAME}->[$i])));
```

```
}
```

```
push (@rows, Tr (@cells));
```

```
while (my @ary = $sth->fetchrow_array ())
```

```
{
```

```
@cells = ();
```

```
foreach my $val (@ary)
```

```
{
```

```
# 如果为非空，则显示值；否则，显示不间断空格
```

```
if (defined ($val) && $val ne "")
```

```
{
```

```
$val = escapeHTML ($val);
```

```
}
```

```

else

{

$val = "&nbsp;";

}

push (@cells, td ($val));

}

push (@rows, Tr (@cells));

}

# 显示带边框的表格

print table ({-border => "1"}, @rows);

}

```

其中，`display_scores()` 函数执行的语句与1.4.9.10节所编写的语句很相似。1.4.9.10节主要介绍了如何编写连接语句。当时，我们是根据给定的日期来进行分数查询，因为日期比事件ID值的意义更明确。相比之下，当我们使用`score_browse.pl`时，我们确切知道的是考试事件ID。之所以这样做，并不是因为我们是按事件ID来进行思考的，而是因为这个脚本已经显示了一个系列的ID供我们选择，同时还带有与其对应的日期和考试类型。很明显，有了这种类型的界面，你就不用详细了解具体的细节了。你不必知道具体的考试事件ID，只需要能识别出考试日期即可。此脚本会将它与正确的ID关联起来。

8.4.6 美史联盟：搜索志趣相投的成员

`db_browse.pl`和`score_browse.pl`脚本让用户可以根据初始页面里的选项列表进行选择，其中的选项都是以超链接形式展现的，它可以带上特定的参数再次调用此脚本。另一种让用户提供信息的方法是，显示一个让用户可以进行填写的表单。当选择范围是一些不易确

定的值时，这种办法更为合适。下一个脚本将展示这种提醒用户进行输入的方法。

在8.3节里，我们构建了一个命令行脚本interests.pl，用于发现具有某种特定兴趣的美史联盟成员。但是，联盟成员无法直接访问这个脚本。联盟秘书必须在命令行执行这个脚本，然后再把结果邮寄给请求这个列表的那位成员。如果能把这种查找功能提供给这些成员，让他们可以自己进行查找，则再好不过了。实现这一点的一种方法便是编写基于Web的脚本。本节将介绍两种表搜索方法：第一种是基于模式匹配的；第二种则使用了MySQL的FULLTEXT搜索功能。

8.4.6.1 利用模式匹配进行搜索

第一个搜索脚本是 ushl_browse.pl，它会显示一个表单，让用户输入一个关键字。当用户提交表单时，会再次调用此脚本，接着会在member表里搜索符合条件的成员，然后显示结果。此脚本的搜索机制是：在输入的关键字的两端，加上“%”通配符，然后执行LIKE模式匹配，找出那些在interests列值里包含这个关键字的行。

此脚本的主要部分是显示关键字表单。它还会检查是否提交了关键字。如果提交了，则执行搜索：

```
my $title = "U.S.Historical League Interest Search";

print header ();

print start_html (-title => $title, -bgcolor => "white");

print h1 ($title);

# 要查找的参数

my $keyword = param ("keyword");

# 显示关键字输入表单。此外，如果定义了$keyword，

# 则搜索并显示拥有该种兴趣的成员列表。
```

```

print start_form (-method => "post");

print p ("Enter a keyword to search for:");

print textfield (-name => "keyword", -value => "", -size => 40);

print submit (-name => "button", -value => "Search");

print end_form ();

# 如果指定了关键字，则连接到服务器，运行搜索

if (defined ($keyword) && $keyword !~ /^\\s*$/ )

{

# ... 建立数据库连接（未显示）...

search_members ($dbh, $keyword);

# ... 断开连接（未显示）...

}

```

此脚本向自己传递信息的办法与db_browse.pl或score_browse.pl略有不同。它没有把参数添加到URL的末尾。事实上，浏览器会负责对表单里的信息进行编码，然后将其作为post请求的一部分进行发送。不过，CGI.pm模块与该信息的具体发送方式毫无关系，因为不管该信息如何被发送，param()函数都会返回该参数值——这是CGI.pm让Web编程变得更容易的又一例证。

search_members()函数完成了关键字的搜索。首先，它会接收两个参数：一个是数据库句柄，另一个是输入的关键字。然后，它会运行搜索查询，把匹配到的成员行显示出来：

```

sub search_members

{

```

```

my ($dbh, $interest) = @_;

print p ("Search results for keyword: " .escapeHTML
($interest));

my $sth = $dbh->prepare (qq{

SELECT * FROM member WHERE interests LIKE ?

ORDER BY last_name, first_name

});

# 在interest 字段的任何位置查找字符串

$sth->execute ("% " . $interest . "%");

my $count = 0;

while (my $ref = $sth->fetchrow_hashref ())

{

html_format_entry ($ref);

++$count;

}

print p ("Number of matching entries: $count");

}

```

当运行ushl_browse.pl脚本时，你将会发现，即使脚本在生成表单时把keyword字段的值设定为一个空串，每次提交的关键字也还会出现在下一个页面中的表单里。这是因为：如果在脚本的执行环境里存在某个字段的值，那么 CGI.pm 模块会将其自动填充到那个字段里。如果你不喜欢这种行为，想让这个字段每次都显示为空白，则需要在textfield()调用里包含一个override参数：

```

    print textfield (-name => "keyword",
- value => "",
- override => 1,
- size => 40);

```

search_members() 函数使用了一个辅助函数html_format_entry() 来显示各个条目。此函数与之前为gen_dir.pl脚本编写的同名函数大同小异（参考8.3.1节）。不过，之前那一版本的函数是通过直接输出标记的方式来生成HTML的，而ushl_browse.pl所用的版本则使用了CGI.pm函数来生成各个标记：

```

sub html_format_entry
{
    my $entry_ref = shift;

    # 对HTML 里的特殊字符进行编码

    foreach my $key (keys (%{$entry_ref}))
    {
        next unless defined ($entry_ref->{$key});

        $entry_ref->{$key} = escapeHTML ($entry_ref->{$key});
    }

    print strong ("Name: " .format_name ($entry_ref)), br ();

    my $address = "";

    $address .= $entry_ref->{street}

```

```

if defined ($entry_ref->{street});

    $address .= ", " . $entry_ref->{city}

if defined ($entry_ref->{city});

    $address .= ", " . $entry_ref->{state}

if defined ($entry_ref->{state});

    $address .= " " . $entry_ref->{zip}

if defined ($entry_ref->{zip});

    print "Address: $address", br ()

if $address ne "";

    print "Telephone: $entry_ref->{phone}", br ()

if defined ($entry_ref->{phone});

    print "Email: $entry_ref->{email}", br ()

if defined ($entry_ref->{email});

    print "Interests: $entry_ref->{interests}", br ()

if defined ($entry_ref->{interests});

    print br ();

}

```

为了把 `first_name`、`last_name` 和 `suffix` 列的值拼在一起，`html_format_entry()` 函数使用了 `format_name()` 函数。它与前面 `gen_dir.pl` 脚本里的同名函数完全一样。

8.4.6.2 利用FULLTEXT索引进行搜索

美史联盟的成员可能会有多种兴趣。如果真是这样，那么在member表的interests列里，要用逗号把这些兴趣隔开：

Revolutionary War,Spanish-American War,Colonial period,Gold rush,Lincoln

可以用ushl_browse.pl脚本来搜索，与任意多个关键字相匹配的行吗？答案是，差不多可以。你可以在搜索表单里输入多个关键字，但要想找到匹配的行，必须构建一个更为复杂的查询来查找与每个关键字匹配的行。一种完成此项兴趣搜索任务的更加灵活的方法是，使用FULLTEXT 索引。本节的ushl_ft_browse.pl 脚本用的就是这种方法。更多与 MySQL的FULLTEXT功能有关的信息请参考2.14节。

在对member表执行FULLTEXT搜索之前，必须确保它是一个MyISAM表。如果你使用了其他的存储引擎来创建member表，则需要使用ALTER TABLE，将其转换为MyISAM表：

```
ALTER TABLE member ENGINE = MyISAM;
```

接下来，需要对member表进行正确的索引。为做到这一点，可以使用下面这条语句：

```
ALTER TABLE member ADD FULLTEXT (interests);
```

这样便可以对interests列执行FULLTEXT搜索。sampdb发行版里的ushl_ft_browse.pl脚本便是在ushl_browse.pl的基础上编写而成的，这两个脚本只在负责构造搜索查询的search_members()函数上有区别。经过修改的search_members()函数如下所示：

```
sub search_members  
  
{  
  
my ($dbh, $interest) = @_;
```



```
print p ("Search results for keyword: " .escapeHTML
($interest));
```

```
my $sth = $dbh->prepare (qq{
SELECT * FROM member WHERE MATCH(interests) AGAINST(?)
ORDER BY last_name, first_name
});
```

在interest 字段的任何位置查找字符串

```
$sth->execute ($interest);
```

```
my $count = 0;
```

```
while (my $ref = $sth->fetchrow_hashref ())
```

```
{
```

```
html_format_entry ($ref);
```

```
++$count;
```

```
}
```

```
print p ("Number of matching entries: $count");
```

```
}
```

与之前的search_members() 函数版本相比，此版本作了以下几处修改。

查询命令使用的是MATCH()...AGAINST() 子句，而非LIKE子句。

不需要使用通配符 “%” 来把关键字转换为模式。

在完成上述改动之后，你可以通过Web浏览器来调用 `ushl_ft_browse.pl` 脚本，并在搜索表单里输入多个关键字（有无逗号都可以）。这个脚本会把与这些关键字中的任何一个相匹配的成员条目找出来。

此脚本还有很多值得改进的地方。此脚本可以基于以下事实来改进：通过建立跨多个列的索引，并修改 `ushl_ft_browse.pl` 脚本来搜索这些列，可以让FULLTEXT搜索同时搜索多个列。例如，你可以删除原来的FULLTEXT 索引，再添加一个同时涉及 `interests`、`last_name` 和 `full_name` 列的索引：

```
ALTER TABLE member DROP INDEX interests;
```

```
ALTER TABLE member ADD FULLTEXT  
(interests,last_name,first_name);
```

为使用这个新的索引，还需要修改 `search_members()` 函数里的那条SELECT语句，将原来的 `MATCH(interests)` 改为 `MATCH(interests, last_name, first_name)`。

对 `ushl_ft_browse.pl` 的另一个更改是：在表单里增加两个单选按钮，让用户在“匹配任意一个关键字”和“匹配所有关键字”两种模式之间进行选择。第一种模式正是现在的脚本所使用的模式。如果想实现第二种模式，则需要修改语句，让它使用一个 `IN BOOLEAN MODE` 类型的FULLTEXT 搜索，再在每个关键字的前面加上一个加号“+”，表明它必须存在于匹配行里。更多关于布尔模式搜索的信息请参考 2.14.2节。

第9章

用PHP语言编写MySQL程序

PHP是一种脚本语言，可用于编写包含内嵌代码的Web页面。无论何时访问页面，内嵌在其中的代码都会被执行，并生成动态内容发送给客户端的Web浏览器。本章将介绍如何编写基于PHP的Web应用程序，并让它可以访问MySQL。关于PHP语言，与C语言和Perl DBI模块的MySQL编程API之间的比较请参考第6章。

本章假设你会把PHP和Apache Web服务器联合起来使用，当然，你也可以用其他服务器。PHP既可以当作 Apache 的模块来使用，也可以像传统的CGI程序那样当作独立的解释器来使用。出于性能上的考虑，请尽量把PHP当作 Apache 的模块来运行。

本章的示例将会使用示例数据库sampdb，会用到那些在第1章里为成绩考评项目和美史联盟创建的表。如果需要这里提到的各种软件，请参考附录A。附录A中还有对如何获得sampdb发行版的指导说明。该发行版的phpapi目录下有本章所开发的各个示例脚本。

PHP提供了以下几种针对MySQL的数据库访问接口。

mysql扩展是最早的MySQL接口。它由一系列以mysql_形式命名的函数构成。其中的绝大部分都由MySQL的同名C语言API函数直接映射而来。此扩展没有提供对MySQL 4.1及以上版本中新增功能的访问支持，因此现在不赞成使用它。

MySQL的增强扩展为 mysqli。此扩展提供了两种调用样式。既可以把它当作一组以mysql_形式命名的函数来使用，也可以通过面向对象的接口来使用它。

PHP数据对象（PHP Data Object， PDO）扩展是一个与特定数据库引擎基本无关的扩展。它提供了一个面向对象的与数据库无关的接口，有点类似于Perl语言的DBI模块。它使用了一个双层架构，其中，顶层是一个统一的接口，底层则由一些适用于不同数据库引擎的驱动程序构成。如果想从某个驱动程序切换到另外一个，只需要修改传递给连接调用的参数即可，这样它们便能调整到你使用的那个驱动程序。

本章将使用PDO来编写PHP脚本。PDO在PHP 5.0及以上的版本里都可以使用，但是本章假定使用的最低版本为PHP 5.1，因为这是PDO第一次与PHP捆绑发行的版本。更多相关信息，请参考<http://www.php.net/pdo>。

MySQL的PDO驱动程序在最初被设计时，需要链接 MySQL的C语言客户端开发库（即libmysqlclient库，详见附录G）。这种设计导致在使用PHP访问MySQL时，需要依赖MySQL发行版的部分内容。另外，新出现的mysqlnd 库是一个本地驱动程序，它实现了与libmysqlclient 一样的通信协议，可用于替代它。在有了mysqlnd 库之后，便可以在未安装MySQL客户端开发库的情况下，通过PHP访问MySQL数据库。自PHP 5.3开始，mysqlnd已包含其中；并且自5.4版本之后，它已成为了默认库。因此，当把PDO与mysqlnd 一起使用时，至少需要使用PHP 5.3版本。

基本上，本章只会对那些需要讨论的PDO 对象和方法进行介绍。它也不会涉及 PDO的MySQL驱动程序。针对其他数据库引擎的驱动程序有很多，但这里不会对它们进行讨论。附录I列出了与PDO接口有关的综合性内容。你也可以查阅PHP使用手册，它描述了PHP的所有功能。该手册可以从PHP官方网站<http://www.php.net/>获得。

PHP脚本的扩展名要能被Web服务器识别，且应该识别为调用PHP解释器来执行。如果服务器无法识别出你所用的扩展名，那么它会把PHP脚本当作普通文本。本章的脚本所用的扩展名都为“.php”。与“如何配置 Apache服务器，使其能够识别出你所用的那个扩展名”有关的信息，请参考附录A。（如果你无法掌控Apache的安装，则需要与系统管理员一起确认一下，应该正确使用哪一个扩展名。）该附录还描述了如何设置Apache服务器，让它把任何名为index.php的脚本当作它所处目录的默认主页，有点像Apache处理index.html文件的方式。

若想使用本章编写的这些脚本，则必须把它们安装到某个Web服务器能够访问的地方。本章所采取的办法是，把美史联盟项目和成绩考评项目放在Apache文档树的顶层目录下，并让它们各自拥有自己的目录ushl和gp。如果你也按这种方式来设置Web服务器，那么现在便应该创建这两个目录。如果服务器运行于本地主机，那么这两个目录里的页面会具有类似下面这样的URL：

<http://localhost/ushl/...>

`http://localhost/gp/...`

例如，如果这两个目录里的主页叫作 `index.php`，则可以像下面这样访问它们：

`http://localhost/ushl/index.php`

`http://localhost/gp/index.php`

如果已对Apache进行了配置，让其把 `index.php` 当作目录的默认页面，那么下面这两个URL与上面的那两个URL是等价的：

`http://localhost/ushl/`

`http://localhost/gp/`

要记得修改本章中的示例URL，指向你自己的Web服务器，而不是 `localhost`。

9.1 PHP概述

PHP的基本操作是，解释脚本并生成一个将被发送到客户端的Web页面。PHP脚本通常夹杂着HTML与可执行代码。HTML会被原样发送给客户端，而PHP代码会被执行，并被替换为它所生成的输出内容。因此，客户端看不到PHP代码，它只能看到最终的HTML页面。（本章的PHP脚本所生成的页面不仅仅符合HTML规范，而且还符合XHTML规范。对XHTML的简介，请参考8.4.2.2节。）

当PHP解释器开始读取某个文件时，它会假设这个文件的内容就像是HTML内容那样的字面文本，并把它遇到的任何内容复制到输出里。当解释器遇到某个特殊的起始标记时，它会从文本复制模式切换到PHP代码模式，并把该文件解释为可执行的PHP代码。当解释器遇到另一个表示代码结束的特殊标记时，它又会从代码模式切换回文本模式。这意味着你可以把静态文本（HTML部分）与动态生成的结果（PHP代码部分的执行结果）混合一起，生成一个会随着脚本的调用环境而变化的Web页面。例如，你可能会用PHP脚本来处理用户已在其中输入数据库搜索参数的表单。根据用户输入的不同内容，每次提交表单时搜索参数可能也会不同，因此当脚本执行搜索，并显示用户所请求的信息时，每一次的结果页面也会不相同。

我们以下面这个极其简单的脚本为例，了解一下PHP是如何工作的：

```
<html>
```

```
<body>

<p>hello,world</p>

</body>

</html>
```

这个脚本实际上非常简单，它根本就没有 PHP 代码！你肯定会问：“这有什么用呢？”问得非常好。答案就是：先建立一个脚本，让它包含你想要生成的页面，然后再往里添加PHP代码——有时，这种做法非常实用。这个脚本完全合法，PHP解释器完全能处理它。

如果想在脚本里包含PHP代码，必须使用特殊的起始标记和结束标记来将它与周围的文本区别开来，这两个标记分别为：<?php和?>。在PHP解释器遇到起始标记<?php时，它会从文本模式切换至 PHP 代码模式，并把遇到的所有内容都当作是可执行代码，一直到遇到结束标记?>为止。在这两个标记之间的代码将会被解释，并被替换为最终的输出结果。现在，把上面那个示例改写一下，让它包含一小段如下所示的PHP代码：

```
<html>

<body>

<p><?php print ("hello, world"); ?></p>

</body>

</html>
```

在这个例子里，代码部分很小，只有一行。当执行这段代码时，它会输出hello, world，这个内容会成为发送给客户端浏览器内容的一部分。因此，此脚本所生成的Web页面，与前面那个完全由HTML构成的脚本所生成的页面相同。

PHP代码可以生成Web页面的任何部分。我们刚看过了一种极端的情况，其中整个脚本完全由HTML文本构成，不包含任何PHP代码。而另一种极端情况是，所有的HTML内容完全由代码生成：

```
<?php

print("<html>\n");
```

```
print("<body>\n");

print("<p>hello, world</p>\n");

print("</body>\n");

print("</html>\n");

?>
```

从上面这三个示例可以看出，在如何产生输出方面，PHP表现得非常灵活。它让你自己来决定，HTML文本与 PHP代码的混合情况。PHP的灵活性还表现在，它不要求 PHP代码全都出现在同一个地方。在整个脚本里，你可以在HTML模式与PHP代码模式之间随意进行切换。

除本章示例使用的<?php和?>以外，PHP还支持其他的标记样式。如果想了解各种可用标记样式，以及如何启用它们，请参考附录I。

可独立执行的PHP脚本

本章的这些示例脚本，原意是想通过 Web服务器调用来生成 Web页面。不过，如果你有可独立运行的PHP 版本，你也可以用它在命令行上执行 PHP脚本。假设有一个名为hello.php的脚本，如下所示：

```
<?php print ("hello, world\n"); ?>
```

如果想在命令行执行这个脚本，可以使用下面这条命令：

```
% php hello.php

hello, world
```

当你在处理某个脚本时，这种做法有时能带来好处，因为你能够立刻发现它是否存在语法错误或者其他方面的问题，而不用在每次修改之后都通过浏览器来请求它。

9.1.1 一个简单的PHP脚本

如果PHP的功能只是利用输出语句生成本质上就是静态HTML的内容，那么它也没什么太大的用处。PHP的强大之处在于，它能够生成动态页面，即输出内容可以因脚本调用的不同而有所不同。本节描述的脚本对这一功能进行了

简单的演示。虽然它相对短小，但却比前面那几个示例多了一些实质性的内容。它展示了，如何从PHP访问MySQL数据库，以及如何在Web页面里使用查询结果。这个脚本构成了美史联盟网站主页的基础框架。在后面，我们会进一步完善这个脚本，但目前它还只能显示一条简短的欢迎消息和联盟成员的当前总人数：

```
<html>

<head>

<title>U.S.Historical League</title>

</head>

<body bgcolor="white">

<hz>U.S.Historical League</hz>

<p>Welcome to the U.S.Historical League Web Site.</p>

<?php

# 美史联盟主页

try

{

    $dbh = new PDO("mysql:host=localhost;dbname=sampdb", "sampadm",
"secret");

    $dbh->setAttribute(PDO::ATTR_ERRMODE,
PDO::ERRMODE_EXCEPTION);

    $sth = $dbh->query ("SELECT COUNT(*) FROM member");

    $count = $sth->fetchColumn (0);

    print ("<p>The League currently has $count members.</p>");

    $dbh = NULL; # 关闭连接
```



```
}  
  
catch (PDOException $e) { } # 清空处理器（进行错误捕捉，但将其忽略掉）  
  
?>  
  
</body>  
  
</html>
```

这条欢迎消息只是静态文本，因此最简单的方法是，将其编写成HTML文本。另一方面，成员总数是动态的，会随着时间的推移发生变化，因此它必须在执行时通过查询 sampdb 数据库里的member 表来确定。为完成此项任务，在脚本的起始标记和结束标记之间的那段代码需要执行以下几个步骤。

（1）连接MySQL服务器，并把sampdb设置为默认数据库。

（2）为后续的PDO调用启用异常捕获机制，以便能够在没有显式测试它们的情况下，可以轻而易举地捕获各种错误。

（3）向MySQL服务器发送一个查询，用于确定美史联盟现在有多少成员（即member表的行数）。

（4）根据查询结果构造一条包含成员总数的消息。

（5）关闭与MySQL服务器的连接。

刚才展示的那个脚本，可以在sampdb 发行版的phpapi/ushl 目录里找到，文件名为index.php。根据需要修改连接参数，并把index.php脚本的副本安装到Web服务器文档树的ushl目录下，你便可以使用下面这两个URL（请根据你自己的Web服务器对主机名和路径名做相应的修改）中的任何一个通过浏览器来访问这个脚本：

<http://localhost/ushl/>

<http://localhost/ushl/index.php>

下面把这个脚本分解一下，看看它是如何工作的。第一步是连接MySQL服务器：

```
$dbh = new PDO("mysql:host=localhost;dbname=sampdb", "sampadm",  
"secret");
```

这里使用了 `new PDO()` 语法来调用PDO类的构造函数。这个构造函数将尝试连接数据库服务器。如果尝试失败，则抛出一个异常；否则，返回一个充当数据库句柄的PDO对象。

`new PDO()` 的第一个参数是一个被称为“数据源名称”（DSN）的字符串。第二和第三个参数分别是用于连接服务器的用户名和密码：`sampadm`和`secret`。那个DSN字符串会告诉PDO使用哪一个驱动程序，后面紧跟着的是该驱动程序特有的连接参数。对于MySQL，驱动程序的名字为`mysql`，连接参数为运行服务器的主机和默认打开的那个数据库。这里所示的DSN表明，MySQL服务器的主机为`localhost`，默认数据库为`sampdb`。紧跟在冒号后面的两个参数是可选的。参数`host`的默认值为`localhost`，因此实际上可以省略这个参数。如果省略参数`dbname`，则表明不打开任何默认数据库。（DSN 还有其他几种形式，并且允许使用其他参数。更多详情请参考附录I。）

你也许会担心把用户名和密码嵌在脚本里会让别人看到。这种担心很有道理。在正常情况下，它们不会出现在发送给客户的结果Web页面里，因为脚本内容会被输出内容替代。可是，万一Web服务器配置不当，未能识别出这个脚本需要调用PHP来处理，那么它会把此脚本当作普通文本来发送，从而导致连接参数暴露。我们将在9.1.2节里处理这个问题。

由 `new PDO()` 返回的数据库句柄，是与MySQL服务器进行交互的手段，如调用要执行的SQL 语句。在连接成功之后，PDO 调用的默认出错处理模式为“静默失败”，这要求你必须显式地检查各个错误。为了简化出错处理环节，可以用下面的脚本来启用PDO的异常捕获机制：

```
$dbh->setAttribute(PDO::ATTR_ERRMODE,  
PDO::ERRMODE_EXCEPTION);
```

在启用异常捕获机制之后，可以用一个`try/catch` 语句块把错误“引导”至某个异常处理例程，并不需要进行显式的测试。如果没有使用`try/catch`，那么发生的异常将终止脚本的运行。

下面这个示例脚本将调用数据库句柄的`query()` 方法，把查询成员人数的命令发送到服务器，然后取回并显示查询结果：

```
$sth = $dbh->query ("SELECT COUNT(*) FROM member");  
  
$count = $sth->fetchColumn (0);
```

```
print("<p>The League currently has $count members.</p>");
```

其中，`query()` 方法负责把查询命令发送到服务器执行。请注意，在查询语句字符串的末尾不需要使用分号“;”，或字符序列“\g”和“\G”。这与我们在mysql客户端程序里调用语句的做法不一样。`query()` 方法适用于执行返回行的语句。（如果要执行对行进行修改的语句，则需要使用另一个方法 `exec()`。）`query()` 方法会返回一个 `PDOStatement` 对象，这是一个用于处理结果集的语句句柄。

具体到上面所示这个查询，其结果集包含的内容仅有一行一列，表示了成员总人数。此脚本通过调用 `$sth` 对象的 `fetchColumn()` 方法，可以获得该行，并提取出其中第一列（即0列）的值。

在输出计数值之后，脚本通过把数据库句柄设置为 `NULL` 的方式，关闭与服务器的连接。这一步是可选的。如果没有关闭与服务器的连接，那么PHP将在脚本执行结束后自动终止它。

在这个示例里，与MySQL交互的代码都出现在一个 `try` 语句块里，因错误而抛出的任何异常都会被与之相应的那个 `catch` 语句块捕获，并得到处理。如果连接操作失败，它会自动抛出一个异常，而 `setAttribute()` 调用会使随后的 `PDO` 调用在失败时抛出异常。不过，这个示例里的 `catch` 语句块为空，因此它等效于捕获并忽略各种错误。这个示例脚本不会输出任何消息，因为对于在主页显示欢迎消息这一功能需求而言，输出成员人数只是一个辅助操作。在这种情形下显示出错信息，很可能会让访问网站的人感到困惑。9.1.8节将对其他的出错处理办法进行讨论。

PHP里的变量

在PHP里，变量不需要事先进行声明，在你使用它的时候它便会立即存在。上面那个主页脚本使用了3个这样的变量，即 `$dbh`、`$sth` 和 `$count`，它们都没有进行声明。（在有些情形下，你可能需要声明变量，如在某个异常处理器里，或者当你需要在某个函数里引用全局变量时。）

所有变量名的前面都带有一个美元字符（\$）。尽管在访问数组或对象时需要费些周折才能访问到其中的各个元素，但是变量的确可用于表示各种类型值。如果变量 `$x` 表示的是一个标量值，如整数或者字符串，那么可以直接通过 `$x` 来访问它。如果 `$x` 表示的是一个以数字为下标的数组，那么需要通过诸如 `$x[0]`、`$x[1]` 这样的形式来访问它的元素。如果 `$x` 表示的是一个以“yellow”或“large”之类的字符串为下标的关联数组，则需要通过诸如 `$x["yellow"]` 或 `$x["large"]` 这样的形式来访问其中的元素。PHP 数组甚至允许把

数字和关联元素混合在一起使用。例如，`$x[1]`和`$x["large"]`都可以是同一数组的元素。如果`$x`表示的是一个对象，则需要通过`$x->property_name`这样的形式来访问它的属性。例如，`$x->yellow`和`$x->large`都可以是`$x`的属性。请注意，数字不能用作对象的属性名，除非用花括号将其括起来，因此在PHP里，`$x->{1}`是合法的，而`$x->1`则不合法。如果需要引用的属性名包含空格或其他非法字符，也可以用花括号将其括起来。

9.1.2 利用PHP库文件实现代码封装

PHP脚本与 DBI脚本的不同之处在于，PHP脚本存放在Web服务器的文档树里，而 DBI脚本通常位于Web文档树外的`cgi-bin`目录里。这会带来一个安全问题：服务器的配置错误会导致Web文档树里的页面以普通文本的形式发送到客户端。这意味着，与DBI脚本相比，PHP脚本里用于连接MySQL服务器的用户名和密码，向外泄露的风险更大。

我们最早编写的那个美史联盟主页脚本也存在这样的问题，因为它里面的MySQL 用户名和密码都是明文。我们现在要利用PHP的两个功能，即函数和包含文件，把这些连接参数转移到脚本以外的某个地方。我们将编写一个名为 `sampdb_connect()` 的函数，用于建立连接和返回数据库句柄，并把这个函数保存到一个包含文件里——这是一个库文件，它不属于主脚本，但可以被主脚本引用。这种办法有以下几个好处。

更易于编写连接建立代码。我们只需要在辅助函数`sampdb_connect()`里编写一次连接参数，用不着在每一个需要连接服务器的脚本里都去编写它。把类似的细节转移到脚本以外的库文件里，还有助于提高脚本的可读性，因为你可以把注意力集中在每个脚本的独特方面，而不用去考虑通用连接建立代码的问题。

包含文件可以供多个脚本使用。这将提高代码的重用性，让代码变得更易维护。全局性的改动可以很容易地应用到访问该文件的所有脚本里。例如，在把 `sampdb` 数据库从`localhost`主机迁移到`boa.example.com`主机时，不需要修改一大堆的单个脚本。而只需在当初定义`sampdb_connect()`函数的那个包含文件里，把主机名参数更改一下即可。

包含文件可以存放在Apache文档树以外的某个地方。这意味着用户将无法直接从他们的浏览器请求包含文件，因此即使Web服务器配置不当，它们的内容也不会泄露出去。如果不想让某些敏感信息被Web服务器发送到网站以外的地方，则可以利用包含文件将其隐藏起来。不过，虽然这种做法可以改善系统的安全性，但并不意味着用户名和密码就百分之百安全了。如果不采取其他一些必要的预防措施，那么可以登录到Web服务器主机（并因此能够访问其文件系统）的其他用户，将仍可以直接读取到你的包含文件。8.4.3节列出

了几个在安装DBI配置文件时需要注意的地方，好让这些文件免遭他人窥视。这些预防措施也同样适用于PHP的包含文件。

如果想使用包含文件，则必须先确定一个存放它们的地方，并且把将那个位置告知PHP。如果系统里已有一个这样的位置，那么你可以继续使用它。如果没有，则需要按下列步骤建立一个位置，用于存放包含文件。

(1) 在Web服务器文档树以外的某个地方，创建一个用于存储PHP包含文件的目录。我使用的目录是`/usr/local/apache/lib/php`，它不在我的文档树`/usr/local/apache/htdocs`下。

(2) 脚本访问包含文件的途径有两种：包含文件的完整路径名；如果设置了PHP的搜索路径，则用它们的基本名（即路径名里的最后一项）。后一种更方便，因为PHP会替我们查找包含文件。PHP在寻找包含文件时使用的搜索路径，可以通过PHP初始化文件`php.ini`里的`include_path`配置项来控制。请在你的系统里找到这个文件（通常位于`/etc/php`和`/usr/local/lib`下），并在该文件里找到含有`include_path`的这一行。如果它没有任何值，则可以把它设置为新创建的那个包含目录的完整路径名：

```
include_path = "/usr/local/apache/lib/php"
```

如果`include_path`已经有值，则可以把那个新目录添加到那个值里：

```
include_path = "/usr/local/apache/lib/php:current_value"
```

对于Unix系统，需要像上面这样使用冒号字符来分隔列在`include_path`里的各个目录。对于Windows系统，则需要使用分号来分隔。

在修改完`php.ini`之后，需要重新启动Apache，更改才能生效。

PHP包含文件的使用方法和C语言里的头文件很相似。例如，PHP可以在多个目录里搜寻包含文件，与C语言预处理器在多个目录里搜寻C语言头文件的方式很像。

(3) 创建要使用的包含文件，并将其放在包含目录里。该文件的名字应该与众不同，我们使用的是`sampdb_pdo.php`。这个文件最终将包含多个函数，但在此时，它只有一个`sampdb_connect()`函数：

```
<?php
```

```
# sampdb_pdo.php - 基于PDO的PHP 脚本所包含的适用于sampdb 数据库  
的公共函数
```

```

# 这个函数使用了我们高度机密的用户名和密码来连接MySQL 服务器，
# 以便使用sampdb 数据库。它还为后续的PDO 调用
# 启用了在出现错误时抛出异常的功能。
# 返回值为new PDO()所产生的数据库句柄。

function sampdb_connect ()

{

$dbh = new PDO("mysql:host=localhost;dbname=sampdb",
"sampadm", "secret");

$dbh->setAttribute(PDO::ATTR_ERRMODE,
PDO::ERRMODE_EXCEPTION);

return ($dbh);

}

?>

```

为了连接数据库服务器，`sampdb_connect()` 函数构造了一个数据源名称，并把它连同MySQL账户的用户名和密码一起传递给了`new PDO()`。接着，它把出错处理模式设置为了在发生PDO错误时抛出异常，并返回一个数据库句柄，用于实现与服务器的更多交互。此函数的用法如下：

```
$dbh = sampdb_connect ();
```

之所以要在`sampdb_connect()` 函数里启用异常捕获机制，是因为在库文件里启用它，比在每一个用到库文件的脚本里启用它，更方便。

通过观察可以发现，`sampdb_pdo.php`文件里的PHP代码都处于脚本标记`<?php`和`?>`之间。这是因为PHP是以文本复制模式来读取包含文件的。如果省略了这些标记，那么PHP会把包含文件的内容当作普通文本发送出去，而不会把它们解释为PHP代码。如果包含文件的内容是你想要生成的HTML输出内容，那么可以这样做。但是如果你想要执行其中的内容，则必须用脚本标记把PHP代码括起来。

(4) 当在脚本里引用包含文件时，可以使用下面语句当中的任何一条：

```
include "sampdb_pdo.php";
```

```
require "sampdb_pdo.php";
```

```
include_once "sampdb_pdo.php";
```

```
require_once "sampdb_pdo.php";
```

PHP会像下面这样处理这4条语句。

`include`和`require` 语句都会包含指定文件的内容，并对其进行计算。两者的区别在于，如果包含文件未找到，那么 `include` 语句会生成一条警告消息，并继续执行，而`require`语句将生成一条出错消息，并终止执行。

`include_once`和`require_once`语句分别类似于`include`和`require`语句，区别在于：如果它们已经读取了指定文件，则不会再次读取它。当这些包含文件还包含其他文件时，这种做法会很有用，因为它能避免因多次包含同一个包含文件，而可能引发的“函数重复定义”错误。

本章的脚本都统一使用`require_once`语句。当PHP遇到文件包含语句时，它会寻找该文件，并读入其中的内容。接下来的脚本代码可以访问该文件里的任何内容。

在`sampdb`发行版的`phpapi`目录里可以找到`sampdb_pdo.php`文件。请根据需要修改这个文件里的连接参数，以便正确连接到你自己的MySQL 服务器。然后，把这个文件复制到你使用的那个包含文件目录里，并设置此文件的访问权限和所有者，以便你的Web服务器可以读取（而其他用户不可读取）它。

现在，我们来修改美史联盟的主页，让它引用包含文件`sampdb_pdo.php`，并通过调用`sampdb_connect()`函数来连接MySQL服务器：

```
<html>
```

```
<head>
```

```
<title>U.S.Historical League</title>
```

```
</head>
```

```
<body bgcolor="white">

<h2>V.S.Historical League</h2>

<p>Welcome to the U.S.Historical League Web Site.</p>

<?php

# 美史联盟主页 - 第2版

require_once "sampdb_pdo.php";

try

{

$dbh = sampdb_connect ();

$sth = $dbh->query ("SELECT COUNT(*) FROM member");

$count = $sth->fetchColumn (0);

print ("<p>The League currently has $count members.</p>");

$dbh = NULL; # 关闭连接

}

catch (PDOException $e) { } # 清空处理器（进行错误捕捉，但将其忽略掉）

?>

</body>

</html>
```

在sampdb发行版的phpapi/ushl目录里，可以找到一个名为index2.php的文件，它便是刚才展示的这个脚本。请把它复制到Web服务器文档树的ushl目录下，并将其重新命名为index.php，以替换原有的那个同名文件。这样一

来，便可用上一个更安全的版本，因为这个新文件没有包含明文形式的MySQL用户名和密码。

你可能会想，虽然使用了包含文件，但主页里的代码并没有明显减少。别急！在sampdb_pdo.php 文件里还可以设置其他函数，从而使它成为一个便捷仓库，其中存放的是我们希望可以在多个脚本里执行的例程。事实上，我们现在便可以创建两个新函数，然后放到这个文件里面。在本章后面编写的每一个Web脚本，都需要在页面的开头和结尾分别生成一组非常模式化的HTML标记。与其在每个脚本里重复编写相同的HTML标记，还不如编写两个函数，html_begin() 和html_end()，来替我们生成。html_begin() 函数有两个参数，分别用于设定页面的窗口标题和文档标题。这两个函数的代码如下所示：

```
function html_begin ($title, $header)

{

print("<html>\n");

print("<head>\n");

if ($title != "")

print ("<title>$title</title>\n");

print("</head>\n");

print("<body bgcolor=\"white\">\n");

if ($header != "")

print ("<h2>$header</h2>\n");

}

function html_end ()

{

print("</body>\n");

print("</html>\n");
```

```
}
```

在把html_begin()和html_end()放入sampdb_pdo.php文件之后，便可以修改美史联盟的主页，让其使用这些函数。下面是改写后的脚本（名字为index3.php）：

```
<?php

# 美史联盟主页 - 第3版

require_once "sampdb_pdo.php";

$title = "U.S.Historical League";

html_begin ($title, $title);

?>

<p>Welcome to the U.S.Historical League Web Site.</p>

<?php

try

{

$dbh = sampdb_connect ();

$sth = $dbh->query ("SELECT COUNT(*) FROM member");

$count = $sth->fetchColumn (0);

print ("<p>The League currently has $count members.</p>");

$dbh = NULL; # 关闭连接

}

catch (PDOException $e) { } # 清空处理器（进行错误捕捉，但将其忽略掉）
```

```
html_end ();
```

```
?>
```

请注意，PHP 代码被分成了两个部分，介于其间的是网站欢迎信息的 HTML 文本。请把index3.php复制到Web服务器文档树的ushl目录下，并将其重新命名为index.php，以替换原有的同名文件。

利用函数来生成页面的开头和结尾，其实是一项很重要的功能。如果要把页头或页脚的外观更改为另一种风格，则只需修改相关的函数即可，而所有使用了它们的脚本都会自动生效。例如，想要在美史联盟的每个页面底部加上一条“Copyright USHL”信息。这时，把信息添加到像html_end()这样的页尾函数里即可轻松实现。

9.1.3 简单的数据检索页面

我们在美史联盟主页里嵌入了一个脚本，它会运行一个只返回一个行（即成员总人数）的查询。下一个脚本将演示如何处理由多个行构成的结果集（如member表的全部内容）。这个PHP脚本与 8.2.2 节所开发的DBI 脚本dump_members.pl 类似，因此可以将其命名为dump_members.php。PHP版本与DBI版本的不同之处在于，它主要用于Web环境，而不太适合于在命令行运行。正因如此，这个PHP脚本会输出HTML格式的内容，而不是输出以制表符分隔的文本。为了把行和列整齐地显示出来， dump_members.php脚本会把各成员行写到一个HTML表格中。下面是脚本的完整代码：

```
<?php

# dump_members.php - 将美史联盟成员信息填入HTML 表格

require_once "sampdb_pdo.php";

$title = "U.S.Historical League Member List";

html_begin ($title, $title);

$dbh = sampdb_connect ();

# 执行语句

$stmt = "SELECT last_name, first_name, suffix, email,"
." street, city, state, zip, phone FROM member ORDER BY last_name";
```

```

$sth = $dbh->query ($stmt);

print("<table>\n");           # 表格开始

# 读取语句的结果，然后进行清理

while ($row = $sth->fetch (PDO::FETCH_NUM))

{

print("<tr>\n");           # 表格行开始

for ($i = 0; $i < $sth->columnCount (); $i++)

{

# 对特殊字符进行转义，并输出表格单元

print ("<td>" . htmlspecialchars ($row[$i]) . "</td>\n");

}

print("</tr>\n");           # 表格行结束

}

print("</table>\n");           # 表格结束

$dbh = NULL;  # 关闭连接

html_end ();

?>

```

其中，`sampdb_connect()` 函数启用了针对PDO错误的异常处理机制，但 `dump_members.php` 脚本并没有包含任何用来处理异常的 `try/catch` 语句块。万一出现错误，会发生什么事情呢？此时，PHP的默认行为是终止脚本的执行，并输出一条问题描述消息。这与我们在美史联盟主页脚本里使用的办法有所不同，当时我们使用了一个空的异常处理例程来捕获并忽略所有的错误。对于那个主页而言，向访问者显示欢迎信息是其主要的用途，而显示现有成员总人数只是一个附加功能，因此在检索不到总人数时不需要输出出错信息。

对于 `dump_members.php` 脚本而言，其意义在于显示从数据库查询出来的结果，因此当它因某种原因无法显示数据库查询结果时，则应该给出一条出错信息，以表明发生了什么样的错误。

在执行查询，从 `member` 表查询到各行之后，此脚本会调用 `fetch()` 方法——它会返回结果集里的下一个行；如果已经到达结果集的末尾，则返回 `FALSE`。`PDO::FETCH_NUM` 参数的作用是，告诉 `fetch()` 返回一个行，其中的各列是按数字索引的。

为了对Web页面里显示的值进行编码，`dump_members.php` 脚本使用 `htmlspecialchars()` 函数对HTML语言里的特殊字符（如“<”、“>”或“&”）进行了转义处理。（如果要对URL里的值进行编码，则需要使用 `urlencode()` 函数。）这两个PHP编码函数与Perl的CGI.pm模块里提供的两个方法 `escapeHTML()` 和 `escape()`（参考8.4.2.3节）很相似。

如果想试用 `dump_members.php` 脚本，需要先把它安装到Web服务器文档树的 `ushl` 目录，然后再使用下面这个URL通过Web浏览器来访问它：

`http://localhost/ushl/dump_members.php`

为了让人知道 `dump_members.php` 脚本，我们还需要在美史联盟的主页脚本里添加一个指向它的链接。下面是修改后的主页脚本（即 `index4.php`）：

```
<?php

# 美史联盟主页 - 第4版

require_once "sampdb_pdo.php";

$title = "U.S.Historical League";

html_begin ($title, $title);

?>

<p>Welcome to the U.S.Historical League Web Site.</p>

<?php

try

{
```

```

$dbh = sampdb_connect ();

$sth = $dbh->query ("SELECT COUNT(*) FROM member");

$count = $sth->fetchColumn (0);

print ("<p>The League currently has $count members.</p>");

$dbh = NULL;  # 关闭连接

}

catch (PDOException $e) { } # 清空处理器（进行错误捕捉，但将其忽略掉）

?>

<p>

You can view the directory of members <a
href="dump_members.php">here</a>.

</p>

<?php

html_end ();

?>

```

参考之前的主页修订，将index4.php复制到Web服务器文档树的ushl目录下，并将其重新命名为index.php，以替换原有的那个同名文件。

这个dump_members.php脚本演示了，如何利用PHP脚本检索MySQL数据库里的信息，并将其转换成Web页面的内容。如果你原意，也可以自行修改这个脚本，让它输出更为详尽的结果。例如，你可以把来自email列的值显示为超链接，而不是静态文本，从而让站点访问者能够更容易地向联盟成员发送电子邮件。sampdb发行版里的dump_members2.php脚本可以做到这一点。它与dump_members.php脚本只是在那个用于获取和显示成员条目的循环里存在差异。原有的那循环语句如下所示：

```

while ($row = $sth->fetch (PDO::FETCH_NUM))
{
    print ("<tr>\n");          # 表格行开始

    for ($i = 0; $i < $sth->columnCount (); $i++)
    {
        # 对特殊字符进行转义，并输出表格单元

        print ("<td>" .htmlspecialchars ($row[$i]) . "</td>\n");
    }

    print ("</tr>\n");        # 表格行结束
}

```

电子邮件地址处于查询结果的第4列，因此ump_members2.php脚本需要对这个列特别对待，即如果它的值不为空，就输出一个超链接：

```

while ($row = $sth->fetch (PDO::FETCH_NUM))
{
    print ("<tr>\n");          # 表格行开始

    for ($i = 0; $i < $sth->columnCount (); $i++)
    {
        print ("<td>");

        # 对特殊字符进行转义，并输出表格单元；

        # 电子邮件处于结果集里的第4 列（即索引3）

        if ($i == 3 && $row[$i] != "")
        {

```

```

        printf("<a href=\"mailto:%s\">%s</a>",
$row[$i],
htmlspecialchars ($row[$i]));
    }

    else
    {

        print (htmlspecialchars ($row[$i]));

    }

    print ("</td>\n");

}

print ("</tr>\n");          # 表格行结束

}

```

9.1.4 处理语句结果

PDO提供了以下几种执行SQL语句的方式。

PDO 对象有两个方法：`exec()`和`query()`。它们都接受一个SQL语句参数，然后立刻执行该语句，并返回结果。

对于那些诸如DELETE、INSERT、REPLACE和UPDATE之类的用于修改行的语句，需要调用`exec()`来执行，这个方法会返回一个计数值，表明语句实际更改（删除、插入、替换或更新）了多少个行。

诸如SELECT这种会生成结果集的语句，需要调用`query()`方法来执行，这个方法将返回一个PDOStatement语句句柄对象。通过这个对象可以访问结果集的信息。例如，可以调用`fetch()`方法来检索结果集里的各个行；可以调用`columnCount()`方法来查明结果集里的有多少个列。

PDO 还支持通过预处理语句的方式分两个阶段执行语句。PDO 对象有一个`prepare()`方法，它可以接受一个SQL语句参数，但是不会立刻执行这条语句，它会做一些初始化处理，并返回一个PDOStatement语句句柄对象。这个

语句句柄有一个可用于执行该语句的`execute()`方法，还有一些用于处理结果集的其他方法。

`prepare()`和`execute()`可以用来处理所有语句。它们并不局限于那些只会修改行的语句，或只会返回行的语句。

预处理语句还提供了两种重要的功能：一是通过反复多次地执行来改善性能；二是可以处理数据值里的特殊字符。更多相关信息请参考9.1.6节。

接下来的几节，将更加详细地讨论PDO的语句执行功能。所用示例程序会假定启用了出错时抛出异常的机制。

9.1.4.1 处理那些修改行的语句

对于修改行的语句，需要使用数据库句柄的`exec()`方法。该方法会返回一个行计数值，用以表明有多少个行受到影响。假设，你需要删除 `member` 表里编号为 149的成员，并报告操作结果。下面这个示例演示了如何确定该语句是否真的删除了行：

```
$count = $dbh->exec("DELETE FROM member WHERE member_id = 149");

if ($count > 0)

    print ("Member 149 was deleted\n");

else

    print ("No record for member 149 was found\n");
```

9.1.4.2 处理那些返回结果集的语句

对于那些会生成结果集的语句，需要使用数据库句柄的`query()`方法。这个方法会返回一个可用于访问结果集的PDOStatement语句句柄对象。这种语句句柄有多个非常有用的方法，如下所示。

`fetch()`方法。如果还有更多的行，那么它返回结果集里的下一行；否则，返回FALSE。

`fetchColumn()`方法。它与`fetch()`方法相似，但只会返回每一个行的一个列值。

`columnCount()` 方法。它会返回结果集里的列数。（没有与之对应的行统计方法。必须先获取行，再对它们进行计数。）

在前面讨论美史联盟主页时给出的那几个示例，演示了如何只调用一次 `fetchColumn()` 方法来获得一个单值结果。当你希望获得包含多个列的多个行时，常用的做法是在循环里多次调用 `fetch()` 来获得这些结果。下面的这个示例演示了一种具体的做法。在获取行时，它也会对这些行进行计数，以确定总共有多少个行。

```
$sth = $dbh->query ("SELECT * FROM member");

# 获取结果集里每一个行

$count = 0;

while ($row = $sth->fetch (PDO::FETCH_NUM))

{

    # 输出行里的各个值，并以逗号进行分隔

    for ($i = 0; $i < $sth->columnCount (); $i++)

        print ($row[$i] . ($i < $sth->columnCount () - 1 ? "," : "\n"));

    $count++;

}

printf ("Number of rows returned: %d\n", $count);
```

`fetch()` 方法有一个参数，能指定返回值的类型。表9-1列出的是一些常用的获取模式。

表9-1 行获取模式的值

参数	返回值
PDO::FETCH_ASSOC	一个数组，其元素需要通过关联索引来访问
PDO::FETCH_NUM	一个数组，其元素需要通过数字索引来访问
PDO::FETCH_BOTH	一个数组，其元素可以通过关联索引或数字索引来访问
PDO::FETCH_OBJ	一个对象，其元素需要通过属性来访问

`fetch()` 方法的参数是可选的。如果未提供参数，则使用默认模式。除非重置了默认设置，否则其默认值为 `PDO::FETCH_BOTH`，表示 `fetch()` 将把每一行返回为一个数组，而其元素既可通过列名来访问，也可以通过数字下标来访问。

如果想在获取行之前设置默认获取模式，那么可以向 `query()` 方法多传递一个参数，也可以调用语句句柄的 `setFetchMode()` 方法。下面两个示例都可以把获取模式设置为 `PDO::FETCH_NUM`，从而对随后的结果集检索操作产生影响：

```
$sth = $dbh->query ($stmt, PDO::FETCH_NUM);

$sth = $dbh->query ($stmt);

$sth->setFetchMode (PDO::FETCH_NUM);
```

如果获取模式为 `PDO::FETCH_ASSOC`，那么 `fetch()` 将把结果集里的下一行返回为一个关联数组。其中的元素名即为查询命令里选取的那些列的名字。例如，如果从 `president` 表里检索 `last_name` 和 `first_name` 的值，则可以像下面这样访问这些列：

```
$stmt = "SELECT last_name, first_name FROM president";

$sth = $dbh->query ($stmt);

while ($row = $sth->fetch (PDO::FETCH_ASSOC))

    printf ("%s %s\n", $row["first_name"], $row["last_name"]);
```

如果获取模式是 `PDO::FETCH_NUM`，则 `fetch()` 将把结果集里的下一个行返回为一个数组，其中的元素需要通过从 0 开始的数字索引来访问。结果集里

的列个数可以通过调用语句句柄的`columnCount()`方法来确定。下面这个简单的循环将获取各个行，并以制表符分隔的格式输出它们的值：

```
$stmt = "SELECT * FROM president";

$sth = $dbh->query ($stmt);

while ($row = $sth->fetch (PDO::FETCH_NUM))

{

    for ($i = 0; $i < $sth->columnCount (); $i++)

        print ($row[$i] . ($i < $sth->columnCount () - 1 ? "\t" :
"\n"));

}
```

如果获取模式为`PDO::FETCH_BOTH`，则`fetch()`将返回一个数组，而其元素既可以通过数字索引来访问，也可以通过列名来访问。这相当于`PDO::FETCH_ASSOC`和`PDO::FETCH_NUM`的组合。

如果获取模式为`PDO::FETCH_OBJ`，则`fetch()`将把结果集里的下一个行返回为一个对象，而其属性需要使用`$row->col_name`语法来访问：

```
while ($row = $sth->fetch (PDO::FETCH_OBJ))

    printf ("%s %s\n", $row->first_name, $row->last_name);
```

如果查询里包含计算列，那该怎么办？例如，你可能要执行一条查询，其返回值为某个表达式的计算结果：

```
SELECT CONCAT(first_name, ', ', last_name) FROM president
```

对于这样的查询，不太适合将行当作对象来获取。其中，查询列的名字即为表达式本身，而它不是合法的属性名。不过，你可以为列取一个别名，以此来提供合法名字。下面这个查询便为这个列取了一个别名 `full_name`：

```
SELECT CONCAT(first_name, ', ', last_name) AS full_name FROM president
```

如果现在把这个查询的每个行当作一个对象来获取，则可以利用这个别名，使用`$row->full_name`语法来访问它所对应的列。

以上几个示例都使用了下面这种形式的行获取循环，将每个行赋给\$row变量：

```
while ($row = $sth->fetch ([fetch_mode]))  
  
    ...handle row ...
```

不过，还有其他几种获取行的办法。一种办法是，获取一个数组，并把结果赋值给一组变量。例如，下面这段代码将把last_name和first_name列赋给\$ln和\$fn变量，并按名在前、姓在后的顺序输出各个姓名：

```
$stmt = "SELECT last_name, first_name FROM president";  
  
$sth = $dbh->query ($stmt);  
  
while (list ($ln, $fn) = $sth->fetch (PDO::FETCH_NUM))  
  
    printf ("%s %s\n", $fn, $ln);
```

你可以任意选择这些变量的名字，只要合法就行，但是它们在list()里的顺序必须与查询命令里选取的各个列的顺序相对应。

另一种办法是，把检索到的各个列值直接赋给 PHP 变量。为实现这一点，需要先用bindParam()把结果集里的各个列与有关变量绑定在一起；然后再使用PDO::FETCH_BOUND模式取回各个行。这样，fetch()将在没有到达结果集末尾时返回TRUE，并且对于获取到的每一个行，它会将其中的列值分别赋予那些绑定变量：

```
$stmt = "SELECT last_name, first_name FROM president";  
  
$sth = $dbh->query ($stmt);  
  
$sth->bindParam (1, $ln);  
  
$sth->bindParam (2, $fn);  
  
while ($sth->fetch (PDO::FETCH_BOUND))  
  
    printf ("%s %s\n", $fn, $ln);
```

如果想把一次性获取的所有行赋值给一个数组，可以使用fetchAll()方法：

```
$rows = $sth->fetchAll ();
```

与`fetch()`一样，`fetchAll()`也可以使用默认获取模式，并且接受一个显式的获取模式参数。

语句句柄可以用作迭代器，而不必显式调用`fetch()`方法：

```
foreach ($sth as $row)
```

```
printf ("%s %s\n", $row["first_name"], $row["last_name"]);
```

默认的获取模式决定了行是如何被返回的。

9.1.5 测试查询结果里的NULL值

PHP使用了PHP语言的NULL值来表示SQL语言的NULL值。要检查SELECT查询所返回的某个列是否为 NULL 值的一种办法是，使用`isnull()`函数。下面这个示例选择并输出了member表里的姓名和电子邮件地址，当电子邮件地址为NULL值时，它将输出 “No email address available”（无电子邮件地址）：

```
$stmt = "SELECT last_name, first_name, email FROM member";
```

```
$sth = $dbh->query ($stmt);
```

```
while (list ($last_name, $first_name, $email) = $sth->fetch  
(PDO::FETCH_NUM))
```

```
{
```

```
printf ("Name: %s %s, Email: ", $first_name, $last_name);
```

```
if (is_null ($email))
```

```
print ("No email address available");
```

```
else
```

```
print ($email);
```

```
print ("\n");
```

```
}
```

另一种测试SQL语言NULL值的办法是，使用完全相等运算符“===”，将值与PHP的NULL常量相比较：

```
if ($email === NULL)

    print ("No email address available");

else

    print ($email);
```

PHP的NULL值等同于一个未设置的值，因此还可以用isset()函数来测试NULL值：

```
if (!isset ($email))

    print ("No email address available");

else

    print ($email);
```

9.1.6 使用预处理语句

前面介绍的exec()和query()方法，都能执行SQL语句，并立刻返回结果集。PDO也可以分步骤来预处理和执行SQL语句。具体做法是：先使用数据库句柄的prepare()方法来获得一个语句句柄，再使用这个语句句柄来执行语句：

```
$sth = $dbh->prepare ($stmt);

$sth->execute ();
```

在执行某条会修改行的语句之后，你便可以调用rowCount()方法来获得实际受影响的行数：

```
$count = $sth->rowCount();
```

如果语句返回了行，则可以使用像 `fetch()`、`columnCount()` 这样的方法来处理。如果想确定结果集里有多少个行，则需要在获取它们时对其进行计数。（`rowCount()` 方法只适用于那些会修改行的语句。）

预处理语句提供了一些重要功能。

语句字符串可以包含占位符，而不仅仅只有字面数据值。在对语句进行预处理之后，需要在执行该语句之前把特定的数据值与相应的占位符绑定在一起，PDO 会负责完成对特殊字符和NULL值的转义和引号处理。把值绑定到占位符的办法有很多种，具体详见9.1.7节。

经过预处理的语句可以反复执行多次。这样可以避免在每次执行时都需要进行预处理而产生额外的开销。当需要多次执行语句时，这种做法能带来好处，因为它可以提供更好的性能。例如，在插入多个行时，可以先用`prepare()`对INSERT语句预处理一次。然后，在一个循环里（每循环一次提供一个行的数据值）反复调用`execute()`执行它，同时使用占位符来将这些值与预处理语句绑定在一起。

9.1.7 利用占位符来处理数据引号问题

在PHP里构造SQL语句串时，一定要对引号问题特别小心，就像用其他类似C和Perl这样的语言开发一样。假设，你正在构造一条往表里插入新行的语句。在这个语句串里，你可能会为每一个插到某个字符串列里的值加上引号：

```
$last = "O'Malley";
```

```
$first = "Brian";
```

```
$expiration = "2013-09-01";
```

```
$stmt = "INSERT INTO member (last_name,first_name,expiration)"
```

```
." VALUES('$last','$first','$expiration)";
```


这里的问题在于，有一个带引号的值（即 O'Malley），其自身也带有一个引号。如果把这条语句发送给MySQL服务器，则会导致一个语法错误。为解决此问题，在C语言程序里，可以调用 `mysql_real_escape_string()` 或 `mysql_escape_string()` 函数。在Perl DBI脚本里，我们可以使用 `quote()` 函数。在PHP里，PDO的数据库句柄有一个 `quote()` 方法，可以达到同样的目的。例如，调用 `quote("O'Malley")` 之后，返回的值为 `'O\'Malley'`。如果在构造语句时使用了 `quote()` 方法，则可以把该方法返回的值直接插到语句串里，而不用自己再添加额外的引号：

```
$last = $dbh->quote ("O'Malley");

$first = $dbh->quote ("Brian");

$expiration = $dbh->quote ("2013-09-01");

$stmt = "INSERT INTO member (last_name,first_name,expiration)"

      ." VALUES($last,$first,$expiration)";
```

不幸的是，与DBI模块里的同名函数相比，PDO的 `quote()` 方法存在某些不足，限制了它的使用。

不是所有驱动程序都实现了这个方法。如果未实现，则在调用它时会返回FALSE，而不会返回一个用引号引起来的字符串。

对于NULL值，当你把单词“NULL”插到语句串里时，可能并不想要带上任何引号。但是，如果把NULL传递给 `quote()` 方法，它将返回一个空串（''）。为解决此问题，必须先进行测试，然后再根据它是否为NULL来区别对待。

因为这些不足，我个人认为应该尽量避免使用 `quote()`，除非你确切知道要处理的字符串数据都是非NULL字符串。一种更好的办法是使用预处理语句。这样一来，你便可以在SQL语句里指定占位符，然后让PDO替你处理所有的引号问题。当你预处理SQL语句时，需要用“?”字符来作为占位标记，以表明数据值所在的位置。当执行该语句时，需要以参数数组的形式来提供数据值：

```
$stmt = "INSERT INTO member (last_name,first_name,expiration)
VALUES(?,?,?);"
```

```
$sth = $dbh->prepare ($stmt);
```

```
$sth->execute (array ("O'Malley", "Brian", "2013-09-01"));
```

PDO会负责处理字符串里的特殊字符，并且能正确处理像数字和NULL那样的非字符串值。

另一种提供数据值的办法是，在调用execute()方法之前，用bindValue()把它们分别绑定到相应的占位符上：

```
$stmt = "INSERT INTO member (last_name,first_name,expiration)
VALUES(?,?,?);"
```

```
$sth = $dbh->prepare ($stmt);
```

```
$sth->bindValue (1, "O'Malley");
```

```
$sth->bindValue (2, "Brian");
```

```
$sth->bindValue (3, "2013-09-01");
```

```
$sth->execute ();
```

前面几个示例使用的都是位置型占位符，即“?”标记。它们都一样，唯一的区别在于它们在语句串里的位置不同而已。PDO还支持名字型占位符风格，即占位符由一个冒号和一个紧跟其后的名字构成。操作流程是：先对要执行的语句进行预处理，然后向execute()方法传递一个关联数组——其中，每一个值都与相应的名字连在一起。

```
$stmt = "INSERT INTO member (last_name,first_name,expiration)
VALUES (:last_name, :first_name, :expiration)";
```

```
$sth = $dbh->prepare ($stmt);
```

```
$sth->execute (array (
    ":last_name" => "O'Malley",
    ":first_name" => "Brian",
    ":expiration" => "2013-09-01"
));
```

另外，还可以在调用`execute()`方法之前，先把每个值与它的占位符名字绑定在一起：

```
$stmt = "INSERT INTO member (last_name,first_name,expiration)
VALUES (:last_name,:first_name,:expiration)";

$sth = $dbh->prepare ($stmt);

$sth->bindValue (":last_name", "O'Malley");

$sth->bindValue (":first_name", "Brian");

$sth->bindValue (":expiration", "2013-09-01");

$sth->execute ();
```

名字型占位符的一个优点是，当有大量参数时，占位符和数据值之间的关联易于跟踪。

9.1.8 出错处理

在与MySQL进行交互时，对出错处理的编排非常关键。如果你认为每一个调用都会成功，那么当真的有错误出现时，你将很难找到脚本不工作的原因。

当你尝试调用`new PDO()`来连接数据库服务器时，如果连接失败，则会抛出一个异常。假设连接成功，并且你得到了一个有效的数据库

句柄，PDO将会根据当前的PDO出错处理模式，处理那些使用了此句柄的操作所发生的错误。出错模式的设置方式如下：

```
$dbh->setAttribute(PDO::ATTR_ERRMODE, mode_value);
```

PDO支持下面三种出错处理模式。

PDO::ERRMODE_SILENT。PDO 只会为那个引起错误的对象设置出错信息，其他什么事情都不做。这是默认的出错处理模式。

PDO::ERRMODE_WARNING。这个类似于静默模式，不同之处在于PDO除了设置出错信息以外，还会抛出一条警告消息。

PDO::ERRMODE_EXCEPTION。在设置出错信息之后，PDO还会抛出一个异常。

对于以上三种情况，如果你确切知道是哪个对象发生了错误，那么可以调用该对象的`errorCode()`或`errorInfo()`方法来获得出错信息。

`errorCode()`会返回一个由 5个字符构成的SQLSTATE 值。如果返回值为PDO::ERR_NONE ("00000")，则表明没有发生任何错误。

`errorInfo()`会返回一个由3个元素构成的数组，其中包括一个SQLSTATE值、一个与驱动程序有关的出错代码和一条出错消息。对于MySQL，后面两个值分别为一个数字形式的出错代码和一条描述性的出错消息。

如果这个句柄操作成功，则返回值可能是一个单元素数组，其中包含的SQLSTATE值为PDO::ERR_NONE ("00000")。

在静默模式或警告模式下，出错处理会对每一个有可能失败的PDO操作的结果进行检查。例如：

```
if (!($sth = $dbh->prepare ("SELECT * FROM non_existent_table")))  
  
    die ("Cannot prepare statement: " . $dbh->errorCode () . "\n");
```

```
else if (!$sth->execute ())
```

```
die ("Cannot execute statement: " . $sth->errorCode () . "\n");
```

请注意，在调用`errorCode()`时，会用到那个发生错误的句柄。对于`errorInfo()`也是如此。

如果启用了异常模式，则在某个PDO操作发生错误时，PHP将抛出一个`PDOException`异常。如果没有发生任何错误，则表明操作成功。否则，如果你没捕捉这个错误的话，那么该异常将导致PHP终止脚本的执行。如果想要捕捉错误，则必须把可能失败的代码放在一个`try`语句块里，并且把相应的出错处理代码放在对应的`catch`语句块里：

```
try
{
    # ...执行数据库操作...
}

catch (PDOException $e)
{
    # ...处理错误...
}
```

那个异常对象（即本示例里的那个`$e`）有它自己的提供出错信息的方法。

`getCode()`方法会返回一个出错代码。

`getMessage()`方法会返回一个包含出错消息的字符串。

下面这个示例启用了异常机制，并且演示了在语句执行失败时如何显示出错信息：

```

$dbh->setAttribute(PDO::ATTR_ERRMODE,
PDO::ERRMODE_EXCEPTION);

try

{

$dbh->exec ("DELETE FROM non_existent_table");

}

catch (PDOException $e)

{

# 输出来自异常对象的出错信息

print ("getCode value: " . $e->getCode () . "\n");

print ("getMessage value: " . $e->getMessage () . "\n");

# 输出来自数据库句柄的出错信息

print ("errorCode value: " . $dbh->errorCode () . "\n");

print ("errorInfo value: " . join (",", $dbh->errorInfo ()) . "\n");

}

```

这个示例不仅会显示来自异常对象（\$e）的出错信息，还会显示来自数据库句柄对象（\$dbh）的出错信息。之所以能做到这一点，是因为，try语句块里使用了那个唯一的PDO句柄\$dbh。如果在try语句块里使用多个PDO句柄，那么将无法知道是catch语句块里的哪一个句柄引发的错误，此时你只能依赖异常对象的方法。另外，你也可以重新调整这段代码，把每个PDO句柄隔离到它自己的try/catch语句块里。

有些PHP函数或操作在发生错误时，不仅会返回一个状态值，还会生成一条出错消息。在Web上下文里，这种出错消息会出现在发送给客户端浏览器的页面里，而这可能不是你想要的结果。如果不想让用户看到函数在一般情况下发生的出错消息（有可能是比较隐蔽的），那么可以在函数名的前面加上一个@运算符。例如，你不想显示名为 `some_func()` 的函数发出的错误信息，以便你可以用一种更合适的方式来报告错误，那么可以像下面这样做：

```
$status = @some_func ();
```

9.2 PHP脚本实战

本章的后续部分将实现我们在第1章里提出的一些到目前仍未完成的目标。

对于成绩考评项目，需要编写一个脚本，可以录入和修改考试与测验分数。

在美史联盟的Web站点上，向访问者提供一个关于美国总统生平事迹的小测验，并且设计成交互式的，以便可以即时生成测验题。

还想让美史联盟成员能够在线修改他们的成员名录条目。这样可以使信息始终保持最新，并且可以减少联盟秘书的工作量。

每个脚本都会生成多个彼此关联的Web页面，因此在对脚本的前后两次调用之间，需要采用在其创建的Web页面里嵌入信息的方式来进行通信。如果对Web页面之间的通信机制还不熟悉，请参考8.4.2.4节。

9.2.1 一个在线录入分数的应用程序

在本节里，我们将把注意力转移到成绩考评项目上，编写一个 `score_entry.php` 脚本，用于管理学生考试和测验的分数。此项目的Web目录为 `gp` 目录，位于Apache文档树的根目录下，与它对应的站点URL为：

```
http://localhost/gp/
```

这个目录目前还是空的，因此当访问者请求这个URL时，可能只会看到一条出错信息——“Page not found”（页面未找到），或者看到一个空目录列表页面。为修正这个问题，可以创建一个名为index.php的简短脚本，并将它放在gp目录下，作为成绩考评项目的主页。下面这个脚本足以满足当前的需要。它包含两个链接：一个指向我们在8.4.5节为成绩考评项目编写的score_browse.pl脚本；另一个指向我们将要编写的score_entry.php脚本。

```
<?php

# 成绩考评项目主页

require_once "sampdb_pdo.php";

$title = "Grade-Keeping Project";

html_begin ($title, $title);

?>

<p>

<a href="/cgi-bin/score_browse.pl">View</a> test and quiz scores

</p>

<p>

<a href="score_entry.php">Enter or edit</a> test and quiz scores

</p>

<?php

html_end ();

?>
```


在sampdb发行版的phpapi/gp目录下可以找到这个index.php脚本。你可以将它复制到Web服务器文档树中的gp目录下。

下面来考虑一下，如何设计和实现score_entry.php脚本。它包含两项功能：一是让我们可以录入一组考试或测验分数；二是让我们可以修改已有的分数。录入功能可以满足往数据库里添加新分数的需要。而编辑功能是今后修改分数所必需的，例如：有学生因生病或其他原因而进行了补考，则需要将他们的补考成绩录入数据库；或者更正我们录入错误的考试成绩。从概念上来讲，这个考分录入脚本的基本操作如下所示。

初始页面会显示一个已知考试事件清单，你既可以从中选择一个事件，也可以指明想要创建一个新的考试事件。

如果你选择创建新的事件，那么这个脚本将会展示一个页面，要求你指定日期和事件类别（是考试，还是测验）。在把新事件添加到数据库之后，脚本将重新显示事件清单页面，这时的页面里就有了那个新事件。

如果从清单里选择一个现有事件，那么脚本将呈现一个考分录入页面，显示考试事件ID、日期、类别、一份班级学生名单，以及一个“提交”按钮。学生名单的每一行会列出每位学生的姓名和他在该次考试中取得的分数。如果选择新的考试事件，那么所有分数都将为空白。如果选择现有考试事件，那么显示出的结果将是你之前录入的那些分数。你可以填充或修改这些分数，然后单击“提交”按钮。此时，脚本将把这些分数输到score表里，或者修改现有的分数。必须把这个操作当成事务来完成，才能确保所有分数都是以整体为单位修改成功的；或者在发生错误时，都是整体取消的。

9.2.1.1 收集PHP里的Web输入

在实现score_entry.php脚本之前，我们必须先讨论一下PHP里的输入参数是如何工作的。这个脚本需要完成多个不同的动作，这意味着它必须把某个状态值从一个页面传递到另一个页面，以便能够弄清在每次调用时应该做什么事情。一种传递状态值的办法是，在URL的末尾传递参数。例如，我们可以像下面这样，把一个名为action的参数添加到这个脚本的URL里：

`http://localhost/gp/score_entry.php?action=value`

参数值也可能是来自用户提交的表单内容。作为表单提交的一部分，由用户浏览器返回的表单里的每个字段都拥有一个名字和一个值。

PHP把输入参数放在特殊数组里，供脚本使用。被编码在URL末尾、作为get请求的一部分发送的参数，被放置在全局数组 `$HTTP_GET_VARS` 和超全局数组 `$_GET` 里。post请求（如其 `method` 属性的值为 `post` 的表单）里接收到的参数，都被放置在全局数组 `$HTTP_POST_VARS` 和超全局数组 `$_POST` 里。

如果不是在PHP脚本的顶层（如在函数定义内部）里使用全局数组，那么必须显式声明它们。超全局数组不需要特别声明便可以在任何层次里访问到。为简单起见，我们将使用超全局数组 `$_GET` 和 `$_POST`。（`$HTTP_GET_VARS` 和 `$HTTP_POST_VARS` 都已被弃用。）

`$_GET` 和 `$_POST` 都是关联数组，其元素的键为各参数的名字。例如，在URL里有一个发送的 `action` 参数，那么可以在PHP脚本里通过 `$_GET["action"]` 获得那个值。假设，有一个表单包含 `name` 和 `address` 两个字段。当用户提交这个表单时，Web服务器将调用脚本来处理这个表单的内容。如果这个表单是以 `get` 请求形式提交的，那么这个脚本可以通过检查变量 `$_GET["name"]` 和 `$_GET["address"]` 的值来弄清用户在表单里输入的具体值。如果这个表单是以 `post` 请求形式提交的，那么这些变量就 `$_POST["name"]` 和 `$_POST["address"]` 里。

对于表单里包含大量字段的情形，为它们一一指定一个唯一的名字显然很不方便。由于PHP支持以数组形式传递表单，因此这件事情会很好处理。如果把字段命名为 `x[0]`、`x[1]` 等，那么PHP将把它们存储在 `$_GET["x"]` 或 `$_POST["x"]` 里，而其值自身即为一个数组。如果把这个数组值赋给变量 `$x`，可以通过 `$x[0]`、`$x[1]` 等方式去访问这个数组的元素。

在大多数情况下，我们不关心参数是用 `get` 提交的，还是用 `post` 提交的，因此我们可以编写一个实用例程 `script_param()`，让它接受一个参数名，并在那两个数组里找出这个参数的值。如果参数不存在，那么例程返回 `NULL`：

```

function script_param ($name)

{

$val = NULL;

if (isset ($_GET[$name]))

    $val = $_GET[$name];

else if (isset ($_POST[$name]))

    $val = $_POST[$name];

if (get_magic_quotes_gpc ())

    $val = remove_backslashes ($val);

return ($val);

}

```

不管输入参数存储在哪个数组里，`script_param()` 函数都能让脚本简单地通过参数名来访问它们的值。在提取出参数值以后，`script_param()` 还可以处理这个参数值，将其传递给 `remove_backslashes()` 函数。这样做的目的在于，要与PHP初始化文件里使用下面这条语句来启用`magic_quotes_gpc`设置的情况相吻合：

```
magic_quotes_gpc = 0n;
```

如果打开了此选项，那么PHP将往这些参数值里添加反斜线，以便把像引号或反斜线这样的特殊字符引起来。这些额外的反斜线会让检查参数值是否合法的工作变得很有难度，因此需要使用 `remove_backslashes()` 函数把它们去除。在PHP里，有可能创建出接受嵌套形式数组的参数，因此它需要使用递归算法来实现：

```
function remove_backslashes ($val)
```

```

{

if (is_array ($val))

{

foreach ($val as $k => $v)

$val[$k] = remove_backslashes ($v);

}

else if (is_null ($val))

$val = stripslashes ($val);

return ($val);

}

```

Web输入参数与register_globals

你可能对PHP的配置选项register_globals并不陌生，其作用是使来自Web的输入参数，直接被注册为脚本里的变量。例如，一个名为 x 的表单字段或 URL 参数，将直接存储到你脚本中一个名为\$x的变量里。不幸的是，在启用这项功能之后，客户端便可以在你的脚本里直接创建出一些超出你意愿的变量。这存在安全风险，因此PHP的开发人员都建议禁用register_globals选项。script_param() 例程故意只使用了专为输入参数提供的数组，这样即增加了安全性，也使得不管register_globals的设置如何，它都可以正常工作。

9.2.1.2 显示和输入分数

既然我们已有了方便提取 Web 输入参数的方法，那么现在便可以利用它来编写score_entry.php脚本了。这个脚本需要在它自己的前后两次调用中交流信息。为做到这一点，我们将使用一个名为action的参数。当脚本执行时，可以像下面这样获得这个参数的值：

```
$action = script_param ("action");
```

如果这个参数未被设置，那么这是脚本的首次调用；否则，它可以根据变量\$action的值来决定做什么事情。下面是score_entry.php脚本的基本框架：

```
<?php

# score_entry.php - 成绩考评项目的分数录入脚本

require_once "sampdb_pdo.php";

# 定义动作常量

define ("SHOW_INITIAL_PAGE", 0);

define ("SOLICIT_EVENT", 1);

define ("ADD_EVENT", 2);

define ("DISPLAY_SCORES", 3);

define ("ENTER_SCORES", 4);

# ...此处放置输入处理函数 ...

$title = "Grade-Keeping Project -- Score Entry";

html_begin ($title, $title);

$dbh = sampdb_connect ();

# 确定要执行的动作（如果未指定动作，

# 则默认显示初始页面）

$action = script_param ("action");
```

```
if (is_null ($action))

$action = SHOW_INITIAL_PAGE;

switch ($action)

{

case SHOW_INITIAL_PAGE:      # 呈现初始页面

display_events ($dbh);

break;

case SOLICIT_EVENT:          # 请求新的考试事件信息

solicit_event_info ();

break;

case ADD_EVENT:              # 往数据库里添加新考试事件

add_new_event ($dbh);

display_events ($dbh);

break;

case DISPLAY_SCORES:         # 显示所选考试事件的分数

display_scores ($dbh);

break;

case ENTER_SCORES:           # 输入新分数或编辑后的分数

enter_scores ($dbh);
```

```

display_events ($dbh);

break;

default:

die ("Unknown action code ($action)\n");

}

$dbh = NULL;  # 关闭连接

html_end ();

?>

```

`$action`变量可以接受多种值，因此我们使用了一个`switch`语句来测试它。在PHP里，`switch`语句与C语言里的`switch`很相似，此处用它来确定脚本应该采取什么样的动作，并调用实现了这个动作的函数。为了避免必须使用文字型的动作值，`switch`语句里使用了一些符号动作名字。这些符号都是在脚本的开头部分利用PHP语言的`define()`构造来初始化的。

下面依次分析处理这些动作的函数。第一个是 `display_events()` 函数，它将呈现一个允许的考试事件列表，所采用的方式是：从MySQL里检索`grade_event`表的各个行，并显示它们。这个列表里的每一个行所列出的信息包括事件 ID、日期和事件类别（考试或测验）。Web页面里的事件ID都是一些超链接，通过它们可以编辑考试的分数。在这些考试事件行的后面，`display_events()` 函数又多添加了一行，其中包含一个用于创建新考试事件的超链接。下面是`display_events()` 函数的代码：

```

function display_events ($dbh)

{

print ("Select an event by clicking on its number, or select\n");

```

```

print ("New Event to create a new grade event:<br /><br />\n");

print ("<table border=\"1\">\n");

# 输出一行表头

print ("<tr>\n");

display_cell ("th", "Event ID");

display_cell ("th", "Date");

display_cell ("th", "Category");

print ("</tr>\n");

# 呈现考试事件列表。将每一个事件ID 与一个链接相关联,

# 用于显示考试事件的分数。

$stmt = "SELECT event_id, date, category
FROM grade_event ORDER BY event_id";

$stmt = $dbh->query ($stmt);

while ($row = $sth->fetch ())
{

print ("<tr>\n");

$url = sprintf ("%s?action=%d&event_id=%d",
script_name (),
DISPLAY_SCORES,
$row["event_id"]);

```



```

        display_cell ("td",
"<a href=\""$url\"">"
$row["event_id"]
."</a>",
FALSE);

display_cell ("td", $row["date"]);

display_cell ("td", $row["category"]);

print ("</tr>\n");

}

# 多添加一个链接，用于创建新的考试事件

print ("<tr align=\"center\">\n");

$url = sprintf ("%s?action=%d",
script_name (),
SOLICIT_EVENT);

display_cell ("td colspan=\"3\"",
"<a href=\""$url\"">Create New Event</a>",
FALSE);

print ("</tr>\n");

print ("</table>\n");

}

```

这些超链接可用于再次调用score_entry.php，我们在构造它们的URL时，使用了一个名为script_name()的函数，它会确定出脚本自身的路径名。script_name()函数非常有用，因为它可以让你不用将脚本的文件名硬编码在代码里。（如果你把脚本的文件名明确地写在代码里，而你之后又重新命名了这个文件，那么这个脚本将无法正常工作。）在sampdb_pdo.php文件里可以找到这个函数。

script_name()函数有点像script_param()函数，因为它也可以访问PHP的超全局数组。不过，它使用的是不一样的数组，因为脚本名称是Web服务器所提供的信息中的一部分，而非输入参数的一部分。下面是script_name()函数的代码：

```
function script_name ()  
  
{  
  
    return ($_SERVER["SCRIPT_NAME"]);  
  
}
```

display_events()函数所使用的display_cell()函数，会生成考试事件表格里的各个单元格：

```
# 显示HTML表格的一个单元格。其中，$tag为标记名（即表头  
# 或数据单元所用的"th"或者"td"）；$value为显示值；  
  
# $encode只能为真或假，表明在显示值之前是否  
# 要对该值进行HTML编码。$encode是可选的，  
# 并且默认为真。
```

```
function display_cell ($tag, $value, $encode = TRUE)  
  
{
```

```

if (strlen ($value) == 0) # 值为空？ 还是未设置？

$value = "&nbsp;";

else if ($encode) # 如果需要，则执行HTML编码

$value = htmlspecialchars ($value);

print("<$tag>$value</$tag>\n");

}

```

如果你在display_events()呈现的表格里选了链接“Create New Event”（创建新事件），那么score_entry.php脚本会被再次调用，同时会带有SOLICIT_EVENT动作。如此一来，它会触发对solicit_event_info()函数的调用，显示一个表单，供你输入新考试事件的日期和类别。下面是solicit_event_info()函数的代码：

```

function solicit_event_info ()

{

printf("<form method=\"post\" action=\""%s?action=%d\">\n",

script_name (),

ADD_EVENT);

print("Enter information for new grade event:<br /><br />\n");

print("Date: ");

print("<input type=\"text\" name=\"date\" value=\"\" size=\"10\"

/>\n");

print("<br />\n");

```

```

print ("Category: ");

print ("<input type=\"radio\" name=\"category\" value=\"T\"");

print (" checked=\"checked\" />Test\n");

print ("<input type=\"radio\" name=\"category\" value=\"Q\"
/>Quiz\n");

print ("<br /><br />\n");

print ("<input type=\"submit\" name=\"submit\" value=\"Submit\"
/>\n");

print ("</form>\n");

}

```

`solicit_event_info()` 函数生成的表单里包含：一个用于输入日期的输入框、一对儿用于指定新事件类别是考试还是测验的单选按钮，以及一个提交按钮。默认的事件类别为'T'（即考试）。（在这里，这个脚本构建表单的方式是输出字面形式的HTML。对于本章后面的脚本，我们将开发一组辅助函数，用于生成各个表单元素。）

当你填写完此表单并提交它时，`score_entry.php` 脚本会被再次调用，只是这次的action值为ADD_EVENT。接着，`add_new_event()` 函数将被调用，往`grade_event`表里输入一个新行：

```

function add_new_event ($dbh)

{

$date = script_param ("date");           # 获得日期和事件类别

$category = script_param ("category");    # 由用户输入

```

if (empty (\$date)) # 确保输入了日期，并且符合ISO 8601 格式的要求

```
die ("No date specified\n");
```

```
if (!preg_match ('/^\d{4}\D\d{1,2}\D\d{1,2}$/', $date))
```

```
die ("Please enter the date in ISO 8601 format (CCYY-MM-DD)\n");
```

```
if ($category != "T" && $category != "Q")
```

```
die ("Bad event category\n");
```

```
$stmt = "INSERT INTO grade_event (date,category) VALUES(?,?)";
```

```
$sth = $dbh->prepare ($stmt);
```

```
$sth->execute (array ($date, $category));
```

```
}
```

`add_new_event()` 函数会调用 `script_param()` 库例程来访问与新事件录入表单里的 `date` 和 `type` 字段相对应的参数值。然后，它会执行几项简单的安全性检查。

日期不应该为空，并且必须按ISO 8601格式输入。`preg_match()` 函数会执行一个针对ISO 8601格式的模式匹配：

```
preg_match ('/^\d{4}\D\d{1,2}\D\d{1,2}$/', $date)
```

这里使用了单引号，目的在于防止把美元字符（\$）和反斜线字符（\）解释为特殊字符。如果日期由3组以非数字字符分隔的数字序列构成，那么测试的结果为真。虽然它不能保证万无一失，但是它很容易添加到脚本里，并且也确实能捕获到许多常见的错误。

为了更加安全，还可以在插入数据前，通过设置SQL模式来启用输入数据限制功能：

```
$dbh->exec ("SET sql_mode = 'TRADITIONAL'");
```

事件类别必须是grade_event表的category列里允许使用的那些值（即'T'或'Q'）当中的某一个。

如果这些参数值看起来没问题，那么 add_new_event() 会往 grade_event 表里输入一个新行。这里的语句执行代码使用到了占位符，其目的在于确保插到查询字符串里的各个数据不会出现引号问题。在执行这条语句之后，add_new_event() 会返回到脚本的主体部分（即switch语句处），再次显示那个事件列表，以便可以选择新的事件，并开始向它输入分数。

当你从display_events() 函数显示的事件列表里选择某个条目时，score_entry.php脚本将调用display_scores() 函数。每一个事件链接都包含有一个被编码为 event_id 参数的事件编号，因此 display_scores() 会获得这个参数值，对它进行检查，以确定它是一个整数，并将它用在一个查询里，检索出每个学生的行和在此次考试中各个学生所拥有的分数。下面是display_scores() 函数的代码：

```
function display_scores ($dbh)
{
    # 获得事件ID 编号，它必须像一个整数

    $event_id = script_param ("event_id");

    if (!ctype_digit ($event_id))

        die ("Bad event ID\n");

    # 查询给定考试事件的分数

    $stmt = "

    SELECT

    student.student_id, student.name, grade_event.date,
```

```

score.score AS score, grade_event.category

FROM student

INNER JOIN grade_event

LEFT JOIN score ON student.student_id = score.student_id

AND grade_event.event_id = score.event_id

WHERE grade_event.event_id = ?

ORDER BY student.name";

$sth = $dbh->prepare ($stmt);

$sth->execute (array ($event_id));

# 获取各个行，并放入一个数组，从而可以知道总的行数

$rows = $sth->fetchAll ();

if (count ($rows) == 0)

    die ("No information was found for the selected
event\n");

printf ("<form method=\"post\" action=\""%s?
action=%d&event_id=%d\">\n",

script_name (),

ENTER_SCORES,

$event_id);

# 把分数输出为一个HTML 表格

```

```
for ($row_num = 0; $row_num < count ($rows); $row_num++)

{

$row = $rows[$row_num];

# 输出考试事件信息和处于第一行的表头

if ($row_num == 0)

{

    printf ("Event ID: %d, Event date: %s, Event category:
%s\n",

$event_id,

$row["date"],

$row["category"]);

    print ("<br /><br />\n");

    print ("<table border=\"1\">\n");

    print ("<tr>\n");

    display_cell ("th", "Name");

    display_cell ("th", "Score");

    print "</tr>\n";

}

print ("<tr>\n");

display_cell ("td", $row["name"]);
```



```

        $col_val = sprintf("<input type=\"text\" name=\"score[%d]\"",
$row["student_id"]);

        $col_val .= sprintf(" value=\"%d\" size=\"5\" /><br />\n",
$row["score"]);

        display_cell("td", $col_val, FALSE);

        print("</tr>\n");

    }

    print("</table>\n");

    print("<br />\n");

    print("<input type=\"submit\" name=\"submit\" value=\"Submit\"
/>\n");

    print "</form>\n";

}

```

`display_scores()` 函数用于检索选中那次考试的分数信息的那个查询，并不是简单的两个表之间的连接，因为那样会漏掉那些没有参加该次考试的学生。尤其是，对于新的考试，这个连接不会查询到任何行，并且那个输入表单也会是空的！实际上，使用LEFT JOIN可以强制把每位学生的行都检索出来。不管这个学生在score表里是否有分数，都没问题。如果某个学生对于某次事件没有分数，那么查询检索到的值将为 NULL。（在2.8.3节有一个查询，它与`display_scores()`函数用于从My SQL检索分数行的这个查询很相似。）

这个脚本会把查询检索到的分数以输入字段的形式放入表单，输入字段的名字类似于score[n]，其中，n为student_id值。你可以输入或编辑这个分数，然后把这个表单提交到数据库里。当浏览器把这个

表单发送回Web服务器时，PHP会把这些输入字段转换为数组的元素，这个数组与名字score相关联，可以按下面这种方式来检索：

```
$score = script_param ("score");
```

这个数组的元素是以学生ID号为键的，因此很容易把每个学生与通过表单提交来的相应考试分数关联起来。对这个表单的处理可能需要执行多条语句（每个学生一条），并且我们不希望只有部分更新操作能成功。在第1章里，我们把score表创建为一个InnoDB表。这样，我们便能获得 InnoDB的事务处理功能所带来的好处。尤其是，把整个数据录入操作作为一个事务来处理之后，我们可以确保这个操作会以原子单元方式执行。如此一来，这个数据更改要么全部成功，要么不会更改任何内容。PDO里的事务处理有这样一个通用结构（假设已启用了针对PDO错误的异常捕获机制）：

```
try

{

$dbh->beginTransaction ();           # 开始事务

# ...执行数据库操作 ...

$dbh->commit ();                     # 事务成功

}

catch (PDOException $e)

{

$dbh->rollback ();                   # 事务失败

}
```

score_entry.php脚本使用了上述结构来保证数据录入操作的整体性。此外，为了防止回滚操作执行失败时终止脚本，将回滚操作被在

了它自己的try/catch块里。

`enter_scores()` 函数负责处理表单内容，确定哪些分数需要更新或删除：

```
function enter_scores ($dbh)

{

# 获得事件ID 编号以及该次事件的分数数组

$event_id = script_param ("event_id");

$score = script_param ("score");

if (!ctype_digit ($event_id)) # 必须像整数

die ("Bad event ID\n");

# 预处理那些要反复执行的语句

$sth_del = $dbh->prepare ("DELETE FROM score

WHERE event_id = ? AND student_id = ?");

$sth_repl = $dbh->prepare ("REPLACE INTO score

(event_id, student_id, score)

VALUES (?, ?, ?)");

# 在一个事务内输入所有分数

try

{

$dbh->beginTransaction ();
```

```
$blank_count = 0;

$nonblank_count = 0;

foreach ($score as $student_id => $new_score)
{

$new_score = trim ($new_score);

if (empty ($new_score))

{

# 如果表单里没有提供学生分数，则把此学生之前

# 在数据库里可能已有的分数删除

++$blank_count;

$sth = $sth_del;

$params = array ($event_id, $student_id);

}

else if (ctype_digit ($new_score)) # 必须像整数

{

# 如果提供了一个分数，则替换

# 数据库里可能已存在的分数

++$nonblank_count;
```

```
$sth = $sth_repl;

$params = array ($event_id, $student_id, $new_score);

}

else

{

throw new PDOException ("invalid score: $new_score");

}

$sth->execute ($params);

}

# 事务执行成功，提交它

$dbh->commit ();

printf ("Number of scores entered: %d<br />\n", $nonblank_count);

printf ("Number of scores missing: %d<br />\n", $blank_count);

}

catch (PDOException $e)

{

printf ("Score entry failed: %s<br />\n",

htmlspecialchars ($e->getMessage ()));
```

执行回滚；不过这里使用了一个空的异常处理器来捕捉回滚失败内容

```
try

{

$dbh->rollback ();

}

catch (PDOException $e) { }

}

print("<br />\n");

}
```

此脚本会通过遍历\$score数组来获得并处理学生ID值以及与它们相关联的考试分数。

如果在把考试分数末尾的空白去掉之后，分数为空，则表明没有输入任何内容。但是，有可能以前曾有一个考试分数，因此脚本会尝试删除它。（可能我们以前错误地为实际并未参加考试的学生输入了考试分数，现在需要删除它。）如果该学生没有任何分数，那么DELETE语句将不会找到任何需要删除的行，但是这不会带来任何坏处。

如果考试分数不为空，那么此函数将对值执行一些基本的输入验证，如果值看上去像一个整数，则接受它。请注意，这里的整数测试方法使用的是模式匹配，而非 PHP的is_int()函数。后者只是用于测试变量的类型是否为整数，可是表单值都被编码成了字符串，因此这里并不适合使用它。is_int()函数在遇到字符串时都返回FALSE，即使这个字符串里只包含数字字符，也是如此。这里需要的是对字符串的内容进行检查。如果从字符串\$str的开头到结尾所包含的每一个字符都是数字，那么下面这个函数将返回TRUE：

`ctype_digit ($str)`

如果考试分数没什么问题，则把它添加到score表里。这里使用的语句是REPLACE，而不是 INSERT，因为我们可能是在替换已存在的考试分数，而不是要输入新的分数。如果某个学生在这次考试里没有任何分数，那么REPLACE语句将添加一个新行，其效果与INSERT语句一样。否则，REPLACE语句会将原有的分数替换为新的分数。

在进入循环之前，这个脚本会调用`beginTransaction()`函数，禁用自动提交模式。在退出循环之后，如果没有发生错误，脚本会提交事务。如果发生了错误，脚本会回滚事务。

`score_entry.php`脚本的内容就这些。现在，你可以通过Web浏览器来录入和编辑所有的考试分数了。这个脚本有一个明显的不足，即没有提供任何安全措施，因此能连接上Web服务器的人就能编辑这些分数。在稍后编写的用于美史联盟成员条目编辑的脚本里，我们将演示一种简单的验证机制，该机制也能用到这个脚本里。

9.2.2 创建交互式的在线测验

美史联盟网站的目标之一是：提供一个在线版的测验，其形式类似于在联盟通讯《Chronicles of U. S. Past》的儿童专区里发布的那些小测验。事实上，我们创建的那个`president`表恰到好处，我们完全可以把它用作历史知识小测验的问题来源。我们现在就来动手做这件事，所用的脚本名称为`pres_quiz.php`。

基本思路是：随机挑选并询问与某位总统有关的问题；然后提示用户输入一个答案，并检查答案是否正确。脚本可以根据`president`表里的信息提出各种类型的问题，但是为简明起见，我们把问题限制在了只询问总统的出生地。另一个简化措施是提供选择题。用户只需要从一组选项里选择一个答案即可，这比让他们敲打键盘输入答案简单很多。对于脚本来说，与通过模式匹配检查用户可能输入的任何内容相比，将用户的选择与正确答案进行比较则显得更加简单。

`pres_quiz.php`脚本必须完成以下两件事。

在首次被调用时，应该利用`president`表里的信息生成并显示一道新的测验题。

在用户提交答案之后，脚本必须核对答案，并向用户反馈答案是否正确。如果答案不正确，那么脚本应该重新显示同样的测验题。否则，它应该生成并显示一道新的测验题。

这个脚本的整体思路非常简单。如果用户没有提交答案，则显示初始测验页面；否则，检查答案是否正确。下面是pres_quiz.php脚本的代码：

```
<?php

# pres_quiz.php - 用于测验用户对总统出生地的掌握情况的脚本

require_once "sampdb_pdo.php";

# ...此处放置测验处理函数 ...

$title = "U.S.President Quiz";

html_begin ($title, $title);

$dbh = sampdb_connect ();

$response = script_param ("response");

if (is_null ($response))      # 第一次调用

    present_question ($dbh);

else                          # 用户提交的表单响应

    check_response ($dbh);

$dbh = NULL;  # 关闭连接

html_end ();

?>
```


为创建测验题，我们需要组合使用ORDER BY RAND()和LIMIT 1，从 president 表里随机选取一个行。例如，想要随机选取一位总统的名字和出生地，则可以使用下面这样的查询来完成：

```
SELECT CONCAT(first_name, ', ', last_name) AS name,  
  
CONCAT(city, ', ', state) AS place  
  
FROM president ORDER BY RAND() LIMIT 1;
```

我们提的问题是：“这位总统的出生地在哪里？”上面的查询提供了名字和出生地。其中，名字即为问题中问到的那位总统，出生地即为该问题的正确答案。我们还需要准备几个不正确的选项，可以用一个类似的查询来获得：

```
SELECT DISTINCT CONCAT(city, ', ', state) AS place  
  
FROM president ORDER BY RAND();
```

我们可以从这个查询的结果中选取前 4个与正确答案不一样的值。为了防止正确答案出现在错误选项中，查询应该使用LIMIT 5，而非 LIMIT 4。这个查询使用了 DISTINCT，以避免可能出现的同一个出生地做为备选答案的情况。如果各位总统的出生地各不相同，就没有必要使用这个DISTINCT，但实际情况并非如此，可以执行下面这条语句来查看实际情况：

```
mysql> SELECT city, state, COUNT(*) AS count FROM president  
  
-> GROUP BY city, state HAVING count > 1;
```

city	state	count
Braintree	MA	2

负责生成测验题和各个备选答案的函数如下所示：

```
function present_question ($dbh)

{

# 执行语句，挑择某位总统，并获得出生地

$stmt = "SELECT CONCAT(first_name, ' ', last_name) AS name,
CONCAT(city, ' ', state) AS place
FROM president ORDER BY RAND() LIMIT 5";

$stmt = $dbh->query ($stmt);

$row = $sth->fetch ();

$name = $row["name"];

$place = $row["place"];

# 构造要显示的各个出生地备选项。

# 建立包含5个出生地的$choices 数组，

# 其中一个为正确答案。

$stmt = "SELECT DISTINCT CONCAT(city, ' ', state) AS place
FROM president ORDER BY RAND() LIMIT 5";

$stmt = $dbh->query ($stmt);

$choices[] = $place;  # 使用正确的选项来初始化数组

while (count ($choices) < 5 && $row = $sth->fetch ())
```

```

{

    if ($row["place"] != $place)

        $choices[] = $row["place"]; # 添加另一个错误选项

}

# 随机排列各个选项，然后显示表单

shuffle ($choices);

display_form ($name, $place, $choices);

}

```

`present_question()` 调用的 `display_form()` 函数会使用一个表单生成测验题，显示一位总统的名字、一组列出备选项的单选按钮和一个提交按钮。这个表单的基本用途是向用户显示测验题，但是它还需要做另外一些事情：它必须把测验题信息显示到客户端；当用户提交答案时，如果信息发回Web服务器，那么脚本要检查这个答案是否正确，如果答案不正确，则重新显示那个测验题。

显示测验题的事情比较好办：把总统的名字和可能的出生地显示出来即可。但检查答案的对错和重新显示测验题的工作则有点儿棘手。这需要访问那个正确答案，还需要重新生成测验题所需要的所有信息。一种做法是使用一组隐藏字段把所有必要的信息都包含在这个表单里。这些字段是表单的一部分，不会显示给用户看，但是用户提交答案时会返回这些字段值。

我们把这些隐藏字段取名为 `name`、`place` 和 `choices`，分别用于保存总统姓名、正确的出生地和备选答案集。使用 `implode()` 函数能很容易地把这些答案选项编码成一个字符串，即在各个值之间插入一个特殊分隔符，把它们连接起来。（如果稍后需要重新显示这道题，分隔符有助于我们使用 `explode()` 函数正确拆解这个字符串。）
`display_form()` 函数负责生成表单：

```

function display_form ($name, $place, $choices)
{
    printf("<form method=\"post\" action=\"%s\">\n", script_name ());

    hidden_field ("name", $name);

    hidden_field ("place", $place);

    hidden_field ("choices", implode ("#", $choices));

    printf ("Where was %s born?<br /><br />\n",
htmlspecialchars ($name));

    for ($i = 0; $i < 5; $i++)
    {

        radio_button ("response", $choices[$i], $choices[$i], FALSE);

        print("<br />\n");

    }

    print ("<br />\n");

    submit_button ("submit", "Submit");

    print ("</form>\n");

}

```

display_form() 函数使用了几个辅助函数来生成各个表单字段。第一个是 hidden_field() 函数，它负责为隐藏字段生成<input>标记：

```

function hidden_field ($name, $value)

```

```

{

    printf("<input type=\"%s\" name=\"%s\" value=\"%s\" />\n",
"hidden",
htmlspecialchars ($name),
htmlspecialchars ($value));

}

```

因为 `hidden_field()` 函数是一个通用性的例程，可能很多脚本都会用到它，因此在逻辑上可以把它放到库文件 `sampdb_pdo.php` 里。请注意，它还使用了 `htmlspecialchars()` 函数来对 `<input>` 标记的属性进行编码，以防止它们包含诸如引号之类的特殊字符。

另外两个辅助函数是 `radio_button()` 和 `submit_button()`，它们的代码如下所示：

```

function radio_button ($name, $value, $label, $checked)

{

    printf("<input type=\"%s\" name=\"%s\" value=\"%s\" %s
/>%s\n",
"radio",
htmlspecialchars ($name),
htmlspecialchars ($value),
($checked ? " checked=\""checked\"" : ""),
htmlspecialchars ($label));

}

```

```

function submit_button ($name, $value)

{

    printf("<input type=\"%s\" name=\"%s\" value=\"%s\" />\n",
"submit",
htmlspecialchars ($name),
htmlspecialchars ($value));

}

```

当用户在备选答案当中挑选了一个出生地并提交表单时，该答案将作为 `response` 参数的值返回到Web服务器。调用`script_param()`函数便可以看到`response`的值。据此可以判断出此脚本是处于首次执行状态，还是有用户正在提交此前显示在表单里的测验题答案。如果是首次调用，那么这个参数不会存在，因此此脚本的主体可以根据这个参数是否存在来确定应该做什么：

```

$response = script_param ("response");

if (is_null ($response))      # 第一次调用

    present_question ($dbh);

else                           # 用户提交的表单响应

    check_response ($dbh);

```

我们还得编写 `check_response()` 函数，将用户所选答案与正确答案进行比较。为此，需要用到隐藏字段 `name`、`place`和`choices` 里的值。我们已经把正确答案编码在表单的`palce`字段里，而用户的选择答案编码在`response` 字段里，因此只需要比较这两个字段的值即可。根据比较结果，`check_response()` 函数会提供一些反馈，然后生成并显示一道新的测验题，或者再次显示刚才那道测验题：

```

function check_response ($dbh)

{

$name = script_param ("name");

$place = script_param ("place");

$choices = script_param ("choices");

$response = script_param ("response");

# 用户选择的出生地正确吗?

if ($response == $place)

{

    print ("That is correct!<br />\n");

    printf ("%s was born in %s.<br />\n",

htmlspecialchars ($name),

htmlspecialchars ($place));

    print ("Try the next question:<br /><br />\n");

    present_question($dbh);

}

else

{

    printf ("\\"%s\" is not correct.    Please try again.<br />
<br

```

```
htmlspecialchars ($response));

    $choices = explode ("#", $choices);

    display_form ($name, $place, $choices);

}

}
```

至此，我们完成了所有的工作。在美史联盟的主页上增加一个指向pres_quiz.php脚本的链接，然后访问者便可以通过这个小测验来测试他们自己的知识水平。（你可以把index5.php从sampdb发行版的phpapi/ushl目录复制到Web服务器文档树里的ushl目录，并将其命名为index.php，以替换原有的那个同名文件。）

使用隐藏字段并不安全

pres_quiz.php脚本依靠隐藏字段来传递信息，这些信息是脚本下次调用时需要的，但用户不会看到它们。对于这种只是为了好玩而编写的脚本，这种方式没什么问题。但是，隐藏字段不应该用于传递任何不能让用户直接查看的信息，因为它们其实毫无安全可言。如果想知道其中的原因，可以把pres_quiz.php脚本安装到Web服务器文档树的ushl目录里，并从浏览器去请求它。然后使用浏览器的“查看源代码”命令，查看这个测验题页面的HTML源代码。你会发现那个隐藏字段place的内容实际包含了当前测验题的正确答案，任何人都可以看到它。这意味着这个小测验很容易出现作弊情况。单就这个特定应用而言，这不是个什么大问题，但这个例子正好说明隐藏字段一点儿也不安全。对于那些需要向用户严格保密的信息，应该选用其他办法，如选用会话，它会把信息保存在服务器端。

9.2.3 在线编辑美史联盟成员信息

要编写的最后一个PHP脚本是edit_member.php，它可以让美史联盟成员在线修改他们自己的成员资料。有了这个脚本，成员便可以随时改正或者更新他们自己的资料，而不必再向联盟办公室递交更改申请。提供这项功能，有助于保持成员信息处于最新状态，同时减少联盟秘书的工作量。

一定要注意，必须确保成员资料只能由其本人或者联盟秘书来修改。这意味着必须增加一些安全措施。为了演示简单的身份验证机制，我们将使用MySQL 来存储各位成员的密码，并且要求成员只有在提供正确的密码之后，才能获得对脚本所给表单进行修改的权限。此脚本的工作流程如下所示。

当首次调用时，edit_member.php会呈现一个登录表单，其中包含成员ID和密码字段。

当提交登录表单时，脚本会检查与成员 ID和密码相关的密码表。如果密码匹配，脚本将从member表里找出这位成员的资料，并显示它，供成员编辑。

当提交编辑表单时，此脚本会利用这个表单的内容在数据库里更新成员的资料。

在开始上述工作之前，需要先分配好密码。比较简便的办法是随机生成这些密码。下面的语句将创建一个名为member_pass的表，并为每位成员创建一个密码，所用方法是根据一个随机数生成一个MD5校验和，并使用结果的前8个字符。在实际应用中，应该让成员自己设置密码，不过这里使用的这个技术可以便捷初始化所有密码：

```
CREATE TABLE member_pass

(

member_id INT UNSIGNED NOT NULL PRIMARY KEY,

password CHAR(8)

);

INSERT INTO member_pass (member_id, password)

SELECT member_id, LEFT(MD5(RAND()), 8) AS password FROM

member;
```

除了为member表里的每位成员生成密码外，还需要在member_pass表里增加一个特殊的“0 号成员”，并将其密码设置为 bigshot，用于管理员（超级）用户。联盟秘书可以用这个密码来获得访问任何成员资料的权限：

```
INSERT INTO member_pass (member_id, password) VALUES(0,
'bigshot');
```

说明

在创建member_pass表之前，应该先删除Web服务器目录下的db_browse.pl脚本。这个脚本是在8.4.4节编写的，它允许任何人查看sampdb数据库里任何表的内容——其中包括member_pass表。因此，它可用于查看联盟成员密码或管理员密码。

在创建好member_pass表之后，就可以开始编写edit_member.php脚本了。此脚本的框架如下所示：

```
<?php

# edit_member.php - 通过Web编辑美史联盟成员资料

require_once "sampdb_pdo.php";

# 定义动作常量

define ("SHOW_INITIAL_PAGE", 0);

define ("DISPLAY_ENTRY", 1);

define ("UPDATE_ENTRY", 2);

# ...此处放置输入处理函数 ...

$title = "U.S.Historical League -- Member Editing Form";

html_begin ($title, $title);
```

```
$dbh = sampdb_connect ();

# 确定要执行的动作（如果未指定动作，
# 则显示初始页面）

$action = script_param ("action");

if (is_null ($action))

$action = SHOW_INITIAL_PAGE;

switch ($action)

{

case SHOW_INITIAL_PAGE:      # 显示初始页面

display_login_page ();

break;

case DISPLAY_ENTRY:          # 显示用于编辑的成员资料

display_entry ($dbh);

break;

case UPDATE_ENTRY:           # 把更新后的资料存入数据库

update_entry ($dbh);

break;

default:

die ("Unknown action code ($action)\n");
```

```
}
```

```
$dbh = NULL;  # 关闭连接
```

```
html_end ();
```

```
?>
```

`display_login_page()` 函数负责生成初始页面，其中包含一个表单，用于要求成员输入其ID和密码：

```
function display_login_page ()
```

```
{
```

```
    printf("<form method=\"post\" action=\""%s?action=%d\">\n",
```

```
    script_name (),
```

```
    DISPLAY_ENTRY);
```

```
    print("Enter your membership ID number and password,\n");
```

```
    print("then select Submit.\n<br /><br />\n");
```

```
    print("<table>\n");
```

```
    print("<tr>");
```

```
    print("<td>Member ID</td><td>");
```

```
    text_field("member_id", "", 10);
```

```
    print("</td></tr>");
```

```
    print("<tr>");
```

```

print("<td>Password</td><td>");

password_field("password", "", 10);

print("</td></tr>");

print("</table>\n");

submit_button("button", "Submit");

print "</form>\n";

}

```

这个表单会把标题和输入框显示在一个HTML表格里，以便让它们排列整齐。因为这里只涉及两个字段，所以效果似乎不太明显。不过，这是一种非常有用的技巧，尤其是在你创建的表单包含有多个不同长度的标题时，因为它可以让表单变得非常整齐。让表单的各个组件整齐排列，有利于用户阅读和理解。

`display_login_page()` 函数还用到了两个辅助函数，你在 `sampdb.php` 库文件里能找到它们。`text_field()` 函数负责生成一个可编辑的文本输入框：

```

function text_field ($name, $value, $size)

{

    printf("<input type=\"%s\" name=\"%s\" value=\"%s\" "
size=\"%s\" />\n",

"text",

htmlspecialchars ($name),

htmlspecialchars ($value),

htmlspecialchars ($size));

```

```
}
```

`password_field()` 函数代码与之相似，不同之处在于`type`属性的值为`password`。

当用户输入成员 ID 和密码并提交表单时，`action` 参数的值将为 `DISPLAY_ENTRY`，而 `edit_member.php` 中的那条 `switch` 语句将在下一次调用脚本时执行 `display_entry()` 函数，以检查用户输入的密码。如果密码匹配则显示该成员的信息：

```
function display_entry ($dbh)

{

# 获得脚本参数。其中，ID 值需要去除空白；

# 但密码不需要这样做，因为密码必须严格匹配。

$member_id = trim (script_param ("member_id"));

$password = script_param ("password");

if (empty ($member_id))

    die ("No member ID was specified\n");

if (!ctype_digit ($member_id))                                # 必须像整数

    die ("Invalid member ID was specified (must be an
integer)\n");

if (empty ($password))

    die ("No password was specified\n");

if (check_pass ($dbh, $member_id, $password)) # 常规数字

    $admin = FALSE;
```

```

else if (check_pass ($dbh, 0, $password))          # 管理员

$admin = TRUE;

else

die ("Invalid password\n");

$stmt = "SELECT

last_name, first_name, suffix, email, street, city,
state, zip, phone, interests, member_id, expiration
FROM member WHERE member_id = ?

ORDER BY last_name";

$stmt = $dbh->prepare ($stmt);

$stmt->execute (array ($member_id));

if (!$row = $sth->fetch ())

die ("No user with member_id = $member_id was found\n");

printf ("<form method=\"post\" action=\"%s?action=%d\">\n",
script_name (),
UPDATE_ENTRY);

# 把成员ID和密码添加为隐藏值，以便下次调用脚本时

# 可以辨别出表单所对应的是哪一条记录，这样就

# 不需要用户再次输入密码了。

```

```
hidden_field ("member_id", $member_id);

hidden_field ("password", $password);

# 格式化语句的结果，以便于编辑

print("<table>\n");

# 把成员ID 显示为静态文本

display_column ("Member ID", $row, "member_id", FALSE);

# 如果用户提供了管理员密码，则$admin 为真；

# 否则，为假。管理用户可以编辑有效期，

# 而常规用户不能。

display_column ("Expiration", $row, "expiration", $admin);

# 把其他值显示为可编辑文本

display_column ("Last name", $row, "last_name");

display_column ("First name", $row, "first_name");

display_column ("Suffix", $row, "suffix");

display_column ("Email", $row, "email");

display_column ("Street", $row, "street");

display_column ("City", $row, "city");

display_column ("State", $row, "state");

display_column ("Zip", $row, "zip");
```



```

display_column ("Phone", $row, "phone");

display_column ("Interests", $row, "interests");

print ("</table>\n");

submit_button ("button", "Submit");

print "</form>\n";

}

```

`display_entry()` 函数做的第一件事是验证密码。如果用户提供的密码，与给定成员ID所对应的存储在`member_pass` 表里的密码相匹配，或者与管理密码（即那个特殊的“0号ID成员”的密码）相匹配，那么 `edit_member.php` 脚本将把这位成员的个人资料显示在一个表单里，供用户编辑。负责密码检查的`check_pass()` 函数会运行一个简单的查询，从`member_pass`表里选出一个行，并把该行中的`password`值与用户在登录表单里提供的密码进行比较：

```

function check_pass ($dbh, $id, $pass)

{

    $stmt = "SELECT password FROM member_pass WHERE member_id
= ?";

    $sth = $dbh->prepare ($stmt);

    $sth->execute (array ($id));

    # 如果密码找到，并且相匹配，则结果为真

    return (($row = $sth->fetch ()) && $row["password"] == $pass);

}

```

假设密码匹配，`display_entry()` 函数会从 `member` 表里检索出与给定成员 ID 相对应的那个行，然后生成一个编辑表单，并用这个行里的各个值来初始化它。大多数字段都是可编辑文本，用户可以更改它们，只有两个例外。第一个，`member_id` 值会显示为静态文本。因为它是唯一标识行的键值，所以不允许改变。第二个，有效期也不允许联盟成员自行修改。（否则，成员本人就能把这个日期修改为将来很远的某个日期，从而导致他们不交费也能延续自己的成员资格。）另一方面，如果用户在登录表单时给出了管理密码，那么脚本需要把这个有效期显示为可编辑字段。假设联盟秘书知道这个密码，那么他便可以为那些更新成员资格的成员更新有效期。

`display_column()` 函数负责显示各字段的标签和值。其参数有：显示在字段值旁边的标签；包含待编辑行的数组；在行内与字段值相对应的列名；一个用于表明该字段显示为可编辑文本框还是静态文本的布尔值。最后那个值为可选的，默认为 `TRUE`。下面是 `display_column()` 函数的代码：

```
function display_column ($label, $row, $col_name, $editable = TRUE)
{
    print("<tr>\n");

    print("<td>" . htmlspecialchars ($label) . "</td>\n");

    print("<td>");

    if ($editable)    # 显示为可编辑字段

        text_field ("row[$col_name]", $row[$col_name], 80);

    else              # 显示为只读文本

        print (htmlspecialchars ($row[$col_name]));

    print("</td>\n");

    print("</tr>\n");
}
```

```
}
```

对于可编辑值，`display_column()` 函数将利用格式为 `row[col_name]` 的名字来生成文本框。这样，当用户提交表单时，PHP 会把所有字段值放到一个数组变量里，而各元素的键即为列名。这样一来，我们便能很容易地提取到表单的内容，并且当要更新数据库里的行时，可以很容易地把每一个字段值与 `member` 表里的列对应起来。例如，把取回的数组放入 `$row` 变量里，我们就可以通过 `$row["phone"]` 来访问电话号码。

`display_entry()` 函数还用隐藏字段把 `member_id` 和 `password` 值嵌入了表单，这样一来，当用户提交编辑过的资料时，这两个值就能传递到 `edit_member.php` 脚本的下一次调用里。那个 ID 能让脚本确定具体需要更新 `member` 表里的哪一行，那个密码使得脚本能够在用户登录之前对其进行身份验证。（请注意，这种简单的、以明文形式来回传递密码的身份验证方法，一般都不太提倡。不过，美史联盟并不是一个对安全性要求很高的组织，因此这个方案完全能满足我们的目的。如果你要执行某些类似金融交易的操作，那么你应该使用更加安全的认证形式。）下面这个函数负责在表单提交时更新成员资料：

```
function update_entry ($dbh)

{

# 获得脚本参数。其中，ID 值需要去除空白；

# 但密码不需要这样做，因为密码必须严格匹配；

# 并且行也不需要，因为它是一个数组。

$member_id = trim (script_param ("member_id"));

$password = script_param ("password");

$row = script_param ("row");

if (empty ($member_id))
```

```

die ("No member ID was specified\n");

if (!ctype_digit ($member_id))                # 必须像整数

    die ("Invalid member ID was specified (must be an
integer)\n");

if (!check_pass ($dbh, $member_id, $password)

    && !check_pass ($dbh, 0, $password))

    die ("Invalid password\n");

# 检查member表的元数据，以确定各个列

# 是否允许NULL值。（请确保“为空性”

# 是以大写形式进行检索的。）

$stmt = "SELECT COLUMN_NAME, UPPER(IS_NULLABLE)
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ?";

$stmt = $dbh->prepare ($stmt);

$stmt->execute (array ("sampdb", "member"));

$nullable = array ();

while ($info = $sth->fetch ())

    $nullable[$info[0]] = ($info[1] == "YES");

# 迭代表单里的每一个字段，使用这些值来

# 构造一条包含占位符的UPDATE语句，

```

并且构造出与这些占位符绑定的数据值数组。

```
$stmt = "UPDATE member ";
```

```
$delim = "SET";
```

```
$params = array ();
```

```
foreach ($row as $col_name => $val)
```

```
{
```

```
$stmt .= "$delim $col_name=?";
```

```
$delim = ",";
```

如果表单值为空，并且对应的列值可以为空，

则将列值更新为NULL。例如，

当有效期列应该为NULL时，这种做法

可以防止出现把空串放入其中的情况。

```
$val = trim ($val);
```

```
if (empty ($val))
```

```
{
```

```
if ($nullable[$col_name])
```

```
$params[] = NULL; # 输入NULL
```

```
else
```

```
$params[] = "";    # 输入空串
```

```

}

else

$params[] = $val;

}

$stmt .= " WHERE member_id = ?";

$params[] = $member_id;

$stmt = $dbh->prepare ($stmt);

$stmt->execute ($params);

printf("<br /><a href=\"%s\">Edit another member record</a>\n",

script_name ());

}

```

这个脚本首先会再次验证密码，确保不会有人故意发送假表单，然后才会更新成员条目。这个更新操作需要特别小心，因为如果表单的某个字段为空白，那么可能需要输入NULL，而不是输入空串。expiration 列便属于此种情况。假设，联盟秘书以管理密码登录（这样可以编辑有效期字段），清空了这个字段，表明该成员具有“终身成员资格”。在数据库里，这种情况所对应的成员资格有效期应该为NULL，而不能为空串（它不是个有效的日期值）。因此，当这种列在表单里为空白时，需要把那些允许为NULL值的列识别出来，并插入NULL（而非空串）。

为解决此问题，update_entry() 函数会检索member表的元数据，并构造一个关联数组——其键为各个列名，以此表明各列是否允许为NULL。这些信息可从 INFORMATION_SCHEMA 数据库的COLUMNS表里获得。我们需要从这个表里获得的值包括：列名和它是否允许为NULL值（即COLUMN_NAME和IS_NULLABLE的那些值）。

至此，`edit_member.php`脚本算是大功告成。你可以将它安装到Web文档树的`ushl`目录，并把密码告知成员，他们就可以通过Web在线修改自己的个人资料了。

第10章

MySQL管理简介

随着MySQL能力的增强，其复杂性也在不断增加。不过在各种数据库系统中，MySQL还算是相对容易使用的，其安装和使用都不难。MySQL的这种简单性使得它非常受欢迎，特别是在那些不是系统管理员、也不想成为系统管理员的人群中更是如此。如果你是一位训练有素的计算机专业人员，对于成功运行MySQL自然会有很大帮助，不过那不是必要条件。

尽管如此，不管你的技术水平如何，MySQL都不会自己运行起来。必须要有人盯着它，才能保证它平稳有效地运行，而且必须要有人知道当问题发生时该怎么办。如果你的工作是保证MySQL在网站上正常运行，那么请继续往下读。

本书的第三部分将讨论MySQL管理员的职责。本章简要说明了MySQL安装管理所涉及的职责。随后的几章提供的是更为详细的实施指导。

如果你是一名新的或者毫无经验的MySQL 管理员，那么请不要被本章列出的一长串职责所吓倒。下面几节列出的每一项任务都很重要，但是你不需要立刻把它们全部学会。只要你愿意，完全可以把本书这部分的各章当作参考，当你真的需要了解它们的时候，再回过头来查询相关内容。

如果你有管理其他数据库系统的经验，那么你会发现，MySQL的安装与它们在很多方面都是类似的，并且你的经验也完全能够派上用场。但是，MySQL的管理工作也有其自己的特定要求。本书的这部分将帮助你熟悉这些内容。

10.1 MySQL组件

MySQL数据库系统由多个组件构成。你应该对这些组件及其用途有所了解，以便你能真正了解自己所管理的这个系统的本质，以及那些能帮助你完成任务的工具。如果多花些时间了解一下自己管理的具体内容，那么你的工作将变得更加简单。为此，你应该对MySQL的下列几个方面有所了解。

(1) MySQL服务器。服务器主程序`mysqld`，是MySQL数据库系统的核心。它负责管理所有数据库和表。在Unix系统里，有几个相关的脚本用于辅助服务器的启动。`mysqld_safe` 程序用于启动和监控服务器，在服务器意外停机时会重新启动它。对于Unix版本的MySQL，当使用运行级目录来启动各项系统服务时，可以使用`mysql.server`脚本来启动。如果想在同一台主机上运行多个服务器实例，那么`mysqld_multi`可以帮助你轻松地管理它们。在Windows系统上，可以选择从命令行运行服务器，也可以选择以Windows服务的方式运行它。

(2) MySQL客户端程序和实用工具。在与服务器进行通信时，可以使用的MySQL程序很多。就系统管理任务而言，下面列出的几个程序是最重要的。

`mysql`。它是一个交互式程序，可用来向服务器发送SQL语句和查看结果。还可用来执行批处理脚本（即包含SQL语句的文本文件）。

`mysqladmin`。这个系统管理程序可用于完成许多任务，如关闭服务器、检查它的配置或者在运行不正常时监视其状态。

`mysqldump`。它是一个用于备份数据库或者把数据库复制到另一个服务器的工具。

`mysqlcheck`和`myisamchk`。这两个程序能帮你完成表的检查、分析和优化，还能帮你修复受损表。`mysqlcheck`主要适用于MyISAM表，有时也能用于其他存储引擎创建的表。`myisamchk`只适用于MyISAM表。

(3) 服务器语言——SQL。你应该可以用服务器自己的语言与它进行对话。举个简单的例子，你可能需要找出用户权限为何没按你所希望的方式工作的具体原因。如果没有能够进入并与服务器直接进行通信的替代程序，那么你可以使用客户端程序mysql来执行SQL语句，检查授权表。

如果你对 SQL还不太了解，那么一定要对其有个基本的了解。对SQL不熟悉只会对你的管理任务造成影响，如果能多花点时间学习一下它，那么会有事半功倍的效果。想要真正掌握SQL需要花费较长的时间，但是基本技能很快便可掌握。有关SQL的指导信息，以及命令行客户端程序mysql的使用，请参考第1章。

(4) MySQL数据目录。服务器会把数据库和状态文件存储在数据目录里。了解数据目录的结构和内容很重要，你能知道服务器是如何使用文件系统来表示数据库和表的，还能知道服务器日志位于何处，以及它们所包含的内容。出于对储存分配和性能的考虑，你还应该了解一下与磁盘分配有关的选项。

10.2 MySQL的常规管理

MySQL的常规管理主要包括：操作MySQL服务器程序mysqld，管理访问服务器的用户。在履行此项职责时，最为重要的工作有以下几个。

(1) 服务器启动和关闭。当系统需要启动和停止时，你应该知道如何通过命令行手动启动和停止服务器，以及如何自动启动和关闭。如果服务器崩溃或者不能正常启动，那么要如何做才能使其重新工作——了解这一点也很重要。

(2) 用户账户维护。你应该了解MySQL账户和Unix或Windows登录账户之间的差别。应该知道如何设置 MySQL 账户，其中包括：指定哪些用户可以连接到服务器，他们可以从哪里连接到服务器，以及他们能做些什么。你还需要知道如何重置已忘记的密码。

(3) 日志维护。你应该了解：哪些类型的日志可用，哪些日志对你有帮助，以及执行日志文件维护的时机和方法。日志轮转和过期对于防止日志填满文件系统来说至关重要。

(4) 服务器配置和优化。MySQL服务器可配置程度很高。可控制的操作特性包括服务器支持的存储引擎、默认字符集和默认时区。

另一个配置问题是服务器优化。用户都希望服务器能以最佳状态运行。提高服务器运行速度最简便的方法是，购置更多的内存或者换一块运行速度更快的磁盘。但是这些增强措施并不能帮你了解服务器的工作。你应当知道有哪些参数可用于调整服务器的操作，以及在实际应用中如何使用它们。在某些站点，查询的工作主要是检索；而在其他站点，主要工作可能是插入和修改。要修改哪些参数，取决于你如何看待自己站点的查询工作情况。

(5) 管理多个服务器。在某些情况下，可能需要在同一台机器上运行多个服务器实例。你可能需要测试某个新发行的MySQL 版本，而同时希望保持现有的生产服务器不动；或者希望通过让每组都拥有其自己的服务器的方式来更好地保护不同用户组的隐私。（后者与因特网服务提供商密切相关。）对于这些情况，你应该知道如何设置多个实例服务器。

(6) 更新 MySQL 软件。MySQL的版本在不断更新。你应该知道，如何通过更新到最新版本来获得服务器的错误修复和新功能。你要知道在什么情况下应推迟更新，并且也要知道如何在发行的稳定版本和开发版本之间进行选择。

10.3 访问控制与安全性

当你负责 MySQL 的安装时，需要确保用户托付给数据库的信息绝对安全——这一点非常重要。MySQL 管理员有责任控制好对数据目录及服务器的访问，并且应该对以下问题有所了解。

(1) 文件系统安全性。在一台计算机里可能会有多个与MySQL的管理任务无关的用户账户。重要的是需要保证这些账户不能访问数据目录。这样可以防止他们在文件系统级对数据造成破坏，如复制或删除数据库的表、读取可能包含有敏感信息的日志。你应该了解：如何设置用于运行 MySQL 服务器的用户账户，如何设置数据目录使得它隶属于该用户，以及如何使用该用户的权限来启动运行服务器。

(2) MySQL 服务器安全性。你必须了解 MySQL 安全系统的工作原理，以便在创建用户账户时，能为其授予合适的MySQL 服务器访问权限。对于那些通过网络连接至服务器的用户，他们应该只做期望他们做的事情。你肯定不想出现因错误理解安全系统而不小心把过高的访问权限授予用户的问题！

10.4 数据库维护、备份和复制

每一个MySQL 数据库管理员都希望能避免处理损坏或者遭破坏的表。但是，希望并不能阻止问题的发生。你应该采取措施降低这种风险，并且应该学会在意外发生时要如何应对。

(1) 预防性维护。你应该有一套定期的预防性维护计划，用于降低数据库出现故障或遭到破坏的可能。当然，备份数据库是必不可少的，但是预防性维护可以减少你求助于备份的机率。

(2) 数据库备份。当发生严重的服务器系统崩溃事件时，数据库备份能起到关键的作用。在数据库崩溃之后，你肯定会希望把数据库恢复到崩溃前的状态，同时尽可能减少数据损失。请注意，数据库备份与常规的系统备份（如在Unix系统上使用dump进行备份）并不相同。在进行系统备份时，表所对应的文件很可能因服务器的活动而处于变化状态，因此恢复那些文件并不能保证表的一致性。对于数据库恢复而言，mysqldump程序生成的备份文件会更有用。它可以让你在无需关闭服务器的情况下创建备份。此外，在硬盘空间用尽的时候，你可能会想要把数据库迁移到另一个地方。

(3) 崩溃恢复。如果尽了最大努力但仍然出现了意外，那么你应该知道如何修复或恢复表。用到崩溃恢复的情况应该会很少，可是一旦需要，那么这将是一件非常痛苦且高度紧张的事情（尤其是在你正忙得不可开交、电话响、门铃也响的时候）。尽管如此，你必须要知道如何处理，否则，你的用户会极度不满！你必须熟练掌握用于检查和修复 MySQL 表的那些程序。一定要知道如何利用备份文件来恢复数据，以及如何利用二进制日志恢复最近一次备份后所发生的那些更改。

(4) 数据库迁移。如果你打算把现有的MySQL迁移到一台速度更快的主机上，那么需要把数据库复制到另一台机器上。你应该熟悉整

个操作的流程，以及产生的要求。数据库文件的具体内容可能依赖于具体的机器。如果真是如此，则不能简单地把它们从一个系统复制到另一个系统。

（5）数据库复制。对数据库进行备份或制作副本，实际上相当于在某个特定的时间点对其状态拍一个快照。另一种可选的办法是使用复制（replication），即建立两个相互合作的服务器，让一个服务器所管理的数据库产生的更改，持续传播到另一个服务器所管理的与之相应的那个数据库。

要使用复制，应该知道：如何把服务器设置为主复制服务器，如何设置用来同步复制主服务器的从服务器。如果出现了问题，导致复制工作停止，那么你必须要知道到什么地方去查找问题，并让复制工作重新开始。

以上几节简要描述了一名 MySQL 管理员应该承担的职责。接下来的几章将对它们进行详细讨论，并且会描述应该遵循的操作流程，以便于大家能有效地履行这些职责。我们首先会讨论 MySQL 的数据目录——它是你要维护的主要资源，你应该熟悉它的结构和内容。然后，我们将依次讨论常规管理任务、MySQL 的安全系统以及数据库的维护和备份。

第11章

MySQL数据目录

在概念上，不同的关系数据库系统在很大程度上都很相似：它们都要负责管理一组数据库，并且每个数据库都包括一组表。但是每个系统都有其自己的数据组织管理的方式，MySQL自然也不例外。默认情况下，MySQL服务器 `mysqld` 会把它管理的所有信息存储在一个被称为 MySQL 数据目录的地方。数据目录存储着所有的数据库，以及提供服务器运作信息的状态文件和日志。如果你负责管理 MySQL 的安装设置，那么熟悉数据目录的结构和使用是好好履行职责的“必修课”。即使你没有负责MySQL的管理工作，你也可以从本章获得好处，因为更好地理解服务器的运作机理绝无坏处。

本章的主要内容如下。

数据目录位置。因为数据目录是MySQL服务器运作的中心，所以你应该知道如何确定它的位置，这样才能高效地对其内容进行管理。

服务器如何组织和提供对其管理的那些数据库及表的访问。这一点对于设置预防性维护计划以及在表出现损坏时进行崩溃恢复都非常重要。

服务器会生成哪些状态文件和日志，以及它们包括的内容。它们的内容提供了与服务器运行状态有关的有用信息，在你遇到问题时，这些内容会很有用。

如何改变数据库目录的默认位置或组织结构。这一点对于系统里的磁盘资源分配管理尤为重要。例如，可以跨驱动器平衡磁盘活动，可以把数据重新放置到具有更多空余空间的文件系统上。你也可以使用这些知识来规划新数据库的存放位置。

对于Unix系统，本章会假定用于执行MySQL管理任务和运行服务器的登录账户已存在。在本书中，该账户的用户和组名都是mysql。12.2.1.1节会讨论为MySQL管理指定登录账户的原因。

11.1 数据目录位置

默认的数据目录位置被编译在服务器中。在Unix系统里，如果通过二进制或源代码发行版安装MySQL，那么数据目录通常默认为/usr/local/mysql/data；如果通过RPM包来安装，则默认目录为/var/lib/mysql。在Windows系统里，默认数据目录通常为C:\ProgramData\MySQL或者C:\Documents and Settings\All Users\Application Data\MySQL，到底是哪一个取决于Windows的具体版本。

如果通过源代码编译MySQL，则可以在运行CMake时使用命令行选项-DMYSQL_DATADIR=dir_name，以此来指定默认的数据目录。

如果想在启动服务器时指定数据目录的位置，则可以使用--datadir=dir_name 选项。如果想要指定一个与编译时内置的那个默认位置不同的位置，那么这种做法比较有用。另一种指定数据目录的方法是将其列在服务器启动时读取的选项文件里。那样，你就不用每次启动服务器时都在命令行上指定它了。

作为一名 MySQL 管理员，你应该知道服务器的数据目录在什么地方。要是你不知道的话（也许你刚从前任那里接手工作，而他没有留下什么有用的资料），这里有几种办法可以找到它。下面会先介绍一种在服务器没有运行时使用的办法，然后会介绍另一种在它运行时使用的办法。

查看服务器启动时读取的选项文件。例如，在Unix系统上查看/etc/my.cnf文件时，你可能会在这个文件的[mysqld]选项组里看到一个datadir行：

```
[mysqld]
```

```
datadir=/path/to/data/directory
```

这个路径名指示的便是数据目录的位置。

如果你还不清楚服务器在什么地方寻找这些选项文件，那么可以调用下面这条命令，并查看帮助消息，在其中的开头部分便列出了选项文件的各个存放位置：

```
% mysqld --verbose --help
```

如果服务器正在运行，那么可以连接到它，向它询问数据目录的位置。服务器维护着许多与其运作有关的系统变量，并且可以报告其中的任何一个值。数据目录的位置保存在datadir变量里，你可以使用SHOW VARIABLES语句或者mysqladmin variables命令来获得它。如果在Unix系统里使用SHOW VARIABLES语句来查看变量值，则会得到类似下面这样的结果：

```
mysql> SHOW VARIABLES LIKE 'datadir';
```

Variable_name	Value
datadir	/usr/local/mysql/data/

从命令行使用mysqladmin得到的结果如下所示：

```
% mysqladmin variables
```

Variable_name	Value
...	
datadir	/usr/local/mysql/data/

...

在Windows上，这个位置则可能类似于C:\ProgramData\MySQL。

如果同时运行着多个服务器实例，则它们将在不同的网络接口上监听（如TCP/IP端口、Unix套接字文件、Windows命名管道或共享内存）。可以使用相应的连接参数选项依次连接每一个服务器来获得数据目录信息。

如果数据目录在某个位置创建好了，而你想把它转移到其他地方，那么可以参考11.3节。

11.2 数据目录结构

MySQL的数据目录存放着服务器管理的所有数据库。一般情况下，它是一个树形结构，是直接利用Unix或Windows的文件系统层次结构实现的。

每个数据库在数据目录下都对应有一个数据库目录。

数据库里的表、视图和触发器都对应于数据库目录中的文件。

某种给定的存储引擎完全可以使用另一种不同的存储结构——它可以不同于常见的基于目录和文件实现的数据库层次结构。例如，InnoDB存储引擎可以把所有数据库的InnoDB表存储在一个公共表空间里。该表空间可以包含一个或多个大文件，这些文件会被当成一个单独统一的数据结构，其中还表示了表和索引。默认情况下，InnoDB会把表空间文件存储在数据目录里。

数据目录也可以包含其他文件。

服务器进程ID（PID）文件。当服务器启动时，它会将自己的进程ID写入文件，以便其他程序在需要向服务器发送信号时可以找到这个值。

服务器生成的状态文件和日志文件。这些文件提供了与服务器运作有关的重要信息，而这些信息对管理员来讲很有价值，特别是在出现了问题，而你想要弄清问题原因的时候。

例如，当某条特殊的语句使得服务器崩溃时，你便可以通过检查日志文件来找到这条“捣乱”的查询。（如果服务器的配置是把日志信息记录到数据库表，而不是记录到文件，那么这些日志表将位于mysql数据库里。）

服务器相关的文件，如DES密钥文件、服务器的SSL证书和密钥文件。大部分的管理员都会把数据目录作为这些文件的存放位置。

11.2.1 MySQL服务器提供的数据库访问方式

当以常见的“客户端/服务器”结构使用MySQL时，数据目录下的所有数据库都会由一个实体（即MySQL服务器mysqld）来管理。客户端程序不会直接操作数据。实际上，服务器是访问数据库的唯一联络点，它就像是处于客户端程序和它们想要使用的数据之间的中间人。图11-1展示了这个结构。

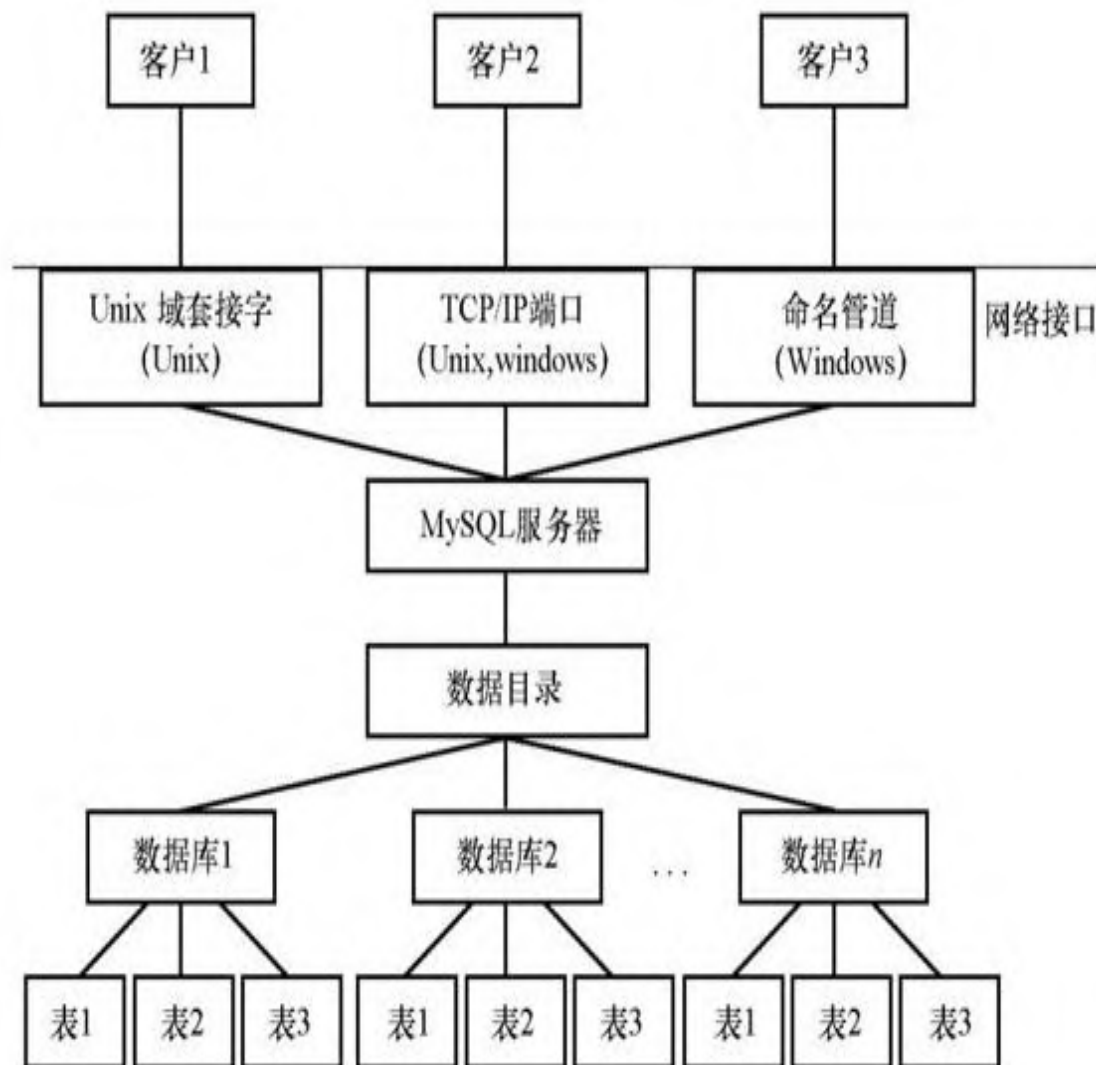


图11-1 MySQL服务器控制数据目录访问的方式

当服务器启动时，它会打开你请求它维护的日志文件，然后通过监听各种类型的网络连接，对外提供一个通向数据目录的网络接口。（更多关于网络接口选择的详细信息请参考12.2.4节。）为访问数据，客户端程序需要建立一个与服务器的连接，然后用SQL语句传递请求，执行所想要的操作，如创建表、查询行或更新行。服务器会执行每一个操作，并把结果返回给该客户端。服务器支持多线程，能够同时为多个客户端提供连接服务。不过，因为每次只能执行一个更新操作，因此服务器实际上会把各个请求排列起来，以便两个客户端决不会在同一时间改变同一个给定的行。

在一般情况下，把服务器当作数据库访问的唯一仲裁者，可以为因多个进程同时访问表而可能造成的各种数据损坏情况提供保障。尽管如此，管理员还是应该有所注意，在许多时候服务器并不能完全对数据目录做到互斥控制。对于MyISAM表的维护、故障诊断、修复和压缩操作，可以使用像

myisamchk那样的直接访问维护工具。这些程序可以直接操作与这些表相对应的文件。因为这些工具程序会更改表的内容，因此在服务器修改表时使用它们来操作表，可能会导致表损坏。

避免出现这类问题的最好办法是，在运行这些表处理工具程序之前，先停止服务器。如果无法做到这一点，则必须知道如何告诉服务器：当你在使用这类工具直接操作某个表的文件时，千万不要去访问该表。与“在使用这些程序时如何与服务器进行协作”相关的指导信息请参考14.2节。另一种不使用myisamchk程序的办法是，使用像CHECK TABLE和REPAIR TABLE这样的语句（或者使用mysqlcheck程序，它可以替你执行这些语句）。这些语句可以通过指导服务器本身来完成表维护操作的方式，避开与服务器进行交互所带来的问题。

11.2.2 数据库在文件系统里的表示

MySQL服务器管理的每个数据库都有其自己的数据库目录。数据库目录是以数据目录的子目录形式存在的，而且每一个目录的名称与它所表示的数据库名相同。例如，如果在你的主机上，DATADIR表示服务器的数据目录位置，那么名为mydb的数据库在Unix或Windows上所对应的数据库目录都为DATADIR/mydb。

SHOW DATABASES语句可以列出位于数据目录里的目录名称。

CREATE DATABASE db_name语句会在数据目录下创建一个名为db_name的数据库目录。它还会在这个数据库目录里创建一个名为db.opt的文件，其中列出了数据库的各种属性，如默认字符集和排序规则。在Unix系统里，数据库目录只隶属于运行服务器的那个登录账户，并且只能被它访问。

DROP DATABASE 语句实现得很简单。DROP DATABASE db_name 语句会删除数据目录里的db_name目录，以及该目录里的所有表文件和其他数据库对象（如视图或触发器）文件。这与使用文件系统级的命令（如Unix系统里的rm命令或Windows系统里的del命令）手动删除数据库目录的效果几乎一样。DROP DATABASE语句与文件系统命令的区别在于以下几点。

DROP DATABASE语句只会根据表和其他数据库对象所使用的文件扩展名来删除文件。如果你在数据库目录里创建了其他文件或子目录，那么服务器不会删除它们。在这种情况下，数据库目录不会被删除，并且DROP DATABASE语句会返回一个错误。这种情况的一种后果是，该数据库的名字仍会显示在SHOW DATABASE语句的输出结果里。为解决此问题，可以先手动删除多余的文件和子目录，然后再次执行DROP DATABASE语句。

删除数据库目录的办法无法安全地删除数据库里的InnoDB表。对于每一个InnoDB表，InnoDB存储引擎会在其系统表空间里维护一个数据字典条目，并且也顺便会将该表的内容存储在那里。如果数据库包含InnoDB表，则必须使用DROP DATABASE语句来删除，这样才能确保InnoDB存储引擎会更新其数据字典，并从表空间里删除该表的内容。

11.2.3 表在文件系统里的表示

MySQL支持多种存储引擎，如InnoDB、MyISAM和MEMORY。对于每一个表，MySQL在磁盘上至少会使用一个文件来表示，即.frm格式文件，其中包含表结构的描述。服务器会负责创建.frm文件，而各个存储引擎会创建一些附加文件，用于保存数据行和索引信息。这些文件的名称和结构会因存储引擎的不同而有所差异。

接下来会对几种具有代表性的存储引擎如何在磁盘上存储文件的特点进行讨论。与“这些存储引擎在功能和行为方面有何差异”有关的更多信息请参考2.6.1节。

InnoDB是默认存储引擎。在数据库目录里，每个InnoDB表都有一个.frm文件，其中包含了表结构的定义。对于表的内容，InnoDB提供了两种表示方式，它们都是基于表空间实现的。

系统表空间。这种表空间由数据目录里的一个或多个大文件构成。表空间的这些组成文件共同形成了一个在逻辑上连续不断的存储区域，其大小为各个组成文件大小之和。默认情况下，InnoDB会将其表存储到这个系统表空间里。对于这类表，唯一与表特定相关的文件是.frm文件。

独立表空间。通过配置，InnoDB可以实现每个表一个表空间的存储模式。在这种情况下，每个InnoDB表在数据库目录里都会有两个与表特定相关的文件，即.frm文件和包含表数据和索引的.ibd文件。

系统表空间还有另一个用途。InnoDB在其内部维护着一个数据字典，其中包含了与每个表有关的信息。这个字典会保存在系统表空间里，因此，即使你使用独立表空间来存储表的内容，系统表空间也是必需的。

MySQL会在包含该表的那个数据库所在的目录里，为每个MyISAM表创建3个文件。其中每个文件的基本名都等同于表名，其扩展名则表明了该文件的用途。例如，一个名为mytbl的MyISAM表将有以下3个文件。

mytbl.frm。格式文件，其中包含对该表结构的描述。

mytbl.MYD。数据文件，其中存储了该表各行的内容。

mytbl.MYI。索引文件，其中包含该表的所有索引信息。

MEMORY 表是一种内存表。每个MEMORY 表在数据库目录里都只有一个描述其格式的.frm文件。除此以外，文件系统里就没有其他与之对应的东西了，因为服务器会把MEMORY表的数据和索引全部存储在内存里，而不是存储在磁盘里。当服务器关闭时，MEMORY 表的内容也会随之丢失。在服务器重启之后，这个表仍然存在（因为它的.frm 文件存在），只是其内容为空。

11.2.4 视图和触发器在文件系统里的表示

在包含视图和触发器对象的数据库目录里，有与对象相关联的文件。

每个视图只有一个.frm文件，其中包含着该视图的定义和其他相关属性。该文件的基本名和视图的名字一样，因此一个名为myview的视图是由一个名为myview.frm的文件表示的。

每个触发器存储在一个.TRG文件里，其中包含着触发器的定义和其他相关属性。触发器文件的基本名与触发器所属的那个表的名字一样。例如，有一个名为 mytrig的触发器，它与一个名为mytbl的表关联在一起，那么存储它的文件将是mytbl.TRG，而不是mytrig.TRG。如果一个表有多个触发器，那么服务器会把它们的所有定义集中存储在同一个.TRG文件里。每个触发器还有一个根据触发器来命名的.TRN 文件，其中包含与这个触发器相关联的那个表的名字。例如，触发器mytrig有一个mytrig.TRN文件，其中包含了表名mytbl。

11.2.5 SQL语句与表文件操作的对应关系

每一种存储引擎都使用了一个.frm 文件来存储表格式（定义），因此SHOW TABLE FROM db_name语句的输出结果与db_name的数据库目录里的那个.frm文件的基本名列表相同。

在创建MySQL所支持的任意类型的表时，需要执行CREATE TABLE语句来定义表的结构，并且其中还要包括ENGINE = engine_name子句，用于表明想要使用哪种存储引擎。如果省略ENGINE子句，那么MySQL将使用默认存储引擎（如果你未更改，则为InnoDB存储引擎）。服务器会为新表创建一个.frm文件，在其中存储该表定义的内部编码，并且会告知相应的存储引擎去创建与该表相关联的其他文件。例如，InnoDB 会创建一个数据字典条目，并在相应的InnoDB 表空间里对其数据和索引信息进行初始化。MyISAM 会创建一个.MYD 数据文件和一个.MYI索引文件；而CSV存储引擎会创建一个.CSV数据文件。在Unix系统里，为表示这个表而创建的所有文件的所有者和访问模式，都被设置成了只允许用来运行 MySQL 服务器的那个登录账户进行访问。

当执行ALTER TABLE语句时，服务器会对表的.frm文件重新编码，以反映出这条语句对其结构的更改；同时，服务器还会修改表的内容（即数据和索引）。在执行语句CREATE INDEX和DROP INDEX时，也会发生这种情况，因为服务器会把它们当作ALTER TABLE的等效语句来处理。如果ALTER TABLE语句更改了表的存储引擎，那么表的内容也会被转换到新的存储引擎，新引擎会使用用于表示表的那些文件的恰当类型重新写入那些内容。

MySQL实现DROP TABLE语句的方式是，删除那些表示表的文件。如果是删除InnoDB表，那么InnoDB存储引擎还会更新它的数据字典，并在InnoDB系统表空间里把与该表相关联的空间标记为空闲。

对于某些存储引擎，如MyISAM，可以通过在表相对应的那个数据库目录里删除文件的方式手动删除该表。对于其他的存储引擎，如InnoDB或MEMORY，由于表的部分内容可能不是以文件系统里的文件来表示的，因此没有与DROP TABLE语句等效的文件系统命令。例如，存储在系统表空间里的InnoDB表，总是有一个.frm文件与之对应，但是删除这个文件并不能彻底删除该表。InnoDB数据字典只能由InnoDB自己更新，并且如果只删除.frm文件，则会导致表的数据和索引“搁置”在系统表空间里。

如果InnoDB表有其自己的独立表空间，那么它会在数据库目录里表示成.frm文件和其自己的.ibd文件。但是，通过删除这些文件的方式来删除表，仍然是不正确的，因为InnoDB存储引擎还是没有机会更新数据字典。因此，必须使用DROP TABLE语句，这样InnoDB存储引擎才能在删除文件的同时更新其数据字典。

11.2.6 操作系统对数据库对象名字的约束

在命名数据库和其他对象（如表）的标识符时，MySQL有一套自己的标识符命名规则。2.2节中详细列出了这些规则，这里再简要概括一下。

不加引号的标识符可以由大小写形式的字母a-z、数字0-9、美元符号“\$”、下划线“_”，以及范围在U+0080到U+FFFF之间的Unicode扩展字符构成。

用反引号引起来的标识符可以包含其他字符（如`odd?name!`）。如果使用SQL保留字作为标识符，则也需要使用反引号将其引起来。如果启用了SQL的ANSI_QUOTES模式，那么这些标识符既可以用反引号，也可以用双引号。

标识符的最大长度为64个字符。

此外，MySQL服务器所在主机的操作系统也可能对标识符施加其他限制。这些限制源于文件系统的命名规则，因为数据库名和表名都对应着目录名和

文件名：在文件系统里，每个数据库都是以数据库目录的形式进行表示的，并且不管使用何种存储引擎，每个表在文件系统里都至少会与一个.frm文件相对应。因此，在命名MySQL标识符的时候还要考虑以下限制。

MySQL允许的数据库名和表名的最大长度为64个字符，但这些名字的长度都不得超过操作系统所允许的最大长度。

底层文件系统是否区分大小写，会对数据库和表的命名与引用产生影响。如果文件系统区分大小写（对于Unix系统通常是这样），那么文件名abc和ABC所指的就是两个不同的文件。如果文件系统不区分大小写（如Windows系统或者Mac OS X系统的扩展文件系统），那么文件名abc和ABC所指的就是同一个文件。如果在某个文件名区分大小写的服务器上开发了一个数据库，并且可能需要把这个数据库移动或复制到另一个文件名不区分大小写的服务器上，则需要对此问题引起注意。

MySQL服务器会对标识符里可能导致文件名出现问题的特殊字符进行编码。这个编码允许在SQL语句里出现的那些名字当中，使用像“/”和“\”这样的字符：在数字和拉丁字母以外的所有字符在文件名里都将被映射成，“@”后面紧跟一个编码后的字符值。例如，字符“?”和“!”的编码值为003f和0021，那么 odd?name! 表所对应的.frm 文件的名称即为 odd@003fname@0021.frm。与表相关联的其他文件的命名方式与此类似。

之前提到过，文件系统区分大小写会对数据库和表的命名产生影响。解决此问题的一种办法是，固定使用一种大小写形式的名字。另一种办法是，在启动服务器时，将lower_case_table_names系统变量设置为1，这个设置有以下两个效果。

在为某个表创建相应的磁盘文件之前，服务器会先把该表的名字转换为小写。

在语句里引用这个表时，服务器会在试图到磁盘上查找这个表之前，把它的名字转换为小写。

这两种做法的结果是，不管文件系统是否区分大小写，所有名字都不会区分大小写。如此一来，在系统间移动数据库和表会变得更加容易。不过，这里提醒一下，如果打算使用这种策略，则必须在开始创建数据库或表之前（而不是之后）通过服务器配置，将lower_case_table_names系统变量设置为1。如果在设置此变量之前，已经创建了数据库或表，而且其名字里包含有大写字母，那么这个设置将无法产生预期的效果，因为磁盘里已经存储了某些不全为小写字母的名字。为避免出现这个问题，可以先把名字里有大写字母的表全部重新命名为小写字母形式，然后再来设置lower_case_table_names系统变量。（可以用ALTER TABLE或RENAME TABLE语

句来重新命名表。)如果你有很多表需要重新命名,或者有许多数据库的名字包含大写字母,那么更简单的办法是先转储这些数据库,然后在设置了lower_case_table_names系统变量之后,再重新创建它们。

(1) 用mysqldump工具将各个数据库转储出来:

```
% mysqldump --database db_name > db_name.sql
```

(2) 用DROP DATABASE语句删除这些数据库。

(3) 关闭服务器,重新配置它,将lower_case_table_names设置为1,然后重启服务器。

(4) 用mysql程序重新加载所有的转储文件:

```
% mysql < db_name.sql
```

由于设置了lower_case_table_names,所有的数据库和表在重新创建之后在磁盘里的名字将全部为小写。

lower_case_table_names实际上还有另外几个可取值,详细信息请参阅附录D。

11.2.7 影响表最大长度的因素

在MySQL里,表的长度是有限的,不过表的长度受很多方面的限制,因此很难精准地确定具体的界限。

操作系统对文件的最大长度有限制。那些用于表示表的文件,如MyISAM表的.MYD和.MYI文件,也会受这种限制的影响。这种限制也适用于构成InnoDB表空间的那些文件。不过,InnoDB系统表空间的总长度很容易超过文件的最大长度。为解决此问题,可以把表空间配置为由多个文件组成,其中每个文件都可以达到最大长度。另一种突破文件大小限制的方法是,在InnoDB表空间里使用磁盘里的原始分区。磁盘原始分区上的表空间组件可以与磁盘分区本身一样大。与配置指导有关的更多信息请参考12.5.3.1节。

除了操作系统的限制以外,MySQL对于表的长度还有其自己的内部限制。这些限制因存储引擎的不同而有所区别。例如:

对于InnoDB存储引擎,InnoDB系统表空间的最大长度为40亿页,其中每页的大小为16 KB。

表空间的最大长度也是存储在表空间里的单个InnoDB表的长度上限。如果把存储引擎InnoDB配置为使用独立表空间，则每个InnoDB表的内容将存储在其自己的.ibd文件里。

此时，InnoDB表的长度将受限于操作系统的文件大小上限。

对于MyISAM表，.MYD和.MYI文件的最大长度默认为256 TB。但是，在创建表的时候，如果使用了AVG_ROW_LENGTH和MAX_ROWS 选项，这些文件的大小上限将可达到65 536 TB。（与 CREATE TABLE 语句有关的描述信息请参考附录 E。）这些选项会对MySQL 内部的行指针宽度产生影响，而这个宽度则决定着表所能容纳的最大行数。当MyISAM表增长到其最大长度，并且出现了与表操作有关的135或136号错误时，可以使用ALTER TABLE语句来增大这些选项值。如果想要直接更改默认的MyISAM指针宽度，可以设置myisam_data_pointer_size系统变量，新设置会对此后创建的表生效。

对于使用不同文件表示数据和索引的存储引擎，只要其中有一个文件达到文件大小上限，那么这个表的长度就会达到上限。对于MyISAM表，其索引情况也会对哪种类型的文件先到达上限造成影响。对于没有索引或者索引比较少的表，很可能是数据文件先到达上限。对于有大量索引的表，则很可能是索引文件先到达上限。

AUTO_INCREMENT 列会隐式限制表所能拥有的行数。例如，若列的类型为TINYINT UNSIGNED，其所能容纳的最大值为 255，那么表可能具有的最大行数也将是 255。更大的整数类型允许有更多的行。更一般的情况是，如果表包含PRIMARY KEY或UNIQUE索引，那么它的行总数将不超过索引中唯一值的数目。

为了确定实际可能达到的最大表长度，必须考虑到所有可能的因素。有效的最大表长度可能由那些因素中的最小值来确定。假设，你想创建一个MyISAM表。当使用默认数据指针大小时，MySQL允许数据和索引文件各自达到256 TB。但是，如果操作系统限制文件大小为2 GB，那么表文件的有效上值也将为2 GB。另外，如果系统支持大于256 TB的文件，那么决定表长度的因素将是MySQL，即那个内部数据指针大小。这个因素是你控制的。

对于存储在系统表空间里的InnoDB表，单个的InnoDB表可以增大到与表空间一样大，而表空间可以跨多个文件，因而可以变得非常大。但是，很有可能你会拥有许多 InnoDB 表，它们都共用同一个空间，这样一来，每个表不仅要受限于表空间的长度，还要受限于为其他表分配了多少表空间。只要表空间未被填满，任何 InnoDB 表都可以不断增大。相反，当表空间被填满时，InnoDB表则不能再增大，除非往这个表空间里增加另外的组件，使之变得更大。另一种做法是，使最后那个表空间组件可以自动扩展，这样，只要

这个表空间组件没有超过系统文件的大小上限，并且还有可用的磁盘空间，那么它便能继续增大。与配置指导有关的更多信息请参考12.5.3.1节。

11.2.8 数据目录结构对系统性能的影响

MySQL数据目录的结构比较容易理解，因为它以一种很自然的方式使用了文件系统的分层结构。同时，这种结构也隐含着会有一些性能方面的问题，特别是对于打开文件（用于表示数据库表的）的操作更是如此。

对于那些使用其自己的文件来单独表示各个表的存储引擎，每次打开表都会需要一个文件描述符。如果某个表由多个文件表示，那么打开这个表将需要多个文件描述符，而不是只需要一个。MySQL服务器会智能地缓存这些文件描述符，但是对于一个繁忙的服务器，在同时为大量的客户端连接提供服务，或者执行引用了多个表的复杂语句时，会很容易耗尽这些描述符。这很可能成为一个问题，因为文件描述符在许多系统里都属于稀缺资源。尤其是在那些把每个进程的默认文件描述符最大个数设置得相当低的系统上，更是如此。如果某个操作系统在这方面设置了一个很低的上限值，并且没有提供增大这个值的手段，那么它就不是运行高负荷MySQL服务器的最佳选择。

对于每个表都用其自己的文件进行表示这一做法，其另一个影响是，表的打开时间会随着表数量的增加而增大。表的打开操作会映射到操作系统提供的文件打开操作上，从而会受到系统目录查找例程效率的约束。通常情况下，这不会有多大问题，但是，当数据库里需要有大量表时，这个问题便需要认真考虑。例如，一个MyISAM表由3个文件表示。如果你需要10 000个MyISAM表，数据库目录将包含30 000文件。在有了这么多文件之后，你便会发现，表的打开操作慢了下来，因为文件打开操作需要花费时间。如果出现了这个问题，你可能会需要考虑使用一种能高效处理大量文件的文件系统。例如，即使有大量的小文件，XFS或JFS仍可以表现出良好的性能。如果不能选择另一种文件系统，则必须根据应用程序的需要重新评估一下表的结构，并且相应地重新组织表。仔细考虑一下是否真的需要这么多的表，很多时候应用程序根本不需要这么多表。为每个用户创建一个表的应用程序，会产生许多表，而这些表都拥有相同的结构。如果想要把这些表组合为一个表，可能只需在其中增加一列，用于标识每一个行属于哪一个用户即可。如果这样能显著减少表的数量，那么应用程序的性能将会得到提高。

在设计数据库时，一定要考虑这种决策是否适用于给定的应用程序。不用刚才描述这种方式合并表的原因如下。

需要增加磁盘空间。合并表可以减少所需表的数量（减少表打开时间），但是会多增加一列（增加对磁盘空间的需求）。这是典型的时间与空间的权衡问题，需要你来决定哪一个因素最为重要。如果速度最重要，那么

你可能愿意牺牲一点额外的磁盘空间。如果空间比较紧张，那么使用多个表可能也是可以接受的，虽然有些延迟。

出于安全性的考虑。这些因素可能会对你合并表的能力和期望产生束缚。让每个用户单独使用一个表的原因在于，每个表只允许具有表级权限的用户进行访问。如果合并了表，则所有用户的数据都会在一个表里。

MySQL 没有提供限制某个给定用户只能访问特定行的功能，因此，在进行表合并时，不能再想着对行进行访问控制。一种可行的办法是使用只能选择当前用户行的视图，并为这些视图授予访问权限。另外，如果所有的数据访问是受应用程序控制的（用户绝不能直接连接到数据库），那么可以合并表，并且使用应用程序逻辑来实施对合并结果的行级访问。

另一种创建大量无需如此多单个文件的表的办法是，使用 InnoDB 表，将它们存储在 InnoDB 系统表空间里。在这种情况下，InnoDB 存储引擎只会与每个表仅有的 .frm 文件相关联，并且会把所有 InnoDB 表的数据和索引信息都存储在一起。这样可以减少表示这些表所需要的磁盘文件数量，从而也会大量减少打开表所需的文件描述符数量。对于表空间里的每一个组成文件，InnoDB 只需要一个描述符即可（在服务器进程存活期间，这是个恒定的值），简单地讲，对于它所打开的任何表，在它读取这个表的 .frm 文件时，只需打开一个描述符。

11.2.9 MySQL 状态文件和日志文件

除了数据库目录之外，MySQL 数据目录还含有许多状态文件和日志文件，如表 11-1 所示。这些文件的默认位置为服务器的数据目录，其中许多默认名是从在表中表示为 HOSTNAME 的服务器主机名称继承来的。二进制日志和中继日志都会被创建为一组带编号（用 nnnnnn 表示）的文件。下面这张表格只列出了服务器级的状态文件和日志文件。个别存储引擎可能还会创建它们自己的日志或其他文件。例如，InnoDB 存储引擎便是如此。

表 11-1 MySQL 状态文件和日志文件

文件类型	默认名	文件内容
进程ID文件	<i>HOSTNAME.pid</i>	服务器进程ID
错误日志	<i>HOSTNAME.err</i>	启动/关闭事件和错误条件
一般查询日志	<i>HOSTNAME.log</i>	连接/断开事件和查询信息
二进制日志	<i>HOSTNAME-bin.nnnnnn</i>	修改数据的语句的二进制表示
二进制日志索引	<i>HOSTNAME-bin.index</i>	当前二进制日志文件名的列表
中继日志	<i>HOSTNAME-relay-bin.nnnnnn</i>	从服务器自主服务器接收到的数据修改信息
中继日志索引	<i>HOSTNAME-relay-bin.index</i>	当前中继日志文件名的列表
主服务器信息文件	<i>master.info</i>	用于连接主服务器的参数
中继信息文件	<i>relay-log.info</i>	中继日志处理的状态
慢查询日志	<i>HOSTNAME-slow.log</i>	处理起来比较耗时的语句文本

对于常规的查询日志和慢查询日志，可以选择让服务器把日志信息写入日志文件，或者写入 `mysql` 数据库里的日志表，或者写到这两个地方。关于日志信息写入表的更多信息请参考12. 8. 6节。

11. 2. 9. 1 进程ID文件

MySQL服务器会在启动时把它的进程ID (PID, process ID) 写入PID文件，并且在关闭时会删除该文件。其他进程可以利用这个文件来确定 MySQL 服务器是否在运行，以及其进程ID是什么（如果在运行的话）。例如，如果操作系统在关闭时调用`mysql.server` 脚本来停止MySQL服务器，那么该脚本会检查PID文件，确定它应该向哪个进程发出终止运行信号。

如果服务器无法创建PID文件，那么它会往出错日志里写入一条信息，然后继续。

11. 2. 9. 2 MySQL日志

MySQL能够维护多种类型的日志文件。大部分日志功能都是可选的，你可以使用服务器启动选项来启用需要的日志，也可指定它们的名字，如果你不喜欢那些默认名字的话。请注意，日志文件有可能会变得非常大，因此千万不要让它们把文件系统的空间都占满。应该定期对日志文件进行过期处理，以保证它们所占用的空间总量总在限定范围之内。

本节只简要地介绍几种常用的日志文件。与日志、控制服务器日志行为的选项以及日志过期处理有关的更多信息请参考12.8节。

错误日志 (error log) 包含的是，服务器在系统发生意外状况时生成的诊断信息。如果服务器启动失败，或者意外退出，那么这个日志会非常有用，因为它通常都包含了出现问题的具体原因。

常规查询日志 (general query log) 包含的是与服务器操作有关的常规信息，其中包括谁在连接服务器、从什么地方连接以及它们都调用了哪些语句。二进制日志还包含语句信息，但是仅限于那些修改数据库内容的语句。当这台服务器在复制结构里属于主服务器时，它还会包含其他的信息，如保持从服务器同步所需要的时间戳。二进制日志的内容是一些以二进制格式记录的“事件”，这些事件可以作为 mysql 客户端程序的输入来执行。配套的二进制日志索引文件会列出服务器当前正在维护的二进制日志文件。

二进制日志对系统崩溃后的数据库恢复工作有着重要意义，因为在进行备份之后，可以通过把二进制日志文件送至服务器的方式重复这些更新操作。这样你便可以把数据库恢复到发生崩溃前的那一刻。即便你建立了复制服务器，二进制日志也能发挥作用，因为它可以把那些必须从主服务器传递到从服务器的更新操作记录下来。更多相关信息请参考第14章。

一定要确保你的日志文件是安全的，并且不能随意被其他用户读取，因为其中的语句文本里可能包含密码之类的敏感信息。例如，下面这条日志记录便会把root用户的密码显示出来，这个可不是任何人都可以访问的信息：

```
080412 16:47:24          44 Query          SET PASSWORD FOR
'root'@'localhost'=PASSWORD('secret')
```

默认情况下，服务器会把日志文件写到数据目录里，因此，确保日志文件安全的一种措施是，只允许 MySQL 系统管理员所使用的登录账户访问服务器主机上的数据目录。有关这个步骤的详细设置请参考13.1.2节。

11.3 迁移数据目录的内容

本章的前几节讨论了默认配置里的数据目录结构，所有的数据库、状态文件和日志文件都位于其中。不过，在确定数据目录内容的存放位置方面，也有一些回旋的余地。MySQL允许你迁移数据目录本身或者其中的某些元素。你想这样做的理由可能有以下几种。

包含数据目录的那个文件系统已满，你需要将其移动到某个拥有更大容量的文件系统上。

如果数据目录位于一个很忙的磁盘驱动器上，那么你可以把它放在活动不太频繁的驱动器上，以平衡跨物理设备的磁盘活动。你可以把数据库和日志文件放在不同的驱动器上，或者基于同样的原因把数据库分散在各个驱动器上。同样地，虽然InnoDB系统表空间在概念上是一个巨大的存储块，但是可将其各个组成文件放置在不同的驱动器上，以提高性能。如果你使用了分区表，那么可以在各个分区来做这件事情。

把数据库和日志分别放在不同的磁盘上，这样有助于减少单个磁盘故障造成的破坏。

本节将讨论数据目录的哪一部分可以移动，以及如何进行移动。

11.3.1 迁移方法

迁移数据目录或其中各个部分的方法有两种。

第一种，在任何平台上，你都可以在服务器启动时，通过命令行或者选项文件，指定一个选项。例如，想要指定数据目录的位置，可以在服务器启动时，在命令行上使用`--datadir=dir_name`选项来指定，或者将下面的行放入某个选项文件中来指定：

```
[mysqld]
```

```
datadir=dir_name
```

一般情况下，服务器选项中的选项文件组名是`[mysqld]`，如上面示例所示。不过，根据具体的情况，其他选项组名可能更为适合。例如，你正在使用`mysqld_multi`运行多个服务器实例，那么这些组名将变成`[mysqldn]`这样的形式，其中，`n`是一个与某个特定的服务器实例相关联的整数。12.2.3节将讨论适用于不同服务器启动方法的不同选项组，并且将对运行多个服务器提供指导。

第二种，在Unix上，可以先移动要迁移的文件或目录，然后在原来位置生成一个符号链接，指向新位置。

这两种方法都不能完全适用于可迁移的所有内容。表11-2总结了哪些是可迁移的，可以采用哪种迁移方法。如果使用的是选项文件，则可以在全局选项文件（如Unix里的`/etc/my.cnf`，或者Windows里的`C:\my.cnf`）中指定各种选项。

表11-2 迁移方法总结

迁移实体	适用的迁移方法
整个数据目录	启动选项或符号链接
各个数据库目录	符号链接
各个数据库表	符号链接

续表

迁移实体	适用的迁移方法
InnoDB表空间文件	启动选项
服务器PID文件	启动选项
日志文件	启动选项

11.3.2 迁移注意事项

在迁移之前，一定要记得备份数据，以便在迁移操作搞砸时进行恢复。而且，应该在执行迁移操作之前，停止 MySQL 服务器，迁移完成之后再重启它。对于某些类型的迁移，如移动数据库目录，有时可能会保持服务器继续运行，但不推荐这样做。如果真要那样，一定要保证服务器不会访问正移动的那个数据库。在移动数据库之前，还应该执行FLUSH TABLES语句，以保证服务器会关闭所有打开的表文件。如果不遵守这几点，可能会导致表损坏。

11.3.3 评估迁移带来的影响

在试图迁移之前，请记得核实一下该操作可能会带来的影响。例如，在 Unix上，可以使用du、df和ls -l 命令来查看磁盘空间信息。不过，你必须正确理解文件系统的结构对所有这些命令来说意味着什么。

下面这个示例展示了对数据目录迁移进行评估时，需要特别注意的某些难以捉摸的“陷阱”。假设，数据目录为/usr/local/mysql/data，而你想要把它转移到/var/mysql，因为df命令的输出表明/var文件系统有更多的可用空间：

```
% df -k /usr /var
```

Filesystem	1K-blocks	Used	Available	Use%	Mounted on
/dev/sda5	28834716K	24078024K	3291968K	88%	/usr
/dev/sda6	28834716K	9175456K	18194536K	34%	/var

为弄清数据目录迁移之后，/usr文件系统中能释放多少空间，可以使用 `du -s` 命令：

```
% du -s /usr/local/mysql/data

3264308K /usr/local/mysql/data
```

这个结果表明，把数据目录从/usr转移到/var，将可以在/usr上释放出约3 GB的空间。真是如此吗？为查明真相，可以对数据目录使用df命令。假设得到了如下所示的输出结果：

```
% df -k /usr/local/mysql/data
```

Filesystem	1K-blocks	Used	Available	Use%	Mounted on
/dev/sda6	28834716K	9175456K	18194536K	34%	/var

奇怪！为什么df命令报告的是/var文件系统中的空间使用情况呢？下面这条 `ls -l` 命令给出了答案：

```
% ls -l /usr/local/mysql/data

lrwxrwxr-x 1 mysql mysql 10 Dec 11 23:46 data -> /var/mysql
```

这个输出显示，/usr/local/mysql/data是一个指向/var/mysql的符号链接。换句话说，数据目录早已被转移到了/var文件系统中，并且被替换为了一个指向该处的符号链接。通过把数据目录转移到/var的方式来在/usr上释放大量的空间相关问题就讲解到这里！

如果想要将某个数据库移动到另一个文件系统，以试图重新分配数据库存储，那么请记住一点：如果你正在使用存储在InnoDB系统表里的InnoDB表，那么这些表的内容并不在数据库目录里。对于主要由像 InnoDB 表这样

的表组成的数据库，在迁移数据库目录时，只需要迁移它们的 .frm 文件即可，不用迁移它们的内容。这不会对存储分布产生大的影响。

上面这些示例给我们的启发是，多花费几分钟来评估迁移可能造成的影响，可以避免做出

许多徒劳无功的事情。

11.3.4 迁移整个数据目录

在迁移数据目录时，要停止 MySQL 服务器，并把数据目录移至新的位置。在移动之后，要使用能明确指示新位置的 `--datadir` 选项来重启服务器。在 Unix 上，除了使用 `--datadir` 以外，还可以在原来的数据目录里创建一个符号链接，让它指向新位置。

11.3.5 迁移单个数据库

服务器总是会在数据目录里查找数据库目录，因此迁移数据库的唯一方法是使用符号链接。这个过程在 Unix 和 Windows 上有所不同。

在 Unix 上，迁移数据库的过程如下：

- (1) 停止正在运行的服务器。
- (2) 把数据库移至新的位置；或者将其复制过去，并删除原来那个。
- (3) 在数据目录中创建一个符号链接，让其具有原来那个数据库的名字，并指向那个新的数据库位置。
- (4) 重启服务器。

下面这个示例展示了如何把 `bigdb` 数据库从 `/usr/local/mysql/data` 目录迁移至 `/var/db`：

```
% mysqladmin -p -u root shutdown

Enter password: *****

% cd /usr/local/mysql/data

% tar cf - bigdb | (cd /var/db; tar xf -)

% rm -rf bigdb
```

```
% ln -s /var/db/bigdb bigdb
```

```
% mysqld_safe &
```

执行这些命令时，要以MySQL管理员的身份登录。

在Windows上，数据库迁移的过程稍有不同，如下所示。

(1) 停止正在运行的服务器。

(2) 把数据库移至新的位置；或者将其复制过去，并删除原来那个。

(3) 在MySQL数据目录中创建一个文件，充当符号链接，让MySQL服务器知道在何处可以找到那个迁移后的数据库目录。这个文件的扩展名为`.sym`，基本名为数据库名。例如，把`sampdb`数据库从`C:\mysql\data\sampdb`移至`E:\mysql-book\sampdb`，则需要创建一个名为`C:\mysql\data\sampdb.sym`的文件，让其包括下面一行内容：

```
E:\mysql-book\sampdb\
```

(4) 保证在重启服务器时启用了符号链接支持功能。Windows 服务器在默认情况下应该已启用了这项功能，但是你也可以在命令行上使用`--symbolic-links`选项来显式启用它，或者将下面几行内容放到某个选项文件里来启用它：

```
[mysqld]
```

```
symbolic-links
```

11.3.6 迁移单个表

只有在满足以下条件时才能迁移单个表。

必须使用Unix系统，并且要迁移的表必须为MyISAM表。

操作系统必须具有一个可工作的`realpath()`系统调用。如果真有，那么下面这个查询的结果将为YES：

```
mysql> SHOW VARIABLES LIKE 'have_symlink';
```

Variable_name	Value
have_symlink	YES

如果上述两个条件都满足，那么你便可以把表的 .MYD 数据文件和 .MYI 索引文件移动到新的位置；接着在原来的数据文件和索引文件所在的数据库目录里创建符号链接，分别指向它们。（.frm文件仍然需要留在原来的数据库目录里。）在执行此操作之前，可以在移动文件时停止服务器；也可以锁定表，以防止服务器使用它，如14.2节所描述的那样。

11.3.7 迁移InnoDB系统表空间

在初次配置 InnoDB 系统表空间时，可以使用innodb_data_home_dir和innodb_data_file_path系统变量，将InnoDB系统表空间的各个组成文件罗列在某个选项文件里。（更多相关细节请参考 12.5.3.1 节。）如果你已经创建了表空间，可能就可以迁移组成它的那些常规文件，例如，把它们分散到不同的文件系统里。因为文件的位置是通过系统变量来指定的，所以你必须按以下步骤来迁移部分或者全部的表空间文件。

- (1) 如果服务器在运行，要停止它。
- (2) 移动需要迁移的表空间文件。
- (3) 修改定义InnoDB配置的那个选项文件，反映出文件迁移后的新位置。
- (4) 重启服务器。

11.3.8 迁移状态文件和日志文件

如果要迁移 PID文件或日志文件，则需要先停止 MySQL服务器，然后使用一个可以指定文件新位置的选项来重启它。例如，为把 PID 文件创建为/tmp/mysql.pid，可以在命令行上使用--pid-file=/tmp/mysql.pid选项，或者在某个选项文件里包含以下内容：

```
[mysqld]

pid-file=/tmp/mysql.pid
```

如果是用绝对路径名来指定文件名，那么服务器会使用该路径名来创建文件。如果使用的是相对路径名，那么服务器将在其自己的数据目录中创建该文件。例如，当指定`--pid-file=mysql.pid`时，这个PID文件即为数据目录下的`mysql.pid`。

有些系统会把各种服务器的PID文件保存在某个特定的目录里，如`/var/run`。你可能也会想要把MySQL的PID文件保存到那里，从而与系统操作的保持一致。类似地，如果你的系统把日志文件都保存在`/var/log`目录里，那么你也可以把MySQL的日志文件放置到那里。不过，许多系统都只允许root用户对这些子目录进行写操作。这意味着，你必须以root用户权限来运行服务器——从安全的角度来讲，这可是个不太好的做法。比较可行的做法是：创建两个子目录`/var/run/mysql`和`/var/log/mysql`，并把它们的所有者设置成你用于运行服务器的那个账户。例如，这个账户的用户名和用户组名都是mysql，以root用户的身份执行以下命令：

```
# mkdir /var/run/mysql

# chown mysql /var/run/mysql

# chgrp mysql /var/run/mysql

# chmod u=rwx,go-rwx /var/run/mysql

# mkdir /var/log/mysql

# chown mysql /var/log/mysql

# chgrp mysql /var/log/mysql

# chmod u=rwx,go-rwx /var/log/mysql
```

然后，服务器就能毫无问题地在这些目录里写入文件了，并且可以使用指定这些文件所在位置的选项来启动服务器。例如：

```
[mysqld]

pid-file = /var/run/mysql/mysql.pid

log-error = /var/log/mysql/log.err

general_log = 1
```

```
general_log_file = /var/log/mysql/querylog
```

```
log-bin = /var/log/mysql/binlog
```

与日志文件选项及其用法有关的更多信息请参考12.8节。

第12章

MySQL的基本管理

本章将讨论你在负责管理网站的MySQL时，为保证MySQL的平稳运行而必须履行的各项职责，包括以下职责。

在MySQL安装完成之后加强安全设置。

确保服务器尽可能长时间地正常运行。

监视和设置服务器的运作参数。

选择插件，启用服务器功能。

维护服务器日志。

调整服务器，以获得更好的性能。

分析底层硬件，解决物理约束，进而提高性能。

运行多个服务器。

确定是否需要将MySQL升级到较新版本，若需要，何时升级。

其他一些比较重要的管理内容将在第13章和第14章进行讨论。

本章将涉及下面几个应用程序，它们对MySQL管理员来讲都是不可或缺的工具。

MySQL服务器mysqld。

服务器启动程序`mysqld_safe`、`mysqld.server`和`mysqld_multi`。

管理工具`mysqladmin`。

MySQL的数据目录里存储着 MySQL服务器的数据库、日志文件和其他信息。对 MySQL的数据目录有所了解，有助于更好地理解本章的大部分内容。更多详细内容请参考第11章。与本章讨论的SQL语句和程序特定相关的更多信息请参考附录E和附录F。

12.1 保护新安装的MySQL

我们先从一项在MySQL 安装之后应该立即执行的管理任务开始，其目的在于要确保服务器只能被授权用户访问。其工作内容包括了解安装过程会创建哪些 MySQL 用户账户，为你需要的那些账户设置密码，删除那些你不需要的账户。

MySQL的安装过程会在服务器的数据目录里生成两个数据库。

数据库`mysql`。它里面包含了用于控制客户端对服务器进行访问的各种权限表。

数据库`test`。它可用于测试。

如果你刚刚完成了MySQL的第一次安装，那么`mysql`数据库的权限表将会包含多个账户，其初始状态是允许任何人在无需身份认证的情况下连接到服务器。这样做很不安全，因此你应该尽快为这些账户分配密码。如果你是在一台已经安装有 MySQL的机器上，选择了另一个位置来安装另一个版本，那么你需要为这个新服务器设置密码。不过，你可能会遇到这样的麻烦：服务器可能会从已有版本创建的选项文件里提取这些密码，详见12.1.2节的讨论。

下面的讨论会假设你还没有建立任何密码。有些 MySQL 安装程序会在安装过程中让你选择创建密码。不过，即使你使用了这种安装程序，这里提供的信息也能帮助你更好地理解初始的MySQL用户账户。

12.1.1 为初始的MySQL账户建立密码

本节将讲述如何检查权限表里存在有哪些账户，以及如何设置它们的密码。具体来讲，本节的示例都将假设你在某台主机名为`cobra.example.com`的机器上运行MySQL服务器，并想从这台机器连接该服务器。你要将本节中出现的主机名，替换成你自己服务器主机的名字。这些示例还会假设你已经启动了MySQL服务器，因为你需要连接它。

MySQL安装过程会在mysql数据库的权限表里设置以下两种账户。

用户名为 root的账户。这些是用于管理的超级用户账户。root 账户拥有全部的权限，可以做任何事情，其中包括删除所有的数据库和关闭服务器。（MySQL和Unix的超级账户名字都是 root，这只是一种巧合。虽然这两个超级账户都拥有最高的权限，但彼此毫无瓜葛。）

用户名为空白的账户。这些是“匿名”账户。有了它们，任何人都可以连接服务器，而不用事先显式地为他们建立账户。匿名用户的权限通常比较少，为的是对它们所能进行的操作加以限制。

MySQL服务器上已有的账户全部都列在mysql数据库的user表里，因此你可以在那里找到那些初始账户。默认情况下，这些账户都没有密码，因为MySQL 认为你肯定会提供自己的密码。因此，管理MySQL安装的第一步应该是建立密码。否则，未授权的用户可以用root身份连接服务器，获得服务器的超级用户访问权限。

在加强初始账户的安全之后，还应该继续创建其他的账户，以使用户社区的成员可以使用你所指定的名字来连接服务器，同时拥有许可范围内的适当权限。13.2 节对如何创建新账户和修改已有账户提供了指导。

user 表里的每一个行都包含有Host值和User值，它们分别代表的是主机（用户可以从那里连接服务器）和用户名（当用户从主机进行连接时必须提供它）。每一个行还包含了Password 列和plugin 列，它们表示的是这个用户的授权方式：如果账户密码不为空，那么Password就是这个账户的密码（它已被散列处理，不会以明文形式显示出来）；如果身份认证插件（服务器为了使用这个账户会调用它来处理身份认证事宜）不为空，则plugin为该插件。此外，user表的行还有很多其他列，用来表示此账户所拥有的各种超级权限。

要想查看存在有哪些账户，以及这些账户是否需要授权，可以用root 身份连接服务器查询user表。这时，应该不需要指定密码，因为一开始，即使是root账户也还没有密码。

```
% mysql -u root
```

```
mysql> SELECT Host, User, Password, plugin FROM mysql.user;
```


Host	User	Password	plugin
localhost	root		
cobra.example.com	root		
127.0.0.1	root		
::1	root		
localhost			
cobra.example.com			

在你自己的服务器上看到的输出结果，与上面所显示的内容可能不完全相同。但是，如果你所看到的每一个账户，其 Password和plugin的内容都为空，就说明它们是不安全的，应该为它们分配密码。

数据目录里的权限表源自何处呢？依据你所使用的平台不同，答案也会有所不同。

在Unix 系统里，mysql_install_db 脚本会在安装过程中完成数据目录的初始化工作。

mysql_install_db脚本的其中一个目标便是在mysql数据库里建立权限表。如果你是在Linux系统上通过RPM软件包来安装MySQL，或者是在Mac OS X系统上使用DMG软件包来进行安装的，那么 mysql_install_db 脚本将会自动运行。否则，你需要自己运行它。更多相关信息请参考F.7节。

在Windows系统里，数据目录和mysql数据库都被包含在预先初始化好的MySQL发行包里。

在所有平台上，大部分的初始账户都是相同的。有一些账户则与特定的平台有关。下表显示的是那些在所有平台里都存在的账户。尽管如前面提到的，所有这些账户在开始时都没有密码，但是安装程序会给你机会选择是否需要建立密码。

主机	用户	超级用户权限
localhost	root	全部
127.0.0.1	root	全部
::1	root	全部
localhost		无

user表里的这些账户允许客户端程序使用的连接方式如下所示。

root账户可以使用主机名localhost、IPv4地址127.0.0.1或IPv6地址::1来连接本地MySQL服务器。例如，你可以使用下面两条命令中的任何一条，以root身份连接服务器：

```
% mysql -h localhost -u root
```

```
% mysql -h 127.0.0.1 -u root
```

对于Host值为::1的账户，可以通过IPv6方式从本地主机进行连接。如果你的系统支持IPv6，就可以像下面这样使用此账户来进行连接：

```
% mysql -h ::1 -u root
```

作为root，你将拥有全部权限，可以执行任何操作。

User值为空白的账户是匿名账户。它允许使用主机名localhost连接本地MySQL服务器，而不用指定任何用户名：

```
% mysql -h localhost
```

匿名用户没有超级用户权限。

在Unix系统里，可以通过运行mysql_install_db脚本来初始化权限表，你会得到两个与主机特定相关的账户：一个是用户root，另一个是匿名用户。下表列出了那些与主机特定相关的账户（假设服务器主机名为cobra.example.com）。

主机	用户	超级用户权限
cobra.example.com	root	全部
cobra.example.com		无

在Windows系统里，你看不到这些账户，因为那些权限表都包括在MySQL发行包的预初始化操作里，并且发行打包程序并不知道你的主机名。

如果你登录到主机cobra.example.com上，那么特定于主机名的那些账户会允许你通过指定主机名，以root用户或者匿名用户身份连接本地服务器：

```
% mysql -h cobra.example.com -u root
```

```
% mysql -h cobra.example.com
```

在Windows里，安装程序可能会让你选择创建一个可以从任何地方进行连接的root账户。这个账户的Host和User值分别为%（与任何主机相匹配）和root，并且拥有所有的超级用户权限。

在所有平台里，另外还有一个权限表（即db 表，这里未显示出来），它包含的权限信息可以让所有的用户（甚至包括匿名用户）使用test 数据库或者任何以 test_作为名字开头的数据库。

本节的其余部分将讲述如何设置root用户和匿名用户的密码。这些示例主要使用了一组具有代表性的账户，但是有些特定的SQL语句实际依赖于你的系统上存在哪些账户。

根据你所使用的密码设置方法的不同，可能还需要让服务器重新加载授权表，以使修改生效。服务器通过使用权限表的内存副本来实现访问控制。对于某些修改 user 表密码的方法，服务器有可能察觉不到你所做的修改，因此你必须显式地告诉它去重新读取权限表。

一种密码分配方法是：以 root 身份连接服务器，确定出哪些账户不安全（因为没有为它们指定身份认证信息），然后对每一个账户使用一次SET PASSWORD语句。假设，你连接到服务器，发现以下账户不安全：

```
% mysql -u root
```

```
mysql> SELECT Host, User FROM mysql.user
```

-> WHERE Password = '' AND plugin = '';

Host	User
localhost	root
cobra.example.com	root
127.0.0.1	root
::1	root
localhost	
cobra.example.com	

可以使用SET PASSWORD 语句分配这些账户的密码。在语句中指定账户名字时，应该采用'user_name'@'host_name'的格式，其中user_name和host_name分别对应于你想要修改的user表行里的User列值和Host列值。（如果User值为空白，那么user_name值为空字符串''。）要想为刚才显示的root用户或匿名用户设置密码，可以使用下面的语句来实现：

```
mysql> SET PASSWORD FOR 'root'@'localhost' = PASSWORD('rootpass');
```

```
mysql> SET PASSWORD FOR 'root'@'cobra.example.com' =  
PASSWORD('rootpass');
```

```
mysql> SET PASSWORD FOR 'root'@'127.0.0.1' = PASSWORD('rootpass');
```

```
mysql> SET PASSWORD FOR 'root'@'::1' = PASSWORD('rootpass');
```

```
mysql> SET PASSWORD FOR ''@'localhost' = PASSWORD('anonpass');
```

```
mysql> SET PASSWORD FOR ''@'cobra.example.com' =  
PASSWORD('anonpass');
```

可以替代SET PASSWORD语句的另一种方法是使用UPDATE语句直接修改user表。可以用这种方法为拥有某个给定User值的所有账户（不管它们的Host值是什么）设置一个密码，并且可以同时修改多个账户。下面这几条语句可以设置所有root账户和匿名用户账户的密码：

```
mysql> UPDATE mysql.user SET Password=PASSWORD('rootpass') WHERE
User='root';
```

```
mysql> UPDATE mysql.user SET Password=PASSWORD('anonpass') WHERE
User=";
```

```
mysql> FLUSH PRIVILEGES;
```

当使用SET PASSWORD语句更改密码时，服务器会觉察到你已对权限表进行了修改，并且会自动重新读取它们，以更新这些表在内存里的副本。如果使用UPDATE语句直接修改user表，则必须显式地通知服务器重新加载这些权限表。这就是在上面的示例里，FLUSH PRIVILEGES语句紧跟在UPDATE语句之后的目的。

另一种处理匿名用户账户的方法是把它们彻底删除掉。除非你有特定的原因需要保留它们，否则强烈建议删除它们。可以使用DROP USER语句来删除匿名用户：

```
mysql> DROP USER "@'localhost';
```

```
mysql> DROP USER "@'cobra.example.com';
```

对于DROP USER语句，服务器会自动重新读取权限表，因此不需要FLUSH PRIVILEGES语句。

删除匿名用户账户的一个好处是，可以提高安全性。另一个好处是，它可以大大简化设置非匿名用户账户的工作。如果想保留匿名用户，那么你可能必须处理在13.4.4节里描述的那个奇怪现象。关于现象背后的更多细节，请仔细阅读那节的内容。

在为账户设置好密码（并根据情况重新加载了权限表）之后，每次连接服务器时将必须给出正确的密码。特别是，在不知道密码的情况下，任何人都无法以root身份连接服务器：

```
% mysql -u root
```

```
ERROR 1045 (28000): Access denied for user 'root'@'localhost'
```

```
(using password: NO)
```

```
% mysql -p -u root
```

```
Enter password: rootpass
```

```
mysql>
```

从此刻起，无论是mysql程序，还是像mysqladmin和mysqldump那样的其他程序，在连接服务器时都需要指定密码。为简明起见，本章后面各节里的很多例子都省略了-u和-p选项。你在连接服务器时，要根据需要加上它们。

12.1.2 为附加服务器设置密码

前面的指导是假设你在以前未安装过MySQL的系统上建立密码。不过，如果你已在某个位置安装了MySQL，并打算为安装在同一台计算机的另一个位置里的新服务器设置密码，那么你可能会发现，当你在不提供密码的情况下试图去连接这个新服务器时，它会拒绝你的连接，并给出下面的出错信息：

```
% mysql -u root
```

```
ERROR 1045 (28000): Access denied for user 'root'@'localhost'
```

```
(using password: YES)
```

奇怪！你并没有指定密码，可是服务器为什么会说它收到了密码呢？这通常表明你在某个地方设置了一个选项文件，其中包含有用于访问以前安装的那个服务器的密码。mysql 会找到这个选项文件，并自动使用列在其中的密码。要想改写它，并显式指定“没有密码”，可以使用--skip-password选项来实现：

```
% mysql -u root --skip-password
```

这一策略也适用于其他的MySQL客户端程序，如mysqladmin。

更多关于使用多个服务器的讨论请参考12.9节。

12.2 安排MySQL服务器的启动和关闭

MySQL管理员的一项基本职责就是，保证mysqld服务器尽可能长时间地运行，以使用户能够访问它。不过，有时候也会需要把服务器停止一下。例如，当你在重定位数据目录时，为了不让服务器在你操作的同时更新表，就需要关闭服务器。既想要保持服务器运行，又想要偶尔能停止它——对于这样的两难问题，本书也无能为力。但是，我们还是要对如何启动和关闭服务器进行讨论，以便让你有能力根据需要执行它们。这些操作过程在Unix和Windows里有很多不同，下面将分别对它们进行讨论。

12.2.1 在Unix里运行MySQL服务器

在Unix里，MySQL服务器可以通过命令行手动启动。也可以将其作为标准启动过程的一部分在操作系统启动时自动运行。（事实上，在正常的操作环境里，当你根据需要把所有内容都设置好之后，便可以按这种方式来启动服务器。）但是在讨论如何启动服务器之前，需要先考虑一下应该使用哪一个登录账户来启动它。在像Unix这样的多用户操作系统里，需要对使用哪个登录账户来运行服务器进行选择。如果手动启动服务器，那么服务器会以登录时的Unix用户身份运行。例如，以paul身份登录并启动服务器，那么它会以paul身份运行。如果使用su命令将用户转换至root，然后再启动服务器，那么它就会以root身份运行。

请记住，在Unix里，MySQL服务器的启动过程要实现以下两个目标。

应该以root以外的其他用户身份来运行服务器。当在说到“作为”给定用户身份运行服务器时，其意思是，服务器进程与该用户Unix登录账户的用户ID是相关联的，并且在文件系统里，它具有该用户对文件的读、写权限。这里隐含着某些安全问题，特别是对于以 root 用户身份运行的进程，因为 root 可以做任何事情，所以非常危险。避免这些危险的一种方法是让服务器放弃一些特定的权限。以root身份启动的进程，能够将它们的用户ID更改为另外的账户，从而放弃 root 的权限，变为一个无特权的普通用户。这样，该进程的危险程度会大大降低。一般情况下，应该对所有进程的权限进行限制，除非它真的需要root访问权限（mysqld不需要）。服务器需要root权限来访问和管理MySQL数据目录的内容，除此以外，几乎没有什么操作再需要root权限。这意味着，如果以root身份启动服务器，则应该告知它在启动过程中需要更改自己的用户ID，以一个无特权用户的身份运行。

每一次都应该以同一个用户身份来运行服务器。当服务器不定时以不同用户权限运行时，会出现权限不一致的问题。这时，在数据目录里创建的文件和目录会拥有不同的所属模式，会导致服务器在以某个用户身份运行时无法访问某些数据库和表。只有固定以同一个用户身份运行服务器，才能从根本上避免这个问题。

12.2.1.1 用无特权的登录账户运行服务器

使用root以外某个单独的无特权账户来从事与MySQL有关的活动具有以下几个好处。

不会有人将服务器作为一个安全漏洞来利用，以获得root访问权限。

服务器创建的文件都属于 root 以外的另一个无特权账户。例如，拥有 FILE 权限的MySQL用户便无法让服务器往 root 所拥有的文件里写入内容。在系统里，像这样的文件越少越好。

当用root 以外的无特权用户身份登录时，执行 MySQL的管理任务会更加安全。如果你不小心以root用户身份执行了某个文件系统操作，那么后果可能会不堪设想。

另外创建一个专门用于MySQL有关活动的账户，在概念上和管理上都会更加清晰，同时也更容易弄清系统里哪些内容是与MySQL有关的。例如，在存放crontab文件的目录里，便单独有一个针对MySQL用户的文件。如果没有这个文件，MySQL的cron作业将会被列在root的crontab文件里，而其他所有事件都会以root身份定期执行。

要想将mysqld设置为以无特权的非root用户运行，可以遵循以下步骤。

(1) 停止正在运行的服务器：

```
% mysqladmin -p -u root shutdown
```

(2) 选择用于运行mysqld的登录账户。你也可以指定一个专用于MySQL的组名。在本书中，使用的用户和组名都是 mysql。如果想要使用不同的名字，可以将本书中所有作为用户名或组名的mysql替换掉。例如，你可能会因为在系统里没有专门的管理权限，而以自己的账户来安装MySQL。在这种情况下，你还会以自己的身份来运行服务器，因此你应该用自己的登录名和组名来替换mysql。

(3) 如有必要，可以使用系统的常规账户创建步骤来为你选择的那个用户名创建登录账户。此时，需要以root身份来完成此项工作。

如果你决定使用名为 mysql的账户来运行服务器，可能就不需要自己创建。如果在Linux系统里使用RPM软件包来安装MySQL，安装过程会自动创建这个账户。Mac OS X系统内部自带一个创建好的mysql账户。其他系统可能也是同样的情况。

(4) 修改 MySQL 数据目录及其下面所有子目录和文件的用户和组的所有权，让它们归mysql用户所用。例如，如果数据目录为/usr/local/mysql/data，就可以像下面这样设置该目录及其内容的所有权：

```
# chown -R mysql /usr/local/mysql/data
```

```
# chgrp -R mysql /usr/local/mysql/data
```

这些命令需要以root身份来运行。

(5) 一种很好的安全预防措施是：将数据目录的访问模式设置为其他人都不可访问。为做到这一点，可以修改其权限，只允许mysql用户访问它。如果数据目录为/usr/local/mysql/data，就可以像下面那样关闭“组”和“其他”权限，从而将该目录下的所有内容都设置为只有mysql可以访问：

```
# chmod -R go-rwx /usr/local/mysql/data
```

最后两步实际上是更为全面的锁定过程的一部分，详细内容请参考13.1.2节。通过查看该节内容，可以获得更多关于设置所有权和进行模式分配的指导信息，尤其是在你的MySQL 是以非标准布局安装的时候。

在完成前面几个步骤之后，请记得在启动服务器时带上--user=mysql选项，这样以root身份来调用时，才能将自己的用户ID切换至mysql。对于以root身份手动运行服务器的情况，以及将服务器设置为在操作系统启动期间被调用的情况，都应该这样做。Unix 系统会以 Unix的root用户来执行启动操作，因此初始化为该过程一部分的所有进程默认都具有root权限。

确保该用户一直被指定的最好方法是：将其列在服务器会读取的某个选项文件里。例如，可以将下列内容放置在/etc/my.cnf中：

```
[mysqld]

user=mysql
```

更多与选项文件有关的信息请参考12.2.3节。

如果碰巧在启动服务器时是以 mysql身份登录的，那么当选项文件中存在有 user 行时，会产生一个警告——只有 root 用户才能使用该选项。这意味着，服务器没有能力更改其用户ID，只会以mysql身份运行。那正是你所希望的事情，因此不用理会这个警告。

12.2.1.2 在Unix里启动服务器

在决定使用哪一个登录账户来运行MySQL之后，关于如何启动服务器还有多种选择。你可以从命令行手动启动它，也可以在系统启动过程中自动启动它。相关的启动方法包括以下几种。

直接调用mysqld。这大概是最不常用的方法。本书不会对它再做过多讨论。只是想说，下面这条命令有助于找出服务器都支持哪些启动选项：

```
% mysqld --verbose --help
```

调用`mysqld_safe`脚本。`mysqld_safe`脚本会为你调用服务器，然后监视它；当服务器意外中断时，还会重启它。`mysqld_safe`脚本通常被用在BSD风格的Unix版本里，在非BSD系统和Mac OS X系统里，也可以通过`mysql.server`来调用它。

`mysqld_safe`脚本会将服务器的出错信息和其他诊断输出内容重定向到数据目录下的某个文件，产生一个出错日志；或者重定向到 `syslog`。如果将错误输出发送给文件，那么`mysqld_safe`脚本会设置该文件的所有权，使其归`--user`选项所指定的那个登录账户所有。在不同的时间使用不同的`--user`值会引起麻烦。问题的具体表现是：当`mysqld_safe`试图写入出错日志文件时，会因“权限不足”而失败。它之所以会成为一个问题，是因为在你想要通过检查出错日志来弄清问题出在哪里时，无法在其中找到任何与导致问题出现的原因有关的有用信息！如果出现了这样的问题，则需要删除出错日志文件，并重新调用`mysqld_safe`。

调用`mysql.server`脚本。`mysql.server`脚本会通过执行`mysqld_safe`来启动服务器。调用此脚本时，可以用参数`start`或`stop`来表明是要启动服务器，还是要停止它。它只是将`mysqld_safe`包装了一下，并且常在采用System V方法将启动和停止脚本分散在多个目录的系统里使用。其中，每个目录都对应于某个特定的运行级，并且包含有当计算机进入或退出那个运行级时会调用的各个脚本。

要想协调多个服务器，可以使用`mysqld_multi`脚本。这个脚本会读取你在其中为多个服务器列出启动参数的那个选项文件。有了它之后，你便可以对每一个服务器的启动或停止进行控制，或者检查它是否正在运行。这个启动脚本比其他脚本复杂，因此本书将在12.9节对其进行详细讨论。

脚本 `mysqld_safe`和`mysqld_multi` 都安装在MySQL 安装目录的`bin` 目录里，而`mysql.server` 脚本可以在`support_files` 目录里找到。要想使用`mysql.server`，你可能需要将其复制到正确的运行级目录里，使其变得可执行。（有些安装方法会替你安装好 `mysql.server`脚本。例如，Linux RPM和Mac OS X DMG软件包便是这么做的。）

说明

通常情况下，你会将`mysql.server`脚本以`mysql`这个名字安装到运行级目录下，但是书仍将继续使用`mysql.server`这个名字来进行讨论，以便能够清楚地表明本书所指的容是什么。如果你使用了其他供应商提供的MySQL RPM，那么它可能也会安装类似的启脚本，只是名字不一样而已（如使用`mysqld`）。

为让启动脚本在系统开机时可以执行，需要做一些安排，安排的具体内容取决于你所使用的操作系统的类型。请仔细阅读下面的示例，使用或调整

与你的系统启动过程最为接近的指导步骤。

对于 BSD风格的系统，在/etc 目录里通常会有几个在启动时初始化服务的文件。这些文件的名字通常是以rc作为开头，并且很可能会有一个名为rc.local（或类似的名字）的文件，专门用于启动本地安装的各种服务。在基于rc的系统里，可以把下面几行内容添加到rc.local里来启动服务器：

```
if [ -x /usr/local/bin/mysqld_safe ]; then  
  
    /usr/local/bin/mysqld_safe &  
  
fi
```

如果你系统上的mysqld_safe路径名与此不同，则需要修改相应行。

对于System V风格的系统，可以安装mysql.server。具体做法是：将它复制到位于/etc下的运行级目录里。如果运行的是Linux，并且是通过RPM包来安装MySQL的，那么操作到此结束。否则，还需要在启动脚本主目录安装这个脚本（其名字可以自定），确保其可执行，并将对它的链接放置到相应的运行级目录里。

运行级目录的布局因系统之间的差异而有所不同，因此必须检查你的系统是如何组织这些目录的。例如，许多Linux的变体在/etc/init.d和/etc/rc.d 下面都有一个目录集。这类Linux 系统通常都有一个用于启动脚本管理的chkconfig 命令，你可以用它来帮助你安装mysql.server 脚本。下面的指导将展示如何将mysql.server 脚本以mysql 这个名字安装到启动目录里。

（1）把mysql.server脚本从当前位置复制到init.d目录里，并使其可执行：

```
# cp mysql.server /etc/init.d/mysql  
  
# chmod +x /etc/init.d/mysql
```

（2）注册脚本，并启用它：

```
# chkconfig --add mysql  
  
# chkconfig mysql on
```

(3) 要想验证该脚本是否已正确启用，可以在运行chkconfig时带上--list选项：

```
# chkconfig --list mysql

mysql                0:off    1:off    2:on     3:on     4:on
5:on                 6:off
```

其输出表明该脚本程序会自动以2、3、4和5这几个运行级别自动执行。

如果没有chkconfig，可以使用手动过程。要想使脚本以级别3的方式运行，可以使用下面这几条命令：

```
# cp mysql.server /etc/init.d/mysql

# cd /etc/init.d

# chmod +x mysql

# cd /etc/rc.d/rc3.d

# ln -s /etc/init.d/mysql S99mysql
```

在Mac OS X里，目录/Library/StartupItems和/System/Library/StartupItems里包含着随操作系统一同启动的那些服务所对应的子目录。MySQL官方网站为Mac OS X系统提供的DMG软件包里，便包含有一个安装程序，它会把针对MySQL服务器的一个启动项添加到其中的一个目录里。

12.2.2 在Windows里运行MySQL服务器

在Windows系统里，可以通过命令行手动启动MySQL服务器。还可以为MySQL创建一个Windows服务。你可以将该项服务设置为在Windows启动时自动运行，并可以通过命令行或者Windows服务管理器去控制它。

针对Windows的MySQL发行版包含有多种服务器程序，各自的构建选项都不相同。本书的各个示例中所使用的服务器名字均为mysqld。你手上的MySQL发行版很可能还包含一个名为mysql-debug的服务器，它与mysqld很像，不同之处在于它带有调试支持功能。如果你不需要调试功能，则可以选择mysqld，因为mysql-debug会占用更多的内存，运行速度更慢。

Windows版本服务器提供的两种连接类型是Unix版本服务器里没有的。

命名管道连接。如果服务器在启动时带有`--named-pipe`选项，就使用此连接。

共享内存连接。如果服务器在启动时带有`--shared-memory`选项，就使用此连接。

与启用这些连接类型有关的更多信息请参考12.2.4节。

12.2.2.1 在Windows里手动运行服务器

要想从命令行手动启动服务器，需要把当前位置改成服务器的安装目录，然后输入下面这条命令：

```
C:\> mysqld
```

可以在命令行或选项文件里指定其他选项请参考12.2.3节。

要想让出错信息进入控制台窗口，而不是进入出错日志（默认为数据目录里的`HOSTNAME.err`文件），可以使用`--console`选项：

```
C:\> mysqld --console
```

当以控制台模式运行服务器时，可以在命令行的`--console`选项之后，或者选项文件里，指定其他的选项。

如果通过命令行来启动服务器，那么只有等到服务器退出时才能看到下一个命令提示符。这是正常的。要想运行客户端程序，只需要另外打开一个控制台窗口即可。

要想停止服务器，可以使用`mysqladmin`：

```
C:\> mysqladmin -p -u root shutdown
```

12.2.2.2 以Windows服务的方式运行服务器

在Windows系统里，可以用下面的命令为MySQL创建一个Windows服务：

```
C:\> "C:\Program Files\MySQL\MySQL Server 5.5\bin\mysqld" --install
```

这条命令使用了MySQL服务器程序的完整路径名。如果你的MySQL服务器安装在其他地方，则需要对这条命令里的路径名做出相应的修改。

服务创建命令本身实际上并不启动mysqld，它只是让mysqld在Windows每次启动时自动运行。如果你希望创建的服务不会自动运行，那么可以把MySQL服务器安装为“手动”服务：

```
C:\> "C:\Program Files\MySQL\MySQL Server 5.5\bin\mysqld" --install-  
manual
```

一般来说，当为MySQL服务器创建Windows服务时，不能在命令行里指定其他的选项，而只能将它们列在选项文件里（参考12.2.3节）。不过，可以把服务名和选项文件指定为参数，详见接下来的讨论。在创建多个MySQL服务时，这种做法尤其有用（参考12.9节）。

当为MySQL创建Windows服务时，其默认的服务名为MySQL（不区分大小写）。也可以在--install选项的后面显式地指定一个服务名：

```
C:\> "C:\Program Files\MySQL\MySQL Server 5.5\bin\mysqld" --install  
service_name
```

每一个Windows服务都必须有一个唯一的名字，因此选择一个不同于MySQL的服务名的原因在于，可以把多个MySQL服务器作为Windows服务来运行。服务名会对服务器在启动时从选项文件里读取哪些选项组产生影响。

如果既没有给出service_name参数，也没有使用服务名MySQL，那么服务器将使用默认服务名（即MySQL），并且会从标准选项文件里读取[mysqld]选项组。

如果service_name参数不同于MySQL，服务器会把这个名字作为服务名，并且会从标准选项文件里读取选项组[mysqld]和[service_name]。

在创建MySQL服务时，如果指定了服务名，那么还可以在命令行指定一个--defaults-file选项，作为最后选项（要在同一行内输入命令）：

```
C:\> "C:\Program Files\MySQL\MySQL Server 5.5\bin\mysqld"  
  
--install service_name --defaults-file=file_name
```

如此一来，又多出了一种提供服务器相关选项的方式。这个语法要求指定服务名。要想使用默认服务名，可以把service_name的值指定为MySQL。服务器会记住这个文件名，并且每次启动都会使用它。服务器只会从该文件里读取选项。默认读取的用户组为[mysqld]；当service_name不为MySQL时，会读取[service_name]组。

虽然可以在服务名的后面加上一个与--defaults-file 不同的选项，但是--defaults-file选项更为灵活，因为你可以将你所需的选项全部放置在文件里。更多与选项文件有关的信息请参考12.2.3节。

在服务器以服务方式安装好之后，你便可以利用服务名来控制它。这个操作可以通过命令行来完成。如果你喜欢图形界面，也可通过Windows服务管理器来完成。服务管理器的位置因Windows版本的不同而有所差异。

要想从命令行启动或停止服务，可以使用下面的命令：

```
C:\> net start MySQL
```

```
C:\> net stop MySQL
```

如果你使用服务管理器，那么它会提供一个窗口，显示服务列表和附加信息（如每个服务是否在运行，是以自动方式运行，还是以手动方式运行）。要想启动或停止MySQL服务器，需要在服务列表里选择其条目，然后选择相关的按钮或菜单项。

也可以在命令行使用mysqladmin shutdown来停止MySQL服务器。

说明

虽然使用服务管理器或者在命令行调用命令都可以控制服务，但是应该避免交错使用这两种法。在从命令行调用服务相关的命令之前，要先关闭服务管理器。

要想从服务列表里删除MySQL服务器，得先停止它（如果它正在运行），然后调用下面的命令：

```
C:\> mysqld --remove
```

这条命令将删除默认服务名为MySQL的那个MySQL服务。要想显式地表明删除哪一个服务，可以在--remove选项的后面指定其名字：

```
C:\> mysqld --remove service_name
```

12.2.3 指定服务器启动选项

在任何一个平台里，都有两种主要的指定启动选项的方法，用于调用MySQL服务器。

在命令行上列出各个选项。在这种情况下，对于所有提供了长、短格式的选项，既可以使用长格式，也可以使用短格式。例如，既可以使用`--user=mysql`，也可以使用`-u mysql`。

将选项列在选项文件里。当采用这种方式指定选项时，需要每行指定一个。只能使用长选项形式，并且前面不能带有短横线：

```
[mysqld]
```

```
user=mysql
```

有关选项文件的格式和语法，以及服务器在何处可以找到它们的一般性讨论，请参考F. 2. 2节。

这两种选项指定方法不会相互排斥。服务器无论是在选项文件里，还是在命令行里，都可以查找选项，只是命令行的选项具有更高的优先级。

通常情况下，最简单的方法是使用选项文件，因为对于采用这种方式指定的选项，不管使用的是哪一种启动方法，在服务器每次启动时它们都能生效。在命令行上列出的选项，只有在手动启动服务器时，或者通过`mysqld-safe`启动时才会起作用。对于`mysql.server`，在命令行列出选项的方式不起作用，因为它只支持参数`start`和`stop`。此外，还存在几种例外情况，如果在命令行使用`--install`、`--install-manual`或者`--remove`来安装或移除以Windows 服务形式存在的服务器，就无法指定启动选项。（有关例外情况的讨论请参考12. 2. 2. 2节。）

服务器查找选项文件的具体位置取决于MySQL的版本（参考F. 2. 2节）。但是，Unix系统里的`/etc/my.cnf`和Windows系统里的`C:\my.ini`都是十分标准的位置。要想弄清服务器查找选项文件的位置，可以运行下面的命令：

```
% mysql --verbose --help
```

如果你想用的选项文件还不存在，可以按普通文本文件格式来创建它。

通常情况下，MySQL服务器启动选项都在`[mysqld]`选项组里。例如，如果想让MySQL服务器以`mysql`身份运行，并使用`/usr/local/mysql`作为基本目录位置，那么可以通过在选项文件里放置下面几行内容来指明：

```
[mysqld]
```

```
user=mysql
```



```
basedir=/usr/local/mysql
```

也可以在命令行上使用选项来启动服务器，实现上述效果：

```
% mysqld --user=mysql --basedir=/usr/local/mysql
```

表 12-1列出的是 My SQL服务器和服务器启动程序所使用的标准选项组。其中，mysqld所对应的那一行也适用于各种名字有所变化（如Windows里的mysqld-debug）的My SQL服务器。

表12-1 My SQL服务器程序使用的选项组

程序	程序使用的选项组
mysqld	[mysqld], [server]
mysqld_safe	[mysqld], [server], [mysqld_safe]
mysql.server	[mysqld], [server], [mysql_server], [mysql.server]

在往选项组里放选项时，要选择在你想用的上下文里会用到的选项组。可以在选项组[mysqld]和[server]里放置适用于 mysqld的各种选项。有了选项组[mysql_safe]、[mysql_server]和[mysql.server]，你就能指定仅适用于某个启动脚本的选项。

在Windows里，如果你不使用默认服务名来为MySQL创建Windows服务，就会对服务器读取的那些选项组产生影响。更多详细信息请参考12. 2. 2. 2节。

mysql.server 脚本读取选项文件只是为了查找选项 basedir、datadir、pid-file和服务-startup-timeout的值。

12. 2. 4 控制服务器的连接监听

MySQL服务器能够在多种网络接口上对连接进行监听，相应的控制方式如下所示。

在所有平台里，服务器都可以对基于IP地址和网络端口的TCP/IP连接进行监听。不过在启动时使用--skip-networking选项，可以跳过这个监听功能。自MySQL 5. 6. 6版本起，默认IP地址为*（即监听所有的IPv4和IPv6接口）。在5. 6. 6版本之前，默认为0. 0. 0. 0（即监听所有的IPv4 接口）。要想指定监听的IP 地址，可以通过在启动服务器时使用--bind-address=addr选项来实现。指定的地址可以是IPv4或IPv6地址，还可以是主机名。当指定的地址是主机名时，服务器会将它解析为 IP 地址，并使用它。下面列出了绑

定不同的地址类型对服务器接受TCP/IP连接方式所产生的各种影响。有些addr值比较特殊，允许在多个网络接口上进行连接。

对于0.0.0.0，服务器会在所有IPv4接口上接受连接。

对于::，服务器会在所有IPv4和IPv6接口上接受连接。

*与::很像，不同之处在于：如果 IPv6不可用，服务器就只会在所有IPv4接口上接受连接。

对于所有的IPv4映射地址，服务器会接受IPv4和IPv6两种地址形式的连接。例如，当服务器绑定到::ffff:192.168.0.3时，允许客户端在进行连接时将--host选项的值指定为::ffff:192.168.0.3或192.168.0.3。

对于其他的IPv4 或 IPv6 地址，服务器只会接收该地址的连接。例如，当服务器绑定到192.168.0.3时，只允许对该地址进行连接。

默认端口号为3306。要想指定一个不同的端口号，可以使用--port选项。

在Unix系统里，服务器会在Unix的域套接字文件上监听本地客户端（它们或者是连接到了特殊的主机名localhost，或者是指定了一protocol=socket选项）的连接。服务器使用的套接字文件不能被关闭。默认的套接字文件通常是/tmp/mysql.sock，不过操作系统发行商提供的MySQL发行版往往会使用不一样的位置。为显式指定套接字文件的路径名，可以使用--socket选项。

对于Windows版本的MySQL服务器，命名管道连接在默认情况下是禁用的。要想启用这个功能，需要在启动服务器时带上--named-pipe 选项。这样，本地客户端便可以通过命名管道来进行连接，方法是指定--protocol=pipe 选项或者连接到特殊的主机名“.”（句点）。默认情况下，管道名称为MySQL（不区分大小写）。使用--socket选项，可以为其指定一个不同的名字。

对于Windows版本的MySQL服务器，共享内存连接在默认情况下是禁用的。要想启用这个功能，需要在启动服务器时带上--shared-memory 选项。在被启用之后，它便会成为本地客户端的默认连接协议。本地客户端也可以使用--protocol=memory 选项来显式指定使用共享内存。默认情况下，共享内存的名字为MYSQL（区分大小写）。使用--shared-memory-base-name选项，可以为其指定一个不同的名字。

在Windows里，命名管道和共享内存连接类型在默认情况下都是禁用的，启用它们的常见做法是，在选项文件的[mysqld]组里添加下面几行内容：

```
[mysqld]

named-pipe

shared-memory
```

对于想要使用TCP/IP协议（即使默认情况下使用的是其他连接协议）连接本地服务器的客户端，应该把主机名指定为 127.0.0.1。该地址是 TCP/IP 环回接口的地址。另一种强制建立TCP/IP连接的方法是，使用--protocol=tcp选项。

如果你只运行了一个服务器，那么比较常见做法是让服务器使用默认的网络设置。如果运行有多个服务器，就必须确保每一个服务器所使用的网络参数都是唯一的。更多相关信息请参考12.9节。

12.2.5 停止服务器

要想手动停止MySQL服务器，可以使用mysqladmin：

```
% mysqladmin -p -u root shutdown
```

对于Unix和Windows，这个操作都有效。在Windows里，如果以服务方式来运行服务器，那么也可以使用服务管理器提供的图形界面来选择和停止服务器；还可以像下面这样，通过命令行手动停止服务器：

```
C:\> net stop MySQL
```

如果已经将服务器设置为在系统开机时自动启动，那么在系统关机时不需要特意做什么便可以停止它。BSD系列的Unix系统在正常关闭时会发送一个TREM信号。程序会对此信号做出适当的响应（如果它们不响应，就会被无情地“杀掉”）。当 mysqld 收到这个信号时，做出的响应是结束运行。

对于使用mysql.server脚本来启动服务器的System V风格的Unix系统，关机进程会在调用该脚本时带上 stop 参数，以告知服务器关闭。你也可以自己调用该脚本，手动关闭服务器。例如，如果你将mysql.server脚本安装为/etc/init.d/mysql，则可以按下面的方法调用它（必须作为root用户来执行此操作）：

```
# /etc/init.d/mysql stop
```

如果以Windows服务方式来运行MySQL服务器，那么在Windows关机时，服务管理器会自动告知MySQL服务器停止运行。如果MySQL服务器不是以服务方式运行的，那么就要在关闭Windows之前，使用mysqladmin shutdown命令手动停止MySQL服务器。

12.2.6 当你无法连接服务器时重新获得对它的控制

在某些情况下，可能需要手动重启MySQL服务器，比如当你发现连接不上MySQL服务器时。如何实现重启便成了一个难题，因为通常情况下，要停止MySQL服务器，得先连接上它，才能再告诉它关机（如执行mysqladmin shutdown命令）。怎么会出现这种情况呢？

首先，MySQL的root密码被设置成了你不知道的某个值。当你更改密码时，可能会出现这种情况。例如，在输入新密码时，不小心键入了某个不可见的控制字符。或者你可能只是忘记了密码。

其次，在Unix里，与localhost的连接默认都是通过一个Unix域套接字文件（如/tmp/mysql.sock）实现的。如果这个套接字文件被删除，那么本地客户端将无法利用它来建立连接。在系统运行某个不定时删除/tmp目录里临时文件的cron作业时，可能会出现这种情况。

如果不能连接服务器的原因仅仅是Unix套接字文件被删除了，那么可以通过重启服务器的方式重新得到该文件。（当服务器重启时，会重建套接字文件。）这里的问题在于，你无法利用套接字文件来建立连接，告知服务器停止运行，因为套接字文件不见了。这时，只能通过建立TCP/IP连接来代替。为做到这一点，在连接至本地服务器时可以利用--protocol=tcp选项，或者把主机值指定为127.0.0.1，而非localhost：

```
% mysqladmin -p -u root --protocol=tcp shutdown
```

```
% mysqladmin -p -u root -h 127.0.0.1 shutdown
```

其中，127.0.0.1 是一个IP地址（它指向的是本地主机的环回接口），因此它会显式地强制使用TCP/IP连接，而不会使用套接字连接。

如果的确是属于cron作业将Unix套接字文件删除的情况，那么这个套接字丢失的问题，只有在你更改 cron 作业时，或是使用位于其他某处的套接字文件时才会再次发生。你可以利用全局选项文件指定一个不同的套接字文件。例如，如果MySQL的基本目录为/usr/local/mysql，那么你可以使用该目录里的套接字文件，方法是把下面几行内容添加到/etc/my.cnf中：

```
[mysqld]
```

```
socket=/usr/local/mysql/mysql.sock
```

```
[client]
```

```
socket=/usr/local/mysql/mysql.sock
```

在修改之后需要重启My SQL服务器，以便在新的位置创建套接字文件。对于服务器和客户端程序，都需要指定Unix套接字文件的路径名，以便它们能使用同一个套接字文件。如果只为服务器设置了路径名，那么客户端程序仍然会到原来的位置查找套接字。这种方法的不足之处在于，它只适用于读选项文件的客户端，可能不适用于某些第三程序。如果你通过源代码重新编译MySQL，那么可以重新配置发行包，使服务器和客户端都能默认使用不同的路径名。这样就会自动对使用客户端开发库的第三程序产生影响，除非它们与原来的库是以静态方式连接的。如果真是如此，你就必须重新编译它们，让它们使用新的开发库。

如果你是因为不记得或不知道 root 密码而不能进行连接的，那么你就得重新获得对服务器的控制，才能再次设置密码。要想做到这一点，需要执行以下几个步骤。

(1) 停止服务器。在Unix里，如果你能以root身份登录服务器主机，就可以使用kill命令来终止服务器。通过查看服务器的PID文件（该文件通常位于数据目录中），或者使用ps命令，找出服务器的进程ID。然后向服务器发送一个TERM信号，试着告诉服务器进程正常关闭：

```
# kill -TERM PID
```

采用这种方式，能正确刷新表和日志。如果服务器被挂起，无法响应正常的终止信号，可使用kill -9来强行终止它：

```
# kill -9 PID
```

使用kill -9是不得已的选择，因为内存中可能还有未刷新的修改内容；这样做还可能会导致表状态不一致。

在Linux里，ps命令可能会显示多个mysqld“进程”，而实际上它们是属于同一个进程的多个线程。因此你只要终止其中的任何一个，便可以将它们全部终止。

如果使用mysql_safe来启动My SQL服务器，它会监视服务器是否是异常终止的。因此，如果使用kill -9来终止My SQL服务器，mysql_safe会立即重

启它。为避免出现这种情况，要在终止mysqld之前先确定mysqld_safe进程的PID，并将其终止。

如果是在Windows里以服务方式来运行服务器的，那么可以在不知道密码的情况下，使用服务管理器或者调用下面的命令将其正常停止：

```
C:\> net stop MySQL
```

在Windows里，可以使用任务管理器（按组合键Alt-Ctrl-Del）强制终止My SQL服务器。与Unix系统里的kill -9一样，这也是在不得已的情况下才能使用的方法。

（2）带上-- skip-grant-tables选项重启My SQL服务器，以禁止使用权限表对连接进行验证。这样，你在进行连接时便可以不用密码，而且拥有所有权限。不过，这也会让你的服务器处于完全开放状态，以至于其他人也可以用同样的方法进行连接。因此，一旦你连接上，就要立即执行FLUSH PRIVILEGES语句：

```
% mysql
```

```
mysql> FLUSH PRIVILEGES;
```

这条FLUSH语句会让My SQL服务器重新读取权限表，并再次把它们用于访问控制。你的连接仍然会被保持，但是My SQL服务器会像往常一样使用权限表对后续其他客户端产生的连接进行验证。FLUSH语句还会重新启用SET PASSWORD语句（它在服务器未使用权限表时是被禁用的）。在重新载入权限表之后，便可以更改root的密码，如12.1节所示。例如：

```
mysql> SET PASSWORD FOR 'root'@'localhost' = PASSWORD('rootpass');
```

如果有多个root账户，请记得更改所有root账户的密码。

（3）在更改root的密码之后，需要关闭My SQL服务器，并按正常的启动过程重启它。现在，你应该可以使用新密码以root身份进行连接了。

如果需要在Unix里使用kill -9或者在Windows里使用任务管理器来强制终止My SQL服务器，那么这种突然的停止操作会使My SQL服务器无法及时将所有未保存的信息保存到磁盘。为处理因这种类型的My SQL服务器关闭而可能产生的问题，比较好的方法是启用My SQL服务器的自动恢复功能。更多相关细节请参考14.3.1节。

12.3 使用系统变量和状态变量

MySQL服务器拥有许多系统变量和状态变量。你可以通过系统变量对服务器进行配置，通过状态变量对MySQL服务器的性能进行监视。本节将大略讲述一下如何使用它们。与各个存储引擎特定相关的配置参数有关的信息请参考12.5节。与服务器优化调整有关的更多讨论请参考12.7节。对各个变量的描述请参考附录D。

系统变量控制着MySQL服务器的运作参数。使用SHOW VARIABLES语句可以显示这些变量。如果这些变量的默认值不太适合，你可以使用更适合MySQL服务器运行环境的值来更改它们，以实现对该服务器的配置。有些系统变量适用于性能调整，如那些控制内存缓冲区大小的变量。例如，你有足够的内存，便可以告诉MySQL服务器，让其为磁盘操作和索引操作提供更大一点的缓冲区。这样，可以在内存里保留更多的信息，同时减少所需的磁盘访问次数。其他的系统变量则会对服务器与客户端之间的交互产生影响。其中包括对SQL模式、默认存储引擎和当前时区进行控制的变量。

MySQL服务器还有一组状态变量，它们提供与MySQL服务器运行时的性能表现相关的信息。使用SHOW STATUS语句可以显示这些变量。状态变量的作用主要表现在：监视服务器和检查通过修改系统变量所产生的配置变化是否达到预期的效果。

12.3.1 查看和设置系统变量值

大部分系统变量都可以在MySQL服务器启动时在命令行或选项文件中进行设置，所用的语法格式与F.2节所描述的各个程序选项的语法格式相同。在运行时，可以使用SHOW VARIABLES来显示系统变量，许多变量还可以使用SET来修改。这种在运行时设置变量的能力让你可以更好地对MySQL服务器的运作进行控制；并且在某些迫不得已需要对MySQL服务器进行重新配置的环境下，它能够让你在避免停止它。例如，你可以测试一下缓冲区大小对MySQL服务器的性能有何影响，而无需在每一次更改时停止并重启MySQL服务器。在运行时所做的更改在MySQL服务器进程终止时会消失，但如果你确定了一个更适合的系统变量值，便可以将其设置到某个选项文件中，从而让该值在MySQL服务器重启之后可以继续使用。

系统变量存在于两个层面上：全局层和特定于会话层。全局变量会对整个服务器的运作产生影响。特定于会话的变量只会对服务器如何处理某个给定的客户端连接或会话产生影响。客户端可以在其自己的会话里局部地改变这些变量，使其能够在不对其他客户端产生影响的同时自定义服务器的行为，以满足自己的特殊要求。对于那些在两个层面都有值的变量，服务器会使用全局值来初始化会话变量。这种情况只会在新客户端会话开始时才会发生。在某个会话里，对全局变量的更改并不会影响到客户端相应会话变量的当前值。

一个系统变量可以同时拥有全局值和会话值、只拥有一个全局值，或者只拥有一个会话值。下面几个示例展示了这几种可能情况。

`sql_mode`变量控制着SQL模式，并在诸多方面影响着对服务器SQL语句的处理。它拥有全局值和会话值。每一个连接上的客户端都可以获得其自己的特定于会话的`sql_mode`变量，它们会初始化为与全局变量相同的值。所有客户端都可以修改会话变量`sql_mode`的值，在其自己的会话中实现对服务器行为的更改，而无其他客户端无关。拥有SUPER权限的客户端也可以更改全局变量`sql_mode`的值。新的全局值会成为在更改之后连入的那些客户端的会话默认值。

`innodb_buffer_pool_size`变量只有全局值。它控制着缓冲InnoDB表数据和索引的那个缓存的大小。这个缓存为所有的客户端所共享，因此没有必要让每个客户端都拥有一个会话值。

`error_count` 变量只有会话值。它表示的是在可能生成错误的当前会话里，最后那条语句所产生的错误数。

附录D列出了所有的系统变量，并且指明了哪些可以在启动时进行设置、哪些可以在运行时进行设置、哪些拥有全局值，以及哪些拥有会话值。下面将讨论如何获取和设置系统变量。

12.3.1.1 查看系统变量值

使用SHOW VARIABLES命令可以查看当前的系统变量值：

```
mysql> SHOW VARIABLES;
```

Variable_name	Value
auto_increment_increment	1
auto_increment_offset	1
autocommit	ON
automatic_sp_privileges	ON
back_log	50
basedir	/usr/local/mysql
big_tables	OFF

...

带上LIKE子句，可以对变量的输出行进行限制，使它们的名字都能与给定的SQL模式相匹配：

```
mysql> SHOW VARIABLES LIKE '%buffer%';
```

-----+-----	
Variable_name	Value
-----+-----	
bulk_insert_buffer_size	8388608
innodb_buffer_pool_instances	1
innodb_buffer_pool_size	134217728
innodb_change_buffering	all
innodb_log_buffer_size	8388608
join_buffer_size	131072
key_buffer_size	8388608

...

要想通过指定通用的条件来选择行，可以使用WHERE子句。下面这条语句可以把超时值小于60秒的行找出来：

```
mysql> SHOW VARIABLES
```

```
-> WHERE Variable_name LIKE '%timeout%' AND Value < 60;
```

Variable_name	Value
connect_timeout	10
innodb_lock_wait_timeout	50
innodb_rollback_on_timeout	OFF
net_read_timeout	30

默认情况下，SHOW VARIABLES 会显示各个会话变量值。要想特定地显示全局变量或者会话变量，可以在语句中加上限定符GLOBAL或者SESSION：

```
SHOW GLOBAL VARIABLES;
```

```
SHOW SESSION VARIABLES;
```

LOCAL是SESSION的同义词。

要想查询单个变量值，可以使用语法格式@@GLOBAL.var_name 来查询全局变量，使用语法格式@@SESSION.var_name 或者@@LOCAL.var_name 来查询会话变量。如果使用不带限定符的@@var_name语法，那么当限定符存在时，使用的是会话变量；否则，使用的是全局值。

限定符关键字和变量名都不区分大小写。

@@-语法具有通用性，可用在SET、SELECT或其他SQL语句里：

```
SELECT @@default_storage_engine AS 'Default storage engine';
```

你也可以查询 INFORMATION_SCHEMA 库的GLOBAL_VARIABLES 表和 SESSION_VARIABLES

表，以获得系统变量的信息。例如：

```
SELECT * FROM INFORMATION_SCHEMA.GLOBAL_VARIABLES
WHERE VARIABLE_NAME LIKE '%binlog%';
```

在命令行，执行mysqladmin variables可以显示出全局系统变量值。

12.3.1.2 在服务器启动时设置系统变量

大部分全局系统变量都可以在My SQL服务器启动时进行设置：只需将变量名当作选项名，并直接对它进行设置。例如，`max_connections` 变量可以对客户端并发连接的最大数量进行控制。要想在`mysqld`命令行把这个变量设置为200，需要使用下面的语法：

```
% mysqld --max_connections=200
```

要想在选项文件里设置这个变量，需要使用下面的语法：

```
[mysqld]
```

```
max_connections=200
```

通常情况下，在选项文件里设置系统变量比在命令行里设置更为方便，因为你不需要在服务器每次启动时都去设置它们。

在服务器启动时，系统变量名称里的短划线和下划线可以混合使用，跟选项名一样。因此，可以在命令行里输入：

```
% mysqld --max-connections=200
```

也可以在选项文件里输入：

```
[mysqld]
```

```
max_connections=200
```

对于表示缓冲区大小或长度的变量，如果指定的数字没有后缀，其值的单位就是字节；若后缀为‘K’、‘M’或‘G’（大小写不限），则分别表示千字节、兆字节或吉字节。下面这几行内容将把连接限制设置为1024，而将InnoDB缓冲池的大小设置为16 MB：

```
[mysqld]
```

```
max_connections=1K
```

```
innodb_buffer_pool_size=16M
```

有些系统变量不能使用启动选项直接进行设置。如果这样做了，My SQL服务器就会生成一条出错信息。在这种情况下，可能会有某个相关的选项可

以使用。例如，不能在启动时直接设置`time_zone`变量，但是可以使用`--default-time-zone`选项。附录D里指明了可以被直接设置的全局系统变量有哪些。对于那些不能直接设置的系统变量，该附录还列出了与之相关的可用于设置该变量的选项（如果存在的话）。

如果你在附录F里查找有关服务器选项的描述，但是未找到，那么可以再查看一下附录D。你要找的那个选项有可能实际上是一个系统变量。

12.3.1.3 在运行时设置系统变量

在运行时设置系统变量的语法取决于你是在设置全局变量还是在设置会话变量。要想设置名为`var_name`的全局变量，可以使用SET语句，具体的格式有下面两种：

```
SET GLOBAL var_name = value;
```

```
SET @@GLOBAL.var_name = value;
```

要想设置会话变量，可以使用类似的语法：

```
SET SESSION var_name = value;
```

```
SET @@SESSION.var_name = value;
```

LOCAL是SESSION的同义词。

如果没有限定符，SET语句会修改会话变量：

```
SET var_name = value;
```

```
SET @@var_name = value;
```

在设置全局变量时，你必须拥有SUPER权限。该设置会一直持续到再次发生改变时或者服务器退出时。在设置会话变量时，不需要特殊权限。该设置会一直持续到再次发生改变时或者当前会话终止时。

要想在一条SET语句里设置多个变量，可以使用逗号将各个赋值隔开：

```
SET SESSION sql_warnings = 0, GLOBAL default_storage_engine = InnoDB;
```

在一次设置多个变量的单条语句里，显式的限定符GLOBAL或SESSION会应用到多个紧随其后的那些自身未包含限定符的变量设置。下面这条语句将设置全局变量 v1和v2，并设置会话变量v3和v4：

```
SET GLOBAL v1 = val1, v2 = val2, SESSION v3 = val3, v4 = val4;
```

与在启动时设置的变量不同，在运行时你不能把名字里的下划线替换为短划线，并且在指定其值时不能使用后缀字母'K'、'M'或'G'。不过，你可以使用表达式，可以在表达式里引用其他变量的值。下面的语句将把全局变量 read_buffer_size的值设置为2 MB，并将该变量的会话值设置为全局值的两倍大：

```
SET GLOBAL read_buffer_size = 2*1024*1024;
```

```
SET SESSION read_buffer_size = 2*@@GLOBAL.read_buffer_size;
```

系统变量可被设置为特殊值DEFAULT。将DEFAULT分配给全局变量之后，会将其设置为编译好的默认值，即使在启动时已被指定了不同的值。将DEFAULT分配给会话变量之后，会将其设置为相应全局变量的当前值。

MySQL支持结构化系统变量的概念，此类变量由一组密切相关的系统变量构成，而这些系统变量是结构化变量的组成成分，也是访问单元。这种类型的变量常用于配置MyISAM的键缓存，详见12.7.2节。

12.3.2 查看状态变量值

服务器维护着许多状态变量，你可以用它们来监视其运作情况。使用SHOW STATUS语句可以显示这些变量。

```
mysql> SHOW STATUS;
```

Variable_name	Value
Aborted_clients	0
Aborted_connects	1
Binlog_cache_disk_use	0
Binlog_cache_use	3
Binlog_stmt_cache_disk_use	0
Binlog_stmt_cache_use	0
Bytes_received	125
Bytes_sent	151

...

状态变量只能由服务器来设置，因此它们对于用户来说都是只读的，并且不能像系统变量那样使用SET来进行修改。

与系统变量一样，状态变量也拥有全局值和特定于会话的值，因此相关的语句也可以使用限定符GLOBAL或SESSION：

```
SHOW GLOBAL STATUS;
```

```
SHOW SESSION STATUS;
```

GLOBAL会显示服务器作为一个整体所拥有的状态（自服务器启动以来所有会话的聚集值）。SESSION会显示当前会话的状态。默认为SESSION。LOCAL是SESSION的同义词。

如果某个变量只有全局值，那么对于GLOBAL和SESSION，你将得到相同的值。

与SHOW VARIABLES一样，将子句LIKE或WHERE与SHOW STATUS一起使用，可以对这些变量的输出进行限制，使它们的名字都能与给定的SQL模式相匹配，或者满足通用的检索条件。例如，想要检查与InnoDB的日志活动相关联的那些状态变量的值，可以使用下面这条语句：

```
mysql> SHOW GLOBAL STATUS LIKE 'innodb%log%';
```

Variable_name	Value
Innodb_log_waits	0
Innodb_log_write_requests	45504
Innodb_log_writes	1234
Innodb_os_log_fsyncs	1384
Innodb_os_log_pending_fsyncs	0
Innodb_os_log_pending_writes	0
Innodb_os_log_written	23465984

查询INFORMATION_SCHEMA库的GLOBAL_STATUS表和SESSION_STATUS表，可以获得状态变量的信息。相对于SHOW STATUS而言，这种做法的一个好处是，可以在计算里使用这些值。例如：

```
SET @queries =
```

```
(SELECT VARIABLE_VALUE FROM  
INFORMATION_SCHEMA.GLOBAL_STATUS
```

```
WHERE VARIABLE_NAME LIKE 'Queries');

SET @uptime =

(SELECT VARIABLE_VALUE FROM
INFORMATION_SCHEMA.GLOBAL_STATUS

WHERE VARIABLE_NAME LIKE 'Uptime');

SELECT @queries/@uptime/60 AS 'Queries per minute';
```

12.4 插件接口

随着时间的推移，MySQL 服务器的架构已变得更加“可插拔”，以至于实现服务器某些功能的代码段可以根据服务器来单独编译、然后存储在外部文件里，并根据需要进行加载（“插入”）。在MySQL里，插件可以实现的功能包括存储引擎、INFORMATION_SCHEMA库的各个表、全文搜索修饰符、复制机制、审计功能和身份认证方法等。

可插拔的服务器架构，让MySQL的DBA可以通过选择性加载插件的方式随意定制服务器的功能，并通过只加载所需插件的方式减轻服务器的负担。它还支持服务器功能的独立开发。例如，第三方开发人员可以把存储引擎创建一个插件，单独进行发布，而不用与服务器的开发周期绑定在一起。

本节将讲述插件接口的一般特征。与身份认证插件特定相关的信息请参考13.2.7节。

插件接口包含以下几个组成部分。

插件文件。服务器插件都是以目标文件方式存储的，这种文件也被称作插件库，因为它们可以包含多个插件。

插件目录。所有的插件文件都必须安装在这个位置。插件目录通常为MySQL安装目录下的lib/plugin，但是可以根据系统变量plugin_dir的值来确定这个位置。要想使用不同的位置，可以在服务器启动时设置plugin_dir的值。

“控制语言”。有了它之后，DBA便可以指示服务器加载哪些插件。它利用服务器选项和SQL语句来处理插件。例如，`--plugin-load`选项用于在服务器启动时加载插件，`INSTALL PLUGIN`语句用于在运行时加载插件。

插件注册。在启动时，服务器会在mysql数据库里检查plugin（插件）表，并自动加载注册的插件。使用--skip-grant-tables选项可以跳过这一操作。要想在这个表里注册插件，可以使用INSTALL PLUGIN。

插件库带有一个与平台特定相关的文件名后缀。例如，在Linux里名为mylib.so的插件文件，在Windows里则为mylib.dll。这里的讨论会把.so作为库的后缀。请你针对自己的系统选择适当的后缀来代替。

举例说明，假设我们有两个插件库对象文件，都安装在服务器的插件目录里：

一个是名为my_engine.so的库，其中包含有一个插件，实现了一个名为MY_ENGINE的存储引擎。

一个是名为info_tables.so的库，其中包含多个插件。这些插件实现了INFORMATION_SCHEMA库的LOCKS表和USERS表，分别用于显示与当前锁和连接客户端有关的信息。

通过插件接口，可以完成如下操作。

在服务器启动时，从文件里单独或成组地加载插件；

在运行时，单独加载或卸载插件；

在运行时，检查哪些插件可用。

要想在服务器启动时加载插件，可以使用选项--plugin-load或--plugin-load-add。这两个选项的值都是一个列表，由一个或多个以分号分隔开的plugin_name=lib_name 或lib_name说明符构成。当同时列出插件名和库名时，服务器只会加载该库里的那个插件。当只列出库名，但没有列出插件名时，服务器会加载该库里的所有插件。例如，想要加载两个库里的所有插件，可以把下面几行内容放到服务器的选项文件里，并重启服务器：

```
[mysqld]
```

```
plugin-load=my_engine.so;info_tables.so
```

这等效于像下面这样更加详尽地单独列出每一个插件：

```
[mysqld]
```

```
plugin-load=my_engine=my_engine.so;locks=info_tables.so;users=info_tables.so
```

要想只加载info_tables.so里的LOCKS插件，可以使用下面的内容：

```
[mysqld]
```

```
plugin-load=locks=info_tables.so
```

插件名的大小写无关紧要。如果你的文件系统区分大小写，文件名的大小写就比较重要。

--plugin-load-add 选项与--plugin-load 选项很相似。两者之间的区别在于，--plugin-load 选项的每个实例都会重置被加载插件的列表，而--plugin-load-add 选项不会。如果多次指定了--plugin-load，那么只有最后一个才会起作用。--plugin-load-add选项的实例会把插件添加到被加载插件的集合里。例如，要想全部加载这三个插件，可以像下面这样，将它们单独列出来：

```
[mysqld]
```

```
plugin-load-add=my_engine=my_engine.so
```

```
plugin-load-add=locks=info_tables.so
```

```
plugin-load-add=users=info_tables.so
```

如果你有很多插件，那么使用--plugin-load-add选项更方便，因为你可以通过多个易于阅读的行来列出它们，并且可以根据需要选择性地对它们进行注释。如果使用--plugin-load选项的话，你就必须使用一个很长的选项值，并且必须编辑它才能更改需要加载的那个插件集合。

对于使用选项--plugin-load 或--plugin-load-add 加载的插件，服务器在重启后不会记得。如果你重启服务器时没带这些选项，那么服务器就不会加载这些插件。

想要在运行时加载插件，可以使用INSTALL PLUGIN语句。此语句的每个实例都可以指定一个插件和包含它的那个库对象文件。例如：

```
INSTALL PLUGIN my_engine SONAME 'my_engine.so';
```

```
INSTALL PLUGIN locks SONAME 'info_tables.so';
```

```
INSTALL PLUGIN users SONAME 'info_tables.so';
```

INSTALL PLUGIN 语句实现的是一种“黏附”式的安装，每个插件只需要执行一次。也就是说，这条语句不仅会加载插件，还会在mysql.plugin表里注册它，这样服务器在随后的重启中就可以自动加载该插件。

可以使用UNINSTALL PLUGIN在运行时卸载插件。例如：

```
UNINSTALL PLUGIN users;
```

INSTALL PLUGIN还能从mysql.plugin表里注销插件（如果它列在其中的话），这样服务器在后续的重启中便不会再次加载它。如果某个插件被设计成不允许在运行时进行卸载，只有在服务器关闭时才能停用，那么执行UNINSTALL PLUGIN语句会导致一个错误。

由于INSTALL PLUGIN和UNINSTALL PLUGIN会修改mysql.plugin表，因此它们分别需要对该表拥有INSERT权限和DELETE权限。

不管你是在服务器启动时加载插件，还是在运行时加载，都可以使用SHOW PLUGINS语句或者INFORMATION_SCHEMA库的PLUGINS表来查看存在哪些插件。例如：

```
mysql> SHOW PLUGINS;
```

Name	Status	Type	Library	License
...				
my_engine	ACTIVE	STORAGE ENGINE	my_engine.so	GPL
...				
LOCKS	ACTIVE	INFORMATION SCHEMA	info_tables.so	GPL
USERS	ACTIVE	INFORMATION SCHEMA	info_tables.so	GPL

如果服务器在启动时加载某个插件的原因是该插件是内建的、注册在mysql.plugin表里的，或者该插件是使用选项--plugin-load 或--plugin-load-add 指定的，那么服务器可能会控制它的激活状态。为此，可以在该插件后面加上一个选项。例如，--my_engine=ON 或--my_engine=OFF可以分别激活或停用存储引擎插件my_engine。可以在插件名后面紧接着指定一个可选值，这些值具体如下。

OFF：不激活插件。等效的选项还有`--disable-plugin_name`、`--skip-plugin_name`和`--plugin_name=0`。

ON：激活插件。如果在插件名后面没有值，就默认为这个值。等效的选项还有`--enable-plugin_name`和`--plugin_name=1`。如果插件加载失败，服务器会在运行时禁用它。

FORCE：类似于ON，但如果插件加载失败，服务器不会启动。

FORCE_PLUS_PERMANENT：类似于FORCE，不过服务器还会阻止在运行时使用UNINSTALL PLUGIN卸载该插件。此选项值是在MySQL 5.5.7里引入的。

这些选项值不区分大小写，也不会将插件名与选项名进行匹配。

INFORMATION_SCHEMA库中PLUGINS表的LOAD_OPTION列指明了已加载插件的激活状态。

12.5 存储引擎配置

MySQL服务器支持多种存储引擎，并且在决定何种引擎可供使用方面高度的可配置。关于MySQL中各存储引擎的一般性讨论请参考2.6.1节。下面将讨论如何通过配置让MySQL服务器使用某些存储引擎，并且特别针对InnoDB提供了一些常规的配置信息。与InnoDB和MyISAM的性能有关的信息请参考12.7.2节。

12.5.1 选择存储引擎

在决定哪些存储引擎可供使用方面，MySQL服务器提供了非常灵活的控制。默认可用且可被禁用的存储引擎是InnoDB（它也是默认存储引擎）。默认可用的且不可禁用的存储引擎有MyISAM、MEMORY、MERGE和CSV。还有其他几种可供使用的存储引擎，你可以根据需要在启动时选择性地启用或禁用它们。例如，你可以禁用不需要的引擎，从而减少服务器的内存占用量。（不过，如果你禁用了某个存储引擎，那么就无法访问由这个存储引擎创建的表。）

对于MySQL安装包里的某个给定的存储引擎，表12-2展示了其默认状态和用于更改其状态的启动选项。更多与启动时控制存储引擎插件状态的选项有关的信息请参考12.4节。

表12-2 存储引擎的配置控制

存储引擎	可用时的默认状态	更改状态的选项
ARCHIVE	启用	--archive=OFF, 表示禁用
BLACKHOLE	启用	--blackhole=OFF, 表示禁用
CSV	启用	不能禁用
FEDERATED	禁用	--federated, 表示启用
InnoDB	启用	--innodb=OFF, 表示禁用
MyISAM	启用	不能禁用
MEMORY	启用	不能禁用
MERGE	启用	不能禁用

要想在运行时看到有哪些可用的存储引擎及其状态，可以使用SHOW ENGINES语句，或者查询INFORMATION_SCHEMA库的ENGINES表。例如：

```
mysql>SELECT ENGINE, SUPPORT FROM
INFORMATION_SCHEMA.ENGINES;
```

ENGINE	SUPPORT
CSV	YES
InnoDB	DEFAULT
MyISAM	YES
MRG_MYISAM	YES
MEMORY	YES

更多相关信息请参考2. 6. 1. 1节。

12. 5. 2 选择默认存储引擎

编译默认的存储引擎是 InnoDB，但你可以在服务器启动时或者运行时通过设置系统变量default_storage_engine来选择不同的默认引擎。例如，要

想让MyISAM成为默认的存储引擎，可以在服务器选项文件里加入下面几行内容：

```
[mysqld]
```

```
default_storage_engine = myisam
```

要想在运行时更改默认存储引擎，可以使用下面这些语句中的某一条：

```
SET GLOBAL default_storage_engine = engine_name;
```

```
SET SESSION default_storage_engine = engine_name;
```

执行第一条语句需要拥有SUPER权限，并且它会为之后建立的所有客户端连接设置默认存储引擎。执行第二条语句不需要拥有任何特殊权限，它只会影响当前客户端的会话，并且任何客户端都可以使用这条语句来更改自己的默认存储引擎。

要想查看全局和当前会话的默认存储引擎，可以使用下面这条语句：

```
SELECT @@GLOBAL.default_storage_engine,  
@@SESSION.default_storage_engine;
```

在MySQL 5.6.3之前，default_storage_engine可同时用于永久表和TEMPORARY表。从5.6.3版本起，它只能用于永久表；对于TEMPORARY表，需要用独立变量default_tmp_storage_engine来指定默认引擎。后者的典型用法是：针对TEMPORARY表使用存储引擎MyISAM或者MEMORY，以避免InnoDB引擎带来的事务开销。

如果在启动服务器时使用了--innodb=OFF来禁用InnoDB引擎，就需要将变量default_storage_engine（在MySQL 5.6里，还包括default_tmp_storage_engine）设置为InnoDB以外的某个存储引擎；否则服务器将无法启动。

12.5.3 配置InnoDB存储引擎

在MySQL 5.5里，InnoDB成为了MySQL的默认存储引擎（以前是MyISAM）。InnoDB管理着一个用于存储表内容和其数据字典的系统表空间。你也可以选择对InnoDB进行配置，让其采用每个表对应一个表空间的工作模式。InnoDB 存储引擎有其自己的日志文件和内存缓冲区。

12.5.3.1 配置InnoDB表空间

在默认的情况下，InnoDB存储引擎不使用每个表对应单独几个文件的模式，而是将所有的InnoDB表都存放在一个单独的系统表空间里。从逻辑上讲，该表空间是一个统一的存储块，InnoDB存储引擎将其当作一个巨大的数据结构。（从某种意义上讲，可以把这个表空间想象为一个虚拟的文件系统。）对于存储在系统表空间里的InnoDB表，唯一与之关联的文件便是.frm格式的文件。这个文件存放在InnoDB表所属的数据库的数据库目录里。系统表空间还包含有存储着与表结构有关的信息的InnoDB数据字典。

也可以让InnoDB存储引擎用自己的表空间文件来表示每个表。要想配置InnoDB存储引擎，使其实现每个表都拥有自己独立的表空间，可以在启动服务器时启用系统变量 `innodb_file_per_table`。在这种情况下，虽然无需太大的系统表空间，但也还是需要的，因为它包含着InnoDB的数据字典。

1. InnoDB系统表空间配置参数

虽然InnoDB系统表空间在逻辑上只是一个单独的存储区域，但它实际上却是由一个或者多个磁盘文件组成的。各个组成文件既可以是常规文件，也可以是原始分区，可以通过此处讲述的各种选项进行配置。可以在服务器的命令行里指定这些选项，但实际上很少这样做。一般都是使用选项文件中某个适合的服务器选项组（如`[mysqld]`或`[server]`）对InnoDB的表空间进行配置，以使服务器在每次启动时都能使用相同的配置。

使用下面的系统变量可以控制系统表空间里文件的数量、大小和位置。

`innodb_data_home_dir`。它用于定义构成表空间的所有组成文件的父目录。其默认值为服务器数据目录。

`innodb_data_file_path`。它用于定义位于InnoDB主目录下的那些表空间组成文件的规格。这个选项的值由多个以分号分隔的文件规格构成。每个规格都包含文件名、文件大小和某些其他可能有的选项，这些选项之间用冒号分隔。表空间各组成文件的总大小不得小于10 MB。默认是一个名为`ibdata1`、大小为10 MB的自扩展文件。

有了这些默认值，即使你没有指定上述两个系统变量，InnoDB也会在服务器的数据目录里创建一个名为`ibdata1`、大小为10 MB的自扩展文件，并把它作为表空间。

假设，你要在数据目录里创建一个表空间——它由两个大小皆为4 GB，名字分别为`innodata1`和`innodata2`的文件构成。在这种情况下，不需要设置`innodb_data_home_dir`，因为数据目录是默认的。文件配置如下：

```
[mysqld]
```

```
innodb_data_file_path=innodata1:4G;innodata2:4G
```

InnoDB存储引擎会按以下规则组合innodb_data_home_dir和innodb_data_file_path的值，以确定表空间文件的路径名：

如果未指定 innodb_data_home_dir，那么默认值就是服务器数据目录的路径名，并且InnoDB会把innodb_data_file_path里的文件名解释为相对于该数据目录的。

如果 innodb_data_home_dir 不为空，那么 InnoDB 会把它解释为 innodb_data_file_path 里的所有文件规格所在的目录，并且会把那些文件名解释为相对于innodb_data_home_dir值的。

如果显式地将 innodb_data_home_dir 设置为空，那么 InnoDB 会把 innodb_data_file_path里的所有文件规格当作绝对路径名。

基于上述规则，以下三组配置所指定的表空间文件集合是完全一样的（假设数据目录为/var/mysql/data）：

```
[mysqld]
```

```
innodb_data_file_path=ibdata1:500M;ibdata2:500M
```

```
[mysqld]
```

```
innodb_data_home_dir=/var/mysql/data
```

```
innodb_data_file_path=ibdata1:500M;ibdata2:500M
```

```
[mysqld]
```

```
innodb_data_home_dir=
```

```
innodb_data_file_path=/var/mysql/data/ibdata1:500M;/var/mysql/data/ibdata2:500M
```

innodb_data_file_path选项的值由多个以分号隔开的文件规格构成，每个规格的各个部分则由冒号隔开。最简单的文件规格语法由一个文件名和一个文件大小构成，不过还可以使用以下几种语法：

```
path:size
```


path:size:autoextend

path:size:autoextend:max:maxsize

第一种格式指定了一个大小固定为 `size` 的文件。其中，`size` 值应该是一个正整数，其后面带有M或G，分别表示以兆字节（megabyte）或吉字节（gigabyte）为单位。第二种格式指定的是一个可自动扩展的文件，即当该文件被填满时，InnoDB会自动以递增方式扩展它。第三种格式与第二种类似，只是它包含了一个用于表明该自动扩展文件允许增长到的最大大小。只有表空间的最后一个组成文件可被列为自扩展的。

默认的自扩展增量值为 8 MB。要想指定其他的增量值，需要设置系统变量 `innodb_autoextend_increment`。

2. 配置InnoDB系统表空间

在一般情况下，系统表空间完全由常规文件构成，并且不会包含任何原始分区（即设备文件）。要想对只包含常规文件的系统表空间进行初始化设置，可以执行以下步骤。

（1）把相应的设置行添加到选项文件里。

（2）确认将被创建的各个表空间组成文件所在的目录都已存在。InnoDB只会创建文件，不会创建目录。

（3）确认那些表空间组成文件都不存在。

（4）启动服务器。InnoDB将会注意到这些文件尚不存在，然后创建并初始化它们。

如果你启动了服务器，同时未显式地配置InnoDB，那么InnoDB会使用默认的配置来创建系统表空间。要想显式地配置表空间，需要先停止服务器，删除与 InnoDB 有关的文件（如表空间文件和日志文件）。接着，指定要使用的配置选项，重启服务器。（在做这件事情之前，请不要创建任何InnoDB表。如果已经创建，那么需要在重新配置之前用`mysqldump`工具把那些表转储出来；在完成配置之后再重新加载它们。）

把原始分区用作InnoDB系统表空间的组成部分要稍微复杂一些。考虑使用这种方式的原因有以下几个。

可以轻而易举地创建非常巨大的表空间。分区组成可以扩展到整个分区，而常规的文件组成会受到操作系统所允许的最大文件大小的限制。

可以保证每个原始分区在磁盘上都拥有一个完全连续的空间，而常规文件却会受到文件系统碎片的影响。为最大限度地减少常规文件的碎片，InnoDB 存储引擎在初始化表空间时，会尽量向有关文件写入足够多的零，以迫使操作系统把空间一次性全部分配给常规文件，而不是以递增方式进行分配。但是，这样只能减少碎片，并不能保证它不再出现。

原始分区通过消除文件系统管理层来减少开销。在某些系统里，这类开销可能并不明显；但是在其他系统里，这种差异却足以让人选用分区。

在 InnoDB 表空间里使用原始分区的一个不利因素是：你的系统备份软件可能只是面向文件系统使用的，而不能面向分区。此时，使用分区会使系统备份变得更加困难。

将原始分区包含在表空间里涉及两个操作：初始配置和随后的重新配置。假设，你有一个 Unix 系统，并且想在其上面使用一个路径名为 `/dev/rds8`、大小为 200 GB 的原始分区。此时，需要为 `innodb_data_home_dir` 选项指定一个值，因为这个分区并不在数据目录下。如果把 `innodb_data_home_dir` 设置为空，就可以把设备文件的完整路径放在 `innodb_data_file_path` 里，以此实现分区配置。具体步骤如下所示。

(1) 使用带有 `newraw` 后缀的大小值来初始配置原始分区。该后缀是要告诉 InnoDB，该文件是一个需要初始化的原始分区：

```
[mysqld]
```

```
innodb_data_home_dir=
```

```
innodb_data_file_path=/dev/rds8:200Gnewraw
```

(2) 启动服务器。InnoDB 会看到 `newraw` 后缀，并初始化这个分区。它还会把这个表空间当作是只读的，因为还没有进行重新配置。

(3) 在 InnoDB 初始化分区之后，停止服务器。

(4) 重新配置分区规格，把后缀 `newraw` 改为 `raw`：

```
[mysqld]
```

```
innodb_data_home_dir=
```

```
innodb_data_file_path=/dev/rds8:200Graw
```

(5) 再次启动服务器。InnoDB现在看到的后缀是raw，而非newraw。它会假定这个分区已被初始化，并且可以按读/写方式来使用此表空间。

如果把原始分区指定为InnoDB表空间的一部分，则需要确保对其设置了访问权限，以便服务器可以对它进行读/写。此外，还需要确保这个分区未被用作其他用途。否则，将会有多个竞争进程，它们每一个都认为自己拥有这个分区，并且可以随意使用它。这样导致的结果就是这些进程会破坏彼此的数据。例如，如果你错误地把交换分区指定给 InnoDB 使用，那么你的系统会变得非常不稳定！

在Windows系统上配置InnoDB系统表空间时，需要把路径名里的反斜线写成单个的斜线（“/”），或者写成两个反斜线（“\\”）。此外，尽管在文件路径名里可能会出现冒号（完整的Windows路径名是以一个驱动器名和一个冒号为开头的），你仍然应该使用冒号来分隔各文件规格的各个部分。遇到冒号时，InnoDB存储引擎会通过查看其后面那个字符来消除这种二义性。如果该字符为一个数字，那么 InnoDB 会将这个规格的下一部分当作一个文件大小；否则，会将其当作路径名的一部分。例如，下面这个配置会创建一个由C盘和D盘上的文件（其大小分别为500 MB和10 GB）构成的表空间：

```
[mysqld]
```

```
innodb_data_home_dir=
```

```
innodb_data_file_path=C:/ibdata1:500M;D:/ibdata2:10G
```

在建立初始表空间时，如果因为InnoDB不能创建某个必要的文件而导致启动失败，那么可以检查出错日志，弄清问题所在。接着，删除 InnoDB 创建的所有文件（不包括可能正在使用的任何原始分区），修正配置错误，并重启服务器。如果已拥有了原始分区，请记得更改规格：在初始化分区时，使用newraw后缀；在启动和停止服务器之后更改回raw。

3. 重新配置InnoDB系统表空间

在初始化InnoDB系统表空间，并开始使用它之后，便不能更改其组成文件的大小。不过，你可在现有的表空间组成文件列表的末尾添加一个新的文件。当表空间被填满时，这样做就很有必要。表空间快被填满的征兆之一是：当 InnoDB 事务应该成功时，它们却总是会失败，并且会回滚。要想确定剩余空间的数量，可以运行下面这条语句，并在其输出结果里检查Data_free的值，其中，tbl_name是包含在系统表空间里的任何一个InnoDB表的名字：

```
mysql> SHOW TABLE STATUS LIKE 'tbl_name';
```

要想通过另外添加组成文件来扩大InnoDB表空间，可以按以下步骤进行。

(1) 停止正在运行的服务器。

(2) 如果表空间的最后一个组成是一个自扩展文件，那么必须先将其规格更改为固定大小，然后在其后面添加另一个文件。为此，需要先确定这个文件的当前实际大小；然后，将其向下舍入近似到1 MB（字节数为1 048 576，而非1 000 000）的倍数，并将其用在文件的规格里。假设，当前有一个文件，其规格内容如下所示：

```
[mysqld]
```

```
innodb_data_file_path = ibdata1:100M:autoextend
```

如果此文件现在的实际大小为121 634 816个字节，其换算后的大小就是 $121\ 634\ 816 \div 1\ 048\ 576 = 116$ MB，使用它来更改文件规格，如下所示：

```
[mysqld]
```

```
innodb_data_file_path = ibdata1:116M
```

(3) 把这个新组成的规格添加到现有文件列表的末尾。如果这个新组成是一个常规文件，则必须保证它还不存在。如果这个组成是一个原始分区，则需要使用前面介绍过的把分区指定为表空间组成部分的步骤（即先使用newraw 后缀，接着在启动和停止服务器之后把它更改回raw），将其添加进表空间。

(4) 重启服务器。

4. 使用单独的（每个表）InnoDB表空间

要想让每个InnoDB表都使用一个表空间，需要启用系统变量innodb_file_per_table：

```
[mysqld]
```

```
innodb_file_per_table=1
```

在启用此变量之后，每个新创建的InnoDB表都将有一个.frm格式文件和一个.ibd数据文件。它们都存放在包含有这个表的那个数据库所在的目录里。如果有必要，InnoDB 会自动扩展.ibd文件。

启用或禁用使用单独的表空间只会对 InnoDB 如何创建新表产生影响。不管对变量 `innodb_file_per_table` 的值如何进行更改，它都可以访问已在系统表空间或者单个表空间里创建的表。

对于使用其自己的 `.ibd` 文件进行存储的表，InnoDB 会启用其他几项功能。

对于未被其他表以外键形式进行引用的表，`TRUNCATE TABLE` 的执行速度更快，并且可以重新申请磁盘空间；但存储在系统表空间里的表不具备这两个特点。

InnoDB 支持多种文件格式，默认格式为 `Antelope`。当启用 `innodb_file_per_table` 时，可以创建使用 `Barracuda` 格式的表。要想实现这一点，还必须设置系统变量 `innodb_file_format`：

```
[mysqld]

innodb_file_per_table=1

innodb_file_format=Barracuda
```

`Barracuda` 支持在创建新的 InnoDB 表时使用 `COMPRESSED` 或者 `DYNAMIC` 行存储格式。与这些行格式的特点相关的信息请参考 5.4 节。

通常情况下，可以在服务器启动时设置系统变量 `innodb_file_per_table` 和 `innodb_file_format`，但也可以在运行时对它们进行更改。这种情况对于那些只应用于创建特定表的设置可能很有用。例如：

```
SET GLOBAL innodb_file_per_table = 1;

SET GLOBAL innodb_file_format = 'Barracuda';

CREATE TABLE ...ENGINE=INNODB ROW_FORMAT=COMPRESSED;

SET GLOBAL innodb_file_format = 'Antelope';

SET GLOBAL innodb_file_per_table = 0;
```

12.5.3.2 InnoDB 存储引擎的变量

前一节讨论了如何配置 InnoDB 的表空间。InnoDB 也有其自己的日志文件和内存缓冲区，以及一些其他的配置参数。下面列出的这几个常用参数，可

以对 InnoDB 存储引擎的一般性操作产生影响。

`innodb_buffer_pool_size`

如果有多余的内存，可以让 InnoDB 的缓冲池变得更大一点，从而减少因访问表数据和表索引导致的磁盘操作。更多相关信息请参考 12.7.2 节。

`innodb_log_buffer_size`

InnoDB 会尽量把与每一个事务有关的信息缓冲在内存里，等到整个事务结束时才把它们一次性地写入磁盘。如果某个事务很大，超出了缓冲区的大小，那么在事务结束之前，需要有更多的磁盘活动多次刷新缓冲区。加大缓冲区的大小可以在内存里缓冲更大的事务，同时不需要提前刷新。缓冲区的默认大小为 1 MB，最大可取值为 8 MB。

`innodb_log_group_home_dir`

InnoDB 有其自己的日志文件，如果服务器启动时，它们还不存在，那么 InnoDB 会自动创建它们。日志文件的名字都是以 `ib_` 作为开头。默认情况下，InnoDB 会在数据目录里创建它们。要想显式指定 InnoDB 写入日志文件的目录路径名，可以使用 `innodb_log_group_home_dir`。（通过这种方式，把日志文件放到另一个与数据目录所在物理驱动器不同的驱动器上，可以分散磁盘的读写活动。）InnoDB 只会创建文件，而不会创建目录，因此在启动服务器之前一定要确保日志文件目录已存在。

`innodb_log_file_size/innodb_log_files_in_group`

当 InnoDB 的日志被写满时，InnoDB 会把缓冲池里的信息写入磁盘。使用较大的 InnoDB 日志文件，可以减少日志文件被填满的频率，从而减少把缓冲池里的信息写入磁盘的次数。（其代价是：较大的日志文件会增加崩溃之后的恢复时间。）修改 `innodb_log_file_size` 可以更改日志文件的大小，修改 `innodb_log_files_in_group` 可以更改日志文件的个数。日志文件的总大小（即这两个值的乘积）是一个十分重要的指标，最大不能超过 4 GB。如果已创建了日志文件，并想要更改它们的大小，则必须完全关闭服务器，以使 InnoDB 能够彻底处理好正在进行中的事务。接着，删除那些日志文件，重新配置，并重启服务器以便让 InnoDB 可以创建新的文件。

12.6 全球化问题

“全球化”（globalization）包含“国际化”（internationalization）和“本地化”（localization）。国际化指的是软件遵照本地习惯使用的能力，即软件可用于各种不同的本地环境。本地化

指的是从软件支持的本地习惯列表中选择使用某组特定的习惯。MySQL的国际化和本地化配置涉及以下几个方面。

服务器默认时区。

默认字符集和排序规则。

显示诊断和出错信息所使用的语言。

月份和日期名称的区域设置。

12.6.1 配置时区支持

MySQL服务器可以通过检查所在系统环境来设置默认时区。大部分情况下，它就是服务器主机的本地时区。不过，你也可以在启动服务器时显式指定时区。此外，服务器还允许每个客户端改写这个默认设置，设置成自己想要的时区。如此一来，应用程序便可以根据客户端程序的运行地点（而不是MySQL服务器的运行地点）来使用时间设置。下面的讨论描述了MySQL支持多个时区的能力。

时区信息保存在下面这两个系统变量里。

`system_time_zone`。它是服务器在启动时确定的在服务器主机上实际使用的时区。这个变量只能以全局系统变量形式存在，并且在运行期间不允许重新设置。在服务器启动之前，如果把环境变量TZ设置为期望的时区，那么会对服务器在启动时如何设置`system_time_zone`产生影响。不过，在某些环境里（如服务器随同系统的启动过程而启动时），难以保证TZ会被设置。在Unix系统里，另一种设置时区的方法是在`mysqld_safe`的启动脚本（并非`mysqld`，因为它不支持这个选项）里指定一个`--timezone`选项。最好的做法是把这个选项放到某个选项文件的`[mysqld_safe]`选项组里，尤其是在你通过`mysql.server`脚本间接调用`mysqld_safe`的时候，因为该脚本不支持命令行选项。例如，要想把`mysqld_safe`指定为美国中部时间，可以在服务器的选项文件里添加如下内容：

```
[mysqld_safe]

timezone=US/Central
```

这个示例展示了一种可以广泛使用的语法格式（例如，它在Linux和Mac OS X上都有效）。另一种常见的语法格式为：

```
[mysqld_safe]
```

timezone=CST6CDT

请根据系统的具体情况选择合适的语法格式。

time_zone。它是 MySQL 服务器的默认时区。默认情况下，其值为 SYSTEM，表示的是“使用system_time_zone的设置”。要想在启动时设置 time_zone，可以使用mysqld的--default-time-zone选项。服务器在运行时会使用全局的time_zone值来为每一个连接上的客户端设置会话级的 time_zone 值，使其成为客户端的默认时区。任何客户端都可以通过设置会话级time_zone变量来重新设置自己的会话级时区。拥有SUPER权限的管理用户可以设置全局的time_zone变量，以更改此后所有连接成功的客户端的默认时区。

要想确定当前的全局和会话级时区值，可以使用下面这条语句：

```
SELECT @@GLOBAL.time_zone, @@SESSION.time_zone;
```

变量time_zone可接受三种类型的值，其中一种还需要额外的管理动作。下面给出的语句会对会话值进行设置。如果你拥有SUPER权限，则可以使用GLOBAL来设置全局值。

要想使用system_time_zone的值，可以把time_zone设置为SYSTEM：

```
SET SESSION time_zone = 'SYSTEM';
```

要想指定相对于UTC（协调世界时）的偏移值，可以使用带正负号的小时和分钟格式：

```
SET SESSION time_zone = '+00:00';      # UTC
```

```
SET SESSION time_zone = '+03:00';      # UTC 前3 小时
```

```
SET SESSION time_zone = '-11:00';      # UTC 后11 小时
```

要想引用区域设置，可以使用命名时区：

```
SET SESSION time_zone = 'US/Central';
```

```
SET SESSION time_zone = 'CST6CDT';
```

```
SET SESSION time_zone = 'Asia/Jakarta';
```


要想使用最后一种方法（即通过名字来设置时区），就必须让服务器能够理解时区的名字，为此需要把操作系统时区文件里的信息加载到 `mysql` 数据库的一组表里。在MySQL的安装过程中，这个操作不会自动完成，因为MySQL只创建表，而不加载表。要想在带有时区文件的系统里手动填充那些时区表，需要确定这些文件的安装目录。然后，使用`mysql_tzinfo_to_sql`程序读取这些文件，根据其内容构造SQL语句，并把这些语句送入`mysql`程序执行。如果时区文件位于`/usr/share/zoneinfo`目录，那么把它们加载到`mysql`数据库的语句将如下所示：

```
% mysql_tzinfo_to_sql /usr/share/zoneinfo | mysql -p -u root mysql
```

在加载完这些表之后，需要重启服务器。这种方法适用于绝大多数的Unix版本。对于没有时区文件集的Windows系统或其他系统，可以通过下面这个网址获得一个压缩包，其中包含有一组预先构建的包含时区信息的MyISAM表：

```
http://dev.mysql.com/downloads/timezones.html
```

把压缩包下载回来，并解压。在MySQL服务器停止的情况下，把那些`.frm`、`.MYD`和`.MYI`文件复制到数据目录下的`mysql`数据库目录里。然后，重启服务器。

12.6.2 选择默认字符集和排序规则

字符集决定了允许出现在字符串里的字符。MySQL支持多种字符集，你可以在服务器、数据库、表、列和字符串常量等级别上选择字符集。对每种字符集，MySQL还支持多种排序规则。排序规则会对字符串的比较和排序操作产生影响。

默认的字符集和排序规则分别为`latin1`和`latin1_swedish_ci`。要想在服务器启动时更改这些值，可以设置系统变量`character_set_server`和`collation_server`。排序规则必须与字符集相兼容（即排序规则名称的开始部分必须与字符集的名称相同）。例如：

```
% mysqld --character_set_server=utf8 \
--collation_server=utf8_icelandic_ci
```

如果是根据源代码来建立服务器，那么可以在运行 `CMake` 时通过选项 `DEFAULT_CHARSET`和`DEFAULT_COLLATION`来更改这些默认值。例如：

```
% cmake -DDEFAULT_CHARSET=utf8 \
```

`-DDEFAULT_COLLATION=utf8_icelandic_ci`

在运行客户端程序的时候，可以通过`--default-character-set` 选项来指明它所使用的字符集。如果在客户端程序默认的位置未找到字符集文件，可以用`--character-sets-dir`选项来告知客户端程序具体的位置。

更多与MySQL里字符集功能有关的信息请参考2.4节。关于创建字符型列和如何使用它们的详细信息请参考第3章。

12.6.3 选择出错信息的显示语言

MySQL服务器可以用多种语言显示诊断信息与出错信息。默认语言为english（英语）。要想知道有哪些语言可以选用，可以查看MySQL安装目录下的share目录。其中，各子目录的名字正好与各种可用的语言相对应。

要想指定出错信息的显示语言，可以把系统变量`lc_messages`配置为适当的区域设置，如`en_US`对应于英语，`de_DE`对应于德语。服务器会将区域设置名称转换为语言名称，并在变量`lc_messages_dir`指出的目录中查找该语言的子目录，从而找到包含出错信息文件的目录。如果默认目录不正确（默认为MySQL安装目录的share目录），需要将`lc_messages_dir`设置为正确的路径。例如，想要把出错信息显示为德语，并指明出错信息位于`/var/mysql/share`目录下，可以像下面这样启动服务器：

```
% mysqld --lc_messages=de_DE --lc_messages_dir=/var/mysql/share
```

也可以在服务器启动时设置这两个系统变量。单个客户端可以在运行时设置`lc_messages`的会话值，以选择另一种语言。当同一服务器的不同客户端需要以不同于默认值的某种语言显示本地化出错信息时，这种做法就很实用。

12.6.4 选择区域设置

MySQL服务器依靠系统变量`lc_time_names`来决定如何显示月份和日期的名称。该变量的值会对许多时态函数（如`DAYNAME()`、`MONTHNAME()`和`DATE_FORMAT()`）的结果产生影响。

默认的区域设置为`en_US`（英语）。要想选择一个不同的区域设置，可以在服务器启动或运行时设置`lc_time_names`的值。各个客户端可以通过在运行时设置`lc_time_names`的会话值来改写默认值。例如，在启动服务器时将`lc_time_names`设置为`es_ES`（西班牙语），会得到下面这样的结果：

```
mysql> SELECT MONTHNAME('2000-07-01');
```

```
-----  
MONTHNAME('2000-07-01')  
-----
```

```
      julio  
-----
```

如果某个客户端想要设置为意大利语，可以像下面这样做：

```
mysql> SET lc_time_names='it_IT';
```

```
mysql> SELECT MONTHNAME('2000-07-01');
```

```
-----  
MONTHNAME('2000-07-01')  
-----
```

```
      luglio  
-----
```

12.7 服务器调整

对 MySQL 服务器以及运行它的那台机器拥有控制权限的管理员，可以进行一些优化或调整操作。例如，可以调整与查询处理有关的服务器参数，某些硬件配置因素对查询处理速度有直接的影响。这些优化有助于提高服务器的整体性能，从而为所有的MySQL用户带来好处。

当进行管理优化时，请牢记以下几项原则。

访问内存数据的速度比访问磁盘数据的速度快。

将数据保存在内存里，尽可能减少磁盘活动。

保留索引的信息比保留行的内容重要。

可以更改的最重要的参数是各个缓存的大小，如用于缓冲索引操作相关信息的表缓存和存储引擎缓存。如果你还有可用的内存空间，可以将它分配给服务器的缓存，让信息能够更长时间地保留在内存里，从而减少磁盘活动。这种做法非常好，因为访问内存信息的速度比读取磁盘信息的速度快。

接下来的几节将讨论几种用于优化服务器操作的方法。

有些参数会影响总体性能，与之相关的信息请参考12.7.1节。

各个存储引擎都有它们自己的参数，你可以利用这些参数来对它们进行配置。相关信息请参考12.7.2节。

MySQL 服务器会将查询结果缓存起来，当处理重复的查询时，可以不用再次执行它们。相关信息请参考12.7.3节。

有几个硬件因素会对MySQL的行为产生影响。相关信息请参考12.7.4节。

与系统变量设置有关的一般性说明请参考12.3.1节。当更改参数值时，请遵循下面这些指导。

一次只更改一个参数。如果同时修改多个变量，则很难对每个更改所产生的影响进行评估。

逐步增大系统变量值。如果大幅度调整某个变量的值，虽然在理论上是越大越好，但是你可能会耗光系统资源，从而导致系统因为你将变量值设置得过高而变得异常快或异常慢。

请不要在MySQL的生产服务器上进行参数调整的试验，比较稳妥的做法是建立一个单独的测试服务器。

让服务器操作变得更有效的其他策略还包括有以下几种。

禁用不需要的存储引擎。服务器不会为禁用的存储引擎分配内存，你可以将它用于其他用途。如果通过源代码来构建 MySQL，那么大部分的存储引擎在配置时便可以不用包含在服务器的二进制程序里。许多包含在服务器里的存储引擎都可以在运行时通过相应的启动选项被禁用。相关细节请参考12.5.1节。

保持简单的访问权限。虽然服务器会将权限表的内容缓存在内存中，但是如果你在mysql数据库的tables_priv、columns_priv或procs_priv表里放置行，那么服务器在检查SQL 语句的权限时，一定会检查它们的内容。如果这些表为空，那么服务器会跳过这些权限级别，从而优化权限检查。相关信息请参考13.4.2节。

12.7.1 用于服务器调整的通用型系统变量

MySQL 服务器有很多可修改的系统变量（如那些控制内存缓冲区大小的变量），使你能够对其操作进行影响。如果你有足够的内存，便可以让服务器针对磁盘操作和索引操作使用更大一点的缓冲区。这样可以在内存里存放更多的信息，从而减少所需的磁盘访问次数。如果你的系统能力有限，则可

以告知服务器使用更小一点的缓冲区。这样可能会使服务器运行得更慢，但可以提高系统的整体性能，防止MySQL服务器占用系统资源，从而损害到其他进程。

下面列出的这几个系统变量能满足常见的性能调整需要。

`back_log`

在处理当前连接时，排队等待连接的最大请求数。如果你的网站非常繁忙，就需要在客户端连接变得很慢时，增大`back_log`的值。

`max_connections`

服务器支持的最大客户端并发连接数。如果服务器很忙，则可能需要增大这个值。例如，有一个活跃的Web服务器，它用你的MySQL服务器处理大量由DBI或PHP脚本生成的语句。当此变量的值设置过低时，网站的访问者可能会发现他们的请求被拒绝。（要想确定已同时打开的最大连接数，可以使用`SHOW STATUS`检查状态变量`Max_used_connections`。）

`table_open_cache`

当服务器打开表文件时，会试图将它们保持在打开状态，以减少必须要完成的文件打开操作和文件关闭操作的数量。为做到这一点，它会在一个表缓存里存放与所有打开文件有关的信息。如果服务器访问了大量表，那么这个表缓存可能会被填满，服务器会关闭那些在最近一段时间未使用的表，从而为那些新打开的表腾出空间。

表缓存的默认大小为 400。对于一个繁忙的服务器来说，这个值可能太小。如果使用了诸如`innodb_file_per_table`那样的功能，或者使用了会增加所需文件描述符数量的分区表，那么这个值也可能太小。如果还有可用的文件描述符，那么增大系统变量 `table_open_cache` 的值，让表缓存变得更大一点，就能让mysqld同时开着更多的表。

要想评估表缓存的有效性，可以检查`Opened_tables`状态指示器：

```
SHOW STATUS LIKE 'Opened_tables';
```

`Opened_tables` 表明了表被打开过的次数，因为它已不在打开状态。（在`mysqladmin status` 命令的输出里，这个值也会显示为`Opens` 值。）如果这个数字保持稳定或者增加得很缓慢，那么这个设置值可能就恰到好处。如果这个数字迅速增大，则意味着缓存太小，为了给其他表腾出空间，必须频繁地关闭一些表。尤其是在表的内容已被更改，并到了必须将这些更改保

存到磁盘时，关闭该表就要付出特别大的代价。大一点的表缓存可以有效地缓解这种性能拖累。

`table_definition_cache`

如果增大`table_open_cache`的值，还需要考虑增大`table_definition_cache`的值。这个系统变量控制着存储表定义（源自`.frm`文件的信息，在打开表时会用到它们）的缓存的大小。

`open_files_limit`

如果增大`max_connections`或`table_open_cache`的值，服务器会需要更多的文件描述符。这样可能会导致许多问题，因为操作系统对每个进程所能使用的文件描述符数量进行了限制。在这种情况下，你需要增大这个限制，或者想办法解决它。首先，尝试为`mysqld`设置`open_files_limit`变量。如果采用这种方式无法把打开文件的数量限制设置得足够高，可能就需要对操作系统进行配置，以允许使用更多的文件描述符。有些系统的配置可以简单地通过编辑系统描述文件并重启的方式进行。其他一些系统的配置则必须通过编辑内核描述文件并以重建内核的方式进行。请查询系统文档，以了解如何处理。

突破单个进程文件描述符限制的解决办法是：为MySQL主服务器建立一个或多个从服务器（参考14.8节）。所有的更新都应该被导向主服务器，但是来自客户端的单纯的检索请求可以分散到所有的服务器。这样可以减少主服务器对客户端的负载和文件描述符的要求。

`max_allowed_packet`

用于客户端通信的那个缓冲区的最大大小可以增大。默认缓冲区大小为1 MB。允许的最大值为1 GB。这个变量并不是一个与性能密切相关的参数，但是把这个值调大，可以防止在处理更大的数据包时发生错误。数据包缓冲区的内存只在需要时才分配。

客户端`mysql`和`mysqldump`都拥有自己的`max_allowed_packet`变量。如果客户端向服务器发送了很长的语句（如包含有大块BLOB或TEXT值的语句），那么可能需要同时在服务器端和客户端增加这个变量的值。例如，在启动服务器时想要将包的大小限制为16 MB，可以把下面这些行添加到服务器的选项文件里：

`[mysqld]`

`max_allowed_packet=16M`

如果只是偶尔需要以16 MB的数据包限制来调用mysql或mysqldump，可以这么做：

```
% mysql --max_allowed_packet=16M ...其他选项...
```

```
% mysqldump --max_allowed_packet=16M ...其他选项...
```

要想一直使用这些设置，可以把下面这些行添加到选项文件里：

```
[mysql]
```

```
max_allowed_packet=16M
```

```
[mysqldump]
```

```
max_allowed_packet=16M
```

有些变量控制的资源是按每个客户端来分配的。如果同时有大量的客户端连接服务器，那么加大这些变量的全局值，可能会导致服务器的资源占用总量急剧上升。为改善性能，可以把变量`read_buffer_size`和`sort_buffer_size`的值加大。它们分别决定了在读取和排序操作时使用的缓冲区大小。不过，要小心：每一个会话都会被分配这些缓冲区，因此如果把这两个变量的值设置得过大，系统性能可能会因资源消耗过大而下降。

请不要把每个会话缓冲区的大小一次增加得太大，最好是能逐步加大，并对更改进行测试。只有这样，才能对每一次更改的效果进行评估，减少性能严重下降的可能性。此外，一定要记得使用真实的测试条件来验证。服务器不是在客户端刚一连接上时就分配这些缓冲区的，而是只有在需要时才分配。例如，对于某个客户端，只有在它执行带有排序操作的查询时，排序缓冲区才会被分配。`join_buffer_size` 变量控制着两个表进行连接时所用的那个缓冲区的大小。如果客户端没有执行连接操作，就不需要连接缓冲区。（反之，如果客户端使用了复杂的涉及多个表的连接操作，那么它便会同时需要多个连接缓冲区。）因此，测试条件应该使用多个同时进行连接并运行复杂语句的客户端，以便可以对服务器内存需求的真实效果进行评估。

对于那些控制着每个客户端分配到的缓冲区大小的变量，不用增加其全局值的做法是：保持这些值不变，并假设各个客户端会根据需要把这些变量的会话值加大。与全局系统变量和会话系统变量之间的差别有关的信息请参考12.3.1节。

12.7.2 存储引擎调整

有些存储引擎带有调整参数，你可以利用它们来获得最佳的服务器负载效果。本节将讨论存储引擎InnoDB和MyISAM中最为重要的与性能相关的资源，即每个引擎在内存中用于缓冲表信息的缓存。与存储引擎的一般配置有关的信息请参考12.5节。

各个存储引擎表信息缓存拥有不同的名字（在InnoDB中叫做缓冲池；在MyISAM中叫做键缓冲），但是每个操作的一般原则都是相同的。

开始时，缓存为空。

当服务器在语句执行期间必须检查表的信息时，它会检查该信息是否已经读入缓存。如果已读入，则服务器会查询内存中的值。否则，存储引擎表会把内容从磁盘读入缓存。

如果缓存在必须读入新值时已满，则必须丢弃旧的信息，以便为新信息腾出空间。默认情况下，每个存储引擎会基于最近最少使用（LRU）算法来确定要丢弃的缓存块。也就是说，存储引擎会选择那些未被使用时间最长的缓存块。这样可以保留频繁使用的缓存块，而丢掉那些最近最不可能再次需要的缓存块。

如果选择丢弃的缓存块未被修改过，那么它的内容会被新读入的值覆盖。否则，在这个缓存块被覆盖之前，其内容必须被存回到原来的那个表。

如果在缓存中找不到请求的值，即值“缺失”，那么必须从磁盘读入。如果能找到，即为“命中”。缓存的目的在于减少对磁盘的访问（即尽量减少请求缺失的比率）。它能大大提高性能，因为访问内存的速度比访问磁盘的速度要快得多。缓存还可以同时减少读和写带来的磁盘 I/O 操作。

因为在缓存中找到的信息不需要再从磁盘里读取，所以对磁盘的读取操作会更少。

因为对缓存信息的修改不需要立即写入磁盘，所以对磁盘的写入操作会更少。将多次的更改缓冲起来，成组地写入磁盘，比单个的写入操作更高效。

虽然小型的缓存也可以带来这些好处，但是较大的缓存会带来更明显的效果。包含经常被用到的索引值的缓存块通常会长时间地停留在缓存里，较大的缓存可以提高命中机率。反过来，它会减少为了给新缓存块腾出空间而丢弃旧缓存块的需要，并且会最小化索引处理所需要的磁盘访问次数。如果服务器管理了许多特定于某种存储引擎的表，并且这种引擎的缓存当前还很小，那么增加它的大小通常是可采取的最为有用的配置更改措施中的一种。

不过，设置缓存大小也需要综合考虑。不要饿着服务器主机上运行的其他需要内存的进程，并且 MySQL 自身也有其他的内存需要，如连接（connection）缓冲、排序和连接（join）缓冲区、临时表，以及查询缓存。同样地，在增加存储引擎的缓存大小时也要小心，不要让它变得太大，以致用光机器上所有可用的内存。这样会导致缓存本身分页，从而失去使用缓存将信息保留在内存里的意义。

接下来的几节将提供与特定引擎（包括 InnoDB和MyISAM）相关的表信息缓存配置的细节。

12.7.2.1 配置InnoDB的缓冲池

InnoDB把它的缓冲池看作一个块列表，该列表被分为新、旧两个子列表，分别包含最近被访问较多和较少的缓存块。InnoDB使用的是中点插入策略，即当把新的缓存块添加到缓冲池时，会将其放置在旧子列表前面（那里正好是新子列表的尾部）。

当查询需要缓存块，或者缓存块的预读取试探器预测到不久便需要用到它时，InnoDB便会将其插入缓冲池。插入缓存块不同于访问它。“访问”只在查询对缓存块的内容进行实际检查时才会发生。

默认情况下，InnoDB会把缓存块插到旧子列表的前面，接着会在插入之后第一次访问这个缓存块时，将其移动到新子列表的前面。如果缓存块是因为查询需要而被插入的，那么第一次访问便会立即发生；但如果缓存块是因为预读取而被插入的，访问可能就不会立即发生。

当访问新子列表里的缓存块时，仅当它还不是太靠前时，才会被移动到这个列表的前面。随着时间的流逝，如果某个缓存块没有任何访问发生，那么它会被逐渐移到列表最后。新子列表里的缓存块会随着时间的推移，越过新旧子列表的分界线。旧子列表里的缓存块会逐步老化到列表的最后，并被丢弃。

下面两个系统变量可以控制InnoDB缓冲池的大小和控制它是否可被分为更小的缓冲池。

`innodb_buffer_pool_size`

分配给缓冲池的总内存，单位为字节。默认大小为128 MB。对于需要大量处理InnoDB表并拥有更多可用内存的系统，可以把这个值增大。一般的指导原则是使用系统内存的70%~80%，此外还需要考虑其他MySQL资源和服务器主机上其他进程所产生的内存需求。

`innodb_buffer_pool_instances`

默认情况下，这个变量值为 1，并且只有一个单独的缓冲池。如果 `innodb_buffer_pool_size` 至少为 1 GB，并且你将 `innodb_buffer_pool_instances` 设置成大于 1，那么 InnoDB 会把这个缓冲池处理成许多个小的缓冲池实例，其中的每一个都独立运作。InnoDB 会把从磁盘读取到的缓存块随机分配给某个给定的缓冲池实例，这会减少竞争，因为在每个实例里试图访问其共享数据结构的会话会更少。（例如，即使一个会话对某个缓冲池拥有独占锁，使用其他缓冲池的工作也仍然可以继续进行。）要想达到最好的效果，需要选择好 `innodb_buffer_pool_size` 和 `innodb_buffer_pool_instances` 的值，以便每个实例的大小都至少为 1 GB。使用多个缓冲池的情况通常仅限于大型系统，如拥有大量内存的 64 位系统。

另外有两个系统变量可以对 InnoDB 缓冲池（如果配置了多个实例，就是对每个缓冲池实例）的 LRU 算法产生影响。

`innodb_old_blocks_pct`

缓冲池的旧子列表所占的百分比。这个百分比是一个近似值，因为 InnoDB 在管理某些操作时，仅仅是通过来回移动指针的方式来表明新旧子列表之间的边界（例如，当某个缓存块从新子列表退化至旧子列表时）。默认值为 37（即缓冲池的 3/8）。允许的取值范围为 5~95。更小的值或更大的值会分别导致新子列表里的缓存块更慢或更快地老化至旧子列表。

`innodb_old_blocks_time`

一个缓存块在第一次访问之后、下次访问之前，需要在旧子列表里待多少毫秒才能移动到新子列表里。默认值为 0，表示的是：插到旧子列表里的缓存块，在第一次访问发生时立即移到新子列表里去。

表扫描往往会快速连续地多次访问表块，此后便不再访问。为防止出现因这种一次性的访问而导致的为填充新子列表而丢弃其他缓存块的现象，可以把 `innodb_old_blocks_time` 的值设置为大于0。当执行备份时（它会涉及大量的一次性扫描），这是个值得推荐使用的设置。另一方面，要想把你希望频繁访问的表的内容预先加载到缓冲池，可以把 `innodb_old_blocks_time` 的值设置为0，并扫描该表以将其加载到缓存里，然后将这个变量值恢复成先前的那个值。

12.7.2.2 配置MyISAM的键缓冲区

MyISAM在存储表时使用了单独的数据文件和索引文件，对处理它们的方式也有所不同。

为缓存从数据文件读取到的数据行，或者是写到数据文件的数据行，MyISAM 依托操作系统，使用了它的文件系统缓存机制。

为处理索引文件，MyISAM 维护了一个键缓冲区，用它来执行基于索引的检索和排序操作，以及索引的创建和修改操作。

对MyISAM存储引擎操作影响最为重要的参数是其键缓冲区的大小。默认大小为8 MB，是个非常保守的值。如果你使用了大量的MyISAM表，并且有多余的内存可以使用，那么把键缓冲区设置得更大一些，应该能在处理索引相关操作时大幅提高MyISAM的性能。

配置缓冲区的方法是，对系统变量 `key_buffer_size` 进行设置。例如，要想把大小设置为512 MB，可以将下面几行内容放置到某个选项文件里：

```
[mysqld]
```

```
key_buffer_size=512M
```

MyISAM键缓冲区在所有MyISAM表之间默认是共享的。如果在缓存中找不到键值，且缓存已满，那么就会产生竞争，此时，必须丢弃当前缓存中的值，下次需要它们时，再从磁盘读取。

如果某个MyISAM表被使用的频率非常高，那么确保其键保留在内存里能带来很多好处，但是在这个缓存里仍然会存在竞争。无论这些键是从同一个表里读取，还是从其他一些表里读取，竞争都会出现。你或许可以通过把键缓存增大到足以完全容纳给定表全部索引的方式，来避免同表竞争问题，但是来自其他表的键仍然会争抢缓存里的空间。

针对此问题，MyISAM提供了一种解决办法。因为它支持多个键缓存，所以可以通过下面这几项功能来对缓存操作进行更多的控制。

可以使用默认的单个键缓存，也可以创建多个缓存。

可以控制总的缓存大小、缓存块大小和缓冲区丢弃算法。

可以将表分配给某些特定的缓存，并把表索引预加载到其中的某个缓存。

如果某个表被使用的频率非常高，你又有足够的内存将其索引加载到缓存里，那么这些功能就会有用处。同表竞争和它表竞争都可以避免，方法是：创建一个足够大的能完全容纳下一个表的索引的缓存，并将此缓存设置为仅供这个表使用。在所有键都加载到缓存之后，便不再需要任何磁盘 I/O 操作。此外，也不需要从缓存中丢弃那些键值，针对该表的键查找也都可以内存里完成。

另一个常见的策略是，将一组被频繁使用的表分配给一个单独的键缓存，以使它们的索引缓存不会与其他表的产生竞争（它是通过默认的缓存进行处理的）。

要想允许对MyISAM键缓存进行配置，服务器需要将每一个缓存都与一组系统变量关联一起。由于这些变量是相关的，因此可以将它们分为多个组成部分，共同组成一个结构化的系统变量。这种结构化变量是简单系统变量的扩展，因此可以使用“缓存名+变量名”的语法形式对它们进行访问：

`cache_name.var_name`

每一个键缓存结构化变量都有下面几个组成部分。

`key_buffer_size`

键缓存的总大小，单位为字节。

`key_cache_block_size`

键缓存的块大小，单位为字节。默认情况下，缓存块的大小为1024字节。

`key_cache_limit`

这个变量会对缓存缓冲区的重用算法产生影响。如果设置为它的默认值100，那么键缓存会使用最近最少使用策略来决定重用哪些缓存缓冲区。如果

设置值小于 100，那么键缓存会使用中点插入策略将缓存分为暖子链和热子链。key_cache_limit的值为暖缓冲区子链占键缓存的百分比，其取值范围为1~100。

有了使用暖子链和热子链的中点插入策略之后，服务器会尽力把最常被访问的缓冲块保留在热子链里。缓冲块会随着被访问次数的增加或减少，在热子链或暖子链之间移动。服务器总是会从暖子链里选择要重用和覆盖的缓冲块。（MyISAM 键缓冲区的热子链和暖子链与InnoDB缓冲池的新旧子列表相似。）

key_cache_age_threshold

它指的是，未使用的缓冲块在被移到暖子链之前，可以在键缓存的热子链里保留多久。值越高，缓冲块被允许停留在热子链里的时间越长。其默认值为300，最小值为100。

有一个默认键缓存，其名字为 default。当应用于默认缓存时，对键缓存组成变量的引用可以不用带上缓存名。因此，key_buffer_size和default.key_buffer_size引用的是同一个变量。键缓存名必须是合法的标识符，并且不区分大小写。它们可以像所有其他标识符一样被引用（参考2.2节）。

要想创建新的键缓存，可以把值赋给与之关联的那个缓存变量的任何组成部分。例如，要想在服务器启动时创建一个名为my_cache、大小为24 MB的缓存，可以将下面这几行添加到服务器的选项文件里：

```
[mysqld]

my_cache.key_buffer_size=24M
```

要想在运行时创建键缓存，可以使用下面这条语句：

```
SET GLOBAL my_cache.key_buffer_size = 24*1024*1024;
```

关键字 GLOBAL 必不可少，因为键缓存是全局性的。访问各组成部分的值时不需要特殊权限，但要想设置它们，就必须拥有SUPER权限。

要想在MyISAM表创建好之后将其分配给某个键缓存，可以使用CACHE INDEX。这条语句会指定一个键缓存以及一个或多个表，将它们分配给该表。下面这条语句将把表 member和president分配给名为my_cache的缓存：

```
CACHE INDEX member, president IN my_cache;
```

之后，你可以将其他表分配给同一个缓存。也可以使用LOAD INDEX INTO CACHE 将表索引预先加载到分配给它们的那个缓存里：

```
LOAD INDEX INTO CACHE member, president;
```

预加载索引不是必需的，但如果这样做了，服务器就会顺序读取各个索引块。与在需要时才等待获取它们的方式相比，这种做法更有效率。

CACHE INDEX和LOAD INDEX INTO CACHE语句要求你对指定表拥有INDEX权限。

要想销毁键缓存，可以将其大小设置为零。所有分配给这个缓存的表都会被重新分配给默认缓存。如果将默认键缓存的大小设置为零，那么就要使用文件系统的缓存来处理分配给它的表索引，这种处理方式跟处理MyISAM数据文件的方式是一样的。这样可以最小化服务器的内存占用，但同时也会降低性能。

键缓存分配只会持续到服务器退出为止。要想实现在服务器每次启动时都能进行分配，可以将适合的CACHE INDEX和LOAD INDEX INTO CACHE语句放置在某个文件里，并在启动服务器时，将系统变量init_file的名字设置为该文件的名字。

12.7.3 使用查询缓存

MySQL服务器可以使用查询缓存来加快处理那些需要重复执行的SELECT语句。最终的性能往往会得到很大的提升。查询缓存有这样几个特点。

某条给定的SELECT 语句在第一次执行时，服务器会记住这条查询语句的文本和它返回的结果。

服务器下次看到这条语句时，就不必再执行了。它可以直接从查询缓存里提取结果，并将其返回给客户端。

查询缓存是以服务器接收到的查询字符串文本为基础的。如果这些查询的文本完全相同，那么就认为它们是相同的。如果这些查询的大小写不同，或者来自使用了不同字符集或通信协议的客户端，那么就认为它们是不同的。就算它们在其他方面都相同，但实际并没有引用相同的表（例如，它们引用了不同数据库中具有相同名字的表）的话，它们也是不同的。

如果某个查询返回的结果不确定，它就不会被缓存。例如，使用NOW()的查询会随着时间的推移返回不同的结果，因此它不会被缓存。

当表被修改时，指向它的所有缓存查询都将变为无效，并且会被丢弃。这样可以防止服务器返回过时的结果。

要想确定服务器是否支持查询缓存，可以检查系统变量 `have_query_cache`：

```
mysql> SHOW VARIABLES LIKE 'have_query_cache';
```

Variable_name	Value
have_query_cache	YES

对于支持查询缓存的服务器，缓存操作依赖于下面三个系统变量的值。

`query_cache_type`。它决定了查询缓存的操作模式。下表展示了所有可能的模式值。

模式	含义
0	不缓存查询结果，不检索缓存结果
1	将可缓存的查询缓存起来，那些以 <code>SELECT SQL_NO_CACHE</code> 为开头的查询除外
2	根据需要只将那些以 <code>SELECT SQL_CACHE</code> 为开头的可缓存查询缓存起来

`query_cache_size`。它决定了分配给缓存的内存数量，单位为字节。

`query_cache_limit`。它设置的是被缓存的结果集大小的最大值。大于这个值的查询结果不会被缓存。

例如，要想启用查询缓存，并为其分配 16 MB 的内存，可以在选项文件里使用以下设置：

```
[mysqld]  
  
query_cache_type=1
```

```
query_cache_size=16M
```

即使`query_cache_type`为零，也会为查询分配`query_cache_size`所表示的那么多数量的内存。为了避免浪费内存，请将这个大小设置为零，除非你计划要启用这种缓存。请注意，在`query_cache_size`的大小设置为零时，即使`query_cache_type`为非零值，查询缓存也会被禁用。

启用查询缓存听起来像是一种提高性能的简单方法，通常情况下也的确如此。不过，在某些情形下，它起不到什么作用，甚至还会影响到性能。

为查询缓存分配过大的内存，会导致服务器在比较当前查询与缓存查询确定是否缓存结果方面花费过多的时间。

在多台处理器系统里，MySQL 服务器可以在不同的处理器核上执行对不同连接的查询。但是，由于对查询缓存的检查只涉及单个线程，因此它会成为运行于不同内核上的两个连接线程之间的竞争点。检查此问题的一种方法是使用`SHOW PROCESSLIST`。如果你经常在`State`列里看到“Waiting for query cache lock”（正在等待查询缓存锁）这样的信息，就表明查询缓存访问存在竞争迹象。此时，最好是能禁用它。

假设已启用了查询缓存，则客户端在开始缓存查询时的状态是由服务器的默认缓存模式所表明的。通过下面这条语句，客户端可以更改其默认的查询缓存模式：

```
SET query_cache_type = val;
```

其中，`val`可以为0、1或2，其含义与在服务器启动时设置`query_cache_type`变量时是相同的。在`SET`语句里，符号值`OFF`、`ON`和`DEMAND`分别对应着0、1和2。

通过在关键字`SELECT`后面添加修饰符的方式，客户端也可以对单个查询的缓存进行控制。如果缓存模式为`ON`或者`DEMAND`，那么在可缓存查询里使用`SELECT SQL_CACHE`语句会导致结果被缓存。使用`SELECT SQL_NO_CACHE`语句会导致结果不被缓存。

抑制缓存可能会为那些从某个不断变化的表里检索信息的查询带来好处。此时，缓存不太可能被大量使用。假设，你正在记录对MySQL里某个表的Web服务器请求，并且会定期对该表运行一组汇总查询。一个非常繁忙的Web服务器，会频繁地向表中插入新行，因此表缓存的任何查询结果都会迅速失效。这意味着，尽管你可能会反复地执行这些汇总查询，但是这个查询缓存对它们来说很可能毫无价值。在这种情况下，使用修饰符`SQL_NO_CACHE`来执行这些查询，明确告知服务器不要缓存它们的结果，会更具意义。

12.7.4 硬件优化

本章前几节讨论了几项在忽略硬件配置的情形下，帮助你提高服务器性能的技术。当然，如果你有更好的硬件，就可以让服务器运行得更快。但是，并非所有与硬件相关的更改都能带来同样的好处。当对可以采取何种硬件改进措施进行评估时，最重要的原则与服务器参数调整所遵循的原则是一致的：尽可能多地把信息放置在快速存储里，并尽可能让它能够在那里待得更长。

下面描述几个硬件配置，修改它们可以提高服务器的性能。

(1) 在计算机上安装更多的内存。这样可以让你配置更大的服务器缓存大小和缓冲区大小，使数据在内存里保留得更久，并减少从磁盘获取信息的需要。

(2) 使用多处理器硬件。对于像MySQL服务器这样的多线程应用程序，多处理器硬件可以同时执行多个线程。

(3) 重新配置系统，移除所有的磁盘交换设备。如果你有足够的RAM，可以在内存文件系统里执行所有的交换操作，就可以移除所有的磁盘交换设备。否则，即使你有足够的RAM 用于交换，有些系统也还会继续在磁盘里进行交换。

(4) 添加速度更快的磁盘，以提高 I/O 延迟。对于活动磁头旋转型磁盘驱动器，寻道时间通常是主要的性能决定因素。横向移动磁头通常都比较缓慢。相比之下，在磁头定位之后，从磁道里读出数据块的速度则会快一些。不过，如果需要在添加更多的内存和获得更快的磁盘之间进行选择，请考虑添加更多的内存。因为内存的速度总是比磁盘的更快，并且添加内存可以让你使用更大的缓存，进而减少磁盘活动。

要想完全避免寻道，可以安装一个固态驱动器（SSD），并配置MySQL来使用它。SSD完全没有寻道时间，因为没有任何部件需要移动。

(5) 通过将磁盘活动分布在不同的物理设备上，充分利用并行特性。如果你可以将读取或写入拆分到多个物理设备上，那么它会比从同一设备里读取和写入的速度更快。例如，如果将数据库存储在一个设备上，而将日志存储在另一个设备上，那么同时往两个设备里写入会比将数据库和日志都写到同一个设备的速度更快。请注意，在同一物理设备上使用不同的分区并不能算作并行操作。之所以说其起不到什么作用，是因为它们仍然会去竞争同一个物理资源（即磁头）。与日志和数据库移动过程有关的描述请参考11.3节。

在把数据迁移到另一个设备之前，需要确保你对系统的负载特征已有所了解。如果某个特定的物理设备上已有其他一些主要活动存在，那么将数据库放置在那里可能会让性能变得更糟。例如，如果你需要处理大量的Web流量，并且还想将数据库移动到Web服务器文档树所在的那个设备上，那么你将可能无法感受到这样做所能带来的任何整体好处。

使用RAID设备也可以让你获得一点并行性的好处。

12.8 服务器日志

MySQL服务器可以生成多种类型的日志。这些日志有助于诊断问题、提高服务器性能、启用复制和恢复崩溃。当服务器开始执行时，它会检查启动选项，看看是否需要记录日志。如果需要记录，则会开启相应的日志功能。下面列出的是对每一种日志的简要描述，接下来的几节将对每一种日志进行更为详细的讲解。

出错日志。这种日志记录的内容包括服务器的启动和关闭，以及与问题或异常条件有关的消息。如果服务器无法启动，就可以查看此类日志。它会在终止之前把消息写到出错日志，以指明出现了什么问题。

普通查询日志。这种日志记录的内容包括客户端连接、从客户端接收到的SQL语句和其他杂项事件。监视服务器的活动很有用处，从中可以了解到：“连接方是谁”、“它来自哪里”，以及“它们在做什么”。当你想要查明客户端在往服务器发送什么样的语句时，使用日志是最方便的，它能为故障诊断或调试提供帮助。

慢查询日志。这种日志可以帮你把那些可能需要被重写、以求获得更好性能的语句识别出来。默认情况下，如果某个查询花费的时间超过了10秒，它便会被认为很慢，并且服务器会将它写到慢查询日志里。有几个系统变量可以进一步控制慢查询日志所记录的内容。

二进制日志及二进制索引文件。这种日志由一个或多个文件构成，记录由UPDATE、DELETE、INSERT、CREATE TABLE、DROP TABLE、GRANT等语句所做的修改。写到二进制日志里的内容都是一些以二进制格式编码的数据修改“事件”。二进制日志文件都伴随有一个索引文件，其中列出了当时存在的那些二进制日志文件。

二进制日志有以下两个用途。

在服务器崩溃之后，它可与备份一起配合完成表的恢复：首先，根据备份文件恢复数据库；然后，使用mysqlbinlog将二进制日志的内容转换为文本

语句，并使用备份之后修改数据库的那些语句作为客户端mysql的输入，把数据库恢复到崩溃时的状态。

它形成了复制的基础。存储在二进制日志里的数据修改事件可被传输到复制从服务器。

中继日志及中继日志索引文件。如果服务器是一个复制从服务器，它就会维护一个中继日志，其中包含的是从主服务器接收到的需要被执行的数据修改事件记录。中继日志文件的格式与二进制日志文件的格式相同，它有一个索引文件，其中列出的是从服务器上存在的日志文件。

在所有日志当中，普通查询日志对于服务器监视是最有用的，因此当你第一次开始使用MySQL时，建议你除了启用其他想要的日志以外，还要记得启用普通查询日志。在对MySQL积累了一定经验之后，你便可以关掉普通查询日志，以减少对服务器磁盘空间的占用。

默认情况下，每个被启用的日志都会以文件形式（可能是一系列的文件）写到数据目录里。对于某些类型的日志，你可以选择把它们写到其他地方。

出错日志可以被发送到syslog。

普通查询日志和慢查询日志可以被写到mysql数据库的表里。

如果你不明确要求，服务器不会创建任何日志，但是，下面两种情况例外。

在Unix里，如果使用mysqld_safe 脚本来启动服务器，那么该脚本会创建出错日志，并告知服务器使用它。

在Windows 里，除非通过指定--console 选项指明需要将诊断信息发送至控制台窗口，而不是发送至文件，否则服务器都会创建出错日志。

要想启用服务器日志记录，可以使用下表显示的那几个选项。如果日志文件名可选（以方括号表示），而你没有提供名字，那么服务器会使用默认文件名把日志文件写到数据目录里。服务器会根据服务器主机的名字（它由后面会讨论到的HOSTNAME表示）生成每一个日志文件的默认名字。如果以相对路径名形式指定日志文件名，那么服务器会将它解释为一个相对于数据目录的名字。要想将日志文件放到其他目录里，可以指定一个完整的路径名。如果日志文件不存在，服务器会自动创建它。但是，服务器不会自动创建存放该文件的那个目录。如有必要，请在启动服务器之前把该目录创建好；否则启动会失败。

选项	用途
<code>--general_log</code>	启用普通日志
<code>--general_log_file=file_name</code>	生成日志文件名
<code>--log-bin[=file_name]</code>	启用二进制日志
<code>--log-bin-index=file_name</code>	二进制日志索引文件
<code>--log_error[=file_name]</code>	启用出错日志
<code>--log_output[=destination]</code>	普通查询/慢查询日志存放位置
<code>--relay-log[=file_name]</code>	启用中继日志
<code>--relay-log-index=file_name</code>	中继日志索引文件
<code>--slow_query_log</code>	启用慢查询日志
<code>--slow_query_log_file=file_name</code>	慢查询日志文件名

其中有些选项实际上是系统变量（标志就是其名字里有下划线），但是你可以在服务器启动时设置它们。也可以在服务器运行时对这些变量进行更改。

可以在`mysqld`或`mysqld_safe`的命令行里指定服务器日志的选项。不过，因为通常在每次启动服务器时你都要以同样的方式指定日志选项，因此最常见的做法是将它们列在选项文件里某个适合的选项组。通常情况下，都是把选项文件放在`[mysqld]`选项组中，但不需要总是放在那里。12.2.3节对适用于服务器和服务器启动程序的各个选项组进行了详细的说明。例如，下面几行内容将启用一个名为`log.err`的出错日志和一个名为`qlog`的普通查询日志，它们都存放在服务器的数据目录里：

```
[mysqld]

log_error = log.err

general_log

general_log_file = qlog
```

除了二进制日志和中继日志之外，其他日志都是以文本格式写入的，可以直接查看。要想显示二进制日志或中继日志文件的内容，可以使用工具程序mysqlbinlog。

除了刚才描述的这些服务器日志，可能还有存储引擎单独创建的其他日志。例如，InnoDB引擎会创建日志，用于崩溃后的自动恢复。你无法控制InnoDB 是否生成日志，但是你可以通过系统变量innodb_log_group_home_dir指定其写入日志的位置。默认位置是数据目录。更多相关信息请参考12.5.3节。

12.8.1 出错日志

服务器使用出错日志来记录它的启动和关闭，记录诊断信息和出错信息。记录的信息量可以通过设置系统变量log_warnings来控制，其取值范围是0~2，表示的是记录的递增级别。

在Unix和Windows里，对某些出错日志特性的处理会有所不同。接下来对它们进行讨论。

12.8.1.1 Unix里的出错日志

在Unix里，mysqld 默认不会创建出错日志，不过它会把诊断输出发送到控制台。如果你直接调用mysqld，那么可以通过设置系统变量log_error将出错输出发送到某个文件，而不是发送到控制台。可以在命令行或某个选项文件的[mysqld]组里设置这个变量。

如果你启动服务器的方式是调用mysql_safe，那么服务器会默认创建出错日志，因为mysql_safe在调用mysqld时，会把服务器的输出重定向到出错日志。默认的出错日志文件名为 HOSTNAME.err。要想使用另外的出错日志名，可以在命令行或者某个选项文件的[mysqld_safe]或[mysqld]组里设置系统变量log_error。（mysqld_safe脚本会查看[mysqld]选项组，使用那里的log_error值，如果能找到的话。）

如果出错日志文件已存在，但是对于用来运行服务器的那个登录账户不可写入，那么服务器启动会失败，并且不会有任何输出写到这个出错日志。如果在不同的时间使用不同的--user值来启动服务器，就会发生这种情况。因此，请始终使用同一个账户，就像12.2.1.1节中讨论的那样。

如果你使用mysql_safe来启动服务器，那么就可以将出错输出发送到syslog，而不发送到日志文件。要想将诊断信息也发送到syslog，可以在启动mysql_safe时带上--syslog选项，而不使用系统变量log_error。当把日志记录到syslog之后，来自mysqld和mysql_safe的消息在被写入时会分别带上

mysqld或mysql_safe字样的标签（以前缀形式）。如果指定了选项--syslog-tag=str，那么这两个标签将分别变成mysqld-str和mysql_safe-str。

如果你使用mysql.server 脚本来启动服务器，那么出错日志将总是会被创建，因为mysql.server 会调用mysql_safe。不过，mysql.server 无法识别出命令行里或其[mysql_server]选项组里的出错日志选项。要想指定mysqld_safe或mysqld所需要的选项，可以使用选项文件组[mysqld_safe]或[mysqld]，具体做法请参考本节前面的描述。

12.8.1.2 Windows里的出错日志

在Windows里，服务器会默认将诊断信息写到数据目录的HOSTNAME.err文件中。要想使用另外的出错日志名，可以在命令行或者某个选项文件的[mysqld]组里设置系统变量log_error。如果你在手动启动服务器时使用了--console 选项，那么它会把诊断输出写到控制台窗口，而不会写到出错日志。（如果服务器是以Windows服务运行的，--console 选项就不会有任何效果，因为这时根本没有控制台可供写入。）

12.8.2 普通查询日志

普通查询日志记录的内容有：客户端何时连接的服务器、客户端发送给服务器的每一条SQL语句以及其他事件（如服务器的启动和关闭）。服务器会把它收到的语句按先后顺序写到这个日志。这个顺序与各条语句执行完毕的顺序可能会有所不同，尤其是在简单语句和复杂语句混杂在一起的时候更是如此。

要想启用普通查询日志，需要启用系统变量general_log。要想指定日志文件名，可以设置系统变量general_log_file。数据目录里默认日志文件的名字为HOSTNAME.log。这些变量可以在服务器启动时设置，运行时更改。

你可以将普通查询日志写入文件、数据库表，或者同时写到这两个地方。更多详细信息请参考12.8.6节。

12.8.3 慢查询日志

慢查询日志记录的是那些需要花费很长时间（默认为10秒）才能执行完毕的查询。因为查询所花费的时间只有等到它执行完毕时才能知道，因此服务器会在它们执行完毕后才将其写到慢查询日志，而不是在刚接收到它们时就写入。慢查询还会使得服务器递增其 Slow-queries状态变量。

服务器写入的慢查询日志是文本格式的，因此可以用任何文件查看程序来查看它，也可以使用工具程序mysqldumpslow来对其内容进行汇总。

慢查询日志有助于把那些在重写时可以进行改进的查询标识出来。不过，在解释其内容时，还应该把系统的正常负载考虑在内。“慢”是以真实的时间（而不是 CPU 时间）来度量的，因此如果你的服务器陷入停顿状态，那么有的查询会被判定为慢，即使它在其他时间运行时不算慢。

要想启用慢查询日志，需要启用系统变量 `slow_query_log`。要想指定日志文件名，可以设置系统变量 `slow_query_log_file`。数据目录里默认日志文件的名为 `HOSTNAME-slow.log`。这些变量可以在服务器启动时设置，运行时更改。

你可以将慢查询日志写入文件、数据库表，或者同时写到这两个地方。更多详细信息请参考12.8.6节。

有几个相关选项和系统变量会影响服务器写到慢查询日志的内容。

查询“慢”的定义是其执行时间超过了系统变量 `long_query_time` 的值，该值的单位为秒（默认值为10秒）。最小值为0。该值可以有小数部分，单位为微秒。如果慢查询被写入日志表，那么查询执行时间的小数部分会被丢弃。

各个查询至少要检查 `min_examined_row_limit` 个行，否则它们不会被记录。默认值为0。

如果服务器启动时带有 `--log-short-format` 选项，那么它会以简短形式把信息写入日志。

如果服务器启动时带有 `--log-slow-admin-statements` 选项，那么它会把执行慢的管理语句（如 `ANALYZE TABLE` 或 `ALTER TABLE`）记录下来。

如果启用了系统变量 `log_queries_not_using_indexes`，服务器还会把那些没有使用索引来执行行查找的查询记录下来。不过，这可能会导致大量的日志输出。自 MySQL 5.6.5 起，可以把系统变量 `log_throttle_queries_not_using_indexes` 设置为非零值，以此表明每分钟可记录到的未使用索引的查询语句的数量上限。

12.8.4 二进制日志

MySQL 服务器使用二进制日志来记录各种数据修改“事件”（如 `INSERT`、`DELETE` 或 `UPDATE` 语句所触发的）。它不会把 `SELECT` 操作记录到这个日志里。像下面这样的 `UPDATE` 语句也不会出现在二进制日志里，因为它实际不会更改任何值：

```
UPDATE t SET i = i;
```

二进制日志的用途包括数据备份和恢复，并且支持服务器之间的复制。（要想把某个服务器设置为主服务器，使其可以被复制到从服务器，必须启用二进制日志。）因为二进制日志可用于服务器之间的复制操作，它包含了某些能为复制操作带来好处的信息，如语句的执行时间戳。

MySQL必须先执行语句才能确定它是否真的修改了数据，因此它会在语句执行完毕之后才把有关的信息写入二进制日志，而不是在刚接收到它们的时候就写入。

与其他类型的日志有所不同，服务器往二进制日志里写入信息的格式不是文本，而是一种更有效率的二进制格式。这意味着这种类型的日志无法直接阅读。要想以可读的文本格式显示二进制日志文件的内容，可以使用工具程序mysqlbinlog。

一个能令复制正确进行的重要特性是：MySQL服务器会按各个事件执行完毕的顺序把它们写入二进制日志，而不是以接收到它们的顺序写入。对于那些包含在事务里的语句，服务器会将它们缓存起来，一直到该事务被成功提交。然后，它才会把在此期间发生的所有事件写入日志。如果事务被回滚，则这个事务不会被写到二进制日志，因为它没有让数据库产生任何变化。

实际上，更为确切的说法是，回滚事务通常都不会被写入二进制日志。如果某个事务对非事务型表（如MyISAM表）进行了修改，那么这些修改将不能被回滚。在这种情况下，即使回滚事务也会被记录到二进制日志，目的是要确保对非事务型表的更改也可以被正确地复制。

启用二进制日志的方法是指定--log-bin选项。如果在指定此选项时没有带上文件名，服务器会以 HOSTNAME-bin 作为基本名，按编号顺序生成二进制日志文件，如 HOSTNAME-bin.000001、HOSTNAME-bin.000002，依此类推。否则，服务器会使用你指定的名字作为文件顺序基本名（如果名字里包含扩展名，扩展名会被忽略掉）。此后，当你每次启动服务器、刷新日志，或者当前日志文件的大小已达到其最大值时，服务器便会按编号顺序生成下一个文件。这个大小由系统变量max_binlog_size的值来确定。

如果启用了二进制日志功能，那么服务器还会创建一个附带的二进制日志索引文件，用来列出现有二进制日志文件的名称。默认的索引基本名与二进制日志文件的基本名一样，扩展名为.index。使用--log-bin-index 选项，可以显式地指定一个名称。如果指定的名称里不包含扩展名，服务器会自动为其加上一个扩展名.index。例如，如果你指定的内容是--log-bin-index=binlog，那么索引文件名就会变成binlog.index。

如果你同时使用选项--log-short-format和--log-bin，那么MySQL写入二进制日志的信息会比较简短。

服务器在把事件写入二进制日志时，既可以使用基于语句的格式，也可以使用基于行的格式。例如，当使用基于语句的记录格式时，一个UPDATE操作便只会被写成一条UPDATE语句；而当使用基于行的记录格式时，同样的UPDATE操作却会根据被更新的各个行所产生的变化来进行记录（参考14.8.3节）。系统变量`binlog_format`可用于指定所要使用的记录类型。此变量的可取值包括STATEMENT、ROW和MIXED（它主要是使用基于行的记录格式，除非在某些情况里必须使用基于语句的格式）。默认值为STATEMENT。

如果将二进制日志用于服务器间的复制，那么在删除某个给定的二进制日志文件之前，要先确认其包含的内容已被复制到所有可用的从服务器，并且的确不需要它了。有关如何进行确认的信息请参考12.8.7.2节。

二进制日志文件和系统备份

如果二进制日志（以及中继日志）因磁盘故障而出现丢失的情况，那么它们对数据库的恢复和复制就起不到任何作用。因此，一定要对文件系统进行定期备份。把这些日志写到另一个与数据库存放位置不同的磁盘上，也是一种不错的做法，只是需要你把它们从服务器默认使用的那个数据目录重定位到另一个目录。为做到这一点，需要使用合适的日志选项来指定一个新的日志存放位置。

12.8.5 中继日志

中继日志需要与复制机制中的从服务器一起使用（参考14.8节）。复制从服务器负责接收来自主服务器的各种数据修改信息（即“事件”），并在接收到它们时将其写到中继日志。从服务器会将中继日志当作一个保存区，其中存储的事件会被保留到它们被从服务器执行为止。

在从服务器上，有两个线程分别负责处理事件的读取和执行。其中，I/O线程负责从主服务器读取事件，并把它们写到中继日志；SQL线程负责读取中继日志文件，执行其中包含的事件，并删除每个处理完毕的文件。这种功能上的不相干让这两个线程可以各自独立地运行。

中继日志与二进制日志有几个共同的特点。

服务器会按编号顺序创建中继日志文件。

有一个索引文件，其中列出的是当前存在的所有中继日志文件集。

中继日志文件的格式与二进制日志文件相同，因此可以使用工具程序`mysqlbinlog`来显示它们的内容。

启用中继日志的方法是指定`--relay-log`选项。如果在指定此选项时没有带上文件名，服务器会以 `HOSTNAME-relay-bin` 作为基本名，按编号顺序生成中继日志文件，如 `HOSTNAME-relay-bin.000001`、`HOSTNAME-relay-bin.000002`，依此类推。否则，服务器会使用你指定的名字作为顺序基本名（如果名字里包含扩展名，扩展名会被忽略掉）。此后，当你每次启动服务器、刷新日志，或者当前日志文件的大小已达到其最大值时，服务器便会按编号顺序生成下一个文件。这个大小由系统变量`max_relay_log_size`的值来确定。

如果启用了中继日志功能，那么服务器还会创建一个附带的中继日志索引文件，用来列出现有中继日志文件的名字。默认的索引文件名与中继日志文件的基本名一样，扩展名为`.index`。使用`--relay-log-index`选项，可以显式地指定一个名字。如果指定的名字里不包含扩展名，服务器会自动为其加上一个扩展名`.index`。例如，如果你指定的内容是`--relay-log-index=relay-log`，那么索引文件名就会变成`relay-log.index`。

12.8.6 使用日志表

当启用了普通查询日志或慢查询日志功能时，你可以选择让服务器把日志输出写入日志文件、写入mysql数据库里的日志表，或者同时写到这两个地方。

要想选择日志的输出目标，可以在启动服务器时设置系统变量`log_output`。该变量的值是一个以逗号分隔的名字列表，取值包括`FILE`（记录到文件）、`TABLE`（记录到表）或`NONE`（不进行记录）。`NONE`（如果存在）会改写所有其他目标值。如果未指定`log_output`，则默认为`FILE`。

系统变量 `log_output` 只是确定了使用哪一个输出目标，但不会启用日志功能。要想启用普通查询日志或慢查询日志，需要启用系统变量`general_log`或`slow_query_log`。

要想在运行时更改日志记录目标，可以设置全局系统变量`log_output`。例如，要想临时禁用日志功能，可以使用下面这条语句：

```
SET GLOBAL log_output='NONE';
```

要想重新启用日志功能，并将文件和表同时作为日志的输出目标，可以像下面这样做：

```
SET GLOBAL log_output='FILE,TABLE';
```

如果启用了日志，服务器会把启动消息写入日志文件，但是不会把查询也写入日志文件，除非选择了FILE作为日志输出目标。

如果把TABLE作为输出目标，服务器会把日志输出写入mysql数据库的general_log和slow_log表。

如果把FILE作为输出目标，日志文件名就由全局系统变量general_log_file和slow_query_log_file 来决定。文件位于数据目录里，默认名分别为 HOSTNAME.log和HOSTNAME-slow.log。如果启用的记录类型为输出到文件，那么更改这两个变量中任何一个的值，便可以更改服务器相应日志记录文件的名字。

日志表的内容只允许查看，不允许修改，除非服务器自己进行更改。因此，你只能对它们使用SELECT语句，而不能使用INSERT、DELETE或UPDATE语句。（不过，可以使用TRUNCATE TABLE语句来清空日志表。）

12.8.7 日志管理

日志记录很重要，但是潜在的问题是会产生大量的信息，甚至有可能会填满你的整个磁盘。如果你的服务器很忙，要处理大量的语句，那么此问题将尤为突出。要想保持最近的几个日志可以在线使用，同时还想防止日志文件无限制地增长，可以使用日志文件过期技术。用于保持日志处于可管理状态的方法有如下几种。

日志轮换法。这种方法适用于文件名固定的日志文件，如出错日志文件、普通查询日志文件和慢查询日志文件。

基于使用期限的过期法。这种方法会把超过某个使用期限的日志文件删除掉。它适用于按编号顺序创建的日志文件，如二进制日志。不过，如果把二进制日志用于服务器复制，就不应该使用这种技术了。

与复制有关的过期法。如果把二进制日志文件用于服务器复制，不能基于使用期限让它们过期。只有在知道它们都已被完全发送至所有从服务器之后，才可以让其过期。因此，这种形式的过期方法是基于哪些二进制日志文件仍在使用的决定的。

复制从服务器会按编号顺序来创建中继日志文件，并且会在处理完它们之后自动将其删除。

为减少存储在磁盘上的中继日志信息量，可以通过减小系统变量max_relay_log_size的值来降低日志文件的最大可允许大小。

日志表截短或轮换法。如果把日志信息记录到mysql数据库的表，就可以把它们截短或者重新命名，并将其替换为空表。

接下来的几节将对如何使用这几种过期方法进行讲解。在sampdb发行版的admin目录里可以找到此处讨论的日志过期脚本示例。

不管你实际选用的是哪一种日志过期技术，都应该把日志文件如何与数据库备份方法融为一体纳入考虑范围。把所有数据恢复操作可能会需要的日志文件都备份起来是种不错的做法，因此在备份它们之前不应该让其过期。

刷新日志

日志刷新经常被用作日志过期或轮换操作的一部分，以确保任何缓冲的日志信息都已被写入磁盘。刷新日志会导致服务器关闭并重新打开日志文件。为此，需要执行 `mysqladmin flush-logs` 命令，或者执行 `FLUSH LOGS` 语句。在Unix里，向服务器发送 `SIGHUP` 信号也可以刷新日志。`mysqladmin refresh` 命令除了会刷新日志之外，还会做其他事情，如刷新表缓存，因此，如果你只是想刷新日志，此命令完成的操作就会有些多余。

服务器会按编号顺序创建二进制日志文件和中继日志文件。因此，刷新日志会导致服务器关闭当前的日志文件，并打开一个新的带有下一个编号的日志文件。

日志刷新不会对存储引擎特有的日志产生影响。

12.8.7.1 轮换固定名字的日志文件

MySQL服务器会将多种类型的日志信息写入固定名字的文件，如出错日志文件、普通查询日志文件和慢查询日志文件。要想让固定名字的日志文件过期，可以使用日志轮换方法。这样可以保证最近的几个日志始终处于在线状态，但要把文件的数量限制在你所选定的范围之内，以防止它们“撑爆”你的磁盘。

现在来看看日志文件轮换的工作原理。假设有一个名为 `qlog` 的普通查询日志文件。在第一次轮换时，将 `qlog` 更名为 `log.1`，并告知服务器把日志写入那个新的 `qlog` 文件。（在服务器打开当前日志文件期间将其更名，然后刷新日志。这个操作会使服务器关闭该文件，并打开一个新文件，从而创建一个使用其原来名字的新日志文件。）在第二次轮换时，把 `qlog.1` 更名为 `qlog.2`、`qlog` 更名为 `qlog.1`，并告知服务器把日志写入另一个新的 `qlog` 文件。按照这种方式，每个文件的名字都会经历 `qlog.1`、`qlog.2` 等这样的轮换。当文件到达某个轮换点时，便可以采用让其前面的文件覆盖它的方式来使其过期。

例如，按每天一次的频率轮换日志，并想要保留一周的日志，那么将保留的文件有qlog.1到qlog.7。在每次轮换时，让qlog.6覆盖qlog.7便可以让qlog.7过期，而qlog.6会变成新的qlog.7。

日志轮换的频率和需要保留的旧日志文件的数量，取决于服务器的繁忙程度（活跃的服务器会产生更多的日志信息）以及你愿意分配给旧日志的磁盘空间量。

下面这个shell脚本rotate_fixed_logs.sh可以用于在Unix里完成对固定名字的日志文件的轮换操作：

```
#!/bin/sh

# rotate_fixed_logs.sh - 轮换固定名字的MySQL 日志文件

# 参数 1: 日志文件名

if [ $# -ne 1 ]; then

echo "Usage: $0 logname" 1>&2

exit 1

fi

logfile=$1

mv $logname.6 $logname.7

mv $logname.5 $logname.6

mv $logname.4 $logname.5

mv $logname.3 $logname.4

mv $logname.2 $logname.3

mv $logname.1 $logname.2

mv $logname $logname.1
```

```
mysqladmin flush-logs
```

此脚本把日志文件名作为其参数。你既可以指定文件的完整路径名，也可以将当前目录切换至日志目录，并指定该目录中的文件名。例如，要想轮换/usr/mysql/data目录中名为qlog的日志文件，可以像下面这样执行此脚本：

```
% rotate_fixed_logs.sh /usr/mysql/data/qlog
```

或者执行下面的命令：

```
% cd /usr/mysql/data
```

```
% rotate_fixed_logs.sh qlog
```

说明

在前几次日志轮换脚本执行时，你还没有整组的轮换日志文件，此时脚本会抱怨说它找不到所有要轮换的文件。这是正常的。

为保证你有重命名日志文件的权限，可以用你运行服务器的那个账户（在本书中为mysql账户）来登录，运行此脚本。请注意，该脚本中的mysqladmin命令没有包括连接参数，如-u或-p。如果调用mysqladmin所使用的相关参数都存储在mysql账户主目录的.my.cnf选项文件里，就不必在这个脚本程序里的mysqladmin命令处指定它们。如果没有使用选项文件，那么mysqladmin命令必须知道如何使用MySQL账户（它拥有足够的刷新日志的权限）连接服务器。为此，可以设置一个权限有限的MySQL账户，让其只能执行刷新命令（也就是说，这个账户只拥有RELOAD权限）。例如，要想将这个用户叫做flush，并为其分配一个flushpass密码，可以使用下列语句：

```
CREATE USER 'flush'@'localhost' IDENTIFIED BY 'flushpass';
```

```
GRANT RELOAD ON *.* TO 'flush'@'localhost';
```

在建立此账户之后，可以像下面这样调整rotate-fixed-logs.sh脚本里的mysqladmin命令：

```
mysqladmin -u flush -pflushpass flush-logs
```

然后通过将其设定为只能由mysql进行访问，以保护这个脚本不会被其他登录账户读取。在以mysql身份登录之后执行下面这条命令：

```
% chmod go-rwx rotate_fixed_logs.sh
```

要想弄清如何使用`rotate_fixed_logs.sh` 脚本来自动轮换和刷新日志，可以参考12.8.7.3节。

在Linux 里，你可能更喜欢使用工具程序 `logrotate` 来安装 `mysql-log-rotate` 脚本（MySQL发行版里自带的），而不用`rotate_fixed_logs.sh` 或编写自己的脚本。可以在RPM发行版的`/usr/share/mysql`里，或MySQL安装或源代码树的`support-files`目录里，查找`mysql-log-rotate`。

在Windows里，可以使用下面这个批处理脚本`rotate_fixed_logs.bat`来完成日志文件的更名操作。

```
@echo off

REM rotate_fixed_logs.bat - 轮换固定名字的MySQL 日志文件

REM 参数 1: 日志文件名

if not "%1" == "" goto ROTATE

@echo Usage: rotate_fixed_logs logname

goto DONE

:ROTATE

set logname=%1

erase %logname%.7

rename %logname%.6 %logname%.7

rename %logname%.5 %logname%.6

rename %logname%.4 %logname%.5

rename %logname%.3 %logname%.4

rename %logname%.2 %logname%.3
```

```
rename %logname%.1 %logname%.2
```

```
rename %logname% %logname%.1
```

```
mysqladmin flush-logs
```

```
:DONE
```

调用rotate_fixed_logs.bat的方式与调用shell脚本rotate_fixed_logs.sh的方式很像，都要带上一个参数来指定需要轮换的那个日志文件。例如，要想轮换C:\mysql\data里名为qlog的日志文件，可以像下面这样执行此脚本：

```
C:\> rotate_fixed_logs C:\mysql\data\qlog
```

或者执行下面的命令：

```
C:\> cd \mysql\data
```

```
C:\> rotate_fixed_logs qlog
```

12.8.7.2 二进制日志文件和中继日志文件的过期处理

固定名字的日志文件可以通过文件名轮换的方法进行过期处理，这是前面刚讨论过的。对于二进制日志和中继日志这样的日志，服务器会按编号顺序来生成日志文件，因此对它们的过期处理会有所不同。

对于二进制日志，有以下两种方法可供选择。

基于日志文件的使用期限（按最后一次修改的时间来评估）来决定其是否过期。这种方法适用于未将二进制日志用于服务器复制的情况。

基于日志文件是否仍在用来决定其是否过期。这种方法适用于将二进制日志用于服务器复制的情况。

要想基于使用期限来让日志文件过期，最简单的办法就是设置系统变量expire_logs_days。例如，要想让那些一周以来未被更改的二进制日志文件过期，可以把下面几行内容添加到选项文件里：

```
[mysqld]
```

```
expire_logs_days=7
```


当`expire_logs_days`的`n`值大于零时，服务器会自动让那些超过`n`天的二进制日志文件过期，并更新二进制日志索引文件。在启动和打开新的日志文件时（如在刷新日志时，或者是在当前文件到达系统变量`max_binlog_size`所指示的大小时），服务器会检查日志文件是否过期。

如果把二进制日志用于服务器复制，就不能使用基于使用期限的过期处理方法，因为使用期限并不是一个复制过的日志文件是否可以被删除的指示器。假设，从服务器处于关闭状态，并且有一个给定的二进制日志文件的内容还没有发送给它。如果从服务器在这个文件到达其过期使用期限时还没有恢复过来，该文件就会被丢弃，复制操作也会因此而失败。为避免此问题，不能让二进制日志文件在其内容被复制到所有从服务器之前过期。

这里有一个难点，由于MySQL复制的异步特性，主服务器自己并不知道有多少从服务器，或者有哪些文件已被传送到各个从服务器。主服务器不会把那些还未发送给已连接的从服务器的二进制日志文件清除掉，但是不保证给定的服务器一定会在某个特定的时间连接上。这意味着你必须知道有哪些服务器属于从服务器，并确定出每一个从服务器正在处理哪些主服务器上的二进制日志文件。为此，需要连接每一个从服务器，执行 `SHOW SLAVE STATUS` 语句，并检查`Master_Log_File`列。所有不再被任何从服务器使用的二进制日志文件都可以被删除。

为弄清其中的工作原理，现假设有下面这样的场景。

本地服务器为主服务器，它有两个从服务器，分别为S1和S2。

在主服务器上，其二进制日志文件的名称分别为`binlog.000038`到`binlog.000042`。

在S1上执行`SHOW SLAVE STATUS`会产生下列结果：

```
mysql> SHOW SLAVE STATUS\G
```

```
...
```

```
Master_Log_File: binlog.000041
```

```
...
```

而在S2上得到的结果是：

```
mysql> SHOW SLAVE STATUS\G
```

...

Master_Log_File: binlog.000040

...

在这种情况下，从服务器仍然需要的编号最小的那个二进制日志为 binlog.000040，因此可以删除所有小于这个编号的日志。为此，可以连接主服务器，并执行下面这条语句：

```
mysql> PURGE MASTER LOGS TO 'binlog.000040';
```

这会导致主服务器删除所有比指定文件编号小的二进制日志文件，在上述情况中，将删除binlog.000038和binlog.000039。

执行语句SHOW SLAVE STATUS和PURGE MASTER LOGS需要拥有SUPER权限。

要想让中继日志文件过期，不需要采取任何特殊措施。复制从服务器会按编号顺序创建中继日志文件。在当前的中继日志文件达到其最大可允许大小时（或者当日志被刷新时），复制从服务器会自动创建一个新的中继日志文件，并且会在处理完旧的文件后自动将其删除。不过，如果中继日志的最大可允许大小很大的话，当前文件也会变得很大。要想使中继日志文件占用较少的磁盘空间，可以将系统变量max_relay_log_size设置为较小的值，以减小中继日志文件的最大可允许大小。

12.8.7.3 自动完成日志过期处理

你可以手动调用日志过期处理脚本，但如果你能让这些命令自动运行，就不需要时时提醒自己去去做这件事情。在Unix里，实现这一操作的一种方法是使用工具程序 cron，建立一个定义有过期调度的crontab文件。（在Windows里，可以使用“计划任务”来代替。）

要想了解更多有关cron的信息，可以使用下列命令查看相关的Unix手册页：

```
% man cron
```

```
% man crontab
```

```
% man 5 crontab
```

假设，你想使用rotate_fixed_logs.sh脚本来轮换一个名为qlog的普通查询日志，并且假设该脚本安装在/home/mysql/bin 目录里，日志文件存放在/var/mysql/data 目录里。接着，先以mysql用户身份登录，然后使用下面的命令编辑mysql用户的crontab文件：

```
% crontab -e
```

该命令允许你编辑当前crontab文件的副本（如果还未建立cron作业，则它可能为空）。像下面这样往文件里添加一行内容：

```
30 4 * * * /home/mysql/bin/rotate_fixed_logs.sh /var/mysql/data/qlog
```

这个条目会告知cron，在每天早上4点半运行这个脚本。这个调度可能会根据需要的不同而有所差异。与条目格式有关的信息，请查看crontab手册页。对于一个会生成大量日志信息的繁忙服务器，可以让其日志轮换频率比活跃度不太高的服务器更快一些。

要想确保日志都会被定期刷新（例如，要生成下一个编号的二进制日志文件），可以通过增加另一个crontab 项来调度 mysqladmin flush_logs 命令定期执行。你可能需要将mysqladmin的完整路径名列出来，以保证cron能够找到它。

12.8.7.4 日志表的过期或轮换处理

如果服务器把普通查询日志或慢查询日志写入了mysql数据库的表里，你就可以截断表，或者采用某种形式的表轮换。

要想截断表，可以使用下面这些语句：

```
USE mysql;
```

```
TRUNCATE TABLE general_log;
```

```
TRUNCATE TABLE slow_log;
```

要想轮换日志表，需要先为它创建一个空副本。然后执行一个原子重命名操作，在一条语句里“置换出”当前表，并用空副本替换它：

```
USE mysql;
```

```
DROP TABLE IF EXISTS general_log_tmp, general_log_old;
```

```
CREATE TABLE general_log_tmp LIKE general_log;
```

```
RENAME TABLE general_log TO general_log_old, general_log_tmp TO  
general_log;
```

```
DROP TABLE IF EXISTS slow_log_tmp, slow_log_old;
```

```
CREATE TABLE slow_log_tmp LIKE slow_log;
```

```
RENAME TABLE slow_log TO slow_log_old, slow_log_tmp TO slow_log;
```

如果你已启用了事件调度器，可以让日志表轮换工作自动进行，方法就是像下面这样创建一个事件。这个事件每天会轮换一次日志表。更改ON SCHEDULE子句可以改变轮换频率。

```
CREATE EVENT mysql.rotate_log_tables
```

```
ON SCHEDULE EVERY 1 DAY
```

```
DO BEGIN
```

```
DROP TABLE IF EXISTS general_log_tmp, general_log_old;
```

```
CREATE TABLE general_log_tmp LIKE general_log;
```

```
RENAME TABLE
```

```
general_log TO general_log_old,
```

```
general_log_tmp TO general_log;
```

```
DROP TABLE IF EXISTS slow_log_tmp, slow_log_old;
```

```
CREATE TABLE slow_log_tmp LIKE slow_log;
```

```
RENAME TABLE
```

```
slow_log TO slow_log_old,
```

```
slow_log_tmp TO slow_log;
```

```
END;
```

12.9 运行多个服务器

大多数人都是在一台给定的机器上运行一个MySQL 服务器，但有时运行多个服务器也很有用或很有必要。

想要测试新版本的服务器，同时想要让生产服务器保持原样。在这种情况下，你要运行不同的服务器二进制文件。

想要尝试使用复制机制，以加深自己对它的了解，但是你只有一台服务器主机，并且必须得在同一台机器上运行主服务器和从服务器。

操作系统通常会对每个进程打开文件描述符的数量加以限制。如果提高这个限制值对你的系统来说比较难（例如，只有编译内核才能做到这一点），那么运行服务器二进制文件的多个实例则是突破这一限制的一种方法。

因特网服务提供商通常会为各个客户提供他们自己的MySQL安装，而这肯定需要多个服务器。这可能会涉及这样一些情况：如果所有客户运行的都是相同版本的MySQL，则需要运行同一个二进制文件的多个实例；如果客户之间运行的是不同版本的MySQL，则需要运行不同的二进制文件。

12.9.1 运行多个服务器的常见问题

运行多个服务器比只运行一个服务器要复杂很多，因为必须防止它们互相干扰。有些问题会在安装时出现的。如果需要同时安装不同的版本，则必须把它们放置到不同的位置。对于预编译好的二进制发行版，你可以通过将它们解压到不同目录的方式来实现。对于需要自己编译的源代码发行版，可以通过设置CMake的CMAKE_INSTALL_PREFIX选项来为每一个发行版指定不同的安装位置。

其他问题则发生在服务器启动时。对于有些参数，每一个服务器进程都必须拥有一个唯一的值。例如，每个服务器都必须针对各个进入连接监听不同的TCP/IP端口，以防止出现冲突。不管你是运行不同的服务器二进制文件，还是运行的同一个二进制文件的多个实例，都需如此。这条原则也适用于其他类型的连接接口，如Unix套接字文件、Windows命名管道或共享内存。如果你启用了日志，则每个服务器还必须把日志信息写入自己的那一组日志文件。让不同的服务器把日志信息写入相同的文件会引发很多问题。

你可以在启动时指定服务器的各个选项（通常情况下是在选项文件里）。或者，如果你运行了几个自己根据源代码编译的服务器二进制文件，那么你也可以在构建过程中为每一个要使用的服务器指定一组不同的参数值。这些值会成为它的内建默认值，并且不需要在启动时显式地指定它们。

不过，如果你是根据源代码来编译的，那么使用选项文件的一个原因就是：采用这种方式指定的参数可以充当文档，对每一个服务器是如何配置的进行显式说明。

接下来的讨论列举了几种类型的选项。如果不依据每个服务器来对其进行设置，可能就会引发冲突。有些选项会影响到其他选项，因此不必针对每个服务器显式地设置所有选项。例如，每个服务器都必须在运行时使用唯一的一组日志文件。但是，数据目录是存放所有这些日志文件的默认位置，因此告知每个服务器使用不同的数据目录，会隐式地导致日志文件被保存在不同的文件集中。

其中有些选项实际上是系统变量（标志就是其名字里有下划线），但是可以在服务器启动时设置它们。

如果你运行着几个不同版本的服务器，那么通常情况下，每个发行版本都会被安装在不同的基本目录下。每一个服务器都应该拥有其自己的数据目录。（在Windows里，这一点通常是强制性的；而在Unix 里，则强烈推荐这样做。）使用下表里的选项可以显式地指定这些值。

选项	用途
--basedir=dir_name	MySQL安装根目录的路径名
--datadir=dir_name	数据目录的路径名

在许多情况下，数据目录是基本目录的子目录，但并不总是如此。例如，因特网服务提供商可能会为其客户提供一套公用的MySQL服务器和客户端程序，但是会针对每一位客户运行一个服务器实例，而该服务器使用的是客户特定相关的数据目录。在这种情况下，所有服务器的基本目录都是一样的，但是各自的数据目录却位于不同位置，很可能就在客户的主目录里。

对于下表里的各个网络接口选项，每一个服务器必须拥有不同的值，目的在于防止多个服务器在同一个接口上进行监听。

选项	用途
<code>--port=port_num</code>	TCP/IP连接端口号
<code>--socket=file_name</code>	Unix域套接字文件的路径名或Windows命名管道名
<code>--shared-memory-base-name=name</code>	共享内存连接所使用的共享内存名（仅限于Windows）

在Windows 里，只有那些为启用命名管道和共享内存连接而在启动时带上选项`--enable-named-pipe`和`--shared-memory`的服务器，才会需要选项`--socket`和`--shared-memory-base-name`。这时，只有一个服务器可以使用默认的命名管道名和共享内存名（分别为MySQL和MYSQL），而其他所有服务器都必须指定不同的名字。

对于状态文件（如PID文件）和日志文件，每个服务器都必须使用不同的文件名。否则，多个服务器之间会因为都想把内容写到同一个文件里而出现竞争。这种竞争导致的不良后果，最乐观的也是使服务器间的记录内容相互混淆，难以分辨，而最糟糕的便是让类似服务器复制这样的操作无法正常进行。服务器会根据下表里各个选项所指明的名字来创建状态文件和日志文件。如果指定的是相对文件名，那么这些文件将被创建在服务器数据目录里。如果每个服务器都使用了不同的数据目录，那么不必指定绝对路径名便可以让每个服务器记录到不同的文件集里。（与日志文件命名有关的更多信息请参考12.8节。）

其中有些选项实际上是系统变量（标志就是其名字里有下划线），但是你可以在服务器启动时设置它们。可以在运行时对这些变量进行更改。

选项	用途
<code>--general_log</code>	启用普通日志
<code>--general_log_file=file_name</code>	生成日志文件名
<code>--log-bin[=file_name]</code>	启用二进制日志
<code>--log-bin-index=file_name</code>	二进制日志索引文件
<code>--log_error[=file_name]</code>	启用出错日志
<code>--log_output[=destination]</code>	普通查询/慢查询日志存放位置
<code>--pid_file=file_name</code>	进程ID文件
<code>--relay-log[=file_name]</code>	启用中继日志
<code>--relay-log-index=file_name</code>	中继日志索引文件
<code>--slow_query_log</code>	启用慢查询日志
<code>--slow_query_log_file=file_name</code>	慢查询日志文件名

每个复制从服务器都必须拥有唯一的一组主服务器信息文件和中继日志信息文件。这些文件被默认创建在数据目录里，可以使用选项`--master-info-file`和`--relay-log-info-file`来对它们进行显式设置。

在Unix里，如果你使用`mysql_safe`来启动服务器，它会创建一个出错日志文件（默认是在数据目录里）。要想显式指定出错日志名，可以设置系统变量 `log_error`。或者，还可以把出错输出发送给`syslog`。相关细节请参考12.8.1节。

必须把每个使用InnoDB的服务器都配置成让其使用自己的系统表空间。另外，InnoDB存储引擎存放其日志的那个目录对于每一个服务器来说都必须是唯一的。默认情况下，InnoDB会将其日志写入数据目录。要想改变这个位置，可以设置系统变量 `innodb_log_group_home_dir`。更多相关信息请参考12.5.3节。

在Unix里，可能有必要为每个服务器指定一个`--user`选项，用以指明运行该服务器所需要的注册账户。这一点与你为不同用户提供单独的MySQL服务器实例的操作很像，其中每一个用户都“拥有”一个独立的数据目录。

在Windows里，每一个以服务方式安装的服务器都必须使用唯一的服务名。

12.9.2 配置和编译不同的服务器

如果想通过源代码来构建不同版本的服务器，必须将它们安装在不同的位置。将它们分隔开的最简单方法是：在运行CMake时设置CMAKE_INSTALL_PREFIX，让每个服务器都有一个不同的安装基本目录。如果把服务器的版本号合并到基本目录名称里，就可以轻易地将哪个目录对应于哪个版本的MySQL 区分出来。本节将展示一种可以实现这一目标的方法。本节还将描述几个我用来分隔我自己的MySQL安装的配置约定。

我把所有的MySQL 安装在一个公共目录/var/mysql 里，把给定的发行包安装在/var/mysql/的某个子目录里，并用这个发行包的版本号来命名该目录。例如，我想用/var/mysql/50525 作为 MySQL 5.5.25的安装基本目录，可以通过在运行 CMake 时带上-DCMAKE_INSTALL_PREFIX=/var/mysql/50525选项来实现。还可以使用其他选项来指定一些附加的与服务器特定相关的值，如 TCP/IP端口号和套接字路径名。我所用的配置是让 TCP/IP端口号等于版本号，将套接字文件放置在基本目录里，并在其中把数据目录命名为data。

为建立这些配置选项，我使用了一个名为config-ver的shell脚本：

```
VERSION=50525

PREFIX="/var/mysql/$VERSION"

PORT="$VERSION"

ENGINES="

-DWITH_INNOBASE_STORAGE_ENGINE=ON

-DWITH_PARTITION_STORAGE_ENGINE=ON

-DWITH_PERFSCHEMA_STORAGE_ENGINE=ON

"

OTHER="

-DCPACK_MONOLITHIC_INSTALL=ON
```

```

-DENABLED_LOCAL_INFILE=ON

-DMYSQL_MAINTAINER_MODE=OFF

-DWITH_SSL=bundled

"

OPTIONS="

-DCMAKE_INSTALL_PREFIX=$PREFIX

-DMYSQL_DATADIR=$PREFIX/data

-DMYSQL_TCP_PORT=$PORT

-DMYSQL_UNIX_ADDR=$PREFIX/mysql.sock

$ENGINES $OTHER

"

cmake $OPTIONS .

```

我会保证用第一行设置正确的版本号，在需要时根据要编译进服务器的存储引擎来修改其他值，等等。在脚本建立好之后，接下来便是用下列命令配置、构建和安装该发行版：

```

% sh config-ver

% make

% make install

```

在安装好给定版本的MySQL 之后，还需要把当前位置切换到安装基本目录，并初始化数据目录和权限表：

```

# cd /var/mysql/50525

# scripts/mysql_install_db --user=user_name

```

其中，`user_name`是用于运行服务器的那个登录账户的名字（如mysql账户）。请在运行这些命令时以root身份或user-name身份登录。

至此，我完成了MySQL安装目录的锁定步骤。相关内容在12.2.1.1节有简要描述，更多细节可参考13.1.2节。

在这之后，剩下的工作便只有设置那些我想要在选项文件里使用的选项，以及编排服务器的启动。12.9.4节对实现这一目标的一种方法进行了讨论。

12.9.3 指定启动选项的策略

在安装好服务器之后，该如何使用每个服务器所需要的正确选项集来启动服务器呢？你有以下几种选择。

如果你是自己构建服务器，那么可以把针对每一个服务器的那一组默认值编译进去，这样在启动时就无需再提供选项。这一做法的缺点是，给定服务器正使用哪些参数并不一定很明显。

可以通过命令行或选项文件在服务器启动时指定各个选项。如果需要指定大量的选项，那么在命令行上把它们写出来会很不实际。将其放在选项文件内反而会更方便些，虽然要想保证每个服务器都能读取到正确的那组选项也非易事。实现这一目标的策略有以下几种。

使用`--defaults_file`选项指定服务器要从中读取所要选项的选项文件，并为每个服务器指定不同的文件。这样，可以把给定服务器所需要的全部选项放入一个文件，以便在一个地方指定其所有的设置。（请注意，当你使用`--defaults_file`选项时，所有平常的选项文件，如`/etc/my.cnf`，都不会被读取。）

把所有服务器共用的那些选项放入一个全局选项文件，如`/etc/my.cnf`，并在命令行使用`--defaults_extra_file`选项来指定一个文件，其中包含与给定服务器特定相关的各种附加选项。例如，可以在`/etc/my.cnf`里使用`[mysqld]`组来指定那些适用于所有服务器的选项。不需要在单个的服务器选项文件里重复这些选项。

一定要让放到公共选项组内的所有选项，都能被你所运行的所有服务器读懂。例如，对于那些早于5.6.3的服务器版本，就不能使用`--plugin-load-add`选项，因为这个选项的概念是从那一个版本才开始引入的。如果把这个选项放到公共选项组里，就会导致早于该版本的服务器启动失败。不过，你可以使用`loose-opt_name`语法来指定那些无法被所有服务器读懂的选项。如果

服务器无法读懂以这种方式提供的选项，就会将这个选项忽略，并在记录下警告信息之后继续执行。有关“loose”选项的更多信息请参考F.2节。

使用`mysqld_multi`脚本来管理多个服务器的启动。这个脚本程序允许你在一个文件里列出所有服务器使用的选项，但是会将每个服务器与这个文件里属于它自己的特定选项组关联在一起。

在Windows 里，你也可以运行多个服务器，不过使用的是特殊的选项文件组命名约定（它们专门用于这种风格的服务器配置）。更多相关信息请参考12.2.2.2节。

接下来的几节将通过演示如何在Unix里使用`mysqld_multi`，以及如何在Windows里运行多个服务器来展示几种应用这些策略的方法。

12.9.4 使用`mysqld_multi`来管理服务器

在Unix里，常用于启动服务器的脚本`mysqld_safe`和`mysqld.server`，在单服务器设置中运行状况都很好。为了使处理几个服务器的工作变得更加容易，需要使用`mysqld_multi`脚本来替代。

`mysqld_multi` 脚本会以分配给每个服务器的特定编号为基础，把服务器各个选项列在某个选项文件的`[mysqldn]`组内，其中，`n` 是上述那个编号。这个选项文件也可以包含一个`[mysqld_multi]`组，在其中列出专供`mysqld_multi`自己使用的那些选项。例如，如果已安装好的MySQL 服务器的版本有 5.1.64、5.5.25和5.6.6，那么可以将它们的选项组分别指定为`[mysqld50164]`、`[mysqld50525]`和`[mysqld50606]`，并可以像下面这样在`/etc/my.cnf`文件里设置各种选项：

```
[mysqld50164]

basedir=/var/mysql/50164

datadir=/var/mysql/50164/data

mysqld=/var/mysql/50164/bin/mysqld_safe

socket=/var/mysql/50164/mysql.sock

port=50164

user=mysql
```

log_error=log.err

general_log

general_log_file=qlog

log-bin=binlog

skip-innodb

[mysqld50525]

basedir=/var/mysql/50525

datadir=/var/mysql/50525/data

mysqld=/var/mysql/50525/bin/mysqld_safe

socket=/var/mysql/50525/mysql.sock

port=50525

user=mysql

log_error=log.err

general_log

general_log_file=qlog

log-bin=binlog

innodb_data_file_path=ibdata1:50M:autoextend

event_scheduler=ON

[mysqld50606]

basedir=/var/mysql/50606

datadir=/var/mysql/50606/data

```
mysqld=/var/mysql/50606/bin/mysqld_safe
```

```
socket=/var/mysql/50606/mysql.sock
```

```
port=50606
```

```
user=mysql
```

```
log_error=log.err
```

```
general_log
```

```
general_log_file=qlog
```

```
log-bin=binlog
```

```
innodb_data_file_path=ibdata1:100M
```

```
lc_messages=fr_FR
```

```
character_set_server=utf8
```

这里为每个服务器设定的布局参数对应于 12.9.2节描述的目录配置。同时，还指定了几个附加的与服务器特定相关的参数，它们分别对应于不同类型的日志、存储引擎等。

要想启动某个给定的服务器，可以在命令行调用`mysqld_multi`，同时将命令字`start`和服务器选项组编号作为调用参数：

```
% mysqld_multi --no-log start 50525
```

其中，选项`--no-log` 会使得状态信息被发送到终端，而不会发送到日志文件。这样可以让你更轻易地看到正在发生的情况。你可以同时指定多个服务器，方法是用逗号将各组编号隔开。采用短划线分隔编号的方式可以指定某段范围的服务器编号。在服务器编号列表里不能有空白。例如：

```
% mysqld_multi --no-log start 50164,50525-50606
```

要想停止服务器或获得服务器是否正在运行的状态报告，可以使用命令字`stop`或`report`，后面紧跟服务器列表。自MySQL 5.6.3起，你还可以使用命令字`reload`（来停止和重启）。对于这些命令，`mysqld_multi`脚本需要调用

mysqladmin来与服务器进行通信，因此还必须指定一个管理账户的用户名和密码：

```
% mysqld_multi --nolog --user=root --password=rootpass stop 50164
```

```
% mysqld_multi --nolog --user=root --password=rootpass report 50164,50606
```

这里的用户名和密码必须适用于所有你想要用一个给定命令来控制的服务器。mysqld_multi 脚本会试着自动确定 mysqladmin的位置，你也可以在选项文件的[mysqld_multi]组里显式指定该路径。另外，你还可以将默认的管理用户名和密码列在stop命令和report命令会使用的那个选项组里。例如：

```
[mysqld_multi]
```

```
mysqladmin=/usr/local/mysql/bin/mysqladmin
```

```
user=leeloo
```

```
password=multipass
```

出于安全的考虑，更好的做法是把密码放在选项文件里，而不是让其暴露在命令行里。但是，如果将密码放在选项文件里，则一定要确保该文件不能被公开地读取！更多相关信息请参考13.1.2.2节。

12.9.5 在Windows里运行多个服务器

在Windows里运行多个MySQL服务器的方法有两种：一是手动启动服务器；二是使用多个Windows服务。当然也可以将这两种方法混和使用。

要想手动启动多个服务器，需要为它们分别创建一个选项文件，用于列出它们各自的参数。例如，要想运行处在不同数据目录，但使用相同程序二进制文件的两个服务器，可以创建两个选项文件。假设，MySQL安装在C:\mysql里，其数据目录位于C:\mysql\data下，而你想运行的第二个服务器实例使用的数据目录位于C:\mysql\data2下。于是，各个选项文件的内容如下所示。

文件C:\my.ini1的内容：

```
[mysqld]
```

```
basedir=C:/mysql
```

```
datadir=C:/mysql/data
```

```
port=3306
```

文件C:\my.ini2的内容:

```
[mysqld]
```

```
basedir=C:/mysql
```

```
datadir=C:/mysql/data2
```

```
port=3307
```

在启动第二个服务器之前，相应的数据目录必须先存在。建立C:\mysql\data2的最简单方法是把它创建成C:\mysql\data的一个副本。使用下列命令（在没有服务器运行的情况下）：

```
C:\> xcopy C:\mysql\data C:\mysql\data2 /E
```

然后，在命令行启动各个服务器，用--defaults-file选项告知各个服务器读取特定于自己的选项文件：

```
C:\> mysql --defaults-file=C:\my.ini1
```

```
C:\> mysql --defaults-file=C:\my.ini2
```

要想把MySQL安装为Windows服务，可以使用--install选项。例如，可以使用下面两条命令中的任何一个，把mysqld安装为服务：

```
C:\> C:\mysql\bin\mysqld --install
```

```
C:\> C:\mysql\bin\mysqld --install service_name
```

这里的--install命令使用了服务器程序的完整路径名。如果你的服务器安装在其他的地方，则需要对这条命令里的路径名做出相应的修改。如果没有给出 service_name 参数或者MySQL的名字，Windows会使用默认的服务名（即为MySQL）；否则，使用给定的名字。（在这两种情况里，服务器会读取哪些选项组呢？与之相关的规则请参考12.2.2.2节。）

假设，你想要运行两个mysqld实例，它们使用的服务名分别为MySQL和mysqlsvc2，使用的共享内存名分别为MYSQL和mysqlsvc2，使用的数据目录与

上一个示例的相同。可以像下面这样在其中一个标准选项文件（如 C:\my.ini）里为每个服务器建立各种选项：

```
# 默认的（“MySQL”）服务所对应的组
```

```
[mysqld]
```

```
basedir=C:/mysql
```

```
datadir=C:/mysql/data
```

```
port=3306
```

```
shared-memory
```

```
shared-memory-base-name=MYSQL
```

```
# “mysqsvc2” 服务所对应的组
```

```
[mysqsvc2]
```

```
basedir=C:/mysql
```

```
datadir=C:/mysql/data2
```

```
port=3307
```

```
shared-memory
```

```
shared-memory-base-name=mysqsvc2
```

这些选项组的先后顺序非常重要。以默认服务名 MySQL 安装的服务器只会读取 [mysqld] 选项组。但是，以非默认服务名 mysqsvc2 安装的服务器会同时读取两个选项组 [mysqld] 和 [mysqsvc2]。如果把 [mysqsvc2] 选项组放在这个选项文件里的第二部分，就可以利用它来改写 [mysqld] 选项组里的所有选项，而所用的值可以只适用于那个以 mysqsvc2 服务方式运行的服务器。

要想创建和启动这些服务，可以使用下面这些命令（同样地，请根据你的实际情况修改服务器的路径）：

```
C:\> C:\mysql\bin\mysqld --install
```

```
C:\> net start MySQL
```

```
C:\> C:\mysql\bin\mysqld --install mysqlsvc2
```

```
C:\> net start mysqlsvc2
```

如果你在创建服务时提供了服务名，那么你还可以将`--defaults-file`选项指定为命令行的最后一个选项：

```
C:\> C:\mysql\bin\mysqld --install service_name --defaults-file=file_name
```

如此一来，又多出了一种提供服务器相关选项的方式。服务器会记住那个文件名，并且会在每次启动时都使用它。服务器只会从该文件的`[mysqld]`组里读取各个选项。

可以使用命令 `mysqladmin shutdown`、`net stop` 或服务管理器来关闭服务器。要想卸载服务，需要先关闭它们（如果它们正在运行的话），然后通过指定`--remove`命令和在安装服务器时所使用的服务名来删除每条服务。如果服务名为默认值（即MySQL），就可以将其省略掉：

```
C:\> mysql --remove
```

```
C:\> mysql --remove mysqlsvc2
```

12.9.6 运行多个服务器的客户端

当有多个服务器在运行时，要想使用客户端程序，在指定适当的连接参数之后便可以连接到所想要的那个服务器。这里包括使用`mysqladmin`来关闭服务器。例如，要想连接到某个正在监听50525端口的服务器，可以运行下面的命令：

```
% mysql --protocol=tcp --port=50525
```

在Windows里，要想使用名为`mysqlsvc2`的共享内存方式连接到服务器，可以运行下面的命令：

```
C:\> mysql --protocol=memory --shared-memory-base-name=mysqlsvc2
```

12.10 升级MySQL

MySQL的开发非常活跃，会定期进行升级。这便向管理员提出了一个问题：当新版本发行时，是否需要升级已安装的MySQL。本节提供的各种指导将有助于你做出选择。

当新版本发行时，要做的第一件事情是，确定它与之前版本有何差异。确保获得新版发行通知的方法是，订阅 MySQL的通告邮件列表。（具体的订阅方法，可以访问 <http://lists.mysql.com/>。）每个通告都会包括一些最新修改注意项，因此这是一种保持不断获悉新版开发情况的很不错的方法。

（另一种方法是，自己查看MySQL的发行说明，了解更新情况。）另外，也可以阅读 MySQL 参考手册里与升级有关的部分，了解相关的发行情况。该部分会指明许多值得你注意的重要问题，以及升级时必须采取的特殊步骤。如果新的发行版本引入的某些行为与旧的发行版本不兼容，那么这些信息就会变得尤为重要。

在查看了发行说明和手册里的升级部分之后，你便可以问自己这样几个问题。

你在当前版本里遇到过新版本所修复的这些问题（包括安全问题）吗？

新版本是否有你想要的或需要的附加功能？

对于你所使用的那些操作，新版本是否在性能上有所改进？

如果对所有这些问题的回答都是否定的，就没有必要进行升级。如果对其中某个问题的回答为肯定的，就可以考虑升级。不过，通常还可以等上几天，关注一个MySQL的一般讨论邮件列表，看看其他人对于该发行版本的情况报告。比如，升级有帮助吗？是否发现了错误或其他问题？这里还有其他几个需要考虑的因素，它们能帮助你做出最终的决定。

稳定系列中的发行版通常是用于修正错误，而不是为了添加新的功能。一般情况下，用稳定版本来升级比用开发版本来升级带来的风险会更小。

如果升级 MySQL，可能还需要升级其他链接了 MySQL的C语言客户端开发库的那些程序。例如，在升级MySQL之后，可能需要重新构建那些依赖MySQL的C语言客户端开发库的其他开发库或应用程序，才能链接到这个新的客户端开发库。（很多示例都会用到Perl的DBD:mysql模块或者PHP语言。当所有与MySQL相关的DBI脚本和PHP脚本，在升级 MySQL 之后开始出现内存转储现象时，这便预示着你需要重新构建它们。）如果你不想重新构建，那最好就不要升级 MySQL。如果你使用的程序采用的是静态链接方式，而非动态链接，那么出现此问题的可能性就会大大降低。但是，你的系统内存需求会增加。

如果你仍然不能确定是否需要升级，那么可以独立于当前的服务器来对新服务器进行测试。在测试时，既可以与生产服务器并行运行，也可以将其安装在不同机器上。如果使用不同的机器，那么两个服务器之间的独立性会更容易维护，因为你会有更大的自由空间，可以根据自己的选择来对它进行配置。如果选择在同一主机上并行运行新服务器和已有服务器，那么在配置时需要保证某些参数的唯一性，如安装位置、数据目录和网络接口（服务器会基于它来监听连接）。更多详细内容请参考12.9节。

不管选择哪一种方式，你都可能会想要用现有数据库的数据副本来测试新的服务器。更多与复制数据库相关的指导请参考14.4节。

要想让测试服务器在执行时拥有不间断的语句源，可以考虑把生产服务器当作一个复制主服务器，而将测试服务器设置为复制从服务器。这样，主服务器执行的那些更新会被发送到从服务器，从而向测试服务器提供持续的输入流。主服务器不会发送任何检索语句到从服务器，但是你可以在从服务器运行客户端程序，调用SELECT语句，弄清它是如何处理它们的。

请试用开发版本！

将开发版本用于生产目的（如业务资产管理），并不是一种明智的做法。但是，本书还是鼓励大家对新版本进行测试，甚至还可以将它与你的生产数据副本一起使用。试用新版本的人越多，新版本被使用得越充分。这样能提高找出 bug 的可能性，让这些 bug 得到修复。bug 报告是推动MySQL开发不断向前发展的重要因素。

第13章

安全性与访问控制

作为一名MySQL管理员，你有责任保证数据库的安全，只能让拥有适当权限的人对数据进行访问。为此，你必须维护好MySQL安装的安全性和完整性。第12章已经涉及一个与安全性相关的话题，即为 MySQL 账户设置密码的重要性。该话题当时是作为安装和运行过程中的一部分被处理的。在本章中，我们将深入查看几个安全性方面的问题。

安全性为什么很重要，你必须提防何种类型的攻击。

你所面临的内部安全风险源自于服务器主机上拥有登录账户的其他用户，你需要管理好文件系统的访问权限。

你所面临的外部安全风险源自于通过网络连接到服务器的客户端，你需要管理好MySQL的用户账户。

内部安全问题主要与其他用户有关，这些用户能直接访问MySQL服务器主机，也就是说，他们有该主机的登录账户。一般情况下，内部安全与文件系统访问权限密切相关。为解决此问题，你需要保护 MySQL 安装的各项内容，让其免遭那些在运行服务器的机器上拥有账户的其他人的攻击。尤其是对于服务器的数据目录，应该只允许用于运行 MySQL 服务器的那个登录账户拥有和控制它。如果这一点无法得到保证，那么你在安全方面所做的其他努力也将白费。例如，虽然你能确保已经按照权限表正确地设置了 MySQL 账户，控制了通过网络进行连接的客户端，但是这些表的完整性依赖于合理的文件系统保护。如果对数据目录内容的访问权限设置得过于宽松，那么其他人便可以将所有与权限表相对应的文件替换掉，实施一套完全不同的客户端访问控制策略。

外部安全问题主要与源自网络的连接有关。必须保护MySQL服务器免受网络连接的攻击，防止未授权客户端对数据库内容进行访问。应该设置好MySQL的权限表，让客户端只有在提供有效名字和密码的情况下才能访问服务器管理的那些数据库。另一种危险是，第三方可能会对网络进行监视，并捕捉服务器和客户端之间的通信。为解决此问题，可以对MySQL进行设置，让其支持使用安全套接层（SSL）协议的连接。

本章将描述大家都应该认真对待的安全性问题，展示如何防止通过文件系统和网络进行的未经授权的访问。本章将会频繁使用到用于运行MySQL服务器和执行其他MySQL相关管理任务的那个登录账户。在本书中，此账户的用户名和组名都是 `mysql`。如果你使用了其他的用户名和组名（如使用了自己的登录账户来运行MySQL服务器），则需要更改各个示例里的名称。

13.1 保护对MySQL文件系统的访问

本节将介绍如何锁住MySQL的安装位置，使其免遭服务器主机上未经授权的用户实施的破坏。本节仅适用于Unix系统。这里假设，当你在Windows上运行服务器时，拥有对机器的完全控制权，并且没有其他本地用户。

MySQL在安装过程中会创建几个目录，其中有些需要不同的保护。例如，服务器程序就不需要MySQL管理账户以外的任何人对其进行访问。相比之下，客户端程序通常允许公开访问，以便其他用户可以运行它们——但对访问能力还是有所限制的，即不能修改或更换它们。

在初次安装完成之后，还需要创建其他一些受保护的文件。对于具体的创建方式，既可以选择将其作为安装之后的配置过程的一部分，由你自己来完成；也可以选择服务器运行时，由服务器来完成。由你负责创建的文件包括选项文件或安全套接层相关的文件。服务器在运行时自己建立的目录和文件包括数据库目录、数据库目录下的文件（与数据库里的表相对应的）、日志文件和Unix套接字文件。

很显然，你需要维护好服务器所维护的那些数据库的保密内容。数据库的所有者需要经常性地考虑如何恰当地做好数据库内容的保密工作。即使他们不做这样的考虑，也应该选择让数据库的内容公开，而不是让其内容因对数据库目录的保护不力而遭泄露。

必须要保证日志文件的安全，因为它们包含了客户端发送给服务器的语句文本。这是一个普遍关注的问题，因为任何具有日志文件访问权限的人都可以监视数据库内容的变化。与日志文件相关的一个更为具体的安全问题是，它们可能包含敏感语句（包含有密码）的文本。MySQL使用了密码加密机制，但是这个加密机制要在设置了密码并且建立了连接之后才起作用。密码

的设置过程会涉及诸如CREATE USER、GRANT或SET PASSWORD之类的语句，而这些语句都会以明文形式记录在某些日志文件里。对这些日志文件具有读取权限的攻击者，只要通过简单的动作便可发现这些敏感性信息，例如，在这些日志文件上运行grep命令，查找类似GRANT或PASSWORD的单词。

有些文件是要允许客户端程序访问的，如Unix套接字文件。通常情况下，都只是允许访问，但不允许完全控制。例如，用户应该能通过套接字文件连接服务器，但是不能删除它，否则会影响其他用户连接服务器。

13.1.1 如何窃取数据

下面这个简短的示例说明了安全的重要性。它强调这样一个事实：你其实并不想让用户直接访问MySQL的数据目录。

MySQL服务器的权限系统非常灵活，其实现依托的是mysql数据库里的权限表。你可以设置这些权限表的内容来允许或拒绝客户端对数据库的访问。这样做能够保障安全，防止客户端在未授权的情况下通过网络对数据进行访问。不过，如果服务器主机上的其他用户可以直接访问数据目录的内容，那么再好的数据库网络访问安全设置都没有意义。除非你很清楚只有你自己可以登录到MySQL服务器主机，否则你必须考虑那台机器上的其他人可能获得服务器数据目录访问权限的情况。

很显然，你肯定不想服务器主机上的其他用户对数据目录里的文件拥有直接写入的访问权限，因为他们可能会破坏状态文件或者数据库的表。但是直接读取访问权限同样很危险。如果能够读到表的文件，那么就能很轻易地窃取这些文件，并让MySQL本身向你展示该表的内容。怎么才能做到呢？这样做。

(1) 在服务器主机上安装一个自己的“流氓”MySQL服务器，使用与官方服务器不同的端口、套接字文件和数据目录。

(2) 运行mysql_install_db，初始化数据目录。这个动作可以让你以MySQL的root用户身份访问你的服务器，建立一个test数据库，用于存储窃取到的那些表。

(3) 访问你想要攻击的那个服务器的数据目录，将你想要窃取的那些表所对应的文件复制到你那个服务器数据目录下的test目录里。这个动作只需要目标数据目录的读取权限。(4) 启动你的那个流氓服务器。一眨眼功夫，这个服务器的test数据库便包含了窃取来的那些表的副本，你可以随意访问它们。使用SHOW TABLES FROM test语句，可以显示这些副本表；使用SELECT *语句，则可以显示其中任何一个表的整个内容。

(5) 更为可恶的是，你可以将许可权限开放给你服务器上的匿名用户账户，这样，任何人都可以从任何主机连接到该服务器，访问你的test 数据库。这相当于是向全世界公开那些窃取来的表。

请仔细想想这一场景，然后换位思考一下。你想要有人这样对付你吗？肯定不想！因此，请利用下面讨论里的各项指导保护好你自己。

13. 1. 2 保护MySQL安装

本节将针对组成 MySQL 安装的那些目录和文件，讲述如何设置其所有权和访问模式。本节的指导所使用的用户名和组名都是mysql，它们将被赋予安装所有权。（选哪一个用户都行，就是不能选root，具体原因请参考12. 2. 1. 1节。）这些指导还会假设MySQL安装的所有内容都位于同一个基本目录下，而不是分散在整个文件系统的不同目录下。在各个示例里，MySQL安装的基本目录都是/usr/local/mysql，数据目录都是其下一级目录/usr/local/mysql/data。

在完成此步操作之后，本书将对如何处理某些非标准的安装布局进行讲解。你的系统布局与本书所描述的布局可能不一样，不过你应该能够做出相应的调整。根据需要更改你自己系统的名称和路径名。如果你运行有多个服务器，则需要对每个服务器都执行此步骤。

通过执行 `ls -la` 命令，可以确定数据目录中是否包含不安全的文件或目录。只需在其中找出“组”或“其他”用户权限已被打开的那些文件或目录即可。下面是一个不安全数据目录的列表，其中列出了一部分数据库目录：

```
% ls -la /usr/local/mysql/data
```

```
total 10148
```

```
drwxrwxr-x    11 mysql  wheel      1024 May   8 12:20 .
```

```
drwxr-xr-x    22 root   wheel      512 May   8 13:31 ..
```

```
drwx-----    2 mysql  mysql      512 Apr 16 15:57 menagerie
```

```
drwxrwxr-x    2 mysql  wheel      512 Jun 25   1998 mysql
```

```
drwx-----    7 mysql  mysql     1024 May   7 10:45 sampdb
```

```
drwxrwxr-x    2 mysql  wheel     1536 Jun 25   1998 test
```



```
drwx----- 2 mysql mysql 1024 May 8 18:43 tmp
```

其中，“.”代表的是正被列出的那个目录，即/usr/local/mysql/data。“..”代表的是父目录/usr/local/mysql。有些数据库目录的用户权限比较合理，如drwx-----，表示其所有者具有读、写和执行权限，其他人无权访问。但是其他一些目录的访问权限模式则过于宽松，如drwxrwxr-x，表示所有其他用户乃至mysql组外的用户，都具有读取和执行这些目录的权限。本示例展示的这个情况是日积月累的结果，即从一个（非常）旧的MySQL安装开始，持续逐步地升级到较新的版本。与新版MySQL服务器相比，旧版服务器所创建的用户权限比较宽松，它们在权限设置方面的要求没那么严格。（从这里显示的内容可以看出，那些限制较为严格的数据库目录——menagerie、sampdb和tmp，其日期都距现在较近。）现在的MySQL服务器都会将其创建的数据库目录的权限设置成，只允许运行服务器的那个账户可以访问它们。

你也可以使用ls -la命令来查看MySQL安装的基本目录/usr/local/mysql。例如，你可能会获得类似如下的结果：

```
% ls -la /usr/local/mysql
```

```
total 44
```

```
drwxrwxr-x 13 mysql mysql 1024 May 7 10:45 .
drwxr-xr-x 24 root wheel 1024 May 1 12:54 ..
drwxr-xr-x 2 mysql mysql 1024 Jul 16 20:58 bin
drwxrwxr-x 12 mysql wheel 1024 May 8 12:20 data
drwxr-xr-x 3 mysql mysql 512 May 7 10:45 etc
drwxr-xr-x 3 mysql mysql 512 May 7 10:45 include
drwxr-xr-x 3 mysql mysql 512 May 7 10:45 lib
drwxr-xr-x 3 mysql mysql 512 May 7 10:45 man
drwxr-xr-x 6 mysql mysql 1024 May 7 10:45 mysql-test
drwxr-xr-x 3 mysql mysql 512 May 7 10:45 scripts
```

```
drwxr-xr-x    3 mysql  mysql          512 May   7 10:45 share
drwxr-xr-x    7 mysql  mysql        1024 May   7 10:45 support-files
```

通过下面几个步骤可以把所有内容锁定，只允许mysql账户进行访问，不过，部分安装内容除外，有时其他用户也需要访问它们。例如，需要对data目录的权限和所有权进行更严格的限制，如同之前指明的那样。

请注意，如果将MySQL服务器和客户端程序连同其他的非MySQL程序一起安装在通用的系统目录里，那么其中的有些步骤就不能使用。（如果使用RPM包来安装MySQL，则通常会发生上述情况。）例如，服务器位于/usr/sbin，而各个客户端位于/usr/bin。在这种情况下，MySQL程序的所有权和权限模式应该会被设置成与这些目录里的其他程序一样。

(1) 如果MySQL服务器正在运行，则需要停止它：

```
% mysqladmin -p -u root shutdown
```

(2) 使用下面的命令，把整个MySQL安装的所有者和组名设置为MySQL管理账户，这些命令必须以root身份执行：

```
# chown -R mysql /usr/local/mysql
```

```
# chgrp -R mysql /usr/local/mysql
```

另一种常见的方法是，让所有除数据目录以外的内容都隶属于root，相关命令如下所示：

```
# chown -R root /usr/local/mysql
```

```
# chgrp -R mysql /usr/local/mysql
```

```
# chown -R mysql /usr/local/mysql/data
```

```
# chgrp -R mysql /usr/local/mysql/data
```

如果你把总的所有者设置为root，则需要以root身份执行下面的大部分步骤。否则，你可以用mysql身份来执行它们。

(3) 对于那些客户端应该能访问的基本目录及其子目录，需要更改它们的模式，以便mysql拥有其全部权限，而其他人只拥有其读取和执行权限。有

可能它们已被这样设置过了；如果没有，则需要更改它们。例如，可以用下列命令中的任何一条来设置基本目录：

```
% chmod 755 /usr/local/mysql
```

```
% chmod u=rwx,go=rx /usr/local/mysql
```

同样地，包含客户端程序的那个bin目录也可以用下列命令中的任何一条来进行设置：

```
% chmod 755 /usr/local/mysql/bin
```

```
% chmod u=rwx,go=rx /usr/local/mysql/bin
```

(4) 更改数据目录以及其下面所有文件和子目录的访问模式，只允许mysql用户访问它们：

```
% chmod -R go-rwx /usr/local/mysql/data
```

这样，除了用于运行 MySQL 服务器的那个登录账户之外，其他账户都不能直接访问数据目录里的内容。

在完成前面的几个步骤之后，你的MySQL 安装基本目录的所有权和用户权限将变成下面这个样子：

```
% ls -la /usr/local/mysql
```

```
total 44
```

```
drwxr-xr-x  13 mysql  mysql    1024 May  7 10:45 .
```

```
drwxr-xr-x  24 root   wheel    1024 May  1 12:54 ..
```

```
drwxr-xr-x   2 mysql  mysql    1024 Jul 16 20:58 bin
```

```
drwx----- 12 mysql  mysql    1024 May  8 12:20 data
```

```
drwxr-xr-x   3 mysql  mysql      512 May  7 10:45 include
```

```
drwxr-xr-x   3 mysql  mysql      512 May  7 10:45 lib
```

```
drwxr-xr-x   3 mysql  mysql      512 May  7 10:45 man
```

```

drwxr-xr-x    6 mysql  mysql      1024 May  7 10:45 mysql-test
drwxr-xr-x    3 mysql  mysql        512 May  7 10:45 scripts
drwxr-xr-x    3 mysql  mysql        512 May  7 10:45 share
drwxr-xr-x    7 mysql  mysql      1024 May  7 10:45 support-files

```

如上所示，所有内容现在都隶属于mysql用户和mysql组。例外情形是那个“..”目录，它代表的是父目录/usr/local/mysql。该目录隶属于root用户，只允许该用户进行修改，这么做有好处，因为你肯定不想让未授权用户能更改MySQL的安装目录。

基本目录下的数据目录甚至拥有更为严格的用户权限设定：

```
% ls -la /usr/local/mysql/data
```

```

total 10148
drwx----- 11 mysql  mysql      1024 May  8 12:20 .
drwxr-xr-x  22 mysql  mysql        512 May  8 13:31 ..
drwx-----  2 mysql  mysql        512 Apr 16 15:57 menagerie
drwx-----  2 mysql  mysql        512 Jun 25  1998 mysql
drwx-----  7 mysql  mysql      1024 May  7 10:45 sampdb
drwx-----  2 mysql  mysql     1536 Jun 25  1998 test
drwx-----  2 mysql  mysql      1024 May  8 18:43 tmp

```

其中，“..”代表数据目录的父目录，即MySQL的基本目录。

有些文件可能需要作为“只有 mysql 能访问数据目录”这一策略的例外情况。例如，想要把数据目录用作Unix套接字文件的存放位置，则必须让该目录的访问权限更开放一点。否则，客户端程序将不能通过这个套接字连接到服务器。另外，也可以采用把Unix套接字文件存放在其他位置（如服务器基本目录）的方法来解决。同样的原则也适用于其他除 mysqld 以外的程序需要对其进行访问的文件，如那些包含全局客户端参数的选项文件。（与实现这些方法有关的更多信息请参考13.1.2.1节。）

如前所述，前面的步骤假定与 MySQL 有关的所有文件都位于一个单独的基本目录下。如果不是这样，则需要找到每个与MySQL有关的目录，并针对每个目录执行相应的操作。例如，如果数据目录位于/var/mysql/data，而不是位于/usr/local/mysql，则需要发出下面几条命令来适当更改安装目录的所有权：

```
# chown -R mysql /usr/local/mysql

# chgrp -R mysql /usr/local/mysql

# chown -R mysql /var/mysql/data

# chgrp -R mysql /var/mysql/data
```

或者，假设在MySQL安装目录下创建了一个innodb目录，用于存放与InnoDB有关的文件。默认情况下，这些文件都在数据目录里。如果把它们放在innodb 目录里，则必须将该目录的访问模式设置成与数据目录的一样。如果想对那些在通常情况下存放在数据目录里的其他文件（如日志文件）进行重定位，这一原则也同样适用。

如果安装根目录下的某些目录实际是指向其他某个地方的符号链接，则会产生另一个复杂的问题。如果你所用版本的chown和chgrp不支持符号链接，则必须跟踪这些链接，并更改它们所指向的那些位置的所有权。一种操作方法是使用find：

```
# find /usr/local/mysql -follow -print | xargs chown mysql

# find /usr/local/mysql -follow -print | xargs chgrp mysql
```

更改访问模式也要考虑这些。例如，如果你的数据目录下有一些符号链，并且chmod不会跟踪它们到实际的位置，则可以使用下面的命令来替代：

```
# find /usr/local/mysql/data -follow -print | xargs chmod go-rwx
```

因为在此刻，该数据目录内容的所有权和访问权限都被设定为只允许mysql登录用户访问，所以你应该保证服务器从现在起，始终是以mysql身份运行的。确保做到这一点的较为容易的方法是，在服务器启动时会读取的/etc/my.cnf文件或其他my.cnf文件里，找到[mysqld]组件，在其中指定该用户：

```
[mysqld]
```

```
user=mysql
```

这样，不管你启动服务器时是以root身份登录，还是以mysql身份登录，服务器都会以mysql身份运行。关于使用特定登录账户运行服务器的更多信息请参考12.2.1.1节。

在保护好MySQL的安装内容之后，需要重新启动服务器。

13.1.2.1 保护Unix套接字文件

对于客户端至 localhost 的连接，服务器使用的是Unix域套接字文件。这个套接字文件通常允许被公开访问，以便客户端程序可以使用它。但是，它不应该放在任何客户端都可以进行删除的目录里。例如，常见的做法是把套接字文件创建在/tmp 目录里，但是在某些 Unix系统里，该目录的权限设置允许用户删除其他人的文件。这意味着任何用户都可以删除这个套接字文件，最后导致的结果便是：客户端程序无法建立与服务器的localhost连接，除非服务器重启再次创建这个套接字文件。更好的做法是为/tmp 目录设置“粘滞位”，这样即使某人在该目录里创建了文件，他也只能删除自己的文件。以 root 身份执行下面的命令，可以设置这个目录的粘滞位：

```
# chmod +t /tmp
```

有些安装程序会把套接字文件放置在数据目录里，而如果把这个数据目录设置为mysql私有，则会导致这样的问题：客户端程序只有以root或mysql身份运行时才能访问套接字文件。在这种情况下，一种解决办法是让这个数据目录变得稍微开放一点。为了让客户端程序在对数据目录没有完全读取权限的情况下，也能够访问套接字文件，可以使用下面这条命令：

```
% chmod go+x /usr/local/mysql/data
```

如果不想像这样让数据目录变得过于开放，则可以采用另一种方法，即让服务器在其他位置创建套接字文件。例如，可以配置MySQL，让其在基本目录里创建套接字文件，指定位置为/usr/local/mysql/mysql.sock。既可以在全局选项文件里指定位置，也可以根据源代码重新编译，将位置构建为默认值。如果选择使用选项文件，请记得在服务器和客户端两边都要指定该位置。

```
[mysqld]
```

```
socket=/usr/local/mysql/mysql.sock
```

```
[client]
```

```
socket=/usr/local/mysql/mysql.sock
```

重新编译比较费事，但是这是一种更为圆满的解决办法，因为使用选项文件只对会检查选项文件的客户端才有作用。（所有的标准 MySQL 客户端都可以，但是第三方程序可能不行。）通过重新编译，新的套接字位置会变成默认值，记录在客户端开发库里。这样，所有使用客户端开发库的程序都将获得这个新的位置，作为自己的默认值，而与是否使用选项文件无关。

13.1.2.2 保护选项文件

使用选项文件存在潜在风险，因为它们会包含一些不该被公开看到的选项。

如果某个选项文件包含有诸如MySQL账户名或密码之类的敏感信息，则不要让它可公共读取。

/etc/my.cnf文件通常允许公共可读，因为它是一个用于指定全局客户端选项的地方。请不要把服务器的选项（如复制机制中使用的密码）放入其中。

每个用户特有的（user-specific）.my.cnf选项文件，应该隶属于它所在主目录对应的那个用户，且只可以被这个用户访问。具体做法是，在你自己的主目录里执行以下命令：

```
% chmod u=rw,go-rwx .my.cnf
```

其他选项文件则需要根据它们的具体用途来设置其访问模式。

一种确保用户选项文件拥有正确的访问模式或所有权的方法是：运行某个程序，让它依次查找每个用户主目录里的.my.cnf 文件，并修正发现的所有问题。下面这个Perl 脚本 `chk_mysql_opt_files.pl`可以做到这一点：

```
#!/usr/bin/perl
```

```
# chk_mysql_opt_files.pl - 检查用户特定的.my.cnf 文件，
```

```
# 确保访问模式和所有权都正确。每个文件都应该隶属于
```

```
# 它所在主目录对应的那个用户。它的访问模式
```

```
# 应该关闭其中的“组”权限和“其他”权限。
```

此脚本必须以root 身份运行。在执行时需要把密码文件作为输入。

如果密码文件为/etc/passwd，则可以像下面这样运行它：

```
# chk_mysql_opt_file.pl /etc/passwd
```

```
use strict;
```

```
use warnings;
```

```
while (<>)
```

```
{
```

```
next if /^#/ || /^\\s*$/;          # 跳过注释、空行
```

```
my ($uid, $home) = (split (/:/, $_))[2,5];
```

```
my $cnf_file = "$home/.my.cnf";
```

```
next unless -f $cnf_file;        # 有.my.cnf 文件吗？
```

```
if ((stat ($cnf_file))[4] != $uid) # 测试所有权
```

```
{
```

```
warn "Changing ownership of $cnf_file to $uid\n";
```

```
chown ($uid, (stat ($cnf_file))[5], $cnf_file);
```

```
}
```

```
my $mode = (stat ($cnf_file))[2];
```

```
if ($mode & 077)                  # 测试“组”和“其他”访问
```

位

```
{
```

```
warn sprintf ("Changing mode of %s from %o to %o\n",
```



```
$cnf_file, $mode, $mode & ~077);  
  
    chmod ($mode & ~077, $cnf_file);  
  
}  
  
}
```

你可以在smpdb发行包的admin目录里找到chk_mysql_opt_files.pl。此脚本必须以root身份运行，这样才能更改其他用户所拥有的那些文件的访问模式和所有权。如果想要自动执行此脚本，则可以把它设置为以root用户身份在每个夜间运行的cron作业。

13.2 管理MySQL用户账户

MySQL管理员必须知道如何设置MySQL账户，其中包括指定哪些用户可以连接到服务器，他们可以从哪里连接到服务器，以及在连接之后他们被允许做些什么事情。MySQL会把这些信息存储在mysql数据库的多个权限表里，可以通过下面这些账户管理SQL语句操作这些表。

CREATE USER、DROP USER和RENAME USER。分别用于创建、删除和重命名MySQL账户。

GRANT。用于指定账户权限（如果账户不存在，则创建它们）。

REVOKE。用于移除MySQL账户的权限。

SET PASSWORD。用于分配账户的密码。

SHOW GRANTS。用于显示账户所拥有的权限。

这些语句会对表13-1中所示的mysql数据库权限表产生影响。

表13-1 MySQL权限表

权限表	内容
user	可连接到服务器的那些用户，以及他们的全局性权限
db	数据库权限
tables_priv	表权限
columns_priv	列权限
procs_priv	存储例程权限
proxies_priv	代理用户权限

还有一个host 权限表，不会受这些账户管理语句的影响，但它已被废弃，因此这里不再对它进行讨论。

当执行CREATE USER语句时，必须指定一个账户名和可选的授权信息（密码或授权方法），服务器会在user表里为这个新账户创建一个行。在GRANT语句里，如果指定的账户不存在，也会发生同样的事情。在使用GRANT语句时，如果指定了全局权限（管理权限或应用于所有数据库的权限），那么它们也会被记录到user表里。如果在GRANT语句里指定的权限只适用于给定的数据库、表、列或存储例程，那么它们会被分别记录到db、tables_priv、columns_priv或procs_priv表里。PROXY权限的分配会被记录在proxies_priv表里。REVOKE语句可以删除权限表里的权限，DROP USER语句可以从各个表中删除与给定账户相关的所有行。

接下来的几节将描述，如何创建和删除 MySQL 用户账户，如何实现权限的授权和撤销，以及如何更改密码或重置丢失的密码。你也可以执行像INSERT和UPDATE 这样的SQL语句，直接处理权限表的内容。不过，使用像CREATE USER和GRANT 这样的账户管理语句，可以使管理工作更便捷，概念更清晰，因为在你描述好自己对各种访问权限所要进行的修改操作后，服务器会将你的请求自动映射成正确的权限表更改操作。不过，尽管使用CREATE USER和GRANT语句比直接修改权限表更容易，但本书还是建议你读一读自13.3节开始的几节。这几节对权限表进行了更详细的描述，能帮助你更好地理解：它们在账户管理语句这一层是如何工作的；客户端在试图连接服务器或执行SQL语句时，服务器将如何使用它们的内容。

说明

有些MySQL版本引入了新的权限表或权限列，而这会改变它们的内部结构。当第一次在机器上安装MySQL时，安装过程创建权限表所使用的结构对

于你正安装的那个MySQL版本来说是最新的。如果把MySQL升级到一个更新的版本，则需要运行mysql_update，用当前版本之后发生的所有修改来升级这些权限表。

默认情况下，mysql_update需要以root用户身份连接本地服务器，因此在调用它时必须像下面这样带着root的密码，然后重启服务器：

```
% mysql_update --password=rootpass
```

13.2.1 MySQL账户的高级管理

使用下面三条语句可以对MySQL账户执行一些高级操作。

CREATE USER语句。用于创建新账户，并选择性地为它设置密码或授权方法：

```
CREATE USER account [auth_info];
```

CREATE USER语句不会分配权限，那是**GRANT**语句分内的事情。

DROP USER语句。用于删除已有账户和所有与之关联的权限：

```
DROP USER account;
```

DROP USER语句不会删除，这个被删账户可以访问到的那些数据库或其中的对象。

RENAME USER语句。用于更改已有账户的名字：

```
RENAME USER from_account TO to_account;
```

如果你拥有全局的**CREATE USER**权限，那么这三条语句就都可以使用。否则，你必须拥有mysql数据库的**INSERT**、**DELETE**或**UPDATE**权限，才能分别使用**CREATE USER**、**DROP USER**或**RENAME USER**语句。

在创建新账户时，一般可以通过回答下面这几个问题来确定使用何种类型的**CREATE USER**语句。

用户名是什么？

这个用户能连接哪个或哪些主机？

如何进行用户认证？

对前面两个问题的回答可用于确定该语句里的那个account值。账户名由用户名和主机名构成。更多与语法管理规则有关的信息请参考13.2.1.1节。

对第三个问题的回答取决于你是要为账户分配密码，还是要指定服务器使用其他的方法来认证这个用户。更多与认证有关的信息请参考13.2.1.3节。

13.2.1.1 指定账户名

在账户管理语句（如CREATE USER）里，account值由用户名和主机名构成，具体格式为'user_name'@'host_name'。在MySQL里，你不仅要指定谁可以连接，而且还要指定可以从什么地方进行连接。这意味着，可以为拥有相同名字、但会从不同位置连接服务器的两个用户，分别建立一个独立的账户。MySQL允许你区别对待它们，并为每一个账户分配彼此毫无瓜葛的权限。服务器会将其中的user_name和host_name值，存入该账户user表行的User列和Host列，还会存入与该账户相关的其他权限表行中。

也可以把账户名指定为CURRENT_USER或CURRENT_USER（），它代表的是当前会话里使用的那个账户。

在MySQL里的用户名，仅仅是在连接服务器时用于表明身份的名字。它与Unix或者Windows的登录名没有任何必然联系。在Unix里，如果连接服务器时没有显式指定用户名，那么客户端程序将默认把登录名当作MySQL用户名，但这只是一种约定。关于那个被用作MySQL超级用户（它可以做任何事情）的名字root，也没什么特别的。你完全可以在权限表里把这个名字更改为superduper，然后再用superduper身份连接服务器，执行各种需要超级用户权限才能完成的操作。

通过选择一个合适的account值，便可以让用户从你想要的某个特定主机或者某个范围的主机进行连接。一种极端的情况是，如果你知道某个用户只会从某台主机进行连接，则完全可以将用户限制在该台主机上：

```
CREATE USER 'boris'@'localhost' IDENTIFIED BY 'frost';
```

```
CREATE USER 'fred'@'ares.mars.net' IDENTIFIED BY 'steam';
```

请注意，其中的主机名部分指的是客户端将从它那里连接到服务器的那台主机。它并不是指客户端将要连接到哪台服务器主机（除非它们碰巧相同）。

你可以做到的最为严格的访问控制形式是，只允许用户从单台主机连接到服务器。而另一种极端的情况是，你的用户可能会经常出差，必须要从世界各地的不同主机连接到服务器。假设该用户的名字为max，则可以使用如下语句让他在任何地方都能连接到服务器：

```
CREATE USER 'max'@'%' IDENTIFIED BY 'mist';
```

其中，“%”字符是一个通配符，其含义与LIKE模式匹配里的含义相同。因此，当%作为主机名指示符时，其代表的是“任意主机”。这是最简单的用户设置方式，但它也是最不安全的。（使用%来指定主机，有时也会导致一些令人头疼的问题，具体原因详见13.4.4节。）

另一个是LIKE通配符（“_”），它可以用在主机值里，匹配任意单个字符。如果需要指定通配符“%”或“_”本身，则需要在其前面加上一个反斜线字符进行转义。

对于中间情况，可以让用户只能从一组有限的主机连接服务器。例如，想让用户mary只能从example.com域里的任意一台主机连接服务器，则需要把主机名指定为%.example.com：

```
CREATE USER 'mary'@'%.example.com' IDENTIFIED BY 'fog';
```

只要愿意，account值的主机部分还允许以IPv4或IPv6地址（而非主机名）的形式进行指定。对于IPv4主机值，你可以指定为文字IP地址（一种包含模式字符的地址），也可以指定为带掩码（它指明哪些位可用于网络编号）的IP地址：

```
CREATE USER 'joe'@'192.168.128.3' IDENTIFIED BY 'water';
```

```
CREATE USER 'ardis'@'192.168.128.%' IDENTIFIED BY 'snow';
```

```
CREATE USER 'rex'@'192.168.128.0/255.255.255.0' IDENTIFIED BY 'ice';
```

上面第一条语句表明，用户只能从IP地址为192.168.128.3的那台主机连接服务器。第二条语句指定的IP模式表示的是C类子网192.168.128。在第三条语句里，192.168.128.0/255.255.255.0指定了一个掩码，该掩码打开了前24位。它能与IP地址前24位为192.168.128的任何一台主机相匹配。可用的掩码值有255.0.0.0、255.255.0.0、255.255.255.0或255.255.255.255。

对于IPv6主机值，除了不支持掩码以外，其他规则都类似。

在账户名里使用主机名localhost，可以让用户从本地主机连接服务器。具体方式如下。

在Unix 里，用户在进行连接时可以使用的主机值包括 localhost、127.0.0.1或::1。对于localhost，连接类型为Unix套接字文件。对于127.0.0.1或::1，连接类型为基于本地主机的IPv4或IPv6环回接口的TCP/IP连接。

在Windows里，用户在进行连接时可以使用的主机值包括localhost、127.0.0.1或::1。如果服务器支持共享内存连接，那么localhost连接的类型默认为共享内存连接；否则，所有连接的类型都是TCP/IP连接。如果服务器支持命名管道连接，那么用户还可以通过指定主机名“.”（句点），以命名管道的方式来建立连接。

如果account值的用户名或主机名部分可以被用作一个不带引号的标识符，则不必将它用引号引起来。如果它包含有像“_”或“%”这样的特殊字符，则必须将它用引号引起来。例如，对于账户名boris@localhost，其用户名和主机名部分不使用引号也是合法的。不过，使用引号是最保险的，本书的示例也都遵循这一原则。在给用户名和主机名加上引号时，既可以使用字符串引号，也可以使用标识符引号。用户名和主机名需要分开使用引号，例如，需要写成'boris'@'localhost'，而不要写成'boris@localhost'。

如果在指定账户时没有给出主机名部分，则等同于把主机名部分设置为%。因此，'max'与'max'@'%'是相同的account 值。这意味着，如果原本想要把某个账户设置为'boris'@'localhost'，但是却将其误写成了'boris@localhost'，那么MySQL也会将其当作是一个合法的账户。发生的情况是，MySQL 将'boris@localhost'整个解释为只包含用户名部分，并会给它加上一个%，以作为其默认的主机名部分，最终得到的账户名将是'boris@localhost'@'%'。为避免这一问题，一定要给账户名里的用户名和主机名分别加上引号。

13.2.1.2 将账户名里的主机值与DNS相匹配

当使用服务器的主机名（而不是使用localhost）从服务器主机进行连接时，通常会遇到一些麻烦。出现这一问题的原因在于，在权限表里指定主机名的方式与DNS域名字解析器把主机名报告给程序的方式不一致。假设，服务器主机的全限定名为cobra.example.com。如果域名解析器报告的是一个未限定的主机名，如 cobra，而权限表的行里包含的是全限定名（或者情况相反），那么这两个主机名肯定不匹配。

为确定你的系统上是否会出现这种问题，可以在连接本地服务器时尝试使用-h选项来指定主机名称：

```
% mysql -h cobra.example.com
```

接着，查看服务器的常规日志文件。当服务器在报告连接尝试时，它会怎样写入主机名呢？名字是全限定的，还是未限定的？根据具体结果，便可以判断出在账户名里需要如何指定主机名部分。

当把IP值作为主机部分指定某个账户时，如果DNS以不同的格式返回该IP值，则可能会遇到类似的匹配问题。例如，对于给定主机，DNS返回的IP值为192.168.10.14，那么它可以与主机部分为192.168.10.14或192.168.10.%的那些账户相匹配，但是无法匹配主机部分为192.168.010.14或192.168.010.%的账户。如果你感到疑惑，可以运行DNS查找工具，核实一下你的DNS解析器会怎样返回值，并使用与账户名一样的格式。

13.2.1.3 指定账户认证方式

CREATE USER语句可借助于可选的auth_info子句来指定账户认证方式：

```
CREATE USER account [auth_info]
```

其中，auth_info子句也是GRANT语句的一部分请参考13.2.2节。

auth_info子句可采用的两种形式如下所示。

账户采用密码认证。语法格式如下：

```
IDENTIFIED BY [PASSWORD] 'password'
```

例如：

```
CREATE USER 'pradeep'@'localhost' IDENTIFIED BY 'aurum';
```

此时，MySQL会管理密码本身，并使用相当于PASSWORD()函数生成的散列格式，将其存储在该账户user行中的Password列里。通常情况下，可以省略PASSWORD关键字，用纯文本指定明文密码。MySQL会在存储值之前，将其转换为给定的散列格式。在特殊情况下，如你想要以已有的散列格式来指定密码，可以在该值的前面加上PASSWORD关键字。当利用SHOW GRANTS的输出重新创建账户时，可能会出现这种情况。（SHOW GRANTS显示的是散列后的密码值，而不是明文密码。）

指定一种不同的认证方法。使用下列语法，指定一个实现了所需方法的认证插件和一个为该插件提供附加信息的可选字符串：

```
IDENTIFIED WITH auth_plugin [AS 'auth_string']
```

例如：

```
CREATE USER 'felipe'@'localhost' IDENTIFIED BY 'myplugin';
```


此时，MySQL 不会直接管理密码，但是会期望插件在连接时能够报告用户认证是否正常。更多细节请参考13.2.7节。目前，与指定密码的方式相比，指定认证插件的方式还比较少见，因为认证插件是最近（在MySQL 5.5.7里）才引入的功能。随着时间的推移，认证插件应该会受欢迎，因为在认证方法的实现方面，它们除了允许使用MySQL本身内部管理的密码类型来进行认证，还允许使用凭据来进行认证。例如，认证插件可以访问操作系统的密码，或者与实现了单点登录机制的某个外部认证服务器进行交互。

当服务器创建账户时，如果不指定带有非空密码的IDENTIFIED BY子句或者IDENTIFIED WITH子句，则客户端会在无认证的情况下用它来连接服务器。这种情况是不安全的，应该尽量避免。

13.2.2 权限分配

为某个账户分配访问权限时，可以使用GRANT语句，其语法格式如下：

```
GRANT privileges [(columns)]  
  
ON what  
  
TO account [auth_info]  
  
[REQUIRE encryption requirements]  
  
[WITH grant or resource management options];
```

如果指定的账户存在，那么 GRANT 语句会修改其权限。如果该账户不存在，那么 GRANT语句会在创建它时带上分配的权限。为避免GRANT语句创建不需要进行认证的新账户（这会是个安全隐患），可以设置sql_mode系统变量，以启用SQL的NO_AUTO_CREATE_USER模式。这样可以防止GRANT语句在未得到认证信息的情况下创建账户。

其中有几个子句是可选的，如无必要，完全可以不用指定它们。通常情况下，最为常用的是下面几个部分。

privileges。它表示的是分配给账户的权限。例如，SELECT权限允许用户执行SELECT语句，SHUTDOWN权限允许用户关闭服务器。每次可以指定多个权限，其间用逗号隔开。

columns。它表示的是受权限影响的列。多个列之间必须以逗号隔开，并且需要列在括号里。它是可选的，并且只用于设置列相关的权限。每个列列

表必须紧跟在每个相关权限名字的后面。

what。它表示的是权限应用的级别。最高级别是全局级，其中给定的任何权限将会应用到所有数据库和所有表。全局权限可被当作超级用户权限。权限也可以应用到特定的级别，其中包括数据库、表、列（如果指定了 columns 子句）或存储例程。

account。它表示的是被授予权限的那个账户。其格式为 'user_name'@'host_name'，请参考13.2.1.1节。

auth_info、REQUIRE子句和WITH子句都是可选的。

auth_info。它表示的是账户密码或认证方法。它是可选的，在已指定账户认证信息的情况下（如使用CREATE USER），不需要使用它。如果使用它，其语法格式与CREATE USER的一样，请参考13.2.1.3节。已有账户的当前密码都会被指定的任意一个密码所替换。

REQUIRE 子句。它会设置账户，要求其在进行连接时必须使用安全的安全套接字层连接。

WITH 子句。它可用于授予 GRANT OPTION 权限，从而让此账户能够把自己的权限转授给其他用户。WITH 子句还可用于指定资源管理选项，从而让你可以对某个账户每小时可以进行多少次连接或可以执行多少条语句加以限制。这些选项有助于防止某些账户过度占用服务器资源。

在指定某个账户可以做哪些事情时，通常可以采用回答下面这几个简单问题的方式来决定使用何种类型的GRANT语句。

应该为该账户分配什么样的访问类型？或者说，该用户应该拥有什么样的权限级别？这些权限应该应用于哪些内容？

是否需要使用安全连接？

该用户是否应该被授予管理员权限？

是否应该对该用户的资源占用量进行限制？

接下来的几节将展示如何回答这些问题，并通过示例来说明如何使用 GRANT 语句的各种子句。

13.2.2.1 定义账户的权限

一个账户可被授予多种权限。这些权限主要分为两类（之前讲过）：管理权限和对象权限。另外，还有两个特殊权限说明符：ALL和USAGE。其中，ALL（或ALL PRIVILEGES）表示的是“所有权限”（GRANT OPTION 除外），USAGE 表示的是“无权限”。USAGE的主要作用在于更改与账户有关的内容，（如资源限制或是否需要 SSL），而不是更改账户的权限。更多的补充说明请参考13.3节，其中会对这些权限与底层权限表之间的关系进行探讨。

下面这些权限适用于管理操作，如控制服务器的运作或用户进行权限授予的能力。通常情况下，在分配这些权限时需要加倍小心，因为它们会让用户对服务器的运作产生影响。例如，SHUTDOWN权限就不应该被随意分配。

CREATE USER

此权限允许使用的语句包括CREATE USER、DROP USER、RENAME USER和REVOKE ALL PRIVILEGES。

FILE

此权限可以让你告诉服务器读取或写入服务器主机的文件。为限定此权限的使用范围，服务器会采取下面几个预防措施。

你只能访问那些公开可读的文件，这样的文件不可能被认定为是受保护的。

任何你想要写入的文件都不能事先存在。这样可以防止你强迫服务器把某些重要的文件（如/etc/passwd 或者其他数据库中的数据库文件）覆盖掉。（例如，如果没有这种限制，那么你便可以将mysql数据库里的那些权限表的内容完全替换掉。）

尽管服务器会有这些预防措施，但是如果没有充足的理由，请不要分配这个权限，因为它极其危险，详情请参考13.2.6节。如果你拥有FILE权限，那么在Unix上一定不要以root登录用户身份运行服务器，因为root用户可以在文件系统的任何地方创建新文件。

以普通的登录账户身份运行服务器，可以确保它只能在该账户可以访问的目录里创建文件。更多相关信息请参考12.2.1.1节。

GRANT OPTION

它可以让你将自己拥有的权限授予其他用户，其中包括GRANT OPTION权限。

PROCESS

MySQL 服务器是多线程的，能够同时为多个客户端连接提供服务。这些线程可以被当作是服务器里运行的进程。有了 PROCESS 权限，你便可以使用 SHOW PROCESSLIST 语句或者mysqladmin processlist命令来查看与当前执行的活动相关的信息。利用这种权限，你可以查看所有活动情况，甚至包括其他用户的活动情况。即使在没有PROCESS权限的情况下，你也可以看到自己的活动情况。

PROXY

此权限使你能够获得另一个用户的权限。换句话说，你可以充当另一个用户的代理，做该用户能做的任何事情。

RELOAD

此权限让你能够执行某些服务器管理操作。在拥有 RELOAD 权限之后，便可以执行像FLUSH和RESET 这样的语句。另外还可以执行这样几个mysqladmin 命令：reload、refresh、flush-hosts、flush-logs、flush-privileges、flush-status、flush-tables和flush-threads。

REPLICATION CLIENT

此权限使你可以使用语句SHOW MASTER STATUS和SHOW SLAVE STATUS来查询主、从服务器的位置和状态。

REPLICATION SLAVE

此权限使客户端能够连接主服务器，发出让从服务器进行更新的请求，且能使用SHOW SLAVE HOSTS和SHOW BINLOG EVENTS语句。该权限只能授予那些连接主服务器的从服务器账户。

SHOW DATABASES

此权限使你能够通过执行SHOW DATABASES语句查看到所有数据库的名称。如果没有此权限，则只能看到你对其拥有权限的那个给定数据库的名称。不过，通过适用于数据库的任意一个全局权限也可以达到这一效果，这些权限包括CREATE TEMPORARY TABLES和LOCK TABLES，它们通常都是全局授予的。如果只想让拥有 SHOW DATABASES 权限的用户执行SHOW DATABASES语句，则可以在启动服务器时带上--skip-showdatabase选项。

SHUTDOWN

此权限使你能够关闭服务器，如使用mysqladmin shutdown命令。

SUPER

此权限使你能够使用KILL语句或mysqladmin kill命令来终止服务器进程。利用此权限，你可以终止任何进程，甚至包括那些属于其他用户的进程。即使在没有SUPER权限的情况下，你也始终可以终止自己的进程。

此权限启用的其他语句包括 SET（用于修改全局系统变量和全局事务的特性）、CHANGE MASTER、PURGE BINARY LOGS、SHOWMASTER STATUS、SHOW SLAVE STATUS、START SLAVE和STOP SLAVE。在有了SUPER权限之后，你还可以在定义视图或存储程序的语句中的DEFINER子句里指定任何账户，利用DES_DECRYPT()函数执行DES解密，该函数是以存储在DES密钥文件里的密钥为基础的。

拥有 SUPER 权限，便可以使用mysqladmin debug，它会在连接服务器时覆盖任何max_connections设置，这样，你就可以在所有常规插槽都被占用的情况下，访问服务器为管理连接预留的连接槽。

下面这些权限适用于针对对象（如数据库、表和存储过程）的操作。它们控制着对服务器所管理的那些数据进行访问的能力。

ALTER

此权限使你可以使用ALTER TABLE 语句。不过根据你想对表进行的具体操作，可能还需要某些附加权限。

ALTER ROUTINE

此权限使你能够更改或删除存储函数和存储过程。

CREATE

此权限使你能够创建数据库和表。它不能让你创建表索引，除非这些索引一开始便定义在CREATE TABLE语句里。

CREATE ROUTINE

此权限使你能够创建存储函数和存储过程。

CREATE TABLESPACE

此权限使你能够创建、删除或更改表空间。

CREATE TEMPORARY TABLES

此权限使你能够利用CREATE TEMPORARY TABLE语句创建临时表。

CREATE VIEW

此权限使你能够创建视图。

DELETE

此权限使你能够删除表里的行。

DROP

此权限使你能够删除数据库和表。它不允许你删除索引。

EVENT

此权限使你能够对事件调度器的各个事件进行操作。

EXECUTE

此权限使你能够执行存储函数和存储过程。

INDEX

此权限使你能够创建或删除表的索引、为键缓存分配索引，以及将索引预加载到键缓存。

INSERT

此权限使你能够将行插入表里。

LOCK TABLES

此权限使你能够通过显式地执行 LOCK TABLES 语句来锁定表。它仅适用于你对其拥有SELECT 权限的表，不过它能让你设置读锁或写锁，而不只是读锁。此权限不适用于服务器在语句执行过程中代表你隐式获得的那些锁。这类锁的设置和释放都是自动的，与LOCK TABLES权限设置无关。

REFERENCES

此权限未使用。

SELECT

此权限使你能够使用SELECT语句检索表里的数据。对于像SELECT NOW()或SELECT 4/2这样的SELECT语句，不需要此权限，因为它们只是在执行表达式计算，并未涉及表。

SHOW VIEW

此权限使你能够使用SHOW CREATE VIEW语句查看视图定义。

TRIGGER

此权限使你能够添加和删除触发器。

UPDATE

此权限使你能够修改表里的行。

有些操作需要多种权限的组合。例如，REPLACE 操作会在执行 DELETE 之后隐式地执行一个INSERT操作，因此它需要DELETE和INSERT权限。想要分配某个权限，你必须先自己拥有该权限，并且还必须拥有GRANT OPTION权限。

你可以在不同的级别上进行权限分配，从全局级别到非常特定的级别。通过ON子句说明符可以对其进行控制，如表 13-2 所示。对于表级的说明符，可以在权限名字的后面指定一个（columns）子句，以在列级别上分配该权限。后面会有示例演示这种语法。

表13-2 权限级别说明符

权限说明符	权限作用级别
ON *.*	全局权限：所有数据库以及其中的所有对象
ON *	针对默认数据库的数据库级权限：若无默认数据库，则出错

续表

权限说明符	权限作用级别
ON db_name.*	数据库权限：指定数据库里的所有对象
ON db_name.tbl_name	表权限：指定表里的所有列
ON tbl_name	表权限：默认数据库里指定表的所有列
ON db_name.routine_name	指定数据库里指定例程的权限
ON account	代理权限：account 为被代理用户

如果存在歧义，想显式地指定权限所要应用的对象类型，可以包含关键字TABLE、FUNCTION或PROCEDURE（如ON TABLE mydb.mytbl或ON FUNCTION mydb.myfunc）。

USAGE权限只能在全局级指定（即需要与ON *.*一起使用）。

权限说明符 ALL（或者 ALL PRIVILEGES）会把给定级别上的所有权限授予账户。例如，在全局级别，它会授予全部权限。在表级别，它只会授予适用于表的权限。ALL只能用于全局、数据库、表或例程权限。对于列权限，必须明确地列出你想要授予的每一种权限。“所有权限”的真实含义是“有一个例外的所有权限”，该例外就是 GRANT和REVOKE 操作所需要的GRANT OPTION权限。

全局权限的作用范围最大，因为它们能应用到任何数据库。为创建一个能做任何事情（包括对其他用户进行授权）的超级用户账户，可以执行以下语句：

```
CREATE USER 'ethel'@'localhost' IDENTIFIED BY 'coffee';
```

```
GRANT ALL ON *.* TO 'ethel'@'localhost' WITH GRANT OPTION;
```

ON *.*子句代表的意思是“所有数据库及其中的所有对象”。出于安全的考虑，在上面这个例子里创建的账户只能从本地主机进行连接。对超级用户能够连接服务器的主机进行限制，是一种明智的做法，因为这样可以防止某组主机进行密码攻击的尝试。

管理权限在本质上具有全局性的特点，因此只有GRANT OPTION和PROXY才可以使用全局权限说明符ON *.*。例如，RELOAD权限允许使用FLUSH语句，因此下面这几条语句将创建一个名为flush的用户，它除了能执行FLUSH语句外什么事情也不能做：


```
CREATE USER 'flush'@'localhost' IDENTIFIED BY 'flushpass';
```

```
GRANT RELOAD ON *.* TO 'flush'@'localhost';
```

这种类型的MySQL 账户对编写某些类型的管理脚本很有用处，比如在文件维护期间执行清除日志操作的管理脚本（参考12.8.7节）。

数据库级的权限适用于某个特定的数据库及其所有对象。在这一级进行权限分配的方法是，使用ON db_name.*子句：

```
CREATE USER 'bill'@'mamba.example.com' IDENTIFIED BY 'rock';
```

```
GRANT ALL ON sampdb.* TO 'bill'@'mamba.example.com';
```

```
CREATE USER 'reader'@'%' IDENTIFIED BY 'dirt';
```

```
GRANT SELECT ON menagerie.* TO 'reader'@'%';
```

上面第一组语句授予用户bill的权限是：当他从主机mamba.example.com连接服务器时，可以对sampdb数据库里的所有表执行所有操作。第二组语句会创建一个名为reader的受限用户，他可以从任何主机进行连接，并可以访问menagerie数据库里的所有表，但是只能使用SELECT语句。也就是说，reader是一个“只读”用户。

在进行授权时，可以同时列出多个权限，而各权限之间必须使用逗号隔开。例如，想要让某个用户能够读取和修改 sampdb 数据库里已有表的内容，但是不能创建新表或删除已有表，则不能将针对此数据库的ALL权限授给这位用户。实际上，只要指定需要启用的特定权限即可：

```
CREATE USER 'jennie'@'%' IDENTIFIED BY 'boron';
```

```
GRANT SELECT,INSERT,DELETE,UPDATE ON sampdb.* TO  
'jennie'@'%';
```

如果想要实现比数据库级更细的访问控制，则可以对各个表，甚至是表里的各个列，进行权限分配。当你想让表中某些列对某位用户不可见时，或者当某位用户只能修改特定的列时，使用各种与列特定相关的权限会很有用处。假设，你在美史联盟工作，现在有位志愿者想要做你的助理，为你减轻一点负担。于是，你决定让这位新助理拥有包含成员信息的member表的只读权限，并且拥有该表expiration列和与地址有关的各个列的UPDATE权限。这样，该助理所拥有的写访问权限便相当有限，仅可以在人们更新其成员资

格时更新其有效期，以及对他们的地址进行更改。用于建立这个MySQL账户的语句如下所示：

```
CREATE USER 'assistant'@'localhost' IDENTIFIED BY 'officehelp';
```

```
GRANT SELECT, UPDATE (expiration,street,city,state,zip)
```

```
ON sampdb.member TO 'assistant'@'localhost';
```

GRANT语句授予那位新助手的权限包括整个member表的读取权限（因为在SELECT后面没有列出任何列），以及紧跟在UPDATE关键字之后的括号里所列出的那些列的更新权限。

如果想要在一条GRANT语句里授予多个列级的权限，则必须在每种权限的名字后面放置一个括号，并把受该权限限制的那些列列在其中。

如果想让某个账户可以对存储例程（即函数和过程）进行操作，则可以使用全局级或数据库级的ALTER ROUTINE和EXECUTE权限，或者针对单个的例程进行权限分配（CREATE ROUTINE权限除外）：

```
CREATE USER 'wilbur'@'localhost' IDENTIFIED BY 'sulfur';
```

```
GRANT CREATE ROUTINE ON sampdb.* TO 'wilbur'@'localhost';
```

```
GRANT EXECUTE ON PROCEDURE sampdb.count_students TO  
'wilbur'@'localhost';
```

PROXY 权限不能应用于特定的操作。实际上，它可以让一个用户成为另一个用户的代理。代理用户拥有被代理用户的所有权限。下面这条语句能够让clint拥有bart的权限：

```
GRANT PROXY ON 'bart'@'localhost' TO 'clint'@'localhost';
```

授予PROXY权限的语法受一定的限制：必须单独指定它，不允许使用REQUIRE子句，并且WITH子句只允许与GRANT OPTION一起使用。更多信息请参考13.2.7节。

在GRANT语句里，如果需要将数据库名、表名或列名用引号引起来，则需要将它们作为标识符（不是作为字符串）引起来。例如：

```
GRANT SELECT, UPDATE (`expiration`,`street`,`city`,`state`,`zip`)
```

```
ON `sampdb`.`member` TO 'assistant'@'localhost';
```

权限表里的行不会“随同”数据库对象的重命名操作一起变化。例如，当更改某个给定表或列的名字时，所有与该表或列相关联的权限都不会被更新。

13.2.2.2 使用“无权限”的USAGE权限

权限说明符 `USAGE` 表示的是“无权限”。它可能看上去不是很有用，但其实不然。它可以让你更改某个账户的特性，而非那些与权限有关的特性，同时还可以保证原有的权限不受影响。`USAGE` 权限的具体用法是：在全局级“授予”此权限、指定账户名，并提供此账户非权限的新特性。例如，想要在不影响某个账户权限的前提下，改变账户密码，要求用户必须使用SSL进行连接，或者想对该账户的连接进行限制，那么可以使用下面这几条语句：

```
GRANT USAGE ON *.* TO account IDENTIFIED BY 'new_password';
```

```
GRANT USAGE ON *.* TO account REQUIRE SSL;
```

```
GRANT USAGE ON *.* TO account WITH  
MAX_CONNECTIONS_PER_HOUR 10;
```

13.2.2.3 要求账户使用安全连接

MySQL允许客户端使用SSL协议来建立安全连接，该协议会对客户端和服务端之间的数据流进行加密，使它们不会以明文形式进行传送。此外，X509可被用作客户端通过SSL连接提供身份认证信息的方式。安全连接提供了一层额外的保护方式，但是其代价就是需要额外的CPU周期来执行加密和解密操作。

可以用`REQUIRE`子句来指定安全连接需求。使用`REQUIRE SSL`则可以要求用户只用SSL进行连接，而不用过多地指定用户必须使用的安全连接类型：

```
CREATE USER 'eladio'@'%.example.com' IDENTIFIED BY 'flint';
```

```
GRANT ALL ON sampdb.* TO 'eladio'@'%.example.com' REQUIRE SSL;
```

如果想更严格一点，可以要求客户端提供一份有效的X509证书：

```
GRANT ALL ON sampdb.* TO 'eladio'@'%.example.com' REQUIRE X509;
```

REQUIRE X509 只会要求证书内容是有效的，而不会对其做过多约束。如果还想更严格一点，则可以使用REQUIRE要求客户端的X509证书必须具备某些特征。这些特征由REQUIRE子句中的ISSUER或SUBJECT选项进行设定。ISSUER和SUBJECT分别代表证书的签发者和持有者。例如，在sampdb发行包的ssl目录里，有一个客户端身份认证文件client-cert.pem，它可用来测试SSL连接。使用openssl命令可以显示出证书的签发者和持有者：

```
% openssl x509 -issuer -subject -noout -in client-cert.pem
```

```
issuer= /C=US/ST=WI/L=Madison/O=sampdb/OU=CA/CN=sampdb
```

```
subject= /C=US/ST=WI/L=Madison/O=sampdb/OU=client/CN=sampdb
```

下面这条GRANT语句表明，当客户端在使用该账户来进行连接时，必须提供与签发者和持有者相匹配的证书：

```
GRANT ALL ON sampdb.* TO 'eladio'@'%.example.com'
```

```
REQUIRE ISSUER
```

```
'/C=US/ST=WI/L=Madison/O=sampdb/OU=CA/CN=sampdb'
```

```
AND SUBJECT
```

```
'/C=US/ST=WI/L=Madison/O=sampdb/OU=client/CN=sampdb';
```

也可以使用REQUIRE子句来表明连接必须使用某种特定的密码类型来进行加密：

```
GRANT ALL ON sampdb.* TO 'eladio'@'%.example.com'
```

```
REQUIRE CIPHER 'DHE-RSA-AES256-SHA';
```

如果想要显式地表明无需使用安全连接，可以使用REQUIRE NONE。这是创建新账户时的默认值，但是对于当前拥有SSL要求的账户，可以通过它来去除该要求。

在使用REQUIRE子句时，还需注意以下几点。

在执行要求账户使用安全连接的GRANT语句时，只会对这个账户建立某种约束。它实际上并不会为客户端程序提供使用该账户进行安全连接的方式。为建立安全连接，必须配置MySQL，使其支持SSL，并且以某种方式启动服务器和客户端程序。更多相关信息请参考13.5节。

如果指定某个账户必须使用SSL进行连接，但服务器或客户端程序中的某一方不支持SSL，那么该账户最终将无法使用。

使用REQUIRE子句，只能表明某个账户是否必须使用安全连接进行连接。只要服务器和客户端程序的配置都支持SSL，即使没有要求使用安全连接，所有的用户仍然会被允许使用安全连接。

如果某个账户在连接服务器时不需要经过外部网络，那么对账户使用REQUIRE子句便没有多大意义。这种连接不会被窥探，因此对它们进行加密不会带来任何好处。像这样的账户所包含的连接类型只会涉及通过UNIX套接字文件、命名管道或共享内存与服务器建立的连接，或者与网络环回接口IP地址127.0.0.1（或::1）建立的连接。这些连接使用的接口都由主机内部处理，并且通信不会跨外部网络。

13.2.2.4 让账户拥有管理权限

如果想让某个账户把自己的权限转授给其他账户，可以指定WITH GRANT OPTION子句。但是，在使用此子句时，你自己必须要拥有GRANT OPTION权限。

让某个账户拥有GRANT OPTION权限的一个原因是，它可以让数据库的所有者控制对该数据库的访问。具体做法是，把对该数据库的全部权限授予其所有者，包括GRANT OPTION权限。例如，想让alicia用户能够从big-corp.com域里的任何一台主机连接服务器，并拥有对sales数据库的所有表进行管理的权限，那么可以像下面这样创建此账户：

```
CREATE USER 'alicia'@'%.big-corp.com' IDENTIFIED BY 'shale';
```

```
GRANT ALL ON sales.* TO 'alicia'@'%.big-corp.com' WITH GRANT  
OPTION;
```

事实上，WITH GRANT OPTION子句可让你把授权访问的权限委托给另一个用户。但是请注意，两个拥有GRANT OPTION权限的用户可以彼此将自己的权限授予对方。如果只向其中一个用户授予了SELECT权限，但是另外一个用户拥有GRANT OPTION权限以及除SELECT权限以外的其他权限，那么第二个用户便可以让第一个用户变得“更强”。

另一种授予GRANT OPTION权限的方法是，把它直接写在GRANT语句的开头部分：

```
GRANT GRANT OPTION ON sales.* TO 'alicia'@'%.big-corp.com';
```

不过，像下面这样的语句却行不通：

```
GRANT ALL,GRANT OPTION ON sales.* TO 'alicia'@'%big-corp.com';
```

在GRANT语句里，ALL只能单独使用，不能与其他权限说明符列在一起。

GRANT OPTION权限可以应用到等于或低于被授予权限级别的所有权限，而不是单个权限。如果你为某个账户授予了某个级别的GRANT OPTION权限，则此账户可以把自己在该级别拥有的所有权限进行转授。对该账户所能转授的具体权限，你无法进行明确地限定。

13.2.2.5 限制账户的资源占用

MySQL的授权系统让你可以对账户每小时连接服务器的次数、每小时可以执行的语句或进行更新的数量进行限制。用户无法通过多个服务器连接的方式来突破这些限制，因为给定账户的所有连接所涉及的语句都会算到该账户头上。

使用WITH子句可以指定这些限制。下面这条语句会建立一个账户，它对sampdb数据库拥有所有访问权限，但是每小时最多只允许连接10次和执行200条语句（其中最多只能更新50次）：

```
CREATE USER 'spike'@'localhost' IDENTIFIED BY 'pyrite';
```

```
GRANT ALL ON sampdb.* TO 'spike'@'localhost'
```

```
WITH MAX_CONNECTIONS_PER_HOUR 10  
MAX_QUERIES_PER_HOUR 200
```

```
MAX_UPDATES_PER_HOUR 50;
```

在WITH子句里，各个资源管理选项的顺序无关紧要。

每个选项的默认值都为零，其含义是“无限制”。因此，如果对某个账户进行了资源限制，那么可以通过把限制值设置为零的方式去除该限制。例如，下面这条语句将取消对spike每小时连接次数的限制：

```
GRANT USAGE ON *.* TO 'spike'@'localhost'
```

```
WITH MAX_CONNECTIONS_PER_HOUR 0;
```

资源上限MAX_USER_CONNECTIONS控制的是账户能够同时建立连接的最大数量。如果这个上限值为零（默认值），那么最大连接数由max_user_connections系统变量控制。该变量为非零值时会对账户能够同时建立的连接数进行限制。

所有拥有RELOAD权限的管理账户都能通过执行FLUSH USER_RESOURCES语句重置当前的计数值。使用FLUSH PRIVILEGES语句也可以。在计数值复位之后，那些达到其每小时上限的账户可以再次进行连接和执行语句。如果对某个账户执行了可以设置其资源上限的GRANT语句，则同样会重置其资源上限值。

13.2.3 显示账户权限

要查看账户所拥有的权限，可以使用SHOW GRANTS语句：

```
SHOW GRANTS FOR 'sampadm'@'localhost' ;
```

如果想查看自己的权限，可以使用下面两条语句当中的任何一条：

```
SHOW GRANTS;
```

```
SHOW GRANTS FOR CURRENT_USER ( ) ;
```

13.2.4 撤销权限

要撤销某个用户的部分或全部权限，可以使用REVOKE语句。REVOKE语句的语法与GRANT语句的有些相似，不同之处在于，前者用FROM子句替换了TO子句，并且没有auth_info、REQUIRE和WITH子句：

```
REVOKE privileges [(columns)] ON what FROM account;
```

例如，下面的GRANT语句会把对sampdb数据库的所有权限授予账户，而REVOKE语句会移除该账户对已有行进行修改的权限：

```
GRANT ALL ON sampdb.* TO 'boris'@'localhost';
```

```
REVOKE DELETE,UPDATE ON sampdb.* FROM 'boris'@'localhost';
```

GRANT OPTION权限并未包括在ALL权限里。要想从自账户权限中撤销它，可以在REVOKE语句中的privileges部分中显式指定它：

```
REVOKE GRANT OPTION ON sales.* FROM 'alicia'@'%big-corp.com';
```

如果想要撤销某个权限，你必须先自己拥有该权限，并且还必须拥有GRANT OPTION权限。下面这条语句可以撤销账户在各个级别所拥有的全部权限：

```
REVOKE ALL, GRANT OPTION FROM account;
```

请注意，这个语法里没有ON子句。在执行它时，必须具备全局级权限CREATE USER或者拥有对mysql数据库的INSERT权限。

如果你撤销账户在数据库、表、列或例程级别上的所有权限，或者撤销它的代理权限，那么MySQL会分别删除权限表db、tables_priv、columns_priv、procs_priv或proxies_priv里与该账户相对应的行。撤销账户的所有全局级权限，会将其在user表里对应的那一行的权限列设置为'N'，而不会删除该行。也就是说，REVOKE语句不会彻底删除一个账户，这意味着该账户仍然能够连接服务器。如果想要彻底删除某个账户，则必须使用DROP USER语句，而不是REVOKE语句（参考13.2.1节）。

有些矛盾的是，有些“撤销”操作必须使用GRANT语句。例如，某个账户必须使用SSL进行连接，但是没有一种REVOKE语法能够撤销该需求。这时，就需要执行GRANT语句，授予账户全局级的USAGE权限（已有权限保持不变），并在其中包含一个REQUIRE NONE子句，以表明不再需要使用SSL进行连接：

```
GRANT USAGE ON *.* TO account REQUIRE NONE;
```

类似地，如果要限制某个用户的资源占用，那么也不能使用REVOKE语句来删除这些限制。这时，只能使用带USAGE的GRANT语句将那个上限值设置为零（即“无限制”）：

```
GRANT USAGE ON *.* TO account
```

```
WITH MAX_CONNECTIONS_PER_HOUR 0  
MAX_QUERIES_PER_HOUR 0
```

```
MAX_UPDATES_PER_HOUR 0;
```

13.2.5 更改密码或重置丢失的密码

更改或重置账户密码的一种办法是，使用UPDATE语句来标识该账户在User表行中的Host值和User值，然后刷新权限：

```
mysql> USE mysql;
```



```
mysql> UPDATE user SET Password=PASSWORD ('silicon')
```

```
-> WHERE User='boris' AND Host='localhost';
```

```
mysql> FLUSH PRIVILEGES;
```

不过，使用SET PASSWORD语句更简单，因为在指定账户时使用的格式与其他账户管理语句的格式相同，并且它不用显式地刷新权限：

```
mysql> SET PASSWORD FOR 'boris'@'localhost' = PASSWORD ('silicon') ;
```

只要你不以匿名用户身份进行连接，便总是可以使用SET PASSWORD语句来修改自己的密码。如果想要修改其他账户的密码，那么你必须拥有mysql数据库的UPDATE权限。

另一种不太常用的密码修改方法是，使用带IDENTIFIED BY子句的GRANT USAGE语句。此时，可以以明文方式指定密码，而不必使用PASSWORD()函数：

```
mysql> GRANT USAGE ON *.* TO 'boris'@'localhost' IDENTIFIED BY  
'silicon';
```

如果你因忘记root 密码而需要重置它，并且无法连接服务器，那么你会遇上大麻烦，因为通常你必须先以root 身份连接服务器才能修改root 密码。如果你不知道密码，则需要先强行停止服务器，然后在不使用权限表进行验证的情况下重启服务器。相关操作过程请参考12.2.6节。

13.2.6 避免访问控制风险

本节将描述在权限分配时需要注意的一些事项，以及因选择不当而带来的各种风险。

避免创建匿名账户（即拥有空用户名的账户）。即使他们没有足够的权限直接造成破坏，允许匿名用户进行连接也可能会使它拥有一些访问权限，从而能够到处收集各种信息（如你所拥有的数据库和表），或者使用SHOW STATUS和SHOW VARIABLES来监视服务器的运行状况。

把不需要身份认证的账户都找出来，然后选择是删除它们，还是要求它们进行身份认证。想要把没有密码也不使用身份认证插件的账户查找出来，需要使用下面这个查询：

```
SELECT Host, User FROM mysql.user WHERE Password = '' AND plugin = '';
```

要删除这类账户，可以使用DROP USER语句。如果要求进行认证，则可以为其分配一个密码：

```
SET PASSWORD FOR account = PASSWORD ('password') ;
```

不要允许用原始的（MySQL 4.1之前）散列格式存储密码，要将它们更改为目前更为安全的散列格式。对于以旧格式存储的那些值，其长度为 16，并且不是以 “*” 字符打头，因此使用下面两条语句中的任何一条便可以轻易地将那些使用旧格式的账户标识出来：

```
SELECT Host, User FROM mysql.user WHERE LENGTH (Password) = 16;
```

```
SELECT Host, User FROM mysql.user WHERE Password NOT LIKE '*/';
```

如果这些语句标识出了某些账户，那么：

（1）如果old_passwords系统变量被设置为1（即为ON），则在重启服务器时禁用它。

（2）随后在重启服务器时，总是启用secure_auth 系统变量。这样可以防止客户端使用OLD_PASSWORD（）把密码重置为旧格式，并防止它使用旧格式密码进行连接。自MySQL 5.6.5起，就不需要这样做了，因为secure_auth在默认情况下就是启用的。

（3）使用SET PASSWORD对拥有旧格式密码的所有账户进行密码重置。

把全局变量sql_mode的值设置为NO_AUTO_CREATE_USER模式，以防止GRANT语句创建不安全的新账户。也就是说，如果某个账户不存在，那么GRANT会执行失败，并且不会创建账户，除非该语句包含一个指定有非空密码的IDENTIFIED BY子句，或者包含一个指定身份认证插件的IDENTIFIED WITH子句。启用NO_AUTO_CREATE_USER并不能阻止客户端在会话过程中禁用该模式，但有助于防止非恶意用户犯错。

除非真的需要在主机名说明符里使用模式，否则，在设置账户时要避免这样做。在扩大给定用户的连接主机范围的同时，也会扩大冒名顶替者的闯入范围。

超级用户权限需要小范围授予。也就是说，不要在user 表的行里启用这些权限。这些权限都是全局性的，会使该用户对服务器的运作产生影响，或者使他能够对所有数据库进行访问。例如，在user表的某个行里启用了DELETE权限，那么与该行相关联的那个账户便可以在任何数据库里删除任何表的任何行。因为在user 表里指定的权限都是超级用户权限，所以在一般情

况下最好是不授予全局权限，可以授予特定级别的权限，以限制用户只能访问特定的数据库或数据库对象，如表或存储例程。对于这一原则，有如下两个例外情况。

首先，像root这样的超级用户和其他管理账户需要全局权限来操作服务器。这些账户往往很少。

其次，有些特定的全局权限通常可以安全地授予。如与临时表创建、表锁定，以及（在可能的情况下）能够使用SHOW DATABASES语句等操作有关的权限。许多安装都会授予这些权限，而其他需要严格控制的权限则不会被随便授予。

不要授予mysql数据库的权限。因为这个数据库包含授权表，对该数据库拥有权限的用户可以修改其中的表，从而获得对其他任何数据库的权限。事实上，授予能让用户修改mysql数据库表的权限，便等同于给了该用户所有的全局权限。如果该用户能够直接修改表，也就意味着他能够执行所有你能想到的账户管理语句。

一定要小心使用GRANT OPTION权限。对于两个拥有不同权限的用户，如果都拥有GRANT OPTION权限，则他们可以让彼此的访问权限变得更强大。

FILE权限特别危险，不要輕易地将它授予其他用户。下面这个示例可以说明拥有FILE权限的用户能够做些什么：

```
CREATE TABLE etc_passwd (pwd_entry TEXT);
```

```
LOAD DATA INFILE '/etc/passwd' INTO TABLE etc_passwd;
```

在执行这些语句之后，该用户便可以通过执行 SELECT 语句，访问到存储在服务器主机上的密码文件的内容：

```
SELECT * FROM etc_passwd;
```

那条LOAD DATA语句里的/etc/passwd可被替换为服务器主机上的任何一个公共可读文件的名字。如果用户是从远程主机进行连接的，其最终效果便是，在授予用户 FILE 权限时，该用户便可以通过网络访问服务器主机上潜在的很大一部分文件系统。

如果在权限上对系统里的那个数据目录限制不够严格，那么 FILE 权限还可能对其中的数据库产生威胁。这也是应该将数据目录的内容设置为只能由服务器读取的一个主要原因。如果与表相对应的那些文件都可以被公开读取，那么在服务器上拥有账户的任何用户都可以读取它们，不仅如此，具有

FILE 权限的任何客户端用户也可以通过网络连接读取到它们！下面几个步骤演示了具体的做法。

(1) 创建一个包含LONGBLOB列的表：

```
USE test;
```

```
CREATE TABLE tmp (b LONGBLOB);
```

(2) 使用该表来读取每个与你想要窃取的那个表相对应的文件的内容。假设，某个用户在other_db数据库里有一个名为x的MyISAM表。该表由x.frm、x.MYD和x.MYI三个文件表示。像下面这样读取这些文件，并将它们复制为test数据库的相应文件：

```
LOAD DATA INFILE './other_db/x.frm' INTO TABLE tmp
```

```
FIELDS ESCAPED BY " LINES TERMINATED BY ";
```

```
SELECT * FROM tmp INTO OUTFILE 'x.frm'
```

```
FIELDS ESCAPED BY " LINES TERMINATED BY ";
```

```
DELETE FROM tmp;
```

```
LOAD DATA INFILE './other_db/x.MYD' INTO TABLE tmp
```

```
FIELDS ESCAPED BY " LINES TERMINATED BY ";
```

```
SELECT * FROM tmp INTO OUTFILE 'x.MYD'
```

```
FIELDS ESCAPED BY " LINES TERMINATED BY ";
```

```
DELETE FROM tmp;
```

```
LOAD DATA INFILE './other_db/x.MYI' INTO TABLE tmp
```

```
FIELDS ESCAPED BY " LINES TERMINATED BY ";
```

```
SELECT * FROM tmp INTO OUTFILE 'x.MYI'
```

```
FIELDS ESCAPED BY " LINES TERMINATED BY ";
```

(3) 在执行这些语句之后，test数据库目录也会包含名为x.frm、x.MYD和x.MYI的文件。换句话说，test数据库包含了x表，而这个表恰好就是从other_db数据库里窃取来的那个表的副本。

为防止这种攻击，可以使用13.1.2节描述的指导方法设置数据目录的许可权限。另外还需要采取一个措施，即避免将 SHOW DATABASE 权限授予其他用户，并且在运行服务器时带上--skip-show-database 选项。这样，可以防止用户对那些他们没有访问权限的数据库使用SHOW DATABASES和SHOW TABLES命令，进而使用户无法找到与他们不该访问的那些数据库和表有关的信息。

如果以root用户身份运行MySQL服务器，那么FILE权限的危险会被进一步放大。首先，那种做法就很不明智，更何况还有一个FILE权限，其带来的后果可想而知。因为root用户可以在文件系统里的任何位置创建文件，而具有FILE 权限的用户（即使是从远程主机进行连接的）也可以让服务器这样做。服务器不会创建已存在的文件，但是有时也可能创建出改变服务器主机操作、或者对服务器的安全产生危害的新文件。例如，只要文件/etc/resolv.conf、/etc/hosts.equiv、/etc/hosts.lpd 或/etc/sudoers中的某一个不存在，那么能够让MySQL服务器创建这些文件的用户便可以完全更改服务器主机的行为。

为避免这些问题，请不要以root身份运行mysqld。（更多相关信息请参考12.2.1.1节。）PROCESS和SUPER权限只应被授予那些被信任的MySQL账户。拥有PROCESS权限的用户，可以使用SHOW PROCESSLIST来查看服务器正在执行的那些语句的文本。这使得用户可以窥探其他用户，从而有可能看到应该保持私密的信息。拥有SUPER权限的用户，可以取消其他用户的语句。利用SUPER权限，用户还可以清空日志文件、执行其他危害服务器运作的操作。

请不要把RELOAD权限授予那些不需要它的人。RELOAD权限允许用户执行FLUSH和RESET语句，而这些语句很可能会被滥用。例如：

二进制日志文件和中继日志文件的名字都是由编号序列构成的。如果启用了这些日志，那么每一条FLUSH LOGS语句都会创建序列中的下一个文件。拥有RELOAD权限的用户可以反复地刷新日志，这会导致服务器创建大量的文件。

拥有RELOAD权限的用户可以通过使用FLUSH PRIVILEGES或FLUSH USER_RESOURCES语句重新加载权限表，使资源管理机制失效。这两条语句都可以将资源管理计数器重置为零。

FLUSH TABLES语句会导致服务器将其打开表的缓存保存至磁盘。反复使用它时，会阻止服务器利用缓存，从而降低服务器的性能。类似地，反复使用RESET QUERY CACHE则会失去查询缓存所带来的好处。

RESET MASTER语句会导致复制主服务器删除其所有的二进制日志文件，即使这些文件仍在使用中也会被删除，这会阻止从服务器正确地复制。

ALTER权限的某些使用方式可能是你意想不到的。假设，你想让某个用户能够访问table1，但是不能访问table2。另一个拥有ALTER权限的用户则可以通过ALTER TABLE将table2更名为table1，从而与你的初衷相违背。

13.2.7 插入式身份认证和代理用户

正如13.2.1.3节所述，在为CREATE USER和GRANT语句提供身份认证信息时，既可以使用IDENTIFIED BY来指定密码，也可以使用IDENTIFIED WITH来指定身份认证插件。本节将对如何使用身份认证插件提供更详细的信息。其中，还将涉及代理用户——这是认证插件所带来的一项附加功能，它允许一个用户获得另一用户的权限。

13.2.7.1 使用身份认证插件

在与 CREATE USER 一起用于指定身份认证插件时，IDENTIFIED WITH 子句的语法格式如下所示（与GRANT的语法格式相同）：

```
CREATE USER account IDENTIFIED WITH auth_plugin [AS 'auth_string']
```

为了让服务器能知道某个身份认证插件的存在，必须先加载它。如果该操作尚未完成，则可以根据12.4节的操作指导加载它。假设auth_plugin已被加载，服务器在user权限表里创建了一个行，并将 auth_plugin和auth_string的值分别存储在了该行的plugin 列和authentication_string列。

当客户端通过该账户尝试连接服务器时，服务器根据user表的plugin值就会知道应该通过指定的插件对该客户端进行认证，接着会调用该插件，并将 authentication_string的值传递给它。该插件会与客户端程序进行通信，以接收客户端的身份认证凭证，并向服务器表明客户端身份认证是否正确。如果不正确，服务器会拒绝该连接。

对于服务器端的身份认证插件，在客户端都有对应的插件，并且每个服务器端插件都会与客户端程序进行通信，确定应该调用哪个客户端插件执行身份认证过程。正因为如此，当用户连接服务器时，并不需要指定使用哪个客户端插件。但是，如果客户端主机的插件目录里没有安装客户端插件，连接就会失败。如果插件位于其他地方，没在客户端程序在默认情况下会去搜索的地方，那么调用客户端程序时可以使用--plugin-dir选项来指定该位置。

如果用`--skip-grant-tables`选项启动服务器，则不会使用身份认证插件，因为这时服务器允许任何客户端无需进行身份认证便可以进行连接。

13.2.7.2 创建代理用户

MySQL里的身份认证插件使用户代理成为可能。这样一来，连接到服务器的用户A（即代理用户），就可以被当成另一个用户B（被代理用户）。如果用户A是用户B的代理，并且现在使用A的账户来连接服务器，那么用户A便可以拥有用户B的所有权限，并且可以执行用户B被允许执行的所有操作。

并不是所有身份认证插件都支持代理。为支持代理功能，在编写插件时，不仅要把用户A是否认证通过返回给服务器，还要返回被代理用户B的名字（用于检查用户A应该具有的权限）。插件如何确定是否需要将代理用户映射成另一个被代理用户以及以何种方式进行映射，完全依赖于插件的具体实现。通常情况下，插件会使用在创建代理用户账户时指定的那个`auth_string`的值。也就是说，当代理用户进行连接时，插件会检查那个认证字符串，并解释其内容，以确定如何将该用户映射到某一个被代理用户。假设，像下面这样创建了一个账户：

```
CREATE USER "@'localhost  
  
IDENTIFIED WITH auth_plugin  
  
AS 'user1=proxied_user1;user2=proxied_user2';
```

在这种情况下，账户名将使用空用户名，并且会匹配来自本地主机的各种连接尝试，其中，用户名与某些更为具体的`localhost`账户实际上并不匹配。对于这样的连接尝试，如果客户端将`--user`的值指定为`user1`或`user2`，那么插件可能会将该客户端分别映射为`proxied_user1`或`proxied_user2`，并且会拒绝来自其他客户端用户名的连接尝试。

目前，只有商业MySQL的发行版本才有支持代理的插件。（更多详细信息请参考MySQL参考手册。）社区的MySQL发行版本并没有这种插件。因此，接下来讨论的内容是如何使用某个假想的插件来创建代理用户。

假设，我们有一个名为`unix_auth`的身份认证插件，它是专为Unix系统而设计的，它的工作方式如下所示。

此插件允许用户使用自己的Unix账户密码来连接MySQL服务器。在使用它时，可以不使用用户代理，但是它支持这种功能。

如果想在无代理的情况下使用此插件，那么在创建账户时就不要指定身份认证字符串：

```
CREATE USER 'user1'@'localhost' IDENTIFIED WITH unix_auth;
```

在这种情况下，user1的连接过程为，先指定user1的Unix密码，然后使用同名的MySQL账户进行身份认证：

```
% mysql -p
```

```
Enter password: ...此处输入Unix用户user1的密码...
```

在命令行里并没有指定用户名。我们假设客户端插件可以通过检查执行环境来确定当前的Unix登录名，并发送到服务器。

如果想在使用插件时支持代理，则可以指定一个身份认证字符串，并在其中列出客户端应该代理的那个MySQL用户的名字。例如：

```
CREATE USER 'user2'@'localhost' IDENTIFIED WITH unix_auth AS  
'my_user';
```

此时，user2会使用user2的Unix密码来连接服务器，而代理后的MySQL账户为my_user。

前面的讨论还不够全面。在代理时，除了需要支持代理的身份验证插件和针对代理用户的账户之外，还要求被代理用户所对应的账户必须存在，并且代理用户对被代理账户需要拥有PROXY权限。下面是一组允许代理时常用的语句序列：

```
CREATE USER proxy_user auth_info;
```

```
CREATE USER proxied_user auth_info;
```

```
GRANT PROXY ON proxied_user TO proxy_user;
```

对于代理用户，auth_info 子句必须指定一个支持用户代理的插件。对于被代理用户，该子句既可以指定一个密码，也可以指定一个身份认证插件。

用于将user2创建为my_user的代理用户的所有语句如下所示：


```
CREATE USER 'user2'@'localhost' IDENTIFIED WITH unix_auth AS  
'my_user';
```

```
CREATE USER 'my_user'@'localhost' IDENTIFIED BY 'my_user_password';
```

```
GRANT PROXY ON 'my_user'@'localhost' TO 'user2'@'localhost';
```

现在，当user2连接MySQL服务器时，不管my_user拥有什么样的权限，在对用户会话进行审核的过程中都会执行这些语句。（请注意，我们为被代理账户分配了一个密码。这样可以防止客户端在未认证的情况下直接连接到该账户。）

你可能会问，为何要这么费力地把user2映射到my_user。为何不直接把权限分配给user2呢？事实上，如果只需要代理一个用户，那么不会有什么问题。但是，假设你有很多Unix用户需要被当作 my_user。于是，你可能会像下面这样为每一个用户创建账户，把它们都映射到my_user：

```
CREATE USER 'user3'@'localhost' IDENTIFIED WITH unix_auth AS  
'my_user';
```

```
GRANT PROXY ON 'my_user'@'localhost' TO 'user3'@'localhost';
```

```
CREATE USER 'user4'@'localhost' IDENTIFIED WITH unix_auth AS  
'my_user';
```

```
GRANT PROXY ON 'my_user'@'localhost' TO 'user4'@'localhost';
```

...

或者，你可能会想要将所有还没有MySQL账户的Unix用户映射到被代理用户my_user。此时，可以建立一个通用或者默认的代理用户来简化整个映射过程：

```
CREATE USER ""@'localhost' IDENTIFIED WITH unixauth AS 'my_user';
```

```
GRANT PROXY ON 'my_user'@'localhost' TO ""@'localhost';
```

将多个代理用户映射到一个被代理用户的方法可以简化权限管理：通过更新被代理账户，便可以同时更新所有人的权限。否则，必须单独更新每个账户的权限。

13.3 权限表结构和内容

MySQL的访问控制系统非常灵活，允许你以各种不同的方法来设置用户权限。通常情况下，可以使用像CREATE USER、GRANT和REVOKE这样的账户管理

语句来完成此项工作，它们会代表你来修改控制客户端访问的各种权限表。不过，你可能会遇到这样的情况：用户权限似乎并没有按你的意图行事。在这种情况下，如果对 MySQL 权限表的结构，以及服务器是如何使用它们来确定访问权限的相关事宜有所了解，则会对上述情况有所帮助。在了解这些内容之后，你便可以通过直接修改权限表的方式来增加、删除或修改用户权限。在你检查这些表时，也能够对各类权限问题进行诊断。

这里假设你已阅读了 13.2.1 节，并且对各种账户管理语句的工作原理已有所了解。这些语句只是提供了一个便捷的“前端”，让你能够建立 MySQL 账户，并为它们分配各种权限，而所有实际动作都是在 MySQL 权限表里进行的。

这些权限表控制着客户端（通过网络与服务器建立连接）对 MySQL 数据库的访问。这些表位于 mysql 数据库里，会在将 MySQL 第一次安装至机器的过程中被初始化。这些表的名字分别是 user、db、tables_priv、columns_priv、procs_priv 和 proxies_priv。服务器对它们的使用情况如下所述。

user 表列出的是可以连接至服务器的各个用户账户，以及每个用户所拥有的全局（超级用户）权限（如果有的话）。需要了解的重点是，在 user 表里启用的所有权限都是全局权限，适用于所有数据库。与如何安全授予这些权限有关的信息请参考 13.2.6 节。

user 表还包含用于身份认证的列，用于使用 SSL 建立安全连接的 SSL 选项列，以及用于防止给定账户独占该服务器的资源管理列。

db 表列出的是哪些账户对哪些数据库拥有权限。此处授予的权限可应用于数据库内的所有对象（其中包括表、存储例程等）。

tables_priv 表列出的是各种表级别的权限。此处指定的权限适用于表里的所有列。

columns_priv 表列出的是列级别的权限。此处指定的权限适用于表中的特定列。

procs_priv 表包含的是适用于各种存储例程（即存储函数和存储过程）的权限。此处指定的权限适用于数据库里的特定例程。

proxies_priv 表表明的是哪些账户可以成为其他账户的代理，并获得它们的权限。

mysql 数据库还有一个名为 host 的权限表——它已过时，因此本书不再对它多加讨论。

下面的几张表格（表 13-3～表 13-6）展示了每个权限表的结构（因版面所限，部分表格按照列的类型进行了拆解）。每个权限表包含两种基本类型的列：一个是访问范围（scope-of-access）列，用于确定何时应用于行；另一个是权限列，用于确定某个行授予了哪些权限。对于系统管理操作和那些与某种特定类型的数据库对象有关的操作，这些权限列可以被进一步细分。user表还有几个用于身份认证、SSL连接和资源管理的列。这些列只存在于 user 表里，因为其作用范围是全局性的。有些权限表还包含其他的杂项列，但是它们并不是我们此处关心的重点，因为它们与账户管理无关。

表13-3 权限表访问范围列

user表	db表	tables_priv表	columns_priv表	procs_priv表	proxies_priv表
Host	Host	Host	Host	Host	Host
User	User	User	User	User	User
	Db	Db	Db	Db	Proxied_host
		Table_name	Table_name	Routine_name	Proxied_user
			Column_name	Routine_type	

表13-4 权限表管理权限列

user表	db表	poxies_priv表
Create_user_priv		
File_priv		
Grant_priv	Grant_priv	With_grant
Process_priv		
Reload_priv		
Repl_client_priv		
Repl_slave_priv		
Show_db_priv		
Shutdown_priv		
Super_priv		

表13-5 权限表对象权限列

user表		db表	
Alter_priv		Alter_priv	
Alter_routine_priv		Alter_routine_priv	
Create_priv		Create_priv	
Create_routine_priv		Create_routine_priv	
Create_tablespace_priv			
Create_tmp_table_priv		Create_tmp_table_priv	
Create_view_priv		Create_view_priv	
Delete_priv		Delete_priv	
Drop_priv		Drop_priv	
Event_priv		Event_priv	
Execute_priv		Execute_priv	
Index_priv		Index_priv	
Insert_priv		Insert_priv	
Lock_tables_priv		Lock_tables_priv	
References_priv		References_priv	
Select_priv		Select_priv	
Show_view_priv		Show_view_priv	
Trigger_priv		Trigger_priv	
Update_priv		Update_priv	
tables_priv表		columns_priv表	procs_priv表
Table_priv			Proc_priv
Column_priv		Column_priv	

表13-6 user表的身份认证列、SSL列和资源管理列

身份认证列	SSL列	资源管理列
Password	ssl_type	max_connections
plugin	ssl_cipher	max_questions
authentication_string	x509_issuer	max_updates
	x509_subject	max_user_connections

权限系统包含的表有`tables_priv`、`columns_priv`和`procs_priv`，分别用于定义以下各项的权限：特定表、列、存储函数和存储过程。因为 MySQL 不提供行级的权限，所以没有`rows_priv`表。例如，无法限制用户只访问那些其列中含有特定值的表中的行。需要这种功能的应用程序必须自己去实现。实现这一功能的一种方法是，创建一个使用`WITH CHECK OPTION`来定义的视图，并为这个视图分配一定的权限。另一种方法是，使用类似 `GET_LOCK()` 和 `RELEASE_LOCK()` 这样的咨询锁定（`advisory locking`）函数实现协同式的行级锁定。相关过程请参考C. 2. 8节。

新版本的MySQL有时会添加新的权限表或权限列。例如，在MySQL 5.5.7里，为实现PROXY权限而引入了 `proxies_priv` 表。当把已安装的MySQL升级至这个版本时，需要在使用这些新权限之前升级这些权限表。相关过程请参考13.2节。

13.3.1 权限表访问范围列

当某个账户试图执行某个给定操作时，访问范围列的操作决定了服务器会使用哪些行来确定最终的权限。权限表的每个行都包括Host列和User列，用于表明该行适用于由特定用户从给定主机发起的连接。例如，对于 `user` 表的某个行，如果其 Host 列和User 列中包含有localhost和bill，那么它会被用于由bill从本地主机发起的连接，而不会被用于由betty发起的连接。其他表还包含一些附加的访问范围列。`db`表包含有一个Db列，用于表明该行适用于哪个数据库。类似地，`tables_priv`表和`columns_priv`表的行里包含的访问范围列，使其访问范围进一步变窄，分别限定在了数据库里的特定表和表里的特定列中。`procs_priv` 表的访问范围列则指定了一个行适用于哪个存储函数或存储过程。

13.3.2 权限表权限列

权限表还包含权限列。对于每一个行，这些权限列表明的是，由访问范围列标识出的那个用户拥有哪些权限。与MySQL所支持的各种权限有关的更多信息请参考13.2.2.1节。在大部分情况下，这些权限名与权限表的权限列名一致。例如，SELECT权限便对应于`Select_priv`列。

在user表和db表里，指定的每种权限都是一个单独列。这些列都被定义为ENUM（'N'，'Y'）类型，其默认值为'N'（即关闭）。例如，Select_priv列的定义如下：

```
Select_priv ENUM（'N','Y'） CHARACTER SET utf8 NOT NULL  
DEFAULT 'N'
```

表tables_priv、columns_priv和procs_priv里的权限都是用SET表示的，从而可以将储存在同一列里的各种权限进行任意组合。tables_priv表的Table_priv列的定义如下：

```
SET（'Select','Insert','Update','Delete','Create','Drop','Grant',  
  
'References','Index','Alter','Create_view','Show_view','Trigger'  
'）  
  
CHARACTER SET utf8 NOT NULL DEFAULT ''
```

columns_priv表的Column_priv列的定义如下：

```
SET（'Select','Insert','Update','References'）  
  
CHARACTER SET utf8 NOT NULL DEFAULT ''
```

列权限的数量比表权限的少一些，因为列级别上有意义的操作较少。例如，可以从表里删除一行，但是不能从一行里删除单个的列。

请注意，INSERT是列级权限。如果你只对表的某些列拥有INSERT权限，那么在插入新行时只能指定这些列的值，而其他各列将被设置为各自的默认值。

procs_priv表里的Proc_priv列的定义如下：

```
SET（'Execute','Alter Routine','Grant'）  
  
CHARACTER SET utf8 NOT NULL DEFAULT ''
```

表tables_priv、columns_priv和procs_priv比表user和db更新，这也是为什么它们会使用更有效的SET表示方式来在单个列里列出多个权限的原因。

user表有一些其他权限表（如File_priv、Process_priv、Reload_priv和Shutdown_priv）没有的管理权限列。这些权限仅存在于 user 表里，因为

它们都是全局权限，与任何特定数据库或表无关。例如，基于默认数据库是哪一个来允许（或不允许）用户关闭服务器是没有意义的。

`proxies_priv`表在表示PROXY权限关系时，会以User列和Host列来指明代理用户账户，以Proxied_user列和Proxied_host列来指明被代理用户账户。

13.3.3 权限表身份认证列

`user` 表有 3个用于指明账户认证方式的列：`Password`、`plugin`和`authentication_string`。对于某个给定账户在`user` 表里对应的行，如果`plugin` 列为空，那么客户端在进行账户认证时会使用`Password`列里的密码。密码可以为空白（空），也可以不为空白；并且不允许使用通配符。空白密码的意思并不是与任何密码都匹配，而是表示用户必须不指定密码。

如果`plugin`列非空，客户端会根据指定插件所实现的身份认证方法来进行认证，并且服务器会将`authentication_string`值作为信息传递给该插件。更多相关信息请参考13.2.1.3节。

密码都是以加密形式（而不是以明文形式）存储的。如果以明文形式将密码存储在`Password`列里，用户将无法进行连接！`CREATE USER`语句、`GRANT`语句和`mysqladmin password`命令可以实现密码的自动加密。如果使用像`INSERT`、`REPLACE`、`UPDATE`或`SET PASSWORD`这样的语句来修改权限表，则密码的指定形式为`PASSWORD ('new_password')`，而不是`'new_password'`。

13.3.4 权限表SSL相关列

在`user` 表里，有几个列适用于基于 SSL的安全认证（参考 13.5 节）。最主要的列是`ssl_type`，它表明账户是否需要安全连接，如若需要，需要什么类型的安全连接。`ssl_type`列的类型为ENUM，有4种可能的取值：

```
ENUM('','ANY','X509','SPECIFIED') CHARACTER SET utf8 NOT NULL  
DEFAULT ''
```

`ssl_type`列的各个枚举值分别具有以下含义。

`''`（空串）。表示不要求账户使用安全连接。它是默认值，用在建立账户时未指定`REQUIRE`子句，或者显式指定了`REQUIRE NONE`的情况下。

`'ANY'`。表示账户必须使用安全连接，不过它可以是任何一种安全连接，这是一种“通用”要求。当在`GRANT`语句里指定`REQUIRE SSL`时，列会被设置为这个值。

'X509'。表示账户必须使用安全连接，而且客户端必须提供一个有效的X509证书。证书的内容是什么无关紧要。当你指定REQUIRE X509时，列会被设置为这个值。

'SPECIFIED'。表示客户端必须使用满足特定要求的安全连接。当REQUIRE子句里指定的是ISSUER、SUBJECT或CIPHER的组合值时，列会被设置为这个值。

对于所有除'SPECIFIED'以外的ssl_type 值，服务器会在验证客户端的连接尝试时，忽略其他SSL相关列里的值。对于'SPECIFIED'，服务器会检查其他列；而对于所有非空值，客户端都必须提供相匹配的信息。

ssl_cipher。如果为非空值，则表示客户端在连接时必须使用密码(cipher)方法。它可用于防止客户端使用脆弱的密码方法。

x509_issuer。如果为非空值，则表示在客户端提供的X509证书里必须有颁发者。

x509_subject。如果为非空值，则表示在客户端提供的X509证书里必须有标题。

在user表里，ssl_cipher、x509_issuer和x509_subject都是BLOB类型的列。

13.3.5 权限表资源管理列

在user表里，可以利用下面各列对任何给定MySQL账户的可用服务器资源的大小进行限定。

max_connections。表示允许给定账户在一小时内连接服务器的最大次数。当该值为0时，表示“无限制”。虽然其名字与max_connections系统变量同名，但两者毫无关联。

max_questions。表示允许给定账户在一小时内执行语句的最大数目。当该值为0时，表示“无限制”。

max_updates。其作用与max_questions相似，但更为具体。它适用于那些修改数据的语句。当该值为0时，表示“无限制”。

max_user_connections。表示允许给定账户同时进行连接的客户端的最大数目。如果该值为0，服务器会根据max_user_connections系统变量的全局值来确定并发连接的上限值。

如果该值大于0，则其优先级会高于`max_user_connections`系统变量。

如果重新启动MySQL服务器，那么所有当前计数会被重置为零。当重新加载权限表或者执行`FLUSH USER_RESOURCES`语句时，也会进行重置（`max_user_connections`值除外）。

与账户限制设置有关的更多信息请参考13.2.2.5节。

13.4 服务器如何控制客户端访问

MySQL服务器对客户端的访问控制分为两个阶段。第一阶段发生在你试图连接服务器的时候。此时，服务器会查看 `user` 表，看能否找到这样一个行，即与你用来连接服务器的主机、你的名字以及你的身份认证凭证（如密码）相匹配的行。如果找不到相匹配的行，你就无法连接。如果能找到，服务器还会检查`user`表的`SSL`列和资源管理列。

如果你已达到每小时连接数目限制或同时连接数目限制，服务器会拒绝该连接。

如果`user`表的行表明需要进行安全连接，那么服务器会确定你所提供的凭证与`SSL`相关列要求的凭证是否相匹配。如果不匹配，服务器会拒绝该连接。

如果所有检查都没问题，那么服务器会建立连接，你可以进入第二阶段。如果你建立的是安全连接，客户端程序和服务器还会对它们之间的通信进行加密。

在第二阶段，服务器会对执行的每一条语句做两项检查。首先，它会检查“每小时执行语句数目”和“每小时更新数目”的限制。其次，服务器会检查权限表，以验证你是否有足够的执行该语句的访问权限。首先检查限制值的原因在于：如果你已经达到上限，就没必要再检查你的权限了。第二阶段将一直持续到你与服务器的连接断开为止。

接下来将详细讨论 MySQL 服务器将权限表的行与以下两项相匹配的规则：连入客户端请求，语句。其中包括权限表访问范围列里的合法值的类型、来自不同权限表的各个权限值如何进行组合，以及服务器在给定权限表内搜索行的顺序。

13.4.1 访问范围列的内容

每个访问范围列都有一些约束规则，用来定义哪些类型值是合法的，以及服务器会如何解释这些值。有些范围列要求的是文字值，但大部分都允许

通配符或其他特定值。

Host

Host列的值可以是主机名或IP地址（IPv4或IPv6都可以）。值localhost代表本地主机。当客户端从本地主机连接一个服务器的本地网络接口时，便属于这一情况，定义如下。

Unix套接字文件，在Unix上。

命名管道或共享内存，在Windows上。

TCP环回接口，即IP地址为127.0.0.1或::1的那个接口。它在所有系统上都有效。如果客户端使用主机的真实名字或IP地址进行连接，就不会与localhost相匹配。假设，本地主机名为cobra.example.com，并且user表里有两个行与名为bob的用户相对应：一个行的Host 值为 localhost，另一个行的值为 cobra.example.com。如果bob在Unix或Windows里使用以下命令中的任意一个从本地主机进行连接，就能与值为localhost的那个行相匹配：

```
% mysql -p -u bob -h localhost
```

```
% mysql -p -u bob -h 127.0.0.1
```

```
% mysql -p -u bob -h ::1
```

此外，如果bob在Windows上像下面这样进行连接，就能与localhost行相匹配：

```
C:\> mysql -p -u bob -h .
```

```
C:\> mysql -p -u bob --protocol=pipe
```

```
C:\> mysql -p -u bob --protocol=memory
```

如果bob使用服务器的主机名（如cobra.example.com）或与主机名相对应的IP地址从本地主机进行连接，就能与 Host 列值为 cobra.example.com 的那个行相匹配。在这两种情况下，连接操作都会使用TCP/IP。

在指定Host的值时，可以使用通配符。可以使用SQL模式字符“%”和“_”，它们的意义与在语句里使用的LIKE操作符一样（不允许使用REGEXP所用的正则表达式类型）。SQL 模式字符既可用于主机名，也可用于 IP 地址。例如，%.example.com 将匹配example.com域里的任何主机，而%.edu将

匹配任何教育机构里的任何主机。同样地， 10.0.%将匹配B类子网10.0里的任何主机，而192.168.3.%将匹配C类子网192.168.3里的任何主机。

如果Host的值为%，则可以与任何主机相匹配，并且允许用户从任何地方进行连接。在权限表里，空的Host值等同于%，但有一个例外：在db表里，空白的Host值表示“检查host表，以获得更多的信息”。不过，host表现在已过时，因此不应再使用空白的db.Host值。

对于IPv4主机值，还可以指定一个带子网掩码的网络编号，以指明客户端IP地址的哪些位必须匹配该网络编号。例如，192.168.128.0/255.255.255.0 指定了一个24-位（24-bit）的网络编号，该编号能匹配IP地址前24位（bit）的值等于192.168.128的任何客户端主机。可把这种编号当作一种通配符。子网掩码的取值必须为 255.0.0.0、255.255.0.0、255.255.255.0 或 255.255.255.255。也就是说，开头的8位必须都设置为1，可以有多个这样的8位，其余位设置为0。

IPv6地址不支持子网掩码。

User

用户名必须为文字值或空白（空）。空白值可以匹配任何名字，即为“匿名”。否则，它必须与指定的名字精确匹配。特别地，当User值为%时，并不是表示空白。它会与名字为%的用户相匹配，这可能不是你想要的。

当根据user表来验证进入连接时，如果第一个匹配的行包含的User值为空白，那么该客户端会被当作匿名用户。

Db

在db表里，指定Db值时，可以使用普通字符，也可以使用SQL模式字符“%”或“_”指定的通配符。当值为%或空白时，表示可以与任何数据库相匹配。在columns_priv、tables_priv和procs_priv表里，Db值必须是普通的数据库名。它们需要与指定的名字精确匹配。不允许使用模式和空白值。

Table_name、Column_name和Routine_name

这些列值必须分别为普通的表名、列名或存储例程名。列值需要与指定的名字精确匹配。不允许使用模式和空白值。

Routine_type

此列的值只能是'FUNCTION'或者'PROCEDURE'，用来表示行中Routine_name列里的名字是适用于存储函数，还是适用于存储过程。利用

Routine_name列和Routine_type列的值，可以唯一性地标识出Db列所指定的那个数据库里的存储例程。

Proxied_host和Proxied_user

这两列存在于proxies_priv表中，该表还拥有可以表示代理用户账户的Host列和用户列。Proxied_host和Proxied_user表示的是被代理账户，其值与前面为Host列和用户列指定的那些值相似。

服务器在处理各个访问范围列时，是否要区分大小写的情况如表13-7所示。需要特别注意的是，Db 列和Table_name 列的值总是区分大小写的，即使SQL语句中数据库名和表名的处理，依赖于运行服务器的那台主机上的文件系统的大小写区分情况（通常情况下，Unix里区分大小写， Windows里不区分大小写）。

表13-7 权限表访问范围列的大小写区分情况

列	是否区分大小写
Host, Proxied_host	否
User, Proxied_user	是
Password	是
Db	是
Table_name	是
Column_name	否
Routine_name	否

13.4.2 语句访问验证

每当执行一条SQL语句时，服务器会检查你是否已经达到你语句资源的上限。这些上限是由存储在user表里的max_questions和max_updates值给定的。如果未达上限，服务器还会检查你是否具备足够的执行这条语句的访问权限。服务器确定你权限的办法是，逐个检查user、db、tables_priv、columns_priv和procs_priv表，直到服务器确定你有足够的权限，或者徒劳地搜索完所有这些表。具体的检查流程如下所示。

(1) 当你开始连接时，服务器会在user 表里检查匹配到的那个行，以确认你拥有哪些全局权限。如果你有这样的全局权限，并且它们足以让你执行该语句，那么服务器便会执行它。(2) 如果你的全局权限不足，服务器会在db表里查找与你相匹配的行。如果服务器找到了，那么它会把该行里的权限添加到你的全局权限里。如果最终结果足以执行那条语句，那么服务器便会执行它。

(3) 如果你的全局权限和你的数据库级权限加到一起还不够的话，服务器会检查tables_priv、columns_priv和procs_priv表，确定你是否有足够的执行该语句的权限。

(4) 如果服务器在检查完所有表之后，发现你仍然没有足够的执行该语句的权限，那么它会拒绝执行该语句。

如果采用布尔方式来表达，服务器对权限表的各种权限进行组合的过程如下：

user OR db OR tables_priv OR columns_priv or procs_priv

以上描述无疑使访问检查看起来像是一个相当复杂的过程，尤其是当你想到服务器会核查源自每个客户端的每一条语句的权限时。其实不然，整个过程相当快，因为服务器实际上并不会针对每一条语句去权限表里查找信息。而是在服务器启动时，就已经将这些表的内容读入内存，然后使用内存里的副本来对语句进行权限检查。这种做法可以显著提升访问检查操作的性能。此外，权限越简单，访问检查速度越快。当服务器把权限表读入内存时，它会留意账户是否存在资源限制，账户是否有表级、列级或例程级权限。如果都没有，它便会知道，在对客户端执行的各条语句进行权限检查时，不用去检查这些类型的信息。这意味着，在整个访问检查过程中，服务器可以省掉某些步骤。

使用权限表在内存里的副本来实现权限检查，会产生一个严重的负面影响：如果你直接修改权限表的内容，那么服务器并不会注意到这个改变。例如，如果你通过使用INSERT 语句向user表添加新行的方式来增加新的MySQL用户，那么此操作自身便不能使新行里指定的那个用户成功连接服务器。这经常令许多新手管理员（甚至包括某些经验丰富的管理员！）很困惑，但解决办法很简单：在直接修改权限表之后，记得告知服务器重新加载它们的内容即可。执行FLUSH PRIVILEGES语句，或者执行mysqladmin flush-privileges或mysqladmin reload命令，即可达到这一效果。

当使用CREATE USER、DROP USER、RENAME USER、GRANT、REVOKE或SET PASSWORD语句来建立或修改用户账户时，可以不用告知服务器重新加载权限

表。服务器会把这些语句映射到相应的权限表修改操作上，然后自动刷新这些表在内存里的副本。

13.4.3 访问范围列匹配顺序

MySQL 服务器会按照某种特定方式对权限表里的行进行排序，然后依次检查这些行，看是否能够找到与进入连接相匹配的行。在哪个行中找到第一个匹配，就确定使用哪个行。了解排序顺序很重要，尤其是user 表的。许多尝试理解MySQL安全性的人，似乎都会卡在这个问题上。

当服务器读取user表的内容时，它会根据Host列和User列的值对行进行排序。排序时是以Host列为主的，因此Host值相同的行会排在一起，然后再根据User值进行排序。不过，它不是按字面顺序排序的，或者更准确地说，仅在一定程度上如此。请牢记这样一个原则：相对于模式，服务器更喜欢使用字面值，相对于特定程度较低的模式，服务器更喜欢使用特定程度较高的模式。这意味着，当从某台名为 `boa.example.com` 的主机进行连接时，如果有两个行的Host值分别为 `boa.example.com` 和 `%.example.com`，那么服务器会优先使用前者。同样地，`%.example.com` 会优于 `%.net`，而 `%.net` 又会优于 `%`。对IP地址的匹配，也是如此。当客户端从某台IP地址为 `192.168.3.14` 的主机进行连接时，所有Host列为下面这些值的行都会被匹配到，但是这些行的优先级顺序如下所示：

`192.168.3.14`

`192.168.3.%`

`192.168.%`

`192.%`

`%`

需要记住的另一个原则是：当服务器尝试匹配user表里的行时，它会先匹配Host值，然后再匹配User值，而不是以相反的顺序进行匹配。

13.4.4 权限难题

本节将描述一个特定场景，说明：为何说在服务器对连接尝试进行验证时，了解服务器以何种顺序搜索user表的行是很有用的。它还展示了如何解决一个对新安装的MySQL来说普遍存在的问题，至少从它在MySQL邮件列表里出现的频率来判断是这样的。该问题是，MySQL管理员在安装部署完一个新服务器时，user表里会默认包含root行和匿名用户行。优秀的管理员通常会为

root账户分配密码，但是对匿名用户常会置之不理（如果不加以提醒），不设置任何密码。现在，假设这位系统管理员想要创建一个新账户，让它所对应的用户可以从多台不同的主机连接到服务器。最省事的做法是：创建新账户时，在GRANT 语句里把“%”作为账户名的主机部分，以便让该用户可以从任何地方进行连接。

```
GRANT ALL ON sampdb.* TO 'fred'@'%' IDENTIFIED BY 'cocoa';
```

上面这条语句的本意是想将sampdb 数据库的所有权限都授予用户fred，使其能够从任何一台主机进行连接。不幸的是，可能出现的结果却是：fred 能够从任何一台主机进行连接，唯独服务器主机除外！假设服务器的主机名为 cobra.example.com，当 fred 尝试从主机boa.example.com进行远程连接时，尝试能成功：

```
% mysql -p -u fred -h cobra.example.com sampdb
```

```
Enter password: cocoa
```

```
mysql>
```

但是当fred尝试从服务器主机cobra.example.com进行本地连接时，尝试会失败，即使fred提供了正确的密码也不行：

```
% mysql -p -u fred -h localhost sampdb
```

```
Enter password: cocoa
```

```
ERROR 1045 (28000) : Access denied for user 'fred'@'localhost'
```

```
(using password: YES)
```

如果你的user表里包含任何默认的匿名用户行，并且其用户名为空，就会出现这个问题。在Unix里，这种行是由初始化脚本 mysql_install_db 创建的；Windows发行版所包含的预初始化user表里也会有这些行。（与user表初始行有关的信息请参考12.1节。）第二个连接尝试之所以失败，是因为当服务器对用户 fred 进行身份认证时，匿名用户的其中一个行在匹配顺序上会优先于用户 fred的行。该匿名用户行要求用户在连接时不使用密码（而不是要求密码cocoa），因此密码匹配会失败。

要弄清为何会出现这种情况，需要考虑两件事情：MySQL的权限表在开始时是如何建立的；服务器在验证客户连接时是如何使用user 表中的行的。例如，在Unix 里，当在主机cobra.example.com上运行mysql_install_db脚本

以初始化权限表时，user表将包含下面这样的行（仅列出了Host列和User列的值）：

Host	User
localhost	root
cobra.example.com	root
127.0.0.1	root
::1	root
localhost	
cobra.example.com	

前面几行使用户能够以 root 身份从本地服务器进行连接。后面两行让用户能够以匿名用户身份从本地服务器进行连接。在管理员使用前面所示的GRANT语句为用户fred创建账户之后，user表将包含下面的行：

Host	User
localhost	root
cobra.example.com	root
127.0.0.1	root
::1	root
localhost	
cobra.example.com	
%	fred

不过，这里所示的行顺序与服务器在验证连接请求时使用的顺序并不相同。事实上，服务器会先按主机进行排序，其后在主机内再按用户进行排序，按特定程度由高到低的顺序来从前到后显示各个值：

Host	User
localhost	root
localhost	
127.0.0.1	root
::1	root
cobra.example.com	root
cobra.example.com	
%	fred

在Host列里，值为localhost的两个行排在一起，而User值为root的那个行排在前面，这是因为它是比空白值更为具体的用户名。Host值为cobra.example.com的行也以同样方式排在一起。此外，因为这些行的Host值都是字面值，不带通配符，所以它们都排列在User值为fred的那个行（其Host值带有一个通配符）之前。值得注意的是，两个匿名用户行在排列顺序上都优先于fred的行。

这样一来，当fred尝试从本地主机进行连接时，其中一个用户名为空的行会在Host列值包含%的那个行之前得到匹配。因为匿名用户行里的空白密码与fred的密码cocoa并不匹配，所以此次连接尝试会失败。这一现象还有一层隐含意思，即fred有可能从本地主机进行连接，但前提是他不要指定任何密码。不幸的是，这样做之后，他将会被验证为一个匿名用户，因而无法获得与fred@%账户相关联的权限。

所有这些都表明，虽然在为从多个主机进行连接的用户创建账户时，使用通配符很方便，但是该用户可能会遇到无法从本地主机进行连接的问题，因为user表里存在有匿名用户行。

对于此问题，有两个解决办法可供选择。第一个办法是为 fred 再创建一个账户，显式地将localhost作为其主机值：

```
GRANT ALL ON sampdb.* TO 'fred'@'localhost' IDENTIFIED BY 'cocoa';
```

如果采用此办法，服务器会像下面这样对user表进行排序：

Host	User
localhost	fred
localhost	root
localhost	
127.0.0.1	root
::1	root
cobra.example.com	root
cobra.example.com	
%	fred

现在，当fred从本地主机进行连接时，同时包含localhost和fred的那个行将在匿名用户行之前得到匹配。当他从其他主机进行连接时，同时包含%和fred的那个行将得到匹配。为fred创建两个账户的弊端在于：每当想要改变他的权限或密码时，必须修改两次。

第二个解决办法要容易得多，只需从user表删除匿名账户即可：

```
DROP USER "@'localhost';
```

```
DROP USER "@'cobra.example.com';
```

user表里的剩余行的顺序排列如下所示：

Host	User
localhost	root
127.0.0.1	root
::1	root
cobra.example.com	root
%	fred

现在，当fred尝试从本地主机进行连接时，就能成功了，因为 user表里已不会再有行比他所对应的行先得到匹配。

为让 MySQL 管理变得更容易，本书推荐从权限表里删除匿名用户账户。个人观点是，这些账户并没有太大的用处，相比之下，它们带来的问题更多。

本节展示的这个难题只是一种特殊情况，但是其中蕴含的道理却是通用的。如果某个给定账户的权限并没有如预期的那样工作，则应该检查权限表，看看里面是否有行包含这样的Host值：它比那个问题用户对应的行更加具体，并且与该用户的连接尝试相匹配。如果真是如此，则说明存在此问题。此时，需要让该用户的行更加具体；或者另外添加一个行，让它覆盖那个更为具体的行。

13.5 使用SSL建立安全连接

MySQL支持使用安全套接层（SSL）协议进行安全、加密连接。默认情况下，启用SSL功能的MySQL安装允许客户端选择是否使用安全连接。管理员也可能会利用GRANT语句来指明某个账户必须使用安全连接。

SSL主要是对服务器的远程连接有好处，它能保护基于网络的传输，防止传输内容被中途窥探。对于通过Unix套接字文件、命名管道、共享内存或网络环回接口IP地址127.0.0.1（或::1）建立的本地主机连接，使用SSL没有什么意义。这些连接的传输信息绝不会离开本地主机。

通过下面的基本步骤，可以在服务器与客户端程序之间建立SSL加密连接。

(1) 确保服务器和客户端程序在编译时，都加入了SSL支持功能。

(2) 在启动服务器时带上选项，用于告诉它到何处找它的证书文件和密钥文件。这些文件是建立安全连接必不可少的。

(3) 为实现客户端程序的安全连接，在调用它时需要带上选项，以便告诉它在哪里找你自己的证书文件和密钥文件。

接下来将详细讲解整个过程。

你的MySQL发行版本必须带有SSL支持功能。为此，可采取的方法包括，设法找到一个支持 SSL功能的二进制发行版本，或者通过源代码自行编译MySQL。大部分平台上的二进制发行版本都支持 SSL。如果需要自己编译MySQL，那么在进行设置时一定要记得带上必要的CMake选项（例如，`-DWITH_SSL=bundled`选项可以启用yaSSL，而`-DWITH_SSL=system`可以启用OpenSSL）。可以执行下面这条语句来验证服务器是否支持SSL：

```
mysql> SHOW VARIABLES LIKE 'have_ssl';
```

-----+-----	
Variable_name	Value
-----+-----	
have_ssl	DISABLED
-----+-----	

如果看到的结果是DISABLED或YES，说明SSL功能可用。DISABLED的意思是：支持SSL功能，但尚未启用。这样就行了。接下来讨论与启用SSL有关的文件。

在完成安装包含SSL支持功能的MySQL之后，服务器与客户端之间便可以安全通信。为建立安全连接，连接的每一端都要使用以下3个文件。

证书机构（Certificate Authority, CA）证书。CA指的是某个被信赖的第三方机构，其证书可被用于验证客户端和服务端证书的真实性。常见的做法是从某个商业机构购买CA证书，但是你也可以自己生成。

证书文件。可以将连接一边的身份认证提供给另一边。这是个公共密钥。

密钥文件。用于对连接传输的数据进行加密和解密。这是个私有密钥。

必须先安装服务器端的证书文件和密钥文件。如果你还没有这样的文件，可以在smpdb发行包的ssl目录里找到几个样板文件。

ca-cert.pem。它是CA证书。

server-cert.pem。它是服务器的证书。

server-key.pem。它是服务器的公共密钥。

首先，把这 3 个文件复制到服务器的数据目录里。然后，在服务器启动时会读取的某个选项文件（如Unix系统里的/etc/my.cnf文件，或者Windows系统里的C:\my.ini文件）里，往[mysqld]选项组里添加几个选项。这些选项的作用是指明证书文件和密钥文件的路径名。例如，如果数据目录为/usr/local/mysql/data，可以像下面这样把各个选项列出来：

```
[mysqld]
```

```
ssl-ca=/usr/local/mysql/data/ca-cert.pem
```

```
ssl-cert=/usr/local/mysql/data/server-cert.pem
```

```
ssl-key=/usr/local/mysql/data/server-key.pem
```

如果愿意，你也可以把证书文件和密钥文件放在其他地方，但是一定要确保该目录只允许服务器进行访问。在安装好SSL文件并修改完选项文件之后，需要重启服务器。

到此，服务器已支持加密连接，而 have_ssl 系统变量的值应该也变成了 YES。不过，客户端程序仍然只能使用未加密的方式连接服务器。为了让客户端程序使用安全连接，还需要指定客户端程序使用的证书文件和密钥文件。smpdb 发行包的ssl 目录也包含有这样的文件。你可以使用同一个CA证书文件（ca-cert.pem）。客户端的证书文件和密钥文件的名称分别为 client-cert.pem和client-key.pem。首先，把这3个文件复制到个人账户下的某个目录里。然后，往客户端程序在执行时会读取的某个选项文件（如Unix系统里你主目录下的.my.cnf文件）里添加几个选项，让其知道这些文件存放在何处。

假设，我想让 mysql 程序使用加密连接。我可以先把各个SSL 文件复制到主目录/home/paul，再把下面几行内容添加到.my.cnf文件：

```
[mysql]
```

```
ssl-ca=/home/paul/ca-cert.pem
```

```
ssl-cert=/home/paul/client-cert.pem
```

```
ssl-key=/home/paul/client-key.pem
```

你可以采用类似的方法建立自己的账户。请注意，一定要确保证书文件和密钥文件只能被你自己访问到。在修改`.my.cnf`文件以指明SSL文件的存放位置之后，便可以调用`mysql`，并执行`\s`或`status`命令。如果输出内容里有SSL这一行，便说明连接已加密：

```
mysql> status;
```

```
-----
```

```
mysql Ver 14.14 Distrib 5.5.21, for Linux (i686)
```

```
Connection id:          5
```

```
Current database:
```

```
Current user:           sampadm@localhost
```

```
SSL:                    Cipher in use is DHE-RSA-AES256-SHA
```

```
...
```

也可以使用下面这条语句来查看，与SSL有关的服务器状态变量的值是什么：

```
SHOW STATUS LIKE 'Ssl%';
```

如果选项文件的`[mysql]`组里有与SSL有关的选项，`mysql`程序就会默认使用SSL连接。如果注释掉这几行，或者从选项文件里删除它们，`mysql`程序就会使用常规的非加密连接。此外，像下面这样调用`mysql`程序，也可忽略SSL选项：

```
% mysql --skip-ssl
```

如果能让其他程序也可以使用SSL，可以把这个`[mysql]`选项组里的SSL选项复制到其他程序特有的选项组。是否应该把这些选项放到通用的`[client]`组呢？或许不应该。这样做会导致所有那些并不知道如何使用SSL的

客户端出现问题。（如果一定要把这些选项放到那里，则应该给它们加上 `loose-`前缀，以便那些不支持SSL的客户端程序可以忽略它们。）

如果不想将SSL选项列在选项文件里，则可以采用另一种方法，即在命令行上指定它们。例如，在那些SSL文件所在的目录里，可以像下面这样调用mysql程序（需要在同一行输入整条命令）：

```
% mysql --ssl-ca=ca-cert.pem --ssl-cert=client-cert.pem  
--ssl-key=client-key.pem
```

不过，在命令行输入所有这些内容通常是件很麻烦的事情。

sampdb发行包里的证书文件和密钥文件完全能满足建立加密连接的需要。不过，因为大家都可以获得它们（任何人都可以获得该发行包），所以由它们建立的连接并不能保证绝对安全。在用这些文件确认SSL正确工作之后，应该将它们替换成你自己生成的文件。与证书文件和密钥文件制作有关的步骤请参考 sampdb 发行包里的ssl/README.txt 文件。当然，也可以考虑购买商业证书。

到此为止，讨论的内容都是账户如何能够把SSL作为备选连接方式。你也可以建立某个账户，禁止其使用未加密连接，并要求它必须使用SSL。当修改已有账户，要求其必须使用SSL连接时，可以使用带REQUIRE子句（它指定连接必须满足的约束条件）的GRANT USAGE语句：

```
GRANT USAGE ON *.* TO 'account' REQUIRE require_options;
```

GRANT USAGE ON *.*语句不会更改账户的权限，只会更改与SSL有关的账户属性。

假设有一个账户，其用户名为laura，需要从主机viper.example.com进行连接。使用下面这条语句便可以要求其连接必须是加密的：

```
GRANT USAGE ON *.* TO 'laura'@'viper.example.com' REQUIRE SSL;
```

为获得更好的安全性，还可以使用REQUIRE X509。此时，用户 laura 在连接时必须提供合法的X509客户端证书。（这可以使用--ssl-cert选项来指定。）只要这份证书是有效的就行，其内容无关紧要。如果想对证书内容做出具体的要求，可以在REQUIRE 子句里使用CIPHER、ISSUER和SUBJECT的组合值。其中，CIPHER 指明的是连接所使用的加密方法类型，ISSUER指明的是客户端证书的签发机构，SUBJECT指明的是证书的持有者。这些子句大大缩小了其他有效证书（otherwise-valid）的范围，因为它们要求证书只能包含这些

特定的内容。下面这条 GRANT 语句要求客户端证书必须由某个特定的机构签发，并且必须使用EXP1024-RC4-SHA加密算法：

```
GRANT USAGE ON *.* TO 'laura'@'viper.example.com'  
  
    REQUIRE ISSUER  
'/C=US/ST=WI/L=Madison/O=sampdb/OU=CA/CN=sampdb'  
  
    CIPHER 'EXP1024-RC4-SHA';
```

如果某个账户当前的设置是要求使用SSL，而你想要取消这一要求，那么可以在GRANT USAGE语句中使用REQUIRE NONE：

```
GRANT USAGE ON *.* TO 'laura'@'viper.example.com' REQUIRE NONE;
```

与REQUIRE子句有关的更多信息请参考13.2.2.3节。

如果在类似Perl或PHP这样的语言里使用MySQL API，那么实现SSL功能支持不仅取决于这些语言的API，还取决于连接的那个MySQL客户端开发库。在编译客户端开发库时，也必须为其加上SSL支持功能，这样它才能支持与服务器的SSL连接。另外，语言的API也必须足够新，以便保证能够使用客户端库的SSL功能。例如，PHP的mysqlnd扩展支持SSL连接，但是早期的mysql扩展却不支持。

第14章

数据库维护、备份和复制

理想情况下，MySQL 自第一次安装好之后便可以一直稳定地运行下去。但是，出于从意外停电到硬件故障再到 MySQL 服务器非正常关闭（如服务器主机崩溃或者你用 `kill -9` 来强制终止它）的种种原因，问题还是可能发生。像这类事件，很多是你无法控制的，并且会导致数据库破坏，通常是表的更新过程中的不完全写入所导致的。

本章将介绍最小化风险的一些具体做法和灾难发生后的一些应对措施。涉及的技术包括数据库备份、表检查、表修复，以及数据丢失后如何进行恢复。本章还会介绍一些用于把数据库从一个服务器转移到另一个服务器的数据库复制过程，因为这些内容与数据库备份技术有很多相似之处。这里涉及的另一种“复制”（copy）技术在英文上称作 replication，它是使用主服务器（master server）上的数据副本来初始化从服务器（slave server），初始化完成后，主服务器的数据变化会传播至从服务器。因此，从服务器像是主服务器的“连续复制”（continuous copy）。这种复制技术还可以用于其他用途。举两个例子：可以把客户端负载分摊到两个服务器上，从而减轻主服务器的负担；在进行备份时，与主服务器相比，从服务器可以轻易地暂停或停止。

14.1 预防性维护的基本原则

本节汇总了一些与数据库系统的预防性维护有关的基本原则。随后几节将讨论实现细则。

为预防数据库出现问题，应该采取以下措施。

启用MySQL服务器提供的自动恢复功能。

有计划地开展预防性维护工作，定期对表进行检查。日常的表检查有助于及时发现各种小问题，并在问题变得更糟之前将其纠正。

建立数据库备份计划。当发生最坏情况时，如果系统出现灾难性的故障，则需要使用以前的备份来恢复系统。还应该启用二进制日志，这样可以记录下你在备份之后又对数据库的内容进行过哪些修改（参考12.8.4节）。二进制日志对备份和复制有很大益处，并且其性能开销几乎可以忽略不计，所以没道理不启用它。

尽管你想尽了办法，如果最终还是出现了表损坏或数据丢失的问题，那么你可以尝试使用下面的方法来处理这些问题。

检查你的表，并尽可能地修复已发现的损坏表。轻微的损坏通常可以使用MySQL的表修复功能来修正。

如果对表进行检查和修复仍不能让服务器恢复运行，则需要使用备份和二进制日志来进行数据恢复。首先，利用备份把表恢复到备份时的状态。然后，再根据备份之后、崩溃发生之前所做的二进制日志，把表恢复到最新状态。

为完成上述任务，可供使用的工具除了MySQL服务器自身的功能以外，还有MySQL发行版自带的其他几种工具。

当服务器启动时，事务型存储引擎会进行自动恢复。对于MyISAM存储引擎，你还可以启用自动表修复功能。在服务器崩溃之后重启时，这些功能都很有用。

使用mysql_dump程序来备份数据库，以后很可能会需要恢复它们。

如果想让服务器按需来执行维护操作，可以使用SQL语句，如CHECK TABLE和REPAIR TABLE。这些语句的命令行接口需要使用mysql_check程序。myisamchk工具也可用于检查表的问题，并采取正确的措施。

有些程序，如mysql_check和mysqldump，需要与MySQL服务器配合起来使用。它们会以客户端身份连接到服务器，然后执行SQL语句，指导服务器执行什么样的表维护操作。相比之下，myisamchk 程序是一个可以直接操作于表示表的文件的独立程序。因为MySQL服务器在运行时也会访问那些文件，所以myisamchk的操作会与MySQL服务器发生竞争。这意味着，你必须采取措施防止myisamchk与服务器相互干扰。例如，你正在使用myisamchk来修复某个表，那么一定要保证服务器在此期间不会对该表执行写入操作。如果你没有这样做，则会导致出现比你正在试图修复的问题还要糟糕得多的问题！

本章所涉及的几项管理任务（从制作备份到执行表修复），都需要与MySQL服务器合作。因此，下一节将讨论如何让MySQL服务器面对困境，随后的各节将讨论如何预防问题发生、如何备份，以及如何在必要时使用各种修复技术和恢复技术。

在Unix系统里，当需要直接操作MySQL数据目录中的表文件或其他文件时，应该以MySQL管理员的身份登录，这样才能有足够的权限去访问这些文件。在本书里，假设这个登录账户的名字为mysql。当然，也可以用root用户的身份来访问这些文件，但是在这种情况下，一定要保证你所处理的文件在你操作的前后都拥有相同的访问模式和所有权属性。

SQL语句支持的选项的完整列表请参考附录E，本章所讨论的各个程序请参考附录F。

14.2 在服务器运行时维护数据库

有些数据库维护操作需要连接服务器，告诉服务器该如何做。如果需要对MyISAM表进行一致性检查或修复，那么一种做法是执行CHECK TABLE或REPAIR TABLE语句（或者调用mysqlcheck程序），让服务器来做具体的事情。在这种情况下，服务器会访问表示这个表的.frm、.MYD和.MYI文件。这可能是能够采用的最好方法，因为当服务器执行这些请求的维护操作时，会负责处理协调访问表所引发的各类问题，所以你就不用再考虑这些问题了。

其他维护操作都是由不依赖于服务器的外部程序来完成的，在这种情况下，你必须考虑与表访问协调有关的问题。例如，另一种检查或修复MyISAM表的做法是调用mysamchk工具程序，这个程序可以在不通过服务器的情况下，直接打开表文件。在mysamchk访问表文件时，你必须防止服务器在此时修改那个表。如果不这样做，那么对这个表的竞争访问则可能对其造成破坏，甚至导致其无法继续使用。很明显，让服务器和mysamchk同时往一个表里写入内容是一件很糟的事情，不过，即使是它们中的一个在读，而另一个在写，也是不妥的。正在进行读取的程序可能会迷糊，搞不清楚这个表是否会同时被另一个程序更改。

在其他一些情况下，也需要防止MySQL服务器访问表。

使用了mysampack程序压缩的MyISAM表。

重定位了MyISAM表的数据文件或索引文件。

重定位了数据库。

使用了会复制表文件的备份技术。为确保备份文件的一致性，必须防止服务器在备份过程中修改表。

使用了把损坏表替换为完好备份副本的恢复方法。在替换该表时，不能让服务器访问它。

如果你不想受到服务器的干扰，最有效的办法是将其彻底关闭。很明显，如果服务器根本没有运行，那么它当然不会访问你正在处理的那个表。但可以理解的是，MySQL管理员通常都不太愿意完全关闭服务器，因为这会使所有的数据库和表（并不仅仅是你想要检查或修复的那几个表）无法使用。

如果想避免停止服务器，同时又想避免在运行的服务器和你在外部执行的操作之间出现相互干扰的问题，那么必须使用锁定协议来与服务器进行协调。MySQL服务器提供了下面两种锁定机制。

内部锁定机制。服务器使用这一机制，可以防止不同客户端的请求相互干扰，例如，要防止一个客户端的SELECT查询被另一个客户端的UPDATE语句干扰。在你使用外部程序处理某个表时，也可以用这种内部锁定机制来阻止客户端访问该表。

外部锁定机制。服务器使用这一机制，可以防止其他程序修改它正在使用的表文件。

这种外部锁定机制的基础是操作系统在文件系统级的锁定功能。通常情况下，服务器使用外部锁定机制的原因在于，在表检查操作期间，可以对诸如myisamchk之类的工具程序进行协调。但是，外部锁定机制在某些系统上不太可靠。另一个限制是，外部锁定机制只能用于对表文件进行只读访问的操作，如表检查。外部锁定机制并不能用于要求对表文件进行读/写访问的操作，如表修复。外部锁定机制是以文件锁定为基础的，但是myisamchk程序执行的修复操作会先把表文件复制为新文件，在处理完之后再用新文件替换原来的表文件。服务器对这些新文件一无所知，这会导致任何想要利用文件锁定机制来协调访问的努力都起不到任何作用。

接下来的讨论只涉及如何使用内部锁定机制来协调服务器的表访问操作。因为外部锁定机制存在问题，所以建议你也不要使用，本书也不再对其做更多的讨论。

14.2.1 为只读或读/写访问锁定单个表

如果要使用服务器的内部锁定机制来防止它访问你正在处理的表，可以使用mysql程序连接服务器，并执行LOCK TABLE语句锁定你想要使用的表。接着，把mysql闲置在那里（也就是说，除了锁定那个表以外，什么事情也不

做），然后根据你的需要处理表文件。在处理完成之后，切换回你的mysql会话，释放刚才的锁，告诉服务器它可以再次使用这个表了。

说明

这里讨论的用于锁定单个表的内部锁定技术，只适用于处理像MyISAM这样的存储引擎所使用的表文件。这种存储引擎在表示每个表时，会使用这个表独有的文件。这些技术并不适用于像InnoDB这样的存储引擎——它们会把与多个表有关的信息存储在某个给定的文件里。例如，InnoDB引擎默认会把所有InnoDB表都表示到构成其系统表空间的那些文件中。（即使通过配置，让每个表单独使用一个表空间，InnoDB仍然会把某些与每个表有关的信息存储在它的数据字典里，而这个数据字典存储在系统表空间里。）

使用哪种锁定协议，取决于你是要对这个表的文件进行只读访问，还是要进行读/写访问。对于只检查或复制文件的操作，只需要进行只读访问。对于需要修改某些文件的操作，如表修复或者使用完好的文件来替换损坏文件，则需要进行读/写访问。

这两种锁定协议都需要使用LOCK TABLE和UNLOCK TABLE语句，前者用来请求锁，后者用来释放锁。它们还需要使用FLUSH TABLE语句来通知服务器刷新（flush）缓存的更改，将缓存的内容写入磁盘，这条语句的另一个作用是告诉服务器在下次访问这个表的时候，需要重新打开它。各个实例使用的是FLUSH TABLE语法，该语法接受一个表名作为参数，并且只会刷新这个特定的表。

你必须在与服务器的一个单独会话里执行所有的LOCK、FLUSH和UNLOCK语句。例如，不能使用mysql程序连接MySQL服务器，锁定表，然后立刻退出，接着又试图再次连接服务器以解除锁定。这种操作之所以行不通，是因为当你退出mysql程序时，服务器会自动解除你所设置的锁定。此时，服务器会认为它又可以随意地使用这个表了，而结果就是你在处理这些表文件时将不再安全。

执行锁定操作的一个简单方法是，同时打开两个窗口。你可以让其中一个窗口持续运行mysql，而在另一个窗口里处理表文件。在非窗口化环境里，你可以一边利用shell的作业控制设备来挂起和恢复mysql进程，一边处理表。

14.2.1.1 为只读访问锁定表

只读锁定协议适用于只需对表文件进行读取的操作，如复制文件或者检查文件的不一致性。对于这种情况，获取读锁就够用了；服务器会阻止其他

客户修改表，但会允许客户读取它。在需要修改表时，请不要使用这种锁定方式。

(1) 在窗口A里，调用mysql程序，并执行下列语句，以获得读锁，然后刷新表：

```
% mysql db_name  
  
mysql> LOCK TABLE tbl_name READ;  
  
mysql> FLUSH TABLE tbl_name;
```

在你检查表时，获取读锁可以防止其他客户端对该表执行写入和修改操作。FLUSH 语句将使服务器关闭表文件，这会刷新可能仍缓存在内存里的未写入的更改。

(2) 在mysql闲置之后，切换到窗口B，开始处理表文件。例如，如果tbl_name是一个MyISAM表，你便可以采用复制其表示文件的方式来进行备份。如果当前目录为表的数据库目录，则需要把这些文件像下面这样复制到备份目录/var/backup下：

```
% cp tbl_name.* /var/backup
```

这个示例只是用于演示。你实际发出的命令取决于你要执行的特定维护操作。

(3) 在完成对表的处理之后，再切换回窗口A里的mysql会话，释放表锁：

```
mysql> UNLOCK TABLE;
```

在整个操作过程中，你很可能会发现还需要对这个表做进一步的处理。例如，在进行表检查时，很可能需要执行一些还原或修复操作。这些操作需要读/写访问，可以使用下一节介绍的协议来安全地获得。

14.2.1.2 为读/写访问锁定表

本节讨论的读/写锁定协议适用于需要对表文件进行修改的场合，如表替换或表修复。为做到这一点，你必须获取一个写锁，以便完全阻止服务器对你正在处理的表进行任何形式的访问。

对表进行读/写访问锁定的操作，与进行读取访问锁定的操作相似，不同之处有两点。第一，除了获取读锁外，还必须获取写锁。因为你要对表进行修改，所以决不能让服务器访问它，哪怕只是读取也不可以。第二，在处理完表之后必须执行第二条FLUSH TABLE语句。如果维护操作重建或者重新修复了索引文件，那么只有等到你再次刷新表缓存之后，服务器才能知道新索引文件的存在。

为读/写访问锁定表的过程如下所示。

(1) 在窗口A里调用mysql程序，执行下面这条语句，以获得写锁，然后刷新表：

```
% mysql db_name

mysql> LOCK TABLE tbl_name WRITE;

mysql> FLUSH TABLE tbl_name;
```

(2) 在mysql闲置之后，切换到窗口B，开始直接处理表文件。例如，如果tbl_name是一个MyISAM表，你便可以根据备份（通过复制表的表示文件得到的）来恢复这个表。如果当前目录为表的数据库目录，则需要像下面这样把备份目录/var/backup里的文件复制过来。

```
% cp /var/backup/tbl_name.*.
```

这个示例只是用于演示。你实际执行的命令取决于你要执行的特定维护操作。

(3) 在完成对表的处理之后，切换回窗口A里的mysql会话，再次刷新表，释放表锁：

```
mysql> FLUSH TABLE tbl_name;

mysql> UNLOCK TABLE;
```

14.2.2 为只读访问锁定所有数据库

如果想要阻止客户端修改任何表，最简单的办法莫过于以只读方式一次性锁定所有数据库里的所有表。使用下面这些语句可以做到这一点：

```
mysql> FLUSH TABLES WITH READ LOCK;
```



```
mysql> SET GLOBAL read_only = ON;
```

其中，FLUSH语句可以获得一个全局性的读锁，而SET语句会进入阻塞状态，一直到其他客户端释放它们所拥有的表锁，并完成所有未完成的事务。在SET语句返回之后，你便可以安全地继续后面的操作，而其他客户端这时也不会对你造成干扰。

如果想再次允许进行修改，可以使用下面这些语句：

```
mysql> SET GLOBAL read_only = OFF;
```

```
mysql> UNLOCK TABLES;
```

当表以这种方式锁定时，其他客户端只能读取数据，但不能更改它们。当你执行某些操作时，如使用mysqldump 程序进行备份，如果想让 MySQL服务器保持“安静”，那么这种方法是个不错的选择。但从另一方面来讲，这种方法对于那些想要更新表的客户端来说很不友好，因此只有在真的有必要时才可以这样锁定服务器。这个办法还无法满足某些操作（例如，对像InnoDB这样的事务型存储引擎所管理的所有表进行二进制备份）的需要，因为这种存储引擎可能还有一些尚未完成的事务，只部分地刷新了它的日志文件。像这样的操作需要你完全停止服务器，才能保证刷新所有的内容，关闭所有的文件。

14.3 基本的预防性维护

本节将介绍一些能帮助你维护数据库完整性的基本策略。

启用MySQL服务器提供的自动恢复功能。

安排预防性的维护工作，定期对表进行检查。

制定一套定期备份数据库的策略，这样，在数据库遭到破坏或出现数据丢失时，可以进行恢复。

本节将对前两项进行讨论。更多关于备份技术的信息请参考14.4节。

14.3.1 运用服务器的自动恢复功能

维护数据库完整性的第一道防线是 MySQL 服务器的崩溃恢复功能。一种恢复（事务型存储引擎的恢复）是服务器启动时自动执行的。另一种恢复（MyISAM的恢复）是可选的，必须显式启用。

当服务器启动时，会执行某些类型的表检查，以帮助处理早些时候因服务器或机器崩溃而导致的问题。MySQL的设计考虑到了对各种问题的恢复操作，因此在一般情况下，你除了重新启动服务器之外什么也不用做。在许多情况下，它会替你完成必要的修正工作。例如，InnoDB存储引擎可以自动检查问题。对于那些已提交的事务，如果它们只存在于重做日志里，而没有经过刷新写入表中，那么它们会前滚（即重做）。对于那些在崩溃时没来得及提交的事务，会利用取消日志进行回滚（即丢弃）。最终的结果保证了InnoDB表始终处于一致的状态，因此它们的内容能够反映出所有在崩溃发生前提交的事务。

如果InnoDB表的自动恢复因为某个无法恢复的问题而失败，那么服务器会把消息写入出错日志，然后退出。如果想要强行启动服务器，可以尝试手动恢复请参考14.7.4节。

对于MyISAM表，服务器提供了一种可选的表恢复功能，但你必须显式启用它。一旦你启用了此功能，每次服务器打开MyISAM表时都会对它进行检查。如果有表在上次使用后没有正确关闭或是被标记为“崩溃”，那么服务器将对它进行检查和修复。如果想要启用MyISAM表的恢复功能，可以通过设置系统变量`myisam_recover_options`来启动服务器。这个变量的值为一个以逗号分隔的列表，其中可以带有一个或者多个选项：BACKUP（如果修复操作会更改这个表，则先为它创建一个备份）、FORCE（即使会丢失多行数据也要强制进行恢复）、QUICK（快速恢复）、DEFAULT（在恢复时不进行任何其他特殊处理）或者 OFF（不进行恢复）。例如，如果想在发现问题时强制进行恢复，但是又需要先创建备份，则可以在服务器的选项文件里放入下面几行内容：

```
[mysqld]
```

```
myisam_recover_options=BACKUP,FORCE
```

作为基本的维护策略，启用MyISAM的自动恢复功能是很有用的，否则，那些服务器发现有问题的MyISAM表将变得不可用，直到你发现这个问题，并手工修复它，它才能再次变得可用。如果你在运行服务器时启用了系统变量`delay_key_write`，或者对单个MyISAM表进行了配置，让其使用“延迟键写入”功能，那么MyISAM的恢复将尤为重要。在这些条件下，在表关闭之前，对索引的修改不会被刷新。虽然这能改善服务器的运行性能，但是这也意味着，在崩溃发生时，那些使用了“延迟键写入”功能的表将仍是打开的，需要为它们修改索引。

14.3.2 制订预防性维护计划

除了启用服务器的自动恢复功能以外，还应该考虑制订一个预防性维护计划。这有助于自动检测问题，让你能够及时采取措施来修正它们。有计划地对表进行定期检查，能够减少求助于备份的可能性。在Unix系统里，实现这种定期检查最简便易行的办法是使用cron作业。通常的做法是，为用于运行服务器的账户创建一个crontab文件，并通过它来调用cron作业。（与设置cron作业有关的信息请参考12.8.7.3节。）

当服务器在线时，mysqlcheck程序可用于对MyISAM和InnoDB表进行检查。假设你想要建立一个定期调用mysqlcheck的维护作业。如果运行服务器的用户为mysql，那么你可以在这个用户的crontab文件里设置定期检查操作，只需往这个文件里添加如下所示的设置项即可。在一行内输入所有的内容，并使用你系统中的mysqlcheck路径：

```
45 3 * * 0 /usr/local/mysql/bin/mysqlcheck  
  
--all-databases --check-only-changed --silent
```

上面这个设置项会告诉cron在每个星期日的3点45分运行mysqlcheck程序。你可以根据需要调整这个时间或计划内容。

--all-databases选项会告诉mysqlcheck去检查所有数据库里的所有表。这种做法可以让你以最简便的方法获得最大的效果。如果想使用适当的选项，让mysqlcheck程序只对特定的数据库或表进行检查，请参阅附录F中的程序描述。

选项--check-only-changed会告诉mysqlcheck跳过所有自最后一次成功检查以来从未修改过的表。选项--silent的作用是在表里未发现错误时抑制信息输出。如果某个cron作业最终产生了输出，它通常会生成一个邮件消息。对于一个用于表检查的作业，如果未发现问题，就没什么理由会收到邮件。请注意，如果数据库里的表所使用的存储引擎是mysqlcheck程序不知道如何进行检查的，那么即使使用了--silent选项，你也会得到一些诊断输出。

最后，请记得检查mysql账户的邮件，或者将其路由到你自己的账户。

说明

当对表进行检查时，不能对它进行更新。如果你无法承受在检查期间阻塞更新操作所带来的后果，那么自动维护策略可能就不适用于那些需要频繁更新的大表。

14.4 数据库备份

为了预防数据丢失造成的损坏，备份数据库很重要。在系统严重崩溃时，备份能够让你以最少的数据损失把表恢复回来。备份也能让你为这样的用户提供帮助：不明智地调用了 `DROP DATABASE`、`DROP TABLE` 或 `DELETE` 语句，并且请求你帮忙恢复数据的用户。

数据库备份对于某些不可恢复性操作（如把一个数据库复制到另一个服务器）也很有用。大部分情况下，都是把数据库迁移到在另一台主机上运行的服务器，但你也可以把数据库迁移到在同一主机上运行的其他服务器。如果你正在测试安装了新版 MySQL 的服务器，并且想使用生产服务器上的真实数据，那么你很可能会这样做。

备份的另一个用途是建立复制（replication）服务器。建立从服务器的第一步是，在某个特定的时间点为主服务器拍一个快照。备份就相当于这个快照，只要把它加载到从服务器，就能让从服务器与主服务器保持一致。此后，用户在主服务器上做出的修改，将通过标准的复制协议原样复制到从服务器。相关内容请参考14.8节。

首先，我们会提供几条掌控备份实践的基本原则，这些原则能够助你决定使用哪一种技术。接着，我们会详细介绍几种特定的备份方法。

数据库备份分为以下两类。

文本格式的备份。可以通过 `mysqldump` 程序把表内容写入转储文件（dump file）来获得。

这些文件主要由两种SQL语句 `CREATE TABLE` 和 `INSERT` 构成，这些语句可以在以后重新加载到服务器，从而恢复表。

二进制备份。可以通过直接复制那些包含表内容的文件来获得。创建此类型备份的办法有很多种。例如，可以使用像 `cp`、`tar` 或 `rsync` 之类的程序。

每一种备份方法都有其自己的优缺点。在挑选备份工具时需要考虑的因素包括是否需要关闭服务器、备份需要花费的时间、备份的可移植性和备份的范围。

`mysqldump` 与 MySQL 服务器是合作关系，因此只要服务器在运行，你就可以使用它。二进制备份方法是在服务器外部进行的文件复制操作。部分此类方法要求你停止服务器。对于那些没有这种要求的备份方法，你必须自己想办法保证服务器不会对你正在复制的表文件进行修改。

mysqldump的备份速度比二进制备份技术的慢，因为转储操作涉及的环节比较多：服务器要先读取表、然后为便于传输转换表的内容，最后通过网络连接把信息发送至mysqldump。二进制备份方法是在文件系统级进行文件复制的，不需要进行数据转换，也不需要通过网络传输。

mysqldump生成的是包含SQL语句的文本文件。这些文件很容易移植到其他机器上，即使是有着不同硬件架构的机器也没问题。因此，可以用它们把数据库从一个服务器复制到另一个服务器。直接复制的二进制备份方法所生成的文件能否移植到其他机器上，取决于表是否使用了与机器无关的存储格式。MyISAM和InnoDB表通常与机器无关。

对于这些存储引擎，可将直接复制的表文件移到运行在另一台有着不同硬件架构的机器上的服务器里。与各种存储引擎的可移植性有关的信息请参考14.4.1节。

mysqldump的输出只包含数据库的内容（表、视图、存储例程等）。它不会备份没有存储在数据库里的信息，如配置文件、日志文件或复制状态文件。二进制备份则可以包括上述内容中的任何部分或全部，因为你可以复制任何你想要的文件做成备份。

无论选用哪种备份方法，都必须遵守以下原则，才能确保在需要恢复数据库内容时，获得最好的结果。

定期进行备份。制订一个计划，并严格执行。

配置服务器，启用其二进制日志功能（参考12.8节）。在服务器崩溃之后，二进制日志能帮助你恢复数据库。基本做法是：用备份文件把数据库恢复到备份时的状态，然后执行二进制日志文件里的内容，重新应用备份之后所发生的更改。这样便可以把数据库还原到崩溃发生时的状态。

备份文件的命名方案，既要一致，又要有意义。诸如backup1、backup2之类的名字并没有多大意义。当需要执行还原操作时，你将浪费大量的时间才能弄清这些文件里面的内容是什么。使用数据库的名字和备份日期来构造备份文件名是种不错的做法。例如，如果你在2013年1月2日转储了sampdb数据库，那么可以把备份文件命名为sampdb-2013-01-02。如果同时运行了多个服务器，那么还应该把服务器的标识也加到名字里。

不要把备份文件和数据库放在同一个文件系统上。首先，这样做可以减少备份文件挤占数据目录所在文件系统空间的可能性。其次，如果将备份文件存放在另一个物理驱动器里，这还能进一步减少因驱动器故障而导致的损失，因为不管哪一个驱动器出现了故障，都不会导致数据目录和备份文件同时遭到破坏。

要把数据库备份文件定期放到文件系统备份中。如果系统彻底崩溃，那么彻底摧毁的将不止是你的数据目录，还有存放数据库备份文件的硬盘驱动器，那时麻烦可就大了。还有一点，别忘了备份日志文件。

定期清理过期的备份文件，防止它们用尽硬盘空间。一种办法是使用文件轮回技术。

12.8.7节讨论了的日志文件的过期问题，那里谈到的处理原则也同样适用于备份文件。

接下来的几节将讨论存储引擎的可移植性，并且会对几种特定的备份方法进行描述。如果你使用的是复制机制，那么14.8.4节描述的方法可以保证你的主服务器完全不会受到干扰。

14.4.1 存储引擎的可移植特点

给定 MySQL 服务器所管理的任何表都可移植到另一个服务器，也就是说，你可以使用`mysqldump`将表转储为一个文本文件，接着将这个转储文件移到运行另一个服务器的机器上，然后加载这个文件，重建这个表。另一种可移植性是“二进制可移植性”，它意味着：你可以把表示这些表的磁盘文件直接复制到另一个机器，将其安装到数据目录下的相应位置，然后期待着那个MySQL服务器能够使用这个表。

一般来说，表的二进制可移植性要求源服务器和目标服务器在功能特性上必须兼容。例如，目标服务器必须支持管理表的存储引擎。如果服务器没有这种存储引擎，就不能访问源服务器上那个存储引擎所创建的表。

有些存储引擎创建的表具有二进制可移植性，有些则不具有。下面重点总结了几种存储引擎的二进制可移植性。

InnoDB和MyISAM表都是以与机器无关的格式存储的，并且都具有二进制可移植性。

试想一下，你的处理器使用了二进制补码的整数运算和IEEE的浮点格式，结果会怎么样呢？除非你的机器真的很古怪，否则在上述两种情况下都不会出现任何问题。（此处的“浮点”指的是FLOAT和DOUBLE。DECIMAL列包含的定点值使用的是可移植的存储格式。）

对于 InnoDB，具备二进制可移植性的附加条件是数据库名和表名都是小写的。InnoDB在其数据字典里会以小写方式存储这些名字，但是在创建`.frm`文件时使用的表名大小写与你在CREATE TABLE语句里使用的大小写一样。如果你在创建数据库或表时使用了带大写字母的名称，然后想要把它们移到文件名区分大小写的平台上，就会导致出现大小写不匹配的问题。

对于InnoDB，在进行二进制可移植性评估时，必须把所有的InnoDB表当作一个整体，而不能只考虑单个表的情况。默认情况下，InnoDB 存储引擎会把所有表的内容存储在共享系统表空间里，而不会存放在与表特定相关的文件里。因此，是 InnoDB表空间文件决定了是否可移植，而不是单个的InnoDB表决定的。这意味着，只要有某个InnoDB表使用了浮点数类型列，就会受到浮点数可移植性约束的限制。即使你通过配置，让InnoDB使用了单个表空间（即每个表对应一个表空间），数据字典条目也会存储在系统表空间里。

CSV表具有二进制可移植性，因为他们的CSV数据文件是纯文本的。

MEMORY表不具有二进制可移植性，因为其内容存储在内存里，而非磁盘上。

不管存储引擎是否具有一般可移植性的特点，在关闭服务器之后，你都不应该试图将表或表空间文件复制到另一台机器，除非这个服务器是彻底关闭的。如果在非正常关闭之后进行复制，那么很难保证表的完整性。表可能需要修复，或者事务信息可能仍然存储在存储引擎的日志文件里，只有应用或回滚这些文件，才能把表恢复到最新。

有时也可以告知正在运行的服务器，让它在复制表的文件时，别对这些表做任何操作。不过，如果服务器正在运行当中，并且正在积极更新表，或者还有很多更改仍然缓存在内存里，那么磁盘上的表内容将处于不断变化之中，并且相关的文件将不会产生可用的表副本。与“在进行表复制时可以避免停止服务器的各种前提条件”有关的讨论请参考14.2节。

14.4.2 用mysqldump程序制作文本备份

使用mysqldump 程序可以创建文本转储文件。默认情况下，转储文件是SQL格式的，其中有多条用于创建那些转储出来的表的CREATE TABLE语句，以及多条包含这些表中行数据的INSERT 语句。如果以后想重建这些转储表，只需要把这个转储文件用作 mysql的输入，将其重新加载到MySQL中即可。例如，想要转储和重新加载某个表（如sampdb.member），可以使用下面这些命令：

```
% mysqldump sampdb member > member.sql
```

```
% mysql sampdb < member.sql
```

请不要用mysqlimport程序来重新加载SQL格式的mysqldump输出。因为mysqlimport只能读取行数据，而不能读取SQL语句。

下面的命令可以把所有数据库里的所有表备份到一个单独的文件里：

```
% mysqldump --all-databases > /archive/mysql/dump-all.2013-01-02
```

不过，如果你的数据量非常大，那么生成的转储文件也会相当大。你可以把每个数据库转储到它自己的文件里，如下所示：

```
% mysqldump mysql > /archive/mysql/mysql.2013-01-02
```

```
% mysqldump sampdb > /archive/mysql/sampdb.2013-01-02
```

```
% ...
```

mysqldump的输出内容类似下面这个样子：

```
-- MySQL dump 10.13  Distrib 5.5.30, for Linux (i686)
```

```
--
```

```
-- Host: localhost      Database: sampdb
```

```
-- -----
```

```
-- Server version      5.5.30-log
```

```
...多条SET 语句 ...
```

```
--
```

```
-- Table structure for table `absence`
```

```
--
```

```
DROP TABLE IF EXISTS `absence`;
```

```
/*!40101 SET @saved_cs_client      = @@character_set_client */;
```

```
/*!40101 SET character_set_client = utf8 */;
```

```
CREATE TABLE `absence` (
```

```
`student_id` int(10) unsigned NOT NULL,
```



```

`date` date NOT NULL,

PRIMARY KEY  (`student_id`,`date`),

CONSTRAINT `absence_ibfk_1` FOREIGN KEY (`student_id`)

REFERENCES `student` (`student_id`)

) ENGINE=InnoDB DEFAULT CHARSET=latin1;

/*!40101 SET character_set_client = @saved_cs_client */;

--

-- Dumping data for table `absence`

--

LOCK TABLES `absence` WRITE;

/*!40000 ALTER TABLE `absence` DISABLE KEYS */;

INSERT INTO `absence` VALUES (3,'2012-09-03'),(5,'2012-09-03'),

(10,'2012-09-06'),(10,'2012-09-09'),(17,'2012-09-07'),(20,'2012-09-07');

/*!40000 ALTER TABLE `absence` ENABLE KEYS */;

UNLOCK TABLES;

```

...其余的转储输出 ...

此文件的其余部分包含了更多的SQL语句，如CREATE TABLE和INSERT。

转储文件通常都很大，因此你可能会想要它们变得更小一点。一种减小转储文件大小的是对其进行压缩。在Windows系统里，可以选用WinZip或与其类似的程序来压缩转储文件，生成一个Zip格式的文件。在Unix系统里，可以使用gzip或bzip2来压缩。你甚至可以使用命令管道在生成备份的同时对其进行压缩：

```
% mysqldump sampdb | gzip > /archive/mysql/sampdb.2013-01-02.gz
```

```
% mysqldump sampdb | bzip2 > /archive/mysql/sampdb.2013-01-02.bz2
```

如果觉得过大的转储文件难以管理，那么可以在mysqldump命令行里，采用数据库名后面紧跟单个表名的方式，实现单个表内容的转储。mysqldump程序只会转储给定数据库里的给定表，而不会转储给定数据库里的所有表，如此一来，便可以得到更小的转储文件，也更易管理。下面这个示例展示了如何将sampdb数据库里的部分表转储为不同的文件：

```
% mysqldump sampdb member president > hist-league.sql
```

```
% mysqldump sampdb student score grade_event absence > gradebook.sql
```

mysqldump有许多选项。下面列出其中比较常用的部分。更多选项，请参考附录F。

通常情况下，在用mysqldump命令行来命名数据库时，可以选择在后面加上具体的表名。如果想要一次性转储多个数据库，可以使用--databases选项。接着，mysqldump会把命令行里的所有名字解释成数据库名，并依次转储这些数据库里的所有表。如果想转储某个服务器里的所有数据库，则可以使用--all-databases选项。此时，不用给出任何数据库名或表名参数。--databases和--all-databases选项都可以使得每个数据库的输出前面带上CREATE DATABASE IF NOT EXISTS和USE语句。

如果想把转储输出加载到另一个服务器，则需要慎重选用--all-databases选项。转储输出里会包含mysql数据库的各个权限表，而你可能并不想替换另一个服务器的权限表。

默认情况下，mysqldump程序将同时转储表的结构（使用CREATE TABLE语句）和表的内容（使用INSERT语句）。如果只想转储这两类信息中的一类，则可以使用--no-create-info或--no-data选项。

--opt 选项可以对转储过程进行优化。它会同时启用其他几个选项，对转储过程进行优化，从而产生更小的转储输出。这些选项也会优化还原过程，因为当你以后重新加载这个转储文件时，整个过程也会更快。例如，--opt的其中一个效果便是让mysqldump写入多行形式的INSERT语句。与单行形式的INSERT语句相比，这种多行语句占用的空间更少，重新加载的速度更快。

--opt选项默认是启用的，因此不需要显式指定它。如果你真的不需要优化转储，可以使用--skip-opt选项来禁用它。

使用`--opt`选项来进行备份是最常见的做法，因为它可以加快备份速度。但需要注意的是，使用`--opt`选项是有代价的。`--opt`选项优化的是备份过程，而不是其他客户端对数据库的访问操作。`--opt`选项会阻止任何人更新你正在转储的表，因为它会同时锁定所有的表。

如果生成的备份文件是用于定期更新另一个数据库的内容（如另一个服务器上的某个数据库）的，那么`--opt`选项会很有用。这是因为`--opt`选项会自动启用`--add-drop-table`选项，从而使`mysqldump`在备份文件的每一条`CREATE TABLE`语句的前面，为每一个相同的表加上一条`DROP TABLE IF EXISTS`语句。当你把这个备份文件加载到第二个数据库时，不会在这个表已经存在的情况下发生错误。如果你想运行第二个的测试服务器不是复制（replication）从服务器，就可以利用这种技术和生产服务器上的数据库数据副本，定期重新加载它。

`--opt`选项的另一个效果是启用`--extended-insert`选项，使`mysqldump`写入多行形式的`INSERT`语句。虽然这种做法可以节省空间，但在你想要一个更可读的转储文件时，它就是一个缺点。如果想产生单行形式的`INSERT`语句，可以使用`--skip-extended-insert`选项。

在为数据库设置检查点时，最好是同时使用`--flush-logs`和`--lock-all-tables`。`--lock-all-tables`可获得全局读锁，而`--flush-logs`可关闭和重新打开日志文件。如果启用了二进制日志功能，那么刷新日志，将会创建一个新的二进制日志文件，其中只会包含那些自检查点之后发生的数据修改。这样便同步了日志与备份的时间。（这种做法的弊端是：在备份期间锁定所有的表，会对那些在此期间需要执行更新操作的客户端带来不便。）

如果想要使用`--flush-logs`把日志的检查点设置为备份的时间，那么最好能每次转储整个数据库。在进行恢复时，通常是提取每个数据库的日志内容。如果你转储的是单个表，那么让日志的检查点与备份文件保持同步会更难。（没有用于从日志里提取各个表的更新内容的选项，因此你只能靠自己提取它们。）

如果要转储InnoDB表，可以使用`--single-transaction`选项，把表都转储到一个事务里，这样就能获得一致的备份。

如果数据库包含存储例程、触发器和事件，那么可以使用`--routines`、`--triggers`和`--events`选项，把它们显式地放到转储输出里。这几个选项都有一`--skip`形式（如`--skip-triggers`），可用于禁止转储相应的对象。默认情况下，转储输出里只会包含触发器（因为它们与表关联在一起），而不会包含存储例程和事件。

当你期望通过建立复制从服务器来在主服务器上生成转储文件时，`--master-data` 选项就能派上用场。使用这个选项，转储文件会包含与主服务器二进制日志坐标有关的信息，在使用该文件的内容对从服务器进行初始化之后，从服务器会由该处开始复制。

14.4.3 制作二进制数据库备份

不使用 `mysqldump` 程序对数据库或表进行备份的方法是直接复制表文件。通常的做法是使用常规的文件系统工具（如 `cp`、`tar` 或 `rsync`），或者使用专门为此开发的程序（如 MySQL Enterprise Backup，这是一个商业程序）。在使用直接复制的备份方法时，需要注意如下两个关键点。

一定要确保表未被使用。当你正在复制某个表时，如果服务器正在更改它，那么复制操作就将毫无意义。为了保证备份的完整性，最好是先停止服务器，再复制文件，最后重启服务器。事实上，有好几种二进制备份方法都要求你停止服务器。如果不想停止服务器（并且备份方法也没有那样要求），那么可以使用只读锁定机制，防止服务器在你正在复制表文件时去修改它们。更多相关信息请参考14.2节。

为了能够恢复你正在备份的那些表，你必须复制恢复操作所需要的全部文件。直接复制法最适用于像 MyISAM 这样的存储引擎，在数据目录里它们都会使用一组具有唯一标识的文件来表示给定的表。如果想备份某个 MyISAM 表，只需复制它的 `.frm`、`.MYD` 和 `.MYI` 文件即可。对于 InnoDB 这样的存储引擎，事情会变得比较复杂：除了必须复制 `.frm` 文件以外，还必须复制所有的表空间文件和 InnoDB 日志文件。

在制作二进制备份时，一定要注意符号链接（如数据目录里指向各数据库目录的符号链接、指向 MyISAM 数据或索引文件的符号链接）可能带来的问题。之所以这会成为一个问题，是因为你使用的文件复制技术可能只会复制那些符号链接，而不会复制它们所指向的数据。

14.4.3.1 制作完整的二进制备份

一个完整的二进制备份至少应该包括存储表内容的所有文件和特定存储引擎所使用的各种日志文件。为保险起见，你还应该复制二进制日志文件。如果服务器是一个复制从服务器，就应该复制中继日志文件、`master.info` 文件和 `relay-log.info` 文件。另外，从服务器还可能会在其临时目录里创建以 `SQL_LOAD-xxx` 形式命名的文件。你应该把这些文件也备份起来，它们都是 `LOAD DATA` 语句所需要的。这些文件都可以在系统变量 `slave_load_tmpdir` 所指定的目录里找到。如果未给出这个变量，那么它默认为系统变量 `tmpdir` 的值。为了在备份时容易找到这些文件，可以创建一个专门让从服务器使用的

目录，并且在启动从服务器时，将`slave_load_tmpdir`设置为这个目录的路径名。

如果想要正确复制上面提到的所有文件，则必须停止服务器，并且必须是彻底关闭，这样各存储引擎才能关闭它们的日志文件，服务器才能关闭任何它正在写入的其他日志。

所有这些听起来让人感觉备份时要做很多事情，但实际上没那么复杂。例如，所有的数据库目录都在数据目录下，并且日志和信息文件都默认在那里创建。在这种情况下，可以通过停止服务器并复制整个数据目录的办法来制作备份。例如，想要创建一个备份，并将其压缩成tar文件，存放在`/archive/mysql`目录里，那么可以把当前位置切换到服务器数据目录，然后备份所有的内容。例如：

```
% cd /usr/local/mysql/data  
  
% tar czf /archive/mysql/backup-all-2013-04-11.tar.gz .
```

14.4.3.2 制作部分的二进制备份

通过复制文件的办法来制作部分的二进制备份，与制作完整的备份很相似，不同之处在于只需要复制整个文件集里的一个子集即可。假设，想要备份位于数据目录`/usr/local/mysql/data`里的`mydb`数据库，并且想把备份存储到归档目录`/archive/mysql`下。那么就可以先停止服务器，然后执行下面的命令：

```
% cd /usr/local/mysql/data  
  
% cp -r mydb /archive/mysql
```

在执行完这些命令之后，`/archive/mysql/mydb`目录下就会有一个`mydb`数据库的副本。

在备份完成之后，需要重启服务器。

在某些情况下，如果你使用只读锁定协议（read-locking protocol）锁定了你想要复制的表，那么可以不用停止服务器就能进行部分备份。例如，如果数据库只包含MyISAM表，就可以采用这种办法。如果前几个示例里使用的`mydb`数据库是这样的数据库，则可以在执行备份命令之前，先给那些表加只读锁，并刷新表，接着，在备份完成之后，释放这些表锁。

14.4.4 备份InnoDB表

对于InnoDB这样的事务型存储引擎，它的表也可以用mysqldump来转储，和其他类型的表一样。在这种情况下，`--single-transaction`选项很有用，它将使mysqldump在一个事务中转储表。对于InnoDB，这样可以确保表在转储期间不会被修改，从而保证了备份的一致性。

如果想要制作二进制InnoDB备份，则必须注意以下几个特殊要求。

InnoDB有其自己的事务管理日志文件，这些日志文件在服务器运行时是激活的。因此，如果想要制作二进制备份，则必须先停止服务器，而且必须是彻底关闭，不能是异常关闭，这样InnoDB才能完成尚未完成的事务，并正确关闭它的日志。

如果想要制作InnoDB表的二进制备份，则必须复制以下文件。

系统表空间文件。

每个表的.frm文件。

每个表的.ibd文件（如果你配置InnoDB时让它使用了单个表空间文件）。

InnoDB日志文件。

系统表空间配置所特有的选项文件。（请为这种选项文件创建一个副本，因为它可用于再次初始化系统表空间。如果不这样做，那么一旦当前选项文件丢失，你将痛苦不堪。）

你也可以使用MySQL Enterprise Backup来制作InnoDB表的二进制备份。MySQL Enterprise Backup是Oracle公司的一个商业程序，它能让你在服务器处于运行状态时进行备份。更多详细信息，请访问<http://www.mysql.com>。

14.5 将数据库复制到另一个服务器

可以通过数据库备份来把某个数据库从一个MySQL 服务器复制到另一个。本节将介绍几种迁移数据库的办法。为便于讨论，我假设要把一个数据库从本地主机的服务器上迁移到远程主机boa.example.com的服务器上。不过，这两个服务器也可以运行在同一台主机上。虽然下面的讨论描述的是如何复制整个数据库，但是你也可以把这些技术用于复制单个表。

接下来讨论如何使用把数据库复制到另一个服务器的两种方法。第一种方法是，把数据库备份为一个文件或一组文件。你可以把这些文件复制到第二台服务器主机上，并把它们加载到第二台主机的MySQL 服务器里。第二种

方法是，通过网络把数据库从一个服务器直接转储到另一个服务器。使用这种方法无需使用任何中间文件。

14.5.1 使用备份文件来复制数据库

使用文本备份文件来复制数据库的操作流程是：先用mysqldump程序创建备份文件，然后将其复制到第二台服务器主机上，最后将其加载到该主机的MySQL 服务器里。下面这个示例演示了如何使用这一流程来复制sampdb数据库。

(1) 创建转储文件：

```
% mysqldump --databases sampdb > sampdb.sql
```

其中，--databases选项将使mysqldump程序为sampdb数据库增加CREATE DATABASE IF NOT EXISTS和USE 语句。这样一来，当你在远程主机上加载这个转储文件时，它就会自动创建sampdb数据库，并把它设置为默认数据库，如此便可以把转储表加载到该数据库里。(2) 把转储文件复制到远程主机。下面这条命令将使用scp 程序，把文件复制到boa.example.com主机的/tmp目录里：

```
% scp sampdb.sql boa.example.com:/tmp
```

(3) 登录远程主机，把转储文件加载到它的MySQL服务器里：

```
% mysql < /tmp/sampdb.sql
```

另一个办法是使用二进制备份技术，即把数据库文件（不是转储文件）从一台主机复制到另一台主机。假设mydb数据库的表全部为MyISAM表，那么表信息就会全部包含在mydb数据库目录下的那些文件里。如果本地数据目录为/usr/local/mysql/data，并且远程主机boa.example.com上的数据目录为/var/mysql/data，那么下列命令将把mydb数据库的目录复制到该主机上：

```
% cd /usr/local/mysql/data
```

```
% scp -r mydb boa.example.com:/var/mysql/data
```

如果想要用这种方式把数据库文件复制到另一台主机上，就必须满足以下要求。

两台机器必须拥有相同的硬件架构，或者你所复制的那些表都使用了二进制可移植的存储引擎。否则，第二台主机上的结果表可能会出现非常奇怪

的内容。

你必须阻止两台主机上的两个服务器尝试更改你正在复制的表。最安全的做法是，在处理表时，停止这两个服务器。

14.5.2 将数据库从一个服务器复制到另一个

上一节展示的mysqldump技术会创建一个转储文件，并把它复制到目标服务器的主机上。另外，你也可以通过网络把mysqldump的输出直接写到另一个服务器，从而不需要中间文件。例如，下面这条命令将把sampdb数据库从本地主机复制到boa.example.com主机的服务器上：

```
% mysqldump --databases sampdb | mysql -h boa.example.com
```

这里，mysql程序会读取转储输出，连接boa.example.com主机上的服务器，并把转储内容加载到该服务器。

如果不能从本地主机访问远程 MySQL 服务器，但是可以通过远程登录进行访问，那么可以使用ssh来远程调用mysql程序：

```
% mysqldump --databases sampdb | ssh boa.example.com mysql
```

如果网络速度慢，那么在把数据库复制到另一台机器时，可以使用--compress 选项来改善性能，因为它能够减少网络传输的数据量：

```
% mysqldump --databases sampdb | mysql --compress -h boa.example.com  
sampdb
```

请注意，--compress选项适用于与远程主机的服务器进行通信的程序，而不适用于与本地主机的服务器进行通信的程序。这里的压缩仅限于网络传输，目标数据库里创建的表不会被压缩。

14.6 检查和修复数据库表

数据库受损的原因很多，受损程度也大不一样。如果运气好，你可能只会遇到一两个表轻微受损的情况（如机器因为断电而关闭）。在这种情况下，服务器在恢复正常运行时通常能够修复这种损坏。如果你比较倒霉，那么就得恢复整个数据目录（如存有数据目录的硬盘报废了）。在其他一些情况下（例如，用户错误地删除了数据库、表或者表的内容），可能也需要恢复。

如果你怀疑某个表被破坏了，则需要对它进行错误检查。如果表没查出任何问题，那么事情就到此为止；否则，请按以下指南进行修复。

先使用一种快速但不那么全面的修复方法。

如果这个方法不管用，那就试试那些更全面（但较慢）的修复方法，直到损坏得到修复或者再也找不到更好的方法为止。

实际上，绝大多数问题都很容易解决，用不着求助于较全面的、较慢的修复方法。

本节描述的表检查和修复过程适用于处理轻微受损的情况。对于更为严重的问题，如表或数据库丢失或无法修复，则需要使用数据库备份和二进制日志来恢复它们。具体操作方法请参考14.7节。

下面大概介绍一下检查和修复InnoDB和MyISAM表的几种办法，随后会对它们进行详细说明。

如果要检查InnoDB表，可以使用CHECK TABLE语句，或者使用mysqlcheck程序（它会替你连接服务器，并执行这条语句）。

如果要修复发现有问题的InnoDB表，可以先使用mysqldump把它转储成文件，然后删除这个表，重新加载那个转储文件来重新创建它。下面的命令序列展示了如何检查、转储和重新加载sampdb数据库里的absence表：

```
% mysqlcheck sampdb absence
```

```
% mysqldump sampdb absence > absence.sql
```

```
% mysql sampdb < absence.sql
```

如果要检查和修复MyISAM表，可以选择以下方法。

使用CHECK TABLE和REPAIR TABLE语句，或者使用mysqlcheck程序（它会替你连接到服务器，并执行这些语句）。

使用myisamchk程序，它可以直接操作表文件。

正如本章前面提到的，在进行表维护操作时，如果你可以选择，是让服务器来做，还是让外部工具程序来做，那么让服务器来做会更简单。这样，你便不用考虑使用锁定协议来协调表的访问了。接下来的几节将描述如何使用CHECK TABLE、REPAIR TABLE和mysqlcheck来获得这种好处。

如果在服务器运行的情况下使用myisamchk程序，就必须使用锁定协议（参考14.2节）。不过，myisamchk程序还提供了以下好处。

当服务器停止时，你可以使用myisamchk。当服务器正在运行时，需要使用CREATE TABLE和REPAIR TABLE。

你可以让myisamchk使用更大的缓冲区，从而加快检查和修复操作的速度。当你的表非常大时，这样做会很有帮助。

关于使用myisamchk进行维护的信息请参考F.3节。

14.6.1 使用CHECK TABLE检查表

CHECK TABLE语句为我们提供了一个对服务器的表进行检查的接口。可用它来检查InnoDB、MyISAM、ARCHIVE和CSV表，还可用它来检查视图。

如果要使用CHECK TABLE，需要列出一个或多个表名，后面可以选择性地加上限定符，用以指明你想进行何种类型的检查。例如，下面这条语句将对3个表进行中等级别的检查，但只有在它们未被正确关闭时才会进行检查：

```
CHECK TABLE tbl1, tbl2, tbl3 FAST MEDIUM;
```

下面是对几种可用检查选项的描述。它们适用于MyISAM表。其他存储引擎可能会忽略它们。更多相关信息请参考附录E。

CHANGED。它表示的是，如果表是正常关闭的，并且自上次检查以来未被更改，则跳过表检查。

EXTENDED。它表示的是，执行扩展检查，以试图确保表完全一致。

FAST。它表示的是，只有在表未正常关闭时，才检查它。

MEDIUM。它表示的是，通过检查索引、扫描数据行的问题，并进行校验。如果未给出选项，则默认为它。

QUICK。它表示的是，只扫描索引，而不扫描数据行。

FOR UPGRADE。它表示的是，确定被检查表是否与当前的MySQL版本相兼容。此选项在升级后很有用。

14.6.2 使用REPAIR TABLE修复表

REPAIR TABLE 语句为我们提供了一个对服务器的表进行修复的接口。可用它来修复MyISAM、ARCHIVE和CSV表。

如果要使用REPAIR TABLE，需要列出一个或多个表名，后面可以选择性地加上限定符，用以指明你想进行何种类型的修复。例如，下面这条语句将对3个表进行快速修复：

```
REPAIR TABLE tbl1, tbl2, tbl3 QUICK;
```

下面是对几种可用修复选项的描述。这些选项适用于MyISAM表。其他存储引擎可能会忽略它们。更多相关信息请参考附录E。

EXTENDED。它表示的是，执行扩展修复，重建索引。

QUICK。它表示的是，只尝试快速修复索引，而不修复数据文件。

USE_FRM。它所表示的是，使用表的.frm 文件，重新初始化索引文件，并确定如何解释数据文件的内容，以便能够重建索引。如果索引丢失或损坏后不可恢复，则此选项非常有用。不过，它应该作为最后的打算，并且只有在当前的MySQL版本与创建表时所使用的MySQL版本一样时才能使用它；否则，有可能会进一步破坏表。

如果没有选项，那么 REPAIR TABLE 会执行常规的表修复操作，这样也能修复大部分的问题，只不过，在只应包含唯一值的索引里会出现重复值。

14.6.3 使用mysqlcheck检查修复表

mysqlcheck程序提供了CHECK TABLE和REPAIR TABLE语句的命令行接口。它会连接服务器，并根据你指定的选项执行相应的语句。因此，mysqlcheck能够检查或修复的表的存储引擎类型，与前面描述的CHECK TABLE和REPAIR TABLE的一样。

通常情况下，在调用mysqlcheck 时，后面需要带上一个数据库名，然后选择性地跟上一个或多个表名。如果只给出了数据库名，那么mysqlcheck将检查数据库里的所有表。

```
% mysqlcheck sampdb
```

如果在数据库名的后面带上表名，那么mysqlcheck将只会检查那些表：

```
% mysqlcheck sampdb president member
```

如果指定了--databases选项，那么mysqlcheck将把所有非选项性参数解释为数据库名，并检查每个数据库里的所有表：

```
% mysqlcheck --databases sampdb test
```

如果指定了--all-databases选项，那么mysqlcheck将检查所有数据库里的所有表。这时，不需要提供任何数据库名或表名参数：

```
% mysqlcheck --all-databases
```

与直接执行CHECK TABLE和REPAIR TABLE语句相比，使用mysqlcheck更加方便，因为那些语句需要你显式地给出每一个你想检查或修复的表的名字。如果需要对某个数据库里的所有表进行检查，那么使用mysqlcheck会更加简单：它会替你找出数据库中表名，并调用与各个表相对应的语句。

默认情况下，mysqlcheck 将对表进行中等检查，但是可以使用选项显式地选择具体的操作类型。下表列出了一些 mysqlcheck 选项和与之相对应的CHECK TABLE 语句的选项。对于CHECK TABLE语句，这些选项只适用于MyISAM表，其他存储引擎可能会忽略它们。

mysqlcheck选项	CHECK TABLE选项
--check-only-changed	CHANGED
--extended	EXTENDED
--fast	FAST
--medium-check	MEDIUM
--quick	QUICK

对于MyISAM、ARCHIVE和CSV表，mysqlcheck 还可以执行表修复操作。下表列出了一些mysqlcheck选项和与之相对应的REPAIR TABLE语句的选项。对于REPAIR TABLE语句，这些选项只适用于MyISAM表，其他存储引擎可能会忽略它们。

mysqlcheck选项	REPAIR TABLE选项
--repair	无选项（执行标准的修复操作）
--repair --extended	EXTENDED
--repair -quick	QUICK
--repair --use-frm	USE_FRM

14.7 使用备份来恢复数据

数据恢复过程涉及两个信息源：备份文件和二进制日志。备份文件既可以是使用像mysql dump这样的工具生成的转储文件，也可以是使用某种二进制备份方法复制出来的文件。

备份文件能够把表恢复到备份时的状态。二进制日志文件里记载着自该次备份之后所执行过的表更改语句。mysqlbinlog可以把这些日志文件转换回文本形式的SQL语句，因此可以使用mysql来执行它们，进而重新应用在备份和问题出现这两个时间点之间所产生的更改。

数据恢复过程因需要恢复的信息量不同而有所差异。事实上，恢复数据库通常比恢复单个表更加容易，因为与应用表的二进制日志相比，应用数据库的二进制日志会更容易。

本节会假设你已经做了数据库的备份，并且启用了二进制日志功能。如果不是这样，那么你将身处险境。在继续往下读之前，你应该立刻启用二进制日志功能，并制作一份新的备份。你肯定不想看到这样的情况发生：因懒惰而没有保存恢复所需要的信息，从而导致表在弄丢之后再也无法找回。如果要启用二进制日志功能，请参考 12.8.4 节。如果要制作备份，请参考 14.4节。

14.7.1 恢复整个数据库

恢复一个或多个数据库的基本步骤如下所示。

（1）复制一份数据库目录的内容。如果以后在恢复过程中做错了什么，或者有什么地方出现了问题，那么你可能会需要这些内容。

（2）使用最近的备份文件重新加载数据库。

如果你的备份是用mysql dump生成的转储文件，那么请依次将每个文件作为mysql的输入，重新加载它们。

如果需要恢复的数据库里有包含权限表的mysql数据库，并且你打算使用转储文件来恢复那些表，那么可以在服务器运行时，使用`--skip-grant-tables`选项来重新加载它们。否则，服务器会抱怨“无法找到权限表”。使用`--skip-networking`选项让服务器在恢复过程中拒绝所有的远程连接，这也是一种不错的做法。在你把表都恢复好之后，需要关闭服务器，再重启它，这样它便可以像往常一样使用这些权限表和监听网络接口。

如果你使用的文件源自二进制备份（如使用tar或cp做的备份），那么就要停止服务器，这样才能保证在恢复数据库时，服务器不会去访问它们。然后，再把备份文件复制到它们原来的位置（通常是在数据目录下），并重启服务器。

（3）利用二进制日志，重新应用在制作备份之后发生的数据更改。具体过程请参考14.7.3节。

14.7.2 恢复单个表

恢复单个表往往要比恢复数据库更难。如果你有一个由mysqldump生成的转储文件，其中只包含那个表，那么只需重新加载这个文件即可。如果转储文件包含有多个表的数据，可以采用这样的方法来恢复其中的某个表：先编辑这个文件，删除其他表的数据，然后再重新加载剩余的内容。这部分很容易。

恢复操作中更难的部分是，需要在二进制日志里把那些与这个表有关的内容提取出来。mysqlbinlog有一个`--database`选项，它可以把输出限制为单个数据库的某些语句，但是没有适用于单个表的选项。在这种情况下，比较有用的策略可能是：首先，恢复更多的内容（如包含这个表的整个数据库），这样肯定会超出实际所需；然后，删除不需要的内容。与只从二进制日志里提取部分内容恢复单个表的做法相比，下面这个操作过程实际更可行。

（1）先找到包含你想要的那个表的数据库，恢复其全部内容，不过这次是恢复到第二个空的数据库。实现方法是：利用备份，重新应用二进制日志。不过，这种做法也会产生两个问题。

mysqldump生成的转储文件可能包含一条用于初始数据库的USE语句。在将这个转储文件用作mysql的输入之前，需要先更改该语句，或者将其删除。

mysqlbinlog的输出可能包含一条或多条初始数据库的USE语句。在将这个文件用作mysql的输入之前，可以先把这个输出保存为一个文件，这样就可以编辑那些USE语句，将其中的数据库换成第二个数据库。

(2) 从第二个数据库开始，使用mysqldump把你感兴趣的那个表转储成文件。

(3) 删除原来的表，并把那个转储文件加载到原来的数据库，重建那个表。如果运行mysqldump 时使用了--opt 或--add-drop-table 选项，那么转储文件里会多出一条 DROP TABLE语句，其作用是在重建该表之前先删除它。

对于 MyISAM表，还有一个办法，即使用mysqldump 把第二个数据库目录里的表文件直接复制到原来的那个数据库目录。为安全起见，在进行复制操作时一定要停止两个服务器。

14.7.3 重新执行二进制日志文件里的语句

在使用备份文件恢复数据库或表之后，还应该重新应用二进制日志文件（它们包含的是自备份之后所有执行过的语句）里的部分内容。这样可以把表恢复到最新状态。

mysqlbinlog 程序可以把二进制日志文件转换为文本形式的语句，使其易于执行：将mysqlbinlog的输出用作mysql的输入。

根据你要从备份中恢复的内容，你可能需要应用二进制日志文件里的所有语句，也可能只需要应用与某个特定数据库有关的语句。你可能还需要选择那些只在某个特定时段执行过的语句。这些事情都可以交给mysqlbinlog去完成。它可以处理多个二进制日志文件，并且可以把自己的输出限制为只与给定数据库或给定时间段有关的语句。

在接下来对应用二进制日志文件的指导里，会假设二进制日志文件的命名格式为binlog.nnnnnn。其中，nnnnnn是一个由六位数构成的扩展名，表示日志文件的序号。如果你的二进制日志文件的基本名不是binlog，那么需要根据实际情况来调整指导内容。此外，这里重点关注的是本地二进制日志文件的使用，这些文件在你执行mysqlbinlog程序的主机上。这个程序也可以读取远程的二进制日志文件，但这里不对它进行讨论。与mysqlbinlog的远程日志处理选项有关的更多详细内容请参考附录F。

如果恢复的数据库是在所有当前二进制日志文件写入之前备份的，那么你还需要应用所有二进制日志文件的内容。具体做法是在这些二进制日志文件所在的目录里，执行下面这条命令：

```
% mysqlbinlog binlog.[0-9]* | mysql
```

在mysqlbinlog命令里，binlog.[0-9]*模式会扩展成二进制日志文件列表，其顺序通常与服务器生成的顺序一样。

如果必须在重新执行日志文件之前对它们进行编辑，那么可以先把它们转换为文本格式，保存在一个文件里。然后，再编辑这个文件，将结果送入mysql。示例如下：

```
% mysqlbinlog binlog.[0-9]* > text_file
```

```
% vi text_file
```

```
% mysql < text_file
```

如果因为有人不小心执行了 DROP DATABASE、DROP TABLE 或 DELETE 语句，而需要执行恢复操作，以及使用日志文件恢复信息，那么就必须使用上面这个策略。在执行日志文件的内容之前，必须先把那条语句从日志文件里移除。

请不要使用mysqlbinlog和mysql 一个接一个地处理二进制日志文件。如果不把它们当作一个整体来处理，那么它们之间的依赖关系可能会被破坏掉。例如，某个日志文件里创建的TEMPORARY表，有可能会被后面的日志文件使用到。如果单独处理各个日志文件，那么在相应的mysql 调用完毕时，每个日志文件所创建的TEMPORARY 表都会被自动删除，无法用在后面日志文件的语句中。

如果只想提取那些只属于特定数据库的语句，则可以使用mysqlbinlog的--database选项：

```
% mysqlbinlog --database=db_name binlog.[0-9]* | mysql
```

mysqlbinlog还有几个选项，可用于提取在特定时段执行过的语句（如在给定备份之后写入的语句）。你可能需要查看一下日志文件的内容，以决定使用什么样的选项。下面是mysqlbinlog的一个输出样本（为适应页面宽度，有些注释行有所删减）：

```
...
```

```
# at 1077
```

```
#121030 16:50:36 server id 1    end_log_pos 106      Query....
```

```
SET TIMESTAMP=1351633836;
```

```
INSERT INTO absence VALUES (3,'2012-09-03');
```



```

# at 1183

#121030 16:50:36 server id 1  end_log_pos 1210  Xid = 386

COMMIT;

# at 1210

#121030 16:50:36 server id 1  end_log_pos 106  Query....

SET TIMESTAMP=1351633836;

INSERT INTO absence VALUES (5,'2012-09-03');

# at 1316

#121030 16:50:36 server id 1  end_log_pos 1343  Xid = 387

COMMIT;

# at 1343

#121030 16:50:36 server id 1  end_log_pos 107  Query....

SET TIMESTAMP=1351633836;

INSERT INTO absence VALUES (10,'2012-09-06');

# at 1450

#071030 16:50:36 server id 1  end_log_pos 1477  Xid = 388

COMMIT;

...

```

假设，想要重新应用从 2012-10-30 16:50:36 开始记录到二进制日志里的修改。下面两种形式都可以把该时间值指定给--start-datetime选项：

```
% mysqlbinlog --start-datetime="2012-10-30 16:50:36" binlog.[0-9]* | mysql
```

```
% mysqlbinlog --start-datetime=20121030165036 binlog.[0-9]* | mysql
```

有一个与之对应的`--stop-datetime`选项，可用于给定截止时间。另外，还有几个基于位置的选项，它们可以接受显示在日志里的位置值。相关信息请参考附录F中对`mysqlbinlog`的描述。

14.7.4 应对InnoDB的自动恢复问题

如果MySQL服务器或者服务器主机崩溃，那么MySQL服务器重启时，InnoDB存储引擎会自动恢复。本节讨论的内容是：当自动恢复遭遇罕见的失败时，该如何处理。

如果InnoDB在服务器启动的过程中，检测到一个无法恢复的问题，那么它会往出错日志里写入一条信息，然后服务器会退出。在这种情况下，把系统变量 `innodb_force_recovery` 设置为1~6之间的某个非零值可以强行启动服务器。如果想要设置这个变量，可以在服务器选项文件的`[mysql]`组里加上一行，并重启服务器：

```
[mysqld]
```

```
innodb_force_recovery=level
```

InnoDB存储引擎默认使用的自动恢复策略比较保守，使用的`level`值都较小。通常推荐使用的启动值为4。在启用这个变量之后，InnoDB会允许添加或删除表，但它们的内容是只读的。在启动服务器之后，应该先用`mysqldump`转储InnoDB表，以尽量多地获得信息。然后从选项文件里删除设置 `innodb_force_recovery` 变量的那行，并正常重启服务器，以便可以重建表的内容，重建方法是：删除表，并根据`mysqldump`的输出文件恢复它们。按照这一过程重建出来的InnoDB表可以保证其内部数据的一致性，并且足以获得令人满意的恢复效果。

如果你使用的是MySQL 5.6.3及以后的版本，那么与 `innodb_force_recovery`一起使用的另一个有用设置是 `innodb_force_load_corrupted`。它会告诉InnoDB，即使这些表被标记为受损，也要加载它们。在某些情况下，这样可以让你获得那些通常会被忽略的表的转储内容。

如果需要恢复所有的InnoDB表，那么必须使用备份。所采用的具体方法取决于备份的类型。

如果是二进制备份，那么你应该具备这样几个文件的副本：系统和单个的表空间文件、InnoDB日志文件、各个表的`.frm`文件，以及用来定义你

InnoDB配置的选项文件。在确认服务器已停止之后，删掉已有的InnoDB文件，把它们替换为备份副本。然后，确保当前服务器的选项文件列出的InnoDB配置，与你保存的选项文件一致。最后，重启服务器。

如果是采用运行mysqldump生成转储文件的方式来备份InnoDB表，那么你应该重新初始化系统表空间和InnoDB日志文件，并把这个转储文件重新加载到InnoDB。

(1) 停止服务器并删除任何已有的与 InnoDB有关的文件，其中包括系统表空间文件、单个表空间文件（不包括原始分区）、InnoDB日志文件，以及所有InnoDB表的.frm文件。

(2) 按原先的设置重新配置系统表空间，并重启服务器。然后，InnoDB存储引擎将重新创建它的系统表空间和日志文件。相关说明请参考 12.5.3.1节。记得，如果使用了原始分区，那么对表空间的初始化将包含两个步骤。

(3) 采用将转储文件当作 mysql 输入的方式重新加载它们。这样便会重建那些InnoDB表。

在利用备份恢复 InnoDB表之后，还需要把二进制日志里自这次备份之后所发生的更新重新应用一下（参考14.7.3节）。如果把InnoDB表的恢复操作作为整个数据库恢复操作的一部分，那么事情会简单很多，因为在这种情况下，你只需要把该次备份后所做的更新应用一遍即可。如果只是恢复InnoDB表，那么应用日志文件会比较麻烦，因为你应该使用只针对那几个表的更新。

14.8 设置复制服务器

实现数据库“复制”（replication）的一种形式是，把数据库简单地复制到另一个服务器。可是，如果原始数据库的内容发生了变化，而你又想让那个副本同样保持最新，那么以后你就不得不重复这个复制操作。如果想让辅助数据库能够随着主数据库内容的变化持续保持更新，那么需要用到MySQL的实时复制功能。它可以让你拥有数据库的副本，并保证原始数据库的更改会自动、及时地传播到这个副本上。

14.8.1 复制的工作原理

在MySQL里，数据库的复制原理如下。

在复制关系中，有两种服务器，一种为主服务器，另一种为从服务器。每个服务器都会分配一个唯一的复制ID。

每个主服务器可以有多个从服务器。一个从服务器可以作为另一个从服务器的主服务器，从而形成一个复制服务器链。

每个从服务器在开始时，其数据库都必须与主服务器上的数据库同步。也就是说，当复制开始时，任何复制到从服务器上的数据库都一定是主数据库的同等副本。之后，在主服务器上所做的更新会传播到从服务器。所有更新不应该直接作用到从服务器里的复制数据库。

更新通信是基于主服务器的二进制日志的，其中记录了需要发送到从服务器的各种更新。因此，在主服务器上必须启用二进制日志功能。存储在二进制日志里的更新被称为“事件”（event）。

每个从服务器都必须有权限连接主服务器，并请求更新。当从服务器连接到它的主服务器时，它会告诉主服务器，自它上次连接以来在主服务器的二进制日志里已进展了多远。这个进展是以复制坐标的方式来表示的，即二进制日志文件名和在该文件里的位置。然后，主服务器将把在二进制日志里记录的、自给定坐标之后发生的事件传送给从服务器。当从服务器读完所有的事件时，就会暂停并等待更多的事件。

当主服务器上有新的更新发生时，它会把这些更新写入主服务器的二进制日志，以便以后再传输给从服务器。

主服务器处理连接从服务器的方式与它处理常规客户端的方式很像，而从服务器的最大连接个数可以通过系统变量`max_connections`来设置。

在从服务器上，服务器会使用两个线程来完成复制任务。I/O线程负责接收来自主服务器的待处理事件，并把它们写到从服务器的中继日志（relay log）。SQL线程负责从中继日志中读出事件，并执行它们。中继日志相当于I/O线程向SQL线程传输数据更新信息的通信桥梁。在处理完一个中继日志文件之后，从服务器会自动将其删除。I/O线程和SQL线程的操作互不影响，它们可以单独启动和关闭。两个线程在功能上的这种“脱节”能带来很大的好处。例如，当你停止了SQL线程时，I/O线程可以继续读取来自主服务器的事件，这样，在你进行备份时，从服务器里便不会有更新操作发生。

复制支持还是一个正在蓬勃发展的领域，因此有时很难跟踪到何时加入了何种与复制有关的功能。你应该考虑到不同服务器版本之间的兼容性问题。总之，我建议你能遵循以下几条指导原则。

在某个给定的MySQL版本系列（如5.5、5.6等）里，尽量使用最新的版本。这样，你可以从中受益良多：获得最丰富的功能集，最大限度地移除限制，消除问题。

尽量让主、从服务器使用的二进制日志格式相匹配。例如，让5.5版本的主服务器与5.5版本的从服务器相匹配，而不要让5.5版本的主服务器与5.1版本的从服务器相匹配；反之亦然。如果主、从服务器的版本不一样，那么需要从旧版的主服务器向新版的从服务器进行复制，反向操作则不行。

主、从服务器在功能上必须兼容。例如，如果主服务器使用的表所要求的存储引擎在从服务器上无法使用，那么你会遇到很多问题。

14.8.2 建立主从复制关系

下面这个流程展示的是如何在两个服务器之间建立主从复制关系。

(1) 确定你想要分配给每个服务器的ID值，并把它们记录到服务器在启动时会读取的选项文件里。这些ID值都必须各不相同，并且每一个都为 $1 \sim 2^{32}-1$ 的整数。这些ID值是每个服务器的启动选项`server_id`所需要的。此外，如果主服务器上未启用二进制日志功能，则需要启用它。如果要想分别在主、从服务器上完成这个操作，则可以在相应的选项组里加上下面几行内容：

```
[mysqld]

server-id=master_server_id

log-bin=binlog_name

[mysqld]

server-id=slave_server_id
```

重启两个服务器，以便使更改生效。

(2) 在主服务器上，创建一个账户，让从服务器可以用它来连接主服务器，并请求更新信息：

```
CREATE USER 'slave_user'@'slave_host' IDENTIFIED BY 'slave_pass';

GRANT REPLICATION SLAVE ON *.* TO 'slave_user'@'slave_host';
```

请记住`slave_user`和`slave_pass`的值，稍后在从服务器连接主服务器时会使用到。如果这个账户只是单纯地用于达到复制的目的，那么不需要再设置其他的权限。不过，如果想在从服务器主机上使用mysql以“手动方式”连接主服务器进行测试，那么可能需要给这个账户授予更多权限。这样，你就

不会感到束手束脚了。（例如，如果只为这个账户授予了REPLICATION SLAVE权限，你可能就无法使用SHOW DATABASES语句来查看主服务器上的数据库名。）

（3）连接主服务器，并通过执行SHOW MASTER STATUS语句确定其当前的复制坐标：

```
mysql> FLUSH TABLES; SHOW MASTER STATUS;
```

File	Position	Binlog_Do_DB	Binlog_Ignore_DB
binlog.000093	1707		

请记住File和Position的值。因为稍后还需要用它们，以便能够把从主服务器读取二进制日志事件的起始位置告知从服务器。

注意

确保自你确定其复制坐标之时起，直到其数据快照传输到从服务器之后的这段时间里，在主服务器上都没有发生任何更新。

（4）必须在从服务器上为将要复制的数据库建立一个副本。通过把主服务器上的数据库复制到从服务器的方式，实现主、从服务器之间的初始同步。为此，一种可行的办法是，在主服务器主机上进行备份，然后把这个备份加载到从服务器上去。另一种办法是，通过网络把所有数据库从主服务器复制到从服务器。与数据库备份和复制技术有关的内容请参考本章的前面部分。

如果你还没有在主服务器上创建任何数据库或表，那么可以省略这一步，因为没有快照可拍。

（5）连接从服务器，并使用CHANGE MASTER语句来配置它。配置包括用于连接主服务器的参数和初始复制坐标。配置语句如下所示：

```
CHANGE MASTER TO
```

```
MASTER_HOST = 'master_host',
```

```
MASTER_USER = 'slave_user',  
  
MASTER_PASSWORD = 'slave_pass',  
  
MASTER_LOG_FILE = 'log_file_name',  
  
MASTER_LOG_POS = log_file_pos;
```

其中，'master_host'是主服务器所在主机的名字。'slave_user'和'slave_pass'应该是之前在主服务器创建的那个账户的用户名和密码，当从服务器连接主服务器并请求主服务器更新信息时会使用到它们。'log_file_name'和'log_file_pos'是通过 SHOW MASTER STATUS语句获得的值。

在Unix系统里，MySQL会使用套接字文件来连接localhost，但是复制操作不支持使用套接字文件。因此，如果主、从服务器都运行在同一台主机上，那么应该使用127.0.0.1来代替localhost，从而确保从服务器会使用TCP/IP连接。

如果主服务器没有监听默认端口，那么可以在CHANGE MASTER 语句里包括一个MASTER_PORT选项来指明端口号。

(6) 让从服务器开始复制。

```
START SLAVE;
```

从服务器将连接主服务器并开始复制。你可以在从服务器上使用SHOW SLAVE STATUS 语句来查看它的工作状态。

从服务器会把CHANGE MASTER参数存储在其数据目录下的一个名为master.info的文件里，用以记录初始复制状态，并且会随着复制的进行不断更新这个文件。如果你以后需要改变复制参数，那么可以连接到从服务器，并再次使用CHANGE MASTER语句。从服务器会用这些新的更改自动更新master.info文件。

存储在master.info文件里的信息包括用于连接主服务器的用户名和密码。这些信息应该是保密的，因此需要确保只让从服务器上的MySQL 管理员的登录账户访问这个文件。例如，可以根据13.1.2节的描述，锁死数据目录的内容。

上面描述的这个流程有一个前提假设，即你想要将主服务器上的所有数据库复制到从服务器。不过，也有可能你并不想让两个服务器使用相同的账

户信息。（例如，你可能会想建立一个私用的复制从服务器，使得即使在主服务器上拥有账户的人也不能随意连接这个从服务器。）如果要在从服务器维护独立于主服务器的账户，需要做下面两件事情。

（1）当把数据库的初始数据快照从主服务器传输到从服务器时，请不要把 mysql 数据库包括在内；或者，在传输之前先备份从服务器的mysql数据库，传输完成之后再恢复它。

（2）让从服务器忽略来自主服务器的mysql数据库的更新。在从服务器的选项文件里加上下列内容，可以做到这一点：

```
[mysqld]
```

```
replicate-wild-ignore-table=mysql.%
```

如果想让从服务器忽略多个数据库里的表，可以多次使用此选项，对每个数据库使用一次。

通过在主服务器的选项文件里使用--binlog-ignore-db 选项，可以在主服务器上（而非从服务器上）排除数据库。这样做的好处是可以减少主、从服务器之间的传输量，但缺点是会导致主服务器上的二进制日志文件会将那些被忽略数据库的信息排除在外。而这些信息在主服务器崩溃后进行数据恢复时是必需的，因此在从服务器上排除数据库的做法更可取。

在建立好复制关系，并开始运作之后，还有几条语句可用于对主、从服务器进行监视或控制。关于这些语句的详细内容请参考附录E。下面是对它们的简要介绍。

SHOW SLAVE STATUS语句。它运行在从服务器上，用于查看复制工作是否正常，以及当前的复制坐标。这些复制坐标可以用于确定主服务器已不需要哪些二进制日志文件。

PURGE BINARY LOGS 语句。它主服务器上运行，用于清理过期的二进制日志文件。可以在每个从服务器执行完SHOW SLAVE STATUS之后使用它，用于确定哪些日志文件不再需要。

STOP SLAVE和START SLAVE语句。它们可用于挂起和继续从服务器上与复制有关的活动。例如，当你正在进行备份时，可以用这些语句让从服务器处于静止状态（参考14.8.4节）。

如前所述，从服务器内部使用了两个线程来管理复制活动。I/O线程负责与主服务器进行通信，接收来自它的更新，并把接收到的更新写到从服务器

的中继日志。SQL线程负责读取中继日志，并执行它所找到的更新。可以通过在STOP SLAVE 或 START SLAVE 语句的末尾加上IO_THREAD或SQL_THREAD，分别挂起或者继续每一个线程。例如，STOP SLAVE SQL_THREAD命令会使从服务器停止执行中继日志里的更新，但从服务器仍可以继续读取来自主服务器的更新，并把它们记录到中继日志。

中继日志文件与二进制日志文件很相似，都是按数字编号生成的。也有一个类似于二进制日志索引文件的中继日志索引文件。默认的中继日志文件和索引文件都在服务器的数据目录里，分别为HOSTNAME-relay-bin. nnnnnnn和HOSTNAME-relay-bin. index。这些默认值可以通过服务器启动选项--relay-log和--relay-log_index来加以改变。更多相关信息请参考12. 8. 5。

14. 8. 3 二进制日志格式

默认情况下，服务器会以SQL语句的形式把数据修改事件写入二进制日志。这被称为基于语句的二进制日志记录（而基于它的复制叫做基于语句的复制）。服务器也可以把更改记录为单独的数据行。这是基于行的日志记录。第三种类型的日志记录是混合格式的，它允许服务器根据实际需要在基于语句的日志记录或者基于行的日志记录之间来回切换。

一般情况下，基于语句的日志记录产生的日志文件比较短小，内容也更易于理解。基于行的日志记录提供了更为细致的更新规范，当原始语句为非确定的，并且在主、从服务器上可能会产生不同效果时，使用这种类型的日志记录有利于复制。

如果要指定日志格式，可以在启动或者运行时设置系统变量binlog_format。其中，允许设置的值有STATEMENT、MIXED和ROW。

14. 8. 4 使用复制从服务器来制作备份

如果你建立了复制从服务器，那么它可以帮助你解决作为 MySQL 管理员所面临的各种取舍问题。

一方面，把服务器的作用发挥到极致很重要，能让用户社区的所有成员都享受到它提供的便利，其中包括允许他们对数据库进行更新。

另一方面，备份也很重要，在进行备份时，最好要防止任何人对数据库进行更改。此外，就恢复效果而言，如果在备份过程中停止服务器或者一次性锁定所有表，并保证备份文件和日志文件的检查点是同步的，那么就可以使备份的作用发挥到最大。

允许随时访问服务器，与在备份时强制全部或部分客户端失去对数据库进行访问的能力，这两者的目标存在冲突。复制从服务器提供了一种摆脱此

困境的解决办法，即备份对象从主服务器转为从服务器。具体操作方法是，先停止从服务器，或者暂停它上面的复制。然后，重启从服务器，或者继续复制，而从服务器会补上备份期间由主服务器所做的一切更新。这样一来，你便不用停止主服务器，或者在备份期间不让客户端对其进行访问了。

下面列出了几种可能的对从服务器进行备份的策略。

如果要对从服务器上的所有数据进行二进制备份，则需要停止从服务器，然后遵循14.4.3.1节给出的指导，并重启服务器。

如果要使用一种无需停止从服务器的备份方法，如使用mysqldump，那么可以在备份期间停止SQL线程，并且在备份完成之后重启该线程。具体操作流程是：先用STOP SLAVE SQL_THREAD语句暂停从服务器上的复制，并且刷新其日志；然后开始备份，并用START SLAVE语句继续复制操作。这样一来，在备份数据库期间，从服务器不会对它们进行任何更改。I/O线程可以继续保持运行，它会把从主服务器接收到的更新事件继续写入中继日志。在备份结束之后，重启SQL线程，它会补上备份期间累积起来的所有更新。

采用这种方法的前提是，不会有客户端对从服务器进行更新。如果你打算使用某种直接复制数据库文件进行二进制备份的方法，那么千万不要使用这种方法。这是因为，虽然SQL线程已停止，但是信息仍然可能缓存在内存里，没有被及时写入磁盘里。

有些备份方法甚至不需要你暂停复制操作。例如，你正在备份一个只包含 MyISAM 表的数据库，那么你可能会带上适当的选项使用mysqldump，以便能一次性锁定所有的表。此时，从服务器可以继续运行，但是在备份期间，它不会试图更新任何已锁定的表。当备份程序结束并释放这些锁时，从服务器会自动继续那个更新过程。

在从服务器处于只读状态时进行备份。

下面这个流程展示的是如何实现上面列表里的最后一个策略。

(1) 让从服务器处于只读状态，防止它被更新：

```
mysql> FLUSH TABLES WITH READ LOCK;
```

```
mysql> SET GLOBAL read_only = ON;
```

(2) 执行备份，如使用mysqldump。（这种技术对于采用直接复制文件和目录的方式所做的二进制备份不起作用，因为事务型存储引擎仍然可能有刷新的事务。）

(3) 再次允许对从服务器进行更新:

```
mysql> SET GLOBAL read_only = OFF;
```

```
mysql> UNLOCK TABLES;
```

附录A

本书所需要的各类软件

本附录介绍的主要内容是如何获得sampdb发行包——它主要用于建立本书的各个示例所使用到的那个样本数据库。为使用这个发行包，必须先要让MySQL运行起来。为此，本附录还将介绍如何获得和安装MySQL及其相关的软件，如Perl的DBI模块和CGI.pm模块、PHP以及Apache。本附录还将对MySQL的安装和必需的Perl与PHP软件的安装提供简要说明，并对如何配置Apache，让其可以连同PHP一起使用提供指导。

对于安装的任何软件包，都需要仔细阅读软件包附带的各种指导说明，并在必要时查阅其他的资源。例如，MySQL的手册里就有一章讲的是各个安装步骤。

A.1 如何获得样本数据库sampdb的发行包

访问<http://www.kitebird.com/mysql-book/>可以获得sampdb发行包。该发行包中包含了建立和访问样本数据库sampdb所需要的各种文件。该发行包的压缩格式有tar和ZIP两种。为解压tar格式的发行包，可以用下面两条命令中的任何一条（如果你的tar版本不支持z选项，则需要使用第二条命令）：

```
% tar zxf sampdb.tar.gz
```

```
% gunzip < sampdb.tar.gz | tar xf -
```

为解压ZIP格式的发行包，可以使用类似WinZip、pkunzip或unzip这样的工具。

在解压 sampdb 发行包时，会创建一个名为 sampdb的目录，其中会包含以下几个文件和子目录。

README.txt文件。它包含的是发行包的使用说明。这是你应该看的第一个文件。发行包的各个子目录可能也会包含带有附加信息的README.txt文件。

在第1章，创建和加载sampdb数据库时使用到的那些文件。

capi目录。其中包含的是第7章用到的各个C语言程序。

perlapi目录。其中包含的是第8章用到的各个Perl DBI脚本。

phpapi目录。其中包含的是第9章用到的各个PHP脚本。

ssl目录。其中包含的是在MySQL客户端程序和服务器之间建立SSL连接时使用到的证书和密钥文件。更多相关信息请参考13.5节。

sampdb目录中还包含有其他几个目录，其中包含的是本书其他章节所提到的文件。想要了解更多信息，请查看README.txt文件。

A.2 如何获得MySQL及相关软件

如果想要利用好本书，你必须安装一套MySQL（如果还没安装的话）。根据计划使用的MySQL访问方式的不同，你可能还需要安装其他软件。

如果想要编写访问MySQL数据库的Perl脚本，则必须安装DBI和DBD:mysql模块。如果打算编写基于Web的DBI脚本，就还得安装CGI.pm模块和一个Web服务器。

如果想要编写本书描述的那些PHP脚本，则必须安装PHP及其数据库访问扩展PHP数据对象（PDO）和一个Web服务器。

对于那些基于Web的脚本，本书使用的是Apache服务器，当然，使用其他服务器也没有任何问题。

如果你在某个提供有MySQL服务的因特网服务提供者那里拥有账户，那么它很可能已安装好所有需要的软件包。此时，你可以直接使用它们。否则，请查看下表，找到每个软件包的主要分发站点。

软件包	网站
MySQL	http://dev.mysql.com/
Perl模块	http://cpan.perl.org/
PHP	http://www.php.net/
Apache	http://httpd.apache.org/

具体的软件包安装版本视具体需要而定。

如果想要获得最大限度的稳定性，就应该保守一点，使用软件包的最新稳定版本。这样，你既不会受到开发版本里试验性代码所带来的影响，又能获得最新的功能和最大数量的bug修复。

如果想要追赶前沿，或者想要获得只有最新的开发版才提供的功能，则可以使用最新的开发版本。

这些软件包的分发站点会指明哪些是稳定版本、哪些是开发版本。它们还会提供每个版本的功能更改列表，帮助你决定哪一个发行版本最合适你。

当前，MySQL的稳定版本是5.5系列，而开发版本是5.6系列。本书使用的MySQL最低版本为5.5，不过在编写本书时由5.6的早期版本所提供的某些功能也有所涉及。对于用于生产目的的站点，本书推荐使用稳定版本，而不使用开发版本，因此对大部分读者的建议是使用MySQL 5.5。如果要体验新功能，可以使用MySQL 5.6。

许多安装包都是以预编译二进制文件形式提供的。二进制发行包通常都是以平台里原生的安装包格式提供的，如Linux里的RPM包，或者Mac OS X里的DMG包。也有发行包是采用其他更为通用的格式提供的，如Unix里的tar文件和Windows里的Zip文档。如果你想要根据源代码来编译软件，或者在你的平台里没有可用的二进制发行包，那么你会需要一个C语言编译器（对于MySQL，则需要C++编译器）。

有些平台有自己的软件包系统。在Unix系统里和像Linux这样的类Unix系统里，这种情况比较常见。此时，你可以使用这种软件包系统来安装所需的所有软件。例如，你可以在FreeBSD里使用port系统，在Gentoo Linux里使用emerge，在Red Hat Linux里使用yum，或者在Debian Linux里使用apt-get。

本附录的剩余部分提供的是一些安装说明，其主要针对的内容是MySQL以及所需的Perl和PHP软件。

A. 3 MySQL安装说明

MySQL的安装涉及以下几个步骤。

(1) 如果想在Unix系统里安装服务器，那么需要先决定好用于运行MySQL的登录账户是哪一个，并根据需要创建该账户。

(2) 获得想要安装的发行包，并进行解压。如果你使用的是源代码，则需要编译并安装它。

(3) 设置环境变量 PATH，让其包括 MySQL安装目录下的bin 目录。这样可以使得对各个MySQL程序的调用变得更加容易，而不需要输入完整的路径名。

(4) 如果你不是要运行服务器，而只打算使用MySQL的客户端程序来访问其他人维护的服务器，则可以跳过剩下的几个步骤。

(5) 初始化数据目录和mysql数据库里的权限表。有些类型的安装包会替你做这一步。其中包括Linux里的RPM包和Mac OS X里的DMG包。在Windows里，没有必要初始化数据目录或授权表，因为它们已包含在发行版的预初始化操作里。

(6) 启动服务器。

(7) 初始化mysql数据库里的其他表。

接下来的几节提供的是对这几个步骤的详细讲解。

在MySQL安装完成之后，你可能还需要执行一些附加操作。

为MySQL账户分配密码，以获得更好的安全性。默认的安装设置允许在不提供密码的情况下连接到MySQL。

让服务器成为普通的系统启动和关闭进程，以便可以跟随系统一起自动启动和停止。将--user选项放在某个选项文件里，可以避免每次启动服务器时必须指定它的麻烦。

启用日志记录，对服务器运行情况的监视、服务器复制和数据恢复过程都有帮助。

进一步配置服务器。例如，可以启用或禁用存储引擎，或者为它们指定调整参数。

在第12章，可以找到针对这些操作和其他管理过程的指导说明。尤其应该阅读一下与密码分配、服务器启动和关闭，以及使用非特权用户账户运行服务器相关的章节。

A. 3. 1 为MySQL用户创建登录账户

这一步操作只有在想要运行MySQL服务器时才会需要。如果只是想使用MySQL客户端软件，则可以跳过这一步。

在Unix系统里，可以使用你系统中的任何一个账户来运行MySQL服务器。但是出于安全和管理方面的考虑，请不要以 `root` 身份来运行服务器。推荐你另外创建一个独立的账户，并把它用于MySQL的管理和运行。这样，你便可以使用这个账户进行登录，并拥有对数据目录的所有权限，从而可以完成维护和故障排查等操作。在MySQL里使用`root`之外的其他账户能带来很多好处，与之相关的更多讨论请参考12. 2. 1. 1节。

本书使用`mysql`作为MySQL管理账户的Unix用户名和用户组名。如果安装MySQL只是为了自己使用，则完全可以用你自己的账户来运行它。这里，你需要把在这本书里看到的所有被作用用户名或用户组名的`mysql`替换为自己的登录名和用户组名。

创建用户账户的过程因系统而异。想要获得具体细节，请查询你自己的文档。（如果你使用的是Linux里的RPM包，那么在安装服务器包时会自动创建一个名为`mysql`的用户作为登录账户。）

在创建用于运行MySQL的账户之前，应该先检查你的系统，看看是否已存在这样的账户。许多Unix系统都会在其标准账户组里包括一个用户名和用户组名都为`mysql`的账户。例如，Mac OS X里会包括一个`mysql`账户（这样便能满足Mac OS X里的那些DMG软件包所做的假设，即假设已存在有一个名为`mysql`的登录账户）。

A. 3. 2 安装MySQL

当在Windows上安装MySQL时，我推荐你使用MySQL安装程序。这个安装程序会引导你完成安装MySQL所需要的各个步骤。

在Unix里，MySQL发行包包含一个或多个下列组件。

`mysqld`服务器。

客户端程序（如`mysql`、`mysqladmin`等）。

客户端编程支持（C语言库文件和头文件）。

有些软件包格式可能会把一个完整的发行包分割成多个组件，因此一定要把你需要的软件包都装上。例如，RPM包可能把发行包划分为服务器、客户端和开发包。如果你只是要连接到运行在另一台机器上的服务器，则不需要安装服务器。不过，客户端软件应该总是需要安装，这样你才能够连接到你所使用的服务器。如果你想要自己编写程序，则还会需要安装开发包。

你也可以通过源代码发行包来构建MySQL。与通过二进制（预构建）发行包来安装的方式相比，采用这种方式会涉及更多的内容，因为你需要编译该软件。不过，这种方式带来的好处是，你可以控制更多的配置参数。例如，可以编译发行包，让其只包含想要的那些存储引擎。

A. 3. 3 设置环境变量PATH

当你在命令提示符下输入命令时，shell（命令解释器）程序会使用环境变量PATH来确定应该到哪里查找程序。如果设置PATH 环境变量，让其包含位于MySQL安装目录下的bin 目录，则调用各个MySQL程序会变得更加方便。这时，只输入命令名即可，不需要指定其完整的路径名。

在Unix里，PATH变量通常是在shell的某个启动文件里设置的，如tcsh的.tcshrc文件、bash的.bashrc或.bash_profile文件。例如，使用的是tcsh，则.tcshrc文件里便可能会有下面这样一行内容：

```
setenv PATH /bin:/usr/bin:/usr/local/bin
```

如果MySQL程序都安装在/usr/local/mysql/bin 目录里，那么可以像下面这样更改PATH的值：

```
setenv PATH /usr/local/mysql/bin:/bin:/usr/bin:/usr/local/bin
```

如果使用的是bash，则它的某个启动文件里应该会包含下面这样的内容：

```
PATH=/bin:/usr/bin:/usr/local/bin
```

可以把这个设置更改为下面这样：

```
PATH=/usr/local/mysql/bin:/bin:/usr/bin:/usr/local/bin
```

在完成对shell启动文件的修改之后，这些新的设置将在下次登录时生效。

在Windows里，设置PATH值的操作步骤如下。

- (1) 在桌面上右击“我的电脑”，选择“属性”。
- (2) 选择“高级”选项卡，并单击“环境变量”。
- (3) 从系统变量列表中选择“路径”，然后单击“编辑”。

(4) 把适当的bin目录添加到这个路径里，在前面加上一个分号，将它与路径里的最后一个目录分隔开。例如，MySQL安装在C:\mysql里，则可以把;C:\mysql\bin添加这个路径里。

重新打开一个新的控制台窗口（或者重启Windows），便可以看到这个更改生效。

A. 3.4 初始化数据目录和权限表

在刚安装好的MySQL可以使用之前，必须对数据目录和mysql数据库里的权限表进行初始化。这些权限表控制着对服务器的访问权限。这一步操作只有在你想要运行MySQL服务器时才需要。如果只是想使用MySQL客户端软件，则可以跳过这一步。如果运行环境是Windows系统，则此步操作也可以跳过，因为这些权限表都包含在Windows发行包的预初始化动作里。

在接下来的指导说明里，DATADIR 代表的是数据目录的路径名。它通常位于MySQL安装的基本目录里，名字为data或者var。一般情况下，本节里的命令都需要以root用户身份来执行。如果以MySQL账户身份（如mysql）登录，或者因为只想自己使用而以自己的账户来安装MySQL，那么不用root身份也可以执行这些命令，并且应该忽略--user选项。这时，可以跳过chown和chgrp命令。

如果想对数据目录、mysql 数据库和默认权限表进行初始化，可以先把当前路径切换到 MySQL的安装目录，然后执行mysql_install_db 脚本。（如果通过RPM包或Mac OS X包来安装，则不需要这样做，因为安装过程会自动执行mysql_install_db脚本。）例如，如果想把MySQL软件安装在/usr/local/mysql目录下，则需要执行下面这两条命令：

```
# cd /usr/local/mysql

# scripts/mysql_install_db --user=mysql
```

在运行mysql_install_db之后，需要更改数据目录里所有文件的所有权（包括用户和用户组）和访问模式。假设，用户和用户组的名字都是mysql，则相关命令如下所示：

```
# chown -R mysql DATADIR
```

```
# chgrp -R mysql DATADIR
```

```
# chmod -R go-rwx DATADIR
```

命令chown和chgrp会把所有权更改为MySQL登录账户的用户和用户组，而chmod命令会把数据目录的访问模式设置为只允许该用户访问。

最后几步实际上是更为综合的锁定过程中的一部分，详细内容请参考13.1.2节。查看该节，可以获得更多让安装变得更安全的指导。

如果mysql_install_db执行失败，可以查阅MySQL参考手册里与安装有关的章节，看看它对你所遇到的问题是怎样描述的。如果mysql_install_db没有完全执行成功，则很可能它所创建的权限表也不完整。这时，应该将它们完全删除，因为mysql_install_db在发现这些权限表存在时不会再去重新创建它们。使用下面这条命令可以删除整个mysql数据库：

```
# rm -rf DATADIR/mysql
```

A. 3.5 启动服务器

在安装好MySQL之后，便可以启动服务器。这个过程在Unix和Windows上会有所不同。

A. 3.5.1 在Unix上启动服务器

本节的命令都需要在MySQL的安装目录里执行。一般情况下，这些命令都需要以root用户身份来执行，并且需要使用--user选项来告知服务器以mysql用户身份来运行。不过，如果你是以MySQL用户身份（如mysql）登录的，或者是以你自己的账户来运行MySQL的，则可以省略这个--user选项。

先把当前路径切换到MySQL的安装基本目录（如/usr/local/mysql），然后使用mysqld_safe来启动服务器：

```
# cd /usr/local/mysql
```

```
# bin/mysqld_safe --user=mysql &
```

你既可以在命令行指定其他选项，也可以在选项文件里指定选项。相关信息请参考12.2.3节。

A. 3.5.2 在Windows上启动服务器

如果你未把MySQL安装到安装程序选定的默认位置，则必须在服务器在启动时会读取的某个选项文件里加上一个[mysqld]选项组，以便服务器能够确定安装目录和数据目录的具体位置。最常用的选项文件是MySQL安装目录里的my.ini文件或C:\my.ini。例如，如果MySQL安装在C:\mysql下，则这个选项组将类似下面这个样子（请注意，路径名里使用的是斜线“/”，而不是反斜线“\”）：

```
[mysqld]

basedir=C:/mysql

datadir=C:/mysql/data
```

如果选择了不同的安装目录，则一定要记得更改这个选项文件里的路径名。

为从命令行手动启动服务器，可以把目录位置更改到服务器安装的地方，然后输入下面这条命令：

```
C:\> mysqld
```

可以在命令行或选项文件里指定其他选项。更多信息请参考12.2.3节。

如果想以控制台模式来运行服务器，以便它能在控制台窗口显示出错信息，可以在启用它时带上--console选项：

```
C:\> mysqld --console
```

当以控制台模式运行服务器时，可以在命令行的--console选项之后，或者在选项文件里指定其他选项。

如果通过命令行来启动服务器，则只有等到服务器退出时才能看到下一个命令提示符。这时，只需要另外打开一个控制台窗口，即可运行MySQL客户端程序。

另一种在Windows上运行服务器的方法是，为MySQL创建一个服务，让其可以随着Windows一起自动启动。更多详细信息请参考12.2.2节。

A.3.6 初始化其他系统表

在启动服务器后，还可以选择性地对mysql数据库里的其他系统表（如时区表和服务器端的帮助表）进行设置。

时区表可用于指定所要使用的时区。要启用它们，可参考12.6.1节的操作指导。

命令行客户端程序mysql可以通过help命令访问服务器端的帮助信息。为启用此命令，必须在mysql数据库里设置帮助表。大部分的安装方法在首次安装时都会自动完成这个设置，并且现在Windows系统里的MySQL发行包也包含预初始化好的帮助表。因此，通常不需要手动设置这些帮助表。

如果你使用的安装方法没有设置帮助表，则可以手动加载它们。操作方法是，先确保服务器正在运行。然后，找到fill_help_tables.sql文件，其中包含了用于创建和加载帮助表的SQL语句。这个文件可能的存放位置包括/usr/share/mysql、MySQL安装目录下的share目录和源代码发行包里的scripts目录。找到这个文件之后，便可以在它所在的目录里执行以下命令：

```
% mysql -p -u root mysql < fill_help_tables.sql
```

上面这条命令会提示你输入MySQL的root账户密码。如果还没有设置密码，则需要省略-p选项。

A.4 Perl DBI安装说明

如果想要编写访问MySQL数据库的Perl脚本，则必须要有可用的DBI软件。如果在你的系统里不存在下面这些内容，则必须安装它们。

DBI模块提供的是通用的DBI驱动程序，而DBD::mysql模块提供的是特定于MySQL的驱动程序。DBI对Perl版本的最低要求为5.6.0（首选为5.6.1）。自DBI 1.611起，Perl版本的最低要求为5.8.1。（如果你还没有安装Perl，那么可以访问<http://www.perl.com/>，下载一个Perl发行包，并在安装DBI支持之前把它安装好。）

必须要有MySQL的C语言客户端开发库和头文件，因为DBD::mysql模块会使用它们。（它们应该已作为MySQL安装过程的一部分安装好了。）

如果要编写基于Web的DBI脚本，则需要安装CGI.pm模块。

为找出某个给定的Perl模块是否已经安装，可以使用perldoc命令：

```
% perldoc DBI
```

```
% perldoc DBD::mysql
```

```
% perldoc CGI
```

如果模块已安装，那么perldoc会显示其文档。否则，可以访问<http://cpan.perl.org>，获得所需的模块。

A.5 PHP和PDO安装说明

本书在编写Web服务器端所使用的脚本时，使用的是带数据库访问扩展“PHP数据对象”（PDO）的PHP。假设，使用的是Apache Web服务器（也可以将其替换为其他服务器）。如果要使用PDO，则必须安装PHP。如果你还没有安装PHP，则可以访问<http://www.php.net>，获得相应的发行包。

MySQL的PDO驱动程序需要链接到MySQL的C语言客户端开发库（即libmysqlclient库，详见附录G）或mysqlnd库。mysqlnd是一个本机驱动，它实现了与libmysqlclient一样的通信协议，并可用于替代它。如果不使用mysqlnd，则除了安装PDO库以外，还必须安装MySQL的客户端开发库。（这个库应该在MySQL的安装过程中便安装好。）

PDO在PHP 5.0及以上的版本里都可以使用，但是本附录假定使用的最低版本为PHP 5.1，因为这是PDO第一次与PHP捆绑发行的版本。当把PDO与mysqlnd一起使用时，至少需要使用PHP 5.3版本。更多相关信息请参考<http://www.php.net/pdo>。

如果通过源代码来编译PHP，则一定要使用包含MySQL和PDO支持的配置选项。对于Windows，PHP的二进制发行包有两种格式：Zip压缩包和.msi 安装程序。如果使用Zip压缩包格式，则需要将其解压缩到你想要安装PHP的位置。使用.msi包会更加方便，因为它可以引导你对Apache进行配置以支持PHP，并且会设置环境变量 PATH，让其包含 PHP的安装位置。不过，如果使用安装程序，则一定要选择扩展安装，否则PDO支持和MySQL支持将不会被安装。

如果在设置PHP时遇到了问题，则可以打开包含在PHP发行包里INSTALL文件，查看其中的“详细安装”（VERBOSE INSTALL）部分。

在PHP和PDO安装完成之后，查看Apache的配置文件httpd.conf。你需要指示Apache在启动时加载PHP模块，以及如何识别PHP脚本。（配置文件httpd.conf可以使用Include指令来包含其他文件。如果在该文件里没有看到下面几段所描述的信息，那么还要检查其他包含文件。）

为告知 Apache 加载 PHP 模块，必须在httpd.conf 里的适当部分包含LoadModule和AddModule 指令（请查看其他类似的指令）。PHP安装可能已经替你添加了这些指令。如果没有，则必须自己动手添加它们。它们看起来应该像下面这个样子，但是LoadModule指令里的路径名可能需要根据你的系统进行相应的调整：

```
LoadModule php5_module libexec/libphp5.so
```

```
AddModule mod_php5.c
```

接下来，编辑httpd.conf文件，告知Apache如何识别PHP脚本。PHP的识别依据的是PHP脚本的文件扩展名。最常用的扩展名是.php，这也是本书的各个示例所使用的扩展名。为把.php用作PHP脚本扩展名，需要把下面这行内容添加到httpd.conf文件里：

```
AddType application/x-httpd-php .php
```

也可以让Apache在URL的末尾没有指定任何文件名时，把index.php文件识别为目录的默认文件。在http.conf里找到下面这样的行：

```
DirectoryIndex index.html
```

将其修改为下面这个样子：

```
DirectoryIndex index.php index.html
```

在编辑完Apache的配置文件之后，停止httpd服务器（如果它正在运行的话），然后将其重启。在许多系统里，可以使用如下命令来完成此操作（以root用户身份来执行）：

```
# /usr/local/apache/bin/apachectl stop
```

```
# /usr/local/apache/bin/apachectl start
```

也可以通过设置，让Apache随同系统的启动和关闭一起自动开始和停止。更多指导说明请参考Apache的帮助文档。通常情况下，可以在系统启动时运行apachectl start命令，在系统关机时运行apachectl stop命令。

附录B

数据类型参考

本附录将介绍 MySQL提供的各种数据类型。与这些类型的应用有关的更多信息请参考第 3章。

数据类型名称的规范遵循以下约定。

语法描述里的可选信息都用方括号（[]）来表示。

M表示的是整型的最大显示宽度、浮点类型和小数类型的精度（有效位数）、BIT类型的位数，以及字符串类型的最大长度。在字符串列的定义里，二进制串的长度单位是字节，非二进制串的长度单位是字符。

对于那些带有小数部分的数字类型，D代表的是小数精度（即小数点后面的位数）。D必须小于等于M；否则，出错。

fsp表示的是那些允许带有小数秒的时态类型的小数秒精度。

在对各种数据类型的描述里都包含有一种或多种下面的信息。

含义：对该类型的简短描述。

可用属性：指的是一些可能会与CREATE TABLE或ALTER TABLE语句里的数据类型有关联的可选属性关键字。这些属性将按字母表的顺序依次列出，但这并不表示在CREATE TABLE或ALTER TABLE 语句（参考附录 E）里也必须按同样的顺序列出它们。在各个数据类型的描述里并没有把包含那些可应用于所有（或近乎所有）数据类型的全局属性列出来。这些全局属性都集中列在下面，而没有将它们单独列在各个类型的描述里。

NULL或NOT NULL可用于任何类型。

DEFAULT default_value 可用在除下面几种类型列以外的所有类型列的定义里：拥有AUTO_INCREMENT 属性的整数列、BLOB和TEXT 列，以及空间类型列。除了 TIMESTAMP和DATETIME（自MySQL 5.6.5起）两种类型外，其他类型的默认值都必须是常数。例如，不能为DATE列指定DEFAULT CURDATE()。

允许长度：适用于字符串数据类型的列，它指的是该类型列的值被允许达到的最大长度。

取值范围：适用于数字类型和时态（日期/时间）类型，它指的是该类型所能表示的最大值。对于整数类型，因为存在有符号和无符号两种情况，所以需要指定两个取值范围，并且每种情况的取值范围也不同。

零值：适用于时态类型，它指的是如果有非法值插入该类型列里时，实际存入的那个零值。（必须将SQL模式设置为允许这种值；否则，有非法值插入时，会出现错误。）

默认值：类型定义没有明确设定DEFAULT 属性时的默认值。这只有在没有启用严格 SQL模式时才适用。如果在严格模式中没有给出DEFAULT子句，并且能接受NULL值，那么会使用默认的NULL来定义列；否则，用非默认值定义。更多相关信息请参考3.2.3节。

存储空间要求：存储有关类型的值所需要使用的字节或字符个数。对于某些类型，该值是固定的。对于其他类型，该数字会因存储在列里的值的长度的不同而有所差异。

比较方式：对于字符串类型，这个值表明了比较操作进行的方式（这对分组、排序和索引操作也有影响）。二进制串类型是使用每个字节的数字值逐个字节地比较。非二进制串类型会根据字符集排序规则逐字符进行比较。

同义词：类型名称的同义词。

备注：与该类型有关的杂项说明。

如果不确定自己的MySQL版本会如何处理给定的列定义，可以这样做：创建一个表，让其包含一个使用你正迷惑的方式进行定义的列；然后使用SHOW CREATE TABLE或DESCRIBE查看MySQL对该定义的报告内容。例如，如果想不起来字符类型属性UNICODE和缩写数据类型SERIAL的作用，可以创建一个使用它们的表；然后告知MySQL显示最终的表定义：

```
mysql> CREATE TABLE t (c CHAR(10) UNICODE, s SERIAL);
```

```
mysql> SHOW CREATE TABLE t\G

***** 1.row *****

Table: t

Create Table: CREATE TABLE `t` (

  `c` char(10) CHARACTER SET ucs2 DEFAULT NULL,

  `s` bigint(20) unsigned NOT NULL AUTO_INCREMENT,

  UNIQUE KEY `s` (`s`)

) ENGINE=InnoDB DEFAULT CHARSET=latin1
```

B.1 数字类型

MySQL提供了精确值和近似值数字数据类型。不同的数字类型有着不同的取值范围，因此要根据想要表示的数值范围来加以选择。此外，还有一种BIT类型，专门用来表示位域值。

整数和定点数（DECIMAL）类型都是精确值数据类型。FLOAT和DOUBLE类型都是近似值数据类型。对于精确值类型，数值在存储和给定时是完全一致的，只要数值和计算结果没有超出这些类型的取值范围，计算过程就将精确地进行而不会产生任何舍入误差。对于近似值类型，计算会产生舍入误差。

对于整数类型的列，如果为其指定了AUTO_INCREMENT 属性，就必须对它进行索引。把NULL值插入一个AUTO_INCREMENT列将导致下一个序列值被插入该列里。通常情况下，新序列值比该列的当前最大值大1。第3章对AUTO_INCREMENT列的精确行为作了详细的描述。（AUTO_INCREMENT属性也可以用于浮点类型列，只是这种做法不太常见。）

属性ZEROFILL和UNSIGNED可以用于除BIT类型以外的数字类型。

如果为某列指定了ZEROFILL属性，那么该列会用前导零将值的长度补齐到列的显示宽度。

如果为某列指定了UNSIGNED属性，那么该列将不允许出现负值。（SIGNED其实也是一种可用属性，但因为数字类型在默认情况下就带有正负号，所以使用该属性没有什么实际效果。）

适用于整数和浮点数据类型的SERIAL DEFAULT VALUE属性是NOT NULL AUTO_INCREMENT UNIQUE的缩写形式。

在某些情况下，指定一种属性会导致另外一种属性也被启用。为数字类型指定ZEROFILL 属性会导致该列被自动设定为 UNSIGNED。指定 AUTO_INCREMENT 属性会导致该列被自动设定为 NOT NULL。

请注意，语句 DESCRIBE和SHOW COLUMNS 会报告说某个AUTO_INCREMENT 列的默认值是NULL，尽管你无法把一个NULL 值插入这个列里去。这说明，你可以通过在创建新行时将AUTO_INCREMENT列的值设置为NULL的方式来生成该列的默认值（即下一个序列编号）。

B. 1. 1 整型

TINYINT[(M)]

含义：非常小的整数。M是最大显示宽度，1~255。如果省略，M的默认值是4（UNSIGNED列的默认值为3）。

可用属性：AUTO_INCREMENT、SERIAL DEFAULT VALUE、UNSIGNED、ZEROFILL。

取值范围：有符号数的取值范围是 $-128 \sim 127$ ($-2^7 \sim 2^7 - 1$)，无符号数的取值范围是 $0 \sim 255$ ($0 \sim 2^8 - 1$)。

默认值：如果列可以为NULL，则默认值为NULL；如果带有NOT NULL属性，则默认值为0。

存储空间要求：1个字节。

同义词：INT1[(M)]。此外，BOOL和BOOLEAN等同于TINYINT(1)。

SMALLINT[(M)]

含义：小整数。M是最大显示宽度，1~255。如果省略，M的默认值是6（UNSIGNED列的默认值为5）。

可用属性：AUTO_INCREMENT、SERIAL DEFAULT VALUE、UNSIGNED、ZEROFILL。

取值范围：有符号数的取值范围是 $-32768 \sim 32767$ ($-2^{15} \sim 2^{15} - 1$)；无符号数的取值范围是 $0 \sim 65535$ ($0 \sim 2^{16} - 1$)。

默认值：如果列可以为NULL，则默认值为NULL；如果带有NOT NULL属性，则默认值为0。

存储空间要求：2个字节。

同义词：INT2[(M)]。

MEDIUMINT[(M)]

含义：中等大小的整数。M是最大显示宽度，1~255。如果省略，M的默认值是9（UNSIGNED列的默认值为8）。

可用属性：AUTO_INCREMENT、SERIAL DEFAULT VALUE、UNSIGNED、ZEROFILL。

取值范围：有符号数的取值范围是 $-8388608 \sim 8388607$ （ $-2^{23} \sim 2^{23}-1$ ）；无符号数的取值范围是 $0 \sim 16777215$ （ $0 \sim 2^{24}-1$ ）。

默认值：如果列可以为NULL，则默认值为NULL；如果带有NOT NULL属性，则默认值为0。

存储空间要求：3个字节。

同义词：INT3[(M)]和MIDDLINT[(M)]。

INT[(M)]

含义：标准大小的整数。M是最大显示宽度，1~255。如果省略，M的默认值是11（UNSIGNED列的默认值为10）。

可用属性：AUTO_INCREMENT、SERIAL DEFAULT VALUE、UNSIGNED、ZEROFILL。

取值范围：有符号数的取值范围是 $-2147483648 \sim 2147483647$ （ $-2^{31} \sim 2^{31}-1$ ）；无符号数的取值范围是 $0 \sim 4294967295$ （ $0 \sim 2^{32}-1$ ）。

默认值：如果列可以为NULL，则默认值为NULL；如果带NOT NULL属性，则默认值为0。

存储空间要求：4个字节。

同义词：INTERGER[(M)]和INT4[(M)]

BIGINT [(M)]

含义：大整数。M是最大显示宽度，1~255。如果省略，M的默认值是20。

可用属性：AUTO_INCREMENT、SERIAL DEFAULT VALUE、UNSIGNED、ZEROFILL。

取值范围：有符号数的取值范围是 $-9223372036854775808 \sim 9223372036854775807$ ($-2^{63} \sim 2^{63}-1$)；无符号数的取值范围是 $0 \sim 18446744073709551615$ ($0 \sim 2^{64}-1$)。

默认值：如果列可以为NULL，则默认值为NULL；如果带NOT NULL属性，则默认值为0。

存储空间要求：8个字节。

同义词：INT8[(M)]。

备注：当 SERIAL 作为一种数据类型的名词时，它是 BIGINT UNSIGNED NOT NULL AUTO_INCREMENT UNIQUE的缩写。

B.1.2 定点类型

DECIMAL[(M, [D])]

含义：定点数。M是有效位数，1~65。D是小数点后面的精确位数，0~30。如果D等于0，列值就没有小数点或小数部分。如果同时省略M和D，则默认值分别为10和0。

可用属性：UNSIGNED、ZEROFILL。

取值范围：给定的DECIMAL列的取值范围由M和D的值以及是否给定UNSIGNED属性来决定。

默认值：如果列可以为NULL，则默认值为NULL；如果带有NOT NULL属性，则默认值为0。

存储空间要求：取决于小数点左右两边的数字个数。无论哪一边，都是每9位数字需要4个字节，最后剩下的数字需要1~4个字节。每个值的存储空间都等于小数点左右两边的存储之和。

同义词：NUMERIC[(M, [D])], DEC[(M, [D])]和FIXED[(M, [D])]

B. 1.3 浮点类型

FLOAT (p)

含义：浮点数。在标准SQL中，精确度p表示至少需要的位数。在MySQL中，p仅用于确定数据类型是单精度的还是双精度的。

p值为0～24的浮点数是单精度浮点数，相当于不带说明符M或D的FLOAT类型。

p值为25～53的浮点数是双精度浮点数，即相当于不带说明符M和D的DOUBLE类型。0～53以外的p值是非法的。

可用属性：UNSIGNED、ZEROFILL。

取值范围：参见本节后面对FLOAT和DOUBLE 类型的描述。

默认值：如果列可以为NULL，则默认值为NULL；如果带有NOT NULL 属性，则默认值为0。

存储空间要求：单精度值需要4个字节，双精度值需要8个字节。

FLOAT [(M, D)]

含义：一种小型浮点数，即单精度浮点数（精度低于DOUBLE类型）。M是有效位数，范围是1～255，D 是小数点后面的精确位数，范围是0～30。如果D 等于0，则列值没有小数点或小数部分。如果省略M和D，则显示宽度和小数精度不确定；所有值都会以硬件系统所支持的最大精度来存储。

可用属性：UNSIGNED、ZEROFILL。

取值范围：最小非零值为 $\pm 1.175494351\text{E}-38$ ，最大非零值为 $\pm 3.402823466\text{E}+38$ 。如果该列指定了UNSIGNED属性，则不允许出现负值。

默认值：如果列可以为NULL，则默认值为NULL；如果带有NOT NULL属性，则默认值为0。

存储空间要求：4个字节。

同义词：FLOAT4 是不带说明符 M 或 D的FLOAT的同义词。如果启用了SQL 模式REAL_AS_FLOAT，则REAL [(M, D)]是FLOAT (M, D)的同义词。

DOUBLE [(M, D)]

含义：一种大型浮点数，即双精度浮点数（精度高于FLOAT）。M和D的含义与FLOAT中的一样。

可用属性：UNSIGNED、ZEROFILL。

取值范围：最小非零值是 $\pm 2.2250738585072014E-308$ ，最大非零值是 $\pm 1.7976931348623157E+308$ 。如果为该列指定了UNSIGNED，就不允许出现负值。

默认值：如果列可以为NULL，则默认值为NULL；如果带NOT NULL属性，则默认值为0。

存储空间要求：8个字节。

同义词：DOUBLE PRECISION[(M, D)]是DOUBLE[(M, D)]的同义词；如果没有启用SQL模式REAL_AS_FLOAT，则其同义词为REAL[(M, D)]。FLOAT8是不带说明符M或D的DOUBLE的同义词。

B. 1. 4 BIT类型

BIT[(M)]

含义：位域值。M应该是一个范围为1~64的整数，用于表明每个BIT值的二进制位数。如果省略，则M默认为1。

可用属性：可用属性：除全局属性外，没有其他特殊属性。

默认值：如果列可以为NULL，则默认值为NULL；如果带有NOT NULL属性，则默认值为0。

存储空间要求：大约为 $(M+7)/8$ 个字节。

B. 2 字符串类型

MySQL 中的字符串类型是通用类型，通常用来存储二进制或者字符（文本）数据。这些字符串类型能够容纳的值的最大长度是不一样的，可以根据你想把它们视为二进制串还是非二进制串来选择。

BINARY、VARBINARY和BLOB类型是二进制串类型。二进制串是一个字节序列，其长度单位是字节。二进制串没有字符集的概念，因此对值的比较是以字节值为基础的。

CHAR、VARCHAR和TEXT类型是非二进制串类型。非二进制串是一个字符序列。它有字符集和排序规则的概念。字符集定义的是允许数据类型使用的各种字符；排序规则定义的是字符排列顺序。在一个非二进制串列的定义里指定的长度表明的是该列能够容纳的最大字符数。

非二进制串值的长度单位通常为字符，但也可以使用字节为单位。若想获得某个非二进制串的字符长度或字节长度，可以分别使用函数CHAR_LENGTH()或LENGTH()。如果某个非二进制串只包含单字节字符，且字符长度为n，那么其字节长度也为n；但如果它包含的是多字节字符，那么其字节长度会大于n。这种差异会影响非二进制串列的存储空间。

固定长度的列，如CHAR(M)，所需的存储空间必须能够存储下M个给定字符集里的最宽字符。例如，utf8字符集里的每个字符需要占用1~3个字节，因此CHAR(M)需要占用M×3个字节。

可变长度的列，如VARCHAR(M)，所需的存储空间除了必须能存储下给定值的所有字符，还要能够放得下存储该字符串字节长度的前缀。对于一个字符集为双字节ucs2的VARCHAR(10)列，它需要0（空串）~20个字节（10字符串），用于存储字符串里的字符；外加1个字节，用于存储其长度前缀。

除ENUM和SET类型外，非二进制串类型（CHAR、VARCHAR、TEXT）也都可以指定字符集和排序规则。

指定字符集的语法是CHARACTER SET charset，其中，charset是字符集名称，如latin1、greek或utf8。CHARSET等同于CHARACTER SET。

指定排序规则的语法是COLLATE collation，其中，collation用于指定给定字符集的某个可用排序规则。

如果没有设定字符集或排序规则，那么MySQL将根据表的默认设置来确定。如果只设定了字符集而没有设定排序规则，MySQL将使用该字符集的默认排序规则。如果只设定了排序规则而没有设定字符集，MySQL将根据排序规则的名称来确定字符集。如果既设定了字符集又设定了排序规则，则排序规则必须和字符集相兼容。例如，latin1_bin排序规则和latin1字符集兼容，但是不兼容于utf8字符集。

binary字符集和BINARY列属性需要特别对待的。

如果为非二进制串类型指定了 CHARACTER SET binary，那么非二进制串类型会被转换为相对应的二进制串类型。也就是说，CHAR类型将变成BINARY类型，VARCHAR类型将变成VARBINARY类型，TEXT类型将变成BLOB类型。ENUM

和SET类型没有相对应的二进制类型，因此CHARACTER SET binary在遇到ENUM和SET类型的时候将变成一个列属性。

BINARY属性相当于为字符集（以_bin结尾的排序规则名称）指定二进制排序规则。例如，定义为CHAR(10) CHARACTER SET utf8 BINARY的列将变成CHAR(10) CHARACTER SET utf8 COLLATE utf8_bin。

对于非二进制串类型，ASCII和UNICODE属性分别是CHARACTER SET latin1和CHARACTER SET ucs2的缩写。

服务器支持的字符集和排序规则可以通过SHOW CHARACTER SET和SHOW COLLATION语句来查询。这些语句将显示出每种字符集的默认排序规则。另外，还可以去查看数据库INFORMATION_SCHEMA里的CHARACTER_SETS表和COLLATIONS表——它们包含的信息相同。

有些长到无法存储在字符串列里的值，对这些值的处理取决于SQL模式设置。如果没有启用严格SQL模式，则超长值会被截短，以便能刚好存放在该列里。如果被截去的字符不全是空格，那么MySQL还将生成一条警告消息。在严格SQL模式下，如果必须截去非空格字符，那么服务器会产生一个错误，并且不会插入任何值。

不同的字符串类型值有不同的尾部填充方式。

对于CHAR类型，如有必要，在存储时会使用空格补齐列的整个长度。在检索时，尾部的空格会被去掉。

对于BINARY类型，如有必要，在存储时会使用0x00字节补齐列的整个长度。在检索时，不会去掉任何内容。

对于VARBINARY、VARCHAR，以及BLOB和TEXT类型，在存储时不会补齐任何内容；在检索时，也不会删除任何内容。

对于ENUM和SET类型，在列定义里列出的成员值的任何尾部空格都将被忽略。因此，存储在列里的值没有任何尾部的空格，因为MySQL会把每个值转换为对应于该列成员的内部数值。这同样会影响到比较操作，因为在对ENUM或SET列里的值进行比较的时候，尾部的空格不参加比较。

B. 2.1 二进制串类型

BINARY[(M)]

含义：固定长度（0～M个字节）的二进制串。M应该是一个0～255的整数。如果省略，则M的默认值是1。

可用属性：除全局属性外，没有其他特殊属性。

允许长度：0～M个字节。

默认值：如果列可以为NULL，则默认值为NULL；如果带有NOT NULL 属性，则默认值为''（空串）。

存储空间要求：M个字节。

比较方式：根据数字字节值依次比较各个字节。

VARBINARY[(M)]

含义：可变长度（0～M个字节）的二进制串。M应该是一个0～65535的整数。

可用属性：除全局属性外，没有其他特殊属性。

允许长度：0～M个字节（如备注所述，长度有可能小于M）。

默认值：如果列可以为 NULL，则默认值为 NULL；如果带有 NOT NULL 属性，则默认值为''（空串）。

存储空间要求：值本身的长度（以字节计算）再加上1或2个字节的前缀。如果列值的最大长度小于256个字节，则前缀需要1个字节；否则，需要2个字节。

比较方式：依次比较各个字节的值。

备注：在实际工作中，VARBINARY 列的最大长度为 65535个字节，在某些情况下，它可能还更小，具体取决于存储引擎内部的行大小限制，以及表里其他列所占用的空间。

TINYBLOB

含义：小的BLOB（二进制串）值。

可用属性：除全局属性外，没有其他特殊属性。

允许长度：0～255（ $0 \sim 2^8 - 1$ ）个字节。

默认值：如果列可以为 NULL，则默认值为 NULL；如果带有 NOT NULL 属性，则默认值为''（空串）。

存储空间要求：值的长度（单位为字节），外加1个字节（用于记录该长度值）。

比较方式：根据数字字节值依次比较各个字节。

BLOB[(M)]

含义：标准大小的BLOB（二进制串）值。

可用属性：除全局属性外，没有其他特殊属性。

允许长度：0~65535 ($0 \sim 2^{16} - 1$) 个字节。如果给出了长度 M，则可以用它来选择一种适当的数据类型，然后丢弃它。如果M的值为1~65535，则数据类型变成BLOB。如果M的值等于或大于65535，则数据类型将根据值的实际长度变成MEDIUMBLOB或LONGBLOB。

默认值：如果列可以为 NULL，则默认值为 NULL；如果带有 NOT NULL 属性，则默认值为''（空串）。

存储空间要求：值的长度（单位为字节），外加2个字节（用于记录该长度值）。

比较方式：根据数字字节值依次比较各个字节。

MEDIUMBLOB

含义：中等大小的BLOB（二进制串）值。

可用属性：除全局属性外，没有其他特殊属性。

允许长度：0~16777215 ($0 \sim 2^{24} - 1$) 个字节。

默认值：如果列可以为 NULL，则默认值为 NULL；如果带有 NOT NULL 属性，则默认值为''（空串）。

存储空间要求：值的长度（单位为字节），外加3个字节（用于记录该长度值）。

比较方式：根据数字字节值依次比较各个字节。

同义词：LONG VARBINARY。

LONGBLOB

含义：大的BLOB（二进制串）值。

可用属性：除全局属性外，没有其他特殊属性。

允许长度：0~4294967295（0~ $2^{32}-1$ ）个字节。

默认值：如果列可以为 NULL，则默认值为 NULL；如果带有 NOT NULL 属性，则默认值为''（空串）。

存储空间要求：值的长度（单位为字节），外加4个字节（用于记录该长度值）。

比较方式：根据数字字节值依次比较各个字节。

B. 2. 2 非二进制串类型

CHAR[(M)]

含义：固定长度（0~M个字符）的非二进制串。M应该是一个0~255的整数。如果省略，M的默认值是1。

可用属性：BINARY、CHARACTER SET、COLLATE。

允许长度：0~M个字符。

默认值：如果列可以为 NULL，则默认值为 NULL；如果带有 NOT NULL 属性，则默认值为''（空串）。

存储空间要求：M个字符，长度是 $M \times w$ 个字节，其中，w是列所使用的字符集里的最宽字符需要占用的字节个数。

比较方式：根据列所使用的排序规则依次比较各个字符。

同义词：NCHAR(M) 和 NATIONAL CHAR(M) 都等同于 CHAR(M) CHARACTER SET utf8。

VARCHAR(M)

含义：可变长度（0~M个字符）的非二进制串。M应该是一个0~65535的整数。

可用属性：BINARY、CHARACTER SET、COLLATE。

允许长度：0～M个字符（如备注所述，长度有可能小于M）。

默认值：如果列可以为 NULL，则默认值为 NULL；如果带有 NOT NULL 属性，则默认值为''（空串）。

存储空间要求：值的长度（单位为字节），外加1或2个字节的前缀（用于记录该长度值）。如果列值的最大长度小于256个字节，则前缀需要1个字节；否则，需要2个字节。

比较方式：根据列所使用的排序规则依次比较各个字符。

同义词：CHAR VARYING(M)、NVARCHAR(M)、NCHAR VARYING(M) 和 NATIONAL CHAR VARYING(M) 都等同于VARCHAR(M) CHARACTER SET utf8。

备注：在实际工作中，VARCHAR列的最大长度为65535个字节，在某些情况下，它可能还更小，具体取决于存储引擎内部的行大小限制、列所使用的字符集是单字节字符集还是多字节字符集，以及表里的其他列所占用的空间。

TINYTEXT

含义：小的TEXT（非二进制串）值。

可用属性：BINARY、CHARACTER SET、COLLATE。

允许长度：0～255（ $0 \sim 2^8 - 1$ ）个字节；如果该值包含的是多字节字符，则字符数会更少。

默认值：如果列可以为 NULL，则默认值为 NULL；如果带有 NOT NULL 属性，则默认值为''（空串）。

存储空间要求：值的长度（单位为字节），外加1个字节（用于记录该长度值）。

比较方式：根据列所使用的排序规则依次比较各个字符。

TEXT[(M)]

含义：标准大小的TEXT（非二进制串）值。

可用属性：BINARY、CHARACTER SET、COLLATE。

允许长度：0～65535 ($0 \sim 2^{16} - 1$) 个字节；如果该值包含的是多字节字符，则字符数会更少。如果给出了长度M，则可以用它来选择一种适当的数据类型，然后丢弃它。如果M的值为1～65535，则数据类型变成TEXT。如果M的值大于或等于65535，则数据类型将根据值的实际长度变成MEDIUMTEXT或LONGTEXT。

默认值：如果列可以为 NULL，则默认值为 NULL；如果带有 NOT NULL 属性，则默认值为''（空串）。

存储空间要求：值的长度（单位为字节），外加2个字节（用于记录该长度值）。

比较方式：根据列所使用的排序规则依次比较各个字符。

MEDIUMTEXT

含义：中等大小的TEXT（非二进制串）值。

可用属性：BINARY、CHARACTER SET、COLLATE。

允许长度：0～16777215 ($0 \sim 2^{24} - 1$) 个字节；如果该值包含的是多字节字符，则字符数会更少。

默认值：如果列可以为 NULL，则默认值为 NULL；如果带有 NOT NULL 属性，则默认值为''（空串）。

存储空间要求：值的长度（单位为字节），外加3个字节（用于记录该长度值）。

比较方式：根据列所使用的排序规则依次比较各个字符。

同义词：LONG VARCHAR。

LONGTEXT

含义：大的TEXT（非二进制串）值。

可用属性：BINARY、CHARACTER SET、COLLATE。

允许长度：0～4294967295 ($0 \sim 2^{32} - 1$) 个字节；如果该值包含的是多字节字符，则字符数会更少。

默认值：如果列可以为 NULL，则默认值为 NULL；如果带有 NOT NULL 属性，则默认值为''（空串）。

存储空间要求：值的长度（单位为字节），外加4个字节（用于记录该长度值）。

比较方式：根据列所使用的排序规则依次比较各个字符。

B. 2. 3 ENUM和SET类型

`ENUM('value1', 'value2', ...)`

含义：枚举值；列的取值只能是值列表中的某个成员。

可用属性：CHARACTER SET、COLLATE。

默认值：如果列可以为 NULL，则默认值为 NULL；如果带有 NOT NULL 属性，则为枚举集合中的第1个成员。

存储空间要求：如果成员个数为 1～255，则占用1个字节；如果为 256～65535，则占用2个字节。

比较方式：根据列值的数值进行比较。

备注：在数据类型定义里，各成员值里的尾部空格将被忽略。

`SET('value1', 'value2', ...)`

含义：集合；列的取值可以是值列表中的0个或者多个成员。

可用属性：CHARACTER SET、COLLATE。

默认值：如果列可以为 NULL，则默认值为 NULL；如果带有 NOT NULL 属性，则默认值为空集（''）。

存储空间要求：1个字节（1～8个成员的集合）、2个字节（9～16个成员）、3个字节（17～24个成员）、4个字节（25～32个成员）或8个字节（33～64个成员）。

比较方式：根据列值的数值进行比较。

备注：在数据类型定义里，各成员值的所有尾部空格会被忽略。

B. 3 时态（日期和时间）类型

MySQL提供了几种表示时态数据的类型。用于表示日期和时间的类型，既可以单独使用，也可以组合使用；并且当不需要完整的日期时，还有一种用于存储年份的类型。对于新建行，有些类型可以被自动初始化成当前的日期和时间；当行里的其他列发生变化时，它们还可以被自动更新为当前的日期和时间。

日期格式里的术语CC、YY、MM、DD分别代表世纪、年、月、日。时间格式里的术语hh、mm、ss分别代表小时、分、秒。此外，MySQL 5.6.4为TIME、DATETIME和TIMESTAMP类型引入了小数秒，并且允许选择性地将小数部分的最大精度设置为6位数（微秒）。时间格式里的术语uuuuuuu表示的是小数部分。（对于MySQL 5.6.4以前的版本，小数秒部分会被忽略。）

在类型语法描述里，fsp 表示的是允许使用小数秒部分的类型的小数秒精度。该精度必须是一个0~6的值，其中，0表示的是无小数秒部分，6表示的是精确到微秒。如果没有指定 fsp，则默认为0。

自MySQL 5.6.4起，对于那些允许使用小数秒部分的类型，其存储空间要求有所变化。对于非小数秒部分的存储，与前面各个类型的描述是一样的。对于小数秒部分的存储，所有类型都是一样的。下表列出了所需要的字节数。

小数秒精度	存储空间要求
0	0个字节
1, 2	1个字节
3, 4	2个字节
5, 6	3个字节

MySQL 5.6.5引入了自动把当前时间戳作为初始值并进行更新的扩展支持。以前，这些属性最多只能用于表的一个TIMESTAMP 列；现在，它们可以用于任何 TIMESTAMP 列，并且还可用于DATETIME列。

DATE

含义：日期值，格式为'CCYY-MM-DD'。

可用属性：除全局属性外，没有其他特殊属性。

取值范围：'1000-01-01'~'9999-12-31'。

零值：'0000-00-00'。

默认值：如果列可以为 NULL，则默认值为 NULL；如果带有 NOT NULL 属性，则默认值为'0000-00-00'。

存储空间要求：3个字节。

DATETIME[(fsp)]

含义：日期和时间值，格式为'CCYY-MM-DD hh:mm:ss[.uuuuuu]'。自MySQL 5.6.4起，允许使用fsp来指定精度为0~6的小数秒。如果省略，则fsp默认为0。

可用属性：在MySQL 5.6.5之前，除全局属性外，无任何属性可用。自MySQL 5.6.5起，DATETIME列可以包含DEFAULT CURRENT_TIMESTAMP或ON UPDATE CURRENT_TIMESTAMP，或者同时包含这两个属性。它们的含义与TIMESTAMP里的相同。更多信息请参考3.2.6.6节。

取值范围：'1000-01-01 00:00:00[.000000]' ~ '9999-12-31 23:59:59[.999999]'。

零值：'0000-00-00 00:00:00[.000000]'。

默认值：如果列可以为 NULL，则默认值为 NULL；如果带有 NOT NULL 属性，则默认值为'0000-00-00 00:00:00[.000000]'。

存储空间要求：在MySQL 5.6.4之前，为8个字节；那以后的版本，还需要外加5个字节，用于存储小数秒。

TIME[(fsp)]

含义：时间值，格式为'hh:mm:ss[.uuuuuu]'（对于负值，格式为'-hh:mm:ss[.uuuuuu]'）。自MySQL 5.6.4起，允许使用fsp来指定精度为0~6的小数秒。如果省略，则fsp默认为0。

可用属性：除全局属性外，没有其他特殊属性。

取值范围：'-838:59:59[.000000]' ~ '838:59:59[.000000]'。

零值：'00:00:00[.000000]'。

默认值：如果列可以为 NULL，则默认值为 NULL；如果带有 NOT NULL 属性，则默认值为'00:00:00[.000000]'。

存储空间要求：在MySQL 5.6.4之前，为3个字节；那以后的版本，还需要外加3个字节，用于存储小数秒。

备注：当有非法值插入 TIME 列时，虽然'00:00:00[.000000]'会被用作零值，但是它同时也是列的正常取值范围内的一个合法值。

`TIMESTAMP[(fsp)]`

含义：时间戳（日期和时间），格式为 'CCYY-MM-DD hh:mm:ss[.uuuuuu]'。自MySQL 5.6.4起，允许使用fsp来指定精度为0~6的小数秒。如果省略，则fsp默认为0。TIMESTAMP类型有以下几种特殊行为。

在表的TIMESTAMP 列里插入NULL 值时，会插入当前的日期和时间，除非列被声明为允许NULL值。

TIMESTAMP列可以有两个自动修改属性。

自动初始化：当创建新行时，该列的默认值为当前时间戳。

自动更新：当更新行时，修改该行里的任何其他列的值，都会使TIMESTAMP列被更新为修改发生时的日期和时间值。

在MySQL 5.6.5之前，每个表最多只能有一个TIMESTAMP列可以拥有这些属性，并且可以指定哪一个列会有此“殊荣”。自MySQL 5.6.5起，任何TIMESTAMP列都可以拥有这些属性，并且可以随意组合。更多详细信息请参考3.2.6.6节。

可用属性：在MySQL 5.6.5之前，一个表最多只能有一个TIMESTAMP列可以拥有DEFAULT CURRENT_TIMESTAMP或ON UPDATE CURRENT_TIMESTAMP属性，或者同时拥有这两个属性。不能为一个TIMESTAMP列设置一种属性，又为另一个TIMESTAMP列设置其他属性；同样，也不能将一个属性用于多个TIMESTAMP列。自MySQL 5.6.5起，所有的TIMESTAMP列都可以拥有这些属性，并且可以随意组合。

在创建新行时，如果没有为TIMESTAMP 列提供值，则DEFAULT CURRENT_TIMESTAMP 属性会使其被设置为当前的日期和时间。当行里的其他列值发生变化时，ON UPDATE CURRENT_TIMESTAMP 属性会使TIMESTAMP 列被更新为当前的日期和时间。CURRENT_TIMESTAMP()和NOW()函数都可以认为等同于CURRENT_TIMESTAMP属性。

如果列定义里的初始关键字TIMESTAMP包含有一个fsp值，那么在用于指定自动属性的子句DEFAULT或ON UPDATE里（如果存在的话），也必须使用相同的值。

可以指定一个DEFAULT常量，从而为TIMESTAMP列分配一个固定的日期和时间值或零值。如果指定NULL属性，则允许TIMESTAMP列存储NULL值。如果没有指定此属性，则将NULL存储到TIMESTAMP列时，会将其设置为当前的日期和时间。

取值范围：'1970-01-01 00:00:01[.000000]'\sim2038年（大约）。

零值：'0000-00-00 00:00:00[.000000]'。

默认值：在创建行时，如果TIMESTAMP列被设置为当前的日期和时间，则DESCRIBE和SHOW COLUMNS语句会把默认值显示为CURRENT_TIMESTAMP；否则，会显示日期和时间的默认值。请参考对“可用属性”项的讨论。

存储空间要求：在MySQL 5.6.4之前，为4个字节；那以后的版本，还需要外加4个字节，用于存储小数秒。

YEAR[(M)]

含义：年份值。如果给出的话，M只能是2或者4，分别代表格式YY或CCYY。如果省略，则M的默认值为4。

可用属性：除全局属性外，没有其他特殊属性。

取值范围：对于YEAR(4)，范围是1901\sim2155，以及0000；对于YEAR(2)，范围相同，但是只显示最后两位数字。如果想要避免存储在YEAR(2)里的值出现歧义，则需要将这些值的范围限制在1970\sim2069；另一种选择是，使用YEAR(4)。

零值：对于YEAR(4)，零值为0000；对于YEAR(2)，零值为00。

默认值：如果列可以为NULL，则默认值为NULL；如果带有NOT NULL属性，则默认值为0000或00。

存储空间要求：1个字节。

备注：自MySQL 5.6.6起，YEAR(2)已被弃用。对于这样的列，可以使用YEAR(4)代替。

附录C

操作符与函数参考

本附录介绍的是用于在SQL 语句里构造表达式的各种操作符和函数。除非特别说明，否则这里列出的所有操作符和函数都至少存在于MySQL 5.5.0之前的版本里。自MySQL 5.5.0以后发生的更改，都在各个操作符和函数的描述里进行了说明。

大部分的操作符和函数示例都使用了下面的格式：

expr	→
result	

其中，表达式`expr`演示的是如何使用操作符或函数，`result`展示的是该表达式的计算结果。例如：

<code>RIGHT('my cat',3)</code>	→ 'cat'
--------------------------------	---------

它表示的是，函数调用`RIGHT('my cat', 3)`将生成一个字符串结果'cat'。如果你要自己尝试本附录里的示例，可以先调用`mysql` 程序，在函数调用的前面输入`SELECT`，函数调用的后面输入一个分号，然后按下Enter 键：

```
mysql> SELECT RIGHT('my cat',3);
```

```
-----  
RIGHT('my cat',3)  
-----
```

```
cat  
-----
```

MySQL并不要求SELECT语句必须要有一个FROM子句——这为我们这些希望通过输入各种表达式来熟悉操作符和函数使用方法的人提供了方便。

对于某些函数，本附录里的示例包含了完整的SELECT语句；否则，难以演示它们。. 2.6节便是采用这种方式编写的，因为那些函数只有引用特定表才会有意义。

函数名和单词形式的操作符（如BETWEEN）都允许使用任何形式的大小写。

操作符和函数的语法描述使用了下面这些约定。

expr。它表示的是一个表达式。根据具体的上下文，它可以是一个数字、字符串或者日期/时间表达式，也可以是一个常量、表列引用或者其他的表达式。

str。它表示的是一个字符串。它可以是一个文字字符串、一个拥有字符串数据类型的表列引用或者一个能生成字符串的表达式。

n。它表示的是一个整数（字母表里邻近n的那些字母也常用来表示整数）。

x。它表示的是一个浮点数（字母表里邻近x的那些字母也常用来表示浮点数）。

其他的参数名很少使用，都是在使用时才定义。

方括号（[]）表示的是可选信息。

竖线字符（|）用于分隔列表里的各个条目。如果列表出现在一对方括号里，则表示可以从中选取一个。如果列表出现在一对花括号（{}）里，则表示必须从中选取一个。

省略号（...）表示的是该省略号前面的那个项目可以重复多次。

表达式的计算过程往往会涉及各种值之间的类型转换。与类型转换发生的具体环境有关的详细讨论，以及与MySQL使用的类型转换规则有关的详细讨论，请参考3.5.2节。

C.1 操作符

操作符能把数据项组合成表达式，执行算术运算、数值比较、二进制位操作、逻辑操作，以及模式匹配。

C.1.1 操作符优先级

操作符有着各种不同的优先级。下面按从高到低的优先级顺序列出了各种运算符。其中，处于同一行的运算符拥有相同的优先级。优先级高的运算符会在优先级低的运算符之前进行计算。优先级相同的运算符则按从左至右的顺序依次进行计算。

INTERVAL

BINARY COLLATE

!

- (负号) ~ (按位求反)

^

* / DIV % MOD

+ -

<< >>

&

|

< <= = <=> <> != >= > IN IS LIKE REGEXP
RLIKE

BETWEEN CASE WHEN THEN ELSE

NOT

AND &&

XOR

OR ||

:=

一元操作符（即负号、按位求反、NOT、BINARY和COLLATE）与操作数的绑定关系比二元操作符紧密。也就是说，它们和表达式里紧随其后的那个项是结合在一起的，而不是和整个表达式的其余部分结合在一起。

-2+3	→
1	
-(2+3)	→ -5

有些操作符的优先级取决于MySQL服务器的SQL模式或MySQL的版本。

如果启用了SQL模式PIPES_AS_CONCAT，那么||将变成一个字符串连接操作符，而不是一个逻辑OR操作符，并且它的优先级将上升至^和一元操作符之间。

NOT操作符的优先级比!操作符的低。如果想让NOT的优先级和!的一样，需要启用SQL模式HIGH_NOT_PRECEDENCE。

C. 1. 2 分组操作符

这些操作符可用于对表达式里的表达项分组，从而控制运算顺序；或者用于将各个值分组成元组。

(...)

括号可以用来对表达式里的各个部分进行分组。它们可以改写默认的操作符优先级，从而改变表达式中表达项的运算顺序（参考C. 1. 1节）。括号还可以改善表达式的清晰度，让表达式更易读。嵌套的括号表达式会以由内到外的顺序进行计算。

1 + 2 * 3 / 4	→
2.5000	

((1 + 2) * 3) / 4	→ 2.2500
-------------------	----------

`(expr[, expr]...)`

`ROW(expr[, expr]...)`

这些行构造符可用于对两组（集合）值进行比较。参与比较的两个元组必须包含相同数量的值。上面给出的两种语法（有或没有ROW关键字）是等效的。例如，如果某个子查询返回的一个行包含3个值，你可以任选以下结构之一，把该行与一个包含3个值的给定元组进行比较：

```
SELECT ...FROM t2 WHERE (0,1,2) = (SELECT col1, col2, col3 FROM ...);
```

```
SELECT ...FROM t2 WHERE ROW(0,1,2) = (SELECT col1, col2, col3 FROM ...);
```

这些行构造符同样可以用在不涉及子查询的上下文里。下面这条语句是合法的：

```
SELECT * FROM president WHERE (first_name,last_name) = ('John','Adams');
```

C. 1. 3 算术操作符

这些操作符用来完成各种标准的算术运算。算术操作符的操作对象是数字，不是字符串（虽然MySQL会自动地把看起来像数字的字符串转换为相应的数值）。

算术操作符遵守以下规则。

在数字上下文里出现的字符串将被转换为双精度数。

如果两个操作数都是整数，那么+、-和*运算将使用64位整数。这意味着，涉及大整数的算术表达式，其运算结果有可能超出64位整数的表示范围。溢出将导致出错。

如果两个操作数都是整数，且其中至少有一个是无符号数，那么计算结果将为无符号数。

如果两个操作数至少有一个是实数，那么+、-、/、*和%所得结果的精度将由精度最大的那个操作数决定。

在需要将结果当作整数的上下文里，通过操作符/实现的除法运算会使用64位整数进行计算。

在使用操作符/对两个精确值进行除法运算的时候，所得结果的小数位数等于被除数的小数位数加上系统变量div_precision_increment的值（默认值为4）。

涉及NULL值的算术运算得到的结果仍然是一个NULL值。

下面是一些算术操作符。

□+

加法操作符。它用于计算两个操作数的和。

2 + 2 → 4

3.2 + 4.7 → 7.9

'43bc' + '21d' → 64

'abc' + 'def' → 0

上面列表里最后那个示例表明的是：MySQL 不像某些程序设计语言那样，把+也用作字符串连接操作符。实际上，在进行运算之前，MySQL 会先将这些字符串转换为数字。那些看起来不像数字的字符串会转换为0。如果要连接字符串，可以使用CONCAT() 函数。

□-

减法或求负操作符。如果出现在两个表达项之间，那么计算的就是两个操作数的差；如果出现在单个表达项的前面，那么计算的就是该操作数的负值（即翻转表达项的正负号）。

10 - 7 → 3

-(10 - 7) → -3

□*

乘法操作符。它用于计算两个操作数的乘积。

2 * 3 → 6

2.3 * -4.5 →
-10.35

□/

除法操作符。它用于计算两个操作数的商值。如果除数为零，则结果为NULL。

3 / 1 →
3.0000

1 / 3 →
0.3333

1 / 0 →
NULL

DIV

整除操作符。它用于计算两个操作数的商值，但是不带小数。如果除数为零，则结果为NULL。如果有一个操作数不是整数，那么这两个操作数都会被当作DECIMAL值来运算，而结果会被转换为整数。如果结果大于BIGINT的上限值，则会出现错误。

3 DIV 1 →
3

1 DIV 3 →
0

1 DIV 0 →
NULL

%或MOD

取模操作符。它计算的是m除以n的余数。操作符语法m % n或m MOD n等价于函数语法MOD(m, n)。与除法相类似，如果除数为0，则结果为NULL。

12 % 4 → 0

12 % 5 → 2

12 % 0 →
NULL

如果两个操作数带有小数部分，那么取模运算所得结构为相除以后得到的那个余数。

14.4 % 3.2 → 1.6

C. 1. 4 比较操作符

如果比较操作的结果为真，比较操作符的返回值就将是 1；如果比较操作的结果为假，比较操作符的返回值就将是0。可以对数值或字符串进行比较。操作数会按以下规则进行转换。

除操作符<=>以外，所有涉及 NULL 值的比较结果都为 NULL。（<=>与=很像，主要区别在于：NULL <=> NULL的结果为真，而NULL = NULL的结果为NULL。）

如果两个操作数都是字符串，那么它们将按字符串方式进行词法比较。对于二进制串，按字节逐个比较各个对应字节的数字。对于非二进制串，使用表示字符串的字符集所对应的排序规则，逐个字符地进行比较。如果这些字符串使用了不同的字符集，那么这个比较可能会导致错误或者产生毫无意义的结果。对于非二进制串和二进制串之间的比较，则按二进制串进行比较。

如果两个操作数都是整数，那么它们将按整数方式进行数字比较。

如果十六进制常量是与数字进行比较，那么会被视为二进制串。

除IN()以外，如果一个操作数为TIMESTAMP（或DATETIME），另一个数为常量，那么这些操作数会被当作TIMESTAMP值进行比较。这种做法很适合于各种ODBC应用程序的比较操作。

如果一个操作数为小数，而另一个操作数为小数或整数，那么这些操作数会按小数进行比较；否则，按双精度数进行比较。

除上述情况外，比较操作中的操作数都将被视为双精度浮点数。请注意，字符串与数字之间的比较也属于这种情况。字符串将被转换为一个双精度数（如果字符串看起来不像数字，那么转换结果将为0）。例如，'14.3'将被转换为14.3，但'L4.3'会被转换为0。

下面的几个比较操作示例说明了这些规则：

2 < 12
→ 1

'2' < '12' →
0

'2' < 12
→ 1

在第一个比较操作里，两个操作数都是整数，因此它们会按数字方式进行比较。在第二个比较操作里，两个操作数都是字符串，因此它们会按字符串方式进行比较。在第三个比较操作里，一个操作数是整数，另一个操作数是字符串，因此它们将被当作双精度浮点数进行比较。

MySQL会按以下规则对字符串进行比较：对于二进制串，会按字节逐个比较各个对应字节的数字。对于非二进制串，使用表示字符串的字符集所对应的排序规则，逐个字符地进行比较。如果这些字符串使用了不同的字符集，那么这个比较可能会导致错误或者产生毫无意义的结果。对于非二进制串和二进制串之间的比较，则按二进制串进行比较。

□=

如果两个操作数相等，则结果为1；否则，结果为0。

1 = 1 →
1

1 = 2 →
0

'abc' = 'abc' → 1

'abc' = 'ABC' → 1

'abc' = 'def' → 0

'abc' = 0 → 1

在进行不区分大小写的字符串比较时，'abc'与'abc'和'ABC'都是相等的。'abc'等于 0的原因在于，MySQL 会先按规则把它转换为数值再进行比

较。由于'abc'看起来不像数值，所以它在和数值进行比较之前会先被转换为0。

在非二进制串的比较操作里，彼此相似但在大小写、重音或其他注音符号上有所区别的字符的“大小”，由操作数的字符集和排序规则决定。

字符串比较操作通常不区分字母的大小写，但如果比较操作涉及二进制串，或是非二进制串使用了二进制排序规则或区分大小写的排序规则，则另当别论。例如，如果你使用BINARY关键字，或者正在对BINARY、VARBINARY或BLOB列里的值进行比较，那么比较操作会区分大小写：

'abc' = 'ABC' → 1

BINARY 'abc' = 'ABC' → 0

BINARY 'abc' = 'abc' → 1

_latin1 'abc' COLLATE latin1_bin = 'ABC' → 0

_latin1 'abc' COLLATE latin1_general_cs = 'ABC' → 0

在二进制串比较操作里，尾部空格会参与比较；但对于非二进制串的比较操作，它们不会参与比较。

BINARY 'a' = 'a ' → 0

'a' = 'a ' → 1

<=>

NULL 安全相等。它与=相似，不同之处在于：只要两个操作数相等，即使它们都为NULL，最终结果也会是1。

1 <=> 1 →
1

1 <=> 2 →
0

NULL <=> NULL
→ 1

NULL = NULL →
NULL

最后两个示例演示的是<=>和=在处理NULL比较时的差异。

<>或!=

如果两个操作数不相等，则计算结果为1；否则，计算结果为0。

3.4 != 3.4 → 0

'abc' <> 'ABC' → 0

BINARY 'abc' <> 'ABC' → 1

'abc' != 'def' → 1

□<

如果左操作数小于右操作数，则计算结果为1；否则，计算结果为0。

3 < 10 → 1

105.4 < 10e+1 → 0

'abc' < 'ABC' → 0

'abc' < 'def' → 1

<=

如果左操作数小于或等于右操作数，则计算结果为1；否则，计算结果为0。

'abc' <= 'a' → 0

'a' <= 'abc' → 1

13.5 <= 14 → 1

(3 * 4) - (6 * 2) <= 0 → 1

□>

如果左操作数大于右操作数，则计算结果为1；否则，计算结果为0。

$\text{PI}() > 3$ → 1

$'\text{abc}' > 'a'$ → 1

$\text{SIN}(0) > \text{COS}(0)$ → 0

>=

如果左操作数大于或等于右操作数，则计算结果为1；否则，计算结果为0。

$'\text{abc}' >= 'a'$ → 1

$'a' >= 'abc'$ → 0

$13.5 >= 14$ → 0

$(3 * 4) - (6 * 2) >= 0$ → 1

expr BETWEEN min AND max

expr NOT BETWEEN min AND max

如果expr落在从min到max的区间内（包括min和max在内），则BETWEEN操作符的计算结果为1；否则，计算结果为0。对于NOT BETWEEN，计算情况则正好相反。如果操作数expr、min和max都是同一种类型，则下面两个表达式等价：

expr BETWEEN min AND max

(min <= expr AND expr <= max)

如果这些操作数不是同一类型，则会进行类型转换，并且这两个表达式不一定等价。在这种情况下，比较操作会使用本节开始部分提供的各种规则。

$'\text{def}' \text{ BETWEEN } 'abc' \text{ AND } 'ghi'$ → 1

'def' BETWEEN 'abc' AND 'def' → 1

13.3 BETWEEN 10 AND 20 → 1

13.3 BETWEEN 10 AND 13 → 0

2 BETWEEN 2 AND 2 →

1

'B' BETWEEN 'A' AND 'a' → 0

BINARY 'B' BETWEEN 'A' AND 'a' → 1

对于使用混合时态类型或使用混合时态类型加字符串的BETWEEN表达式，最好使用CAST() 确保所有操作数的类型都相同。

CASE [expr] WHEN expr1 THEN result1 ... [ELSE default] END

当第一个表达式expr存在时，CASE就会把它与每一个WHEN后面的表达式进行比较。对于第一个相等的表达式，与之相应的那个THEN值会成为计算结果。这特别适用于需要把一个给定值与一组值进行比较的情况。

CASE 0 WHEN 1 THEN 'T' WHEN 0 THEN 'F' END → 'F'

CASE 'F' WHEN 'T' THEN 1 WHEN 'F' THEN 0 END → 0

当第一个表达式expr不存在时，CASE将计算WHEN表达式。对于第一个结果为真（非零，非 NULL）的表达式，与之相应的THEN 值会成为计算结果。这特别适用于需要判断“不等于”关系或者需要对任意条件进行测试的场合。

CASE WHEN 1=0 THEN 'absurd' WHEN 1=1 THEN 'obvious' END

→ 'obvious'

如果没有匹配到任何WHEN表达式，那么ELSE的值便是计算结果。如果不存在ELSE子句，那么CASE的计算结果为NULL。

CASE 0 WHEN 1 THEN 'true' ELSE 'false' END → 'false'

CASE 0 WHEN 1 THEN 'true' END → NULL

CASE WHEN 1=0 THEN 'true' ELSE 'false' END → 'false'

CASE WHEN 1/0 THEN 'true' END →
NULL

CASE表达式的返回类型由默认返回值的聚合类型决定。

CASE 1 WHEN 0 THEN 0 ELSE 1 END → 1

CASE 1 WHEN 0 THEN '0' ELSE '1' END → '1'

但是默认返回类型也受上下文影响，上下文可能会引起类型转换，将返回类型转换为字符串、数字等。

请注意，CASE表达式与E. 2. 1节中的CASE语句有所不同。

expr IN (value1, value2, ...)

expr NOT IN (value1, value2, ...)

如果 expr 与列表中的某个值相等，则 IN() 的计算结果为 1；否则，结果为 0。对于 NOT IN()，计算情况正好相反。下面两个表达式是等价的：

expr NOT IN (value1, value2, ...)

NOT (expr IN (value1, value2, ...))

如果列表里的值全部为常量，那么MySQL会对它们进行排序，并使用折半查找法（其查找速度非常快）来计算IN()测试。

3 IN (1,2,3,4,5) → 1

'd' IN ('a','b','c','d','e') → 1

'f' IN ('a','b','c','d','e') → 0

3 NOT IN (1,2,3,4,5) → 0

'd' NOT IN ('a','b','c','d','e') → 0

'f' NOT IN ('a','b','c','d','e') → 1

`expr IS {FALSE | TRUE | UNKNOWN}`

这些语句用于测试 `expr` 的逻辑值是否为假、是否为真，或者是否为未知，并且会返回 0（假）或1（真）。当值为0时，表示为假；非0和非NULL表示为真；NULL表示未知。

	<code>2 IS FALSE</code>	<code>→ 0</code>
	<code>2 IS TRUE</code>	<code>→</code>
1		
	<code>2 IS UNKNOWN</code>	
<code>→ 0</code>		
	<code>NULL IS FALSE</code>	<code>→ 0</code>
	<code>NULL IS TRUE</code>	<code>→</code>
0		
	<code>NULL IS UNKNOWN</code>	
<code>→ 1</code>		

`expr IS NULL`

`expr IS NOT NULL`

如果`expr`的值为NULL，则`IS NULL`的计算结果为1；否则，结果为0。`IS NOT NULL`的计算结果正好相反。下面两个表达式是等价的：

`expr IS NOT NULL`

`NOT (expr IS NULL)`

`IS NULL`和`IS NOT NULL`专门用来判断`expr`的值是否为NULL。普通的比较操作符（`=`、`<>`和`!=`）无法进行这种判断。（不过，可以用操作符`<=>`来测试NULL值。）

	<code>NULL IS NULL</code>	<code>→</code>
1		

0	0 IS NULL	→
0	NULL IS NOT NULL	→
	0 IS NOT NULL	→ 1
	NOT (0 IS NULL)	→ 1
	NOT (NULL IS NULL)	→ 0

C. 1. 5 位操作符

本节介绍用来完成各种位操作的操作符。位操作必须用BIGINT 值（64位整数）完成，这就限制了这类操作的最大范围。位操作的结果是64位无符号值；如果操作数里有NULL值，则位运算的结果为NULL。

□&

两个操作数的按位AND（与）运算。

1	1 & 1	→
0	1 & 2	→
5	7 & 5	→

□|

两个操作数的按位OR（或）运算。

	1 1	→ 1
	1 2	→ 3
	1 2 4 8	→ 15

1 | 2 | 4 | 8 | 15 → 15

□^

两个操作数的按位XOR（异或）运算。

1 ^ 1 → 0

1 ^ 0 → 1

255 ^ 127 →

128

<<

把左操作数的各位左移，移动次数由右操作数指定。如果右操作数为负值，则运算结果为0。

1 << 2 → 4

2 << 2 → 8

1 << 63 →

9223372036854775808

1 << 64 →

0

最后两个例子演示了64位计算的极限情况。

>>

把左操作数的各位右移，移动次数由右操作数指定。如果右操作数为负值，则运算结果为0。

16 >> 3 →

2

16 >> 4 →

1

16 >> 5 →
0

□ ~

对随后的操作数逐位取反。即把所有的0位翻转为1，而把所有的1位翻转为0。

~0
→ 18446744073709551615

~(-1) →
0

~~(-1) →
18446744073709551615

C. 1.6 逻辑操作符

逻辑操作符（即“布尔操作符”）用来测试表达式是否成立。如果逻辑操作的结果为真，则返回1；结果为假，则返回0；结果未知，则返回NULL。逻辑操作符把非零、非NULL操作数解释为真，把操作数0解释为假，NULL解释为未知。

逻辑操作符要求操作数为数字，因此字符串操作数在计算过程开始之前会被转换为数字。

在MySQL里，!、||和&&表示逻辑操作，与C语言里的含义一样。特别地，||不像标准SQL语言里规定的那样用来完成字符串连接操作。如果要连接字符串，需要使用CONCAT()函数。如果想把||当作字符串连接操作符来使用，需要启用SQL模式PIPES_AS_CONCAT。

NOT或!

逻辑非操作符。如果随后的操作数为假，则计算结果为 1；如果操作数为真，则计算结果为0。但NOT NULL仍将为NULL。

NOT 0
→ 1

NOT 1
→ 0

NOT NULL
→ NULL

NOT 3
→ 0

NOT NOT 1
→ 1

NOT '1' →
0

NOT '0' →
1

NOT 'abc' → 1

在上面的最后几个例子里，字符串操作数将被转换为数字后再计算。

操作符NOT的优先级可以按照C. 1. 1节的做法进行更改。

AND或&&

逻辑“与”操作符。如果两个操作数都为真（非零，非NULL），则计算结果为1；如果有一个操作数为假，则计算结果为0；否则，计算结果为NULL（结果不确定）。

4 AND 2 →
1

0 AND 0 →
0

0 AND 3 →
0

1 AND NULL →
NULL

0 AND NULL →
0

NULL AND NULL
→ NULL

OR或||

逻辑“或”操作符。如果两个操作数里有一个为真（非零，非NULL），则计算结果为1；如果两个操作数都为假，则计算结果为0；否则，为NULL（结果不确定）。

4 OR 2 → 1

0 OR 3 → 1

0 OR 0 → 0

1 OR NULL
→ 1

0 OR NULL
→ NULL

NULL OR NULL
→ NULL

XOR

逻辑“异或”操作符。如果有且仅有一个操作数为真（非零，非NULL），则计算结果为1；否则，计算结果为0。如果操作数中有NULL值，计算结果为NULL（未知）。

0 XOR 0 →
0

0 XOR 9 →
1

7 XOR 0	→
1	
5 XOR 2	→
0	

C. 1. 7 类型转换操作符

类型转换操作符能够把数据值从一种类型转换为另一种类型。

`_charset str`

操作符 `_charset` 常被叫作“引导符”。它会导致紧跟其后面的字符串常量或列值会使用给定的字符集进行解释。`charset` 必须是服务器所支持的某个字符集的名字。例如，下面的表达式将分别使用 `latin2`、`utf8` 字符集解释字符串 `'abcd'`：

`_latin2 'abcd'`

`_utf8 'abcd'`

对于多字节字符集的引导符，如果其操作数尾部字节的个数不够创建一个完整的字符，则转换结果的尾部可能会有一些空格。

`BINARY str`

`BINARY` 操作符用来把紧随其后的操作数转换为一个二进制串。对结果的比较将使用每个字节的数值逐个字节地进行。如果紧随其后的操作数是一个数值，就先把它转换为字符串形式：

<code>'abc' = 'ABC'</code>	→ 1
----------------------------	-----

<code>'abc' = BINARY 'ABC'</code>	→ 0
-----------------------------------	-----

<code>BINARY 'abc' = 'ABC'</code>	→ 0
-----------------------------------	-----

<code>'2' < 12</code>	→ 1
--------------------------	-----

<code>'2' < BINARY 12</code>	→ 0
---------------------------------	-----

在最后那个示例里，`BINARY` 会强制进行一次由数值到字符串的转换。然后，这两个操作数会按二进制串进行比较。

`str COLLATE collation`

操作符COLLATE会使给定的字符串str拥有给定的排序规则（该排序规则必须是str所属字符集所对应的合法排序规则中的一种）。COLLATE会对某些操作（如比较、排序、分组以及DISTINCT）产生影响。

```
SELECT ...WHERE utf8_str COLLATE utf8_icelandic_ci > 'M';
```

```
SELECT MAX(greek_str COLLATE greek_general_ci) FROM ...;
```

```
SELECT ...GROUP BY latin1_str COLLATE latin1_german2_ci;
```

```
SELECT ...ORDER BY sjis_str COLLATE sjis_bin;
```

```
SELECT DISTINCT latin2_str COLLATE latin2_croatian_ci FROM ...;
```

C. 1. 8 模式匹配操作符

MySQL提供了两种模式匹配机制：一种是使用LIKE 操作符的SQL模式匹配，另一种是使用REGEXP 操作符的正则表达式模式匹配。只有整个字符串得到匹配时，SQL 模式才算匹配成功。而只要能在字符串里找到匹配模式，正则表达式模式就算匹配成功。

3. 5. 1. 1节提供了更多与模式匹配有关的讨论和示例。

```
str LIKE pattern [ESCAPE 'c']
```

```
str NOT LIKE pattern [ESCAPE 'c']
```

LIKE是SQL模式匹配操作的操作符，当匹配字符串pattern与整个字符串表达式str匹配时，它的计算结果将是 1。如果没有得到匹配，LIKE 操作符的计算结果将是 0。NOT LIKE操作符的计算情况则正好相反。下面两个表达式是等价的：

```
str NOT LIKE pattern [ESCAPE 'c']
```

```
NOT (str LIKE pattern [ESCAPE 'c'])
```

只要两个操作数里有一个是NULL，计算结果就为NULL。

在SQL模式里，有两个字符是有着特殊含义的通配符：

“%”。它能与除NULL以外的任意字符序列（包括空串）相匹配。

“_”（下划线）。它能与任何一个字符相匹配。

SQL模式允许混合使用这两种通配符：

'catnip' LIKE 'cat%' → 1

'dogwood' LIKE '%wood' → 1

'bird' LIKE '____' → 1

'bird' LIKE '___' → 0

'dogwood' LIKE '%wo__' → 1

如果有一个操作数是二进制串，那么LIKE会按二进制串对字符串进行比较。如果操作数是非二进制串，则使用操作数的排序规则进行比较。

'abc' LIKE 'ABC' → 1

BINARY 'abc' LIKE 'ABC' → 0

'abc' LIKE BINARY 'ABC' → 0

'abc' LIKE 'ABC' COLLATE latin1_general_ci → 1

'abc' LIKE 'ABC' COLLATE latin1_general_cs → 0

因为通配符“%”能够与任何一个字符序列相匹配，所以它能与空串相匹配。

" LIKE '%' → 1

'cat' LIKE 'cat%' → 1

MySQL允许使用LIKE操作符对数值表达式进行匹配。

50 + 50 LIKE '1%' → 1

200 LIKE '2__' → 1

如果想对通配符字符进行匹配，就必须给它们加上一个前导的反斜线字符“\”以取消其特殊含义。

'100% pure' LIKE '100%' → 1

'100% pure' LIKE '100\%' → 0

'100% pure' LIKE '100\% pure' → 1

如果要解释“\”，则需要启用SQL模式 NO_BACKSLASH_ESCAPES。或者，重新定义转义字符，指定一个ESCAPE子句。

'100% pure' LIKE '100^%' ESCAPE '^' → 0

'100% pure' LIKE '100^% pure' ESCAPE '^' → 1

如果启用了NO_BACKSLASH_ESCAPES，ESCAPE子句就不能指定空串。

str REGEXP pattern

str NOT REGEXP pattern

REGEXP是正则表达式模式匹配操作的操作符。只要能在字符串表达式str里找到匹配模式字符串pattern，它的计算结果就为1；否则，它的计算结果将为0。NOT REGEXP操作符的计算情况正好与REGEXP操作符相反。下面两个表达式是等价的：

str NOT REGEXP pattern

NOT (str REGEXP pattern)

只要两个操作数里有一个为NULL，计算结果就为NULL。

如果有一个操作数是二进制串，那么 REGEXP 会按二进制串对字符串进行比较。如果操作数是非二进制串，则使用操作数的排序规则进行比较。

'abc' REGEXP 'ABC' → 1

BINARY 'abc' REGEXP 'ABC' → 0

'abc' REGEXP BINARY 'ABC' → 0

'abc' REGEXP 'ABC' COLLATE latin1_bin → 0

'abc' COLLATE latin1_bin REGEXP 'ABC' → 0

REGEXP操作符目前尚不支持多字节字符集，只适用于单字节字符集。

正则表达式与Unix实用工具程序grep和sed 使用的模式很相似。下表列出了各种允许使用的模式序列。

元素	含义
^	匹配字符串的开头
\$	匹配字符串的结尾
.	匹配任何单个的字符，包括换行符
[...]	匹配方括号内的任何一个字符
[^ ...]	匹配没有出现在方括号内的任何一个字符
e*	匹配模式元素e的0次或更多次出现
e+	匹配模式元素e的1次或更多次出现
e?	匹配模式元素e的0次或1次出现
e1 e2	匹配模式元素e1或e2
e{m}	匹配模式元素e的m次出现
e{m, }	匹配模式元素e的m次或更多次出现
e{, n}	匹配模式元素e的0次到n次出现
e{m, n}	匹配模式元素e的m次到n次出现
(...)	把括号中的模式元素当作一个元素来对待
其他	非特殊字符将与自身匹配

正则表达式模式不要求匹配模式与整个字符串相匹配，只要能在字符串里找到匹配模式就足够了。

'cats and dogs' REGEXP 'dogs' → 1

'cats and dogs' REGEXP 'cats'	→ 1
'cats and dogs' REGEXP 'c.*a.*d'	→ 1
'cats and dogs' REGEXP 'o'	→ 1
'cats and dogs' REGEXP 'x'	→ 0

^和\$分别用来匹配字符串的开头和末尾。

'abcde' REGEXP 'b'	→ 1
'abcde' REGEXP '^b'	→ 0
'abcde' REGEXP 'b\$'	→ 0
'abcde' REGEXP '^a'	→ 1
'abcde' REGEXP 'e\$'	→ 1
'abcde' REGEXP '^a.*e\$'	→ 1

[...]和[^...]构造用来设定字符类。在字符类里，可以用短划线（-）来设定字符区间，短划线的两端是该字符区间的第一个字符和最后一个字符。例如，[a-z]将匹配任何一个从“a”到“z”的小写字符，而[0-9]将匹配任何一个十进制数字。

'bin' REGEXP '^b[aciou]n\$'	→ 1
'bxn' REGEXP '^b[aciou]n\$'	→ 0
'oboeist' REGEXP '^ob[aciou]+st\$'	→ 1
'wolf359' REGEXP '[a-z]+[0-9]+'	→ 1
'wolf359' REGEXP '[0-9a-z]+'	→ 1
'wolf359' REGEXP '[0-9]+[a-z]+'	→ 0

如果想把字符“]”放到一个字符类里，它就必须是该字符类的第一个字符。如果想把字符“-”放到一个字符类里，它就必须是该字符类的第一个或

者最后一个字符。如果想把字符“^”放到一个字符类里，它就不能是“[”后面的第一个字符。

此外，还有几个特殊的正则表达式POSIX字符类构造（用于处理排序序列）和多个等效分类可以使用，如下表所示。

分类	含义
<code>[:alnum:]</code>	字母和数字字符
<code>[:alpha:]</code>	字母字符
<code>[:blank:]</code>	空白符（空格或制表符）
<code>[:cntrl:]</code>	控制字符
<code>[:digit:]</code>	十进制数字（0-9）
<code>[:graph:]</code>	图形字符（非空白字符）
<code>[:lower:]</code>	小写字母字符
<code>[:print:]</code>	图形或空格字符
<code>[:punct:]</code>	标点符号字符
<code>[:space:]</code>	空格、制表符、换行符或回车符
<code>[:upper:]</code>	大写字母字符
<code>[:xdigit:]</code>	十六进制数字（0-9, a-f, A-F）

POSIX 字符类构造器常用在字符类里，并且类名里包含有方括号字符“[”和“]”，因此在书写引用了这些名字的字符类表达式里，一定要记录包含足够的方括号。

<code>'abc' REGEXP '[:space:]'</code>	→ 0
<code>'a c' REGEXP '[:space:]'</code>	→ 1
<code>'abc' REGEXP '[:digit][:punct:]'</code>	→ 0
<code>'a0c' REGEXP '[:digit][:punct:]'</code>	→ 1
<code>'a,c' REGEXP '[:digit][:punct:]'</code>	→ 1

在字符类里，特殊标记[:<:]和[:>:]分别用于匹配单词边界的开始和结尾。单词字符通常可被认为是 `alnum` 字符类里的所有字符或下划线字符。对于由一个或多个单词字符构成的单词，在其前面和后面都不能是单词字符。

```
'a few words' REGEXP '[:<:]few[:>:]' → 1
```

```
'a few words' REGEXP '[:<:]fe[:>:]' → 0
```

对于正则表达式字符串里的转义序列，MySQL 使用了与 C 语言类似的语法。例如，“`\n`”、“`\t`”、“`\\`”会分别解释为换行符、制表符和反斜线字符（`\`）。如果需要在匹配模式里使用这几个字符，则必须双写反斜线字符（即把它们分别写成“`\\n`”、“`\\t`”、“`\\\\`”）。语法分析器在对查询语句进行分析时会去掉一个反斜线字符；在模式匹配进行过程中，会继续解释剩下的转义序列。

```
str RLIKE pattern
```

```
str NOT RLIKE pattern
```

`RLIKE`和`NOT RLIKE`分别等同于`REGEXP`和`NOT REGEXP`。

C.2 函数

调用函数可以进行计算，并返回值。默认情况下，在调用函数时函数名与紧随其后的左括号之间不允许出现空格；否则，会出现错误：

```
mysql> SELECT NOW();
```

```
-----  
NOW()  
-----
```

```
2013-01-08 15:34:46  
-----
```

```
mysql> SELECT NOW ();
```

```
ERROR 1630 (42000): FUNCTION    NOW does not exist
```

如果启用了SQL模式`IGNORE_SPACE`，服务器会允许在内建函数的名字后面出现空格，但是这种做法的副作用是所有的函数名都会被视为保留字。某些程序可以通过其他方式选择性地允许这种行为。例如，在启动mysql时，可以带上`--ignore-space`选项；在C语言程序里，可以在调用函数`mysql_real_connect()`时带上`CLIENT_IGNORE_SPACE`选项。

大部分情况下，可以用逗号来分隔某个函数的多个输入参数。函数参数的前后也允许出现空格。下面两行都是合法的：

```
CONCAT('abc','def')
```

```
CONCAT('abc', 'def')
```

有些函数使用的是另一种语法格式，如函数`TRIM()`或`EXTRACT()`：

```
TRIM(' ' FROM 'x')                                → 'x'
```

```
EXTRACT(YEAR FROM '2018-01-01')                    → 2018
```

下面将具体介绍每个函数的语法格式。

C. 2. 1 比较函数

这些函数用于对多个值进行比较。

ELT(n, str1, str2, ...)

此函数的返回值是字符串列表str1, str2, ... 里的第n个字符串。如果n为NULL、第n个字符串为 NULL 或不存在，则返回 NULL。第一个字符串的索引为 1。函数 ELT() 和FIELD() 之间为互补关系。

ELT(3,'a','b','c','d','e') → 'c'

ELT(0,'a','b','c','d','e') → NULL

ELT(6,'a','b','c','d','e') → NULL

ELT(FIELD('b','a','b','c'),'a','b','c') → 'b'

FIELD(arg0, arg1, arg2, ...)

在参数列表 arg1, arg2, ... 里找到与 arg0 相匹配的那个参数并返回该参数的索引（从 1 开始）。如果没能找到匹配或者arg0为NULL，则返回0。如果所有的参数都是字符串，则进行字符串比较；如果所有的参数都是数值，则进行数值比较；其他情况进行双精度浮点比较。函数FIELD() 和ELT() 之间为互补关系。

FIELD('b','a','b','c') → 2

FIELD('d','a','b','c') → 0

FIELD(NULL,'a','b','c') → 0

FIELD(ELT(2,'a','b','c'),'a','b','c') → 2

GREATEST(expr1, expr2, ...)

返回最大参数。其中的“最大”是根据以下规则定义的。

只要有一个参数为NULL，结果就为NULL。

如果是在整数上下文里调用此函数，或者它的输入参数全都为整数，那么所有输入参数将按整数方式进行比较。

如果是在浮点数上下文里调用此函数，或者它的输入参数全都为浮点数，则所有输入参数将按浮点数方式进行比较。

如果以上两条规则都不适用，则所有输入参数将使用C. 1. 4节描述的字符串比较规则按字符串方式进行比较。

GREATEST(2,3,1) → 3

GREATEST(38.5,94.2,-1) → 94.2

GREATEST('a','ab','abc') → 'abc'

GREATEST(1,3,5) → 5

GREATEST('A','b','C') → 'C'

GREATEST(BINARY 'A','b','C') → 'b'

IF (expr1, expr2, expr3)

若表达式expr1为真（非0，非NULL），则返回expr2；否则，返回expr3。IF () 函数的返回类型由后面的测试决定：如果expr2或expr3是字符串，则返回字符串；如果两个参数中有一个是浮点值，则返回浮点值；如果两个参数中有一个是整数，则返回整数。

IF(1,'true','false') → 'true'

IF(0,'true','false') → 'false'

IF(NULL,'true','false') → 'false'

IF(1.3,'nonzero','zero') → 'nonzero'

IF(0.3 <> 0,'nonzero','zero') → 'nonzero'

请注意，IF () 函数与E. 2. 1节描述的IF语句有所不同。

IFNULL (expr1, expr2)

若表达式expr1的值为NULL，则返回expr2；否则，返回expr1。IFNULL () 函数将根据自己调用时的上下文决定是返回数字，还是返回字符串。

IFNULL(NULL,'null') → 'null'

IFNULL('not null','null') → 'not null'

INTERVAL (n, n1, n2, ...)

如果 $n < n_1$ ，则返回0；如果 $n < n_2$ ，则返回1；依此类推；如果 n 为NULL，则返回-1。即是说，INTERVAL () 函数会在其余参数所定义的各个区间里找到第一个参数的位置。所有参数都必须是整数。因为此函数使用的是快速折半查找法，所以参数 n_1 、 n_2 、...的顺序必须严格递增（即 $n_1 < n_2 < \dots$ ）。否则，INTERVAL () 的行为无法预测。

INTERVAL(2,0,1,3) → 2

INTERVAL(7,1,3,5,7,9) → 4

ISNULL (expr)

若表达式expr的值为NULL，则返回1；否则，返回0。

ISNULL(NULL) → 1

ISNULL(0) → 0

ISNULL(1) → 0

LEAST (expr1, expr2, ...)

返回最小参数。其中的“最小”是按与GREATEST () 函数相同的规则定义的。

LEAST(2,3,1) → 1

LEAST(38.5,94.2,-1) → -1

LEAST('a','ab','abc') → 'a'

NULLIF (expr1, expr2)

如果两个表达式的值不相同，返回expr1；若相同，则返回NULL。

NULLIF(3,4) → 3

NULLIF(3,3) →
NULL

STRCMP(str1, str2)

此函数将根据第一个参数str1是大于、等于还是小于第二个参数str2来返回1、0或-1。只要两个参数中有一个为NULL值，此函数就会返回NULL。只要两个操作数中有一个是二进制串，STRCMP()就按二进制串比较。如果两个操作数都是非二进制串，则按操作数排序规则比较。

STRCMP('a','a') → 0

STRCMP('a','A') → 0

STRCMP(BINARY 'a','A') → 1

STRCMP('A' COLLATE latin1_general_ci,'a') → 0

STRCMP('A' COLLATE latin1_general_cs,'a') → -1

C. 2. 2 类型转换函数

此类函数能够把数值从一种类型转换为另一种类型。

CAST(expr AS type)

把表达式的expr 值转换为给定的类型。其中，type 可以是 BINARY[(n)]（二进制串）、CHAR[(n)]（非二进制串）、DATE、DATETIME、TIME、SIGNED [INTEGER]、UNSIGNED [INTEGER]或DECIMAL[(M[, D])]

CAST(304 AS BINARY) →
'304'

CAST(-1 AS UNSIGNED) →
18446744073709551615

CAST(13 AS DECIMAL(5,2)) → 13.00

如果参数type为BINARY或CHAR，则可以指定可选长度n，这样可以把转换结果的长度限定在n个字节或n个字符以内。如果参数type是BINARY，则少于n个字节的值将用尾缀的零值字节（0x00）补齐到长度等于n为止。

在需要使用CREATE TABLE ... SELECT 语句来创建一个新表的时候，人们经常会利用CAST() 函数把某些列强行设置为指定的类型。

```
mysql> CREATE TABLE t SELECT CAST(20130101 AS DATE) AS date_val;
```

```
mysql> SHOW COLUMNS FROM t;
```

Field	Type	Null	Key	Default	Extra
date_val	date	YES		NULL	

```
mysql> SELECT * FROM t;
```

date_val
2013-01-01

函数CAST() 与CONVERT() 相似，但是前者遵循的是标准SQL语法，而后者遵循的却是ODBC语法。

CONVERT(expr, type)

CONVERT(expr USING charset)

CONVERT() 函数的第一种形式与CAST() 函数的功能完全相同，只是使用的语法稍微有些不同外。参数expr和type的用法也完全一致。第二种（即USING）形式会把值转换成具有指定字符集格式的字符串。

CONVERT(304,BINARY) →
'304'

CONVERT(-1,UNSIGNED) →
18446744073709551615

CONVERT('abc' USING utf8); → 'abc'

C. 2. 3 数值函数

如果你传递的参数超出了数值函数的允许范围或者是一个无效值，那么函数会返回NULL。

ABS(x)

返回x的绝对值。

ABS(13.5) →
13.5

ABS(-13.5) →
13.5

ACOS(x)

返回x的反余弦值。如果x不在-1~1的区间内，则返回NULL。

ACOS(1) →
0

ACOS(0) →
1.5707963267949

ACOS(-1) →
3.1415926535898

ASIN(x)

返回x的正弦值。如果x不在-1~1的区间内，则返回NULL。

ASIN(1) →
1.5707963267949

ASIN(0) →
0

ASIN(-1) →
-1.5707963267949

ATAN(x)

ATAN(y, x)

只带一个输入参数的ATAN()函数将返回 x 的反正切值。带两个参数的函数形式等同于ATAN2()函数。

ATAN(1) →
0.78539816339745

ATAN(0) →
0

ATAN(-1) →
-0.78539816339745

ATAN2(y, x)

相当于ATAN2(y/x)，但它要根据两个输入参数的正负符号来判断返回值将落在哪一个坐标象限里。

ATAN2(1,1) →
0.78539816339745

ATAN2(1,-1) →
2.3561944901923

ATAN2(-1,1) →
-0.78539816339745

ATAN2(-1,-1) →
-2.3561944901923

CEILING(x)

CEIL(x)

返回不小于 x 的最小整数。如果参数 x 是某种精确数值类型的，返回值也将有着同样的类型。否则，返回值将有着某种浮点（近似）数值类型。即使此函数的返回值没有小数部分，也是如此。

CEILING(3.8) → 4

CEILING(-3.8) → -3

COS(x)

返回x的余弦值。其中，x是一个弧度值。

COS(0) → 1

COS(PI()) →
-1

COT(x)

返回x的余切值。其中，x是一个弧度值。

COT(PI()/4) → 1

CRC32(str)

根据参数（它被当成一个字符串）计算循环冗余校验值。返回值是一个32位（即0~2³²—1之间）的无符号值；如果输入参数为NULL，则返回NULL。

CRC32('xyz') →
3951999591

CRC32('0') →
4108050209

CRC32(0) →
4108050209

CRC32(NULL) →
NULL

DEGREES(x)

返回弧度值x的角度值。

DEGREES(PI())	→
180	

DEGREES(PI()*2)	→
360	

DEGREES(PI()/2)	→ 90
-----------------	------

DEGREES(-PI())	→
-180	

EXP(x)

返回 e^x （e的x次方）。e是自然对数的底数。

EXP(1)	→
2.718281828459	

EXP(2)	→
7.3890560989307	

EXP(-1)	→
0.36787944117144	

1/EXP(1)	→
0.36787944117144	

FLOOR(x)

返回不大于x的最大整数。如果参数x是某种精确数值类型的，返回值也将有着同样的类型；否则，返回值将有着某种浮点（近似）数值类型，即使此函数的返回值没有小数部分。

FLOOR(3.8)	→ 3
------------	-----

FLOOR(-3.8)	→ -4
-------------	------

LN(x)

此函数等同于LOG() 函数。

LOG(x)

LOG(b, x)

只带一个输入参数的LOG(x) 函数将返回x以e为底的自然对数。

LOG(0)	→
NULL	

LOG(1)	→
0	

LOG(2)	→
0.69314718055995	

LOG(EXP(1))	→ 1
-------------	-----

两个参数形式的将返回x以b为底的对数。

LOG(10,100)	→ 2
-------------	-----

LOG(2,256)	→ 8
------------	-----

你可以利用公式LOG(x)/LOG(b) 来计算出x以b为底的对数。

LOG(100)/LOG(10)	→ 2
------------------	-----

LOG10(100)	→ 2
------------	-----

LOG10(x)

返回x以10为底的对数。

LOG10(0)	→
NULL	

LOG10(10)	→
1	

LOG10(100) → 2

LOG2(x)

返回x以2为底的对数。

LOG2(0) →
NULL

LOG2(255) →
7.9943534368589

LOG2(32767) →
14.99995597177

LOG2()能够让你了解以位计算的数值“宽度”。人们经常利用此函数来估算某个值的存储空间占用量。

MOD(m, n)

MOD()函数执行的是模运算。MOD(m, n)的函数语法等价于操作符语法 $m \% n$ 或 $m \text{ MOD } n$ （参考C.1.3节）。

PI()

返回圆周率 π 。

PI() →
3.141593

POW(x, y)

POWER(x, y)

返回 x^y ，即x的y次方。

POW(2,3) →
8

POW(2,-3) →
0.125

POW(4,0.5) → 2

POW(16,0.25) → 2

RADIANS (x)

返回角度值x的弧度值。

RADIANS(0) →

0

RADIANS(360) →

6.2831853071796

RADIANS(-360) →

-6.2831853071796

RAND ()

RAND (n)

RAND () 函数返回的是一个介于0.0和1.0之间的随机浮点值。参数n（如果给定了）应该为一个整数。如果参数为常量，那么 RAND () 函数将以这个常量作为随机数发生器的种子值。如果需要按照同样的顺序为结果集里的某个列生成一组重复的随机数，只要用同样的值作为种子就可以达到目的。

RAND() →

0.1036697114852

RAND() →

0.5725383884949

RAND(10) →

0.65705152196535

RAND(10) →

0.65705152196535

如果该参数不是常量，那么每次调用RAND ()时都会将它作为种子值。
(例如，如果你指定了某个列，那么对于每一个记录行，该列的值都会做为该记录行的随机数产生器的种子。)

种子操作与客户端特定相关。一个客户端调用RAND (n) 来提供给随机数发生器的种子，并不会对其他客户端调用该函数返回的随机数产生影响。

如果RAND () 出现在WHERE子句中，那么每次执行该子句时，它都会被调用。

ROUND (x)

ROUND (x, d)

ROUND () 函数将返回x的值，但只保留到小数点后面的d位数字。如果d等于零或没有给出，结果将没有小数点或小数部分。此函数的返回值和它的第一个参数有着同样的数值类型，所以如果该参数是一个整数的话，返回结果将没有任何小数部分。以字符串形式给出的数值将先转换为双精度浮点数，再进行舍入处理。

ROUND(15.3) →
15

ROUND(15.5) →
16

ROUND(-33.27834,2) →
-33.28

ROUND(1,4) →
1

ROUND('1',4) →
1.0000

如果d是一个负数，那么ROUND () 函数会去掉小数部分，把从小数点开始往左算起的ABS (d) 位数字设置为零。

ROUND(123456,-2) →
123500

ROUND () 函数对x参数的舍入处理遵循以下规则。

对于近似值数值，舍入行为仍取决于其底层的算术函数库。

对于精确值数值，大于或等于0.5的小数部分将按远离零的方向舍入。小于0.5的小数部分将按接近零的方向舍入。例如，1.5和-1.5将被分别舍入为2和-2，而1.49和-1.49将被舍入为1和-1。

有关精确值和近似值的更多讨论请参考3.1.1.1节。

SIGN(x)

根据x的值是负数、0还是正数，分别返回-1、0或1。

	SIGN(15.803)	→ 1
0	SIGN(0)	→
-1	SIGN(-99)	→

SIN(x)

返回x的正弦值，其中，x是一个弧度值。

	SIN(0)	→ 0
	SIN(PI()/2)	→ 1

SQRT(x)

返回x的非负平方根。

25	SQRT(625)	→
1.5	SQRT(2.25)	→
NULL	SQRT(-1)	→

TAN(x)

返回x的正切值，其中，x是一个弧度值。

TAN(0) →
0

TAN(PI()/4) → 1

TRUNCATE(x, d)

返回小数部分被截短为d位数字的x值。如果d等于0，返回值里将不包含小数点和小数部分。如果d大于x的小数位，x的小数部分将用0来补足到指定的位数。

TRUNCATE(1.23,1) → 1.2

TRUNCATE(1.23,0) → 1

TRUNCATE(1.23,4) →
1.2300

如果d是负数，TRUNCATE()会去掉小数部分，把小数点左边的ABS(d)个数变为0。

TRUNCATE(123456.789,-3) →
123000

C. 2. 4 字符串函数

本节绝大多数字符串函数的返回值仍将是一个字符串。有些函数，比如LENGTH()，其参数为字符串，但返回的是数值。对于那些根据字符串位置处理字符串类的函数，我们将把字符串的头一个字符（字符串的最左端）称为第1个字符（而不是第0个字符）。

有些字符串函数能够支持多字节字符：CHAR_LENGTH()、INSERT()、INSTR()、LCASE()、LEFT()、LOCATE()、LOWER()、LTRIM()、MID()、POSITION()、REPLACE()、REVERSE()、RIGHT()、RPAD()、RTRIM()、SUBSTRING()、SUBSTRING_INDEX()、TRIM()、UCASE()和UPPER()。

ASCII(str)

返回字符串str最左端的那个字节的ASCII编码，整数值，范围为0～255。如果str是一个空串，则返回0；如果str为NULL，则返回NULL。str应该只包含8位的字符。

ASCII('abc') → 97

ASCII("") → 0

ASCII(NULL) →
NULL

BIN(n)

返回以字符串表示的参数n的二进制表示形式。下面两个表达式是等价的，更多详细信息请参考对CONV()函数的介绍。

BIN(65) →
'1000001'

CONV(65,10,2) →
'1000001'

CHAR(n1, n2, ... [USING charset])

它会把参数 n1, n2, ... 解释为一组数字字符编码值，并根据它们用当前字符集里的对应字符构成一个字符串作为其返回值。大于 255的字符编码值将被解释为多字节字符的字符编码。如果没有使用USING 选项，那么 CHAR()函数的返回值将是一个二进制串。如果使用USING 选项，那么返回值将由该选项所指定的字符集里的字符构成。如果结果不是指定字符集里的合法字符，那么CHAR()函数将返回一条警告消息（如果启用了SQL严格模式，那么结果将是NULL）。NULL参数会被忽略。

CHAR(65) →
'A'

CHAR(97) →
'a'

CHAR(89,105,107,101,115,33) → 'Yikes!'

CHAR_LENGTH(str)

CHARACTER_LENGTH(str)

这两个函数与LENGTH()差不多，但参数长度以字符计算，而非字节。
(一个多字节字符的长度为1。)

CHARSET(str)

返回给定字符串的字符集名；如果参数为NULL，则返回NULL。

CHARSET('abc') →
'latin1'

CHARSET(CONVERT('abc' USING utf8)) → 'utf8'

CHARSET(123) →
'binary'

COALESCE(expr1, expr2, ...)

返回输入参数中的第一个非NULL元素；如果所有元素全都为NULL，则返回NULL。

COALESCE(NULL,1/0,2,'a',45+97) → '2'

COALESCE(NULL,1/0) →
NULL

COERCIBILITY(str)

返回字符串 str的排序可压缩性；如果该参数非法，则返回 NULL。所谓“可压缩性”(coercibility)，指的是某个字符串在涉及其他字符串的表达式里改变其排序规则的程度。下表列出了各种返回值，它们都按可压缩性从低到高的顺序排列。

可压缩性	含义
0	排序规则是显式设定的，不能转换
1	未设定排序规则
2	排序规则是隐式设定的
3	排序规则由 USER() 等系统值设定
4	排序规则可压缩
5	排序规则可忽略（如设置为 NULL）

COERCIBILITY(_utf8 'abc' COLLATE utf8_bin) → 0

COERCIBILITY('abc') → 4

COLLATION(str)

返回给定字符串str的排序规则名称；如果该参数非法，则返回NULL。

COLLATION(_latin2 'abc') →
'latin2_general_ci'

COLLATION(CONVERT('abc' USING utf8) COLLATE utf8_bin)
→ 'utf8_bin'

CONCAT(str1, str2, ...)

返回一个由自身所有参数连接而成的字符串；只要有一个参数为NULL，就返回NULL。只要有一个参数是二进制串，结果就是一个二进制串；如果所有参数都是非二进制串，结果就是一个非二进制串。数字参数都可以转换为字符串。自MySQL 5.5.3起，这种转换得到的是非二进制串；那以前的版本，转换结果为二进制串。

CONCAT('abc','def') → 'abcdef'

CONCAT('abc') →
'abc'

CONCAT('abc',NULL) →
NULL

CONCAT('Hello',' ','goodbye') → 'Hello, goodbye'

连接字符串的另一个办法是相邻排列，即一个接一个地写出它们。

'three' 'blind' 'mice' →
'threeblindmice'

'abc' 'def' = 'abcdef' → 1

CONCAT_WS(delim, str1, str2, ...)

与CONCAT()类似，返回值为由第2个及后续输入参数合并在一起而得到的一个字符串，各输入参数之间用字符串delim加以分隔。如果delim为NULL，则返回NULL；但参加连接的字符串里的NULL值会被忽略。

CONCAT_WS(',', 'a', 'b', '', 'd') → 'a,b,,d'

CONCAT_WS('*-', 'lemon', 'lime', NULL, 'grape') → 'lemon*-*lime*-*grape'

CONV(n, from_base, to_base)

给定一个以from_base为底的数值n，返回一个以to_base为底的数值的字符串表示。如果有参数为NULL，则返回NULL。参数from_base和to_base必须是一个2~36之间的整数。n将被当作BIGINT（64位整数）值，但不能被指定为字符串，因为底数大于10的数值可能会包含非十进制数字。（这也正是CONV()函数为何返回一个字符串的原因。底数在11~36之间的数值经过转换得到的字符可能是从A到Z的字母。）如果n不是一个合法的以from_base为底的数字，那么此函数的返回结果将为0。例如，如果from_base为16，n为'abcdefg'，则结果为0，因为g不是一个合法的十六进制数字。

数值n里的非十进制字符既可以是大小写字母，也可以是小写字母。但结果里的非十进制数字都是大写。

下面的例子将把十六进制表示的14转换为二进制：

CONV('e',16,2) → '1110'

下面的例子将把二进制表示的255转换为八进制：

CONV(11111111,2,8) → '377'

CONV('11111111',2,8) → '377'

默认情况下，数值 `n` 将被视为一个无符号数。如果给定的`to_base` 是一个负数，那么`CONV()` 函数将把`n`视为一个有符号数。

CONV(-10,10,16) →
'FFFFFFFFFFFFFFF6'

CONV(-10,10,-16) → '-A'

EXPORT_SET(`n`, `on`, `off`[, `delim`, [`bit_count`]])

返回一个由字符串`on`和`off`构成、以字符串`delim`为分隔符的字符串。默认的分隔符是逗号。`on`用来表示值`n`中被置位（即等于1）的每个位，`off`用来表示`n`中没被置位（即等于0）的每个位。结果中最左边的字符串对应于`n` 中最下边的位。`bit_count` 是对`n` 值进行如此转换的最大位数。`bit_count`的默认值是64，这也是它的最大值。只要输入参数中有NULL值，此函数的返回值就为NULL。

EXPORT_SET(7,'+', '-',5) → '++++--'

EXPORT_SET(0xa,'1','0',",",6) → '010100'

EXPORT_SET(97,'Y','N',';',8) →
'Y,N,N,N,N,Y,Y,N'

FIND_IN_SET(`str`, `str_list`)

`str_list`是由一些以逗号分隔的子串（即类似于MySQL中的SET值）构成的一个字符串。`FIND_IN_SET()` 将返回字符串`str`在`str_list`中的下标。如果`str`没有出现在`str_list`里，则返回0；只要输入参数中有NULL值，就返回NULL。第一个子串的下标是1。

FIND_IN_SET('cow','moose,cow,pig') → 2

FIND_IN_SET('dog','moose,cow,pig') → 0

FORMAT(`x`, `d`[, `locale`])

使用一个类似 'nn, nnn. nnn' 的格式，将数值x格式化成小数点后带d位小数的数字，返回值是一个字符串。如果d等于0，则返回值中将不包含小数点或小数部分。

```
FORMAT(1234.56789,3) →  
'1,234.568'
```

```
FORMAT(999999.99,2) →  
'999,999.99'
```

```
FORMAT(999999.99,0) →  
'1,000,000'
```

请注意最后一个例子中的数值舍入行为。

本地语言决定了小数点、千位分隔符以及分隔符之间的分组方式。默认本地语言为 'en_US'，它可以使用可选的 `locale` 参数来改写。所有允许值等同于系统变量 `lc_time_names` 的值。

`FROM_BASE64(str)`

转换一个base-64 编码字符串，并返回原始值；如果参数不是 base-64 编码字符串，则返回NULL。此函数是 `TO_BASE64()` 的逆操作。

```
TO_BASE64('hello') →  
'aGVsbG8='
```

```
FROM_BASE64(TO_BASE64('hello')) → 'hello'
```

`FROM_BASE64()` 是在MySQL 5.6.1里引入的。

`HEX(n)`

`HEX(str)`

如果输入参数是一个数值n，那么`HEX()`函数将返回n的十六进制表示，返回值是一个字符串。下面两个表达式是等价的（更多详细信息请参考对`CONV()`函数的介绍）：

```
HEX(65) →  
'41'
```

CONV(65,10,16) → '41'

如果输入参数是一个字符串，那么 HEX() 函数将把参数中的每一个字符转换为一个两位数的十六进制数，组成一个字符串返回。HEX() 是 UNHEX() 的逆操作。

HEX('255') →
'323535'

HEX('abc') →
'616263'

UNHEX(HEX('abc')) →
'abc'

INSERT(str, pos, len, ins_str)

把字符串str从第pos个位置开始的len个字符替换为ins_str，得到的字符串即为返回值。如果pos超出字符串str的长度范围，则返回原来的字符串；如果有输入参数为NULL，则返回NULL。

INSERT('nighttime',6,4,'fall') → 'nightfall'

INSERT('sunshine',1,3,'rain or') → 'rain or shine'

INSERT('sunshine',0,3,'rain or') → 'sunshine'

INSTR(str, substr)

INSTR() 类似于带两个输入参数的LOCATE() 函数，只是参数的顺序相反。下面的表达式是等价的：

INSTR(str,substr)

LOCATE(substr,str)

LCASE(str)

等同于LOWER() 函数。

LEFT(str, len)

返回字符串str最左面的len个字符；如果len大于str的长度，则返回整个字符串str。如果len为NULL或者小于1，则返回一个空串。如果str为NULL，则返回NULL。

LEFT('my left foot',2) → 'my'

LEFT(NULL,10) →
NULL

LEFT('abc',NULL) →
NULL

LEFT('abc',0) → ''

LENGTH(str)

返回字符串str的长度，以字节为单位。多字节字符长度大于1。如果要确定字符长度，可以使用CHAR_LENGTH()。

LENGTH('abc') → 3

LENGTH(CONVERT('abc' USING ucs2)) → 6

LENGTH('') → 0

LENGTH(NULL)
→ NULL

LOCATE(substr, str)

LOCATE(substr, str, pos)

带有两个参数形式的LOCATE()函数，会返回字符串substr在字符串str里第一次出现的位置；如果substr没有出现在字符串str里，则返回0。如果有输入参数为NULL，则返回NULL。如果给出了位置参数pos，LOCATE()函数将在字符串str里以pos为起点去寻找字符串substr。如果有一个操作数是二进制串，LOCATE()将按二进制串比较字符串；如果str和substr都不是二进制串，比较操作将按操作数排序规则进行。

LOCATE('b','abc') → 2

LOCATE('b','ABC') → 2

LOCATE(BINARY 'b','ABC') → 0

LOCATE('b' COLLATE latin1_general_ci,'ABC') → 2

LOCATE('b' COLLATE latin1_general_cs,'ABC') → 0

LOWER(str)

返回把字符串str里的字符全都转换为小写字母后得到的一个字符串；如果str为NULL，则返回NULL。

LOWER('New York, NY') → 'new
york, ny'

LOWER(NULL)
→ NULL

LOWER()函数是根据其参数字符集的排序规则来进行大小写转换的。如果它的参数是一个二进制串，LOWER()函数将原封不动地返回该参数本身，因为二进制串没有任何字符集和排序规则。

LOWER(BINARY 'New York, NY') → 'New
York, NY'

LOWER(0x414243) →
'ABC'

如果这不是你想要的结果，可以把它的参数强制转换为一个有着适当排序规则的非二进制串。

LOWER(CONVERT(BINARY 'New York, NY' USING latin1))
→ 'new york, ny'

LOWER(_latin1 0x414243) → 'abc'

LPAD(str, len, pad_str)

返回在字符串str的左侧用字符串pad_str补足到长度等于len个字符时得到的一个字符串。如果某个参数为NULL，则返回NULL。

LPAD('abc',12,'def') → 'defdefdefabc'

LPAD('abc',10, '.') → '.....abc'

如果字符串str的长度已经超过len个字符，那么LPAD() 将把字符串str截短为len个字符。

LPAD('abc',2, '.') → 'ab'

LTRIM(str)

返回去掉字符串str最左端的（打头的）空格后得到的那个字符串；如果str为NULL，则返回NULL。

LTRIM(' abc ') → 'abc '

MAKE_SET(n, bit0_str, bit1_str, ...)

根据整数值n和字符串bit0_str, bit1_str, ... 构造出一个SET 值（即一个以逗号来分隔各个子串的字符串）。n 值中每一个被置位（即等于 1）的位所对应的子串都将被包括在返回值里。（如果设置了0位，则结果会包含 bit0_str，如此等等。）如果n等于0，则返回一个空串。如果n为NULL，则返回NULL。在构造结果字符串时，列表中值为NULL的字符串会被忽略。

MAKE_SET(8,'a','b','c','d','e') → 'd'

MAKE_SET(1|2|4,'a','b','c','d','e') → 'a,b,c'

MAKE_SET(2+16,'a','b','c','d','e') → 'b,e'

MAKE_SET(-1,'a','b','c','d','e') → 'a,b,c,d,e'

在最后一个例子里，因为n值—1翻转了所有位，所以返回值包含了每一个子串。

MATCH(col_list) AGAINST(str [search_mode])

MATCH 将使用一个FULLTEXT 索引来执行一次搜索操作。MATCH 列表由一个或多个列名构成，各列名以逗号分隔。它们必须是组成正被搜索的那张表的FULLTEXT 索引的列。AGAINST() 里的str 参数表示的是想要给定的那些列里进行查找的一个或者多个单词。这些单词是由字母、数字、单引号或者下划线字符构成的字符序列。MATCH 里允许出现括号，但AGAINST里不允许出现括号。

默认情况下，是以自然语言模式执行搜索。可以显式地将search_mode参数设置为下面这些值：

IN NATURAL LANGUAGE MODE

IN BOOLEAN MODE

[IN NATURAL LANGUAGE MODE] WITH QUERY EXPANSION

对于自然语言搜索，MATCH() 给出的是被搜索单词在每个行里的相关性排序。这些排序值都是非负的浮点数：如果排序值为 0，则表示在表里没有找到被搜索单词；如果排序值为正，则表明至少找到了一个被搜索单词。那些在表中一半以上的行里都有的单词（它们的相关性被认为是零，因为它们的出现次数太多了）会被忽略。此外，MySQL 内部还有一个停用词（比如“the”和“but”）列表，其中的单词都会被当作不相关。

如果搜索模式为IN BOOLEAN MODE，那么搜索结果将以被搜索单词是否出现过为依据，而不是以它们的出现频率为依据。在布尔搜索方式中，你还可以通过给被搜索单词加上以下几种操作符来影响搜索操作的具体行为。

打头的+或-分别表示该单词必须出现或必须不出现。

打头的<或>会减少或增加该单词对相关度计算结果的贡献。

打头的~将使给定单词在相关度计算里的贡献值变换正负号，但不会像-前导字符那样把包含该单词的行完全排除在外。

出现在被搜索单词尾部的*被看作是一个通配符。例如，act*将匹配act、acts、action等。

短语搜索的实现方式是用双引号将该短语括起来（如"phase"）。只有该短语存在，且排列顺序一致，才能算匹配成功。

多个被搜索单词可以用括号分组为一个表达式。括号表达式可以嵌套。

在布尔搜索方式中，那些不带修饰符的被搜索单词都是可选的，与普通语言搜索方式中的含义相同。

布尔模式搜索在表没有相应的FULLTEXT索引时也能进行，但速度往往会非常慢。

如果搜索模式为WITH QUERY EXPANSION或IN NATURAL LANGUAGE MODE WITH QUERY EXPANSION，那么要先使用搜索字符串进行搜索，然后使用搜索字符串和初次搜索最为匹配的少量信息再次搜索，才能完成自然语言搜索。这样能找出内容与初次搜索字符串相关的数据行。

更多与FULLTEXT搜索相关的信息请参考2.14节。

MID(str, pos, len)

MID(str, pos)

带3个参数形式的MID()将返回字符串str从位置pos开始且长度为len个字符的那一个子串。2个参数形式的MID()将返回字符串str从位置pos到最后一个字符的那一个子串。如果某个参数为NULL，则返回NULL。

MID('what a dull example',8,4) → 'dull'

ID('what a dull example',8) → 'dull example'

事实上，MID()等同于SUBSTRING()函数，并且可以利用SUBSTRING()函数支持的各种语法形式。

OCT(n)

返回包含参数n的八进制表示形式的一个字符串。下面两个表达式是等价的（更多详细信息请参考对CONV()函数的介绍）：

OCT(65) →
'101'

CONV(65,10,8) →
'101'

OCTET_LENGTH(str)

此函数等同于LENGTH()函数。

ORD(str)

返回字符串str第一个字符的排位序号；如果str为NULL，则返回NULL。
如果第一个字符是一个单字节字符，ORD()函数将等价于ASCII()函数。

ORD('abc') → 97

ASCII('abc') → 97

对于多字节字符，ORD()函数将返回一个根据该字符的各个字节（b1～bn，顺序为从右至左）确定的值：

$b1 + (b2 \times 256) + (b3 \times 256 \times 256) + \dots$

POSITION(substr IN str)

本函数相当于只带两个输入参数的LOCATE()函数。下面的表达式是等价的：

POSITION(substr IN str)

LOCATE(substr, str)

QUOTE(str)

按SQL语句的使用要求在输入参数str 里正确地添加各种引号后得到的一个字符串。此函数在编写能生成其他查询语句的查询语句时非常有用。对于非NULL值，返回值里的每个单引号、反斜线、Ctrl+Z字符和NUL字符（即0值字节）都会带有一个进行转义的引导斜线，而整个结果会用单引号引起来。如果 str 为 NULL，则返回值为不带单引号的单词“NULL”。

QUOTE("Let's go!") → 'Let\'s
go!'

QUOTE(NULL)
→ NULL

REPEAT(str, n)

返回值是把字符串str重复n次后得到的一个字符串。如果n是非正数，则返回一个空串；如果其中有一个参数为NULL，则返回NULL。

REPEAT('x',10) →
'xxxxxxxxxx'

REPEAT('abc',3) →
'abcabcabc'

REPLACE(str, from_str, to_str)

返回值是把字符串str中的子串from_str全部替换为to_str后得到的一个字符串。如果to_str是空串，则效果相当于删除所有的from_str。如果from_str是空串，REPLACE()将不对字符串str做任何改变。如果有参数为NULL，则返回NULL。

REPLACE('abracadabra','a','oh') →
'ohbrohcohdohbroh'

REPLACE('abracadabra','a','') → 'brcdbr'

REPLACE('abracadabra','','x') → 'abracadabra'

REVERSE(str)

返回值是前后颠倒字符串str 里的所有字符后得到的一个字符串。若str为NULL，则返回NULL。

REVERSE('abracadabra') →
'arbadacarba'

REVERSE('tararA ta tar a raT') → 'Tar a rat at
Ararat'

RIGHT(str, len)

返回字符串str最右边的len个字符；如果没有足够的字符，则返回整个字符串。如果len为NULL或小于1，则返回一个空串。如果有参数为NULL，则返回NULL。

RIGHT('rightmost',4) → 'most'

RPAD(str, len, pad_str)

返回在字符串 `str` 的右侧用子串 `pad_str` 补足到长度等于 `len` 个字符时得到的一个字符串。如果有参数为 `NULL`，则返回 `NULL`。

`RPAD('abc',12,'def')` →
'abcdefdefdef'

`RPAD('abc',10,'.')` → 'abc.....'

如果字符串 `str` 的长度已经超过 `len` 个字符，则 `RPAD()` 将把字符串 `str` 截短为 `len` 个字符。

`RPAD('abc',2,'.')` → 'ab'

`RTRIM(str)`

返回去掉字符串 `str` 最右端（尾部）的空格后得到的那个字符串；如果 `str` 为 `NULL`，则返回 `NULL`。

`RTRIM('abc')` →
'abc'

`SOUNDEX(str)`

`expr1 SOUNDS LIKE expr2`

返回根据字符串 `str` 计算出来的一个桑迪克斯串；如果 `str` 为 `NULL`，则返回 `NULL`。字符串 `str` 中非字母数字字符都将被忽略。不在从 A 到 Z 范围以内的非字母国际字符都将被视为元音。对于有多字节字符的字符串或英语之外的其他语言，`SOUNDEX()` 的结果可能是无意义的。

`SOUNDEX('Cow')` →
'C000'

`SOUNDEX('Cowl')` →
'C400'

`SOUNDEX('Howl')` →
'H400'

`SOUNDEX('Hello')` →
'H400'

SOUNDS LIKE操作符等价于SOUNDEX() 函数。

SPACE (n)

返回一个由 n个空格构成的字符串。如果 n 不是一个正数，则返回一个空串；如果 n 为NULL，则返回NULL。

SPACE(6) → "

SPACE(0) → "

SPACE(NULL) →

NULL

SUBSTR (arguments)

SUBSTR () 等同于SUBSTRING () 。参数格式也一样。

SUBSTRING (str, pos)

SUBSTRING (str, pos, len)

SUBSTRING (str FROM pos)

SUBSTRING (str FROM pos FOR len)

返回字符串str从位置pos开始的一个子串；如果有参数为NULL，则返回NULL。如果给出了len参数，则作为返回值子串的长度将是len个字符；否则，将返回字符串str从位置pos开始直到最后一个字符的子串。

SUBSTRING('abcdef',3) → 'cdef'

SUBSTRING('abcdef',3,2) → 'cd'

下面的表达式是等价的：

SUBSTRING(str,pos,len)

SUBSTRING(str FROM pos FOR len)

MID(str,pos,len)

`SUBSTRING_INDEX(str, delim, n)`

返回字符串`str`的一个子串。如果`n`是正值，`SUBSTRING_INDEX()`函数将按从左向右的顺序找到字符串 `delim`的第 `n` 次出现，并返回该位置左侧的全部内容。如果 `n` 为负值，`SUBSTRING_INDEX()`函数将按从右向左的顺序找到 `delim`的第`n`次出现，并返回该位置右侧的全部内容。如果`SUBSTRING_INDEX()`函数在字符串`str`没有里找到`delim`，则原样返回字符串`str`。如果有参数为`NULL`，则返回`NULL`。

`SUBSTRING_INDEX('jar-jar','j',-2)` → 'ar-jar'

`SUBSTRING_INDEX('sampadm@localhost','@',1)` → 'sampadm'

`SUBSTRING_INDEX('sampadm@localhost','@',-1)` → 'localhost'

`TO_BASE64(str)`

返回参数的base-64编码；如果参数为`NULL`，则返回`NULL`。数字参数会被当作字符串来处理。此函数是`FROM_BASE64()`的逆操作。

`TO_BASE64('abc')` →
'YWJj'

`TO_BASE64('123')` →
'MTIz'

`TO_BASE64(123)` →
'MTIz'

`TO_BASE64()`是在MySQL 5.6.1里引入的。

`TRIM([trim_str FROM] str)`

`TRIM([|LEADING | TRAILING | BOTH] [trim_str] FROM] str)`

第一种形式，返回的是从字符串`str`的首部和尾部去掉字符串`trim_str`后得到的结果。对于第二种形式，如果指定了`LEADING`，则`TRIM()`会将开头的`trim_str`去掉。如果指定了`TRAILING`，则`TRIM()`会将尾部的`trim_str` 去掉。如果指定了`BOTH`，则`TRIM()`会将首尾的`trim_str`去掉。如果没有指定`LEADING`、`TRAILING`或`BOTH`，则默认为`BOTH`。如果没有指定`trim_str`，则`TRIM()`将会把空格去掉。

TRIM('^' FROM '^^^xyz^^') → 'xyz'

TRIM(LEADING '^' FROM '^^^xyz^^') → 'xyz^^'

TRIM(TRAILING '^' FROM '^^^xyz^^') → '^^^xyz'

TRIM(BOTH '^' FROM '^^^xyz^^') → 'xyz'

TRIM(BOTH FROM ' abc ') →
'abc'

TRIM(' abc ') →
'abc'

UCASE(str)

此函数等同于UPPER() 函数。

UNHEX(expr)

参数被解释为包含几对十六进制数字的字符串。每对数字被转换为一个字符，返回值是一个由这些字符组成的二进制串。UNHEX() 是HEX() 的逆操作。

UNHEX('414243') →
'ABC'

HEX(UNHEX('414243')) →
'414243'

UNHEX(HEX('ABC')) →
'ABC'

UNHEX(414243) →
'ABC'

CHARSET(UNHEX('414243')) →
'binary'

UPPER(str)

返回把字符串str里的字符全都转换为大写字母后得到的一个字符串；如果str为NULL，则返回NULL。

```
UPPER('New York, NY') →  
'NEW YORK, NY'
```

```
UPPER(NULL) →  
NULL
```

关于二进制串的大小写转换问题请参考前面对LOWER()函数的描述。

WEIGHT_STRING(str [AS type(n)] [LEVEL levels] [flags])

以二进制串的形式返回str参数的权重字符串。权重字符串是MySQL在进行字符串比较和排序操作的时候使用的一种内部表示形式。两个权重字符串相同的字符串在比较操作中将被认为是相等的，或者更准确地说，它们有着与它们的权重字符串相同的相对顺序。此函数的AS选项会把str参数转换为某种给定的类型和长度，LEVEL选项会指定返回哪一个排序规则级别的权重字符串。flags值在现阶段还没有任何具体的实现。

如果str 参数是一个二进制串，其权重字符串将与str 相同。如果str 参数是一个非二进制串，它就会有一种排序规则，结果字符串也将包含着该种排序规则的权重。如果str为NULL，则返回结果为NULL。下面几个例子为了把权重字符串显示为可打印格式而使用了HEX()函数：

```
HEX(WEIGHT_STRING(BINARY 'Hello')) →  
'48656C6C6F'
```

```
HEX(WEIGHT_STRING('Hello')) →  
'48454C4C4F'
```

```
HEX(WEIGHT_STRING(_utf8'Hello')) →  
'00480045004C004C004F'
```

可以用AS子句把参数str强制转换为某种给定的类型和长度。如果想把str强制转换为一个n个字符长的CHAR字符串，可以使用AS CHAR(n)。如有必要，可在该字符串的尾部用空格补齐。AS BINARY(n) 将把str强制转换为一个n个字节长的二进制串，必要时用0x00补齐。n必须大于或等于1。如果str的长度大于n，那么它将被截短而不是被补齐。

一种排序规则可能会有多个级别。默认情况下，此函数的返回结果将包含所有级别的权重。如果只想返回某几个特定级别的权重，就需要用到LEVEL子句。levels值可以是一个整数、由几个以逗号分隔的整数所构成的列表，或者由两个以短划线分隔的整数所设定的区间。列表中各个级别必须按递增的顺序给出。区间里的第二个级别如果小于第一个级别将被视为等于第一个级别。排序规则的级别从 1 开始，并有一个最大值，超出这个范围的levels值将按相关规则被换算成这个范围内的某个值。

还可以在列表中级别值的后面给出一个修饰符：ASC，返回未做任何修改的权重（默认情况）；DESC，返回逐位反转的权重；REVERSE，返回把 str 前后颠倒而得到的那个字符串值的权重。

HEX(WEIGHT_STRING('abc' LEVEL 1 ASC)) → '414243'

HEX(WEIGHT_STRING('abc' LEVEL 1 DESC)) → 'BEBDBC'

HEX(WEIGHT_STRING('abc' LEVEL 1 REVERSE)) → '434241'

WEIGHT_STRING() 是在MySQL 5.6.0里引入的。

C.2.5 日期/时间函数

日期/时间函数的参数有多种类型。一般情况下，接受 DATE 参数的函数也接受 DATETIME 或TIMESTAMP类型的参数，并且会忽略其中的时间部分。接受TIME参数的函数通常也接受DATETIME或TIMESTAMP类型的参数，并且会忽略其中的日期部分。

本节中的大部分函数都能把数字形式的输入参数解释为时态值。

MONTH('1906-04-18') → 4

MONTH(19060418) → 4

类似地，对于那些返回时态值的函数，MySQL会根据上下文将返回值转换为字符串或数值。

CURDATE()
→ '2012-08-26'

CONCAT('Today is ', CURDATE()) → 'Today is
2012-08-26'

`CURDATE() + 0` →
20120826

在把时间值或日期/时间值转换为数值的时候，转换结果将有一个“.000000”形式的微秒部分。如果想去掉这个部分，把转换结果强制转换为一个整数即可。

`NOW() + 0`
→ 20120806163317.000000

`CAST(NOW() AS UNSIGNED)` →
20120806163317

`CURTIME() + 0` →
163317.000000

`CAST(CURTIME() AS UNSIGNED)` →
163317

有几个用来提取日期部分的函数，在遇到“不完整的”日期时会返回0。例如，`MONTH()`和`DAYOFMONTH()`函数在遇到参数'2013-00-00'的时候都将返回0。用在`DATE_FORMAT()`函数里的日期部分的格式说明符也是如此。

如果提供给日期/时间函数的日期值或时间值不合法，就不能指望能够得到合理的结果。一定要先验证参数。

`ADDDATE(date, INTERVAL expr interval)`

`ADDDATE(date, expr)`

对于第一种语法，`ADDDATE()`函数通过 `date` 参数读入一个日期值或一个日期/时间值，给它加上一段时间间隔，最后返回计算的结果。它等价于`ADD_DATE()`函数。

`ADDDATE('2004-12-01', INTERVAL 1 YEAR)` → '2005-12-01'

对于第二种语法，`ADDDATE()`函数通过 `date` 参数读入一个日期值或一个日期/时间值，给它加上一个天数，最后返回计算的结果。

ADDDATE('2004-12-01',365) → '2005-12-01'

第二种语法可以改写为第一种语法，如下所示：

ADDDATE(date,expr) = ADDDATE(date,INTERVAL expr DAY)

ADDTIME (expr1, expr2)

把两个表达式相加并返回其结果。expr1 应该是一个时间值或一个日期/时间值，expr2 应该是一个时间值。

ADDTIME('06:30:00.5','12:30:00.4') →
'19:00:00.900000'

ADDTIME('2004-01-01 00:00:00','12:30:00') → '2004-01-01
12:30:00'

CONVERT_TZ (date, from_zone, to_zone)

给定日期值或日期/时间值 date，CONVERT_TZ() 函数把它视为 from_zone 时区里的一个值，把它转化为 to_zone 时区里的一个值，最后返回转换结果。只要这些参数里有一个是非法的，就返回 NULL。在指定时区值时可以参考 12.6.1 节的描述。为了让此函数正确工作，转换结果必须落在 TIMESTAMP 数据类型的表示范围内。

CONVERT_TZ('2019-02-11 00:00:00','US/Central','US/Eastern')
→ '2019-02-11 01:00:00'

CONVERT_TZ('2019-02-11','+00:00','-03:00') → '2019-02-10
21:00:00'

CURDATE ()

返回会话时区的当前日期，它是一个 'CCYY-MM-DD' 格式的DATE值。

CURDATE()
→ '2012-08-26'

CURRENT_DATE ()

它等价于CURDATE()函数。那对括号是可选的。

CURRENT_TIME([fsp])

它等价于CURTIME()函数。那对括号是可选的，除非指定了fsp参数。

CURRENT_TIMESTAMP([fsp])

它等价于NOW()函数。那对括号是可选的，除非指定了fsp参数。

CURTIME([fsp])

返回会话时区的当前时间，它是一个'hh:mm:ss'格式的TIME值。自MySQL 5.6.4开始，可以给出可选参数fsp来指定返回值应该拥有的小数秒精度（0到6位）。在MySQL 5.6.4之前，所有的参数都会被忽略。

CURTIME() →
'15:01:02'

CURTIME(2) →
'15:01:02.20'

DATE(expr)

返回expr的日期部分，应该是一个日期表达式或日期/时间表达式。

DATE('2008-03-12') → '2008-
03-12'

DATE('2008-03-12 16:15:00') → '2008-03-12'

DATE_ADD(date, INTERVAL expr interval)

返回日期值或日期/时间值date加上一段时间间隔后得到的结果。其中，expr给出了将与date值进行加减（若expr为负，则减去；否则就加上）的数值，interval表明了如何解释这段时间间隔。如果date是一个DATE值，则返回值也将是一个DATE值，有关参数中的时间部分不参加计算。否则，返回值将是一个DATETIME值。如果date不是一个合法的日期值，结果将为NULL。

DATE_ADD('2009-12-01',INTERVAL 1 YEAR) → '2010-12-
01'

DATE_ADD('2009-12-01',INTERVAL 60 DAY) → '2010-01-30'

DATE_ADD('2009-12-01',INTERVAL -3 MONTH) → '2009-09-01'

DATE_ADD('2009-12-01 08:30:00',INTERVAL 12 HOUR) → '2009-12-01 20:30:00'

下表格列出了interval的各种可取值、它们的含义，以及为每种间隔类型指定具体值时使用的格式。关键字INTERVAL和说明符interval不区分大小写。

间隔类型	含义	值格式
MICROSECOND	微秒	uuuuuu
SECOND	秒	ss
SECOND_MICROSECOND	秒和微秒	'ss.uuuuuu'
MINUTE	分	mm
MINUTE_SECOND	分和秒	'mm:ss'
MINUTE_MICROSECOND	分、秒和微秒	'mm:ss.uuuuuu'
HOUR	小时	hh
HOUR_MINUTE	小时和分	'hh:mm'
HOUR_SECOND	小时、分和秒	'hh:mm:ss'
HOUR_MICROSECOND	小时、分、秒和微秒	'hh:mm:ss.uuuuuu'
DAY	日	DD
DAY_HOUR	日和小时	'DD hh'
DAY_MINUTE	日、小时和分	'DD hh:mm'
DAY_SECOND	日、小时、分和秒	'DD hh:mm:ss'
DAY_MICROSECOND	日、小时、分、秒和微秒	'DD hh:mm:ss.uuuuuu'
WEEK	周	WW
MONTH	月	MM
QUARTER	季度	QQ
YEAR	年	YY
YEAR_MONTH	年和月	'YY-MM'

添加到日期的表达式expr既可以指定为数值，也可以指定为字符串；但如果它里面包含有非数字的字符，那就必须指定为一个字符串。表达式expr中的分隔符可以是任何一种标点符号。

DATE_ADD('2005-12-01',INTERVAL '2:3' YEAR_MONTH) → '2008-03-01'

`DATE_ADD('2005-12-01',INTERVAL '2-3' YEAR_MONTH) → '2008-03-01'`

表达式 `expr` 的各组成部分将按 `interval` 限定的输入格式从右向左进行匹配。例如，`HOUR_SECOND` 限定的输入格式是 `'hh:mm:ss'`。如果表达式 `expr` 的值是 `'15:21'`，那它将被解释为 `'00:15:21'` 而不是 `'15:21:00'`。

`DATE_ADD('2003-12-01 12:00:00',INTERVAL '15:21' HOUR_SECOND)`

`→ '2003-12-01 12:15:21'`

如果 `interval` 是 `YEAR`、`MONTH` 或 `YEAR_MONTH`，而计算结果中的日部分大于结果月的天数，那么 `DATE_ADD()` 函数将把日自动调整为当月的最大日期值。

`DATE_ADD('2003-12-31',INTERVAL 2 MONTH) → '2004-02-29'`

日期加法还支持使用下面这种语法格式：

`'2003-12-31' + INTERVAL 2 MONTH → '2004-02-29'`

`INTERVAL 2 MONTH + '2003-12-31' → '2004-02-29'`

`DATE_FORMAT(date, format)`

按格式字符串 `format` 对日期或日期/时间值 `date` 进行格式化，返回一个结果字符串。此函数能把 MySQL 所支持的日期值或日期/时间值重新格式化为其他所需的格式。

`DATE_FORMAT('2014-12-01','%M %e, %Y') → 'December 1, 2014'`

`DATE_FORMAT('2014-12-01','The %D of %M') → 'The 1st of December'`

下表格列出了允许在格式化字符串里使用的说明符。月、日说明符的数值表示范围从零开始，因为不完整的日期需要用零来表示其月或日，如'2004-00-13'或'1998-12-00'。系统变量`lc_time_names`决定了用于生成月名和日名的说明符的语言。

每种格式代码的前面必须要有“%”字符。如果那些包含在格式化字符串里的字符未出现在这张表里，那么它们会被原封不动地复制到输出结果里。

说明符	含义
%f	六位数字形式的微秒 (000000, 000001, ...)
%S 或 %s	两位数字形式的秒 (00, 01, ..., 59)
%i	两位数字形式的分 (00, 01, ..., 59)
%H	两位数字形式的小时, 24小时制 (00, 01, ..., 23)
%h 或 %I	两位数字形式的小时, 12小时制 (00, 01, ..., 12)
%k	数值形式的小时, 24小时制 (0, 1, ..., 23)
%l	数值形式的小时, 12小时制 (0, 1, ..., 12)
%T	24小时制的时间 (hh:mm:ss)
%r	12小时制的时间 (hh:mm:ss AM或者hh:mm:ss PM)
%p	AM或者PM
%W	星期名 (Sunday, Monday, ..., Saturday)
%a	简写形式的星期名 (Sun, Mon, ..., Sat)
%d	两位数字形式的日 (00, 01, ..., 31)
%e	数值形式的日 (0, 1, 2, ..., 31)
%D	带英语后缀的日 (0th, 1st, 2nd, 3rd, ...)
%w	数值形式的星期 (0=Sunday, 1=Monday, ..., 6=Saturday)
%j	三位数字形式的天数 (001, 002, ..., 366)
%U	周 (00, ..., 53), 以Sunday (星期日) 作为每周的第一天
%u	周 (00, ..., 53), 以Monday (星期一) 作为每周的第一天
%V	周 (01, ..., 53), 以Sunday (星期日) 作为每周的第一天
%v	周 (01, ..., 53), 以Monday (星期一) 作为每周的第一天
%M	月名 (January, February, ..., December)
%b	简写形式的月名 (Jan, Feb, ..., Dec)

说明符	含义
<code>%m</code>	两位数字形式的月 (00, 01, 02, ..., 12)
<code>%c</code>	数值形式的月 (0, 1, 2, ..., 12)
<code>%Y</code>	四位数字形式的年
<code>%y</code>	两位数字形式的年
<code>%X</code>	以Sunday (星期日) 作为每周第一天的年, 以四位数字表示
<code>%x</code>	以Monday (星期一) 作为每周第一天的年, 以四位数字表示
<code>%%</code>	%字符本身

如果对DATE值使用时间说明符, 则该值的时间部分将被当作'00:00:00'。

`DATE_FORMAT('2014-12-01','%i')` → '00'

`DATE_SUB(date, INTERVAL expr interval)`

`DATE_SUB()` 执行日期计算的方式与`DATE_ADD()` 函数相同, 只不过前者是从日期或日期/时间值date上减去expr。更多信息请参考`DATE_ADD()`。

`DATE_SUB('2009-12-01',INTERVAL 1 MONTH)` → '2009-11-01'

`DATE_SUB('2009-12-01',INTERVAL '13-2' YEAR_MONTH)` → '1996-10-01'

`DATE_SUB('2009-12-01 04:53:12',INTERVAL '13-2' MINUTE_SECOND)`
→ '2009-12-01 04:40:10'

日期减法还支持使用下面这种语法格式:

`'2009-12-01' - INTERVAL 1 MONTH` → '2009-11-01'

使用这种语法时, 必须把INTERVAL子句放在减法操作符的右侧, 因为你不能用一段间隔减去一个日期。

DATEDIFF (expr1, expr2)

计算并返回两个表达式相距的天数，两个表达式都应该是日期值或日期/时间值。如果第一个参数给出的时间晚于第二个，此函数的返回值将是一个正数。两个参数里的时间部分都将被忽略。

DATEDIFF('1987-01-01','1987-01-08') → -7

DATEDIFF('1987-01-08','1987-01-01') → 7

DATEDIFF('1987-01-01 12:00:00','1987-01-08') → -7

DATEDIFF('1987-01-08','1987-01-01 12:00:00') → 7

DAY (date)

它等价于DAYMONTH() 函数。

DAYNAME (date)

返回包含日期值 date的星期名的字符串；如果名字无法确定，则返回NULL。系统变量lc_time_names决定了名字语言。

DAYNAME('2004-12-01') →
'Wednesday'

DAYNAME('1900-12-01') →
'Saturday'

DAYNAME('1900-12-00') →
NULL

DAYOFMONTH (date)

返回表示日期值date是一个月中第几日的数字值，范围为0~31（为0时，表示没有“日”部分）。

DAYOFMONTH('2002-12-01') → 1

DAYOFMONTH('2002-12-25') → 25

DAYOFMONTH('2002-12-00') → 0

DAYOFWEEK(date)

返回表示日期值date是一周中第几天的数字值。按照ODBC标准的规定，这个数字值的范围是1~7，其中1代表Sunday（星期日）、7代表Saturday（星期六）。请参考WEEKDAY()函数。

DAYOFWEEK('2004-12-05') → 1

DAYNAME('2004-12-05') →
'Sunday'

DAYOFWEEK('2004-12-18') → 7

DAYNAME('2004-12-18') →
'Saturday'

DAYOFYEAR(date)

返回表示日期值date是一年中第几天的数字值，取值范围是1~366。

DAYOFYEAR('2002-12-01') → 335

DAYOFYEAR('2004-12-31') → 366

EXTRACT(interval FROM datetime)

从日期/时间值datetime里根据interval（可以是DATE_ADD()允许的任何
一个）返回一个值。

EXTRACT(YEAR FROM '2002-12-01 13:42:19') → 2002

EXTRACT(MONTH FROM '2002-12-01 13:42:19') → 12

EXTRACT(DAY FROM '2002-12-01 13:42:19') → 1

EXTRACT(HOUR_MINUTE FROM '2002-12-01 13:42:19') → 1342

EXTRACT(SECOND FROM '2002-12-01 13:42:19') → 19

EXTRACT()函数还可用于那些已部分缺失的日期。

EXTRACT(YEAR FROM '2004-00-12') → 2004

EXTRACT(MONTH FROM '2004-00-12') → 0

EXTRACT(DAY FROM '2004-00-12') → 12

FROM_DAYS(n)

把从公元0年1月1日开始计算的天数n（通常是调用TO_DAYS()函数后的返回值）转换为相应的日期，并返回这个日期值。

TO_DAYS('2009-12-01') →
734107

FROM_DAYS(734107 + 3) →
'2009-12-04'

FROM_DAYS() 函数只适用于公历日期（起始于公元1582年）。

FROM_UNIXTIME(unix_timestamp)

FROM_UNIXTIME(unix_timestamp, format)

给定一个Unix时间戳值unix_timestamp，比如它是由UNIX_TIMESTAMP()函数返回的，函数将以'CCYY-MM-DD hh:mm:ss'格式的DATETIME值返回会话时区的日期/时间。如果给定format参数，则返回值会是一个如同使用DATE_FORMAT()函数格式化的字符串。

UNIX_TIMESTAMP() →
1328381427

FROM_UNIXTIME(1328381427) →
'2012-02-04 12:50:27'

FROM_UNIXTIME(1328381427, '%Y') → '2012'

GET_FORMAT(val_type, format_type)

根据format_type 参数的要求把val_type 参数转换为一个可以用在DATE_FORMAT()、TIME_FORMAT()和STR_TO_DATE()函数里的格式化字符串。val_type参数用来给出一种日期类型，它可以是DATE、TIME、DATETIME 或TIMESTAMP。format_type 参数表明应该返回哪种风格的格式化字符串，它可以是'EUR'（欧洲）、'INTERNAL'（MySQL的内部表示形式）、'ISO'（是ISO 9075，而非ISO 8601）、'JIS'（日本工业标准）或'USA'（美国）。

GET_FORMAT() 函数针对val_type和format_type的每种组合所返回的格式化字符串，如下表所示。

val_type	format_type	格式化字符串
DATE	'EUR'	'%d.%m.%Y'
DATE	'INTERNAL'	'%Y%md'
DATE	'ISO'	'%Y-%m-%d'
DATE	'JIS'	'%Y-%m-%d'
DATE	'USA'	'%m.%d.%Y'
TIME	'EUR'	'%H.%i.%s'
TIME	'INTERNAL'	'%H%i%s'
TIME	'ISO'	'%H:%i:%s'
TIME	'JIS'	'%H:%i:%s'
TIME	'USA'	'%h:%i:%s %p'
DATETIME	'EUR'	'%Y-%m-%d %H.%i.%s'
DATETIME	'INTERNAL'	'%Y%md%H%i%s'
DATETIME	'ISO'	'%Y-%m-%d %H:%i:%s'
DATETIME	'JIS'	'%Y-%m-%d %H:%i:%s'
DATETIME	'USA'	'%Y-%m-%d %H.%i.%s'

请注意，适用于 DATETIME 类型的格式化字符串'EUR'和'USA'的日期部分，和适用于DATE类型的格式化字符串'EUR'和'USA'的日期部分是不一样的。

hour(time)

返回时间值time里的小时数，其取值范围是0~23。

last_day(date)

hour('12:31:58') → 12

hour(123158) →

返回由参数date所给定的月份里的最后一天的日期。参数date应该是一个日期值或一个日期/时间值。

LAST_DAY('2003-07-01') → '2003-07-31'

LAST_DAY('2003-07-01 12:30:00') → '2003-07-31'

LOCALTIME([fsp])

LOCALTIMESTAMP([fsp])

这两个函数等价于NOW()函数。那对括号是可选的，除非指定了fsp参数。

MAKEDATE(year, day_of_year)

根据给定的年份和天数返回一个日期值；如果day_of_year的值小于1，则返回NULL。

MAKEDATE(2010,365) → '2010-12-31'

MAKEDATE(2010,367) → '2011-01-02'

MAKEDATE(2010,0) → NULL

MAKETIME(hour, minute, second)

用给定的小时、分和秒构造并返回一个时间值；如果有参数超出取值范围，则返回 NULL。分和秒的值必须在0~59。小时值可以超出此范围。如果小时值为负，则返回结果也为负。

MAKETIME(0,0,0) → '00:00:00'

MAKETIME(12,59,59) → '12:59:59'

MAKETIME(12,59,60) →
NULL

MAKETIME(-12,59,59) →
'-12:59:59'

MICROSECOND(expr)

返回给定时间值或日期/时间值里的微秒部分。返回值的取值范围是0～999999。

MICROSECOND('00:00:00.000001'); → 1

MICROSECOND('2004-06-30: 23:59:59.5'); → 500000

MINUTE(time)

返回时间值time中的分钟值，范围是0～59。

MINUTE('12:31:58') → 31

MINUTE(123158) → 31

MONTH(date)

返回日期值date中的月份值，范围是0～12（0表示日期部分没有月份表示）。

MONTH('2002-12-01') → 12

MONTH(20021201) →
12

MONTH('2002-00-01') → 0

MONTHNAME(date)

以字符串形式返回日期值 date 中的月份名字；对于没有月份部分的日期，则返回 NULL。系统变量lc_time_names决定了月份名字语言。

MONTHNAME('2002-12-01') →
'December'

MONTHNAME(20021201) →
'December'

MONTHNAME('2002-00-01') →
NULL

NOW([fsp])

返回会话时区的当前日期和时间，它是一个'CCYY-MM-DD hh:mm:ss'格式的DATETIME值。自MySQL 5.6.4开始，可以给出fsp参数，以指定返回值应该拥有的小数秒精度（0~6位）。在MySQL 5.6.4以前，所有的参数都会被忽略。

NOW()
→ '2012-02-04 12:51:22'

NOW()函数返回的是它所在的语句刚开始执行的那一时刻的日期/时间值，不管那条语句会执行多长时间。如果是包含在某个存储函数或触发器里的NOW()函数，它返回的是该函数或触发器刚开始执行的那一时刻的日期/时间值。建议把这种行为与SYSDATE()的行为做个对比。

PERIOD_ADD(period, n)

返回时间段值 period 加上 n个月后得到的结果。返回值的格式是CCYYMM。输入参数period的格式可以是CCYYMM或YYMM（它们都不是日期值）。

PERIOD_ADD(201002,12) →
201102

PERIOD_ADD(0802,-3) →
200711

PERIOD_DIFF(period1, period2)

返回时间段参数值相减得到的差，即它们之间相隔的月份数。这些参数的格式可以是CCYYMM或YYMM（它们都不是日期值）。

PERIOD_DIFF(200302,200202) → 12

PERIOD_DIFF(200711,0802) → -3

QUARTER (date)

返回日期值date确定的日子所在的季节值，范围是1～4。

QUARTER('2008-12-01') → 4

QUARTER('2009-01-01') → 1

SECOND (time)

返回时间值time中的秒值，范围是0～59。

SECOND('12:31:58') → 58

SECOND(123158) → 58

SEC_TO_TIME (second)

给定秒数second，以'hh:mm:ss'格式的TIME值返回相应的时间值。

SEC_TO_TIME(29834) →
'08:17:14'

STR_TO_DATE (str, format_str)

使用格式化参数format_str对字符串参数str进行解释，并根据format_str参数里的格式化说明符返回一个TIME、DATE或DATETIME值。你可以利用此函数来解释非ISO格式的时态值。STR_TO_DATE()函数是DATE_FORMAT()函数的逆向操作，DATE_FORMAT()函数描述里列出的格式说明符也都可以用在STR_TO_DATE()函数里。如果str值非法或者无法用给定的格式字符串解释，则返回结果为NULL。

STR_TO_DATE('3/16/1960','%m/%d/%Y') → '1960-03-16'

STR_TO_DATE('12.20.32','%H:%i:%s') → '12:20:32'

STR_TO_DATE('3/16/1960 12:20:32','%m/%d/%Y %H:%i:%s')
→ '1960-03-16 12:20:32'

STR_TO_DATE('3/16/1960','%m-%d-%Y') → NULL

SUBDATE(date, INTERVAL expr interval)

SUBDATE(date, expr)

对于第一种语法格式，SUBDATE() 函数通过 date 参数读入一个日期值或日期/时间值，用它减去一段时间间隔，最后返回计算结果。此函数等同于 DATE_SUB() 函数。

SUBDATE('2009-12-01',INTERVAL 1 MONTH) → '2009-11-01'

对于第二种语法格式，SUBDATE() 函数通过 date 参数读入一个日期值或日期/时间值，用它减去一个天数，最后返回计算结果。这与ADDDATE() 函数的相应语法很相似。

SUBDATE('2009-12-01',30) → '2009-11-01'

SUBTIME(expr1, expr2)

用第一个表达式减去第二个表达式并返回其结果。expr1 应该是一个时间值或日期/时间值，expr2应该是一个时间值。

SUBTIME('06:30:00.5','12:30:00.4') → '-05:59:59.900000',

SUBTIME('2009-01-01 00:00:00','12:30:00') → '2008-12-31 11:30:00'

SYSDATE()

返回会话时区的当前日期和时间，它是一个'CCYY-MM-DD hh:mm:ss'格式的DATETIME值。此函数类似于 NOW() 函数，不同之处在于：SYSDATE() 函数返回的是它执行时的日期和时间值；而NOW() 函数返回的是它所在的语句开始执行时的时间。（更多细节请参考NOW() 函数的描述。）如果想让 SYSDATE() 函数的行为和NOW() 函数的一样，则需要在启动服务器时带上--sysdate-is-now选项。

TIME(expr)

返回expr参数中的时间部分，expr应该是一个时间值或一个日期/时间值。

TIME('16:15:00') →
'16:15:00'

TIME('2005-03-12 16:15:00') → '16:15:00'

TIME_FORMAT(time, format)

根据format参数给定的格式化字符串格式对time参数的值进行格式化，并返回结果字符串。此函数也接受 DATETIME 或 TIMESTAMP 类型的参数。这里使用的格式化字符串和DATE_FORMAT()函数里使用的相似，只是仅允许使用与时间有关的说明符。其他说明符将导致返回NULL值或0。

TIME_FORMAT('12:31:58','%H %i') → '12 31'

TIME_FORMAT(123158,'%H %i') → '12 31'

TIME_TO_SEC(time)

把参数time给出的时间值换算为相应的秒数，并返回该秒数。此函数的返回值可以传递到SEC_TO_TIME()函数，重新转换回一个时间值。

TIME_TO_SEC('08:17:14') → 29834

SEC_TO_TIME(29834) →
'08:17:14'

如果time参数给出的是一个DATETIME或TIMESTAMP值，那么TIME_TO_SEC()函数将忽略日期部分。

TIME_TO_SEC('2012-03-26 08:17:14') → 29834

TIMEDIFF(expr1, expr2)

计算并返回表达式 expr1和expr2的时间差，这两个表达式分别表示开始时间和结束时间。它们应该都是时间值，或者都是日期/时间值，时间值和日期/时间值不能混用。

TIMEDIFF('00:00:00','09:30:45') → '-09:30:45'

TIMEDIFF('09:30:45','00:00:00') → '09:30:45'

TIMESTAMP(expr1[, expr2])

如果只有一个参数，那么函数将把 `expr1` 参数给定的日期值或日期/时间值转换为一个 `DATETIME` 值。如果有两个参数，那么函数将把参数 `expr2` 给出的时间值加到参数 `expr1` 上，并返回计算结果 `TIMESTAMP` 值。

```
TIMESTAMP('1985-12-14');                                → '1985-12-14 00:00:00'
```

```
TIMESTAMP('1985-12-14 09:00:00');                       → '1985-12-14 09:00:00'
```

```
TIMESTAMP('1985-12-14','18:00:00');                     → '1985-12-14 18:00:00'
```

```
TIMESTAMP('1985-12-14 09:00:00','18:00:00');           → '1985-12-15 03:00:00'
```

```
TIMESTAMP('1985-12-14 09:00:00','-18:00:00');           → '1985-12-13 15:00:00'
```

`TIMESTAMPADD(interval, expr1, expr2)`

把 `expr1` 解释为一个以 `interval` 参数值为单位的整数，把该整数与 `expr2` 参数给出的日期值或日期/时间值相加，最后返回计算结果。允许使用的 `interval` 参数值是 `MICROSECOND`、`SECOND`、`MINUTE`、`hour`、`DAY`、`WEEK`、`MONTH`、`QUARTER` 和 `YEAR`。这些值在使用时都可以加上一个 `SQL_TSI_` 前缀。

```
TIMESTAMPADD(DAY,12,'1995-07-01')                       → '1995-07-13'
```

```
TIMESTAMPADD(MONTH,12,'1995-07-01')                     → '1996-07-01'
```

```
TIMESTAMPADD(SQL_TSI_MONTH,12,'1995-07-01')             → '1996-07-01'
```

`TIMESTAMPDIFF(interval, expr1, expr2)`

计算 `expr1` 和 `expr2` 参数所给出的日期值或日期/时间值之间的差，然后以 `interval` 参数值为单位返回计算结果。这里允许使用的 `interval` 参数值和 `TIMESTAMPADD()` 函数描述里给出的一样。

```
TIMESTAMPDIFF(DAY,'1995-07-01','1995-08-01')           → 31
```

`TIMESTAMPDIFF(MONTH,'1995-07-01','1995-08-01')` → 1

`TO_DAYS(date)`

把date参数所给出的日期值转换为从公元0年开始计算的天数，并返回结果数值。此函数的返回值可以传递到FROM_DAYS()，重新转换回一个日期值。

`TO_DAYS('2010-12-01')` →
734472

`FROM_DAYS(734472)` →
'2010-12-01'

如果给出的是一个DATETIME或TIMESTAMP值，TO_DAYS()函数将忽略时间部分。

`TO_DAYS('2010-12-01 12:14:37')` → 734472

TO_DAYS()函数只适用于公历日期（起始于公元1582年）。

`TO_SECONDS(expr)`

返回给定日期或日期/时间表达式所对应的自 0 年以来的秒数；如果参数无效，则返回NULL。

`TO_SECONDS('1790-07-17')` →
56504044800

`UNIX_TIMESTAMP()`

`UNIX_TIMESTAMP(date)`

如果不带任何参数，此函数将返回从 UTC 基准时间'1970-01-01 00:00:00'开始计算的秒数。如果通过date参数给出一个日期值，此函数将返回基准时间与date参数日期之间的秒数。date参数可以用多种方法给出：以DATE、DATETIME或TIMESTAMP形式给出，或者以CCYYMMDD或YYMMDD格式的数值形式给出。服务器将把date参数值解释为一个会话时区中的值并转换为一个UTC格式的值，但来自TIMESTAMP列的值例外（因为存放在TIMESTAMP列里的值已经是UTC时间的格式了）。

`UNIX_TIMESTAMP()` →
1328381580

UNIX_TIMESTAMP('2007-12-01') →
1196488800

UNIX_TIMESTAMP(20071201) →
1196488800

UTC_DATE()

返回当前的UTC日期，它是一个'CCYY-MM-DD'格式的DATE值。那对括号是可选的。

UTC_DATE() →
'2012-08-26'

UTC_TIME([fsp])

返回当前的UTC时间，它是一个'hh:mm:ss'格式的TIME值。自MySQL 5.6.4开始，可以给出fsp参数，以指定返回值应该拥有的小数秒精度（0到6位）。在MySQL 5.6.4之前，所有的参数都会被忽略。那对括号是可选的，除非指定了fsp参数。

UTC_TIME() →
'20:00:18'

UTC_TIME(3) →
'20:00:18.465'

UTC_TIMESTAMP([fsp])

返回当前的UTC日期/时间，它是一个'CCYY-MM-DD hh:mm:ss'格式的DATETIME值。自MySQL 5.6.4开始，可以给出fsp参数，以指定返回值应该拥有的小数秒精度（0~6位）。在MySQL 5.6.4之前，所有的参数都会被忽略。那对括号是可选的，除非指定了fsp参数。

UTC_TIMESTAMP() →
'2012-08-26 20:48:23'

WEEK(date[, mode])

如果只向此函数传递一个date参数，它将返回一个0到53之间的整数值，表明由date参数所给定的日期落在该年的第几周里。假定每周的第一天是星

期日。如果向此函数传递两个参数，它将返回一个有着同样含义的整数值，但要根据mode参数的值来决定以星期几为每周的第一天，返回值的取值范围是0~53还是1~53。下表列出了所有允许使用的mode 参数值及其含义。

模式	起始日	返回值范围	含义
0	星期日	0~53	第一周是第一个包含星期日的周
1	星期一	0~53	第一周是第一个多于3天的周
2	星期日	1~53	第一周是第一个包含星期日的周
3	星期一	1~53	第一周是第一个多于3天的周
4	星期日	0~53	第一周是第一个多于3天的周
5	星期一	0~53	第一周是第一个包含星期一的周
6	星期日	1~53	第一周是第一个多于3天的周
7	星期一	1~53	第一周是第一个包含星期一的周

如果mode参数值缺失，则此函数会使用系统变量default_week_format。

WEEK('2003-12-08') → 49

WEEK('2003-12-08',0) → 49

WEEK('2003-12-08',1) → 50

如果WEEK() 函数的返回值是0，则表明date参数所给出的日期早于该年的第一个星期起始日（星期日或星期一，视mode参数值而定）。

WEEK('2005-01-01') → 0

DAYNAME('2005-01-01') →
'Saturday'

WEEK('2006-01-01',1) → 0

DAYNAME('2006-01-01') →
'Sunday'

WEEKDAY (date)

返回一个整数值以表明由 date 参数所给定的日期是星期几；如果无法确定，则返回NULL。返回值的取值范围是0（对应于星期一）到6（对应于星期日）。另请参考DAYOFWEEK() 函数。

WEEKDAY('2002-12-08') → 6

DAYNAME('2002-12-08') →
'Sunday'

WEEKDAY('2002-12-16') → 0

DAYNAME('2002-12-16') →
'Monday'

WEEKDAY('2002-12-00') →
NULL

WEEKOFYEAR (date)

此函数等同于WEEK (date, 3)。

YEAR (date)

返回日期值date所包含的年份。

YEAR('1974-12-01') → 1974

YEAR(19741201) →
1974

YEARWEEK (date [, mode])

返回一个CCYYMM格式的整数值以表明由date参数所给定的日期落在该年的第几周里。如果还给出了mode参数，其含义与WEEK() 函数里的相同。

YEARWEEK('2006-01-01') →
200601

YEARWEEK('2006-01-01',0) → 200601

YEARWEEK('2006-01-01',1) → 200552

对于一年里的第一周或最后一周，结果里的年可能与参数里的年不相同。

WEEK('2008-01-01') → 0

YEARWEEK('2008-01-01') →
200752

C. 2. 6 汇总函数

汇总（或“聚集”）函数的返回值是根据一组值计算出来的一个结果。这个结果是根据查询结果中的非NULL值统计出来的，只有COUNT(*)函数例外，它将统计所有的行。汇总函数既可以对整个查询结果集进行统计，也可以对查询结果按某种规则进行分组（比如查询语句里使用了 GROUP BY子句的情况）后得到的各个子集进行统计。具体请参考1. 4. 9. 9节。

本节的示例要用到一个下面这样的mytbl表：它有一个整数列mycol，各行在这个列的值依次是1、3、5、5、7、9、9和NULL。

```
mysql> SELECT mycol FROM mytbl;
```

```
-----  
mycol  
-----  
  
1  
  
3  
  
5  
  
5  
  
7  
  
9  
  
9  
  
NULL  
-----
```

AVG([DISTINCT] expr)

返回表达式 `expr` 计算结果的平均值，涉及查询结果集里的全体非 `NULL` 值；如果没有非`NULL`值，则返回`NULL`。

```
SELECT AVG(mycol) FROM mytbl →  
5.5714
```

```
SELECT AVG(mycol)*2 FROM mytbl → 11.1429
```

```
SELECT AVG(mycol*2) FROM mytbl → 11.1429
```

`DISTINCT`子句可以让`AVG()` 返回各个`expr`值的平均值。

`BIT_AND(expr)`

返回表达式`expr`计算结果的按位与操作结果，涉及查询结果集里的全体非`NULL`值；如果没有非`NULL`值，则返回`~0`。

```
SELECT BIT_AND(mycol) FROM mytbl → 1
```

`BIT_OR(expr)`

返回表达式`expr`计算结果的按位或操作结果，涉及查询结果集里的全体非`NULL`值；如果没有非`NULL`值，则返回`0`。

```
SELECT BIT_OR(mycol) FROM mytbl → 15
```

`BIT_XOR(expr)`

返回表达式`expr`计算结果的按位异或操作结果，涉及查询结果集里的全体非`NULL`值；如果没有非`NULL`值，则返回`0`。

```
SELECT BIT_XOR(mycol) FROM mytbl → 5
```

`COUNT(expr)`

`COUNT(*)`

`COUNT(DISTINCT expr1, expr2, ...)`

有一个表达式参数时，返回查询结果集里的非`NULL`值的个数；如果没有非`NULL`值，则返回`0`。`COUNT(*)`将返回查询结果集里全体行（不管它们是不是`NULL`值）的个数。

```
SELECT COUNT(mycol) FROM mytbl → 7
```

```
SELECT COUNT(*) FROM mytbl → 8
```

对于MyISAM表，不带WHERE子句的COUNT(*)将被优化成直接返回FROM子句所指定的表里的总行数。如果FROM子句指定了多个表，COUNT(*)将返回各表的总行数的乘积。

```
SELECT COUNT(*) FROM mytbl AS m1 INNER JOIN mytbl AS m2  
→ 64
```

使用DISTINCT，COUNT()可以统计各种不同非NULL值的个数。

```
SELECT COUNT(DISTINCT mycol) FROM mytbl → 5
```

```
SELECT COUNT(DISTINCT MOD(mycol,3)) FROM mytbl → 3
```

如果给出了多个表达式，则COUNT(DISTINCT)的返回值是全体表达式非NULL 值计算结果的各不相同组合的总数。

```
GROUP_CONCAT([DISTINCT] val_list [ORDER BY ...] [SEPARATOR  
str])
```

此函数把字符串列表 val_list 中的所有非 NULL 值连接在一起并返回其结果。如果val_list列表中没有非NULL值，则返回NULL。其中，DISTINCT用于剔除重复出现的字符串，ORDER BY用于对结果进行排序，SEPARATOR用于指定字符串之间的分隔符。默认情况下，此函数不剔除重复值、不对结果排序，也不使用逗号分隔值。

```
mysql> CREATE TABLE t (name CHAR(10));
```

```
mysql> INSERT INTO t VALUES('dog'),('cat'),('rat'),('dog'),('rat');
```

```
mysql> SELECT GROUP_CONCAT(name) FROM t;
```

```
-----  
GROUP_CONCAT(name) |
```

```
-----  
dog, cat, rat, dog, rat  
-----
```

```
mysql> SELECT GROUP_CONCAT(name SEPARATOR '!') FROM t;
```

```
-----  
GROUP_CONCAT(name SEPARATOR ':')
```

```
-----  
dog:cat:rat:dog:rat  
-----
```

```
mysql> SELECT GROUP_CONCAT(name ORDER BY name DESC) FROM  
t;
```

```
-----  
GROUP_CONCAT(name ORDER BY name DESC)
```

```
-----  
rat, rat, dog, dog, cat  
-----
```

```
mysql> SELECT GROUP_CONCAT(DISTINCT name ORDER BY name)  
FROM t;
```

```
-----  
GROUP_CONCAT(DISTINCT name ORDER BY name)
```

```
-----  
cat, dog, rat  
-----
```

GROUP_CONCAT() 函数返回值的最大长度受限于系统变量 group_concat_max_len。通过更改这个变量，可以获得更长的连接结果。

MAX([DISTINCT] expr)

根据表达式 expr 对被选中的行里的所有非 NULL 值进行计算，并返回计算结果中的最大值；如果没有非NULL值，则返回NULL。MAX()函数还可以用于字符串值或时态值，此时它将返回词法意义上或时态意义上的最大值。

SELECT MAX(mycol) FROM mytbl → 9

DISTINCT关键字将导致MAX()函数在筛选最大值时剔除重复的expr值（这并不会改变返回结果）。

MIN([DISTINCT] expr)

根据表达式 expr 对被选中的行里的所有非 NULL 值进行计算，并返回计算结果中的最小值。如果没有非NULL值，则返回NULL。MIN()函数还可以用于字符串值或时态值，此时它将返回词法意义上或时态意义上的最小值。

SELECT MIN(mycol) FROM mytbl → 1

DISTINCT 关键字将导致 MIN()函数在筛选最小值时剔除重复的expr 值（这并不会改变返回结果）。

STD(expr)

STDDEV(expr)

STDDEV_POP(expr)

根据表达式expr对被选中的行里的所有非NULL值进行计算，并返回计算结果的总体标准偏差；如果没有非NULL值，则返回NULL。

SELECT STDDEV_POP(mycol) FROM mytbl → 2.7701

STDDEV_SAMP(expr)

根据表达式expr对被选中的行里的所有非NULL值进行计算，并返回计算结果的样本标准偏差；如果没有非NULL值，则返回NULL。

SELECT STDDEV_SAMP(mycol) FROM mytbl →
2.9921

SUM([DISTINCT] expr)

根据表达式expr对被选中的行里的所有非NULL值进行计算，并返回计算结果的累加值；如果没有非NULL值，则返回NULL。

```
SELECT SUM(mycol) FROM mytbl → 39
```

DISTINCT关键字将导致SUM()函数在计算累加值时剔除重复的expr值。

VARIANCE(expr)

VAR_POP(expr)

根据表达式 expr 对被选中的行里的所有非 NULL 值进行计算，并返回计算结果的总体方差；如果没有非NULL值，则返回NULL。

```
SELECT VAR_POP(mycol) FROM mytbl → 7.6735
```

VAR_SAMP(expr)

根据表达式 expr 对被选中的行里的所有非 NULL 值进行计算，并返回计算结果的样本方差；如果没有非NULL值，则返回NULL。

```
SELECT VAR_SAMP(mycol) FROM mytbl → 8.9524
```

C. 2. 7 安全函数和压缩函数

这些函数用来完成各种与安全有关的操作，如字符串的加密或压缩。在这类函数中，有一些是成对出现的，其中一个用来进行加密，另一个则用来解密。这些成对出现的函数通常都要使用同一个字符串来充当密钥或密码。要想在解密后得到原先的数据，就必须使用在加密它们时使用的同一个密钥来进行解密；否则，解密结果将毫无价值。

当使用会返回二进制串的加密函数时，如果想要将结果保存在数据库里，可以使用二进制串数据类型（如VARBINARY或某种BLOB类型）的列。这样可以防止出现二进制串类型可能有的问题，如字符集转换或尾部空格截断。

某些加密函数返回的是 ASCII 字符串，并且该返回值是一个连接字符集里的非二进制串。这些函数包括：MD5()、OLD_PASSWORD()、PASSWORD()、SHA()、SHA1()和SHA2()。这些函数在MySQL 5.5.3之前（对于SHA2()，则是在MySQL 5.5.6之前），返回二进制串。

AES_DECRYPT(str, key_str)

对于已加密的通过调用AES_ENCRYPT()获得的字符串str,使用密钥字符串key_str进行解密,返回结果字符串。如果其中一个参数为NULL,则返回NULL。

```
AES_DECRYPT(AES_ENCRYPT('secret','scramble'),'scramble')
```

→ 'secret'

```
AES_ENCRYPT(str, key_str)
```

采用高级加密标准(Advanced Encryption Standard, AES)以密钥字符串key_str对字符串str进行加密,它使用的密钥长度是128位。返回的结果为一个二进制串;如果输入参数中有NULL值,则返回NULL。本函数的加密结果可以用AES_DECRYPT()函数和同一个密钥解密为原先的字符串。

```
COMPRESS(str)
```

把参数字符串的压缩版本按二进制串格式返回;如果没有用压缩库编译服务器,则返回NULL。

```
DECODE(str, key_str)
```

给定调用ENCODE()后得到的加密字符串str,用密钥字符串key_str对其进行解密。返回结果字符串;如果字符串str为NULL,则返回NULL。

```
DECODE(ENCODE('secret','scramble'),'scramble') → 'secret'
```

```
DES_DECRYPT(str[, key_str])
```

对用DES_ENCRYPT()函数进行加密而得到的二进制串str进行解密。如果系统上没有启用SSL支持功能或者解密操作失败,DES_DECRYPT()将返回NULL。

如果指定了key_str参数,则它会被用作解密密钥;否则,DES_DECRYPT()函数将使用服务器的DES 密钥文件里的密钥来解密该字符串。密钥编号将根据该加密字符串的第一个字节的第0~6位来确定。该密钥文件的位置由服务器启动时使用的--des-key-file 选项指定。

如果str看起来不像个加密串,则DES_DECRYPT()会原样返回该字符串。(例如,如果字符串str的第一个字节的第7位未被置位,则会出现这种情况。)

只有拥有SUPER权限的用户才能使用单参数的DES_DECRYPT()函数。

```
DES_ENCRYPT(str[, {key_num|key_str}])
```

采用DES加密对字符串str 进行加密，返回一个二进制串的加密结果。本函数的加密结果可以用DES_DECRYPT()函数解密。如果系统上没有启用SSL支持功能或者加密操作失败，DES_ENCRYPT()将返回NULL。

如果给出了 key_str 参数，DES_ENCRYPT()函数就将把它用作加密密钥。如果给出了key_num参数，则它应该是一个0~9的整数，表示服务器DES密钥文件里的条目编号。在这种情况下，加密密钥会从该条目获得。如果没有指定key_str或key_num参数，则会使用DES密钥文件里的第一个密钥来进行加密。（它并不一定等同于将key_num参数设置为0时。）

结果字符串的第一个字节能让我们知道字符串是如何加密的。这个字节的第7位会被置位，而0~6位则构成了一个密钥编号。如果这个编号是一个0~9的数字，则说明加密工作是用服务器DES密钥文件里对应于该编号的那个密钥来完成的。如果这个编号等于127，则说明加密工作是用key_str 参数来完成的。例如，如果加密时使用的密钥编号为 3，加密结果字符串第一个字节的值就应该是131（即128+3）。如果加密时使用的是key_str参数，加密结果字符串第一个字节的值就应该是255（即128+127）。

对于使用密钥编号的加密操作，服务器会读取 DES 密钥文件，查找相应的密钥字符串。该密钥文件由服务器启动时使用的--des-key-file 选项指定。密钥在密钥文件里的存放格式如下所示：

```
key_num key_str
```

每个key_num 都是一个0~9的数字，而key_str 则是与之对应的加密密钥。key_num与 key_str 之间要用至少一个空白符加以分隔。密钥文件里的各行允许按任意先后顺序出现。

与 DES_DECRYPT()函数不同的是，DES_ENCRYPT()函数并不要求用户必须拥有 SUPER 权限才能使用来自DES密钥文件里的密钥。（任何人都可以使用DES密钥文件里的密钥去加密自己的信息，但只有那些有足够权限的用户才能对其解密。）

```
ENCODE(str, key_str)
```

用密钥字符串key_str对字符串str进行加密，返回一个二进制串的加密结果。本函数的加密结果可以用DECODE()函数和同一个密钥解密为原先的字符串。

```
ENCRYPT(str[, salt])
```

返回字符串str的加密结果字符串；如果输入参数中有NULL值，则返回NULL。这是一个不可逆的加密过程。salt 参数（如果给出的话）必须是一个由两个或更多字符组成的字符串。对于一个给定的salt 值，不管你对字符串str 进行多少次加密，其结果都将是一样的。如果不带salt 参数，MySQL将使用随机值，同样的ENCRYPT() 函数调用产生的结果却不相同。

```
ENCRYPT('secret','AB') →  
'ABS5SGh1EL6bk'
```

```
ENCRYPT('secret','AB') →  
'ABS5SGh1EL6bk'
```

```
ENCRYPT('secret') →  
'ByUthKNv3.LsE'
```

```
ENCRYPT('secret') →  
'Hyx4rhb7Qdvpk'
```

ENCRYPT() 函数使用Unix 操作系统的底层 crypt() 系统调用来完成加密操作，所以针对crypt() 的各项规定也同样适用于它。特别地，在某些系统上，crypt() 只对字符串的前 8个字符加密。如果你的系统上没有 crypt() 或者它不可用，ENCRYPT() 函数的返回值将永远是NULL。

str不应该是字符集ucs2、utf16、utf16le或utf32里的字符串。crypt() 会将NUL字节解释为参数终止符，但是这些字符集允许字符串里出现NUL。

MD5(str)

返回用RSA Data Security, Inc. 的MD5消息摘要算法为字符串str生成的一个128位的校验和。这个返回值是由32个十六进制数字构成的一个非二进制串；如果字符串str为NULL，那么返回值也将是NULL。请参考本节引言里关于字符串返回类型的那部分内容。

```
MD5('secret') → '5ebe2294ecd0e0f08eab7690d2a6ee69'
```

OLD_PASSWORD(str)

此函数返回的是PASSWORD() 所返回的那个加密密码值的MySQL 4.1以前的版本格式。请参考本节引言里关于字符串返回类型的那部分内容。

PASSWORD(str)

返回根据字符串str计算出来的一个加密密码字符串，其形式与MySQL权限表里使用的一样。这是一个不可逆的加密过程。请参考本节引言里关于字符串返回类型的那部分内容。

```
PASSWORD('secret')          →  
'*14E65567ABDB5135D0CFD9A70B3032C179A49EE7'
```

PASSWORD()函数所使用的算法与 Unix 用来加密账户密码的算法是不一样的。如果你想要的是后一种加密效果，就应该使用ENCRYPT()函数。

如果系统变量 old_passwords的值为 1，则 PASSWORD()函数返回的是一个使用MySQL 4.1版之前的散列算法而得到的密码。在这种情况下，PASSWORD()函数和OLD_PASSWORD()函数将返回同样的值。

```
SHA1(str)
```

```
SHA(str)
```

返回用安全散列算法（Secure Hash Algorithm, SHA）为字符串str生成的一个160位的校验和。这个返回值是由 40个十六进制数字构成的一个非二进制串；但如果字符串 str 为NULL，那么返回值也将是NULL。请参考本节引言里关于字符串返回类型的那部分内容。

```
SHA1('secret')              → 'e5e9fa1ba31ecd1ae84f75caaa474f3a663f05f4'
```

SHA1()比MD5()更安全，但是没有SHA2()安全。

```
SHA2(str, hash_length)
```

此函数和SHA1()函数相似，但安全性更高。它会计算第一个参数的散列值，而结果值的位长度由第二个参数指定。散列值的长度只能是224、256、384或512。最后的结果是一个以十六进制数字表示的具有指定位长度的非二进制串。如果参数为 NULL，或者散列值长度无效，则结果为NULL。请参考本节引言里关于字符串返回类型的那部分内容。

```
SHA2('secret',224)  
→ '95c7fbca92ac5083afda62a564a3d014fc3b72c9140e3cb99ea6bf12'
```

SHA2()是在MySQL 5.5.5里引入的。

```
UNCOMPRESS(str)
```

给定一个用COMPRESS()函数压缩而来的字符串，UNCOMPRESS()函数将返回其原始字符串。如果参数不是一个压缩字符串，或是在编译MySQL服务器时没有把支持压缩功能的函数库包括进去，它将返回NULL。

UNCOMPRESSED_LENGTH(str)

给定一个用COMPRESS()函数压缩而来的字符串，此函数将返回其原始未压缩字符串的长度。如果在编译服务器时没有把支持压缩功能的函数库包括进去，它将返回NULL。

C. 2. 8 建议性锁定函数

本节里的各个函数实现的是建议性（advisory）锁定，也被称作协同（cooperative）锁定。它们可用于编写根据某个商定好的锁名的状态协同操作的应用程序。基础函数是 GET_LOCK() 和 RELEASE_LOCK()，分别用于获得锁和释放锁。另外两个函数是 IS_FREE_LOCK() 和 IS_USED_LOCK()，分别用于查询锁的状态和确定哪个客户端拥有指定的锁。

从本质上讲，建议性锁定只是让你锁定的一个名字，它仅仅是一个字符串而已。建议性锁是私有的，只能由当前拥有它的那个客户端来释放；但也是全局性的，任何客户端都可以查询锁名的状态。

要想获得建议性锁，可以使用 GET_LOCK(str, timeout) 函数，其中，str 参数是锁名，timeout 参数是超时值，单位为秒。如果在超时时间内成功地获得锁，则 GET_LOCK() 将返回1；如果因超时导致锁定失败，则返回0；如果发生错误，则返回NULL。

timeout 参数值是尝试获得建议性锁的等待时间，而不是锁的持续时间。在获得建议性锁之后，它在被释放之前将一直发挥效力。

下面的调用将尝试获得一个名为 'Nellie' 的建议性锁，最多等待10秒的时间：

```
GET_LOCK('Nellie',10)
```

建议性锁只作用于字符串名字本身。它不锁定数据库、表或表里的行或列。换句话说，建议性锁不会阻止任何其他客户端对数据库里的表做任何事情，因此 GET_LOCK() 函数的锁定只是建议性的——它只是让其他合作客户端能够通过它来确定建议性锁定是否在生效。

已经获得建议性锁的客户端将阻止其他客户端（对一个与MySQL服务器同时建立了多个会话的多线程客户端而言，就是其他线程）锁定该名字。假设

客户端1已经锁定了字符串'Nellie'。当客户端2试图锁定同一个字符串时，客户端2将被阻塞，直到客户端1释放这个建议性锁或是客户端2的倒计时时间结束。如果客户端1在客户端2的倒计时时间结束前释放这个建议性锁，那么客户端2就能成功获得这个锁；否则，客户端2会失败。

因为两个会话不能同时锁定同一个字符串，我们就可以在应用程序里把一个字符串用作某个建议性锁的名字，根据该建议性锁的状态去判断能否安全地进行与该名字相关的操作。例如，我们可以给表里的行设置一个建议性锁，让应用程序根据其状态去协调对该行的操作。

如果想显式地释放某个锁，可以在调用RELEASE_LOCK()函数时带上锁名：

```
RELEASE_LOCK('Nellie')
```

如果锁释放成功，则RELEASE_LOCK()函数会返回1；如果锁属于另一个会话，则返回0；如果锁不存在，则返回NULL。

每个客户端会话一次只能锁定一个字符串，所以当持有某个锁的客户端发出另一个GET_LOCK()调用时，它当前持有的锁将自动释放。此时，在新锁获得之前，旧锁会被释放；即使锁名完全一样，也是如此。当客户端与服务器之间的会话终止时，锁也会被释放掉。因此，如果某个运行时间非常长的客户端会话由于不活动而超时，那么服务器会释放其占有的所有锁。

如果想测试锁名的状态，可以使用以下两种方法。

调用IS_FREE_LOCK(str)函数。如果指定的名字未被锁定，则返回1；如果已锁定，则返回0；如果发生错误，则返回NULL。

调用IS_USED_LOCK(str)函数。如果某个客户端正占用该锁名，则返回该客户端的连接ID。如果不存在该锁，则返回NULL。

还可以使用GET_LOCK(str, 0)，通过简单的轮询来迅速确定str 是否已被锁定，这个过程无需等待。不过，这个简便方法有一个副作用：如果那个字符串当前未被锁定，那么你发出的调用就会锁定它，这意味着你必须适时发出RELEASE_LOCK()调用。

如果锁名参数为NULL，那么所有的建议性锁定函数都将返回NULL。

```
GET_LOCK(str, timeout)
```

在timeout秒的超时时间内，以str字符串为名请求一个建议性锁。如果在超时时间内成功地获得锁，则 GET_LOCK() 返回 1；如果因超时导致锁定失

败，则返回 0；如果发生错误，则返回NULL。

`IS_FREE_LOCK(str)`

检查以`str`为名的建议性锁的状态。如果该名字可被锁定，则返回1；如果已被锁定，则返回0；如果发生错误，返回NULL。

`IS_USED_LOCK(str)`

如果某个客户端正占用锁 `str`，则返回该客户端的连接 ID；如果不存在该锁，则返回NULL。

`RELEASE_LOCK(str)`

释放`str`指定的那个建议性锁。如果释放成功，则返回1；如果该锁正由另一个会话占用，则返回0；如果该锁不存在，则返回NULL。

C. 2. 9 IP地址函数

本节的这些函数主要用于处理和测试IP地址，其中包括IPv4和IPv6（自MySQL 5.6.3起）两种形式。

`INET_ATON(str)`

如果用点分四组表示法给定字符串形式的IPv4 地址，则返回相应的整数形式的网络字节序；如果参数不是一个有效的IPv4地址，则返回NULL。

`INET_ATON('64.28.67.70')` →
1075594054

`INET_ATON('255.255.255.255')` →
4294967295

`INET_ATON('256.255.255.255')` → NULL

`INET_ATON('www.mysql.com')` →
NULL

`INET_NTOA(n)`

如果用网络字节序给定整数形式的IPv4 地址，则返回相应的用点分四组表示法表示的字符串；如果参数非法，则返回NULL。

INET_NTOA(1075594054) →
'64.28.67.70'

INET_NTOA(2130706433) →
'127.0.0.1'

INET6_ATON(expr)

如果以字符串形式给定IPv6或IPv4地址，则以二进制串的形式返回相应的网络字节序的数字值；如果参数非法，则返回NULL。对于返回的非NULL值，其类型为VARINARY(16)或VARINARY(4)，分别对应于IPv6地址或IPv4地址。如果要将结果转换为可打印形式，可以使用HEX()。

HEX(INET6_ATON('::1')) →
'00000000000000000000000000000001'

HEX(INET6_ATON('192.168.10.14')) → 'C0A80A0E'

INET6_ATON()是在MySQL 5.6.3里引入的。

INET6_NTOA(expr)

如果用二进制串给出数字形式的IPv6或IPv4地址，则返回相应的字符串表示；如果参数非法，则返回NULL。

INET6_NTOA(INET6_ATON('::1')) → '::1'

INET6_NTOA(INET6_ATON('192.168.10.14')) →
'192.168.10.14'

INET6_NTOA()是在MySQL 5.6.3里引入的。

IS_IPV4(expr)

如果参数是一个有效的IPv4地址字符串，则返回1；否则，返回0。

IS_IPV4('::1') → 0

IS_IPV4('192.168.10.14') → 1

IS_IPV4()是在MySQL 5.6.3里引入的。

IS_IPV4_COMPAT(expr)

IS_IPV4_MAPPED(expr)

这些函数可用于解释与 INET6_ATON() 的返回值形式相同的IPv6地址，即数字形式的二进制串。前者用于确定参数表示的是不是一个有效的兼容IPv4的IPv6地址字符串；后者用于确定参数表示的是不是一个有效的IPv4映射的IPv6地址字符串。如果确实如此，则它们会返回 1；否则，返回 0。兼容IPv4 或 IPv4 映射的地址格式分别为::ipv4_addr或::ffff:ipv4_addr。

IS_IPV4_COMPAT(INET6_ATON('::1')) → 0

IS_IPV4_COMPAT(INET6_ATON('::192.168.10.14')) → 1

IS_IPV4_COMPAT(INET6_ATON('::ffff:192.168.10.14'))

→ 0

IS_IPV4_MAPPED(INET6_ATON('::1')) → 0

IS_IPV4_MAPPED(INET6_ATON('::192.168.10.14')) → 0

IS_IPV4_MAPPED(INET6_ATON('::ffff:192.168.10.14'))

→ 1

IS_IPV4_COMPAT() 和 IS_IPV4_MAPPED() 是在MySQL 5.6.3里引入的。

IS_IPV6(expr)

如果参数是一个有效的IPv6地址字符串，则返回1；否则，返回0。对于IPv4地址，此函数会返回0。

IS_IPV6('::1') → 1

IS_IPV6('192.168.10.14') → 0

IS_IPV6() 是在MySQL 5.6.3里引入的。

C. 2. 10 XML函数

本节介绍的函数可用于处理表示XML片段的字符串和XPath表达式：一个用于从XML片段里提取文本；另一个用于把XML片段中的匹配元素替换为另一个字符串，并返回替换结果。这些XML字符串参数包含的XML标记必须正确配对和嵌套。

这些函数使用的是XPath 1.0规范（请参考<http://www.w3.org/TR/xpath> 提供的规范）。MySQL对Xpath的支持并不完备。相关限制请参考MySQL参考手册。

EXTRACTVALUE (xml_str, xpath_expr)

根据XPath表达式对XML字符串进行处理，返回该XPath表达式在该XML字符串里匹配到的XML元素的第一个文本节点的内容。如果该XPath表达式匹配到了多个XML元素的话，此函数将以空格为分隔符把所有匹配元素的第一个文本节点连接在一起作为返回结果。

`EXTRACTVALUE('<a><b>B</b><c>C</c></a>', '//b')` → 'B'

`EXTRACTVALUE('<a><b>B1</b><b>B2</b><b>B3</b></a>', '//b')`
→ 'B1 B2 B3'

如果没有找到任何匹配，此函数将返回一个空串（这和匹配到一个XML元素、但该XML元素不包含任何文本内容的情况是一样的）。

UPDATEXML (xml_str, xpath_expr, xml_new)

根据XPath表达式对XML字符串进行处理，把匹配到的XML元素替换为xml_new参数值并返回。如果没有找到任何匹配或是匹配到了多个XML元素，此函数将返回该XML字符串本身而不对它做任何改动。

C. 2. 11 空间函数

MySQL提供了对空间值进行操作的各种函数。虽然这类函数很多，但是可以将高度相似的函数分成一组。例如，点、线、多边形和其他空间类型转换函数都很相似。基于此原因，并且也是为了节省篇幅，本附录没有再介绍它们，请大家根据需要自行查阅MySQL参考手册。

C. 2. 12 杂项函数

本节的各个函数不属于前面几节介绍的任何一类函数。

BENCHMARK (n, expr)

对表达式expr重复计算n次。BENCHMARK()函数和其他函数不太一样，因为人们通常只在mysql客户端程序里使用它。它的返回值总是为0，因此没有任何实际的用途。比较有意思的值是mysql在执行下面查询语句之后显示出来的执行时间：

```
mysql> SELECT BENCHMARK(1000000,PASSWORD('secret'));
```

```

-----
BENCHMARK(1000000,PASSWORD('secret'))
-----
0
-----

```

1 row in set (2.35 sec)

这个时间只能大致评估 MySQL服务器对给定表达式的计算操作到底有多快，因为它是客户端端的时钟时间，不是服务器端的CPU 时间。这个时间可能受到多种因素的影响，如服务器端的负载情况、BENCHMARK() 查询到达时MySQL 服务器是正在运行还是被切换出了内存，等等。为了获得一个有代表性的值，应该重复执行多次BENCHMARK() 查询。

BIT_COUNT(n)

用BIGINT值（64位整数）返回参数n中被设置为1的二进制位的个数。

0	BIT_COUNT(0)	→
1	BIT_COUNT(1)	→
1	BIT_COUNT(2)	→
3	BIT_COUNT(7)	→
64	BIT_COUNT(-1)	→

BIT_COUNT(NULL) →
NULL

BIT_LENGTH(str)

返回给定参数str以二进制位为计算单位的长度；如果str为NULL，则返回NULL。

BIT_LENGTH('abc') → 24

BIT_LENGTH('a long string') → 104

BIT_LENGTH(CONVERT('abc' USING ucs2)) → 48

CONNECTION_ID()

返回MySQL服务器分配给当前客户端会话的连接标识符。每个客户端都会分配到一个独一无二的连接标识符。

CONNECTION_ID()
→ 10146

CURRENT_USER()

连接 MySQL 服务器时，MySQL 服务器会使用mysql.user 表里的某个特定的账户行对你的连接进行身份验证。CURRENT_USER() 函数将以一个'user_name@host_name' 格式的utf8字符串的形式返回那个行的User和Host列的值。那对括号是可选的。

CURRENT_USER() →
'sampadm@localhost'

SUBSTRING_INDEX(CURRENT_USER(), '@', 1) →
'sampadm'

CURRENT_USER() 函数可用于检测 MySQL 服务器把你当成了谁。也许和你在连接 MySQL 服务器时给出的用户名不一样，因为你有可能“碰巧”通过了服务器使用其他账户对你进行的身份验证。需要特别注意的是，如果 MySQL 服务器认为你是一个匿名用户，那么此函数返回值里的用户名部分将是空的，然而 USER() 函数返回值里的用户名部分永远是你建立连接时给出的用户名。

在视图或存储程序里，CURRENT_USER() 函数默认会返回与该对象的 DEFINER 属性相对应的账户。对于视图和定义时带有 SQL SECURITY INVOKER 特性的存储例程，CURRENT_USER() 会返回调用用户的账户。

DATABASE()

以 utf8 字符串的形式返回默认数据库的名字；如果没有默认数据库，则返回 NULL。如果在存储例程里调用DATABASE()，则它会返回该例程相关联的那个数据库。

```
DATABASE() →  
'sampdb'
```

FOUND_ROWS()

返回刚执行的那条SELECT语句在没有LIMIT子句的情况下返回的行的个数。例如，下面这条语句最多只能返回10个行：

```
mysql> SELECT * FROM mytbl LIMIT 10;
```

使用下面这些语句，可以查出前一条SELECT 语句在没有LIMIT 子句的情况下会返回多少个行：

```
mysql> SELECT SQL_CALC_FOUND_ROWS * FROM mytbl LIMIT 10;
```

```
mysql> SELECT FOUND_ROWS();
```

DEFAULT(col_name)

INSERT语句允许指定DEFAULT关键字来往新行里插入列的默认值。但是这个关键字不能用在某些特定的表达式或其他上下文里。例如，MySQL不允许在UPDATE语句里使用DEFAULT 关键字把某个列重置为默认值。DEFAULT() 函数可以在这类场合帮到你。如果给定一个列名，DEFAULT() 函数会返回该列的默认值；如果该列没有默认值，则产生错误。

```
UPDATE counts SET counter = DEFAULT(counter)
```

```
WHERE max_time > expire_time;
```

LAST_INSERT_ID()

LAST_INSERT_ID(expr)

如果不带任何参数，那么此函数会返回最近一个INSERT 语句成功生成的那个第一个AUTO_INCREMENT值；如果此前尚未生成过这样的值，则返回0。如果语句插入了多个行，那么LAST_INSERT_ID()会返回其中第1行的值。如果出现错误，那么LAST_INSERT_ID()的值为未定义。

如果带有expr参数，LAST_INSERT_ID()的结果将为该参数的值，但是该值会被当作像是自动生成的一样——这一点在生成序列值时非常有用。

更多详细信息请参考 3.4节。对于这两种形式的LAST_INSERT_ID()函数，MySQL服务器会根据每个会话来维护这个值。该值不能被其他客户端更改，即使那些会导致自己创建新的自动生成值的客户端也无法更改。

LOAD_FILE(file_name)

读取file_name 文件并把它的内容返回为一个字符串。如果文件名是一个文本字符串，则它会用character_set_filesystem 给定的那个字符集进行解释。这个文件必须存放在MySQL 服务器主机里，必须以一个绝对（完整）路径名的形式给出，必须是全局可读的，以确保你不会读取受保护的文件。如果secure_file_priv 系统变量的值不为空，该变量值应该是一个目录，而你打算读取的文件必须存放在该目录里。因为这个文件必须存放在服务器主机里，所以你还必须具备FILE权限。只要有一项条件没有满足，LOAD_FILE()函数就将返回NULL。

MASTER_POS_WAIT(log_file, pos[, timeout])

此函数主要用于测试复制机制中的服务器。当你在从服务器上执行此函数的时候，它会阻塞从服务器上的其他操作，让从服务器专注于读取和处理来自主服务器的事件，直到到达log_file和pos参数所给定的复制位置为止。如果还给出了可选的timeout参数，它会为 MASTER_POS_WAIT()函数设置一个等待秒数的上限。如果这个参数值小于或等于 0，则表示没有超时。

MASTER_POS_WAIT()函数返回的是事件数，表示距离给定的复制位置还有多少个日志文件事件需要处理。如果从服务器已经到达该位置，则此函数立刻返回 0。如果此函数的返回值是-1，则可能是因为它设置的超时时间已到、发生了错误，或者主服务器信息未被初始化。如果此函数的返回值是NULL，则表明从服务器的SQL线程在该函数所设置的等待期内没有运行或是已停止。

NAME_CONST(name, value)

此函数仅供MySQL内部使用（例如，为了把语句写入二进制日志）。它将返回name 参数所给定的列名和value参数。这两个参数必须是常数。

ROW_COUNT ()

此函数与MySQL的C语言API函数库中的mysql_affected_rows() 函数相似。其返回值取决于前一条语句的类型。

对于SELECT以外的数据操作语句：返回受影响的（即插入的、删除的或更新的）行数。

对于SELECT语句：如果语句返回的是一个结果集，则此函数会返回-1；否则，会返回检索到目标的行数。例如，语句SELECT ... INTO var_name会返回1，而SELECT ... INTO file_name则会返回写入文件里的行数。

对于数据定义语句（如CREATE TABLE）：此函数会返回0。

如果语句产生了错误，则ROW_COUNT() 会返回-1。

SCHEMA ()

它等价于DATABASE()。

SESSION_USER ()

它等价于USER()。

SLEEP (seconds)

让当前客户端休眠seconds秒，并返回0；如果在休眠时被意外打断，则返回1。seconds参数可以带小数部分。

SYSTEM_USER ()

它等价于USER()。

USER ()

返回一个utf8 字符串——它表示的是客户端在连接MySQL服务器时指定的用户名，以及该客户端所在的那台主机。返回值是一个格式为'user_name@host_name' 的字符串。

USER() → 'paul@localhost'

SUBSTRING_INDEX(USER(), '@', 1) → 'paul'

SUBSTRING_INDEX(USER(), '@', -1) →
'localhost'

UUID()

返回一个“全局唯一标识符”。其意图是想让每次调用UUID()函数后得到的返回值都不同。返回值的唯一性无法保证，但是出现重复值的概率却是极低的。

UUID() → '4550868e-3c1f-1027-
9cc8-78fa7f8d46b6'

此函数的返回值是由一个128位的数字生成的一个由5组十六进制数构成的utf8字符串。前4组应该是唯一时态值，最后一组应该是唯一空间值。前3组来自一个时间戳。第4组用于保证这个时间戳的唯一性（如因采用夏令时而导致的时间变化）而特意增加的。第5组是一个IEEE 802节点编号。它可能是根据服务器主机上具有唯一性的值（如网络接口地址）生成的。如果无法获得这样的唯一值，则使用一个随机生成的48位数代替。

UUID_SHORT()

此函数类似于UUID()，但是统一标识符是一个64位的无符号整数，而非一个字符串。

UUID_SHORT() →
94344395712626688

为保证值的唯一性，则必须满足这样几个条件：系统变量server_id的值在各个复制服务器之间必须是唯一的，并且位于0~255；在两次服务器重启期间，不能将服务器系统的时间往回调；在两次服务器重启期间，调用UUID_SHORT()的平均速率不能超过1600万次/秒。

VALUES(col_name)

此函数需要与INSERT ... ON DUPLICATE KEY UPDATE语句一起使用。在UPDATE子句里，VALUES(col_name)返回的是插入指定列（它不能有重复的键值，否则会出现错误）的值。在此情况之外，此函数会返回 NULL。此函数可以用于构造另一个与原始插入值有关的另一个插入值。

VERSION()

此函数返回的是一个描述当前MySQL服务器版本信息的utf8字符串。

VERSION()

→

'5.5.28-log'

此值由一个版本编号以及（可能的）一个或多个后缀构成（如-log用于表明启用日志记录；-debug用于表明服务器正运行在调试模式下）。

附录D

系统变量、状态变量和用户变量参考

本附录介绍的主要内容是以下几种类型的MySQL变量。

用于对服务器进行配置或者通过对它们进行检查即可获得当前配置信息的系统变量。

提供与服务器当前操作状态有关信息的状态变量。

可以定义、赋值，以及可以在表达式里引用的用户变量。

代表缓冲区大小或长度的变量值通常都是以字节为单位。例外情况会单独说明。

除非特别说明，否则这里列出的变量都至少存在于 MySQL 5.5.0之前的版本里。自那之后引入的、或者在含义上发生变化的变量都会另外加以说明。

D.1 系统变量

系统变量提供的是各种与服务器配置和功能有关的信息。大部分的系统变量都可以在服务器启动时进行设置。在运行时，每一个系统变量都拥有一个全局值或会话值，或者同时拥有这两个值。许多系统变量都是动态的，也就是说，可以在服务器正在运行时修改它们的值。这些类型的信息都会在每个变量的描述里（与变量名同处一行）指明。

对于那些可以在服务器启动时设置的变量，看到的是“启动”一词，后面紧跟着“直接设置”或某个选项。“直接设置”的意思是：可以在命令行上或者是在选项文件里，使用与变量名同名的选项直接设置该变量。

(F. 2. 1. 2节描述了具体的语法格式。) 否则, 在“启动”的后面需要紧跟用来设置变量的那个选项。例如, 需要使用`--default-time-zone` 选项来设置`time_zone`变量。对于某个给定的选项, 可以在附录F对`mysqld`程序的描述里找到其含义。

如果要指明某个变量的作用范围, 则会看到“启动”一词, 而后面紧跟着的是“全局”或“会话”(或者两者皆有), 用以表明该变量是否拥有GLOBAL形式或SESSION形式(或者是否同时拥有这两种形式)。运行时只读的变量通常只有一个全局值。

对于那些在服务器运行时可以进行修改的变量, 则会看到“动态”一词。

通过`SHOW VARIABLES`语句或`mysqladmin variables`命令, 可以查看这些系统变量。也可以检查数据库`INFORMATION_SCHEMA`里的`GLOBAL_VARIABLES`表和`SESSION_VARIABLES`表, 以获得系统变量的信息。要想显示单个变量值, 可以对于全局变量使用`SELECT @@GLOBAL. var_name`, 对会话变量使用`SELECT @@SESSION. var_name`或`SELECT @@var_name`。

使用`SET`语句可以设置系统变量。如果要设置全局变量, 则必须要拥有`SUPER`权限。设置会话变量通常不需要特别的权限。例外情况会特别注明。

系统变量的名字不区分大小写。

与运行时设置系统变量和查看其具体值有关的更多信息请参考12. 3. 1节。

此处描述的一些变量只在一些特定的配置里或者一些特定的平台上才有。例如, 在编译服务器时, 必须得支持调试功能, 才能使用`debug`变量。只有Windows平台上才有`named_pipe`。

下面列出的是一些比较通用的系统变量。与InnoDB存储引擎有关的变量将单独用一节来描述。

`auto_increment_increment` (启动: 直接设置; 作用范围: 全局、会话; 动态)

在服务器每次生成一个新序列值时, `AUTO_INCREMENT`值的递增量。取值范围是1~65535; 默认值为1。

`auto_increment_offset` (启动: 直接设置; 作用范围: 全局、会话; 动态)

AUTO_INCREMENT序列的起始值。取值范围是1~65535；默认值为1。

autocommit（启动：直接设置；作用范围：全局、会话；动态）

事务处理的自动提交模式。默认值为 1，因此自动提交功能是启用的，并且语句会立即生效。本质上，每条语句都是其自身的事务。将这个值设置为 0，可以禁用自动提交功能，如此一来，后续语句便只有等到提交完成（可以使用COMMIT语句，或者将autocommit设置为1来完成提交）之后才能生效。如果提交还未发生，则可以通过ROLLBACK 来取消事务里的语句。将 autocommit 设置为1，可以重新启用自动提交（并且会隐式提交所有挂起的事务）。

auto_sp_privileges（启动：直接设置；作用范围：全局；动态）

如果此变量为 1（默认值），那么在你创建存储例程时，MySQL 服务器将自动授予你EXECUTE和ALTER ROUTINE 权限，让你随后可以执行、修改或删除该例程。当删除例程时，MySQL服务器会收回那些权限。如果 automatic_sp_privileges的值为0，则上述权限的自动授予和回收机制将不起作用。

back_log（启动：直接设置；作用范围：全局）

当前连接正在处理时，排队中的最大等待连接请求数量。

basedir（启动：直接设置；作用范围：全局）

MySQL 安装根目录的路径名。如果其他路径名是以相对路径名的形式给出的，那么这些路径名会相对于此目录进行解析。

big_tables（启动：直接设置；作用范围：全局、会话；动态）

将此变量设置为1，可以使得在处理大型的结果集时，将所有临时结果保存到磁盘，而不是将它们保留在内存里。虽然这样做会使性能变得慢一点，但是那些要求大型临时表的SELECT 语句不会产生“表已满”的错误。其默认值为 0（即把临时表保存在内存里）。通常情况下，不需要设置此变量，因为服务器会根据需要自动将结果保存到磁盘里。

bind_address（启动：直接设置；作用范围：全局）

服务器在监听TCP/IP客户端连接时所绑定的那个IP地址。自MySQL 5.6.6起，其默认值为*（即监听所有的IPv4和IPv6接口）；那以前的版本，默认值为0.0.0.0（即监听所有的IPv4接口）。更多信息请参考12.2.4节。此变量是在MySQL 5.6.1里引入的。

`binlog_cache_size`（启动：直接设置；作用范围：全局；动态）

用于存储SQL语句（在将它们刷新到二进制日志之前，它们是事务的一部分）的那个缓存的大小。（只有当事务被提交，或者事物里包括有对非事务型表进行更新的语句时，这些语句才会被刷新。如果只回滚某个更新事务型表的事务，那么这些语句将被丢弃。）

`binlog_checksum`（启动：直接设置；作用范围：全局；动态）

是否在往二进制日志写入的事件里包含校验和。可取值包括：NONE（不写入校验和）和CRC32。此变量是在MySQL 5.6.2里引入的。在MySQL 5.6.6之前，默认值为NONE；之后，默认值为CRC32。

`binlog_direct_non_transactional_updates`（启动：直接设置；作用范围：全局、会话；动态）

可以同时更新事务型表和非事务型表的事务，可能会导致主服务器里的更新与从服务器里的更新不一致，因为对非事务型语句的更新，在它们出现在二进制日志之前，对于其他会话来讲是可见的。启用此变量（默认是禁用的）会导致非事务型更新被立即写到二进制日志里，而不是被缓存起来，一直到事务提交之后才写入。启用此变量，仅对于那些使用基于语句的日志记录进行复制的语句才有效。

`binlog_format`（启动：直接设置；作用范围：全局、会话；动态）

二进制的日志记录格式。其可取值包括：STATEMENT、ROW 或 MIXED，分别代表的是基于语句的日志记录格式、基于行的日志记录格式和混合型日志记录格式。如果使用混合格式，则 MySQL 服务器将根据具体情况在基于语句的和基于行的日志记录格式之间自动切换。默认格式为 STATEMENT。在运行时，如果要更改此变量或者会话值，客户端必须拥有SUPER权限。

`binlog_row_image`（启动：直接设置；作用范围：全局、会话；动态）

基于行的二进制日志记录会使用之前和之后的“图像”来描述原始行和更改行。`binlog_row_image` 控制着这些图像将包含多少列。FULL（默认值）表示记录所有列。MINIMAL只会记录那些需要确定行的那些列和被更改过的列。NOBLOB与FULL很像，它去掉了无关紧要的BLOB和TEXT列。此变量是在MySQL 5.6.2里引入的。

`binlog_rows_query_log_events`（启动：直接设置；作用范围：全局、会话；动态）

在启用基于行的二进制日志记录之后，启用此变量可以让有助于调试的信息包含在二进制日志里。此变量是在MySQL 5.6.2里引入的。

`binlog_stmt_cache_size`（启动：直接设置；作用范围：全局；动态）

在事务内执行非事务语句所需要使用的缓存的大小。默认大小为 32K。此变量是在MySQL 5.5.9里引入的。

`bulk_insert_buffer_size`（启动：直接设置；作用范围：全局、会话；动态）

用于帮助优化往MyISAM表里批量插入语句的缓存的大小。这里包括LOAD DATA语句、多行INSERT语句和INSERT INTO ... SELECT语句。当值为0时将禁用优化。

`character_set_client`（作用范围：全局、会话；动态）

客户端向服务器发送的那些SQL语句的字符集。

`character_set_connection`（作用范围：全局、会话；动态）

“客户端-服务器”连接的字符集。它主要用于解释字符串常量（带有前导符的字符串不包括在内），以及那些“从数字转换到字符串”而得到的那些字符串的字符集。

`character_set_database`（作用范围：全局、会话；动态）

默认数据库（如果有的话）的字符集。如果没有默认的数据库（例如，如果客户在连接MySQL服务器时没有指定默认数据库），此变量将被设置为系统变量 `character_set_server` 的值。每当用户选择一个不同的数据库，MySQL服务器便会自动设置 `character_set_database` 的值。

`character_set_filesystem`（启动：直接设置；作用范围：全局、会话；动态）

文件系统的字符集。这个字符集用来解释SQL语句里的文件名字符串，如LOAD DATA语句里的数据文件的名称。服务器在访问文件之前，会先把文件名从 `character_set_client` 变量指定的字符集转换为 `character_set_filesystem` 变量指定的那个字符集。默认为binary（即不会进行转换）。

`character_set_results`（作用范围：全局、会话；动态）

服务器发送给客户端的那些查询结果的字符集。

`character_set_server`（启动：直接设置；作用范围：全局、会话；动态）

服务器的默认字符集。

`character_set_system`（作用范围：全局）

系统字符集。其值总是 `utf8`。这是各种元数据（如数据库名、表名和列名）所使用的字符集。像 `DATABASE()`、`CURRENT_USER()`、`USER()` 和 `VERSION()` 这样的函数也是使用这个字符集。

`character_set_dir`（启动：直接设置；作用范围：全局）

存放字符集文件的那个目录。

`collation_connection`（作用范围：全局、会话；动态）

连接的字符集排序规则。

`collation_database`（作用范围：全局、会话；动态）

数据库的字符集排序规则，如果有的话。如果没有默认的数据库（例如，如果客户在连接MySQL服务器时没有指定默认数据库），此变量将被设置为系统变量 `collation_server` 的值。每当用户选择一个不同的数据库，MySQL 服务器便会自动设置 `collation_database` 的值。

`collation_server`（启动：直接设置；作用范围：全局、会话；动态）

服务器默认字符集所对应的默认排序方式。

`completion_type`（启动：直接设置；作用范围：全局、会话；动态）

事务的完成类型。此变量的值为 0（默认值）或 `NO_CHAIN` 时，不会影响 `COMMIT` 和 `ROLLBACK`。此变量的值为 1 或 `CHAIN` 时，将导致 `COMMIT` 和 `ROLLBACK` 分别等价于 `COMMIT AND CHAIN` 和 `ROLLBACK AND CHAIN` 语句。此变量的值为 2 或者 `RELEASE` 时，将使它们分别等价于 `COMMIT RELEASE` 和 `ROLLBACK RELEASE`。`AND CHAIN` 的含义是，当一个事务结束时，服务器将以同样的隔离级别自动开始一个新的事务。`AND RELEASE` 的含义是，当一个事务结束时，服务器将终止会话。

`concurrent_insert`（启动：直接设置；作用范围：全局；动态）

对于在数据文件的中间没有空洞的MyISAM表，MySQL服务器允许在对已有行进行检索的同时在该表的末尾插入新行。此变量控制着服务器是否允许并发插入。此变量的值为 0（或NEVER）时，会禁用此功能。此变量的值为 1（或AUTO）时，会启用此功能。此变量的值为2（或 ALWAYS）时，会对所有的MyISAM表启用并发插入，不管它们的数据文件里是否有空洞；此时，新行将被添加到正被使用的那个表的末尾，或者插到空洞处。默认值为1。

`connect_timeout`（启动：直接设置；作用范围：全局；动态）

`mysqld`在开始建立连接时等待数据包的秒数。默认值为10。

`core_file`（启动：直接设置；作用范围：全局）

当发生致命错误时，服务器在退出前是否需要生成一个核心文件。此变量是在MySQL 5.6.2里引入的。

`datadir`（启动：直接设置；作用范围：全局）

MySQL数据目录的路径名。

`date_format`（作用范围：全局）

此变量未被使用。

`datetime_format`（作用范围：全局）

此变量未被使用。

`debug`（作用范围：全局、会话；动态）

调试输出的类型。请参考F.2.1里对`--debug`的描述。此变量只有在服务器编译时带有调试支持时才存在。

`default_storage_engine`（启动：直接设置；作用范围：全局、会话；动态）

在创建表时，如果没有带上 `ENGINE = engine_name` 选项或使用了一个不被支持的`engine_name` 值，就使用此变量指定的引擎作为默认存储引擎。存储引擎的名字不区分大小写。默认为InnoDB。

`default_tmp_storage_engine`（启动：直接设置；作用范围：全局、会话；动态）

与default_storage_engine相似，但它适用于那些使用CREATE TEMPORARY TABLE创建的表。此变量是在MySQL 5.6.3里引入的。

default_week_format（启动：直接设置；作用范围：全局、会话；动态）

当调用WEEK()或YEARWEEK()函数时，如果不带可选的mode参数，则默认认为此变量设置的那个模式值。

delay_key_write（启动：直接设置；作用范围：全局；动态）

对于使用DELAY_KEY_WRITE选项创建的MyISAM表，服务器是否需要延迟键写入。此变量有以下3种可取值。

ON（默认值）。此时，服务器会根据DELAY_KEY_WRITE选项的值进行操作：如果表是用DELAY_KEY_WRITE=1选项创建的，则延迟键写入；如果表是用DELAY_KEY_WRITE=0选项创建的，则不延迟。

OFF。它表示的是，对任何表都不延迟键写入，不管它们是如何定义的。

ALL。它表示的是，对任何表都总是延迟键写入，不管它们是如何定义的。

比较常见的做法是，在运行复制从服务器时将delay_key_write设置为ALL，以提高MyISAM表的性能（不管这些表最初是如何被创建的，都延迟其键写入）。

delayed_insert_limit（启动：直接设置；作用范围：全局；动态）

它指的是该表的延迟行处理器在检查表是否有新的SELECT语句到来之前插入的源自INSERT DELAYED语句的行数量。如果有语句到达，则将插入操作挂起，以便检索操作能够得到执行。

delayed_insert_timeout（启动：直接设置；作用范围：全局；动态）

当INSERT DELAYED操作的处理器插入完排队的各个行时，它会等待delayed_insert_timeout秒，检查一下是否有新的INSERT DELAYED行到达。如果有行到达，则继续插入；否则，终止插入操作。

delayed_queue_size（启动：直接设置；作用范围：全局；动态）

每个表针对INSERT DELAYED语句可以排入队列的行数量。当队列排满时，则会有更多针对该表的INSERT DELAYED语句被阻塞，一直到队列拥有足

够的空间为止。

`div_precision_increment`（启动：直接设置；作用范围：全局、会话；动态）

在使用操作符/对两个精确数值做除法运算时，此变量表示的是需要增加的精度位数。例如，当 `div_precision_increment` 的值为 4 或 6 时，`.1/.7` 的结果分别为 `.14286` 或 `0.1428571`。此变量的取值范围是 0~30，默认为 4。

`eq_range_index_dive_limit`（启动：直接设置；作用范围：全局、会话；动态）

对于使用非唯一索引的相等比较处理，它指的是优化器使用索引统计代替索引等分进行行估算的范围上限。当值为 0 时，将禁用统计。默认为 10。此变量是在 MySQL 5.6.5 里引入的，在此之前，不管范围值为多少，都会使用索引等分。

`error_count`（作用范围：会话）

这是一个只读会话变量，它表示的是可能产生错误的最后那条语句所产生的错误数量。

`event_scheduler`（启动：直接设置；作用范围：全局；动态）

事件调度器的状态。此变量的可取值包括：OFF、ON 或 DISABLED。在启动 MySQL 服务器时，如果把此变量设置为 DISABLED，那么事件调度器的状态将无法在 MySQL 服务器运行时被更改。在启动 MySQL 服务器时，如果把此变量设置为 ON 或 OFF，那么在 MySQL 服务器运行时只能让事件调度器的状态在这两个值之间进行更改。默认值为 OFF。

`expire_logs_days`（启动：直接设置；作用范围：全局；动态）

此变量的默认值为 0。如果将其设置成其他的值，则 MySQL 服务器将自动删除在 `expire_logs_days` 天之前创建的二进制日志文件，并更新该二进制日志的索引文件。MySQL 服务器将在它每次启动以及每次打开一个新的二进制日志文件时执行这个到期检查。

`external_user`（作用范围：会话）

这是个只读会话变量，可以通过用于认证当前用户的身份认证插件进行设置。此变量是在 MySQL 5.5.7 里引入的。

`flush`（启动：直接设置；作用范围：全局；动态）

它用于表明服务器是否会在每次更新后将 MyISAM 表刷新到磁盘。这样可以降低在出现系统崩溃时表被损坏的风险，但是它会严重降低性能。因此，只有在使用不稳定的系统时它才会有用处。默认值是OFF。

`flush_time`（启动：直接设置；作用范围：全局；动态）

如果此变量为非零值，那么MySQL服务器会每隔`flush_time`秒关闭一次表，以便将尚未写入磁盘的更改刷新到磁盘。如果你的系统不够稳定，经常会死机或重启，那么使用此方法强行更新表，便可以减少表受损或数据丢失的概率，但是会降低性能。在Unix 系统上，其默认值为0；在Windows系统上，其默认值为1800（即30分钟）。

`foreign_key_checks`（作用范围：全局、会话；动态）

将此变量设置为0或1将分别禁用或启用InnoDB表的外键检查。默认进行检查。禁用外键检查有时会非常有用。例如，可以按不同于表的外键关系所要求的顺序恢复一个创建和加载表的转储文件。在加载完表之后，可以重新启用外键检查。

`ft_boolean_syntax`（启动：直接设置；作用范围：全局；动态）

支持使用IN BOOLEAN MODE模式的进行FULLTEXT搜索的操作符列表。

`ft_max_word_len`（启动：直接设置；作用范围：全局）

包括在FULLTEXT索引里的单词所允许的最大长度。更长的单词会被忽略掉。在你更改此变量的值时，应该重建所有拥有这些FULLTEXT索引的MyISAM表的索引。默认值为84。

`ft_min_word_len`（启动：直接设置；作用范围：全局）

包括在FULLTEXT索引里的单词所允许的最小长度。更短的单词会被忽略掉。在你更改此变量的值，应该重建所有拥有这些FULLTEXT索引的MyISAM表的索引。默认值为4。

`ft_query_expansion_limit`（启动：直接设置；作用范围：全局）

此变量用于那些使用WITH QUERY EXPANSION子句完成的全文检索。它决定了在每次搜索的第二阶段所能使用的“最佳匹配”的数量。

`ft_stopword_file`（启动：直接设置；作用范围：全局）

用于FULLTEXT索引的停用词文件。默认使用内建的停用词列表。如果要禁用停用词，可以将此变量的值设置为空串。在你更改此变量的值或者停用词列表的内容时，应该重建所有拥有这些FULLTEXT索引的MyISAM表的索引。

`general_log`（启动：直接设置；作用范围：全局；动态）

它用于指定是否要启用普通查询日志的日志记录。如果要启用，`log_output`变量会控制日志的输出目标。

`general_log_file`（启动：直接设置；作用范围：全局；动态）

它指的是普通查询日志文件的名字。在启用了FILE日志记录目标时，会用到它。它默认位于数据目录里的HOSTNAME.log中，其中，HOSTNAME是服务器主机的名字。如果其文件名是以相对路径形式给出的，则服务器会将它解释为相对于数据目录。

`group_concat_max_len`（启动：直接设置；作用范围：全局、会话；动态）

GROUP_CONCAT() 函数返回值的长度上限（默认为1024）。

`have_compress`（作用范围：全局）

它表示的是，服务器用于实现COMPRESS()和UNCOMPRESS()函数所需要的那个zlib压缩库是否可用。如果不可用，那么这两个函数将无法使用。

`have_crypt`（作用范围：全局）

它表示的是，服务器用于实现CRYPT()函数所需要的那个crypt()系统调用是否可用。如果不可用，那么该函数将无法使用。

`have_dynamic_loading`（作用范围：全局）

它表示的是，服务器是否支持动态加载各种插件。

`have_geometry`（作用范围：全局）

它表示的是，空间数据类型是否可用。

`have_openssl`（作用范围：全局）

它表示的是，服务器是否允许客户端使用SSL来建立加密连接。
`have_openssl` 等同于`have_ssl`。

`have_query_cache`（作用范围：全局）

它表示的是，查询缓存是否可用。

`have_rtree_keys`（作用范围：全局）

它表示的是，RTREE索引是否可用于SPATIAL索引。

`have_ssl`（作用范围：全局）

请参考`have_openssl`的描述。

`have_symlink`（作用范围：全局）

如果服务器在启动时带有`--skip-symbolic-links`选项，则此变量为DISABLED；否则，其可取值为 YES 或 NO，但是实际的含义取决于具体的系统平台。在Unix 里，它表示MyISAM 表是否支持表符号链接。在Windows 里，它表示是否支持数据库符号链接。请参考11.3节。

`host_cache_size`（启动：直接设置；作用范围：全局；动态）

它指的是主机缓存的大小，这个缓存的用途是保存与客户端连接尝试以及尝试是否成功有关的信息。连接失败时主机通常会被阻塞，一直到该缓存被刷新。在运行时更改此变量的值会刷新该缓存（与FLUSH HOSTS的效果一样）。默认值为128。此变量是在MySQL 5.6.5里引入的。

`hostname`（作用范围：全局）

服务器的主机名。MySQL服务器会在启动时自动确定此变量的值。

`identity`（作用范围：会话；动态）

等同于会话变量`last_insert_id`。

`ignore_db_dirs`（启动：直接设置；作用范围：全局）

此变量会被`--ignore-db-dir`选项的实例进行设置，它表示的是数据库目录里的目录不会被SHOW DATABASES或数据库INFORMATION_SCHEMA里的表当作数据库目录。默认为空。此变量是在MySQL 5.6.3里引入的。

`init_connect`（启动：直接设置；作用范围：全局；动态）

如果此变量的值不为空，那么它指定的是一条或多条以分号分隔的SQL语句（它们会在每个客户端连接到服务器时被执行）。此变量可以用于修改客户端的初始会话环境。对于具备SUPER 权限的用户，init_connect 变量将被忽略，目的是为了避免此变量值里包含的不正确或不恰当的句子导致管理员用户无法连接到服务器更正该错误。

init_file（启动：直接设置；作用范围：全局）

如果此变量的值不为空，那么它指定的是一个文件名，该文件名对应的文件中包括的是服务器在启动时执行的SQL 语句。如果文件名是以相对路径形式指定的，那么服务器会将它解释为相对于数据目录。此文件应该是每行一条语句，并且这些语句不应该以“;”或其他终止符作为结尾。

init_slave（启动：直接设置；作用范围：全局；动态）

对于主复制服务器，如果此变量的值不为空，那么它指定的是在每个从复制服务器进行连接时都会被执行的语句。该值的内容应该是一条或多条SQL语句，其间用分号分隔。

innodb_xxx

请参考D.1.1节。

insert_id（作用范围：会话；动态）

设置此变量，可以指定下一个INSERT 语句在插入一个AUTO_INCREMENT列时使用的值。它用于二进制日志的处理。

interactive_timeout（启动：直接设置；作用范围：全局、会话；动态）

此变量指的是，在服务器自己认为可以将交互式客户端会话关闭之前，该会话能够保持空闲的秒数。对于非交互式客户端，需要使用wait_timeout变量来代替。

join_buffer_size（启动：直接设置；作用范围：全局、会话；动态）

它指的是连接（它们是在未使用索引的情况下执行的，并且需要进行全表扫描）和特定类型的索引扫描所使用的缓冲区的最小大小。

keep_files_on_create（启动：直接设置；作用范围：全局、会话；动态）

如果在创建MyISAM表的CREATE TABLE 语句里明确给出了DATA DIRECTORY 或INDEX DIRECTORY 选项，并且服务器在给定的目录里分别找到了一个已有的数据文件或索引文件，那么它会返回一个错误。当没有使用DATA DIRECTORY或INDEX DIRECTORY 选项来指定数据文件或索引文件的存放位置时，keep_files_on_create 变量会控制服务器如何创建 MyISAM表。如果此变量的值为 OFF（默认值），并且服务器找到了一个已有的.MYD数据文件或.MYI索引文件，那么它会覆盖该文件。如果此变量的值为ON，服务器会返回一个错误。

key_buffer_size（启动：直接设置；作用范围：全局；动态）

它指的是用于缓存MyISAM表索引块的缓冲区大小。这个缓冲区在各个会话之间是共享的。此变量和其他的键缓存变量（key_cache_age_threshold、key_cache_block_size和key_cache_limit）是以分组的形式存在的，并且可以作为结构化系统变量的组成部分进行访问。为了更好地控制键缓存的使用，可以创建多个键缓存。更多相关信息请参考12.7.2节。

key_cache_age_threshold（启动：直接设置；作用范围：全局；动态）

它指的是，在被移到暖子链之前，未使用的缓冲块在MyISAM 键缓存的热子链里可以保留多久。值越高，允许缓冲块在热子链里保留的时间越长。默认值为300。最小值为100。

key_cache_block_size（启动：直接设置；作用范围：全局；动态）

它指的是MyISAM键缓存的块大小。默认情况下，一个缓存块的大小为1024字节。

key_cache_limit（启动：直接设置；作用范围：全局；动态）

如果此变量被设置为它的默认值100，则MyISAM键缓存将使用“最近最少使用”策略来解决缓存缓冲区的重用问题。如果设置的值小于 100，则键缓存将使用“中间插入”策略，而此变量的值即为暖缓冲子链使用的键缓存百分比值。此变量的取值范围是1~100。

language（启动：直接设置；作用范围：全局）

此变量已过时。请使用lc_messages和lc_messages_dir来代替。

large_files_support（作用范围：全局）

它指的是，MySQL服务器是否能够处理大文件。

`large_page_size`（作用范围：全局）

如果启用了大页面支持，则此变量指的是大内存页的大小。如果未启用，则其值为0。

`large_pages`（启动：直接设置；作用范围：全局）

它指的是是否启用了大内存页支持功能。只有Linux系统才支持大页面。

`last_insert_id`（作用范围：会话；动态）

设置此变量，可以指定`LAST_INSERT_ID()`的返回值。它用于二进制日志的处理。

`lc_messages`（启动：直接设置；作用范围：全局、会话；动态）

出错信息的本地化语言。服务器会将本地化名字转换为语言名字，并在`lc_messages_dir`变量指定的目录里查找该语言名字所对应的目录，以便找到错误信息文件的位置。请参考12.6.3节。

`lc_messages_dir`（启动：直接设置；作用范围：全局）

存放出错信息文件的目录。

`lc_time_names`（启动：直接设置；作用范围：全局、会话；动态）

它指的是函数`DATE_FORMAT()`、`DAYTIME()`和`MONTHNAME()`显示日期和月份名字所使用的本地化语言。默认值为`en_US`，但是也可以设置为POSIX风格的语言名称，如`es_AR`（西班牙语/阿根廷）或`zh_HK`（中文/中国香港）。更多相关信息，请参考12.6.4节。

`license`（作用范围：全局）

它指的是服务器所使用的许可类型。例如，如果服务器遵从的是GNU GPL（通用公共许可证）条款，则其值为GPL。

`local_infile`（启动：直接设置；作用范围：全局；动态）

它指的是服务器是否允许在`LOAD DATA`语句里使用`LOCAL`修饰符。

`lock_wait_timeout`（启动：直接设置；作用范围：全局、会话；动态）

它指的是等待每个元数据锁的秒数。如果语句阻塞的时间超过了等待锁的时间，就会出错。允许的取值范围是1~31 536 000（默认为1年）。此变量是在MySQL 5.5.3里引入的。

`locked_in_memory`（启动：--memlock；作用范围：全局）

服务器是否锁定在内存里。

`log`（作用范围：全局）

此变量已在MySQL 5.6里被弃用并删除。请使用`general_log`来代替。

`log_bin`（作用范围：全局）

是否启用二进制日志记录功能。请注意，--log-bin选项设置的是`log_bin_basename`，而非`log_bin`。

`log_bin_basename`（作用范围：全局）

二进制日志文件的完整路径名。此变量是在MySQL 5.6.2里引入的。

`log_bin_index`（作用范围：全局）

二进制日志索引文件的名字。此变量是在MySQL 5.6.2里引入的。

`log_bin_trust_function_creators`（启动：直接设置；作用范围：全局；动态）

如果要创建或修改存储函数，则必须拥有相应的CREATE ROUTINE权限或ALTER ROUTINE权限。不过，如果启用了二进制日志记录功能，并且`log_bin_trust_function_creators`的值为0（默认值），那么还必须拥有SUPER权限，并且声明该存储函数是确定性的（即不会修改任何数据）。如果想禁用这些附加的要求，可以将此变量值设置为1。

`log_error`（启动：直接设置；作用范围：全局）

错误日志文件的名字。如果不设置此变量，服务器会把出错信息输出到控制台终端。如果在服务器启动时指定了此变量，但未设置具体值，则日志文件名为数据目录里的HOSTNAME.err，其中，HOSTNAME为服务器主机的名字。如果其文件名是以相对路径形式给出的，则服务器会将它解释为相对于数据目录。如果在指定文件名时没有带扩展名，mysqld会添加一个扩展名.err。

`log_output`（启动：直接设置；作用范围：全局；动态）

如果启用了普通查询日志和慢查询日志，则此变量指定的是它们的输出目标。此变量列出的是一组以逗号分隔的输出目标名。允许目标包括：`TABLE`、`FILE`和`NONE`。如果指定了此变量，则`NONE`值会禁用日志记录功能，并且它比其他的值优先级更高。默认值为`FILE`。设置系统变量`general_log`或`slow_query_log`之后，可以启用或是禁用相应的日志功能。设置系统变量`general_log_file`或`slow_query_log_file`，可以指定相应日志文件的名称。

`log_queries_not_using_indexes`（启动：直接设置；作用范围：全局；动态）

如果启用了慢查询日志，则此变量将控制那些不使用索引的查询是否应该被记录。如果进行记录，将使得日志文件迅速变得较大。请参考`log_throttle_queries_not_using_indexes`。

`log_slave_updates`（启动：直接设置；作用范围：全局）

对于复制从服务器从其主服务器的二进制日志里接收到的更新，此变量控制着从服务器是否把这些更新记录到它自己的二进制日志里。默认情况下，从服务器的更新记录功能是禁用的。但是在链式复制设置里，允许将一个从服务器当作另一个从服务器的主服务器。

`log_throttle_queries_not_using_indexes`（启动：直接设置；作用范围：全局；动态）

如果启用了`log_queries_not_using_indexes`，则它指的是每分钟最多向慢查询日志里写入的语句数量。默认值为0，表示“无限制”。此变量是在MySQL 5.6.5里引入的。

`log_slow_queries`（启动：直接设置；作用范围：全局）

此变量已在MySQL 5.6里被弃用并删除。请使用`slow_query_log`来代替。

`log_warnings`（启动：直接设置；作用范围：全局、会话；动态）

它指的是把非关键性警告信息记录到出错日志的记录级别。如果此变量的值为0，则不记录这些警告消息；如果值为1（默认值），则启用记录。如果此变量的值大于1，则会提高日志记录的级别，以包括那些与异常中止连接和“拒绝访问”错误有关的信息。

`long_query_time`（启动：直接设置；作用范围：全局、会话；动态）

此变量值的单位为秒。如果某个查询命令的执行时间大于了这个值（以及检查的记录会超过`min_examined_row_limit`行），则它会被认为是一个“慢”查询，并且会导致状态变量`Slow_queries` 增加。此外，如果已启用慢查询日志记录功能，则服务器会将该查询写入该日志。

最小值和默认值分别为0和10。该值可以包含小数部分（即微秒）。不过，只有日志目标是一个文件，而不是`mysql.slow_query`表时，才会记录小数部分。

`low_priority_updates`（启动：直接设置；作用范围：全局、会话；动态）

如果启用此变量，则对于使用表级锁定机制的存储引擎，更新操作的优先级将低于检索操作。那些会修改表内容的语句（`DELETE`、`INSERT`、`REPLACE`、`UPDATE`）将一直等待，直到表上没有`SELECT`语句正在执行或者正在等待执行。如果有新的`SELECT`语句到来时，另一条`SELECT`语句还在执行，那么新语句会立即执行，而不是等待低优先级的修改语句。对于那些支持`LOW_PRIORITY`选项的语句（如`INSERT`和`UPDATE`），启用此变量与指定该选项的效果一样。对于单独的`INSERT`语句，可以通过`HIGH_PRIORITY`选项来取消此变量的效果，并把插入操作恢复到正常的优先级。

`lower_case_file_system`（作用范围：全局）

此变量用来表明包含数据目录的文件系统是否区分文件名的大小写。如果此变量的值为`ON`，则文件名不区分大小写。（即可以把`ON`的含义理解为文件名的大写和小写版本都是一样的。）如果此变量的值为`OFF`，则文件名区分大小写。

`lower_case_table_names`（启动：直接设置；作用范围：全局）

此变量控制的是，在执行`CREATE DATABASE`和`CREATE TABLE` 语句时，如何处理数据库名和表名所对应的目录名和文件名。当执行这些语句时，它还控制着服务器会如何进行名字比较。下表列出了各种可能的取值。

取值	含义
0	按照CREATE DATABASE和CREATE TABLE语句里给出的名字来创建磁盘文件。名字的比较会区分大小写。在文件名区分大小写的系统里，它是默认设置
1	在创建数据库和表时，名字强制转换为小写形式。名字的比较不区分大小写
2	以小写形式保存名字，但是在进行比较时不区分大小写。也就是说，创建的文件名与那些CREATE语句里指定的一样，但是不以区分大小写的形式进行比较。只有在文件名不区分大小写的系统里才会使用此变量的值

当没有显式设置lower_case_table_names变量时，如果数据目录所在的文件系统不区分文件名的大小写，那么 MySQL服务器将自动把此变量设置为2。将此变量设置为非零值，也会使得表的别名不区分大小写。

master_info_repository（启动：直接设置；作用范围：全局；动态）

此变量表示的是，从服务器是将主服务器的日志信息写入文件，还是写入表。如果该值为FILE（默认值），则从服务器日志文件由--master-info-file 选项指定。如果该值为TABLE，则服务器会把日志记录到mysql.slave_master_info表中。此变量是在MySQL 5.6.2里引入的。

master_verify_checksum（启动：直接设置；作用范围：全局；动态）

此变量表示的是，在主服务器把从二进制日志里读取到的事件发送到从服务器之前，它是否验证这些事件的校验和。默认值为OFF。此变量是在MySQL 5.6.2里引入的。

max_allowed_packet（启动：直接设置；作用范围：全局、会话；动态）

它指的是服务器与客户端之间通信时使用的那个缓冲区的最大大小。该缓冲区的初始大小为net_buffer_length个字节，但是会根据需要增大到max_allowed_packet个字节。这个值也会限制MySQL服务器所能处理的字符串的最大长度。此变量的默认值和最大值分别是1 MB和1 GB。它的会话值是只读的。

max_binlog_cache_size（启动：直接设置；作用范围：全局；动态）

它指的是二进制日志缓存的最大大小。在事务被提交之前，构成事务的那些语句都会保存在二进制日志缓存里，它们要等到事务被提交时才会被写

入二进制日志。如果某个事务超出了此变量所设定的最大长度，就必须被刷新到一个临时硬盘文件里。那些影响非事务语句的语句会被单独缓存；请参考max_binlog_stmt_cache_size。

max_binlog_size（启动：直接设置；作用范围：全局；动态）

它指的是二进制日志文件的最大大小。如果当前二进制日志文件达到了此大小，则MySQL服务器将关闭它，并开始使用下一个文件。此变量的取值范围为 4 KB~1 GB。默认值为1 GB。

如果max_relay_log_size被设置为0，那么max_binlog_size变量也会对从服务器的中继日志文件的大小进行限制。

max_binlog_stmt_cache_size（启动：直接设置；作用范围：全局；动态）

此变量与 max_binlog_cache_size 相似，但是仅限于那些影响非事务表的语句（它们会被单独缓存起来）。此变量是在MySQL 5.5.9里引入的。

max_connect_errors（启动：直接设置；作用范围：全局；动态）

它指的是，主机在被阻止继续进行连接之前，允许连接失败的次数。这是为了防止有人从该主机试图攻击数据库系统。如果想清空这个主机缓存，并重新启用被阻塞的主机，可以使用FLUSH HOSTS 语句或者 mysqladmin flush-hosts 命令。请参考状态变量Connect_errors_xxx。

max_connections（启动：直接设置；作用范围：全局；动态）

它指的是客户端连接的最大并发数。默认值为151。

max_delayed_threads（启动：直接设置；作用范围：全局、会话；动态）

它指的是为处理 INSERT DELAYED 语句而允许创建的最大线程数。当达到最大处理器数时，所有新接收到的这类语句都将被当作非 DELAYED 语句。客户端可以把此变量的值设置为0，以便在自己的会话里禁用DELAYED插入。

max_error_count（启动：直接设置；作用范围：全局、会话；动态）

它指的是服务器可以保存的出错信息、警告信息和提醒信息的最大数量。（这些信息总是会被统计。此变量只会影响相关信息的存储数量，并且可供SHOW ERRORS和SHOW WARNINGS使用。）

`max_heap_table_size`（启动：直接设置；作用范围：全局、会话；动态）

它指的是新建MEMORY表的最大允许长度。更改此变量的值不会影响那些已有表，除非使用ALTER TABLE或者TRUNCATE TABLE语句来修改它们。此变量可以用于防止服务器使用过多的内存。此变量还会对服务器如何处理其内部使用的各种内存表产生影响；请参考对`tmp_table_size`变量的描述。

`max_insert_delayed_threads`（启动：使用`--max-delayed-threads` 选项；作用范围：全局、会话；动态）

此变量等同于`max_delayed_threads`。

`max_join_size`（启动：直接设置；作用范围：全局、会话；动态）

当执行多表连接时，MySQL 优化器会估算它需要检查多少个行组合。如果估算值超过了`max_join_size` 行，则会出现一个错误。如果你习惯于编写会返回大量行的SELECT 语句，则可以使用此变量。这个限制不会应用于存储在查询缓存里的查询结果，因为缓存里的查询结果不需再次执行该查询即可被返回。

此变量需要与 `sql_big_selects` 变量一起使用，详见对该变量描述的讨论。将`max_join_size`设置为DEFAULT以外的某个值，会自动将`sql_big_selects`设置为0。

`max_length_for_sort_data`（启动：直接设置；作用范围：全局、会话；动态）

查询优化器将使用此变量来确定应该为ORDER BY操作执行哪一种filesort动作。

`max_prepared_stmt_count`（启动：直接设置；作用范围：全局；动态）

它指的是服务器可以同时维护的预处理语句的最大数。将此变量的值设置得更小一点，可将其用于限制服务器占用的内存。此变量的可取值范围是0～1 000 000，默认值为16 382。当将其值设置为0时，可以完全禁用预处理语句。

`max_relay_log_size`（启动：直接设置；作用范围：全局；动态）

它指的是从服务器中继日志文件的最大长度。如果当前中继日志文件达到了此大小，则 MySQL服务器将关闭它，并开始使用下一个文件。如果此变

量的值是 0，则服务器将使用`max_binlog_size` 变量来控制中继日志文件的最大长度。此变量的取值范围为4 KB~1GB；默认值为0。

`max_seeks_for_key`（启动：直接设置；作用范围：全局、会话；动态）

查询优化器在进行基于键的查找时会使用此变量。如果某个索引的差异度比较低（只有很少量的唯一值），那么优化器可能会认为在执行键查找操作时需要进行多次查找，从而会采用全表扫描的方式来代替。把此变量设置成某个小一点的值，则会使优化器认为在执行键查找时最多只需要指定数量的索引寻找，从而使其更倾向于选择使用索引，而不会执行全表扫描。

`max_sort_length`（启动：直接设置；作用范围：全局、会话；动态）

使用每个数据值的前 `max_sort_length`个字节来进行排序。默认值为1024。如果被排序值在这么多的字节内是唯一的，那么将此变量设置为更小一点的值可以获得更少的比较次数，同时也不会损失精确度。反之，如果被排序值在这么多的字节内不是唯一的，那么增加这个值可以获得更好的区分效果。

`max_sp_recursion_depth`（启动：直接设置；作用范围：全局、会话；动态）

它指的是每个存储过程的最大递归深度。默认值为0（不允许递归），最大值为255。

`max_tmp_tables`（启动：直接设置；作用范围：全局、会话；动态）

此变量未被使用。

`max_user_connections`（启动：直接设置；作用范围：全局、会话；动态）

它指的是每个账户同时进行连接的客户端最大个数。默认值为0，表示“无限制”。并发连接数还会受到`max_connections`变量的全局限制。

此变量的会话值是只读的。它的会话值与全局值是相等的，除非账户在`user`表里的行拥有一个非零的`MAX_USER_CONNECTIONS`值。此时，该会话值将从账户的行里获得。

如果需要为某个特定的账户指定连接限制，可以使用`GRANT`语句。

`max_write_lock_count`（启动：直接设置；作用范围：全局；动态）

在对某个表使用了`max_write_lock_count`个写锁定之后，对于那些正在等待获得该表读锁的语句，服务器将适当提升它们的优先级。

`metadata_locks_cache_size`（启动：直接设置；作用范围：全局）

它指的是服务器为元数据锁所使用的那个缓存的大小。此变量是在MySQL 5.5.19里引入的。

`min_examined_row_limit`（启动：直接设置；作用范围：全局、会话；动态）

一个查询至少需要检查`min_examined_row_limit`个行才被允许记录到慢查询日志。默认值为0。

`myisam_data_pointer_size`（启动：直接设置；作用范围：全局；动态）

它指的是MyISAM索引文件的行指针的字节大小。此变量的取值范围是2~7，默认为6。

对于个别的表，这个指针大小会受到指定的那个`MAX_ROWS`表选项的影响。

`myisam_max_sort_file_size`（启动：直接设置；作用范围：全局；动态）

对于`REPAIR TABLE`、`ALTER TABLE`或`LOAD DATA`等语句，在对MyISAM表进行重建时，既可以使用一个临时文件，也可以使用键缓存。具体使用哪一种方法由这个变量的值来决定。如果临时文件的大小可能会大于这个值，则需要使用键缓存。

`myisam_mmap_size`（启动：直接设置；作用范围：全局）

它指的是压缩后的MyISAM 表文件在进行内存映射时使用的最大内存量。此变量是在MySQL 5.5.1里引入的。

`myisam_recover_options`（启动：直接设置；作用范围：全局）

它指的是自动修复 MyISAM 表时使用的模式。如果启用它，那么当服务器打开某个MyISAM 表时，如果该表被标记为崩溃或者在上次使用时未正确关闭，那么它会尝试修复该表。这个变量的值应该为一个以逗号分隔的列表，其中可以带有这样一些选项：`BACKUP`（如果修复操作会更改这个表，则先为它创建一个备份）、`FORCE`（即使丢失了多行数据也要强制进行恢复）、

QUICK（快速恢复）、DEFAULT（在恢复时不进行任何其他特殊的处理）或者OFF（不进行恢复；这是默认值）。

如果在运行服务器时启用了系统变量`delay_key_write`，或者对单个MyISAM表进行了配置，启用了延迟索引写入功能，那么启用表的自动修复功能是种不错的做法。

`myisam_repair_threads`（启动：直接设置；作用范围：全局、会话；动态）

它指的是在修复操作过程中用来创建MyISAM表索引的线程个数。默认值为1，表示的是使用单个线程进行修复。将此变量的值设置为大于1，实现多线程修复——此功能仍处于体验阶段。

`myisam_sort_buffer_size`（启动：直接设置；作用范围：全局、会话；动态）

在执行类似ALTER TABLE、CREATE INDEX和REPAIR TABLE这样的操作期间，需要分配缓冲区，以便对MyISAM表的索引进行排序。此变量指定的便是该缓冲区的大小。

`myisam_stats_method`（启动：直接设置；作用范围：全局、会话；动态）

此变量表示的是，服务器在为MyISAM表统计其索引键的分布概率时，应该把NULL值视为相同，还是视为不同。可设置的值包括有：`nulls_equal`（所有的NULL值在同一个组里）、`nulls_unequal`（每个NULL值单独成为一组）或`nulls_ignored`（忽略NULL值）。

`myisam_use_mmap`（启动：直接设置；作用范围：全局；动态）

它指的是服务器在读写MyISAM表时是否需要使用内存映射。默认值为OFF。

`named_pipe`（启动：直接设置）

它指的是服务器是否允许客户端使用命名管道连接。只有Windows支持这种连接方式。默认值为OFF。

`net_buffer_length`（启动：直接设置；作用范围：全局、会话；动态）

它指的是服务器与客户端程序进行通信时使用的连接和结果缓冲区的初始大小。此缓冲区可以扩展到`max_allowed_packet`个字节大小。此变量的取

值范围是1 KB~1 MB；默认为16 KB。它的会话值是只读的。

`net_read_timeout`（启动：直接设置；作用范围：全局、会话；动态）

它指的是在连接出现超时之前等待客户端发送数据的秒数。

`net_retry_count`（启动：直接设置；作用范围：全局、会话；动态）

它指的是中断的读取操作被重试的次数。

`net_write_count`（启动：直接设置；作用范围：全局、会话；动态）

它指的是在连接出现超时之前将数据写入客户端所需要等待的秒数。

`new`（启动：直接设置；作用范围：全局、会话；动态）

此变量未被使用。

`old`（启动：直接设置；作用范围：全局）

它是一个用于解决兼容问题的变量，可以启用某些较旧的行为特性。目前，它会导致索引提示无法应用到ORDER BY或GROUP BY子句。

`old_alter_table`（启动：直接设置；作用范围：全局、会话；动态）

对于某些ALTER TABLE操作（在MySQL 5.0里，它们会需要表的临时副本），当前服务器并不需要临时副本。启用此变量可以让服务器使用临时副本。

`old_passwords`（启动：直接设置；作用范围：全局、会话；动态）

从MySQL 4.1版开始，MySQL服务器开始支持一种较以前更为安全的密码散列方法。仍然支持那些拥有旧风格散列密码的已有账户，但是默认情况下此变量是禁用的，并且新的密码会使用新的散列方法。启用此变量可以使密码散列操作使用4.1 版本之前的旧方法。这个变量可能会对服务器降级或将账户移动到旧服务器有好处。

`open_files_limit`（启动：直接设置；作用范围：全局）

此变量指的是 MySQL服务器试图保留的文件描述符的个数。如果在服务器启动时将此变量设置为一个非零值，但是发现服务器显示的实际值却小于指定值，那么该实际值表示的是操作系统所允许的文件描述符的最大数。

（如果服务器显示的值为 0，则表示操作系统不允许mysqld更改文件描述符

的个数。) 如果在服务器启动时没有设置此变量或是将其设置为0, 那么MySQL服务器会把`max_connections × 5`和`max_connections + table_open_cache × 2`这两者当中较大的那个数作为保留的文件描述符个数。`open_files_limit`变量独立地控制着文件描述符的分配, 与`innodb_open_files`变量控制的文件描述符无关。

`optimizer_prune_level` (启动: 直接设置; 作用范围: 全局、会话; 动态)

查询优化器将分析多个执行计划并从中选出一个最佳的。此变量决定着优化器如何处理中间计划。如果此变量的值是 1 (默认值), 那么优化器将只对各候选计划将要检查的行个数进行估算, 并根据估算结果丢弃中间计划。如果此变量被设置为 0, 则优化器将在所有计划中执行穷举搜索。

`optimizer_search_depth` (启动: 直接设置; 作用范围: 全局、会话; 动态)

此变量控制着查询优化器搜索执行计划的深度。如果此变量的值是 0, 则优化器将自动选择一个合理的深度值。默认行为是执行穷举搜索。

`optimizer_switch` (启动: 直接设置; 作用范围: 全局、会话; 动态)

此变量控制的是可以被关闭或打开的优化器策略。其值为一个以逗号隔开的`flag=value`设置列表, 其中, `value`的值可以为`on`或`off`。你也可以将标志设置为`default`, 以表示不管其默认值为何值都需要返回它; 或者将`optimizer_switch` 自己设置为`default`, 将所有的标志都恢复成默认值。对于各种策略的描述请参考MySQL参考手册。

`optimizer_trace_xxx` (启动: 直接设置; 作用范围: 全局、会话)

这些以`optimizer_trace`作为开头的系统变量主要用于跟踪查询优化器的内部操作。这些变量是在MySQL 5.6.3里引入的。

`performance_schema_xxx`

这些以 `performance_schema` 作为开头的系统变量主要是与搜索和分析服务器性能数据所使用的那个Performance Schema有关。更多信息请参考MySQL参考手册。

`pid_file` (启动: 直接设置; 作用范围: 全局)

在`mysqld` 启动时, 它会将其进程ID (即PID) 写入一个文件里。此变量指定的是那个PID文件的路径名。此文件可能被其他进程用于确定服务器的进

程编号，通常是用于向服务器发送信号。例如，当mysql.server向服务器发送关闭信号时，会读取这个文件。如果其文件名是以相对路径形式给出的，服务器会将它解释为相对于数据目录。默认为host_name.pid。

plugin_dir（启动：直接设置；作用范围：全局）

它指的是存放插件的那个目录的路径名。

port（启动：直接设置；作用范围：全局）

它指的是服务器用来监听客户端连接的TCP/IP端口号。

preload_buffer_size（启动：直接设置；作用范围：全局、会话；动态）

此变量决定着MySQL服务器在使用LOAD INDEX语句预加载有关索引时会分配一个多大的缓冲区。

protocol_version（作用范围：全局）

服务器所使用的客户端/服务器协议的版本号。

proxy_user（作用范围：全局）

这是一个只读的会话变量。在启用了代理功能时，它表示的是代理用户；否则为 NULL。此变量是在MySQL 5.5.7里引入的。

pseudo_thread_id（作用范围：会话；动态）

此变量仅供MySQL服务器内部使用。

query_alloc_block_size（启动：直接设置；作用范围：全局、会话；动态）

它指的是在分析和执行SQL语句时分配的临时内存块大小。

query_cache_limit（启动：直接设置；作用范围：全局；动态）

它指的是查询结果的最大缓存大小，超过这一大小的查询结果将不会被缓存。默认大小为1MB。

query_cache_min_res_unit（启动：直接设置；作用范围：全局；动态）

它指的是为把查询结果存入查询缓存里而分配的内存块大小。默认大小为4 KB。

query_cache_size（启动：直接设置；作用范围：全局；动态）

它指的是用来缓存查询结果的内存大小。当把此变量设置为 0 时，即使 query_cache_type 不为 OFF，也要禁用查询缓存。反之，如果把此变量设置为一个非零值，那么即使query_cache_type的值为OFF，也要强行分配那么多的内存。取值为1024的倍数。

query_cache_type（启动：直接设置；作用范围：全局、会话；动态）

当query_cache_size的值大于0时，它指的是查询缓存的操作模式。下表列出了各种可能的取值。

模式	含义
0	不缓存查询结果，也不检索缓存结果
1	将可缓存的查询缓存起来，那些以SELECT SQL_NO_CACHE作为开头的查询除外
2	根据需要只将那些以SELECT SQL_CACHE作为开头的可缓存查询缓存起来

如果使用SET语句来设置query_cache_type变量，那么符号值OFF、ON和DEMAND分别对应于模式0、模式1和模式2。

query_cache_wlock_invalidate（启动：直接设置；作用范围：全局、会话；动态）

当此变量为0（默认值）时，即使某个客户对某个表进行了WRITE锁定，其他客户端也能检索该表已被缓存的查询结果。如果把此变量设置为 1，那么在某个客户端对某个表获得WRITE锁期间，该表已被缓存的查询结果将变得不可用，其他客户端必须等待该锁被释放。

query_prealloc_size（启动：直接设置；作用范围：全局、会话；动态）

它指的是为分析和执行 SQL 语句而分配的那个缓冲区的长度。与那些由 query_alloc_block_size变量控制的内存分配块不同，这个缓冲区在前后两条语句之间不会被释放。

rand_seed1和rand_seed2（作用范围：会话；动态）

它们是复制RAND()函数时内部使用的只读会话变量。

`range_alloc_block_size` (启动: 直接设置; 作用范围: 全局、会话; 动态)

它指的是在进行范围优化时分配的内存的块大小。

`read_buffer_size` (启动: 直接设置; 作用范围: 全局、会话; 动态)

它指的是对表进行顺序扫描的那个线程所使用的缓存区的大小。缓冲区会根据每个客户端的需要进行分配。

`read_only` (启动: 直接设置; 作用范围: 全局; 动态)

此变量控制的是从服务器是否会以只读方式来处理客户连接。默认情况下, `read_only`的值为OFF, 此时, 客户端的更新都会以平常的方式(即它们都有权限这样做)被接受。如果把此变量设置为 ON, 那么只有从主服务器那里接收到的语句(对于从服务器)或者拥有SUPER权限的客户端所执行的语句才能执行更新操作; SET PASSWORD要求SUPER权限。`read_only`不会应用于TEMPORARY表。

如果你明确地拥有表锁或者还有尚未执行完毕的事务, 那么你不能启用此变量。在其他客户端拥有表锁或是还有尚未执行完毕的事务时, 如果你试图激活此变量, 那么你的请求将被阻塞, 一直到这些锁被释放或是事务执行完毕为止。当你的请求被阻塞时, 如果其他客户端试图要获得新的表锁或是试图开始新的事务, 那么它们也将被阻塞。这些阻塞条件对FLUSH TABLES WITH READ LOCK语句没有作用, 因为它获得的是全局读锁, 不是表锁。

`read_rnd_buffer_size` (启动: 直接设置; 作用范围: 全局、会话; 动态)

它指的是在对行进行排序之后, 顺序读取各行时使用的那个缓冲区的大小。缓冲区会根据每个客户端的需要进行分配。

`relay_log` (启动: 直接设置; 作用范围: 全局)

中继日志文件的名字。

`relay_log_basename` (作用范围: 全局)

中继日志文件的完整路径名。此变量是在MySQL 5.6.2里引入的。

`relay_log_index` (作用范围: 全局)

中继日志索引文件的名字。

`relay_log_info_file`（启动：直接设置；作用范围：全局）

中继日志信息文件的名字；默认为`relay-log.info`。

`relay_log_info_repository`（启动：直接设置；作用范围：全局；动态）

此变量表示的是，从服务器是将中继日志信息写入文件，还是写入表。如果该值为 `FILE`（默认值），那么从服务器会把日志记录到`--relay-log-info-file` 选项所指定的文件里。如果该值为 `TABLE`，那么服务器会把日志记录到 `mysql.slave_relay_log_info_file`表里。此变量是在MySQL 5.6.2里引入的。

`relay_log_purge`（启动：直接设置；作用范围：全局；动态）

如果此变量被设置为 1（默认值），从服务器会在用完一个中继日志文件之后立刻删除它。如果把此变量设置为0，中继日志文件将不会被自动删除。

`relay_log_recovery`（启动：直接设置；作用范围：全局）

此变量在从服务器崩溃之后非常有用。在启动时启用它，可以使从服务器删除所有的还未处理的中继日志，并再次从主服务器获取它们。此变量默认是禁用的。

`relay_log_space_limit`（启动：直接设置；作用范围：全局）

全体中继日志文件的最大允许大小。当达到这个限制时，从服务器的I/O线程会等待SQL线程处理更多的事件，并通过删除日志文件的方式释放空间。

`report_host`、`report_password`、`report_port`、`report_user`（作用范围：全局）在从复制服务器上，可以设置这些变量，以表示从服务器进行连接时应该报告给主服务器的那些值。它们不需要与从服务器在建立连接时实际使用的那些值相对应。当`SHOW SLAVE HOSTS` 语句在主服务器上执行时，这些值会显示在该语句的输出里。只有在主服务器启动时带有`--show-slave-auth-info` 选项，才会报告 `report_user`和`report_password`的值。

`secure_auth`（启动：直接设置；作用范围：全局；动态）

它表示的是，服务器是否只允许那些拥有 MySQL 4.1 里引入的密码格式的用户进行连接。如果此变量被设置为`OFF`，那么服务器将允许那些使用4.1

版本之前的旧密码格式的账户进行连接。在MySQL 5.6.5之前，此变量的默认值为OFF；之后的默认值为ON。

`secure_file_priv`（启动：直接设置；作用范围：全局）

如果把此变量设置为目录路径名，服务器将只允许 `LOAD DATA`和`SELECT ... INTO OUTFILE` 语句以及 `LOAD_FILE()` 函数对该目录里的文件进行操作。默认情况下，此变量的值为空（没有上述限制）。

`server_id`（启动：直接设置；作用范围：全局；动态）

MySQL服务器的复制ID编号。如果编号值为0，则表示该服务器不参与复制；否则，这个值必须是一个 $1 \sim 2^{32}-1$ 的整数。每一个服务器的复制ID编号都必须是唯一的。

`server_uuid`（作用范围：全局）

服务器自动生成的UUID值。此变量是在MySQL 5.6.0里引入的。

`shared_memory`（启动：直接设置）

它指的是服务器是否允许客户端使用共享内存连接。只有 Windows 支持这种连接方式。默认值为OFF。

`shared_memory_base_name`（启动：直接设置）

它指的是用来建立共享内存连接的共享内存的名字。默认名字为MYSQL（区分大小写）。

`skip_external_locking`（启动：直接设置；作用范围：全局）

它指的是外部锁定机制（即文件系统级的锁定机制）是否被禁止。

`skip_name_resolve`（启动：直接设置；作用范围：全局）

此变量默认是禁用的。启用它之后，可以禁止主机名解析，并且在权限表里必须通过 IP 地址或使用localhost来指定这些主机。

`skip_networking`（启动：直接设置；作用范围：全局）

如果禁用此变量（默认值），则服务器允许使用TCP/IP连接。如果启用它，则会禁用TCP/IP连接。客户端只可以从本地主机进行连接，且必须使用非 TCP/IP接口。Unix客户端可以使用Unix套接字文件。Windows客户端可以使用共享内存或命名管道（如果这些连接类型被启用了的话）连接。

`skip_show_databases`（启动：直接设置；作用范围：全局）

如果禁用此变量（默认值），则所有用户都可以执行 SHOW DATABASES 语句。如果用户拥有SHOW DATABASE 权限，则它可以看到所有的数据库；否则，用户只能看到它自己权限能够访问的那些数据库。如果启用此变量，则只有拥有 SHOW DATABASE 权限的用户才能使用SHOW DATABASES语句，并且它会显示所有的数据库。

`slave_allow_batching`（启动：直接设置；作用范围：全局；动态）

启用从服务器的批请求功能。它仅适用于MySQL集群。

`slave_checkpoint_group`（启动：直接设置；作用范围：全局；动态）

如果启用了多线程从服务器执行（请参考`slave_parallel_threads`），则此变量控制的是从服务器在获得一个检查点之前执行的最大事务数。默认值为 512。允许的取值范围为32~512 KB。此变量是在MySQL 5.6.3里引入的。检查点依赖于`slave_checkpoint_group`和`slave_checkpoint_period`；超过任何一个限制都会导致从服务器回到检查点，并重置与它们两个关联在一起的那些计数器。

`slave_checkpoint_group`（启动：直接设置；作用范围：全局；动态）

如果启用了多线程从服务器执行（请参考`slave_parallel_threads`），则此变量控制的是从服务器在获得一个检查点之前执行的最大时间（单位为毫秒）。默认值为 300。允许的取值范围为 1~4 GB。此变量是在MySQL 5.6.3 里引入的。请参考 `slave_checkpoint_group`。

`slave_compressed_protocol`（启动：直接设置；作用范围：全局；动态）

此变量决定着是否要对从服务器和主服务器之间的通信进行压缩。这需要主、从服务器都支持使用压缩协议才行。

`slave_load_mode`（启动：直接设置；作用范围：全局；动态）

它指的是从服务器的执行模式，取值可以为 STRICT（默认）或 IDEMPOTENT。后面那个设置对于涉及循环或多个主服务器的复制拓扑可能很有用，可用于防止出现“键重复”和“键未找到”的错误。

`slave_load_tmpdir`（启动：直接设置；作用范围：全局）

如果服务器是一个复制从服务器，那么它指的是用于处理LOAD DATA 语句的那个目录的路径名。此变量的默认值是系统变量`tmpdir`的值。

`slave_max_allowed_packet`（启动：直接设置；作用范围：全局；动态）

与`max_allowed_packet`类似，但它适用于从服务器的SQL线程和I/O线程。默认大小为1GB。

`slave_net_timeout`（启动：直接设置；作用范围：全局；动态）

它指的是在连接出现超时之前等待主服务器发送数据的秒数。

`slave_parallel_workers`（启动：直接设置；作用范围：全局；动态）

在从服务器上，将此变量设置为大于 0，可以启用多线程复制。通常情况下会执行从从服务器上接收到的所有事件的那个SQL 线程，实际上会变成执行这些事件的那个工作线程`slave_parallel_workers`的坐标。默认值为0。允许的取值范围为0~1024。此变量是在MySQL 5.6.3里引入的。

`slave_pending_jobs_size_max`（启动：直接设置；作用范围：全局；动态）

如果启用了从服务器多线程执行（请参考 `slave_parallel_threads`），则此变量控制的是工作线程用于缓存未处理事件的最大内存量。默认大小为16 MB。允许的取值范围为1 KB~18 EB。此变量是在MySQL 5.6.3里引入的。

`slave_skip_errors`（启动：直接设置；作用范围：全局）

此变量用于指定一个错误列表；在该列表里的错误出现时，从服务器会忽略它们，而不是将复制过程挂起。（不过，与利用这个选项来忽略错误的做法相比，还是找出问题的根源并彻底解决更好。）如果此变量的值为`all`，则会忽略所有的错误。否则，此变量的值应该是以逗号分隔的一个或者多个出错编号。

`slave_sql_verify_checksum`（启动：直接设置；作用范围：全局；动态）

它指的是从服务器的SQL 线程是否应该验证中继日志里读取到的事件的校验和。如果校验和错误，复制过程会停止。默认值为`OFF`。此变量是在MySQL 5.6.2里引入的。

`slave_transaction_retries`（启动：直接设置；作用范围：全局；动态）

它指的是从服务器重试失败事务的次数。事务失败的原因包括：死锁或者已经到达存储引擎的超时限制。默认值为10。

`slave_type_conversions`（启动：直接设置；作用范围：全局；动态）

这个变量表示的是在从服务器上进行基于行的复制时允许的各种类型转换。默认值为空串（即不允许转换）。当其值为非空时，它应该是一个以逗号分隔的值列表，其中的值为一个或多个`ALL_LOSSY`（允许丢失信息的转换）或`ALL_NON_LOSSY`（允许那些不丢失信息的转换）。此变量是在MySQL 5.5.3里引入的。

`slow_launch_time`（启动：直接设置；作用范围：全局；动态）

它指的是创建“慢”线程所占用的秒数。任何线程的创建时间超过这个值时，都会导致状态变量`Slow_launch_threads`的值加1。

`slow_query_log`（启动：直接设置；作用范围：全局；动态）

它用于指定是否要启用慢查询日志记录。如果要启用，则 `log_output` 变量会控制日志的输出目标。

`slow_query_log_file`（启动：直接设置；作用范围：全局；动态）

它指的是慢查询日志文件的名称。在启用了FILE日志目标时，会用到它。它的默认值是数据目录里的`HOSTNAME-slow.log`文件，其中，`HOSTNAME`是服务器主机的名称。如果其文件名是以相对路径形式给出的，则服务器会将其解释为相对于数据目录。

`socket`（启动：直接设置；作用范围：全局）

它指的是Unix域套接字的路径名或Windows中命名管道的名称。

`sort_buffer_size`（启动：直接设置；作用范围：全局、会话；动态）

供那些用来完成排序操作（GROUP BY 或ORDER BY）的线程使用的缓冲区的大小。如有必要，每个客户端都能分配到一个这样的缓冲区。一般来说，如果你有许多个客户端会在同一时间进行排序操作，就不应该把这个值设置得很大（超过1 MB）。

`sql_auto_is_null`（作用范围：全局、会话；动态）

如果此变量设置为1，那么最近生成的`AUTO_INCREMENT`值可以选择使用`WHERE col_name IS NULL`形式的子句来进行查询，其中，`col_name`是`AUTO_INCREMENT`列的名称。有些ODBC 程序会使用此功能。如果要禁用它，可以将此变量设置为 0。默认是 0（在MySQL 5.5.3之前默认为1）。

`sql_big_selects`（作用范围：全局、会话；动态）

服务器会把此变量与系统变量`max_join_size`一起使用。如果`sql_big_selects`为1（默认），服务器会接受那些返回任意大小结果集的查询。如果`sql_big_selects`为0，服务器会拒绝那些有可能返回大量行的查询。在这种情况下，执行连接操作时，会使用`max_join_size`的值：服务器会对它需要检查的行组合数量进行估算，如果该值超过了`max_join_size`的值，那么服务器将返回一个错误，而不会执行该查询。

将max_join_size设置为DEFAULT以外的某个值，会自动将sql_big_selects设置为0。

sql_buffer_result（作用范围：全局、会话；动态）

将此变量设置为1，会导致服务器使用内部临时表来保存SELECT语句的结果。带来的效果是，服务器可以快速地释放由生成结果的那个表所占用的锁。默认值为0。

sql_log_bin（作用范围：全局、会话；动态）

将这个变量的会话值设置为0或1，可以禁用或启用当前客户端会话的二进制日志记录。在运行时，客户端必须拥有 SUPER 权限才能更改这个变量，即使是更改其会话值也是如此。该变量只有在启用二进制日志记录的情况下才会发挥作用。

sql_log_off（作用范围：全局、会话；动态）

将这个变量的会话值设置为0或1，可以启用或禁用当前客户端会话的普通查询日志记录。在运行时，客户端必须拥有 SUPER 权限才能更改这个变量，即使是更改其会话值也是如此。该变量只有在启用普通查询日志记录的情况下才会发挥作用。

sql_mode（启动：直接设置；作用范围：全局、会话；动态）

服务器的SQL模式。这个变量将改变 MySQL服务器的某些行为，使它更符合 SQL语言标准或是与其他数据库服务器或老版本的MySQL服务器保持兼容。此变量的值应该是一个空串（这将清除以前设置的SQL模式）或者是由下面将要介绍的一个或多个模式值以逗号分隔而构成的一系列值。自MySQL 5.6.6起，其默认值为NO_ENGINE_SUBSTITUTION；而对于之前的版本，其值为空串。有些模式值很简单，它们可以单独使用以启用某种特定的行为。其他的则是复合模式，每种复合SQL模式涵盖多种简单SQL模式，这使得用户可以方便地一次设置多种SQL模式。模式值不区分大小写。

术语“严格模式”（strict mode）指的是这样一种 sql_mode 设置：它启用了STRICT_TRANS_TABLES和STRICT_ALL_TABLES，以便让服务器在对数据进行检查时更加严格。本书的3.3节对严格模式和其他几个会对输入数据处理产生影响的SQL模式进行了详细的讨论。

下面列出了一些简单的SQL模式值。

ALLOW_INVALID_DATES

在严格模式里，禁止对DATE和DATETIME值进行全面的日期有效性检查。唯一的要求是月份的取值范围必须为1～12，日期的取值范围必须为1到31。但TIMESTAMP值是个例外：不管是否启用了这个SQL模式，它们都必须都是有效的。

ANSI_QUOTES

把双引号字符解释为标识符（如数据库名、表名和列名）使用的引号字符，而不是供字符串使用的引号字符。（不管这个模式是否启用，都可以用反引号来当作名字的引号字符。）

□ ERROR_FOR_DIVISION_BY_ZERO

对于行插入或修改操作，即使是在严格模式下，以零为除数的除法（或求余数）运算得到的结果通常是 NULL，且不会返回任何警告信息。启用ERROR_FOR_DIVISION_BY_ZERO模式将更改这种行为。如果没有启用严格模式，那么以零为除数时结果将为NULL值，但会返回一条警告信息；如果启用了严格模式，那么在执行INSERT或UPDATE语句期间遇到以零为除数的情况时，将产生一个错误，并且该语句会失败。如果想要禁止在插入或更新时出现错误，并且产生一个NULL值和警告信息，可以使用INSERT IGNORE或UPDATE IGNORE。

HIGH_NOT_PRECEDENCE

此模式可以更改NOT操作符的优先级，使其与!操作符的优先级相同。

IGNORE_SPACE

让服务器忽略内建函数名与引入参数表的那个左括号“(”之间的空格。默认情况下，那个左括号必须紧跟在函数名的后面，其中间不允许有间隔。此模式会使函数名被当作保留字。

NO_AUTO_CREATE_USER

防止GRANT语句创建不安全的新账户。也就是说，如果某个账户不存在，那么GRANT会执行失败，并且不会创建账户，除非该语句包含一个指定有非空密码的IDENTIFIED BY子句，或者包含有一个指定身份验证插件的IDENTIFIED WITH子句。

NO_AUTO_VALUE_ON_ZERO

通常情况下，把0插入一个AUTO_INCREMENT列，等效于插入NULL值：MySQL将自动生成下一个序列编号，并把它保存在该列里。如果启用了此模

式，那么往AUTO_INCREMENT列里插入0将会把数字0存入该列。

NO_BACKSLASH_ESCAPES

如果启用了这个模式，那么反斜线字符（“\”）将被当作一个没有特殊含义的普通字符，而不是当作一个字符串的转义字符。

NO_DIR_IN_CREATE

忽略CREATE TABLE和ALTER TABLE语句里的DATA DIRECTORY和INDEX DIRECTORY表选项。

NO_ENGINE_SUBSTITUTION

当服务器遇到带有ENGINE选项的CREATE TABLE或ALTER TABLE语句时，该模式所指定的存储引擎不可用——此时，这个模式便决定着服务器将如何处理它们。如果启用了此模式，那么会出现一个错误，并且该表不会被创建（或更改）。如果禁用了此模式，那么允许替换为默认的存储引擎。

NO_FIELD_OPTIONS

让SHOW CREATE TABLE语句的输出不包含MySQL独有的与列有关的选项，其目的是为了这条语句的输出有更好的可移植性。

NO_KEY_OPTIONS

让SHOW CREATE TABLE语句的输出不包含MySQL独有的与索引有关的选项，其目的是为了这条语句的输出有更好的可移植性。

NO_TABLE_OPTIONS

让SHOW CREATE TABLE语句的输出不包含MySQL独有的与表有关的选项，其目的是为了这条语句的输出有更好的可移植性。

NO_UNSIGNED_SUBTRACTION

默认情况下，如果在参与整数减法运算的两个操作数里有一个是无符号整数，那么计算结果就将是一个无符号整数。启用这个模式，可以使上述运算的结果是一个带符号整数——这样做，可以与MySQL 4.0版之前的行为保持兼容。

NO_ZERO_DATE

在严格模式下，拒绝接受'0000-00-00'作为一个有效日期值。通常情况下，MySQL允许存储“零”日期值。这个模式可以通过使用INSERT IGNORE语句代替INSERT语句的办法来覆盖。

NO_ZERO_IN_DATE

在严格模式下，拒绝接受月或日部分是零的日期值。（年份是零的日期值是允许的。）通常情况下，MySQL允许存储这样的日期值。在非严格模式下，或者如果用户发出的是INSERT IGNORE语句，MySQL将把这样的日期值保存为'0000-00-00'。

ONLY_FULL_GROUP_BY

通常情况下，MySQL允许 SELECT 语句的输出列列表里带有非聚合型列，或者使用不是列在GROUP BY子句里的HAVING子句。例如：

```
SELECT a, b, COUNT(*) FROM t GROUP BY a;
```

ONLY_FULL_GROUP_BY标志要求非聚合型输出列（或HAVING列）都被列在GROUP BY里：

```
SELECT a, b, COUNT(*) FROM t GROUP BY a, b;
```

PAD_CHAR_TO_FULL_LENGTH

通常情况下，服务器在检索CHAR列值时会删除其尾部空格。如果启用了这个模式，MySQL服务器将禁止删除CHAR列的尾部空格，这样，检索到的值的长度就等于列的定义宽度。

PIPES_AS_CONCAT

如果启用了这个模式，那么||将被解释为字符串连接操作符，而不会解释为逻辑或。

REAL_AS_FLOAT

如果启用了这个模式，那么数据类型REAL将与FLOAT同义，而不是等同于DOUBLE。

STRICT_ALL_TABLES

如果启用了这个模式，那么所有的存储引擎都将对输入数据做更严格的检查，这将导致MySQL拒绝接受绝大多数非法值。如果想要更加严格，可以使

用TRADITIONAL模式。

STRICT_TRANS_TABLES

如果启用了这个模式，那么事务型存储引擎将对输入数据值做严格的检查，这将导致MySQL 拒绝接受绝大多数非法值。在此基础上，只要有可能（例如，遇到插入单个行的INSERT语句），非事务型存储引擎也将对输入数据做严格的检查。如果想要更加严格，可以使用TRADITIONAL模式。

下表列出的是复合SQL模式，以及每种复合模式所包含的模式内容。

复合模式	组成模式
ANSI	ANSI_QUOTES, IGNORE_SPACE, PIPES_AS_CONCAT, REAL_AS_FLOAT
DB2	ANSI_QUOTES, IGNORE_SPACE, NO_FIELD_OPTIONS, NO_KEY_OPTIONS, NO_TABLE_OPTIONS, PIPES_AS_CONCAT
MAXDB	ANSI_QUOTES, IGNORE_SPACE, NO_AUTO_CREATE_USER, NO_FIELD_OPTIONS, NO_KEY_OPTIONS, NO_TABLE_OPTIONS, PIPES_AS_CONCAT
MSSQL	ANSI_QUOTES, IGNORE_SPACE, NO_FIELD_OPTIONS, NO_KEY_OPTIONS, NO_TABLE_OPTIONS, PIPES_AS_CONCAT
MYSQL323	HIGH_NOT_PRECEDENCE, NO_FIELD_OPTIONS
MYSQL40	HIGH_NOT_PRECEDENCE, NO_FIELD_OPTIONS
ORACLE	ANSI_QUOTES, IGNORE_SPACE, NO_AUTO_CREATE_USER, NO_FIELD_OPTIONS, NO_KEY_OPTIONS, NO_TABLE_OPTIONS, PIPES_AS_CONCAT
POSTGRESQL	ANSI_QUOTES, IGNORE_SPACE, NO_FIELD_OPTIONS, NO_KEY_OPTIONS, NO_TABLE_OPTIONS, PIPES_AS_CONCAT
TRADITIONAL	ERROR_FOR_DIVISION_BY_ZERO, NO_AUTO_CREATE_USER, NO_ZERO_DATE, NO_ZERO_IN_DATE, STRICT_ALL_TABLES, STRICT_TRANS_TABLES

之所以称之为TRADITIONAL 模式，是因为它启用这样的模式——它们使得MySQL在处理输入值时，可以表现得像那些会拒绝无效数据的传统数据库一样。它有点像严格模式，但是对于更加严格的检查又包含了几个附加的约束。

sql_log_off（作用范围：全局、会话；动态）

将此变量设置为0或1（默认值），可以控制服务器是禁止还是记录Note级的警告。

sql_quote_show_create（作用范围：全局、会话；动态）

此变量控制的是，在语句SHOW CREATE TABLE和SHOW CREATE DATABASE的输出结果里是否需要将标识符（数据库名、表名、列名和索引名）用引号引起来。默认为 1（即需要使用引号）。将此变量设置为 0，可以不要求使用引号。当生产用于其他数据库服务器的CREATE TABLE语句时，不使用引号会带来好处。如果禁用引号，则一定要确保表没有使用保留字或者没有包含特殊字符。

如果禁用了 SQL 模式 ANSI_QUOTES，引用标识符将使用反撇号（`）；否则，会使用双引号（"）。

`sql_safe_updates`（作用范围：全局、会话；动态）

如果将此变量设置为1，那么只有这些被修改的行是通过键值来确定的，或者使用了LIMIT子句，服务器才会接受UPDATE和DELETE。当使用默认值0时，无这种限制。

`sql_select_limit`（作用范围：全局、会话；动态）

此变量指定的是SELECT语句返回的最大行数。如果显式地给出了LIMIT子句，那么该子句的优先级要高于此变量。默认值为每个表所能容纳的最大行数。如果你曾经更改过此变量的值，可以使用值DEFAULT来恢复其默认值。

对于存储例程或那些不会把行返回给客户端的SELECT操作（如子查询、INSERT INTO ... SELECT和CREATE TABLE ... SELECT语句），此变量没有任何作用。

`sql_slave_skip_counter`（作用范围：全局；动态）

拥有SUPER权限的用户可以把此变量当作GLOBAL变量设置为n，告诉复制从服务器跳过接下来从其主服务器接收到的n个事件。如果最后的位置位于一组事件（如某个事务的一组事件）的中间，那么从服务器还会跳过该组里的其余事件。

`sql_warnings`（作用范围：全局、会话；动态）

如果将此变量设置为 1，那么 MySQL 会报告警告统计信息（即使对于单个行的插入操作也会报告）。对于默认值0，则只会针对插入多个行的INSERT语句报告警告统计信息。

`ssl_xxx`

`ssl_xxx` 变量表示的是与服务器启动时给定的各个`--ssl-xxx` 选项相对应的值。（例如，`ssl_ca`表示的是`--ssl-ca`选项的值。）如果在启动服务器时

没有给定--ssl-xxx选项，那么相应的ssl_xxx 变量的值将为空串。如果不支持使用SSL，则这些变量的值都将是NULL。

storage_engine（作用范围：全局、会话；动态）

自MySQL 5.5.3起，它已被弃用。请使用default_storage_engine来代替。

stored_program_cache（启动：直接设置；作用范围：全局；动态）

它指的是服务器会为每个连接缓存多少存储例程。默认值为256。此变量是在MySQL 5.5.21里引入的。

sync_binlog（启动：直接设置；作用范围：全局；动态）

当把此变量设置为0时（默认值），服务器不会把二进制日志刷新到磁盘。当此变量被设置为一个正整数n的时候，服务器每对二进制日志进行n次写操作就会把日志刷新到磁盘。在这种情况下，系统发生崩溃时，此变量的值越小，有关数据的安全系数越大，但这同时对系统性能的负面影响也越大。

sync_frm（启动：直接设置；作用范围：全局；动态）

当启用此变量时（默认值），服务器会在数据创建时将每个非临时表的.frm 文件刷新到磁盘。

sync_master_info（启动：直接设置；作用范围：全局；动态）

对于从服务器，如果这个变量为0（默认值），那么从服务器不会强制将它的master.info文件同步到磁盘。实际上，正常的文件系统会执行刷新操作。如果该值大于 0，那么从服务器会在每个事件组处理完之后将该文件同步到磁盘。

sync_relay_log（启动：直接设置；作用范围：全局；动态）

对于从服务器，如果这个变量为 0（默认值），那么从服务器不会强制将其中继日志同步到磁盘。实际上，正常的文件系统会执行刷新操作。如果该值大于 0，那么从服务器会在每个写入组处理完之后将中继日志同步到磁盘。如果启用了自动提交，那么每条语句都会发生一次写入操作；否则，是每个事务发生一次写入操作。

sync_relay_log_info（启动：直接设置；作用范围：全局；动态）

此变量与sync_master_info很像，不同之处在于它处理的是relay-log.info文件。

system_time_zone（作用范围：全局）

服务器的系统时区。服务器在它启动时会尝试通过询问操作系统的办法来确定此变量的值。要想显式地设置这个值，可以设置 TZ 环境变量，或是为 mysqld_safe 脚本指定--timezone选项。

table_definition_cache（启动：直接设置；作用范围：全局；动态）

它指的是服务器在其定义缓存里可以存放的表定义（来自.frm文件）个数。

table_open_cache（启动：直接设置；作用范围：全局；动态）

它指的是服务器能够同时打开的表的最大个数。

table_open_cache_instances（启动：直接设置；作用范围：全局）

它指的是用于对打开的表缓存进行分区的实例数量。默认为 1（即不分区）。在高负荷情况下，将缓存分开可能会减少会话之间的资源竞争。此变量是在MySQL 5.6.6里引入的。

thread_cache_size（启动：直接设置；作用范围：全局；动态）

它指的是线程缓存所能容纳的最大线程数。当客户端断开与MySQL服务器的连接时，如果这个缓存还没有满，那么客户端曾经使用过的线程就将被放入这个缓存。只要这个缓存里还有线程，新建的连接就会重复使用它们而不是去创建新的线程。当MySQL服务器为每个当前连接的客户端分别使用一个线程的时候，就要用到线程缓存。

thread_concurrency（启动：直接设置；作用范围：全局）此变量已过时。

thread_handling（启动：直接设置；作用范围：全局）

它指的是服务器用于处理客户端连接的线程模型。此变量的值可以为no-threads（单个连接线程）或one-thread-per-connection（每个当前连接的客户端使用一个线程）。默认为one-thread-per-connection。

thread_stack（启动：直接设置；作用范围：全局）

它指的是每个线程的栈大小。

`time_format`（作用范围：全局）

此变量未被使用。

`time_zone`（启动：使用`--default-time-zone`选项；作用范围：全局、会话；动态）

它指的是MySQL服务器的当前时区。如果此变量的值为SYSTEM，则MySQL服务器将使用`system_time_zone` 变量的值作为它的当前时区。客户端可以通过修改此变量的会话值，设置其自己会话的时区。

`timestamp`（作用范围：会话；动态）

通过设置此变量，可以指定当前会话的TIMESTAMP 值。它用于二进制日志的处理。`timestamp`的值会对`NOW()`的返回值产生影响，但是不会影响到`SYSDATE()`的返回值。

`tmp_table_size`（启动：直接设置；作用范围：全局、会话；动态）

它指的是MySQL内部使用的各种临时表（即服务器在处理SQL语句的过程中自动创建的表）的最大允许长度。如果某个临时表的长度超过了`max_heap_table_size`和`tmp_table_size`当中较小的那个值，那么服务器会把它从内部内存表转换为MyISAM表，保存到磁盘上。如果有足够多的内存的话，那么增大此变量的值可以使服务器在内存里维护更大的临时表，而不必把它们转换为磁盘文件格式。

`tmpdir`（启动：直接设置；作用范围：全局）

它指的是服务器用于创建临时文件的那个目录的路径名。此变量的值可以是一个目录列表，它们将轮换着使用。在Unix里，目录名之间使用冒号（:）隔开；在Windows里，目录名之间则需要用分号（;）隔开。

`transaction_alloc_block_size`（启动：直接设置；作用范围：全局、会话；动态）

它指的是为处理这样一些语句而分配的临时内存的块大小：在事务提交之时，在将事务写入二进制日志之前，需要存储为事务一部分的语句。

`transaction_prealloc_size`（启动：直接设置；作用范围：全局、会话；动态）

它指的是为处理构成某个事务的语句而分配的那个缓冲区的大小。与那些由 `trans-action_alloc_block_size` 变量控制的内存分配块不同，这个缓冲区在前后两条语句之间不会被释放。

`tx_isolation`（启动：使用`--transaction-isolation`选项；作用范围：全局、会话；动态）

它指的是事务隔离的默认级别。

`tx_read_only`（启动：使用`--transaction-read-only`选项；作用范围：全局、会话；动态）

默认事务访问模式是否为只读。默认值为OFF（即允许读/写）。此变量是在MySQL 5.6.5里引入的。

`unique_checks`（作用范围：全局、会话；动态）

将此变量设置为 0或 1，将分别禁用或启用对 InnoDB表里的辅助索引进行唯一性检查。当把数据导入 InnoDB表时，禁用这些检查可以提升性能，但是只有事先知道这些数据值不会与唯一性需求冲突时，才可以这样做。

`updatable_views_with_limit`（启动：直接设置；作用范围：全局、会话；动态）

当禁用此变量时，服务器会禁止对那些在其底层表里没有使用主键的视图进行更新操作（即执行UPDATE 或DELETE 语句），即使更新操作里包含有LIMIT 1 子句，把更新范围限制在了单个行里也不行。当启用此变量时（默认值），服务器会允许更新操作，并产生一条警告信息。

`version`（作用范围：全局）

它指的是 MySQL服务器的版本。此变量的值由一个版本编号以及（可能）一个或多个后缀构成，如`-log`用于表明启用日志记录；`-debug`用于表明服务器正运行在调试模式下。

`version_comment`（作用范围：全局）

它指的是在配置服务器时指定给CMake的`-DWITH_COMMENT` 选项的值。如果在配置时没有指定任何注释，则此变量的默认值为"Source distribution"。

`version_compile_machine`（作用范围：全局）

它指的是MySQL的编译机器（即硬件类型）。这个值是在构建MySQL时的配置阶段确定的。

`version_compile_os`（作用范围：全局）

它指的是MySQL的编译操作系统。这个值是在构建MySQL时的配置阶段确定的。

`wait_timeout`（启动：直接设置；作用范围：全局、会话；动态）

此变量指的是，在服务器自己认为可以将非交互式客户端关闭之前，该客户端能够保持空闲的秒数。对于交互式客户端，需要使用 `interactive_timeout` 变量的值来代替。

`warning_count`（作用范围：会话）

这是一个只读会话变量，它表示的是，最后那条语句所产生的诸如错误、警告和注意等信息的数量。

InnoDB系统变量

下面这些系统变量都与InnoDB存储引擎有关。

`ignore_builtin_innodb`（作用范围：全局）

此变量未被使用。

`innodb_adaptive_flushing`（启动：直接设置；作用范围：全局；动态）

它表示的是，InnoDB 是否会试图通过使用工作负载水平更改缓冲池里脏页刷新频率的方式来避免I/O爆炸。默认值为ON。

`innodb_adaptive_flushing_lwm`（启动：直接设置；作用范围：全局；动态）

它指的是会使 InnoDB启用自适应刷新功能的那个重做日志容量的最低百分比点数。默认值为10。此变量是在MySQL 5.6.6里引入的。

`innodb_adaptive_hash_index`（启动：直接设置；作用范围：全局；动态）

它指的是，InnoDB 是否要使用自适应哈希索引。也就是说，它是否要监视索引查找，并动态构建散列索引，如果这样做，则有可能提高性能。默认值为ON。

`innodb_adaptive_max_sleep_delay`（启动：直接设置；作用范围：全局；动态）

它指的是，InnoDB是否要使用工作负载水平来调整 `innodb_thread_sleep_delay`。当值为 0 时，会禁止调整。非零值最大可达 1 000 000，它指定的是 InnoDB 调整 `innodb_thread_sleep_delay`的上限值（单位为微秒）。默认值为 150000。此变量是在MySQL 5.6.3里引入的。

`innodb_additional_mem_pool_size`（启动：直接设置；作用范围：全局）

它指的是InnoDB存储引擎用来存放各种内部数据结构的内存池的大小。

`innodb_autoextend_increment`（启动：直接设置；作用范围：全局；动态）当某个自扩展表空间快要被填满时，InnoDB 存储引擎将自动使用此变量的值作为递增量去增加那个表空间的大小。指定值的单位为MB。此变量的默认值是8，最大值是1000。

`innodb_autoinc_lock_mode`（启动：直接设置；作用范围：全局）

此变量控制的是InnoDB用于生成AUTO_INCREMENT值的锁定算法。允许取值包括0、1（默认值）和2。一般情况下，当有多个事务需要同时生成自动增量值时，使用这些值可以增加锁定的扩展性、获得更好的并发性，并减少阻塞。对于会生成多个这种值的多行INSERT语句，模式0和1可以保证生成的值在该语句内部是连续的。模式2无法保证这种连续性：因为它允许在语句之间交叉分配各个值，虽然这样对于某个给定的语句生成的值是单调，但是它们不一定连续。这也会对复制产生影响。模式0和1对基于语句的复制来讲是安全的；但是模式2分配的值是不确定的，因此该模式是不安全的。（所有的模式对基于行的复制都是安全的。）

`innodb_buffer_pool_dump_at_shutdown`（启动：直接设置；作用范围：全局；动态）它指的是，InnoDB 是否会在关闭时将缓冲池转储至磁盘。默认值为 OFF。此变量是在MySQL 5.6.3里引入的。

`innodb_buffer_pool_dump_now`（启动：直接设置；作用范围：全局；动态）

将此变量设置为ON，会使得InnoDB立即将缓冲池转储至磁盘。默认值为OFF。此变量是在MySQL 5.6.3里引入的。

`innodb_buffer_pool_filename`（启动：直接设置；作用范围：全局；动态）

它指的是 InnoDB 用于缓冲池转储和加载操作的那个文件的名称。默认为数据目录里的`ib_buffer_pool`文件。此变量是在MySQL 5.6.3里引入的。

`innodb_buffer_pool_instances`（启动：直接设置；作用范围：全局）

它指的是，如果`innodb_buffer_pool_size`的值至少为1 GB，则需要将InnoDB缓冲池划分为`innodb_buffer_pool_instances`个区域。此变量的默认值是1（单个缓冲池），最大值是 64。为达到最好的效果，需要对`innodb_buffer_pool_size`和`innodb_buffer_pool_instances`的值进行选择，以便每个实例都至少为 1 GB。此变量是在MySQL 5.5.4里引入的。

`innodb_buffer_pool_load_abort`（启动：直接设置；作用范围：全局；动态）

将此变量设置为ON，会导致InnoDB中断当前正从磁盘加载的所有缓冲池。默认值为OFF。此变量是在MySQL 5.6.3里引入的。

`innodb_buffer_pool_load_at_startup`（启动：直接设置；作用范围：全局）

它指的是，InnoDB是否会在启动时从磁盘加载缓冲池。默认值为OFF。此变量是在MySQL 5.6.3里引入的。

`innodb_buffer_pool_load_now`（启动：直接设置；作用范围：全局；动态）

将此变量设置为 ON，会导致 InnoDB 立即从磁盘加载缓冲池。默认值为OFF。此变量是在MySQL 5.6.3里引入的。

`innodb_buffer_pool_size`（启动：直接设置；作用范围：全局）

它指的是InnoDB用于缓冲表数据和索引的那个缓存的大小。默认大小为128 MB。

`innodb_change_buffer_max_size`（启动：直接设置；作用范围：全局；动态）

它指的是InnoDB为更改缓冲区预留的缓冲池最大百分比。默认值为25。允许的取值范围为0~50。此变量是在MySQL 5.6.2里引入的。

innodb_change_buffering（启动：直接设置；作用范围：全局；动态）

它指的是，InnoDB 缓冲表会如何更改辅助索引的延迟写操作。通过将写操作缓冲的方式，InnoDB 有时可以把它们分成一组，然后顺序地写入，而不是以随机方式写入；这样有利于提高的I/O性能。下表列出了各种可能的取值。

取值	含义
all	缓存所有的更改操作；这个是默认值
none	不缓存更改操作
changes	缓存插入和删除操作
deletes	缓存那些把索引记录标记为删除的更改操作
inserts	缓存插入操作
purges	缓存那些被“垃圾回收器”标记为删除的清理操作

innodb_checksum_algorithm（启动：直接设置；作用范围：全局；动态）

InnoDB 会在其表空间的每一个块里写入校验和值。此变量控制的是用于计算校验和值的算法。下表列出了各种可能的取值。strict_xxx的值与它们的对应变量的很相似，主要差异在于：InnoDB在表空间里遇到多种校验和类型时会停止。

取值	含义
none, strict_none	校验值为常量
crc32, strict_crc32	使用CRC32算法
innodb, strict_innodb	默认算法

此变量是在MySQL 5.6.3里引入的。

`innodb_checksums`（启动：直接设置；作用范围：全局）

它指的是，InnoDB表是否启用检验计算。默认值为ON。自MySQL 5.6.3起，此变量已被弃用，请使用`innodb_checksum_algorithm`。

`innodb_commit_concurrency`（启动：直接设置；作用范围：全局；动态）

此变量表示的是可以同时提交多少个线程。当该值为0时（默认值），表示的是“无限制”。

`innodb_concurrency_tickets`（启动：直接设置；作用范围：全局；动态）

当某个线程想进入 InnoDB时，只有当线程数量小于 `innodb_commit_concurrency` 变量设置的上限时它才能成功。否则，该线程将排队等候直到线程的数量降低到那个上限以下。一旦线程被允许进入，它将可以不受限制地离开和重新进入 InnoDB，这种自由往返的最大次数由 `innodb_concurrency_tickets`变量的值决定。默认值为500。

`innodb_data_file_path`（启动：直接设置；作用范围：全局）

它指定的是InnoDB表空间组件的文件。与该变量值的格式有关的讨论请参考12.5.3.1节。

`innodb_data_home_dir`（启动：直接设置；作用范围：全局）

它指的是相对于 InnoDB表空间组件文件存放位置的目录路径名。如果此变量为空，则组件文件名将被解释为绝对路径名。

`innodb_doublewrite`（启动：直接设置；作用范围：全局）

它指的是，InnoDB是否启用了双写缓冲区。默认值是ON。

`innodb_fast_shutdown`（启动：直接设置；作用范围：全局；动态）

当此变量的值为0或1时，它表示的是InnoDB是否使用其较快速的关机方法（它会跳过某些正常情况下会执行的操作）。当取值为2时，InnoDB会刷新其日志，然后停止。

`innodb_file_format`（启动：直接设置；作用范围：全局；动态）

如果启用了`innodb_file_per_table`，则它指的是InnoDB新表所使用的格式。默认格式为Antelope；另一个允许值为Barracuda。使用Barracuda可以启用不被Antelope支持的功能，如COMPRESSED行格式。

`innodb_file_format_check`（启动：直接设置；作用范围：全局）

InnoDB 系统表空间包含有一个标志，它表示的是表空间里使用的最高版本的文件格式。此变量会在服务器启动时设置，主要用于控制 InnoDB是否要检查这个标志，以确定此格式版本比 InnoDB支持的那个版本更高。如果启用此变量（默认值），并且该格式版本更高，那么启动会失败，并产生一个错误。如果该格式版本不够高，那么 InnoDB 会将`innodb_file_format_max`设置成该格式。

`innodb_file_format_max`（启动：直接设置；作用范围：全局；动态）

请参考`innodb_file_format_check`的描述。

`innodb_file_io_threads`（启动：直接设置）

此变量未被使用。

`innodb_file_per_table`（启动：直接设置；作用范围：全局；动态）

如果此变量被设置为0（默认值），InnoDB将在其系统表空间里创建新表。如果此变量被设置为1，InnoDB 将为每个新表分别创建一个独立表空间：在数据库目录里为每一个新表单独创建一个`.ibd`文件来存放该表的内容。在这种情况下，系统表空间只会用于InnoDB数据目录条目，而不会用于数据或索引存储。

此变量只影响InnoDB如何创建新表。不管如何更改此变量的值，InnoDB存储引擎总是可以访问已经在系统表空间或独立表空间里创建的表。

`innodb_flush_log_at_trx_commit`（启动：直接设置；作用范围：全局；动态）

这个变量控制着InnoDB日志的刷新行为。下表列出了各种可能的取值。

取值	含义
0	每秒写入一次日志，并刷新到磁盘一次
1	每提交一次写入一次日志，并刷新到磁盘一次
2	每提交一次写入一次日志，但是每秒刷新到磁盘一次

只有在将此变量设置为1（默认值）时，InnoDB才能保证ACID特性。否则，如果出现崩溃的情况，那么会付出大约1秒的代价，即最近的事务会丢失。如果将此变量设置为0，则可以减少 InnoDB将缓存数据保存到磁盘的次数。不过，如果出现崩溃，则这样会大大增加潜在丢失最近提交事务的风险。

`innodb_flush_method`（启动：直接设置；作用范围：全局）

此变量给出的是 InnoDB 用来刷新文件的方法。它只适用于 Unix 系统。可取值包括：`fdatsync`（使用`fsync()`来刷新数据文件和日志文件）、`O_DSYNC`（使用`fsync()`来刷新数据文件，并使用`O_SYNC`来打开和刷新日志文件）和`O_DIRECT`（使用`fsync()`来刷新数据文件和日志文件，视情况选用`O_DIRECT`或`directio()`来打开数据文件）。默认值为`fdatsync`。在Windows里，此变量的值总是为`async_unbuffered`。

`innodb_flush_neighbors`（启动：直接设置；作用范围：全局；动态）

它表示的是，InnoDB 是单独刷新缓冲池的脏面，还是连同位于同一范围（页面组）内的相邻页面一起刷新。刷新相邻页面，可以将写操作结合在一起，减少磁盘设备旋转过程中的寻道时间开销。这个变量是在MySQL 5.6.3里引入的，是一个布尔量，其默认值为 `ON`。自MySQL 5.6.6起，这个变量成为了三态变量，允许的值包括：0（不刷新相邻页面）、1（刷新相邻页面）和2（刷新同一范围里的所有相邻页面）。

`innodb_force_load_corrupted`（启动：直接设置；作用范围：全局）

它表示的是，InnoDB 在启动时是否加载已标记为损坏的表。默认值为 `OFF`，因此这些表通常情况下会被忽略。如果启用此变量，则可能允许恢复那些在禁用此变量时无法访问的数据；在恢复之后，可以在禁用此变量的情况下重启服务器。更多相关信息请参考14.7.4节。此变量是在MySQL 5.5.18里引入的。

`innodb_force_recovery`（启动：直接设置；作用范围：全局）

此变量的值通常为0，但是可以被设置为1~6的某个值，以便让服务器在崩溃后即使InnoDB恢复失败也可以再次启动。与如何使用此变量有关的信息请参考14.7.4节。

`innodb_ft_xxx`

在MySQL 5.6.4之前，对全文搜索的支持仅限于MyISAM表。这些以`innodb_ft`作为开头的系统变量都与InnoDB表的全文搜索支持有关。该项功能是在MySQL 5.6.4引入的。更多信息，请参考MySQL参考手册。

`innodb_io_capacity`（启动：直接设置；作用范围：全局；动态）

它指的是InnoDB对于后台任务每秒执行I/O操作次数的近似限制。默认值为200，最小值为100。对于慢速旋转的磁盘，可能需要将这个值调低一点。对于SSD磁盘，可以将其适当调高。请参考`innodb_io_capacity_max`。

`innodb_io_capacity_max`（启动：直接设置；作用范围：全局；动态）

如果`innodb_io_capacity`的值在紧急情况下不够高，那么`innodb_io_capacity_max`会成为InnoDB可以将该限制扩展到的最大值。其默认值为`innodb_io_capacity`默认值的两倍，它受限制于服务器所使用的最低值2000。此变量是在MySQL 5.6.6里引入的。

`innodb_large_prefix`（启动：直接设置；作用范围：全局；动态）

InnoDB索引的最大索引前缀长度通常是767字节。如果启用此变量，那么对于那些使用COMPRESSED或DYNAMIC行格式的表，允许前缀最高达到3072个字节。默认值为OFF。此变量是在MySQL 5.5.14里引入的。

`innodb_lock_wait_timeout`（启动：直接设置；作用范围：全局、会话；动态）

它指的是等待某个事务锁的秒数。如果无法获得该锁，则InnoDB会回滚该事务。

`innodb_locks_unsafe_for_binlog`（启动：直接设置；作用范围：全局）

此变量会对InnoDB如何处理索引行锁定产生影响，但是自MySQL 5.6.3起，它已被弃用，因此请不要再使用它。为获得与此变量相同的效果（非严格的锁定），但是更加灵活，可以针对特定的会话或事务，使用SET TRANSACTION将事务隔离级别设置为READ COMMITTED。更多详细信息请参考2.12.3节。

`innodb_log_buffer_size`（启动：直接设置；作用范围：全局）

它指的是InnoDB事务日志缓冲区的大小。此变量的取值范围通常是1 MB～8 MB；默认为1 MB。

`innodb_log_file_size`（启动：直接设置；作用范围：全局）

它指的是每个InnoDB 日志文件的大小。`innodb_log_file_size`和`innodb_log_files_in_group`的乘积决定了InnoDB日志的总大小。

`innodb_log_files_in_group`（启动：直接设置；作用范围：全局）

它指的是InnoDB维护的日志文件个数。`innodb_log_file_size`和`innodb_log_files_in_group`的乘积决定了InnoDB日志的总大小。

`innodb_log_group_home_dir`（启动：直接设置；作用范围：全局）

它指的是InnoDB应该将其日志文件写入的那个目录的路径名。

`innodb_lru_scan_depth`（启动：直接设置；作用范围：全局；动态）

InnoDB 会使用一个后台操作来查找需要从其缓冲池刷新到磁盘的脏页。这个变量控制的是这个操作能够查看到的页面列表（按最近最少使用的顺序排序）的长度。对默认值 1024的合理更改包括：对于拥有繁重写操作工作负载和大型缓冲池的服务器，可以减小这个值；而对于I/O能力还有盈余的服务器，可以增加这个值。此变量是在MySQL 5.6.3里引入的。

`innodb_max_dirty_page_pct`（启动：直接设置；作用范围：全局；动态）

它指的是，在InnoDB认为需要将日志刷新到磁盘之前，InnoDB允许脏页占用其缓冲池的百分比。允许的取值范围是0～100；默认值为90。

`innodb_max_dirty_pages_pct_lwm`（启动：直接设置；作用范围：全局；动态）

它指的是，InnoDB 为降低脏页的占用比例，而启用预刷新功能所参考的缓冲池脏页最低占用百分比。默认值为 0，表示禁止预刷新。允许的取值范围为 0～99。此变量是在MySQL 5.6.6里引入的。

`innodb_max_purge_lag`（启动：直接设置；作用范围：全局；动态）

InnoDB 维护着一个清除线程，它可以清除删除操作或更新操作标记的待删除的行。如果小批量行被插入和删除的频率差不多相同，那么清除线程可能会落后，进而导致大量待删除行不能及时清除，而占用着本应该释放的空间。`innodb_max_purge_lag` 变量控制着如何延迟 INSERT、DELETE 和 UPDATE 语句，让它们适当放慢速度以便清除线程能够追上它们的进度。默认为 0（即不延迟）。如果把此变量设置为一个非零值，那么延迟大约是 $((n/\text{innodb_max_purge_lag}) \times 10) - 5$ 毫秒，其中的 n 是在执行过程中会把一些行标记为待删除的事务的数量。

`innodb_max_purge_lag_delay`（启动：直接设置；作用范围：全局；动态）

它指的是可以由 `innodb_max_purge_lag` 产生的延迟间隔（单位为毫秒）。默认为 0（即无间隔）。此变量是在 MySQL 5.6.5 里引入的。

`innodb_mirrored_log_groups`（启动：直接设置；作用范围：全局）

它指的是 InnoDB 维护的日志文件组的个数。其值总是为 1。

`innodb_monitor_disable`、`innodb_monitor_enable`、`innodb_monitor_reset`、`innodb_monitor_reset_all`（启动：直接设置；作用范围：全局）

这此变量控制的是对 `INFORMATION_SCHEMA.innodb_metrics` 表的操作。更多信息请参考 MySQL 参考手册。这些变量是在 MySQL 5.6.2 里引入的。

`innodb_old_blocks_pct`（启动：直接设置；作用范围：全局；动态）

它指的是 InnoDB 缓冲池的旧子列表所占百分比。默认值为 37（即缓冲池的 3/8）。允许的取值范围为 5~95。更多信息请参考 12.7.2.1 节。

`innodb_old_blocks_time`（启动：直接设置；作用范围：全局；动态）

一个缓存块在第一次访问之后、下次访问之前，需要在 InnoDB 缓冲池的旧子列表里待多少毫秒才能移动到新子列表里。默认值为 0。表示的是，插到旧子列表里的缓存块，在第一次访问时会立即移到新子列表里去。更多信息请参考 12.7.2.1 节。

`innodb_open_files`（启动：直接设置；作用范围：全局）

如果将 `innodb_file_per_table` 变量设置为 1，将启用独立表空间。`innodb_open_files` 变量控制着需要为 InnoDB 存储引擎保留多少个文件描述符用于同时打开多个 `.ibd` 文件。最小值为 10，默认值为 300。在

`innodb_open_files`变量控制下分配的文件描述符与在`open_files_limit` 变量控制下分配的文件描述符互不相干：前者用来打开`.ibd` 文件，后者供表缓冲区使用。

`innodb_page_size`（启动：直接设置；作用范围：全局）

它指的是InnoDB表空间里的页面大小。默认大小为16 KB。允许值有4 KB、8 KB和16 KB。该设置只有在InnoDB初始化表空间的时候才会生效，因此应该在初始化MySQL之前，或者在删除并重新创建InnoDB表空间文件之前设置它。此变量是在MySQL 5.6.4里引入的。

`innodb_print_all_deadlocks`（启动：直接设置；作用范围：全局；动态）

它指的是，InnoDB 是否会将诊断信息写到与事务死锁有关的错误日志里。默认值为 OFF。此变量是在MySQL 5.6.2里引入的。

`innodb_purge_batch_size`（启动：直接设置；作用范围：全局；动态）

它指的是，重做日志记录的数量。这些记录会引发清除操作，将更改过的缓冲池数据块刷新到磁盘上。默认值为20。此变量是在MySQL 5.5.4里引入的。

`innodb_purge_threads`（启动：直接设置；作用范围：全局）

它指的是，InnoDB 使用了多少后台线程来实现清除操作（将所有事务都不再需要的待删除行删除掉）。默认值为0。此变量是在MySQL 5.5.4里引入的。

`innodb_random_read_ahead`（启动：直接设置；作用范围：全局；动态）

它指的是，InnoDB 是否要试图预测何时会需要位于某个范围内（即页面组）的页面，并为它们执行一个异步的预读取操作。预测的依据是该范围内的其他页面是否已被读取，而它们的读取顺序无关紧要。默认值为OFF。

`innodb_read_ahead_threshold`（启动：直接设置；作用范围：全局；动态）

如果InnoDB检测到某个模式的顺序页面访问是由`innodb_read_ahead_threshold` 或多个来自同一范围（页面组）的页面构成，那么它会在下一个范围里执行一个异步预读取操作。默认值为56。允许的取值范围为0~64。

`innodb_read_io_threads`（启动：直接设置；作用范围：全局）

它指的是InnoDB在执行读取操作时会使用多少个线程。取值范围是1～64；默认值为4。

`innodb_replication_delay`（启动：直接设置；作用范围：全局；动态）

如果在从服务器上已达到`innodb_thread_concurrency`表明的那个限制，那么此变量便是复制线程的延迟时间（单位为毫秒）。默认值为0。

`innodb_rollback_on_timeout`（启动：直接设置；作用范围：全局）

此变量控制着 InnoDB存储引擎在事务超时时的行为。如果此变量的值是OFF（默认值），InnoDB存储引擎将只回滚最后一条语句；如果此变量的值是ON，InnoDB存储引擎将回滚整个事务。

`innodb_rollback_segments`（启动：直接设置；作用范围：全局；动态）

它指的是，在一个事务里，InnoDB 在系统表空间里会使用多少回滚段。默认值为128。此变量是在MySQL 5.5.11里引入的。在5.6.3里，它被替换成了`innodb_undo_logs`。

`innodb_sort_buffer_size`（启动：直接设置；作用范围：全局）

它指的是InnoDB在索引创建期间用于合并排序的缓冲区大小（单位为字节）。默认大小为1 MB。在MySQL 5.6.4里最小值为512 KB；在MySQL 5.6.5及以下的版本里，最小值为64 KB。此变量是在MySQL 5.6.4里引入的。在MySQL 5.6.4之前，使用的是固定大小1 MB。

`innodb_spin_wait_delay`（启动：直接设置；作用范围：全局；动态）

它指的是，旋转锁的两次轮询之间的最大等待值。这个值没有单位，但是值越大意味着等待得越久。默认值为6，最小值为0。

`innodb_stats_method`（启动：直接设置；作用范围：全局；动态）

此变量表示的是，服务器在为InnoDB表统计其索引键的分布概率时，应该把NULL 值视为相同，还是视为不同。可设置的值包括有：
`nulls_equal`（所有的NULL值在同一个组里）、`nulls_unequal`（每个NULL值单独成为一组）或`nulls_ignored`（忽略NULL值）。

`innodb_stats_on_metadata`（启动：直接设置；作用范围：全局；动态）

它指的是，InnoDB是否要更新与那些与表元数据访问语句（如SHOW INDEX或SHOW TABLE STATUS）有关的统计信息，或者更新那些与访问STATISTICS 表和执行 TABLES INFORMATION_SCHEMA 语句所得到的各张表有关的统计信息。其效果与运行 ANALYZE TABLE相类似。默认值为ON。

`innodb_stats_persistent_sample_pages`（启动：直接设置；作用范围：全局；动态）它指的是，用于估计统计信息的InnoDB 样本的索引页数量。默认值为 20。如果启用了`innodb_analyze_is_persistent`，那么此设置会被忽略掉。否则，InnoDB 会使用`innodb_stats_transient_sample_pages`的值。此变量是在MySQL 5.6.2里引入的。

`innodb_stats_sample_pages`（启动：直接设置；作用范围：全局；动态）

在MySQL 5.5里，它指的是用于估计统计信息的InnoDB样本的索引页数量。默认值为8。自MySQL 5.6.3起，此变量已被弃用，请使用`innodb_stats_transient_sample_pages`。

`innodb_stats_transient_sample_pages`（启动：直接设置；作用范围：全局；动态）

它指的是，用于估计统计信息的InnoDB 样本的索引页数量。默认值为8。如果启用了`innodb_analyze_is_persistent`，此设置会被忽略。否则，InnoDB 会使用`innodb_stats_persistent_sample_pages`的值。此变量是在MySQL 5.6.2里引入的。

`innodb_strict_mode`（启动：直接设置；作用范围：全局、会话；动态）

它指的是，InnoDB 是否对表和索引的创建和修改语句的语法进行较严格要求。如果启用了此变量，那么 InnoDB会把有冲突的子句当作错误；否则，它们会被当作警告。这个有点类似于启用了严格的SQL模式。

`innodb_support_xa`（启动：直接设置；作用范围：全局、会话；动态）

它表示的是，InnoDB在XA事务里是否支持两阶段提交模式。默认值为ON；但是如果为了获得更好的性能而不使用XA事务，则可以将其设置为OFF。

`innodb_sync_spin_loops`（启动：直接设置；作用范围：全局；动态）

它表示的是，线程在被挂起之前等待InnoDB释放互斥信号的次数。

`innodb_table_locks`（启动：直接设置；作用范围：全局、会话；动态）

此变量控制的是，当禁用自动提交模式时，InnoDB会如何处理LOCK TABLE 语句，以获得InnoDB表的写锁。如果此变量的值是ON（默认值），InnoDB将申请到一个内部表锁。如果此变量的值是 OFF，InnoDB 要一直等到没有任何其他线程锁定那个表时才能执行 LOCK TABLE 语句。禁用此变量可以在一定程度上防止应用程序在自动提交模式已被禁用的情况下发出LOCK TABLE语句时遭遇死锁。

`innodb_thread_concurrency`（启动：直接设置；作用范围：全局；动态）

它指的是InnoDB尝试维护的线程数量上限。当值为0时（默认值），表示的是“无限制”。允许的取值范围为0~1000。

`innodb_thread_sleep_delay`（启动：直接设置；作用范围：全局；动态）

它指的是InnoDB线程在被放入InnoDB等待队列之前休眠的时间（单位为毫秒）。默认值为10 000（即10秒）；值为0时，表示的是“不休眠”。

`innodb_undo_directory`（启动：直接设置；作用范围：全局）

如果 `innodb_undo_logs`和`innodb_undo_tablespaces` 都是非零值，则它指的是InnoDB 在其中创建独立恢复日志表空间的那个目录。默认值为“.”，它表示的是 InnoDB在其中创建其他日志文件的那个默认目录。此变量是在MySQL 5.6.3里引入的。

`innodb_undo_logs`（启动：直接设置；作用范围：全局；动态）

它指的是，在一个事务里，InnoDB 在系统表空间里会使用多少回滚段。默认值为 128。此变量是在MySQL 5.6.3里引入的。它用于替换 `innodb_rollback_segments`。

`innodb_undo_tablespaces`（启动：直接设置；作用范围：全局）

它指的是 InnoDB 针对独立恢复日志所使用的表空间文件数量。默认值为 0。此变量是在MySQL 5.6.3里引入的。

`innodb_use_native_aio`（启动：直接设置；作用范围：全局）

它指的是，在Linux里是否使用异步I/O子系统。默认值为ON。此变量是在MySQL 5.5.4里引入的。

`innodb_use_sys_malloc`（启动：直接设置；作用范围：全局）

它指的是，InnoDB 是否要使用系统内存分配器。默认值为 ON。如果禁用此变量，会导致InnoDB使用其自己的分配器。

`innodb_version`（作用范围：全局）

它指的是InnoDB存储引擎的版本号。

`innodb_write_io_threads`（启动：直接设置；作用范围：全局）

它指的是InnoDB在执行写入操作时会使用多少个线程。取值范围是1～64；默认值为4。

`timed_mutexes`（启动：直接设置；作用范围：全局；动态）

是否要搜集InnoDB的互斥时间信息。默认值为OFF。

D.2 状态变量

状态变量提供的是各种与服务器当前运作状态有关的信息。通过使用 `SHOW STATUS` 语句或执行 `mysqladmin extended-status` 命令可以查看这些状态变量。状态变量（与系统变量相似）的值可分为全局级和会话级，分别对应于所有客户端的总和值和当前客户端的值。如果某个状态变量只有一个全局值，则全局变量和会话变量都将返回相同的值。另外，还可以通过在 `INFORMATION_SCHEMA` 数据库里查询表 `GLOBAL_STATUS` 和 `SESSION_STATUS` 来获得状态变量信息。

关于如何在运行时查看状态变量的更多信息请参考12.3.2节。

状态变量名不区分大小写。

下面列出的是一些比较通用的状态变量。随后的几节描述的是几组彼此相关的状态变量，其中涉及的类型包括：InnoDB存储引擎、查询缓存和SSL。

`Aborted_clients`

因客户端未正确关闭而被中断的客户端连接个数。

`Aborted_connects`

连接服务器的尝试失败的次数。

Binlog_cache_disk_use

因其大小超过系统变量binlog_cache_size的限制而使用临时磁盘文件的那些事务的数量。

Binlog_cache_use

因其大小未超过系统变量binlog_cache_size的限制而保存在二进制日志缓存里的那些事务的数量。

Binlog_stmt_cache_disk_use

因语句缓存已满而存储在临时文件里的非事务语句的数量。此变量是在MySQL 5.5.9里引入的。

Binlog_stmt_cache_use

存储在语句缓存里的非事务语句数量。此变量是在MySQL 5.5.9里引入的。

Bytes_received

服务器从所有客户端（对于全局值）或当前客户端（对于会话值）接收到的字节总数。

Bytes_sent

服务器向所有客户端（对于全局值）或当前客户端（对于会话值）发送出去的字节总数。

Com_xxx

服务器维护着一组状态变量，它们的作用是计数器，用于表示特定类型的语句（或命令）已执行的次数。这种类型的变量有几十个，并且它们的名字都相似，因此这里没有将它们逐一地列出来。每个语句计数器的变量名都以Com_作为开头，并且都有一个后缀，用于表明该计数器所对应的语句类型。例如，变量Com_select和Com_drop_table分别表示的是服务器已执行了多少条SELECT语句和DROP TABLE语句。

Compression

表明是否对利用客户端/服务器协议实现的通信进行压缩。

Connection_errors_xxx

这些变量用于跟踪在客户端尝试进行连接时出现的各种类型的错误。这些变量是在MySQL 5.6.5里引入的。

Connections

尝试连接服务器的次数（包括成功或失败的）。

Created_tmp_disk_tables

服务器在处理语句时在磁盘上创建的内部临时表个数。

Created_tmp_files

服务器创建的临时文件个数。

Created_tmp_tables

服务器在处理语句时创建的内部临时表个数。

Delayed_errors

在处理INSERT DELAYED行时发生的错误个数。

Delayed_insert_threads

INSERT DELAYED处理器的个数。

Delayed_writes

已写入的INSERT DELAYED行的个数。

Flush_commands

已执行的表刷新操作次数。

Handler_commit

请求提交事务的次数。

`Handler_delete`

请求从表删除行的次数。

`Handler_external_lock`

此变量与锁定操作数量有关，主要是在表访问的开始和结束时起作用。其值除以2，即为锁定操作的次数。此变量是在MySQL 5.6.2里引入的。

`Handler_mrr_init`

有些存储引擎自己实现了“多范围读取”（Multi-Range Read）优化策略。此变量表示的是服务器使用该实现的次数。此变量是在MySQL 5.6.1里引入的。

`Handler_prepare`

为两阶段提交所做的准备次数。

`Handler_read_first`

请求从索引里读取第一行的次数。

`Handler_read_key`

请求基于索引值读取行的次数。

`Handler_read_key`

请求基于索引值读取行的次数。

`Handler_read_last`

请求从索引里读取最后一行的次数。此变量是在MySQL 5.5.7里引入的。

`Handler_read_next`

按索引顺序请求读取下一行的次数。

`Handler_read_prev`

按索引逆序请求读取上一行的次数。

`Handler_read_rnd`

根据位置请求读取一行的次数。

`Handler_read_rnd_next`

请求读取下一行的次数。如果这个值很大，则说明你可能在执行大量的需要全表扫描的语句，或者执行大量的未正确使用索引的语句。

`Handler_rollback`

请求事务回滚的次数。

`Handler_savepoint`

请求创建事务保存点的次数。

`Handler_savepoint_rollback`

请求回滚到事务保存点的次数。

`Handler_update`

在表里请求更新行的次数。

`Handler_write`

在表里请求插入行的次数。

`Innodb_xxx`

请参考D. 2. 1节。

`Key_blocks_not_flushed`

在键缓存里已被修改、但仍未刷新到磁盘的缓存块个数。

`Key_blocks_unused`

键缓存里尚未使用的缓存块个数。

`Key_blocks_used`

键缓存里被同时使用的缓存块最大个数。

`Key_reads_requests`

请求从键缓存里读取数据块的次数。

`Key_reads`

从磁盘里读取索引块的次数。

`Key_write_requests`

请求把数据块写入键缓存的次数。

`Key_writes`

把索引块写入键缓存的次数。

`Last_query_cost`

查询优化器最近一次的查询开销计算。这个值只对未使用UNION或子查询的查询才有用。如果还没计算出查询开销，则这个值为0。这个值只对使用查询缓存的查询才有效。

`Last_query_partial_plans`

它适用于最近执行的查询，指的是优化器在准备执行计划时迭代的次数。此变量是在MySQL 5.6.5里引入的。

`Max_used_connections`

同时打开的最大连接数。

`Not_flushed_delayed_rows`

等待通过INSERT DELAYED语句写入的行个数。

`Open_files`

打开文件的个数。

`Open_streams`

处于打开状态的文件流的个数。一个流即为一个用`fopen()`函数打开的文件。它只适用于日志文件。

`Open_table_definitions`

缓存的.frm文件的个数。

`Open_tables`

处于打开状态的表个数，其中不包括TEMPORARY表。

`Opened_files`

服务器已打开文件的次数。（有些存储引擎不会递增这个计数器的值。）

`Opened_files`

服务器使用内部库函数`my_open()`打开文件的次数。

`Opened_table_definitions`

服务器已打开.frm文件的次数。

`Opened_tables`

服务器已打开表的次数。如果这个值很大，则应该考虑加大表缓存。

Performance_schema_xxx

这些以Performance_schema开头的状态变量都与用于采集和分析服务器性能数据的Performance Schema程序有关。更多详细信息请参考MySQL参考手册。

Prepared_stmt_count

预处理语句的个数。

Qcache_xxx

请参考D. 2. 2节。

Questions

服务器接收到的语句个数（其中即包括成功的语句也包括不成功的语句）。Questions和Uptime的比值即为每秒处理的语句数量。

Select_full_join

“完全”连接（即未使用索引的连接操作）的次数。

Select_full_range_join

对参照表进行范围搜索时执行连接的次数。

Select_range

使用第一个表的范围执行连接的次数。

Select_range_check

在利用范围搜索获取辅助表的行时执行连接的次数。

Select_scan

在对第一张表进行全表扫描时执行连接的次数。

`Slave_heartbeat_period`

复制操作使用的心跳间隔（单位为秒）。

`Slave_last_heartbeat`

该TIMESTAMP值表示的是从主服务器接收最后一个心跳包的时间。
此变量是在MySQL 5.6.1里引入的。

`Slave_open_tmp_tables`

从SQL线程曾经打开的临时表个数。

`Slave_received_heartbeats`

自上一条CHANGE MASTER语句以来或者自从服务器重启或重置以来，从主服务器接收到的心跳包个数。

`Slave_retried_transactions`

从SQL线程重试事务的次数。

`Slave_running`

从I/O线程和SQL线程是否都在运行中。

`Slow_launch_threads`

创建时间超过`slow_launch_time`秒的线程数。

`Slow_queries`

查询时间超过`long_query_time`秒的查询的个数。

`Sort_merge_passes`

排序算法执行的合并的次数。

Sort_range

使用范围的排序操作的个数。

Sort_rows

排好序的行数。

Sort_scan

使用全表扫描的排序操作的个数。

Ssl_xxx

请参考D. 2. 3节。

Table_locks_immediate

无需等待便能立刻得到满足的表锁请求的数量。

Table_locks_waited

只有在等待之后才能得到满足的表锁请求的数量。如果这个值很大，则表明表锁存在大量的竞争问题。

Table_open_cache_hits、Table_open_cache_misses、Table_open_cache_overflows这些变量提供的是与打开表缓存的操作相关的统计信息。“命中”和“缺失”分别指的是在缓存里找到和未找到表。“溢出”指的是缓存超出table_open_cache所表示的那个大小限制的次数。这些变量是在MySQL 5.6.6里引入的。

Tc_log_max_pages_used

事务协调恢复日志文件所使用的页面的最大数量。

Tc_log_page_size

事务协调恢复日志文件的页面大小。

`Tc_log_page_waits`

在事务协调恢复日志文件里，两阶段提交操作必须等待空闲页面的次数。

`Threads_cached`

线程缓存里的线程数量。

`Threads_connected`

处于打开状态的连接数量。

`Threads_created`

服务器为处理客户端连接而创建线程的次数。

`Threads_running`

处于激活状态（非休眠状态）的线程个数。

`Uptime`

服务器自开始运行以来持续运行的秒数。

`Uptime_since_flush_status`

自最近一次执行FLUSH STATUS语句以来经过的秒数。

D. 2. 1 InnoDB状态变量

下面这些状态变量显示的是与 InnoDB 存储引擎操作有关的信息。其中许多都可以在SHOW ENGINE INNODB STATUS语句的输出里获得，但是SHOW STATUS语句的输出更容易读懂。

`Innodb_available_undo_logs`

可用的InnoDB恢复日志个数。（系统变量innodb_undo_logs表示的是当前激活的数量。）此变量是在MySQL 5.6.5里引入的。

`Innodb_buffer_pool_pages_data`

InnoDB缓冲池里包含数据的页面数量。此计数包括那些从没被修改过的干净页面和那些包含修改数据的脏页。

`Innodb_buffer_pool_dump_status`

缓冲池转储操作的当前状态。此变量是在MySQL 5.6.3里引入的。

`Innodb_buffer_pool_load_status`

缓冲池加载操作的当前状态。此变量是在MySQL 5.6.3里引入的。

`Innodb_buffer_pool_pages_dirty`

InnoDB缓冲池里包含修改过的数据的页面数量。

`Innodb_buffer_pool_pages_flushed`

已执行过刷新请求的InnoDB缓冲池的页面数量。

`Innodb_buffer_pool_pages_free`

InnoDB缓冲池里的空闲页面数。

`Innodb_buffer_pool_pages_latched`

InnoDB缓冲池里正在被读取或写入的页面数，或者不能被刷新和释放以再次使用的页面数。只有在构建MySQL时定义了UNIV_DEBUG才会显示。

`Innodb_buffer_pool_pages_misc`

InnoDB缓冲池里的内部操作所分配到的页面数。

`Innodb_buffer_pool_pages_total`

InnoDB缓冲池里的页面总数。

Innodb_buffer_pool_read_ahead

InnoDB预读后台线程读入InnoDB缓冲池的页面数。

Innodb_buffer_pool_read_ahead_evicted

被InnoDB预读后台线程读入的、因未被查询使用而被驱逐的页面数量。

Innodb_buffer_pool_read_requests

InnoDB发起的逻辑读请求的数量。

Innodb_buffer_pool_reads

因不能从InnoDB缓冲池进行逻辑读取而采取的单页读取操作的数量。

Innodb_buffer_pool_wait_free

当把数据写入将被刷新到磁盘的缓冲池时，InnoDB必须等待的次数。写入操作通常是在后台进行的，但是在InnoDB需要读取页面或者创建新页时，如果没有可用页面，就必须等待。

Innodb_buffer_pool_write_requests

往InnoDB缓冲池写入的次数。

Innodb_data_fsyncs

由InnoDB执行的“同步到磁盘”（sync-to-disk）操作的个数。

Innodb_data_pending_fsyncs

挂起的InnoDB数据“同步到磁盘”操作的个数。

Innodb_data_pending_reads

挂起的InnoDB“数据读取”操作的个数。

Innodb_data_pending_writes

挂起的InnoDB“数据写入”操作的个数。

Innodb_data_read

InnoDB读取的字节数。

Innodb_data_reads

InnoDB“数据读取”操作的个数。

Innodb_data_writes

InnoDB“数据写入”操作的个数。

Innodb_data_written

InnoDB写入的字节数。

Innodb_dblwr_pages_written

写入InnoDB双写缓冲区的页面数。

Innodb_dblwr_writes

写入InnoDB双写缓冲区的次数。

Innodb_have_atomic_builtins

表明MySQL在构建时是否启用了原子操作。

Innodb_log_waits

当把数据写入将被刷新的日志缓冲池时，InnoDB必须等待的次数。

Innodb_log_write_requests

InnoDB日志文件的写请求数。

InnoDB_log_writes

InnoDB日志文件的写入次数。

InnoDB_num_open_files

InnoDB已打开的文件个数。此变量是在MySQL 5.6.2里引入的。

InnoDB_os_log_fsyncs

InnoDB日志文件“同步到磁盘”操作的个数。

InnoDB_os_log_pending_fsyncs

挂起的InnoDB日志文件“同步到磁盘”操作的个数。

InnoDB_os_log_pending_writes

挂起的InnoDB日志文件写入操作的个数。

InnoDB_os_log_written

写入InnoDB日志文件的字节数。

InnoDB_page_size

InnoDB所使用的内编译页面大小。通过它，可以把一个以页为单位的测量值转换为以字节为单位的值。默认大小为16 KB。

InnoDB_pages_created

InnoDB创建的页数。

InnoDB_pages_read

InnoDB读取的页数。

InnoDB_pages_written

InnoDB写入的页数。

Innodb_row_lock_current_waits

InnoDB正在等待获得的行锁个数。

Innodb_row_lock_time

获得InnoDB行锁所花费的总时间（单位为毫秒）。

Innodb_row_lock_time_avg

获得一个InnoDB行锁所需要花费的平均时间（单位为毫秒）。

Innodb_row_lock_time_max

获得InnoDB行锁所需要花费的最大时间（单位为毫秒）。

Innodb_row_lock_waits

InnoDB为获得行锁而等待的时间。

Innodb_rows_deleted

从InnoDB表删除的行数。

Innodb_rows_inserted

插入InnoDB表的行数。

Innodb_rows_read

从InnoDB表读取的行数。

Innodb_rows_updated

在InnoDB表里更新的行数。

Innodb_truncated_status_writes

SHOW ENGINE INNODB STATUS的输出内容被截断了多少次。此变量是在MySQL 5.5.7里引入的。

D. 2. 2 查询缓存状态变量

下面这些状态变量显示的是与查询缓存操作有关的信息。

`Qcache_free_blocks`

查询缓存里的空闲内存块数量。

`Qcache_free_memory`

查询缓存里的空闲内存总量。

`Qcache_hits`

保存在缓存里的满足查询请求的查询数量。

`Qcache_inserts`

在查询缓存里注册过的查询数量。

`Qcache_lowmem_prunes`

为了给新的查询结果腾出空间而从查询缓存里丢弃的缓存查询结果的数量。

`Qcache_not_cached`

无法缓存的或者因用户使用了`SQL_NO_CACHE`关键字而被禁止缓存的查询的数量。

`Qcache_queries_in_cache`

缓存里注册的查询数量。

`Qcache_total_blocks`

查询缓存里的内存块数量。

D. 2. 3 SSL状态变量

以下变量提供的是与SSL管理代码有关的信息。其中的许多变量都可以反映出当前会话的状态；并且当会话实际并非安全连接时，它们将为空白。只有服务器在构建时包含了SSL支持，这些变量才可用。

`Ssl_accept_renegotiates`

在服务器模式里开始重新协商的次数。

`Ssl_accepts`

在服务器模式里开始的SSL/TLS握手的次数。

`Ssl_callback_cache_hits`

在服务器模式里从外部会话缓存成功地检索到的会话数量。

`Ssl_cipher`

会话所使用的SSL密码（协议）。（如果不存在有效的密码，则为空。）可以利用此变量来判断会话是否是已加密。

`Ssl_cipher_list`

可用的SSL密码协议列表。

`Ssl_client_connects`

在客户端模式里开始的SSL/TLS握手的次数。

`Ssl_connect_renegotiates`

在客户端模式里开始重新协商的次数。

`Ssl_ctx_verify_depth`

SSL上下文的验证深度。

`Ssl_ctx_verify_mode`

SSL上下文的验证模式。

`Ssl_default_timeout`

默认的SSL会话超时时间。

`Ssl_finished_accepts`

在服务器模式里成功建立的SSL/TLS会话数。

`Ssl_finished_connects`

在客户端模式里成功建立的SSL/TLS会话数。

`Ssl_server_not_after`

SSL证书的最后有效日期。此变量是在MySQL 5.6.3里引入的。

`Ssl_server_not_before`

SSL证书的开始有效日期。此变量是在MySQL 5.6.3里引入的。

`Ssl_session_cache_hits`

在会话缓存里找到的SSL会话数。

`Ssl_session_cache_misses`

在会话缓存里未找到的SSL会话数。

`Ssl_session_cache_mode`

服务器所使用的SSL缓存类型。

`Ssl_session_cache_overflows`

因缓存已满而被删除的会话数。

`Ssl_session_cache_size`

SSL会话缓存里存储的会话数。

`Ssl_session_cache_timeouts`

已超时的会话数。

`Ssl_session_reused`

是否有会话再次利用了之前的某个会话。

`Ssl_used_session_cache_entries`

会话缓存里的会话数。

`Ssl_verify_depth`

SSL验证深度。

`Ssl_verify_mode`

SSL验证模式。

`Ssl_version`

会话的SSL协议版本。

D.3 用户定义的变量

用户定义的变量（简称为“用户变量”）可以被赋值；然后，在其他语句里引用它们。

用户定义的变量的名字由“@”字符和一个紧随其后的标识符构成，遵守的规则与合法标识符的规则相似（参考2.2节）。不过，用户变量名有一点与标识符不相同，它可以包含“.”，同时无需用引号引起来。用户变量名不区分大小写。

用户变量的赋值方式有两种：在SET语句里使用操作符“=”或“:=”；在其他语句（如SELECT）里使用操作符“:=”。可以在同一条语句里对多个变量进行赋值。

```
mysql> SET @x = 0, @y = 2;
```

```
mysql> SET @color := 'red', @size := 'large';
```

```
mysql> SELECT @x, @y, @color, @size;
```

@x	@y	@color	@size
0	2	red	large

```
mysql> SELECT @count := COUNT(*) FROM member;
```

@count := COUNT(*)
102

用户变量可以被赋值为整数、小数、浮点数、字符串或 NULL 值，并且可以通过任意形式的表达式（包括引用了其他变量的表达式）进行赋值。如果在访问某个用户变量时它还未被赋值，则其默认值为NULL。

用户变量的值只在当前会话里有效。也就是说，当会话终止时，这些值都将会消失。

在返回多个行的SELECT 语句里，每一行都会执行变量赋值操作。最终的结果值为最后那行所赋予的值。

字符串类型的用户变量所拥有的字符集和排序规则与给它们赋值的那些值的相同：

```
mysql> SET @s = CONVERT('abc' USING latin2) COLLATE  
latin2_czech_cs;
```

```
mysql> SELECT CHARSET(@s), COLLATION(@s);
```

CHARSET(@s) COLLATION(@s)	
latin2	latin2_czech_cs

附录E

SQL语法参考

本附录将介绍 MySQL 提供的各种 SQL 语句的语法。如无特别说明，这里列出的语句早在MySQL 5.5.0里就有了。在该版本之后产生的变更则加了说明。

本附录的内容包括以下3个部分。

复合语句之外的SQL语句。

复合语句所使用的SQL语句，都需要使用BEGIN和END 来编写，并且可用于编写存储程序：在服务器端存储的函数、过程、触发器和事件。

在SQL代码里编写注释的语法。注释的用途是编写描述性文字（它们会被MySQL服务器忽略掉），以及隐藏MySQL相关的关键字（它们会被MySQL执行，但是会被其他数据库服务器忽略掉）。

在对SQL语法进行描述时，本书使用了以下这些约定。

方括号（[]）用于表示可选信息。

竖线字符（|）用于分隔列表里的各个条目。如果列表出现在一对方括号里，则表示可以从中选取一个。如果列表出现在一对花括号（{}）里，则表示必须从中选取一个。

省略号 (...) 表示的是该省略号前面的那个项目可以重复多次。

n表示的是一个整数。

'str'表示的是一个字符串。对于使用单引号括起来的值（如'file_name'或'pattern'），表示的是一些更为特定类型的值，如文件名或模式。

本附录不会涉及那些与用户定义的函数（User-Defined Function, UDF）和XA事务相关的语句或子句，以及那些与 MySQL 集群或那些不常使用到的存储引擎（如 FEDERATED）特定相关的语句或子句。

对于那些通用的同义词，本书则将它们统一列在了下面，而没有将其随处放置。

下面两种的格式都可用于指定字符集：

CHARACTER SET charset

CHARSET [=] charset

这些形式可以用在表和列的定义里，也可以用在CREATE DATABASE和ALTER DATABASE语句里。

SCHEMA和SCHEMAS分别与DATABASE和DATABASES同义，并且可以在任何地方使用后面两个关键字的语句里随意替换。例如，在创建数据库时，既可以使用CREATE DATABASE语句，也可以使用CREATE SCHEMA语句。

COLUMNS和FIELDS同义。

E.1 SQL语句语法（非复合语句）

本节将对MySQL的各种SQL语句（复合语句除外，参考E.2节）的语法和含义进行描述。如果在执行语句时没有足够的权限，那么该语句的执行会失败。例如，如果没有访问db_name数据库的权限，那么执行USE db_name语句时会失败。

ALTER DATABASE

ALTER DATABASE [db_name] db_attr ...

ALTER DATABASE db_name UPGRADE DATA DIRECTORY
NAME

这条语句可用于更改数据库的属性，或者升级数据库目录名的编码。它要求对数据库拥有ALTER权限。

对于第一种语法，db_attr的允许值（即数据库属性值）与CREATE DATABASE条目里列出的属性值相同。如果省略了数据库名，那么此语句会使用默认数据库。如果没有默认数据库，则会出现错误。

当升级MySQL 5.1之前的版本时，需要使用UPGRADE DATA DIRECTORY NAME语法。如果数据库目录名称里包含有特殊字符，那么它会根据需要将该名字重新编码为MySQL当前使用的那个文件系统编码。

ALTER EVENT

ALTER

[DEFINER = definer_name]

EVENT event_name

[ON SCHEDULE schedule]

[ON COMPLETION [NOT] PRESERVE]

[RENAME TO new_event_name]

[ENABLE | DISABLE [ON SLAVE]]

[COMMENT 'str']

[DO event_stmt]

这条语句用于更改现有事件，使其具有新的定义。RENAME TO子句可用于重命名事件。其他的子句会在CREATE EVENT条目里进行描述。在使用此语句时，必须对该事件所属数据库拥有EVENT权限。

ALTER FUNCTION, ALTER PROCEDURE

ALTER {FUNCTION | PROCEDURE} routine_name [characteristic]

...

characteristic:

COMMENT 'str'

| {CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA}

| LANGUAGE SQL

| SQL SECURITY {DEFINER | INVOKER}

这些语句用于更改存储例程的特性。在使用它们时，需要对指定例程拥有 ALTER ROUTINE 权限。这些特性会在CREATE FUNCTION和CREATE PROCEDURE语句里进行介绍。

ALTER TABLE

ALTER [IGNORE] TABLE tbl_name [action [, action] ...]

ALTER TABLE可用于重命名表或更改它们的结构。要使用此命令时，需要指定表名，同时还可以指定需要对该表执行的哪些操作。如果该操作会使更改后的表里的唯一性索引出现重复键值，那么还需要使用关键字IGNORE。如果没有IGNORE，则ALTER TABLE语句的作用会被取消。如果带有IGNORE，则那些拥有键值重复的行将被删除。

在ALTER TABLE 操作过程中，其他客户端依然可以从原始表里读取数据。那些试图更新该表的客户端将被阻塞，一直到该操作执行完

成为止；而此时，所有更新将会应用到那个新表上。

其中，`action`指定的是各种修改操作——它们会按顺序依次执行。与操作描述里说明的情况一样，有些操作不能与其他动作组合使用。如果没有指定任何操作，则`ALTER TABLE`无任何效果。

对于包含`index_option`子句的索引定义操作，有些存储引擎允许指定索引算法或其他索引定义修饰符。与不同 MySQL 版本所支持的各种索引值有关的更多详细信息，请参考 `CREATE INDEX` 条目。与索引创建有关的其他信息请参考 2.6.4 节。

`action`的值可以是以下的任何一个。

`table_option`

用于指定表选项——它可以作为`CREATE TABLE`语句的 `table_option` 部分。用于分隔相连表选项的那个逗号可以被省略。

```
ALTER TABLE score ENGINE=MyISAM CHECKSUM=1;
```

```
ALTER TABLE sayings CHARACTER SET utf8;
```

对于不同的MySQL版本或不同的存储引擎，允许指定的表选项会有所不同（请参考`CREATE TABLE`语句的描述）。如果试图要更改某个表，让其使用某种不可用的存储引擎，那么`ALTER TABLE`语句的最终效果将取决于对SQL模式`NO_ENGINE_SUBSTITUTION`的具体设置。

表选项`[DEFAULT] CHARACTER SET`可以更改默认的表字符集，但是不能将现有列转换为该字符集。如果想转换列的字符集，则必须使用`CONVERT TO CHARACTER SET`操作。

`ALTER TABLE`语句会忽略表选项`DATA DIRECTORY`和`INDEX DIRECTORY`。

```
ADD [COLUMN] col_name col_definition [FIRST | AFTER  
col_name]
```

为表增加一列。其中，col_name是列名，col_definition是列的定义——其格式与CREATE TABLE语句的列定义格式完全相同。如果指定了FIRST关键字，则该列将成为表的第一列；如果指定了AFTER col_name，则该列将被放置在所指定列的后面。如果未指定列的位置，则它会成为表的最后一列。

```
ALTER TABLE t ADD name CHAR(20);
```

```
ALTER TABLE t ADD id INT UNSIGNED NOT NULL PRIMARY  
KEY FIRST;
```

```
ALTER TABLE t ADD birth DATE AFTER name;
```

```
ADD [COLUMN] (create_definition,...)
```

为表增加列或索引。每个create_definition 都是一个列定义或索引定义，其格式与CREATE TABLE语句所使用的格式完全相同。

```
ADD [CONSTRAINT [name]] FOREIGN KEY [fk_name]
```

```
(index_columns) reference_definition
```

为指定的表增加一个外键定义。这种更改操作只适用于 InnoDB 表。新外键由index_columns 列表里列出的列构成，该列表由给定表里的一个或多个以逗号分隔的列构成。如果指定了 CONSTRAINT 部分，则其中列出的任何名字都将被忽略。fk_name 是外键ID。如果指定了它，那么它会被忽略，除非 InnoDB会自动为外键创建一个索引——此时，fk_name将变成该索引的名字。reference_definition部分用来定义新外键与父表的关系。相关语法，请参考CREATE TABLE语句条目的描述。

```
ALTER TABLE child
```

```
ADD FOREIGN KEY (par_id) REFERENCES parent (par_id) ON  
DELETE CASCADE;
```

动作ADD FOREIGN KEY和DROP FOREIGN KEY不能出现在同一条ALTER TABLE语句里。

```
ADD FULLTEXT [INDEX | KEY] [index_name]
```

```
(index_columns) [index_option] ...
```

为MyISAM表或InnoDB表（自MySQL 5.6.4起）增加一个FULLTEXT索引。该索引会根据index_columns 里列出的列来创建，这些列都是表里的一个或多个非二进制串列，它们之间需要用逗号隔开。关于index_name的指定，请参考ADD INDEX动作。

```
ALTER TABLE poetry ADD FULLTEXT (author,title,stanza);
```

```
ADD {INDEX | KEY} [index_name]
```

```
(index_columns) [index_option] ...
```

为表增加一个索引。此索引会根据index_columns里列出的列来创建，这些列都是给定表里的一个或多个列，它们之间需要用逗号隔开。如果没有给出索引名字index_name，MySQL会自动选择第一个索引列的名字作为此索引的名字。

```
ADD [CONSTRAINT [name]] PRIMARY KEY
```

```
(index_columns) [index_option] ...
```

根据给定列添加一个主键。在指定主键时需要带上PRIMARY。关于index_columns的指定，请参考ADD INDEX动作。其中的每个列都必须定义为NOT NULL。如果主键已经存在，则此操作会导致出错。

```
ALTER TABLE president ADD PRIMARY KEY (last_name,  
first_name);
```

```
ADD SPATIAL [INDEX | KEY] [index_name]
```

```
(index_columns) [index_option]...
```

为MyISAM表添加一个SPATIAL索引。该索引会根据index_columns里列出的列来创建，这些列都是表里的一个或多个空间类型列，它们

之间用逗号隔开。其中的每个列都必须定义为NOT NULL。关于index_name的指定，请参考ADD INDEX动作。

```
ALTER TABLE coordinates ADD SPATIAL (x,y);
```

```
ADD [CONSTRAINT [name]] UNIQUE [INDEX | KEY]
```

```
[index_name]
```

```
(index_columns) [index_option] ...
```

为tbl_name表增加一个唯一值索引。关于index_name和index_columns的指定，请参考ADD INDEX动作。

```
ALTER TABLE absence ADD UNIQUE id_date (student_id, date);
```

```
ALTER [COLUMN] col_name {SET DEFAULT value | DROP  
DEFAULT}
```

修改指定列的默认值：既可以修改为指定的值，也可以删除当前的默认值。对于后一种情况，有可能会赋予新的隐含默认值，如3.2.3节所述。

```
ALTER TABLE grade_event ALTER category SET DEFAULT 'Q';
```

```
ALTER TABLE grade_event ALTER category DROP DEFAULT;
```

```
CHANGE [COLUMN] old_col_name new_col_name col_definition
```

```
[FIRST | AFTER col_name]
```

更改指定列的名称和定义。old_col_name和new_col_name分别是该列的当前名和新名，而col_definition 是该列的新定义。col_definition的格式与CREATE TABLE 语句所使用的格式相同，可以包含任何列属性，如NULL、NOT NULL和DEFAULT。请注意，如果想更改定义，而不是更改名字，则必须把相同的名字指定两遍。FIRST和AFTER的作用与ADD COLUMN的相同。

```
ALTER TABLE student CHANGE name name VARCHAR(40);
```

```
ALTER TABLE student CHANGE name student_name CHAR(30)  
NOT NULL;
```

```
CONVERT TO CHARACTER SET charset [COLLATE collation]
```

把给定表的默认字符集和所有的非二进制字符列转换为给定的新字符集。其中，binary的charset 值会把各个列转换为相应的二进制串数据类型，DEFAULT 会把表转换为使用其所在数据库的字符集。也可以用COLLATE子句来指定一种排序规则。如果省略了COLLATE子句，则将使用新字符集的默认排序规则。

```
DISABLE KEYS
```

对于MyISAM表，其非唯一性索引通常会在该表发生变化时及时更新，但此动作将禁止这种更新行为。可以使用ENABLE KEYS来重新启用索引更新功能。

```
ALTER TABLE score DISABLE KEYS;
```

```
DISCARD TABLESPACE
```

此动作适用于使用独立表空间的InnoDB表。本操作将删除用于存储表内容的tbl_name. idb文件。此动作不能与其他动作一起使用。

```
DROP [COLUMN] col_name [RESTRICT | CASCADE]
```

从表里删除指定的列，同时，如果该列还是某个索引的组成部分，它也将被从该索引中剔除。如果构成某个索引的所有列全被剔除了，那么该索引也将被删除。

```
ALTER TABLE president DROP suffix;
```

虽然关键字RESTRICT和CASCADE可以被解析，但是它们会被忽略，因而不会有任何作用。

```
DROP FOREIGN KEY fk_name
```

删除指定的外键定义。ADD FOREIGN KEY和DROP FOREIGN KEY 操作不能出现在同一条ALTER TABLE语句里。

```
DROP {INDEX | KEY } index_name
```

从表里删除给定的索引。

```
ALTER TABLE member DROP INDEX name;
```

```
DROP PRIMARY KEY
```

从表里删除主键。如果该表根本没有主键，此动作将报告出错。

```
ALTER TABLE president DROP PRIMARY KEY;
```

```
ENABLE KEYS
```

对于MyISAM表，重新启用被DISABLE KEYS子句禁用的非唯一性索引自动更新机制。

```
ALTER TABLE score ENABLE KEYS;
```

```
FORCE
```

执行“空”操作——它会在不更改表结构的情况下重建它。此动作在MySQL 5.5.11之前的版本里没有任何效果。

```
IMPORT TABLESPACE
```

此动作适用于使用独立表空间的InnoDB 表。本操作会把表所在的数据库目录里的tbl_name.ibd文件与该表关联起来。.ibd文件必须由同一台导入它的服务器创建。（有可能出现这样的情况，该表原有的.ibd文件已经随同DISCARD TABLESPACE被删掉了。）此动作不能与其他动作一起使用。

```
MODIFY [COLUMN] col_name col_definition [FIRST | AFTER  
col_name]
```

改变列的定义。`col_name` 是待修改的列名。列定义 `col_definition` 的格式与 `CREATE TABLE` 语句所使用的格式相同，包括所有的列特性，如 `NULL`、`NOT NULL` 和 `DEFAULT`。`FIRST` 和 `AFTER` 的作用与 `ADD COLUMN` 的相同。

```
ALTER TABLE student MODIFY name VARCHAR(40) DEFAULT "
NOT NULL;
```

```
ORDER BY col_list
```

表里的行将根据 `col_list` 里列出的列来排序，这些列都是该表里的一个或多个列，它们间之用逗号隔开。如果指定了多个子句，则本子句应该为最后一个。默认的排序顺序是升序。可以在各列名的后面加上关键字 `ASC` 或 `DESC` 来显式地指定是按升序排序，还是按降序排序。以这种方式对表进行排序能够提高按同样顺序检索行的后续查询的性能。不过，如果表在进行完 `ORDER BY` 操作之后又发生了修改，便会打乱之前排好的顺序，所以此动作只对那些今后不再会发生修改的表有用。

```
ALTER TABLE score ORDER BY event_id, student_id;
```

```
RENAME [TO | AS] new_tbl_name
```

把表重命名为 `new_tbl_name`。如果重新命名的是一个 `InnoDB` 表，其他的表又依赖于该 `InnoDB` 表的外键关系，`InnoDB` 存储引擎将调整那些依赖关系指向重新命名后的表。

```
ALTER TABLE president RENAME TO prez;
```

`ALTER TABLE` 语句还可以用来调整分区设置。关于 `CREATE TABLE` 语句的条目对以下动作描述中的 `partition_scheme` 和 `partition_definition` 术语的含义作出了定义。如果这些分区选项中的某一个出现在了 `ALTER TABLE` 语句里，则不能再使用任何其他分区选项。

```
partition_scheme
```

根据给定的分区描述对表进行分区。如果表此前没有分区，则它将变成有分区的；否则，新分区将取代老分区。

```
ADD PARTITION [partition_definition]
```

给一个已经有分区的表增加一个新的分区。

```
{ANALYZE | CHECK | OPTIMIZE | REBUILD | REPAIR |  
TRUNCATE}
```

```
PARTITION {partition_name [, partition_name] ... | ALL}
```

对指定的分区执行指定的动作。其中的每一个动作都允许使用ALL来代替分区名列表，以实现对所有分区进行操作。TRUNCATE不会处理子分区。

```
COALESCE PARTITION n
```

将某个分区表的分区减少为不超过n个，将被删除的分区里的数据合并到余下的分区里。此动作只适用于 HASH 或 KEY 分区。如果想删除 LIST 或 RANGE 分区，可以使用DROP PARTITION。

```
DROP PARTITION partition_name [, partition_name] ...
```

删除指定的分区。DROP操作只适用于LIST或RANGE分区，被删除的分区里的数据将丢失。

如果想减少HASH或KEY分区的数量，可以使用COALESCE PARTITION。

```
EXCHANGE PARTITION partition_name WITH TABLE tbl_name2
```

将指定分区表tbl_name里的指定分区与未分区表tbl_name2进行交换。除了分区之外，这两个表必须拥有相同的结构。tbl_name2里的行不能处于分区定义之外，并且不作为任何外键引用。

```
REMOVE PARTITIONING
```

删除所有的分区，最终会得到一个未分区的表。


```
REORGANIZE PARTITION partition_name [, partition_name]
...
```

```
INTO (partition_definition [, partition_definition] ...)
```

使用新的分区定义对给定分区重新分区。

```
ALTER VIEW
```

```
ALTER
```

```
[ALGORITHM = {MERGE | TEMPTABLE | UNDEFINED}]
```

```
[DEFINER = definer_name]
```

```
[SQL SECURITY = {DEFINER | INVOKER}]VIEW view_name
[(col_list)] AS select_stmt
```

```
[WITH [CASCADED | LOCAL] CHECK OPTION]
```

修改已有视图，使其具有指定的定义。各个子句的含义与CREATE VIEW 语句条目所描述的相同。

执行ALTER VIEW语句时，要求对视图拥有CREATE VIEW权限和DROP权限，并且要求对用于定义视图的那条SELECT 语句所使用到的每一个列拥有一定的权限。ALTER VIEW 语句只允许定义视图的那个用户或是拥有SUPER权限的用户使用。

```
ANALYZE TABLE
```

```
ANALYZE
```

```
[NO_WRITE_TO_BINLOG | LOCAL]
```

```
{TABLE | TABLES} tbl_name [, tbl_name] ...
```

该语句会让MySQL对每个指定的表进行分析，统计并保存每个表里索引的键值分布情况。它适用于InnoDB和MyISAM表。ANALYZE TABLE要

求对每个表都拥有SELECT和INSERT权限。

如果启用了二进制日志记录，那么在不给出NO_WRITE_TO_BINLOG或LOCAL选项时，MySQL会把ANALYZE TABLE语句写到二进制日志里。

在分析完成之后，SHOW INDEX的输出里的Cardinality 列会给出索引里有多少个互不相同的值。利用这条语句得到的分析结果，优化器就能在今后的查询里更快地完成某些特定的连接操作。

表的分析操作需要用到读锁，这样可以防止该表在分析期间被更新。如果该表已被分析过，并且之后也未被修改过，那么再次执行ANALYZE TABLE时，将不会做任何事情。

ANALYZE TABLE语句与CHECK TABLE语句产生的输出格式相同。

BEGIN

BEGIN [WORK]

这个语句是START TRANSACTION的同义词；请参考该语句条目。

BEGIN和END也可以一起用在存储程序里，用于创建复合语句（参考E.2节）。

BINLOG

BINLOG 'str'

BINLOG语句是由mysqlbinlog生成的，因此'str'是二进制日志事件的base-64编码（它是可打印的形式）。当再次执行时，服务器会对该字符串进行解码，以恢复数据更改事件。此语句需要拥有SUPER权限。

CACHE INDEX

CACHE INDEX

tbl_index_spec [, tbl_index_spec] ...

IN cache_name

tbl_index_spec:

tbl_name

[PARTITION (partition_name [, partition_name] ... | ALL)]

[[INDEX | KEY] (index_name [, index_name] ...)]

把一个或多个MyISAM表与给定的键缓存（它必须事先存在）关联起来。必须对这条语句列出的每一个表拥有INDEX权限。键缓存区的默认名字是default。表的索引可以稍后用LOAD INDEX语句加载到相应的缓存里。目前，虽然CACHE INDEX语句的语法允许只为特定的索引分配键缓存，但实际情况却是该语句将把各个表的所有索引与指定的缓存关联起来。

下面的语句将为member表的索引分配一个名为member_cache的键缓存：

```
CACHE INDEX member IN member_cache;
```

对于分区表，PARTITION子句允许对特定分区进行缓存分配。

CACHE INDEX语句与CHECK TABLE语句产生的输出格式相同。

与MyISAM键缓存管理有关的更多信息请参考12.7.2节。

CALL

CALL routine_name([proc_param [, proc_param] ...])

CALL routine_name[()]

调用有给定名字的存储过程。可选的参数列表由一个或多个以逗号分隔的参数值构成。只要参数列表里有OUT或INOUT参数，被调用的存储过程就可以通过它们返回值。

当某个存储过程返回时，我们可以通过调用`ROW_COUNT()`函数来获得它最后执行过的修改行的语句所影响的行数。在C语言程序里，我们可以通过调用`mysql_affected_rows()`函数来获得这个值。

如果某个存储过程不带任何参数，则该过程名字后面的那对括号可以省略。

CHANGE MASTER

CHANGE MASTER TO option [, option] ...

改变主从复制机制中从服务器上的配置参数，以表明要使用哪个主服务器主机、如何与之建立连接，或者使用哪些日志。参数保存在从服务器的`master.info`和`relay_log.info`文件中，用于随后的从服务器重启。其中的许多参数也都可以`SHOW SLAVE STATUS`的输出里看到。

每个option都是用`param = value`格式来指定参数定义的。所有的定义参数如下所示。

IGNORE_SERVER_IDS = (server_id_list)

它可以让从服务器忽略 `server_id_list` 里列出的任何服务器的事件，`server_id_list`是一个由零个或多个以逗号隔开的服务器 ID 列表。如果要清除一组被忽略的服务器，则可以指定一个空列表。

MASTER_BIND = 'interface'

它指的是连接主服务器时绑定的IP地址。此选项是在MySQL 5.6.1引入的。

MASTER_CONNECT_RETRY = n

试图连接主服务器的各次尝试之间的等待秒数。

MASTER_DELAY = n

它指的是复制延迟的秒数。从服务器会在一个事件在主服务器执行之后等待至少n秒才会执行下一个事件。默认值为0。此选项是在

MySQL 5.6.0引入的。

`MASTER_HEARTBEAT_PERIOD = interval`

当经过心跳间隔的时长之后，如果主服务器没有任何要写入二进制日志的事件，那么它会向从服务器发送一个心跳包。此选项用于告知主服务器，心跳间隔的是多少（单位为秒）。这个值可以包含小数部分（单位为微秒）。当值为 0 时，会禁用心跳功能。其默认值为 `slave_net_timeout/2`。语句 `RESET SLAVE` 可以将心跳间隔重置为默认值。

`MASTER_HOST = 'host_name'`

它指的是运行主服务器的那台主机。

`MASTER_LOG_FILE = 'file_name'`

主服务器的某个二进制日志文件的名称，从服务器将从这个文件开始或者继续去建立复制机制。

`MASTER_LOG_POS = n`

主服务器的某个二进制日志文件中的某个位置，从服务器将从这个位置开始或者继续去建立复制机制。

`MASTER_PASSWORD = 'pass_val'`

连接主服务器时使用的密码。

`MASTER_PORT = n`

连接主服务器时使用的TCP/IP端口号。

`MASTER_RETRY_COUNT = n`

它指的是从服务器在放弃之前尝试连接主服务器的次数。此选项是在MySQL 5.6.1里引入的。

`MASTER_SSL = {0 | 1}`

MASTER_SSL_CA = 'file_name'

MASTER_SSL_CAPATH = 'dir_name'

MASTER_SSL_CERT = 'file_name'

MASTER_SSL_CIPHER = 'str'

MASTER_SSL_CRL = 'file_name'

MASTER_SSL_CRLPATH = 'dir_name'

MASTER_SSL_KEY = 'file_name'

MASTER_SSL_VERIFY_SERVER_CERT = {0 | 1}

这些选项指定的是与主服务器建立 SSL连接时的参数。它们与 F. 1. 2. 1节描述的--ssl-xxx选项具有相同的含义。从服务器会把这些选项的值保存在它的master.info 文件里，但是它们不会有任何效果，除非从服务器启用了 SSL 支持。选项 MASTER_SSL_CRL和 MASTER_CRLPATH是在MySQL 5. 6. 3里开始引入的。

MASTER_USER = 'user_name'

连接主服务器时使用的账户名。自MySQL 5. 6. 4起，将这个值设置为NULL或空字符串，或者在指定MASTER_PASSWORD时不设置它，都会出现错误。

RELAY_LOG_FILE = 'file_name'

从服务器的中继日志文件名。

RELAY_LOG_POS = n

从服务器中继日志文件里的当前位置。

没有在语句里指定的参数将保留它们的当前值，但有一种情况除外：在更改 MASTER_HOST 或MASTER_PORT时，通常表明的是要切换到

一个不同的主服务器上，所以如果指定其中的某一个选项，那么MASTER_LOG_FILE和MASTER_LOG_POS的值会被设置成主服务器的第一个二进制日志文件的起始部分。

不能在同一条语句里混合使用MASTER_LOG_FILE、MASTER_LOG_POS选项与 RELAY_LOG_FILE、RELAY_LOG_POS选项。

如果没有指定RELAY_LOG_FILE或RELAY_LOG_POS选项，那么CHANGE MASTER语句会删除现有的所有中继日志文件，并创建一个新的文件。

CHECK TABLE

```
CHECK {TABLE | TABLES} tbl_name [, tbl_name] ...[option] ...
```

检查表的错误。这条语句适用于MyISAM表、InnoDB表、ARCHIVE表，以及CSV表。CHECK TABLE 还可以检查视图定义的问题，如引用了不存在的表。它要求必须拥有对检查表或视图的SELECT权限。

对于InnoDB表，如果服务器发现问题，会在错误日志里编写一条信息后终止运行，防止发生进一步的错误。对于MyISAM表，CHECK TABLE也更新索引统计信息。

每个option 值可以是以下选项当中的某一个。如果没有进行特别说明，这些选项都适用于MyISAM表；而其他存储引擎可能会忽略它们。

CHANGED。它表示的是，如果表正常关闭，并且自上次检查以来未被更改，则跳过表检查。

EXTEND。它表示的是，执行扩展检查，以试图确保表完全一致。这是最全面的检查，但也是执行速度最慢的。例如，它将检查每个索引中的每个键是否都指向一个数据行。

FAST。它表示的是，仅当表正确关闭时检查它。

MEDIUM。它表示的是，检查索引、扫描行的问题，并进行校验和验证。如果未给出选项，则默认为它。

QUICK。它表示的是，只扫描索引，而不扫描数据行。此选项适用于InnoDB表和MyISAM表。

FOR UPGRADE。它表示的是，确定被检查表是否与当前的MySQL版本相兼容，所以此选项在升级后很有用。只要发现有不兼容的地方，MySQL 服务器便会执行全面检查。如果全面检查不成功，则应该尝试修复该表。除非有不兼容之处并且全面检查未能成功，否则MySQL服务器将根据当前的MySQL版本升级表的.frm 文件。此选项的适用范围并不仅限于MyISAM表。

如果不是使用FOR UPGRADE选项来检查表，并且在检查MyISAM表时没有给出QUICK、MEDIUM或EXTENDED 选项中的任何一个，那么CHECK TABLE 语句将默认使用MEDIUM 选项来检查那些行长度可变的表。如果行长度是固定的，并且指定了CHANGED或FAST选项，那么CHECK TABLE 语句将默认使用QUICK选项进行检查；否则，默认使用MEDIUM选项。

在某些情况下，CHECK TABLE也可能会修改表，但是这种修改仅仅是设置一个内部标志而已。例如，有一个表被标记为损坏，或者未正常关闭，但是检查时并未发现任何问题，那么CHECK TABLE会将该表标记为完好。

CHECK TABLE语句会返回一些关于操作结果的信息。例如：

```
mysql> CHECK TABLE t;
```

Table	Op	Msg_type	Msg_text
test.t	check	status	OK

ANALYZE TABLE、CACHE INDEX、LOAD INDEX INTO CACHE、OPTIMIZE TABLE和REPAIR TABLE语句也会以这种格式返回信息。其中，Table 列表示的是操作处理的表。Op 列表示的是语句所执行那个操作的操作。Msg_type和Msg_text列则是关于这次操作结果的信息

（如果这两个输出列里的值表明表状态不佳或是尚未更新到最新，那么你应该修复它）。

CHECKSUM TABLE

CHECKSUM {TABLE | TABLES} tbl_name [, tbl_name] ...

[QUICK | EXTENDED]

报告表的校验和。对于分区表，此条语句在MySQL 5.6.4之前的版本里会返回 0，除非指定了EXTENDED。CHECKSUM TABLE要求对检查的每个表拥有SELECT权限。

```
mysql> CHECKSUM TABLE president;
```

Table	Checksum
sampdb.president	3032762697

如果给定的表不存在，则Checksum值为NULL，并且会生成一条警告信息。

默认情况下，只要存储引擎支持，这条语句将报告实时校验和。（实时校验和会随着表的每次修改而更新。）对于MyISAM表，你可以通过在相应的CREATE TABLE或ALTER TABLE语句里加上CHECKSUM = 1选项来启用实时校验和更新。

如果在CHECKSUM TABLE 语句里使用了QUICK 选项，则该语句将在给定表有实时校验和的情况下返回报告；而其他情况都将返回NULL。如果使用的是EXTENDED选项，则MySQL会读取整个表来计算其校验和并返回。此操作会随着表长度的增加而变得越来越慢。

COMMIT

COMMIT [WORK] [AND [NO] CHAIN] [[NO] RELEASE]

提交当前事务里的各条语句对表的修改，把那些修改永久性地记录到数据库里。COMMIT语句只适用于支持事务处理的存储引擎。（对于非事务型存储引擎，语句在执行完毕后会立刻提交。）

可选关键字WORK目前没有任何效果。CHAIN和RELEASE子句对MySQL服务器在事务结束时的处理有影响。如果使用了AND CHAIN子句，则MySQL服务器将在当前事务结束后以相同的隔离级别开始执行另一个事务。如果使用了RELEASE子句，则MySQL服务器将在当前事务结束后终止当前会话。为CHAIN或RELEASE子句加上NO之后，将分别使MySQL服务器不会立即开始新事务，或不会终止当前会话。在没有这些子句的情况下，COMMIT语句的行为将取决于系统变量completion_type的值。默认情况下，MySQL服务器不会应用CHAIN和RELEASE。

如果事先没有借助START TRANSACTION语句或者采用把autocommit变量设置为0的办法来禁用自动提交功能，那么COMMIT语句将没有任何效果。

有些语句无法成为事务的组成部分，它们会隐式结束当前事务，就像执行了COMMIT语句那样。通常情况下，那些用来创建、更改或删除数据库或其中的对象的数据定义语言（Data Definition Language，DDL）语句，以及那些与锁定有关的语句都不能成为事务的一部分。例如，在事务进行过程中调用了下列语句当中的任何一条，那么服务器会在执行这条语句之前提交当前事务：

ALTER TABLE

CREATE INDEX

DROP DATABASE

DROP INDEX

DROP TABLE

LOCK TABLES

RENAME TABLE

SET autocommit = 1 (如果尚未设置为1)

TRUNCATE TABLE

UNLOCK TABLES (如果表当前未被锁定)

从MySQL参考手册可以查到：当前使用的MySQL版本里都有哪些语句会导致MySQL服务器隐式地提交当前事务。

CREATE DATABASE

CREATE DATABASE [IF NOT EXISTS] db_name [db_attr] ...

db_attr:

[DEFAULT] CHARACTER SET [=] charset

| [DEFAULT] COLLATE [=] collation

用给定名字创建一个数据库。必须要拥有该数据库的CREATE 权限。如果要创建的数据库已经存在，那么执行这条语句会导致一个错误；如果指定了 IF NOT EXISTS 子句，则数据库将不会被创建，但是这条语句也不会报告出错。

在数据库名之后可以给出可选的CHARACTER SET和COLLATE属性，为数据库指定默认的字符集和排序规则。这些属性用于没有显式指定字符集或排序规则的表。charset可以是字符集名，或者DEFAULT—表示使用当前的服务器字符集。collation可以是排序规则名称，或者DEFAULT—表示使用当前服务器的排序规则。

如果没有给出上述任何属性，新数据库将使用服务器级字符集和排序规则。如果只有指定了CHARACTER SET属性，但是没有给出COLLATE属性，那么新建数据库将使用给定字符集的默认排序规则。如果只有指定了COLLATE属性，但是没有给出CHARACTER SET属性，那么MySQL服务器会根据你给定的排序规则来确定字符集。如果想要同时给

出CHARACTER SET和COLLATE属性，那么指定的排序规则和字符集必须相互兼容。

MySQL会把数据库的属性保存在数据库子目录中的db.opt文件里。

CREATE EVENT

CREATE

[DEFINER = definer_name]

EVENT [IF NOT EXISTS] event_name

ON SCHEDULE schedule

[ON COMPLETION [NOT] PRESERVE]

[ENABLE | DISABLE | DISABLE ON SLAVE]

[COMMENT 'str']

[DO event_stmt]

schedule:

AT datetime

| EVERY expr interval [STARTS datetime] [ENDS datetime]

为事件调度器创建一个名为event_name的新事件。在创建时，必须要拥有新事件所属数据库的EVENT权限。默认情况下，新事件将被创建在默认数据库里。如果想在特定的数据库里创建新事件，则必须以db_name.event_name格式给出它的名字。

DEFINER子句的作用是在执行某个事件时确定其安全上下文（即用来核查其访问权限的账户），详细情况请参考4.3节。默认使用执行CREATE EVENT语句的那个用户的账户。

子句ON SCHEDULE的作用是为新事件安排执行时间表（假设事件调度器正在运行中）。在这个子句的各种格式里，datetime 代表的是日期/时间值。CURRENT_TIMESTAMP() 函数（或它的同义词）可以用来表示当前日期和时间。datetime 表达式可以使用INTERVAL expr interval 运算来加上或减去一个时态间隔。详细的语法格式描述，请参考C. 2. 5节中的DATE_ADD() 函数条目。这里的interval值不应该使用任何与微秒有关的限定符。

在ON SCHEDULE子句里，AT形式的设置项将使得该事件在指定时刻执行且仅执行一次。EVERY形式的设置项将使得该事件定期重复执行。重复时间由一个数值和一个用来表明如何解释该数值的interval修饰符构成（例如，5 HOUR或'1:30' MINUTE_SECOND）。默认情况下，新事件的首次执行发生在它刚被创建时，然后再按照设定的时间间隔重复执行。可以用STARTS 子句来设定事件的首次执行时间，用ENDS子句来设定事件从何时起不再执行。在ON SCHEDULE子句里，不应该出现表引用，也不应该出现指向存储函数或用户定义函数的引用。

DO 子句用来给出事件发生时将要执行的语句。它应该只包含一条SQL语句。如果需要使用多条语句，必须用关键字BEGIN和END把它们括起来以构成一个复合语句。（更多信息请参考E. 2节。）默认情况下，事件在它最后一次执行完毕之后将被删除。ON COMPLETION NOT PRESERVE 子句显式指定了这种行为。ON COMPLETION PRESERVE子句将导致事件不会被删除。

选项ENABLE和DISABLE分别表示的是事件在被创建出来后的初始状态是“启用”（按计划运行）还是“禁用”（不运行）。DISABLE ON SALVE选项的含义是，事件在创建它的那个MySQL服务器上是用启用的，但是在复制它的任何从服务器上都是禁用的。

在事件创建时生效的sql_mode系统变量的值会被保存起来并在事件发生时恢复生效。

事件既不需要输入参数，也不会产生输出信息。也就是说，无法向事件传递参数，并且输出信息（如SELECT语句生成的结果集）将被删除。

CREATE FUNCTION、CREATE PROCEDURE

CREATE

[DEFINER = definer_name]

FUNCTION routine_name ([func_param [, func_param] ...])

RETURNS type

[characteristic] ...

routine_stmt

CREATE

[DEFINER = definer_name]

PROCEDURE routine_name ([proc_param [, proc_param] ...])

[characteristic] ...

routine_stmt

func_param:

param_name type

proc_param:

[IN | OUT | INOUT] param_name type

characteristic:

COMMENT 'str'

| {CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES
SQL DATA}

| [NOT] DETERMINISTIC

| LANGUAGE SQL

| SQL SECURITY {DEFINER | INVOKER}

这些语句可用来创建新的存储例程（函数和过程）。在使用这些语句创建存储例程时，必须拥有CREATE ROUTINE权限。

默认情况下，新例程将被创建在当前的默认数据库里。如果想在某个数据库里创建一个新例程，就应该以 db_name.routine_name的格式来给出它的名字。在同一个数据库里，任意两个函数或两个过程不允许有同样的名字，但函数可以和过程有同样的名字。

存储函数的参数是通过给出参数名及其类型的方式定义的。参数类型可以是任何一种有效的MySQL数据类型。参数的作用是在函数被调用时把参数值传递给它，但参数值的变化在函数返回时对调用者而言是不可见的。（即函数的参数会被当作IN参数。）

在定义函数的时候，参数列表后面必须加上一条RETURNS语句，以表明返回值的数据类型。

存储过程的参数也是通过给出参数名及其类型的方式定义的，但在参数名的前面还可以加上IN、OUT或INOUT，以表明该参数是仅用于输入、仅用于输出，还是可以同时用于输入和输出。如果没加，则默认为IN。

IN 参数用来把值传递给存储过程。这种参数可以在存储过程里修改，但在存储过程终止之后从调用程序里看不出任何改变。

OUT 参数不是用来向存储过程传递值的。它在存储过程里的初始值是 NULL，允许在存储过程里修改。OUT参数的最终结果在存储过程终止之后对调用程序而言是可见的。

INOUT参数可以把一个值传递给存储过程，其最终结果在存储过程终止之后对调用者而言是可见的。

可以指定一个或多个characteristic值，它们之间以空格隔开的下列选项构成。

COMMENT

例程的描述性说明。此说明信息可以通过用于显示例程信息的SHOW语句显示出来。

CONTAINS SQL、NO SQL、READS SQL DATA、MODIFIES SQL DATA

这些特性提供的是与例程如何访问数据有关的提示。在MySQL里，这些特性不会对“服务器实际上允许例程执行什么样的语句”产生任何影响。

CONTAINS SQL：例程包含SQL语句。如果没有指定数据访问特性，则默认为这个值。

NO SQL：例程不包含SQL语句。

READS SQL DATA：例程包含的是只读SQL语句，不能修改数据。

MODIFIES SQL DATA：例程包含可修改数据的SQL语句。

DETERMINISTIC、NOT DETERMINISTIC

DETERMINISTIC表示的是，函数在使用同样的参数值调用时，总是会产生相同的结果。NOT DETERMINISTIC 表示的是，结果不一定相同。例如，使用NOW()作为返回值的函数可能是不确定性的。

LANGUAGE SQL

表明的是例程语言。此选项会被解析，但会被忽略。MySQL只支持用SQL语言编写存储例程。

SQL SECURITY

此特性与DEFINER子句一起使用，决定着例程执行时的安全上下文（即用于核查其访问权限的账户），请参考4.3节。如果没有给出DEFINER，则默认使用执行CREATE语句的那个用户的账户。就哪些权限

会被检查而言，该账户必须对能够被调用的那个例程拥有 EXECUTE 权限。默认情况下，MySQL会自动将EXECUTE和ALTER ROUTINE权限分配给例程的创建用户，并且会在例程被删除时恢复它们。如果要关闭这种行为，可以禁用系统变量automatic_sp_privileges。

routine_stmt 是例程主体，是一个SQL语句。它应该是单条 SQL 语句。如果需要使用多条语句，必须用关键字BEGIN和END把它们括起来以构成一个复合语句（参考E.2节）。

函数要向调用者返回一个值，所以在函数体内必须包含至少一条 RETURN 语句。不过，函数无法执行那些会生成一个结果集的语句。

在创建例程时，系统变量sql_mode的当前值会被保存起来，以便在例程执行时使用。

CREATE INDEX

CREATE [UNIQUE | FULLTEXT | SPATIAL] INDEX index_name

ON tbl_name (index_columns) [index_option] ...

index_option:

index_type

| COMMENT 'str'

| KEY_BLOCK_SIZE [=] n

| WITH PARSER parser_name

index_type: USING {BTREE | HASH | RTREE}

给tbl_name表增加一个名为index_name的新索引。新索引会根据index_columns里给出的列来创建，这些列是表里的一个或多个列，它们之间用逗号隔开。CREATE INDEX语句在MySQL内部是被当作ALTER TABLE语句来处理的。更多详细信息请参考ALTER TABLE条目。如果需

要为某个表创建多个索引，最好使用ALTER TABLE 语句。只用一条语句就可以添加所有的索引，这比逐个创建要快得多。

默认情况下，创建的是非唯一性索引。UNIQUE、FULLTEXT或SPATIAL关键字可以用来表明索引的类型。CREATE INDEX 语句不能用来创建 PRIMARY KEY，必须使用ALTER TABLE 语句来创建。

FULLTEXT索引可用于MyISAM表或InnoDB表（自MySQL 5.6.4起），但是只能用于非二进制串列（CHAR、VARCHAR、TEXT）。SPATIAL索引只适用于MyISAM表，并且只适用于具有NOT NULL属性的空间类型列。

在索引定义的末尾，可以使用下面这些index_option值。

index_type。它表示的是索引算法，该算法可用于多种存储引擎。算法值包括适用于MyISAM表和InnoDB表的BTREE、适用于MEMORY表的HASH或BTREE，以及MyISAM表里用于SPATIAL索引的RTREE。

COMMENT 'str'。它提供的是对索引的描述性说明（最多1024个字符）。此选项是在MySQL 5.5.3里引入的。

KEY_BLOCK_SIZE [=] n。它会建议存储引擎使用n个字节作为新索引的键块长度。n值为0时表示使用默认长度。

WITH PARSER parser_name。它只适用于FULLTEXT索引。其作用是，为新索引指定一个全文解析器插件。与解析器插件有关的详细内容请参考MySQL参考手册。

与索引创建有关的信息请参考2.6.4节。

CREATE TABLE

CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name

{

(create_definition,...) [table_option] ...

[partition_scheme] [trailing_select]

| [(create_definition,...) [table_option] ...

[partition_scheme] trailing_select

| LIKE tbl_name2

| (LIKE tbl_name2)

}

table_option: (参考后面的讨论)

trailing_select:

[IGNORE | REPLACE] [AS] select_stmt

create_definition:

col_name col_definition [reference_definition]

| [CONSTRAINT [name]] PRIMARY KEY

[index_name]

(index_columns) [index_option] ...

| [CONSTRAINT [name]] UNIQUE [INDEX | KEY]

[index_name]

(index_columns) [index_option] ...

| {INDEX | KEY} [index_name]

(index_columns) [index_option] ...

| {FULLTEXT | SPATIAL} [INDEX | KEY]

[index_name] (index_columns) [index_option] ...

| [CONSTRAINT [name]] FOREIGN KEY [fk_name]

(index_columns) [reference_definition]

| CHECK (expr)

col_definition:

data_type

[NOT NULL | NULL] [DEFAULT default_value]

[AUTO_INCREMENT] [PRIMARY KEY] [UNIQUE [KEY]]

[COMMENT 'str']

index_option: (参考后面的讨论)

reference_definition:

REFERENCES tbl_name (index_columns)

[MATCH FULL | MATCH PARTIAL | MATCH SIMPLE]

[ON DELETE reference_action]

[ON UPDATE reference_action]

reference_action:

RESTRICT | CASCADE | SET NULL | NO ACTION | SET DEFAULT

partition_scheme:

PARTITION BY

```

{
RANGE (expr)

    | RANGE COLUMNS (col_list)

    | LIST (expr)

    | LIST COLUMNS (col_list)

    | [LINEAR] HASH (expr)

    | [LINEAR] KEY (col_list)

}

[PARTITIONS n]

[SUBPARTITION BY

{

[LINEAR] HASH (expr)

| [LINEAR] KEY (col_list)

}

[SUBPARTITIONS n]

]

[(partition_definition [, partition_definition] ...)]

partition_definition:

PARTITION partition_name

```

[VALUES {LESS THAN {(expr) | MAXVALUE} | IN (value_list)}]

[partition_option] ...

[(subpartition_definition [, subpartition_definition] ...)]

subpartition_definition:

SUBPARTITION subpartition_name

[partition_option] ...

partition_option: (参考后面的讨论)

CREATE TABLE语句会在默认数据库里创建一个新的名为tbl_name的表。如果需要在指定数据库里创建表，则必须以 db_name.tbl_name 格式给出表名。执行此语句时，要求对该表拥有CREATE权限。

一般情况下，如果想要创建的表已存在，那么这条语句将执行失败，并且会报告错误。但是，这里有两种例外情况。首先，如果指定了 IF NOT EXISTS 子句，表将不会被创建，这条语句也不会报告出错。其次，如果指定了TEMPORARY，并且原来那个表不是一个临时表，那么这条语句会创建一个临时表，并且在临时表存在期间，客户端将无法看到原来那个名为tbl_name的表。原来那个表之所以仍然能被其他客户端看到，那是因为临时表只对创建它的客户端可见的。如果用 DROP TABLE 语句删除临时表，或者把它重新命名，那么可以再次看到原有的那个表。在创建临时表时，必须拥有CREATE TEMPORARY TABLE 权限。

如果指定了TEMPORARY关键字，则临时表将一直存在，直到当前客户端会话结束或者使用DROP TABLE语句将其删除。

列表 create_definition 列出的是要创建的各个列和索引。如果新表是通过结尾的SELECT语句创建的，那么该列表是可选的。table_option 值用于指定表的各种属性。如果需要将表分区存储，可以使用partition_scheme来定义各种分区特性。如果指定了结尾的 select_stmt（以任意的SELECT语句形式），则新表将使用它所返回的

结果集来创建。结尾的LIKE子句会把新表创建为已有表的一个空副本。后面有几节会更加完整地描述这些子句。

列和索引的定义。create_definition可以是一个列定义、一个索引定义、一个FOREIGN KEY子句或一个CHECK子句。CHECK子句可以被解析，但会被忽略。FOREIGN KEY子句的处理方式相类似，但是InnoDB表除外。

列定义 col_definition的开头是一个数据类型 data_type，后面通常还会有几个可选的关键字。该类型可以是附录 B里列出的任何一种。每种列类型都有一些特有的属性，详细讨论请参考附录B。允许出现在数据类型之后的其他可选关键字如下所示。

NULL或NOT NULL

用来表明该列是否允许包含NULL值。如果这两个关键字都没有给出，则默认为NULL。

DEFAULT default_value

用来设定该列的默认值。它不能用于 BLOB 或 TEXT 类型、空间类型，或者带有AUTO_INCREMENT属性的列。除了TIMESTAMP和DATETIME（自MySQL 5.6.5起），列的默认值都必须是一个常数，并且可以指定为数字、字符串或者NULL。在没有包含DEFAULT子句时，MySQL也可以分配默认值——与之相关的规则，参考3.2.3节。

AUTO_INCREMENT

这个关键字只能用在整数和浮点类型的列上。AUTO_INCREMENT列的特殊之处在于：当你往其中插入NULL值时，实际插入的数据值将是列序列的下一个编号值。它通常等于该列里的当前最大编号值再加上1。默认情况下，AUTO_INCREMENT列的实际取值将从1开始编号。

（有些存储引擎可以通过表选项AUTO_INCREMENT来设定起始编号值。请参考下面对表选项的介绍。）AUTO_INCREMENT列必须被索引，而且不能为NOT NULL。每个表最多只能有一个AUTO_INCREMENT列。

[PRIMARY] KEY

用来表明该列是一个PRIMARY KEY。PRIMARY KEY列必须是NOT NULL的，因此如果你省略这个项的话，MySQL会向列定义中添加NOT NULL。

UNIQUE [KEY]

用来表明该列是一个UNIQUE索引。

COMMENT 'str'

列的描述性注释（最多1024个字符）。此属性可以用SHOW CREATE TABLE和SHOW FULL COLUMNS语句来查看。

子句PRIMARY KEY、UNIQUE、INDEX、KEY、FULLTEXT和SPATIAL的用途是创建索引。子句PRIMARY KEY和UNIQUE所创建的索引不允许包含相同的值。INDEX和KEY互为同义词，它们所创建的索引允许包含彼此相同的值。新索引会根据 index_columns 里给出的列来创建，这些列是表里的一个或多个列，它们之间用逗号隔开。如果没有给出索引名字 index_name，则MySQL会自动选择第一个索引列的名字作为此索引的名字。

FULLTEXT索引可用于MyISAM表或InnoDB表（自MySQL 5.6.4起），但是仅用于非二进制串列（CHAR、VARCHAR、TEXT）。SPATIAL索引只适用于MyISAM表，并且只适用于具有NOT NULL属性的空间类型列。

在索引定义的最后，index_option的取值与CREATE INDEX条目所述的取值完全一致。与索引创建有关的更多信息请参考2.6.4节。

表选项。每个table_option值指定一个表特性，如下所示。每个选项设置都能应用于所有存储引擎，除非有特别说明。各个设置值之间可以用空白或逗号隔开。

AUTO_INCREMENT [=] n

为表生成一个起始编号值。此选项可用于 InnoDB 表、MyISAM 表和MEMORY 表。对于InnoDB表，如果在生成任意AUTO_INCREMENT值之前重启服务器，这个项将失效。

AVG_ROW_LENGTH [=] n

表的行平均长度。对于MyISAM表，MySQL将根据AVG_ROW_LENGTH和MAX_ROWS的乘积来确定数据文件的最大长度。MyISAM存储引擎内部使用的行指针的宽度可以是2~7个字节。

这个指针的默认宽度足以创建出长度高达256 TB的表来。如果需要用更大的表（并且你的操作系统也支持如此之大的文件），可以利用MAX_ROWS和AVG_ROW_LENGTH表选项来调整MyISAM存储引擎内部使用的行指针的宽度。这两个选项值的乘积越大，存储引擎内部使用的行指针的宽度也就越大，文件的大小可高达 65 536 TB。反之，乘积越小，可使用的指针越小。如果表本身的尺寸比较小，那么这种做法也不能节省多少空间，但是如果有很多小尺寸的表，则累积起来的节省总量会非常可观。

如果要直接设置数据指针的大小，可以在创建表之前设置系统变量 `myisam_data_pointer_size`。

`[DEFAULT] CHARACTER SET [=] charset`

表的默认字符集。`charset` 可以是某个字符集的名字，也可以是DEFAULT（即使用数据库的字符集）。如果在定义字符串列时没有显式表明它使用哪一个字符集，那么该列里的数据值将使用本选项所确定的字符集。在下面示例里，`c1`和`c2`将分别使用字符集`sjis`和`ujis`：

```
CREATE TABLE t
(
  c1 CHAR(50) CHARACTER SET sjis,
  c2 CHAR(50)
) CHARACTER SET ujis;
```

这个表选项也会应用到后续的表修改操作（它是ALTER TABLE在更改未明确指明字符集的字符型列定义时产生的）。

`CHECKSUM [=] {0 | 1}`

如果此选项被设置为1，MySQL将为表生成一个校验和；修改表时也会修改校验和。校验和会给表的更新操作稍微增加一点儿开销，但能提高表检查操作的工作效率。（此选项只适用于MyISAM表。）

`[DEFAULT] COLLATE [=] collation`

表的默认字符集排序规则。collation 可能是一个排序规则名称，也可能是 DEFAULT（即使用表的字符集的默认排序规则）。

`COMMENT [=] 'str'`

表的描述性注释（最大为2048个字符）。通过SHOW CREATE TABLE和SHOW TABLE STATUS语句可以查看注释内容。

`DATA DIRECTORY [=] 'dir_name'`

此选项只适用于MyISAM表和Unix。它用来规定数据文件（即.MYD文件）必须写到指定的目录里去。'dir_name'必须是一个完整的路径名。此选项只能工作在服务器未使用--skip-symlink-links选项启动的场合里。在某些Unix变体上，符号链接不是线程安全的，并且默认是被禁用的。对于分区表，此选项会被忽略。如果启用了系统变量keep_files_on_create，那么当指定目录里已存在该表的.MYD文件时，会发生一个错误。

`DELAY_KEY_WRITE [=] {0 | 1}`

如果此选项被设置为1，表的键缓存将只是偶尔刷新，而不是每次插入操作之后都立刻刷新。这能提高性能，但如果发生崩溃，可能需要修复数据库。（此选项只适用于MyISAM表。）

`ENGINE [=] engine_name`

为表指定一种存储引擎。如果未对MySQL服务器进行配置，则默认存储引擎为InnoDB。如果想在启用服务器时使用另一种默认引擎，可以参考12.5.2节的相关指导。SHOW ENGINES语句可用于查询给定的MySQL服务器所支持的存储引擎。如果在创建某个表时为它指定的存储引擎不可用，则此语句的执行效果取决于对SQL模式

NO_ENGINE_SUBSTITUTION的设置。对各种存储引擎的描述请参考2.6.1节。

INDEX DIRECTORY [=] 'dir_name'

此选项与DATA DIRECTORY相似(并且拥有相同的限制),但指定的是编写索引文件(.MYI)的目录。

INSERT_METHOD [=] {NO | FIRST | LAST}

此选项用来设定MERGE表将如何插入行。NO 表示根本不允许插入行。FIRST 或LAST 则表示行将被插到构成该MERGE表的那些MyISAM表中的第一个或最后一个。

KEY_BLOCK_SIZE [=] n

建议存储引擎使用n个字节作为索引的键块长度。n值为0表示使用默认的长度。如果某个索引定义了自己的KEY_BLOCK_SIZE选项,该选项的设置值将取代相应的表级默认值。

MAX_ROWS [=] n

向存储引擎表明打算存放到表里的行的最大个数。表在创建之后将至少允许这么多行。关于这个值的使用情况,请参考对AVG_ROW_LENGTH选项的描述。

MIN_ROWS [=] n

向存储引擎表明打算存放到表里的行的最小个数。此选项主要用于 MEMORY 表,它能向MEMORY存储引擎提供一些内存优化方面的提示。

PACK_KEYS [=] {0 | 1 | DEFAULT}

此选项控制着MyISAM表中的索引压缩功能,即是否需要对相似的索引值进行压缩。索引压缩功能会增加表更新操作的开销,但能改善检索操作的性能。0表示不压缩。1表示对字符串(Char、VARCHAR、BINARY和VARBINARY)值以及数值型索引值压缩。DEFAULT表示只对长字符串列进行压缩。

ROW_FORMAT [=]

{DEFAULT | COMPACT | COMPRESSED | DYNAMIC | FIXED | REDUNDANT}

行存储格式。其中，DEFAULT 允许存储引擎选择默认格式。如果指定的行格式不能被使用，则存储引擎可以忽略此选项。例如，如果表包含BLOB或TEXT列，就不能使用FIXED。使用SHOW TABLE STATUS，并检查Row_format的值，可以看到存储引擎实际使用的格式类型。

对于 InnoDB 表，默认格式是 COMPACT。对于更早一点的原始 InnoDB 格式，可以使用REDUNDANT 来显式指定。如果启用那些允许 InnoDB创建Barracuda文件格式的表的系统变量，那么还可以选择其他的行存储格式。为此，可设置 innodb_file_per_table=1和 innodb_file_format=Barracuda（参考 12.5.3.1 节的第 4 小节）。然后，可以将ROW_FORMAT的值指定为COMPRESSED或DYNAMIC。更多与这些行格式的特点相关的信息请参考5.4节。

对于MyISAM表，选项值DYNAMIC 或FIXED 分别对应于可变长度和固定长度的行格式。

如果使用myisampack程序来压缩MyISAM表（它也会使其成为只读的），则myisampack会将行格式设置为COMPRESSED。

UNION [=] (tbl_list)

此选项只适用于MERGE表。它列出了构成MERGE表的各个MyISAM表，以逗号分隔。

结尾的SELECT语句。如果指定了select_stmt子句（作为结尾的SELECT命令），则MySQL将使用该子句所返回的结果集来创建新表。对于那些会导致唯一性索引出现重复值的行，MySQL将按以下原则处理：如果指定了 IGNORE，则表示忽略后出现的行；如果指定了 REPLACE，则用后出现的行替换先出现的行。如果这两个关键字都没有给出，则此语句将异常终止，并报告出错。

结尾的LIKE子句。如果在结尾指定了LIKE tbl_name2子句，则新创建的表将是tbl_name2表的一个空白副本。你必须拥有tbl_name2表

的SELECT权限。新表将包括同样的列定义、索引定义和表选项，但是不会复制DATA DIRECTORY和INDEX DIRECTORY表选项，也不复制外键定义。

外键支持。InnoDB存储引擎提供了外键支持机制。外键必须通过在子表里使用FOREIGN KEY关键字进行定义，在这个关键字的后面是一个可选的外键ID，然后是构成这一外键的列清单，最后是一个REFERENCES定义。外键ID即使给出也会被忽略，除非InnoDB自动为外键创建一个索引，也就是说，fk_name将成为索引名称。在外键的REFERENCES定义里，需要列出该外键所用到的父表和列，并规定父表中的行被删除时应该采取什么动作。默认动作是防止删除或更新父表或子表，这会危害引用的整体性。动作RESTRICT和NO ACTION都表示不指定任何动作。子句ON DEFAULT和ON UPDATE可用于指定显式动作。InnoDB实现的动作是CASCADE（删除或更新相应的子表行）和SET NULL（把相应的子表行中的外键列设置为 NULL）。SET DEFAULT 动作没有实现，InnoDB会报告出错。

REFERENCE定义中的MATCH子句可以被解析，但是不会被使用。（不过，应该避免使用MATCH，因为它会导致ON DELETE和ON UPDATE被忽略。）如果为InnoDB之外的存储引擎定义了外键，则整个定义都将被忽略。

与外键有关的更多讨论请参考2.13节。

分区选项。MySQL支持对表分区。我们可以在定义表的时候对它进行分区并安排其数据被存储到不同的分区。接下来的讨论将对表分区的定义语法进行简要的描述。对此问题的深入讨论和相关示例请参考2.6.2.5节，更多信息请参考MySQL参考手册。

在表分区的描述里，开始是 PARTITION BY；接着是一个用于为每个表行计算值的分区函数，或者是一个列名列表。行的函数值或列值可用来确定把行存储到哪一个分区。分区定义还可以包含下面这些可选的组成部分。

PARTITIONS n子句用来表明表有多少个分区，其中，n应该是一个正整数。如果指定了此子句，并且还指定了一些 partition_definition 子句，则必须要有n个这样的定义。

每个表的分区个数是1024，其中包括子分区。

一条关于如何把分区进一步划分成子分区的描述。

一组用来定义各个分区的`partition_definition` 子句。每个 `partition_definition`子句定义了一个分区特性。除了为分区提供的一个名字，它还可以包括一个用来描述都有哪些分区函数值将被映射到该分区的`VALUES`子句、其他分区选项和一组子分区定义。每个 `subpartition_definition` 子句与之相似，只不过它用于描述子分区，并且不允许包含`VALUES`子句或子分区定义。

下面列出了将表里的行分配到不同分区的方法。在下面的描述里，`expr`代表由表里的一个或多个列构成的表达式，`col_list`是一个由1~16个以逗号分隔的列名构成的列表。列名只能来自将被创建的那个表。

`RANGE (expr)` 分区函数将把每个分区和表达式 `expr`的可取值范围的一个子集关联在一起。这种分区函数必须和一个包含着 `VALUES LESS THAN` 子句的分区定义搭配使用，并按照该子句所给出的整数上限把函数值映射到不同的分区。（`NULL`不允许作为限制，`NULL`值将被映射到第一个分区。）对于首尾相连的分区，它们的`VALUES`子句所给出的上限值必须是按递增顺序列出的。最后一个分区可以使用 `MAXVALUES` 关键字作为其分区函数值，其含义是所有未落入此前各分区的函数值都将属于这最后一个分区。

```
CREATE TABLE t (income BIGINT, ...)
```

```
PARTITION BY RANGE (income)
```

```
(
```

```
PARTITION p0 VALUES LESS THAN (10000),
```

```
PARTITION p1 VALUES LESS THAN (30000),
```

```
PARTITION p2 VALUES LESS THAN (75000),
```

```
PARTITION p3 VALUES LESS THAN MAXVALUE
```

);

RANGE COLUMNS (col_list) 分区函数与 RANGE(expr) 相似，但是在分区时，它使用的不是一个单个的表达式，而是一个列名列表。每个分区的 VALUES LESS THAN 子句会为每个列列出一个非 NULL 字面值，以定义分区允许的上限值。例如，如果分区定义的开始部分是 PARTITION BY RANGE COLUMNS (col1, col2, col3)，那么每个 VALUES 子句的形式就为 VALUES LESS THAN (val1, val2, val3)，用于指定每个列的最大值。在 col_list 里的每个列的类型都必须是整数、字符串或时态，不能是 BIT、BLOB、TEXT、ENUM 或 SET。VALUES LESS THAN 子句里的每一个字面值都必须具有与 col_list 里的相应列相同的数据类型。

LIST(expr) 分区函数将把每个分区和一系列值关联在一起。这种分区函数必须和一个包含着 VALUES IN 子句的分区定义搭配使用，并按照该子句所给出的整数列表把函数值映射到不同的分区。允许使用 NULL，但是不能使用 MAXVALUE。如果 expr 表达式的计算结果为 NULL 值，则 VALUES 列表里也必须有一项为 NULL。

```
CREATE TABLE t (id INT NULL, ...)

PARTITION BY LIST(id)

(

PARTITION p0 VALUES IN (1, 2, 3),

PARTITION p1 VALUES IN (4, 5, 6, NULL)

);
```

LIST COLUMNS (col_list) 分区函数与 RANGE COLUMNS(col_list) 类似，主要差别在于里面的那个列表不同。对于允许使用的列类型方面，有类似的条件要求。需要使用 VALUES IN 来代替 VALUES LESS THAN，但是对于指定的各个值的类型，也有类似的条件要求。（与 LIST 的差异在于，LIST 要求通过整数值来分区，而 LIST COLUMNS 的 VALUES IN 子句可以指定非整数值。）每个 VALUES IN 子句可以提供一

个或多个带括号的列表值。列表里的每个值都必须拥有相同数量的列，但每个子句里的列表数量可以各不相同。例如，如果有一个表包含两个INT列，名字分别为i1和i2，并且想要将它们用于分区，那么可以像下面这样使用LIST COLUMNS：

```
PARTITION BY LIST COLUMNS (i1, i2)

(

PARTITION p1 VALUES IN ((NULL, NULL)),

PARTITION p2 VALUES IN ((0, 0), (0, 1), (0, 2)),

PARTITION p3 VALUES IN ((1, 0), (1, 1))

)
```

HASH (expr) 分区函数将根据以行的内容计算出来的expr值把行存储到相应的分区。典型的做法是把HASH() 函数和用来指定创建多少分区的PARTITION n子句搭配使用。行是基于expr除以n的余数来进行分配的。

```
CREATE TABLE t (d DATE, ...)

PARTITION BY HASH(TO_DAYS(d))

PARTITIONS 5;
```

可以在HASH() 形式的分区表达式的前面加上LINEAR，这会使得该函数的算法有所变化。使用LINEAR的一个好处是可以让某些分区管理操作（如通过ALTER TABLE语句来增加或删除分区）变得更有效率。但是，这有可能会让行在各个分区上的分布程度比没有使用LINEAR时的情况更差。

KEY (col_list) 分区函数的效果类似于 HASH() 分区函数，但是你可以指定数据列（通过它们来计算散列值），并且 MySQL 服务器会负责提供散列函数。KEY() 的前面可以加上LINEAR。

如果包含的分区定义使用了 `HASH()` 或 `KEY()`，那么这些分区定义不应该有 `VALUES` 子句。`VALUES`子句只能与`RANGE()`和`LIST()`搭配使用。

`expr`表达式必须是确定性的，这样可以确保同样的输入总是会得到同样的结果。例如，在`expr`表达式里可以使用`ABS()`函数，但不允许使用`RAND()`函数。如果使用了一个不允许使用的函数，`CREATE TABLE`语句将返回一条出错信息。

对于`RANGE()`或`LIST()`，表达式`expr`的计算结果必须是一个整数或`NULL`值。对于`HASH()`，表达式`expr`的计算结果必须是一个非`NULL`非负的整数，因此如果表达式引用了某个非整数列，那么它必须将列值转换为整数。例如，如果`d` 是一个`DATE` 列，那么你可以使用`TO_DAYS(d)`函数把日期转换为天数，以保证`HASH(TO_DAYS(d))`是一个有效的散列函数。

对于`KEY()`，其参数都是列名，但这些列不必是整数类型。

每个`partition_option`值对应着一个附加的分区特性，如下面所示。（虽然在以下描述里使用的术语是“分区”，但这些操作也可以用在子分区的定义里。）

```
COMMENT [=] 'str'
```

为分区加上一个描述性的注释。

```
DATA DIRECTORY [=] 'dir_name', INDEX DIRECTORY [=]  
'dir_name'
```

这两个选项类似于前面描述过的同名的表选项。它们分别用来设定该分区里的数据或索引将被存储到什么地方。默认的存储位置是该表所在数据库的数据库目录。

```
MAX_ROWS [=] n、 MIN_ROWS [=] n
```

这些选项用来表明你计划存储在该分区中的行的最大值和最小值。`n`必须是一个正整数。

```
[STORAGE] ENGINE [=] engine_name
```

用来处理该分区的存储引擎。表分区不支持混合使用多种存储引擎，所以必须为同一个表的所有分区指定同一种存储引擎。

以下语句演示了CREATE TABLE语句的一些常见用法。

创建一个包含3个列的表。id列是一个PRIMARY KEY，在last_name和first_name列上还有一个多列索引：

```
CREATE TABLE customer

(

    id                SMALLINT UNSIGNED NOT NULL
    AUTO_INCREMENT,

    last_name        CHAR(30) NOT NULL,

    first_name       CHAR(20) NOT NULL,

    PRIMARY KEY (id),

    INDEX (last_name, first_name)

);
```

创建一个临时表；为获得更好的访问速度，使其成为一个MEMORY表：

```
CREATE TEMPORARY TABLE tmp_table

(id MEDIUMINT NOT NULL UNIQUE, name CHAR(40))

ENGINE = MEMORY;
```

创建一个新表作为另一个表的空白副本：

```
CREATE TABLE prez_copy LIKE president;
```

用另一个表的内容创建一个新表：

```
CREATE TABLE prez_copy SELECT * FROM president;
```

用另一个表的部分内容创建一个新表：

```
CREATE TABLE prez_alive SELECT last_name, first_name, birth  
FROM president WHERE death IS NULL
```

对于通过使用结尾的SELECT 语句创建和填充的表，如果为其指定了列定义，那么列定义将在表内容插到该表里以后生效。例如，可以像下面这样把一个选定的列定义为新表里的PRIMARY KEY：

```
CREATE TABLE new_tbl (PRIMARY KEY (a)) SELECT a, b, c  
FROM old_tbl;
```

还可以对新表里的列进行定义以覆盖那些基于结果集特性的默认定义：

```
CREATE TABLE new_tbl  
  
(a INT UNSIGNED NOT NULL AUTO_INCREMENT, b DATE,  
PRIMARY KEY (a))
```

```
SELECT a, b, c FROM old_tbl;
```

```
CREATE TRIGGER
```

```
CREATE
```

```
[DEFINER = definer_name]
```

```
TRIGGER trigger_name trigger_time trigger_event
```

```
ON tbl_name
```

FOR EACH ROW trigger_stmt

把触发器和表关联在一起，如此一来，当该表上发生特定的事件时，该触发器将被激活并执行它所包含的语句。默认情况下，tbl_name 表来自当前的默认数据库。如果针对的是某个特定的数据库里的一个表，必须按照 db_name.tbl_name的格式来给出它的名字。使用CREATE TRIGGER语句时，需要对与触发器相关联的那个表拥有TRIGGER权限。

当触发器被激活时，DEFINER子句将被用来确定其安全上下文（即用于核查其访问权限的账户），请参考4.3节。默认使用执行CREATE TRIGGER语句的那个用户的账户。相关账户对该表必须拥有TRIGGER权限；如果想在触发器的定义里使用NEW或OLD引用tbl_name表里的列，则必须拥有tbl_name表的SELECT权限；如果想在触发器的定义里使用SET NEW.col_name修改表里的列，则必须拥有tbl_name表的UPDATE权限。此外，该账户还必须拥有为了正常执行触发器定义里的语句而必不可少的各项权限。

trigger_time 值既可以是 BEFORE，也可以是 AFTER；分别表示的是应该在激活触发器的那条语句对每个行进行处理之前或之后执行触发器所包含的语句。

trigger_event值应该是INSERT、UPDATE或DELETE，用以表明哪种语句将导致触发器被激活。

trigger_stmt 部分是由 SQL语句构成的触发器主体。它应该只包含一条 SQL语句。如果需要使用多条语句，必须用关键字BEGIN和END把它们括起来以构成一个复合语句（参考E.2节）。

在DELETE或UPDATE触发器里，可以用OLD.col_name语法来引用将被删除或更新的旧行里的列。类似地，在INSERT或UPDATE触发器里，可以用NEW.col_name语法来引用将被插入或更新的新行里的列。OLD和NEW关键字都不区分大小写。

在BEFORE触发器里，可以使用SET语句修改新行里的值：

SET NEW.col_name = value

在触发器创建时生效的sql_mode系统变量的值会被保存起来并在触发器被激活时恢复生效。触发器不带任何参数，并且像存储函数那样，无法执行会产生一个结果集的语句。

CREATE USER

CREATE USER account [auth_info]

[, account [auth_info]] ...

auth_info:

IDENTIFIED BY [PASSWORD] 'password'

| IDENTIFIED WITH auth_plugin [AS 'auth_string']

创建一个或多个MySQL账户。这条语句要求拥有全局级 CREATE USER 权限，或者对 mysql数据库拥有INSERT权限。

对于每个账户，在mysql.user表里创建的行都没有任何权限。如果某个账户已经存在，则会看到一条出错信息。账户名必须以'user_name'@'host_name'的格式给出，请参考 13.2.1.1 节中的描述。

如果指定了auth_info子句，则它表示的是，新账户的身份认证是使用密码，还是使用身份认证插件。更多详细信息请参考13.2.1.3节。当客户端使用此账户连接服务器时，如果没有指定带有非空密码的IDENTIFIED BY子句，或者没有指定IDENTIFIED WITH子句，则连接不会进行身份认证。这种情况并不安全，应该尽量避免。

CREATE USER 'myname'@'localhost' IDENTIFIED BY 'mypass';

CREATE VIEW

CREATE [OR REPLACE]

[ALGORITHM = {MERGE | TEMPTABLE | UNDEFINED}]

[DEFINER = definer_name]

[SQL SECURITY = {DEFINER | INVOKER}]VIEW view_name
[(col_list)] AS select_stmt

[WITH [CASCADED | LOCAL] CHECK OPTION]

创建一个视图。默认情况下，如果存在同名的视图，则会看到一条出错信息。如果指定了 OR REPLACE子句，则新视图将替换旧视图。

如果指定了 col_list，新视图将由该列表里的列构成。新视图里的每个列都必须名列其中。如果省略了col_list，新视图将由其视图定义里的SELECT语句所选取的列构成。

select_stmt是一条用来定义视图的SELECT语句。在该语句里可以引用表或其他视图。

如果想创建视图，则必须对它拥有CREATE VIEW权限，对select_stmt语句所选取的各列拥有权限，并且对select_stmt语句里引用到其他地方的每一个列都拥有SELECT权限。如果使用了OR REPLACE子句，则还必须拥有对该视图的DROP权限。

在调用视图时，DEFINER和SQL SECURITY子句将被用来确定其安全上下文（即用于核查其访问权限的账户），请参考4.3节。默认使用执行CREATE VIEW语句的那个用户的账户。

ALGORITHM子句决定着如何处理视图。如果被设置为MERGE，当你发出一条引用了该视图的语句时，MySQL 将把该视图的定义合并到那条语句里。执行的是最终那条语句。如果被设置为TEMPTABLE，MySQL 在用到该视图的时候将把它创建为一个临时表。如果被设置为UNDEFINED，MySQL服务器将根据具体情况自行选择如何处理。默认设置是UNDEFINED。

WITH CHECK OPTION 子句适用于可更新视图（它指的是那些可以与UPDATE 或其他表修改语句一起用于更新底层表的视图）。它允许利用视图插入或修改底层表里的记录行，只是这些记录都满足SELECT里的WHERE子句所指定的条件为真）。关键字CASCADED和LOCAL可用于在视图定义里引用了其他视图。当使用CASCADED时，检查操作会延伸到

底层视图。当使用LOCAL时，检查操作则仅限于当前视图。如果这两个关键字都没有给出，则MySQL将默认使用CASCADED。

DEALLOCATE PREPARE

{DEALLOCATE | DROP} PREPARE stmt_name

释放此前使用PREPARE语句进行预处理而得到的那条名为stmt_name的预处理语句。被释放的预处理语句将不能再执行。

DELETE

DELETE [LOW_PRIORITY] [QUICK] [IGNORE]

FROM tbl_name

[PARTITION (partition_name [, partition_name] ...)]

[WHERE where_expr] [ORDER BY ...] [LIMIT n]

DELETE [LOW_PRIORITY] [QUICK] [IGNORE]

tbl_list

FROM tbl_refs

[WHERE where_expr]

DELETE [LOW_PRIORITY] [QUICK] [IGNORE]

FROM tbl_list

USING tbl_refs

[WHERE where_expr]

tbl_list:

tbl_name[.*]

[PARTITION (partition_name [, partition_name] ...)]

[, tbl_name[.*]

[PARTITION (partition_name [, partition_name] ...)]

] ...

DELETE语句可以删除表里的行，并返回被删除的行数。自MySQL 5.6.2起，DELETE语句支持使用针对分区表的PARTITION子句，以表明需要从哪个分区或子分区删除行。此时，如果将被删除的某个行不在指定的分区里，则它会保持不变。

第一种形式的DELETE语句会删除tbl_name表里的行。第二种和第三种形式可以从多个表里删除行，或者根据涉及多个表的条件来删除行。tbl_refs的语法与SELECT的类似，区别在于不能把子查询指定为表。

与WHERE子句里指定的条件相匹配的行会被删除。

```
DELETE FROM score WHERE event_id = 14;
```

```
DELETE FROM member WHERE expiration < CURDATE();
```

如果省略了 WHERE 子句，则表中的全体行都将被删除。（如果不需要行统计，那么还可以使用另一种清空表的方式，即使用TRUNCATE TABLE。）

LOW_PRIORITY选项将使DELETE语句的执行被延迟到没有客户端读取该表为止。此选项只对使用表级别锁定的存储引擎（如MyISAM或MEMORY）有效。

对于MyISAM表，给出QUICK 选项能加快语句的执行速度。MyISAM存储引擎将不执行通常要执行的索引树的叶合并工作。

如果使用了 IGNORE 修饰符，那么在行被删除时发生的错误将被忽略。这些错误会生成警告信息。

如果给出LIMIT子句，则其中的n值指定的是将要删除的最大行数。

如果给出ORDER BY子句，则各个行会按最终排序顺序被删除。如果将它与LIMIT子句结合使用，能更精确地控制将要删除哪些行。ORDER BY子句的语法与SELECT语句的相同。

DELETE 语句的第二种和第三种形式特别适用于从多个表一次删除有关行。它们使你能够根据表之间的连接关系确定需要删除哪些行。对于那些需要删除行的表，其名字的指定形式可以是tbl_name，也可以是tbl_name.*（这种形式是为了与ODBC保持兼容）。

tbl_refs 子句用来给出为了判断应该删除哪些行而需要关联的表。在这个子句里，可以在引用表时为它们声明一个别名。DELETE语句的其他部分可以引用表别名，但是不能声明它们。

如果想从t1表里删除id值与t2表里的id值相匹配的那些行，则可以使用第一种多表语法形式，如下所示：

```
DELETE t1 FROM t1 INNER JOIN t2 WHERE t1.id = t2.id;
```

也可以使用第二种语法形式，如下所示：

```
DELETE FROM t1 USING t1 INNER JOIN t2 WHERE t1.id = t2.id;
```

多表DELETE语句不允许有ORDER BY或LIMIT子句。另外，WHERE子句也不允许包含会从被删除行的那个表查询行的子查询。

DESCRIBE

```
{DESCRIBE | DESC} tbl_name [col_name | 'pattern']
```

```
{DESCRIBE | DESC} stmt
```

带表名或视图名的DESCRIBE语句产生的输出报告与SHOW COLUMNS语句的相同。更多信息请参考SHOW条目。在这种语法里，如果给定列，DESCRIBE语句的输出报告中将只包含关于该列的信息。结尾的字符串会被解释为匹配模式（如LIKE操作符），并且会将输出限制在名字与该模式相匹配的那几个列里。

下面这条语句将显示president表里的last_name列的描述信息：

```
DESCRIBE president last_name;
```

下面这条语句将显示president表里的last_name和first_name列的描述信息：

```
DESCRIBE president '%name';
```

DESCRIBE 后面带上语句之后等同于 EXPLAIN 语句。在MySQL 5.6.3 之前，该语句必须为SELECT。自MySQL 5.6.3起，该语句可以为SELECT、DELETE、INSERT、REPLACE或UPDATE。更多信息请参考EXPLAIN条目。

DO

```
DO expr [, expr] ...
```

对表达式进行计算，但不返回计算结果。这使得DO 语句用起来比SELECT 语句方便，它不需要处理结果集。例如，DO语句可用来设置变量，或者调用那些你只关心其副作用而不关心返回值的函数。

```
DO @sidea := 3, @sideb := 4, @sidec :=  
SQRT(@sidea*@sidea+@sideb*+@sideb);
```

```
DO RELEASE_LOCK('mylock');
```

DROP DATABASE

```
DROP DATABASE [IF EXISTS] db_name
```

删除指定的数据库及其内容。执行此语句时，对指定数据库必须拥有DROP权限。

IF EXISTS子句的作用是禁止显示数据库不存在时会产生的出错信息。在这种情况下，会生成一个警告。

数据库是由数据目录下的目录表示的。服务器只会删除它可以识别为其自身创建的文件和目录（如.frm 文件）。此语句不会删除其他文件和目录。如果在数据目录里放入了非表文件，则这些文件不会被DROP DATABASE语句删除。这种情况会导致数据库目录无法被删除，而且DROP DATABASE执行会失败。在这种情况下，SHOW DATABASES将继续列出数据库。为改正这个问题，需要手动删除所有的其他文件和子目录，然后再次执行DROP DATABASE语句。

DROP DATABASE语句成功执行后，会返回一个计数值——其表示的是实际删除的表或视图的个数。（这个计数值其实是它实际删除的.frm文件的个数，但二者的含义是一样的。）

DROP EVENT

DROP EVENT [IF EXISTS] event_name

删除指定的事件。IF EXISTS子句的作用是禁止显示事件不存在时会产生的出错信息。如果带有IF EXISTS子句，MySQL将生成一条警告信息。执行此操作时，必须拥有事件所属数据库的EVENT权限。

DROP FUNCTION、DROP PROCEDURE

DROP {FUNCTION | PROCEDURE} [IF EXISTS] routine_name

删除指定的存储函数或存储过程。这些语句要求拥有指定例程的ALTER ROUTINE权限。

IF EXISTS子句的作用是禁止显示例程不存在时会产生的出错信息。在这种情况下，会生成一个警告。

DROP INDEX

```
DROP INDEX index_name ON tbl_name
```

从tbl_name表里删除名为index_name的索引。MySQL将把这条语句当作一条ALTER TABLE DROP INDEX语句来处理。更多详细信息请参考ALTER TABLE条目。若要使用DROP INDEX语句删除一个PRIMARY KEY，必须用一个标识符把后者引起来。

```
DROP INDEX `PRIMARY` ON tbl_name;
```

```
DROP TABLE
```

```
DROP [TEMPORARY] {TABLE | TABLES} [IF EXISTS]
```

```
tbl_name [, tbl_name] ...
```

```
[RESTRICT | CASCADE]
```

将指定表从所在数据库中删除。如果指定TEMPORARY关键字，只删除TEMPORARY表。

IF EXISTS 子句的作用是禁止显示表不存在时会产生的出错信息。在这种情况下，会生成一个警告。

关键字RESTRICT和CASCADE可以正常解析，但是都会被忽略，因而它们不会有任何效果。

```
DROP TRIGGER
```

```
DROP TRIGGER [IF EXISTS] trigger_name
```

删除触发器。默认情况下，是删除默认数据库里的触发器。如果要删除指定数据库里的触发器，则可以按db_name.trigger_name格式指定触发器名字。使用DROP TRIGGER语句时，需要对与触发器相关联的那个表拥有TRIGGER权限。

IF EXISTS 子句的作用是禁止显示触发器不存在时会产生的出错信息。在这种情况下，会生成一个警告。

如果表有触发器，那么删除该表也会删除它的触发器。

DROP USER

DROP USER account [, account] ...

DROP USER 语句会在权限表里删除所有与指定的各个账户相关联的行。这条语句要求拥有全局级CREATE USER权限，或者对mysql数据库拥有DELETE权限。

DROP USER 语句会删除指定的账户和与之关联的所有权限，但是不会删除由它创建的数据库和其他对象。每个账户的指定格式为'user_name'@'host_name'，请参考13.2.1.1节。如果账户不存在，则会报告出错。

```
DROP USER 'myname'@'localhost';
```

DROP VIEW

DROP VIEW [IF EXISTS]

view_name [, view_name] ...

[RESTRICT | CASCADE]

将给定视图从其所在数据库中删除。必须拥有该视图的DROP权限。

IF EXISTS 子句的作用是禁止显示视图不存在时会产生的出错信息。在这种情况下，会生成一个警告。

关键字RESTRICT和CASCADE可以正常解析，但是都会被忽略，因而它们不会有任何效果。

EXECUTE

EXECUTE stmt_name [USING @var_name [, @var_name] ...]

执行之前用PREPARE 预处理过的名为 stmt_name的预处理语句。如果预处理语句包含任何占位符标记，那么必须给出USING子句。这个子句应该提供一个用逗号分隔的用户变量列表，为子句中每个后续的占位符提供值。

EXPLAIN

EXPLAIN tbl_name [col_name | 'pattern']

EXPLAIN

[EXTENDED | PARTITIONS | FORMAT = {TRADITIONAL | JSON}]

{SELECT ...| DELETE ...| INSERT ...| REPLACE ...| UPDATE ...}

这条语句的第一种形式相当于一**条DESCRIBE tbl_name**语句。更多信息请参考DESCRIBE条目。EXPLAIN语句的第二种形式能够让我们了解到MySQL将如何执行出现在EXPLAIN关键字之后的那条SELECT语句：

```
EXPLAIN SELECT score.* FROM score INNER JOIN grade_event
ON score.event_id = grade_event.event_id AND
grade_event.event_id = 14;
```

在MySQL 5.6.3之前，该语句必须为 SELECT。自 MySQL 5.6.3起，该语句可以为 SELECT、DELETE、INSERT、REPLACE或UPDATE。

在EXPLAIN关键字之后，可以指定一个可选的指示器，指明想要产生的输出类型：

EXTENDED选项将使EXPLAIN语句生成更多的执行计划信息。在EXPLAIN语句执行完毕之后立刻执行SHOW WARNINGS语句可以查看到这些信息。

PARTITIONS选项将会多生成一个输出列，其中包含的是与表分区有关的信息。

FORMAT表明的是用“传统”格式（默认）产生输出，还是用JSON格式产生输出。JSON输出包含了扩展的和可应用的分区信息。此选项是在MySQL 5.6.5里引入的。

在默认输出格式里，EXPLAIN输出一个或多个行，其中包含以下列。

id

这个输出行所对应的SELECT 语句的ID 编号。如果语句包含子查询，或者语句中使用了UNION，那么语句中可以有多于一个SELECT子句。

select_type

这个输出行所对应的SELECT语句类型如下表所示。

类型	含义
SIMPLE	不带UNION或子查询部分的SELECT语句
PRIMARY	最外层或最左侧的SELECT语句
UNION	UNION里的第二条或随后的SELECT子句
DEPENDENT UNION	和UNION相似，但需要依赖于某个外层查询
UNION RESULT	UNION的结果
SUBQUERY	子查询中的第一个SELECT子句
DEPENDENT SUBQUERY	和SUBQUERY相似，但需要依赖于某个外层查询
DERIVED	FROM子句里的子查询
UNCACHEABLE SUBQUERY	无法缓存的子查询结果
UNCACHEABLE UNION	无法缓存的子查询UNION的第二条或随后的SELECT子句

table

输出行引用的那个表。

partitions

将要使用的分区。只有出现PARTITIONS选项时才显示这列。对于非分区表，这个值为NULL。

type

MySQL将执行的连接类型。这些类型（从优到劣）包括system、const、eq_ref、ref、ref_or_null、index_merge、unique_subquery、index_subquery、range、index和ALL。排在前面的类型有着更强的限制性，这意味着MySQL在检索过程中检查的行会相对少一些。

possible_keys

MySQL认为在名称出现在table列里的那个表里查找行时可能会用到的索引。如果这个输出列里的值是NULL，则表明没有找到索引。

key

MySQL在指定表里查找行时实际用到的索引。（如果 MySQL使用了 index_merge 连接类型，这里可能会列出几个键，因为优化器会使用几个索引来处理查询。）如果这个输出列里的值是NULL，则表明没有在该表里找到这样的索引。

key_len

实际使用的索引的长度。如果 MySQL会使用索引的最左前缀，则此数字可能会小于全文索引行的长度。

ref

MySQL用来与索引值进行比较的值。单词const 或'???'表示的是对常数进行比较。如果是某个列的名称，则表示逐个比较列。

rows

MySQL 为完成查询而需要在表里检查的行数的估算值。这个输出列里的值的乘积就是所有表中必须检查的行的各种可能组合的估算值。

filtered

它指的是将与前面的表进行连接的行的估算百分比。如果存在EXTENDED 选项，则会显示此列。

Extra

其他与执行计划相关的信息。这个值或者为空白，或者包含一个或多个下面这样的值。

Using filesort：需要将索引值写到文件中并且排序，以便按排序顺序检索相关行。

Using index：MySQL可以不必检查数据行，只使用索引信息就能检索表信息。

Using temporary：必须创建的临时表。

Using where：利用SELECT语句中WHERE子句里的信息查询行。

此外，还有很多其他的值并未列在这里。与Extra值有关的更多信息请参考MySQL参考手册。

FLUSH

FLUSH [NO_WRITE_TO_BINLOG | LOCAL] option [, option] ...

刷新MySQL服务器内部使用的缓存。执行此语句时，必须拥有RELOAD权限；对于某些操作，可能还会需要其他的权限，后面会有特别说明。

如果启用了二进制日志记录，那么在不给出NO_WRITE_TO_BINLOG或LOCAL选项时，MySQL会把大部分的FLUSH语句写到二进制日志里。在任何情况下都不会被记录的异常有FLUSH LOGS、FLUSH MASTER、FLUSH SLAVE，以及两种形式的FLUSH TABLES WITH READ LOCK。这些内容都不会被记录，以避免在复制从服务器上出现问题。

每个option值都只能是下面列表中的某一项。对于TABLES选项，TABLE与之等价。在同一个FLUSH语句里，TABLES选项都不能与其他选项一起使用。

DES_KEY_FILE

重新加载DES_ENCRYPT()和DES_DECRYPT()函数使用的DES密钥文件。

HOSTS

刷新主机缓存。如果来自某个客户端主机的连接所导致的错误会超过max_connect_errors个，并且服务器开始阻塞来自客户端主机的连接，那么这个选项是很有必要的。

[log_type] LOGS

通过关闭并重新打开日志文件的方式刷新日志文件。如果启用了二进制日志或中继日志，那么此选项会导致在编号日志文件序列里的下一个文件被打开。自MySQL 5.5.3起，可以在LOGS关键字的前面加上一个说明符，以表明需要刷新哪个特定的日志。BINARY、ERROR、GENERAL、RELAY或SLOW将分别刷新二进制日志、出错日志、普通查询日志、从服务器中继日志或慢查询日志。ENGINE会告诉存储引擎刷新它们维护的、可以刷新的日志。InnoDB会刷新其日志，并且记录检查点。

PRIVILEGES

重新加载权限表。如果通过GRANT或REVOKE命令修改了这些表，MySQL服务器将自动同步更新它们在内存中的复制；但如果是通过INSERT或UPDATE等

语句来直接修改权限表，就必须利用此选项明确地让 MySQL 重新加载它们。此选项还会影响账户资源管理的限制；从这一点上来讲，它类似于 USER_RESOURCES 选项。

QUERY CACHE

刷新查询缓存以对其进行碎片整理，但不清除这个缓存里的语句。（如果想清除这个缓存，需要使用 RESET QUERY CACHE。）

STATUS

重新初始化服务器状态变量。

TABLES [tbl_name [, tbl_name] ...]

如果没有给出任何表名，则关闭表缓存里所有已经打开的表。如果以逗号分隔列出了一个或多个表名，则只刷新指定的表（而不是整个表缓存）。自 MySQL 5.5.3 起，当 LOCK TABLES 因为 READ 锁而正发挥作用时，不能使用此操作。对于读锁，可以使用相应的 WITH READ LOCK 语法，并且刷新表。

如果查询缓存是可操作的，则 FLUSH TABLES 语句也会刷新查询缓存。

TABLES WITH READ LOCK

刷新所有数据库里的所有表，然后为它们加上一个全局读锁——该锁将一直保持到调用 UNLOCK TABLES 语句为止。这条语句仍允许客户端读取表里的内容，但将阻塞对这些表的修改操作。如果想把服务器的内容完整地备份下来，使用本选项将确保表不会在备份操作期间发生任何改变。从客户端的角度看，这种做法的坏处在于，禁止更新的周期会更长一些。

TABLES tbl_name [, tbl_name] ... WITH READ LOCK

刷新指定的表，并获得它们的读锁——这些锁定会被一直占有，直到执行 UNLOCK TABLES、LOCK TABLES 或 START TRANSACTION 语句为止。对于其中的每个表，除拥有 RELOAD 权限外，还必须拥有 LOCK TABLES 权限。此操作仅适用于基本表。它会忽略 TEMPORARY 表，并产生一个错误，以便于查看具体情况。此语法是在 MySQL 5.5.3 里引入的。

USER_RESOURCES

重置当前账户资源管理限制（如 MAX_QUERYS_PER_HOUR）的每小时配额。这样，那些资源使用量已经达到配额上限的账户就又可以使用了。此选项不会影响 MAX_USER_CONNECTIONS 的限制，它不是对每小时配额的限制。

GRANT

GRANT priv_type [(col_list)] [, priv_type [(col_list)]] ...

ON [TABLE | FUNCTION | PROCEDURE]

{*. * | * | db_name. * | db_name. tbl_name

| tbl_name | db_name.routine_name}

TO account [auth_info] [, account [auth_info]] ...

[REQUIRE security_options]

[WITH grant_or_resource_options]

GRANT PROXY ON account

TO account [auth_info] [, account [auth_info]] ...

[WITH GRANT OPTION]

auth_info:

IDENTIFIED BY [PASSWORD] 'password'

| IDENTIFIED WITH auth_plugin [AS 'auth_string']

GRANT 语句将把访问权限授予一个或者多个MySQL账户。如果想要使用这条语句，则必须拥有GRANT OPTION权限和你将要授予的权限。

授予 PROXY 权限的语法比授予其他权限的语法更严格：该权限必须单独指定，不允许使用REQUIRE子句，并且WITH子句只能与GRANT OPTION一起使用。

每一个priv_type值都会指定一个要被授予的权限，从下表里选择。权限ALL和PROXY需要单独使用。对于其他的权限，可以使用以逗号分隔的列表形式来指定它们。ALL 表示的是在指定级别上所有其他可用权限的组合，但是GRANT OPTION 除外，它必须单独授予或通过增加一个WITH GRANT OPTION子句来授予。例如，在表级别，ALL只会授予那些适用于表的权限。

权限名称	权限允许的操作
ALTER	更改表和索引
ALTER ROUTINE	更改或删除存储函数和过程
CREATE	创建数据库和表
CREATE ROUTINE	创建存储函数和过程
CREATE TABLESPACE	创建、删除或更改表空间
CREATE TEMPORARY TABLES	使用TEMPORARY关键字创建临时表
CREATE USER	使用高级账户管理语句
CREATE VIEW	创建视图
DELETE	从表中删除行
DROP	删除数据库、表和其他对象
EVENT	创建、删除和更改事件调度器事件
EXECUTE	执行存储函数和过程
FILE	读、写服务器主机上的文件
GRANT OPTION	把本账户的权限授予其他账户
INDEX	创建或删除索引
INSERT	往表中插入新行
LOCK TABLES	用LOCK TABLES语句显式锁定表
PROCESS	查看服务器上运行的线程的信息
PROXY	用户代理
REFERENCES	未使用（供以后使用）
RELOAD	重新加载权限表，或刷新日志或缓存
REPLICATION CLIENT	询问主、从服务器的位置
REPLICATION SLAVE	用作复制从服务器
SELECT	检索表中的行
SHOW DATABASES	用SHOW DATABASES语句查看所有数据库名称
SHOW VIEW	用SHOW CREATE VIEW语句查看视图定义
SHUTDOWN	关闭服务器
SUPER	终止线程，执行其他超级用户操作
TRIGGER	创建或删除触发器
UPDATE	修改表行
ALL [PRIVILEGES]	所有操作（GRANT和REVOKE除外）
USAGE	特殊的“无权限”权限

其中，PROXY权限是在MySQL 5.5.7里引入的。

LOCK TABLES权限只能作用于你对其拥有SELECT权限的那些表，但它允许你放置任何种类的锁，而不仅限于读锁。

自己总是可以查看或者终止自己的线程。PROCESS或SUPER权限分别允许你查看或终止任何账户的线程，而不仅限于你自己的线程。ALTER ROUTINE和CREATE ROUTINE只适用于存储例程，而不能用于用户定义函数（UDF）。

ON子句负责设定权限的作用范围，如下表所示。

权限说明符	权限作用级别
ON *.*	全局权限：所有数据库以及其中的所有对象
ON *	针对默认数据库的数据库级权限；若无默认数据库，则出错
ON db_name.*	数据库权限：指定数据库里的所有对象
ON db_name.tbl_name	表权限：指定表里的所有列
ON tbl_name	表权限：默认数据库里指定表的所有列
ON db_name.routine_name	指定数据库里指定例程的权限
ON account	代理权限：account为被代理用户

在显式指定权限所要应用的对象类型时，如果存在歧义，则可以包含关键字TABLE、FUNCTION或PROCEDURE（如ON TABLE mydb.mytbl或ON FUNCTION mydb.myfunc）。

如果要授予表或列权限，则要求表必须已存在。

当使用ALL 作为权限名时，只能将当前授权级别上的可用权限授予对方。例如，RELOAD 权限只能当作全局级权限，所以只有在ON *.*级别上使用GRANT ALL语句时才能把它授予对方，在ON db_name.*级别上则不行。在后一种情况里，实际授予的只有数据库级别的全部权限。ALL 还可以用来授予全局级、数据库级、表级或例程级的全部权限。

USAGE表示的是“无权限”。它应该只在全局级别使用。

GRANT OPTION作用于给定级别的所有权限。例如，如果你把某给定数据库的SELECT和INSERT权限授予了某个账户，你不能让该账户把其中某个权限授予其他人。

当某个表的名字出现在ON子句里时，还可以在权限后面加上一条 (col_list) 子句，把该权限进一步授予到列，(col_list) 子句由一个或多个以逗号分隔的列名构成。（这只适用于INSERT、REFERENCE、SELECT和UPDATE权限，只有这几种权限允许授予列。）

TO子句用来列出将获得权限的一个或多个账户。账户名必须以'user_name'@'host_name'的格式给出，请参考13.2.1.1节。

每个账户名的后面可以紧跟一个可选的auth_info子句，用于指定该账户是使用密码进行身份认证，还是使用认证插件来进行认证。其语法与CREATE USER的语法相同，请参考该语句的描述。更多详细信息请参考13.2.1.3节。为已有账户指定的密码会替换它的当前密码。

如果指定账户不存在，则GRANT会创建它。在这种情况下，如果不指定身份认证信息（带有非空密码的IDENTIFIED BY子句或者IDENTIFIED WITH子句），那么客户端会使用该账户来连接服务器。除非GRANT 语句包含了身份认证信息，否则在要想阻止GRANT 创建账户时，必须启用SQL模式NO_AUTO_CREATE_USER。

如果打算给数据库名、表名、列名和例程名加上引号，必须使用标识符引号字符。用户名和主机名则既可以使用标识符引号字符，也可以使用字符串引号字符。例如：

```
GRANT INSERT (`mycol`) ON `test`.`t` TO 'myuser'@'localhost';
```

如果给出REQUIRE子句，那么它将要求被授权账户必须使用安全连接，并指定客户端应提供何种类型的信息。在REQUIRE关键字的后面可以给出以下选项。

NONE。不要求安全连接。

SSL。常用的连接类型，指定账户必须通过SSL来连接MySQL服务器。

X509。用户必须提供一个有效的X509证书。在这种情况下，客户端可以提供任何 X509证书；与有效性相比，其内容反而没那么重要。

利用下面的一个或多个选项，可以要求连接具有某些特性。

CIPHER 'str'。在建立连接时，必须把'str'当作加密密钥。

ISSUER 'str'。客户端证书必须用'str'作为证书签发者的值。

SUBJECT 'str'。客户端证书必须用'str'作为证书主题的值。

如果需要给出一个以上的上述选项，可以用AND 把它们分隔开（这是可选的）。先后顺序无关紧要。

WITH子句（如果给出的话）用于规定指定账户是否可以把自己的权限转授给其他用户，并对账户的资源消耗作出限制。其各种选项如下所示。可以同时使用多个选项，先后顺序无关紧要。

GRANT OPTION。账户可以把自己的权限转授给其他账户，其中甚至包括授予权限。

MAX_CONNECTIONS_PER_HOUR n。账户每小时可以连接n次服务器。

MAX_QUERIES_PER_HOUR n。账户每小时可以执行n条语句。

MAX_UPDATES_PER_HOUR n。账户每小时可以执行n条修改数据的语句。

MAX_USER_CONNECTIONS n。账户最多可以和服务器同时建立n个连接。

对于选项 MAX_CONNECTIONS_PER_HOUR、MAX_QUERIES_PER_HOUR和MAX_UPDATES_PER_HOUR，值为0表示“没有限制”。对于MAX_USER_CONNECTIONS选项，值为0表示“以系统变量max_user_connections的值为准”。

下面是一些演示GRANT语句各种用法的语句。关于其他示例请参考13.2.2节。与启用SSL有关的信息请参考13.5节。对于其中的每一种情况，没有给出IDENTIFIED子句，因为假定CREATE USER已经为该账户创建并分配了密码。

让名为paul的账户可以从任何一台主机访问sampdb数据库里的所有表。下面两条语句是等价的，因为省略账户名中的主机名部分相当于用%指定主机名：

```
GRANT ALL ON sampdb.* TO 'paul';
```

```
GRANT ALL ON sampdb.* TO 'paul'@'%';
```

授予账户只读权限，使其只能读取 menagerie 数据库里的表。用户lookonly 可以从xyz.com域里的任何一台主机进行连接：

```
GRANT SELECT ON menagerie.* TO 'lookonly'@'%xyz.com';
```

授予账户所有权限，但是仅限于 sampdb 数据库里 member 表的所有权限。用户member_mgr只能从指定的那台主机进行连接：


```
GRANT ALL ON sampdb.member TO 'member_mgr'@'boa.snake.net';
```

授予账户超级用户权限，其中包括能够对其他用户进行授权。此用户必须从本地主机连接服务器：

```
GRANT ALL ON *.* TO 'superduper'@'localhost' WITH GRANT OPTION;
```

授予匿名用户完全访问menagerie数据库的权限：

```
GRANT ALL ON menagerie.* TO ''@'localhost';
```

授予报告生成者（report-generator）账户权限，使其可以执行存储在admin数据库里的例程：

```
GRANT EXECUTE ON admin.* TO 'report_generator'@'localhost';
```

授予账户完全访问privatedb数据库的权限，但是要求连接是通过带有有效X509证书的SSL建立的：

```
GRANT ALL ON privatedb.* TO 'paranoid'@'%mydomain.com' REQUIRE X509;
```

授权账户有限访问权限，使得该账户每小时只能发出100条语句，其中，最多可以有10条更新语句：

```
GRANT ALL ON test.* TO 'caleb'@'localhost'
```

```
WITH MAX_QUERIES_PER_HOUR 100 MAX_UPDATES_PER_HOUR 10;
```

授予clint权限，使其成为bart的代理：

```
GRANT PROXY ON 'bart'@'localhost' TO 'clint'@'localhost';
```

HANDLER

```
HANDLER tbl_name OPEN [[AS] alias_name]
```

```
HANDLER tbl_name READ
```

```
{FIRST | NEXT}
```

```
[where_clause] [limit_clause]
```

```
HANDLER tbl_name READ index_name
```

```
{FIRST | NEXT | PREV | LAST }
```

```
[where_clause] [limit_clause]
```

```
HANDLER tbl_name READ index_name
```

```
{< | <= | = | => | >} (expr_list)
```

```
[where_clause] [limit_clause]
```

```
HANDLER tbl_name CLOSE
```

HANDLER语句提供了一个面向InnoDB和MyISAM存储引擎的底层接口，它可以绕过优化器直接访问表内容。如果想要通过HANDLER接口访问某个表，则首先需要用HANDLER ... OPEN语句去打开它。这个表在你执行HANDLER ... CLOSE语句（显式关闭这个表）之前，或者在本次会话终止之前将一直处于打开状态。当表处于打开状态时，可以用HANDLER ... READ语句访问其内容。

HANDLER 没有提供任何针对数据并发修改情况的保护措施。它不锁定表，所以用HANDLER 打开的表不会阻塞其他客户端对其修改，而这些改变不一定能反映在你从表里读出来的行里。

请注意，有些语句可能会重置打开的处理器，并且会导致它定位不准，例如那些在打开了表的处理器会话里的语句，或者任何会话里的FLUSH TABLES语句。

```
INSERT
```

```
INSERT [DELAYED | LOW_PRIORITY | HIGH_PRIORITY] [IGNORE]
```

```
[INTO] tbl_name
```

```
[PARTITION (partition_name [, partition_name] ...)]
```

```
[(col_list)]
```

```
{VALUES | VALUE} (expr [, expr] ...) [, (...)] ...
```

```
[ON DUPLICATE KEY UPDATE col_name=expr [, col_name=expr] ...]
```

```

INSERT [DELAYED | LOW_PRIORITY | HIGH_PRIORITY] [IGNORE]

[INTO] tbl_name

[PARTITION (partition_name [, partition_name] ...)]

SET col_name=expr [, col_name=expr] ...

[ON DUPLICATE KEY UPDATE col_name=expr [, col_name=expr] ...]

INSERT [LOW_PRIORITY | HIGH_PRIORITY] [IGNORE]

[INTO] tbl_name

[PARTITION (partition_name [, partition_name] ...)]

[(col_list)]

{SELECT ... | (SELECT ...)}

[ON DUPLICATE KEY UPDATE col_name=expr [, col_name=expr] ...]

```

把行插到已有的tbl_name 表里面，并且返回实际插入的行数。自 MySQL 5.6.2起，INSERT针对分区表支持使用PARTITION子句，以表明行需要插到哪一个分区或子分区。在这种情况下，如果有某个行未被插到指定的分区，则会发生错误。

INSERT语句的语法有以下三种形式。

INSERT 语句的第一种形式要求把待插入的行全都写在VALUES()列表里。如果指定了col_list，则VALUES()列表必须为表中每列指定一个值。如果col_list由以逗号分隔的一个或者多个列名构成，则每一列的值必须在VALUES()列表中指定。列列表里没有列出的列将被设置为它们的默认值。可以指定多个值列表，即允许你使用一条INSERT语句插入多个行。

```

INSERT INTO absence (student_id, date) VALUES(14,'2011-11-03'),
(34,NOW());

```

其中，col_list和VALUES()可以为空，即允许你往表里插入一条各列全部是其默认值的行：

```
INSERT INTO t () VALUES();
```

INSERT语句的第二种形式把SET子句指定的列插到表里，SET子句负责把列设置为相应的表达式的值。没有出现在SET子句里的列将被设置为默认值。

```
INSERT INTO absence SET student_id = 14, date = '2011-11-03';
```

```
INSERT INTO absence SET student_id = 34, date = NOW();
```

单词DEFAULT可以用在VALUES列表里或者SET子句里，将列设置为其默认值（在不知道默认值是什么的情况下）。更一般地，为了在表达式中引用列的默认值，可以使用DEFAULT(col_name)。如果默认值为NULL，则下列语句将把i列设置为0；否则，会设置为1。

```
INSERT INTO t SET i = IF(DEFAULT(i) IS NULL,1,0);
```

INSERT语句的第三种形式先执行SELECT语句，然后再把检索到的行插到tbl_name表里。列的个数必须等于tbl_name里的列数，或者col_list指定的列数（如果指定了列列表）。如果指定了列列表，则那些没有出现在列表里的列将被设置为其默认值。

```
INSERT INTO score (student_id, score, event_id)
```

```
SELECT student_id, 100 AS score, 15 AS event_id FROM student;
```

不能使用子查询从你正在插入行的那个表里选取行。

如果在执行INSERT 语句时启用了严格SQL模式，那么将其定义里没有DEFAULT 子句的列忽略掉，或者使用DEFAULT来指定它的值，都会导致错误。

如果插入一个行会导致唯一性索引中出现重复的键值，那么 INSERT 语句将终止执行并报告出错，剩余的行将不再被插入。加上IGNORE关键字将使它忽略这样的行，继续执行，不会报告出错。在严格SQL模式下，IGNORE 关键字还会使得INSERT 语句把会导致它终止执行的数据转换错误当作一个不那么致命的警告来处理。在这种情况下，列会被设置为最接近的合法值。

ON DUPLICATE KEY UPDATE子句用于解决新插入的行导致某个唯一值索引发生键值重复的冲突问题。带有这个子句的INSERT 语句在遇到上述问题时将被转换为一条UPDATE 语句，它将使用UPDATE 关键字后面的列赋值表达式对已有行的列进行修改。如果确实发生了一次这样的更新，INSERT语句返回的受影响行的计数值将会是2，而不是1。

选项DELAYED、LOW_PRIORITY和HIGH_PRIORITY会对INSERT语句的执行调度产生影响。

DELAYED。它会导致行被放入一个延迟插入队列，而INSERT语句则会立刻返回，以便客户端可以继续执行，无须等待。但是，在这种情况下，LAST_INSERT_ID()不会为表里的任何AUTO_INCREMENT列返回AUTO_INCREMENT值。DELAYED插入只适用于MyISAM表、MEMORY 表和ARCHIVE 表。DELAYED 在INSERT INTO ... SELECT 语句和INSERT INTO ... ON DUPLICATE KEY UPDATE 语句里会被忽略掉。如果DELAYED与存储过程或触发器搭配使用，使得INSERT语句引用了访问表或触发器的存储过程，或者在某个存储函数或触发器内调用了INSERT语句，那么它也会被忽略。视图或分区表都不支持DELAYED。

InnoDB是默认存储引擎，不支持DELAYED。另外，自MySQL 5.6.6起，DELAYED已被弃用，在将来的某个时候，它将不复存在，因此最好避免使用它。

LOW_PRIORITY。它会使INSERT语句延迟，一直到没有任何客户端读取该表为止。

HIGH_PRIORITY。它会使服务器选项--low-priority-updates 对于单条语句不起作用。（如果在启动服务器时使用了--low-priority-updates 选项，它将降低INSERT和其他更新语句的优先级。）HIGH_PRIORITY 选项的另一种用途是预防INSERT 语句与访问同一个的表的SELECT语句并发执行。

选项LOW_PRIORITY和HIGH_PRIORITY只适用于那些使用表级锁定的存储引擎，如MyISAM或MEMORY。

INSTALL PLUGIN

```
INSTALL PLUGIN plugin_name SONAME 'plugin_lib'
```

从插件库对象文件里安装某个插件，该对象文件必须存在于plugin_dir系统变量指定的那个目录里。此语句还会在mysql.plugin表里注册该插件，以便服务器在随后的重启过程中可以加载它。INSTALL PLUGIN要求对mysql.plugin表拥有INSERT权限。更多信息请参考12.4节。

KILL

```
KILL [CONNECTION | QUERY] thread_id
```

终止thread_id指定的服务器线程。你必须拥有SUPER权限才能终止不属于你自己的线程。KILL语句每次只能终止一个线程。mysqladmin kill命令也

可以完成相同的操作，但是它允许在命令行上同时给出多个线程的ID值。

CONNECTION选项的效果和没有选项时一样：终止有着给定ID的线程。QUERY选项将终止给定线程正在执行的所有语句，但不终止该线程本身。

LOAD DATA

```
LOAD DATA [LOW_PRIORITY | CONCURRENT] [LOCAL] INFILE  
'file_name'
```

```
[IGNORE | REPLACE]
```

```
INTO TABLE tbl_name
```

```
[PARTITION (partition_name [, partition_name] ...)]
```

```
[CHARACTER SET charset]
```

```
[field_options] [line_options]
```

```
[IGNORE n {LINES | ROWS}]
```

```
[(col_or_user_var_name, ...)]
```

```
[SET col_name = expr [, col_name = expr] ...]
```

LOAD DATA语句从file_name文件里读取输入记录，并把它们批量加载到tbl_name表里。这要比使用一组INSERT语句来完成的速度快。自MySQL 5.6.2起，LOAD DATA针对分区表支持使用PARTITION子句，以表明需要将行插到哪一个分区或子分区。在这种情况下，如果有某个行未被插到指定的分区，则会发生错误。

LOAD DATA语句将返回一个格式如下所示的信息字符串：

```
Records: n Deleted: n Skipped: n Warnings: n
```

如果警告计数值不为零，则可以使用SHOW WARNINGS语句来查看到底出了什么问题。

LOW_PRIORITY 选项将使这条语句的执行被延迟到没有客户端读取该表为止。此选项只适用于那些使用表级锁定的存储引擎，如MyISAM或MEMORY。

CONCURRENT选项只适用于MyISAM表。如果表的中间没有空闲块，则新行会加载到表的末尾。此时，其他客户可以在往表里加载行时检索该表。

如果不带LOCAL关键字，则服务器会直接在服务器主机上读取文件。在这种情况下，你必须拥有FILE权限，并且文件必须位于默认数据库的目录中，或者是完全可读的。如果secure_file_priv系统变量的值不为空，该变量值应该是一个目录，而你打算读取的文件必须存放在该目录里。如果带LOCAL关键字，则客户端将读取客户端主机上的文件，并把其内容通过网络发送给服务器。在这种情况下，并不要求拥有FILE 权限，但是该文件对于客户端用户必须是可读的。LOCAL 可以选择性地被禁用或启用。如果在服务器端禁用了LOCAL，你就不能在客户端使用它。如果在服务器端启用了LOCAL，那么它在客户端默认是禁用的，因此必须显式启用它。例如，当使用mysql程序时，你可以通过--local-infile标志来启用LOCAL功能。

如果在LOAD DATA语句里省略LOCAL，MySQL服务器将按以下规则来定位文件：

如果'file_name'是一个绝对路径名，则服务器将从根目录直接查找文件。

如果'file_name'是一个相对路径名，则需要根据该名字是否只包含一个部分来进行解释。如果是，则服务器会在默认数据库的数据库目录里去查找文件。如果文件名包含有多个组成部分，则服务器将从它自己的数据目录里查找文件。

如果带有LOCAL，则文件名的解释规则如下。

如果'file_name'是一个绝对路径名，则客户端将从根目录直接查找文件。

如果'file_name'是一个相对路径名，则客户端将从当前目录开始查找文件。

对于Windows系统，文件名中的反斜线既可以写成一个斜线字符（/），也可以写成双反斜线（\\）。

默认情况下，MySQL会使用系统变量character_set_database所指定的字符集来解析文件的内容。如果想显式表明文件的字符集，可以用CHARACTER SET 子句。（不过，该字符集不能是ucs2、utf16、utf16le或utf32——这也表示你不能使用这些字符集来加载文件。）

对于那些会导致唯一性索引里出现重复值的行，MySQL 将按以下规则处理：如果指定了IGNORE，则忽略后出现的行；如果指定了REPLACE，则后出现的行将替换掉已有的行。如果两个选项都没有给出，则出现错误，并且忽略尚未加载的行。如果指定了 LOCAL，则文件传输过程不会被中断，因此，如果这两个复制处理选项都没有给出的话，则默认行为类似于IGNORE。

子句field_options和line_options 用于设定数据的格式。（这两个子句里的可用选项也适用于SELECT ... INTO OUTFILE语句的对应子句。）下面是这两个子句的语法：

field_options:

[FIELDS

[TERMINATED BY 'str']

[[OPTIONALLY] ENCLOSED BY 'char']

[ESCAPED BY 'char']]

line_options:

[LINES

[STARTING BY 'str']

[TERMINATED BY 'str']]

'str'和'char'的值允许包含下表里列出的各种转义序列，用以表示特殊字符。在给出这些转义序列时，大小写应该与表中所示一致。

转义序列	含义
\0	NUL (零值字节)
\b	退格
\n	换行符 (换行键)
\r	回车
\s	空格
\t	制表符
\'	单引号
\"	双引号
\\	反斜线
\Z	Ctrl+Z (Windows系统中的EOF字符)

还可以用十六进制常数来表示任意字符。例如，`LINES TERMINATED BY 0x02` 表示各个行是以Ctrl+B (ASCII 2) 字符结尾的。

如果指定了FIELDS子句，那么至少还要再给出TERMINATED BY、ENCLOSED BY或ESCAPED BY选项中的一个。如果同时给出多个选项，则它们的顺序可以是任意的。类似地，如果指定了 LINES子句，那么至少还要再给出STARTING BY或TERMINATED BY选项中的一个，不过，如果同时给出这两个选项，则它们的顺序可以是任意的。如果同时指定了FIELDS和LINES，则FIELDS必须放在LINES的前面。

FIELDS子句的各个选项的用法如下所示。

TERMINATED BY值指定的是用于分隔行里的各个值的（一个或多个）字符。

ENCLOSED BY 值指定的是引号字符——如果它存在于多个字段值的末尾，则它会被去掉。无论是否存在OPTIONALLY，都会这样处理。对于输出（`SELECT ... INTO OUTFILE`），由ENCLOSED BY字符会把输出行里的各个字段值括起来。如果指定了OPTIONALLY，则CHAR列和VARCHAR列的值会使用引号引起来。

如果想要在输入字段值里包含ENCLOSED BY字符的实例，则必须双写这个引号符或者在其前面加上ESCAPED BY字符。否则，引号符便会被解释为字段值的结束标记。对于输出语句，MySQL将自动在字段值中ENCLOSED BY字符的实例前面加上一个ESCAPED BY字符。

ESCAPED BY值指定的是如何对特殊字符进行转义。在下面的例子里，假设转义字符是反斜线（\）。对于输入语句，不带引号的转义序列\N（反斜线-N）将被解释为NULL。转义序列\0（反斜线-ASCII码值0）将被解释为一个零值字节。对于其他经转义的字符，去掉转义字符之后剩余的字符将会作为字符本身。例如，\"将被解释为一个双引号，即使字段值包含在双引号之内。

对于输出语句，转义字符将把NULL值编码为不带引号的转义序列\N，把零值字节编码为\0。另外，ENCLOSED BY字符和ESCAPED BY字符的实例将像字段开头的（多个）字符和行末尾的字符串一样被加上一个前导的转义字符。但是，如果ESCAPED BY字符是空的（ESCAPED BY ''），则不进行转义处理。（在这种情况下，NULL写作NULL，而不是\N。）如果想把转义字符设定为反斜线“\”，就必须双写它（ESCAPED BY '\\’）。

子句LINES的各个选项用法如下所示。

STARTING BY值指定的是一个或多个行起始字符。这个值及其前面的所有内容（同一行里）都会被当作行的开头部分。

TERMINATED BY值指定的是一个或多个行结尾字符。

如果子句FIELDS和LINES都没有给出，则默认情况如下所示：

```
FIELDS TERMINATED BY '\t' ENCLOSED BY " ESCAPED BY '\\'
```

```
LINES STARTING BY " TERMINATED BY '\n'
```

换句话说，同一行内的字段将以制表符分隔且不带引号，反斜线将被当作转义字符，各行数据将以换行符结尾。

如果FIELDS子句所设定的TERMINATED BY字符和ENCLOSED BY字符都是空的，那么数据文件将使用固定宽度的行格式，并且各字段值之间也没有分隔符。列值将根据该列的显示宽度从数据文件里被读出（对于输出语句，则是写入）。例如，对于输入，会把VARCHAR(15)列和MEDIUMINT(5)列分别当作一个15-字符宽和一个8-字符宽的字段读入。对于输出，则会把它们分别写为一个15个字符宽和一个8个字符宽的字段。NULL值将写为由空格构成的字符串。

输入数据文件里的NULL值用不带引号的转义序列\N表示。如果FIELDS ENCLOSED BY字符不为空，那么所有的非NULL输入值就都将用给定的包围字符（enclosed-by character）引起来，而不带引号的单词NULL也将被解释为一个NULL值。

如果指定了IGNORE n LINES子句，则输入的前n行将被删除。（ROWS是LINES的同义词。）例如，如果数据文件的第一行是列标题，而你又不想把它们放到数据库表里去，则需要使用IGNORE 1 LINES子句：

```
LOAD DATA LOCAL INFILE 'mytbl.txt' INTO TABLE mytbl IGNORE 1 LINES;
```

默认情况下，输入行将被假定为包含表里每一列的一个值。如果指定了由一个或者多个以逗号分隔的列名构成的列表，则输入行应该包含每个指定列的一个值。列表中没有指定的列将被设置为它们的默认值。如果输入行提供的数据值的个数少于列的个数，则那些值缺失的列会被设置为它们的默认值。

如果在执行LOAD DATA时启用了严格SQL模式，那么对于其定义里没有DEFAULT子句的列，如果其值缺失，则会出错。

列表可以包含列名或用户变量名，SET子句可以对要加载到表里的输入值做一些必要的处理。例如，下面这条语句将把第一个输入列加载到表里的col1列，忽略第二个输入列，把第三个和第四个输入列的和加载到col2列，再使用UUID()函数为col3列提供一个即时生成的值：

```
LOAD DATA LOCAL INFILE 'mytbl.txt' INTO TABLE mytbl  
  
(col1,@skip,@addend1, @addend2)  
  
SET col2 = @addend1 + @addend2, col3 = UUID();
```

SET子句可以包含一个或多个以逗号分隔的赋值表达式。每个赋值表达式的左侧必须是表里的一个列名。MySQL不允许使用用户变量来读取固定宽度的输入行，因为它无法根据用户变量来确定各列的宽度。标量子查询可以用来提供列值，但子查询所查询的表和正被加载数据的表不能是同一个。

如果你有一个在Windows平台上创建的以制表符分隔各字段的文本文件，那么你可以使用默认的列分隔符，但是输入行很可能是以“回车/换行符”结束的。为了加载这样的文件，需要指明那些行是以“回车/换行符”结束的：

```
LOAD DATA LOCAL INFILE 'mytbl.txt' INTO TABLE mytbl
```

```

LINES TERMINATED BY '\r\n';

```

有一部分 Windows 程序在创建数据文件时会沿用MS-DOS 年代的奇怪约定，在文件末尾加上Ctrl+Z字符作为文件结束标记，加载这样的数据文件可能会在数据库里留下一个异常的行。要解决这个问题，可以换个没有这种“坏习惯”的程序来创建文件，也可以在加载文件后及时删掉那个异常的行。

采用以逗号分隔的值（Comma-Separated Value, CSV）格式的文件以逗号作为字段之间的分隔符，字段本身或许还用双引号引了起来。假设各输入行以换行符结束，用来加载这类数据文件的LOAD DATA语句应该是如下所示的样子：

```

LOAD DATA LOCAL INFILE 'mytbl.txt' INTO TABLE mytbl

```

```

FIELDS TERMINATED BY ',' ENCLOSED BY '"';

```

十六进制表示法对于指定任意控制字符来讲很有用。下面这条语句可以用来加载各字段之间以Ctrl+A（ASCII码值为1）字符分隔、输入行以Ctrl+B（ASCII码值为2）字符结束的数据文件：

```

LOAD DATA LOCAL INFILE 'mytbl.txt' INTO TABLE mytbl

```

```

FIELDS TERMINATED BY 0x01 LINES TERMINATED BY 0x02;

```

```

LOAD INDEX INTO CACHE

```

```

LOAD INDEX INTO CACHE

```

```

tbl_index_spec [, tbl_index_spec] ...

```

```

tbl_index_spec:

```

```

tbl_name

```

```

[PARTITION (partition_name [, partition_name] ... | ALL)]

```

```

[[INDEX | KEY] (index_name [, index_name ] ...)]

```

```

[IGNORE LEAVES]] ...

```

把给定的MyISAM表的索引加载到分配给该表的键缓存中。如果没有使用CACHE INDEX语句为某个表特意分配一个缓存，则该表的索引将被加载到默认的键缓存中。默认情况下，所有的索引块都将被加载；如果指定了IGNORE LEAVES子句，则只有索引树中非叶节点的索引块会被加载。

对于分区表， PARTITION子句允许加载特定分区的缓存。

类似于CACHE INDEX语句的情况，LOAD INDEX INTO CACHE语句的语法允许只加载个别的索引，但是它目前的实现把一个表的所有索引都加载了。

必须对这条语句列出的每一个表都拥有INDEX权限。

LOAD INDEX INTO CACHE语句与CHECK TABLE语句产生的输出格式相同。

与MyISAM键缓存管理有关的更多信息请参考12.7.2节。

LOAD XML

```
LOAD XML [LOW_PRIORITY | CONCURRENT] [LOCAL] INFILE  
'file_name'
```

```
[IGNORE | REPLACE]
```

```
INTO TABLE tbl_name
```

```
[PARTITION (partition_name [, partition_name] ...)]
```

```
[CHARACTER SET charset]
```

```
[ROWS IDENTIFIED BY '<tag_name>']
```

```
[IGNORE n {LINES | ROWS}]
```

```
[(col_or_user_var_name, ...)]
```

```
[SET col_name = expr [, col_name = expr] ...]
```

这条语句类似于LOAD DATA，主要差异如下：

被加载的文件应该为XML文件。

ROWS IDENTIFIED BY 子句（如果给出了）用于指定标签（包括尖括号），在文件的开始和结束处标识行的位置。默认值为'<row>'。

在输入XML文件里指定目标表的各个行的格式有以下三种。在同一个文件里，对于不同的行可以使用不同的格式。

在<row>元素内部，属性名和属性值分别对应的是列名和列值。

```
<row col_name1="value1" col_name2="value2" ... />
```

在<row>元素内部，子元素标签名和内容分别对应的是列名和列值。

```
<row>
```

```
<col_name1>value1</col_name1>
```

```
<col_name2>value2</col_name2>
```

```
...
```

```
</row>
```

在<row>元素内部，子元素属性值和内容分别对应的是列名和列值。

```
<row>
```

```
<field name="col_name1">value1</field>
```

```
<field name="col_name2">value2</field>
```

```
...
```

```
</row>
```

LOCK TABLE

LOCK {TABLE | TABLES}

tbl_name [[AS] alias_name] lock_type

[, tbl_name [[AS] alias_name] lock_type] ...

获取指定表或视图的锁，如有必要，则会一直等到获得所有的锁。如果你锁定了视图，则此语句会锁定该视图使用的那个基表。此语句要求对每个被锁定的表或视图都要拥有 LOCK TABLES 权限和SELECT权限。

每个lock_type的值都必须是下列之一。

READ [LOCAL]：获取一个读锁。这种锁将阻塞其他客户端对表的写操作，但允许其他客户端对表进行读操作。

READ LOCAL是READ锁的一个变体，是专为并发插入操作而设计的。它只能用在MyISAM表上，并且要求MyISAM表不能有任何因删除或更新操作而产生的空闲块。READ LOCAL允许你显式地锁定一个表，不过也允许其他客户端对它进行并发插入操作。（如果表内部存在空闲块，这种锁将被视为一个普通的READ锁。）

[LOW_PRIORITY] WRITE：获取一个写锁。这样会阻止所有客户端对表进行读取或写入。修饰符LOW_PRIORITY可用于更改锁获取行为，但是自MySQL 5.5.3起，它便没有任何作用。

LOCK TABLE 语句会释放你当前拥有的所有锁。因此，如果想要锁定多个表，则必须用一条LOCK TABLE语句来将它们全部锁定。在使用LOCK TABLE获得锁之后，你不能引用任何未锁定的表。当会话终止时，服务器会自动释放它所拥有的所有锁。

LOCK TABLE 语句允许给待锁定的表取一个别名，以便在随后的查询里再引用这个表时，可以用别名来锁定它。如果在同一条查询里多次引用了某个表，则必须为该表的每一次引用分别获取一个锁，同时根据需要锁定别名。必须在同一条语句里获得所有的锁。

```
LOCK TABLE student READ, score WRITE, grade_event READ;
```

```
LOCK TABLE member READ;
```

```
LOCK TABLE t AS t1 READ, t AS t2 READ;
```

如果某个事务正在执行，则LOCK TABLE会引发一个隐式提交。如果使用START TRANSACTION来开始一个事务，则使用LOCK TABLE获取的表锁可以被隐式地释放掉。

```
OPTIMIZE TABLE
```

```
OPTIMIZE [NO_WRITE_TO_BINLOG | LOCAL]
```

{TABLE | TABLES} tbl_name [, tbl_name] ...

语句DEL TETE、REPLACE和UPDATE会使表的内部出现未使用区域。如果要将其算计在内，则可以使用OPTIMIZE TABLE 语句来执行表优化操作。它可用于处理 InnoDB 表、MyISAM 表和ARCHIVE表。OPTIMIZE TABLE要求对每个表都拥有SELECT和INSERT权限。

如果启用了二进制日志记录，那么在不给出NO_WRITE_TO_BINLOG或LOCAL选项时，MySQL会把OPTIMIZE TABLE语句写入二进制日志里。

对于InnoDB表，OPTIMIZE TABLE会被映射成ALTER TABLE，以更新表索引统计，并释放聚簇索引里未使用的空间。

对于MyISAM表，OPTIMIZE TABLE会执行以下几个动作。

对表进行碎片整理，以消除浪费掉的空间，并缩小表的大小。

把因碎片化而散布在各处的可变长行的内容合并在一起，让各行的内容无间断地存放在同一处。

如有必要，将索引页面排序。

更新内部表统计。

针对MyISAM表执行OPTIMIZE TABLE语句，类似于带--check-only-changed、--quick、--sort-index和--analyze选项执行myisamchk程序。不过，在使用myisamchk时，必须设法阻止服务器在你检查表的过程中访问它们。在使用OPTIMIZE TABLE语句时，服务器除了会完成自己的工作，还会替你阻止其他客户端在表正在被优化时访问它。

对于ARCHIVE表，OPTIMIZE TABLE语句将对表进行分析并重新压缩它们以减少其存储空间占用量。

OPTIMIZE TABLE语句与CHECK TABLE语句产生的输出格式相同。

PREPARE

PREPARE stmt_name FROM {'str' | @var_name}

对一条语句进行预处理并把它命名为 stmt_name。经过预处理的语句可以用EXECUTE 语句来执行，用DEALLOCATE PREPARE语句来释放。如果已经存在一个同名的预处理语句，PREPARE语句将先释放那条旧语句后再处理新语句。这些语句名不区分大小写。

可以使用字符串字面量或用户变量来指定待预处理的语句。允许使用PREPARE语句来进行预处理的语句随着时间的推移正变得越来越多。最初只有CREATE TABLE、DELETE、DO、INSERT、REPLACE、SELECT、SET、UPDATE语句以及绝大多数SHOW语句。其他语句已被添加进来，通过查询所用MySQL版本的MySQL 参考手册，可以看到都包含了哪些语句。PREPARE、EXECUTE和DEALLOCATE PREPARE语句不能被预处理。

可以在预处理语句里使用“?”字符作为占位符。当以后执行该语句时，再为其中的占位符提供相应的数据值。占位符使得预处理语句可以接受输入参数，以便可以每次使用不同的数据值去执行同一条预处理语句。标识或SQL关键字不能用占位符。

PREPARE、EXECUTE和DEALLOCATE语句共同构成了SQL语言中的预处理语句接口。请不要把它们与第7章和附录G里讨论的二进制API混为一谈，彼此在效率方面也不相同。

PURGE BINARY LOGS

```
PURGE { BINARY | MASTER } LOGS { TO 'log_name' | BEFORE 'date' }
```

把服务器上早于给定文件或给定日期（'CCYY_MM_DD hh:mm:ss'格式）生成的二进制日志文件全部删除，重置二进制日志索引文件，使它只列出那些未被删除的日志文件。通常情况下，可以在主服务器的每一个从服务器上运行SHOW SLAVE STATUS语句之后使用这个语句，以确定还有哪些日志文件仍在使用中。此语句要求拥有SUPER权限。

下面这条语句将删除二进制日志文件 binlog.000001 到 binlog.000009，并且会使binlog.000010成为剩余文件中的第一个：

```
PURGE BINARY LOGS TO 'binlog.000010';
```

RELEASE SAVEPOINT

```
RELEASE SAVEPOINT savepoint_name
```

释放当前事务中名为的savepoint_name的保存点；如果该保存点不存在，则返回一条出错消息。释放还原点不会导致事务被提交或回滚。

RENAME TABLE

```
RENAME { TABLE | TABLES } tbl_name TO new_tbl_name
```

```
[, tbl_name TO new_tbl_name] ...
```

重命名一个或者多个表。这条语句与 ALTER TABLE... RENAME语句相似，但RENAME TABLE语句能够同时对多个表进行重命名并能在命名过程中锁定它们。如果需要确保给定表在重命名的过程中不会被其他客户端修改，就应该使用RENAME TABLE语句。

如果重命名的是一个InnoDB表，其他的表又依赖于该 InnoDB表的外键关系，InnoDB存储引擎将调整那些依赖关系指向重命名后的表。

RENAME TABLE语句不能用来重命名TEMPORARY表。

如果某个表上有触发器，那么在试图把它重命名到另一个数据库里去时会导致一个错误。

RENAME TABLE语句也可以用于视图，但是不能把视图重命名到另一个数据库里去。

RENAME USER

```
RENAME USER from_account TO to_account
```

```
[, from_account TO to_account] ...
```

重命名一个或多个MySQL账户。这条语句要求拥有全局级CREATE USER权限，或者对mysql数据库拥有UPDATE权限。

RENAME USER 语句将把每个from_account 重命名为相应的to_account。如果from_account 不存在或是某个to_account 已经存在，则报告出错。账户名必须以'user_name'@'host_name'的格式给出，请参考13.2.1.1节。

```
RENAME USER 'myname'@'localhost' TO 'yourname'@'localhost';
```

RENAME USER会更新原账户所拥有的权限，以应用到新的账户。不过，它不会更改指向原账户的任何对象定义。例如，所有带DEFINER子句（指向原账户）的存储程序都必须被更新，以指向那个新的账户。

REPAIR TABLE

```
REPAIR [NO_WRITE_TO_BINLOG | LOCAL]
```

```
{TABLE | TABLES} tbl_name [, tbl_name] ...[option] ...
```

这条语句的用途是对受损表进行修复。它可用于修复 MyISAM 表、ARCHIVE 表和CSV 表。REPAIR TABLE要求对每个表都拥有SELECT和INSERT权限。

如果启用了二进制日志记录，那么在不给出NO_WRITE_TO_BINLOG或LOCAL选项时，MySQL会把REPAIR TABLE语句写入二进制日志里。

不带任何选项的REPAIR TABLE语句执行的是普通的表修复操作。除了应该在只包含唯一值的索引里出现了重复值这一情况无法处理之外，该操作可以修复大部分的问题。下面列出的是各种允许使用的option值。这些选项适用于MyISAM表，其他存储引擎可能会忽略它们。

EXTENDED。它所表示的是，执行扩展修复，重建索引。

QUICK。它所表示的是，只是快速修复索引，而不管数据文件。

USE_FRM。它所表示的是，使用表的.frm 文件，重新初始化索引文件，并确定如何解释数据文件的内容，以便能够重建索引。如果索引丢失或损坏后不可恢复，则此选项非常有用。不过，它应该作为最后的打算，并且只有在当前的MySQL版本与创建表时所使用的MySQL版本一样时才使用它。否则，有可能会进一步破坏表。USE_FRM选项不能用于分区表。

REPAIR TABLE语句与CHECK TABLE语句产生的输出格式相同。

REPLACE

REPLACE [DELAYED | LOW_PRIORITY]

[INTO] tbl_name

[PARTITION (partition_name [, partition_name] ...)]

[(col_list)]

{VALUES | VALUE} (expr [, expr] ...) [, (...)] ...

REPLACE [DELAYED | LOW_PRIORITY]

[INTO] tbl_name

[PARTITION (partition_name [, partition_name] ...)]

[(col_list)]

{SELECT ... | (SELECT ...)}

REPLACE [DELAYED | LOW_PRIORITY]

[INTO] tbl_name

[PARTITION (partition_name [, partition_name] ...)]

SET col_name=expr [, col_name=expr] ...

REPLACE语句的基本动作与INSERT相似，不同之处在于：如果将被插入的行会导致表的唯一性索引里出现重复键值，MySQL将先删除旧行，然后再插入新行。因此，REPLACE语句的语法里不存在IGNORE选项。同样，REPLACE不支持ON DUPLICATE KEY UPDATE。更多信息请参考INSERT条目。REPLACE要求对该个表拥有INSERT和DELETE权限。

自MySQL 5.6.2起，REPLACE针对分区表支持使用PARTITION子句，以表明行需要插到哪一个分区或子分区。在这种情况下，如果有某个行未被插到指定的分区，则会发生错误。

如果表有多个唯一性索引，则有可能发生一条REPLACE语句删除多个行的情况。当新行同时与几个唯一性索引中的值匹配时，MySQL会先删除所匹配的那几个行，然后再插入新行。

RESET

RESET option [, option] ...

RESET语句对日志或缓存信息的影响与FLUSH语句相同。此语句要求拥有RELOAD权限。

option的各种取值及其含义如下所示。

MASTER。它会删除复制主服务器上已有的二进制日志文件，重新创建一个新文件并把它编号为000001，再重置二进制日志的索引文件，使它只列出新文件。

QUERY CACHE。它会清除查询缓存，把当前注册在其中的查询命令全部除掉。如果只想消除查询缓存中的碎片而不想清除它的全部内容，就应该使用FLUSH QUERY CACHE语句。

SLAVE。告诉服务器（如果它是作为一个从服务器），删除所有已有中继日志文件，开始一个新中继日志，“忘记”其复制坐标（也就是说，它当前使用的复制二进制日志的文件名和这些文件中的读写位置）。

REVOKE

```
REVOKE priv_type [(col_list)] [, priv_type [(col_list)] ...]
```

```
ON [TABLE | FUNCTION | PROCEDURE]
```

```
{*. * | * | db_name. * | db_name. tbl_name
```

```
| tbl_name | db_name.routine_name}
```

```
| tbl_name | db_name.routine_name}
```

```
FROM account [, account] ...
```

```
REVOKE ALL [PRIVILEGES], GRANT OPTION
```

```
FROM account [, account] ...
```

```
REVOKE PROXY ON account
```

```
FROM account [, account] ...
```

REVOKE 语句用于撤销（一个或多个）给定账户的权限。其中，账户名的指定格式必须为 'user_name'@'host_name'，请参考13.2.1.1节。如果账户不存在，则会导致错误。

在第一种语法里，子句priv_type、col_list和ON的指定方式与GRANT语句一样。如果想要使用这条语句，则必须同时拥有GRANT OPTION权限和想要撤销的那些权限。

第二种语法有一个固定的权限列表，但是没有ON子句。它将撤销所有给定账户的全部权限。第二种语法要求拥有全局级CREATE USER权限，或者对mysql数据库拥有UPDATE权限。

第三种语法形式会从FROM子句指定的各个账户里来撤销ON所指定的那个账户的PROXY权限。

REVOKE 语句不会从mysql.user 权限表删除账户行。这意味着即使把某个账户的权限全都撤销了，该账户也可以用来连接MySQL服务器。要想彻底删除某个账户，必须使用DROP USER语句（或者从mysql.user表手动删除对应于该账户的行）。

撤销member_mgr用户对sampdb数据库里的member表进行修改的权限：

```
REVOKE INSERT,DELETE,UPDATE ON sampdb.member
```

```
FROM 'member_mgr'@'boa.snake.net';
```

撤销本地主机上的匿名用户对menagerie数据库里的某个表的全部权限：

```
REVOKE ALL ON menagerie.pet FROM '@'localhost';
```

ALL能撤消除GRANT OPTION权限以外的全部权限。如果想把GRANT OPTION权限也撤销，则必须显式地这么做：

```
REVOKE GRANT OPTION ON menagerie.pet FROM '@'localhost';
```

撤销superduper@localhost账户在所有级别上的全部权限：

```
REVOKE ALL, GRANT OPTION FROM 'superduper'@'localhost';
```

根据proxy_user撤销proxied_user账户的PROXY权限：

```
REVOKE PROXY ON proxied_user FROM proxy_user;
```

ROLLBACK

```
ROLLBACK [WORK] [AND [NO] CHAIN] [[NO] RELEASE]
```

```
ROLLBACK [WORK] TO [SAVEPOINT] savepoint_name
```

回滚当前事务里的语句所做出的修改，把各有关表恢复到修改前的状态。这只适用于支持事务处理的存储引擎。（对于非事务型存储引擎，语句在执行完毕后会立刻被提交，因而无法回滚。）

可选的WORK关键字目前没有任何作用。CHAIN和RELEASE子句的作用与COMMIT语句的同名子句相同。

如果指定了TO SAVEPOINT子句，那么这条语句将只把当前事务回滚到给定的保存点。

如果没在事先使用START TRANSACTION语句或是通过把autocommit变量设置为0的办法禁用自动提交模式，那么ROLLBACK语句将什么也不做。

SAVEPOINT

SAVEPOINT savepoint_name

创建一个名为savepoint_name的事务保存点。任何与之同名的已有保存点都将被删除。在当前事务里，可以用ROLLBACK TO SAVEPOINT语句把当前事务回滚到给定的保存点。

SELECT

SELECT

[select_option] ...

select_expr [, select_expr] ...

[FROM tbl_refs

[WHERE where_expr]

[GROUP BY {col_name | expr | position}

[ASC | DESC], ... [WITH ROLLUP]]

[HAVING where_expr]

[ORDER BY {col_name | expr | position}

[ASC | DESC], ...]

[LIMIT {[skip_count,] show_count | show_count OFFSET skip_count}]

[PROCEDURE procedure_name([param_list])]

[

```

INTO OUTFILE 'file_name'

[CHARACTER SET charset]

[field_options] [line_options]

| INTO DUMPFILE 'file_name'

| INTO var_name [, var_name] ...

]

[FOR UPDATE | LOCK IN SHARE MODE] ]

```

SELECT语句一般用来从一个或多个表里检索行。不过，因为SELECT语句中的各个组成部分都是可选的（关键字SELECT除外，且必须至少要有有一个select_expr子句），所以还可以用它来编写表达式计算语句：

```
SELECT 'one plus one =', 1+1;
```

为了与那些要求SELECT语句必须有一个FROM子句的数据库系统保持兼容，MySQL准备了一个DUAL关键字来充当伪表：

```
SELECT 'one plus one =', 1+1 FROM DUAL;
```

子查询是嵌套在一个SELECT语句里的另一个SELECT语句，在2.9节里可以找到很多例子。子查询还可以用在DELETE和UPDATE语句的WHERE子句里，或者和INSERT和REPLACE语句一起使用。不过，你不能使用子查询来查询正被修改的那个表。

每个select_option值可以为下面选项中的一项。

ALL、DISTINCT、DISTINCTROW：这几个关键字控制着是否需要返回重复的行。ALL 表示将返回所有的行，它是默认值。DISTINCT和DISTINCTROW 表示将从结果集里剔除重复的行。

HIGH_PRIORITY：指定 HIGH_PRIORITY 可以使语句拥有更高的优先级，如果原本要等待的话。如果有客户端正在用SELECT语句读取某个表，而且SELECT语句有HIGH_PRIORITY选项，那么对这个表进行写操作的其他语句（如INSERT和UPDATE）必须等待这个读操作完成之后才能执行，这条SELECT语句有高于那些写操作语句的优先级。不过，因为此选项会延缓写语句的执行，所以应该只在那些会很快执行完成并且需要立刻执行的SELECT语句里才使用

此选项。此选项只适用于那些使用表级锁定的存储引擎，如 MyISAM 或 MEMORY。

SQL_BUFFER_RESULT：在等待将整个查询结果发回到客户端的同时，告诉服务器将查询结果缓冲到一个单独的临时表里，而不是锁定SELECT里指定的（一个或多个）表。这能使服务器早点儿释放锁，从而使其他客户端更快地访问那些表。不过，使用此选项会消耗更多的磁盘空间和内存。

SQL_CACHE、**SQL_NO_CACHE**：如果查询结果是可以缓存的，并且查询缓存是以DEMAND模式操作的，那么 **SQL_CACHE** 选项将导致 **SELECT** 语句的查询结果被缓存起来。

SQL_NO_CACHE选项将禁用各种查询结果的缓存机制。这两个选项是互斥的，不能同时出现在子查询里，也不能出现在UNION的第一个SELECT里。

SQL_CALC_FOUND_ROWS：通常情况下，带有LIMIT子句的SELECT语句所返回的行计数值将等于实际返回的行的个数。如果在这类 **SELECT** 语句里使用 **SQL_CALC_FOUND_ROWS** 选项，则服务器将把不带 **LIMIT** 时返回的行数也统计出来。如果想知道这个计数值到底是多少，则需要在该SELECT语句返回后立刻执行一条**SELECT FOUND_ROWS()** 语句。

SQL_BIG_RESULT、**SQL_SMALL_RESULT**：这两个关键字反映结果集尺寸是小还是大——根据这一信息，优化器可以更高效地处理查询语句操作。

STRAIGHT_JOIN：强制表必须按它们在FROM子句中的先后顺序进行连接。如果你认为优化器做出的选择不够好，可以利用此选项来指定表检索顺序。

select_expr 表达式列举SELECT 语句将要返回的输出列，多个输出列之间要用逗号彼此分隔开。输出列可以是表中的列，也可以是表达式（包括标量子查询）。还可以利用**AS alias_name**语法（**AS**关键字是可选的）给输出列起一个别名。该别名将成为输出中的列名，并可以用在**GROUP BY**、**ORDER BY** 和**HAVING**子句里。但是，不能在**WHERE**子句里使用列的别名。

特殊符号* 表示的是“FROM 子句所指定的那个表里的所有列”，而 **tbl_name.***表示的是“tbl_name表里的所有列”。

FROM子句用来列出SELECT语句将从中选取行的表。MySQL支持下面这种连接语法：

tbl_refs:

tbl_ref[, tbl_ref]...

tbl_ref:

tbl_factor

| join_tbl

tbl_factor:

tbl_name

| (subquery) [AS] alias_name

| (tbl_refs)

| { OJ tbl_ref LEFT OUTER JOIN tbl_ref ON conditional_expr }

join_tbl:

tbl_ref [INNER | CROSS] JOIN tbl_factor [join_condition]

| tbl_ref STRAIGHT_JOIN tbl_factor [ON conditional_expr]

| tbl_ref {LEFT | RIGHT} [OUTER] JOIN tbl_ref join_condition

| tbl_ref NATURAL [{LEFT | RIGHT} [OUTER]] JOIN tbl_factor

join_condition:

ON conditional_expr

| USING (col_list)

每个tbl_name都可以有一个别名或者索引提示。此外，自MySQL 5.6.2起，SELECT语句支持使用针对分区表的PARTITION 子句，以表明需要从那里查询行的分区或子分区。也就是说，完整的表引用语法实际上看起来像下面这个样子：

tbl_name

[PARTITION (partition_name [, partition_name] ...)]

[[AS] alias_name]

[{USE | IGNORE | FORCE} {INDEX | KEY}

[FOR {JOIN | ORDER BY | GROUP BY}]

(index_list)]

如果需要在FROM子句里为表取别名，则可以使用tbl_name alias_name或者tbl_name AS alias_name语法。别名机制使我们能够在查询命令中的其他地方利用别名来引用表里的列。

还可以在FROM子句里使用子查询的结果集来充当表，但是必须把子查询用方括号括起来，并为它取一个别名，以便在SELECT语句中的其他地方引用这个“表”：

```
SELECT * FROM (SELECT 1) AS t;
```

子句USE INDEX、IGNORE INDEX和FORCE INDEX的作用是为优化器提供索引提示。在优化器不能正确选择在连接中使用哪个索引的情况下，这些子句可以帮助优化器做出正确的选择。USE INDEX子句告诉优化器只能从index_list列表中的索引里选择。IGNORE INDEX子句告诉优化器不要使用哪些索引。FORCE INDEX子句的含义和USE INDEX子句的相似，但是前者告诉优化器，与使用所列的索引相比，选用全表扫描的代价非常高。

index_list应该是由一个或多个以逗号分隔的索引名构成的一个列表，例外情况是：将index_list用于USE时，它可以为空，表示“不使用索引”。其中列出的每一个索引都应该是来自给定表的索引名，或者是代表给定表的PRIMARY KEY的PRIMARY关键字。

索引提示只用于选取行和连接表，不用于处理ORDER BY或GROUP BY子句。如果只想将索引提示应用于查询和连接，可以使用FOR JOIN子句。

每个tbl_name引用允许有多个索引提示。不过，对于同一个引用不能同时使用USE INDEX和FORCE INDEX。

对于FULLTEXT索引，索引提示没有任何作用，带有FOR JOIN修饰符或没有FOR修饰符的布尔模式搜索除外。

连接型 SELECT 语句将按照以下描述从给定的表选取行。实际返回到客户端的行的个数会受限于WHERE、HAVING或LIMIT子句。

如果只列出一个表，则SELECT会从该表里检索行。

如果 FROM 子句以逗号为间隔列举了多个表，则 SELECT 将返回这些表里的行的全排列组合。如果没有NO或USING子句，则使用JOIN、CROSS JOIN或CROSS JOIN与使用逗号等价。STRAIGHT_JOIN与此类似，但强制MySQL优化器必须按表在FROM子句中的先后顺序来连接表。如果你认为MySQL优化器做出的选择不是最佳的，可以使用此选项。

与逗号操作符不同，使用JOIN、CROSS JOIN 或 INNER JOIN 执行的连接可以用ON 或USING()子句来约束表间的匹配。匹配行是根据 ON conditional_expr 或 USING (col_list)子句中的条件来确定的。conditional_expr 是可以用在WHERE 子句中的表达式的形式。col_list 由一个或多个以逗号分隔的列名组成，每个列名必须是在两个连接的表中都出现的列的名字。

在从表连接的表中检索行时，LEFT JOIN将强制性地为左表里的每一个行生成一个行，哪怕右表里没有与之匹配的行也是如此。当没有匹配的时候，来自右表的列将被返回为 NULL值。表名后面的ON或USING()子句与JOIN、CROSS JOIN或INNER JOIN里的一样。LEFT OUTER JOIN和LEFT JOIN 是等效的。OJ 语法与此类似，这个语法是为了使 MySQL与ODBC 标准保持兼容而引入的。OJ 语法中的花括号不是元字符，它们必须原样出现在相应的SELECT语句中。

NATURAL LEFT JOIN等价于LEFT JOIN USING (col_list)，其中，col_list必须把同时出现在两个表里的列全都列举出来。

RIGHT JOIN类型与相应的LEFT JOIN类型相似，只是把表的角色对调了一下。

逗号连接的优先级低于其他类型的连接。混合使用逗号连接和其他类型的连接有可能导致“Unknown column”（未知列）错误。把逗号替换为INNER JOIN通常可以解决这样的问题。

MySQL将根据WHERE子句所给出的条件表达式从FROM子句所列举的表里选取行。不满足该条件表达式的行将不会出现在SELECT语句的查询结果里。结果集还要受到HAVING和LIMIT子句的进一步限制。列别名是不允许用在WHERE子句里的。

GROUP BY子句的语法和ORDER BY子句的相似。GROUP BY col_list子句将根据col_list所给定的列对结果集里的行进行归组。当在select_expr子句里使用了像COUNT()或MAX()这样的汇总函数时，就需要使用GROUP BY子句。ORDER BY col_list规定结果集应按指定列排序。在这两个子句中，可以使用列名、列的别名，或者列在select_expr表达式里的位置来引用列。列的位置序号是从 1 开始的无符号整数，但使用列的位置是非标准的，已经弃用。还

可以用表达式来分组，或根据表达式结果排序。例如，ORDER BY RAND() 是以随机顺序为行排序的。

在GROUP BY或ORDER BY子句里，可以在任何一个列名后面加上ASC或DESC关键字以表明该列应该按照递增或递减的顺序来排序。如果没有这两个关键字中的任何一个，则列将默认使用递增顺序。这两个关键字还可以用在GROUP BY子句里，因为MySQL中的GROUP BY子句不仅会对行进行分组，还会对分组结果进行排序。如果同时指定了GROUP BY和ORDER BY子句，则检索结果的排序顺序将由ORDER BY子句决定。要想禁用GROUP BY子句的这种隐式排序效果（这将减少不必要的排序开销），可以使用ORDER BY NULL子句。

在GROUP BY子句的末尾还可以给出WITH ROLLUP。它将在每个行分组的后面加上一个关于该分组的“小计”行，还将在输出报告的末尾加上一个“总计”行。

MySQL将根据HAVING子句所给出的次要条件表达式，对那些已经满足WHERE子句所给出的主要条件表达式（并且已根据GROUP BY子句分组）的行做进一步的筛选。不满足HAVING条件的行将不会出现在SELECT语句的查询结果里。HAVING子句非常适合因带有汇总函数而无法用在WHERE子句里的条件表达式。但是，如果某个条件表达式在WHERE子句里或HAVING子句里都是合法的，则应该把它放在WHERE子句里，因为只有WHERE子句里的条件表达式才会得到优化器的分析和优化。

LIMIT子句的用途是从结果集里进一步选取它的某个组成部分。这个子句可以带一个或者两个参数，这些参数必须是整数常数。LIMIT n将返回结果集里的前n个行。LIMIT m, n将跳过结果集里的前m个行，返回随后的n个行。

PROCEDURE子句用于指定一个存储过程。结果集里的数据在被返回到客户端之前，会先被发送到该存储过程。可选参数列表 param_list，是一个用逗号分隔的值列表，这些值将被传递到指定的代码模块里去。可以通过PROCEDURE ANALYSE()子句来获得该SELECT语句所选取的列里的数据的特性。更多信息请参考5.3节。

INTO格式指定了查询结果的目的地。在语句里指定INTO的另一个比较靠前的位置是紧跟在select_expr列表之后。如果使用了USE子句，则该语句就不能用作内嵌的SELECT。

SELECT语句的结果可以使用INTO outfile 'file_name'子句写入名为file_name的文件中。如果要指定这些值被写入时使用的字符集，则需要包含一个CHARACTER SET子句。如果该子句被省略，或者字符集为binary，则不会进行任何字符转换。file_options和line_options子句的语法与LOAD DATA语句的相应子句的相同。更多信息请参考LOAD DATA条目。

INTO DUMPFILE 'file_name'与INTO OUTFILE类似，但是它只会写入单行，并且在输出写出整个内容期间不会进行任何解释。也就是说，它写入的是不带分隔符、引号和结束符的原始值。如果想将BLOB数据（如图像或其他二进制数据）写到文件，那么它会有用。

INTO OUTFILE和INTO DUMPFILE 选项都是用来确定输出文件位置的，所使用的规则与不带LOCAL选项的LOAD DATA语句的相同。在执行此操作时，必须拥有FILE权限，输出文件必须尚不存在，并且该文件是由服务器在运行服务器的主机上使用全局访问模式创建的。该文件的所有权设置为运行服务器的那个账户。

如果INTO 关键字的后面是一个以逗号分隔的变量名列表，则SELECT 语句将把检索结果存入那些变量。每一个变量都可以是一个@var_name 形式的用户定义变量，也可以是包含这条SELECT语句的存储例程的某个参数或局部变量。这种查询必须选取且只能选取一个行，而每个输出列都必须有一个与之对应的变量。

FOR UPDATE和LOCK IN SHARE MODE子句将锁定SELECT语句所选取的行。这些锁定会强制保持到当前事务被提交或者回滚为止。这些锁定在多语句事务里非常有用。在支持行级锁定的表（如InnoDB表）上使用FOR UPDATE子句将在选取的行上施加一个独占性的写锁。使用LOCK IN SHARE MODE子句将在行上施加一个读锁，即允许其他客户端去读这些行、但不允许修改这些行。请注意，如果查询优化器发现没有用于检查行的索引，就必须扫描（并锁定）表中的所有行。

下面是一些演示SELECT语句各种用法的示例。更多示例请参考第1章和第2章。

选取某表的全部内容：

```
SELECT * FROM president;
```

选取全部内容，但是按名字排序：

```
SELECT * FROM president ORDER BY last_name, first_name;
```

把出生日期晚于'1900-01-01'的总统查出来：

```
SELECT * FROM president WHERE birth >= '1900-01-01';
```

执行相同的查询，但是按出生日期排序：

```
SELECT * FROM president WHERE birth >= '1900-01-01' ORDER BY birth;
```

查看member表里的行涉及哪些州：

```
SELECT DISTINCT state FROM member;
```

选取member表里的行并把它们的列写到一个文件里，各列值之间以逗号分隔：

```
SELECT * INTO OUTFILE '/tmp/member.txt'
```

```
FIELDS TERMINATED BY ',' FROM member;
```

把某次考试前5名学生选取出来：

```
SELECT * FROM score WHERE event_id = 9 ORDER BY score DESC  
LIMIT 5;
```

SET

```
SET assignment [, assignment] ...
```

```
assignment: var_name {= | :=} expr
```

SET 语句用来对系统变量、用户定义变量和存储程序参数（或存储程序局部变量）进行赋值。附录D提供了有关系统变量和用户定义变量的信息。E. 2. 2节描述了存储程序局部变量的声明语法。SET也可用于一些其他设置，本条目后面将会介绍。

其他以SET开头的语句（SET PASSWORD和SET TRANSACTION）将在本附录后面的其他条目里单独介绍。

当SET用于为变量赋值时，每个赋值语句中的var_name就是待赋值的变量，expr表达式指定的是要赋给变量的值。赋值运算符可以是=或者是:=。

SET语句可以用于对用户定义的变量（它们使用@var_name语法来指定）进行赋值：

```
SET @day = CURDATE(), @time = CURTIME();
```

SET还可以给系统变量赋值，其中许多是动态的，你可以在服务器仍在运行时修改它们。MySQL提供的动态系统变量分为两个级别：全局级系统变量的

有效范围是整个服务器，它们对所有的客户端都有影响；会话级系统变量（也叫做局部系统变量）的有效范围只局限于某个特定的客户端会话。对于那些同时存在于系统级、会话级的变量，每个新建的客户连接都会根据相应全局级变量的值得到一个初始的会话级变量设置。任何一个客户端都可以修改它自己的会话级变量，但只有那些拥有SUPER权限的用户才能修改全局级变量。

设置系统变量的语法有很多种。如果要设置全局级变量（以全局的sql_mode值为例），则可以用下面两种形式中的任何一种：

```
SET GLOBAL sql_mode = 'ANSI_QUOTES';
```

```
SET @@GLOBAL.sql_mode = 'ANSI_QUOTES';
```

如果要设置会话级变量，则需要用SESSION代替GLOBAL：

```
SET SESSION sql_mode = 'ANSI_QUOTES';
```

```
SET @@SESSION.sql_mode = 'ANSI_QUOTES';
```

也可以把LOCAL用作SESSION的同义词。

如果在SET语句里没有给出GLOBAL、SESSION或LOCAL关键字，则SET语句将默认修改会话级变量：

```
SET sql_mode = 'ANSI_QUOTES';
```

```
SET @@sql_mode = 'ANSI_QUOTES';
```

如果想要查看各个系统变量的值，则可以使用SHOW VARIABLES 语句。你也可以检查INFORMATION_SCHEMA库里的GLOBAL_VARIABLES表和SESSION_VARIABLES表。如果要显示单个变量值，则可以使用SELECT：

```
SELECT @@GLOBAL.sql_mode, @@SESSION.sql_mode,  
@@LOCAL.sql_mode;
```

关于使用系统变量的更多信息请参考12.3.1节。

下面是可以通过SET语句加以控制的一些其他设置。

```
SET CHARACTER SET {charset | DEFAULT}
```


把会话变量`character_set_client`和`character_set_results`设置为给定的字符集，把会话变量`character_set_connection`设置为`character_set_database`的值。这些变量影响着发送至服务器和从其发出的字符数据的转换。`charset`值不能为`ucs2`、`utf16`、`utf16le`或`utf32`。

`SET CHARACTER SET DEFAULT`可以把字符集恢复成默认设置。

`SET NAMES {charset [COLLATE collation] | DEFAULT}`

把会话变量`character_set_client`、`character_set_connection`和`character_set_results` 设置为给定的字符集，把 `collation_connection` 设置为 `character_set_connection`的默认排序规则。`COLLATE` 子句可以用于显式指定排序规则。`charset`和`collation`可以是带引号或不带引号的字符串。这些变量影响着发送至服务器和从其发出的字符数据的转换。`charset`值不能为`ucs2`、`utf16`、`utf16le`或`utf32`。

`SET NAMES DEFAULT`可以把字符集恢复成默认设置。

`SET PASSWORD`

`SET PASSWORD [FOR account] = PASSWORD('pass_val')`

`SET PASSWORD [FOR account] = OLD_PASSWORD('pass_val')`

`SET PASSWORD [FOR account] = 'encrypted_pass_val'`

`SET PASSWORD` 语句用来修改某个MySQL账户的密码。你总是可以修改自己的密码，但是以匿名用户身份连接服务器的情况例外。如果想要修改其他账户的密码，则必须对mysql数据库拥有UPDATE权限。如果启用了系统变量`read_only`，则必须拥有SUPER权限。

如果不带FOR子句，则这条语句将设置当前账户的密码。如果带有FOR子句，那么它会设置指定账户的密码，指定格式为'`user_name`'@'`host_name`'，如13.2.1.1节所述。

密码值'`pass_val`'应该用`PASSWORD()`函数进行标准化加密，或者使用`OLD_PASSWORD()`函数进行老式（MySQL 4.1 版之前）加密。如果没有使用这两个函数中的任何一个，则相应的'`encrypted_pass_val`'值必须是一个已经加过密的密码字符串。

`SET PASSWORD = PASSWORD('secret');`

```
SET PASSWORD FOR 'paul' = PASSWORD('secret');
```

```
SET PASSWORD FOR 'paul'@'localhost' = PASSWORD('secret');
```

```
SET PASSWORD FOR 'bill'@'%.bigcorp.com' = PASSWORD('old-sneep');
```

SET TRANSACTION

SET [GLOBAL | SESSION] TRANSACTION

trans_characteristic [, trans_characteristic] ...

trans_characteristic:

ISOLATION LEVEL level

| READ WRITE

| READ ONLY

此语句用来设置事务处理特性。它可以接受一个可选的作用范围指示器，以指明这些特性如何应用到事务里。

语句作用范围的设置方式如下。

如果指定了GLOBAL选项，那么它将设置全局特性（服务器范围的），并且它们会成为此后连接服务器的所有客户端的默认级别。

如果指定了 SESSION 选项，那么它将设置会话（客户端特有的）特性，并且它们会应用于当前会话里随后发生的事务。

如果这两个选项都没有给出，那么它将只为当前会话里的下一个事务设置特性。如果还有事务正在处理中，则不允许此执行设置。

使用GLOBAL选项时，必须拥有SUPER权限。任何客户端都可以更改自己的会话级特性或下一个事务的特性。

自MySQL 5.6.3起，此语句可以接受多个以逗号隔开的特性。在MySQL 5.6.3之前，此语句只允许指定ISOLATION LEVEL特性。

如果给出了ISOLATION LEVEL特性，则level所指明的事件级别应该是下列值中的一个。

READ UNCOMMITTED：某给定事务可以看到其他事务对行做的修改，不管它们是否已经提交。

READ COMMITTED：它只允许某个事务看到其他事务已经提交的行修改。

REPEATABLE READ：如果某个事务两次执行同一条SELECT语句，其结果是可重复的。也就是说，即使有其他事务在同时插入或修改行，这个事务所看到的结果也是一样的。

SERIALIZABLE：这个隔离级别与REPEATABLE READ很相似，但对事务的隔离更彻底，即对于某个事务正在选择的行，只有等到该事务完成才能被其他事务修改。

ISOLATION LEVEL特性可用于InnoDB存储引擎。默认的隔离级别是REPEATABLE READ。非事务型存储引擎没有隔离级别的概念。

有关事务隔离和隔离级别的进一步讨论请参考2.12.3节。

READ WRITE和READ ONLY特性决定了事务访问模式，即是否允许事务修改表。（不管是哪种访问模式，TEMPORARY 临时表总是可以被修改。）这些特性是互斥的，不能在同一条语句里使用。它们是在MySQL 5.6.5里引入的。

SHOW

SHOW {BINARY | MASTER} LOGS

SHOW BINLOG EVENTS

SHOW CHARACTER SET

SHOW COLLATION

SHOW COLUMNS

SHOW CREATE DATABASE

SHOW CREATE EVENT

SHOW CREATE {FUNCTION | PROCEDURE}

SHOW CREATE TABLE

SHOW CREATE TRIGGER

SHOW CREATE VIEW

SHOW DATABASES

SHOW ENGINE

SHOW ENGINES

SHOW ERRORS

SHOW EVENTS

SHOW {FUNCTION | PROCEDURE} STATUS

SHOW GRANTS

SHOW INDEX

SHOW MASTER STATUS

SHOW OPEN TABLES

SHOW PLUGINS

SHOW PRIVILEGES

SHOW PROCESSLIST

SHOW SLAVE HOSTS

SHOW SLAVE STATUS

SHOW STATUS

SHOW TABLE STATUS

SHOW TABLES

SHOW TRIGGERS

SHOW VARIABLES

SHOW WARNINGS

这些 SHOW 语句提供的是与数据库及其里面的各种对象（如表或存储程序）有关的信息，或者是与服务器操作有关的信息。有几种 SHOW语句允许使用一个可选的FROM db_name子句，从而可以指定数据

库，以了解与之相关的信息。如果没有给出这个子句，则使用默认数据库。对于所有支持使用FROM来指定表名或数据库名的语句，也可以使用IN来代替。

有几种形式的SHOW 语句允许使用可选的LIKE 'pattern'子句，用于限制输出内容，使其中的值都能与给定的模式相匹配。'pattern'可被理解为一个SQL模式，并且可以包含通配符“%”或“_”。

INFORMATION_SCHEMA库为我们提供了另外一种获得数据库元数据的方法，许多通过SHOW语句获得的信息现在还可以从INFORMATION_SCHEMA库的各个表里查到。在此基础上，那些支持LIKE 'pattern'子句的SHOW语句，还可以改用一条WHERE子句对它将显示的行进行筛选。更多相关信息请参考2.7节。

SHOW BINARY LOGS

SHOW {BINARY | MASTER} LOGS

此语句可以显示当前二进制日志文件的名称和大小。在主复制服务器上，可用它在每个从服务器执行PURGE BINARY LOGS之前、运行SHOW SLAVE STATUS之后，确定主服务器二进制日志文件当前所在的位置。

此语句要求具有SUPER权限；自MySQL 5.5.25起，拥有SUPER权限或REPLICATION CLIENT权限也可以。

SHOW BINLOG EVENTS

SHOW BINLOG EVENTS [IN 'file_name'] [FROM position]

[LIMIT [skip_count,] show_count]

此语句用于显示二进制日志里的事件。它要求拥有REPLICATION SLAVE权限。如果要显示从服务器中继日志里的事件，则需要使用SHOW RELAYLOG EVENTS。

各个事件与SQL语句大致上是一致。如果省略了文件名，则会使用第一个二进制日志文件。如果省略了的位置，则此语句会从文件的开

头处开始读取。如果给出了LIMIT子句，则可限定要显示的行数。其语法与SELECT的LIMIT子句相同。

SHOW BINLOG EVENTS的输出内容包含以下几列。

Log_name：二进制日志的文件名。

Pos：事件在日志文件中的位置。

Event_type：事件的类型，如Query，它表示的是将被执行的语句。

Server_id：记录事件的服务器的ID编号。

End_log_pos：在日志文件里事件之后的下一个字节的位置。

Info：事件信息，如Query事件的语句文本。

SHOW CHARACTER SET

SHOW CHARACTER SET [LIKE 'pattern' | WHERE where_expr]

显示服务器当前支持的字符集。如果包含 LIKE 子句，则只显示其名称与给定模式相匹配的字符集的信息。如果包含一条WHERE子句，则只输出那些能够满足给定表达式的行。

SHOW CHARACTER SET语句的输出包含以下几列。

Charset：简写的字符集名。这些简写的名字可以直接用在SQL语句里。

Description：一个较长的描述性字符集名。

Default collation：字符集默认排序规则的名称。

Maxlen：该字符集里“最宽”字符的长度，以字节单位。对于多字节字符集，这个值大于1。对于单字节字符集，所有字符都只占用一个字节，所以这个值是1。

SHOW COLLATION

SHOW COLLATION [LIKE 'pattern' | WHERE where_expr]

显示每一种字符集的所有可用排序规则。如果包含一条 LIKE 子句，则只显示其名称与给定模式相匹配的排序规则的信息。如果包含一条WHERE子句，则只输出那些能够满足给定表达式的行。

SHOW COLLATION语句的输出包含以下几列。

Collation: 排序规则的名字。

Charset: 与排序规则相关联的字符集的名字。

Id: 排序规则的ID编号。

Default: 如果这种排序规则是相关字符集的默认排序规则，这里将是 Yes；否则，为空白。

Compiled: 如果这种排序规则已被编译到服务器里，这里将是 Yes；否则，为空白。

Sortlen: 一个与内存耗用量有关的开销因数，在按照这种排序规则对值排序时，服务器将根据这个因数为其内部的字符串转换操作分配内存。

SHOW COLUMNS

SHOW [FULL] COLUMNS {FROM | IN} tbl_name

[{FROM | IN} db_name] [LIKE 'pattern' | WHERE where_expr]

显示给定表或视图的列。这条语句的输出只包含那些你拥有权限的列。SHOW FIELDS 是SHOW COLUMNS的同义词。如果包含FULL关键字，这条语句将显示Collation、Privilege和Comment输出字段。如果包含一条 LIKE 子句，则只显示其名称与给定模式相匹配的列的信息。如果包含一条WHERE子句，则只输出那些能够满足给定表达式的行。

如果要指定包含给定表的数据库的名字，可以使用一条 FROM db_name 子句或以db_name.tbl_name格式写出表名：

```
SHOW COLUMNS FROM president;
```

```
SHOW COLUMNS FROM president FROM sampdb;
```

```
SHOW COLUMNS FROM sampdb.president;
```

SHOW COLUMNS的输出可以提供下面这些与表中每个列有关的信息。Collation、Privileges和Comment的值只有在给出了关键字FULL的时候才会被显示。

Field：列名。

Type：列的数据类型。在类型名的后面可能还会列出一些相关的类型属性。

Collation：非二进制串列的排序规则名称；如果是其他类型的列，这里将是 NULL。排序规则名称里隐含着字符集名。

Null：如果列允许包含NULL值，则为Yes；否则，为NO。

Key：列是否被索引。PRI表示的是所有PRIMARY KEY列，UNI表示的是UNIQUE索引的第一列，MUL表示的是（非唯一性）索引（各个值允许在列里多次出现）的第一列。如果该值为空白，则此列未被索引；或者已索引过，但是与其他的标志符不相符。

Default：该列的默认值。当值为NULL时，表示的是默认值NULL，或者该列的定义里没有包含DEFAULT子句。

Extra：提取与该列有关的信息。对于带 AUTO_INCREMENT 属性的列，该值为auto_increment；对于带有 ON UPDATE CURRENT_TIMESTAMP 属性的列，该值为 on update CURRENT_TIMESTAMP；其他情况下为空。

Privileges：你针对该列所拥有的权限。

Comment：定义该列时在COMMENT子句里给出的注释信息。

SHOW CREATE

SHOW CREATE DATABASE [IF NOT EXISTS] db_name

SHOW CREATE EVENT event_name

SHOW CREATE FUNCTION func_name

SHOW CREATE PROCEDURE proc_name

SHOW CREATE TABLE tbl_name

SHOW CREATE TRIGGER trigger_name

SHOW CREATE VIEW view_name

SHOW CREATE obj_type语句将显示创建给定对象的CREATE obj_type语句。只有对某个对象拥有适当的权限，才能显示其 CREATE 语句。这条语句的几种形式还可以显示一些关于给定对象的其他信息，如当初创建该对象时的sql_mode值。

对SHOW CREATE DATABASE语句而言，如果它包含IF NOT EXIST子句，那么它输出的CREATE DATABASE语句也将包含该子句。

SHOW DATABASES

SHOW DATABASES [LIKE 'pattern' | WHERE where_expr]

显示服务器主机上当前可以使用的数据库。如果包含 LIKE 子句，则只显示其名称与给定模式相匹配的数据库的信息。如果包含 WHERE子句，则只输出那些能够满足给定表达式的行。

如果没有SHOW DATABASES权限，那么你能只能看到你拥有的访问权限范围内的数据库。如果在启动服务器时使用了--skip-show-database选项，并且拥有SHOW DATABASES权限，那么你能看到所有的数据库；否则，你看不到任何数据库。

SHOW ENGINE

SHOW ENGINE engine_name info_type

这条语句会显示与存储引擎有关的信息。它要求拥有PROCESS权限。

SHOW ENGINE INNODB MUTEX

显示与InnoDB互斥机制有关的信息。

SHOW ENGINE INNODB STATUS

显示与InnoDB存储引擎内部操作状态有关的信息。

SHOW ENGINE PERFORMANCE_SCHEMA STATUS

显示与Performance Schema（它被实现为一个存储引擎）的内部操作有关的信息。

SHOW ENGINES

SHOW [STORAGE] ENGINES

显示服务器当前支持的各个存储引擎。对于每一个存储引擎，输出内容包含下面几列，用以表明支持级别、提供对存储引擎特性的简要描述。

Engine：存储引擎名称（如InnoDB、MyISAM等）。

Support：存储引擎的支持级别。YES表示支持；NO表示不支持；DISABLED表示支持，但是在运行时会被禁用；DEFAULT表示存储引擎是默认存储引擎。默认存储引擎总是可用。

Comment：关于存储引擎的描述性文字。

Transactions：引擎是否支持事务处理。

XA：引擎是否支持分布式事务处理。

Savepoints: 引擎是否支持部分事务回滚。

SHOW ERRORS

SHOW ERRORS [LIMIT [skip_count,] show_count]

SHOW COUNT(*) ERRORS

SHOW ERRORS语句类似于SHOW WARNINGS语句，但是只会显示问题很严重的出错信息。SHOW COUNT(*) ERRORS 语句类似于 SHOW COUNT(*) WARNINGS 语句，但显示的是系统变量error_count的值而不是warning_count的值。更多信息请参考SHOW WARNINGS的描述。

SHOW EVENTS

SHOW EVENTS [{FROM | IN} db_name]

[LIKE 'pattern' | WHERE where_expr]

这条语句将显示关于默认数据库里的事件的信息，如果带有 FROM 子句，则显示关于给定数据库里的事件的信息。它要求对包含这些事件的那个数据库拥有ALTER权限。如果包含LIKE子句，则只显示其名称与给定模式相匹配的事件的信息。如果包含WHERE子句，则只输出那些能够满足给定表达式的行。

SHOW FUNCTION STATUS、SHOW PROCEDURE STATUS

SHOW {FUNCTION | PROCEDURE} STATUS

[LIKE 'pattern' | WHERE where_expr]

这些语句将显示与默认数据库里的存储函数或存储过程有关的描述性信息。如果包含 LIKE 子句，则只显示其名称与给定模式相匹配的存储例程的信息。如果包含WHERE子句，则只输出那些能够满足给定表达式的行。

SHOW GRANTS

```
SHOW GRANTS [FOR account]
```

此语句可以显示与给定账户有关的权限授予方面的信息。账户的指定格式为 'user_name'@'host_name'，如13.2.1.1节所述。

```
SHOW GRANTS FOR 'root'@'localhost';
```

```
SHOW GRANTS FOR "@'cobra.example.com';
```

还可以使用下列语句中的任何一个去查看当前使用的MySQL用户账户拥有的权限：

```
SHOW GRANTS FOR CURRENT_USER();
```

```
SHOW GRANTS FOR CURRENT_USER;
```

```
SHOW GRANTS;
```

当使用SHOW GRANTS语句查看当前用户的权限时，如果存储过程在SQL SECURITY DEFINER上下文里执行，那么它输出的将是与该存储过程的定义者有关的信息，而不是与其调用者有关的信息。

```
SHOW INDEX
```

```
SHOW {INDEX | INDEXES | KEYS} {FROM | IN} tbl_name
```

```
[{FROM | IN} db_name] [WHERE where_expr]
```

显示关于表的索引的信息。必须对表里列拥有相关的权限。

如果想要指定包含给定表的数据库的名字，可以使用FROM db_name 子句，或者以db_name.tbl_name的格式写出表名：

```
SHOW INDEX FROM score;
```

```
SHOW INDEX FROM score FROM sampdb;
```

```
SHOW INDEX FROM sampdb.score;
```

如果包含WHERE子句，则只输出那些能够满足给定表达式的行。

SHOW INDEX语句的输出包含以下几列。

Table：包含该索引的表的名字。

Non_unique：1表示该索引允许包含重复的值；0表示不允许。

Key_name：索引名。

Seq_in_index：列在索引中的序号。索引列的序号从1开始编号。

Column_name：应用当前输出行的索引中列的名字。

Collation：列在索引中的排序顺序。它的可取值是A（升序）、D（降序）或NULL（不排序）。MySQL目前还不支持按降序排序的键。

Cardinality：索引里唯一值的近似数量。当myisamchk在运行时带有一analyze选项，它便能够更新MyISAM表里的这个值。ANALYZE TABLE语句能更新InnoDB表和MyISAM表里的这个值。OPTIMIZE TABLE语句能更新MyISAM里的这个值。

Sub_part：如果只对列的前缀进行索引，那么这个输出列将给出该前缀以字节计数的长度。如果是对整个列进行索引，那么这个输出列里的值将是NULL。

Packed：键的压缩方式；如果值为NULL，则表示未压缩。

Null：如果列允许包含NULL值，则为YES；否则，为空白。

Index_type：用于索引列的算法，如BTREE、FULLTEXT或HASH。

Comment：保留供该索引的内部注释之用。

Index_comment：包含在索引定义里的注释。

SHOW MASTER STATUS

SHOW MASTER STATUS

这条语句需要在复制主服务器上使用。输出内容包含下面几列，用于显示与二进制日志文件的状态有关的信息。

File: 二进制日志的文件名。

Position: 服务器在该文件里的当前写位置。

Binlog_Do_DB: 一个以逗号分隔的数据库列表，这些数据库是通过--binlog-do-db选项被显式地复制到二进制日志的；如果没有数据库，则列表为空。

Binlog_Ignore_DB: 一个以逗号分隔的数据库列表，这些数据库是通过--binlog-ignore-db选项被显式地排除在二进制日志之外的；如果没有数据库，则列表为空。

SHOW OPEN TABLES

SHOW OPEN TABLES [{FROM | IN} db_name]

[LIKE 'pattern' | WHERE where_expr]

显示注册在表缓存里、且处于打开状态的非TEMPORARY表的列表（要求对这些表拥有一定的权限）。如果包含LIKE子句，则只显示其名称与给定模式相匹配的表的信息。如果包含WHERE子句，则只输出那些能够满足给定表达式的行。

SHOW OPEN TABLES语句的输出包含以下几列。

Database: 包含给定表的数据库的名字。

Table: 表名。

In_use: 表目前被使用的次数。

Name_locked: 这个表当前是否有一个名字锁，名字锁是在不访问其内容的情况下使用某个表（例如，对于RENAME TABLE语句）的必要条件。

SHOW PLUGINS

SHOW PLUGINS

显示与已安装插件有关的信息。

SHOW PLUGINS语句的输出包含以下几列。

Name：插件名。

Status：插件状态，其中包括ACTIVE、INACTIVE、DISABLED和DELETED。

Type：插件类型，如STORAGE ENGINE。

Library：插件库对象文件；如果插件是内建的，则为NULL。

License：插件的许可类型。

SHOW PRIVILEGES

SHOW PRIVILEGES

显示可被授予的各种权限，以及与每种权限的用途有关的信息。

SHOW PROCESSLIST

SHOW [FULL] PROCESSLIST

显示关于当前正在执行的服务器活动的信息。如果拥有PROCESS权限，那么这条语句将显示所有的信息；否则，它将只显示与你自己的活动有关的信息。

SHOW PROCESSLIST语句的输出包含以下几列。

Id：客户端的进程ID号。

User：与进程相关联的那个账户的用户名。

Host：产生客户端连接的那台主机。

db: 进程的默认数据库；如果没有，则为NULL。Command: 正在执行的那条命令的类型。

Time: 过程正在执行的语句所花费的时间，单位为秒。

State: 在处理SQL语句时，与MySQL正忙着的事情有关的信息。

Info: 正被执行的那条语句的前面100个字符；如果未给出关键字FULL，则会显示整条语句的内容。

SHOW RELAYLOG EVENTS

SHOW RELAYLOG EVENTS [IN 'file_name'] [FROM position]

[LIMIT [skip_count,] show_count]

这条语句类似于SHOW BINLOG EVENTS，但是它显示的是从服务器的中继日志内容。

SHOW SLAVE HOSTS

SHOW SLAVE HOSTS

这条语句需要在复制主服务器上使用，用于显示与当前注册的各个从服务器有关的信息。它要求拥有REPLICATION SLAVE权限。

SHOW SLAVE HOSTS语句的输出包含以下几列。

Server_id: 从服务器ID。

Host: 从服务器主机。

User: 从服务器用来建立当前连接的账户的用户名。

Password: 从服务器用来建立当前连接的账户的密码。

Port: 从服务器监听的端口。除非在启动从服务器时设置了系统变量 report_port，否则Port列的值为0（在MySQL 5.5.23之前为3306）。

Master_id: 主服务器ID。

Slave_UUID: 从服务器的UUID值。此列是在MySQL 5.6.0里引入的。

SHOW SLAVE STATUS

SHOW SLAVE STATUS

这条语句需要在从服务器上使用，用于显示与从服务器的复制工作状态有关的信息。其中许多值都可以映射到使用CHANGE MASTER 语句指定的各个参数。SHOW SLAVE STATUS 语句的输出包含下面几列。

Slave_IO_State: 从服务器I/O线程的状态。它与SHOW PROCESSLIST针对该I/O线程显示的State值是一样的。

Master_Host: 从服务器连接到的那台主服务器主机。

Master_User: 用来连接主服务器的账户的用户名。

Master_Port: 用来连接主服务器的端口号。

Connect_Retry: 试图连接主服务器的各次尝试之间的等待秒数。

Master_Log_File: 主服务器当前使用的二进制日志文件的名称。

Read_Master_Log_Pos: 从服务器的I/O线程在主服务器的二进制日志里的当前读位置。

Relay_Log_File: 当前中继日志文件的名称。

Relay_Log_Pos: 从服务器SQL线程在当前中继日志中的读位置。

Relay_Master_Log_File: SQL线程最近执行的事件所在的主服务器上的二进制日志文件的名称。

Slave_IO_Running: 从服务器的I/O线程是否正在运行。

Slave_SQL_Running: 从服务器的SQL线程是否正在运行。

Replicate_Do_DB、Replicate_Ignore_DB、Replicate_Do_Table、Replicate_Ignore_Table、Replicate_Wild_Do_Table、Replicate_Wild_Ignore_Table：一个以逗号分隔的数据库或表列表，它们可通过--replicate-do-db、--replicate-ignore-db、--replicate-do-table、--replicate-ignore-table、--replicate-wild-do-table和--replicate-wild-ignore-table选项，分别显式地包含在复制过程中或被排除在外；如果没有内容，则为空白。

Last_Errno、Last_Error：这些列是Last_SQL_Errno和Last_SQL_Error的别名。

Skip_Counter：当前从服务器应该跳过的那些来自主服务器的日志事件的数量。（通过设置全局系统变量sql_slave_skip_counter，可以使从服务器跳过事件。）

Exec_Master_Log_Pos：从服务器的SQL线程在主服务器的二进制日志里的当前执行位置。

Relay_Log_Space：中继日志文件的总长度。

Until_Condition：START SLAVE 语句的UNTIL 子句所给出的条件，该条件决定着SQL线程应该在何时停止读取和执行事件。

None：不指定UNTIL子句。

Master：从服务器将不断读取，一直到其SQL线程到达主服务器二进制日志的指定位置为止。

Relay：从服务器将不断读取，一直到其SQL线程到达中继日志的指定位置为止。

如果 Until_Condition 列的值为 Master 或 Relay，那么 Until_Log_File和Until_Log_Pos列的值表明的是文件名和SQL线程将在那里停止执行的位置。

Until_Log_File、Until_Log_Pos：请参考Until_Condition的描述。

Master_SSL_Allowed: 是否可以使用SSL连接主服务器。Yes 表示可以使用SSL连接, No表示不能使用SSL连接, Ignored表示允许使用SSL连接但从服务器在编译时没有启用SSL支持。

Master_SSL_CA_File、Master_SSL_CA_Path、Master_SSL_Cert、Master_SSL_Cipher、Master_SSL_Key、Master_SSL_Verify_Server_Cert、Master_SSL_Crl、Master_SSL_Crtpath: 连接主服务器时使用的SSL 参数。Master_SSL_Crl和Master_SSL_Crtpath是在MySQL 5.6.3里引入的。

Seconds_Behind_Master: 它指的是当前时间与从服务器的SQL 线程最近一次执行的主服务器事件所记录的那个时间戳之间的差值(单位为秒)。如果SQL线程已经赶上了I/O线程并处于闲等状态, 则这个值为零; 如果还没有执行过任何事件或者从服务器的参数刚被CHANGE MASTER或RESET SLAVE语句修改过, 则这个值为NULL。

Last_IO_Errno、Last_IO_Error: I/O 线程最近一次错误的编号和信息。如果没有发生任何错误, 则它们的值为0和空串。服务器会把非空信息值写入其错误日志。

Last_SQL_Errno、Last_SQL_Error: 类似于 Last_IO_Errno和Last_IO_Error, 但是它们适用于SQL线程。

Replicate_Ignore_Server_Ids: 需要忽略事件的那些服务器的ID。

Master_Server_Id: 主服务器的server_id值。

Master_UUID: 主服务器的server_uuid值。此列是在MySQL 5.6.1里引入的。

Master_Info_File: 从服务器的master.info文件的名称。此列是在MySQL 5.6.1里引入的。

SQL_Delay: 复制延迟时间(单位为秒)。此列是在MySQL 5.6.1里引入的。

SQL_Remaining_Delay: 从服务器在执行下一个事件之前仍然要等待的延迟秒数；如果不等待，则为NULL。此列是在MySQL 5.6.1里引入的。

Slave_SQL_Running_State: 从服务器SQL线程的状态。它与SHOW PROCESSLIST显示的SQL线程的State值相同。此列是在MySQL 5.6.1里引入的。

Master_Retry_Count: 从服务器在放弃连接主服务器之前的尝试次数。此列是在MySQL 5.6.1里引入的。

Master_Bind: 从服务器绑定的网络接口。此列是在MySQL 5.6.1里引入的。

Last_IO_Error_Timestamp: 最近一次I/O线程出错的时间。此列是在MySQL 5.6.1里引入的。在MySQL 5.6.2之前，时间戳是Last_IO_Error值的一部分。

Last_SQL_Error_Timestamp: 最近一次SQL线程出错的时间。此列是在MySQL 5.6.2里引入的。在MySQL 5.6.2之前，时间戳是Last_SQL_Error值的一部分。

SHOW STATUS

SHOW [GLOBAL | SESSION] STATUS

[LIKE 'pattern' | WHERE where_expr]

显示服务器的状态变量和它们的值。这些变量提供的是与服务器的运行状态有关的信息。如果包含LIKE子句，则只显示其名称与给定模式相匹配的变量的信息。如果包含WHERE子句，则只输出那些能够满足给定表达式的行。

MySQL服务器可以显示全局级（服务器范围的）或会话级（客户端特定的）的状态变量值。它们分别代表所有客户端的总和和当前客户端的值。默认情况下，SHOW语句将显示给定变量在会话级别的值。如果要显式查看全局值或者会话值，可以指定一个范围指示符：

```
SHOW GLOBAL STATUS;
```

```
SHOW SESSION STATUS;
```

如果某个变量只有全局值，那么对于 GLOBAL和SESSION，你将得到相同的值。LOCAL 是SESSION的同义词。

也可以查询INFORMATION_SCHEMA库里的GLOBAL_STATUS表和SESSION_STATUS表，以获得状态变量的信息。

与如何使用状态变量有关的更多信息请参考12.3.2节。与各个变量有关的描述请参考附录D。

```
SHOW TABLE STATUS
```

```
SHOW TABLE STATUS [{FROM | IN} db_name]
```

```
[LIKE 'pattern' | WHERE where_expr]
```

显示与数据库里的非TEMPORARY表有关的描述性信息（要求对这些表拥有一定的权限）。此语句也可以显示数据库里的视图，但是所有列都为NULL（除了Name和Comment，它们分别为视图名和view）。如果包含LIKE子句，则只显示其名称与给定模式相匹配的表的信息。如果包含WHERE子句，则只输出那些能够满足给定表达式的行。

SHOW TABLE STATUS语句的输出包含下面几列。

Name：表名。

Engine：存储引擎（如InnoDB、MyISAM等）。

Version：表的.frm文件的版本号。

Row_format：行存储格式。它来自为CREATE TABLE指定的表选项ROW_FORMAT。

Rows：表里的行数。对于某些存储引擎，如InnoDB，这个数字只是一个近似值。

Avg_row_length: 表的各个行的平均字节数。

Data_length: 表的数据文件的实际字节长度。

Max_data_length: 表的数据文件所能增长到的最大长度。

Index_length: 索引文件的实际字节长度。

Data_free: 数据文件中尚未使用的字节数。如果这个值很大, 可以执行OPTIMIZE TABLE语句来优化它。对于InnoDB表, 此列表示的是InnoDB表空间(表存在在其中)里的空闲空间数量。(此表可能在系统表空间里, 也可能拥有其自己的表空间。)

Auto_increment: 表的AUTO_INCREMENT列将生成的下一个值。

Create_time: 表创建时间。

Update_time: 最近一次表修改时间; 如果存储引擎不维护这个值, 则它为NULL。

Check_time: 最近一次表检查或修复时间; 如果存储引擎不维护这个值, 或者如果表从未被检查或修复过, 则它为NULL。

Collation: 表的排序规则。排序规则的名字隐含了所用字符集的名字。

Checksum: 表的校验和; 如果此前未被计算过, 则它为NULL。

Create_options: 在当初创建表的CREATE TABLE语句或随后的ALTER TABLE语句里作为table_option值而给出的额外选项。

Comment: 在创建表时给出的任何注释文本。

SHOW TABLES

SHOW [FULL] TABLES [{FROM | IN} db_name]

[LIKE 'pattern' | WHERE where_expr]

显示数据库里的非TEMPORARY表的名字（要求对这些表拥有一定的权限）。此语句也会显示视图名字。可以指定关键字FULL，以显示一系列信息，用于表明每个行引用的是表（BASE_TABLE），还是视图（VIEW）。如果包含 LIKE 子句，则只显示其名称与给定模式相匹配的表的信息。如果包含WHERE子句，则只输出那些能够满足给定表达式的行。

```
SHOW TRIGGERS
```

```
SHOW TRIGGERS [FROM db_name]
```

```
[LIKE 'pattern' | WHERE where_expr]
```

这条语句可以显示与默认数据库里的各个触发器有关的信息；如果带有 FROM 子句，则显示给定数据库里的触发器信息。输出内容只会包含对其拥有 TRIGGER 权限的那些表的信息。如果包含LIKE 子句，则只显示其名称与给定模式匹配的触发器的信息。如果包含WHERE子句，则只输出那些能够满足给定表达式的行。

```
SHOW VARIABLES
```

```
SHOW [GLOBAL | SESSION] VARIABLES
```

```
[LIKE 'pattern' | WHERE where_expr]
```

显示系统变量及其值。这些变量提供的是各种与服务器配置和功能有关的信息。如果包含LIKE子句，则只显示其名称与给定模式相匹配的变量的信息。如果包含WHERE子句，则只输出那些能够满足给定表达式的行。

服务器可以显示全局级（服务器范围的）或会话级（客户端特定的）的系统变量的值。默认情况下，SHOW VARIABLES会把各个会话变量显示出来。（在MySQL 5.5.3之前，它显示的是会话值；如果没有会话值，则显示全局值。）如果要显式查看全局值或会话值，可以指定一个范围指示符：

```
SHOW GLOBAL VARIABLES;
```



```
SHOW SESSION VARIABLES;
```

LOCAL是SESSION的同义词。也可以使用SELECT语句来检索某个动态变量的值：

```
SELECT @@GLOBAL.sql_mode, @@SESSION.sql_mode,  
@@LOCAL.sql_mode;
```

使用SELECT语句的好处是可以方便地在特定的上下文里处理查询结果。

你也可以检查INFORMATION_SCHEMA库里的GLOBAL_VARIABLES表和SESSION_VARIABLES表，以获得系统变量的信息。

关于如何使用系统变量的更多信息请参考12.3.1节。与各个变量有关的描述请参考附录D。

```
SHOW WARNINGS
```

```
SHOW WARNINGS [LIMIT [skip_count,] show_count]
```

```
SHOW COUNT(*) WARNINGS
```

SHOW WARNINGS语句将显示最近一条语句的出错信息、警告信息或其他提示性信息。如果最近一条语句执行成功，SHOW WARNINGS语句将返回一个空集。

SHOW COUNT (*) WARNINGS显示的是保存在系统变量warning_count里的信息计数值。（还有一个与此相关的error_count变量，只统计出错信息个数。）warning_count变量的值有可能大于SHOW WARNINGS语句所显示的信息的个数。系统变量max_error_count限制的是可以被保存起来供SHOW WARNINGS语句显示的信息个数，但是warning_count统计的是所有信息，而不管它们是否被保存。

如果给出了LIMIT子句，则可限定要显示的行数。其语法与SELECT的LIMIT子句相同。

```
START SLAVE
```

```
START SLAVE [thread_option [, thread_option] ...]
```

```
START SLAVE [SQL_THREAD] UNTIL
```

```
MASTER_LOG_FILE = 'file_name', MASTER_LOG_POS = position
```

```
START SLAVE [SQL_THREAD] UNTIL
```

```
RELAY_LOG_FILE = 'file_name', RELAY_LOG_POS = position
```

```
START SLAVE [USER = 'user_name'] [PASSWORD = 'pass_val']
```

```
[DEFAULT_AUTH = 'auth_plugin'] [PLUGIN_DIR = 'dir_name']
```

这条语句，和STOP SLAVE语句，控制着从服务器上复制线程的操作。不带任何选项时，START SLAVE 将同时启动 I/O 线程和SQL 线程，而 STOP SLAVE 将同时中止这两个线程。可以指定thread_option 值，以表明需要启动或停止哪些线程。

IO_THREAD：启动或停止I/O线程，该线程负责从主服务器读取事件并把它们写入中继日志。

SQL_THREAD：启动或停止SQL线程，该线程负责从中继日志读出事件并执行。

如果没有指定任何线程或SQL_THREAD选项，则可以使用UNTIL子句。根据这个子句所给出的日志文件和位置选项，从服务器在启动后将一直运行到它的SQL线程到达主服务器的二进制日志或是从服务器的中继日志里的指定位置为止。如果 SQL 线程已经在运行，则从服务器将忽略 UNTIL子句并生成一条警告信息。如果该子句包含SQL_THREAD 选项，则从服务器将只启动SQL线程；否则，同时启动I/O和SQL两个线程。

自MySQL 5.6.4起，START SLAVE可以指定身份认证插件所需要的参数。这个语法允许使用下面这些选项；必须按下面的顺序来指定它们。

USER：用户名。不能给定为NULL或空串；如果存在PASSWORD选项，则必须指定它。

PASSWORD：用户密码。

DEFAULT_AUTH：身份认证插件名。如果省略，则默认为MySQL本地密码认证方式。

PLUGIN_DIR：插件所在的目录。

START TRANSACTION

START TRANSACTION

[trans_characteristic [, trans_characteristic] ...]

trans_characteristic:

WITH CONSISTENT SNAPSHOT

| READ WRITE

| READ ONLY

通过禁用自动提交模式的方式开始一个事务，一直到遇见下一条COMMIT或ROLLBACK语句为止。在自动提交模式被禁用的情况下，要执行的语句将作为一个整体被提交或回滚。

在事务被提交或回滚之后，自动提交模式将恢复到它在START TRANSACTION语句开始执行之前的状态。如果要显式地设置自动提交模式，可以设置系统变量autocommit（如2.12.1节所述）。

如果带有 WITH CONSISTENT SNAPSHOT 特性，会使事务以一致性读取方式开始执行。对于InnoDB，这个子句不会更改当前的隔离级别，因此只有隔离级别为 REPEATABLE READ 或SERIALIZABLE时，它才会发挥作用。

READ WRITE和READ ONLY特性决定了事务的访问模式，即是否允许事务修改表。（不管是哪种访问模式，TEMPORARY 临时表总是可以被修改。）这些特性是互斥的，并且不能在同一条语句里使用。它们是在MySQL 5.6.5里引入的。

START TRANSACTION语句会隐式释放当前客户端此前通过LOCK TABLE语句获得、但尚未释放的所有表锁。在某个事务进行的过程中执行START TRANSACTION语句将导致该事务被隐式地提交。

STOP SLAVE

STOP SLAVE [thread_option [, thread_option] ...]

这条语句，和START SLAVE语句，控制着从服务器上复制线程的操作。关于thread_option的各个允许值的详细信息请参考START SLAVE条目。

TRUNCATE TABLE

TRUNCATE [TABLE] tbl_name

TRUNCATE TABLE将通过删除并重建表的方式快速清空表的内容。此操作比单独删除每个行的方式快。要求拥有表的DROP权限。

InnoDB不允许使用这条语句；如果有外键引用了这个表，该语句会返回一个错误。

这条语句不是事务安全的。如果在事务里执行TRUNCATE TABLE语句，或者在你拥有显式表锁的时候执行该语句，会出现错误。

UNINSTALL PLUGIN

UNINSTALL PLUGIN plugin_name

卸载指定的插件，并将它从 mysql.plugin 表里注销掉（如果它列在其中的话），以便服务器在后续的重启中不会再加载它。INSTALL PLUGIN要求对mysql.plugin表拥有DELETE权限。更多相关信息请参考12.4节。

UNION

select_stmt

UNION [DISTINCT | ALL] select_stmt

[UNION [DISTINCT | ALL] select_stmt] ...

[ORDER BY col_list] [LIMIT [skip_count,] show_count]

UNION用来把多条SELECT语句的检索结果合并在一起。每条SELECT语句必须在它自己的结果集里生成出同样个数的输出列。在最终结果里，各输出列的名字将根据第一条 SELECT 语句里的列名来确定。各输出列的数据类型将根据所选表相应列中的全体数据值来确定。

可以在UNION关键字的后面加上一个DISTINCT关键字，以剔除重复的输出行；也可以加上一个关键字ALL，以保留重复的行，并返回所有选取的行。如果关键字DISTINCT和ALL都没有给出，则默认行为是剔除重复的行。DISTINCT联合操作的优先级比它左侧的ALL联合操作的更高：

```
mysql> SELECT 1 UNION ALL SELECT 2 UNION ALL SELECT 1;
```

```
+---+
```

```
| 1 |
```

```
+---+
```

```
| 1 |
```

```
| 2 |
```

```
| 1 |
```

```
+---+
```

```
mysql> SELECT 1 UNION ALL SELECT 2 UNION SELECT 1;
```

```
+---+
```

```
| 1 |
```

```
+---+
```

```
| 1 |
```

```
| 2 |
```

```
+---+
```

如果想要让各个的SELECT语句都带上ORDER BY和LIMIT子句，则必须把每一个SELECT语句都用圆括号括起来。（只有存在LIMIT子句时，单个SELECT语句里的ORDER BY子句才可用于确定 LIMIT 子句会应用于哪些行。它不会对最终 UNION 结果里的行顺序产生影响。）如果需要把ORDER BY或LIMIT子句当作一个整体应用于UNION，则需要将每一条SELECT语句用圆括号括起来，并把ORDER BY或LIMIT添加到最后那个右括号的后面。在这种情况下，在ORDER BY子句中列出的所有列都应该会引用到第一个SELECT语句里的各个列的名字。

UNLOCK TABLE

UNLOCK {TABLE | TABLES}

这条语句会释放当前客户端所占有的所有表锁。如果客户端开始了事务，或者客户端会话终止，则服务器会自动释放该客户端所占有的表锁。

UPDATE

UPDATE [LOW_PRIORITY] [IGNORE]

tbl_name

[PARTITION (partition_name [, partition_name] ...)]

SET col_name=expr [, col_name=expr] ...

[WHERE where_expr] [ORDER BY ...] [LIMIT n]

UPDATE [LOW_PRIORITY] [IGNORE] tbl_refs

SET col_name=expr [, col_name=expr] ...

[WHERE where_expr] [ORDER BY ...] [LIMIT n]

UPDATE 语句的第一种语法将对tbl_name 表里已有行的内容进行修改。自MySQL 5.6.2起， UPDATE支持针对分区表使用PARTITION子句，以表明行需要更新到哪一个分区或子分区。在这种情况下，如果将被更新的行不在指定的分区里，那么它会保持不变。

UPDATA 语句的第二种语法类似于第一种，但是允许一次给出多个表以进行一次多表修改。tbl_refs的语法和SELECT的相似（包括对紧跟在每个表名后面的那个PARTITION子句也一样），二者的区别在于：这里不再允许把子查询当作表。

只有用WHERE子句所给出的表达式筛选出来的那些行才会被修改。对于选出来的每一个行，在SET子句里给出的每一列将被设置为相应表达式的值。

```
UPDATE member SET expiration = NULL, phone = '197-602-4832'
```

```
WHERE member_id = 14;
```

WHERE子句允许包含子查询，但不允许子查询从正被修改的表选取行。

如果没有给出任何WHERE子句，那么表里的所有行都将被修改。

默认情况下，UPDATE语句返回的是它实际更新的行的个数。如果某个行的列在修改前后没有发生任何实质性变化，UPDATA语句将不把它统计在内。如此说来，把某个列设置为它的当前值不算影响了行。

如果真的需要UPDATE语句返回一个数值告诉你到底有多少个行和它的WHERE子句相匹配（不管它实际修改了多少个行），那么应该在与服务器建立连接时给出CLIENT_FOUND_ROWS选项。请参考附录G里的mysql_real_connect()函数条目。

LOW_PRIORITY 子句将使这条语句的执行被延迟到没有客户端读取该表为止。此选项只适用于那些使用表级锁定的存储引擎，如MyISAM或MEMORY。

如果更新行会导致某个唯一性索引出现重复的键值，那么UPDATE语句将终止执行并报告出错，剩余的行将不再被更新。加上IGNORE将使这些行不被更新，并且不会报告出错。在严格SQL模式下，IGNORE关键字还将使得INSERT语句把会导致它终止执行的数据转换错误当作一个不那么致命的警告来处理。在这种情况下，列会被更新成最近的合法值。

如果带有ORDER BY子句，UPDATE语句将按给定顺序依次修改行。这个子句的语法与SELECT语句的相同。

如果给出LIMIT子句，则其中的n值指的是将被更新的最大行数。

对于操作多个表的UPDATE语句，其WHERE子句可以基于表之间的连接指定条件，并且其SET子句可以对多个表里的列进行更新。例如，下面的语句将更新t1表里的id值与t2表里的id值相匹配的那些行，把quantity值从t2表复制到t1表：

```
UPDATE t INNER JOIN t2 SET t.quantity = t2.quantity WHERE t.id = t2.id;
```

USE

USE db_name

把db_name 设定为默认数据库（未显式给出数据库名的表、视图或存储程序引用的数据库）。在USE 语句执行成功之后，服务器将把会话级系统变量 character_set_database和collation_database设置为数据库的字符集和排序规则。

如果给定数据库不存在或者是你没有访问它的权限，那么USE语句会失败。

E. 2 SQL语句语法（复合语句）

本节描述的内容是用来编写复合语句的语句的语法。复合语句由关键字BEGIN和END之间的一条或多条语句构成，可用于编写保存在服务器端的存储程序：函数、过程、触发器和事件。

程序体内部的每条语句都必须以分号（;）字符结尾。如果打算使用mysql 程序来创建一个包含有多条语句的存储例程，则必须临时性地重定义mysql的语句分隔符，以确保mysql本身不会去解释那些“;”字符。使用delimiter命令可以做到这一点。在挑选语句分隔符时一定要保证它不会出现在用来定义存储例程的语句里。与定义存储程序有关的更多信息请参考4. 2. 1节。

E. 2. 1 控制结构语句

本节里将介绍一些能把语句归聚为语句块并提供各种流程控制构造的语句。在这些语句的语法表达式里，每个stmt_list代表一个由一条或多条语句构成的语句块，每条语句都以一个分号字符（;）结束。

有些构造可以带有标签（如BEGIN、LOOP、REPEAT和WHILE）。标签不区分大小写，但必须遵守以下规则。

如果标签出现在某个构造的开头，那么与之同名的标签也需要出现在该构造的末尾。

如果某个构造的开头没有与之匹配的标签，那么该标签不能出现在这个构造的末尾。

```
BEGIN ... END
```

```
BEGIN [stmt_list] END
```

```
label: BEGIN [stmt_list] END [label]
```

BEGIN...END构造会创建一个语句块，其中可以包含多条语句。如果某个存储程序体需要包含多条语句，就必须把它们放在一个BEGIN块里。此外，如果BEGIN块包含有DECLARE语句，则它们必须出现在BEGIN块的开头部分。

CASE

CASE [expr]

WHEN expr1 THEN stmt_list1

[WHEN expr2 THEN stmt_list2] ...

[ELSE stmt_list]

END IF

CASE语句提供了一种分支型的流程控制构造。这条语句与C. 1. 4节描述的CASE运算符不同。

如果指定了初始表达式expr，那么CASE语句会将它与跟在每个WHEN后面的表达式进行比较。在找到第一个匹配时，相应的THEN 值后面的语句列表将被执行。在将某个给定值与一组值进行比较时，这个会比较有用。

如果没有给出初始表达式expr，那么CASE语句将计算WHEN表达式。在遇到第一个值为真的WHEN表达式时，相应的THEN关键字后面的语句列表将被执行。这个适用于执行“不等于”测试或测试任意条件。

如果没有找到匹配的WHEN表达式，则执行ELSE子句的语句列表，如果有的话。

IF

IF expr1 THEN stmt_list1

[ELSEIF expr2 THEN stmt_list2] ...

[ELSE stmt_list]

END IF

IF语句提供了一种分支型的流程控制构造。这条语句不同于C. 2. 1节里介绍的IF () 函数。

如果IF关键字后面的表达式为真，则执行第一个THEN关键字后面的语句列表。否则，依次对各ELSEIF 关键字后面的表达式进行计算。在遇到第一个值为真的ELSEIF 表达式时，执行相应的THEN关键字后面的语句列表。如果所有表达式的计算结果都为真，则执行ELSE子句里的语句列表，如果有的话。

ITERATE

ITERATE label

ITERATE语句用来开始循环的下一次迭代，它只能在循环构造内部使用。它可以出现在LOOP、REPEATE和WHILE语句内。

LEAVE

LEAVE label

LEAVE 语句用来退出一个带有给定标签的流程控制构造。该语句只能出现在带有给定标签的构造的内部。

LOOP

LOOP stmt_list END LOOP

label: LOOP stmt_list END LOOP [label]

这条语句用来创建一个执行循环。循环内的语句将反复执行，直到控制权被转出该循环。

REPEAT

REPEAT stmt_list UNTIL expr END REPEAT

label: REPEAT stmt_list UNTIL expr END REPEAT [label]

这条语句用来创建一个执行循环。循环内的语句将反复执行，直到表达式expr的值为真为止。

RETURN

RETURN expr

RETURN语句只能用在存储函数里（不能用在存储过程、触发器或事件里）。它会终止存储函数的执行，表达式expr的值将被传递给调用这个函数的语句。一个存储函数里至少要有一条RETURN语句。

WHILE

WHILE expr DO stmt_list END WHILE

label: WHILE expr DO stmt_list END WHILE [label]

这条语句用来创建一个执行循环。循环内的语句将反复执行，只要表达式expr的值为真。

E. 2.2 声明语句

DECLARE

DECLARE var_name [, var_name] ...type [DEFAULT value]

DECLARE condition_name CONDITION FOR named_condition

named_condition: {SQLSTATE [VALUE] sqlstate_value |
mysql_errno}

DECLARE cursor_name CURSOR FOR select_stmt

DECLARE handler_type

HANDLER FOR handler_condition [, handler_condition] ...

statement

handler_type: {CONTINUE | EXIT}

handler_condition:

SQLSTATE [VALUE] sqlstate_value

| mysql_errno

| condition_name

| SQLWARNING

| NOT FOUND

| SQLEXCEPTION

DECLARE语句的变体用来声明局部变量、条件、游标（cursor）和处理程序（handler）。DECLARE语句只能出现在BEGIN块的开头。如果有多条DECLARE语句，它们必须按以下顺序出现。

（1）变量和条件声明

（2）游标声明

（3）处理程序声明

如果在DECLARE关键字的后面以逗号为分隔符列出一组变量，就能把那些变量声明为可以在当前例程里使用的局部变量。局部变量只能在声明它们的BEGIN块里以及嵌套在该BEGIN块内部的其他语句块里使用。

在DECLARE语句里还可以用DEFAULT子句对局部变量进行初始化。如果没有DEFAULT子句，初始值将为NULL。如果需要在例程里把值赋给

局部变量，可以使用SET 语句、SELECT ... INTO语句，或者FETCH ... INTO语句。

DECLARE ... CONDITION语法用来为条件创建名字。
condition_name可以在DECLARE ... HANDLER语句、SIGNAL语句和RESIGNAL语句里引用。named_condition可以是一个带引号的5-字符字符串表示的SQLSTATE值，也可以是一个数值形式的MySQL专用的错误代码。

DECLARE ... CURSOR语法用来声明游标，并把它与给定的SELECT语句相关联，该SELECT语句不得包含INTO子句。游标可以用OPEN语句打开，和FETCH语句一起使用可以检索行，可以用CLOSE语句关闭。

DECLARE ... HANDLER可以把一个或多个条件与一条语句关联在一起，从而只要其中某个条件发生，该语句便会被执行（该语句可以是一条BEGIN ... END复合语句）。那个handler_type值决定着处理程序执行完毕后将会发生的事情。如果handler_type的值是CONTINUE，则继续执行下一条语句；如果是EXIT，则立刻退出当前BEGIN块。

handler_condition可以是任何一种以下类型的值。

由带引号的5-字符字符串表示的SQLSTATE值。这个值不应该是
由'00'开头的，因为那表示的是“成功”而不是“错误”。

数字形式的MySQL专用的错误代码。这个值不应该是零，因为那表示的是“成功”而不是“错误”。

使用DECLARE ... CONDITION语句提前声明的命名条件。

SQLWARNING，它代表的是所有以'01'开头的SQLSTATE值。

NOT FOUND，它代表的是所有以'02'开头的SQLSTATE值。

SQLEXCEPTION，它代表的是所有不以'01'、'02'或'03'作为开头SQLSTATE值。

与DECLARE的各个类型有关的示例请参考E. 2. 3节。

E. 2. 3 游标语句

本节里的这些语句可用来打开或关闭游标，以及在打开期间用来获取行。在MySQL里，游标都是只读的，并且是每次只能移动到结果集的下一行（也就是说，它们无法滚动）。下面这个的例子演示了如何使用与游标相关的所有语句。它还展示了DECLARE的所有变体——它们通常与游标循环一起使用。

```
CREATE PROCEDURE p ()  
  
BEGIN  
  
    DECLARE more_data INT DEFAULT TRUE;  
  
    DECLARE id INT;  
  
    DECLARE no_data CONDITION FOR SQLSTATE '02000';  
  
    DECLARE curs CURSOR FOR SELECT member_id FROM  
sampdb.member;  
  
    DECLARE CONTINUE HANDLER FOR no_data  
  
    BEGIN  
  
        SET more_data = FALSE;  
  
    END;  
  
    -- 打开游标，使用游标获取行，一直到No Data出现为止  
  
    OPEN curs;  
  
    WHILE more_data DO  
  
        FETCH curs INTO id;  
  
    END WHILE;
```

CLOSE curs;

END;

CLOSE

CLOSE cursor_name

关闭给定的游标，它在被关闭前必须是打开的。当一个BEGIN 块结束时，在这个BEGIN 块里声明的游标都将自动关闭。

FETCH

FETCH [[NEXT] FROM] cursor_name INTO var_name [, var_name] ...

利用给定游标可以把结果集里的下一行提取到在关键字 INTO 后面列出的（一个或多个）变量里。该游标必须是打开的。如果已经没有可供提取的行，则会出现一个错误，并且SQLSTATE的值会变成02000（即表示No Data，无数据）。

OPEN

OPEN cursor_name

打开给定的游标以便在后面的FETCH语句里使用它。

E. 2. 4 条件处理语句

在SQL 语句执行过程中，它们会产生与发生条件有关的诊断信息。这些信息可以通过 GET DIAGNOSTICS来获得。也可以通过SIGNAL和RESIGNAL来显式地触发这些条件。本节将介绍这些语句。与指定条件和条件处理程序的声明语法有关的信息请参考E. 2. 2节。

GET DIAGNOSTICS

GET [CURRENT] DIAGNOSTICS

{

var_name = stmt_info_type

[, var_name = stmt_info_type] ...

| CONDITION n

var_name = condition_info_type

[, var_name = condition_info_type] ...

}

stmt_info_type:

NUMBER

| ROW_COUNT

condition_info_type:

CLASS_ORIGIN

| SUBCLASS_ORIGIN

| RETURNED_SQLSTATE

| MESSAGE_TEXT

| MYSQL_ERRNO

| CONSTRAINT_CATALOG

| CONSTRAINT_SCHEMA

| CONSTRAINT_NAME

| CATALOG_NAME

| SCHEMA_NAME

| TABLE_NAME

| COLUMN_NAME

| CURSOR_NAME

这条语句可用于检查诊断区域的内容，其中包括语句生成的两类信息：语句本身的信息（如受影响的行数或发生的条件数）和与该语句生成的那些条件有关的信息（如错误信息和错误代码）。这个诊断区域可以包含与多个条件有关的信息。GET DIAGNOSTICS是在MySQL 5.6.4里引入的。

CURRENT关键字在MySQL里没有意义，可以省略掉。标准SQL拥有一个诊断区域栈。MySQL只有一个诊断区域，它始终是当前的。

每个GET DIAGNOSTICS实例都会返回语句信息或者条件信息（带上CONDITION子句）。例如，为把获取到的受影响行数和语句的条件数放到用户定义变量里，可以像下面这样做：

```
GET DIAGNOSTICS
```

```
@affected_rows = ROW_COUNT, @condition_count = NUMBER;
```

如果想获得条件2的SQLSTATE值和错误信息，可以运行下面的语句：

```
GET DIAGNOSTICS CONDITION 2
```

```
@sqlstate = RETURNED_SQLSTATE, @message =  
MESSAGE_TEXT;
```

其中，CONDITION子句对应的条件编号必须是一个整数，范围是1到条件总数。指定它的方式包括文本值、用户定义变量、系统变量、存储程序局部变量、存储函数参数或存储过程参数。系统变量max_error_count控制着诊断区域可以存储多少个条件。warning_count变量指明的是累计出现了多少个错误、警告和注意类型

的条件。这个值可以超过 NUMBER，因为它会对条件进行统计，看看是否有足够的空间存储它们。error_count 与之类似，但是它只会统计错误类型条件。

这条语句的剩下部分指明的是要获取什么样的信息，它由一个或多个以逗号分隔的变量赋值表达式组成。每个变量 var_name 都可以是用户定义变量、存储程序局部变量、存储函数参数或存储过程参数。允许分配给这些变量的类型名将在下面介绍。其中，数字项都是整数值；其他的值都是字符串。

对于诊断区域里的语句信息，可以使用下面几项。

NUMBER：诊断区域里的条件数量。有可能没有任何条件，在这种情况下，NUMBER为0。如果要确定有多少条件，可以使用下面这条语句：

```
GET DIAGNOSTICS @condition_count = NUMBER;
```

ROW_COUNT：语句影响到的行数。

对于诊断领域里的条件信息，可以使用下列各项。除了 RETURNED_SQLSTATE之外，这些项目也可以用在 SIGNAL 和 RESIGNAL 的 SET 子句里。这两条语句会根据它们指定的条件间接设置 RETURNED_SQLSTATE 的值。

CLASS_ORIGIN 和 SUBCLASS_ORIGIN：它们分别是 RETURNED_SQLSTATE 值的类和子类。

如果该值是在 SQL 标准文档 ISO 9075-2 里定义的，则类为 'ISO 9075'；否则，为 'MySQL'。如果类为 'ISO 9075'，或者是以 '000' 结尾的 RETURNED_SQLSTATE，则子类为 'ISO 9075'；否则，为 'MySQL'。

RETURNED_SQLSTATE：SQLSTATE 字符串。

MESSAGE_TEXT：错误信息。

MYSQL_ERRNO：MySQL 错误代码。

CONSTRAINT_CATALOG、CONSTRAINT_SCHEMA和CONSTRAINT_NAME：冲突约束的目录、模式和名称。在MySQL里总是为空。

CATALOG_NAME、SCHEMA_NAME、TABLE_NAME和COLUMN_NAME：条件的相关目录、模式、表和列。在MySQL里总是为空。

CURSOR_NAME：条件的相关游标名。在MySQL里总是为空。

这些“总是为空”的值对于GET DIAGNOSTICS也总是为空。它们可以被SIGNAL或RESIGNAL设置为非空，因为这些语句会将值赋予各个条件区域成员。

那些使用表的语句在开始执行时会对诊断区域进行清理。如果某一条语句在执行期间出现了问题，则其他的语句会根据上一条语句对条件的诊断区域进行清理。GET DIAGNOSTICS和RESIGNAL属于例外，它们会把自己的条件添加到诊断区域，而不是对其进行清理。可以使用GET DIAGNOSTICS、SHOW WARNINGS或SHOW ERRORS来检索或显示条件信息，而不用对诊断区域进行清理。

RESIGNAL

RESIGNAL [condition_value]

[SET condition_info_type = value

[, condition_info_type = value] ...]

condition_value, condition_info_type, value:

see SIGNAL description

RESIGNAL 语句与 SIGNAL 相似，不过它主要用于传递先前生成的条件信息，并且有可能对其进行修改或添加。在条件处理器里，如果你发现某个条件在外部上下文里可以比在当前上下文里处理得更好，这一点会很有用。与 SIGNAL 有所不同的是，RESIGNAL 只有在某些条件处理器激活的情况下才可以使用。

关于RESIGNAL的语法术语的解释请参考SIGNAL的描述。RESIGNAL语句也基本相同，主要差异在于condition_value是可选的。

不带condition_value的RESIGNAL会传递已有的条件信息。如果存在SET子句，则该子句里的赋值操作会修改条件信息的指定部分。

带有condition_value的RESIGNAL会把最近那个条件的副本添加到条件区域的末尾，并且会根据condition_value和SET子句里存在的赋值表达式来对它进行修改。

SIGNAL

SIGNAL condition_value

[SET condition_info_type = value

[, condition_info_type = value] ...]

condition_value:

SQLSTATE [VALUE] sqlstate_value

| condition_name

condition_info_type:

CLASS_ORIGIN

| SUBCLASS_ORIGIN

| MESSAGE_TEXT

| MYSQL_ERRNO

| CONSTRAINT_CATALOG

| CONSTRAINT_SCHEMA

| CONSTRAINT_NAME

| CATALOG_NAME

| SCHEMA_NAME

| TABLE_NAME

| COLUMN_NAME

| CURSOR_NAME

这条语句能让你发送错误提示，在语句执行期间，这一点有助于解决问题，并且能够提供更为详细的信息。

SIGNAL 关键字后面的那个condition_value 表示的是错误提示的条件，其指定方式既可以是5-字符SQLSTATE字符串，也可以是使用DECLARE ... CONDITION 语句声明的条件名。与各个SQLSTATE值及DECLARE语句的语法有关的信息请参考E. 2. 2节。

其中的SET子句是可选的，用于给各个条件信息项进行赋值；你可以通过这个子句对条件进行“描述”。SET 子句由一个或多个以逗号分隔的赋值表达式构成，其中的每一个都会指定一个信息的类型和分配给它的价值。下面这段代码可以测试除法运算的除数。如果出现了被零除的问题，它会提示出错，同时设置MySQL的错误代码和错误信息。

```
IF divisor <> 0 THEN
```

```
SET ratio = numerator / divisor;
```

```
ELSE
```

```
SIGNAL SQLSTATE '22012'
```

```
SET MYSQL_ERRNO = 1365, MESSAGE_TEXT = 'Division by zero attempt';
```

END IF;

在每一个赋值表达式里，用于赋值的那个值value不能是普通的表达式。它只能被指定为文本值、用户定义变量、系统变量、存储程序局部变量、存储函数参数或存储过程参数。

与允许条件信息类型名称及其含义有关的更多信息请参考GET DIAGNOSTICS的描述。

E. 3 注释语法

MySQL允许你在SQL代码里穿插一些注释，以此对存储在文件里的那些语句进行说明。MySQL服务器支持以下三种类型的注释。

从“#”到该行末尾的所有内容都会被当作注释。这种语法与绝大多数的Unix shell程序以及许多种脚本编程语言（如Perl、PHP和Ruby）使用的注释语法相同。

```
# this is a single line comment
```

在“/*”和“*/”之间的所有内容都会被当作注释。这种形式的注释允许跨越多行。这种语法与C语言所使用的注释语法相同。

```
/* this is a single line comment */
```

```
/* this
```

```
is a multiple line
```

```
comment
```

```
*/
```

可以用两个短划线加一个空格（“--”）或者两个短划线加一个控制字符（如换行符）来开始一条注释。从双短划线到行尾的所有文字都将被视为注释内容。

```
--
```

```
-- This is a comment
```

```
--
```

MySQL的双短划线注释风格与标准SQL的注释风格不太一样，后者只要求以两个短划线开头，不要求后面必须有一个空格。MySQL要求必须在短划线的后面加上一个空格是为了避免二义性。否则，像5--7这样的表达式很可能会被误认为包含一条注释。因为不太可能写出像5--7这样的表达式，所以这是一个有用的启发式。不过，这仅仅是一个启发式，当从其他数据库将包含有两个短划线的代码导入MySQL时，一定要对这一点加以注意。

在执行语句时，服务器会忽略其中的注释，但以“/*!”开始的C语言风格注释却是个例外。我们可以把MySQL独有的关键字“隐藏”在以“/*!”开头（不是以“/*”开头）的C风格注释里。在遇到这种特殊形式的注释时，MySQL将识别出其中的关键字并执行相应的动作，而其他的数据库服务器却会把它们当作注释的一部分忽略掉。这种安排增加了代码的可移植性，至少对支持C语言风格注释的其他服务器来说是这样的：在MySQL环境里执行的代码，在编写时可能使用了MySQL独有的函数；但当把代码拿到其他数据库环境里去使用时，也用不着修改。下面两条语句在其他数据库服务器看来是等价的，但是当MySQL遇到第二条语句时，会执行INSERT LOW_PRIORITY操作：

```
INSERT INTO mytbl (id,date) VALUES(13,'2013-09-28');
```

```
INSERT /*! LOW_PRIORITY */ INTO mytbl (id,date)
VALUES(13,'2013-09-28');
```

C语言风格的注释可以用来实现版本控制。具体做法是，在“/*!”序列的后面加上5个数字的版本号。如果MySQL服务器的版本低于该注释所给出的版本，它将忽略这条注释。除非MySQL服务器的版本是5.6.2，或者更高，否则下面这条DELETE语句里的注释会被忽略（只有MySQL 5.6.2及以后版本的服务器才能理解DELETE的PARTITION子句）：

```
DELETE FROM mytbl /*!50602 PARTITION (p1) */ WHERE date <
'2010-01-01';
```


附录F

MySQL程序参考

本附录的主要内容是MySQL程序调用，同时还对下面列出的几个程序进行了详细讲解。与各个程序相对应的章节所包含的内容有程序用途、程序调用语法、所支持的选项，以及内部使用的各种变量。如无特别说明，这里列举的选项和变量最早可以追溯到的MySQL版本是MySQL 5.5.0。在那之后产生的变化都会单独进行说明。

`myisamchk`

用于检查和修复MyISAM表、分析键的分布情况、禁用或启用索引。

`mysql`

用于与MySQL服务器进行交互，或者以批处理模式执行文件里的语句。

`mysql.server`

用于启动和停止MySQL服务器。

`mysql_config`

用于显示编译基于MySQL的程序所使用的各种特定标志。

`mysql_install_db`

用于初始化服务器的数据目录和权限表。

`mysql_upgrade`

用于安装完较新版MySQL之后的数据库升级。

`mysqladmin`

用于完成各种管理操作。

`mysqlbinlog`

用于显示文本格式的二进制日志和中继日志。

`mysqlcheck`

用于表的检查、修复、优化和分析。

`mysqld`

MySQL服务器。它必须处于运行状态，这样客户端程序才能访问数据库。

`mysqld_multi`

用于同时启动和停止多个服务器。

`mysqld_safe`

用于启动和监视MySQL服务器。

`mysqldump`

用于转储数据库的内容。

`mysqlimport`

用于将数据批量载入表中。

`mysqlshow`

用于显示关于数据库或表的信息。

`perror`

用于显示错误代码的含义。

语法描述里的可选信息都是用方括号（[]）来表示的。

F.1 显示程序的帮助信息

本附录在随后介绍每个程序时，都会列出它们当前所能支持的各个选项。如果某个程序无法识别出在其描述里列举的某个选项，则该程序可能是比较早的版本，或者该选项在较新版本里可能已被弃用或移除。

如果想要获得程序支持的选项列表，可以查看该程序的帮助信息，采用这种方式，可以快速获得与程序本身有关的各种信息。对于服务器程序（即 `mysqld`），在调用它时可以带上 `--version` 选项和 `--help` 选项。对于其他程序，只需要使用 `--help` 选项即可。例如，如果不太清楚如何使用 `mysqlimport`，可以像下面这样调用命令来获得帮助信息：

```
% mysqlimport --help
```

选项 `-?` 等同于选项 `--help`，但是你的 `shell`（命令解释器）程序可能会把字符 `"?"` 解释为文件名通配符：

```
% mysqlimport -?
```

```
mysqlimport: No match.
```

如果遇到这种情况，可以试试下面这条命令：

```
% mysqlimport -\?
```

有些选项只会在特定条件下才会显示帮助信息。例如，对于SSL有关的选项，只有在把SSL支持功能编译进程序以后才会被显示出来；而对于 `--pipe` 这种只有 Windows 系统独有的选项，则会存在于 Windows 系统里。

MySQL 程序的帮助信息还会显示出它们查找选项文件的默认位置，并列出所支持的各个变量。

F.2 指定程序选项

大部分的MySQL程序都能理解会影响它们的操作的那几个选项。选项既可以在命令行上指定，也可以在选项文件里给出。此外，有一些选项可以通过设置环境变量来指定。在命令行上指定的选项优先于其他方式指定的选项，在选项文件里指定的选项优先于通过环境变量设置的选项。

大部分的选项都既有长形式（完整单词）又有短形式（单字符）。前面介绍过的那对选项 `--help` 和 `-?` 就是这样的例子。对于后面带有值的长格式选项，其指定格式为 `--name=val` 或 `--name val`，其中，`name` 是选项名对，`val` 是

该选项值。如果短形式选项的后面带有值，那么在大部分情况下，选项和值之间都可以用空白来隔开。例如，当你指定用户名时，`-usampadm`和`-usampadm` 是等价的。选项`-p`（密码）是个例外；那个密码值是可选的，但如果给出该值，则该值必须紧随`-p`之后，它们之间不能有空格。

选项名都是区分大小写的，但是选项值区不区分大小写都行。例如，像用户名和密码这样的值都是区分大小写的，但是选项`--protocol`的值则不区分大小写。在进行 TCP/IP 连接时，`--protocol=tcp`与`--protocol=TCP`都是等价的。

许多选项都是“布尔型”的，其值为开或关。这些选项都有一个基本形式和一组相关的公认标准形式，如下表所示。

选项	含义
<code>--name</code>	基本形式；表示启用该选项
<code>--enable-name</code>	<code>--enable-</code> 前缀；表示启用该选项
<code>--disable-name</code>	<code>--disable-</code> 前缀；表示禁用该选项
<code>--skip-name</code>	<code>--skip-</code> 前缀；表示禁用该选项
<code>--name=1</code>	<code>=1</code> 后缀；表示启用该选项
<code>--name=0</code>	<code>=0</code> 后缀；表示禁用该选项

例如，许多 MySQL 客户端程序都支持使用客户端/服务器协议压缩功能。对于这些程序，如果指定`--compress` 选项，可以启用压缩功能；如果忽略它，则不会使用压缩功能。不过，采用其他方式也可以达到同样的效果：`--enable-compress`和`--compress=1` 也可以启用压缩功能，而`--disable-compress`、`--skip-compress`和`--compress=0`则会禁用压缩功能。

对于那些默认启用的选项，显式禁用选项的格式非常有用。对于协议压缩，采用忽略`--compress`选项的方式便可以禁用它。但是，对于那些默认启用的选项则不然。例如，`mysqldump`的`--quote-names`选项默认是启用的。你不能采用将其省略的方式来禁用名字引用功能，但是可以通过指定`--skip-quote-names`、`--disable-quote-names`或`--quote-names=0`的方式来实现。

本附录在描述程序时使用了指示词“（布尔值）”来明确说明哪些选项的解释优先级更高——也就是说，表中所示的前缀选项和后缀选项都支持。

当感到疑惑时，可以查看程序的帮助消息，弄清它所支持的选项形式（参考F.1节）。

MySQL程序还有其他几个标准的选项处理功能，如下所示。

长选项可以截短为无歧义的前缀，这样在指定名字很长的选项时会变得更加容易。如果指定的前缀长度不足以消除歧义，那么你调用的程序会给出提示，并且列出与该前缀匹配的各个选项：

```
% mysql --h
```

```
mysql: ambiguous option '--h' (help, html)
```

程序变量可以在命令行上设置，也可以在选项文件里（需要将变量名当作选项名）设置。尽管变量名里有下划线，没有短划线，但是对于命令行或选项文件里的选项名或变量名，既可以使用下划线，也可以使用短划线。

（不过，对于命令行里的长选项，其前面的双短划线不能换作破折号。）更多相关信息请参考F.2.1.2节。

MySQL 支持选项前缀`--loose-`。当使用不同版本的程序（它们理解的选项集不同）时，它可以让整个事情变得更加简单。有了`--loose`之后，无法识别的选项只会引发一个警告，而不会导致程序出错终止。例如，mysql 自 5.5.3 版本之后允许使用`--auto-vertical-output`选项，但是之前的版本则不支持。如果把该选项指定为`--loose-auto-vertical-output`，则所有版本的mysql都会根据自己是否能够理解该选项来决定是使用它，还是忽略它。

MySQL服务器，mysqld，支持使用`--maximum-`前缀，其作用是指定用户可修改的那个变量可被设置的最大值。例如，服务器允许用户通过更改`sort_buffer_size`变量来设置排序缓冲区的大小。如果想把这个变量的值设置为最大上限 64 MB，可以在启动服务器时带上选项`--maximum-sort_buffer_size=64MB`。

F.2.1 MySQL程序的标准选项

有几个选项的含义是标准的，几乎所有的MySQL程序都能以同样的方式来解释它们。为了避免在多个程序的描述里重复它们，本节将对它们进行一次性的讲解；在每个程序的“支持的标准选项”一节将单独对程序所能理解的各个选项进行介绍。在该节里，将只列出长格式名称，不过各个程序也都能理解相应的短格式选项，除非另有说明。

下面列出的是对多个标准选项的描述。如果MySQL在编译时未被重新配置，则使用的默认值即为这里显示的那些值。

`--bind-address=addr`

对于客户端程序，此选项是自 MySQL 5.6.1以后引入的，它指明的是连接服务器时绑定的那个IP地址。这在拥有多个网络接口的机器上非常有用。对于服务器，此选项的含义比较复杂。

请参考F.12.2节。

`--character-sets-dir=dir_name`

指的是用于存放字符集文件的目录。

`--compress`或`-C`（布尔值）

此选项仅用于客户端程序，其作用是要求使用压缩后的客户端/服务器通信协议，如果客户端和服务器都支持它的话。

`--debug=debug_options`或`-# debug_options`

用于打开调试输出。如果 MySQL 在编译时启用了调试支持机制，否则此选项不起作用。

`debug_options`字符串由用冒号分隔的选项构成。常见的值是`d:t:o,``file-name`，它能启用调试功能，打开函数调用的进入和退出跟踪机制，以及将输出内容发送到`file_name`文件。

如果希望进行更多的调试，应该查阅DBUG库的用户手册，获得所要使用的那些选项的描述信息。该手册位于MySQL源发行包的`dbug`目录里。

`--debug-check`（布尔值）

在程序退出时检查内存和打开文件的使用情况。

`--debug-info`（布尔值）

与`--debug-check`相类似，不过它还会显示与内存和CPU使用情况有关的信息。

`--default-auth=plugin_name`

对于客户端，指的是所用的身份认证插件。应该没必要使用此选项，因为服务器会基于指定的客户端用户名来发送插件名。此选项是在MySQL 5.5.7里引入的。

`--default-character-set=charset`

设置默认字符集。charset值不能为ucs2、utf16、utf16le或utf32。

`--help`或`-?`

显示帮助信息并退出。请参考F.1节。

`--host=host_name`或`-h host_name`

只有客户端程序才能使用此选项。它指明了要连接到的主机（即运行服务器的那台主机）。默认值为localhost。

`--password[=pass_val]`或`-p[pass_val]`

只有客户端程序才能使用此选项，它指明了连接服务器时使用的密码。如果在选项名后面没有指定 pass_val，则程序会提示你输入密码。如果在-p之后指定了 pass_val，则它必须紧跟在选项名的后面，中间不留任何空白。换句话说，此选项的短形式必须写成-ppass_val，而不能写成-p pass_val。

如果想要显式指定“无密码”，可以使用--skip-password。当某个选项文件包含了你想忽略的密码时，可以采用这种方式。

`--pipe` 或 `-W`

指定使用命名管道方式连接服务器。此选项只能用于 Windows里的客户端程序，并且只能用于连接那些支持命名管道功能的Windows服务器。

`--plugin-dir=dir_name`

对于服务器，它指明的是服务器端插件所在的位置。对于客户端，它指明的是客户端插件所在的位置。此选项是在MySQL 5.5.7里引入的。

`--port=port_num`或`-P port_num`

对于mysqld，此选项指定的是TCP/IP连接的监听端口号。默认端口号是3306。对于客户端程序，它指定的是通过TCP/IP连接服务器时要使用的端口号。

`--protocol=protocol_type`

只有客户端程序才能使用此选项，它指明了与服务器建立的连接的类型。protocol_type的值可以为tcp（使用TCP/IP）、socket（使用Unix套接

字文件)、pipe (使用Windows命名管道) 或memory (使用共享内存)。这个值不区分大小写。

有些连接类型是与平台特定相关的, 或者只能用于连接与客户端程序同处一台主机的本地服务器。

套接字、命名管道和共享内存连接都只能用于连接本地服务器。

套接字连接只能用于Unix。

命名管道和共享内存连接只能用于Windows。

TCP/IP连接可用于任何平台, 并且既可用于连接本地服务器, 也可用于连接远程服务器。

选项--protocol可与其他用于提供服务器连接信息的选项一起使用。

对于TCP/IP连接, 选项--host和--port分别指定的是主机名和TCP/IP端口号。

对于套接字和命名管道连接, --socket选项在Unix上指定的是Unix套接字文件名, 在Windows上指定的是命名管道名称。

对于共享内存连接, --share-memory-base-name选项指定的是共享内存名称。

--shared-memory-base-name=name

指定用于共享内存连接的共享内存名。默认值为MYSQL。这个值区分大小写。

--silent或-s

用于告诉程序以静默模式运行。这并不表示程序是完全静默的, 而只是表示程序会比正常情况下生成更少的输出。有些程序允许多次指定此选项, 从而使其变得更静默。

--socket=file_name或-S file_name

对于Unix上的客户端程序, 它指的是在连接主机名为localhost的服务器时所使用的Unix套接字文件的完整路径名。默认Unix套接字文件名为/tmp/mysql.sock。如果MySQL主机上的文件名需要区分大小写, 那么这个路径名也要区分大小写。对于Windows里的客户端程序, 它指的是通过命名

管道连接MySQL服务器时使用的命名管道名。默认管道名为MySQL。管道名不区分大小写。

`--user=user_name`或`-u user_name`

对于mysql，它指的是运行服务器时使用的那个Unix账户的名字或用户ID。如果想让此选项起作用，则服务器必须以root身份来启动，因为只有这样才能将其用户ID更改为你所指定的那个账户。对于客户端程序，它指的是在连接服务器时所用的MySQL用户名。默认为登录名（在Unix里）或ODBC（在Windows里）。

`--verbose`或`-v`

用于告知程序以详细模式运行。此时，程序会比正常情况产生更多的输出。有些程序允许多次指定此选项，从而使输出变得更详细。

`--version`或`-V`

用于告知程序显示其版本信息字符串，然后退出。

F. 2. 1. 1 SSL标准选项

下面这些选项主要用于建立安全连接。只有在编译 MySQL时带上了 SSL功能支持时才能使用它们。与建立安全连接有关的更多信息请参考13.5节。

`--ssl`（布尔值）

启用SSL 连接。其他所有的SSL 选项都隐含带有`--ssl` 选项。此选项更为常见的用法是使用`--skip-ssl`来禁用SSL。

`--ssl-ca=file_name`

证书授权文件的路径名。

`--ssl-capath=dir_name`

用于保存证书认证所需的受信任证书的目录的路径名。

`--ssl-cert=file_name`

证书文件的路径名。

`--ssl-cipher=str`

这是一个字符串，其中列出的是可用于对连接发送的通信进行加密的SSL密码。字符串的值应该是一个或者多个以逗号分隔的密码类型。

`--ssl-crl=file_name`

证书撤销列表文件的路径名。此选项是在MySQL 5.6.3里引入的。

`--ssl-crlpath=dir_name`

存放证书撤销列表文件的路径名。此选项是在MySQL 5.6.3里引入的。

`--ssl-key=file_name`

密钥文件的路径名。

`--ssl-verify-server-cert`（布尔值）

此选项只适用于客户端程序，其目的是让客户端在从服务器收到的证书里检查Common Name字段的值。如果这个值不同于客户端连接的那台服务器主机，则放弃连接尝试。

F.2.1.2 设置程序变量

有些MySQL程序具有允许用户进行设置的变量（操作参数）。一种设置变量的方式是，将其名字当作一个选项来处理。例如，想要在调用mysql时将变量connect_timeout设置为10，则可以使用下面这条命令：

```
% mysql --connect_timeout=10
```

这种语法还允许将变量名里的下划线（_）写成短横线（-），从而使变量选项看起来更像其他的选项：

```
% mysql --connect-timeout=10
```

对于表示缓冲区大小或长度的变量，如果可以将值指定为没有后缀的数字，单位为字节；或者为值指定后缀'K'、'M'或'G'，分别表示千字节、兆字节或吉字节。这些后缀都不区分大小写，因此也可以将它们写成'k'、'm'或'g'。

各个程序的变量都列在本附录对它们的描述里，并且都可以显示在程序的帮助信息里（参考F.1节）。

F.2.2 选项文件

大部分MySQL程序都支持使用选项文件。选项文件提供了一种存储程序选项的方式，有了它就不用每次执行程序时都在命令行上输入选项了。选项文件里指定的选项都可以被命令行上显式指定的不同值改写。一种例外情况是：对于--user选项，mysqld使用的是它的第一个实例，其目的是防止选项文件里的实例被命令行上的命令改写。

MySQL 程序可以在多个地方寻找选项文件，但是当某个选项文件不存在时通常不会被视为错误。这意味着，你通常必须自己创建选项文件。选项文件必须是文本文件，如果在字处理器里创建选项文件，则一定要以普通文本格式保存它，而不能以字处理器的默认文档格式保存。

在Unix里，下表展示的各个选项文件都是按顺序（如果存在的话）被读取的。另外，如果某个选项文件是以--defaults-extra-file选项给出的，则它会在紧挨着~/.my.cnf的前面被读取。

文件名	内容
/etc/my.cnf	全局选项
/etc/mysql/my.cnf	全局选项
SYSCONFDIR/my.cnf	全局选项
\$MYSQL_HOME/my.cnf	特定于服务器的选项
~/.my.cnf	特定于用户的选项

其中，~表示的是你的主目录路径名。SYSCONFDIR 来自在构建 MySQL 时给 CMake 设置的-DSYSCONFDIR 选项，其默认值为编译在发行包里的那个安装目录下的etc 目录。\$MYSQL_HOME是一个环境变量，可设置为一个包含服务器相关选项文件的目录，供 mysqld_safe 使用。如果未设置它，则 mysqld_safe会试着自动设置它，以便在MySQL安装目录下查找my.cnf文件。

在Windows里，下表展示的各个选项文件都是按顺序（如果存在的话）被读取的。另外，如果某个选项文件是以--defaults-extra-file选项给出的，则它会在下表这些选项之后被读取。

文件名	内容
WINDIR\my.ini, WINDIR\my.ini	全局选项
C:\my.ini, C:\my.cnf	全局选项
INSTALLDIR\my.ini, INSTALLDIR\My.ini	全局选项

其中，WINDIR是Windows目录的路径名。INSTALLDIR是MySQL安装目录的路径名。

全局选项文件可以被所有对选项文件感兴趣的MySQL程序使用。在Unix里，特定于用户的选项文件只能被该用户运行的程序读取。

Windows用户在使用选项文件时，需要特别注意以下问题。

Windows路径名的各个组成部分是以反斜线字符（\）进行分隔的，而MySQL会把该字符当作转义字符。解决此问题的方法是，把反斜线写成斜线字符（/）或写成的两个反斜线（\\）。

在Windows里，文件名在显示时可能会把扩展名隐藏起来。如果创建了一个名为my.cnf的选项文件，则该名字可能会显示为 my。如果未注意到这一点，则可能会把它更改为my.cnf。此时，你会发现此选项文件不再发挥作用。这是因为实际上将该文件的名称从my.cnf改成了my.cnf.cnf！

有几个选项与选项文件处理有关，它们都是标准选项，能被大部分的MySQL识别。下面列出的是它们的具体含义。如果想要使用它们中的任何一个，它必须是命令行中的第一个选项，--print-defaults例外，它允许放在--defaults-file或--defaults-extra-file的后面。

`--defaults-extra-file=file_name`

除了有常规选项文件之外，还可以借助它再多指定一个选项文件。此文件的读取顺序是在全局和服务端选项文件之后、用户选项文件之前。文件必须存在，且可读；否则会出错。

`--defaults-file=file_name`

用于指定唯一的选项读取文件。一般情况下，程序会依次在多个位置寻找选项文件（如之前所述）。但是，如果指定了--default-file选项，则只会读取这里指定的那个文件。文件必须存在，且可读；否则会出错。

`--defaults-group-suffix=suffix`

读取惯用选项组，以及那些名字由惯用名加上指定后缀构成的选项组。

`--no-defaults`

禁止使用任何选项文件。此外，此选项还会导致像`--defaults-file`这样的选项文件相关的选项无法被识别。

`--print-defaults`

把在命令行调用程序时没有带任何选项也会被使用到的选项值显示出来。最终的显示值主要是从选项文件（以及环境变量）获得。在检查某个选项文件的设置是否正确时，`--print-defaults`选项非常有用。此外，如果发现MySQL程序使用了某个你从未指定的选项，则此选项也能派上用场：可以使用`--print-defaults`选项来检查某些选项是否来自某个选项文件。

在程序的帮助信息里可以看到程序默认查找选项文件的位置（参考F.1节）。默认需要读取哪些文件，会受`--defaults-file`、`--defaults-extra-file`或`--no-defaults`的影响。

选项文件里的选项是以分组（或分区）形式给出的。示例如下：

[client]

user=sampadm

password=secret

[mysql]

skip-auto-rehash

[mysqlshow]

status

其中，选项组的名字必须用方括号括起来，且不区分大小写。`[client]`比较特殊，它里面指定的选项适用于所有的客户端程序。而其他的选项组则通常只与某个具体的客户端程序相对应。在上面的示例里，`[mysql]`选项组表示的是它适用于mysql客户端程序；而`[mysqlshow]`选项组表示的是它适用于mysqlshow。标准的MySQL客户端程序会在`[client]`选项组和与客户端名同名

的选项组里查找选项。例如，mysql 会到选项组[client]和[mysql]里查找选项，而 mysqlshow 则会到[client]和[mysqlshow]里去查找。

一定要避免将只能由单个客户端理解的选项放置到[client]组里。例如，skip- auto-rehash只适用于mysql。如果把此选项放在[client]组里，则会导致像mysqlimport这样的客户端程序不再工作。（它们会在帮助信息后面显示出一条出错信息。）因此，需要把skip-auto-rehash放在[mysql]组里。

跟在某个组名后面的所有选项都与该组相关。一个选项文件可以包含任意数量的选项组，并且靠后的选项组具有更高的优先级。如果某个程序在多个选项组里多次找到某个选项，它会使用最后一次出现的那个值。一种例外情况是：对于--user选项，mysqld使用的是它的第一个实例，其目的是防止选项文件里的实例在命令行被改写掉。

命令行里的每个选项在选项文件里都可以指定为单独的一行。这一行的第一个单词是选项名，其形式是不带前导破折号的长名格式。例如，为了在命令行上指定压缩选项，可以使用-C 或--compress；但是在选项文件里，只能使用compress。程序支持的所有长格式选项都可以列在选项文件里。如果选项需要值，可以用等号(=)字符将名字和值隔开。

请看下面这行命令：

```
% mysql --compress --user=sampadm --max_allowed_packet=16M
```

如果想在选项文件的[mysql]组里指定同样的设置信息，可以像下面这样做：

```
[mysql]

compress

user=sampadm

max_allowed_packet=16M
```

你可以用单引号或双引号把选项值引起来。当值包含空格时，这种做法很有用效。

选项文件行里的起始空格会被忽略掉，围绕在将选项名和选项值分隔开的“=”字符两边的空格也会被忽略掉。空行和以“#”或“;”打头的行会被

当作注释忽略掉。在一行中间，也可以用“#”（但不能用“;”字符）字符来引导一条注释。

下表列出的是一些转义序列。它们可用在选项文件的各个值里，用于指定某些特殊字符。

序列	含义
<code>\b</code>	退格符
<code>\n</code>	换行符
<code>\r</code>	回车符
<code>\s</code>	空格
<code>\t</code>	制表符
<code>\\</code>	反斜线

选项文件可以包含读取其他选项文件的指令。

```
!include file_name
```

读取指定的选项文件。

```
!includedir dir_name
```

读取指定目录下的所有选项文件。Unix里扩展名为.cnf的文件和Windows里扩展名为.ini或.cnf的文件都会被识别为选项文件。文件的读取顺序不确定。

那些被包含文件需要遵循常规的选项文件语法。只有来自当前包含位置的那个选项组的选项才会被使用。

F. 2. 2. 1 让用户特定的选项文件专人专用

在Unix里，位于你主目录下的用户特定选项文件.my.cnf应该只能由你个人拥有；并且其访问模式应该设置为600或400，从而使其他用户无法读取它。你肯定不想让自己的MySQL用户名和密码被其他人看到。为了让用户特定的选项文件专人专用，可以在你的主目录里执行下面两条命令中的任何一条：

```
% chmod 600 .my.cnf
```

```
% chmod go-rwx .my.cnf
```

对于客户端程序mysql所生成的历史文件（即主目录里的.mysql_history文件），同样要多加注意。需要以同样的方式来对它加以保护。

F. 2. 2. 2 使用my_print_defaults来检查选项

工具my_print_defaults可用来检测程序从选项文件读取到的选项内容。它会搜索选项文件，并把在指定选项组里找到的各个选项显示出来。例如，mysql 程序使用了[client]组和[mysql]组里的选项。为弄清各个选项文件里都有哪些选项会应用到 mysql 程序上，可以像下面这样调用my_print_defaults：

```
% my_print_defaults client mysql
```

同样地，服务器程序mysqld 使用了[mysqld]和[server]组的选项。为确定各个选项文件里都有哪些选项，可以使用下面的命令：

```
% my_print_defaults mysqld server
```

F. 2. 3 环境变量

MySQL程序会检查一些环境变量的值，以获得选项设置。环境变量拥有比较低的优先级，通过它们指定的选项值可以被选项文件里的选项或者命令行里的选项改写。

MySQL程序会检查以下环境变量。

LANG、LC_ALL

如果通过这两个环境变量中的某一个来指定区域设置，则MySQL客户端程序会使用这个值来设置默认字符集。这一点在效果上类似于使用--defaultcharacter-set选项。

MYSQL_DEBUG

在调试的时候可以使用此选项。此变量只有在编译MySQL时启用了调试支持功能时才有效。

设置MYSQL_DEBUG变量类似于使用--debug选项。

MYSQL_PWD

它指的是与 MySQL 服务器建立连接时使用的密码。设置 MYSQL_PWD 变量类似于使用--password选项。

使用MYSQL_PWD 变量来保存密码存在安全风险，因为系统上的其他用户可以轻易发现它的值。例如，在某些系统里，使用ps工具就能显示出其他用户的环境变量设置。

MYSQL_TCP_PORT

对于 mysqld，它指的是 TCP/IP连接所使用的监听端口号。对于客户端程序，它指的是在连接服务器时所用的端口号。设置MYSQL_TCP_PORT 变量类似于使用--port选项。

MYSQL_UNIX_PORT

对于mysqld，它指的是本地连接所监听的那个套接字文件。对于客户端程序，它指的是与运行在localhost 上的服务器建立套接字文件连接时所使用的Unix套接字文件的路径名。设置MYSQL_UNIX_PORT变量类似于使用--socket选项。

TMPDIR

它指的是用于创建临时文件的那个目录的路径名。设置此变量类似于使用--tmpdir选项，差异在于这里只能指定一个目录。

USER

它指的是与服务器建立连接时使用的MySQL用户名。此变量只能由运行在Windows下的客户端程序使用。设置此类似于使用--user选项。

客户端mysql还会另外检查下面三个环境变量。

MYSQL_HISTFILE

在Unix里，它指的是用于存储在交互操作期间产生的命令行历史命令的那个文件的名称。如果此变量未设置，则默认为你主目录下的.mysql_history。

MYSQL_HOST

它指的是与 MySQL 服务器建立连接时需要连接的那个主机。设置此变量类似于使用 `--host` 选项。

MYSQL_PS1

它指的是用于代替 `mysql >` 的提示字符串。此字符串可以包含 F. 4. 5 节列出的各种特殊序列。

F. 3 myisamchk

`myisamchk` 工具执行 MyISAM 表的维护操作。（分区表不能用它，要用 `mysqlcheck`。）其中，包括检查和修复受损的表、显示表信息、分析索引键值的分布情况，以及禁用或启用索引。

MyISAM 存储引擎管理的表拥后缀分别为 `.MYD` 和 `.MYI` 的数据文件名和索引文件名。如果使用 `myisamchk` 来操作错误类型的表，它会显示一条警告信息，并忽略该表。

在调用 `myisamchk` 时，可以带上多个要检查的表的名字：

```
myisamchk [options] tbl_name[.MYI] ...
```

如果没有指定任何选项，`myisamchk` 会检查列出的那些表是否存在错误。如果指定了选项，它会根据指定选项的含义处理这些表。如果想要执行某个可能会修改表内容的操作，最好是能够先将其备份一下。

用于指定表的参数既可以是表名，`tbl_name`，也可以是该表的索引文件名，`tbl_name.MYI`。下面的命令是等效的：

```
% myisamchk member
```

```
% myisamchk member.MYI
```

如果命令解释器支持使用文件名模式，使用索引文件名会带来方便。如果要检查当前目录里的所有 MyISAM 表，可以使用一个通配符，让其指定所有的索引文件名：

```
% myisamchk *.MYI
```

`myisamchk` 程序不会假设表文件的存放位置。如果它们不在当前子目录里，则必须指定它们所在目录的路径名。因为表文件并未被假定存放在服务

器的数据目录里，所以你可以把它们复制到另一个目录里，然后再对那些副本进行操作，而不用对那些原始文件进行操作。

`myisamchk` 程序的许多操作也可以通过向服务器发送 SQL 语句来实现。这些语句包括：`ANALYZE TABLE`、`CHECK TABLE`、`OPTIMIZE TABLE`和`REPAIR TABLE`。你可以直接执行这些语句，也可以使用`mysqlcheck` 程序（它可以为多种SQL表维护语句提供命令行接口）。一般而言，使用这些语句或`mysqlcheck`会比使用`myisamchk`更简单、更安全。更多信息请参考14.6节。

当使用`myisamchk`维护表时，有一点需要注意，即必须防止服务器同时访问该表。之所以这样要求，是因为服务器和`myisamchk`都会直接访问表文件。如果允许它们同时访问表，可能会破坏表。在使用`myisamchk`之前，请记得查阅14.2节，其中讨论了如何防止服务器在`myisamchk`正在处理某个表时使用该表。

当使用`myisamchk`维护那些包含FULLTEXT索引的表时，如果满足以下两个条件，则也必须特别小心。

使用`myisamchk`执行某个修改索引的操作，其中包括分析和修复操作。

对于那些 FULLTEXT 相关的系统变量（如 `ft_max_word_len`、`ft_min_word_len` 或`ft_stopword_file`），运行的服务器使用的都是非默认值。

当上面这两个条件都具备时，必须使用合适的选项告诉 `myisamchk` 应该使用哪个FULLTEXT参数，因为它并不知道服务器正在使用哪些值。如果不这样做，`myisamchk`会使用与服务器期望值并不相同的参数值，并且FULLTEXT搜索会返回错误的结果。假设，对于最小单词长度和停用词文件，在运行服务器时使用了下面的非默认选项设置：

```
[mysqld]
```

```
ft_min_word_len=2
```

```
ft_stopword_file=/var/mysql/data/my-stopwords
```

在这种情况下，在对包含FULLTEXT 索引的表执行任何会修改索引的操作时，必须把这些值指定给`myisamchk`。可以通过在命令行使用`-ft_min_word_len`选项和`--ft_stopword_list`选项来指定。但是，最好的方法是将这些值记录在选项文件里，这样才不会忘记使用它们。选项组的使用方法与服务器上的使用方法相似：

```
[myisamchk]
```

```
ft_min_word_len=2
```

```
ft_stopword_file=/var/mysql/data/my-stopwords
```

在维护表时，为避免出现 FULLTEXT 参数匹配错误的问题，可以使用像 REPAIR TABLE 或 ANALYZE TABLE 这样的语句。接着，服务器会完成索引的修改操作，并且由于它知道自己在使用哪些 FULLTEXT 参数，因此它会把这些参数用到对那些包含 FULLTEXT 索引的表的维护操作上。

F. 3.1 myisamchk 支持的标准选项

```
--character-sets-dir  --help  --verbose  
  
--debug  --silent  --version
```

对这些选项的概要描述请参考 F. 2.1 节。

其中，--silent 选项表示的是只显示出错信息。--verbose 选项表示的是，当同时指定选项 --check、--description 或 --extend-check 时显示更多的信息。选项 --silent 和 --verbose 可以被多次指定，以加强效果。

标准选项 --help 输出的帮助信息会按选项功能进行分组。myisamchk 还支持 --HELP 和 --H 选项，用于按字母顺序显示所有的选项。

F. 3.2 myisamchk 特有的选项

有些选项需要用到索引号。索引从 1 开始编号。如果要确定某表的索引号，可以执行 SHOW INDEX 语句，或者 mysqlshow --keys 命令。输出结果里的 Key_name 列所列出的各个索引的顺序与 myisamchk 看到的顺序相同。

```
--analyze 或 -a
```

对键的分布进行分析。这有助于服务器更加快速地完成基于索引的查找和连接操作。如果要获得分析之后的键分布信息，可以带上选项 --description 和 --verbose 再次运行 myisamchk 程序。

```
--backup 或 -B
```

对于其他修改数据（.MYD）文件的选项，此选项会要求对这些文件进行备份，备份文件名字的形式为 tbl_name-time.BAK。其中，time 是一个表示时

间戳的数字。myisamchk会在表文件所在的目录里创建备份文件。

`--block-search=n`或`-b n`

从表的第n块开始显示1块的行。此选项仅用于调试。

`--check`或`-c`

检查表中的错误。当未指定任何选项时，这个将是默认动作。

`--check-only-changed`或`-C`

只有在表自上次检查以来从未被更改过的情况下，才会对其进行检查。

`--correct-checksum`

对于使用CHECKSUM=1选项创建的表，此选项可用于确保该表的校验和信息正确无误。

`--data-file-length=n`或`-D n`

它指的是重建已满的数据文件时容许增加到的最大长度（单位为字节）。（当文件达到MySQL或操作系统设置的大小上限时，通常会出现这种情况。当行数量达到内部表结构的大小限制时，同样会发生这种情况。）

此选项只有在与`--recover`或`--safe-recover`一起使用时才有效。

`--description`或`-d`

用于显示表的描述性信息。

`--extend-check`或`-e`

对表做进一步的检查。此选项很少会被使用到，因为myisamchk通常少量的检查便能将错误查找出来。

`--fast`或`-F`

只有在表未正常关闭的情况下，才对其进行检查。例如，在mysqld 打开许多表的同时，服务器主机机器出现崩溃，于是mysqld便没有机会关闭它们，这时便需要此选项。

`--force`或`-f`

即使表的临时文件已经存在，也强制对该表进行检查或修复。一般情况下，如果myisamchk发现已经存在有一个名为 tbl_name.TMD的文件，则它会在显示一条出错信息之后退出，因为这表明可能有另外一个myisamchk 程序实例正在运行。但是，文件可能也还存在，因为你在以前调用的myisamchk 仍在执行的时候，强行终止了它，在这种情况下，文件也可以安全移除。如果确认属于此种情况，可以使用--force选项告知myisamchk程序，即使临时文件已存在也要强制运行。（另一种做法是手动移除临时文件。）

如果你在检查表时使用了--force 选项，那么当 myisamchk 发现表有问题时，会带上--recover 选项自动重启。此外，myisamchk 还会更新该表的状态，跟--update-state选项所做的一样。

--information或-i

显示与表内容有关的统计信息。

--keys-used=n或-k n

此选项需要与--recover选项配合使用。其中，n是一个位掩码，它表明的是需要使用哪些索引。第1个索引对应的是第0位。（例如，6的二进制表示为110，表明应该使用第2个和第3个索引。）值为0时，表示禁用所有的索引，这样做可以改善INSERT、DELETE和UPDATE操作的性能。重新打开这些索引（即指定一个掩码，启用与每个索引相对应的那个二进制位），会恢复正常的索引行为。

--max-record-length=n

对于大于n个字节的行，如果不能为其分配内存，则将其忽略掉。

--medium-check或-m

使用一种比--extend-check更快的方法对表进行检查，只是检查不太彻底。这种检查模式能满足大部分的情况。中等检查模式的工作原理是：先计算索引键值的CRC值，然后将它们与数据文件的索引列所计算出来的CRC值进行比较。

--parallel-recover或-p

执行的恢复操作与--recover选项的一样，但是它会使用多线程并行重建索引。这种方式比使用非并行方式重建索引的速度更快，但是此选项目前处于试验阶段。

--quick或-q（布尔值）

当此选项与`--recover`配合使用时，可以达到比单独使用`--recover`选项更快的修复效果。当同时指定这两个选项时，数据文件并不会被更改。如果想在找到重复键值时强制修改数据文件，可以指定两个`--quick`选项。

`--read-only`或`-T`

此选项可以让表不被标记为已检查。

`--recover`或`-r`

对表进行常规的修复操作。除了只允许包含唯一值的索引里出现了重复值的情况无法处理以外，它可以修复大部分的问题。

`--safe-recover`或`-o`

此选项执行恢复操作的速度，比使用`--recover`的慢，但是它能够解决一些`--recover`选项无法解决的问题。此外，`--safe-recover`使用的磁盘空间也比`--recover`少。

`--set-auto-increment [=n]`或`-A[n]`

设置 `AUTO_INCREMENT` 计数器，使后续的序列编号值从 `n` 开始（如果表的各行里包含有`AUTO_INCREMENT` 值等于`n`的行，则从更大的值开始）。如果没有指定`n` 值，则此选项将把下一个`AUTO_INCREMENT`值，设置成一个比该表里存储的当前最大编号值还要大的值。

如果把`n`指定在`-A`之后，则它们之间不允许有空格；否则，该值将无法得到正确识别。

如果想在不用`myisamchk`的情况下设置MyISAM表的`AUTO_INCREMENT`值，可以执行下面形式的语句：

```
ALTER TABLE tbl_name AUTO_INCREMENT = n;
```

`--set-collation=collation`

在对表索引项进行重建和排序时，可以用此选项来指定排序规则。排序规则名隐含了所用的字符集名。

`--sort-index`或`-S`

对索引块进行排序以加快随后的检索操作所产生的顺序块的读取速度。

`--sort-records=n`或`-R n`

根据数据行在第 `n` 个索引中的顺序对它们进行排序。随后基于指定索引的检索操作应该会变得更快。当第一次对表执行这种操作时，它可能非常慢，因为各个行都还是无序的。`ALTER TABLE ... ORDER BY`所做的事情和`--sort-records`一样，而且在速度上通常会更快。

`--sort-recover`或`-n`

对表强制执行排序恢复，即使执行此操作所必需的临时文件可能会变得很大也一样执行。

`--start-check-pos=n`

从位置`n`开始读取数据文件。此选项仅用于调试操作。

`--tmpdir=dir_name`或`-t dir_name`

它用来指定存放临时文件的目录的路径名。此选项的默认值为环境变量`TMPDIR`的值；如果该变量未设置，则使用`/tmp`的值。此选项的值可以是一个目录列表，值将被循环使用。在Unix里，目录名之间要用冒号隔开；在Windows里，目录名之间要用分号隔开。

`--unpack`或`-u`

对`myisampack` 压缩后的表进行解压缩。此选项可以用于把压缩后的只读表转换为可修改形式。它不能与`--quick`或`--sort-records`一起使用。

`--update-state`或`-U`

对存储在表内部的状态标志进行更新。完好的表将被标志为一切正常，而对于有错误的表，则会被标记为需要修复。对于那些完好的表，利用此选项可以使随后带上`--check-only-changed`选项调用`myisamchk`的操作变得更加高效。

`--wait`或`-w`

如果表被锁定，则等待其变得可用为止。如果不使用`-wait`选项，`myisamchk`等待选项的时间是10秒，然后，如果还是未获得锁，则会显示一条出错信息。

F.3.3 `myisamchk`的变量

下面这些myisamchk变量都可以根据F. 2. 1. 2节的说明进行设置。

对于那些包含 FULLTEXT 索引的表，需要对 myisamchk 程序的开始描述部分里所讲述的注意事项多加注意。

`decode_bits`

在对压缩表进行解码时所使用的二进制位的个数。该值越大，解码操作也越快，但是会占用较多的内存。一般情况下，默认值9就能满足需要。

`ft_max_word_len`

包括在FULLTEXT索引里的单词的最大长度。更长的单词会被忽略掉。默认值为84。

`ft_min_word_len`

包括在FULLTEXT索引里的单词的最小长度。更短的单词会被忽略掉。默认值为4。

`ft_stopword_file`

用于FULLTEXT索引的停用字文件。默认使用内置的停用字列表。

`key_buffer_size`

索引块所使用的缓存区长度。（它只适用于--safe-recover，而不能用于--recover 或--sort-recover。）默认大小为512 KB。

`key_cache_block_size`

键缓冲的块大小。默认大小为1 MB。

`myisam_block_size`

.MYI文件里的索引块的块大小。默认大小为1 MB。

`read_buffer_size`

读取缓冲区的长度。默认大小为256 KB。

`sort_buffer_size`

键值排序操作所使用的缓冲区长度。（它只适用于`--recover`或`--sort-recover`，而不能用于`--safe-recover`。）默认大小为2 MB。

`sort_key_blocks`

此变量与表索引所使用的B-树结构的深度有关。值默认为16，通常不需要更改它。

`stats_method`

在计算索引键值的分布统计时，是否需要将NULL值当作相等。可设置的值有`nulls_equal`（所有的NULL 值在同一个组里）、`nulls_unequal`（每个NULL 值单独成为一组）或`nulls_ignored`（忽略NULL值）。

`write_buffer_size`

写缓冲区的长度。默认大小为256 KB。

F.4 mysql

客户端程序mysql可以用于连接服务器、执行SQL语句、查看查询结果。

`mysql [options] [db_name]`

如果指定了 `db_name` 参数，则该数据库会成为本次会话的默认数据库。如果未指定 `db_name`参数，则mysql在启动时将没有默认数据库，此时，你必须使用数据库名来限定所有的表引用，或者使用`USE db_name`语句来指定一个默认数据库。

mysql 可以交互运行。也可以将它用在批处理模式下，执行保存在文件里的语句，如果你将命令行的输入重定向到了该文件的话。例如：

```
% mysql -u sampadm -p -h cobra.snake.net sampdb < my_sql_file
```

在交互模式下，mysql 会显示一个mysql>提示符，以表明正等待输入。如果要执行语句，可以用键盘键入它（如有必要，可以跨多个行），然后，键入一个分号（;）或\g以表明该语句结束。mysql 会把该语句发送到服务器，然后显示结果，并再次显示提示符以表明正在等待下一条语句。\\G也可以用作语句结束标志，但是它使语句结果被“垂直”格式化（即每个输出行对应一个列值）。

不同的mysql 提示符表明等待输入的内容也会不一样，如下表所示。mysql>提示符是主提示符，它显示在每条语句的最开始。其他的提示符都是

辅助提示符，当需要获得当前语句的更多内容行时，它们会被显示出来。

提示符	含义
mysql>	等待输入新语句的第一行
->	等待输入当前语句的下一行
'>	在当前语句里等待输入完一个单引号字符串
">	在当前语句里等待输入完一个双引号字符串
`>	在当前语句里等待输入完一个引起来的标识符
/*>	等待输入完成一个/*...*/注释

其中，提示符'>和">表明的是，你在上一行输入了一个单引号或双引号字符串，但是尚未输入与之配对的结束引号。类似地，`>提示符表明的是一个还未最终引起的标识符。/*>提示符表明的是看到了一个用/*作为开头的/*...*/注释，但是还没有输入结尾的*/。一般情况下，当看到这些提示符时，很可能是因为你忘记了结束字符串、标识符或注释。如果属于此种情况，那么想要退出这种字符串收集模式时，需要根据提示符输入相应的引号或注释结束符，然后输入\c以取消当前语句。

在Unix里，当以交互模式使用mysql 时，它会将语句保存到一个历史文件里。默认情况下，此文件位于主目录里，名字为.mysql_history。你可以通过环境变量MYSQL_HISTFILE来显式指定它。对此文件的访问应该有所限制，以防止其他用户看到它的内容，请参考F. 2. 2. 1节。如果不想保存历史文件，可以删除.mysql_history 文件；然后，创建一个名为.mysql_history的指向/dev/null的符号链接，或者将MYSQL_HISTFILE设置/dev/null。

有些选项会禁止使用历史文件。一般情况下，那些表明以非交互方式使用mysql的选项（如--batch、--html和--quick）都会如此。

在支持输入编辑库的系统里，语句可以从命令历史里调出来再次执行，也可以进行编辑修改。在Windows平台上，mysql程序并没有提供行编辑功能。但是，Windows自己支持多种编辑命令，因此在mysql里也可以利用它们。与编辑命令有关的信息请参考1. 5. 2. 1节。

F. 4. 1 mysql支持的标准选项

--bind-address

--help

--silent

--character-sets-dir	--host	--socket
--compress	--password	--user
--debug	--pipe	--
verbose		
--debug-check	--port	--version
--debug-info	--protocol	
--default-character-set	--shared-memory-base-name	

对这些选项的概要描述请参考F. 2. 1节。mysql也支持F. 2. 1. 1节列出的那些标准SSL选项。指定多个--silent和--verbose选项，可以加强它们的效果。

-l等同于-help。

F. 4. 2 mysql特有的选项

--auto-rehash（布尔值）

在mysql启动时，它会对数据库名、表名和列名进行散列处理，目的是要构造一个可以快速完成这些名字的数据结构：当输入语句时，只要键入名字的开始部分，然后按一下Tab键，mysql便会自动完成这个名字（如果没有歧义的话）。再按一下Tab键，会看到各种可能的完成内容。默认情况下，名字散列处理的开关是打开的，但是只有选择了默认数据库之后，它才会起作用。使用--skip-auto-rehash选项可以禁止散列计算，如此一来可以使mysql的启动速度更快，尤其是在有很多表的时候。

如果禁用了散列计算，在启动mysql之后又想要启用名字自动完成功能，则可以在mysql>提示符处使用rehash命令。

--auto-vertical-output（布尔值）

对于超出最大宽度的查询结果，可以使用自动垂直输出样式。请参考--vertical。此选项是在MySQL 5. 5. 3里引入的。

--batch或-B

以批处理方式运行。mysql 会以制表符分隔的格式（每一个行都单独占用一个显示行，各个列值之间用制表符隔开）显示查询结果。这种格式的输出内容非常适合导入进其他的程序（如电子表格应用）做进一步的处理。默认情况下，查询结果会包含一个列标题作为第一行。如果想屏蔽掉这些标题，可以使用--skip-column-names选项。

----binary-mode（布尔值）

禁止将\r\n转换成\n，以及将\0解释成语句终止符。此选项主要适用于这样的情形：使用mysql来处理mysqlbinlog的输出（其中可能包含像BLOB值那样的任意数据）。当以非交互方式使用mysql时，此选项也会禁止各种命令（charset和delimiter除外）。此选项是在MySQL 5.6.3里引入的。

--column-names（布尔值）

把列名显示成查询结果里的列标题。使用--skip-column-names 可以禁止显示列名。通过两次指定--silent也可以达到同样的效果。

--column-type-info（布尔值）

在查询输出里包含结果集元数据。

--comments或-c（布尔值）

对于那些包含注释的语句，可以在把这些语句发送到服务器时包含注释。默认情况下，所有注释都会被去除掉（与指定--skip-comment的效果一样）。

--database=db_name或-D db_name

把db_name当作默认数据库。

--delimiter=str

设置语句分隔符。默认分隔符为分号（“;”）。

--execute=stmt或-e stmt

执行语句并退出。将语句用引号引起来，可以防止shell程序（命令解释器）将其当作多个命令行参数。如果要指定多条语句，可以在stmt值里使用分号将它们分隔开。

--force或-f（布尔值）

通常情况下，当mysql从文件读取语句时，如果发生错误，它会立即退出。此选项可以使mysql继续处理语句，而无视发生的错误。

`--html`或`-H`（布尔值）

生成HTML输出。

`--i-am-a-dummy`（布尔值）

此选项等同于`--safe-updates`。

`--ignore-spaces`或`-i`

让服务器忽略内置函数名与引入参数表的那个左括号“(”之间的空格。默认情况下，那个左括号必须紧跟在函数名的后面，其中间不允许有间隔。此选项还会使函数名被当作保留字。`--init-command=stmt`

指定一条在连接上MySQL服务器之后（如果启用了自动重连，则可能是在重新连接之后）执行的语句。与指定语句字符串有关的更多信息，请参考对`--execute`的描述。

`--line-numbers`（布尔值）

在出错信息里显示行号。此选项默认是打开的。如果想禁止行号，可以使用`--skip-line-numbers`选项。

`--local-infile`（布尔值）

启用或者禁用LOAD DATA LOCAL语句。LOCAL功能虽然已经存在，但默认是禁用的。如果LOAD DATA LOCAL语句导致了某个错误，则可以在带上`--local-infile`选项调用mysql之后进行重试。此选项还可以用于禁用LOCAL功能（如果它启用了的话），如使用选项`--disable-local-infile`。

如果服务器经过配置不允许使用LOCAL功能，则此选项将不会有任何效果。

`--named-commands`或`-G`（布尔值）

在所有输入行的开头部分启用长形式的mysql内部命令。如果使用`--skip-named-commands`选项禁用了此功能，则长格式命令只允许出现在主提示符的后面，而不允许出现在辅提示符的后面。也就是说，它们不能出现在多行语句的第二行及后面的行里。

`--no-auto-rehash`或`-A`

此选项等同于`--skip-auto-rehash`。请参考`--auto-rehash`的描述。

`--no-beep`或`-b`（布尔值）

当发生错误时不发出蜂鸣报警声。

`--one-database`或`-o`

当通过二进制日志文件更新数据库时可以使用此选项。它会告诉mysql只更新默认数据库（即命令行指定的数据库），并且忽略对其他数据库的更新。如果在命令行没有指定数据库，则不会更新任何内容。

`--pager [=program]`

它指定的是分页程序的名字（如`/bin/more` 或 `/bin/less`），该程序用于分页显示很长的查询结果。如果没有指定`program`，则mysql将根据环境变量`PAGER`的值来决定使用哪一个分页程序。输出分页功能在批处理模式下不可用，并且在Windows里也不可用。如果要禁用分页，可以使用`--skip-pager`选项。

`--prompt=str`

将主提示符从`mysql>`更改成`str`所定义的字符串。此字符串可以包含F. 4. 5节列出的各种特殊序列。

`--quick`或`-q`

通常情况下，mysql会先从服务器检索出整个结果集，然后再显示它。此选项会实现每检索一行便立即显示的效果，这样可以减少内存占用，同时也可能使某些大型的语句成功执行（如果不这样做则可能会失败）。不过，不应该在交互模式下使用此选项，因为如果用户暂停输出或者将mysql程序挂起，那么服务器会继续等待，这会影响到与其他客户端的交互。

`--raw`或`-r`（布尔值）

在输出列值时不对任何特殊字符做转义处理。此选项通常与`--batch`选项一起使用。

`--reconnect`（布尔值）

如果连接中断，则自动重新连接服务器。此选项默认是启用的。如果要禁用它，可以使用`--skip-reconnect`。

自动重新连接在有些情况下会造成麻烦。例如，当前所有处于激活状态的事务都会回滚，并且会话变量的值都会丢失，而且没有任何提示。

`--safe-updates`或`-U`（布尔值）

此选项会对某些操作施加一些限制，并且能为MySQL新手带来好处：

只有通过键值识别出被修改的各个行之后，或者只有使用了LIMIT子句之后，才允许进行更新操作（那些修改数据的语句）。这样有助于防止语句意外地更改或删除全部（或大部分）的表内容。

除非使用了 LIMIT 子句，否则通过非连接检索方式产生的结果集会被限制在1 000行以内。对于涉及连接的检索，如果优化程序判断到它检查的行可能会超过100万个，那么它会拒绝该操作。这样有助于防止意外生成过大的查询结果。

这些限制可以通过设置变量`select_limit`和`max_join_size`来进行修改。

`--secure-auth`（布尔值）

禁止连接到服务器，除非它支持在MySQL 4.1里引入的密码散列算法。此选项自MySQL 5.6.7开始默认是启用的，而在之前的版本里默认是禁用的。

`--show-warnings`（布尔值）

自动显示所有警告（就好像是在每条语句之后执行SHOW WARNINGS一样）。

`--sigint-ignore`（布尔值）

此选项用于忽略SIGINT信号，信号通常是通过键入Ctrl+C发送的。Ctrl+C会告知mysql终止当前语句。如果该语句无法被终止，或者在该语句被终止之前又输入了一个Ctrl+C，则mysql会退出。使用`--sigint-ignore`可以防止mysql像刚才描述的那样解释Ctrl+C。

`--skip-column-names`或`-N`

请参考`--column-names`的描述。

`--skip-line-numbers`或`-L`

请参考--line-numbers的描述。

--table或-t（布尔值）

以表格形式生成输出，每一个行里的各个值都以表格线进行分隔，且垂直对齐。当mysql不以批处理模式运行时，这是其默认输出格式。

--tee=file_name

将全部输出信息的副本追加到指定文件中。如果要禁止输出复本，可以使用--skiptee选项。此选项在批处理模式下不起作用。

--unbuffered或-n（布尔值）

在每一条语句执行完成之后，便刷新用于与服务器进行通信的缓冲区。

--vertical或-E

按垂直方式显示查询结果，即将查询结果中的每一行（row）都格式化为一个输出行（line），其中每输出行对应一列。（每一输出行由列名和值组成）。显示的每个行的前面都带有一个输出行，用以指明结果集里的行号。当某个查询产生的输出行很长时，垂直显示格式可能会很有用。如果不指定此选项，也可以针对各个查询启用垂直显示格式，方法是使用\G（而不是使用“;”或“\g”）作为查询语句的结束符。请参考--auto-vertical-output。

--wait或-w

如果无法与服务器建立连接，则等待并重试。

--xml或-X（布尔值）

生成XML输出。

F. 4. 3 mysql的变量

下面这些mysql变量都可以根据F. 2. 1. 2节的说明进行设置。

connect_timeout

它指的是，当尝试与服务器建立连接时，在超时之前需要等待的秒数。默认值为0。

`max_allowed_packet`

它指的是服务器与客户端之间通信时使用的缓冲区的最大大小。默认值为16 MB，最大值为1 GB。

`max_join_size`

它指的是，当指定`--safe-updates`选项时，执行连接操作所能产生的最大行数。对于那些服务器认为会需要检查超过`max_join_size`个行的连接操作，它会拒绝执行。默认值为1 000 000。

`net_buffer_length`

它指的是服务器与客户端之间通信时使用的缓冲区的初始大小。此缓冲区可以扩张到`max_allowed_packet`个字节大小。默认大小为16 KB。

`select_limit`

如果指定了`--safe-updates`选项，则它指的是SELECT语句返回的最大行数。默认值为1 000。

F.4.4 mysql命令

除了允许向MySQL服务器发送SQL语句之外，mysql还实现了多个供自己使用的命令。其中，每一个命令都必须在独立行里给出。大部分的命令都有由一个单词构成的长格式形式和一个由反斜线加单个字母构成的短格式形式。长格式的命令不区分大小写。短格式的命令则必须以下面列表所示的大小写形式进行指定。对于长格式命令，行末尾的分号不是必须的，但允许加上；但是对于短格式命令，则应该将其忽略掉。

如果禁用命名命令（例如，使用`--disable-named-commands`选项），则只有出现在主提示符`mysql>`里的长格式命令才能被识别出来。如果以非交互式方式使用mysql，并且在调用它时带有`--binary-mode`选项，则除了`charset`和`delimiter`以外的所有命令都会被禁用。

`clear`或`\c`

清除（取消）当前语句（即你正在键入的那条语句）。此命令无法取消已发送给服务器以及mysql正在显示其输出的语句。

`charset [charset]`或`\C [charset]`

更改默认字符集，并向服务器发送一条SET NAMES语句，以便服务器也使用该字符集。

`connect [db_name [host_name]]或\r [db_name [host_name]]`

连接（或者再次连接）指定主机上的指定数据库。如果省略了数据库名或主机名，则会使用当前mysql会话中最近一次用过的那个值。

`delimiter str或\d str`

设置语句分隔符。默认分隔符为分号（“;”）。存储程序分析器只能把分号识别为分隔符，因此在定义存储程序时此命令可用于为mysql重新定义分隔符。相关示例请参考4.2.1节。

在分隔符里最好避免使用反斜线，因为MySQL会把反斜线当作转义字符。

`edit或\e`

编辑当前语句。mysql会检查环境变量EDITOR和VISUAL来确定使用哪一个编辑器。如果这两个变量都没有设置，mysql会使用vi编辑器。此选项在Windows里不可用。

`ego或\G`

把当前语句发送到服务器，并按垂直格式显示查询结果。

`exit`

等同于quit命令。

`go 、 \g或;`

把当前语句发送到服务器，并显示结果。

`help [ topic ]、 \h [ topic ]、 ? [ topic ]或\? [ topic ]`

显示帮助信息，其中描述的是可用的mysql命令。

如果mysql数据库里的帮助表已被加载，你就可以用help来获得服务器端帮助：使用help contents可以获得帮助类别列表；使用help category可以获得特定类别的帮助内容；使用help keyword 可以获得与特定关键字（如SELECT 或UPDATE）有关的帮助内容。与帮助表加载有关的说明请参考A.3.6节。

`nopager`或`\n`

禁止分页显示，并把输出内容发送到标准输出设备。此命令在Windows里不可用。

`notee`或`\t`

停止将输出内容写到外部文件。

`nowarning`或`\w`

停止自动显示每条语句所生成的任何警告信息。

`pager [program]`或`\P [program]`

把输出内容发送给通过`program`或通过环境变量`PAGER`（如果设置了此变量，并且没有给定`program`）指定的分页显示程序。此命令在Windows里不可用。

`print`或`\p`

显示当前语句（即语句自身的文本内容，而非执行它后得到的结果）。

`prompt [arguments]`或`\R [arguments]`

重新定义主提示符`mysql>`。从`prompt`关键字后面第一个空格开始，所有的内容都将成为提示符字符串的一部分，包括其他空格。此字符串可以包含特殊序列（参考 F.4.5 节）。如果想把提示符恢复为默认值，可以运行不带任何参数的`prompt`或`\R`命令。

`quit`或`\q`

退出`mysql`程序。

`rehash`或`\#`

重新计算自动完成数据库名、表名和列名所需要的信息。请参考`--auto-rehash`选项有关的描述。

`source file_name`或`\. file_name`

读取并执行包含在指定文件里的语句。对于包含反斜线（“\”）路径名分隔符的Windows文件名，必须双写反斜线或者使用斜线字符（“/”）来代替。

`status`或`\s`

检索并显示服务器的状态信息。此命令可用于检查服务器版本、默认数据库、SSL密码等。

`system command`或`\! command`

用默认命令解释器执行`command`命令。此命令在Windows里不可用。

`tee [file_name]`或`\T [file_name]`

把输出内容复制到指定文件的末尾。如果省略了`file_name`，则使用上一个文件。

`use db_name`或`\u db_name`

选择指定数据库作为默认数据库。

`warnings`或`\W`

自动显示每条语句所生成的任何警告信息。

F.4.5 mysql提示符定义序列

环境变量MYSQL_PS1、--prompt选项或prompt命令都可用于重定义mysql输出的主提示符mysql>。例如，想要在提示符里包含默认数据库的名字，可以使用下面的prompt命令，然后，选择不同的数据库，看看提示符是如何随当前选择一起变化的：

```
% mysql

mysql> prompt \d>\_

PROMPT set to '\d>\_'

(none)> USE sampdb;

Database changed

sampdb> USE test;

Database changed

test>
```

紧跟在prompt 关键字后面的是提示符定义字符串。在提示符定义里，以反斜线开头的转义序列表示的是特殊的提示符选项。序列\d和_分别代表的是默认数据库名和空格。（如果使用环境变量或--prompt选项来设置提示符，则需要在指定提示符字符串时双写反斜线字符。）下表列出的是各种可用的选项。

序列	含义
\c	当前输入行号
\d	默认数据库名，如果未选择数据库，则为“(none)”
\D	完整的日期和时间
\h	当前主机
\l	当前分隔符
\m	分钟

序列	含义
\o	月份
\O	月份名称, 3个字母
\p	当前端口号、套接字、命名管道名或共享内存名
\P	时间值的am或pm标志
\r	小时 (12小时制)
\R	小时 (24小时制)
\s	秒
\S	分号
\t	制表符
\u	当前用户名, 不带主机名
\U	当前用户名, 带主机名
\v	服务器版本
\w	星期名字, 3个字母
\Y	年 (2位数字)
\Y	年 (4位数字)
\'	单引号
\"	双引号
_	空格符
\	空格符 (这个序列是一个反斜线加一个空格)
\\	反斜线
\n	换行
\x	字符x (x代表的是任何未在上面列出的字符)

F.5 mysql.server

mysql.server可以通过调用mysql_safe脚本来启动和关闭mysqld服务器。mysql.server是一个shell脚本, 并且只有在Unix系统里才可用。

mysql.server 支持的命令行参数包括start和stop:

```
mysql.server start
```

```
mysql.server stop
```

通常情况下，mysql.server脚本安装在Unix系统的某个运行级目录里（位于/etc目录里）。（对于安装后的版本，其名字通常为mysql，而非mysql.server。）系统在启动时，带上start参数调用该脚本便可以启动服务器。系统在关机时，带上stop参数来调用该脚本便可以关闭服务器。此脚本也可以通过手动方式带上相应的参数来启动或关闭服务器。

mysql.server支持的选项

mysql.server 脚本对标准MySQL选项的支持非常有限。它不能从命令行读取任何标准的选项。在选项文件中的[mysql_server]和[mysql.server]组里，它可以读取basedir、datadir和pid_file选项，并把它们传递给mysql_safe。它还可以读取service-startup-timeout，其值为一个数值，表示的是该脚本等待服务器启动的时间（单位为秒）。默认值为900。当值为0时，表示的是“不等待”；为负值时，表示的是“永远等待”。

F.6 mysql_config

mysql_config工具能为开发使用C语言编写的MySQL应用程序带来好处。可以通过调用它来获得各种正确的标志，这些标志是编译C语言源文件或链接MySQL开发库所需要的：

```
mysql_config [options]
```

相关示例请参考7.1节。

mysql_config特有的选项

```
--cflags
```

显示访问MySQL头文件所必需的那些包含目录（include-directory）标志，以及其他可能会需要的C语言编译器标志。

```
--cxxflags
```

与--cflags相似，但是它是用于C++编译器的。此选项是在MySQL 5.6.4里引入的。

`--embedded`或`--embedded-libs`

这两个选项等同于`--libmysqld-libs`选项。

`--include`

显示访问MySQL头文件所必需的那些包含目录标志。

`--libmysqld-libs`

显示连接`libmysqld`（嵌入式服务器开发库）所必需的那些库标志。

`--libs`

显示连接客户端开发库所必需的那些库标志。

`--libs_r`

显示连接线程安全的客户端开发库所必需的那些库标志。

`--plugindir`

显示默认的插件目录。

`--port`

显示默认的TCP/IP端口号。

`--socket`

显示默认的Unix套接字文件路径名。

`--variable=var_name`

显示`var_name`的值（可能的值有`pkgincludedir`、`pkglibdir`或`plugindir`）。

`--version`

显示MySQL版本字符串。

F.7 `mysql_install_db`

`mysql_install_db` 脚本可以创建服务器的数据目录、初始化包含权限表的mysql 数据库，以及创建空的test数据库：

```
mysql_install_db [options]
```

`mysql_install_db`脚本将使用root和匿名用户的初始账户来填写权限表。参考第12章可以获得更多详细的与这些账户有关的信息，以及有关如何通过建立密码来加强数据库安装版本安全的信息。

`mysql_install_db` 脚本在Windows系统里不可用的，也不需要它，因为Windows发行版本包含有一个预初始化的数据目录。

F. 7. 1 `mysql_install_db`支持的标准选项

```
--help          --user          --verbose
```

对这些选项的概要描述请参考F. 2. 1节。

在Unix 系统里，`--user` 选项的作用是使用给定用户的登录账户来运行服务器。如果以 Unix系统的root用户身份运行`mysql_install_db`脚本，可以确保服务器创建的所有目录和文件都隶属于该用户。

F. 7. 2 `mysql_install_db`特有的选项

你可以在命令行上使用本节提到的各个选项，并且可以通过在选项文件的[mysql]组里放置合适的条目来设置它们的值。此脚本也会读取[mysql_install_db]选项组，这一点对于那些只能被`mysql_install_db`脚本所理解（而无法被mysql程序所理解）的选项（如`--ldata`和`--force`）特别有用。

`mysql_install_db`脚本会将所有无法识别的选项传递给mysql。

```
--basedir=dir_name
```

MySQL基本目录的路径名。

```
--datadir=dir_name或--ldata=dir_name
```

MySQL数据目录的路径名。

```
--force
```

即使无法确定当前主机名，也要强制运行。此时，主机的IP地址会被用于创建权限表里的记录项。这意味着，如果要使用客户端程序，则需要指定IP地址，而不能用主机名（localhost除外）。

`--skip-name-resolve`

在权限表里只使用IP地址，而不使用主机名。如果没有可用的DNS服务器，则此选项非常有用。

F.8 mysql_upgrade

此程序应该在将MySQL升级到较新版本之后使用。默认情况下，它会以MySQL的root用户身份连接到本地服务器，因此在调用它时需要带上root的密码：

```
% mysql_upgrade --password=rootpass
```

mysql_upgrade程序会检查MySQL安装的数据库，找出与新版本不兼容的地方，并尽可能修复它。其中包括升级mysql数据库里的表（如处理新的权限）和修复那些包含不正确格式数据（可能会导致新版服务器出现问题）的表。在运行 mysql_upgrade 之后，需要重新启动服务器，以便让所有的更改生效。

mysql_upgrade 程序会调用mysqlcheck 来执行检查和修复操作。有关其功能的更多信息请参考该程序的描述。

F.8.1 mysql_upgrade支持的标准选项

<code>--character-sets-dir</code>	<code>--default-character-set</code>	<code>--port</code>
<code>--compress</code>	<code>--help</code>	<code>--</code>
protocol		
<code>--debug</code>	<code>--host</code>	<code>--</code>
shared-memory-base-name		
<code>--debug-check</code>	<code>--password</code>	<code>--socket</code>
<code>--debug-info</code>	<code>--pipe</code>	<code>--user</code>
<code>--default-auth</code>	<code>--plugin-dir</code>	<code>--verbose</code>

对这些选项的概要描述请参考F. 2. 1节。mysql_upgrade也支持F. 2. 1. 1节列出的那些标准SSL选项。

与大部分MySQL程序不同，--user的默认值为root。--verbose选项默认是启用的。使用--skip-verbose可以禁用它。

F. 8. 2 mysql_upgrade特有的选项

--basedir=dir_name或-b dir_name此选项未使用。

--datadir=dir_name或-d dir_name

此选项未使用。

--force或-f（布尔值）

对于当前版本的MySQL，即使已执行过mysql_upgrade，也还是要强行升级。

--tmpdir=dir_name或-t dir_name

它用来指定存放临时文件的目录路径名。

--upgrade-system-tables或-s（布尔值）

只升级mysql数据库里的表。

--write-binlog（布尔值）

将mysql_upgrade执行过的SQL语句写入二进制日志。使用此选项会导致语句被复制。

F. 9 mysqladmin

mysqladmin程序通过与服务器的通信可以完成各种管理操作。你可以利用mysqladmin来获取服务器信息、控制其操作、设置密码，以及创建或删除数据库：

mysqladmin [options] command ...

F. 9. 1 mysqladmin支持的标准选项

--bind-address

--default-character-set

--protocol

--character-sets-dir	--help	--shared-memory-base-name
--compress	--host	--silent
--debug	--password	--socket
--debug-check	--pipe	--user
--debug-info	--plugin-dir	--verbose
--default-auth	--port	--version

对这些选项的概要描述请参考F. 2. 1节。mysqladmin 也支持F. 2. 1. 1节列出的那些标准SSL选项。

使用--silent选项之后，如果mysqladmin无法连接到服务器，它会静默地退出。在有些命令里，--verbose选项可以使mysqladmin输出更多的信息。

F. 9. 2 mysqladmin特有的选项

--count=n或-c n

在指定了--sleep 选项时，它表示的是迭代次数。如果指定了--sleep，但是没有指定--count，则mysqladmin将一直迭代下去（除非你中断它）。

--force或-f（布尔值）

此选项有两个作用。首先，它会导致mysqladmin在执行drop db_name命令时不会进行确认。其次，当在命令行上同时指定多条命令时，即使出现了错误，mysqladmin 也会尝试执行每一条命令。正常情况下，mysqladmin会在第一个错误之后退出。

--no-beep或-b（布尔值）

当错误发生时不发出蜂鸣报警声。

--relative或-r（布尔值）

与--sleep 选项一起使用，显示当前值与上一个值之间的差异。此选项只能用于extend-status命令。

`--sleep=n`或`-i n`

每隔n秒重复执行一次命令行里指定的命令。

`--vertical`或`-E`（布尔值）

此选项与`--relative`相似，只是输出内容按垂直格式显示。

`--wait[=n]`或`-w[n]`

如果无法与服务器建立连接，则等待一定时间后重试。如果未指定具体值，则 n 的默认值为1。如果使用`-w`来指定n值，则它们之间不允许有空格；否则，该值将无法正确解释。

F. 9. 3 mysqladmin的变量

下面这些mysqladmin变量都可以根据F. 2. 1. 2节的说明进行设置。

`connect_timeout`

它指的是，当尝试与服务器建立连接时，在超时之前需要等待的秒数。默认值为43 200。

`shutdown_timeout`

对于shutdown命令，它表示的是等待成功关闭服务器的秒数。默认值为3 600。

F. 9. 4 mysqladmin命令

在命令行的任何选项后面，都可以指定一个或者多个下列命令。其中，每条命令的名字都可以被缩短为无歧义的前缀形式。例如，`processlist`命令可以缩短为`process`或`proc`，但不能简写为`p`。

其中一些命令有着与之等效的SQL语句，相关的描述里会特别注明。与这些语句含义相关的信息请参考附录E。

`create db_name`

使用给定的名字创建一个新数据库。此命令与`CREATE DATABASE db_name`语句相似。

`debug`

让服务器将调试信息转储到错误日志中。

`drop db_name`

删除指定的数据库以及该数据库里的所有表。如果没有给定`--force` 选项，则mysqladmin程序在执行这些命令时会要求进行确认。此命令与`DROP DATABASE db_name`语句相似。

`extended-status`

显示服务器状态变量的名字和值。此命令与`SHOW STATUS`语句相似。

`flush-hosts`

刷新主机缓存。此命令与`FLUSH HOSTS`语句相似。

`flush-logs`

刷新（关闭后再打开）日志文件。此命令与`FLUSH LOGS`语句相似。

`flush-privileges`

重新加载权限表。此命令与`FLUSH PRIVILEGES`语句相似。

`flush-status`

清理状态变量。（这会将多个计数器重置为0。）此命令与`FLUSH STATUS`语句相似。

`flush-tables`

刷新表缓存。此命令与`FLUSH TABLES`语句相似。

`flush-threads`

刷新线程缓存。

`kill id, id, ...`

终止指定标识号的服务器线程。如果同时指定多个标识号，则它们之间将不允许有任何空格，以免与下面的另一个`kill`命令相混淆。使用`mysqladmin processlist`命令可以看到当前正在运行的那些线程的信息。此命令与针对各个线程ID调用`KILL`语句相似。

`old-password new_password`

此命令与password命令相似，但是它会密码保存为MySQL 4.1之前使用的密码散列格式。

`password new_password`

更改你连接服务器时服务器授权给你的那个账户的密码。（能够使用此账户连接到服务器，即说明

你已知道当前密码。）新密码将被设置为new-password。此命令和SET PASSWORD语句相似。在Unix里，如果密码包含了命令解释器认为特殊的字符，你可以在mysqladmin命令里用单引号或双引号把密码引起来。在Windows里，应该只能使用双引号。Windows命令解释器不会把单引号识别为参数引用字符，因此如果使用了单引号，那么它们将成为密码的一部分！自MySQL 5.5.3起，如果password是命令行里的最后一个命令，则new_password值是可选的，如果它不存在，mysqladmin会提示输入它。

`ping`

检查MySQL服务器是否正在运行。

`processlist`

以列表形式显示当前的服务器活动。此命令与 SHOW PROCESSLIST 语句相似。如果带上--verbose选项，则与SHOW FULL PROCESSLIST语句相似。

`refresh`

刷新表缓存和各权限表，并且关闭和重新打开日志文件。如果服务器是复制机制中的主服务器，则此命令将告知它删除在二进制日志索引文件里列出的二进制日志文件，并把其索引截短为0。如果服务器是从服务器，则此命令将告知它忘记自己在主机日志里的位置。

`reload`

重新加载权限表。此命令与FLUSH PRIVILEGES语句相似。

`shutdown`

关闭MySQL服务器。

`start-slave`

启动复制从服务器。此命令与START SLAVE语句相似。

`status`

按简短格式显示服务器的状态信息。

`stop_slave`

停止复制从服务器。此命令与STOP SLAVE语句相似。

`variables`

显示服务器各变量的名字和值。此命令与SHOW GLOBAL VARIABLES语句相似。

`version`

检索并显示服务器的版本信息串。它的返回信息与SQL函数VERSION()的相同。

F.10 mysqlbinlog

mysqlbinlog程序将以可读格式显示二进制日志文件的内容：

`mysqlbinlog [options] file_name ...`

默认情况下，mysqlbinlog可以直接读取本地日志文件，不用连接服务器。它也可能会连接到服务器，并要求它通过连接把日志文件发送过来。更多信息请参考--read-from-remote-server选项的描述。

二进制日志的格式一直处于不断变化当中。为避免兼容性问题，请尽可能使用最靠近服务器版本的mysqlbinlog版本。

mysqlbinlog还可以读取复制从服务器创建的中继日志文件，因为中继日志的格式和二进制日志的相同。

F.10.1 mysqlbinlog支持的标准选项

<code>--bind-address</code>	<code>--help</code>	<code>--shared-memory-base-name</code>
<code>--character-sets-dir</code>	<code>--host</code>	<code>--socket</code>

--debug	--password	--user
--debug-check	--plugin-dir	--verbose
--debug-info	--port	--version
--default-auth	--protocol	

对这些选项的概要描述请参考F. 2. 1节。

--verbose选项会使mysqlbinlog把行事件显示为注释掉的SQL语句。如果指定两次，则此选项还会生成描述列元数据的注释。

F. 10.2 mysqlbinlog特有的选项

--base64-output=value

控制如何以base-64编码格式将输出显示为BINLOG语句。这些语句通常用于那些无法用SQL来描述的行事件。允许的选项值如下所示。

AUTO或UNSPEC：在必要时自动使用BINLOG语句。

NEVER：不生成 BINLOG 语句；如果遇到无法显示的行事件，则在退出时会显示错误，而不会生成BINLOG语句。

DECODE_ROWS：在使用时如果带上了--verbose，则此值与 NEVER 相似，不同之处在于退出时不会显示错误。

ALWAYS：总是尽可能地显示BINLOG语句。自MySQL 5.6.1开始，此选项值不再可用。

这些选项值不区分大小写。如果没有指定此选项，则默认为 AUTO。如果计划再次执行mysqlbinlog的输出，则只有这个值是安全的。其他值应该仅作调试或测试之用。

--binlog-row-event-max-size=n

基于行的事件所允许的最大大小。mysqlbinlog会尝试把行分组成不会超过这个值的事件。该值会被截断成最靠近256倍数的某个非零值。默认大小为4 GB。此选项是在MySQL 5.6.0里引入的。

--database=db_name或-d db_name

从日志文件里只提取与指定数据库有关的语句。此选项只在读取本地日志时有效。

`--disable-log-bin`或`-D`（布尔值）

在输出里包含这样的语句——它们可以在日志里禁用数据更新语句的二进制日志记录功能。这样可以防止这些语句在再次执行时又被记录到二进制日志里。

`--force-if-open`或`-F`（布尔值）

即使那些二进制日志文件未正确关闭（或正被使用当中），也要强行读取它们。此选项默认是启用的。使用`--skip-force-if-open`可以禁用它。

`--force-read`或`-f`（布尔值）

此选项控制着mysqlbinlog在从二进制日志里读取到无法识别的事件时将采取的具体行动。默认的情况下，它会停止。如果启用了此选项，则mysqlbinlog将在记录警告信息并丢弃该事件之后继续执行。

`--hexdump`或`-H`（布尔值）

在输出里包含十六进制/ASCII格式的事件转储内容。

`--local-load=dir_name` 或 `-l dir_name`

为创建处理LOAD DATA LOCAL语句所需要的临时文件而创建的目录。mysqlbinlog并不会删除这些文件，因为你不可能再次立即执行其输出，并且在再次执行时还需要这些文件。不再需要它们时可以手动删除它们。

`--offset=n`或`-o n`

跳过日志文件中的前n个事件。

`--raw`（布尔值）

此选项需要与`--read-from-remote-server`一起使用，以便从指定服务器请求二进制日志文件，数据格式为它们原始的二进制格式，而非默认的文本格式。这种做法的一种用途是备

份服务器的二进制日志文件。（在这种情况下，可以使用`--result-file`来控制输出文件的命名。）此选项是在MySQL 5.6.0里引入的。

`--read-from-remote-server`或`-R`（布尔值）

通过与服务器建立网络连接来读取二进制日志文件，并要求它通过该连接发送这些日志。为此，需要使用`--read-from-remote-server` 选项，并且根据需要使用`--host`、`--password`、`--protocol`、`--socket`和`--user` 选项来指定连接参数。如果没有给出`--read-from-remote-server`选项，则这些选项将被忽略。

`--result-file=file_name`或`-r file_name`

如果未指定`--raw`，则将输出内容写到指定的文件。在指定`--raw` 时，这个选项表示的是如何指定服务器传输过来的那些二进制日志文件的本地副本。此时，输出文件的名称与主机主日志文件的名称相同，只是会带上一个`--result-file`的值所指定的前缀。该值可以从需要写入这些文件的目录开始。例如，对于 2012-01-28 制作的备份文件，便可以使用`--result-file=2012-01-28-`来进行标明，并且这些输出文件会存在当前目录里，而它们的文件名则是以该日期开头。如果想要把文件写到`/var/backup` 目录里，可以使用`--resultfile=/var/backup/2012-01-28-`。

`--server-id=n`

只转储由服务器创建的指定ID的事件。

`--set-charset=charset`

在输出里包含一条SET NAMES语句。

`--short-form`或`-s`

只显示日志里记载的语句；会忽略日志里的那些与语句有关的其他信息，并且不显示基于行的事件。

`--start-datetime=date_time`

开始读取发生时间等于或晚于指定时间 `date_time`的那些二进制日志事件。必须用运行mysqlbinlog的那台主机所在的时区里的合法DATETIME格式来指定`date_time`的值。根据命令解释器的需要，决定是否需要将这个值引起来。

`--start-position=n`或`-j n`

从命令行上指定的第一个日志文件里的指定位置开始读取二进制日志事件。

`--stop-datetime=date_time`

停止读取发生时间等于或晚于指定时间 `date_time` 的那些二进制日志事件。必须用运行mysqlbinlog的那台主机所在的时区里的合法DATETIME格式来指定`date_time`的值。根据命令解释器的需要，决定是否需要将这个值引起来。

`--stop-never`（布尔值）

此选项与`--read-from-remote-server`一起使用时，可以保持与服务器的连接。它可以用来按服务器写入日志事件的方式显示这些事件；或者与`--raw`一起使用，实现连续的实时日志备份。此选项是在MySQL 5.6.0里引入的。

`--stop-never-slave-server-id=server_id`

当使用`--stop-never`时，报告给服务器的服务器ID。默认ID为65535，但是如果有从服务器或mysqlbinlog的另一个实例正在使用该ID，则需要显式地指定一个ID。此选项是在MySQL 5.6.0里引入的。

`--stop-position=n`

在命令行上指定的最后一个日志文件里的指定位置，停止读取二进制日志事件。

`--to-last-log`或`-R`（布尔值）

当从服务器读取日志文件时（需要`--read-from-remote-server`选项），此选项会使程序一直读到服务器的最后一个二进制日志文件，而不是在命令行上指定的最后一个日志文件的末尾。使用`--to-last-log`选项，可以确保获得服务器上的所有二进制日志信息。（不过，如果正在把这些事件发送到同一个服务器进行处理，那么此选项将导致一个无限循环操作。）

`--verify-binlog-checksum`或`-c`（布尔值）

在事件里启用校验和验证功能。此选项是在MySQL 5.6.1里引入的。

F.10.3 mysqlbinlog的变量

下面的mysqlbinlog变量可以根据F.2.1.2节的说明进行设置。

`open_files_limit`

保留的文件描述符个数。默认值是64。

F. 11 mysqlcheck

mysqlcheck是一个用于检查和修复表的客户端程序。它为管理性的CHECK TABLE、ANALYZE TABLE、OPTIMIZE TABLE和REPAIR TABLE语句提供了一个命令行接口。

所有的myisamchk选项都适用于MyISAM表。mysqlcheck程序也可以对InnoDB表进行检查和分析。

mysqlcheck有三种运行模式：

```
mysqlcheck [options] db_name [tbl_name] ...
```

```
mysqlcheck [options] --databases db_name ...
```

```
mysqlcheck [options] --all-databases
```

在第一种模式里，mysqlcheck会检查指定数据库里的指定表。如果没有指定表，则mysqlcheck会检查数据库里的所有表。在第二种模式里，mysqlcheck 程序会把所有的参数都当作数据库名，并依次检查各个数据库里的所有表。在第三种模式里，mysqlcheck 会将依次检查所有数据库里的所有表。

F. 11.1 mysqlcheck支持的标准选项

--bind-address	--default-character-set	--protocol
--character-sets-dir	--help	--shared-memory-base-name
--compress	--host	--silent
--debug socket	--password	--
--debug-check	--pipe	--user
--debug-info	--plugin-dir	--verbose
--default-auth	--port	--version

对这些选项的概要描述请参考F. 2. 1节。mysqlcheck也支持F. 2. 1. 1节列出的那些标准SSL选项。

F. 11.2 mysqlcheck特有的选项

下列这些mysqlcheck支持的选项可用于控制它如何处理表。在这个选项列表之后，你将会看到一段描述，其中介绍的是这些选项与相应的SQL语句之间的等价关系。

`--all-databases`或`-A`（布尔值）

处理所有数据库里的所有表。

`--analyze`或`-a`

通过一条 `ANALYZE TABLE` 语句来完成表的分析。（例如，分析键值的分布情况。）分析结果有助于查询优化器快速完成基于索引的查找和连接操作。

`--all-in-1`或`-1`（布尔值）

如果没有使用此选项，则mysqlcheck将针对每个表单独执行语句。此选项会使mysqlcheck将按数据库对表进行分组，并在单条语句里列出每个数据库的所有表。

`--auto-repair`（布尔值）

如果检查发现表存在问题，则在检查阶段完成之后进行第二阶段——修复它们。

`--check`或`-c`

执行`CHECK TABLE`语句，进行错误检查。如果没有显式指定动作，这个将是默认动作。

`--check-only-changed`或`-C`

只检查那些自从上次检查以来发生改变或者未正确关闭的表。

`--check-upgrade`或`-g`

检查表是否与当前的MySQL版本兼容。如果使用了`--auto-repair` 选项，在发现数据不兼容时，会尝试自动修复。此选项会启用`--fix-db-names`和`--`

fix-table-names选项。

--databases或-B（布尔值）

把所有参数都解释为数据库的名字，并检查各个数据库里的所有表。

--extended或-e（布尔值）

对表做进一步的检查。如果与--repair 一起使用，则mysqlcheck 会使用一种比单独使用--repair选项更全面但也更加缓慢的修复方法。

--fast或-F（布尔值）

只有在表未正常关闭的情况下，才对其进行检查。

--fix-db-names（布尔值）

检查数据库的名字，并根据MySQL 5.0和5.1版本之间的名字编码差异逐个进行转换。

--fix-table-names（布尔值）

检查表和视图的名字，并根据MySQL 5.0和5.1版本之间的名字编码差异逐个进行转换。

--force或-f（布尔值）

即使出现了错误也要继续执行。

--medium-check或-m

使用一种比--extended 更快（但是没那么彻底）的方法对表进行检查。这种检查模式能满足大部分的情况。

--optimize或-o

通过执行OPTIMIZE TABLE语句来完成表的优化。

--quick或-q（布尔值）

对于表检查操作，此选项将跳过数据行之间的链接检查。如果与--repair选项一起使用，此选项将会只修复索引文件，而不会对数据文件进行更改。

`--repair`或`-r`

通过执行REPAIR TABLE语句来完成表的修复。除了只允许包含唯一值的索引里出现了重复值的情况无法处理以外，这种修复模式可以修复大部分的问题。

`--tables`

改写`--databases`选项，使得随后的所有参数都被解释为表名。

`--use-frm`（布尔值）

当与`--repair`一起使用时，它可以对表进行修复：使用该表的.frm文件重新初始化索引文件，并确定如何解释数据文件的内容，从而重建这些索引。如果索引丢失或损坏后不可恢复，则此选项将非常有用。不过，它应该作为最后的打算，并且只有在当前的MySQL版本与创建表时所使用的MySQL版本一样时才能使用它。否则，有可能进一步破坏表。

`--write-binlog`（布尔值）

将ANALYZE TABLE、OPTIMIZE TABLE和REPAIR TABLE语句写到二进制日志里（这意味着，它们将被发送到从服务器）。此选项默认是启用的。使用`--skip-write-binlog`可以禁用它。

下面几个表格展示了mysqlcheck的各个选项与它所执行的SQL语句之间的对应关系。

表检查选项（只适用于InnoDB表和MyISAM表）：

选项	相应语句
<code>--check</code>	<code>CHECK TABLE tbl_list</code>
<code>--check-only-changed</code>	<code>CHECK TABLE tbl_list CHANGED</code>
<code>--extended</code>	<code>CHECK TABLE tbl_list EXTENDED</code>
<code>--fast</code>	<code>CHECK TABLE tbl_list FAST</code>
<code>--medium-check</code>	<code>CHECK TABLE tbl_list MEDIUM</code>
<code>--quick</code>	<code>CHECK TABLE tbl_list QUICK</code>

InnoDB并不支持多种类型的检查，因此对于InnoDB表，mysqlcheck 会把上表里的所有选项当成`--check`。

表分析选项（只适用于MyISAM表和InnoDB表）：

选项	相应语句
<code>--analyze</code>	<code>ANALYZE TABLE tbl_list</code>

表修复选项（只适用于MyISAM表）：

选项	相应语句
<code>--repair</code>	<code>REPAIR TABLE tbl_list</code>
<code>--repair --quick</code>	<code>REPAIR TABLE tbl_list QUICK</code>
<code>--repair --extended</code>	<code>REPAIR TABLE tbl_list EXTENDED</code>
<code>--repair --use-frm</code>	<code>REPAIR TABLE tbl_list USE FRM</code>

表优化选项（只适用于MyISAM表）：

选项	相应语句
<code>--optimize</code>	<code>OPTIMIZE TABLE tbl_list</code>

F. 12 mysqld

`mysqld`是MySQL服务器程序。它用于向客户端程序提供数据库访问功能，因此它必须处于运行状态，否则客户端程序无法使用该服务器管理的那些数据库。当 `mysqld` 启动的时候，它会打开网络接口进行监听，并等待客户的连接。`mysqld`是一个多线程程序——它会使用不同线程来处理各个客户连接的方式，在多个客户端之间提供并发处理能力。

常见的启动方式是在服务器名字的后面紧跟各种想要的选项：

```
mysqld [options]
```

在Windows里，可以把服务器安装成一个服务。例如，可以像下面这样把服务器安装为系统启动时自动运行，或者删除安装的服务：

```
C:\> "C:\Program Files\MySQL\MySQL Server 5.5\bin\mysqld" --install
```

```
C:\> mysqld --remove
```

这个安装命令使用了服务器程序的完整路径名。如果你的服务器安装在其他的地方，则需要对这条命令里的路径名做出相应的修改。默认的服务名为 MySQL。你可以在选项的后面带一个服务名（请在同一行里输入每一条命令）：

```
C:\> "C:\Program Files\MySQL\MySQL Server 5.5\bin\mysqld" -install  
service_name
```

```
C:\> mysqld --remove service_name
```

采用这种方式，可以以多个不同的服务名运行多个服务器。如果没有提供 service_name 参数或者 MySQL 的服务名，则服务器会用 MySQL 作服务名，并且会从标准选项文件里读取 [mysqld] 选项组。如果指定的 service_name 参数不同于 MySQL，则服务器会将该名字用作服务名，并从标准选项文件里读取 [mysqld] 和 [service_name] 选项组。

可以在服务名的后台紧跟 --defaults-file 选项，用于指定另一个服务器在启动时可以读取

的选项文件（请在同一行里输入每一条命令）：

```
C:\> "C:\Program Files\MySQL\MySQL Server 5.5\bin\mysqld" -  
install  
service_name --defaults-file=file_name
```

在这种情况下，不允许省略参数 service_name。

刚才对 --install 选项的说明也适用于 --install-manual 选项。

F. 12.1 mysqld 支持的标准选项

--character-sets-dir	--port	--user
--debug	--shared-memory-base-name	--verbose
--help	--socket	--
version		

对这些选项的概要描述请参考F. 2. 1节。mysqld也支持F. 2. 1. 1节列出的那些标准SSL选项。

--help选项只会显示一段简要的使用说明信息。使用下面的命令可以看到完整的帮助信息：

```
% mysqld --verbose --help
```

详细的帮助信息也会把那些可以在命令行进行设置的系统变量显示出来。对这些变量的描述请参考附录D。

虽然支持使用--socket选项，但却不支持与之对应的短形式（-S）。在Windows里，如果服务器支持命名管道连接，则--socket会设置管道名称。

在Unix里，如果使用了--user选项，则服务器将使用该用户名或数字形式的用户ID来运行。在这种情况下，服务器启动时，会从密码文件里查找该账户的用户 ID和用户组 ID，然后更改自己的用户ID和用户组ID。如此一来，服务器便会以与该用户相关联的权限（而非root权限）来运行。（如果想让--user 选项起作用，服务器必须以root 用户身份来启动。否则，它不能更改自己的用户ID，并且会产生一个警告信息。）

F. 12. 2 mysqld特有的选项

在所有的MySQL程序当中，mysqld的选项集涉及范围最广。不过，如果将mysqld --verbose--help的输出与这里描述的选项列表相对比，你便会发现，帮助输出里包含的许多项目在这个列表里都找不到。这是因为帮助输出内容里还包括这样一些“选项”：它们实际上都是一些可以在服务器启动时进行设置的系统变量。限于篇幅，如果某个选项拥有相同名称的系统变量，那么这里将会省略该选项。如果你在此处未看到你正在查找的服务器选项，那么你可以尝试在附录D里找找看。例如，mysqld的帮助信息会列出一个用于启用查询日志的--general-log选项。你可以在对系统变量general_log的描述里找到与之相关的信息。

下面第一组选项列表描述的是一些通用选项。随后的几个选项列表则与Windows和复制机制特定相关。

--allow-suspicious-udfs（布尔值）

允许MySQL服务器加载老式的用户定义函数（UDF）——它们可能只定义了与函数名有关的符号，而没有定义任何与标准支持例程有关的符号。为避免加载那些有可能并非真正 UDF的函数，此功能默认是被禁用的。

`--ansi`或`-a`

告知服务器在遇到某些类型的语法时使用标准 SQL行为，而不使用MySQL特定的语法。此选项可以使服务器更兼容于标准。

此选项等价于将系统变量 `sql_mode` 设置为包含这样一些模式值：`REAL_AS_FLOAT`、`PIPES_AS_CONCAT`、`ANSI_QUOTES`、`IGNORE_SPACE`和`ONLY_FULL_GROUP_BY`。

`--archive[=state]`

控制ARCHIVE存储引擎插件的激活状态，如果它包含在MySQL的发行包里的话。与state允许值有关的更多信息请参考`--plugin_name`的描述。

`--basedir=dir_name`或`-b dir_name`

MySQL安装根目录的路径名。对于许多其他路径，如果它们是以相对路径名的形式给出的，则它们都是相对于此目录。

`--bind-address=addr`

绑定到给定的IP地址，以监听TCP/IP客户端连接。自MySQL 5.6.6版本起，默认IP地址为*（即监听所有的IPv4和IPv6接口）。在5.6.6版本之前，默认为0.0.0.0（即监听所有的IPv4接口）。更多相关信息请参考12.2.4节。

`--binlog-row-event-max-size=n`

基于行的事件所允许的最大大小。mysqld会尽量把行分组成不会超过这个值的事件。该值会截断成最靠近256倍数的某个非零值。自MySQL 5.6.6起，其默认值为8 KB；而对于之前的版本，其值为1 KB。

`--blackhole[=state]`

控制BLACKHOLE存储引擎插件的激活状态，如果它包含在MySQL的发行包里的话。与state允许值有关的更多信息请参考`--plugin_name`的描述。

`--bootstrap`

此选项是在第一次安装MySQL时使用的安装脚本。

`--character-set-client-handshake`（布尔值）

告诉服务器使用客户端提供的字符集信息。此选项是默认启用的。使用`--skip-character-set-client-handshake`选项可以忽略这些信息，MySQL 4.0 允许这样做。

`--character-set-server=charset`或`-C charset`

服务器的默认字符集。

`--chroot=dir_name`或`-r dir_name`

在支持服务器时，以指定目录作为它的根目录。请参考Unix手册页里的`chroot()`，获得更多有关在`chroot()`环境里运行的信息。

`--collation-server=collation`

服务器默认字符集所对应的默认排序规则。

`--core-file`

当发生致命错误时，MySQL服务器会在退出之前生成一个内核文件。

`--datadir=dir_name`或`-h dir_name`

MySQL数据目录的路径名。

`--default-time-zone=tz_name`

把服务器的默认时区设置为tz_name。与时区值有关的描述请参考12.6.1节。此选项会设置系统变量time_zone，而不会设置system_time_zone。

`--delay-key-write[=val]`

设定服务器用于处理MyISAM文件的键值延迟写入模式。val的取值有三种：ON（根据每个表的具体情况延迟键写入，即根据每个表在创建时使用的DELAY_KEY_WRITE 选项值。如果没有给定选项值，则默认值就是它）、OFF（对任何MyISAM表都不进行延迟键写入）或 ALL（对所有的MyISAM表进行延迟键写入）。OFF和ALL 对所有的MyISAM表都一视同仁，无论它们在创建时是如何进行定义的。

`--des-key-file=file_name`

它指的是，用于存放DES_ENCRYPT()和DES_DECRYPT()函数的DES密钥的那个文件的名字。与文件格式有关的更多描述请参考附录C里DES_ENCRYPT()函数的条目。

`--exit-info[=n]或-T[n]`

让MySQL服务器在退出时生成调试信息。如果把n指定在-T之后，则它们之间不允许有空格；否则，该值将无法得到正确识别。

`--external-locking`（布尔值）

在某些系统（如Linux）里，外部锁定机制（即文件系统级的锁定）默认是禁用的，使用此选项可以启用该机制。

外部锁定比较麻烦，因为它在某些系统里无法可靠地工作，并且只对那些只读表的操作（如表检查）起作用。

`--federated[=state]`

控制FEDERATED存储引擎插件的激活状态，如果它包含在MySQL的发行包里的话。与state允许值有关的更多信息请参考--plugin_name的描述。

`--gdb`

设置信号处理器，方便使用gdb进行调试。

`--ignore-db-dir=dir_name`

在数据目录中指定一个目录的名字，使其不会被SHOW DATABASES语句或INFORMATION_SCHEMA库里的表当作数据库目录。此选项可以多次指定。采用这种方式指定的各个目录，在运行时作为系统变量ignore_db_dirs的值。例外情况是，当指定为空值时，会清除该目录列表，同时改写之前的所有实例。此选项是在MySQL 5.6.3里引入的。

`--innodb[=state]`

控制存储引擎插件 InnoDB的激活状态。与 state 允许值有关的信息请参考--plugin_name的描述。因为InnoDB是默认存储引擎，所以此选项也是默认启用的。如果不想使用InnoDB表，可以使用--innodb=OFF来禁用它。此时，还必须设置系统变量default_storage_engine（自MySQL 5.6.3起，还需要设置default_tmp_storage_engine）来指定不同的默认存储引擎；否则，服务器无法启动。参考12.5.2节。

`--innodb-status-file`（布尔值）

将SHOW INNODB STATUS信息定期写到innodb_status.nnnnnn指定的位于数据目录里的那个文件。其中，nnnnnn是服务器的进程ID号。除了在完全关闭服务器以外，这些文件都不会被删除。你需要间歇性地删除那些不再需要的文件。

`--innodb-xxx`

有许多其他InnoDB参数都可以被当作系统变量，在服务器启动时进行设置。请参考D.1.1节。--language=lang_name或-L lang_name

出错信息显示语言。此选项已过时。请使用系统变量lc_messages和lc_message_dir来指定出错信息的区域设置和文件位置。

`--log[=file_name]或-l[file_name]`

启用普通查询日志。在MySQL 5.6里，此选项已被弃用和删除。请使用系统变量general_log和general_log_file来代替。

`--log-bin[=file_name]`

启用二进制日志。file_name指定的是二进制日志文件的基本名。如果未给定它，则日志文件名为数据目录里的HOSTNAME-bin.nnnnnn；其中，HOSTNAME是服务器主机名，nnnnnn是一个序列号（服务器每创建一个新的日志文件，它便会增加1）。如果file_name是以相对路径形式指定的，则服务器会将它解释为相对于数据目录。

`--log-bin-index=file_name`

启用二进制日志索引文件。如果未给出file_name，则默认名字与二进制日志文件的基本名相同，扩展名为.index。如果file_name是以相对路径形式指定的，则服务器会将它解释为相对于数据目录。

`--log-isam[=file_name]`

启用索引文件日志记录。这个只适用于调试MyISAM操作。如果你没有给定具体的名字，则默认为数据目录里的myisam.log。

`--log-raw`（布尔值）

禁止将语句重写到普通查询日志、慢查询日志和二进制日志，以防止密码以明文方式出现。这个选项主要用于调试和测试。通常情况下，你肯定不会希望将密码暴露在服务器日志里。此选项是在MySQL 5.6.3里引入的。

`--log-short-format`（布尔值）

如果启用了二进制日志和慢查询日志，则往它们当中写入较少的信息。

`--log-slow-admin-statements`（布尔值）

诸如ALTER TABLE或OPTIMIZE TABLE语句所执行的那些管理性操作可能会很慢，但是它们在默认情况下并不会被记录到慢查询日志里。此选项会使它们在处理比较慢时被记录到慢查询日志里。

`--log-slow-queries[=file_name]`

启用慢查询日志。在MySQL 5.6 里，此选项已被弃用和删除。请使用系统变量slow_query_log和slow_query_log_file来代替。

`--log-tc=file_name`

事务坐标日志文件（用于XA事务）的路径名。此选项未使用。

`--log-tc-size=n`

事务坐标日志文件的大小。

`--log-warnings[=n]或-W[n]`

将某些非关键性警告信息写入错误日志。此选项是默认启用的。如果在给定此选项时没有设置具体值，也会启用警告；使用0或者1，也可以分别禁用或启用警告。在没有带具体值的情况下两次指定此选项，或者将值指定为2，都可以启用对连接中断和“拒绝访问”错误的日志记录功能。如果使用-W来指定n值，则它们之间不允许有空格；否则，该值将无法正确解释。

`--memlock`（布尔值）

如果可能，则在内存里锁定服务器。此选项只有在类似 Solaris或Linux这样的系统里（它们可以在内存里锁定进程）才会起作用，并且要求服务器必须以root用户身份运行。

`--mysam-block-size=size`

MyISAM表索引块的块大小。

`--old-style-user-limits`（布尔值）

MySQL账户可以对它们的活动进行限制，如13.2.2.5节所述。在对账户进行限制评估时，不会考虑该账户是从哪台特定主机连接的。但是，使用`--old-style-user-limits`选项可以启用早期的限制评估方法。（在MySQL 5.0.3之前，限制是按账户连接的主机单独进行评估的。）

`--plugin-load=plugin_list`

加载选项值指明的那些插件，它是由一个或多个形如 `plugin_name=lib_name` 或 `lib_name` 的说明符构成的一个列表，各个说明符之间用分号隔开。当同时列出插件和库的名字时，会导致服务器只加载该库里的那个插件。当只列出库名，但没有列出插件名字时，会导致服务器加载该库里的所有插件。`--plugin-load`会重置要被加载的插件列表，因此如果多次指定它，则只有最后那个实例才会起作用。更多相关信息请参考12.4节。

`--plugin-load-add=plugin_list`

与`--plugin-load`相似，但是它会把插件列表添加到将被加载的那个插件列表里，而不会首先重置该列表。此选项是在MySQL 5.6.3里引入的。

`--plugin_name[=state]`

如果某个插件在服务器启动时会被加载的原因是，它是内置的、注册在 `mysql.plugin`表里或者使用选项`--plugin-load`或`--plugin-load-add`指定的，那么它的激活状态完全可以进行控制。具体做法是，在该插件的后面加上一个选项。允许的state值有OFF（不激活）、ON（激活；它是忽略state时

的默认值)、FORCE (当遇到错误时, 也要激活, 并提示启用失败) 和 FORCE_PLUS_PERMANENT (与FORCE相似, 但是阻止插件在运行时被卸载掉)。关于插件加载的更多信息请参考12.4节。

`--port-open-timeout=n`

在启动时, 服务器在其TCP/IP端口变得可用之前应该等待的时间 (单位为秒)。默认为0 (即不等待)。

`--safe-mode`

此选项已过时。

`--safe-user-create` (布尔值)

如果某用户不具有对mysql.user权限表的INSERT权限, 则不允许创建账户。

`--skip-grant-tables` (布尔值)

不使用权限表来验证客户端连接。这样将允许任何客户端做任何事情。它也会禁止使用CREATE USER、DROP USER、RENAME USER、GRANT、REVOKE和SET PASSWORD语句; 并且阻止服务器加载mysql.plugin 表里注册的任何插件。如果想让服务器再次使用权限表, 可以执行FLUSH PRIVILEGES语句, 或者mysqladmin flush-privileges命令, 或者不带--skip-grant-tables重新启动它。

`--skip-host-cache`

禁止使用主机名缓存。自MySQL 5.6.5起, 此选项的效果与设置host_cache_size=0相似, 不同之处在于, 使用--skip-host-cache之后, 缓存不能在运行时被再次启用。

`--skip-stack-trace`

在出现操作失败时, 不输出栈跟踪信息。

`--symbolic-links`或`-s` (布尔值)

在Unix系统里, 此选项将允许使用MyISAM表数据文件和索引文件的符号链接 (即使用DATA DIRECTORY和INDEX DIRECTORY表创建选项)。在Windows里, 此选项将允许使用数据库目录的符号链接。关于这些技术的讨论请参考

第11章。在Windows里，数据库符号链接支持默认是启用的。使用`--skip-symbolic-links`选项可以禁用它。

`--sysdate-is-now`（布尔值）

`SYSDATE()`函数返回的是该函数被调用时的日期和时间，而`NOW()`函数返回的是语句开始执行时的时间。`--sysdate-is-now`选项可以使`SYSDATE()`的行为与`NOW()`的相似。

`--tc-heuristic-recover=str`

此选项未被使用。

`--temp-pool`（布尔值）

如果设置此选项，则服务器会使用少量的一组名字来命名临时文件，而不是为每个临时文件创建一个唯一的名字。在Linux里（它是唯一适用的系统），这种做法能避免某些缓存问题。此选项默认是启用的。使用`--skip-temp-pool`选项可以禁用它。

`--tmpdir=dir_name`或`-t dir_name`

它用来指定存放临时文件的目录的路径名。此选项的值可以是由一个目录列表给出的，这些值以循环的方式使用。在Unix里，目录名之间使用冒号隔开；在Windows里，目录名之间则需要用分号隔开。

`--transaction-isolation=level`

设定事务隔离的默认级别。`level`的可取值包括`READ-UNCOMMITTED`、`READ-COMMITTED`、`REPEATABLE-READ`和`SERIALIZABLE`。

`--transaction-read-only`（布尔值）

默认事件访问模式是否为只读。此选项默认是禁用的，即表示默认访问模式为读/写。此选项是在MySQL 5.6.5里引入的。

F.12.2.1 Windows选项

本节的各个选项只能用于运行在Windows里的服务器。服务名和命名管道名不区分大小写。共享内存名区分大小写。

`--console`（布尔值）

显示一个控制台窗口，用于显示出错信息。如果另外指定了`--log-error`，出错信息会被写入日志文件，而不会被输出到控制台。

`--install [service_name]`

把服务器安装为服务，并让它在Windows启动时自动运行。如果没指定`service_name`，则默认服务名为MySQL。

`--install-manual [service_name]`

把服务器安装为服务，但不让它在Windows启动时自动运行。你必须自己显式启动该服务。

如果没指定`service_name`，则默认服务名为MySQL。

`--named-pipe`（布尔值）

对于支持命名管道功能的MySQL服务器，命名管道连接默认是禁用的。此选项可以启用命名管道连接。默认管道名为MySQL。管道名可以通过`--socket`选项更改。

`--remove [service_name]`

删除服务器服务。如果没指定`service_name`，则默认服务名为MySQL。

`--shared-memory`（布尔值）

启用对共享内存连接。默认共享内存名为 MySQL。这个名字可以通过`--shared-memory-base-name`选项来更改。

`--standalone`

以独立程序方式（而非以服务方式）运行服务器。

F. 12. 2. 2 复制机制选项

本节的各个选项都与MySQL的复制功能有关。

`--show-slave-auth-info`选项会影响到主服务器上SHOW SLAVE HOSTS的输出，如附录E所述。

`--abort-slave-event-count=n`

此选项是MySQL测试套件在对复制进行测试时使用的。

`--binlog-do-db=db_name`

告诉复制主服务器只记录对指定数据库的更新。其他数据库都不参加复制。如果想记录多个数据库的数据更新，可以针对每个数据库重复一次此选项。

`--binlog-ignore-db=db_name`

告诉复制主服务器不要记录对指定数据库的更改。如果想忽略多个数据库的数据修改，则可以针对每个数据库重复一次此选项。

请注意，使用此选项会导致二进制日志不能包含给定数据库在发生崩溃时进行恢复所需要的信息。为避免此问题，可以在从服务器上使用 `--replication-ignore-db` 来代替它。

`--disconnect-slave-event-count=n`

此选项是MySQL测试套件在对复制进行测试时使用的。

`--master-info-file=file_name`

对于从服务器而言，它指的是用于存储当前复制状态信息的那个文件的名称。该文件包括的内容有复制坐标（主服务器上的二进制日志的文件名和位置）、主服务器主机、用户名、密码、端口号、连接重试间隔时间，以及SSL选项值。该文件的默认名为数据目录里的 `master.info`。如果 `file_name` 是以相对路径形式指定的，则服务器会将它解释为相对于数据目录。

`--master-retry-count=n`

对于复制从服务器而言，它指的是从服务器在放弃之前尝试连接主服务器的次数。在MySQL 5.6里，此选项已过时。请使用 `CHANGE MASTER TO` 语句的 `MASTER_RETRY_COUNT` 选项。

`--max-binlog-dump-events=n`

此选项是MySQL测试套件在对复制进行测试时使用的。

`--relay-log=file_name`

对于复制从服务器而言，此选项指定的是中继日志文件的基本名。（从服务器的I/O线程会负责把来自主服务器的更新存储到中继日志里；而SQL线程会从中继日志里读取语句，并执行它们。）默认情况下，中继日志的文件名为 `HOSTNAME-relay-bin.nnnnnn`（位于数据目录里）；其中，`HOSTNAME` 是

服务器主机名，nnnnnn 是一个序列号（服务每创建一个新的日志文件，它便会增加1）。

```
--relay-log-index=file_name
```

对于复制从服务器而言，它指的是中继日志索引文件的名称。该文件的默认名为HOSTNAME-relay-bin.index（位于数据目录里）；其中HOSTNAME是该服务器主机的主机名。如果file_name是以相对路径形式指定的，则服务器会将它解释为相对于数据目录。

```
--relay-log-info-file=file_name
```

对于复制从服务器而言，它指的是中继日志信息文件的名称。默认日志文件的名称为relay-log.info，位于数据目录里。

```
--replicate-do-db=db_name
```

告诉复制从服务器只复制指定的数据库。如果要限制一组数据库的复制，则可以针对每个数据库重复一次此选项。

```
--replicate-do-table=db_name.tbl_name
```

告诉复制从服务器只复制指定的表，指定格式为db_name.tbl_name。如果要限制一组表的复制，可以针对每个表重复一次此选项。

```
--replicate-ignore-db=db_name
```

告诉复制从服务器不要复制指定的数据库。如果想忽略多个数据库，可以针对每个数据库重复一次此选项。

```
--replicate-ignore-table=db_name.tbl_name
```

告诉复制从服务器不要复制指定的表。如果想忽略多个表，可以针对每个表重复一次此选项。

```
--replicate-rewrite-db=master_db->slave_db
```

告诉复制从服务器把某个数据库当成另外一个数据库来处理。对主服务器上的那个源数据库master_db的更改，将作为更新复制到从服务器的slave_db数据库。重写操作只会在master_db为默认数据库的时候才能使用，而且只能用于对该数据库里的表进行操作的语句。在命令行指定此选项时，其值应该用引号引起来，以防止命令解释器将字符“>”当作一个输出重

定向操作符。此选项可以多次指定。服务器将按顺序尝试它们，并且会使用master_db值匹配到的第一个规则。

此选项会在其他的--replication-xxx 选项所指定的操作被测试之前应用，因此如果你使用它，那些选项应该把slave_db用作数据库名。

`--replicate-same-server-id` (布尔值)

如果启用了此选项，服务器将不会跳过包含其自身服务器ID的那些复制事件。此选项默认是禁用的，其目的是防止发生循环复制的现象，但是在某些特殊的场合也可以启用它。

`--replicate-wild-do-table=pattern`

告诉复制从服务器只复制其名字与指定模式相匹配的那些表。如果要限制一组模式的复制，可以针对每个模式重复一次此选项。

`--replicate-wild-ignore-table=pattern`

告诉复制从服务器不要复制其名字与指定模式相匹配的那些表。如果想忽略多个模式，可以针对每个模式重复一次此选项。

`--show-slave-auth-info` (布尔值)

它会让主服务器在SHOW SLAVE HOSTS语句的输出里显示从服务器的用户名和密码。

`--skip-slave-start`

它会让服务器不自动启动从服务器线程。这些线程必须通过START SLAVE语句手动启动。

`--sporadic-binlog-dump-fail` (布尔值)

此选项是MySQL测试套件在对复制进行测试时使用的。

F. 12.3 mysqld的变量

如果要查看完整的帮助信息（其中会显示出mysqld默认使用的那些系统变量的值），可以使用下面这条命令：

```
% mysqld --verbose --help
```

如果要查看当前正执行的mysqld正使用的那些系统变量的值，可以使用下面这条命令：

```
% mysqladmin variables
```

查看当前系统变量值的方法还包括执行SHOW VARIABLES 语句，查看INFORMATION_SCHEMA库里的GLOBAL_VARIABLES表和SESSION_VARIABLES表。各个系统变量的描述可以在附录D里找到。系统变量的值可以在启动时根据F. 2. 1. 2节的说明进行设置。此外，许多系统变量还可以被动态修改，请参考12. 3. 1节和附录E里的SET语句条目。

F. 13 mysqld_multi

mysqld_multi脚本大大简化了在同一台主机上运行多个mysqld服务器的工作。既可以用它来启动或停止服务器，也可以用它来检测它们是否正在运行：

```
mysqld_multi [options] command server_list
```

其中，参数command的值可以是start、stop或report。自MySQL 5.6.3起，也允许使用reload（停止后再重启）。参数 server_list 表示的是需要操纵的那些服务器。有关如何使用mysqld_multi脚本的更多信息请参考12. 9. 4节。

F. 13.1 mysqld_multi支持的标准选项

```
--help          --silent          --verbose  
  
--password      --user           --version
```

对这些选项的概要描述请参考F. 2. 1节。

当 mysqld_multi 需要停止服务器或者确定它是否在运行时，它会把选项--user和--password的值传递给 mysqladmin。对于--password 选项，那个密码值不是可选的，与大部分的MySQL程序不一样。

F. 13.2 mysqld_multi特有的选项

```
--example
```

显示一个样本选项文件，示范适用于mysqld_multi脚本的各种选项文件组。

`--log=file_name`

`mysqld_multi` 用来记录其动作的日志文件名。如果日志文件已经存在，新的日志信息将被追加到它的末尾。默认的日志文件为数据目录里的 `mysqld_multi.log`。使用 `--no-log` 选项可以禁用日志记录功能。

`--mysqladmin=file_name`

所用的那个 `mysqladmin` 程序的路径名。如果 `mysqld_multi` 无法自己找到 `mysqladmin`，或者想要使用某个特定版本的 `mysqladmin` 程序，就需要使用此选项。

`--mysqld=file_name`

所用的那个 `mysqld` 程序的路径名。如果 `mysqld_multi` 无法自己找到 `mysqladmin`，或者想要使用某个特定版本的 `mysqld` 程序，就需要使用此选项。你也可以把 `mysqld` 或 `mysqld_safe` 的路径名作为此选项的值。

`--no-log`

显示日志输出，而不是把它写到日志文件里。如果想在屏幕上看到日志输出内容，则必须使用此选项，因为默认是把日志写入文件。

`--tcp-ip`

默认情况下，`mysqld_multi` 会尝试使用 Unix 套接字文件来连接服务器。此选项则会要求使用 TCP/IP 来建立连接。如果某个服务器正在运行，但是其套接字文件已经被删除，并且该服务器只能通过 TCP/IP 来访问，则此选项非常有用。

F.14 `mysqld_safe`

`mysqld_safe` 用于启动和监视 `mysqld` 服务器：

`mysqld_safe [options]`

如果服务器意外停止，`mysqld_safe` 会重启它。`mysqld_safe` 是一个 shell 脚本，并且只能用于 Unix 系统。

F.14.1 `mysqld_safe` 支持的标准选项

`--help`

`--plugin-dir`

对这些选项的概要描述请参考F. 2. 1节。

F. 14. 2 mysqld_safe特有的选项

如果在命令行指定那些mysqld_safe并不支持的选项，它会将它们传递至mysqld。

mysqld_safe脚本支持的其自己的选项如下所示。

`--basedir=dir_name`

MySQL基本目录的路径名。

`--core-file-size=n`

将服务器崩溃时生成的内核文件大小限制为n个字节。

`--datadir=dir_name`

MySQL数据目录的路径名。

`--ledir=dir_name`

在“libexec”目录中查找服务器。

`--log-error=file_name`

用来保存出错日志信息的那个文件。此选项的解释方式与mysqld解释系统变量log_error的值的方式一致。

`--malloc-lib[=lib_name]`

mysqld 在实现的malloc()应该使用的那个库，它并非系统库。此选项会修改环境变量LD_PRELOAD，从而对mysqld的动态链接产生影响。对于Linux系统，lib_name的值可以为tcmalloc，其目的是要使用与MySQL发行版本绑定在一起的那个库。如果此选项的值为某个目录名，则mysqld_safe会修改LD_PRELOAD，将该目录添加到修改值的前面。

`--mysqld=file_name`

mysqld程序的路径名。

`--mysqld-version=suffix`

此选项的值是一个后缀串。如果指定了此选项，则它会被添加到mysqld的基本名末尾，二者之间用一根短划线连接，从而生成mysqld_safe应该启动的那个服务器的名字。

`--nice=N`

使用nice程序将服务器的调度优先级设置为N。

`--open-files-limit=n`

mysqld应该保留的文件描述符的个数。

`--pid-file=file_name`

mysqld的进程ID文件的名字。

`--port=port_num`

服务器应该监听的那个TCP/IP连接端口号。

`--skip-kill-mysqld`

在启动一个新的mysqld 进程之前，不要试图终止任何当前正在运行的mysqld 进程。如果想要运行同一个mysqld程序的多个实例，则此选项非常有用。此选项只在Linux里有效。

`--skip-syslog`

要求服务器不要把出错输出发送到syslog，而应该将其发送到日志文件。默认是发送到日志文件。

`--socket=file_name`

Unix套接字文件的路径名。

`--syslog`

要求将出错输出发送到syslog，适用于那些拥有logger程序的系统。

`--syslog-tag=tag`

当出错输出发送到syslog时，来自mysqld-safe和mysqld的信息都会带上一个相应的程序名前缀。--syslog-tag选项会分别将前缀修改为mysqld-safe-tag和mysqld-tag。

`--timezone=tz_name`

把服务器的系统时区设置为 `tz_name`。如果服务器无法自动确定系统时区，此选项可能会很有用。

`--user=user_name`或`--user=uid`

用于运行MySQL服务器的那个系统账户的用户名或者数字形式的用户ID。

F. 15 `mysqldump`

`mysqldump`程序能够把数据库表的内容写到文本文件里。这些文件有许多用途，如数据库备份、将数据库转移到另一个服务器或者根据现有数据库的内容建立一个测试数据库。

默认情况下，每个转储表的输出都由一条CREATE TABLE语句（用于创建这个表）以及紧随其后的一组INSERT语句（用于加载该表的内容）构成。如果给定了一`--tab`选项，那么该表的内容会被写到一个数据文件里（各个值之间都用制表符隔开，每个行占用一输出行），而用来创建表的SQL语句会被写到另一个文件里。

`mysqldump`的运行模式有以下三种：

`mysqldump [options] db_name [tbl_name] ...`

`mysqldump [options] --databases db_name ...`

`mysqldump [options] --all-databases`

在第一种模式里，`mysqldump`会转储指定数据库里的指定表。如果没有指定表，则`mysqldump`会转储数据库里的所有表。在第二种模式里，`mysqldump`会把所有的参数都当作数据库名，并依次转储各个数据库里的所有表。在第三种模式里，`mysqldump`会转储所有数据库里的所有表。如果使用了`--database` 或`--all-database`，那么输出会在每个数据库表语句的前面包含CREATE DATABASE IF NOT EXISTS和USE语句。

下面是`mysqldump`的一种常见用法：

`% mysqldump db_name > backup_file`

如果想将备份文件导入MySQL，需要使用`mysql`，而不能使用`mysqlimport`：

```
% mysql db_name < backup_file
```

mysqldump会忽略INFORMATION_SCHEMA数据库；并且只有在命令行上显式指定它时，才能将其转储。在这种情况下，还必须指定--skip-lock-tables。

F. 15.1 mysqldump支持的标准选项

--bind-address	--default-character-set	--protocol
--character-sets-dir	--help	--shared-memory-base-name
--compress	--host	--socket
--debug	--password	--user
--debug-check	--pipe	--verbose
--debug-info	--plugin-dir	--version
--default-auth	--port	

对这些选项的概要描述请参考F. 2. 1节。mysqldump也支持F. 2. 1. 1节列出的那些标准SSL选项。

F. 15.2 mysqldump特有的选项

下面这些选项控制着mysqldump的各种操作。F15. 3节描述了各种与--tab选项一起使用的选项，用以指明各种数据文件的格式。

--add-drop-database (布尔值)

在每条CREATE DATABASE语句之前添加一条DROP DATABASE IF EXISTS语句。

--add-drop-table (布尔值)

在每条CREATE TABLE语句之前添加一条DROP TABLE IF EXIST语句。

--add-drop-trigger (布尔值)

在每条CREATE TRIGGER语句之前添加一条DROP TRIGGER IF EXISTS语句。此选项是在MySQL 5.6.0里引入的。

--add-locks (布尔值)

在每个表的INSERT语句集合前后分别加上LOCK TABLE和UNLOCK TABLE语句。

--all-databases或-A (布尔值)

转储所有数据库里的所有表。此选项也使转储输出针对每个数据库都包含CREATE DATABASE和USE语句。

--all-tablespaces或-Y (布尔值)

转储所有的表空间。只与MySQL集群相关。

--allow-keywords (布尔值)

允许使用关键字作为列名。

--apply-slave-statements (布尔值)

此选项通常需要与--dump-salve 选项一起使用。它会导致转储输出在CHANGE MASTER 语句的前面和加上一条STOP SLAVE语句，后面加上一条START SLAVE语句。此选项是在MySQL 5.5.3里引入的。

--comments或-i (布尔值)

在输出结果里包含一些附加的注释信息，如mysqldump版本号、每组INSERT语句都应用于哪些表，等等。此选项默认是启用的。使用--skip-comments选项可以禁用它。

--compact (布尔值)

生成比较简洁的输出，其中不包含注释，而只包含那些会设置系统变量的与版本有关的注释。此选项还会启用--skip-add-drop-table、--skip-set-charset、--skip-disable-keys和--skip-add-locks选项。

--compatible=mode

此选项将导致mysqldump修改其输出结果，以便可以与标准SQL、其他数据库服务器或旧版的MySQL服务器保持兼容。mode的值用于指定兼容模式。它

可以是一个以逗号分隔的列表，其中的取值如下表所示。

选项	兼容性
ANSI	与ANSI兼容
DB2	与DB2兼容
MAXDB	与MaxDB兼容
MSSQL	与MS SQL Server兼容
MYSQL323	与MySQL 3.23兼容
MYSQL40	与MySQL 4.0兼容
ORACLE	与OPACLE兼容
POSTRESQL	与PostgreSQL兼容
NO_FIELD_OPTIONS	禁用MySQL特有的与列有关的选项
NO_KEY_OPTIONS	禁用MySQL特有的与索引有关的选项
NO_TABLE_OPTIONS	禁用MySQL特有的与表有关的选项

此选项对于MySQL 4.1之前的版本不起作用。

`--complete-insert`或`-c`（布尔值）

在写入INSERT语句时，列出要被插入的每一个列。

`--create-options`或`-a`（布尔值）

在mysqldump 生成的CREATE TABLE 语句里包含附加注释信息，如存储引擎、AUTO_INCREMENT的起始值等。你可以按 CREATE TABLE的语法格式使用 table_option的值来指定这些信息（请参考附录E）。

此选项默认是启用的。使用`--skip-create-options`选项可以禁用它。

`--databases`或`-B`（布尔值）

把所有参数都解释为数据库名，并转储各个数据库里的所有表。此选项也使转储输出针对每个数据库都包含CREATE DATABASE IF NOT EXISTS和USE语句。

`--delayed-insert`（布尔值）

写入INSERT DELAYED语句，而非INSERT语句。如果正在把MyISAM表的转储文件加载到其他数据库，并且想尽量减小可能会在那个数据库上发生的对其他语句进行操作所产生的影响，那么使用--delayed-insert有助于实现该目标。

`--delete-master-logs`

在生成转储输出之后通过执行一条FLUSH MASTER 语句，将MySQL服务器上的二进制日志文件删除，并创建一个新的文件。只有你能确保已有的二进制日志文件可以被删除时，才能使用此选项。此选项会启用--master-date选项。

`--disable-keys`或`-K`（布尔值）

往输出内容里添加ALTER TABLE ... DISABLE KEYS和ALTER TABLE ... ENABLE KEYS语句，以禁止在处理 INSERT 语句时更新那些非唯一性索引。如此一来，能够在加载表之后立即创建索引，从而加快每个MyISAM表的索引创建速度。

`--dump-date`（布尔值）

在输出末尾添加一个转储日期注释。

`--dump-salve[=n]`

此选项类似于--master-data，但是它主要用于转储“复制从服务器”，并且它还将在转储输出文件里生成一条 CHANGE MASTER 语句，用以表明从服务器在主服务器里（并非从服务器本身）的二进制日志坐标。关于如何使用选项参数的更多描述，请参考与--master-data选项有关的描述。--dump-salve选项是在MySQL 5.5.3里引入的。

`--events`或`-E`（布尔值）

在转储里包含“事件调度器”的事件。

`--extended-insert`或`-e`（布尔值）

写入多行 (multiple-row) INSERT语句。这种语句的加载效率比单行语句的高。

`--flush-logs`或`-F` (布尔值)

在转储表之前，先刷新服务器的日志文件。默认情况下，会按数据库刷新这些日志，以便于创建检查点。这样做可以让恢复操作变得更加容易，因为你知道，这些二进制日志文件是在备份给定数据库之后所做的那个检查点之后创建的。将它与`--lock-all-tables` 或`--master-data`一起使用，可以只在所有表都锁定之后才刷新这些日志。此选项要求具有RELOAD权限。

`--flush-privileges` (布尔值)

如果转储内容要包含 `mysql` 数据库，在转储完该数据库之后还需要在输出内容里加上一条FLUSH PRIVILEGES语句。

`--force`或`-f` (布尔值)

即使出现了错误也要继续执行。

`--hex-blob` (布尔值)

将BINARY、VARBINARY和BLOB列转储为十六进制常量。例如，在给定此选项后，`mysqldump`将把字符串值"MySQL"写成0x4D7953514C。

`--ignore-table=db_name.tbl_name`

跳过指定表或视图的转储输出。如果想忽略多个表，可以针对每个表重复一次此选项。

`--include-master-host-port` (布尔值)

在使用`--dump-slave`选项生成的输出内容里，为CHANGE MASTER语句加上MASTER_HOST和MASTER_PORT选项，用以指定从服务器所对应的主服务器的主机名和端口号。此选项是在MySQL 5.5.3里引入的。

`--insert-ignore` (布尔值)

写入INSERT IGNORE语句，而非INSERT语句。

`--lock-all-tables`或`-x` (布尔值)

用FLUSH TABLES WITH READ LOCK 语句锁定所有数据库里的所有表。此选项将禁用--single-transaction和--lock-tables选项。

`--lock-tables`或`-l`（布尔值）

对于每个转储的数据库，在导出它们之前，先使用Lock TABLES ... READ LOCAL锁定所有导出表。此选项适用于MyISAM表，因为在转储过程中READ LOCAL锁允许并发插入操作。对于InnoDB表，首选--single-transaction选项。

`--log-error=`*file_name*

把警告信息和出错信息写到指定文件的末尾。

`--master-data`[=*value*]

此选项能帮助制作用于建立从服务器的备份。利用此选项，mysqldump可以向服务器发送一条SHOW MASTER STATUS语句，以获取它上面的当前二进制日志文件的名称和位置；并利用这些结果向包含相同文件名和位置的输出写入一条CHANGE MASTER语句。最终的结果是，当你把转储文件加载到某个从服务器时，它会将从服务器同步到被转储服务器的正确复制坐标位置，并在转储时从该处开始复制。只有在服务器启用了二进制日志功能的情况下，此选项才能发挥作用。默认情况下，在写入CHANGE MASTER语句时不会带上注释。--master-data可以接受一个可选值，用于显式控制语句的注释。当值为1时，将生成一条不带注释的语句；当值为2时，将生成一条带注释的语句。

--master-data 要求具有 RELOAD 权限。如果没有给定--single-transaction，则此选项会自动启用--lock-all-tables。

`--no-autocommit`（布尔值）

在一个事务里为每个表写 INSERT 语句。与自动提交模式执行每条语句的方式相比，采用此种方式输出的结果在加载时会更有效率。

`--no-create-db`或`-n`（布尔值）

不写入CREATE DATABASE语句。通常情况下，当使用--databases或--all-databases选项时，它们会自动被添加到输出里。

`--no-create-info`或`-t`（布尔值）

不写入CREATE TABLE语句。它只对只转储表的情况有用。

`--no-data`或`-d`（布尔值）

不写入表数据。它只对只转储`CREATE TABLE`语句的情况有用。

`--no-set-names`或`-N`

等同于`--skip-set-charset`。

`--no-tablespaces`或`-y`（布尔值）

不转储表空间。只与MySQL集群相关。

`--opt`

优化表的转储速度，并生成一个针对重载速度进行优化的转储文件。此选项会启用`--add-drop-table`、`--add-locks`、`--create-options`、`--disable-keys`、`--extended-insert`、`--lock-tables`、`--quick`和`--set-charset` 选项。此选项默认是启用的。使用`--skip-opt`选项可以禁用它。

`--order-by-primary`（布尔值）

按主键或者第一个唯一性索引（如果存在的话）的顺序来转储表的行。这种方式是以牺牲性能为代价为每个表生成一个排序的转储输出。

`--quick`或`-q`（布尔值）

默认情况下，`mysqldump`会先把表的全部内容读到内存里，然后再将其输出。此选项将使每个行在读入内存之后立刻写到输出里，这样可以大大减少对内存的占用。但是，如果使用了此选项，千万不要在操作过程中将`mysqldump`挂起。如果这样做的话，将会导致服务器进入等待状态，从而影响与其他客户端的正常交互。

`--quote-names`或`-Q`（布尔值）

把表名和列名用反撇号（```）字符括起来。如果这些名字是MySQL的保留字或者其中包含有特殊字符，则此选项会很有用。此选项默认是启用的。使用`--skip-quote-names`选项可以禁用它。

`--replace`

在生成语句时，将`INSERT`替换为`REPLACE`。

`--result-file=file_name`或`-r file_name`

将输出写到指定的文件。此选项只能用于Windows，它可以防止换行符自动转换为一对回车/换行符。

`--routines`或`-R`（布尔值）

在转储输出中包含存储函数和存储过程。

`--set-charset`（布尔值）

将SET NAMES charset语句写到输出里，其中charset的默认值是utf8。字符集可以用`--default-character-set`选项来更改。`--set-charset`选项默认是启用的。使用`--skip-set-charset`选项可以禁用它。

`--single-transaction`（布尔值）

此选项能够让 InnoDB 表的转储保持不变。其中心思想是，将所有表转储到同一个事务里。mysqldump会使用事务隔离层REPEATABLE READ来生成一个一致性的转储，同时不会阻塞其他客户端。（对于非事务性表，在转储过程中它们仍然可能会发生变化。）此选项会禁用`--lock-all-tables`选项。

`--skip-opt`

此选项拥有`--opt`选项的反面效果，它默认是启用的。

`--tab=dump_dir`或`-T dump_dir`

此选项将使mysqldump程序为每个表生成两个文件，同时将dump_dir用作存放这些文件的位置。该目录必须已经存在。对于每个名为tbl_name的表，都会生成一个dump_dir/tbl_name.txt文件（其中包含表的数据）和一个dump_dir/tbl_name.sql文件（其中包含用于创建表的CREATE TABLE语句）。在使用此选项时，你必须拥有FILE权限。

默认情况下，这些输出的数据文件里的每一行都会带一个换行符，各列值之间以制表符进行分隔。此格式可以通过后面F. 15.3节描述的选项来更改。

只有清楚地了解`--tab`选项的工作原理，才不会对它产生混淆。原理如下。

其中一些文件将被输出到服务器主机上，而另一些文件则会被输出到客户端主机上。

dump_dir目录会用于服务器主机上的*.txt文件和客户端主机上的*.sql文件。如果这两个主机不同，则输出文件将被分别创建在不同的主机上。为了避免出现搞不清输出文件存放位置的问题，当使用此选项时，最好是在服务器主机上运行 mysqldump 程序，以便所有文件都被创建在同一台机器上。

.txt文件隶属于运行服务器所使用的那个账户，而.sql文件则归你所有。造成这一现象的原因在于：服务器自己会输出*.txt文件；而CREATE TABLE语句会被服务器发送至mysqldump程序——它负责输出*.sql文件。

--tables

改写--databases选项，使得随后的所有参数都被解释为表名。

--triggers (布尔值)

在转储输出里包括触发器。触发器默认是被包含的。如果要将其排除在外，可以使用--skip-triggers选项。

--tz-utc (布尔值)

在连接到服务器之后把时区设置为UTC，并且在输出里包含一条SET TIME_ZONE='+00:00' 语句。这种做法带来的效果是：在转储和重新加载数据时，如果它们之间的时区不相同，可以防止本地时区的来回转换，从而不会使TIMEATAMP的值发生变化。此选项默认是启用的。使用--skip-tz-utc选项可以禁用它。

--where=where_expr或-w where_expr

只转储满足由where_expr所指定的那个WHERE条件的行。这个条件应该用引号引起来，以防止命令解释器错误地把它当作多个命令行参数。

--xml或-X

生成XML格式的输出，而不是生成一组SQL语句。

F. 15.3 mysqldump的数据格式选项

如果你使用--tab 或-T 选项来为每个表生成一个独立的数据文件，那么可以使用以下几个选项。这些选项的值应该用适当的引号字符括起来。这些选项与LOAD DATA语句所使用的数据格式选项很相似。更多相关信息请参才附录E里的LOAD DATA条目。

--fields-enclosed-by=char

指定用于将列值括起来的字符，通常是引号。默认情况下，这些列值都不会被括起来。此选项不能与`--fields-optionally-enclosed-by`选项一起使用。

`--fields-escaped-by=char`

为转义特殊字符指定转义字符。默认没有转义字符。

`--fields-optionally-enclosed-by=char`

指定用于将列值括起来的字符，通常是引号。这个字符适用于非数字列。默认情况下，这些列值都不会被括起来。此选项不能与`--fields-enclosed-by`选项一起使用。

`--fields-terminated-by=str`

指定数据文件所使用的（一个或多个）列值分隔符。默认情况下，列值之间都是通过制表符分隔的。

`--lines-terminated-by=str`

指定在输出行末尾写入的字符。默认为换行符。

F. 15. 4 mysqldump的变量

下面这些mysqldump变量都可以根据F. 2. 1. 2节的说明进行设置。

`max_allowed_packet`

它指的是服务器与客户端之间通信时使用的缓冲区的最大大小。默认值为 24 MB，最大值为1 GB。

`net_buffer_length`

它指的是服务器与客户端之间通信时使用的缓冲区的初始大小。此缓冲区可以扩张到`max_allowed_packet`个字节大小。默认值略小于1 MB。

F. 16 mysqlimport

客户端程序 `mysqlimport` 是一个批量加载程序，它可以把文本文件的内容读到已有的表里。它向SQL语句LOAD DATA 提供一个命令行接口，并且它是一种非常高效地将多行数据输到表的方式。命令格式如下：

`mysqlimport [options] db_name file_name ...`其中，`db_name` 参数指定的是包含要加载进数据的那些表的数据库。要加载的表由文件名参数`file_name`确定。对于每一个文件名，从名字里第一个句点开始的扩展名部分将被截去，而剩下的主名部分将被视为该文件将被加载进的那个表的名字。例如，`mysqlimport` 将把名为`president.txt`的文件的内容加载到`president`表里。

`mysqlimport`只能读取数据文件。它不能用于读取由`mysqldump`生成的SQL格式的转储文件。对于该格式的文件，需要使用`mysql`。

F. 16.1 `mysqlimport`支持的标准选项

<code>--bind-address</code>	<code>--default-character-set</code>	<code>--protocol</code>
<code>--character-sets-dir</code>	<code>--help</code>	<code>--shared-memory-base-name</code>
<code>--compress</code>	<code>--host</code>	<code>--silent</code>
<code>--debug</code>	<code>--password</code>	<code>--socket</code>
<code>--debug-check</code>	<code>--pipe</code>	<code>--user</code>
<code>--debug-info</code>	<code>--plugin-dir</code>	<code>--verbose</code>
<code>--default-auth</code>	<code>--port</code>	<code>--version</code>

对这些选项的概要描述请参考 F. 2. 1节。`mysqlimport` 也支持 F. 2. 1. 1节列出的那些标准 SSL选项。

F. 16.2 `mysqlimport`特有的选项

下面这些选项主要用于控制`mysqlimport`如何处理输入文件。在随后的 F. 16. 3节，将会对那些用于表明输入文件数据格式的选项进行描述。

`--columns=col_list`或`-C col_list`

指定表的各个列，它们与数据文件里的列相对应。`mysqlimport`会将输入行的各个值载入这里列出的列里，并且会将其他列设置为它们的默认值。在`col_list` 里，各个列名之间用逗号隔开。

`--delete`或`-d`（布尔值）

在把数据加载到表里之前，先清空表。

`--force`或`-f`（布尔值）

即使出现了错误也要继续加载各个行。

`--ignore`或`-i`

如果某个输入行包含的唯一键值已存在于表里，则保留已有的行，并丢弃这个输入行。选项`--ignore`与`--replace`是互斥的，不能同时使用。

`--ignore-lines=n`

忽略数据文件的前`n`行。例如，此选项可用于跳过最开始的那行列标签。

`--local`或`-L`（布尔值）

默认情况下，`mysqlimport`会让服务器去读取数据文件——这意味着，该文件必须位于服务器主机上，并且你必须拥有 `FILE` 权限。通过指定`--local`选项，可以告诉 `mysqlimport`自己读取该数据文件，并将其发送给服务器。虽然在服务器主机以外的其他机器上运行`mysqlimport`会显得慢一点，但是它可行；除此之外，即使你在服务器主机上没有`FILE`权限也可以进行操作。

如果服务器被配置为禁止使用`LOAD DATA LOCAL`语句，则此选项不会有效果。

`--lock-tables`或`-l`（布尔值）

在把数据加载到表里之前，先锁定这些表。

`--low-priority`（布尔值）

对那些生成之后用于将数据加载到表的语句使用优先级修饰符 `LOW_PRIORITY`。

`--replace`或`-r`（布尔值）

如果某个输入行包含的唯一键值已存在于表里，则使用输入行来替换已有行。选项`--ignore`与`--replace`是互斥的，不能同时使用。

`--use-threads =n`

使用n个线程并行加载文件。

F. 16.3 mysqlimport的数据格式选项

默认情况下，mysqlimport会假设数据文件包含的数据都是这样的格式：每一行都有一个换行结束符，各个列值之间用制表符隔开。这种格式可以通过以下选项加以更改。这些选项的值应该用适当的引号括起来。这些选项与LOAD DATA语句所使用的数据格式选项很相似。更多相关信息请参考附录E里的LOAD DATA条目。

`--fields-enclosed-by=char`

指定用于将列值括起来的字符，通常是引号。默认情况下，这些值都假定不会被任何字符括起来。此选项不能与`--fields-optionally-enclosed-by`一起使用。

`--fields-escaped-by=char`

指定用于转义特殊字符的转义字符。默认没有转义字符。

`--fields-optionally-enclosed-by=char`

指定有可能用于将列值括起来的字符，通常是引号。此选项不能与`--fields-enclosed-by`一起使用。

`--fields-terminated-by=str`

指定分隔各个列值的（一个或多个）字符。默认情况下，各个列值之间假定都是通过制表符分隔的。

`--lines-terminated-by=str`

指定分隔各个输入行的（一个或多个）字符。默认情况下，各行假定都是以换行符结束的。

F. 17 mysqlshow

mysqlshow可用于列出数据库、数据库里的表，或者与表的各个列或索引有关的信息。它提供了一个面向SQL语句SHOW的命令行接口：

`mysqlshow [options] [db_name [tbl_name [col_name]]]`

如果没有指定数据库名，mysqlshow将列出服务器主机上的所有数据库。如果只指定了数据库名，但没有指定表名，mysqlshow将列出该数据库里的所有表。如果指定了数据库名和表名，但没有指定列名，它会列出该表里所有列。如果指定了所有这些名字，mysqlshow将显示与指定列有关的信息。

最后那个参数可以包含SQL通配符“%”和“_”，其意义与操作符LIKE所使用的通配符一样。输出结果将仅限于那些与该通配符相匹配的值。如果最后那个参数包含shell的通配符“*”或“?”，那么它们将分别被当作“%”和“_”。

F. 17.1 mysqlshow支持的标准选项

--bind-address	--default-character-set	--protocol
--character-sets-dir	--help	--shared-memory-base-name
--compress	--host	--socket
--debug	--password	--user
--debug-check	--pipe	--verbose
--debug-info	--plugin-dir	--version
--default-auth	--port	

对这些选项的概要描述请参考F. 2. 1节。mysqlshow也支持F. 2. 1. 1节列出的那些标准SSL选项。

其中，--verbose选项会导致输出结果（每个数据库的表、每个表的行等）里包含其他的列。此选项可以多次指定。

F. 17.2 mysqlshow特有的选项

--count（布尔值）

在输出结果里包含每个表的行数。对于某些存储引擎，统计行数可能会比较慢。

--keys或-k（布尔值）

除了显示与列有关的信息外，还要显示与表索引有关的信息。此选项只有在指定了表名参数的时候才有意义。

`--show-table-type`或`-t`（布尔值）

在输出结果里包含一列，用于显示每个表（BASE TABLE 或 VIEW）的类型，与 SHOW FULL TABLES的输出结果类似。

`--status`或`-i`（布尔值）

显示与通过SHOW TABLE STATUS语句显示出的结果一样的表信息。

F. 18 perror

`perror`用于显示出错代码所对应的出错信息：

`perror [options] [err_code] ...`

可以用它来确定MySQL程序所返回的出错代码的具体含义。

`% perror 142`

MySQL error: 142=Unknown character set used

`perror`支持的标准选项

`--help` `--silent` `--verbose` `--version`

对这些选项的概要描述请参考F. 2. 1节。

`--silent` 选项表示只显示出错信息，而不显示出错代码。默认选项为`--verbose`，即同时显示出错代码和出错信息。

`--info`和`-l`都等同于`--help`。